৩০টি অধ্যায় • ৩০টি পরীক্ষিত Project • Browser থেকে Backend, Testing, Security, Deployment ও System Design

JavaScript Mastery

শূন্য থেকে প্রফেশনাল

Moynul Hasan

সম্পূর্ণ সংস্করণ • প্রকাশ-পূর্ব Proof, অক্টোবর ২০২৬

স্বত্ব ও প্রকাশনা তথ্য

JavaScript Mastery — শূন্য থেকে প্রফেশনাল

লেখক: **Moynul Hasan**

© ২০২৬ Moynul Hasan। সর্বস্বত্ব সংরক্ষিত। লেখকের লিখিত অনুমতি ছাড়া এই বইয়ের কোনো অংশ কোনো মাধ্যমে পুনরুৎপাদন, সংরক্ষণ বা বিতরণ করা যাবে না।

সংস্করণ: সম্পূর্ণ সংস্করণ, প্রকাশ-পূর্ব Proof (সম্পাদকীয় Proofread-এর জন্য), অক্টোবর ২০২৬

ISBN: **[প্রকাশকের কাছ থেকে প্রাপ্তব্য — এখনো নির্ধারিত হয়নি]**

প্রকাশক: **[এখনো নির্ধারিত হয়নি]**

প্রথম প্রকাশের তারিখ: **[এখনো নির্ধারিত হয়নি]**

Companion Code-এর License: **[লেখক নির্ধারণ করবেন — এখনো নির্ধারিত হয়নি]**

দায়মুক্তি: এই বইয়ের সব Code, Project ও উদাহরণ শিক্ষামূলক উদ্দেশ্যে তৈরি। প্রতিটি অধ্যায়ে স্পষ্ট বলা আছে কোন অংশ Production-এর জন্য যথেষ্ট নয়। বাস্তব প্রয়োগের আগে নিজস্ব Security Review, Testing ও Code Review করুন। লেখক ও প্রকাশক এই Code-এর ব্যবহারজনিত কোনো ক্ষতির জন্য দায়ী নন।

Trademark: JavaScript যুক্তরাষ্ট্রে Oracle Corporation-এর Trademark। Node.js OpenJS Foundation-এর Trademark। TypeScript ও Windows Microsoft Corporation-এর, Chrome Google LLC-এর, এবং Docker Docker Inc.-এর Trademark। বইয়ে উল্লিখিত অন্যান্য Product ও Service-এর নাম তাদের নিজ নিজ মালিকের Trademark হতে পারে; এখানে শুধু পরিচয়ের জন্য ব্যবহৃত।

যাচাই: বইয়ের প্রতিটি চালানো-যোগ্য Listing ও প্রতিটি Project-এর Test Windows 10, Node.js 22.16.0 ও Chrome 155-এ চালিয়ে দেখা হয়েছে। যা চালানো সম্ভব হয়নি (যেমন Docker Build বা PostgreSQL), তা সংশ্লিষ্ট অধ্যায়ে স্পষ্টভাবে চিহ্নিত।

লেখক পরিচিতি

Moynul Hasan বর্তমানে M&N Holidays-এ IT Officer হিসেবে কর্মরত। তাঁর শিক্ষার ক্ষেত্র Computer Science and Engineering (CSE), Daffodil International University। এই বইয়ের অনেক উদাহরণ ও Project ভ্রমণ-ব্যবসার প্রেক্ষাপটে সাজানো—Package, Booking, Visa ও গ্রাহকসেবা।

নাম	Moynul Hasan
পেশা	IT Officer, M&N Holidays
শিক্ষা	Computer Science and Engineering (CSE), Daffodil International University
ঠিকানা	Bogura, Bangladesh
যোগাযোগ	01745334044

ভূমিকা

JavaScript আজ শুধু Web Page-কে চলমান করার ভাষা নয়। Browser, Server, Mobile, Desktop ও Cloud—সব জায়গায় এর ব্যবহার। বাংলাভাষী শিক্ষার্থীদের অনেকেই এই ভাষা শিখতে চান, কিন্তু ভালো মানের উপকরণের বেশির ভাগই ইংরেজিতে, আর বাংলায় যা পাওয়া যায় তা প্রায়ই মৌলিক Syntax-এ থেমে যায়। এই বইয়ের লক্ষ্য সেই ফাঁক পূরণ করা: একেবারে প্রথম `console.log()` থেকে শুরু করে পরীক্ষিত, নিরাপদ ও রক্ষণযোগ্য পেশাদার Application পর্যন্ত একটি ধারাবাহিক পথ।

বইটির একটি মূল নীতি আছে: **যা দেখানো হয়েছে, তা চালিয়ে দেখা হয়েছে।** প্রতিটি Code Listing একটি আসল File থেকে নেওয়া, প্রতিটি Output Panel আসল চালনার ফল, এবং প্রতিটি অধ্যায়ের Project-এর স্বয়ংক্রিয় Test আছে। যেখানে কিছু চালানো সম্ভব হয়নি—যেমন Docker Build বা একটি আলাদা Database Server—সেখানে তা স্পষ্টভাবে বলা হয়েছে। Production-এর জন্য কী যথেষ্ট নয়, তা-ও প্রতিটি Project-এর README ও অধ্যায়ে লেখা।

আরেকটি নীতি: প্রযুক্তিগত শব্দ ইংরেজিতেই রাখা হয়েছে (Function, Promise, Transaction), কারণ কাজের জগতে, Documentation-এ ও Error-বার্তায় আপনি এগুলোই দেখবেন; ব্যাখ্যা বাংলায়। প্রতিটি অধ্যায়ের শেষে একটি পরিভাষার সারণি আছে, আর বইয়ের শেষে সব পরিভাষার একটি একত্রিত কোষ।

বইয়ের বহু উদাহরণ ভ্রমণ-ব্যবসার প্রেক্ষাপটে: Package, Booking, Visa বা Loyalty Point—বাস্তব সমস্যা, বাস্তব নিয়ম। আশা করি সেগুলো আপনার নিজের Project-এর জন্যও অনুপ্রেরণা হবে।

শুভকামনাসহ, **Moynul Hasan**

বইটি সম্পর্কে

এই বই কার জন্য

- যাঁরা Programming একেবারে নতুন শুরু করছেন এবং বাংলায় ধাপে ধাপে শিখতে চান।
- যাঁরা JavaScript-এর মৌলিক বিষয় জানেন, কিন্তু Asynchronous Programming, Testing, Security, Architecture বা Backend-এ আত্মবিশ্বাসী হতে চান।
- যাঁরা চাকরির Interview বা Portfolio-র জন্য বাস্তবসম্মত প্রস্তুতি নিতে চান।

বইটির গঠন

বইটি ৩০টি অধ্যায়ে সাজানো; প্রতিটি অধ্যায় আগের অধ্যায়ের ওপর দাঁড়িয়ে:

অংশ	অধ্যায়	বিষয়
ভিত্তি	১-৯	প্রথম পদক্ষেপ, Variable ও Data Type, Operator, Loop, Function, Array, Object, String, DOM ও Event
আধুনিক JavaScript	১০-১৮	Asynchronous Programming, Error Handling, Modules, OOP, Closure, Browser API, Debugging ও Testing, Regular Expression, Functional Programming
পেশাদার Engineering	১৯-২৪	Performance, Security, Design Pattern, Testing Mastery, Profiling, TypeScript
Backend থেকে System Design	২৫-৩০	Node.js Backend, Database, Real-Time, Deployment, Distributed System, Full-Stack Capstone ও Career Roadmap

প্রতিটি অধ্যায়ের কাঠামো

প্রতিটি অধ্যায় একই ক্রমে সাজানো, যাতে আপনি জানেন কোথায় কী পাবেন: শেখার লক্ষ্য; পূর্বশর্ত; অধ্যায়টি কেন গুরুত্বপূর্ণ; ধাপে ধাপে ধারণা ও চালানো-যোগ্য উদাহরণ; একটি পূর্ণাঙ্গ Project ও তার ব্যাখ্যা; সাধারণ ভুল; Debugging নির্দেশিকা; পেশাদার অভ্যাস; প্রাথমিক, মধ্যম ও চ্যালেঞ্জ স্তরের অনুশীলনী ও তাদের সমাধান; দশটি MCQ ও ব্যাখ্যাসহ উত্তর; সারসংক্ষেপ; পরিভাষা; এবং আরও পড়ার তালিকা।

বইটি কীভাবে ব্যবহার করবেন

ক্রমানুসারে পড়ুন, কিন্তু হাতে-কলমে

অধ্যায়গুলো ক্রমানুসারে পড়ার জন্য লেখা। প্রতিটি উদাহরণ নিজে লিখে চালান—Copy-Paste নয়। একটি Output Panel-এর সঙ্গে নিজের ফল মিলিয়ে দেখুন; না মিললে কারণ খুঁজুন। সেটিই সবচেয়ে বড় শিক্ষা।

Code Panel পড়ার নিয়ম

- **Panel-এর শিরোনাম:** File-এর নাম, ভাষা (JavaScript, HTML, Terminal ইত্যাদি) এবং কোথায় চলবে (চলবে: Node.js, Browser, বা দুটিই)।
- **অংশ N / M (Part N of M):** একটি দীর্ঘ File কয়েকটি Panel-এ ভাগ করা; সব অংশ মিলে সম্পূর্ণ File। `// ...` চিহ্ন বলে এখানে আগের বা পরের অংশ জোড়া লাগবে।
- **নির্বাচিত অংশ (Selected Excerpt):** File-এর শুধু প্রাসঙ্গিক অংশ; সম্পূর্ণ File Companion Code-এ।
- **Output / Error Message:** ঠিক আগের Listing চালালে যা দেখা যায়, আসল চালনা থেকে নেওয়া। সময় বা যন্ত্রভেদে বদলায় এমন মান (যেমন Millisecond) কোথাও কোথাও বাদ রাখা হয়েছে, এবং তা লেখায় বলা আছে।
- **Terminal:** Command Line-এ লেখার Command।
- **দীর্ঘ Line:** পাতার প্রস্থে ভেঙে একটু ডানে সরিয়ে দেখানো; আসল File-এ সেটি এক Line।

অনুশীলনী ও MCQ

প্রতিটি অধ্যায়ের অনুশীলনী তিন স্তরে: প্রাথমিক, মধ্যম ও চ্যালেঞ্জ। আগে নিজে চেষ্টা করুন, তারপর সমাধান দেখুন। MCQ-গুলোর উত্তর ব্যাখ্যাসহ দেওয়া; ভুল হলে ব্যাখ্যাটি পড়ে সংশ্লিষ্ট অনুচ্ছেদে ফিরে যান।

প্রযুক্তিগত প্রয়োজন

বিষয়	প্রয়োজন	মন্তব্য
Node.js	২২.৫ বা নতুন	অধ্যায় ১-২৫ Node.js ২০-এও চলে; অধ্যায় ২৬, ২৭-এর Outbox, ২৯ ও ৩০-এর জন্য `node:sqlite` লাগে (২২.৫+)। যাচাই: Node.js 22.16.0
Browser	আধুনিক Chrome, Edge বা Firefox	যাচাই: Chrome 155
Code Editor	যেকোনো (যেমন Visual Studio Code)	—
npm Package	অধিকাংশ Project-এ লাগে না	অধ্যায় ২৪: typescript, @types/node; অধ্যায় ২৫: express; অধ্যায় ২৭: ws; অধ্যায় ৩০: ঐচ্ছিক Type Check-এর জন্য typescript
Database	আলাদা Server লাগে না	SQLite Node.js-এর অন্তর্নির্মিত; PostgreSQL-এর Code (অধ্যায় ২৬) ঐচ্ছিক ও চালানো হয়নি
Docker	ঐচ্ছিক	অধ্যায় ২৮-এর Dockerfile ও Compose File; এই বইয়ের যাচাইয়ে Docker Build চালানো হয়নি
Operating System	Windows, macOS বা Linux	যাচাই শুধু Windows 10-এ

Browser-এর Project-গুলো `file://` থেকে নয়, একটি Local HTTP Server থেকে খুলুন (যেমন Project Folder-এ `npx serve` বা `python -m http.server 8000`), কারণ ES Module ও `fetch` সাধারণত `file://`-এ ঠিকমতো চলে না।

Companion Code

বইয়ের সঙ্গে একটি Companion Code Archive দেওয়া আছে: `JavaScript_Mastery_Companion_Code.zip`। এর গঠন:

Folder-এর কাঠামো

```
JavaScript_Mastery_Companion_Code/
├── README.md                প্রয়োজন, চালানোর নিয়ম, যাচাই-পরিবেশ
├── chapter-01/
│   ├── project/
│   │   └── interactive-greeting-app/
│   └── examples/            অধ্যায়ের উদাহরণ ও সমাধানের File
├── chapter-02/
│   ├── project/student-result-calculator/
│   └── examples/
├── ...
└── chapter-30/
    ├── project/travel-master-pro/
    └── examples/
```

প্রতিটি Project Folder-এ একটি README আছে, যেখানে চালানোর Command, প্রয়োজনীয় Node.js সংস্করণ ও সীমাবদ্ধতা লেখা। সাধারণ নিয়ম:

১. ZIP খুলে সংশ্লিষ্ট অধ্যায়ের Folder-এ যান।

২. `package.json`-এ Dependency থাকলে প্রথমে `npm install` চালান (মাত্র কয়েকটি Project-এ লাগে)।

৩. Test চালাতে Project Folder-এ `npm test` বা `node --test`; Browser Project হলে Local HTTP Server দিয়ে `index.html` খুলুন।

বইয়ের Listing-এর ওপরে বা লেখায় যে Path দেখানো (যেমন `project/travel-live/src/gateway.mjs` বা `examples/02-sse-basic.mjs`), তা সংশ্লিষ্ট অধ্যায়ের Folder থেকে হুবহু একই: `chapter-27/project/travel-live/src/gateway.mjs`। উদাহরণের File-গুলো Project-এর Code `../project/...` Path দিয়ে Import করে, তাই Folder-এর গঠন বদলাবেন না। পরিশিষ্ট খ-তে প্রতিটি অধ্যায়ের Project-এর একটি তালিকা আছে।

সূচিপত্র

অধ্যায় ১ JavaScript-এর জগতে প্রথম পদক্ষেপ ২৪
শেখার লক্ষ্য ২৪
পূর্বশর্ত ২৫
এই অধ্যায় কেন গুরুত্বপূর্ণ ২৫
১.১ Programming কী ২৫
১.২ JavaScript কী এবং কেন শিখবেন ২৫
১.৩ এই বই থেকে কী শিখবেন ২৭
১.৪ Development Environment প্রস্তুত করা ২৭
১.৫ প্রথম JavaScript Program ৩০
১.৬ JavaScript লেখার মৌলিক নিয়ম ৩৩
১.৭ Variable, Function এবং শর্ত: সংক্ষিপ্ত পরিচয় ৩৪
১.৮ Error Message পড়তে শেখা ৩৬
১.৯ Web Page-এ JavaScript ব্যবহার ৩৯
১.১০ সাধারণ ভুল ৪১
১.১১ Debugging নির্দেশিকা ৪২
১.১২ পেশাদার অভ্যাস ৪৪
১.১৩ Project: Interactive Greeting App ৪৪
১.১৪ অনুশীলনী ৫১
১.১৫ অনুশীলনীর সমাধান ৫২
১.১৬ MCQ: নিজেকে যাচাই করুন ৫৭
১.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৫৮
১.১৮ অধ্যায়ের সারসংক্ষেপ ৫৯
১.১৯ গুরুত্বপূর্ণ পরিভাষা ৬০
১.২০ আরও পড়ুন ৬১

অধ্যায় ২ Variables, Data Types এবং Memory ৬২
শেখার লক্ষ্য ৬২
পূর্বশর্ত ৬৩
এই অধ্যায় কেন গুরুত্বপূর্ণ ৬৩
২.১ Variable: মানের নাম ৬৩
২.২ let, const এবং var ৬৪
২.৩ Data Type এবং typeof ৬৬
২.৪ Number-এর আচরণ ৬৭
২.৫ undefined এবং null ৬৯
২.৬ Dynamic Typing ৬৯
২.৭ Type Conversion ৭০
২.৮ সমতা যাচাই: === এবং == ৭৩
২.৯ Memory Model: মান এবং Reference ৭৪
২.১০ সাধারণ ভুল ৭৭
২.১১ Debugging নির্দেশিকা ৭৮
২.১২ পেশাদার অভ্যাস ৭৯
২.১৩ Project: Student Result Calculator ৭৯
২.১৪ অনুশীলনী ৮৪
২.১৫ অনুশীলনীর সমাধান ৮৫
২.১৬ MCQ: নিজেকে যাচাই করুন ৯০
২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৯২
২.১৮ অধ্যায়ের সারসংক্ষেপ ৯৩
২.১৯ গুরুত্বপূর্ণ পরিভাষা ৯৩
২.২০ আরও পড়ুন ৯৪

অধ্যায় ৩ Operators এবং Decision Making ৯৫
শেখার লক্ষ্য ৯৫
পূর্বশর্ত ৯৬
এই অধ্যায় কেন গুরুত্বপূর্ণ ৯৬
৩.১ Expression এবং Operator ৯৬
৩.২ Arithmetic Operator ৯৭
৩.৩ Assignment Operator ৯৮
৩.৪ Comparison Operator ১০০
৩.৫ Logical Operator ১০০
৩.৬ Truthy এবং Falsy ১০২
৩.৭ if, else if এবং else ১০৪
৩.৮ switch ১০৫
৩.৯ Ternary Operator ১০৭
৩.১০ Operator Precedence ১০৭
৩.১১ Input Validation ১০৯
৩.১২ সাধারণ ভুল ১১০
৩.১৩ Debugging নির্দেশিকা ১১১
৩.১৪ পেশাদার অভ্যাস ১১২
৩.১৫ Project: Shopping Discount Calculator ১১২
৩.১৬ অনুশীলনী ১১৭
৩.১৭ অনুশীলনীর সমাধান ১১৮
৩.১৮ MCQ: নিজেকে যাচাই করুন ১২৪
৩.১৯ MCQ-এর উত্তর ও ব্যাখ্যা ১২৬
৩.২০ অধ্যায়ের সারসংক্ষেপ ১২৭
৩.২১ গুরুত্বপূর্ণ পরিভাষা ১২৭
৩.২২ আরও পড়ুন ১২৮

অধ্যায় ৪ Loops এবং Iteration ১২৯
শেখার লক্ষ্য ১২৯
পূর্বশর্ত ১৩০
এই অধ্যায় কেন গুরুত্বপূর্ণ ১৩০
৪.১ Loop কেন প্রয়োজন ১৩০
৪.২ for Loop ১৩১
৪.৩ Array: তালিকার প্রথম পরিচয় ১৩২
৪.৪ Loop-এর সীমা এবং Off-by-One Error ১৩৩
৪.৫ while Loop ১৩৪
৪.৬ do...while Loop ১৩৫
৪.৭ break এবং continue ১৩৬
৪.৮ Nested Loop ১৩৮
৪.৯ Infinite Loop এবং তার প্রতিরোধ ১৩৯
৪.১০ Loop দিয়ে Data প্রক্রিয়াকরণ ১৪০
৪.১১ সাধারণ ভুল ১৪২
৪.১২ Debugging নির্দেশিকা ১৪৩
৪.১৩ পেশাদার অভ্যাস ১৪৪
৪.১৪ Project: Cart Analyzer ১৪৪
৪.১৫ অনুশীলনী ১৪৯
৪.১৬ অনুশীলনীর সমাধান ১৫১
৪.১৭ MCQ: নিজেকে যাচাই করুন ১৫৭
৪.১৮ MCQ-এর উত্তর ও ব্যাখ্যা ১৫৮
৪.১৯ অধ্যায়ের সারসংক্ষেপ ১৫৯
৪.২০ গুরুত্বপূর্ণ পরিভাষা ১৬০
৪.২১ আরও পড়ুন ১৬০

অধ্যায় ৫ Functions এবং Scope ১৬১
শেখার লক্ষ্য ১৬১
পূর্বশর্ত ১৬২
এই অধ্যায় কেন গুরুত্বপূর্ণ ১৬২
৫.১ Function কেন প্রয়োজন ১৬২
৫.২ Function Declaration ১৬৩
৫.৩ Parameter এবং Argument ১৬৩
৫.৪ Return Value ১৬৪
৫.৫ Default Parameter ১৬৬
৫.৬ Function Expression ১৬৭
৫.৭ Arrow Function ১৬৭
৫.৮ Scope: Block, Function এবং Global ১৬৯
৫.৯ Lexical Scope ১৭১
৫.১০ Hoisting ১৭২
৫.১১ পুনর্ব্যবহারযোগ্য Logic ১৭৩
৫.১২ সাধারণ ভুল ১৭৬
৫.১৩ Debugging নির্দেশিকা ১৭৭
৫.১৪ পেশাদার অভ্যাস ১৭৮
৫.১৫ Project: Invoice Calculator ১৭৯
৫.১৬ অনুশীলনী ১৮৩
৫.১৭ অনুশীলনীর সমাধান ১৮৫
৫.১৮ MCQ: নিজেকে যাচাই করুন ১৯০
৫.১৯ MCQ-এর উত্তর ও ব্যাখ্যা ১৯২
৫.২০ অধ্যায়ের সারসংক্ষেপ ১৯৩
৫.২১ গুরুত্বপূর্ণ পরিভাষা ১৯৩
৫.২২ আরও পড়ুন ১৯৪

অধ্যায় ৬ Arrays এবং Data Processing ১৯৫
শেখার লক্ষ্য ১৯৫
পূর্বশর্ত ১৯৬
এই অধ্যায় কেন গুরুত্বপূর্ণ ১৯৬
৬.১ Array কেন প্রয়োজন ১৯৬
৬.২ Array তৈরি করা ১৯৭
৬.৩ Index দিয়ে পড়া ও বদলানো ১৯৮
৬.৪ Element যোগ ও বাদ: push, pop, shift, unshift ১৯৯
৬.৫ slice এবং splice ২০০
৬.৬ for...of ২০১
৬.৭ map ২০৩
৬.৮ filter ২০৪
৬.৯ find ২০৫
৬.১০ reduce ২০৫
৬.১১ Mutation বনাম নতুন Array ২০৭
৬.১২ Data রূপান্তরের ধারা ২০৯
৬.১৩ সাধারণ ভুল ২১০
৬.১৪ Debugging নির্দেশিকা ২১১
৬.১৫ পেশাদার অভ্যাস ২১২
৬.১৬ Project: Inventory Dashboard ২১২
৬.১৭ অনুশীলনী ২১৭
৬.১৮ অনুশীলনীর সমাধান ২১৮
৬.১৯ MCQ: নিজেকে যাচাই করুন ২২৩
৬.২০ MCQ-এর উত্তর ও ব্যাখ্যা ২২৫
৬.২১ অধ্যায়ের সারসংক্ষেপ ২২৬
৬.২২ গুরুত্বপূর্ণ পরিভাষা ২২৬
৬.২৩ আরও পড়ুন ২২৭

অধ্যায় ৭ Objects: Properties, Methods, Destructuring এবং Data Modeling ২২৮
শেখার লক্ষ্য ২২৮
পূর্বশর্ত ২২৯
এই অধ্যায় কেন গুরুত্বপূর্ণ ২২৯
৭.১ Object কেন প্রয়োজন ২২৯
৭.২ Object তৈরি করা ২৩০
৭.৩ Dot এবং Bracket Notation ২৩১
৭.৪ Property পড়া, যোগ, বদল এবং মোছা ২৩২
৭.৫ Method এবং this ২৩৩
৭.৬ Nested Object এবং Optional Chaining ২৩৫
৭.৭ Destructuring ২৩৬
৭.৮ Object-এর ওপর Loop ২৩৮
৭.৯ Spread, Rest এবং Shallow Copy ২৩৮
৭.১০ Reference, সমতা এবং Freeze ২৪০
৭.১১ JSON: তথ্য সংরক্ষণ ও আদান-প্রদান ২৪১
৭.১২ Data Modeling: তথ্যের গঠন নকশা করা ২৪৩
৭.১৩ সাধারণ ভুল ২৪৪
৭.১৪ Debugging নির্দেশিকা ২৪৪
৭.১৫ পেশাদার অভ্যাস ২৪৫
৭.১৬ Project: Student Management System ২৪৬
৭.১৭ অনুশীলনী ২৫২
৭.১৮ অনুশীলনীর সমাধান ২৫৩
৭.১৯ MCQ: নিজেকে যাচাই করুন ২৫৯
৭.২০ MCQ-এর উত্তর ও ব্যাখ্যা ২৬০
৭.২১ অধ্যায়ের সারসংক্ষেপ ২৬১
৭.২২ গুরুত্বপূর্ণ পরিভাষা ২৬২
৭.২৩ আরও পড়ুন ২৬২

অধ্যায় ৮ Strings এবং Modern Text Processing ২৬৩
শেখার লক্ষ্য ২৬৩
পূর্বশর্ত ২৬৪
এই অধ্যায় কেন গুরুত্বপূর্ণ ২৬৪
৮.১ String কী ২৬৪
৮.২ String লেখা ২৬৫
৮.৩ Length, Index এবং Unicode ২৬৬
৮.৪ অংশ পড়া: at এবং slice ২৬৮
৮.৫ খোঁজা ২৬৮
৮.৬ পরিষ্কার করা ও সাজানো ২৬৯
৮.৭ Case Conversion এবং তুলনা ২৭০
৮.৮ split এবং join: শব্দে ভাগ করা ২৭১
৮.৯ Unicode Normalization ২৭২
৮.১০ Regular Expression-এর পরিচয় ২৭৩
৮.১১ Text যাচাই ২৭৫
৮.১২ নিরাপদ Rendering: XSS ২৭৬
৮.১৩ সাধারণ ভুল ২৭৭
৮.১৪ Debugging নির্দেশিকা ২৭৭
৮.১৫ পেশাদার অভ্যাস ২৭৮
৮.১৬ Project: Content Search Utility ২৭৯
৮.১৭ অনুশীলনী ২৮৪
৮.১৮ অনুশীলনীর সমাধান ২৮৫
৮.১৯ MCQ: নিজেকে যাচাই করুন ২৯১
৮.২০ MCQ-এর উত্তর ও ব্যাখ্যা ২৯২
৮.২১ অধ্যায়ের সারসংক্ষেপ ২৯৩
৮.২২ গুরুত্বপূর্ণ পরিভাষা ২৯৪

৮.২৩ আরও পড়ুন ২৯৪

অধ্যায় ৯ DOM Manipulation, Events এবং Interactive Web Applications ২৯৫

শেখার লক্ষ্য ২৯৫
পূর্বশর্ত ২৯৬
এই অধ্যায় কেন গুরুত্বপূর্ণ ২৯৬
৯.১ DOM: Page-এর গাছ ২৯৬
৯.২ Script কখন চলে ২৯৮
৯.৩ Element বদলানো ২৯৯
৯.৪ Element তৈরি, যোগ ও সরানো ৩০১
৯.৫ Event: ব্যবহারকারীর কাজে সাড়া ৩০২
৯.৬ Event Object এবং Propagation ৩০৩
৯.৭ Event Delegation ৩০৫
৯.৮ Form এবং Validation ৩০৬
৯.৯ State এবং UI আলাদা রাখা ৩০৮
৯.১০ localStorage: নিরাপদে সংরক্ষণ ৩০৯
৯.১১ সাধারণ ভুল ৩১১
৯.১২ Debugging নির্দেশিকা ৩১১
৯.১৩ পেশাদার অভ্যাস ৩১২
৯.১৪ Project: Task Manager ৩১২
৯.১৫ অনুশীলনী ৩২৪
৯.১৬ অনুশীলনীর সমাধান ৩২৫
৯.১৭ MCQ: নিজেকে যাচাই করুন ৩৩১
৯.১৮ MCQ-এর উত্তর ও ব্যাখ্যা ৩৩৩
৯.১৯ অধ্যায়ের সারসংক্ষেপ ৩৩৪
৯.২০ গুরুত্বপূর্ণ পরিভাষা ৩৩৪
৯.২১ আরও পড়ুন ৩৩৫

অধ্যায় ১০ Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop ৩৩৬

শেখার লক্ষ্য ৩৩৬
পূর্বশর্ত ৩৩৭
এই অধ্যায় কেন গুরুত্বপূর্ণ ৩৩৭
১০.১ Synchronous এবং Asynchronous ৩৩৭
১০.২ Event Loop: ভেতরের নিয়ম ৩৩৮
১০.৩ Callback: প্রথম পদ্ধতি ৩৪০
১০.৪ Promise: ভবিষ্যতের ফলের প্রতিশ্রুতি ৩৪১
১০.৫ async এবং await ৩৪৩
১০.৬ fetch: Server থেকে তথ্য আনা ৩৪৪
১০.৭ একাধিক কাজ: একটির পর একটি, নাকি একসঙ্গে ৩৪৬
১০.৮ বাতিল করা, সময়সীমা এবং Race Condition ৩৪৮
১০.৯ Retry, Timeout এবং Backoff ৩৫০
১০.১০ সাধারণ ভুল ৩৫১
১০.১১ Debugging নির্দেশিকা ৩৫৩
১০.১২ পেশাদার অভ্যাস ৩৫৩
১০.১৩ Project: User Directory API Explorer ৩৫৪
১০.১৪ অনুশীলনী ৩৬৩
১০.১৫ অনুশীলনীর সমাধান ৩৬৪
১০.১৬ MCQ: নিজেকে যাচাই করুন ৩৭২
১০.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৩৭৪
১০.১৮ অধ্যায়ের সারসংক্ষেপ ৩৭৫
১০.১৯ গুরুত্বপূর্ণ পরিভাষা ৩৭৫
১০.২০ আরও পড়ুন ৩৭৬

অধ্যায় ১১ Error Handling, Debugging এবং Defensive Programming ৩৭৭
শেখার লক্ষ্য ৩৭৭
পূর্বশর্ত ৩৭৮
এই অধ্যায় কেন গুরুত্বপূর্ণ ৩৭৮
১১.১ তিন ধরনের ত্রুটি ৩৭৮
১১.২ Error Object ৩৭৯
১১.৩ throw, try, catch এবং finally ৩৮০
১১.৪ নিজের Error তৈরি ৩৮২
১১.৫ প্রত্যাশিত বনাম অপ্রত্যাশিত ত্রুটি ৩৮৪
১১.৬ Defensive Programming: Trust Boundary ৩৮৫
১১.৭ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা ৩৮৭
১১.৮ Stack Trace পড়া এবং DevTools ৩৮৭
১১.৯ সাধারণ ভুল ৩৮৯
১১.১০ Debugging নির্দেশিকা ৩৮৯
১১.১১ পেশাদার অভ্যাস ৩৯০
১১.১২ Project: OrderGuard — Order Validation ৩৯০
১১.১৩ অনুশীলনী ৩৯৯
১১.১৪ অনুশীলনীর সমাধান ৪০০
১১.১৫ MCQ: নিজেকে যাচাই করুন ৪০৮
১১.১৬ MCQ-এর উত্তর ও ব্যাখ্যা ৪১০
১১.১৭ অধ্যায়ের সারসংক্ষেপ ৪১০
১১.১৮ গুরুত্বপূর্ণ পরিভাষা ৪১১
১১.১৯ আরও পড়ুন ৪১২

অধ্যায় ১২ Modern JavaScript (ES6+), Modules এবং Clean Code Practices ৪১৩
শেখার লক্ষ্য ৪১৩
পূর্বশর্ত ৪১৪
এই অধ্যায় কেন গুরুত্বপূর্ণ ৪১৪
১২.১ Modern JavaScript: কী বদলেছে ৪১৪
১২.২ কয়েকটি নতুন সুবিধা ৪১৫
১২.৩ Module কেন: Global Scope-এর সমস্যা ৪১৬
১২.৪ export এবং import ৪১৭
১২.৫ Module-এর নিয়ম ৪১৯
১২.৬ Browser-এ Module ৪২১
১২.৭ Node.js-এ Module ৪২২
১২.৮ Clean Code: পাঠযোগ্যতা আগে ৪২৩
১২.৯ Module সাজানো: Pure Core, Side Effect সীমানায় ৪২৫
১২.১০ সাধারণ ভুল ৪২৬
১২.১১ Debugging নির্দেশিকা ৪২৬
১২.১২ পেশাদার অভ্যাস ৪২৭
১২.১৩ Project: Modular Shopping Cart ৪২৭
১২.১৪ অনুশীলনী ৪৩৮
১২.১৫ অনুশীলনীর সমাধান ৪৩৯
১২.১৬ MCQ: নিজেকে যাচাই করুন ৪৪৫
১২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৪৪৬
১২.১৮ অধ্যায়ের সারসংক্ষেপ ৪৪৭
১২.১৯ গুরুত্বপূর্ণ পরিভাষা ৪৪৮
১২.২০ আরও পড়ুন ৪৪৮

অধ্যায় ১৩ Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation ৪৪৯
শেখার লক্ষ্য ৪৪৯
পূর্বশর্ত ৪৫০
এই অধ্যায় কেন গুরুত্বপূর্ণ ৪৫০

১৩.১ Factory Function থেকে Class ৪৫০
১৩.২ Class-এর গঠন ৪৫১
১৩.৩ Prototype Chain: Class-এর ভেতরে কী ঘটে ৪৫২
১৩.৪ Encapsulation: Private Field ৪৫৫
১৩.৫ Getter, Setter এবং Static ৪৫৬
১৩.৬ Inheritance: extends এবং super ৪৫৮
১৩.৭ Polymorphism: একই প্রশ্ন, ভিন্ন উত্তর ৪৬০
১৩.৮ this হারানো: Method আলাদা করে পাঠালে ৪৬২
১৩.৯ Composition বনাম Inheritance ৪৬৩
১৩.১০ সাধারণ ভুল ৪৬৫
১৩.১১ Debugging নির্দেশিকা ৪৬৫
১৩.১২ পেশাদার অভ্যাস ৪৬৬
১৩.১৩ Project: Booking Management Lab ৪৬৬
১৩.১৪ অনুশীলনী ৪৭৫
১৩.১৫ অনুশীলনীর সমাধান ৪৭৬
১৩.১৬ MCQ: নিজেকে যাচাই করুন ৪৮৪
১৩.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৪৮৬
১৩.১৮ অধ্যায়ের সারসংক্ষেপ ৪৮৬
১৩.১৯ গুরুত্বপূর্ণ পরিভাষা ৪৮৭
১৩.২০ আরও পড়ুন ৪৮৭

অধ্যায় ১৪ Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming ৪৮৯
শেখার লক্ষ্য ৪৮৯
পূর্বশর্ত ৪৯০
এই অধ্যায় কেন গুরুত্বপূর্ণ ৪৯০
১৪.১ Function একটি মান ৪৯০
১৪.২ Closure: মনে রাখা অবস্থা ৪৯১
১৪.৩ Higher-Order Function: Function-কে মোড়ানো ৪৯২
১৪.৪ call, apply এবং bind ৪৯৪
১৪.৫ Function Composition: pipe এবং compose ৪৯৪
১৪.৬ Currying এবং Partial Application ৪৯৫
১৪.৭ Pure Function, Immutability এবং Side Effect ৪৯৬
১৪.৮ Memoization ৪৯৮
১৪.৯ Debounce এবং Throttle ৪৯৯
১৪.১০ সাধারণ ভুল ৫০১
১৪.১১ Debugging নির্দেশিকা ৫০১
১৪.১২ পেশাদার অভ্যাস ৫০২
১৪.১৩ Project: Sales Function Lab ৫০২
১৪.১৪ অনুশীলনী ৫০৯
১৪.১৫ অনুশীলনীর সমাধান ৫১০
১৪.১৬ MCQ: নিজেকে যাচাই করুন ৫১৬
১৪.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৫১৮
১৪.১৮ অধ্যায়ের সারসংক্ষেপ ৫১৮
১৪.১৯ গুরুত্বপূর্ণ পরিভাষা ৫১৯
১৪.২০ আরও পড়ুন ৫১৯

অধ্যায় ১৫ JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications ৫২০
শেখার লক্ষ্য ৫২০
পূর্বশর্ত ৫২১
এই অধ্যায় কেন গুরুত্বপূর্ণ ৫২১
১৫.১ ভাষা এবং পরিবেশ ৫২১
১৫.২ URL এবং URLSearchParams ৫২২
১৫.৩ Web Storage: localStorage এবং sessionStorage ৫২৪

১৫.৪ সংরক্ষিত তথ্যের Version এবং Migration ৫২৭
১৫.৫ fetch, Request এবং Response ৫২৮
১৫.৬ History API: অবস্থা URL-এ ৫৩০
১৫.৭ Blob, File এবং Download ৫৩১
১৫.৮ নিরাপত্তা ও গোপনীয়তা ৫৩২
১৫.৯ সাধারণ ভুল ৫৩২
১৫.১০ Debugging নির্দেশিকা ৫৩৩
১৫.১১ পেশাদার অভ্যাস ৫৩৩
১৫.১২ Project: Bookmark Vault ৫৩৪
১৫.১৩ অনুশীলনী ৫৪৭
১৫.১৪ অনুশীলনীর সমাধান ৫৪৮
১৫.১৫ MCQ: নিজেকে যাচাই করুন ৫৫৩
১৫.১৬ MCQ-এর উত্তর ও ব্যাখ্যা ৫৫৪
১৫.১৭ অধ্যায়ের সারসংক্ষেপ ৫৫৫
১৫.১৮ গুরুত্বপূর্ণ পরিভাষা ৫৫৬
১৫.১৯ আরও পড়ুন ৫৫৬

অধ্যায় ১৬ JavaScript Error Handling, Debugging, Testing এবং Software Reliability ৫৫৭
শেখার লক্ষ্য ৫৫৭
পূর্বশর্ত ৫৫৮
এই অধ্যায় কেন গুরুত্বপূর্ণ ৫৫৮
১৬.১ Software Reliability কী ৫৫৮
১৬.২ Error-এর শ্রেণিবিন্যাস: code, retryable এবং cause ৫৫৯
১৬.৩ Invariant: ভেতরের নিয়মের পাহারা ৫৬০
১৬.৪ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা ৫৬১
১৬.৫ Debugging: একটি পুনরাবৃত্তিযোগ্য পদ্ধতি ৫৬২
১৬.৬ Test-এর ধরন এবং Testing Pyramid ৫৬৩
১৬.৭ node:test গভীরভাবে ৫৬৪
১৬.৮ Test Double: Stub, Spy, Mock এবং Fake ৫৬৬
১৬.৯ ভালো Test-এর বৈশিষ্ট্য ৫৬৭
১৬.১০ সাধারণ ভুল ৫৬৮
১৬.১১ Debugging নির্দেশিকা ৫৬৮
১৬.১২ পেশাদার অভ্যাস ৫৬৮
১৬.১৩ Project: BookingGuard ৫৬৯
১৬.১৪ অনুশীলনী ৫৭৭
১৬.১৫ অনুশীলনীর সমাধান ৫৭৮
১৬.১৬ MCQ: নিজেকে যাচাই করুন ৫৮৫
১৬.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৫৮৭
১৬.১৮ অধ্যায়ের সারসংক্ষেপ ৫৮৭
১৬.১৯ গুরুত্বপূর্ণ পরিভাষা ৫৮৮
১৬.২০ আরও পড়ুন ৫৮৮

অধ্যায় ১৭ JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing ৫৮৯
শেখার লক্ষ্য ৫৮৯
পূর্বশর্ত ৫৯০
এই অধ্যায় কেন গুরুত্বপূর্ণ ৫৯০
১৭.১ আগে যা শিখেছেন, এখন যা নতুন ৫৯০
১৭.২ Character Class, Quantifier এবং সীমানা ৫৯১
১৭.৩ Group, Backreference এবং Alternation ৫৯২
১৭.৪ Lookahead এবং Lookbehind ৫৯৩
১৭.৫ Regex-এর Method এবং Flag-এর গভীরে ৫৯৪
১৭.৬ Unicode এবং বাংলা ৫৯৬
১৭.৭ Normalize, Validate, Parse: একটি ক্রম ৫৯৭
১৭.৮ Email, Phone এবং Password: Regex-এর সীমা ৫৯৮

১৭.৯ ReDoS: যখন একটি Regex Server আটকে দেয় ৫৯৯
১৭.১০ নিরাপদ Input প্রক্রিয়া: Regex কোথায় থামবে ৬০১
১৭.১১ সাধারণ ভুল ৬০১
১৭.১২ Debugging নির্দেশিকা ৬০২
১৭.১৩ পেশাদার অভ্যাস ৬০২
১৭.১৪ Project: FormShield ৬০২
১৭.১৫ অনুশীলনী ৬০৯
১৭.১৬ অনুশীলনীর সমাধান ৬১০
১৭.১৭ MCQ: নিজেকে যাচাই করুন ৬১৫
১৭.১৮ MCQ-এর উত্তর ও ব্যাখ্যা ৬১৭
১৭.১৯ অধ্যায়ের সারসংক্ষেপ ৬১৭
১৭.২০ গুরুত্বপূর্ণ পরিভাষা ৬১৮
১৭.২১ আরও পড়ুন ৬১৮

অধ্যায় ১৮ Functional Programming, Immutability এবং Advanced Data Transformation ৬২০
শেখার লক্ষ্য ৬২০
পূর্বশর্ত ৬২১
এই অধ্যায় কেন গুরুত্বপূর্ণ ৬২১
১৮.১ আগে যা শিখেছেন, এখন যা নতুন ৬২১
১৮.২ Referential Transparency ৬২২
১৮.৩ Nested তথ্যের Immutable হালনাগাদ ৬২২
১৮.৪ flatMap এবং reduce-এর গভীরে ৬২৬
১৮.৫ Grouping, Aggregation এবং Pivot ৬২৮
১৮.৬ Join: তালিকা জোড়া দেওয়া ৬২৯
১৮.৭ Sorting: Stability, বহু Key এবং বাংলা ৬৩০
১৮.৮ Result Type: ভুলসহ তথ্যের প্রবাহ ৬৩১
১৮.৯ Asynchronous Pipeline ৬৩২
১৮.১০ সাধারণ ভুল ৬৩৩
১৮.১১ Debugging নির্দেশিকা ৬৩৪
১৮.১২ পেশাদার অভ্যাস ৬৩৪
১৮.১৩ Project: LedgerFlow ৬৩৫
১৮.১৪ অনুশীলনী ৬৪৩
১৮.১৫ অনুশীলনীর সমাধান ৬৪৪
১৮.১৬ MCQ: নিজেকে যাচাই করুন ৬৪৯
১৮.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৬৫১
১৮.১৮ অধ্যায়ের সারসংক্ষেপ ৬৫১
১৮.১৯ গুরুত্বপূর্ণ পরিভাষা ৬৫২
১৮.২০ আরও পড়ুন ৬৫২

অধ্যায় ১৯ JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency ৬৫৪
শেখার লক্ষ্য ৬৫৪
পূর্বশর্ত ৬৫৫
এই অধ্যায় কেন গুরুত্বপূর্ণ ৬৫৫
১৯.১ আগে যা শিখেছেন, এখন যা নতুন ৬৫৫
১৯.২ Performance-এর মাত্রা ৬৫৬
১৯.৩ Big-O: বৃদ্ধির হার ৬৫৬
১৯.৪ নির্ভরযোগ্যভাবে মাপা ৬৫৭
১৯.৫ Main Thread: দীর্ঘ কাজ টুকরোয় ভাগ ৬৫৮
১৯.৬ DOM: হালনাগাদ একত্রে, Layout Thrashing নয় ৬৬০
১৯.৭ Memory এবং Garbage Collection ৬৬১
১৯.৮ Cache: LRU, মেয়াদ এবং ঝুঁকি ৬৬২
১৯.৯ Network: Byte-ই সবচেয়ে ধীর ৬৬৩
১৯.১০ সাধারণ ভুল ৬৬৩

১৯.১১ Debugging নির্দেশিকা ৬৬৪
১৯.১২ পেশাদার অভ্যাস ৬৬৪
১৯.১৩ Project: PerfLab ৬৬৫
১৯.১৪ অনুশীলনী ৬৭২
১৯.১৫ অনুশীলনীর সমাধান ৬৭৩
১৯.১৬ MCQ: নিজেকে যাচাই করুন ৬৭৯
১৯.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৬৮১
১৯.১৮ অধ্যায়ের সারসংক্ষেপ ৬৮১
১৯.১৯ গুরুত্বপূর্ণ পরিভাষা ৬৮২
১৯.২০ আরও পড়ুন ৬৮২

অধ্যায় ২০ JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices ৬৮৩
শেখার লক্ষ্য ৬৮৩
পূর্বশর্ত ৬৮৪
এই অধ্যায় কেন গুরুত্বপূর্ণ ৬৮৪
২০.১ আগে যা শিখেছেন, এখন যা নতুন ৬৮৪
২০.২ Threat Model ও Trust Boundary ৬৮৫
২০.৩ XSS: অন্যের Browser-এ আপনার Page দিয়ে Script ৬৮৫
২০.৪ Input Validation বনাম Output Encoding ৬৮৭
২০.৫ SQL Injection ও Parameterized Query ৬৮৮
২০.৬ Authentication বনাম Authorization ৬৮৯
২০.৭ Password: Hash, Salt এবং ধ্রুব-সময়ের তুলনা ৬৯০
২০.৮ Session, Cookie এবং CSRF ৬৯১
২০.৯ CORS, Same-Origin Policy এবং Content Security Policy ৬৯২
২০.১০ Prototype Pollution ৬৯৩
২০.১১ Secret, Dependency এবং Log ৬৯৫
২০.১২ Project: SecureTrip ৬৯৬
২০.১৩ সাধারণ ভুল ৭১২
২০.১৪ Debugging নির্দেশিকা ৭১২
২০.১৫ পেশাদার অভ্যাস ৭১৩
২০.১৬ অনুশীলনী ৭১৩
২০.১৭ অনুশীলনীর সমাধান ৭১৪
২০.১৮ MCQ: নিজেকে যাচাই করুন ৭২০
২০.১৯ MCQ-এর উত্তর ও ব্যাখ্যা ৭২১
২০.২০ অধ্যায়ের সারসংক্ষেপ ৭২২
২০.২১ গুরুত্বপূর্ণ পরিভাষা ৭২৩
২০.২২ আরও পড়ুন ৭২৩

অধ্যায় ২১ JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure ৭২৪
শেখার লক্ষ্য ৭২৪
পূর্বশর্ত ৭২৫
এই অধ্যায় কেন গুরুত্বপূর্ণ ৭২৫
২১.১ আগে যা শিখেছেন, এখন যা নতুন ৭২৫
২১.২ Design Pattern বনাম Architecture ৭২৬
২১.৩ Factory: তৈরির নিয়ম এক জায়গায় ৭২৬
২১.৪ Strategy: if/else-এর শিকলের বদলে বিনিময়যোগ্য নিয়ম ৭২৭
২১.৫ Observer: Publisher জানে না কে শুনছে ৭২৮
২১.৬ Repository, Adapter এবং Dependency Injection ৭২৯
২১.৭ Module-Singleton: সুবিধা ও ঝুঁকি ৭৩০
২১.৮ State Machine: Workflow-কে তথ্য হিসেবে ৭৩১
২১.৯ Layered Architecture এবং নির্ভরতার দিক ৭৩১
২১.১০ Project: VisaFlow ৭৩২
২১.১১ সাধারণ ভুল ৭৪৬

২১.১২ Debugging নির্দেশিকা ৭৪৬
২১.১৩ পেশাদার অভ্যাস ৭৪৬
২১.১৪ অনুশীলনী ৭৪৭
২১.১৫ অনুশীলনীর সমাধান ৭৪৮
২১.১৬ MCQ: নিজেকে যাচাই করুন ৭৫৫
২১.১৭ MCQ-এর উত্তর ও ব্যাখ্যা ৭৫৭
২১.১৮ অধ্যায়ের সারসংক্ষেপ ৭৫৭
২১.১৯ গুরুত্বপূর্ণ পরিভাষা ৭৫৮
২১.২০ আরও পড়ুন ৭৫৮

অধ্যায় ২২ JavaScript Testing Mastery: Unit Testing, Integration Testing, Mocking, TDD এবং E2E ৭৫৯
শেখার লক্ষ্য ৭৫৯
পূর্বশর্ত ৭৬০
এই অধ্যায় কেন গুরুত্বপূর্ণ ৭৬০
২২.১ আগে যা শিখেছেন, এখন যা নতুন ৭৬০
২২.২ কোন স্তরে কী যাচাই করবেন ৭৬০
২২.৩ Arrange–Act–Assert এবং Table-driven Test ৭৬১
২২.৪ Equivalence Partitioning এবং Boundary Value Analysis ৭৬২
২২.৫ Property-based Testing ৭৬৩
২২.৬ Test-Driven Development: Red, Green, Refactor ৭৬৪
২২.৭ node:test-এর Mock: Function, Method ও সময় ৭৬৬
২২.৮ Integration Test: আসল HTTP, নিয়ন্ত্রিত পৃথিবী ৭৬৭
২২.৯ Async Test, Cleanup এবং Flaky Test ৭৬৭
২২.১০ Coverage: কী বলে, কী বলে না ৭৬৮
২২.১১ Mutation Testing: Test কি আসলেই ত্রুটি ধরে? ৭৬৮
২২.১২ End-to-end: আসল Browser-এ ৭৭০
২২.১৩ Project: CurrencyDesk ৭৭১
২২.১৪ সাধারণ ভুল ৭৮৪
২২.১৫ Debugging নির্দেশিকা ৭৮৪
২২.১৬ পেশাদার অভ্যাস ৭৮৫
২২.১৭ অনুশীলনী ৭৮৫
২২.১৮ অনুশীলনীর সমাধান ৭৮৬
২২.১৯ MCQ: নিজেকে যাচাই করুন ৭৯৩
২২.২০ MCQ-এর উত্তর ও ব্যাখ্যা ৭৯৫
২২.২১ অধ্যায়ের সারসংক্ষেপ ৭৯৫
২২.২২ গুরুত্বপূর্ণ পরিভাষা ৭৯৬
২২.২৩ আরও পড়ুন ৭৯৭

অধ্যায় ২৩ JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance ৭৯৮
শেখার লক্ষ্য ৭৯৮
পূর্বশর্ত ৭৯৯
এই অধ্যায় কেন গুরুত্বপূর্ণ ৭৯৯
২৩.১ আগে যা শিখেছেন, এখন যা নতুন ৭৯৯
২৩.২ মাপার বিন্দু: User Timing API ৮০০
২৩.৩ CPU Profile: সময় আসলে কোথায় যায় ৮০১
২৩.৪ Event Loop Delay এবং Memory ৮০২
২৩.৫ Browser Rendering Pipeline ৮০৪
২৩.৬ Virtual Scrolling ৮০৬
২৩.৭ Web Worker: Main Thread মুক্ত রাখা ৮০৬
২৩.৮ Project: TravelSearch Explorer ৮০৮
২৩.৯ সাধারণ ভুল ৮১৯
২৩.১০ Debugging নির্দেশিকা ৮১৯
২৩.১১ পেশাদার অভ্যাস ৮২০

২৩.১২ অনুশীলনী ৮২০
২৩.১৩ অনুশীলনীর সমাধান ৮২১
২৩.১৪ MCQ: নিজেকে যাচাই করুন ৮২৬
২৩.১৫ MCQ-এর উত্তর ও ব্যাখ্যা ৮২৮
২৩.১৬ অধ্যায়ের সারসংক্ষেপ ৮২৯
২৩.১৭ গুরুত্বপূর্ণ পরিভাষা ৮২৯
২৩.১৮ আরও পড়ুন ৮৩০

অধ্যায় ২৪ JavaScript থেকে TypeScript: Static Typing, Type Safety এবং Professional Development Workflow ৮৩১

শেখার লক্ষ্য ৮৩১
পূর্বশর্ত ৮৩২
এই অধ্যায় কেন গুরুত্বপূর্ণ ৮৩২
২৪.১ আগে যা শিখেছেন, এখন যা নতুন ৮৩২
২৪.২ Dynamic Typing-এর একটি চেনা ভুল, এবং TypeScript-এর উত্তর ৮৩৩
২৪.৩ Project-এর কাঠামো এবং `tsconfig.json` ৮৩৪
২৪.৪ Type Inference, Literal Type এবং Union ৮৩৫
২৪.৫ null, undefined এবং Strict Mode ৮৩৬
২৪.৬ unknown বনাম any: Trust Boundary ৮৩৭
২৪.৭ Discriminated Union, `Result` এবং Exhaustiveness ৮৩৯
২৪.৮ Generics: একবার লেখা, অনেক Type-এ ৮৪০
২৪.৯ `keyof` এবং Utility Type ৮৪১
২৪.১০ Branded Type: একই Type, ভিন্ন অর্থ ৮৪২
২৪.১১ পেশাদার Workflow: JavaScript থেকে ধাপে ধাপে TypeScript ৮৪৩
২৪.১২ Project: TypeSafe Claims ৮৪৫
২৪.১৩ সাধারণ ভুল ৮৫৮
২৪.১৪ Debugging নির্দেশিকা ৮৫৯
২৪.১৫ পেশাদার অভ্যাস ৮৫৯
২৪.১৬ অনুশীলনী ৮৫৯
২৪.১৭ অনুশীলনীর সমাধান ৮৬০
২৪.১৮ MCQ: নিজেকে যাচাই করুন ৮৬৬
২৪.১৯ MCQ-এর উত্তর ও ব্যাখ্যা ৮৬৭
২৪.২০ অধ্যায়ের সারসংক্ষেপ ৮৬৮
২৪.২১ গুরুত্বপূর্ণ পরিভাষা ৮৬৮
২৪.২২ আরও পড়ুন ৮৬৯

অধ্যায় ২৫ JavaScript Backend Development with Node.js: REST APIs, Express, Databases, Authentication এবং Production Architecture ৮৭০

শেখার লক্ষ্য ৮৭০
পূর্বশর্ত ৮৭১
এই অধ্যায় কেন গুরুত্বপূর্ণ ৮৭১
২৫.১ আগে যা শিখেছেন, এখন যা নতুন ৮৭১
২৫.২ Browser JavaScript বনাম Node.js Server ৮৭২
২৫.৩ REST: Resource, Method ও Status Code ৮৭২
২৫.৪ Express: Route ও Middleware ৮৭৩
২৫.৫ Error Handling এবং Express 4-এর Async ফাঁদ ৮৭৫
২৫.৬ Layered Architecture: Route, Service, Repository ৮৭৬
২৫.৭ Authentication ও Authorization: Token ও Role ৮৭৭
২৫.৮ নির্ভরযোগ্যতা: Pagination, Idempotency, Concurrency ৮৭৮
২৫.৯ Observability এবং Production-এর অভ্যাস ৮৭৮
২৫.১০ Database: Repository-র পেছনে ৮৭৯
২৫.১১ Project: LeadDesk API ৮৭৯
২৫.১২ সাধারণ ভুল ৮৯৪
২৫.১৩ Debugging নির্দেশিকা ৮৯৫

২৫.১৪ পেশাদার অভ্যাস ৮৯৫
২৫.১৫ অনুশীলনী ৮৯৫
২৫.১৬ অনুশীলনীর সমাধান ৮৯৬
২৫.১৭ MCQ: নিজেকে যাচাই করুন ৯০৩
২৫.১৮ MCQ-এর উত্তর ও ব্যাখ্যা ৯০৪
২৫.১৯ অধ্যায়ের সারসংক্ষেপ ৯০৫
২৫.২০ গুরুত্বপূর্ণ পরিভাষা ৯০৫
২৫.২১ আরও পড়ুন ৯০৬

অধ্যায় ২৬ Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture ৯০৭

শেখার লক্ষ্য ৯০৭
পূর্বশর্ত ৯০৮
এই অধ্যায় কেন গুরুত্বপূর্ণ ৯০৮
২৬.১ আগে যা শিখেছেন, এখন যা নতুন ৯০৮
২৬.২ Relational Model এবং Normalization ৯০৯
২৬.৩ Constraint: Database নিজেই নিয়ম পাহারা দেয় ৯১০
২৬.৪ Transaction: সব, নয়তো কিছুই নয় ৯১১
২৬.৫ Isolation, Lock এবং Lost Update ৯১৩
২৬.৬ Index, Query Plan এবং N+1 ৯১৪
২৬.৭ Pagination, Connection Pool ও Cache ৯১৬
২৬.৮ Migration ও Backup ৯১৭
২৬.৯ SQLite থেকে PostgreSQL, এবং ORM ৯১৭
২৬.১০ Security ও Observability ৯১৯
২৬.১১ Project: TravelData Pro ৯১৯
২৬.১২ সাধারণ ভুল ৯৩২
২৬.১৩ Debugging নির্দেশিকা ৯৩৩
২৬.১৪ পেশাদার অভ্যাস ৯৩৩
২৬.১৫ অনুশীলনী ৯৩৩
২৬.১৬ অনুশীলনীর সমাধান ৯৩৪
২৬.১৭ MCQ: নিজেকে যাচাই করুন ৯৩৯
২৬.১৮ MCQ-এর উত্তর ও ব্যাখ্যা ৯৪১
২৬.১৯ অধ্যায়ের সারসংক্ষেপ ৯৪২
২৬.২০ গুরুত্বপূর্ণ পরিভাষা ৯৪২
২৬.২১ আরও পড়ুন ৯৪৩

অধ্যায় ২৭ Real-Time JavaScript Applications ৯৪৪

শেখার লক্ষ্য ৯৪৪
পূর্বশর্ত ৯৪৫
এই অধ্যায় কেন গুরুত্বপূর্ণ ৯৪৫
২৭.১ আগে যা শিখেছেন, এখন যা নতুন ৯৪৫
২৭.২ Real-Time-এর চারটি পথ ৯৪৬
২৭.৩ Server-Sent Events ৯৪৭
২৭.৪ WebSocket: Handshake ও Lifecycle ৯৪৯
২৭.৫ Socket.IO: Event, Room ও Acknowledgement ৯৫০
২৭.৬ Event Contract ও Versioning ৯৫২
২৭.৭ Event-Driven Architecture ও Transactional Outbox ৯৫৪
২৭.৮ Delivery Semantics, Ordering ও Duplicate ৯৫৬
২৭.৯ Reconnect, Heartbeat ও Backpressure ৯৫৮
২৭.১০ একাধিক Server: Pub/Sub ও Scaling ৯৬০
২৭.১১ নিরাপত্তা Checklist ৯৬২
২৭.১২ Project: TravelLive ৯৬৩
২৭.১৩ সাধারণ ভুল ৯৮১
২৭.১৪ Debugging নির্দেশিকা ৯৮২

২৭.১৫ পেশাদার অভ্যাস ৯৮২
২৭.১৬ অনুশীলনী ৯৮৩
২৭.১৭ অনুশীলনীর সমাধান ৯৮৪
২৭.১৮ MCQ: নিজেকে যাচাই করুন ৯৯১
২৭.১৯ MCQ-এর উত্তর ও ব্যাখ্যা ৯৯৩
২৭.২০ অধ্যায়ের সারসংক্ষেপ ৯৯৪
২৭.২১ গুরুত্বপূর্ণ পরিভাষা ৯৯৪
২৭.২২ আরও পড়ুন ৯৯৫

অধ্যায় ২৮ Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring ৯৯৬

শেখার লক্ষ্য ৯৯৬
পূর্বশর্ত ৯৯৭
এই অধ্যায় কেন গুরুত্বপূর্ণ ৯৯৭
২৮.১ আগে যা শিখেছেন, এখন যা নতুন ৯৯৭
২৮.২ Localhost থেকে Production: কী বদলায় ৯৯৮
২৮.৩ Twelve-Factor ও Configuration ৯৯৮
২৮.৪ Build Artifact, Lockfile ও পুনরুৎপাদনযোগ্যতা ১০০১
২৮.৫ Docker: Image, Container ও Dockerfile ১০০১
২৮.৬ Docker Compose: Local-এ Production-এর অনুকরণ ১০০৪
২৮.৭ CI: প্রতিটি পরিবর্তন যাচাই ১০০৪
২৮.৮ CD: Staging, Rollout ও Rollback ১০০৭
২৮.৯ Liveness, Readiness ও Graceful Shutdown ১০০৯
২৮.১০ Logs, Metrics ও Traces ১০১১
২৮.১১ SLI, SLO ও Alerting ১০১৫
২৮.১২ Secret, Supply Chain ও Infrastructure-এর নিরাপত্তা ১০১৬
২৮.১৩ Backup, Disaster Recovery ও Incident Response ১০১৭
২৮.১৪ Project: DeployReady Travel Operations API ১০১৭
২৮.১৫ সাধারণ ভুল ১০২৬
২৮.১৬ Debugging নির্দেশিকা ১০২৭
২৮.১৭ পেশাদার অভ্যাস ১০২৭
২৮.১৮ অনুশীলনী ১০২৮
২৮.১৯ অনুশীলনীর সমাধান ১০২৮
২৮.২০ MCQ: নিজেকে যাচাই করুন ১০৩৪
২৮.২১ MCQ-এর উত্তর ও ব্যাখ্যা ১০৩৬
২৮.২২ অধ্যায়ের সারসংক্ষেপ ১০৩৬
২৮.২৩ গুরুত্বপূর্ণ পরিভাষা ১০৩৭
২৮.২৪ আরও পড়ুন ১০৩৮

অধ্যায় ২৯ Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture ১০৩৯

শেখার লক্ষ্য ১০৩৯
পূর্বশর্ত ১০৪০
এই অধ্যায় কেন গুরুত্বপূর্ণ ১০৪০
২৯.১ আগে যা শিখেছেন, এখন যা নতুন ১০৪০
২৯.২ System Design আসলে কী ১০৪১
২৯.৩ Capacity Planning: অনুমান নয়, হিসাব ১০৪২
২৯.৪ Monolith, Modular Monolith ও Microservices ১০৪২
২৯.৫ Domain-Driven Design ও Bounded Context ১০৪৪
২৯.৬ Synchronous বনাম Asynchronous যোগাযোগ ১০৪৫
২৯.৭ Distributed Transaction ও Saga ১০৪৫
২৯.৮ Idempotency: Key ও Fingerprint ১০৪৭
২৯.৯ Consistency, CAP ও Data-র মালিকানা ১০৪৮
২৯.১০ নির্ভরযোগ্যতার Pattern ১০৪৯

২৯.১১ পর্যবেক্ষণ ও নিরাপত্তা ১০৫২
২৯.১২ Project: TravelSaga Booking Orchestrator ১০৫২
২৯.১৩ Failure Lab ১০৬৫
২৯.১৪ সাধারণ ভুল ১০৬৫
২৯.১৫ Debugging নির্দেশিকা ১০৬৬
২৯.১৬ পেশাদার অভ্যাস ১০৬৬
২৯.১৭ অনুশীলনী ১০৬৭
২৯.১৮ অনুশীলনীর সমাধান ১০৬৮
২৯.১৯ MCQ: নিজেকে যাচাই করুন ১০৭৫
২৯.২০ MCQ-এর উত্তর ও ব্যাখ্যা ১০৭৬
২৯.২১ অধ্যায়ের সারসংক্ষেপ ১০৭৭
২৯.২২ গুরুত্বপূর্ণ পরিভাষা ১০৭৮
২৯.২৩ আরও পড়ুন ১০৭৮

অধ্যায় ৩০ JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap ১০৭৯
শেখার লক্ষ্য ১০৭৯
পূর্বশর্ত ১০৮০
এই অধ্যায় কেন গুরুত্বপূর্ণ ১০৮০
৩০.১ JavaScript Mastery মানে কী ১০৮০
৩০.২ Capstone: TravelMaster Pro-র Requirement ১০৮১
৩০.৩ Architecture: যা বানানো হয়েছে, আর যা লক্ষ্য ১০৮২
৩০.৪ মূল সিদ্ধান্ত ১: Trust Boundary-তে যাচাই ১০৮৩
৩০.৫ মূল সিদ্ধান্ত ২: Password ও Session ১০৮৫
৩০.৬ মূল সিদ্ধান্ত ৩: Database, দাম, Idempotency ও Inventory ১০৮৭
৩০.৭ Transport: src/http.mjs ১০৯৫
৩০.৮ Frontend: public/app.mjs ১০৯৯
৩০.৯ Test-এর পিরামিড ও Quality Gate ১১০৩
৩০.১০ Project: TravelMaster Pro চালানো ও প্রমাণ ১১০৫
৩০.১১ Production-এর জন্য উন্নয়ন-পরিকল্পনা ১১০৮
৩০.১২ Architecture Decision Record ১১০৯
৩০.১৩ ১০০ নম্বরের মূল্যায়ন ১১০৯
৩০.১৪ পেশাদার Roadmap: ১২ সপ্তাহ ১১১০
৩০.১৫ সাধারণ ভুল ১১১১
৩০.১৬ Debugging নির্দেশিকা ১১১২
৩০.১৭ পেশাদার অভ্যাস ১১১২
৩০.১৮ অনুশীলনী ১১১৩
৩০.১৯ অনুশীলনীর সমাধান ১১১৩
৩০.২০ MCQ: নিজেকে যাচাই করুন ১১১৭
৩০.২১ MCQ-এর উত্তর ও ব্যাখ্যা ১১১৯
৩০.২২ অধ্যায়ের সারসংক্ষেপ ১১১৯
৩০.২৩ বই শেষ, শেখা নয় ১১২০
৩০.২৪ গুরুত্বপূর্ণ পরিভাষা ১১২০
৩০.২৫ আরও পড়ুন ১১২১

পরিশিষ্ট ক — পরিভাষা-কোষ ১১২২

পরিশিষ্ট খ — Companion Project-এর তালিকা ১১৩৩

পরিশিষ্ট গ — যাচাই-পরিবেশ ১১৩৫

পরিশিষ্ট ঘ — তথ্যসূত্র ও আরও পড়ুন ১১৩৬

অধ্যায় ১

০১

JavaScript-এর জগতে প্রথম পদক্ষেপ

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Programming বলতে কী বোঝায় এবং JavaScript কোন ধরনের কাজে ব্যবহৃত হয়, তা নিজের ভাষায় ব্যাখ্যা করতে।
- JavaScript কোথায় চলে (Browser ও Node.js) এবং Java-র সঙ্গে এর পার্থক্য কী, তা বলতে।
- নিজের Computer-এ Browser, VS Code এবং Node.js দিয়ে একটি কাজের পরিবেশ তৈরি করতে।
- Browser Console এবং Node.js—দুই জায়গাতেই JavaScript Code চালাতে।
- String, Semicolon, Comment এবং Case Sensitivity-র মৌলিক নিয়ম মেনে Code লিখতে।
- একটি Error Message পড়ে ভুলের ধরন ও জায়গা খুঁজে বের করতে।
- একটি HTML Page-এ JavaScript File যুক্ত করে Button Click-এ Page-এর লেখা বদলাতে।
- সম্পূর্ণ একটি ছোট Application—**Interactive Greeting App**—নিজে তৈরি করে চালাতে।

পূর্বশর্ত

এই অধ্যায়ের জন্য আগে থেকে Programming জানার প্রয়োজন নেই। আপনার যা লাগবে:

- একটি Computer (Windows, macOS অথবা Linux) এবং Software Install করার অনুমতি।
- File ও Folder তৈরি করা, নাম বদলানো এবং Keyboard-এ টাইপ করার সাধারণ অভ্যাস।
- Software Download করার জন্য Internet সংযোগ।
- প্রতিটি উদাহরণ নিজে টাইপ করে চালিয়ে দেখার ধৈর্য।

এই অধ্যায় কেন গুরুত্বপূর্ণ

নতুন ভাষা শেখার সময় সবচেয়ে বড় বাধা সাধারণত ভাষাটি নয়; বাধা হলো প্রথম Program-টি চালাতে না পারা। কোথায় Code লিখতে হবে, কীভাবে চালাতে হবে, লাল রঙের Error দেখালে কী করতে হবে—এই প্রশ্নগুলোর উত্তর জানা না থাকলে শেখা শুরুতেই থেমে যায়।

এই অধ্যায়ে তাই তিনটি ভিত্তি তৈরি করা হবে। প্রথমত, আপনি জানবেন JavaScript কী এবং কোথায় ব্যবহৃত হয়। দ্বিতীয়ত, আপনার Computer-এ এমন একটি পরিবেশ প্রস্তুত হবে, যা বইয়ের বাকি সব অধ্যায়ে কাজে লাগবে। তৃতীয়ত, আপনি Code লেখা, চালানো এবং ভুল খুঁজে ঠিক করার পুরো চক্রটি নিজের হাতে একবার সম্পন্ন করবেন।

অধ্যায়ের শেষে আপনার হাতে একটি সম্পূর্ণ কাজ করা Web Application থাকবে। এটি ছোট, কিন্তু এর প্রতিটি লাইন আপনি বুঝবেন।

১.১ Programming কী

Computer নিজে থেকে কিছু করে না; তাকে প্রতিটি কাজ ধাপে ধাপে বলে দিতে হয়। এই ধাপে ধাপে লেখা নির্দেশনার সমষ্টিকে **Program** বলা হয়, আর Program লেখার কাজকে বলা হয় **Programming**।

একটি রান্নার প্রণালির কথা ভাবুন। সেখানে উপকরণ, পরিমাণ এবং কাজের ক্রম নির্দিষ্টভাবে লেখা থাকে। ক্রম উল্টে গেলে বা একটি ধাপ বাদ পড়লে ফল ঠিক আসে না। Program-ও তেমনই: Computer নির্দেশনাগুলো ঠিক যেভাবে লেখা আছে সেভাবেই পালন করে, আপনি কী বোঝাতে চেয়েছিলেন তা অনুমান করে না।

মানুষের ভাষায় নির্দেশ দিলে Computer বোঝে না, কারণ সে ভাষায় একই কথার অনেক অর্থ হতে পারে। তাই নির্দিষ্ট নিয়মে বাঁধা ভাষা ব্যবহার করা হয়, যাকে বলে **Programming Language**। JavaScript তেমনই একটি ভাষা।

১.২ JavaScript কী এবং কেন শিখবেন

JavaScript একটি Programming Language, যা ১৯৯৫ সালে Web Page-কে সচল ও ব্যবহারকারীর কাজে সাড়া দিতে সক্ষম করার জন্য তৈরি হয়েছিল। আজ এটি Web-এর মূল তিনটি প্রযুক্তির একটি:

প্রযুক্তি	কাজ	উদাহরণ
HTML	Page-এর কাঠামো ও বিষয়বস্তু	শিরোনাম, অনুচ্ছেদ, Button, Form
CSS	Page-এর চেহারা	রং, Font, ফাঁকা জায়গা, বিন্যাস
JavaScript	Page-এর আচরণ	Button চাপলে কী হবে, Form ঠিকভাবে পূরণ হয়েছে কি না

একটি বাড়ির সঙ্গে তুলনা করলে HTML হলো দেয়াল ও ঘর, CSS হলো রং ও সাজসজ্জা, আর JavaScript হলো বিদ্যুতের সুইচ—যা চাপলে কিছু একটা ঘটে।

JavaScript কোথায় চলে

JavaScript Code চালানোর জন্য একটি **Runtime** প্রয়োজন, অর্থাৎ এমন একটি Software যা Code পড়ে সেটি কার্যকর করে। সবচেয়ে প্রচলিত দুটি Runtime হলো:

- **Browser:** Chrome, Edge, Firefox, Safari—প্রতিটি Browser-এর ভেতরে JavaScript চালানোর ব্যবস্থা আছে। এখানে JavaScript Web Page-এর সঙ্গে কাজ করে।
- **Node.js:** এটি Browser-এর বাইরে, সরাসরি আপনার Computer বা Server-এ JavaScript চালায়। Node.js দিয়ে Server, Command-line Tool এবং নানা ধরনের স্বয়ংক্রিয় কাজ তৈরি করা যায়।

ভাষা একই, কিন্তু পরিবেশ আলাদা। Browser-এ Web Page থাকে, তাই সেখানে Page-এর Button বা লেখা নিয়ে কাজ করা যায়। Node.js-এ Web Page নেই, তবে File পড়া-লেখা বা Server চালানোর সুবিধা আছে। এই পার্থক্য পুরো বইজুড়ে মনে রাখতে হবে।

JavaScript এবং Java এক নয়

নামের মিল থাকলেও JavaScript এবং Java সম্পূর্ণ আলাদা দুটি ভাষা। এদের নিয়ম, চালানোর পদ্ধতি এবং ব্যবহারের ক্ষেত্র ভিন্ন। নামের এই মিল ঐতিহাসিক; প্রযুক্তিগত কোনো সম্পর্ক এতে বোঝায় না। Java শিখলে JavaScript জানা হয় না, আবার JavaScript শিখলেও Java জানা হয় না।

বাস্তবে JavaScript দিয়ে কী তৈরি হয়

- Online দোকানের Cart, যেখানে পণ্য যোগ করলে মোট দাম সঙ্গে সঙ্গে বদলায়।
- Form, যা ভুল Email বা ফাঁকা ঘর থাকলে জমা দেওয়ার আগেই সতর্ক করে।
- Map, Chat, Video Call এবং Online Editor-এর মতো Browser-ভিত্তিক Application।
- Server-side API, যা Mobile App বা Website-কে Data সরবরাহ করে।
- Developer-দের ব্যবহারের Tool, যেমন Code পরীক্ষা বা Build করার Program।

Frontend, Backend এবং Full-Stack

Web Application সাধারণত দুটি অংশে ভাগ করা হয়।

- **Frontend:** ব্যবহারকারী যা দেখেন এবং যার সঙ্গে সরাসরি কাজ করেন। এটি Browser-এ চলে এবং HTML, CSS ও JavaScript দিয়ে তৈরি।
- **Backend:** পর্দার আড়ালের অংশ, যা Server-এ চলে। Data সংরক্ষণ, হিসাব-নিকাশ এবং নিরাপত্তা যাচাই এখানে হয়।

যিনি দুটো অংশেই কাজ করতে পারেন, তাঁকে **Full-Stack Developer** বলা হয়। JavaScript-এর একটি বড় সুবিধা হলো একই ভাষা দিয়ে Frontend (Browser) এবং Backend (Node.js)—দুই দিকেই কাজ করা যায়।

১.৩ এই বই থেকে কী শিখবেন

বইটি ৩০টি অধ্যায়ে চারটি ধাপে সাজানো:

অধ্যায়	ধাপ	বিষয়
১–৯	মৌলিক ভিত্তি	Variable, Data Type, Operator, Loop, Function, Array, Object, String, DOM ও Event
১০–১৮	আধুনিক JavaScript	Asynchronous Programming, Error Handling, Module, OOP, Browser API, Testing
১৯–২৪	পেশাদার Engineering	Performance, Security, Design Pattern, TypeScript
২৫–৩০	Backend থেকে System Design	Node.js Backend, Database, Deployment এবং পূর্ণাঙ্গ Project

এই অধ্যায়ে কয়েকটি বিষয়—যেমন Variable, Function এবং DOM—শুধু পরিচয় পর্যায়ে আসবে, যাতে আপনি প্রথম Project-টি তৈরি করতে পারেন। প্রতিটি বিষয়ের বিস্তারিত আলোচনা পরের অধ্যায়গুলোতে আছে।

১.৪ Development Environment প্রস্তুত করা

Code লেখা ও চালানোর জন্য যে Software-গুলো লাগে, সেগুলোকে একসঙ্গে **Development Environment** বলা হয়। এই বইয়ের জন্য তিনটি জিনিস প্রয়োজন: একটি আধুনিক Browser, একটি Code Editor এবং Node.js। তিনটিই বিনামূল্যে পাওয়া যায়।

> **সতর্কতা:** Software সব সময় নির্মাতার নিজস্ব Website থেকে Download করুন। অন্য কোনো Website-এর Installer-এ ক্ষতিকর Program থাকতে পারে।

ধাপ ১: একটি আধুনিক Browser

Google Chrome, Microsoft Edge অথবা Mozilla Firefox—যেকোনো একটি হালনাগাদ Browser যথেষ্ট। আপনার Computer-এ এর কোনোটি আগে থেকে থাকলে নতুন করে Install করার দরকার নেই; শুধু সেটি Update করে নিন। এই বইয়ের Screenshot ও বর্ণনায় Chrome ব্যবহার করা হয়েছে, তবে Edge এবং Firefox-এও একই কাজ প্রায় একইভাবে করা যায়।

ধাপ ২: Visual Studio Code

Code যেকোনো সাধারণ Text Editor-এ লেখা যায়, কিন্তু একটি **Code Editor** কাজটি অনেক সহজ করে। এটি Code-এর বিভিন্ন অংশ আলাদা রঙে দেখায়, বন্ধনী মেলাতে সাহায্য করে এবং ভেতরেই Terminal চালানোর সুবিধা দেয়। এই বইয়ে Visual Studio Code (সংক্ষেপে VS Code) ব্যবহার করা হয়েছে।

১. `https://code.visualstudio.com` ঠিকানায় যান।
২. আপনার Operating System-এর জন্য Installer Download করুন।
৩. Installer চালিয়ে নির্দেশনা অনুসরণ করুন। Default Option-গুলো সাধারণত ঠিক থাকে।

> **নোট:** Microsoft Word বা অনুরূপ Word Processor-এ Code লিখবেন না। এগুলো লেখার সঙ্গে অদৃশ্য Formatting যোগ করে এবং Quotation Mark বদলে দেয়, ফলে Code চলে না।

ধাপ ৩: Node.js

১. `https://nodejs.org` ঠিকানায় যান।
২. **LTS** চিহ্নিত সংস্করণটি Download করুন। LTS-এর পূর্ণরূপ Long Term Support; এই সংস্করণ দীর্ঘ সময় ধরে রক্ষণাবেক্ষণ করা হয় এবং শেখার জন্য এটিই উপযুক্ত।
৩. Installer চালিয়ে Default Option রেখে Install সম্পন্ন করুন।

Node.js-এর সঙ্গে **npm** নামে আরেকটি Tool স্বয়ংক্রিয়ভাবে Install হয়। npm দিয়ে অন্যদের তৈরি Code Package ব্যবহার করা যায়। এই অধ্যায়ে npm ব্যবহার করা হবে না, তবে এটি ঠিকভাবে Install হয়েছে কি না তা যাচাই করে রাখা ভালো।

> **নোট:** Node.js-এর সংস্করণ নিয়মিত বদলায়, তাই এই বইয়ে নির্দিষ্ট কোনো সংখ্যা Install করতে বলা হয়নি। Website-এ যেটি LTS হিসেবে চিহ্নিত, সেটিই নিন। এই অধ্যায়ের উদাহরণগুলো লেখকের Computer-এ Node.js v22.16.0 দিয়ে চালিয়ে দেখা হয়েছে।

ধাপ ৪: Terminal খুলে Install যাচাই করা

Terminal হলো এমন একটি জানালা, যেখানে Mouse দিয়ে Click করার বদলে লিখে Computer-কে নির্দেশ দেওয়া হয়। প্রতিটি নির্দেশকে **Command** বলে। VS Code-এর ভেতরেই একটি Terminal আছে:

- Menu থেকে **Terminal > New Terminal** বেছে নিন, অথবা

- Keyboard-এ `Ctrl` এবং `` ` `` (Backtick, সাধারণত `Esc` Key-র নিচে থাকে) একসঙ্গে চাপুন।

Terminal খুললে নিচের দুটি Command আলাদাভাবে লিখে প্রতিবার `Enter` চাপুন।

Terminal

```
node --version
npm --version
```

প্রতিটি Command-এর পর একটি সংস্করণ সংখ্যা দেখা যাবে। লেখকের Computer-এ ফল ছিল এমন:

Output

```
v22.16.0
10.9.2
```

আপনার সংখ্যা ভিন্ন হলে সমস্যা নেই; সংখ্যা দেখানোই প্রমাণ যে Install সফল হয়েছে। যদি `command not found` অথবা `is not recognized` জাতীয় বার্তা আসে, তাহলে VS Code সম্পূর্ণ বন্ধ করে আবার খুলুন। তাতেও কাজ না হলে Computer Restart করে পুনরায় চেষ্টা করুন, কারণ নতুন Install করা Program অনেক সময় Restart-এর পর Terminal-এ পাওয়া যায়।

ধাপ ৫: Project Folder তৈরি করা

বইয়ের সব কাজ একটি নির্দিষ্ট Folder-এ রাখলে পরে খুঁজে পাওয়া সহজ হয়।

১. Documents বা সুবিধাজনক কোনো জায়গায় `javascript-mastery` নামে একটি Folder তৈরি করুন।
২. তার ভেতরে `chapter-01` নামে আরেকটি Folder তৈরি করুন।
৩. VS Code-এ **File > Open Folder** থেকে `chapter-01` Folder-টি খুলুন।

Folder ও File-এর নামে ফাঁকা জায়গা এবং বাংলা অক্ষর ব্যবহার না করাই ভালো। ছোট হাতের ইংরেজি অক্ষর, সংখ্যা এবং হাইফেন (`-`) ব্যবহার করুন, যেমন `chapter-01`। এতে Terminal-এ কাজ করার সময় জটিলতা কমে।

ধাপ ৬: File তৈরি, সংরক্ষণ এবং File Extension

VS Code-এর বাঁ পাশের Explorer অংশে **New File** চিহ্নে Click করে File তৈরি করা যায়। নামের শেষের অংশটিকে **File Extension** বলে; এটি দেখে Computer ও Editor বোঝে File-টি কোন ধরনের।

Extension	File-এর ধরন	উদাহরণ
`.html`	Web Page-এর কাঠামো	`index.html`
`.css`	Web Page-এর চেহারা	`style.css`
`.js`	JavaScript Code	`app.js`

File-এ কিছু লেখার পর `Ctrl + S` (macOS-এ `Cmd + S`) চেপে Save করতে হয়। VS Code-এ File-এর নামের পাশে একটি গোল বিন্দু দেখা গেলে বুঝবেন, পরিবর্তন এখনো Save হয়নি।

> **সতর্কতা:** নতুনদের সবচেয়ে প্রচলিত ভুলগুলোর একটি হলো Save না করে Code চালানো। তখন Computer পুরোনো File-টিই চালায় এবং মনে হয় পরিবর্তন কাজ করছে না।

Windows অনেক সময় File Extension লুকিয়ে রাখে। ফলে `app.js` ভেবে তৈরি করা File-এর আসল নাম `app.js.txt` হয়ে যেতে পারে। File Explorer-এর **View** Menu থেকে **File name extensions** চালু করে নিলে আসল নাম দেখা যায়। VS Code-এর ভেতর থেকে File তৈরি করলে এই সমস্যা হয় না।

প্রয়োজনীয় Terminal Command

নিচের কয়েকটি Command জানলেই এই বইয়ের কাজ চালানো যাবে। VS Code-এর Terminal Windows-এ সাধারণত PowerShell এবং macOS ও Linux-এ zsh বা bash ব্যবহার করে; নিচের Command-গুলো এসব জায়গাতেই কাজ করে।

কাজ	Windows (PowerShell)	macOS / Linux
এখন কোন Folder-এ আছি দেখা	`pwd`	`pwd`
Folder-এর File-এর তালিকা দেখা	`ls`	`ls`
কোনো Folder-এ প্রবেশ করা	`cd chapter-01`	`cd chapter-01`
এক ধাপ ওপরের Folder-এ যাওয়া	`cd ..`	`cd ..`
নতুন Folder তৈরি করা	`mkdir practice`	`mkdir practice`
Terminal-এর লেখা মুছে ফেলা	`cls`	`clear`
একটি JavaScript File চালানো	`node hello.js`	`node hello.js`

VS Code-এ কোনো Folder খোলা থাকলে নতুন Terminal সেই Folder থেকেই শুরু হয়। তাই শুরুতে সাধারণত `cd` ব্যবহার করার প্রয়োজন পড়ে না।

১.৫ প্রথম JavaScript Program

নতুন কোনো ভাষা শেখার প্রথম Program সাধারণত পর্দায় একটি বাক্য দেখায়। JavaScript-এ এ কাজে `console.log()` ব্যবহার করা হয়।

JavaScript — চলবে: Browser Console ও Node.js

```
console.log('Hello, JavaScript!');
```

Output

```
Hello, JavaScript!
```

এই এক লাইনের প্রতিটি অংশের অর্থ আছে:

অংশ	অর্থ
`console`	**Console**—Developer-দের জন্য বার্তা দেখানোর জায়গা।
`.`	Console-এর ভেতরের একটি সুবিধা ব্যবহার করা হচ্ছে, তা বোঝায়।
`log`	"লিখে রাখো"—Console-এ একটি বার্তা দেখানোর নির্দেশ।
`( )`	বন্ধনীর ভেতরে থাকে যা দেখাতে হবে।
`'Hello, JavaScript!'`	Quotation Mark-এ ঘেরা লেখা, যাকে **String** বলে।
`;`	নির্দেশটি এখানে শেষ।

`console.log()` ব্যবহারকারীর জন্য নয়; এটি Developer-এর নিজের দেখার জন্য। Program কোন পথে চলছে বা কোনো মান কত, তা যাচাই করতে এটি বারবার কাজে লাগবে।

পদ্ধতি ১: Browser Console-এ চালানো

প্রতিটি আধুনিক Browser-এ **Developer Tools** (সংক্ষেপে DevTools) নামে একটি অংশ থাকে, যার ভেতরে Console আছে।

১. Browser-এ একটি নতুন Tab খুলুন।
২. `F12` চাপুন, অথবা Windows ও Linux-এ `Ctrl + Shift + I`, macOS-এ `Cmd + Option + I` চাপুন।
৩. খোলা অংশের ওপরের দিক থেকে **Console** Tab বেছে নিন।
৪. `>` চিহ্নের পাশে ওপরের লাইনটি টাইপ করে `Enter` চাপুন।

বার্তাটির নিচে `undefined` লেখা দেখা যাবে। এটি কোনো ভুল নয়। Console প্রতিটি নির্দেশের ফলাফল দেখায়; `console.log()` শুধু বার্তা দেখায় এবং কোনো ফলাফল ফেরত দেয় না বলে `undefined` (অর্থাৎ "কোনো মান নেই") দেখানো হয়। আপাতত এটি উপেক্ষা করুন।

> **পরামর্শ:** শুরুর দিকে Code Copy-Paste না করে নিজে টাইপ করুন। টাইপ করার সময় যে ছোট ভুলগুলো হয়, সেগুলো ঠিক করতে গিয়েই ভাষার নিয়ম সবচেয়ে ভালোভাবে শেখা হয়।

Browser Console ছোট পরীক্ষার জন্য চমৎকার, কিন্তু Tab বন্ধ করলে লেখা Code হারিয়ে যায়। স্থায়ীভাবে Code রাখতে File ব্যবহার করতে হয়।

পদ্ধতি ২: Node.js দিয়ে File চালানো

১. VS Code-এ `chapter-01` Folder-এর ভেতরে `hello.js` নামে একটি File তৈরি করুন।

২. File-এ নিচের লাইনটি লিখে Save করুন।

hello.js · JavaScript — চলবে: Node.js

```
console.log('Hello, JavaScript!');
```

৩. Terminal খুলে নিচের Command চালান।

Terminal

```
node hello.js
```

Output

```
Hello, JavaScript!
```

`node hello.js` Command-টির অর্থ: "Node.js, `hello.js` File-এর Code চালাও।" লক্ষ করুন, এখানে `undefined` দেখায়নি। কারণ Node.js File চালানোর সময় শুধু সেই লেখাই দেখায়, যা আপনি `console.log()` দিয়ে দেখাতে বলেছেন।

একাধিক লাইন এবং বিভিন্ন ধরনের মান

Program-এর লাইনগুলো ওপর থেকে নিচে, একটির পর একটি চলে। `values.js` নামে নতুন File তৈরি করে নিচের Code লিখুন।

values.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('আমার প্রথম JavaScript Program');
console.log(2026);
console.log(10 + 5);
console.log('10 + 5');
```

Output

```
আমার প্রথম JavaScript Program
2026
15
10 + 5
```

শেষ দুটি লাইনের পার্থক্য গুরুত্বপূর্ণ। Quotation Mark ছাড়া `10 + 5` একটি হিসাব, তাই JavaScript যোগ করে `15` দেখায়। Quotation Mark-এর ভেতরে `'10 + 5'` শুধুই লেখা, তাই সেটি হুবহু দেখানো হয়। Quotation Mark থাকা বা না থাকার ওপর নির্ভর করে JavaScript একই অক্ষরগুলোকে সম্পূর্ণ ভিন্নভাবে বোঝে।

১.৬ JavaScript লেখার মৌলিক নিয়ম

প্রতিটি ভাষার যেমন ব্যাকরণ আছে, Programming Language-এরও তেমন নিয়ম আছে। এই নিয়মগুলোকে **Syntax** বলে। Syntax ভুল হলে Program চলে না।

String এবং Quotation Mark

লেখা বা অক্ষরের সমষ্টিকে JavaScript-এ **String** বলে। String তিন ধরনের চিহ্ন দিয়ে লেখা যায়:

strings.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('Single Quote দিয়ে লেখা String');
console.log("Double Quote দিয়ে লেখা String");
console.log(`Backtick দিয়ে লেখা String`);
console.log("It's a good day to learn.");
console.log('শিক্ষক বললেন, "চেষ্টা চালিয়ে যান।"');
```

Output

```
Single Quote দিয়ে লেখা String
Double Quote দিয়ে লেখা String
Backtick দিয়ে লেখা String
It's a good day to learn.
শিক্ষক বললেন, "চেষ্টা চালিয়ে যান।"
```

তিনটি নিয়ম মনে রাখুন:

- String যে চিহ্ন দিয়ে শুরু হয়, সেই চিহ্ন দিয়েই শেষ করতে হয়।
- String-এর ভেতরে একটি Quotation Mark দরকার হলে বাইরে অন্য ধরনেরটি ব্যবহার করুন। চতুর্থ লাইনে `It's`-এর ভেতরে Single Quote আছে বলে বাইরে Double Quote দেওয়া হয়েছে।
- এই বইয়ে সাধারণ String-এর জন্য Single Quote ব্যবহার করা হয়েছে। Backtick-এর একটি বিশেষ সুবিধা আছে, যা অনুচ্ছেদ ১.৭-এ দেখানো হবে।

Statement এবং Semicolon

JavaScript-এর একটি সম্পূর্ণ নির্দেশকে **Statement** বলে। `console.log('Hello');` একটি Statement। বাংলা বাক্যের শেষে যেমন দাঁড়ি বসে, Statement-এর শেষে তেমন Semicolon (`;`) বসে।

JavaScript অনেক ক্ষেত্রে Semicolon ছাড়াও Statement-এর শেষ বুঝে নিতে পারে, তাই কিছু Code-এ Semicolon দেখা যায় না। তবে এই স্বয়ংক্রিয় ব্যবস্থা কিছু বিশেষ ক্ষেত্রে অপ্রত্যাশিত ফল দেয়। এ কারণে এই বইয়ে প্রতিটি Statement-এর শেষে Semicolon দেওয়া হয়েছে, এবং আপনাকেও সেই অভ্যাস করার পরামর্শ দেওয়া হচ্ছে।

Comment

Comment হলো Code-এর ভেতরে মানুষের জন্য লেখা টীকা। JavaScript Comment চালায় না।

comments.js · JavaScript চলবে: Browser Console ও Node.js

```
// এটি এক লাইনের Comment। JavaScript এই লাইন চালায় না।
console.log('প্রথম লাইন চলল।'); // লাইনের শেষেও Comment লেখা যায়

/*
  এটি একাধিক লাইনের Comment।
  বড় ব্যাখ্যা লেখার সময় কাজে লাগে।
*/
console.log('Program শেষ।');
```

Output

```
প্রথম লাইন চলল।
Program শেষ।
```

`//` থেকে লাইনের শেষ পর্যন্ত সবকিছু Comment। `/*` থেকে `*/` পর্যন্ত যত লাইনই থাকুক, সবই Comment। ভালো Comment বলে **কেন** কাজটি করা হচ্ছে; Code নিজেই বলে **কী** করা হচ্ছে।

Case Sensitivity

JavaScript বড় হাতের ও ছোট হাতের অক্ষরকে আলাদা হিসেবে ধরে। একে **Case Sensitivity** বলে। তাই `console`, `Console` এবং `CONSOLE` তিনটি ভিন্ন নাম। একইভাবে `log` এবং `Log` এক নয়। শুরুর দিকের অনেক Error-এর কারণ শুধু একটি অক্ষরের আকার।

১.৭ Variable, Function এবং শর্ত: সংক্ষিপ্ত পরিচয়

অধ্যায়ের শেষের Project তৈরি করতে তিনটি ধারণার প্রাথমিক পরিচয় প্রয়োজন। এখানে ঠিক ততটুকুই দেখানো হলো; বিস্তারিত আলোচনা অধ্যায় ২, ৩ এবং ৫-এ আছে।

Variable: মানকে নাম দেওয়া

Variable হলো একটি নাম, যার সঙ্গে একটি মান যুক্ত থাকে। একবার নাম দিলে পরে সেই নাম ব্যবহার করেই মানটি পাওয়া যায়।

Function: পুনর্ব্যবহারযোগ্য নির্দেশনা

Function হলো নাম দেওয়া একগুচ্ছ নির্দেশনা। একবার লিখে রাখলে সেটি যতবার প্রয়োজন, ততবার চালানো যায়।

variables-functions.js · JavaScript — চলবে: Browser Console ও Node.js

```
const learnerName = 'Nusrat';
let completedLessons = 0;

console.log(learnerName);
console.log(completedLessons);

completedLessons = 1;
console.log(completedLessons);

function createWelcome(name) {
  return `স্বাগতম, ${name}!`;
}

console.log(createWelcome(learnerName));
console.log(createWelcome('Tanvir'));
```

Output

```
Nusrat
0
1
স্বাগতম, Nusrat!
স্বাগতম, Tanvir!
```

ধাপে ধাপে ব্যাখ্যা:

- `const learnerName = 'Nusrat';` — `learnerName` নামে একটি Variable তৈরি হলো, যার মান `'Nusrat'`। `const` দিয়ে তৈরি Variable-এ পরে নতুন মান বসানো যায় না।
- `let completedLessons = 0;` — `let` দিয়ে তৈরি Variable-এর মান পরে বদলানো যায়। সপ্তম লাইনে সেটিই করা হয়েছে।
- `=` চিহ্নের অর্থ এখানে "সমান" নয়; এর অর্থ "ডান পাশের মানটি বাঁ পাশের নামে রাখো।"
- `function createWelcome(name) { ... }` — `createWelcome` নামে একটি Function তৈরি হলো। বন্ধনীর ভেতরের `name` হলো **Parameter**: Function চালানোর সময় যে মান দেওয়া হবে, সেটি এই নামে পাওয়া যাবে। একাধিক Parameter দরকার হলে কমা দিয়ে আলাদা করা হয়, যেমন `(name, city)`।
- `return` — Function-এর ফলাফল ফেরত দেয়।
- `` `স্বাগতম, ${name}!` `` — Backtick-এ লেখা String-এর ভেতরে `${ }` দিয়ে Variable-এর মান বসানো যায়। এটিই Backtick-এর বিশেষ সুবিধা।
- `createWelcome(learnerName)` — Function-টি চালানো হলো। একে Function **Call** করা বলে। একই Function দুটি ভিন্ন নাম দিয়ে দুবার Call করা হয়েছে।

নাম লেখার সময় এই বইয়ে একটি রীতি মানা হয়েছে: প্রথম শব্দ ছোট হাতের অক্ষরে, পরের প্রতিটি শব্দের প্রথম অক্ষর বড় হাতের—যেমন `learnerName`। এই রীতিকে camelCase বলা হয়।

শর্ত: অবস্থা বুঝে সিদ্ধান্ত নেওয়া

অনেক সময় Program-কে অবস্থা অনুযায়ী ভিন্ন কাজ করতে হয়। এ জন্য `if` ব্যবহার করা হয়।

condition.js · JavaScript — চলবে: Browser Console ও Node.js

```
function checkName(name) {
  const cleanName = name.trim();

  if (cleanName === '') {
    return 'নাম লেখা হয়নি।';
  }

  return `নাম পাওয়া গেছে: ${cleanName}`;
}

console.log(checkName('  Karim  '));
console.log(checkName('     '));
```

Output

```
নাম পাওয়া গেছে: Karim
নাম লেখা হয়নি।
```

- `name.trim()` — String-এর শুরু ও শেষের ফাঁকা জায়গা বাদ দেয়। `'  Karim  '` হয়ে যায় `'Karim'`, আর শুধু ফাঁকা জায়গার String হয়ে যায় `''`, অর্থাৎ ফাঁকা String।
- `===` — দুই পাশের মান হুবহু এক কি না, তা যাচাই করে। এটি `=` থেকে আলাদা: `=` মান রাখে, `===` তুলনা করে।
- `if (শর্ত) { ... }` — শর্ত সত্য হলে তবেই দ্বিতীয় বন্ধনীর ভেতরের Code চলে।
- `return` চলার সঙ্গে সঙ্গে Function শেষ হয়ে যায়। তাই নাম ফাঁকা হলে নিচের `return` পর্যন্ত আর যাওয়া হয় না।

১.৮ Error Message পড়তে শেখা

Code-এ ভুল হলে JavaScript একটি **Error Message** দেখায়। নতুনরা অনেক সময় লাল লেখা দেখে ঘাবড়ে যান, অথচ এই বার্তাই সবচেয়ে বড় সহায়ক: এতে ভুলের ধরন, জায়গা এবং কারণ লেখা থাকে।

নিচে চারটি ইচ্ছাকৃত ভুল এবং Node.js-এ সেগুলোর আসল বার্তা দেখানো হলো। প্রতিটি নিজে চালিয়ে দেখুন।

ভুল ১: শেষের Quotation Mark নেই

e1-missing-quote.js · JavaScript — চলবে: Node.js

```
console.log('Hello, JavaScript!);
```

Error Message

```
e1-missing-quote.js:1
console.log('Hello, JavaScript!);
            ^^^^^^^^^^^^^^^^^^^^

SyntaxError: Invalid or unexpected token
```

ভুল ২: নামের বানান ভুল

e2-misspelled-name.js · JavaScript — চলবে: Node.js

```
consol.log('Hello, JavaScript!');
```

Error Message

```
e2-misspelled-name.js:1
consol.log('Hello, JavaScript!');
^

ReferenceError: consol is not defined
```

ভুল ৩: অক্ষরের আকার ভুল

e3-wrong-case.js · JavaScript — চলবে: Node.js

```
console.Log('Hello, JavaScript!');
```

Error Message

```
e3-wrong-case.js:1
console.Log('Hello, JavaScript!');
        ^

TypeError: console.Log is not a function
```

ভুল ৪: শেষের বন্ধনী নেই

e4-missing-parenthesis.js · JavaScript — চলবে: Node.js

```
console.log('Hello, JavaScript!';
```

Error Message

```
e4-missing-parenthesis.js:1
console.log('Hello, JavaScript!';
            ^^^^^^^^^^^^^^^^^^^^

SyntaxError: missing ) after argument list
```

ওপরের প্রতিটি বার্তার পরে Node.js আরও কয়েকটি লাইন দেখায়, যেগুলো এখানে বাদ দেওয়া হয়েছে। সেগুলো নিয়ে অনুচ্ছেদ ১.১১-এ আলোচনা আছে।

একটি Error Message-এর চারটি অংশ

১. **File-এর নাম ও লাইন নম্বর** — `e2-misspelled-name.js:1` মানে ভুলটি এই File-এর ১ নম্বর লাইনে।

২. **সংশ্লিষ্ট লাইন** — যে লাইনে সমস্যা, সেটি হুবহু দেখানো হয়।

৩. `^` **চিহ্ন** — লাইনের ঠিক কোন জায়গায় সমস্যা ধরা পড়েছে, তা দেখায়।

৪. **Error-এর ধরন ও বর্ণনা** — যেমন `ReferenceError: consol is not defined`।

তিনটি প্রচলিত Error-এর ধরন

ধরন	অর্থ	সাধারণ কারণ
`SyntaxError`	Code-এর গঠন ভাষার নিয়ম মানছে না	Quotation Mark বা বন্ধনী বাদ পড়া
`ReferenceError`	এমন একটি নাম ব্যবহার করা হয়েছে, যা JavaScript চেনে না	নামের বানান ভুল
`TypeError`	কোনো মানকে এমনভাবে ব্যবহার করা হয়েছে, যা সম্ভব নয়	অক্ষরের আকার ভুল; যা নেই তাকে Function হিসেবে চালানো

`SyntaxError`-এর একটি বিশেষ বৈশিষ্ট্য আছে: এটি থাকলে File-এর কোনো লাইনই চলে না, কারণ JavaScript চালানোর আগে পুরো File-এর গঠন যাচাই করে। অন্য দুই ধরনের Error-এ ভুলের আগের লাইনগুলো চলে, তারপর Program থেমে যায়।

> **নোট:** Browser Console-এ একই ভুলের বার্তার শুরুতে `Uncaught` শব্দটি যুক্ত থাকে, যেমন `Uncaught ReferenceError: consol is not defined`। Chrome, Edge এবং Node.js একই বর্ণনা দেখায়; Firefox একই ভুলের জন্য কিছুটা ভিন্ন শব্দ ব্যবহার করে, তবে Error-এর ধরন একই থাকে।

১.৯ Web Page-এ JavaScript ব্যবহার

এ পর্যন্ত JavaScript শুধু Console-এ লেখা দেখিয়েছে। এবার একটি Web Page তৈরি করে JavaScript দিয়ে সেই Page-এর লেখা বদলানো হবে। `chapter-01` Folder-এর ভেতরে `first-page` নামে একটি নতুন Folder তৈরি করুন এবং তার ভেতরে নিচের দুটি File রাখুন।

HTML Document-এর মৌলিক কাঠামো

index.html · HTML — চলবে: Browser

```
<!DOCTYPE html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>আমার প্রথম Web Page</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>আমার প্রথম Web Page</h1>
  <p id="status">এখনো Button চাপা হয়নি।</p>
  <button id="change-button" type="button">লেখা বদলান</button>
</body>
</html>
```

HTML লেখা হয় **Tag** দিয়ে। অধিকাংশ Tag জোড়ায় আসে: `<p>` দিয়ে শুরু, `</p>` দিয়ে শেষ। শুরু থেকে শেষ পর্যন্ত পুরো অংশটিকে একটি **Element** বলে।

অংশ	কাজ
`<!DOCTYPE html>`	Browser-কে জানায়, এটি আধুনিক HTML Document।
`<html lang="bn">`	পুরো Page-এর মূল Element; `lang="bn"` জানায় Page-এর ভাষা বাংলা।
`<head>`	Page সম্পর্কিত তথ্য, যা সরাসরি পর্দায় দেখা যায় না।
`<meta charset="utf-8">`	বাংলাসহ সব ভাষার অক্ষর ঠিকভাবে দেখানোর জন্য প্রয়োজন।
`<meta name="viewport" ...>`	Mobile-এর পর্দায় Page-কে সঠিক আকারে দেখায়।
`<title>`	Browser Tab-এ যে নাম দেখা যায়।
`<body>`	Page-এ যা কিছু দেখা যায়, সব এর ভেতরে থাকে।
`id="status"`	Element-টির একটি অনন্য নাম, যা দিয়ে JavaScript তাকে খুঁজে পাবে।

`<script>` Element এবং External File

JavaScript Code HTML File-এর ভেতরে সরাসরি লেখা যায়, কিন্তু আলাদা File-এ রাখাই উত্তম অভ্যাস। তাতে কাঠামো (HTML) এবং আচরণ (JavaScript) আলাদা থাকে, এবং দুটোই পড়া ও ঠিক করা সহজ হয়। আলাদা File-কে **External File** বলে।

`<script src="script.js" defer></script>` লাইনটি Browser-কে বলে: "`script.js` নামের File-টি এনে চালাও।" `src`-এর মানটি File-এর **Path**, অর্থাৎ ঠিকানা। দুটি File একই Folder-এ থাকলে শুধু File-এর নাম লিখলেই হয়।

script.js · JavaScript — চলবে: Browser

```
const statusText = document.querySelector('#status');
const changeButton = document.querySelector('#change-button');

function showMessage() {
  statusText.textContent = 'Button চাপা হয়েছে। JavaScript কাজ করছে!';
}

changeButton.addEventListener('click', showMessage);
console.log('script.js সফলভাবে Load হয়েছে।');
```

File Explorer থেকে `index.html` File-টিতে Double-click করলে সেটি Browser-এ খুলবে। Page-এ একটি শিরোনাম, একটি বাক্য এবং একটি Button দেখা যাবে। Button-এ Click করলে বাক্যটি বদলে যাবে, আর Console খুললে `script.js সফলভাবে Load হয়েছে।` বার্তাটি দেখা যাবে।

DOM: Page-এর সঙ্গে JavaScript-এর সংযোগ

Browser HTML File পড়ে তার প্রতিটি Element-কে মনে রাখার জন্য নিজের ভেতরে একটি কাঠামো তৈরি করে। এই কাঠামোকে **DOM** (Document Object Model) বলে। JavaScript সরাসরি HTML File বদলায় না; সে DOM বদলায়, আর Browser সেই পরিবর্তন পর্দায় দেখায়। JavaScript-এ `document` নামটি দিয়ে পুরো Page-এর DOM পাওয়া যায়।

এই পর্যায়ে DOM নিয়ে তিনটি কাজ জানলেই যথেষ্ট।

১. একটি Element খুঁজে নেওয়া। `document.querySelector('#status')` Page থেকে সেই Element-টি খুঁজে আনে, যার `id` হলো `status`। `#` চিহ্নটি বোঝায় যে `id` দিয়ে খোঁজা হচ্ছে। খুঁজে পাওয়া Element-টি `statusText` Variable-এ রাখা হয়েছে, যাতে পরে ব্যবহার করা যায়।

২. Element-এর লেখা বদলানো। `statusText.textContent = '...'` সেই Element-এর ভেতরের লেখা নতুন লেখা দিয়ে বদলে দেয়।

৩. Click-এ সাড়া দেওয়া। ব্যবহারকারীর কোনো কাজ—যেমন Click করা বা Key চাপা—কে **Event** বলে। `changeButton.addEventListener('click', showMessage)` লাইনটির অর্থ: "এই Button-এ `click` Event ঘটলে `showMessage` Function-টি চালাও।" লক্ষ করুন, এখানে `showMessage`-এর পরে বন্ধনী নেই। বন্ধনী

দিলে Function-টি তখনই চলে যেত; বন্ধনী ছাড়া লিখলে শুধু Function-টির নাম জানিয়ে রাখা হয়, আর Browser Click-এর সময় সেটি চালায়।

> **মনে রাখুন:** Page-এ লেখা দেখানোর জন্য `textContent` ব্যবহার করুন। `innerHTML` নামে আরেকটি উপায় আছে, যা লেখাকে HTML হিসেবে ব্যাখ্যা করে। ব্যবহারকারীর দেওয়া লেখা `innerHTML` দিয়ে দেখালে তার ভেতরের ক্ষতিকর Code চলে যেতে পারে। `textContent` সবকিছুকে সাধারণ লেখা হিসেবেই দেখায়, তাই এটি নিরাপদ। নিরাপত্তা নিয়ে বিস্তারিত আলোচনা অধ্যায় ২০-এ আছে।

`defer` কেন প্রয়োজন

Browser HTML File ওপর থেকে নিচে পড়ে। `<script>` Tag-টি `<head>`-এর ভেতরে, অর্থাৎ `<body>`-র আগে। `defer` না থাকলে Browser Script-টি পাওয়ামাত্র চালিয়ে দেয়, যখন `<p>` বা `<button>` এখনো তৈরিই হয়নি। ফলে `document.querySelector()` কিছু খুঁজে পায় না এবং `null` ফেরত দেয়। `null` মানে "কিছু নেই"।

`defer` শব্দটি সরিয়ে Page-টি আবার খুললে Console-এ এই বার্তা দেখা যায়:

Error Message

```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
```

বার্তাটির অর্থ: "যার `addEventListener` ব্যবহার করতে চাওয়া হয়েছে, সেটি `null`।" অর্থাৎ Button-টি খুঁজে পাওয়া যায়নি। `defer` যোগ করলে Browser পুরো HTML পড়া শেষ করে তারপর Script চালায়, ফলে সব Element তখন প্রস্তুত থাকে।

> **নোট:** আরেকটি পুরোনো সমাধান হলো `<script>` Tag-টি `</body>`-র ঠিক আগে রাখা। তাতেও কাজ হয়। এই বইয়ে `<head>`-এর ভেতরে `defer` সহ `<script>` ব্যবহার করা হয়েছে।

১.১০ সাধারণ ভুল

নিচের ভুলগুলো প্রায় সব নতুন শিক্ষার্থীই কোনো না কোনো সময় করেন। তালিকাটি চিনে রাখলে সমস্যা দ্রুত ধরা যায়।

ভুল	কী দেখা যায়	সমাধান
File Save করা হয়নি	পরিবর্তন করার পরও পুরোনো ফল দেখায়	`Ctrl + S` চেপে Save করুন; File-এর নামের পাশের বিন্দুটি লক্ষ করুন
File-এর নাম মিলছে না	Page-এ JavaScript কাজ করে না; Console-এ File খুঁজে না পাওয়ার বার্তা	`src`-এর নাম এবং আসল File-এর নাম অক্ষরে অক্ষরে মিলিয়ে দেখুন
Path ভুল	ওপরের মতোই	File অন্য Folder-এ থাকলে Folder-সহ লিখুন, যেমন `js/script.js`

ভুল	কী দেখা যায়	সমাধান
Quotation Mark বা বন্ধনী বাদ পড়েছে	`SyntaxError`	যে লাইন দেখানো হয়েছে, সেখানে প্রতিটি শুরুর চিহ্নের শেষের চিহ্ন আছে কি না দেখুন
অক্ষরের আকার ভুল	`TypeError` অথবা `ReferenceError`	`console.log`, `querySelector`, `textContent`-এর বানান মিলিয়ে দেখুন
`id` মিলছে না	`Cannot read properties of null`	HTML-এর `id` এবং JavaScript-এর `#id` হুবহু এক কি না দেখুন
`defer` নেই	`Cannot read properties of null`	`<script>` Tag-এ `defer` যোগ করুন
ভুল Folder থেকে Command চালানো	Node.js `Cannot find module` বার্তা দেখায়	`pwd` ও `ls` দিয়ে দেখুন File-টি বর্তমান Folder-এ আছে কি না
File Extension ভুল	File চলে না বা Browser-এ Code-টিই লেখা হিসেবে দেখায়	নামটি `.js` বা `.html` দিয়ে শেষ হয়েছে কি না দেখুন

`src`-এ ভুল নাম দিলে—যেমন `script.js`-এর বদলে `scripts.js`—Chrome-এর Console-এ এই ধরনের বার্তা আসে:

Error Message

```
Failed to load resource: net::ERR_FILE_NOT_FOUND
```

এই বার্তা কোনো JavaScript Error নয়; এটি জানায় যে Browser File-টিই খুঁজে পায়নি। তাই এ ক্ষেত্রে Code নয়, File-এর নাম ও অবস্থান যাচাই করতে হয়।

১.১১ Debugging নির্দেশিকা

Program-এর ভুলকে **Bug** বলা হয়, আর ভুল খুঁজে বের করে ঠিক করার কাজকে বলে **Debugging**। এটি Programming-এর স্বাভাবিক অংশ; অভিজ্ঞ Developer-রাও প্রতিদিন এ কাজ করেন।

কোথায় দেখবেন

- **Browser-এ চলা Code:** DevTools-এর Console Tab। লাল রঙের বার্তার ডান পাশে File-এর নাম ও লাইন নম্বর থাকে; সেখানে Click করলে সংশ্লিষ্ট লাইনটি দেখা যায়।
- **Node.js-এ চলা Code:** VS Code-এর Terminal। Error Message সেখানেই দেখানো হয়।

Stack Trace-এর প্রাথমিক পরিচয়

Error ঘটার সময় JavaScript দেখায়, কোন Function থেকে কোন Function Call হয়ে ভুলের জায়গায় পৌঁছানো হয়েছে। এই তালিকাকে **Stack Trace** বলে।

e5-stack-trace.js · JavaScript চলবে: Node.js

```
function showLesson() {
  console.log(lessonTitle);
}

function startChapter() {
  showLesson();
}

startChapter();
```

Error Message

```
e5-stack-trace.js:2
  console.log(lessonTitle);
              ^

ReferenceError: lessonTitle is not defined
    at showLesson (e5-stack-trace.js:2:15)
    at startChapter (e5-stack-trace.js:6:3)
    at Object.<anonymous> (e5-stack-trace.js:9:1)
```

`at` দিয়ে শুরু হওয়া লাইনগুলো ওপর থেকে নিচে পড়ুন:

- প্রথম লাইন বলছে, ভুলটি ঘটেছে `showLesson` Function-এর ভেতরে, File-এর ২ নম্বর লাইনের ১৫ নম্বর অক্ষরে।
- দ্বিতীয় লাইন বলছে, `showLesson`-কে Call করা হয়েছিল `startChapter` থেকে, ৬ নম্বর লাইনে।
- তৃতীয় লাইন বলছে, `startChapter`-কে Call করা হয়েছিল File-এর ৯ নম্বর লাইন থেকে।

এরপর Node.js আরও কিছু লাইন দেখায়, যেগুলো `node:internal` দিয়ে শুরু। সেগুলো Node.js-এর নিজস্ব Code-এর, আপনার নয়; আপাতত উপেক্ষা করুন। সব সময় **ওপরের দিকে, আপনার নিজের File-এর নাম থাকা প্রথম লাইনটি** থেকে খোঁজা শুরু করুন।

এখানে ভুলের কারণ হলো `lessonTitle` নামে কোনো Variable তৈরিই করা হয়নি।

Debugging Checklist

Code প্রত্যাশামতো কাজ না করলে নিচের ক্রমে যাচাই করুন:

১. File Save করা হয়েছে তো?
২. সঠিক File-টিই চালানো বা খোলা হচ্ছে তো? Browser-এ Page-টি Reload (`F5`) করা হয়েছে?
৩. Console অথবা Terminal-এ কোনো Error Message আছে কি?
৪. বার্তায় কোন File এবং কোন লাইনের কথা বলা হয়েছে?
৫. সেই লাইনে Quotation Mark, বন্ধনী, বানান এবং অক্ষরের আকার ঠিক আছে তো?
৬. HTML-এর `id` এবং JavaScript-এ ব্যবহৃত নাম হুবহু মিলছে তো?

৭. কোনো Error না থাকলেও ফল ভুল হলে, সন্দেহজনক জায়গায় `console.log()` বসিয়ে দেখুন Code সেখানে পৌঁছাচ্ছে কি না এবং Variable-এর মান কত।

> **পরামর্শ:** একসঙ্গে অনেক কিছু বদলাবেন না। একটি পরিবর্তন করুন, Save করুন, চালিয়ে দেখুন। তাহলে কোন পরিবর্তনে সমস্যা মিটল বা তৈরি হলো, তা পরিষ্কার বোঝা যায়।

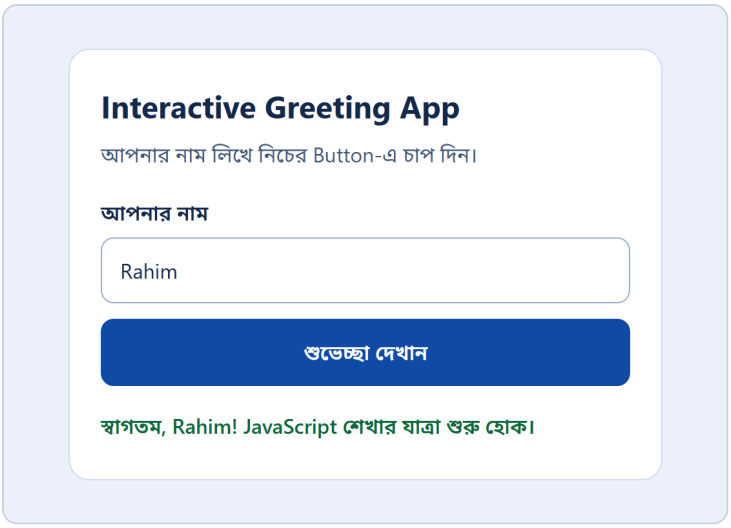
১.১২ পেশাদার অভ্যাস

শুরু থেকেই কয়েকটি অভ্যাস গড়ে তুললে পরে বড় Project-এ কাজ করা অনেক সহজ হয়।

- **অর্থবহ নাম দিন।** `x` বা `data1`-এর চেয়ে `learnerName` বা `statusText` অনেক বেশি তথ্য দেয়।
- **একই রীতি বজায় রাখুন।** Quotation Mark, Semicolon এবং Indentation (লাইনের শুরুর ফাঁকা জায়গা) পুরো File-এ একই রকম রাখুন। এই বইয়ে Single Quote, Semicolon এবং দুই Space-এর Indentation ব্যবহার করা হয়েছে।
- **HTML, CSS এবং JavaScript আলাদা File-এ রাখুন।**
- **অল্প লিখে বারবার চালান।** বিশ লাইন লিখে একবার চালানোর চেয়ে দুই-তিন লাইন পরপর চালানো ভালো; ভুল হলে খোঁজার জায়গা ছোট থাকে।
- **ব্যবহারকারীর লেখা দেখাতে `textContent` ব্যবহার করুন।**
- **Error Message পুরোটা পড়ুন।** শুধু লাল রং দেখে অনুমান করবেন না।
- **`const` দিয়ে শুরু করুন।** মান বদলানোর প্রয়োজন হলে তবেই `let` ব্যবহার করুন।
- **`console.log()` ব্যবহারে সংযত থাকুন।** Debugging শেষে অপ্রয়োজনীয় `console.log()` মুছে ফেলুন, এবং Password-এর মতো গোপন তথ্য কখনো Console-এ দেখাবেন না।

১.১৩ Project: Interactive Greeting App

এ পর্যন্ত শেখা সবকিছু একত্র করে এবার একটি সম্পূর্ণ Application তৈরি করা হবে। ব্যবহারকারী নিজের নাম লিখে Button চাপবেন, আর Page তাঁকে নাম ধরে শুভেচ্ছা জানাবে।



চিত্র ১.১ — সম্পূর্ণ Interactive Greeting App: নাম লিখে Button চাপার পরের অবস্থা

Project-এর চাহিদা

- নাম লেখার জন্য একটি Input ঘর এবং তার একটি স্পষ্ট Label থাকবে।
- Button-এ Click করলে অথবা Input-এ `Enter` চাপলে শুভেচ্ছা-বার্তা দেখাবে।
- নামের আগে-পরে ফাঁকা জায়গা থাকলে তা বাদ যাবে।
- নাম ফাঁকা থাকলে শুভেচ্ছার বদলে একটি সতর্কবার্তা দেখাবে।
- Mobile ও বড় পর্দা—দুই জায়গাতেই Page ঠিকভাবে দেখা যাবে।
- বাইরের কোনো Library বা Package ব্যবহার করা হবে না।

File-এর কাঠামো

`chapter-01` Folder-এর ভেতরে `interactive-greeting-app` নামে একটি Folder তৈরি করুন এবং তাতে তিনটি File রাখুন।

Folder-এর কাঠামো

```
interactive-greeting-app/
├── index.html
├── style.css
└── app.js
```

index.html

index.html · HTML — চলবে: Browser

```
<!DOCTYPE html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <title>Interactive Greeting App</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main class="card">
    <h1>Interactive Greeting App</h1>
    <p class="intro">আপনার নাম লিখে নিচের Button-এ চাপ দিন।</p>

    <form id="greeting-form">
      <label for="name">আপনার নাম</label>
      <input id="name" name="name" type="text" maxlength="60" autocomplete="name">
      <button type="submit">শুভেচ্ছা দেখান</button>
    </form>

    <p id="message" class="message" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

আগের উদাহরণের তুলনায় এখানে কয়েকটি নতুন অংশ আছে:

- `<link rel="stylesheet" href="style.css">` — CSS File-টি Page-এর সঙ্গে যুক্ত করে।
- `<main>` — Page-এর মূল বিষয়বস্তু চিহ্নিত করে।
- `<form>` — Input এবং Button-কে একটি Form হিসেবে একত্র করে। Form ব্যবহারের সুবিধা হলো, Button-এ Click করা এবং Input-এ `Enter` চাপা—দুটোই একই **Submit** Event তৈরি করে। ফলে Keyboard ব্যবহারকারীদের জন্য আলাদা Code লিখতে হয় না।
- `<label for="name">` — এই লেখাটি `id="name"` Input-এর Label। `for` এবং `id`-র মান মিললে Label-এ Click করলে Input সক্রিয় হয়, এবং Screen Reader (দৃষ্টিপ্রতিবন্ধী ব্যবহারকারীদের জন্য পর্দার লেখা পড়ে শোনানো Software) Input-টির উদ্দেশ্য জানাতে পারে।
- `maxlength="60"` — নামের দৈর্ঘ্য সর্বোচ্চ ৬০ অক্ষরে সীমিত রাখে।
- `type="submit"` — এই Button চাপলে Form Submit হয়।
- `role="status"` এবং `aria-live="polite"` — এই অনুচ্ছেদের লেখা বদলালে Screen Reader সেটি ব্যবহারকারীকে পড়ে শোনায়। যাঁরা পর্দা দেখতে পান না, তাঁরা এভাবেই ফলাফল জানতে পারেন।

style.css

CSS এই বইয়ের মূল বিষয় নয়, তাই ১০০ লাইনের পুরো File-টি এখানে ছাপা হয়নি। সম্পূর্ণ, চালানোর উপযোগী File-টি এই অধ্যায়ের Companion Source Code-এ নিচের Path-এ আছে:

`project/interactive-greeting-app/style.css`

File-টি সেখান থেকে Copy করে আপনার `interactive-greeting-app` Folder-এ রাখুন। Companion Source Code হাতের কাছে না থাকলেও Project-টি চলবে: `style.css` ছাড়া Page সাদামাটা দেখাবে, কিন্তু নাম লেখা, Button এবং বার্তা ঠিকভাবেই কাজ করবে।

নিচে File-টির চারটি শিক্ষণীয় অংশ আলাদাভাবে দেখানো হলো। এগুলো **সম্পূর্ণ File নয়**; `/* ... */` চিহ্নিত জায়গায় আরও নিয়ম বাদ দেওয়া হয়েছে। পুরো File-এ এ ছাড়াও `body`, `h1`, `label`, `input` এবং `button`-এর সাধারণ নিয়ম আছে।

১. Card-এর প্রস্থ: ছোট ও বড় পর্দায় মানিয়ে নেওয়া

style.css · CSS নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Browser

```css
/* ... */

.card {
  width: 100%;
  max-width: 460px;
  padding: 28px 24px;
  background: #ffffff;
  border: 1px solid #d5e0f0;
  border-radius: 16px;
}

/* ... */
```

`.card`-এর `width: 100%` এবং `max-width: 460px` — ছোট পর্দায় Card পুরো প্রস্থ নেয়, বড় পর্দায় ৪৬০ Pixel-এর বেশি চওড়া হয় না। এভাবেই Page-টি বিভিন্ন আকারের পর্দায় মানিয়ে যায়।

২. Keyboard ব্যবহারকারীর জন্য স্পষ্ট Focus

style.css · CSS নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Browser

```css
/* ... */

input:focus-visible,
button:focus-visible {
  outline: 3px solid #12a5c9;
  outline-offset: 2px;
}

/* ... */
```

`:focus-visible` — Keyboard দিয়ে কোনো ঘরে গেলে তার চারপাশে একটি স্পষ্ট রেখা দেখায়, যাতে বোঝা যায় এখন কোন ঘরটি সক্রিয়।

৩. বার্তার ধরন অনুযায়ী রং

style.css · CSS নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Browser

```
/* ... */

.message.success {
  color: #0b6b3a;
}

.message.error {
  color: #b42318;
}

/* ... */
```

`.message.success` এবং `.message.error` — বার্তার ধরন অনুযায়ী রং বদলায়। কোন রংটি দেখাবে, তা JavaScript ঠিক করে।

৪. খুব ছোট পর্দার জন্য আলাদা নিয়ম

style.css · CSS নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Browser

```
/* ... */

@media (max-width: 400px) {
  .card {
    padding: 22px 16px;
  }

  h1 {
    font-size: 1.3rem;
  }
}
```

`@media (max-width: 400px)` — খুব ছোট পর্দায় ফাঁকা জায়গা ও শিরোনামের আকার কিছুটা কমায়।

app.js

app.js · JavaScript চলবে: Browser

```
'use strict';

// নাম থেকে শুভেচ্ছা-বার্তা তৈরি করে। নাম ফাঁকা হলে ফাঁকা String ফেরত দেয়।
function createGreeting(name) {
  const cleanName = name.trim();

  if (cleanName === '') {
    return '';
  }
```

```
  return `স্বাগতম, ${cleanName}! JavaScript শেখার যাত্রা শুরু হোক।`;
}

const form = document.querySelector('#greeting-form');
const nameInput = document.querySelector('#name');
const message = document.querySelector('#message');

// Form Submit হলে (Button Click অথবা Enter) এই Function চলে।
function handleSubmit(event) {
  event.preventDefault();

  const greeting = createGreeting(nameInput.value);

  if (greeting === '') {
    message.textContent = 'অনুগ্রহ করে আপনার নাম লিখুন।';
    message.className = 'message error';
    nameInput.focus();
    return;
  }

  message.textContent = greeting;
  message.className = 'message success';
}

form.addEventListener('submit', handleSubmit);
```

Code-এর ধাপে ধাপে ব্যাখ্যা

`'use strict';` — File-এর প্রথম লাইনের এই String JavaScript-কে কঠোর নিয়মে চলতে বলে। এতে কিছু ভুল, যা সাধারণ অবস্থায় নীরবে উপেক্ষিত হতো, Error হিসেবে ধরা পড়ে। নতুনদের জন্য এটি উপকারী, কারণ ভুল দ্রুত চোখে পড়ে।

`createGreeting(name)` — এই Function-টি অনুচ্ছেদ ১.৭-এর `checkName`-এর মতোই কাজ করে। এটি নাম থেকে আগে-পরের ফাঁকা জায়গা বাদ দেয়; নাম ফাঁকা হলে ফাঁকা String ফেরত দেয়, না হলে শুভেচ্ছা-বার্তা ফেরত দেয়। লক্ষ করুন, এই Function Page-এর কিছুই স্পর্শ করে না: এটি শুধু একটি String নেয় এবং একটি String ফেরত দেয়। হিসাবের অংশকে এভাবে Page থেকে আলাদা রাখলে সেটি আলাদাভাবে পরীক্ষা করা সহজ হয়।

তিনটি `querySelector` — Form, Input এবং বার্তার অনুচ্ছেদ—এই তিনটি Element খুঁজে তিনটি Variable-এ রাখা হয়েছে।

`handleSubmit(event)` — Form Submit হলে Browser এই Function-টি চালায় এবং ঘটনাটির তথ্য `event` Parameter-এ দেয়।

- `event.preventDefault();` — Form Submit হলে Browser-এর স্বাভাবিক আচরণ হলো Page-টি নতুন করে Load করা। তাতে লেখা বার্তা মুছে যেত। এই লাইনটি সেই স্বাভাবিক আচরণ বন্ধ করে।
- `nameInput.value` — Input ঘরে ব্যবহারকারী এই মুহূর্তে যা লিখেছেন।

- `if (greeting === '')` — `createGreeting` ফাঁকা String ফেরত দিলে বোঝা যায় নাম দেওয়া হয়নি। তখন সতর্কবার্তা দেখানো হয়, বার্তার `className` বদলে লাল রং প্রয়োগ করা হয়, এবং `nameInput.focus()` দিয়ে Cursor আবার Input ঘরে ফিরিয়ে দেওয়া হয়। এরপর `return` Function-টি সেখানেই শেষ করে।
- শেষ দুই লাইন — নাম ঠিক থাকলে শুভেচ্ছা-বার্তা দেখানো হয় এবং সবুজ রং প্রয়োগ করা হয়।

`form.addEventListener('submit', handleSubmit);` — Form-এ `submit` Event ঘটলে `handleSubmit` চালানোর ব্যবস্থা করে। এই একটি লাইনই Button Click এবং `Enter` —দুটোকেই সামলায়।

চালানো এবং প্রত্যাশিত আচরণ

File Explorer থেকে `index.html`-এ Double-click করুন। তারপর নিচের প্রতিটি ক্ষেত্র নিজে পরীক্ষা করুন।

আপনি যা করবেন	যা দেখা যাওয়ার কথা
`Rahim` লিখে Button-এ Click	সবুজ রঙে: স্বাগতম, Rahim! JavaScript শেখার যাত্রা শুরু হোক।
নাম লিখে `Enter` চাপা	একই শুভেচ্ছা-বার্তা
কিছু না লিখে Button-এ Click	লাল রঙে: অনুগ্রহ করে আপনার নাম লিখুন।; Cursor Input ঘরে ফেরে
শুধু কয়েকটি Space লিখে Click	একই সতর্কবার্তা
`  Karim  ` (আগে-পরে Space) লিখে Click	স্বাগতম, Karim! …—বাড়তি Space ছাড়া
`<b>Test</b>` লিখে Click	লেখাটি হুবহু দেখায়; মোটা অক্ষর হয় না

শেষ সারিটি `textContent`-এর নিরাপত্তা প্রমাণ করে: ব্যবহারকারী HTML Tag লিখলেও সেটি Tag হিসেবে কার্যকর হয় না।

> **নোট:** এই Project-এর সঙ্গে দেওয়া Source Code-এ দুই ধরনের Automated Test আছে: `createGreeting` Function-এর জন্য ৮টি Test এবং আসল Browser-এ পুরো Page-এর আচরণের জন্য ১২টি Test। ওপরের টেবিলের প্রতিটি সারি Chrome-এ স্বয়ংক্রিয়ভাবে যাচাই করা হয়েছে। Test কীভাবে লিখতে হয়, তা অধ্যায় ১৬ ও ২২-এ শেখানো হবে; আপাতত Test চালানোর Command Project-এর `README.md` File-এ দেওয়া আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Page সাদামাটা দেখায়, কোনো রং বা Card নেই	`style.css` Load হয়নি	`href="style.css"` এবং File-এর নাম মিলিয়ে দেখুন
Button চাপলে Page ঝলক দিয়ে আগের অবস্থায় ফেরে	JavaScript চলছে না, অথবা `event.preventDefault()` নেই	Console-এ Error দেখুন; `src="app.js"` এবং `defer` যাচাই করুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Console-এ `Cannot read properties of null`	`id` মিলছে না অথবা `defer` নেই	`greeting-form`, `name`, `message` —তিনটি `id` মিলিয়ে দেখুন
বার্তা দেখায় কিন্তু রং বদলায় না	CSS-এ Class-এর নাম ভিন্ন	`message success` এবং `message error` বানান মিলিয়ে দেখুন
বাংলা অক্ষরের জায়গায় অদ্ভুত চিহ্ন	`<meta charset="utf-8">` নেই অথবা File অন্য Encoding-এ Save হয়েছে	Tag-টি যোগ করুন; VS Code-এর নিচের ডান কোণে `UTF-8` আছে কি না দেখুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project; বাস্তব ব্যবহারকারীদের জন্য তৈরি Application নয়। এখানে কোনো Data সংরক্ষণ করা হয় না, Page বন্ধ করলে সবকিছু মুছে যায়, এবং Server-এর সঙ্গে কোনো যোগাযোগ নেই। বাস্তব Application-এ Server-side যাচাই, বিস্তৃত Browser ও Device পরীক্ষা এবং আরও পূর্ণাঙ্গ Accessibility পর্যালোচনা প্রয়োজন হয়। এই বিষয়গুলো বইয়ের পরবর্তী অধ্যায়গুলোতে ধাপে ধাপে আসবে।

১.১৪ অনুশীলনী

প্রতিটি অনুশীলনী এই অধ্যায়ে শেখা বিষয় দিয়েই সমাধান করা যায়। আগে নিজে চেষ্টা করুন, তারপর অনুচ্ছেদ ১.১৫-এর সমাধানের সঙ্গে মিলিয়ে দেখুন।

প্রাথমিক স্তর

১. `about-me.js` নামে একটি File তৈরি করুন। তাতে তিনটি `console.log()` দিয়ে আপনার নাম, আপনার শহর এবং JavaScript শেখার একটি উদ্দেশ্য দেখান। Node.js দিয়ে চালান।

২. না চালিয়ে বলুন, নিচের দুটি লাইন কী দেখাবে: `console.log(20 - 5);` এবং `console.log('20 - 5');`। তারপর চালিয়ে মিলিয়ে নিন।

৩. নিচের তিন লাইনের প্রতিটিতে একটি করে ভুল আছে। ভুলগুলো খুঁজে ঠিক করুন, যাতে Program তিনটি লাইনই দেখায়।

fix-me.js · JavaScript — চলবে: Node.js

```
Console.log('শুরু');
console.log('মাঝখান);
console.log('শেষ';
```

৪. `hello.js` File-এ এক লাইনের একটি এবং একাধিক লাইনের একটি Comment যোগ করুন। চালিয়ে নিশ্চিত হোন যে Output বদলায়নি।

৫. Terminal ব্যবহার করে `chapter-01` Folder-এর ভেতরে `practice` নামে একটি Folder তৈরি করুন, তাতে প্রবেশ করুন এবং আপনি কোন Folder-এ আছেন তা দেখান।

মধ্যম স্তর

৬. `createFarewell` নামে একটি Function লিখুন, যা একটি নাম নিয়ে `বিদায়, <নাম>! আবার দেখা হবে।` String-টি ফেরত দেয়। দুটি ভিন্ন নাম দিয়ে Function-টি Call করে ফলাফল দেখান।

৭. `describeLesson` নামে একটি Function লিখুন, যা একটি শিরোনাম নেয়। শিরোনাম ফাঁকা হলে (অথবা শুধু ফাঁকা জায়গা হলে) `শিরোনাম দেওয়া হয়নি।` ফেরত দেবে; না হলে `আজকের পাঠ: <শিরোনাম>` ফেরত দেবে।

৮. অনুচ্ছেদ ১.৯-এর `first-page` উদাহরণে `<h1>` Element-এ `id="page-title"` যোগ করুন। Button চাপলে অনুচ্ছেদের লেখার পাশাপাশি শিরোনামটিও বদলে `JavaScript শেখা চলছে` হবে।

৯. `first-page` উদাহরণ থেকে `defer` সরিয়ে দিন এবং Console-এ কী বার্তা আসে তা নিজের ভাষায় ব্যাখ্যা করুন। তারপর `defer` ফিরিয়ে না এনে অন্য কোন উপায়ে সমস্যাটি সমাধান করা যায়, তা প্রয়োগ করে দেখান।

চ্যালেঞ্জ স্তর

১০. **গণনা:** Greeting App-এ একটি নতুন অনুচ্ছেদ যোগ করুন, যা দেখাবে এ পর্যন্ত কতবার সফলভাবে শুভেচ্ছা দেখানো হয়েছে—যেমন `মোট শুভেচ্ছা: 3 বার`। ফাঁকা নাম দিলে সংখ্যা বাড়বে না।

১১. **মুছে ফেলা:** Greeting App-এ `মুছে ফেলুন` লেখা দ্বিতীয় একটি Button যোগ করুন। এটি চাপলে Input ঘর এবং বার্তা দুটোই ফাঁকা হবে এবং Cursor Input ঘরে ফিরবে।

১২. **দুটি তথ্য:** Greeting App-এ শহরের নাম লেখার জন্য দ্বিতীয় একটি Input (Label-সহ) যোগ করুন। নাম ও শহর দুটোই থাকলে দেখাবে `স্বাগতম, <নাম>! <শহর> থেকে JavaScript শেখার যাত্রা শুরু হোক।`; শুধু নাম থাকলে `স্বাগতম, <নাম>!`; নাম ফাঁকা থাকলে আগের মতোই সতর্কবার্তা।

১.১৫ অনুশীলনীর সমাধান

একই সমস্যার একাধিক সঠিক সমাধান থাকতে পারে। আপনার Code নিচের সঙ্গে হুবহু না মিললেও প্রত্যাশিত ফল দিলে সেটি সঠিক।

সমাধান ১

about-me.js · JavaScript চলবে: Node.js

```
console.log('আমার নাম Sadia Rahman।');
console.log('আমি Rajshahi শহরে থাকি।');
console.log('আমি নিজের Website তৈরি করার জন্য JavaScript শিখছি।');
```

সমাধান ২ — প্রথম লাইন `15` দেখায়, কারণ Quotation Mark ছাড়া `20 - 5` একটি হিসাব। দ্বিতীয় লাইন `20 - 5` দেখায়, কারণ Quotation Mark-এর ভেতরের সবকিছু String।

সমাধান ৩ — তিনটি ভুল: প্রথম লাইনে `Console`-এর `C` বড় হাতের; দ্বিতীয় লাইনে শেষের Quotation Mark নেই; তৃতীয় লাইনে শেষের বন্ধনী নেই।

fix-me.js · JavaScript — চলবে: Node.js

```
console.log('শুরু');
console.log('মাঝখান');
console.log('শেষ');
```

Output

```
শুরু
মাঝখান
শেষ
```

এখানে একটি শিক্ষণীয় বিষয় আছে। মূল File-টি চালালে Node.js শুধু দ্বিতীয় লাইনের `SyntaxError` দেখায়, প্রথম লাইনের ভুলটি নয়। কারণ `SyntaxError` থাকলে কোনো লাইনই চলে না। সেটি ঠিক করার পর পরের ভুল ধরা পড়ে। ভুল তাই একটি একটি করে ঠিক করতে হয়।

সমাধান ৪

hello.js · JavaScript — চলবে: Node.js

```
// আমার প্রথম Program
/*
  এই File-টি Console-এ একটি বার্তা দেখায়।
  Comment যোগ করলে Output বদলায় না।
*/
console.log('Hello, JavaScript!');
```

সমাধান ৫

Terminal

```
mkdir practice
cd practice
pwd
```

শেষ Command-টি যে Path দেখায়, তার শেষ অংশ `practice` হবে।

সমাধান ৬

farewell.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createFarewell(name) {
  return `বিদায়, ${name}! আবার দেখা হবে।`;
}

console.log(createFarewell('Rahim'));
console.log(createFarewell('Nusrat'));
```

Output

```
বিদায়, Rahim! আবার দেখা হবে।
বিদায়, Nusrat! আবার দেখা হবে।
```

সমাধান ৭

describe-lesson.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describeLesson(title) {
  const cleanTitle = title.trim();

  if (cleanTitle === '') {
    return 'শিরোনাম দেওয়া হয়নি।';
  }

  return `আজকের পাঠ: ${cleanTitle}`;
}

console.log(describeLesson('  console.log()  '));
console.log(describeLesson('   '));
```

Output

```
আজকের পাঠ: console.log()
শিরোনাম দেওয়া হয়নি।
```

সমাধান ৮ — HTML-এ `<h1 id="page-title">আমার প্রথম Web Page</h1>` লিখুন এবং `script.js` নিচের মতো বদলান।

script.js · JavaScript চলবে: Browser

```
const pageTitle = document.querySelector('#page-title');
const statusText = document.querySelector('#status');
const changeButton = document.querySelector('#change-button');

function showMessage() {
  pageTitle.textContent = 'JavaScript শেখা চলছে';
  statusText.textContent = 'Button চাপা হয়েছে। JavaScript কাজ করছে!';
}

changeButton.addEventListener('click', showMessage);
```

সমাধান ৯ — `defer` না থাকলে Console-এ `Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')` দেখা যায়। কারণ Script চলার সময় Button তখনো তৈরি হয়নি, তাই `querySelector` `null` ফেরত দিয়েছে। বিকল্প সমাধান হলো `<script src="script.js"></script>` লাইনটি `<head>` থেকে সরিয়ে `</body>`-র ঠিক আগে বসানো। তখন Script চলার আগেই সব Element তৈরি হয়ে যায়।

সমাধান ১০ — `index.html`-এ বার্তার অনুচ্ছেদের নিচে `<p id="counter" class="intro">মোট শুভেচ্ছা: 0 বার</p>` যোগ করুন। `app.js`-এ দুটি নতুন লাইন Variable-গুলোর সঙ্গে, এবং দুটি লাইন `handleSubmit`-এর শেষে যোগ করুন।

app.js · JavaScript যোগ করার অংশ (Fragment) চলবে: Browser

```
const counter = document.querySelector('#counter');
let greetingCount = 0;

// handleSubmit Function-এর শেষে, সফল শুভেচ্ছা দেখানোর পরে:
greetingCount = greetingCount + 1;
counter.textContent = `মোট শুভেচ্ছা: ${greetingCount} বার`;
```

`greetingCount`-এর মান বদলাতে হয় বলে এখানে `let` ব্যবহার করা হয়েছে। লাইন দুটি ফাঁকা নামের `return`-এর পরে থাকায় সতর্কবার্তার ক্ষেত্রে সংখ্যা বাড়ে না।

সমাধান ১১ — `index.html`-এ Submit Button-এর নিচে `<button id="reset-button" type="button">মুছে ফেলুন</button>` যোগ করুন। `type="button"` গুরুত্বপূর্ণ: এটি না দিলে Form-এর ভেতরের Button চাপলে Form Submit হয়ে যায়।

app.js · JavaScript যোগ করার অংশ (Fragment) চলবে: Browser

```
const resetButton = document.querySelector('#reset-button');

function handleReset() {
  nameInput.value = '';
  message.textContent = '';
  message.className = 'message';
  nameInput.focus();
}

resetButton.addEventListener('click', handleReset);
```

সমাধান ১২ — `index.html`-এ নামের Input-এর নিচে `<label for="city">আপনার শহর</label>` এবং `<input id="city" name="city" type="text" maxlength="60">` যোগ করুন। তারপর `createGreeting`-এ দ্বিতীয় Parameter যোগ করুন।

app.js · JavaScript যোগ করার অংশ (Fragment) চলবে: Browser

```
function createGreeting(name, city) {
  const cleanName = name.trim();
  const cleanCity = city.trim();

  if (cleanName === '') {
    return '';
  }

  if (cleanCity === '') {
    return `স্বাগতম, ${cleanName}!`;
  }

  return `স্বাগতম, ${cleanName}! ${cleanCity} থেকে JavaScript শেখার যাত্রা শুরু হোক।`;
}

const cityInput = document.querySelector('#city');

// handleSubmit-এর ভেতরে createGreeting Call করার লাইনটি বদলান:
const greeting = createGreeting(nameInput.value, cityInput.value);
```

১.১৬ MCQ: নিজেকে যাচাই করুন

প্রতিটি প্রশ্নের একটিই সঠিক উত্তর। উত্তর ও ব্যাখ্যা অনুচ্ছেদ ১.১৭-এ আছে।

> **প্রশ্ন ১.** Web Page-এ JavaScript-এর প্রধান ভূমিকা কী?
>
> ক) Page-এর কাঠামো তৈরি করা
>
> খ) Page-এর রং ও Font ঠিক করা
>
> গ) Page-এর আচরণ নিয়ন্ত্রণ করা
>
> ঘ) Page-কে Internet-এ প্রকাশ করা

> **প্রশ্ন ২.** Browser-এর বাইরে JavaScript চালাতে এই বইয়ে কোনটি ব্যবহার করা হয়েছে?
>
> ক) Node.js
>
> খ) HTML
>
> গ) CSS
>
> ঘ) VS Code

> **প্রশ্ন ৩.** JavaScript এবং Java সম্পর্কে কোন বক্তব্যটি সঠিক?
>
> ক) JavaScript হলো Java-র সংক্ষিপ্ত সংস্করণ
>
> খ) Java শিখলে JavaScript আপনাআপনি জানা হয়ে যায়
>
> গ) দুটি একই ভাষার দুটি নাম
>
> ঘ) দুটি সম্পূর্ণ আলাদা ভাষা

> **প্রশ্ন ৪.** `console.log('8 + 2');` কী দেখাবে?
>
> ক) `10`
>
> খ) `8 + 2`
>
> গ) `'8 + 2'`
>
> ঘ) একটি Error

> **প্রশ্ন ৫.** নিচের কোন লাইনটি কোনো Error ছাড়া চলবে?
>
> ক) `console.log('Hello);`
>
> খ) `Console.log('Hello');`
>
> গ) `console.log("Hello");`
>
> ঘ) `console.log('Hello';`

প্রশ্ন ৬. `consol.log('Hi');` চালালে কোন ধরনের Error দেখা যায়?

ক) `SyntaxError`

খ) `ReferenceError`

গ) `TypeError`

ঘ) কোনো Error দেখা যায় না

প্রশ্ন ৭. `<script src="app.js" defer></script>`-এ `defer`-এর কাজ কী?

ক) Script-টি পুরোপুরি বন্ধ রাখে

খ) Script-টি দুবার চালায়

গ) HTML পড়া শেষ হওয়ার পর Script চালায়

ঘ) Script-টিকে CSS File-এ রূপান্তর করে

প্রশ্ন ৮. ব্যবহারকারীর লেখা Page-এ নিরাপদে দেখাতে কোনটি ব্যবহার করা উচিত?

ক) `textContent`

খ) `innerHTML`

গ) `console.log`

ঘ) `querySelector`

প্রশ্ন ৯. `document.querySelector('#name')` কী খুঁজে আনে?

ক) `name` নামের একটি JavaScript File

খ) `name` নামের একটি Function

গ) Page-এর সব Element

ঘ) সেই Element, যার `id` হলো `name`

প্রশ্ন ১০. `'  '.trim() === ''` —এই তুলনার ফল কী?

ক) সত্য, কারণ `trim()` শুরু ও শেষের ফাঁকা জায়গা বাদ দেয়

খ) মিথ্যা, কারণ String-টিতে তিনটি অক্ষর আছে

গ) `SyntaxError`

ঘ) `undefined`

১.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–খ · ৭–গ · ৮–ক · ৯–ঘ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	HTML কাঠামো দেয়, CSS চেহারা দেয়, আর JavaScript আচরণ নিয়ন্ত্রণ করে।
২	ক	Node.js হলো Browser-এর বাইরে JavaScript চালানোর Runtime। VS Code একটি Editor; এটি নিজে Code চালায় না।
৩	ঘ	নামের মিল ঐতিহাসিক। দুটি ভাষার নিয়ম ও ব্যবহার আলাদা।
৪	খ	Quotation Mark-এর ভেতরের সবকিছু String, তাই হিসাব হয় না। Output-এ Quotation Mark দেখায় না।
৫	গ	Double Quote দিয়ে শুরু ও শেষ হওয়া String বৈধ। ক-তে শেষের Quotation Mark নেই, খ-তে `C` বড় হাতের, ঘ-তে শেষের বন্ধনী নেই।
৬	খ	`consol` নামে কিছু নেই, তাই JavaScript নামটি চিনতে পারে না: `ReferenceError: consol is not defined`।
৭	গ	`defer` থাকলে Browser আগে পুরো HTML পড়ে, তারপর Script চালায়। ফলে Element-গুলো তখন প্রস্তুত থাকে।
৮	ক	`textContent` সবকিছুকে সাধারণ লেখা হিসেবে দেখায়। `innerHTML` লেখাকে HTML হিসেবে ব্যাখ্যা করে, যা ঝুঁকিপূর্ণ।
৯	ঘ	`#` চিহ্ন `id` বোঝায়। `querySelector` মিলে যাওয়া প্রথম Element-টি ফেরত দেয়।
১০	ক	শুধু ফাঁকা জায়গার String `trim()`-এর পর ফাঁকা String হয়ে যায়, যা `''`-এর সঙ্গে হুবহু মেলে।

১.১৮ অধ্যায়ের সারসংক্ষেপ

- Program হলো Computer-কে দেওয়া ধাপে ধাপে নির্দেশনা; JavaScript সেই নির্দেশনা লেখার একটি ভাষা।
- Web-এ HTML কাঠামো, CSS চেহারা এবং JavaScript আচরণের দায়িত্ব পালন করে।
- JavaScript Browser-এ এবং Node.js-এ চলে। ভাষা একই, পরিবেশ ভিন্ন। JavaScript এবং Java আলাদা ভাষা।
- কাজের পরিবেশে তিনটি জিনিস লাগে: একটি আধুনিক Browser, VS Code এবং Node.js-এর LTS সংস্করণ।
- `console.log()` Developer-এর দেখার জন্য Console-এ বার্তা দেখায়।
- String Quotation Mark-এ লেখা হয়; Statement-এর শেষে Semicolon বসে; Comment চালানো হয় না; JavaScript Case-sensitive।
- `const` ও `let` দিয়ে Variable তৈরি হয়; Function নির্দেশনাকে পুনর্ব্যবহারযোগ্য করে; `if` দিয়ে শর্ত অনুযায়ী সিদ্ধান্ত নেওয়া হয়।
- Error Message-এ File, লাইন নম্বর, ভুলের জায়গা এবং ধরন থাকে। `SyntaxError`, `ReferenceError` এবং `TypeError` সবচেয়ে প্রচলিত।

- `<script src="..." defer>` দিয়ে External JavaScript File যুক্ত করা হয়। `defer` নিশ্চিত করে যে Script চলার আগে Page প্রস্তুত।
- `querySelector` দিয়ে Element খোঁজা হয়, `textContent` দিয়ে নিরাপদে লেখা বদলানো হয়, এবং `addEventListener` দিয়ে Event-এ সাড়া দেওয়া হয়।

পরের অধ্যায়ে Variable এবং Data Type নিয়ে বিস্তারিত আলোচনা করা হবে: JavaScript কী কী ধরনের মান নিয়ে কাজ করে এবং সেগুলো কীভাবে সংরক্ষণ করে।

১.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Program	Computer-এর জন্য লেখা ধাপে ধাপে নির্দেশনা।
Runtime	যে Software JavaScript Code চালায়, যেমন Browser বা Node.js।
Frontend / Backend	Application-এর ব্যবহারকারী-মুখী অংশ / Server-এ চলা অংশ।
Terminal	লিখে Command দেওয়ার জানালা।
Console	Developer-এর জন্য বার্তা ও Error দেখানোর জায়গা।
Syntax	ভাষার গঠনগত নিয়ম।
String	Quotation Mark-এ ঘেরা লেখা।
Statement	একটি সম্পূর্ণ নির্দেশ।
Comment	মানুষের জন্য লেখা টীকা, যা চালানো হয় না।
Variable	একটি নাম, যার সঙ্গে একটি মান যুক্ত।
Function	নাম দেওয়া, পুনর্ব্যবহারযোগ্য একগুচ্ছ নির্দেশনা।
Parameter	Function চালানোর সময় দেওয়া মান যে নামে পাওয়া যায়।
Element	শুরুর Tag থেকে শেষের Tag পর্যন্ত HTML-এর একটি অংশ।
DOM	Browser-এর তৈরি Page-এর কাঠামো, যা JavaScript পড়তে ও বদলাতে পারে।
Event	ব্যবহারকারীর কাজ বা Browser-এর ঘটনা, যেমন Click বা Submit।
Bug / Debugging	Program-এর ভুল / ভুল খুঁজে ঠিক করার কাজ।
Stack Trace	Error পর্যন্ত পৌঁছাতে কোন Function থেকে কোনটি Call হয়েছে, তার তালিকা।

১.২০ আরও পড়ুন

নিচের উৎসগুলো সংশ্লিষ্ট প্রযুক্তির নির্মাতা বা রক্ষণাবেক্ষণকারী প্রতিষ্ঠানের নিজস্ব Documentation। অধিকাংশ লেখা ইংরেজিতে, তবে এই অধ্যায়ের পরিভাষাগুলো জানা থাকলে পড়া সহজ হবে।

- **MDN Web Docs — JavaScript Guide:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide` — JavaScript ভাষার ধাপে ধাপে নির্দেশিকা।
- **MDN Web Docs — Learn Web Development:** `https://developer.mozilla.org/en-US/docs/Learn_web_development` — HTML, CSS ও JavaScript একসঙ্গে শেখার পাঠক্রম।
- **Node.js Documentation:** `https://nodejs.org/en/learn` — Node.js Install করা এবং ব্যবহার শুরুর নির্দেশনা।
- **Visual Studio Code Documentation:** `https://code.visualstudio.com/docs` — Editor এবং Integrated Terminal ব্যবহারের নির্দেশনা।
- **Chrome DevTools Documentation:** `https://developer.chrome.com/docs/devtools` — Console এবং অন্যান্য Developer Tool-এর ব্যবহার।

অধ্যায় ২

Variables, Data Types এবং Memory

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- `let`, `const` এবং `var`-এর পার্থক্য ব্যাখ্যা করে সঠিকটি বেছে নিতে।
- JavaScript-এর Primitive Type-গুলো চিনতে এবং `typeof` দিয়ে যাচাই করতে।
- `undefined` এবং `null`-এর পার্থক্য বুঝে সঠিক জায়গায় ব্যবহার করতে।
- String থেকে Number-এ এবং অন্য Type-এ নিরাপদে রূপান্তর করতে।
- `===` এবং `==`-এর পার্থক্য ব্যাখ্যা করতে।
- Primitive মান এবং Object Reference মেমোরিতে কীভাবে আলাদা আচরণ করে, তা বুঝতে।
- Type-সংক্রান্ত সাধারণ Bug খুঁজে বের করতে।
- একটি সম্পূর্ণ **Student Result Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ১ সম্পন্ন করা: `console.log()`, String, Function-এর প্রাথমিক পরিচয় এবং `if`।
- Node.js এবং VS Code Install করা আছে, এবং Terminal থেকে `node file.js` চালাতে পারেন।
- `chapter-02` নামে একটি নতুন Folder, যেখানে এই অধ্যায়ের File-গুলো রাখবেন।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি Program আসলে তথ্য নিয়ে কাজ করে: নাম, নম্বর, দাম, তারিখ, হ্যাঁ অথবা না। এই তথ্য কোথায় রাখা হবে, কোন ধরনের তথ্য কীভাবে আচরণ করে এবং এক ধরন থেকে আরেক ধরনে গেলে কী ঘটে—এই তিনটি প্রশ্নের উত্তরই এই অধ্যায়ের বিষয়।

বিষয়গুলো সহজ মনে হলেও বাস্তব Program-এর বহু Bug এখান থেকেই আসে। একটি Form থেকে আসা `'80'` এবং `'90'` যোগ করলে `170` না হয়ে `8090` হয়ে যায়। একটি তালিকা Copy করে বদলালে মূল তালিকাটিও বদলে যায়। ফাঁকা ঘরের মান শূন্য ধরে নেওয়া হয়। প্রতিটি সমস্যার কারণ এই অধ্যায়ে পরিষ্কার হবে।

অধ্যায়ের শেষে আপনি একটি ফলাফল হিসাবের Program লিখবেন, যেখানে Text হিসেবে আসা নম্বর যাচাই করে সংখ্যায় রূপান্তর করা হয়, তারপর মোট, গড় এবং Grade বের করা হয়।

২.১ Variable: মানের নাম

অধ্যায় ১-এ আপনি দেখেছেন, **Variable** হলো একটি নাম, যার সঙ্গে একটি মান যুক্ত থাকে। মান যুক্ত করার কাজটিকে **Assignment** বলে, আর `=` চিহ্নটি Assignment Operator।

variables.js · JavaScript — চলবে: Browser Console ও Node.js

```
const studentName = 'Sara';
let totalMarks = 240;

console.log(studentName);
console.log(totalMarks);

totalMarks = 250;
console.log(totalMarks);
```

Output

```
Sara
240
250
```

প্রথম দুই লাইনে দুটি Variable তৈরি করা হয়েছে। Variable তৈরি করাকে **Declaration** বলে। ষষ্ঠ লাইনে `totalMarks` -এ নতুন মান রাখা হয়েছে; একে **Reassignment** বলে।

Variable-এর নাম রাখার নিয়ম

নিয়ম	সঠিক	ভুল
অক্ষর, সংখ্যা, `_` এবং `$` ব্যবহার করা যায়	`total1`, `_count`	`total-marks`
সংখ্যা দিয়ে শুরু করা যায় না	`mark1`	`1mark`
ফাঁকা জায়গা থাকতে পারে না	`studentName`	`student name`
ভাষার নিজস্ব শব্দ ব্যবহার করা যায় না	`result`	`const`, `return`
বড় ও ছোট হাতের অক্ষর আলাদা	`total` এবং `Total` দুটি ভিন্ন নাম	—

নিয়ম মানলেই নাম ভালো হয় না। `x` বা `data` নাম থেকে কিছু বোঝা যায় না; `totalMarks` বা `studentName` পড়লেই বোঝা যায় ভেতরে কী আছে। এই বইয়ে camelCase রীতি ব্যবহার করা হয়েছে।

২.২ let, const এবং var

JavaScript-এ Variable তৈরি করার তিনটি শব্দ আছে। পার্থক্যগুলো এক নজরে:

বৈশিষ্ট্য	`const`	`let`	`var`
নতুন মান রাখা যায়	না	হ্যাঁ	হ্যাঁ
তৈরির সময় মান দেওয়া বাধ্যতামূলক	হ্যাঁ	না	না
একই নামে আবার তৈরি করা যায়	না	না	হ্যাঁ
কোথায় কার্যকর	যে `{ }`-এর ভেতরে তৈরি	যে `{ }`-এর ভেতরে তৈরি	পুরো Function জুড়ে

`const` : যে নামের মান বদলাবে না

const-error.js · JavaScript — চলবে: Node.js

```
const passMark = 33;
passMark = 40;
```

Error Message

```
TypeError: Assignment to constant variable.
```

`const` দিয়ে তৈরি নামে দ্বিতীয়বার মান রাখতে গেলে JavaScript Program থামিয়ে দেয়। এটি অসুবিধা নয়, বরং সুরক্ষা: যে মান বদলানোর কথা নয়, সেটি ভুল করে বদলে গেলে সঙ্গে সঙ্গে জানা যায়।

`let` : যে নামের মান বদলাবে

গণনা, যোগফল বা অবস্থা—যে মান Program চলার সময় বদলায়, তার জন্য `let`।

Block এবং Scope

দ্বিতীয় বন্ধনী `{ }`-এর ভেতরের অংশকে **Block** বলে। কোনো Variable Program-এর যে অংশ থেকে ব্যবহার করা যায়, সেই অংশকে তার **Scope** বলে। `let` এবং `const` দিয়ে তৈরি Variable শুধু নিজের Block-এর ভেতরে কার্যকর।

block-scope.js · JavaScript — চলবে: Browser Console ও Node.js

```
let count = 1;

if (true) {
  let count = 2;
  console.log(count);
}

console.log(count);
```

Output

```
2
1
```

Block-এর ভেতরের `count` একটি সম্পূর্ণ আলাদা Variable। Block শেষ হলে সেটি আর থাকে না, এবং বাইরের `count` অপরিবর্তিত থাকে। Scope নিয়ে বিস্তারিত আলোচনা অধ্যায় ৫-এ আছে।

`var` : পুরোনো পদ্ধতি

`var` JavaScript-এর শুরুর দিনের পদ্ধতি। পুরোনো Code পড়ার জন্য এটি চেনা দরকার, কিন্তু এর দুটি আচরণ সহজেই Bug তৈরি করে।

var-problems.js · JavaScript — চলবে: Browser Console ও Node.js

```
var city = 'Dhaka';
var city = 'Sylhet';
console.log(city);

if (true) {
  var leaked = 'Block-এর ভেতরে তৈরি';
}

console.log(leaked);
```

Output

```
Sylhet
Block-এর ভেতরে তৈরি
```

প্রথমত, একই নামে দুবার `var` লিখলে কোনো Error হয় না; দ্বিতীয়টি নীরবে প্রথমটিকে মুছে দেয়। বড় File-এ এভাবে অন্যের Variable নষ্ট হয়ে যেতে পারে। দ্বিতীয়ত, `var` Block মানে না: `if`-এর ভেতরে তৈরি `leaked` বাইরে থেকেও পাওয়া যাচ্ছে।

> **মনে রাখুন:** প্রথমে `const` ব্যবহার করুন। মান বদলানোর প্রয়োজন হলে তবেই `let`। নতুন Code-এ `var` ব্যবহার করবেন না।

তৈরি করার আগে ব্যবহার

`let` বা `const` দিয়ে তৈরি Variable তার Declaration-এর লাইনের আগে ব্যবহার করা যায় না।

before-declaration.js · JavaScript — চলবে: Node.js

```javascript
console.log(score);
let score = 95;
```

Error Message

```
ReferenceError: Cannot access 'score' before initialization
```

বার্তাটি স্পষ্ট: `score` আছে, কিন্তু এখনো প্রস্তুত নয়। সমাধান হলো Variable আগে তৈরি করা, পরে ব্যবহার করা।

২.৩ Data Type এবং typeof

প্রতিটি মানের একটি ধরন আছে, যাকে **Data Type** বলে। ধরন ঠিক করে দেয় মানটি নিয়ে কী করা যাবে: সংখ্যা গুণ করা যায়, লেখা যায় না।

JavaScript-এ সাতটি **Primitive Type** আছে। Primitive মানে মৌলিক: এগুলো একটি করে সরল মান, যা ভেঙে আর ছোট করা যায় না।

Type	কী রাখে	উদাহরণ
`string`	লেখা	`'Dhaka'`
`number`	পূর্ণসংখ্যা ও দশমিক সংখ্যা	`42`, `3.14`
`boolean`	সত্য অথবা মিথ্যা	`true`, `false`
`undefined`	এখনো কোনো মান দেওয়া হয়নি	`undefined`

Type	কী রাখে	উদাহরণ
`null`	ইচ্ছাকৃতভাবে "কিছু নেই"	`null`
`bigint`	অত্যন্ত বড় পূর্ণসংখ্যা	`9007199254740993n`
`symbol`	অনন্য পরিচয়চিহ্ন	`Symbol('id')`

প্রথম পাঁচটি প্রতিদিন লাগবে। `bigint` ও `symbol` বিশেষ ক্ষেত্রে ব্যবহৃত হয়; এখন শুধু নাম জেনে রাখুন। Primitive নয় এমন সব মান **Object**, যা অনুচ্ছেদ ২.৯-এ আসবে।

কোনো মানের Type জানতে `typeof` ব্যবহার করুন।

typeof.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(typeof 'Dhaka');
console.log(typeof 42);
console.log(typeof 3.14);
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof 10n);
console.log(typeof Symbol('id'));
console.log(typeof null);
```

Output

```
string
number
number
boolean
undefined
bigint
symbol
object
```

দুটি বিষয় লক্ষ করুন। পূর্ণসংখ্যা ও দশমিক সংখ্যা—দুটোরই Type `number`; JavaScript-এ এদের আলাদা Type নেই। আর `typeof null` দেখায় `object`, যদিও `null` একটি Primitive। এটি ভাষার একটি পুরোনো ত্রুটি, যা পুরোনো Website ভেঙে যাওয়ার আশঙ্কায় আর ঠিক করা হয়নি। কোনো মান `null` কি না জানতে `typeof` নয়, `=== null` ব্যবহার করুন।

২.৪ Number-এর আচরণ

Computer দশমিক সংখ্যা নির্দিষ্ট পরিমাণ জায়গায় রাখে, তাই কিছু সংখ্যা হুবহু রাখা সম্ভব হয় না। ফলে কয়েকটি ফলাফল প্রথমবার অবাক করে।

numbers.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(0.1 + 0.2);
console.log(10 / 3);
console.log(5 / 0);
console.log(Number('abc'));
console.log(typeof NaN);
console.log(NaN === NaN);
console.log(Number.isNaN(Number('abc')));
```

Output

```
0.30000000000000004
3.3333333333333335
Infinity
NaN
number
false
true
```

- `0.1 + 0.2` ঠিক `0.3` হয় না। এটি JavaScript-এর ত্রুটি নয়; প্রায় সব Programming Language একই পদ্ধতিতে দশমিক সংখ্যা রাখে।
- শূন্য দিয়ে ভাগ করলে Error হয় না; ফল হয় `Infinity`।
- `NaN`-এর পূর্ণরূপ Not a Number। যে হিসাবের অর্থপূর্ণ সংখ্যা-ফল নেই, তার ফল `NaN`। অদ্ভুত শোনালেও এর Type `number`।
- `NaN` নিজের সমানও নয়, তাই `=== NaN` দিয়ে একে ধরা যায় না। সব সময় `Number.isNaN()` ব্যবহার করুন।

> **সতর্কতা:** টাকার হিসাবে দশমিক সংখ্যা ব্যবহার করবেন না। `0.1 + 0.2`-এর মতো সামান্য ভুল হাজার হাজার হিসাবে জমে বড় হয়ে যায়। টাকাকে পয়সায় (পূর্ণসংখ্যায়) রেখে হিসাব করুন, দেখানোর সময় টাকায় রূপান্তর করুন।

দশমিকের পরে নির্দিষ্ট সংখ্যক ঘর দেখাতে `toFixed()` ব্যবহার করা হয়। এটি একটি String ফেরত দেয়, তাই শুধু দেখানোর জন্য ব্যবহার করুন, হিসাবের জন্য নয়।

to-fixed.js · JavaScript চলবে: Browser Console ও Node.js

```
const average = 220 / 3;

console.log(average);
console.log(average.toFixed(2));
console.log(typeof average.toFixed(2));
```

Output

```
73.33333333333333
73.33
string
```

২.৫ undefined এবং null

দুটোই "মান নেই" বোঝায়, কিন্তু কারণ আলাদা।

- `undefined` — JavaScript নিজে দেয়। Variable তৈরি হয়েছে, কিন্তু কোনো মান রাখা হয়নি।
- `null` — Programmer ইচ্ছা করে দেন। অর্থ: "এখানে জেনেশুনেই কিছু রাখা হয়নি।"

undefined-null.js · JavaScript চলবে: Browser Console ও Node.js

```
let email;
const middleName = null;

console.log(email);
console.log(middleName);
console.log(email === undefined);
console.log(middleName === null);
```

Output

```
undefined
null
true
true
```

একটি নিবন্ধন Form-এর কথা ভাবুন। Email-এর ঘরটি এখনো পূরণ করা হয়নি: অবস্থাটি `undefined`। ব্যবহারকারী জানিয়েছেন তাঁর মধ্য নাম নেই: অবস্থাটি `null`। প্রথমটি "জানা নেই", দ্বিতীয়টি "জানা আছে যে নেই"।

নিজে কখনো কোনো Variable-এ `undefined` রাখবেন না। "কিছু নেই" বোঝাতে হলে `null` ব্যবহার করুন; তাহলে `undefined` দেখলেই বোঝা যাবে কোথাও মান দিতে ভুল হয়েছে।

২.৬ Dynamic Typing

JavaScript-এ Type থাকে মানের, Variable-এর নয়। একই Variable-এ প্রথমে সংখ্যা, পরে লেখা রাখা যায়। এই বৈশিষ্ট্যকে **Dynamic Typing** বলে।

dynamic-typing.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
let value = 42;
console.log(typeof value);

value = 'বিয়াল্লিশ';
console.log(typeof value);

value = true;
console.log(typeof value);
```

Output

```
number
string
boolean
```

এতে Code লেখা সহজ হয়, কিন্তু দায়িত্বও বাড়ে: কোনো Variable-এ এই মুহূর্তে কোন Type-এর মান আছে, তা JavaScript আপনাকে আগে থেকে জানায় না। তাই একটি Variable-কে একটিই কাজে এবং একটিই Type-এ ব্যবহার করা ভালো অভ্যাস। `totalMarks` -এ সব সময় সংখ্যাই রাখুন।

২.৭ Type Conversion

এক Type-এর মানকে অন্য Type-এ বদলানোকে **Type Conversion** বলে। এটি দুইভাবে ঘটে: আপনি নিজে করলে **Explicit Conversion**, আর JavaScript নিজে থেকে করলে **Implicit Conversion** বা Coercion।

Explicit Conversion: নিজে রূপান্তর করা

explicit-conversion.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
console.log(Number('42'));
console.log(Number('  7  '));
console.log(Number('12px'));
console.log(Number(''));
console.log(Number(true));
console.log(Number(null));
console.log(Number(undefined));
console.log(typeof String(42));
console.log(Boolean(0));
console.log(Boolean(''));
console.log(Boolean('0'));
```

Output

```
42
7
NaN
0
1
0
NaN
string
false
false
true
```

রূপান্তর	নিয়ম
`Number(text)`	পুরো লেখাটি সংখ্যা হলে সেই সংখ্যা; আগে-পরের ফাঁকা জায়গা উপেক্ষিত; অন্যথায় `NaN`
`Number('')`	`0` — ফাঁকা String শূন্য হয়ে যায়
`String(value)`	মানটির লিখিত রূপ
`Boolean(value)`	`0`, `''`, `null`, `undefined`, `NaN` হলে `false`; বাকি সব `true`

`Boolean('0')`-এর ফল `true`, কারণ `'0'` একটি ফাঁকা নয় এমন String। কোন মান সত্য আর কোনটি মিথ্যা হিসেবে গণ্য হয়, তা অধ্যায় ৩-এ বিস্তারিত আসবে।

> **সতর্কতা:** `Number('')` এর ফল `0`। Form-এর ফাঁকা ঘরকে সরাসরি `Number()` দিলে সেটি শূন্য নম্বর হয়ে যাবে, আর Program বুঝতেই পারবে না যে ঘরটি পূরণ করা হয়নি। রূপান্তরের আগে সব সময় ফাঁকা কি না যাচাই করুন।

parseInt এবং parseFloat

`Number()` কঠোর: লেখার কোনো অংশ সংখ্যা না হলে পুরো ফলই `NaN`। কখনো কখনো লেখার শুরুর সংখ্যাটুকু দরকার হয়, যেমন `'12px'` থেকে `12`। সে কাজে `parseInt()` এবং `parseFloat()` ব্যবহার করা হয়।

parse-int.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Number('12px'));
console.log(parseInt('12px', 10));
console.log(parseFloat('3.5kg'));
console.log(parseInt('80.9', 10));
console.log(parseInt('abc', 10));
```

Output

```
NaN
12
3.5
80
NaN
```

`parseInt()` বাঁ দিক থেকে পড়ে এবং প্রথম যে অক্ষরটি সংখ্যার অংশ নয়, সেখানে থেমে যায়। দশমিক বিন্দুও তার কাছে থামার জায়গা, তাই `'80.9'` থেকে `80` পাওয়া যায়; এটি সংখ্যাকে নিকটতম মানে আনে না, শুধু কেটে ফেলে। দ্বিতীয় মান `10` জানায় যে সংখ্যাটি দশভিত্তিক; এটি সব সময় লিখুন।

এই নমনীয়তা যাচাইয়ের কাজে বিপজ্জনক। কেউ নম্বরের ঘরে `'80abc'` লিখলে `parseInt()` সেটিকে নীরবে `80` হিসেবে গ্রহণ করবে। তাই ব্যবহারকারীর দেওয়া মান যাচাই করতে `Number()` ব্যবহার করুন, আর `parseInt()` রাখুন সেই ক্ষেত্রের জন্য, যেখানে লেখার শেষে একক থাকাই স্বাভাবিক।

Implicit Conversion: JavaScript যখন নিজে রূপান্তর করে

implicit-conversion.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('5' + 2);
console.log('5' - 2);
console.log('5' * '2');
console.log(1 + 2 + '3');
console.log('1' + 2 + 3);
```

Output

```
52
3
10
33
123
```

নিয়মটি `+` চিহ্নকে ঘিরে। `+`-এর দুটি কাজ: সংখ্যা যোগ করা এবং String জোড়া লাগানো। দুই পাশের যেকোনো একটি String হলে JavaScript অন্যটিকেও String বানিয়ে জোড়া লাগায়। `-` এবং `*`-এর এমন দ্বৈত ভূমিকা নেই, তাই সেখানে String-কে সংখ্যায় রূপান্তর করা হয়।

শেষ দুই লাইন বাঁ থেকে ডানে হিসাব হয়। `1 + 2 + '3'`-তে প্রথমে `1 + 2` হয় `3`, তারপর `3 + '3'` হয় `'33'`। `'1' + 2 + 3`-তে প্রথম ধাপেই String তৈরি হয়ে যায়: `'12'`, তারপর `'123'`।

Implicit Conversion-এর ওপর নির্ভর করবেন না। যেখানে সংখ্যা দরকার, সেখানে `Number()` দিয়ে নিজে রূপান্তর করুন।

রূপান্তরের ফল যাচাই করা

validate-number.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(Number.isNaN(Number('abc')));
console.log(Number.isInteger(42));
console.log(Number.isInteger(42.5));
console.log(Number.isInteger(Number('80')));
```

Output

```
true
true
false
true
```

`Number.isNaN()` জানায় রূপান্তর ব্যর্থ হয়েছে কি না। `Number.isInteger()` জানায় মানটি পূর্ণসংখ্যা কি না; `NaN` এবং দশমিক সংখ্যা—দুটোর জন্যই এটি `false` দেয়। Project-এ এই দুটিই কাজে লাগবে।

২.৮ সমতা যাচাই: === এবং ==

দুটি মান সমান কি না যাচাইয়ের দুটি Operator আছে।

- `===` (Strict Equality) — Type এবং মান দুটোই মিলতে হবে।
- `==` (Loose Equality) — Type আলাদা হলে আগে রূপান্তর করে, তারপর তুলনা করে।

equality.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(0 === '0');
console.log(0 == '0');
console.log('' == 0);
console.log(null == undefined);
console.log(null === undefined);
console.log(5 !== '5');
```

Output

```
false
true
true
true
false
true
```

`==`-এর রূপান্তরের নিয়মগুলো মনে রাখা কঠিন এবং ফলাফল প্রায়ই অপ্রত্যাশিত: ফাঁকা String আর শূন্য "সমান" হয়ে যায়। তাই এই বইয়ে সব সময় `===` ব্যবহার করা হয়েছে। "সমান নয়" বোঝাতে `!==` ব্যবহার করুন; শেষ লাইনে সংখ্যা `5` এবং String `'5'` আলাদা Type, তাই ফল `true`।

২.৯ Memory Model: মান এবং Reference

এ পর্যন্ত দেখা সব মান ছিল Primitive। এবার **Object**-এর পরিচয়: Object হলো একাধিক তথ্যকে একসঙ্গে রাখার একটি মান। দ্বিতীয় বন্ধনীর ভেতরে নাম ও মানের জোড়া লিখে Object তৈরি করা হয়, এবং ডট (`.`) দিয়ে ভেতরের তথ্য পড়া হয়। Object নিয়ে পূর্ণ আলোচনা অধ্যায় ৭-এ; এখানে শুধু মেমোরির আচরণটি দেখা হবে।

Primitive: মান Copy হয়

primitive-copy.js · JavaScript — চলবে: Browser Console ও Node.js

```
let a = 5;
let b = a;

b = 9;

console.log(a);
console.log(b);
```

Output

```
5
9
```

`let b = a` লেখার সময় `5` মানটির একটি Copy `b`-তে রাখা হয়েছে। এরপর `a` এবং `b`-র মধ্যে কোনো সম্পর্ক নেই।

Object: ঠিকানা Copy হয়

object-reference.js · JavaScript — চলবে: Browser Console ও Node.js

```
const first = { name: 'Nila' };
const second = first;

second.name = 'Nila Islam';

console.log(first.name);
console.log(first === second);

const third = { name: 'Nila Islam' };
console.log(first === third);
```

Output

```
Nila Islam
true
false
```

Object তৈরি হলে সেটি মেমোরির কোনো এক জায়গায় থাকে, আর Variable-এ রাখা হয় সেই জায়গার ঠিকানা। এই ঠিকানাকে **Reference** বলে। `const second = first` Object-টি Copy করে না; শুধু ঠিকানাটি Copy করে। ফলে দুটি নাম একই Object দেখায়, এবং একটির মাধ্যমে বদলালে অন্যটিতেও পরিবর্তন দেখা যায়।

	Primitive	Object
Variable-এ যা থাকে	মানটি নিজে	মানটির ঠিকানা (Reference)
`b = a` করলে	মান Copy হয়	ঠিকানা Copy হয়; Object একটিই থাকে
`===` যা তুলনা করে	মান	ঠিকানা

শেষ সারিটি `first === third`-এর ফল ব্যাখ্যা করে। দুটি Object-এর ভেতরের তথ্য একই, কিন্তু এরা মেমোরির দুটি আলাদা জায়গায় আছে। ঠিকানা আলাদা, তাই ফল `false`।

`const` এবং Object

const-object.js · JavaScript চলবে: Browser Console ও Node.js

```
const profile = { name: 'Rafi' };

profile.name = 'Rafi Ahmed';

console.log(profile.name);
```

Output

```
Rafi Ahmed
```

`const` থাকা সত্ত্বেও কোনো Error হয়নি। কারণ `const` শুধু নিশ্চিত করে যে `profile` নামটি সব সময় একই Object-এর ঠিকানা ধরে রাখবে। Object-এর ভেতরের তথ্য বদলানো নিষেধ নয়। নিষেধ হলো `profile = { ... }` লিখে নামটিকে অন্য Object-এ সরিয়ে নেওয়া।

Function-এ মান পাঠালে কী ঘটে

একই নিয়ম Function Call-এর সময়ও খাটে। Function-এ Primitive পাঠালে মানের Copy যায়; Object পাঠালে ঠিকানার Copy যায়।

pass-to-function.js · JavaScript চলবে: Browser Console ও Node.js

```
function addBonus(mark) {
  mark = mark + 5;
  return mark;
}

const mark = 70;
const updated = addBonus(mark);
console.log(mark);
console.log(updated);

function promote(student) {
  student.level = 2;
}

const student = { name: 'Mina', level: 1 };
promote(student);
console.log(student.level);
```

Output

```
70
75
2
```

`addBonus`-এর ভেতরের `mark` বাইরের `mark`-এর একটি Copy; ভেতরে বদলালেও বাইরের মান `70`-ই থাকে। কিন্তু `promote` যে ঠিকানা পেয়েছে, সেটি বাইরের Object-টিরই ঠিকানা। তাই Function-এর ভেতরে করা পরিবর্তন বাইরেও দেখা যায়।

এই আচরণ জানা না থাকলে খুঁজে পাওয়া কঠিন Bug তৈরি হয়: একটি Function "শুধু হিসাব করার জন্য" একটি Object নিল, আর অজান্তে সেটি বদলে দিল। অভ্যাস হিসেবে, Function-এর ভেতরে পাওয়া Object বদলাবেন না; প্রয়োজন হলে নতুন Object তৈরি করে ফেরত দিন।

আলাদা Copy তৈরি করা

সত্যিকারের আলাদা Object দরকার হলে তিনটি ডট (`...`) ব্যবহার করে নতুন Object তৈরি করা যায়।

object-copy.js · JavaScript চলবে: Browser Console ও Node.js

```
const original = { name: 'Arif', city: 'Dhaka' };
const copy = { ...original };

copy.city = 'Khulna';

console.log(original.city);
console.log(copy.city);
console.log(original === copy);
```

Output

```
Dhaka
Khulna
false
```

`{ ...original }` একটি নতুন Object তৈরি করে এবং পুরোনোটির তথ্যগুলো তাতে বসায়। এই Copy শুধু প্রথম স্তরের জন্য: Object-এর ভেতরে আরেকটি Object থাকলে সেটির ঠিকানাই Copy হয়। বিষয়টি অধ্যায় ৭-এ আবার আসবে।

> **নোট:** যে Object-এর দিকে আর কোনো Variable নির্দেশ করে না, JavaScript সেটির মেমোরি নিজে থেকেই ফিরিয়ে নেয়। এই ব্যবস্থার নাম Garbage Collection। আপনাকে নিজে মেমোরি মুক্ত করতে হয় না।

২.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
String হিসেবে আসা সংখ্যা যোগ করা	`'80' + '90'` হয় `8090`	আগে `Number()` দিয়ে রূপান্তর করুন
ফাঁকা ঘরকে সরাসরি `Number()` দেওয়া	ফাঁকা ঘর `0` হয়ে যায়	রূপান্তরের আগে `=== ''` যাচাই করুন
`=== NaN` দিয়ে যাচাই	সব সময় `false`	`Number.isNaN()` ব্যবহার করুন
`const`-এ নতুন মান রাখা	`TypeError: Assignment to constant variable.`	মান বদলাতে হলে `let` ব্যবহার করুন
Declaration-এর আগে ব্যবহার	`ReferenceError: Cannot access ... before initialization`	Variable আগে তৈরি করুন
`==` ব্যবহার	`'' == 0` হয় `true`	`===` ব্যবহার করুন
Object "Copy" করতে `=` ব্যবহার	মূল Object-ও বদলে যায়	`{ ...object }` দিয়ে নতুন Object তৈরি করুন

ভুল	কী দেখা যায়	সমাধান
`typeof` দিয়ে `null` যাচাই	ফল আসে `object`	`=== null` ব্যবহার করুন
`toFixed()`-এর ফল দিয়ে হিসাব	সংখ্যা জোড়া লেগে যায়	হিসাব শেষ করে তবেই `toFixed()` দিন

২.১১ Debugging নির্দেশিকা

Type-সংক্রান্ত Bug-এ সাধারণত কোনো Error Message আসে না; Program চলে, কিন্তু ফল ভুল হয়। তাই এখানে মূল কৌশল হলো মান এবং তার Type একসঙ্গে দেখা।

debug-types.js · JavaScript — চলবে: Browser Console ও Node.js

```
const bangla = '80';
const english = '90';

console.log(bangla + english);
console.log('bangla:', bangla, typeof bangla);

const total = Number(bangla) + Number(english);
console.log('total:', total, typeof total);
```

Output

```
8090
bangla: 80 string
total: 170 number
```

`console.log()`-এ কমা দিয়ে একাধিক মান দেওয়া যায়। প্রথমে একটি Label, তারপর মান, তারপর `typeof`—এই তিনটি একসঙ্গে দেখলে সমস্যাটি পরিষ্কার হয়: `80` দেখতে সংখ্যার মতো, কিন্তু Type `string`।

ফল ভুল হলে এই ক্রমে যাচাই করুন:

১. সন্দেহজনক প্রতিটি Variable-এর মান এবং `typeof` একসঙ্গে দেখুন।

২. ফল `NaN` হলে পেছনের দিকে যান: কোন রূপান্তর বা হিসাবটি প্রথম `NaN` তৈরি করেছে?

৩. দুটি সংখ্যা জোড়া লেগে গেলে খুঁজুন কোনটি String হিসেবে এসেছে।

৪. ফাঁকা ঘর শূন্য হয়ে গেলে দেখুন `Number()`-এর আগে ফাঁকা যাচাই আছে কি না।

৫. কোনো Object অপ্রত্যাশিতভাবে বদলালে খুঁজুন আর কোন Variable একই Object-এর Reference ধরে আছে।

৬. `ReferenceError` এলে দেখুন Variable-টি ব্যবহারের আগে এবং একই Block-এ তৈরি হয়েছে কি না।

২.১২ পেশাদার অভ্যাস

- `const` **দিয়ে শুরু করুন।** কোন মান বদলায় আর কোনটি বদলায় না, তা Code পড়েই বোঝা যায়।
- **একটি Variable, একটি উদ্দেশ্য, একটি Type।** একই নামে কখনো সংখ্যা, কখনো লেখা রাখবেন না।
- **বাইরে থেকে আসা সব তথ্যকে String ধরে নিন।** Form, File বা Network থেকে আসা মান যাচাই করে তবেই রূপান্তর করুন।
- **নিজে রূপান্তর করুন।** `Number()`, `String()` স্পষ্টভাবে লিখুন; Implicit Conversion-এর ওপর ভরসা করবেন না।
- **সব সময়** `===` **এবং** `!==`**।**
- **"নেই" বোঝাতে** `null`**।** `undefined` যেন শুধু ভুলের সংকেত হয়।
- **বিশেষ সংখ্যাকে নাম দিন।** Code-এর মাঝখানে `33` না লিখে `const PASS_MARK = 33;` লিখুন। যে মান পুরো Program-এ স্থির, তার নাম বড় হাতের অক্ষরে লেখা একটি প্রচলিত রীতি।
- **টাকার হিসাব পূর্ণসংখ্যায়।**

২.১৩ Project: Student Result Calculator

এবার একটি Program লিখবেন, যা একজন শিক্ষার্থীর তিনটি বিষয়ের নম্বর থেকে মোট, গড়, Grade এবং উত্তীর্ণ হওয়ার অবস্থা বের করে। নম্বরগুলো ইচ্ছা করেই String হিসেবে দেওয়া হবে, কারণ বাস্তবে Form বা File থেকে তথ্য এভাবেই আসে।

Project-এর চাহিদা

- শিক্ষার্থীর নাম এবং বাংলা, ইংরেজি ও গণিতের নম্বর String হিসেবে নেওয়া হবে।
- প্রতিটি নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে। ফাঁকা, দশমিক বা সীমার বাইরের নম্বর গ্রহণ করা হবে না।
- মোট নম্বর, দুই দশমিক ঘরে গড় এবং Grade দেখাতে হবে।
- যেকোনো একটি বিষয়ে ৩৩-এর কম পেলে ফল হবে অনুত্তীর্ণ এবং Grade `F`।
- কোনো নম্বর ভুল হলে হিসাবের বদলে কোন বিষয়ের নম্বর ভুল, তা জানাতে হবে।

এখানে ব্যবহৃত Grade-এর সীমাগুলো শেখার জন্য একটি উদাহরণ; এটি কোনো শিক্ষা বোর্ডের সরকারি নিয়ম নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
student-result-calculator/
├── result.js
├── README.md
└── tests/
    └── result.test.js
```

`result.js` File-টি আপনি নিজে লিখবেন। `README.md` এবং Test File এই অধ্যায়ের Companion Source Code-এর `project/student-result-calculator/` Folder-এ আছে।

result.js

File-টি ১০৪ লাইনের, তাই তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` চিহ্নিত লাইনগুলো লিখতে হবে না, সেগুলো শুধু বোঝায় যে File-এর আরও অংশ আছে।

ধাপ ১: Text থেকে নিরাপদে নম্বর তৈরি

result.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const PASS_MARK = 33;

// Text থেকে নম্বর তৈরি করে। নম্বর গ্রহণযোগ্য না হলে null ফেরত দেয়।
function toMark(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const mark = Number(clean);

  if (Number.isInteger(mark) === false) {
    return null;
  }

  if (mark < 0) {
    return null;
  }

  if (mark > 100) {
    return null;
  }

  return mark;
}

// ...
```

`toMark` এই অধ্যায়ের মূল শিক্ষাগুলো এক জায়গায় আনে। প্রথমে ফাঁকা কি না যাচাই, কারণ `Number('')` শূন্য দেয়। তারপর রূপান্তর। `Number.isInteger(mark) === false` একসঙ্গে দুটি সমস্যা ধরে: `'abc'` থেকে আসা `NaN` এবং `'80.5'`-এর মতো দশমিক সংখ্যা। শেষে সীমা যাচাই; `<` মানে "ছোট" এবং `>` মানে "বড়" (তুলনার সব Operator অধ্যায় ৩-এ আছে)। কোনো যাচাইয়ে আটকে গেলে Function `null` ফেরত দেয়—অর্থাৎ "গ্রহণযোগ্য নম্বর নেই"।

ধাপ ২: গড় থেকে Grade

result.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// গড় নম্বর থেকে Grade ঠিক করে।
function getGrade(average) {
  if (average >= 80) {
    return 'A+';
  }

  if (average >= 70) {
    return 'A';
  }

  if (average >= 60) {
    return 'A-';
  }

  if (average >= 50) {
    return 'B';
  }

  if (average >= 40) {
    return 'C';
  }

  if (average >= PASS_MARK) {
    return 'D';
  }

  return 'F';
}

// ...
```

`>=` মানে "বড় অথবা সমান"। যাচাইগুলো বড় থেকে ছোট ক্রমে সাজানো, আর `return` চলামাত্র Function শেষ হয়। তাই গড় `75` হলে প্রথম শর্ত মেলে না, দ্বিতীয়টি মেলে, এবং `'A'` ফেরত দিয়ে Function সেখানেই থামে।

ধাপ ৩: পুরো ফলাফল

result.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// নাম ও তিনটি নম্বর (Text) থেকে ফলাফলের একটি বাক্য তৈরি করে।
function calculateResult(name, banglaText, englishText, mathText) {
  const bangla = toMark(banglaText);
```

```
  const english = toMark(englishText);
  const math = toMark(mathText);

  if (bangla === null) {
    return 'বাংলার নম্বর সঠিক নয়।';
  }

  if (english === null) {
    return 'ইংরেজির নম্বর সঠিক নয়।';
  }

  if (math === null) {
    return 'গণিতের নম্বর সঠিক নয়।';
  }

  const total = bangla + english + math;
  const average = total / 3;
  let grade = getGrade(average);
  let status = 'উত্তীর্ণ';

  if (bangla < PASS_MARK) {
    grade = 'F';
    status = 'অনুত্তীর্ণ';
  }

  if (english < PASS_MARK) {
    grade = 'F';
    status = 'অনুত্তীর্ণ';
  }

  if (math < PASS_MARK) {
    grade = 'F';
    status = 'অনুত্তীর্ণ';
  }

  const shown = average.toFixed(2);

  return `${name.trim()}: মোট ${total}, গড় ${shown}, Grade ${grade}, ${status}`;
}

console.log(calculateResult('Sara', '80', '90', '70'));
console.log(calculateResult('Rafi', '100', '100', '20'));
console.log(calculateResult('Mina', '75', '', '60'));
```

Output

```
Sara: মোট 240, গড় 80.00, Grade A+, উত্তীর্ণ
Rafi: মোট 220, গড় 73.33, Grade F, অনুত্তীর্ণ
ইংরেজির নম্বর সঠিক নয়।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **রূপান্তর আগে, হিসাব পরে।** তিনটি নম্বরই প্রথমে `toMark` দিয়ে যাচাই ও রূপান্তর করা হয়। এরপরের পুরো হিসাবে শুধু `number` Type-এর মান থাকে, তাই `+` কখনো String জোড়া লাগাবে না।
- **`=== null` দিয়ে ব্যর্থতা ধরা।** `toMark` ব্যর্থ হলে `null` দেয়। শূন্য একটি বৈধ নম্বর, তাই "নম্বর নেই" বোঝাতে `0` ব্যবহার করা যেত না; এখানেই `null`-এর প্রয়োজন।
- **`const` এবং `let`।** `total` ও `average` একবার হিসাব হয়ে আর বদলায় না, তাই `const`। `grade` ও `status` পরে বদলাতে পারে, তাই `let`।
- **কোনো বিষয়ে অনুত্তীর্ণ হলে।** গড় যত ভালোই হোক, যেকোনো একটি নম্বর `PASS_MARK`-এর নিচে হলে `grade` ও `status` বদলে দেওয়া হয়। Rafi-র গড় ৭৩.৩৩, কিন্তু গণিতে ২০ পাওয়ায় Grade `F`।
- **`toFixed(2)` সবার শেষে।** হিসাব শেষ হওয়ার পর শুধু দেখানোর জন্য গড়কে String-এ রূপান্তর করা হয়েছে।

তিনটি `if` Block-এ একই দুই লাইন তিনবার লেখা হয়েছে। এটি পুনরাবৃত্তি, এবং অধ্যায় ৩-এ শেখা Logical Operator দিয়ে এটিকে একটি শর্তে আনা যাবে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal-এ `student-result-calculator` Folder থেকে চালান:

Terminal

```
node result.js
```

যে মান দেবেন	যা দেখা যাওয়ার কথা
`'Sara', '80', '90', '70'`	Sara: মোট 240, গড় 80.00, Grade A+, উত্তীর্ণ
`'Rafi', '100', '100', '20'`	Rafi: মোট 220, গড় 73.33, Grade F, অনুত্তীর্ণ
`'Mina', '75', '', '60'`	ইংরেজির নম্বর সঠিক নয়।
কোনো নম্বর `'80.5'`, `'abc'`, `'-5'` বা `'101'`	সংশ্লিষ্ট বিষয়ের নম্বর সঠিক নয়।
নম্বরের আগে-পরে ফাঁকা জায়গা, যেমন `' 80 '`	স্বাভাবিকভাবে গৃহীত

Companion Source Code-এ এই Project-এর ১৪টি Automated Test আছে। সেগুলো চালানোর Command `README.md`-তে দেওয়া আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
মোট নম্বর `8090`-এর মতো জোড়া লাগা	`toMark` ছাড়া সরাসরি যোগ করা হয়েছে	তিনটি নম্বরই `toMark` দিয়ে রূপান্তর করুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ফাঁকা নম্বর শূন্য হিসেবে গৃহীত	`clean === ''` যাচাইটি নেই বা `Number()` -এর পরে আছে	যাচাইটি রূপান্তরের আগে রাখুন
গড় `NaN`	কোনো নম্বর যাচাই ছাড়া হিসাবে ঢুকেছে	`=== null` যাচাই তিনটি আছে কি না দেখুন
সব সময় `F` দেখায়	`<` এবং `>` উল্টে গেছে	`bangla < PASS_MARK` ঠিক আছে কি না দেখুন
`TypeError: Assignment to constant variable.`	`grade` বা `status` `const` দিয়ে তৈরি	`let` ব্যবহার করুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। নম্বরগুলো Code-এর ভেতরেই লেখা; ব্যবহারকারীর কাছ থেকে নেওয়া হয় না, এবং কোনো ফলাফল সংরক্ষণ করা হয় না। বিষয়ের সংখ্যা তিনটিতে স্থির, কারণ তালিকা নিয়ে কাজ করার উপায় (Array এবং Loop) এখনো শেখানো হয়নি; অধ্যায় ৪ ও ৬-এর পর যেকোনো সংখ্যক বিষয় সামলানো যাবে। শিক্ষার্থীর নাম যাচাই করা হয় না (অনুশীলনী ১২ দেখুন)।

২.১৪ অনুশীলনী

প্রতিটি অনুশীলনী এই অধ্যায় এবং অধ্যায় ১-এর বিষয় দিয়েই সমাধান করা যায়।

প্রাথমিক স্তর

১. তিনটি Variable তৈরি করুন: আপনার নাম, বয়স এবং আপনি শিক্ষার্থী কি না (`true` বা `false`)। প্রতিটির `typeof` দেখান।

২. না চালিয়ে বলুন এই তিনটি কী দেখাবে: `'10' + 5`, `'10' - 5` এবং `10 + 5 + '5'`। তারপর চালিয়ে মিলিয়ে নিন।

৩. নিচের Code-এ একটি ভুল আছে। ভুলটি ঠিক করুন, যাতে `50` দেখায়।

fix-total.js · JavaScript — চলবে: Node.js

```
const total = 0;
total = total + 50;
console.log(total);
```

৪. একটি Variable মান ছাড়া তৈরি করুন এবং আরেকটিতে `null` রাখুন। দুটির মান দেখান এবং পার্থক্যটি নিজের ভাষায় লিখুন।

৫. না চালিয়ে বলুন: `Number('')`, `Number(' ')` এবং `Number('12a')` কী দেবে। তারপর চালিয়ে দেখুন।

মধ্যম স্তর

৬. `toAge` নামে একটি Function লিখুন, যা Text নেয়। Text-টি ০ থেকে ১২০-এর মধ্যে পূর্ণসংখ্যা হলে সেই সংখ্যা ফেরত দেবে; অন্যথায় `null`।

৭. একটি Object তৈরি করে দুটি Variable-এ রাখুন। একটির মাধ্যমে তথ্য বদলে দেখান যে অন্যটিতেও বদলেছে। তারপর `...` দিয়ে আলাদা Copy তৈরি করে দেখান যে এবার মূলটি বদলায় না।

৮. `0.1 + 0.2 === 0.3` কেন `false`, তা ব্যাখ্যা করুন। একই হিসাব পয়সায় (পূর্ণসংখ্যায়) করে দেখান যে তুলনাটি তখন `true` হয়।

৯. `250 / 3`-এর ফল এক দশমিক ঘরে দেখান এবং ফলটির `typeof` দেখান। ফলটির সঙ্গে `1` যোগ করলে কী হয় এবং কেন?

চ্যালেঞ্জ স্তর

১০. **চতুর্থ বিষয়:** চারটি নম্বর (সংখ্যা) নিয়ে মোট ও গড় ফেরত দেয়, এমন একটি Function লিখুন। গড় দুই দশমিক ঘরে দেখান।

১১. **পাস নম্বর:** পাস নম্বরকে একটি নামযুক্ত `const`-এ রেখে `isPassed` Function লিখুন। পাস নম্বর ৩৩ থেকে ৪০ করলে কোন ফল বদলায়, তা দেখান।

১২. **নাম যাচাই:** `cleanName` Function লিখুন, যা নামের আগে-পরের ফাঁকা জায়গা বাদ দেয়, এবং নাম ফাঁকা হলে `null` ফেরত দেয়।

২.১৫ অনুশীলনীর সমাধান

একই সমস্যার একাধিক সঠিক সমাধান থাকতে পারে।

সমাধান ১

about-me.js · JavaScript — চলবে: Browser Console ও Node.js

```
const myName = 'Tanvir';
const age = 21;
const isStudent = true;

console.log(typeof myName);
console.log(typeof age);
console.log(typeof isStudent);
```

Output

```
string
number
boolean
```

সমাধান ২ — যথাক্রমে `105`, `5` এবং `155`। প্রথমটিতে `+`-এর এক পাশে String থাকায় জোড়া লেগেছে। দ্বিতীয়টিতে `-` String-কে সংখ্যায় রূপান্তর করেছে। তৃতীয়টিতে আগে `10 + 5` হয়ে `15`, তারপর `15 + '5'` জোড়া লেগে `'155'`।

সমাধান ৩ — `total`-এর মান বদলায়, তাই `const` নয়, `let` প্রয়োজন।

fix-total.js · JavaScript — চলবে: Browser Console ও Node.js

```js
let total = 0;
total = total + 50;
console.log(total);
```

Output

```
50
```

সমাধান ৪

empty-values.js · JavaScript — চলবে: Browser Console ও Node.js

```js
let phone;
const fax = null;

console.log(phone);
console.log(fax);
```

Output

```
undefined
null
```

`phone`-এ কোনো মান দেওয়া হয়নি, তাই JavaScript নিজে `undefined` দিয়েছে। `fax`-এ ইচ্ছা করে `null` রাখা হয়েছে, যার অর্থ "Fax নম্বর নেই"।

সমাধান ৫ — যথাক্রমে `0`, `0` এবং `NaN`। ফাঁকা String এবং শুধু ফাঁকা জায়গার String—দুটোই শূন্য হয়, কারণ `Number()` আগে-পরের ফাঁকা জায়গা বাদ দেয়। `'12a'` পুরোটা সংখ্যা নয়, তাই `NaN`।

সমাধান ৬

to-age.js · JavaScript — চলবে: Browser Console ও Node.js

```
function toAge(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const age = Number(clean);

  if (Number.isInteger(age) === false) {
    return null;
  }

  if (age < 0) {
    return null;
  }

  if (age > 120) {
    return null;
  }

  return age;
}

console.log(toAge(' 25 '));
console.log(toAge('২৫ বছর'));
console.log(toAge('150'));
console.log(toAge(''));
```

Output

```
25
null
null
null
```

সমাধান ৭

shared-object.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const book = { title: 'JavaScript Mastery' };
const sameBook = book;

sameBook.title = 'JS Mastery';
console.log(book.title);

const separateBook = { ...book };
separateBook.title = 'অন্য বই';
console.log(book.title);
```

Output

```
JS Mastery
JS Mastery
```

প্রথমবার `book.title` বদলেছে, কারণ `sameBook` একই Object-এর Reference। দ্বিতীয়বার বদলায়নি, কারণ `separateBook` একটি নতুন Object।

সমাধান ৮ — Computer `0.1` এবং `0.2` হুবহু রাখতে পারে না, তাই যোগফল `0.30000000000000004` হয়। পূর্ণসংখ্যায় এই সমস্যা নেই।

paisa.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
console.log(0.1 + 0.2 === 0.3);
console.log(10 + 20 === 30);
```

Output

```
false
true
```

সমাধান ৯

one-decimal.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const result = (250 / 3).toFixed(1);

console.log(result);
console.log(typeof result);
console.log(result + 1);
```

Output

```
83.3
string
83.31
```

`toFixed()` String ফেরত দেয়, তাই `+ 1` যোগ না করে জোড়া লাগিয়েছে। হিসাব সংখ্যায় শেষ করে তারপর `toFixed()` ব্যবহার করতে হয়।

সমাধান ১০

four-subjects.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function summarize(bangla, english, math, science) {
  const total = bangla + english + math + science;
  const average = total / 4;

  return `মোট ${total}, গড় ${average.toFixed(2)}`;
}

console.log(summarize(80, 90, 70, 85));
```

Output

```
মোট 325, গড় 81.25
```

সমাধান ১১

pass-mark.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const PASS_MARK = 40;

function isPassed(mark) {
  if (mark >= PASS_MARK) {
    return true;
  }

  return false;
}

console.log(isPassed(35));
console.log(isPassed(40));
```

Output

```
false
true
```

পাস নম্বর ৩৩ থাকলে `isPassed(35)` হতো `true`। সংখ্যাটি এক জায়গায় নাম দিয়ে রাখায় শুধু একটি লাইন বদলাতে হয়েছে।

সমাধান ১২

clean-name.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
function cleanName(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  return clean;
}

console.log(cleanName('  Sara  '));
console.log(cleanName('     '));
```

Output

```
Sara
null
```

২.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. নতুন Code-এ Variable তৈরির জন্য প্রথম পছন্দ কোনটি হওয়া উচিত?

ক) `var`

খ) `let`

গ) `const`

ঘ) যেকোনো একটি; কোনো পার্থক্য নেই

প্রশ্ন ২. `typeof null`-এর ফল কী?

ক) `object`

খ) `null`

গ) `undefined`

ঘ) `number`

প্রশ্ন ৩. `Number('')`-এর ফল কী?

ক) `NaN`

খ) `undefined`

গ) `null`

ঘ) `0`

প্রশ্ন ৪. `'5' + 2`-এর ফল কী?

ক) `7`

খ) `'52'`

গ) `NaN`

ঘ) একটি Error

প্রশ্ন ৫. একটি Variable তৈরি করা হয়েছে, কিন্তু কোনো মান দেওয়া হয়নি। তার মান কী?

ক) `null`

খ) `0`

গ) `undefined`

ঘ) ফাঁকা String

প্রশ্ন ৬. কোনো মান `NaN` কি না যাচাইয়ের সঠিক উপায় কোনটি?

ক) `value === NaN`

খ) `Number.isNaN(value)`

গ) `typeof value === 'NaN'`

ঘ) `value == 'NaN'`

প্রশ্ন ৭. `0 === '0'`-এর ফল কী এবং কেন?

ক) `true`, কারণ মান একই

খ) `true`, কারণ String সংখ্যায় রূপান্তরিত হয়

গ) `false`, কারণ শূন্য কোনো কিছুর সমান নয়

ঘ) `false`, কারণ Type আলাদা

প্রশ্ন ৮. `const a = { n: 1 }; const b = a; b.n = 2;` চালানোর পর `a.n`-এর মান কত?

ক) `2`

খ) `1`

গ) `undefined`

ঘ) Error হবে, কারণ `a` একটি `const`

প্রশ্ন ৯. `(10 / 4).toFixed(1)`-এর ফলের Type কী?

ক) `number`

খ) `bigint`

গ) `string`

ঘ) `boolean`

প্রশ্ন ১০. `{ }`-এর ভেতরে `let` দিয়ে তৈরি Variable কোথা থেকে ব্যবহার করা যায়?

ক) পুরো File থেকে

খ) শুধু সেই Block-এর ভেতর থেকে

গ) শুধু Block-এর বাইরে থেকে

ঘ) শুধু Function-এর ভেতর থেকে

২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–খ · ৭–ঘ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	`const` ভুল করে মান বদলে যাওয়া আটকায়। মান বদলানোর প্রয়োজন হলে তবেই `let`।
২	ক	এটি ভাষার একটি পুরোনো ত্রুটি। `null` যাচাই করতে `=== null` ব্যবহার করতে হয়।
৩	ঘ	ফাঁকা String শূন্য হয়ে যায়। তাই রূপান্তরের আগে ফাঁকা কি না যাচাই জরুরি।
৪	খ	`+`-এর এক পাশে String থাকলে অন্য পাশটিও String হয়ে জোড়া লাগে।
৫	গ	মান না দিলে JavaScript নিজে `undefined` দেয়। `null` Programmer ইচ্ছা করে দেন।
৬	খ	`NaN` নিজের সমানও নয়, তাই `===` দিয়ে ধরা যায় না। `typeof NaN` হলো `number`।
৭	ঘ	`===` Type এবং মান দুটোই মেলায়। `number` এবং `string` আলাদা Type।
৮	ক	`b` একই Object-এর Reference ধরে আছে। `const` Object-এর ভেতরের তথ্য বদলানো আটকায় না।
৯	গ	`toFixed()` দেখানোর জন্য একটি String ফেরত দেয়।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	খ	`let` এবং `const` Block-এর ভেতরেই কার্যকর। `var` এই নিয়ম মানে না।

২.১৮ অধ্যায়ের সারসংক্ষেপ

- Variable হলো মানের নাম। প্রথমে `const`; মান বদলালে `let`; নতুন Code-এ `var` নয়।
- `let` ও `const` নিজের Block-এর ভেতরে কার্যকর, এবং Declaration-এর আগে ব্যবহার করা যায় না।
- সাতটি Primitive Type: `string`, `number`, `boolean`, `undefined`, `null`, `bigint`, `symbol`। বাকি সব Object।
- `typeof` মানের Type জানায়; ব্যতিক্রম `typeof null`, যা `object` দেখায়।
- দশমিক হিসাব সব সময় নিখুঁত নয়; `NaN` ধরতে `Number.isNaN()`।
- `undefined` মানে মান দেওয়া হয়নি; `null` মানে ইচ্ছাকৃতভাবে কিছু নেই।
- Type থাকে মানের, Variable-এর নয়; তবু একটি Variable-এ একটিই Type রাখা উচিত।
- নিজে রূপান্তর করুন, এবং `Number('')` যে `0` দেয় তা মনে রাখুন।
- সব সময় `===` ও `!==`।
- Primitive-এ মান Copy হয়; Object-এ ঠিকানা (Reference) Copy হয়। `const` Object-এর ভেতরের তথ্য বদলানো আটকায় না।

পরের অধ্যায়ে Operator এবং সিদ্ধান্ত গ্রহণ: হিসাব, তুলনা, একাধিক শর্ত জোড়া লাগানো এবং `if`, `else`, `switch` দিয়ে Program-কে ভিন্ন পথে চালানো।

২.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Declaration	একটি Variable তৈরি করা।
Assignment / Reassignment	Variable-এ মান রাখা / নতুন মান রাখা।
Block	`{ }`-এর ভেতরের Code।
Scope	Program-এর যে অংশ থেকে একটি Variable ব্যবহার করা যায়।
Data Type	মানের ধরন, যা ঠিক করে মানটি নিয়ে কী করা যাবে।
Primitive	একটি সরল মান: string, number, boolean, undefined, null, bigint, symbol।
Object	একাধিক তথ্য একসঙ্গে রাখার মান।

পরিভাষা	অর্থ
Dynamic Typing	Type মানের সঙ্গে থাকে; একই Variable-এ ভিন্ন Type-এর মান রাখা যায়।
Type Conversion	এক Type-এর মানকে অন্য Type-এ বদলানো।
Explicit / Implicit Conversion	নিজে করা রূপান্তর / JavaScript-এর নিজে থেকে করা রূপান্তর।
NaN	Not a Number: অর্থহীন সংখ্যা-হিসাবের ফল।
Strict Equality	`===` : Type এবং মান দুটোই মেলায়।
Reference	মেমোরিতে একটি Object-এর ঠিকানা।
Garbage Collection	অব্যবহৃত Object-এর মেমোরি স্বয়ংক্রিয়ভাবে ফিরিয়ে নেওয়া।

২.২০ আরও পড়ুন

- **MDN — Grammar and types:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types` — Variable ও Type-এর পূর্ণ বিবরণ।
- **MDN — JavaScript data types and data structures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures` — প্রতিটি Type-এর বিস্তারিত।
- **MDN — typeof:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof` — `typeof`-এর সব সম্ভাব্য ফল।
- **MDN — Equality comparisons and sameness:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Equality_comparisons_and_sameness` — `==` এবং `===`-এর তুলনা।

অধ্যায় ৩

০৩

Operators এবং Decision Making

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Arithmetic Operator দিয়ে হিসাব করতে এবং Remainder-এর ব্যবহার বুঝতে।
- Assignment Operator দিয়ে Variable-এর মান সংক্ষেপে বদলাতে।
- Comparison ও Logical Operator দিয়ে শর্ত লিখতে।
- কোন মান Truthy এবং কোনটি Falsy, তা বলতে এবং এর ফাঁদ এড়াতে।
- `if`, `else if`, `else`, `switch` এবং Ternary Operator দিয়ে সিদ্ধান্ত নিতে।
- Operator Precedence বুঝে বন্ধনী দিয়ে হিসাবের ক্রম স্পষ্ট করতে।
- ব্যবহারকারীর দেওয়া মান ধাপে ধাপে যাচাই করতে।
- একটি সম্পূর্ণ **Shopping Discount Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ২ সম্পন্ন করা: `const` ও `let`, Data Type, `Number()` দিয়ে রূপান্তর, `===` এবং `null`।
- অধ্যায় ১-এর Function ও `if`-এর প্রাথমিক পরিচয়।
- `chapter-03` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আপনার Program ওপর থেকে নিচে সোজা পথে চলেছে। বাস্তব Program-কে প্রতিনিয়ত সিদ্ধান্ত নিতে হয়: কেনাকাটা তিন হাজার টাকার বেশি হলে Delivery ফ্রি; Coupon সঠিক হলে ছাড়; পরিমাণ শূন্য হলে Order গ্রহণ করা হবে না। প্রতিটি সিদ্ধান্তের পেছনে থাকে একটি শর্ত, আর শর্ত লেখা হয় Operator দিয়ে।

এই অধ্যায়ের দুটি অংশ। প্রথম অংশে Operator: যেসব চিহ্ন দিয়ে হিসাব, তুলনা এবং একাধিক শর্ত জোড়া লাগানো হয়। দ্বিতীয় অংশে সিদ্ধান্ত গ্রহণ: শর্তের ফল অনুযায়ী Program-কে ভিন্ন পথে চালানো।

শেষে আপনি একটি দোকানের মূল্য-হিসাবের Program লিখবেন, যেখানে দাম, পরিমাণ, Coupon এবং সদস্যপদ মিলিয়ে চূড়ান্ত পরিশোধযোগ্য টাকা বের করা হয়।

৩.১ Expression এবং Operator

যে Code থেকে একটি মান পাওয়া যায়, তাকে **Expression** বলে। `5 + 3` একটি Expression, যার মান `8`। `price > 1000`-ও একটি Expression, যার মান `true` অথবা `false`।

Operator হলো সেই চিহ্ন, যা মান নিয়ে কাজ করে; আর যে মানগুলোর ওপর কাজ হয়, সেগুলো **Operand**। `5 + 3`-তে `+` Operator, `5` ও `3` Operand।

expression.js · JavaScript — চলবে: Browser Console ও Node.js

```
const unitPrice = 850;
const quantity = 3;
const subtotal = unitPrice * quantity;

console.log(subtotal);
console.log(subtotal >= 2000);
```

Output

```
2550
true
```

`unitPrice * quantity` একটি Expression; তার মান `subtotal`-এ রাখা হয়েছে। Expression যেকোনো জায়গায় বসানো যায়, যেখানে একটি মান দরকার: Variable-এ, `console.log()`-এর ভেতরে, অথবা শর্ত হিসেবে।

৩.২ Arithmetic Operator

Operator	কাজ	উদাহরণ	ফল
`+`	যোগ	`7 + 3`	`10`
`-`	বিয়োগ	`7 - 3`	`4`
`*`	গুণ	`7 * 3`	`21`
`/`	ভাগ	`7 / 2`	`3.5`
`%`	ভাগশেষ (Remainder)	`7 % 3`	`1`
`**`	ঘাত	`2 ** 3`	`8`

প্রথম চারটি পরিচিত। ভাগের ফল দশমিক হতে পারে; JavaScript-এ আলাদা "পূর্ণসংখ্যা ভাগ" নেই।

Remainder: ছোট Operator, অনেক কাজ

`%` ভাগ করার পর যা অবশিষ্ট থাকে, তা দেয়। এটি দিয়ে বেশ কিছু প্রশ্নের উত্তর সহজে পাওয়া যায়।

remainder.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(10 % 2);
console.log(7 % 2);
console.log(125 % 60);
console.log(-7 % 3);
```

Output

```
0
1
5
-1
```

- কোনো সংখ্যাকে `2` দিয়ে ভাগ করে ভাগশেষ `0` হলে সংখ্যাটি জোড়; `1` হলে বিজোড়।
- ১২৫ মিনিটে পূর্ণ ঘণ্টা বাদ দিলে `125 % 60`, অর্থাৎ ৫ মিনিট থাকে।
- ঋণাত্মক সংখ্যার ভাগশেষ ঋণাত্মক হয়: `-7 % 3` এর ফল `-1`।

পূর্ণসংখ্যায় আনা

টাকার হিসাবে শতাংশ বের করলে দশমিক আসতে পারে। `Math.round()` সংখ্যাকে নিকটতম পূর্ণসংখ্যায় আনে; ঠিক মাঝামাঝি (`.5`) হলে ওপরের দিকে নেয়।

math-round.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Math.round(42.4));
console.log(Math.round(42.5));
console.log(Math.round(850 * 5 / 100));
```

Output

```
42
43
43
```

শেষ লাইনটি ৮৫০ টাকার ৫ শতাংশ। লক্ষ করুন, শতাংশকে `0.05` না লিখে `5 / 100` হিসেবে লেখা হয়েছে। অধ্যায় ২-এ দেখেছেন, দশমিক সংখ্যা হুবহু রাখা যায় না; আগে পূর্ণসংখ্যায় গুণ করে পরে ভাগ করলে সেই ঝুঁকি কমে।

৩.৩ Assignment Operator

`=` ডান পাশের মান বাঁ পাশের Variable-এ রাখে। প্রায়ই Variable-এর বর্তমান মানের সঙ্গে কিছু যোগ বা বিয়োগ করে আবার সেখানেই রাখতে হয়। এ কাজের সংক্ষিপ্ত রূপ আছে।

সংক্ষিপ্ত রূপ	পূর্ণ রূপ
`stock += 5`	`stock = stock + 5`
`stock -= 2`	`stock = stock - 2`
`stock *= 2`	`stock = stock * 2`
`stock /= 2`	`stock = stock / 2`
`stock++`	`stock = stock + 1`
`stock--`	`stock = stock - 1`

assignment.js · JavaScript — চলবে: Browser Console ও Node.js

```
let stock = 10;

stock += 5;
console.log(stock);

stock -= 2;
console.log(stock);

stock *= 2;
console.log(stock);

stock++;
console.log(stock);

stock--;
console.log(stock);
```

Output

```
15
13
26
27
26
```

মান বদলানো হচ্ছে, তাই `stock` অবশ্যই `let` দিয়ে তৈরি।

`++` এবং `--` Variable-এর আগে বা পরে বসানো যায়, এবং দুই ক্ষেত্রে আচরণ সামান্য আলাদা।

increment.js · JavaScript — চলবে: Browser Console ও Node.js

```
let a = 5;

console.log(a++);
console.log(a);
console.log(++a);
```

Output

```
5
6
7
```

`a++` আগে পুরোনো মান দেয়, তারপর বাড়ায়; `++a` আগে বাড়ায়, তারপর নতুন মান দেয়। এই সূক্ষ্ম পার্থক্য ভুলের উৎস। তাই `++` ও `--` সব সময় আলাদা লাইনে লিখুন, অন্য হিসাবের ভেতরে নয়।

৩.৪ Comparison Operator

Comparison Operator দুটি মান তুলনা করে এবং ফল হিসেবে একটি Boolean দেয়।

Operator	অর্থ	উদাহরণ	ফল
`===`	সমান (Type-সহ)	`5 === 5`	`true`
`!==`	সমান নয়	`5 !== 5`	`false`
`>`	বড়	`10 > 5`	`true`
`<`	ছোট	`10 < 5`	`false`
`>=`	বড় অথবা সমান	`10 >= 10`	`true`
`<=`	ছোট অথবা সমান	`10 <= 9`	`false`

সীমা নিয়ে কাজ করার সময় `>` এবং `>=`-এর পার্থক্যই সবচেয়ে বেশি ভুল ঘটায়। "৩০০০ টাকা বা তার বেশি হলে Delivery ফ্রি" মানে `>= 3000`; `> 3000` লিখলে ঠিক ৩০০০ টাকার ক্রেতা সুবিধাটি পাবেন না।

String-ও তুলনা করা যায়, তবে নিয়মটি আলাদা।

compare-strings.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('apple' < 'banana');
console.log(10 < 9);
console.log('10' < '9');
```

Output

```
true
false
true
```

String তুলনা হয় অভিধানের মতো, বাঁ দিক থেকে অক্ষর ধরে ধরে। `'10'` এবং `'9'`-এর প্রথম অক্ষর `'1'` ও `'9'`; `'1'` আগে আসে, তাই `'10' < '9'` সত্য। সংখ্যা হিসেবে এটি ভুল। এ কারণেই অধ্যায় ২-এর নিয়মটি গুরুত্বপূর্ণ: তুলনা বা হিসাবের আগে String-কে `Number()` দিয়ে সংখ্যায় রূপান্তর করুন।

৩.৫ Logical Operator

একাধিক শর্ত জোড়া লাগাতে বা একটি শর্ত উল্টে দিতে Logical Operator ব্যবহার করা হয়।

Operator	নাম	কখন সত্য
`&&`	AND (এবং)	দুই পাশই সত্য হলে

Operator	নাম	কখন সত্য
`||`	OR (অথবা)	অন্তত এক পাশ সত্য হলে
`!`	NOT (নয়)	মানটি মিথ্যা হলে

logical.js · JavaScript — চলবে: Browser Console ও Node.js

```
const age = 20;
const hasId = true;
const isWeekend = false;
const isHoliday = true;

console.log(age >= 18 && hasId);
console.log(isWeekend || isHoliday);
console.log(!hasId);
console.log(age >= 18 && !isWeekend);
```

Output

```
true
true
false
true
```

- প্রথম লাইন: বয়স ১৮ বা বেশি **এবং** পরিচয়পত্র আছে—দুটোই সত্য, তাই ফল সত্য।
- দ্বিতীয় লাইন: সপ্তাহান্ত **অথবা** ছুটির দিন—একটি সত্য হলেই যথেষ্ট।
- তৃতীয় লাইন: `!` সত্যকে মিথ্যা এবং মিথ্যাকে সত্য করে।

অধ্যায় ২-এর Project-এ একই দুই লাইন তিনটি আলাদা `if`-এ লিখতে হয়েছিল। এখন সেটি একটি শর্তে লেখা যায়: `bangla < 33 || english < 33 || math < 33`।

Short-Circuit: প্রয়োজন না হলে থেমে যাওয়া

`&&`-এর বাঁ পাশ মিথ্যা হলে পুরো ফল মিথ্যা হবেই, তাই JavaScript ডান পাশ আর দেখে না। একইভাবে `||`-এর বাঁ পাশ সত্য হলে ডান পাশ দেখা হয় না। এই আচরণকে **Short-Circuit** বলে।

আরেকটি বিষয়: `&&` এবং `||` সব সময় `true` বা `false` ফেরত দেয় না; যে Operand-এ গিয়ে থামে, সেটির মানই ফেরত দেয়।

short-circuit.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log('' || 'Guest');
console.log('Sara' || 'Guest');
console.log(0 || 10);
console.log(0 ?? 10);
console.log(null ?? 10);
```

Output

```
Guest
Sara
10
0
10
```

`||` দিয়ে একটি বিকল্প মান দেওয়া যায়: নাম ফাঁকা হলে `'Guest'`। কিন্তু তৃতীয় লাইনে সমস্যা: `0` একটি বৈধ মান হতে পারে, অথচ `||` সেটিকে বাদ দিয়ে `10` দিয়েছে। এ জন্য `??` আছে, যা শুধু `null` বা `undefined` হলেই বিকল্পটি নেয়। শূন্য বা ফাঁকা String যেখানে বৈধ মান, সেখানে `??` ব্যবহার করুন।

৩.৬ Truthy এবং Falsy

শর্তের জায়গায় Boolean ছাড়া অন্য মান বসালে JavaScript সেটিকে নিজে থেকে সত্য বা মিথ্যা হিসেবে ধরে নেয়। যে মানগুলো মিথ্যা হিসেবে গণ্য হয়, সেগুলো **Falsy**; বাকি সব **Truthy**।

Falsy মান মাত্র কয়েকটি, তাই এগুলো মনে রাখাই সহজ:

Falsy মান	অর্থ
`false`	মিথ্যা
`0`	শূন্য
`''`	ফাঁকা String
`null`	ইচ্ছাকৃতভাবে কিছু নেই
`undefined`	মান দেওয়া হয়নি
`NaN`	অর্থহীন সংখ্যা

truthy-falsy.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Boolean(0));
console.log(Boolean(''));
console.log(Boolean(null));
console.log(Boolean('0'));
console.log(Boolean('false'));
console.log(Boolean(' '));
console.log(Boolean({}));
```

Output

```
false
false
false
true
true
true
true
```

শেষ চারটি সত্য। `'0'` ও `'false'` ফাঁকা নয় এমন String; একটি ফাঁকা জায়গাও একটি অক্ষর; আর যেকোনো Object, এমনকি ফাঁকা Object-ও, Truthy।

Truthy-র ফাঁদ

zero-trap.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describeMark(mark) {
  if (mark) {
    return `নম্বর: ${mark}`;
  }

  return 'নম্বর দেওয়া হয়নি';
}

console.log(describeMark(75));
console.log(describeMark(0));
```

Output

```
নম্বর: 75
নম্বর দেওয়া হয়নি
```

শূন্য একটি বৈধ নম্বর, কিন্তু `if (mark)` সেটিকে "নেই" হিসেবে ধরেছে, কারণ `0` Falsy। আপনি যা জানতে চান, ঠিক সেটিই লিখুন: নম্বর দেওয়া হয়েছে কি না জানতে চাইলে `mark !== null`।

> **মনে রাখুন:** সংখ্যা বা String-এর ক্ষেত্রে `if (value)` লেখার আগে ভাবুন: শূন্য বা ফাঁকা String কি এখানে বৈধ মান হতে পারে? হলে স্পষ্ট তুলনা লিখুন।

৩.৭ if, else if এবং else

`if` শর্ত সত্য হলে একটি Block চালায়। শর্ত মিথ্যা হলে অন্য কিছু করতে চাইলে `else` যোগ করুন; মাঝখানে আরও শর্ত থাকলে `else if`।

shipping.js · JavaScript — চলবে: Browser Console ও Node.js

```
function shippingMessage(amount) {
  if (amount >= 5000) {
    return 'Delivery ফ্রি';
  } else if (amount >= 2000) {
    return 'Delivery-তে অর্ধেক ছাড়';
  } else {
    return 'সাধারণ Delivery';
  }
}

console.log(shippingMessage(6000));
console.log(shippingMessage(2500));
console.log(shippingMessage(500));
```

Output

```
Delivery ফ্রি
Delivery-তে অর্ধেক ছাড়
সাধারণ Delivery
```

JavaScript শর্তগুলো ওপর থেকে নিচে যাচাই করে এবং **প্রথম যেটি সত্য হয়, শুধু সেই Block-টি চালায়**। বাকিগুলো আর দেখা হয় না। কোনোটিই সত্য না হলে `else`-এর Block চলে।

শর্তের ক্রম গুরুত্বপূর্ণ

wrong-order.js · JavaScript — চলবে: Browser Console ও Node.js

```
function wrongMessage(amount) {
  if (amount >= 2000) {
    return 'Delivery-তে অর্ধেক ছাড়';
  } else if (amount >= 5000) {
    return 'Delivery ফ্রি';
  } else {
    return 'সাধারণ Delivery';
  }
}

console.log(wrongMessage(6000));
```

Output

```
Delivery-তে অর্ধেক ছাড়
```

৬০০০ টাকার ক্রেতা ফ্রি Delivery পাওয়ার কথা, কিন্তু `6000 >= 2000` সত্য হওয়ায় প্রথম Block-ই চলেছে, আর দ্বিতীয় শর্তে কখনো পৌঁছানো হয়নি। সীমা নিয়ে কাজ করার সময় সবচেয়ে কঠোর শর্তটি সবার আগে লিখুন।

৩.৮ switch

একটি মানকে অনেকগুলো নির্দিষ্ট মানের সঙ্গে মেলাতে হলে লম্বা `else if`-এর চেয়ে `switch` পড়তে সহজ।

payment.js · JavaScript — চলবে: Browser Console ও Node.js

```
function paymentLabel(method) {
  switch (method) {
    case 'card':
      return 'Card-এ পরিশোধ';
    case 'cash':
      return 'পণ্য হাতে পেয়ে পরিশোধ';
    case 'mobile':
      return 'Mobile Banking';
    default:
      return 'অজানা পদ্ধতি';
  }
}

console.log(paymentLabel('cash'));
console.log(paymentLabel('cheque'));
```

Output

```
পণ্য হাতে পেয়ে পরিশোধ
অজানা পদ্ধতি
```

`switch` বন্ধনীর মানটিকে প্রতিটি `case`-এর সঙ্গে `===` দিয়ে মেলায়। যেটি মেলে, সেখান থেকে Code চলা শুরু হয়। কোনোটি না মিললে `default` চলে। `default` সব সময় লিখুন, যাতে অপ্রত্যাশিত মান নীরবে পার না হয়ে যায়।

break এবং Fallthrough

ওপরের উদাহরণে প্রতিটি `case` `return` দিয়ে শেষ হয়েছে। `return` না থাকলে প্রতিটি `case`-এর শেষে `break` লিখতে হয়; না লিখলে JavaScript থামে না, পরের `case`-এর Code-ও চালিয়ে যায়। একে **Fallthrough** বলে।

fallthrough.js · JavaScript — চলবে: Browser Console ও Node.js

```
function dayType(day) {
  let type = 'কর্মদিবস';

  switch (day) {
    case 'fri':
    case 'sat':
      type = 'সাপ্তাহিক ছুটি';
      break;
    case 'sun':
      type = 'সপ্তাহের প্রথম কর্মদিবস';
      break;
    default:
      break;
  }

  return type;
}

console.log(dayType('fri'));
console.log(dayType('sat'));
console.log(dayType('mon'));
```

Output

```
সাপ্তাহিক ছুটি
সাপ্তাহিক ছুটি
কর্মদিবস
```

এখানে Fallthrough ইচ্ছা করে ব্যবহার করা হয়েছে: `'fri'`-এর নিজস্ব কোনো Code নেই, তাই সেটি `'sat'`-এর Code-এ গড়িয়ে পড়ে। একাধিক মানের জন্য একই কাজ করতে এটি একটি পরিচ্ছন্ন উপায়। কিন্তু ভুল করে `break` বাদ পড়লে একই আচরণ Bug হয়ে দাঁড়ায়।

	if / else if	switch
উপযুক্ত	সীমা, পরিসর, জটিল শর্ত (`>=`, `&&`)	একটি মানকে নির্দিষ্ট কিছু মানের সঙ্গে মেলানো
তুলনা	যেকোনো Expression	শুধু `===`

৩.৯ Ternary Operator

দুটি মানের মধ্যে একটি বেছে নেওয়ার সংক্ষিপ্ত উপায় **Ternary Operator**। এর তিনটি অংশ: শর্ত, সত্য হলে মান, মিথ্যা হলে মান।

ternary.js · JavaScript — চলবে: Browser Console ও Node.js

```
const stock = 0;
const badge = stock > 0 ? 'মজুত আছে' : 'শেষ হয়ে গেছে';

console.log(badge);

const isMember = true;
const discountPercent = isMember ? 5 : 0;

console.log(discountPercent);
```

Output

```
শেষ হয়ে গেছে
5
```

`শর্ত ? মান১ : মান২` —শর্ত সত্য হলে পুরো Expression-এর মান `মান১`, নইলে `মান২`। এটি একটি Expression, তাই সরাসরি `const`-এ রাখা যায়; `if` / `else` দিয়ে একই কাজ করতে `let` এবং চার-পাঁচ লাইন লাগত।

Ternary ব্যবহার করুন শুধু সরল, দুই-পথের সিদ্ধান্তে। একটি Ternary-র ভেতরে আরেকটি বসালে Code পড়া কঠিন হয়ে যায়; সে ক্ষেত্রে `if` / `else` লিখুন।

৩.১০ Operator Precedence

একটি Expression-এ একাধিক Operator থাকলে কোনটি আগে হিসাব হবে, তা ঠিক করে **Operator Precedence**। গণিতের মতোই গুণ ও ভাগ যোগ-বিয়োগের আগে হয়।

precedence.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(2 + 3 * 4);
console.log((2 + 3) * 4);
console.log(10 - 4 - 3);
console.log(2 ** 3 ** 2);
console.log(true || false && false);
console.log((true || false) && false);
```

Output

```
14
20
3
512
true
false
```

এই অধ্যায়ের Operator-গুলোর ক্রম, আগে থেকে পরে:

ক্রম	Operator
১	`( )` বন্ধনী
২	`!`, `++`, `--`
৩	`**`
৪	`*`, `/`, `%`
৫	`+`, `-`
৬	`<`, `<=`, `>`, `>=`
৭	`===`, `!==`
৮	`&&`
৯	`||`, `??`
১০	`? :` (Ternary)
১১	`=`, `+=`, `-=` ইত্যাদি

একই স্তরের Operator সাধারণত বাঁ থেকে ডানে হিসাব হয়: `10 - 4 - 3` মানে `(10 - 4) - 3`। ব্যতিক্রম `**`, যা ডান থেকে বাঁয়ে: `2 ** 3 ** 2` মানে `2 ** 9`।

সবচেয়ে গুরুত্বপূর্ণ সারি দুটি হলো ৮ ও ৯: `&&` সব সময় `||`-এর আগে হিসাব হয়। তাই `true || false && false`-এ আগে `false && false` হয়, তারপর `true || false`।

> **পরামর্শ:** এই তালিকা মুখস্থ করার প্রয়োজন নেই। যেখানে `&&` এবং `||` একসঙ্গে আছে, অথবা ক্রম নিয়ে সামান্য সন্দেহও আছে, সেখানে বন্ধনী দিন। বন্ধনী Code-কে ধীর করে না, কিন্তু পাঠকের ভুল বোঝা আটকায়।

৩.১১ Input Validation

ব্যবহারকারীর দেওয়া মান ব্যবহারের আগে যাচাই করাকে **Input Validation** বলে। অধ্যায় ২-এ এটি একাধিক আলাদা `if` দিয়ে করা হয়েছিল। Logical Operator দিয়ে একই কাজ আরও সংক্ষেপে হয়।

validate-quantity.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isValidQuantity(text) {
  const clean = text.trim();

  if (clean === '') {
    return false;
  }

  const quantity = Number(clean);

  return Number.isInteger(quantity) && quantity >= 1 && quantity <= 100;
}

console.log(isValidQuantity('5'));
console.log(isValidQuantity(''));
console.log(isValidQuantity('0'));
console.log(isValidQuantity('2.5'));
console.log(isValidQuantity('abc'));
console.log(isValidQuantity('101'));
```

Output

```
true
false
false
false
false
false
```

শেষ `return`-এ তিনটি শর্ত `&&` দিয়ে জোড়া: পূর্ণসংখ্যা হতে হবে, **এবং** ১ বা বেশি, **এবং** ১০০ বা কম। Comparison-এর ফল নিজেই একটি Boolean, তাই সেটি সরাসরি ফেরত দেওয়া যায়; `if (...) { return true; } else { return false; }` লেখার দরকার নেই।

Guard Clause: আগে সমস্যাগুলো সরিয়ে ফেলা

Function-এর শুরুতেই অগ্রহণযোগ্য অবস্থাগুলো যাচাই করে `return` করে দেওয়ার ধরনটিকে **Guard Clause** বলে। ওপরের `if (clean === '')` একটি Guard Clause। এতে মূল হিসাব `if`-এর ভেতরে ঢুকে যায় না, আর Function-এর নিচের অংশে নিশ্চিত থাকা যায় যে মানগুলো ঠিক আছে।

যাচাইয়ের একটি নির্ভরযোগ্য ক্রম:

১. মানটি আছে কি না (ফাঁকা নয়)।
২. সঠিক Type-এ রূপান্তর করা যায় কি না।
৩. মানটি অনুমোদিত সীমার মধ্যে কি না।
৪. অন্য মানের সঙ্গে মিলিয়ে নিয়মটি মানছে কি না (যেমন: এই Coupon-এর জন্য ন্যূনতম কেনাকাটা)।

> **সতর্কতা:** Browser বা ব্যবহারকারীর Computer-এ করা যাচাই শুধু সুবিধার জন্য। দাম, ছাড় বা টাকার হিসাব সব সময় Server-এ আবার করতে হয়, কারণ ব্যবহারকারীর দিকের Code বদলে ফেলা সম্ভব। বিষয়টি অধ্যায় ২০-এ বিস্তারিত আছে।

৩.১২ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
শর্তে `===`-এর বদলে `=` লেখা	শর্ত সব সময় সত্য, আর Variable-এর মান বদলে যায়	তুলনার জন্য `===`
`>` এবং `>=` গুলিয়ে ফেলা	ঠিক সীমার মানে ভুল ফল	নিয়মটি "বা তার বেশি" কি না পড়ে দেখুন; সীমার মান দিয়ে পরীক্ষা করুন
`else if`-এর ভুল ক্রম	পরের শর্তে কখনো পৌঁছানো যায় না	কঠোরতম শর্ত আগে
`switch`-এ `break` বাদ পড়া	পরের `case`-এর Code-ও চলে	প্রতিটি `case` `break` বা `return` দিয়ে শেষ করুন
`if (value)` দিয়ে শূন্য যাচাই	বৈধ `0` বাদ পড়ে	`value !== null` বা স্পষ্ট তুলনা
`||` দিয়ে বিকল্প মান, যেখানে `0` বৈধ	`0`-এর জায়গায় বিকল্প বসে	`??` ব্যবহার করুন
String হিসেবে সংখ্যা তুলনা	`'10' < '9'` সত্য	আগে `Number()`
`&&` ও `||` বন্ধনী ছাড়া মেশানো	হিসাবের ক্রম প্রত্যাশামতো নয়	বন্ধনী দিন
`1 < x < 10` লেখা	সব সময় সত্য	`x > 1 && x < 10`

প্রথম ভুলটি চালিয়ে দেখা শিক্ষণীয়।

assign-in-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
let coupon = 'NONE';

if (coupon = 'SAVE10') {
  console.log('Coupon প্রয়োগ হয়েছে');
}

console.log(coupon);
```

Output

```
Coupon প্রয়োগ হয়েছে
SAVE10
```

`coupon = 'SAVE10'` তুলনা নয়, Assignment। এটি `coupon`-এ `'SAVE10'` রাখে, আর Assignment-এর মান হয় সেই String, যা Truthy। ফলে শর্ত সব সময় সত্য, এবং Variable-টিও নীরবে বদলে গেছে। Coupon যাচাইয়ের মতো জায়গায় এই একটি অক্ষরের ভুল মানে সবাইকে ছাড় দেওয়া।

৩.১৩ Debugging নির্দেশিকা

শর্ত-সংক্রান্ত Bug-এ Program কোনো Error দেখায় না; শুধু ভুল পথে যায়। তাই প্রশ্ন হলো: শর্তটির মান আসলে কী হচ্ছে?

একটি জটিল শর্তকে ভেঙে প্রতিটি অংশ আলাদাভাবে দেখুন।

debug-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
const subtotal = 4800;
const code = 'SAVE20';

console.log('code মিলেছে:', code === 'SAVE20');
console.log('ন্যূনতম পূরণ:', subtotal >= 5000);
console.log('পুরো শর্ত:', code === 'SAVE20' && subtotal >= 5000);
```

Output

```
code মিলেছে: true
ন্যূনতম পূরণ: false
পুরো শর্ত: false
```

এক নজরে বোঝা যাচ্ছে কোন অংশটি শর্তকে মিথ্যা করছে।

Program ভুল পথে গেলে এই ক্রমে যাচাই করুন:

১. শর্তের প্রতিটি অংশের মান আলাদাভাবে `console.log()` করুন।

‹‹‹

২. তুলনার দুই পাশের `typeof` দেখুন; একটি String আর একটি Number নয় তো?
৩. ঠিক সীমার মান দিয়ে পরীক্ষা করুন: সীমাটি, তার এক কম এবং এক বেশি।
৪. `else if`-এর ক্রম দেখুন: ওপরের কোনো শর্ত কি নিচেরটিকে ঢেকে দিচ্ছে?
৫. `switch`-এ প্রতিটি `case`-এর শেষে `break` বা `return` আছে কি না দেখুন।
৬. শর্তে `=` আর `===` মিলিয়ে দেখুন।
৭. প্রতিটি Block-এর শুরুতে একটি `console.log()` বসিয়ে দেখুন Program আসলে কোন পথে ঢুকছে।

৩.১৪ পেশাদার অভ্যাস

- **আপনি যা বোঝাতে চান, ঠিক তা-ই লিখুন।** `if (count)` নয়, `if (count > 0)`।
- **Guard Clause দিয়ে শুরু করুন।** অগ্রহণযোগ্য অবস্থাগুলো আগে সরিয়ে দিলে মূল Logic সমতল ও পাঠযোগ্য থাকে।
- **গভীর Nesting এড়িয়ে চলুন।** একটি `if`-এর ভেতরে আরেকটি, তার ভেতরে আরেকটি—এমন হলে শর্তগুলো `&&` দিয়ে জোড়া লাগান অথবা আলাদা Function-এ নিন।
- **জটিল শর্তকে নাম দিন।** `const qualifiesForFreeDelivery = payable >= 3000;` লিখে তারপর `if (qualifiesForFreeDelivery)`।
- **সন্দেহ হলে বন্ধনী।**
- **`switch`-এ সব সময় `default`।**
- **সীমার সংখ্যাগুলোকে নাম দিন।** `3000` বা `120` Code-এর মাঝখানে না লিখে `const` করুন।
- **`++` ও `--` আলাদা লাইনে।**
- **ব্যবসার নিয়ম আগে ভাষায় লিখে নিন।** "Coupon-এর ছাড় আগে, সদস্য-ছাড় পরে"—ক্রম বদলালে টাকার অঙ্ক বদলায়।

৩.১৫ Project: Shopping Discount Calculator

একটি Online দোকানের মূল্য-হিসাবের অংশটি লিখবেন। দাম ও পরিমাণ Text হিসেবে আসবে; সঙ্গে থাকবে একটি Coupon Code এবং ক্রেতা সদস্য কি না।

Project-এর চাহিদা

এগুলো এই শিক্ষামূলক Project-এর জন্য তৈরি নিয়ম।

- দাম শূন্য বা তার বেশি পূর্ণসংখ্যা (টাকায়)। পরিমাণ ১ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা।
- Coupon `SAVE10` দিলে ১০ শতাংশ ছাড়। `SAVE20` দিলে ২০ শতাংশ ছাড়, তবে মোট কেনাকাটা কমপক্ষে ৫০০০ টাকা হতে হবে।

- Coupon Code ছোট বা বড় হাতের অক্ষরে এবং আগে-পরে ফাঁকা জায়গাসহ লেখা হলেও গৃহীত হবে। Coupon না দিলে কোনো ছাড় নেই। অচেনা Code দিলে তা জানাতে হবে; নীরবে উপেক্ষা করা যাবে না।
- সদস্য হলে Coupon-এর ছাড়ের পর অবশিষ্ট টাকার ওপর আরও ৫ শতাংশ ছাড়।
- প্রতিটি ছাড় নিকটতম পূর্ণ টাকায় আনা হবে।
- সব ছাড়ের পর টাকার অঙ্ক ৩০০০ বা তার বেশি হলে Delivery ফ্রি; নইলে ১২০ টাকা।

File-এর কাঠামো

Folder-এর কাঠামো

```
shopping-discount-calculator/
├── discount.js
├── README.md
└── tests/
    └── discount.test.js
```

discount.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: স্থির মান এবং Text থেকে পূর্ণসংখ্যা

discount.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```javascript
'use strict';

const DELIVERY_FEE = 120;
const FREE_DELIVERY_FROM = 3000;
const MEMBER_PERCENT = 5;
const SAVE20_MINIMUM = 5000;

// Text থেকে শূন্য বা তার বেশি পূর্ণসংখ্যা তৈরি করে; না পারলে null।
function toWholeNumber(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const value = Number(clean);

  if (!Number.isInteger(value) || value < 0) {
    return null;
  }

  return value;
}
```

```
// ...
```

চারটি সংখ্যাকে শুরুতেই নাম দেওয়া হয়েছে। দোকান Delivery-র খরচ বদলালে একটি লাইন বদলালেই চলবে। `toWholeNumber` অধ্যায় ২-এর `toMark`-এর মতোই, তবে শেষ দুটি যাচাই এখন `||` দিয়ে এক শর্তে: পূর্ণসংখ্যা **নয়, অথবা** ঋণাত্মক—যেকোনো একটি হলেই বাতিল।

ধাপ ২: Coupon থেকে ছাড়ের শতাংশ

discount.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// Coupon Code থেকে ছাড়ের শতাংশ দেয়; অচেনা Code হলে null।
function getCouponPercent(code) {
  switch (code) {
    case '':
      return 0;
    case 'SAVE10':
      return 10;
    case 'SAVE20':
      return 20;
    default:
      return null;
  }
}

// ...
```

একটি মানকে কয়েকটি নির্দিষ্ট মানের সঙ্গে মেলানো— `switch`-এর আদর্শ ব্যবহার। ফাঁকা Code বৈধ এবং তার ছাড় শূন্য; অচেনা Code-এর জন্য `default` থেকে `null`। শূন্য শতাংশ একটি বৈধ উত্তর, তাই "অচেনা" বোঝাতে `0` ব্যবহার করা যেত না।

ধাপ ৩: পুরো হিসাব

discount.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// দাম, পরিমাণ, Coupon ও সদস্যপদ থেকে হিসাবের একটি বাক্য তৈরি করে।
function calculateOrder(priceText, quantityText, couponText, isMember) {
  const price = toWholeNumber(priceText);
  const quantity = toWholeNumber(quantityText);

  if (price === null) {
    return 'দাম সঠিক নয়।';
  }
```

```
  if (quantity === null || quantity < 1 || quantity > 100) {
    return 'পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।';
  }

  const subtotal = price * quantity;
  const code = couponText.trim().toUpperCase();
  const couponPercent = getCouponPercent(code);

  if (couponPercent === null) {
    return 'Coupon Code সঠিক নয়।';
  }

  if (code === 'SAVE20' && subtotal < SAVE20_MINIMUM) {
    return 'SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।';
  }

  const couponDiscount = Math.round(subtotal * couponPercent / 100);
  const afterCoupon = subtotal - couponDiscount;
  const memberPercent = isMember ? MEMBER_PERCENT : 0;
  const memberDiscount = Math.round(afterCoupon * memberPercent / 100);
  const payable = afterCoupon - memberDiscount;
  const delivery = payable >= FREE_DELIVERY_FROM ? 0 : DELIVERY_FEE;
  const discount = couponDiscount + memberDiscount;
  const total = payable + delivery;

  return `মোট ${subtotal}, ছাড় ${discount}, Delivery ${delivery}, দিতে হবে ${total}`;
}

console.log(calculateOrder('1500', '2', 'SAVE10', false));
console.log(calculateOrder('2500', '3', ' save20 ', true));
console.log(calculateOrder('850', '1', '', true));
console.log(calculateOrder('1200', '2', 'SAVE20', false));
console.log(calculateOrder('1200', '0', '', false));
console.log(calculateOrder('1200', '2', 'FREE50', false));
```

Output

```
মোট 3000, ছাড় 300, Delivery 120, দিতে হবে 2820
মোট 7500, ছাড় 1800, Delivery 0, দিতে হবে 5700
মোট 850, ছাড় 43, Delivery 120, দিতে হবে 927
SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।
পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।
Coupon Code সঠিক নয়।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **চারটি Guard Clause।** দাম, পরিমাণ, Coupon-এর অস্তিত্ব এবং Coupon-এর শর্ত—অনুচ্ছেদ ৩.১১-এর ক্রমেই যাচাই করা হয়েছে। এরপরের অংশে সব মান নিশ্চিতভাবে সঠিক।

- **`quantity === null || quantity < 1 || quantity > 100`** — `||`-এর Short-Circuit এখানে কাজে লাগছে: `quantity` `null` হলে বাকি দুটি তুলনা আর করা হয় না।
- **`couponText.trim().toUpperCase()`** — `toUpperCase()` String-এর সব অক্ষর বড় হাতের করে দেয়। ফলে `' save20 '` এবং `'SAVE20'` একই Code হিসেবে গণ্য হয়।
- **`code === 'SAVE20' && subtotal < SAVE20_MINIMUM`** — নিয়মটি দুটি মান মিলিয়ে: Code-টি `SAVE20` **এবং** কেনাকাটা সীমার নিচে।
- **দুটি Ternary** — সদস্য হলে ৫ শতাংশ, নইলে শূন্য; পরিশোধযোগ্য অঙ্ক সীমা ছুঁলে Delivery শূন্য, নইলে ১২০। দুটিই সরল দুই-পথের সিদ্ধান্ত।
- **হিসাবের ক্রম।** Coupon-এর ছাড় পুরো অঙ্কের ওপর; সদস্য-ছাড় তার পরের অঙ্কের ওপর। দ্বিতীয় উদাহরণে ৭৫০০ থেকে ২০ শতাংশ (১৫০০) বাদ গিয়ে ৬০০০, তার ৫ শতাংশ (৩০০) বাদ গিয়ে ৫৭০০।
- **`Math.round()`** — তৃতীয় উদাহরণে ৮৫০-এর ৫ শতাংশ ৪২.৫, যা ৪৩ টাকায় আনা হয়েছে।

প্রথম উদাহরণটি লক্ষ করার মতো: ৩০০০ টাকার কেনাকাটায় ছাড়ের পর থাকে ২৭০০, যা ফ্রি Delivery-র সীমার নিচে। নিয়মে বলা আছে "সব ছাড়ের পর", তাই ১২০ টাকা যোগ হয়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node discount.js
```

দাম, পরিমাণ, Coupon, সদস্য	যা দেখা যাওয়ার কথা
`'1500', '2', 'SAVE10', false`	মোট 3000, ছাড় 300, Delivery 120, দিতে হবে 2820
`'2500', '3', ' save20 ', true`	মোট 7500, ছাড় 1800, Delivery 0, দিতে হবে 5700
`'850', '1', '', true`	মোট 850, ছাড় 43, Delivery 120, দিতে হবে 927
`'1200', '2', 'SAVE20', false`	SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।
পরিমাণ `'0'`, `'101'`, `'2.5'` বা ফাঁকা	পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।
অচেনা Coupon, যেমন `'FREE50'`	Coupon Code সঠিক নয়।

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে; প্রতিটি সীমার মান (যেমন ঠিক ৩০০০ এবং ২৯৯৯) আলাদাভাবে পরীক্ষা করা হয়েছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ঠিক ৩০০০ টাকায় Delivery ফ্রি হয় না	`>=`-এর জায়গায় `>`	`payable >= FREE_DELIVERY_FROM`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ছোট হাতের `save10` বাতিল হয়	`toUpperCase()` নেই	`couponText.trim().toUpperCase()`
সব Coupon-এ একই ছাড়	`switch`-এ `return` বা `break` বাদ পড়েছে	প্রতিটি `case` `return` দিয়ে শেষ করুন
সদস্য-ছাড় বেশি আসছে	ছাড়টি `subtotal`-এর ওপর হিসাব হচ্ছে	`afterCoupon`-এর ওপর হিসাব করুন
ছাড়ে দশমিক সংখ্যা	`Math.round()` নেই	প্রতিটি ছাড় `Math.round()` করুন
ফাঁকা পরিমাণে `TypeError`	`null`-এর যাচাই পরে লেখা	`quantity === null` শর্তটি সবার আগে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project এবং নিয়মগুলো উদাহরণ মাত্র। এখানে একটিই পণ্য হিসাব করা হয়; একাধিক পণ্যের Cart অধ্যায় ৪-এ আসবে। Coupon-এর তালিকা Code-এর ভেতরে লেখা। সবচেয়ে গুরুত্বপূর্ণ: বাস্তব দোকানে মূল্য ও ছাড়ের চূড়ান্ত হিসাব Server-এ হতে হয়; ক্রেতার Computer-এ করা হিসাব শুধু আগাম ধারণা দেওয়ার জন্য।

৩.১৬ অনুশীলনী

প্রাথমিক স্তর

১. না চালিয়ে বলুন: `17 % 5`, `2 + 3 * 2` এবং `(2 + 3) * 2`-এর ফল কত। তারপর চালিয়ে মিলিয়ে নিন।

২. `isEven` নামে একটি Function লিখুন, যা সংখ্যা জোড় হলে `true` এবং বিজোড় হলে `false` ফেরত দেয়।

৩. `let points = 50;` দিয়ে শুরু করে সংক্ষিপ্ত Assignment Operator ব্যবহার করে ১০ যোগ করুন, তারপর দ্বিগুণ করুন, তারপর ১ কমান। প্রতিটি ধাপের পর মান দেখান।

৪. না চালিয়ে বলুন কোনগুলো Truthy: `'0'`, `0`, `' '`, `''`, `null`, `'null'`।

৫. নিচের Function-এ একটি ভুল আছে, যার কারণে `canVote(15)`-ও `true` দেয়। ভুলটি খুঁজে ঠিক করুন।

can-vote.js · JavaScript — চলবে: Browser Console ও Node.js

```
function canVote(age) {
  if (age >= 18 || age < 18) {
    return true;
  }

  return false;
}

console.log(canVote(15));
```

মধ্যম স্তর

৬. `getGrade` নামে একটি Function লিখুন, যা `else if` ব্যবহার করে নম্বর থেকে Grade দেয়: ৮০ বা বেশি `A+`, ৬০ বা বেশি `A`, ৩৩ বা বেশি `B`, নইলে `F`।

৭. `seasonOf` নামে একটি Function লিখুন, যা `switch` ব্যবহার করে মাসের নম্বর (১–১২) থেকে ঋতু দেয়: ১২, ১, ২ হলে `শীত`; ৩, ৪, ৫ হলে `গ্রীষ্ম`; ৬, ৭, ৮, ৯ হলে `বর্ষা`; ১০, ১১ হলে `হেমন্ত`; অন্য কিছু হলে `অজানা`।

৮. `ticketPrice` নামে একটি Function লিখুন: বয়স ৫-এর কম হলে `0`, ৫ থেকে ১২ হলে `50`, বাকিদের `100`। তারপর সদস্য হলে দাম অর্ধেক করুন (Ternary ব্যবহার করুন)।

৯. `isValidAge` নামে একটি Function লিখুন, যা Text নিয়ে `true` দেয় শুধু তখনই, যখন সেটি ০ থেকে ১২০-এর মধ্যে পূর্ণসংখ্যা। একটিমাত্র `return`-এ `&&` ব্যবহার করুন (ফাঁকা যাচাইয়ের Guard Clause বাদে)।

চ্যালেঞ্জ স্তর

১০. **নতুন Coupon:** `FLAT200` নামে একটি Coupon-এর ছাড় হিসাব করার Function লিখুন: কেনাকাটা ১০০০ টাকা বা বেশি হলে সরাসরি ২০০ টাকা ছাড়, নইলে শূন্য।

১১. **Delivery-র ধাপ:** `deliveryFee` Function লিখুন: অঙ্ক ৩০০০ বা বেশি হলে `0`; ১৫০০ বা বেশি হলে `60`; নইলে `120`। ঠিক সীমার মানগুলো দিয়ে পরীক্ষা করুন।

১২. **সর্বোচ্চ ছাড়:** `cappedDiscount` Function লিখুন, যা শতাংশ-ছাড় হিসাব করে, কিন্তু ছাড় কখনো ১০০০ টাকার বেশি হতে দেয় না।

৩.১৭ অনুশীলনীর সমাধান

সমাধান ১ — যথাক্রমে `2`, `8` এবং `10`। ১৭-কে ৫ দিয়ে ভাগ করলে ২ অবশিষ্ট থাকে। দ্বিতীয়টিতে গুণ আগে হয়; তৃতীয়টিতে বন্ধনী আগে।

সমাধান ২

is-even.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isEven(number) {
  return number % 2 === 0;
}

console.log(isEven(10));
console.log(isEven(7));
```

Output

```
true
false
```

সমাধান ৩

points.js · JavaScript — চলবে: Browser Console ও Node.js

```
let points = 50;

points += 10;
console.log(points);

points *= 2;
console.log(points);

points--;
console.log(points);
```

Output

```
60
120
119
```

সমাধান ৪ — Truthy: `'0'`, `' '` এবং `'null'`, কারণ তিনটিই ফাঁকা নয় এমন String। Falsy: `0`, `''` এবং `null`।

সমাধান ৫ — `age >= 18 || age < 18` যেকোনো সংখ্যার জন্য সত্য, কারণ দুটির একটি সব সময় মিলবেই। `||`-এর পরের অংশটি বাদ দিতে হবে।

can-vote.js · JavaScript — চলবে: Browser Console ও Node.js

```
function canVote(age) {
  return age >= 18;
}

console.log(canVote(15));
console.log(canVote(18));
```

Output

```
false
true
```

সমাধান ৬

get-grade.js · JavaScript — চলবে: Browser Console ও Node.js

```
function getGrade(mark) {
  if (mark >= 80) {
    return 'A+';
  } else if (mark >= 60) {
    return 'A';
  } else if (mark >= 33) {
    return 'B';
  } else {
    return 'F';
  }
}

console.log(getGrade(80));
console.log(getGrade(79));
console.log(getGrade(33));
console.log(getGrade(32));
```

Output

```
A+
A
B
F
```

সমাধান ৭

season.js · JavaScript — চলবে: Browser Console ও Node.js

```
function seasonOf(month) {
  switch (month) {
    case 12:
    case 1:
    case 2:
      return 'শীত';
    case 3:
    case 4:
    case 5:
      return 'গ্রীষ্ম';
    case 6:
    case 7:
    case 8:
    case 9:
      return 'বর্ষা';
    case 10:
    case 11:
      return 'হেমন্ত';
    default:
      return 'অজানা';
  }
}

console.log(seasonOf(1));
console.log(seasonOf(7));
console.log(seasonOf(13));
```

Output

```
শীত
বর্ষা
অজানা
```

সমাধান ৮

ticket.js · JavaScript — চলবে: Browser Console ও Node.js

```
function ticketPrice(age, isMember) {
  let price = 100;

  if (age < 5) {
    price = 0;
  } else if (age <= 12) {
    price = 50;
  }

  return isMember ? price / 2 : price;
}

console.log(ticketPrice(4, false));
console.log(ticketPrice(12, false));
console.log(ticketPrice(30, true));
```

Output

```
0
50
50
```

সমাধান ৯

valid-age.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isValidAge(text) {
  const clean = text.trim();

  if (clean === '') {
    return false;
  }

  const age = Number(clean);

  return Number.isInteger(age) && age >= 0 && age <= 120;
}

console.log(isValidAge('25'));
console.log(isValidAge('0'));
console.log(isValidAge('121'));
console.log(isValidAge(''));
```

Output

```
true
true
false
false
```

সমাধান ১০

flat200.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function flat200Discount(subtotal) {
  return subtotal >= 1000 ? 200 : 0;
}

console.log(flat200Discount(999));
console.log(flat200Discount(1000));
```

Output

```
0
200
```

সমাধান ১১

delivery-fee.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function deliveryFee(amount) {
  if (amount >= 3000) {
    return 0;
  } else if (amount >= 1500) {
    return 60;
  } else {
    return 120;
  }
}

console.log(deliveryFee(3000));
console.log(deliveryFee(2999));
console.log(deliveryFee(1500));
console.log(deliveryFee(1499));
```

Output

```
0
60
60
120
```

সমাধান ১২

capped-discount.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const MAX_DISCOUNT = 1000;

function cappedDiscount(subtotal, percent) {
  const discount = Math.round(subtotal * percent / 100);

  return discount > MAX_DISCOUNT ? MAX_DISCOUNT : discount;
}

console.log(cappedDiscount(3000, 10));
console.log(cappedDiscount(20000, 10));
```

Output

```
300
1000
```

৩.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `17 % 5`-এর ফল কত?

ক) `3`

খ) `2`

গ) `3.4`

ঘ) `12`

প্রশ্ন ২. `let a = 5; console.log(a++);` কী দেখাবে?

ক) `6`

খ) `4`

গ) `5`

ঘ) একটি Error

প্রশ্ন ৩. `'10' < '9'`-এর ফল কী?

ক) `true`, কারণ String অক্ষর ধরে তুলনা হয়

খ) `false`, কারণ ১০ বড়

গ) `NaN`

ঘ) একটি Error

প্রশ্ন ৪. নিচের কোনটি Falsy?

ক) `'0'`

খ) `' '`

গ) `'false'`

ঘ) `0`

প্রশ্ন ৫. `true || false && false`-এর ফল কী?

ক) `false`

খ) `true`

গ) `undefined`

ঘ) একটি Error

প্রশ্ন ৬. `0 ?? 10`-এর ফল কী?

ক) `10`

খ) `null`

গ) `0`

ঘ) `false`

প্রশ্ন ৭. `else if`-এর একাধিক শর্ত সত্য হলে কোন Block চলে?

ক) সবগুলো

খ) শেষ যেটি সত্য

গ) কোনোটিই নয়

ঘ) প্রথম যেটি সত্য

প্রশ্ন ৮. `switch`-এ একটি `case`-এর শেষে `break` বা `return` না থাকলে কী হয়?

ক) পরের `case`-এর Code-ও চলে

খ) একটি Error হয়

গ) `default` চলে

ঘ) `switch` থেকে বেরিয়ে যায়

> **প্রশ্ন ৯.** `const fee = total >= 3000 ? 0 : 120;`-তে `total` যদি `3000` হয়, `fee` কত?
>
> ক) `120`
>
> খ) `3000`
>
> গ) `0`
>
> ঘ) `undefined`

> **প্রশ্ন ১০.** `if (coupon = 'SAVE10')` লিখলে কী ঘটে?
>
> ক) `coupon`-এর সঙ্গে `'SAVE10'` তুলনা হয়
>
> খ) `coupon`-এ `'SAVE10'` রাখা হয় এবং শর্ত সত্য হয়
>
> গ) সব সময় একটি Error হয়
>
> ঘ) শর্ত সব সময় মিথ্যা হয়

৩.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–গ · ৭–ঘ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	১৭-কে ৫ দিয়ে ভাগ করলে ভাগফল ৩ এবং ভাগশেষ ২। `%` ভাগশেষ দেয়।
২	গ	`a++` আগে পুরোনো মান দেয়, তারপর বাড়ায়। এরপর `a`-এর মান হয় `6`।
৩	ক	`'1'` অভিধানে `'9'`-এর আগে আসে। সংখ্যা হিসেবে তুলনা করতে আগে `Number()` দরকার।
৪	ঘ	বাকি তিনটি ফাঁকা নয় এমন String, তাই Truthy।
৫	খ	`&&` আগে হিসাব হয়: `false && false` হলো `false`; তারপর `true || false` হলো `true`।
৬	গ	`??` শুধু `null` বা `undefined` হলে বিকল্প নেয়। `0` একটি বৈধ মান হিসেবে থেকে যায়।
৭	ঘ	শর্তগুলো ওপর থেকে নিচে যাচাই হয়; প্রথম সত্যটির Block চালিয়ে বাকিগুলো বাদ দেওয়া হয়।
৮	ক	একে Fallthrough বলে। ইচ্ছাকৃত না হলে এটি একটি Bug।
৯	গ	`>=` মানে বড় অথবা সমান, তাই শর্ত সত্য এবং প্রথম মানটি (`0`) বেছে নেওয়া হয়।
১০	খ	একটি `=` হলো Assignment। String-টি Truthy, তাই শর্ত সব সময় সত্য হয়।

৩.২০ অধ্যায়ের সারসংক্ষেপ

- Expression থেকে একটি মান পাওয়া যায়; Operator মান নিয়ে কাজ করে।
- Arithmetic: `+`, `-`, `*`, `/`, `%`, `**`। ভাগশেষ দিয়ে জোড়-বিজোড় ও চক্রাকার হিসাব করা যায়।
- `+=`, `-=`, `++`, `--` Variable-এর মান সংক্ষেপে বদলায়; `++` ও `--` আলাদা লাইনে লিখুন।
- Comparison-এর ফল Boolean। সীমার ক্ষেত্রে `>` আর `>=`-এর পার্থক্য খেয়াল করুন; String-এর তুলনা অভিধানের নিয়মে হয়।
- `&&` দুই পাশই সত্য চায়, `||` যেকোনো একটি, `!` উল্টে দেয়। `&&` আগে হিসাব হয়।
- প্রধান Falsy মান ছয়টি: `false`, `0`, `''`, `null`, `undefined`, `NaN`। শূন্য বৈধ হলে `if (value)` ব্যবহার করবেন না।
- `else if`-এ প্রথম সত্য শর্তটিই চলে, তাই কঠোরতম শর্ত আগে।
- নির্দিষ্ট মান মেলাতে `switch`; প্রতিটি `case`-এ `break` বা `return`, এবং সব সময় `default`।
- সরল দুই-পথের সিদ্ধান্তে Ternary।
- সন্দেহ হলে বন্ধনী।
- যাচাইয়ের ক্রম: আছে কি না, Type, সীমা, তারপর অন্য মানের সঙ্গে নিয়ম।

পরের অধ্যায়ে Loop: একই কাজ বারবার করা। সেখানে একাধিক পণ্যের Cart হিসাব করা হবে।

৩.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Expression	যে Code থেকে একটি মান পাওয়া যায়।
Operator / Operand	যে চিহ্ন কাজ করে / যে মানের ওপর কাজ হয়।
Remainder	ভাগ করার পর অবশিষ্ট অংশ (`%`)।
Comparison Operator	দুটি মান তুলনা করে Boolean দেয়।
Logical Operator	`&&`, `||`, `!`: শর্ত জোড়া লাগায় বা উল্টে দেয়।
Short-Circuit	ফল নিশ্চিত হয়ে গেলে বাকি অংশ হিসাব না করা।
Truthy / Falsy	শর্তে যে মান সত্য / মিথ্যা হিসেবে গণ্য হয়।
Fallthrough	`switch`-এ `break` না থাকায় পরের `case`-এ চলে যাওয়া।
Ternary Operator	`শর্ত ? মান১ : মান২`।
Operator Precedence	কোন Operator আগে হিসাব হবে, তার নিয়ম।
Input Validation	ব্যবহারের আগে বাইরে থেকে আসা মান যাচাই করা।

পরিভাষা	অর্থ
Guard Clause	Function-এর শুরুতে অগ্রহণযোগ্য অবস্থা ধরে `return` করা।

৩.২২ আরও পড়ুন

- **MDN — Expressions and operators:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators` — সব Operator-এর বিবরণ।
- **MDN — if...else:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/if...else` — শর্তযুক্ত Statement।
- **MDN — switch:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/switch` — `switch`, `break` এবং Fallthrough।
- **MDN — Operator precedence:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence` — পূর্ণ Precedence তালিকা।
- **MDN — Falsy:** `https://developer.mozilla.org/en-US/docs/Glossary/Falsy` — Falsy মানের সম্পূর্ণ তালিকা।

অধ্যায় ৪

08

Loops এবং Iteration

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একই কাজ বারবার করার জন্য `for`, `while` এবং `do...while` Loop লিখতে।
- কোন অবস্থায় কোন Loop উপযুক্ত, তা বেছে নিতে।
- একটি Array-র প্রতিটি মান Index ধরে পড়তে।
- Loop-এর শুরু ও শেষের সীমা ঠিক রেখে Off-by-One Error এড়াতে।
- `break` ও `continue` দিয়ে Loop-এর গতিপথ নিয়ন্ত্রণ করতে।
- Nested Loop লিখতে এবং তার খরচ বুঝতে।
- Infinite Loop চিনতে, থামাতে এবং প্রতিরোধ করতে।
- Loop দিয়ে যোগফল, গণনা, সর্বোচ্চ ও সর্বনিম্ন বের করতে।
- একটি সম্পূর্ণ **Cart Analyzer** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৩ সম্পন্ন করা: Comparison ও Logical Operator, `if` / `else`, `+=` এবং `++`।
- অধ্যায় ২-এর Object-এর প্রাথমিক পরিচয়: `{ name: 'Pen' }` এবং ডট দিয়ে তথ্য পড়া।
- `chapter-04` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৩-এর Discount Calculator একটি পণ্যের হিসাব করত। বাস্তব Cart-এ পাঁচটি, পঞ্চাশটি বা পাঁচশটি পণ্য থাকতে পারে। প্রতিটির জন্য আলাদা লাইন লেখা সম্ভব নয়, আর আগে থেকে জানাও যায় না কয়টি পণ্য থাকবে।

Loop এই সমস্যার সমাধান: একটি কাজ একবার লিখে যতবার প্রয়োজন ততবার চালানো। তালিকার প্রতিটি পণ্যের দাম যোগ করা, প্রতিটি শিক্ষার্থীর ফল দেখানো, একটি শর্ত পূরণ না হওয়া পর্যন্ত চেষ্টা করে যাওয়া—সবই Loop-এর কাজ। Program যে গতিতে হাজার হাজার তথ্য সামলাতে পারে, তার ভিত্তি এটিই।

Loop-এর সঙ্গে দুটি নতুন ধরনের ভুলও আসে: একবার বেশি বা একবার কম চলা, এবং কখনো না থামা। এই অধ্যায়ে দুটোই চিনে নেবেন।

8.১ Loop কেন প্রয়োজন

পাঁচটি পণ্যের নম্বর দেখাতে হলে পাঁচবার `console.log()` লেখা যায়। পাঁচশটি হলে? Loop একই কাজ এভাবে করে:

first-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let i = 1; i <= 5; i++) {
  console.log(`পণ্য ${i}`);
}
```

Output

```
পণ্য 1
পণ্য 2
পণ্য 3
পণ্য 4
পণ্য 5
```

`5`-এর জায়গায় `500` লিখলেই পাঁচশ লাইন দেখাবে; Code এক লাইনও বাড়বে না। Loop-এর ভেতরের Code একবার চলাকে একটি **Iteration** বলে। ওপরের Loop-এ পাঁচটি Iteration হয়েছে।

৪.২ for Loop

কতবার চলবে তা আগে থেকে জানা থাকলে `for` সবচেয়ে উপযুক্ত। এর প্রথম বন্ধনীতে Semicolon দিয়ে আলাদা করা তিনটি অংশ থাকে।

অংশ	উদাহরণ	কখন চলে	কাজ
Initialization	`let i = 1`	শুরুতে একবার	গণনার Variable তৈরি
Condition	`i <= 5`	প্রতিটি Iteration-এর আগে	সত্য হলে Loop চলে, মিথ্যা হলে থামে
Update	`i++`	প্রতিটি Iteration-এর পরে	Variable বদলায়

গণনার Variable-টিকে **Counter** বলা হয়। `i` নামটি একটি পুরোনো রীতি; ছোট Loop-এ এটি গ্রহণযোগ্য।

ধাপগুলো একটি একটি করে অনুসরণ করলে Loop-এর আচরণ পরিষ্কার হয়।

sum-to-ten.js · JavaScript — চলবে: Browser Console ও Node.js

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
  sum += i;
}

console.log(sum);
```

Output

```
55
```

Iteration	`i`	শর্ত `i <= 10`	`sum` (Iteration-এর পরে)
১	1	সত্য	1
২	2	সত্য	3
৩	3	সত্য	6
…	…	…	…
১০	10	সত্য	55
—	11	মিথ্যা	Loop শেষ

এ ধরনের টেবিলকে Trace Table বলে। Loop প্রত্যাশামতো কাজ না করলে হাতে এমন একটি টেবিল তৈরি করা সবচেয়ে নির্ভরযোগ্য Debugging কৌশলগুলোর একটি।

`sum` Loop-এর বাইরে তৈরি করা হয়েছে, কারণ এটি সব Iteration জুড়ে মান ধরে রাখে। Loop-এর ভেতরে তৈরি করলে প্রতিবার নতুন করে শূন্য হয়ে যেত। আর `i` শুধু Loop-এর ভেতরেই কার্যকর; Loop শেষ হলে সেটি আর থাকে না।

ভিন্ন ধাপে এবং উল্টো দিকে

Update অংশে যেকোনো পরিবর্তন লেখা যায়।

steps.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let i = 2; i <= 10; i += 2) {
  console.log(i);
}

for (let i = 3; i >= 1; i--) {
  console.log(`বাকি ${i}`);
}
```

Output

```
2
4
6
8
10
বাকি 3
বাকি 2
বাকি 1
```

উল্টো দিকে গোনার সময় শর্তটিও উল্টে যায়: `i >= 1`, এবং Update হয় `i--`।

৪.৩ Array: তালিকার প্রথম পরিচয়

Loop-এর সবচেয়ে বড় ব্যবহার তালিকা নিয়ে কাজ করা। JavaScript-এ একাধিক মানকে ক্রমানুসারে একসঙ্গে রাখার মান হলো **Array**। তৃতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করে মানগুলো লেখা হয়। Array নিয়ে পূর্ণ আলোচনা অধ্যায় ৬-এ; এখানে শুধু Loop-এর জন্য যতটুকু দরকার।

array-intro.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];

console.log(prices[0]);
console.log(prices[2]);
console.log(prices.length);
console.log(prices[3]);
```

Output

```
120
950
3
undefined
```

- Array-র প্রতিটি মানকে একটি **Element** বলে, আর তার অবস্থানের নম্বরকে **Index**।
- **Index শূন্য থেকে শুরু হয়।** প্রথম Element `prices[0]`, দ্বিতীয়টি `prices[1]`।
- `prices.length` Element-এর সংখ্যা দেয়।
- শেষ Element-এর Index তাই `length - 1`। এখানে তিনটি Element, শেষ Index `2`।
- যে Index-এ কিছু নেই, সেখান থেকে পড়লে Error হয় না; `undefined` পাওয়া যায়।

শেষ দুটি বিষয় মনে রাখুন; অনুচ্ছেদ ৪.৪-এর ভুলটি এখান থেকেই আসে।

Loop দিয়ে Array পড়া

loop-array.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i < prices.length; i++) {
  console.log(`Index ${i}: ${prices[i]}`);
  total += prices[i];
}

console.log(`মোট: ${total}`);
```

Output

```
Index 0: 120
Index 1: 15
Index 2: 950
মোট: 1085
```

এই ধরনটি মুখস্থ রাখার মতো: `let i = 0` থেকে শুরু, `i < array.length` পর্যন্ত, `i++` করে এগোনো। Array-তে আরও Element যোগ হলেও Loop বদলাতে হবে না, কারণ শর্তটি `length` থেকে নিজেই সীমা জেনে নেয়।

8.8 Loop-এর সীমা এবং Off-by-One Error

Loop একবার বেশি বা একবার কম চললে যে ভুল হয়, তাকে **Off-by-One Error** বলে। এটি Programming-এর সবচেয়ে প্রচলিত ভুলগুলোর একটি, এবং প্রায় সব সময় দুটি জায়গা থেকে আসে: শুরুর মান এবং শর্তের Operator।

off-by-one.js · JavaScript চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i <= prices.length; i++) {
  console.log(prices[i]);
  total += prices[i];
}

console.log(total);
```

Output

```
120
15
950
undefined
NaN
```

শর্তে `<`-এর জায়গায় `<=` লেখায় Loop চারবার চলেছে। চতুর্থবার `i` হয়েছে `3`, আর `prices[3]`-এ কিছু নেই। `undefined` যোগ করায় পুরো যোগফল `NaN` হয়ে গেছে। কোনো Error Message আসেনি; শুধু ফল ভুল।

উল্টো ভুলটিও ঘটে: `let i = 1` দিয়ে শুরু করলে প্রথম Element বাদ পড়ে।

লেখা	কোন Index-গুলো পড়া হয় (৩টি Element)	ফল
`i = 0; i < length`	0, 1, 2	সঠিক
`i = 0; i <= length`	0, 1, 2, 3	একবার বেশি; শেষে `undefined`
`i = 1; i < length`	1, 2	প্রথমটি বাদ
`i = 0; i < length - 1`	0, 1	শেষটি বাদ

> **মনে রাখুন:** Index শূন্য থেকে শুরু, তাই Array-র Loop-এ `i = 0` এবং `i < array.length`। মানুষের জন্য ক্রমিক নম্বর দেখাতে হলে দেখানোর সময় `i + 1` লিখুন; Loop-এর সীমা বদলাবেন না।

সীমা ঠিক আছে কি না যাচাইয়ের সহজ উপায়: নিজেকে তিনটি প্রশ্ন করুন। প্রথম Iteration-এ Counter-এর মান কত? শেষ Iteration-এ কত? মোট কয়বার চলবে?

8.৫ while Loop

কতবার চলতে হবে তা আগে থেকে জানা না থাকলে `while` ব্যবহার করা হয়। এতে শুধু একটি শর্ত থাকে; শর্ত যতক্ষণ সত্য, Loop চলে।

savings.js · JavaScript চলবে: Browser Console ও Node.js

```
const target = 5000;
const monthlySaving = 1200;
let balance = 0;
let months = 0;

while (balance < target) {
  balance += monthlySaving;
  months++;
}

console.log(`${months} মাসে জমা হবে ${balance} টাকা`);
```

Output

```
5 মাসে জমা হবে 6000 টাকা
```

প্রশ্নটি ছিল "কত মাস লাগবে?"—উত্তরটিই অজানা, তাই `for` দিয়ে আগে থেকে বলা যায় না কতবার চলবে। `while` প্রতিবার শর্ত দেখে: লক্ষ্যে পৌঁছানো হয়নি? তাহলে আরেক মাস।

`for`-এর তিনটি অংশ `while`-এও থাকে, শুধু আলাদা জায়গায়: Variable তৈরি Loop-এর আগে, শর্ত বন্ধনীতে, আর Update Loop-এর ভেতরে। Update-টি লিখতে ভুলে গেলে শর্ত কখনো মিথ্যা হয় না; বিষয়টি অনুচ্ছেদ ৪.৯-এ আছে।

	for	while
উপযুক্ত	Iteration-এর সংখ্যা জানা; Array ধরে চলা	সংখ্যা অজানা; একটি অবস্থা না হওয়া পর্যন্ত চলা
Counter	Loop-এর শিরোনামেই	নিজে আলাদাভাবে সামলাতে হয়

৪.৬ do...while Loop

`while` আগে শর্ত দেখে, তারপর চলে; শর্ত শুরুতেই মিথ্যা হলে একবারও চলে না। `do...while` উল্টো: আগে একবার চলে, তারপর শর্ত দেখে।

do-while.js · JavaScript চলবে: Browser Console ও Node.js

```
const stock = 0;

while (stock > 0) {
  console.log('while: একবার চলল');
}

do {
  console.log('do...while: একবার চলল');
} while (stock > 0);
```

Output

```
do...while: একবার চলল
```

শর্ত দুই ক্ষেত্রেই মিথ্যা, কিন্তু `do...while`-এর ভেতরের Code একবার চলেছে। `while`-এর বন্ধনীর পরে Semicolon লক্ষ করুন; `do...while`-এ এটি প্রয়োজন।

যে কাজ অন্তত একবার করতেই হবে, সেখানে এটি উপযুক্ত: ব্যবহারকারীকে একটি প্রশ্ন করা এবং উত্তর সঠিক না হওয়া পর্যন্ত আবার করা; একটি Menu দেখানো; একটি সংযোগের চেষ্টা করা।

attempts.js · JavaScript চলবে: Browser Console ও Node.js

```js
const maxAttempts = 3;
let attempt = 0;

do {
  attempt++;
  console.log(`চেষ্টা ${attempt}`);
} while (attempt < maxAttempts);
```

Output

```
চেষ্টা 1
চেষ্টা 2
চেষ্টা 3
```

বাস্তবে `do...while` তুলনামূলক কম লাগে। অধিকাংশ ক্ষেত্রে `for` বা `while`-ই যথেষ্ট।

৪.৭ break এবং continue

Loop-এর স্বাভাবিক গতি বদলানোর দুটি উপায় আছে।

- **`break`** — Loop সম্পূর্ণ থামিয়ে দেয়। Loop-এর পরের লাইন থেকে Program চলতে থাকে।
- **`continue`** — শুধু বর্তমান Iteration-এর বাকি অংশ বাদ দেয়, এবং পরের Iteration-এ চলে যায়।

break: পাওয়া গেলে থেমে যাওয়া

break.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280, 1500];
let checked = 0;

for (let i = 0; i < prices.length; i++) {
  checked++;

  if (prices[i] > 500) {
    console.log(`প্রথম দামি পণ্য: ${prices[i]} (Index ${i})`);
    break;
  }
}

console.log(`যাচাই করা হয়েছে ${checked}টি`);
```

Output

```
প্রথম দামি পণ্য: 950 (Index 2)
যাচাই করা হয়েছে 3টি
```

যা খোঁজা হচ্ছিল তা তৃতীয় Element-এই পাওয়া গেছে, তাই বাকি দুটি আর দেখা হয়নি। তালিকা বড় হলে এই সাশ্রয় তাৎপর্যপূর্ণ।

continue: অপ্রয়োজনীয়টি বাদ দেওয়া

continue.js · JavaScript — চলবে: Browser Console ও Node.js

```
const amounts = [50, -10, 30, 0, 20];
let total = 0;
let skipped = 0;

for (let i = 0; i < amounts.length; i++) {
  if (amounts[i] <= 0) {
    skipped++;
    continue;
  }

  total += amounts[i];
}

console.log(`মোট: ${total}, বাদ পড়েছে: ${skipped}`);
```

Output

```
মোট: 100, বাদ পড়েছে: 2
```

শূন্য বা ঋণাত্মক মান পেলে `continue` যোগ করার লাইনটি এড়িয়ে পরের Element-এ চলে যায়। এটি অধ্যায় ৩-এর Guard Clause-এরই Loop-সংস্করণ: অগ্রহণযোগ্য অবস্থা আগে সরিয়ে দিলে মূল কাজ `if`-এর ভেতরে ঢুকে যায় না।

> **সতর্কতা:** `while` Loop-এ `continue` ব্যবহারের সময় খেয়াল রাখুন, Counter-এর Update যেন `continue`-এর আগে থাকে। নইলে Update বাদ পড়ে এবং Loop আর থামে না। `for` Loop-এ এই ঝুঁকি নেই, কারণ `continue`-এর পরেও Update অংশটি চলে।

৪.৮ Nested Loop

একটি Loop-এর ভেতরে আরেকটি Loop থাকলে তাকে **Nested Loop** বলে। বাইরের Loop-এর প্রতিটি Iteration-এ ভেতরের Loop শুরু থেকে শেষ পর্যন্ত পুরোটা চলে।

nested.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let row = 1; row <= 3; row++) {
  let line = '';

  for (let column = 1; column <= 4; column++) {
    line += `${row * column} `;
  }

  console.log(line);
}
```

Output

```
1 2 3 4
2 4 6 8
3 6 9 12
```

বাইরের Loop তিনবার চলে, আর প্রতিবার ভেতরের Loop চারবার। ভেতরের লাইনটি মোট ৩ × ৪ = ১২ বার চলেছে। `line` বাইরের Loop-এর ভেতরে কিন্তু ভেতরের Loop-এর বাইরে তৈরি, তাই প্রতিটি সারির জন্য এটি নতুন করে ফাঁকা হয়।

দুটি Loop-এ আলাদা Counter ব্যবহার করুন এবং অর্থবহ নাম দিন: `row` ও `column` পড়লেই বোঝা যায় কোনটি কী। দুটোতেই `i` লিখলে ভেতরেরটি বাইরেরটিকে ঢেকে দেয়।

Nested Loop-এর খরচ

Nested Loop-এ কাজের পরিমাণ গুণ হয়ে বাড়ে।

তালিকার আকার	একটি Loop	দুটি Nested Loop
10	10 বার	100 বার

তালিকার আকার	একটি Loop	দুটি Nested Loop
1,000	1,000 বার	10,00,000 বার
1,00,000	1,00,000 বার	1,000 কোটি বার

ছোট তালিকায় পার্থক্য বোঝা যায় না; বড় তালিকায় Program কয়েক সেকেন্ড থেকে কয়েক ঘণ্টায় চলে যেতে পারে। Nested Loop লেখার আগে ভাবুন: ভেতরের Loop-টি কি সত্যিই প্রয়োজন, নাকি একটি Loop-এই কাজ হয়? Performance নিয়ে বিস্তারিত আলোচনা অধ্যায় ১৯-এ আছে।

৪.৯ Infinite Loop এবং তার প্রতিরোধ

যে Loop-এর শর্ত কখনো মিথ্যা হয় না, সেটি থামে না। একে **Infinite Loop** বলে। Program আটকে যায়, Computer-এর Processor পুরো ব্যস্ত থাকে, এবং Browser-এ হলে Page সাড়া দেওয়া বন্ধ করে।

নিচের Code-টি একটি Infinite Loop। **এটি চালানোর দরকার নেই;** চালিয়ে ফেললে Terminal-এ `Ctrl + C` চেপে থামান।

infinite.js · JavaScript — চলবে: Node.js

```
let i = 0;

while (i !== 10) {
  i += 3;
}

console.log('এই লাইনে কখনো পৌঁছানো হবে না');
```

`i`-এর মান হয় 3, 6, 9, 12, 15...। এটি কখনো ঠিক `10` হয় না, তাই `i !== 10` সব সময় সত্য। শর্তটি `i < 10` হলে `12`-তে গিয়ে Loop থেমে যেত।

Infinite Loop-এর তিনটি সাধারণ কারণ:

কারণ	উদাহরণ	সমাধান
Update নেই	`while (i < 5) { console.log(i); }`	Loop-এর ভেতরে `i++`
Update ভুল দিকে	`for (let i = 0; i < 5; i--)`	`i++`
শর্ত ঠিক একটি মানের অপেক্ষায়	`while (i !== 10)` যখন `i` তিন করে বাড়ে	`!==`-এর বদলে `<` বা `>`

নিরাপত্তা-সীমা

যে `while` Loop-এ শর্ত বাইরের কোনো অবস্থার ওপর নির্ভর করে, সেখানে সর্বোচ্চ Iteration-এর একটি সীমা বসানো একটি ভালো অভ্যাস।

safety-limit.js · JavaScript — চলবে: Browser Console ও Node.js

```
const MAX_STEPS = 1000;
let i = 0;
let steps = 0;

while (i !== 10) {
  i += 3;
  steps++;

  if (steps >= MAX_STEPS) {
    console.log('থামানো হলো: শর্তটি মিথ্যা হচ্ছে না');
    break;
  }
}

console.log(`মোট ধাপ: ${steps}`);
```

Output

```
থামানো হলো: শর্তটি মিথ্যা হচ্ছে না
মোট ধাপ: 1000
```

একই ভুল শর্ত, কিন্তু এবার Program আটকে না থেকে সমস্যাটি জানিয়ে দিয়েছে। সীমাটি ভুলকে লুকায় না; বরং ধরা সহজ করে।

৪.১০ Loop দিয়ে Data প্রক্রিয়াকরণ

Loop-এর অধিকাংশ কাজ কয়েকটি পরিচিত ধরনের মধ্যে পড়ে। ধরনগুলো চিনে রাখলে নতুন সমস্যায় কোনটি লাগবে তা সহজে বোঝা যায়। প্রতিটিতে Loop-এর আগে একটি Variable তৈরি হয়, যাতে ফল জমা হয়; একে **Accumulator** বলে।

patterns.js · JavaScript — চলবে: Browser Console ও Node.js

```
const marks = [72, 88, 45, 91, 60];

let sum = 0;
let passed = 0;
let highest = marks[0];
let lowest = marks[0];

for (let i = 0; i < marks.length; i++) {
  const mark = marks[i];
```

```
  sum += mark;

  if (mark >= 60) {
    passed++;
  }

  if (mark > highest) {
    highest = mark;
  }

  if (mark < lowest) {
    lowest = mark;
  }
}

console.log(`যোগফল: ${sum}`);
console.log(`গড়: ${sum / marks.length}`);
console.log(`৬০ বা বেশি: ${passed} জন`);
console.log(`সর্বোচ্চ: ${highest}, সর্বনিম্ন: ${lowest}`);
```

Output

```
যোগফল: 356
গড়: 71.2
৬০ বা বেশি: 4 জন
সর্বোচ্চ: 91, সর্বনিম্ন: 45
```

ধরন	শুরুর মান	প্রতিটি Iteration-এ
যোগফল	`0`	`sum += value`
গণনা	`0`	শর্ত মিললে `count++`
সর্বোচ্চ	প্রথম Element	বড় হলে বদলে দেওয়া
সর্বনিম্ন	প্রথম Element	ছোট হলে বদলে দেওয়া
খোঁজা	`null` বা `-1`	পাওয়া গেলে রেখে `break`

সর্বোচ্চ ও সর্বনিম্নের শুরুর মান হিসেবে `0` নয়, প্রথম Element নেওয়া হয়েছে। `0` দিয়ে শুরু করলে সব নম্বর ঋণাত্মক হলে সর্বোচ্চ ভুল আসত, আর সর্বনিম্ন সব সময় `0`-ই থেকে যেত।

একটি Loop-এই চারটি হিসাব করা হয়েছে। চারটি আলাদা Loop লিখলেও ফল একই হতো, কিন্তু তালিকাটি চারবার পড়তে হতো।

Object-এর তালিকা

বাস্তব তথ্য সাধারণত Object-এর Array হিসেবে আসে: প্রতিটি Element একটি Object।

students.js · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', mark: 80 },
  { name: 'Rafi', mark: 28 },
  { name: 'Mina', mark: 65 },
];

for (let i = 0; i < students.length; i++) {
  const student = students[i];
  const status = student.mark >= 33 ? 'উত্তীর্ণ' : 'অনুত্তীর্ণ';

  console.log(`${i + 1}. ${student.name}: ${status}`);
}
```

Output

```
1. Sara: উত্তীর্ণ
2. Rafi: অনুত্তীর্ণ
3. Mina: উত্তীর্ণ
```

`students[i]` একটি Object দেয়; সেটিকে `student` নামে রেখে ডট দিয়ে তার তথ্য পড়া হয়েছে। ক্রমিক নম্বর দেখাতে `i + 1` ব্যবহার করা হয়েছে, কিন্তু Loop-এর সীমা শূন্য থেকেই। Project-এ ঠিক এই ধরনটিই ব্যবহার করা হবে।

৪.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
শর্তে `<=` (`i <= array.length`)	শেষে `undefined`; যোগফল `NaN`	`i < array.length`
Index `1` থেকে শুরু	প্রথম Element বাদ পড়ে	`let i = 0`
`while`-এ Update নেই	Program আটকে যায়	Loop-এর ভেতরে Counter বদলান
Accumulator Loop-এর ভেতরে তৈরি	প্রতিবার শূন্য হয়ে যায়; শুধু শেষ মানটি থাকে	Loop-এর আগে তৈরি করুন
Counter `const` দিয়ে তৈরি	`TypeError: Assignment to constant variable.`	`let i`
Nested Loop-এ একই Counter	বাইরের Loop এলোমেলো চলে	আলাদা নাম: `row`, `column`
সর্বোচ্চ খুঁজতে `0` দিয়ে শুরু	ঋণাত্মক সংখ্যায় ভুল ফল	প্রথম Element দিয়ে শুরু
`break`-এর বদলে `continue`, বা উল্টো	খোঁজা থামে না, অথবা আগেই থেমে যায়	`break` থামায়, `continue` এড়িয়ে যায়

ভুল	কী দেখা যায়	সমাধান
Loop-এর ভেতরে Array-র বাইরের Index	`undefined`	Index সব সময় `0` থেকে `length - 1`

৪.১২ Debugging নির্দেশিকা

Loop-এর Bug ধরার মূল উপায় হলো প্রতিটি Iteration-এ কী ঘটছে তা দেখা।

debug-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i < prices.length; i++) {
  total += prices[i];
  console.log(`i=${i} মান=${prices[i]} total=${total}`);
}
```

Output

```
i=0 মান=120 total=120
i=1 মান=15 total=135
i=2 মান=950 total=1085
```

এটি Trace Table-এর স্বয়ংক্রিয় রূপ। Counter, বর্তমান মান এবং Accumulator—তিনটি একসঙ্গে দেখলে বোঝা যায় কোন Iteration-এ হিসাব ভুল পথে গেছে।

Loop প্রত্যাশামতো কাজ না করলে:

১. Counter, বর্তমান Element এবং Accumulator প্রতি Iteration-এ দেখান।

২. প্রথম এবং শেষ Iteration আলাদাভাবে লক্ষ করুন; Off-by-One সেখানেই ধরা পড়ে।

৩. Iteration-এর সংখ্যা গুনুন: এটি কি `array.length`-এর সমান?

৪. ফল `NaN` হলে খুঁজুন কোন Iteration-এ `undefined` বা String যোগ হয়েছে।

৫. Program আটকে গেলে `Ctrl + C` চেপে থামান, তারপর দেখুন Counter আদৌ বদলাচ্ছে কি না এবং সঠিক দিকে বদলাচ্ছে কি না।

৬. ছোট তালিকা দিয়ে পরীক্ষা করুন: শূন্য, এক এবং দুই Element-এর Array।

৭. ফাঁকা Array দিলে কী হয় দেখুন। `marks[0]` তখন `undefined`, এবং গড়ের হিসাবে শূন্য দিয়ে ভাগ হয়।

৪.১৩ পেশাদার অভ্যাস

- **সঠিক Loop বেছে নিন।** সংখ্যা জানা থাকলে বা Array ধরে চললে `for`; একটি অবস্থার অপেক্ষায় থাকলে `while`।
- **সীমা `length` থেকে নিন।** `i < 3` নয়, `i < prices.length`।
- **Counter-এর অর্থবহ নাম দিন** যখন Loop ছোট নয় বা Nested: `row`, `column`, `attempt`।
- **Loop-এর ভেতরে বর্তমান Element-কে একটি নাম দিন।** `const item = items[i];` লিখলে বাকি Code পড়া সহজ হয়।
- **Loop-এর ভেতরের কাজ ছোট রাখুন।** বড় হলে একটি Function-এ নিয়ে যান।
- **যে `while`-এর শেষ নিশ্চিত নয়, তাতে নিরাপত্তা-সীমা দিন।**
- **Loop চলাকালে যে Array ধরে চলছেন সেটি বদলাবেন না।** Element যোগ বা বাদ দিলে Index সরে যায়।
- **ফাঁকা তালিকার কথা ভাবুন।** শূন্য Element-এ আপনার Code কী করবে?
- **অপ্রয়োজনীয় Nested Loop এড়িয়ে চলুন।**

৪.১৪ Project: Cart Analyzer

একটি Cart-এর সব পণ্য বিশ্লেষণ করে সারসংক্ষেপ তৈরি করবেন: মোট টাকা, মোট পণ্যের সংখ্যা, সবচেয়ে বড় অঙ্কের পণ্য এবং একটি নির্দিষ্ট অঙ্কের বেশি দামের পণ্যগুলোর নাম।

Project-এর চাহিদা

- Cart হলো Object-এর একটি Array। প্রতিটি Object-এ পণ্যের নাম (`name`), একটির দাম (`price`) এবং পরিমাণ (`quantity`) আছে।
- একটি পণ্য গ্রহণযোগ্য, যদি নাম ফাঁকা না হয়, দাম শূন্য বা তার বেশি পূর্ণসংখ্যা হয়, এবং পরিমাণ ১ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হয়।
- অগ্রহণযোগ্য পণ্য হিসাবে ধরা হবে না, তবে কয়টি বাদ পড়ল তা জানাতে হবে।
- প্রতিটি পণ্যের জন্য দাম × পরিমাণ দেখাতে হবে।
- মোট টাকা, মোট পরিমাণ, সবচেয়ে বড় অঙ্কের পণ্য এবং একটি নির্দিষ্ট সীমার সমান বা বেশি অঙ্কের পণ্যগুলোর নাম দেখাতে হবে।
- ফাঁকা Cart দিলে Program ভেঙে পড়বে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
cart-analyzer/
├── cart.js
├── README.md
└── tests/
    └── cart.test.js
```

cart.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: একটি পণ্য যাচাই

cart.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const MAX_QUANTITY = 100;

// একটি পণ্য গ্রহণযোগ্য কি না যাচাই করে।
function isValidItem(item) {
  const hasName = item.name.trim() !== '';
  const hasPrice = Number.isInteger(item.price) && item.price >= 0;
  const hasQuantity = Number.isInteger(item.quantity) &&
    item.quantity >= 1 && item.quantity <= MAX_QUANTITY;

  return hasName && hasPrice && hasQuantity;
}

// ...
```

তিনটি শর্তের প্রতিটিকে একটি নাম দেওয়া হয়েছে, তারপর `&&` দিয়ে জোড়া লাগানো হয়েছে। এক লাইনে সব লিখলে একই কাজ হতো, কিন্তু পড়া কঠিন হতো। একটি Expression দীর্ঘ হলে Operator-এর পরে ভেঙে পরের লাইনে লেখা যায়, যেমন `hasQuantity`-তে করা হয়েছে।

ধাপ ২: পুরো Cart বিশ্লেষণ

cart.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// Cart-এর সব পণ্য একবার ঘুরে একটি সারসংক্ষেপ তৈরি করে।
function analyzeCart(items, threshold) {
  let total = 0;
  let units = 0;
  let skipped = 0;
```

```
  let highestName = '';
  let highestAmount = 0;
  let bigItems = '';

  for (let i = 0; i < items.length; i++) {
    const item = items[i];

    if (!isValidItem(item)) {
      skipped++;
      continue;
    }

    const amount = item.price * item.quantity;

    total += amount;
    units += item.quantity;

    if (amount > highestAmount) {
      highestAmount = amount;
      highestName = item.name;
    }

    if (amount >= threshold) {
      bigItems += bigItems === '' ? item.name : `, ${item.name}`;
    }
  }

  return {
    total: total,
    units: units,
    skipped: skipped,
    highestName: highestName,
    highestAmount: highestAmount,
    bigItems: bigItems,
  };
}

// ...
```

এই Function-এ অনুচ্ছেদ ৪.১০-এর প্রায় সব ধরন একসঙ্গে আছে: যোগফল (`total`, `units`), গণনা (`skipped`), সর্বোচ্চ (`highestAmount`), এবং অগ্রহণযোগ্য পণ্য বাদ দিতে `continue`। ফলগুলো একটি Object-এ ভরে ফেরত দেওয়া হয়েছে, কারণ একটি Function একটিই মান ফেরত দিতে পারে; একাধিক তথ্য পাঠাতে সেগুলোকে একটি Object-এ রাখতে হয়।

ধাপ ৩: Report দেখানো

cart.js · JavaScript (অংশ ৩ / ৩ (Part 3 of 3)) চলবে: Node.js

```
// ...

// প্রতিটি পণ্যের লাইন এবং শেষে সারসংক্ষেপ দেখায়।
function printReport(items, threshold) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const label = `${i + 1}. ${item.name}`;
    const amount = item.price * item.quantity;

    console.log(`${label}: ${item.price} x ${item.quantity} = ${amount}`);
  }

  const summary = analyzeCart(items, threshold);

  console.log(`মোট: ${summary.total} টাকা, পরিমাণ: ${summary.units}`);
  console.log(`সবচেয়ে বড় অঙ্ক: ${summary.highestName} (${summary.highestAmount})`);
  console.log(`${threshold} টাকা বা বেশি: ${summary.bigItems}`);
  console.log(`বাদ পড়েছে: ${summary.skipped}টি`);
}

const cart = [
  { name: 'Notebook', price: 120, quantity: 3 },
  { name: 'Pen', price: 15, quantity: 10 },
  { name: 'School Bag', price: 950, quantity: 1 },
  { name: 'Calculator', price: 650, quantity: 0 },
  { name: 'Water Bottle', price: 280, quantity: 2 },
];

printReport(cart, 500);
```

Output

```
1. Notebook: 120 x 3 = 360
2. Pen: 15 x 10 = 150
3. School Bag: 950 x 1 = 950
4. Calculator: 650 x 0 = 0
5. Water Bottle: 280 x 2 = 560
মোট: 2020 টাকা, পরিমাণ: 16
সবচেয়ে বড় অঙ্ক: School Bag (950)
500 টাকা বা বেশি: School Bag, Water Bottle
বাদ পড়েছে: 1টি
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **সঠিক সীমা।** দুটি Loop-ই `let i = 0` থেকে `i < items.length` পর্যন্ত। ক্রমিক নম্বরের জন্য দেখানোর সময় `i + 1`।

- `const item = items[i];` — প্রতিটি Iteration-এ বর্তমান পণ্যটিকে একটি নাম দেওয়া হয়েছে। `const` এখানে ঠিক আছে, কারণ প্রতিটি Iteration-এ এটি নতুন করে তৈরি হয়।
- `continue` **দিয়ে Guard।** Calculator-এর পরিমাণ শূন্য, তাই `isValidItem` মিথ্যা দেয়; `skipped` এক বাড়ে এবং বাকি হিসাব বাদ পড়ে। মোট টাকা ও পরিমাণে এটি ধরা হয়নি।
- **সর্বোচ্চ খোঁজা।** এখানে শুরুর মান `0` নেওয়া নিরাপদ, কারণ যাচাইয়ের পর কোনো অঙ্ক ঋণাত্মক হতে পারে না। দুটি পণ্যের অঙ্ক সমান হলে প্রথমটিই থাকে, কারণ শর্তটি `>`, `>=` নয়।
- **নামের তালিকা তৈরি।** `bigItems` একটি String, যাতে নামগুলো জোড়া লাগে। প্রথম নামের আগে কমা বসবে না, তাই Ternary দিয়ে দেখা হয়েছে String-টি এখনো ফাঁকা কি না। অধ্যায় ৬-এ Array দিয়ে এ কাজ আরও সহজে করা যাবে।
- **হিসাব আর দেখানো আলাদা।** `analyzeCart` কিছু দেখায় না; শুধু হিসাব করে ফেরত দেয়। `printReport` দেখানোর কাজ করে। এই বিভাজনের কারণে `analyzeCart`-কে আলাদাভাবে পরীক্ষা করা যায়।

`printReport` প্রতিটি পণ্যের লাইন দেখায়, অগ্রহণযোগ্যটিসহ; বাদ পড়ার বিষয়টি শুধু সারসংক্ষেপে ধরা পড়ে। অনুশীলনী ১১-তে এটি উন্নত করবেন।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node cart.js
```

অবস্থা	যা দেখা যাওয়ার কথা
ওপরের পাঁচটি পণ্য, সীমা ৫০০	মোট 2020 টাকা, পরিমাণ 16; সবচেয়ে বড় অঙ্ক School Bag (950); বাদ পড়েছে 1টি
সীমা ৫০০	School Bag, Water Bottle
সীমা ৯৫০	শুধু School Bag (সীমার সমান অঙ্কও ধরা হয়)
ফাঁকা Cart `[]`	মোট 0 টাকা, পরিমাণ 0; কোনো Error নয়
পরিমাণ `0`, `101` বা `2.5`	পণ্যটি বাদ পড়ে
নাম ফাঁকা বা দাম ঋণাত্মক	পণ্যটি বাদ পড়ে

Companion Source Code-এ এই Project-এর ১৫টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`TypeError: Cannot read properties of undefined (reading 'name')`	Loop-এর শর্তে `<=`	`i < items.length`
মোট `NaN`	কোনো পণ্যের দাম বা পরিমাণ সংখ্যা নয় এবং যাচাই বাদ পড়েছে	`isValidItem`-এর `continue` Block আছে কি না দেখুন
মোট শুধু শেষ পণ্যের সমান	`total` Loop-এর ভেতরে তৈরি	Loop-এর আগে `let total = 0;`
নামের তালিকার শুরুতে কমা	ফাঁকা যাচাইয়ের Ternary নেই	`bigItems === '' ? ... : ...`
অগ্রহণযোগ্য পণ্যও মোটে ধরা হচ্ছে	`continue` নেই	`skipped++`-এর পরে `continue;`
Program আটকে যায়	`i++` বাদ পড়েছে	`for`-এর তৃতীয় অংশ দেখুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Cart-এর তথ্য Code-এর ভেতরে লেখা এবং সংখ্যা হিসেবেই দেওয়া; Text থেকে রূপান্তর এখানে করা হয়নি (অধ্যায় ২ ও ৩-এর কৌশল প্রয়োগ করে তা যোগ করা যায়)। প্রতিটি `item` যে একটি Object এবং তাতে `name` আছে, তা ধরে নেওয়া হয়েছে; তা না হলে Program Error দেবে। ছাড় ও Delivery-র হিসাব নেই। টাকার অঙ্ক পূর্ণসংখ্যায় ধরা হয়েছে।

৪.১৫ অনুশীলনী

প্রাথমিক স্তর

১. `for` Loop দিয়ে ১ থেকে ১০০ পর্যন্ত সংখ্যার যোগফল বের করুন।

২. `while` Loop দিয়ে ৫ থেকে ১ পর্যন্ত উল্টো গুনে শেষে `শুরু!` দেখান।

৩. `[3, 8, 11, 16, 19]` Array থেকে শুধু জোড় সংখ্যাগুলো দেখান।

৪. না চালিয়ে বলুন: `for (let i = 0; i < 5; i += 2)` কয়বার চলবে এবং `i`-এর মানগুলো কী হবে?

৫. নিচের Code-এ একটি Off-by-One Error আছে। ভুলটি ঠিক করুন, যাতে তিনটি নামই দেখায় এবং `undefined` না আসে।

names-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (let i = 1; i <= names.length; i++) {
  console.log(names[i]);
}
```

মধ্যম স্তর

৬. `[40, 75, 12, 98, 63]` Array-র সর্বোচ্চ মান এবং সেটির Index বের করুন।

৭. একটি Array-তে প্রথম `0` পাওয়ার আগ পর্যন্ত সংখ্যাগুলো যোগ করুন; `0` পেলে থেমে যান। `[5, 10, 0, 20]` দিয়ে পরীক্ষা করুন।

৮. Nested Loop দিয়ে নিচের মতো একটি ত্রিভুজ দেখান (চার সারি):

Output

```
*
**
***
****
```

৯. `countPassed` নামে একটি Function লিখুন, যা নম্বরের Array এবং পাস নম্বর নিয়ে কতজন উত্তীর্ণ তা ফেরত দেয়। ফাঁকা Array দিলে `0` ফেরত দেবে।

চ্যালেঞ্জ স্তর

১০. **সবচেয়ে সস্তা:** পণ্যের Array থেকে সবচেয়ে কম দামের পণ্যের নাম বের করার Function লিখুন। ফাঁকা Array দিলে `null` ফেরত দেবে।

১১. **বাদ পড়া চিহ্নিত:** পরিমাণ শূন্য বা তার কম হলে পণ্যের লাইনের জায়গায় `(বাদ)` দেখায়, এমন একটি Function লিখুন।

১২. **কত মাসে দ্বিগুণ:** একটি অঙ্ক প্রতি মাসে ৫ শতাংশ করে বাড়লে কত মাসে দ্বিগুণ হয়, তা `while` দিয়ে বের করুন। প্রতি মাসের শেষে অঙ্কটি `Math.round()` করুন, এবং নিরাপত্তা-সীমা হিসেবে সর্বোচ্চ ১০০০ মাস রাখুন।

৪.১৬ অনুশীলনীর সমাধান

সমাধান ১

sum-hundred.js · JavaScript — চলবে: Browser Console ও Node.js

```
let sum = 0;

for (let i = 1; i <= 100; i++) {
  sum += i;
}

console.log(sum);
```

Output

```
5050
```

সমাধান ২

countdown.js · JavaScript — চলবে: Browser Console ও Node.js

```
let count = 5;

while (count >= 1) {
  console.log(count);
  count--;
}

console.log('শুরু!');
```

Output

```
5
4
3
2
1
শুরু!
```

সমাধান ৩

even-only.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [3, 8, 11, 16, 19];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    console.log(numbers[i]);
  }
}
```

Output

```
8
16
```

সমাধান ৪ — তিনবার। `i`-এর মান হয় `0`, `2` এবং `4`। এরপর `i` হয় `6`, তখন `6 < 5` মিথ্যা, তাই Loop থামে।

সমাধান ৫ — দুটি ভুল একসঙ্গে: শুরু `1` থেকে (প্রথম নাম বাদ) এবং শর্তে `<=` (শেষে `undefined`)। সঠিক সীমা `0` থেকে `< length`।

names-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (let i = 0; i < names.length; i++) {
  console.log(names[i]);
}
```

Output

```
Sara
Rafi
Mina
```

সমাধান ৬

highest-index.js · JavaScript — চলবে: Browser Console ও Node.js

```
const values = [40, 75, 12, 98, 63];
let highest = values[0];
let highestIndex = 0;

for (let i = 1; i < values.length; i++) {
  if (values[i] > highest) {
    highest = values[i];
    highestIndex = i;
  }
}

console.log(`সর্বোচ্চ ${highest}, Index ${highestIndex}`);
```

Output

```
সর্বোচ্চ 98, Index 3
```

এখানে `i = 1` থেকে শুরু করা ইচ্ছাকৃত এবং সঠিক: প্রথম Element আগেই `highest`-এ নেওয়া হয়েছে, তাই সেটির সঙ্গে নিজেকে তুলনা করার দরকার নেই।

সমাধান ৭

until-zero.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [5, 10, 0, 20];
let sum = 0;

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] === 0) {
    break;
  }

  sum += numbers[i];
}

console.log(sum);
```

Output

```
15
```

সমাধান ৮

triangle.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let row = 1; row <= 4; row++) {
  let line = '';

  for (let star = 1; star <= row; star++) {
    line += '*';
  }

  console.log(line);
}
```

Output

```
*
**
***
****
```

ভেতরের Loop-এর সীমা বাইরের Counter-এর ওপর নির্ভর করে (`star <= row`), তাই প্রতিটি সারিতে একটি করে তারা বাড়ে।

সমাধান ৯

count-passed.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countPassed(marks, passMark) {
  let passed = 0;

  for (let i = 0; i < marks.length; i++) {
    if (marks[i] >= passMark) {
      passed++;
    }
  }

  return passed;
}

console.log(countPassed([72, 28, 45, 33], 33));
console.log(countPassed([], 33));
```

Output

```
3
0
```

ফাঁকা Array-র `length` শূন্য, তাই Loop একবারও চলে না এবং শুরুর মান `0`-ই ফেরত যায়। আলাদা কোনো যাচাই লাগেনি।

সমাধান ১০

cheapest.js · JavaScript — চলবে: Browser Console ও Node.js

```
function cheapestName(items) {
  if (items.length === 0) {
    return null;
  }

  let cheapest = items[0];

  for (let i = 1; i < items.length; i++) {
    if (items[i].price < cheapest.price) {
      cheapest = items[i];
    }
  }

  return cheapest.name;
}

const items = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
  { name: 'Bag', price: 950 },
];

console.log(cheapestName(items));
console.log(cheapestName([]));
```

Output

```
Pen
null
```

সমাধান ১১

mark-skipped.js · JavaScript — চলবে: Browser Console ও Node.js

```
function printLines(items) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];

    if (item.quantity <= 0) {
      console.log(`${i + 1}. ${item.name}: (বাদ)`);
      continue;
    }

    console.log(`${i + 1}. ${item.name}: ${item.price * item.quantity}`);
  }
}

printLines([
  { name: 'Pen', price: 15, quantity: 10 },
  { name: 'Calculator', price: 650, quantity: 0 },
]);
```

Output

```
1. Pen: 150
2. Calculator: (বাদ)
```

সমাধান ১২

double.js · JavaScript — চলবে: Browser Console ও Node.js

```
const MAX_MONTHS = 1000;
const start = 10000;
let amount = start;
let months = 0;

while (amount < start * 2 && months < MAX_MONTHS) {
  amount = Math.round(amount * 105 / 100);
  months++;
}

console.log(`${months} মাসে ${amount}`);
```

Output

```
15 মাসে 20791
```

নিরাপত্তা-সীমাটি এখানে শর্তের অংশ: অঙ্ক দ্বিগুণ হয়নি **এবং** সর্বোচ্চ মাস পেরোয়নি। শতাংশের হিসাবে আগে গুণ, পরে ভাগ করা হয়েছে।

৪.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `for` Loop-এর Update অংশ (যেমন `i++`) কখন চলে?

ক) Loop শুরুর আগে একবার

খ) প্রতিটি Iteration-এর পরে

গ) প্রতিটি Iteration-এর আগে

ঘ) শুধু শর্ত মিথ্যা হলে

প্রশ্ন ২. পাঁচটি Element-এর Array-তে শেষ Element-এর Index কত?

ক) `5`

খ) `6`

গ) `4`

ঘ) `0`

প্রশ্ন ৩. `do...while` Loop-এর ভেতরের Code সর্বনিম্ন কতবার চলে?

ক) একবার

খ) শূন্যবার

গ) দুইবার

ঘ) শর্তের ওপর নির্ভর করে

প্রশ্ন ৪. `break` কী করে?

ক) বর্তমান Iteration বাদ দিয়ে পরেরটিতে যায়

খ) Program বন্ধ করে দেয়

গ) Loop আবার শুরু থেকে চালায়

ঘ) Loop সম্পূর্ণ থামিয়ে দেয়

প্রশ্ন ৫. `continue` কী করে?

ক) Loop সম্পূর্ণ থামিয়ে দেয়

খ) বর্তমান Iteration-এর বাকি অংশ বাদ দিয়ে পরেরটিতে যায়

গ) Function থেকে মান ফেরত দেয়

ঘ) Counter শূন্য করে দেয়

প্রশ্ন ৬. `for (let i = 0; i <= arr.length; i++)` লিখলে কী সমস্যা হয়?

ক) প্রথম Element বাদ পড়ে

খ) Loop একবারও চলে না

গ) শেষে একটি অতিরিক্ত Iteration হয়, যেখানে `arr[i]` হলো `undefined`

ঘ) কোনো সমস্যা হয় না

প্রশ্ন ৭. কতবার চলতে হবে তা আগে থেকে জানা না থাকলে কোন Loop সবচেয়ে উপযুক্ত?

ক) `while`

খ) `for`

গ) `switch`

ঘ) Nested `for`

প্রশ্ন ৮. বাইরের Loop ৪ বার এবং ভেতরের Loop ৫ বার চললে ভেতরের Code মোট কতবার চলে?

ক) ৯ বার

খ) ৫ বার

গ) ৪ বার

ঘ) ২০ বার

প্রশ্ন ৯. নিচের কোনটি Infinite Loop তৈরি করে?

ক) `for (let i = 0; i < 5; i++) { }`

খ) `let i = 0; while (i < 5) { console.log(i); }`

গ) `let i = 5; while (i > 0) { i--; }`

ঘ) `for (let i = 5; i > 0; i--) { }`

প্রশ্ন ১০. সব সংখ্যা ঋণাত্মক হতে পারে, এমন একটি Array-র সর্বোচ্চ মান খুঁজতে শুরুর মান কী হওয়া উচিত?

ক) `0`

খ) `null`

গ) Array-র প্রথম Element

ঘ) `Infinity`

৪.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	ক্রমটি হলো: শর্ত, ভেতরের Code, Update, আবার শর্ত।
২	গ	Index শূন্য থেকে শুরু, তাই শেষ Index হলো `length - 1`।
৩	ক	`do...while` আগে চলে, পরে শর্ত দেখে।
৪	ঘ	`break`-এর পর Loop-এর পরের লাইন থেকে Program চলে।
৫	খ	Loop চলতে থাকে; শুধু এই Iteration-এর বাকি অংশ বাদ যায়।
৬	গ	`i` যখন `length`-এর সমান, তখন সেই Index-এ কিছু নেই।
৭	ক	`while` শুধু একটি শর্ত দেখে এবং সেটি সত্য থাকা পর্যন্ত চলে।
৮	ঘ	বাইরের প্রতিটি Iteration-এ ভেতরের Loop পুরোটা চলে: ৪ × ৫।
৯	খ	`i` কখনো বদলায় না, তাই `i < 5` সব সময় সত্য।
১০	গ	`0` দিয়ে শুরু করলে সব সংখ্যা ঋণাত্মক হলে ভুল করে `0`-ই সর্বোচ্চ থেকে যায়।

৪.১৯ অধ্যায়ের সারসংক্ষেপ

- Loop একটি কাজ বারবার চালায়; প্রতিবার চলা একটি Iteration।
- `for`-এ তিনটি অংশ: Initialization, Condition, Update। সংখ্যা জানা থাকলে বা Array ধরে চললে এটি ব্যবহার করুন।
- Array ক্রমানুসারে মান রাখে; Index শূন্য থেকে শুরু, শেষ Index `length - 1`।
- Array-র Loop: `let i = 0; i < array.length; i++`।
- Off-by-One Error আসে শুরুর মান বা শর্তের Operator থেকে; প্রথম ও শেষ Iteration যাচাই করুন।
- `while` শর্ত সত্য থাকা পর্যন্ত চলে; `do...while` অন্তত একবার চলে।
- `break` Loop থামায়; `continue` বর্তমান Iteration এড়িয়ে যায়।
- Nested Loop-এ কাজ গুণ হয়ে বাড়ে।
- Infinite Loop-এর কারণ: Update নেই, ভুল দিকে, অথবা শর্ত কখনো মিথ্যা হয় না। `Ctrl + C` দিয়ে থামান; নিরাপত্তা-সীমা দিয়ে প্রতিরোধ করুন।
- যোগফল, গণনা, সর্বোচ্চ, সর্বনিম্ন এবং খোঁজা—Loop-এর পাঁচটি মৌলিক ধরন, প্রতিটিতে একটি Accumulator থাকে।

পরের অধ্যায়ে Function এবং Scope: Code-কে পুনর্ব্যবহারযোগ্য অংশে ভাগ করা, এবং কোন Variable কোথা থেকে দেখা যায় তার নিয়ম।

৪.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Loop	একই Code বারবার চালানোর ব্যবস্থা।
Iteration	Loop-এর ভেতরের Code-এর একবার চলা।
Counter	Loop কতবার চলেছে তা ধরে রাখার Variable।
Array	ক্রমানুসারে একাধিক মান রাখার মান।
Element / Index	Array-র একটি মান / তার অবস্থানের নম্বর (শূন্য থেকে শুরু)।
Off-by-One Error	Loop একবার বেশি বা একবার কম চলার ভুল।
Nested Loop	একটি Loop-এর ভেতরে আরেকটি Loop।
Infinite Loop	যে Loop কখনো থামে না।
Accumulator	Loop জুড়ে ফল জমা রাখার Variable।
Trace Table	প্রতিটি Iteration-এ Variable-এর মান লিখে রাখার টেবিল।

৪.২১ আরও পড়ুন

- **MDN — Loops and iteration:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration` — সব ধরনের Loop-এর বিবরণ।
- **MDN — for:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for` — `for` Statement-এর পূর্ণ নিয়ম।
- **MDN — while:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/while` — `while` Statement।
- **MDN — break:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/break` — `break`-এর ব্যবহার।

অধ্যায় ৫

Functions এবং Scope

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Function Declaration, Function Expression এবং Arrow Function লিখতে ও তাদের পার্থক্য বলতে।
- Parameter ও Argument-এর পার্থক্য ব্যাখ্যা করতে।
- `return` দিয়ে মান ফেরত দিতে এবং `return` ও `console.log()`-এর পার্থক্য বুঝতে।
- Default Parameter ব্যবহার করতে।
- Block Scope, Function Scope এবং Global Scope চিনতে।
- Lexical Scope কীভাবে ঠিক করে কোন Variable কোথা থেকে পাওয়া যাবে, তা বুঝতে।
- Hoisting-এর মৌলিক আচরণ ব্যাখ্যা করতে।
- বড় কাজকে ছোট, পুনর্ব্যবহারযোগ্য Function-এ ভাগ করতে।
- একটি সম্পূর্ণ **Invoice Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৪ সম্পন্ন করা: `for` Loop এবং Array-র প্রাথমিক ব্যবহার।
- অধ্যায় ২-এর `let`, `const` ও Block; অধ্যায় ৩-এর শর্ত ও Ternary।
- অধ্যায় ১ থেকে এ পর্যন্ত আপনি Function ব্যবহার করে আসছেন; এই অধ্যায়ে সেটির পূর্ণ ব্যাখ্যা।
- `chapter-05` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

গত চারটি অধ্যায়ে আপনি `toMark`, `calculateOrder`, `analyzeCart`-এর মতো Function লিখেছেন, কারণ Function ছাড়া অর্থপূর্ণ Program লেখা প্রায় অসম্ভব। কিন্তু সেগুলো ছিল একটি নির্দিষ্ট ছাঁচে লেখা। Function আসলে কীভাবে কাজ করে, কত রকমভাবে লেখা যায় এবং Function-এর ভেতরের Variable বাইরে থেকে কেন দেখা যায় না—এসব এখনো ব্যাখ্যা করা হয়নি।

Function হলো Program গঠনের মূল একক। একটি বড় সমস্যাকে ছোট ছোট নামযুক্ত কাজে ভাগ করা, প্রতিটিকে আলাদাভাবে পরীক্ষা করা এবং একই কাজ বহু জায়গায় ব্যবহার করা—পেশাদার Code-এর এই তিনটি বৈশিষ্ট্যই Function থেকে আসে।

এর সঙ্গে জড়িত **Scope**: কোন নাম Program-এর কোন অংশ থেকে ব্যবহার করা যায়, তার নিয়ম। Scope না বুঝলে "Variable-টি তো আছে, তবু খুঁজে পাচ্ছে না কেন" ধরনের Error বারবার ফিরে আসে।

৫.১ Function কেন প্রয়োজন

একটি দোকানের Program-এ তিন জায়গায় পণ্যের মোট দাম হিসাব করতে হয়: Cart-এ, Invoice-এ এবং Report-এ। তিন জায়গায় একই হিসাব আলাদাভাবে লিখলে নিয়ম বদলানোর সময় তিন জায়গাতেই বদলাতে হবে, আর একটি বাদ পড়লেই Bug।

why-functions.js · JavaScript — চলবে: Browser Console ও Node.js

```
function calculateLineTotal(price, quantity) {
  return price * quantity;
}

console.log(calculateLineTotal(120, 3));
console.log(calculateLineTotal(15, 10));
```

Output

```
360
150
```

হিসাবটি একবার লেখা হয়েছে, একটি নাম দেওয়া হয়েছে, এবং দুবার ব্যবহার করা হয়েছে। Function তিনটি সুবিধা দেয়:

- **পুনর্ব্যবহার।** একবার লিখে যতবার প্রয়োজন ব্যবহার।
- **নাম।** `price * quantity`-এর চেয়ে `calculateLineTotal` বেশি স্পষ্টভাবে বলে কী করা হচ্ছে।
- **এক জায়গায় পরিবর্তন।** নিয়ম বদলালে শুধু Function-টি বদলাতে হয়।

৫.২ Function Declaration

ওপরের লেখার ধরনটিকে **Function Declaration** বলে। এর চারটি অংশ:

অংশ	উদাহরণে	কাজ
`function` শব্দ	`function`	JavaScript-কে জানায় একটি Function তৈরি হচ্ছে
নাম	`calculateLineTotal`	এই নাম দিয়ে Function-টি ব্যবহার করা হবে
Parameter-এর তালিকা	`(price, quantity)`	Function যে মানগুলো গ্রহণ করে
Body	`{ return price * quantity; }`	যে Code চলবে

Function তৈরি করলেই সেটি চলে না। চালাতে হলে নামের পরে বন্ধনী দিয়ে **Call** করতে হয়: `calculateLineTotal(120, 3)`। বন্ধনী ছাড়া শুধু নাম লিখলে Function-টি চলে না; নামটি তখন Function-টিকেই বোঝায়, যা অনুচ্ছেদ ৫.৬-এ কাজে লাগবে।

Function-এর নাম একটি ক্রিয়া দিয়ে শুরু করা প্রচলিত রীতি, কারণ Function কিছু একটা করে: `calculateTotal`, `formatPrice`, `isValid`, `getGrade`।

৫.৩ Parameter এবং Argument

দুটি শব্দ প্রায়ই একই অর্থে ব্যবহৃত হয়, কিন্তু এরা আলাদা।

- **Parameter** — Function তৈরির সময় বন্ধনীতে লেখা নাম। এগুলো Function-এর ভেতরের Variable।
- **Argument** — Function Call করার সময় দেওয়া আসল মান।

`function calculateLineTotal(price, quantity)`-তে `price` ও `quantity` Parameter। `calculateLineTotal(120, 3)`-তে `120` ও `3` Argument। Call-এর সময় Argument-গুলো ক্রম অনুযায়ী Parameter-এ বসে: প্রথমটি প্রথমে, দ্বিতীয়টি দ্বিতীয়তে।

সংখ্যা না মিললে

অনেক ভাষায় Argument-এর সংখ্যা ভুল হলে Error হয়। JavaScript-এ হয় না।

argument-count.js · JavaScript চলবে: Browser Console ও Node.js

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3));
console.log(add(2));
console.log(add(2, 3, 4));
```

Output

```
5
NaN
5
```

- কম Argument দিলে বাকি Parameter-গুলো `undefined` হয়। `2 + undefined` এর ফল `NaN`।
- বেশি দিলে অতিরিক্তগুলো নীরবে উপেক্ষিত হয়।

কোনো ক্ষেত্রেই Error Message আসে না। তাই Function Call-এর সময় Argument-এর সংখ্যা ও ক্রম মিলিয়ে নেওয়া আপনার দায়িত্ব। ক্রম উল্টে গেলেও (`calculateLineTotal(3, 120)`) JavaScript কিছু বলবে না।

৫.৪ Return Value

`return` দুটি কাজ করে: Function-এর ফলাফল Call-এর জায়গায় ফেরত পাঠায়, এবং Function সেখানেই শেষ করে দেয়।

নতুনদের সবচেয়ে প্রচলিত বিভ্রান্তি হলো `return` এবং `console.log()` গুলিয়ে ফেলা।

return-vs-log.js · JavaScript চলবে: Browser Console ও Node.js

```
function showTotal(price, quantity) {
  console.log(price * quantity);
}

function getTotal(price, quantity) {
  return price * quantity;
}

const shown = showTotal(120, 3);
const returned = getTotal(120, 3);

console.log(shown);
console.log(returned);
console.log(getTotal(120, 3) + 50);
```

Output

```
360
undefined
360
410
```

`showTotal` পর্দায় `360` দেখায়, কিন্তু কিছু ফেরত দেয় না; তাই `shown`-এর মান `undefined`। যে Function-এ `return` নেই, অথবা `return`-এর পরে কোনো মান নেই, সেটি সব সময় `undefined` ফেরত দেয়।

`getTotal` কিছু দেখায় না, কিন্তু মানটি ফেরত দেয়। সেই মান Variable-এ রাখা যায়, অন্য হিসাবে ব্যবহার করা যায় (শেষ লাইনে `+ 50`), অথবা অন্য Function-এ পাঠানো যায়।

	`console.log()`	`return`
কার জন্য	Developer-এর চোখের জন্য	Program-এর বাকি অংশের জন্য
মানটি পরে ব্যবহার করা যায়	না	হ্যাঁ
Function থামায়	না	হ্যাঁ

> **মনে রাখুন:** যে Function হিসাব করে, সেটি ফল `return` করবে; দেখাবে না। দেখানোর কাজ আলাদা Function-এ রাখুন। তাহলে একই হিসাব পর্দায়, File-এ বা Test-এ—সব জায়গায় ব্যবহার করা যায়।

return Function থামিয়ে দেয়

`return` চলার পর Function-এর আর কোনো লাইন চলে না। অধ্যায় ৩-এর Guard Clause এই আচরণের ওপরই দাঁড়িয়ে।

early-return.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describeStock(quantity) {
  if (quantity <= 0) {
    return 'শেষ হয়ে গেছে';
  }

  if (quantity < 5) {
    return 'অল্প বাকি';
  }

  return 'মজুত আছে';
}

console.log(describeStock(0));
console.log(describeStock(3));
console.log(describeStock(40));
```

Output

```
শেষ হয়ে গেছে
অল্প বাকি
মজুত আছে
```

একটি Function একটিই মান ফেরত দিতে পারে। একাধিক তথ্য ফেরত দিতে হলে সেগুলোকে একটি Object বা Array-তে রেখে ফেরত দিতে হয়, যেমন অধ্যায় ৪-এর `analyzeCart` করেছিল।

৫.৫ Default Parameter

কোনো Argument না দিলে Parameter-টি `undefined` হয়। **Default Parameter** দিয়ে এমন ক্ষেত্রের জন্য একটি পূর্বনির্ধারিত মান ঠিক করে রাখা যায়।

default-parameter.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greet(name = 'অতিথি') {
  return `স্বাগতম, ${name}!`;
}

console.log(greet('Sara'));
console.log(greet());
console.log(greet(undefined));
console.log(greet(null));
```

Output

```
স্বাগতম, Sara!
স্বাগতম, অতিথি!
স্বাগতম, অতিথি!
স্বাগতম, null!
```

Default মান বসে শুধু তখনই, যখন Argument দেওয়া হয়নি অথবা `undefined` দেওয়া হয়েছে। `null` একটি ইচ্ছাকৃত মান (অধ্যায় ২), তাই `null` দিলে Default বসে না।

Default Parameter-গুলো তালিকার শেষে রাখুন। `function price(amount, discount = 0)` ঠিক আছে: শুধু `price(500)` লিখলেই চলে। Default-টি প্রথমে থাকলে তাকে এড়িয়ে দ্বিতীয়টি দেওয়ার সহজ উপায় থাকে না।

default-discount.js · JavaScript চলবে: Browser Console ও Node.js

```
function finalPrice(amount, discountPercent = 0) {
  return amount - Math.round(amount * discountPercent / 100);
}

console.log(finalPrice(500));
console.log(finalPrice(500, 10));
```

Output

```
500
450
```

৫.৬ Function Expression

JavaScript-এ Function নিজেই একটি মান, ঠিক সংখ্যা বা String-এর মতো। তাই Function-কে একটি Variable-এ রাখা যায়। এভাবে লেখাকে **Function Expression** বলে।

function-expression.js · JavaScript চলবে: Browser Console ও Node.js

```
const add = function (a, b) {
  return a + b;
};

console.log(add(2, 3));
console.log(typeof add);
```

Output

```
5
function
```

`function` -এর পরে কোনো নাম নেই; নামটি আসে Variable থেকে। এটি একটি Assignment Statement, তাই শেষে Semicolon আছে। Call করার নিয়ম একই।

Function যেহেতু একটি মান, তাকে অন্য Function-এর Argument হিসেবেও পাঠানো যায়। এই ক্ষমতা অনুচ্ছেদ ৫.১১-এ এবং অধ্যায় ৬-এ বারবার কাজে লাগবে।

৫.৭ Arrow Function

Arrow Function হলো Function Expression লেখার সংক্ষিপ্ত রূপ। `function` শব্দের বদলে Parameter-এর পরে `=>` (তীরের মতো দেখতে) বসে।

arrow-functions.js · JavaScript চলবে: Browser Console ও Node.js

```
const double = (number) => number * 2;

const isAdult = (age) => age >= 18;

const describe = (name, mark) => {
  const status = mark >= 33 ? 'উত্তীর্ণ' : 'অনুত্তীর্ণ';
  return `${name}: ${status}`;
};

const greeting = () => 'স্বাগতম!';

console.log(double(21));
console.log(isAdult(15));
console.log(describe('Sara', 80));
console.log(greeting());
```

Output

```
42
false
Sara: উত্তীর্ণ
স্বাগতম!
```

Arrow Function-এর Body দুইভাবে লেখা যায়:

- **একটি Expression, `{ }` ছাড়া।** Expression-টির মান নিজে থেকেই ফেরত যায়; `return` লিখতে হয় না। `double` এবং `isAdult` এভাবে লেখা।
- **`{ }` সহ একাধিক লাইন।** তখন `return` নিজে লিখতে হয়, যেমন `describe`-এ। `{ }` দিয়ে `return` বাদ দিলে Function `undefined` ফেরত দেবে।

Parameter না থাকলে ফাঁকা বন্ধনী `()` লিখতে হয়।

তিনটি রূপ পাশাপাশি

রূপ	উদাহরণ	উপযুক্ত ব্যবহার
Function Declaration	`function add(a, b) { return a + b; }`	Program-এর প্রধান, নামযুক্ত Function
Function Expression	`const add = function (a, b) { return a + b; };`	পুরোনো Code-এ দেখা যায়
Arrow Function	`const add = (a, b) => a + b;`	ছোট Function; অন্য Function-কে পাঠানোর Function

এই বইয়ে একটি সরল রীতি মানা হয়েছে: যে Function-এর একাধিক ধাপ আছে, সেটি Function Declaration; ছোট, এক লাইনের হিসাব Arrow Function।

> **নোট:** Arrow Function এবং সাধারণ Function-এর মধ্যে আরেকটি পার্থক্য আছে, যা `this` নামের একটি বিশেষ শব্দের সঙ্গে জড়িত। সেটি Object ও Class-এর আলোচনায়, অধ্যায় ৭ ও ১৩-তে আসবে। এ পর্যন্ত লেখা কোনো Code-এ তার প্রভাব নেই।

৫.৮ Scope: Block, Function এবং Global

Scope হলো Program-এর সেই অংশ, যেখান থেকে একটি Variable ব্যবহার করা যায়। JavaScript-এ তিন স্তরের Scope আছে।

Scope	কোথায় তৈরি Variable	কোথা থেকে পাওয়া যায়
Global	সব Function ও Block-এর বাইরে	File-এর সব জায়গা থেকে
Function	একটি Function-এর ভেতরে	শুধু সেই Function-এর ভেতর থেকে
Block	একটি `{ }`-এর ভেতরে (`if`, `for` ইত্যাদি), `let` বা `const` দিয়ে	শুধু সেই Block-এর ভেতর থেকে

scopes.js · JavaScript — চলবে: Browser Console ও Node.js

```
const shopName = 'Boi Ghar';

function printReceipt() {
  const total = 500;

  if (total > 100) {
    const note = 'বড় কেনাকাটা';
    console.log(shopName, total, note);
  }

  console.log(typeof note);
}

printReceipt();
console.log(typeof total);
```

Output

```
Boi Ghar 500 বড় কেনাকাটা
undefined
undefined
```

- `shopName` Global Scope-এ; Function ও Block-এর ভেতর থেকেও পাওয়া যায়।
- `total` Function Scope-এ; `printReceipt`-এর বাইরে এর অস্তিত্ব নেই।
- `note` Block Scope-এ; `if`-এর Block শেষ হলেই আর পাওয়া যায় না।

এখানে `typeof` ব্যবহার করা হয়েছে, কারণ যে নাম নেই তার ওপর `typeof` Error না দিয়ে `'undefined'` দেয়। সরাসরি ব্যবহার করতে গেলে Program থেমে যায়:

out-of-scope.js · JavaScript চলবে: Node.js

```
function printReceipt() {
  const total = 500;
}

printReceipt();
console.log(total);
```

Error Message

```
ReferenceError: total is not defined
```

নিয়মটি একমুখী: **ভেতর থেকে বাইরের Variable দেখা যায়, বাইরে থেকে ভেতরের নয়।**

Scope কেন উপকারী

Scope একটি সীমাবদ্ধতা মনে হলেও এটি আসলে সুরক্ষা। দুটি আলাদা Function-এ `total` নামে Variable থাকতে পারে, এবং একটি অন্যটিকে প্রভাবিত করে না। প্রতিটি Function নিজের নাম নিজে বেছে নিতে পারে, বাকি Program-এ কী নাম ব্যবহৃত হয়েছে তা না জেনেই।

উল্টো দিকে, Global Variable সব জায়গা থেকে বদলানো যায়। বড় Program-এ একটি Global মান ভুল হয়ে গেলে কোন Function সেটি বদলেছে, তা খুঁজে বের করা কঠিন। তাই Global Scope-এ শুধু স্থির মান (`const`) এবং Function রাখুন।

Parameter-ও Function Scope-এর Variable: `printReceipt(amount)` লিখলে `amount` শুধু ওই Function-এর ভেতরেই থাকে।

৫.৯ Lexical Scope

একটি Function-এর ভেতরে আরেকটি Function লেখা যায়। ভেতরের Function বাইরের Function-এর Variable দেখতে পায়। JavaScript একটি নাম খোঁজে ভেতর থেকে বাইরের দিকে: আগে নিজের Scope-এ, না পেলে তার বাইরের Scope-এ, এভাবে Global পর্যন্ত। এই ধারাবাহিক খোঁজার পথকে **Scope Chain** বলে।

scope-chain.js · JavaScript — চলবে: Browser Console ও Node.js

```
const rate = 5;

function outer() {
  const rate = 10;

  function inner() {
    return rate;
  }

  return inner();
}

console.log(outer());
console.log(rate);
```

Output

```
10
5
```

`inner` নিজের ভেতরে `rate` পায়নি, তাই এক ধাপ বাইরে `outer`-এ খুঁজেছে এবং সেখানে `10` পেয়ে থেমে গেছে। Global `rate` পর্যন্ত যাওয়ার দরকার হয়নি। ভেতরের Scope-এ একই নামের Variable থাকলে সেটি বাইরেরটিকে ঢেকে দেয়; একে **Shadowing** বলে। বাইরের `rate` অপরিবর্তিত থাকে।

কোথায় লেখা, সেটিই নির্ধারক

সবচেয়ে গুরুত্বপূর্ণ নিয়ম: একটি Function কোন Variable দেখতে পাবে, তা নির্ভর করে Function-টি Code-এর **কোথায় লেখা** তার ওপর; কোথা থেকে **Call করা হয়েছে** তার ওপর নয়। এই নিয়মের নাম **Lexical Scope**।

lexical-scope.js · JavaScript চলবে: Browser Console ও Node.js

```
const currency = 'টাকা';

function format(amount) {
  return `${amount} ${currency}`;
}

function report() {
  const currency = 'ডলার';
  return format(100);
}

console.log(report());
```

Output

```
100 টাকা
```

`format`-কে Call করা হয়েছে `report`-এর ভেতর থেকে, যেখানে `currency`-র মান `'ডলার'`। তবু ফল `টাকা`। কারণ `format` লেখা হয়েছে Global Scope-এ; তার Scope Chain-এ `report`-এর ভেতরের Variable নেই। `format` নিজের ভেতরে `currency` না পেয়ে সরাসরি Global-এ খুঁজেছে।

এর ব্যবহারিক অর্থ: একটি Function পড়েই বলা যায় সেটি কোন Variable ব্যবহার করবে; কে তাকে Call করছে তা জানার দরকার নেই।

৫.১০ Hoisting

JavaScript একটি File চালানোর আগে পুরোটা একবার পড়ে নেয় এবং কোন কোন নাম তৈরি হবে তা জেনে রাখে। এর ফলে কিছু নাম তাদের লেখার লাইনের আগেও পরিচিত থাকে। এই আচরণকে **Hoisting** বলে।

তিন ধরনের Declaration তিনভাবে আচরণ করে।

hoisting.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(square(4));

function square(number) {
  return number * number;
}

console.log(oldStyle);
var oldStyle = 'var দিয়ে তৈরি';
console.log(oldStyle);
```

Output

```
16
undefined
var দিয়ে তৈরি
```

- **Function Declaration** পুরোপুরি Hoist হয়: নাম এবং Body দুটোই। তাই `square` লেখার আগের লাইনেই Call করা গেছে।
- `var` আংশিক Hoist হয়: নামটি পরিচিত, কিন্তু মান তখনো `undefined`। Error আসে না, ভুল মান আসে—যা আরও বিপজ্জনক।
- `let` ও `const` দিয়ে তৈরি নাম (Function Expression ও Arrow Function-সহ) Declaration-এর লাইনের আগে ব্যবহার করা যায় না।

arrow-before.js · JavaScript চলবে: Node.js

```
console.log(cube(3));

const cube = (number) => number * number * number;
```

Error Message

```
ReferenceError: Cannot access 'cube' before initialization
```

Declaration	লেখার আগে ব্যবহার করলে
`function name() { }`	কাজ করে
`var name`	`undefined`
`let name`, `const name`	`ReferenceError`

Hoisting-এর ওপর নির্ভর করে Code লিখবেন না। Function Declaration-এর এই সুবিধা একটি কাজে লাগে: File-এর ওপরে মূল ধাপগুলো এবং নিচে সহায়ক Function-গুলো রাখা যায়, যাতে পাঠক আগে বড় ছবিটি দেখেন।

৫.১১ পুনর্ব্যবহারযোগ্য Logic

Function লেখা সহজ; ভালো Function লেখা একটি দক্ষতা। তিনটি নীতি বেশির ভাগ ক্ষেত্রে পথ দেখায়।

একটি Function, একটি কাজ

একটি Function যদি যাচাই করে, হিসাব করে এবং ফল দেখায়, তবে তার তিনটির যেকোনো একটি আলাদাভাবে ব্যবহার করা যায় না। কাজগুলো ভাগ করলে প্রতিটি অংশ আলাদাভাবে ব্যবহার ও পরীক্ষা করা যায়।

small-functions.js · JavaScript চলবে: Browser Console ও Node.js

```
function percentOf(amount, percent) {
  return Math.round(amount * percent / 100);
}

function applyDiscount(amount, percent) {
  return amount - percentOf(amount, percent);
}

function addTax(amount, percent) {
  return amount + percentOf(amount, percent);
}

const discounted = applyDiscount(1000, 10);
const payable = addTax(discounted, 5);

console.log(discounted);
console.log(payable);
```

Output

```
900
945
```

`percentOf` দুটি ভিন্ন Function ব্যবহার করছে। শতাংশ হিসাবের নিয়ম বদলালে (যেমন পূর্ণ টাকায় আনার পদ্ধতি) এক জায়গায় বদলালেই দুটোই ঠিক হয়ে যাবে।

Pure Function

যে Function একই Argument-এ সব সময় একই ফল দেয় এবং বাইরের কিছু বদলায় না, তাকে **Pure Function** বলে। ওপরের তিনটিই Pure। এর বিপরীতে, বাইরের Variable বদলানো, পর্দায় দেখানো বা File লেখাকে **Side Effect** বলে।

pure-vs-impure.js · JavaScript চলবে: Browser Console ও Node.js

```
let runningTotal = 0;

function addToTotal(amount) {
  runningTotal += amount;
  return runningTotal;
}

function add(a, b) {
  return a + b;
}

console.log(addToTotal(100));
console.log(addToTotal(100));
console.log(add(100, 100));
console.log(add(100, 100));
```

Output

```
100
200
200
200
```

`addToTotal(100)` দুবার Call করে দুটি ভিন্ন ফল পাওয়া গেছে, কারণ এটি বাইরের `runningTotal`-এর ওপর নির্ভর করে এবং সেটি বদলায়। এর ফল বুঝতে হলে আগে কতবার Call হয়েছে তা জানতে হয়। `add` শুধু নিজের Argument-এর ওপর নির্ভর করে; এটি বোঝা, পরীক্ষা করা এবং পুনর্ব্যবহার করা সহজ।

Side Effect ছাড়া Program অর্থহীন—কোনো এক সময় ফল দেখাতেই হবে। লক্ষ্য হলো হিসাবের অংশ Pure রাখা এবং Side Effect-গুলোকে অল্প কয়েকটি Function-এ সীমাবদ্ধ করা।

Function-কে Argument হিসেবে পাঠানো

Function একটি মান, তাই একটি Function আরেকটি Function গ্রহণ করতে পারে। এতে "কী করতে হবে" আর "কোন নিয়মে করতে হবে" আলাদা করা যায়।

function-as-argument.js · JavaScript চলবে: Browser Console ও Node.js

```
const tenPercent = (amount) => Math.round(amount * 10 / 100);
const flatFifty = (amount) => 50;

function finalPrice(amount, discountRule) {
  return amount - discountRule(amount);
}

console.log(finalPrice(1000, tenPercent));
console.log(finalPrice(1000, flatFifty));
```

Output

```
900
950
```

`finalPrice` জানে না ছাড়ের নিয়ম কী; সে শুধু পাওয়া Function-টি Call করে। নতুন নিয়ম যোগ করতে `finalPrice` বদলাতে হবে না। লক্ষ করুন, `tenPercent` পাঠানো হয়েছে বন্ধনী ছাড়া: Function-টিকে পাঠানো হচ্ছে, তার ফলাফলকে নয়। অন্য Function-কে পাঠানো এমন Function-কে **Callback** বলে। অধ্যায় ৬-এর `map`, `filter` এবং `reduce` সম্পূর্ণভাবে এই ধারণার ওপর দাঁড়িয়ে।

৫.১২ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`return` লিখতে ভুলে যাওয়া	Function `undefined` ফেরত দেয়	হিসাবের ফল `return` করুন
`return`-এর বদলে `console.log()`	পর্দায় দেখায়, কিন্তু মান ব্যবহার করা যায় না	দেখানো আর ফেরত দেওয়া আলাদা
`{ }`-সহ Arrow Function-এ `return` নেই	`undefined`	`return` যোগ করুন, অথবা `{ }` সরান
বন্ধনী ছাড়া Call (`getTotal`)	Function-টির Code দেখায়, ফল নয়	`getTotal()`
Argument-এর ক্রম ভুল	ভুল ফল, কোনো Error নয়	Parameter-এর ক্রমের সঙ্গে মিলিয়ে নিন
কম Argument দেওয়া	`NaN` বা `undefined`	সব Argument দিন, অথবা Default Parameter ব্যবহার করুন
`null`-এ Default বসবে ধরে নেওয়া	`null` থেকে যায়	Default শুধু `undefined`-এর জন্য
Function-এর ভেতরের Variable বাইরে ব্যবহার	`ReferenceError: ... is not defined`	মানটি `return` করুন

ভুল	কী দেখা যায়	সমাধান
Arrow Function তৈরির আগে Call	`ReferenceError: Cannot access ... before initialization`	আগে তৈরি করুন
Function-এর ভেতরে Global Variable বদলানো	একই Call-এ ভিন্ন ভিন্ন ফল	মান Argument হিসেবে নিন, ফল `return` করুন

তৃতীয় ভুলটি বিশেষভাবে ধরা কঠিন, কারণ Code দেখতে ঠিকই লাগে।

arrow-no-return.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wrongDouble = (number) => {
  number * 2;
};

const rightDouble = (number) => number * 2;

console.log(wrongDouble(21));
console.log(rightDouble(21));
```

Output

```
undefined
42
```

৫.১৩ Debugging নির্দেশিকা

Function-সংক্রান্ত অধিকাংশ Bug দুটি প্রশ্নে ধরা পড়ে: Function-টি কী পেয়েছে, এবং কী ফেরত দিয়েছে?

debug-function.js · JavaScript — চলবে: Browser Console ও Node.js

```
function applyDiscount(amount, percent) {
  console.log('পেয়েছে:', amount, percent, typeof amount);

  const result = amount - Math.round(amount * percent / 100);

  console.log('ফেরত দিচ্ছে:', result);
  return result;
}

applyDiscount('1000', 10);
```

Output

```
পেয়েছে: 1000 10 string
ফেরত দিচ্ছে: 900
```

`typeof` দেখাচ্ছে `amount` একটি String হিসেবে এসেছে। এখানে ফল ঠিক এসেছে, কারণ `*` ও `-` String-কে সংখ্যায় রূপান্তর করে; কিন্তু পরে কেউ `+` ব্যবহার করলে জোড়া লেগে যাবে। Function-এর শুরুতে Argument দেখা এমন লুকানো সমস্যা ধরিয়ে দেয়।

Function প্রত্যাশামতো কাজ না করলে:

১. Function-এর প্রথম লাইনে প্রতিটি Parameter-এর মান ও `typeof` দেখান।
২. `return`-এর ঠিক আগে ফলটি দেখান।
৩. ফল `undefined` হলে দেখুন `return` আছে কি না, এবং সব পথে আছে কি না।
৪. Function-টি আদৌ Call হচ্ছে কি না দেখুন; নাম লিখে বন্ধনী দিতে ভুলে যাননি তো?
৫. `ReferenceError` এলে নামটির বানান এবং Scope দেখুন: Variable-টি কোন `{ }`-এর ভেতরে তৈরি?
৬. Error Message-এর Stack Trace পড়ুন (অধ্যায় ১): কোন Function থেকে কোনটি Call হয়েছে।
৭. Function-টিকে একা, নির্দিষ্ট Argument দিয়ে Call করে দেখুন। Pure Function হলে এটি সহজ; সেটিই Pure Function-এর বড় সুবিধা।

৫.১৪ পেশাদার অভ্যাস

- **একটি Function, একটি কাজ।** নামে "এবং" বসাতে হলে (`validateAndSave`) সেটি সম্ভবত দুটি Function।
- **নাম ক্রিয়া দিয়ে শুরু করুন;** সত্য-মিথ্যা ফেরত দেওয়া Function-এর নাম `is` বা `has` দিয়ে: `isValid`, `hasStock`।
- **Function ছোট রাখুন।** এক পর্দায় না আঁটলে ভাগ করার কথা ভাবুন।
- **Parameter কম রাখুন।** তিন-চারটির বেশি হলে ক্রম মনে রাখা কঠিন; তখন একটি Object নিন।
- **হিসাব আর দেখানো আলাদা রাখুন।**
- **Argument হিসেবে পাওয়া Object বদলাবেন না** (অধ্যায় ২)। নতুন মান তৈরি করে ফেরত দিন।
- **Global Variable-এর ওপর নির্ভর করবেন না।** যা দরকার Argument হিসেবে নিন।
- **সব পথে একই ধরনের মান ফেরত দিন।** কখনো সংখ্যা, কখনো String ফেরত দিলে Call-কারীকে প্রতিবার যাচাই করতে হয়।
- **Function-এর ওপরে এক লাইনের Comment** লিখুন: এটি কী নেয় এবং কী ফেরত দেয়।

৫.১৫ Project: Invoice Calculator

একটি দোকানের Invoice তৈরির Program লিখবেন: প্রতিটি পণ্যের লাইন, উপমোট, ছাড়, কর এবং সর্বমোট। এখানে মূল লক্ষ্য হিসাব নয়, বরং কাজটিকে ছোট ছোট Function-এ ভাগ করা।

Project-এর চাহিদা

- প্রতিটি পণ্যের নাম, একটির দাম এবং পরিমাণ থাকবে।
- **টাকার অঙ্ক পয়সায়, পূর্ণসংখ্যায় রাখা হবে** (১২০.৫০ টাকা = ১২০৫০ পয়সা), যাতে দশমিক হিসাবের ত্রুটি না হয়। দেখানোর সময় টাকায় রূপান্তর করা হবে।
- ছাড় ও করের হার শতাংশে দেওয়া যাবে; না দিলে দুটোই শূন্য ধরা হবে।
- ছাড় হিসাব হবে উপমোটের ওপর; কর হিসাব হবে ছাড়ের পরের অঙ্কের ওপর। প্রতিটি নিকটতম পয়সায় আনা হবে।
- হিসাবের Function কিছু দেখাবে না; শুধু ফল ফেরত দেবে।

এখানে করের হারটি একটি উদাহরণ; এটি কোনো দেশের প্রকৃত কর-নিয়ম নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
invoice-calculator/
├── invoice.js
├── README.md
└── tests/
    └── invoice.test.js
```

invoice.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: তিনটি ছোট সহায়ক Function

invoice.js · JavaScript (অংশ ১ / ৩ (Part 1 of 3)) চলবে: Node.js

```
'use strict';

const PAISA_PER_TAKA = 100;

// পয়সাকে টাকার লিখিত রূপে আনে: 12050 থেকে '120.50 টাকা'।
const formatTaka = (paisa) => `${(paisa / PAISA_PER_TAKA).toFixed(2)} টাকা`;

// একটি পণ্যের লাইনের মোট: দাম x পরিমাণ (পয়সায়)।
const lineTotal = (item) => item.pricePaisa * item.quantity;

// একটি অঙ্কের নির্দিষ্ট শতাংশ, নিকটতম পয়সায়।
function percentOf(amount, percent) {
  return Math.round(amount * percent / 100);
}

// ...
```

তিনটিই Pure Function, এবং প্রতিটি একটি কাজ করে। প্রথম দুটি এক লাইনের হিসাব, তাই Arrow Function। `formatTaka`-ই একমাত্র জায়গা, যেখানে পয়সা টাকায় রূপান্তরিত হয়; বাকি সব হিসাব পূর্ণসংখ্যায় থাকে।

ধাপ ২: Invoice-এর হিসাব

invoice.js · JavaScript (অংশ ২ / ৩ (Part 2 of 3)) চলবে: Node.js

```
// ...

// পণ্যের তালিকা ও হার থেকে Invoice-এর চারটি অঙ্ক (পয়সায়) হিসাব করে।
function calculateInvoice(items, discountPercent = 0, taxPercent = 0) {
  let subtotal = 0;

  for (let i = 0; i < items.length; i++) {
    subtotal += lineTotal(items[i]);
  }

  const discount = percentOf(subtotal, discountPercent);
  const taxable = subtotal - discount;
  const tax = percentOf(taxable, taxPercent);

  return {
    subtotal: subtotal,
    discount: discount,
    tax: tax,
    total: taxable + tax,
  };
}

// ...
```

দুটি Default Parameter থাকায় `calculateInvoice(items)` লিখলেই ছাড় ও কর ছাড়া হিসাব পাওয়া যায়। Function-টি কিছু দেখায় না এবং `items` বদলায় না; চারটি অঙ্ক একটি Object-এ ফেরত দেয়।

ধাপ ৩: Invoice দেখানো

invoice.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// প্রতিটি লাইন এবং শেষে চারটি অঙ্ক দেখায়।
function printInvoice(items, discountPercent = 0, taxPercent = 0) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const label = `${item.name} x ${item.quantity}`;

    console.log(`${label}: ${formatTaka(lineTotal(item))}`);
  }

  const invoice = calculateInvoice(items, discountPercent, taxPercent);

  console.log(`উপমোট: ${formatTaka(invoice.subtotal)}`);
  console.log(`ছাড় (${discountPercent}%): ${formatTaka(invoice.discount)}`);
  console.log(`কর (${taxPercent}%): ${formatTaka(invoice.tax)}`);
  console.log(`সর্বমোট: ${formatTaka(invoice.total)}`);
}

const items = [
  { name: 'Notebook', pricePaisa: 12050, quantity: 3 },
  { name: 'Pen', pricePaisa: 1500, quantity: 10 },
  { name: 'School Bag', pricePaisa: 95000, quantity: 1 },
];

printInvoice(items, 10, 5);
console.log(formatTaka(calculateInvoice(items).total));
```

Output

```
Notebook x 3: 361.50 টাকা
Pen x 10: 150.00 টাকা
School Bag x 1: 950.00 টাকা
উপমোট: 1461.50 টাকা
ছাড় (10%): 146.15 টাকা
কর (5%): 65.77 টাকা
সর্বমোট: 1381.12 টাকা
1461.50 টাকা
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Function-এর স্তর।** `printInvoice` Call করে `calculateInvoice`-কে, সেটি Call করে `lineTotal` ও `percentOf`-কে। প্রতিটি স্তর নিচের স্তরের নাম ব্যবহার করে, বিস্তারিত না জেনেই। ওপরের Function পড়লে কী হচ্ছে বোঝা যায়; নিচেরগুলো পড়লে কীভাবে।
- **Side Effect এক জায়গায়।** পুরো File-এ `console.log()` আছে শুধু `printInvoice`-এ এবং শেষ লাইনে। বাকি চারটি Function Pure, তাই প্রতিটিকে আলাদাভাবে পরীক্ষা করা যায়।
- **Scope।** `subtotal`, `discount`, `taxable`, `tax` —সবই `calculateInvoice`-এর Function Scope-এ। `printInvoice` এগুলো সরাসরি দেখতে পায় না; পায় শুধু ফেরত আসা Object-টি। `taxable` ফেরত দেওয়া হয়নি, তাই বাইরে থেকে সেটি পাওয়ার কোনো উপায় নেই।
- **Lexical Scope।** `formatTaka` নিজের ভেতরে `PAISA_PER_TAKA` পায় না; এক ধাপ বাইরে Global Scope-এ পায়। এটি একটি `const`, তাই এই নির্ভরতা নিরাপদ।
- **Hoisting।** `formatTaka` ও `lineTotal` Arrow Function, তাই ব্যবহারের আগে লেখা হয়েছে। `printInvoice`-এর ভেতরে এগুলোর ব্যবহার সমস্যা নয়, কারণ `printInvoice` Call হয় File-এর একেবারে শেষে, যখন সবকিছু তৈরি।
- **হিসাব।** উপমোট ১,৪৬,১৫০ পয়সা। ১০ শতাংশ ছাড় ১৪,৬১৫। বাকি ১,৩১,৫৩৫-এর ৫ শতাংশ ৬,৫৭৬.৭৫, যা নিকটতম পয়সায় ৬,৫৭৭। সর্বমোট ১,৩৮,১১২ পয়সা, অর্থাৎ ১৩৮১.১২ টাকা।

শেষ লাইনে একই `calculateInvoice` শুধু `items` দিয়ে Call করা হয়েছে; Default Parameter-এর কারণে ছাড় ও কর শূন্য, এবং ফল উপমোটের সমান।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node invoice.js
```

Call	যা পাওয়ার কথা
`calculateInvoice(items, 10, 5)`	উপমোট 146150, ছাড় 14615, কর 6577, সর্বমোট 138112 (পয়সায়)
`calculateInvoice(items)`	ছাড় 0, কর 0, সর্বমোট 146150
`calculateInvoice([])`	চারটি অঙ্কই 0; কোনো Error নয়
`percentOf(131535, 5)`	6577 (6576.75 নিকটতম পয়সায়)
`formatTaka(12050)`	120.50 টাকা
`formatTaka(5)`	0.05 টাকা

Companion Source Code-এ এই Project-এর ১৪টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`ReferenceError: Cannot access 'formatTaka' before initialization`	Arrow Function-টি ব্যবহারের পরে লেখা	সহায়ক Function-গুলো File-এর ওপরে রাখুন
সর্বমোট `NaN`	`discountPercent` বা `taxPercent`-এর Default নেই এবং Argument দেওয়া হয়নি	`= 0` যোগ করুন
`invoice.subtotal` হলো `undefined`	`calculateInvoice`-এ `return` নেই, বা Object-এর নামে বানান ভুল	`return { ... }` এবং নামগুলো মিলিয়ে দেখুন
অঙ্ক একশ গুণ বড় বা ছোট	কোথাও টাকা আর পয়সা মিশে গেছে	সব হিসাব পয়সায়; টাকায় শুধু `formatTaka`
কর বেশি আসছে	কর `subtotal`-এর ওপর হিসাব হচ্ছে	`taxable`-এর ওপর হিসাব করুন
`formatTaka` দশমিকের পরে অনেক ঘর দেখায়	`toFixed(2)` নেই	`(paisa / 100).toFixed(2)`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। পণ্যের তথ্য যাচাই করা হয় না: দাম ও পরিমাণ সঠিক পূর্ণসংখ্যা বলে ধরে নেওয়া হয়েছে, এবং শতাংশ ০ থেকে ১০০-এর মধ্যে আছে কি না তা দেখা হয় না (অধ্যায় ৩ ও ৪-এর কৌশলে তা যোগ করা যায়; অনুশীলনী ১২ দেখুন)। করের হার উদাহরণমূলক। Invoice সংরক্ষণ বা ছাপানোর ব্যবস্থা নেই।

৫.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `square` নামে একটি Function Declaration লিখুন, যা একটি সংখ্যার বর্গ ফেরত দেয়। একই কাজ একটি Arrow Function দিয়েও লিখুন।

২. না চালিয়ে বলুন নিচের Code কী দেখাবে, এবং কেন।

predict-return.js · JavaScript চলবে: Browser Console ও Node.js

```
function sayHello(name) {
  console.log(`Hello, ${name}`);
}

const result = sayHello('Sara');
console.log(result);
```

৩. `fullName` নামে একটি Function লিখুন, যা নামের প্রথম ও শেষ অংশ নিয়ে মাঝখানে একটি ফাঁকা জায়গা দিয়ে জোড়া লাগিয়ে ফেরত দেয়।

৪. `power` নামে একটি Function লিখুন, যা একটি সংখ্যা ও ঘাত নেয়; ঘাত না দিলে `2` ধরা হবে।

৫. নিচের Code-এ একটি Scope-সংক্রান্ত ভুল আছে। ভুলটি ঠিক করুন, যাতে `450` দেখায়।

scope-bug.js · JavaScript চলবে: Node.js

```
function calculate(amount) {
  const discounted = amount - 50;
}

calculate(500);
console.log(discounted);
```

মধ্যম স্তর

৬. `isLeapYear` নামে একটি Arrow Function লিখুন, যা বছর অধিবর্ষ হলে `true` দেয়। নিয়ম: ৪ দিয়ে বিভাজ্য এবং ১০০ দিয়ে নয়; অথবা ৪০০ দিয়ে বিভাজ্য।

৭. `average` নামে একটি Function লিখুন, যা সংখ্যার একটি Array নিয়ে গড় ফেরত দেয়; ফাঁকা Array হলে `0`।

৮. না চালিয়ে বলুন নিচের Code কী দেখাবে। Lexical Scope দিয়ে উত্তরটি ব্যাখ্যা করুন।

predict-scope.js · JavaScript চলবে: Browser Console ও Node.js

```
const label = 'বাইরের';

function show() {
  return label;
}

function run() {
  const label = 'ভেতরের';
  return show();
}

console.log(run());
```

৯. `applyTwice` নামে একটি Function লিখুন, যা একটি সংখ্যা ও একটি Function নেয় এবং Function-টি সংখ্যার ওপর পরপর দুবার প্রয়োগ করে।

চ্যালেঞ্জ স্তর

১০. **Delivery যোগ:** `totalWithDelivery` Function লিখুন, যা Invoice-এর সর্বমোট (পয়সায়) এবং Delivery-র খরচ নেয়; খরচ না দিলে ৬০০০ পয়সা ধরা হবে; সর্বমোট ৩,০০,০০০ পয়সা বা বেশি হলে Delivery ফ্রি।

১১. **নিয়ম পাঠানো:** `priceAfter` Function লিখুন, যা একটি অঙ্ক এবং ছাড়ের নিয়ম (একটি Function) নেয়। তিনটি নিয়ম তৈরি করুন: কোনো ছাড় নয়, ১৫ শতাংশ, এবং সরাসরি ২০০০ পয়সা।

১২. **হার যাচাই:** `isValidPercent` Function লিখুন, যা মানটি ০ থেকে ১০০-এর মধ্যে সংখ্যা হলে `true` দেয়। তারপর `safePercentOf` লিখুন, যা হার সঠিক না হলে `null` ফেরত দেয়।

৫.১৭ অনুশীলনীর সমাধান

সমাধান ১

square.js · JavaScript চলবে: Browser Console ও Node.js

```
function square(number) {
  return number * number;
}

const squareArrow = (number) => number * number;

console.log(square(7));
console.log(squareArrow(7));
```

Output

```
49
49
```

সমাধান ২ — প্রথমে `Hello, Sara`, তারপর `undefined`। `sayHello` বার্তাটি দেখায়, কিন্তু কিছু `return` করে না; তাই `result`-এর মান `undefined`।

সমাধান ৩

full-name.js · JavaScript — চলবে: Browser Console ও Node.js

```
function fullName(first, last) {
  return `${first} ${last}`;
}

console.log(fullName('Sadia', 'Rahman'));
```

Output

```
Sadia Rahman
```

সমাধান ৪

power.js · JavaScript — চলবে: Browser Console ও Node.js

```
function power(base, exponent = 2) {
  return base ** exponent;
}

console.log(power(5));
console.log(power(2, 10));
```

Output

```
25
1024
```

সমাধান ৫ — `discounted` Function-এর ভেতরে তৈরি, তাই বাইরে থেকে পাওয়া যায় না। মানটি `return` করে বাইরে একটি Variable-এ নিতে হবে।

scope-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
function calculate(amount) {
  return amount - 50;
}

const discounted = calculate(500);
console.log(discounted);
```

Output

```
450
```

সমাধান ৬

leap-year.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isLeapYear = (year) =>
  (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0;

console.log(isLeapYear(2024));
console.log(isLeapYear(1900));
console.log(isLeapYear(2000));
console.log(isLeapYear(2025));
```

Output

```
true
false
true
false
```

`&&` ও `||` একসঙ্গে থাকায় বন্ধনী দেওয়া হয়েছে, যদিও `&&` এমনিতেই আগে হিসাব হয়।

সমাধান ৭

average.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function average(numbers) {
  if (numbers.length === 0) {
    return 0;
  }

  let sum = 0;

  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
  }

  return sum / numbers.length;
}

console.log(average([80, 90, 70]));
console.log(average([]));
```

Output

```
80
0
```

সমাধান ৮ — বাইরের। `show` লেখা হয়েছে Global Scope-এ, তাই সে `label` খোঁজে নিজের ভেতরে এবং তারপর Global-এ। `run`-এর ভেতরের `label` তার Scope Chain-এ নেই, যদিও Call করা হয়েছে সেখান থেকেই।

সমাধান ৯

apply-twice.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function applyTwice(value, operation) {
  return operation(operation(value));
}

const addTen = (number) => number + 10;
const double = (number) => number * 2;

console.log(applyTwice(5, addTen));
console.log(applyTwice(5, double));
```

Output

```
25
20
```

সমাধান ১০

delivery.js · JavaScript — চলবে: Browser Console ও Node.js

```
const FREE_DELIVERY_FROM = 300000;

function totalWithDelivery(total, deliveryFee = 6000) {
  return total >= FREE_DELIVERY_FROM ? total : total + deliveryFee;
}

console.log(totalWithDelivery(138112));
console.log(totalWithDelivery(138112, 12000));
console.log(totalWithDelivery(300000));
```

Output

```
144112
150112
300000
```

সমাধান ১১

price-after.js · JavaScript — চলবে: Browser Console ও Node.js

```
const noDiscount = (amount) => 0;
const fifteenPercent = (amount) => Math.round(amount * 15 / 100);
const flat2000 = (amount) => 2000;

function priceAfter(amount, discountRule) {
  return amount - discountRule(amount);
}

console.log(priceAfter(50000, noDiscount));
console.log(priceAfter(50000, fifteenPercent));
console.log(priceAfter(50000, flat2000));
```

Output

```
50000
42500
48000
```

সমাধান ৫২

safe-percent.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isValidPercent = (value) =>
  typeof value === 'number' && value >= 0 && value <= 100;

function safePercentOf(amount, percent) {
  if (!isValidPercent(percent)) {
    return null;
  }

  return Math.round(amount * percent / 100);
}

console.log(safePercentOf(1000, 10));
console.log(safePercentOf(1000, 150));
console.log(safePercentOf(1000, '10'));
```

Output

```
100
null
null
```

`NaN`-এর `typeof` হলো `number`, কিন্তু `NaN >= 0` মিথ্যা, তাই `NaN`-ও বাতিল হয়।

৫.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `function add(a, b) { return a + b; }`-এ `a` এবং `b` কী?

ক) Argument

খ) Parameter

গ) Return Value

ঘ) Global Variable

প্রশ্ন ২. যে Function-এ `return` নেই, সেটি কী ফেরত দেয়?

ক) `null`

খ) `0`

গ) `undefined`

ঘ) একটি Error

প্রশ্ন ৩. `function greet(name = 'অতিথি')`-কে `greet(null)` দিয়ে Call করলে `name`-এর মান কী হয়?

ক) `null`

খ) `'অতিথি'`

গ) `undefined`

ঘ) ফাঁকা String

প্রশ্ন ৪. `const double = (n) => { n * 2; };` লিখে `double(5)` Call করলে ফল কী?

ক) `10`

খ) `5`

গ) একটি Error

ঘ) `undefined`

প্রশ্ন ৫. `function add(a, b) { return a + b; }` থাকলে `add(2)`-এর ফল কী?

ক) `2`

খ) `NaN`

গ) একটি Error

ঘ) `undefined`

প্রশ্ন ৬. একটি Function-এর ভেতরে `const` দিয়ে তৈরি Variable কোথা থেকে পাওয়া যায়?

ক) শুধু সেই Function-এর ভেতর থেকে

খ) File-এর সব জায়গা থেকে

গ) যে Function তাকে Call করেছে সেখান থেকেও

ঘ) শুধু Global Scope থেকে

প্রশ্ন ৭. Lexical Scope অনুযায়ী, একটি Function কোন Variable দেখতে পাবে তা কিসের ওপর নির্ভর করে?

ক) Function-টি কোথা থেকে Call করা হয়েছে

খ) Function-টি কতবার Call করা হয়েছে

গ) Function-টি Code-এর কোথায় লেখা হয়েছে

ঘ) Function-টির নাম

প্রশ্ন ৮. কোনটি তার লেখার লাইনের আগে Call করা যায়?

ক) `const f = () => 1;`

খ) `const f = function () { return 1; };`

গ) `let f = () => 1;`

ঘ) `function f() { return 1; }`

প্রশ্ন ৯. Pure Function-এর বৈশিষ্ট্য কোনটি?

ক) একই Argument-এ একই ফল দেয় এবং বাইরের কিছু বদলায় না

খ) সব সময় `console.log()` ব্যবহার করে

গ) কোনো Parameter নেয় না

ঘ) Global Variable বদলায়

প্রশ্ন ১০. `finalPrice(1000, tenPercent)`-এ `tenPercent` বন্ধনী ছাড়া লেখা হয়েছে। এর অর্থ কী?

ক) এটি একটি ভুল; বন্ধনী দিতে হবে

খ) Function-টি নিজেই পাঠানো হচ্ছে, তার ফলাফল নয়

গ) Function-টি সঙ্গে সঙ্গে চলে যায়

ঘ) `tenPercent` একটি String

৫.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Function তৈরির সময় লেখা নামগুলো Parameter। Call-এর সময় দেওয়া মানগুলো Argument।
২	গ	`return` না থাকলে, বা মান ছাড়া `return` থাকলে, ফল `undefined`।
৩	ক	Default শুধু `undefined`-এর জন্য বসে। `null` একটি ইচ্ছাকৃত মান।
৪	ঘ	`{ }` থাকলে `return` নিজে লিখতে হয়। এখানে হিসাব হয়, কিন্তু ফেরত যায় না।
৫	খ	`b` হয় `undefined`; `2 + undefined` এর ফল `NaN`। কোনো Error হয় না।
৬	ক	এটি Function Scope। বাইরে থেকে পেতে হলে মানটি `return` করতে হয়।
৭	গ	Scope ঠিক হয় লেখার জায়গা থেকে; Call-এর জায়গা কোনো প্রভাব ফেলে না।
৮	ঘ	শুধু Function Declaration পুরোপুরি Hoist হয়। বাকিগুলো `let` বা `const`-এর নিয়ম মানে।
৯	ক	Pure Function শুধু নিজের Argument-এর ওপর নির্ভর করে, তাই বোঝা ও পরীক্ষা করা সহজ।
১০	খ	বন্ধনী দিলে Function তখনই চলত। বন্ধনী ছাড়া নামটি Function-টিকেই বোঝায়।

৫.২০ অধ্যায়ের সারসংক্ষেপ

- Function হলো নামযুক্ত, পুনর্ব্যবহারযোগ্য Code; নাম দেয়, পুনরাবৃত্তি কমায় এবং পরিবর্তন এক জায়গায় আনে।
- Parameter হলো Function-এর নিজস্ব নাম; Argument হলো Call-এর সময় দেওয়া মান। সংখ্যা না মিললে Error হয় না।
- `return` মান ফেরত দেয় এবং Function থামায়। `console.log()` শুধু দেখায়।
- Default Parameter শুধু `undefined`-এর ক্ষেত্রে বসে।
- Function একটি মান: Variable-এ রাখা যায় (Function Expression), সংক্ষেপে লেখা যায় (Arrow Function), অন্য Function-কে পাঠানো যায় (Callback)।
- Arrow Function-এ `{ }` না থাকলে মান নিজে ফেরত যায়; থাকলে `return` লিখতে হয়।
- Scope তিন স্তরের: Global, Function, Block। ভেতর থেকে বাইরে দেখা যায়, বাইরে থেকে ভেতরে নয়।
- Lexical Scope: Function কী দেখবে তা ঠিক হয় কোথায় লেখা তা থেকে।
- Hoisting: Function Declaration আগে Call করা যায়; `var` তখন `undefined`; `let` ও `const` Error দেয়।
- ভালো Function ছোট, একটি কাজ করে, এবং যতটা সম্ভব Pure।

পরের অধ্যায়ে Array এবং Data প্রক্রিয়াকরণ: তালিকা তৈরি, বদলানো, এবং `map`, `filter`, `reduce` দিয়ে রূপান্তর—যেখানে এই অধ্যায়ের Callback সরাসরি কাজে লাগবে।

৫.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Function Declaration	`function name() { }` দিয়ে তৈরি Function।
Function Expression	Variable-এ রাখা Function।
Arrow Function	`=>` দিয়ে লেখা সংক্ষিপ্ত Function।
Call	Function চালানো।
Parameter / Argument	Function-এর গ্রহণের নাম / Call-এ দেওয়া মান।
Return Value	Function যে মান ফেরত দেয়।
Default Parameter	Argument না দিলে যে মান ব্যবহৃত হয়।
Scope	যে অংশ থেকে একটি Variable ব্যবহার করা যায়।
Scope Chain	নাম খোঁজার পথ: ভেতর থেকে বাইরের দিকে।
Shadowing	ভেতরের একই নামের Variable বাইরেরটিকে ঢেকে দেওয়া।
Lexical Scope	Scope ঠিক হয় Code কোথায় লেখা তা থেকে।

পরিভাষা	অর্থ
Hoisting	কিছু Declaration-এর, লেখার লাইনের আগেই পরিচিত থাকা।
Pure Function	একই Argument-এ একই ফল; বাইরের কিছু বদলায় না।
Side Effect	বাইরের অবস্থা বদলানো বা বাইরে কিছু দেখানো।
Callback	অন্য Function-কে Argument হিসেবে পাঠানো Function।

৫.২২ আরও পড়ুন

- **MDN — Functions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions` — Function-এর পূর্ণ নির্দেশিকা।
- **MDN — Arrow function expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions` — Arrow Function-এর নিয়ম ও সীমা।
- **MDN — Default parameters:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters` — Default Parameter।
- **MDN — Scope:** `https://developer.mozilla.org/en-US/docs/Glossary/Scope` — Scope-এর সংজ্ঞা।
- **MDN — Hoisting:** `https://developer.mozilla.org/en-US/docs/Glossary/Hoisting` — Hoisting-এর ব্যাখ্যা।

অধ্যায় ৬

Arrays এবং Data Processing

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Array তৈরি করতে এবং Index দিয়ে Element পড়তে ও বদলাতে।
- `push`, `pop`, `shift` ও `unshift` দিয়ে Element যোগ ও বাদ দিতে।
- `slice` এবং `splice`-এর পার্থক্য ব্যাখ্যা করতে।
- `for...of` দিয়ে Array-র প্রতিটি মান পড়তে।
- `map`, `filter` ও `find` দিয়ে Data রূপান্তর, বাছাই ও খোঁজার কাজ করতে।
- `reduce` দিয়ে একটি তালিকা থেকে একটি সারমান বের করতে।
- কোন Method মূল Array বদলায় এবং কোনটি নতুন Array দেয়, তা চিনতে।
- একাধিক Method জুড়ে বাস্তব Data রূপান্তর করতে।
- একটি সম্পূর্ণ **Inventory Dashboard** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫ সম্পন্ন করা: Arrow Function, Callback এবং Pure Function।
- অধ্যায় ৪-এর `for` Loop এবং Array-র প্রাথমিক পরিচয় (Index, `length`)।
- অধ্যায় ২-এর Reference: Object ও Array-তে ঠিকানা Copy হয়, মান নয়।
- `chapter-06` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

প্রায় সব Program-এর কেন্দ্রে থাকে তালিকা: পণ্যের তালিকা, শিক্ষার্থীর তালিকা, বার্তার তালিকা, লেনদেনের তালিকা। অধ্যায় ৪-এ আপনি Loop দিয়ে তালিকা ঘুরে যোগফল, গণনা ও সর্বোচ্চ বের করেছেন। সেটি কাজ করে, কিন্তু প্রতিবার Counter, সীমা এবং Accumulator নিজে সামলাতে হয়, আর সেখানেই Off-by-One Error ঢোকে।

JavaScript-এর Array এই সাধারণ কাজগুলোর জন্য তৈরি Method দেয়। "প্রতিটি দামে কর যোগ করো", "যেগুলোর মজুত আছে শুধু সেগুলো রাখো", "সব মিলিয়ে মোট বের করো"—প্রতিটির জন্য একটি করে Method আছে, এবং Code পড়লেই উদ্দেশ্য বোঝা যায়।

এই অধ্যায়ের দ্বিতীয় বড় বিষয় **Mutation**: কোন কাজ মূল তালিকাটিকে বদলে ফেলে, আর কোনটি নতুন তালিকা তৈরি করে। এই পার্থক্য না জানলে একটি তালিকা "অজান্তে" বদলে যাওয়ার Bug বারবার দেখা দেয়।

৬.১ Array কেন প্রয়োজন

তিনটি পণ্যের দাম রাখতে তিনটি Variable লাগে: `price1`, `price2`, `price3`। তিনশটি হলে? আর সংখ্যাটি যদি আগে থেকে জানা না থাকে?

Array একটি Variable-এ ক্রমানুসারে যেকোনো সংখ্যক মান রাখে। তালিকা বড়-ছোট হতে পারে, এবং পুরো তালিকাটি একটি নাম দিয়ে Function-এ পাঠানো যায়।

why-arrays.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950];

console.log(prices);
console.log(prices.length);
console.log(typeof prices);
console.log(Array.isArray(prices));
```

Output

```
[ 120, 15, 950 ]
3
object
true
```

Array আসলে একটি বিশেষ ধরনের Object, তাই `typeof` দেখায় `object`। কোনো মান Array কি না জানতে `Array.isArray()` ব্যবহার করুন।

> **নোট:** `console.log()` একটি Array-কে কীভাবে দেখাবে, তা পরিবেশভেদে সামান্য আলাদা। এই অধ্যায়ের Output Node.js-এর; Browser Console একই তথ্য একটু ভিন্ন সাজে দেখায়। তাই Array সরাসরি দেখানো উদাহরণগুলো Node.js-এ চালান।

৬.২ Array তৈরি করা

সবচেয়ে প্রচলিত উপায় **Array Literal**: তৃতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করা মান।

creating.js · JavaScript — চলবে: Node.js

```
const empty = [];
const fruits = ['আম', 'কলা', 'লিচু'];
const mixed = [42, 'লেখা', true, null];
const product = [{ name: 'Pen', price: 15 }];

console.log(empty);
console.log(fruits);
console.log(mixed);
console.log(product[0].name);
```

Output

```
[]
[ 'আম', 'কলা', 'লিচু' ]
[ 42, 'লেখা', true, null ]
Pen
```

- Array ফাঁকা হতে পারে; পরে তাতে Element যোগ করা যায়।
- একই Array-তে ভিন্ন Type-এর মান রাখা সম্ভব, কিন্তু উচিত নয়। একটি Array-তে একই ধরনের জিনিস রাখুন: সব দাম, অথবা সব পণ্য।
- Element একটি Object-ও হতে পারে। `product[0]` Object-টি দেয়, তারপর `.name` তার তথ্য পড়ে।

Array-র নাম বহুবচনে রাখুন (`prices`, `students`), আর একটি Element-এর নাম একবচনে (`price`, `student`)। তাহলে Code পড়েই বোঝা যায় কোনটি তালিকা আর কোনটি একটি মান।

৬.৩ Index দিয়ে পড়া ও বদলানো

অধ্যায় ৪ থেকে মনে করুন: Index শূন্য থেকে শুরু, এবং শেষ Index হলো `length - 1`।

indexing.js · JavaScript — চলবে: Node.js

```
const fruits = ['আম', 'কলা', 'লিচু'];

console.log(fruits[0]);
console.log(fruits[fruits.length - 1]);
console.log(fruits.at(-1));
console.log(fruits[5]);

fruits[1] = 'কাঁঠাল';
console.log(fruits);

console.log(fruits.includes('লিচু'));
console.log(fruits.indexOf('লিচু'));
console.log(fruits.indexOf('জাম'));
```

Output

```
আম
লিচু
লিচু
undefined
[ 'আম', 'কাঁঠাল', 'লিচু' ]
true
2
-1
```

- **শেষ Element** পাওয়ার দুটি উপায়: `fruits[fruits.length - 1]`, অথবা সংক্ষেপে `fruits.at(-1)`। `at()`-এ ঋণাত্মক সংখ্যা শেষ থেকে গোনে।
- **সীমার বাইরের Index** Error দেয় না; `undefined` দেয়।
- **Index-এ Assignment** সেই জায়গার মান বদলে দেয়। `fruits` একটি `const`, তবু এটি সম্ভব, কারণ `const` শুধু নামটিকে একই Array-র সঙ্গে বেঁধে রাখে; Array-র ভেতরের মান বদলানো আটকায় না (অধ্যায় ২)।
- **`includes()`** জানায় মানটি আছে কি না। **`indexOf()`** তার Index দেয়, আর না পেলে `-1`।

> **সতর্কতা:** `length` -এর চেয়ে বড় Index-এ মান রাখবেন না (যেমন তিন Element-এর Array-তে `fruits[10] = 'জাম'`)। JavaScript মাঝের জায়গাগুলো ফাঁকা রেখে Array-টিকে এগারো Element-এর বানিয়ে দেবে। শেষে যোগ করতে `push()` ব্যবহার করুন।

৬.৪ Element যোগ ও বাদ: push, pop, shift, unshift

Array-র দুই প্রান্তে Element যোগ ও বাদ দেওয়ার চারটি Method আছে।

Method	কাজ	কোন প্রান্তে	ফেরত দেয়
`push(value)`	যোগ করে	শেষে	নতুন `length`
`pop()`	বাদ দেয়	শেষ থেকে	বাদ পড়া Element
`unshift(value)`	যোগ করে	শুরুতে	নতুন `length`
`shift()`	বাদ দেয়	শুরু থেকে	বাদ পড়া Element

push-pop.js · JavaScript — চলবে: Node.js

```
const queue = ['Sara', 'Rafi'];

const newLength = queue.push('Mina');
console.log(newLength);
console.log(queue);

const last = queue.pop();
console.log(last);

queue.unshift('Nila');
console.log(queue);

const first = queue.shift();
console.log(first);
console.log(queue);
```

Output

```
3
[ 'Sara', 'Rafi', 'Mina' ]
Mina
[ 'Nila', 'Sara', 'Rafi' ]
Nila
[ 'Sara', 'Rafi' ]
```

চারটিই **মূল Array-টিকে বদলে ফেলে**। `queue.push('Mina')`-এর পর `queue` আর আগের মতো নেই। এ ধরনের পরিবর্তনকে **Mutation** বলে, এবং বিষয়টি অনুচ্ছেদ ৬.১১-এ ফিরে আসবে।

ফেরত আসা মানটিও লক্ষ করুন: `push()` নতুন Array ফেরত দেয় না, দেয় নতুন `length`। তাই `const result = queue.push('X')` লিখে `result`-কে Array ভাবা একটি প্রচলিত ভুল।

`push` ও `pop` দ্রুত, কারণ শেষ প্রান্তে কাজ করলে বাকি Element-গুলো নিজের জায়গায় থাকে। `shift` ও `unshift`-এ প্রতিটি Element-এর Index এক ঘর সরে, তাই বড় Array-তে এগুলো তুলনামূলক ধীর।

৬.৫ slice এবং splice

নাম প্রায় এক, কাজ সম্পূর্ণ আলাদা। এই দুটির পার্থক্যই এই অধ্যায়ের মূল ধারণার সবচেয়ে পরিষ্কার উদাহরণ।

slice: একটি অংশের Copy

`slice(start, end)` মূল Array না বদলে তার একটি অংশ নতুন Array হিসেবে ফেরত দেয়। `start` Index থেকে শুরু, `end` Index-এর **আগে** শেষ।

slice.js · JavaScript — চলবে: Node.js

```
const letters = ['a', 'b', 'c', 'd', 'e'];

console.log(letters.slice(1, 3));
console.log(letters.slice(2));
console.log(letters.slice(-2));
console.log(letters.slice());
console.log(letters);
```

Output

```
[ 'b', 'c' ]
[ 'c', 'd', 'e' ]
[ 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e' ]
```

- `slice(1, 3)` — Index ১ ও ২; Index ৩ অন্তর্ভুক্ত নয়।
- `slice(2)` — Index ২ থেকে শেষ পর্যন্ত।
- `slice(-2)` — শেষ দুটি।
- `slice()` — পুরো Array-র একটি Copy।

শেষ লাইন দেখাচ্ছে `letters` অপরিবর্তিত।

splice: মূল Array কেটে বদলানো

`splice(start, deleteCount, ...items)` মূল Array-র `start` Index থেকে `deleteCount` টি Element সরিয়ে দেয়, এবং চাইলে সেই জায়গায় নতুন Element বসায়। এটি সরানো Element-গুলো ফেরত দেয়।

splice.js · JavaScript — চলবে: Node.js

```
const letters = ['a', 'b', 'c', 'd', 'e'];

const removed = letters.splice(1, 2);
console.log(removed);
console.log(letters);

letters.splice(1, 0, 'x');
console.log(letters);

letters.splice(0, 1, 'z');
console.log(letters);
```

Output

```
[ 'b', 'c' ]
[ 'a', 'd', 'e' ]
[ 'a', 'x', 'd', 'e' ]
[ 'z', 'x', 'd', 'e' ]
```

- `splice(1, 2)` — Index ১ থেকে দুটি সরানো হলো।
- `splice(1, 0, 'x')` — কিছু সরানো হয়নি (`0`); Index ১-এ `'x'` ঢোকানো হলো।
- `splice(0, 1, 'z')` — একটি সরিয়ে তার জায়গায় `'z'` বসানো হলো।

	slice	splice
মূল Array বদলায়	না	হ্যাঁ
দ্বিতীয় Argument	শেষ Index (অন্তর্ভুক্ত নয়)	কয়টি সরাতে হবে
ফেরত দেয়	অংশটির নতুন Array	সরানো Element-গুলো
ব্যবহার	অংশ পড়া, Copy করা	মাঝখান থেকে সরানো, ঢোকানো, বদলানো

মনে রাখার সহজ উপায়: `splice`-এ একটি অতিরিক্ত `p` আছে, এবং এটিই Array-টিকে **p**রিবর্তন করে।

৬.৬ for...of

Index-এর প্রয়োজন না থাকলে Array-র প্রতিটি মান পড়ার সবচেয়ে পরিষ্কার উপায় `for...of`।

for-of.js · JavaScript চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (const name of names) {
  console.log(name);
}
```

Output

```
Sara
Rafi
Mina
```

প্রতিটি Iteration-এ `name` -এ পরের Element-টি আসে। Counter নেই, সীমা নেই, `names[i]` নেই; তাই Off-by-One Error-এর সুযোগও নেই। `break` ও `continue` এখানে স্বাভাবিকভাবেই কাজ করে।

for-of-break.js · JavaScript চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280];
let firstExpensive = null;

for (const price of prices) {
  if (price > 500) {
    firstExpensive = price;
    break;
  }
}

console.log(firstExpensive);
```

Output

```
950
```

কখন	কোন Loop
শুধু মান দরকার	`for...of`
Index-ও দরকার, বা নির্দিষ্ট পরিসর	`for (let i = 0; ...)`
রূপান্তর, বাছাই বা সারমান	`map`, `filter`, `reduce` (নিচে)

> **সতর্কতা:** `for...in` নামে প্রায় একই চেহারার আরেকটি Loop আছে। সেটি মান নয়, Index (String হিসেবে) দেয়, এবং Object-এর জন্য তৈরি। Array-তে `for...in` ব্যবহার করবেন না।

৬.৭ map

`map` একটি Array-র প্রতিটি Element-কে রূপান্তর করে একই দৈর্ঘ্যের একটি **নতুন Array** তৈরি করে। আপনি একটি Callback দেন (অধ্যায় ৫), যা একটি Element নিয়ে তার নতুন রূপ ফেরত দেয়; `map` সেটি প্রতিটি Element-এর ওপর চালায়।

map.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950];

const withTax = prices.map((price) => price + Math.round(price * 15 / 100));
const labels = prices.map((price) => `${price} টাকা`);

console.log(withTax);
console.log(labels);
console.log(prices);
```

Output

```
[ 138, 17, 1093 ]
[ '120 টাকা', '15 টাকা', '950 টাকা' ]
[ 120, 15, 950 ]
```

একই কাজ `for` Loop দিয়ে করতে একটি ফাঁকা Array তৈরি করে প্রতিটি ফল `push` করতে হতো। `map` সেই ছাঁচটি নিজে সামলায়; আপনাকে শুধু বলতে হয় একটি Element-এর কী হবে।

Object-এর তালিকা থেকে একটি তথ্য বের করে আনা `map`-এর সবচেয়ে প্রচলিত ব্যবহার।

map-objects.js · JavaScript — চলবে: Node.js

```
const products = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
];

const names = products.map((product) => product.name);

console.log(names);
console.log(names.join(', '));
```

Output

```
[ 'Notebook', 'Pen' ]
Notebook, Pen
```

`join()` একটি Array-র Element-গুলোকে মাঝখানে একটি String বসিয়ে জোড়া লাগায়। অধ্যায় ৪-এ নামের তালিকা বানাতে যে Ternary লিখতে হয়েছিল, `join(', ')` সেটি এক ধাপে করে।

`map`-এর দুটি নিয়ম:

- Callback-কে অবশ্যই একটি মান ফেরত দিতে হবে। `{ }` ব্যবহার করে `return` ভুলে গেলে ফলের Array `undefined`-এ ভরে যায়।
- ফলের দৈর্ঘ্য সব সময় মূলের সমান। কিছু বাদ দিতে হলে সেটি `filter`-এর কাজ।

৬.৮ filter

`filter` প্রতিটি Element-এর ওপর একটি শর্ত পরীক্ষা করে এবং যেগুলোতে শর্ত সত্য, শুধু সেগুলো নিয়ে একটি **নতুন Array** তৈরি করে। Callback-টি `true` বা `false` ফেরত দেয়।

filter.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950, 280];

console.log(prices.filter((price) => price >= 200));
console.log(prices.filter((price) => price > 5000));

const products = [
  { name: 'Notebook', stock: 40 },
  { name: 'Pen', stock: 0 },
  { name: 'Bag', stock: 5 },
];

const available = products.filter((product) => product.stock > 0);

console.log(available.length);
console.log(available.map((product) => product.name));
```

Output

```
[ 950, 280 ]
[]
2
[ 'Notebook', 'Bag' ]
```

- ফলের Array মূলের চেয়ে ছোট বা সমান হয়; Element-গুলো অপরিবর্তিত থাকে এবং ক্রম বজায় থাকে।
- কিছু না মিললে ফল একটি ফাঁকা Array, `undefined` বা `null` নয়। তাই ফলের ওপর নিশ্চিন্তে `length` বা `map` ব্যবহার করা যায়।

`map` এবং `filter`-এর পার্থক্য এক বাক্যে: `map` প্রতিটি Element **বদলায়**, সংখ্যা একই থাকে; `filter` কিছু Element **বেছে নেয়**, Element বদলায় না।

৬.৯ find

`find` শর্ত মেলা **প্রথম Element-টি** ফেরত দেয় এবং সেখানেই থেমে যায়। কিছু না মিললে `undefined`।

find.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { id: 'P1', name: 'Notebook', price: 120 },
  { id: 'P2', name: 'Pen', price: 15 },
  { id: 'P3', name: 'Bag', price: 950 },
];

const pen = products.find((product) => product.id === 'P2');
const missing = products.find((product) => product.id === 'P9');

console.log(pen.name);
console.log(missing);
console.log(products.findIndex((product) => product.id === 'P3'));
```

Output

```
Pen
undefined
2
```

	filter	find
ফেরত দেয়	মেলা সব Element-এর Array	মেলা প্রথম Element-টি নিজে
কিছু না মিললে	`[]`	`undefined`
ব্যবহার	"যেগুলো..."	"যেটি..." (যেমন id দিয়ে একটি খোঁজা)

`find`-এর ফল ব্যবহারের আগে সেটি `undefined` কি না যাচাই করুন। ওপরে `missing.name` লিখলে Program `TypeError` দিয়ে থেমে যেত। `findIndex()` একই কাজ করে, কিন্তু Element-এর বদলে তার Index দেয়, আর না পেলে `-1`।

৬.১০ reduce

`map` ও `filter` একটি Array থেকে আরেকটি Array তৈরি করে। `reduce` পুরো Array-কে গুটিয়ে **একটি মান** তৈরি করে: যোগফল, সর্বোচ্চ, অথবা অন্য কোনো সারমান।

এটি অধ্যায় ৪-এর Accumulator ধরনটিরই Method-রূপ। Callback দুটি মান নেয়: এ পর্যন্ত জমা ফল (Accumulator) এবং বর্তমান Element; আর ফেরত দেয় নতুন জমা ফল। `reduce`-এর দ্বিতীয় Argument হলো Accumulator-এর শুরুর মান।

reduce.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];

const total = prices.reduce((sum, price) => sum + price, 0);

console.log(total);
```

Output

```
1085
```

ধাপ	sum (জমা ফল)	price (বর্তমান)	ফেরত দেয়
শুরু	0	—	—
১	0	120	120
২	120	15	135
৩	135	950	1085

প্রতিটি ধাপে Callback যা ফেরত দেয়, পরের ধাপে সেটিই `sum` হয়ে আসে। শেষ ধাপের ফলটি `reduce`-এর ফল।

Object-এর তালিকায় এটি বিশেষ কাজে লাগে।

reduce-objects.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Bag', price: 950, stock: 5 },
];

const stockValue = products.reduce(
  (sum, product) => sum + product.price * product.stock,
  0,
);

console.log(stockValue);
console.log([].reduce((sum, value) => sum + value, 0));
```

Output

```
9550
0
```

> **মনে রাখুন:** `reduce` -এ শুরুর মান (এখানে `0`) সব সময় দিন। না দিলে প্রথম Element-টিই শুরুর মান হয়ে যায়; Object-এর তালিকায় তাতে ভুল ফল আসে, এবং ফাঁকা Array-তে Program `TypeError` দিয়ে থেমে যায়। শুরুর মান দিলে ফাঁকা Array-তে সেই মানটিই ফেরত আসে, যেমন শেষ লাইনে।

`reduce` শক্তিশালী, কিন্তু সহজে দুর্বোধ্য হয়ে যায়। যোগফল বা একটি সারমানের জন্য এটি ব্যবহার করুন। যে কাজ `map` বা `filter` দিয়ে হয়, তা `reduce` দিয়ে করবেন না।

৬.১১ Mutation বনাম নতুন Array

কোনো কাজ মূল Array-টিকে বদলে ফেললে সেটি **Mutating**; মূলটি অক্ষত রেখে নতুন Array ফেরত দিলে **Non-mutating**।

মূল Array বদলায় (Mutating)	নতুন মান দেয় (Non-mutating)
`push`, `pop`, `shift`, `unshift`	`slice`, `map`, `filter`
`splice`	`find`, `reduce`, `includes`, `indexOf`, `join`
`sort`, `reverse`	`[...array]` (Spread)
Index-এ Assignment (`arr[0] = x`)	`at`

এটি কেন গুরুত্বপূর্ণ? কারণ Array একটি Reference (অধ্যায় ২)। একটি Function-এ Array পাঠালে Function-টি সেই একই Array-র ঠিকানা পায়।

mutation-bug.js · JavaScript চলবে: Node.js

```
function addBonusBad(marks) {
  marks.push(5);
  return marks;
}

const original = [70, 80];
const updated = addBonusBad(original);

console.log(original);
console.log(original === updated);

function addBonusGood(marks) {
  return [...marks, 5];
```

```
}

const safe = [70, 80];
const result = addBonusGood(safe);

console.log(safe);
console.log(result);
```

Output

```
[ 70, 80, 5 ]
true
[ 70, 80 ]
[ 70, 80, 5 ]
```

`addBonusBad` যে তালিকাটি পেয়েছিল, সেটিই বদলে দিয়েছে। যে Code `original` তৈরি করেছিল, সে জানেই না তার তালিকায় একটি নতুন মান ঢুকে গেছে। `addBonusGood` একটি নতুন Array তৈরি করে ফেরত দেয়; মূলটি অক্ষত।

`[...marks]` হলো **Spread**: এটি একটি Array-র Element-গুলোকে নতুন Array-তে ছড়িয়ে দেয়। `[...marks, 5]` মানে "আগের সব Element, তারপর `5`"।

Mutation ছাড়া তিনটি সাধারণ কাজ

কাজ	Mutating উপায়	Non-mutating উপায়
শেষে যোগ	`items.push(x)`	`[...items, x]`
একটি বাদ	`items.splice(i, 1)`	`items.filter(...)`
একটি বদলানো	`items[i] = x`	`items.map(...)`

immutable-update.js · JavaScript চলবে: Node.js

```
const stock = [
  { id: 'P1', count: 40 },
  { id: 'P2', count: 3 },
];

const afterSale = stock.map((item) =>
  item.id === 'P1' ? { ...item, count: item.count - 1 } : item,
);

console.log(afterSale[0].count);
console.log(stock[0].count);
```

Output

```
39
40
```

`map` প্রতিটি Element দেখে: id মিললে `{ ...item, count: ... }` দিয়ে একটি **নতুন Object** তৈরি করে, যাতে পুরোনো সব তথ্য আছে এবং `count` নতুন; না মিললে পুরোনো Object-টিই ফেরত দেয়। মূল `stock` অপরিবর্তিত। এখানে `item.count = ...` লিখলে মূল Object-টিই বদলে যেত, কারণ নতুন Array-র ভেতরেও একই Object-গুলোর ঠিকানা থাকে।

কোনটি ব্যবহার করবেন? নিজের তৈরি, Function-এর ভেতরের একটি স্থানীয় Array-তে `push` করা সম্পূর্ণ ঠিক আছে। কিন্তু Argument হিসেবে পাওয়া Array, অথবা Program-এর একাধিক জায়গা থেকে ব্যবহৃত Array বদলাবেন না; নতুন Array তৈরি করুন।

৬.১২ Data রূপান্তরের ধারা

`map` ও `filter` Array ফেরত দেয়, তাই একটির ফলের ওপর সরাসরি আরেকটি Method Call করা যায়। একে **Method Chaining** বলে। প্রতিটি ধাপ একটি ছোট, স্পষ্ট কাজ করে।

chaining.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Pen', price: 15, stock: 0 },
  { name: 'School Bag', price: 950, stock: 5 },
  { name: 'Calculator', price: 650, stock: 0 },
];

const availableNames = products
  .filter((product) => product.stock > 0)
  .map((product) => product.name)
  .join(', ');

const availableValue = products
  .filter((product) => product.stock > 0)
  .reduce((sum, product) => sum + product.price * product.stock, 0);

console.log(availableNames);
console.log(availableValue);
```

Output

```
Notebook, School Bag
9550
```

প্রথম ধারাটি পড়ুন ওপর থেকে নিচে: যেগুলোর মজুত আছে সেগুলো **বাছো**, প্রতিটির নাম **নাও**, কমা দিয়ে **জোড়া লাগাও**। প্রতিটি Method নতুন লাইনে লিখলে ধাপগুলো আলাদাভাবে পড়া যায়।

বাস্তব Data প্রক্রিয়াকরণের অধিকাংশ কাজ এই তিন ধাপের কোনো সমন্বয়:

প্রশ্ন	Method
কোনগুলো দরকার?	`filter`
প্রতিটির কোন রূপ দরকার?	`map`
সব মিলিয়ে কী?	`reduce`
নির্দিষ্ট একটি কোথায়?	`find`

সাধারণত `filter` আগে, `map` পরে লিখুন: আগে তালিকা ছোট করে নিলে পরের ধাপে কম Element নিয়ে কাজ করতে হয়।

৬.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`map`-এর Callback-এ `return` নেই	`[ undefined, undefined ]`	`{ }` সরান অথবা `return` লিখুন
`filter` আশা করে `map` ব্যবহার	`[ true, false, true ]`	বাছাইয়ের জন্য `filter`
`push()`-এর ফলকে Array ভাবা	একটি সংখ্যা পাওয়া যায়	`push` নতুন `length` ফেরত দেয়
`slice` ও `splice` গুলিয়ে ফেলা	মূল Array অপ্রত্যাশিতভাবে বদলায়	`splice` বদলায়, `slice` বদলায় না
`find`-এর ফল যাচাই ছাড়া ব্যবহার	`TypeError: Cannot read properties of undefined`	আগে `=== undefined` যাচাই করুন
`reduce`-এ শুরুর মান নেই	ফাঁকা Array-তে `TypeError`; Object-এ ভুল ফল	দ্বিতীয় Argument দিন
`=` দিয়ে Array "Copy"	দুটি নাম একই Array দেখায়	`[...array]`
দুটি Array `===` দিয়ে তুলনা	একই মান হলেও `false`	`===` ঠিকানা তুলনা করে; Element ধরে তুলনা করুন
Argument-এর Array-তে `push` বা `sort`	Call-কারীর তালিকা বদলে যায়	নতুন Array তৈরি করুন
`sort()` Callback ছাড়া সংখ্যায়	`[ 100, 40, 9 ]`	`sort((a, b) => a - b)`

শেষ ভুলটি এতই প্রচলিত যে আলাদাভাবে দেখা দরকার।

sort-trap.js · JavaScript চলবে: Node.js

```
const scores = [40, 100, 9];

console.log([...scores].sort());
console.log([...scores].sort((a, b) => a - b));
console.log(scores);
```

Output

```
[ 100, 40, 9 ]
[ 9, 40, 100 ]
[ 40, 100, 9 ]
```

`sort()` Element-গুলোকে String হিসেবে তুলনা করে, তাই `'100'` আসে `'40'`-এর আগে (অধ্যায় ৩-এর String তুলনা)। সংখ্যা সাজাতে একটি তুলনাকারী Callback দিতে হয়। `sort` মূল Array বদলায়, তাই এখানে আগে `[...scores]` দিয়ে Copy করা হয়েছে; শেষ লাইন দেখাচ্ছে `scores` অক্ষত। Sorting নিয়ে বিস্তারিত অধ্যায় ১৮-এ।

৬.১৪ Debugging নির্দেশিকা

Method Chain-এ সমস্যা হলে পুরো ধারাটি একসঙ্গে বোঝার চেষ্টা করবেন না। প্রতিটি ধাপের ফল আলাদা Variable-এ নিয়ে দেখুন।

debug-chain.js · JavaScript চলবে: Node.js

```
const products = [
  { name: 'Notebook', stock: 40 },
  { name: 'Pen', stock: 0 },
];

const available = products.filter((product) => product.stock > 0);
console.log('filter-এর পর:', available.length);

const names = available.map((product) => product.name);
console.log('map-এর পর:', names);

console.log(JSON.stringify(available));
```

Output

```
filter-এর পর: 1
map-এর পর: [ 'Notebook' ]
[{"name":"Notebook","stock":40}]
```

`JSON.stringify()` একটি Object বা Array-কে এক লাইনের String-এ পরিণত করে। বড় বা গভীর Data এক নজরে দেখতে এবং দুটি Array-র বিষয়বস্তু মেলাতে এটি কাজে লাগে।

Array নিয়ে কাজ প্রত্যাশামতো না হলে:

১. Chain ভেঙে প্রতিটি ধাপের ফল ও তার `length` দেখুন।
২. ফলে `undefined` থাকলে দেখুন Callback কিছু `return` করছে কি না।
৩. Callback-এর ভেতরে `console.log()` বসিয়ে দেখুন প্রতিটি Element-এ কী আসছে।
৪. মূল Array অপ্রত্যাশিতভাবে বদলে গেলে খুঁজুন কোথায় `push`, `splice`, `sort` বা Index-এ Assignment হয়েছে।
৫. দুটি Variable একই Array কি না `===` দিয়ে যাচাই করুন।
৬. ফাঁকা Array এবং এক Element-এর Array দিয়ে পরীক্ষা করুন।
৭. `TypeError: ... is not a function` এলে দেখুন যার ওপর Method Call করছেন সেটি আদৌ Array কি না (`Array.isArray()`); হয়তো সেটি `push()`-এর ফেরত দেওয়া সংখ্যা।

৬.১৫ পেশাদার অভ্যাস

- **উদ্দেশ্য অনুযায়ী Method বেছে নিন।** রূপান্তরে `map`, বাছাইয়ে `filter`, একটি খুঁজতে `find`, সারমানে `reduce`, শুধু কিছু করতে `for...of`।
- **Argument হিসেবে পাওয়া Array বদলাবেন না।** নতুন Array ফেরত দিন।
- **`reduce`-এ সব সময় শুরুর মান দিন।**
- **Callback ছোট ও Pure রাখুন।** `map`-এর Callback-এর ভেতরে বাইরের Variable বদলাবেন না।
- **Callback-এর Parameter-এর অর্থবহ নাম দিন:** `product`, `price` — `x` বা `item1` নয়।
- **Chain-এর প্রতিটি ধাপ নতুন লাইনে।** তিন-চার ধাপের বেশি হলে মাঝের ফলকে একটি নাম দিন।
- **একটি Array-তে এক ধরনের মান।**
- **Array-র নাম বহুবচনে।**
- **ফাঁকা তালিকার আচরণ ভেবে নিন।** `filter` ফাঁকা Array দেয়, `find` দেয় `undefined`, শুরুর মানসহ `reduce` দেয় শুরুর মানটি।

৬.১৬ Project: Inventory Dashboard

একটি দোকানের মজুতের তালিকা সামলানোর Program লিখবেন: পণ্য যোগ, বাদ, মজুত বদলানো, খোঁজা, কম মজুতের তালিকা এবং মোট মূল্য। প্রতিটি কাজ মূল তালিকা না বদলে নতুন তালিকা ফেরত দেবে।

Project-এর চাহিদা

- প্রতিটি পণ্য একটি Object: `id`, `name`, `price` (টাকায়, পূর্ণসংখ্যা) এবং `stock`।

- পণ্য যোগ, বাদ এবং মজুত বদলানোর Function মূল Array বা তার Object বদলাবে না; নতুন Array ফেরত দেবে।
- মজুত কখনো শূন্যের নিচে নামবে না।
- id দিয়ে একটি পণ্য খোঁজা যাবে; না পেলে `undefined`।
- মজুত একটি সীমার সমান বা কম হলে পণ্যটি "মজুত কম" তালিকায় আসবে; সীমা না দিলে ৫ ধরা হবে।
- পুরো মজুতের মোট মূল্য (দাম × মজুত-এর যোগফল) দেখাতে হবে।

File-এর কাঠামো

Folder-এর কাঠামো

```
inventory-dashboard/
├── inventory.js
├── README.md
└── tests/
    └── inventory.test.js
```

inventory.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: তালিকা বদলানোর Function

inventory.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const LOW_STOCK_LIMIT = 5;

// নতুন পণ্যসহ একটি নতুন Array ফেরত দেয়।
const addProduct = (products, product) => [...products, product];

// নির্দিষ্ট id-র পণ্য বাদ দিয়ে একটি নতুন Array ফেরত দেয়।
const removeProduct = (products, id) =>
  products.filter((product) => product.id !== id);

// id দিয়ে একটি পণ্য খোঁজে; না পেলে undefined।
const findProduct = (products, id) =>
  products.find((product) => product.id === id);

// একটি পণ্যের মজুত বদলে নতুন Array ফেরত দেয়। মজুত শূন্যের নিচে নামে না।
function changeStock(products, id, change) {
  return products.map((product) => {
    if (product.id !== id) {
      return product;
    }

    const next = product.stock + change;
```

```
    return { ...product, stock: next < 0 ? 0 : next };
  });
}

// ...
```

অনুচ্ছেদ ৬.১১-এর তিনটি Non-mutating ধরনই এখানে আছে: যোগ করতে Spread, বাদ দিতে `filter`, বদলাতে `map`। `changeStock`-এর Callback-এ একাধিক ধাপ আছে, তাই `{ }` এবং `return` ব্যবহার করা হয়েছে। যে পণ্যের id মেলে না, সেটি অপরিবর্তিত ফেরত যায়; যেটি মেলে, তার একটি নতুন Object তৈরি হয়।

ধাপ ২: তথ্য বের করার Function

inventory.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// মজুত সীমার সমান বা কম, এমন পণ্যের Array।
const lowStock = (products, limit = LOW_STOCK_LIMIT) =>
  products.filter((product) => product.stock <= limit);

// পুরো মজুতের মোট মূল্য (টাকায়)।
const totalValue = (products) =>
  products.reduce((sum, product) => sum + product.price * product.stock, 0);

// প্রতিটি পণ্যের এক লাইনের বিবরণ।
const describe = (products) =>
  products.map((p) => `${p.id} ${p.name}: ${p.stock} x ${p.price}`);

// ...
```

তিনটি প্রশ্ন, তিনটি Method: কোনগুলো (`filter`), সব মিলিয়ে কত (`reduce`), প্রতিটির কোন রূপ (`map`)। `lowStock`-এ একটি Default Parameter আছে। `describe`-এর Callback-এ লাইন ছোট রাখতে Parameter-এর নাম `p` রাখা হয়েছে; এক লাইনের ছোট Callback-এ এটি গ্রহণযোগ্য।

ধাপ ৩: Dashboard দেখানো

inventory.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// পণ্যের তালিকা এবং সারসংক্ষেপ দেখায়।
function printDashboard(products) {
  for (const line of describe(products)) {
    console.log(line);
  }

  const lowNames = lowStock(products).map((product) => product.name);
```

```
  console.log(`পণ্য: ${products.length}টি`);
  console.log(`মোট মূল্য: ${totalValue(products)} টাকা`);
  console.log(`মজুত কম: ${lowNames.join(', ')}`);
}

const initial = [
  { id: 'P1', name: 'Notebook', price: 120, stock: 40 },
  { id: 'P2', name: 'Pen', price: 15, stock: 3 },
  { id: 'P3', name: 'School Bag', price: 950, stock: 5 },
];
const calculator = { id: 'P4', name: 'Calculator', price: 650, stock: 12 };

let products = addProduct(initial, calculator);
products = changeStock(products, 'P1', -38);
products = removeProduct(products, 'P2');

printDashboard(products);
console.log(`শুরুর তালিকায় এখনো ${initial.length}টি পণ্য`);
console.log(`শুরুর তালিকায় Notebook-এর মজুত: ${initial[0].stock}`);
```

Output

```
P1 Notebook: 2 x 120
P3 School Bag: 5 x 950
P4 Calculator: 12 x 650
পণ্য: 3টি
মোট মূল্য: 12790 টাকা
মজুত কম: Notebook, School Bag
শুরুর তালিকায় এখনো 3টি পণ্য
শুরুর তালিকায় Notebook-এর মজুত: 40
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **তিনটি পরিবর্তন, কোনো Mutation নয়।** `addProduct`, `changeStock` ও `removeProduct`—প্রতিটি একটি নতুন Array ফেরত দেয়, যা আবার `products`-এ রাখা হয়েছে। `products` নামটি নতুন Array-র দিকে সরানো হচ্ছে বলে এটি `let`; কোনো Array নিজে বদলাচ্ছে না।
- **প্রমাণ শেষ দুই লাইনে।** তিনটি পরিবর্তনের পরও `initial`-এ তিনটি পণ্য, এবং Notebook-এর মজুত ৪০। `changeStock`-এ `{ ...product, stock: ... }`-এর বদলে `product.stock = ...` লিখলে `initial[0].stock`-ও ২ হয়ে যেত, কারণ দুটি Array একই Object-এর ঠিকানা ধরে থাকত।
- **মজুত ৪০ থেকে ৩৮ কমে ২।** আরও বেশি কমাতে চাইলে Ternary সেটিকে শূন্যে আটকে দিত।
- **সীমার সমান মানও ধরা হয়।** School Bag-এর মজুত ঠিক ৫, এবং শর্তটি `<=`, তাই এটি "মজুত কম" তালিকায় আছে।
- **`for...of` দেখানোর জন্য।** `printDashboard` প্রতিটি লাইন দেখায়; এখানে কোনো মান তৈরি হচ্ছে না, শুধু একটি কাজ হচ্ছে, তাই `map` নয়, `for...of`।

- **Function-এর স্তর।** `printDashboard` ছাড়া সব Function Pure। অধ্যায় ৫-এর Invoice Calculator-এর মতোই, দেখানোর কাজ এক জায়গায়।

`findProduct` এই Demo-তে Call করা হয়নি; এটি Test-এ এবং অনুশীলনী ১০-এ ব্যবহৃত হয়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node inventory.js
```

Call	যা পাওয়ার কথা
`addProduct(initial, calculator)`	চারটি পণ্যের নতুন Array; `initial` -এ তিনটিই থাকে
`changeStock(products, 'P1', -38)`	P1-এর মজুত ২; বাকিগুলো অপরিবর্তিত
`changeStock(products, 'P1', -500)`	P1-এর মজুত ০ (ঋণাত্মক হয় না)
`changeStock(products, 'P9', 5)`	কোনো পরিবর্তন নয়; অচেনা id উপেক্ষিত
`removeProduct(products, 'P2')`	P2 ছাড়া নতুন Array
`findProduct(products, 'P9')`	`undefined`
`lowStock(initial)`	Pen (৩) এবং School Bag (৫)
`totalValue([])`	`0`

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে, যার কয়েকটি বিশেষভাবে যাচাই করে যে মূল Array ও তার Object বদলায়নি। চালানোর Command `README.md` -তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`initial` -এর মজুতও বদলে যাচ্ছে	`changeStock` -এ `product.stock = ...`	`{ ...product, stock: ... }` দিয়ে নতুন Object
`products.map is not a function`	`products` -এ `push()` -এর ফল (একটি সংখ্যা) রাখা হয়েছে	`[...products, product]`
Dashboard-এ `undefined` লাইন	`map` -এর Callback-এ `return` নেই	`{ }` ছাড়া লিখুন, বা `return` দিন
মোট মূল্য `NaN`	`reduce` -এর শুরুর মান নেই, বা কোনো `price` সংখ্যা নয়	দ্বিতীয় Argument `0` ; তথ্য যাচাই করুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ঠিক ৫ মজুতের পণ্য তালিকায় নেই	`<=` -এর জায়গায় `<`	`product.stock <= limit`
বাদ দেওয়ার পরও পণ্য থাকছে	ফলটি Variable-এ রাখা হয়নি	`products = removeProduct(products, id);`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Program-এর ভেতরে লেখা এবং কোথাও সংরক্ষণ করা হয় না। পণ্যের তথ্য যাচাই করা হয় না, এবং একই id দুবার যোগ করা আটকানো হয় না (অনুশীলনী ১০ দেখুন)। অচেনা id-তে মজুত বদলাতে চাইলে কোনো বার্তা দেওয়া হয় না। দাম পূর্ণ টাকায় ধরা হয়েছে; পয়সাসহ হিসাবে অধ্যায় ৫-এর পদ্ধতি প্রয়োগ করতে হবে।

৬.১৭ অনুশীলনী

প্রাথমিক স্তর

১. পাঁচটি শহরের নাম দিয়ে একটি Array তৈরি করুন। প্রথম, শেষ এবং মাঝের শহরটি দেখান।

২. একটি ফাঁকা Array দিয়ে শুরু করে `push` দিয়ে তিনটি সংখ্যা যোগ করুন, তারপর `pop` দিয়ে একটি বাদ দিন। প্রতিটি ধাপের পর `length` দেখান।

৩. না চালিয়ে বলুন: `['a', 'b', 'c', 'd'].slice(1, 3)` কী দেয়, এবং মূল Array-তে কয়টি Element থাকে?

৪. `for...of` দিয়ে `[10, 20, 30]` -এর প্রতিটি মানের দ্বিগুণ দেখান।

৫. নিচের Code-এ `doubled` -এ `[ 2, 4, 6 ]` আসার কথা, কিন্তু আসছে না। ভুলটি ঠিক করুন।

map-bug.js · JavaScript — চলবে: Node.js

```
const numbers = [1, 2, 3];

const doubled = numbers.map((number) => {
  number * 2;
});

console.log(doubled);
```

মধ্যম স্তর

৬. `[72, 28, 45, 91, 60]` নম্বরগুলো থেকে `filter` দিয়ে উত্তীর্ণদের (৩৩ বা বেশি) বের করুন, এবং `map` দিয়ে প্রতিটির সঙ্গে ৫ নম্বর Bonus যোগ করুন।

৭. `reduce` দিয়ে `[120, 15, 950, 280]` -এর যোগফল এবং গড় বের করুন।

৮. শিক্ষার্থীদের একটি Array থেকে (`name` ও `mark` সহ Object) `find` দিয়ে প্রথম সেই শিক্ষার্থীকে খুঁজুন, যার নম্বর ৯০ বা বেশি। না পাওয়া গেলে `কেউ নেই` দেখান।

৯. `withoutFirst` নামে একটি Function লিখুন, যা একটি Array-র প্রথম Element বাদ দিয়ে নতুন Array ফেরত দেয়, কিন্তু মূল Array বদলায় না।

চ্যালেঞ্জ স্তর

১০. **নকল id আটকানো:** `addUnique` Function লিখুন, যা পণ্য যোগ করে, কিন্তু একই id আগে থেকে থাকলে মূল Array-টিই অপরিবর্তিত ফেরত দেয়।

১১. **দামি পণ্যের নাম:** পণ্যের Array থেকে যেগুলোর মোট মূল্য (দাম × মজুত) ৪০০০ টাকা বা বেশি, সেগুলোর নাম কমা দিয়ে আলাদা করে দেখান। `filter`, `map` ও `join` ব্যবহার করুন।

১২. **দাম বাড়ানো:** `raisePrices` Function লিখুন, যা সব পণ্যের দাম একটি নির্দিষ্ট শতাংশ বাড়িয়ে নতুন Array ফেরত দেয়। মূল Array এবং তার Object যেন না বদলায়, তা দেখান।

৬.১৮ অনুশীলনীর সমাধান

সমাধান ১

cities.js · JavaScript — চলবে: Browser Console ও Node.js

```
const cities = ['Dhaka', 'Bogura', 'Sylhet', 'Khulna', 'Rajshahi'];

console.log(cities[0]);
console.log(cities.at(-1));
console.log(cities[2]);
```

Output

```
Dhaka
Rajshahi
Sylhet
```

সমাধান ২

push-three.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [];

numbers.push(10);
console.log(numbers.length);

numbers.push(20);
numbers.push(30);
console.log(numbers.length);

numbers.pop();
console.log(numbers.length);
```

Output

```
1
3
2
```

সমাধান ৩ — `[ 'b', 'c' ]`: Index ১ ও ২, কারণ শেষ Index অন্তর্ভুক্ত নয়। মূল Array-তে চারটি Element-ই থাকে, কারণ `slice` মূল Array বদলায় না।

সমাধান ৪

double-each.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (const value of [10, 20, 30]) {
  console.log(value * 2);
}
```

Output

```
20
40
60
```

সমাধান ৫ — Callback-এ `{ }` আছে কিন্তু `return` নেই, তাই প্রতিটি Element-এর জন্য `undefined` ফেরত যাচ্ছে। `{ }` সরিয়ে দিলে Expression-টির মান নিজে থেকেই ফেরত যায়।

map-fixed.js · JavaScript — চলবে: Node.js

```
const numbers = [1, 2, 3];

const doubled = numbers.map((number) => number * 2);

console.log(doubled);
```

Output

```
[ 2, 4, 6 ]
```

সমাধান ৬

passed-bonus.js · JavaScript — চলবে: Node.js

```
const marks = [72, 28, 45, 91, 60];

const result = marks
  .filter((mark) => mark >= 33)
  .map((mark) => mark + 5);

console.log(result);
```

Output

```
[ 77, 50, 96, 65 ]
```

সমাধান ৭

sum-average.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280];

const sum = prices.reduce((total, price) => total + price, 0);

console.log(sum);
console.log(sum / prices.length);
```

Output

```
1365
341.25
```

সমাধান ৮

top-student.js · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', mark: 80 },
  { name: 'Mina', mark: 93 },
  { name: 'Rafi', mark: 95 },
];

const top = students.find((student) => student.mark >= 90);

console.log(top === undefined ? 'কেউ নেই' : top.name);
```

Output

```
Mina
```

Rafi-র নম্বর বেশি, কিন্তু `find` প্রথম যাকে পায় তাকেই ফেরত দেয় এবং সেখানে থেমে যায়।

সমাধান ৯

without-first.js · JavaScript — চলবে: Node.js

```
const withoutFirst = (items) => items.slice(1);

const queue = ['Sara', 'Rafi', 'Mina'];

console.log(withoutFirst(queue));
console.log(queue);
```

Output

```
[ 'Rafi', 'Mina' ]
[ 'Sara', 'Rafi', 'Mina' ]
```

`shift()` একই Element বাদ দিত, কিন্তু মূল Array বদলে ফেলত।

সমাধান ১০

add-unique.js · JavaScript — চলবে: Browser Console ও Node.js

```
function addUnique(products, product) {
  const existing = products.find((item) => item.id === product.id);

  if (existing !== undefined) {
    return products;
  }

  return [...products, product];
}

const products = [{ id: 'P1', name: 'Notebook' }];
const added = addUnique(products, { id: 'P2', name: 'Pen' });
const duplicate = addUnique(added, { id: 'P2', name: 'Pencil' });

console.log(added.length);
console.log(duplicate.length);
console.log(duplicate === added);
```

Output

```
2
2
true
```

সমাধান ১১

valuable.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Pen', price: 15, stock: 3 },
  { name: 'School Bag', price: 950, stock: 5 },
];

const valuable = products
  .filter((product) => product.price * product.stock >= 4000)
  .map((product) => product.name)
  .join(', ');

console.log(valuable);
```

Output

```
Notebook, School Bag
```

সমাধান ১২

raise-prices.js · JavaScript চলবে: Browser Console ও Node.js

```
function raisePrices(products, percent) {
  return products.map((product) => ({
    ...product,
    price: product.price + Math.round(product.price * percent / 100),
  }));
}

const products = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
];
const raised = raisePrices(products, 10);

console.log(raised[0].price);
console.log(raised[1].price);
console.log(products[0].price);
```

Output

```
132
17
120
```

Arrow Function থেকে সরাসরি একটি Object ফেরত দিতে হলে Object-টিকে প্রথম বন্ধনীতে ঘিরতে হয়: `=> ({ ... })`। নইলে JavaScript `{ }`-কে Function-এর Body ভাবে।

৬.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `const a = ['x', 'y', 'z'];` হলে `a.at(-1)` কী দেয়?

ক) `'x'`

খ) `undefined`

গ) `'z'`

ঘ) একটি Error

প্রশ্ন ২. `push()` কী ফেরত দেয়?

ক) Array-র নতুন `length`

খ) নতুন Array

গ) যোগ করা Element

ঘ) `undefined`

প্রশ্ন ৩. কোনটি Array-র শুরু থেকে একটি Element বাদ দেয়?

ক) `pop()`

খ) `unshift()`

গ) `push()`

ঘ) `shift()`

প্রশ্ন ৪. `slice` এবং `splice`-এর মূল পার্থক্য কী?

ক) `slice` শুধু String-এ কাজ করে

খ) `splice` মূল Array বদলায়; `slice` বদলায় না

গ) `slice` মূল Array বদলায়; `splice` বদলায় না

ঘ) কোনো পার্থক্য নেই

প্রশ্ন ৫. `[1, 2, 3].map((n) => n * 2)`-এর ফল কী?

ক) `6`

খ) `[ 1, 2, 3 ]`

গ) `[ 2, 4, 6 ]`

ঘ) `[ true, true, true ]`

প্রশ্ন ৬. `filter`-এ কোনো Element শর্ত না মিললে কী ফেরত আসে?

ক) `undefined`

খ) `null`

গ) `false`

ঘ) একটি ফাঁকা Array

প্রশ্ন ৭. `find` কী ফেরত দেয়?

ক) শর্ত মেলা প্রথম Element

খ) শর্ত মেলা সব Element-এর Array

গ) শর্ত মেলা Element-এর সংখ্যা

ঘ) `true` বা `false`

প্রশ্ন ৮. `[10, 20, 30].reduce((sum, n) => sum + n, 0)`-এর ফল কী?

ক) `[ 10, 30, 60 ]`

খ) `60`

গ) `30`

ঘ) `0`

প্রশ্ন ৯. নিচের কোনটি মূল Array বদলায় **না**?

ক) `sort()`

খ) `splice()`

গ) `filter()`

ঘ) `push()`

প্রশ্ন ১০. `const b = a;` লেখার পর `b.push(1)` করলে `a`-র কী হয় (`a` একটি Array)?

ক) কিছু হয় না; `b` একটি আলাদা Copy

খ) `a`-তেও `1` যোগ হয়, কারণ দুটি নাম একই Array দেখায়

গ) একটি Error হয়

ঘ) `a` ফাঁকা হয়ে যায়

৬.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–ঘ · ৭–ক · ৮–খ · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	`at()`-এ ঋণাত্মক সংখ্যা শেষ থেকে গোনে; `-1` মানে শেষ Element।
২	ক	`push` মূল Array বদলায় এবং নতুন `length` দেয়; নতুন Array নয়।
৩	ঘ	`shift` শুরু থেকে বাদ দেয়; `pop` শেষ থেকে। `unshift` ও `push` যোগ করে।
৪	খ	`slice` একটি অংশের Copy দেয়; `splice` মূল Array কেটে বদলায়।
৫	গ	`map` প্রতিটি Element রূপান্তর করে একই দৈর্ঘ্যের নতুন Array দেয়।
৬	ঘ	`filter` সব সময় একটি Array দেয়; কিছু না মিললে সেটি ফাঁকা।
৭	ক	`find` প্রথম মিলেই থেমে যায়। সবগুলো পেতে `filter`। না পেলে `undefined`।
৮	খ	শুরুর মান `0`; তারপর ১০, ৩০, ৬০। `reduce` একটি মান দেয়, Array নয়।
৯	গ	`filter` নতুন Array দেয়। বাকি তিনটি মূল Array বদলায়।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	খ	`=` ঠিকানা Copy করে, Array নয়। আলাদা Copy-র জন্য `[...a]`।

৬.২১ অধ্যায়ের সারসংক্ষেপ

- Array ক্রমানুসারে মান রাখে; Index শূন্য থেকে; শেষ Element `at(-1)`।
- `push` / `pop` শেষ প্রান্তে, `unshift` / `shift` শুরুর প্রান্তে যোগ ও বাদ দেয়; চারটিই মূল Array বদলায়।
- `slice` একটি অংশের Copy দেয়; `splice` মূল Array কেটে বদলায়।
- শুধু মান দরকার হলে `for...of`; Index দরকার হলে `for`।
- `map` প্রতিটি Element রূপান্তর করে; দৈর্ঘ্য একই থাকে।
- `filter` শর্ত মেলা Element বেছে নেয়; না মিললে ফাঁকা Array।
- `find` প্রথম মিলটি দেয়; না মিললে `undefined`।
- `reduce` পুরো Array-কে একটি মানে গুটিয়ে আনে; শুরুর মান সব সময় দিন।
- Mutating Method মূল Array বদলায়; Non-mutating Method নতুন মান দেয়। Argument হিসেবে পাওয়া Array বদলাবেন না।
- Mutation ছাড়া: যোগ করতে `[...items, x]`, বাদ দিতে `filter`, বদলাতে `map` ও `{ ...item }`।
- `filter`, `map`, `reduce` জুড়ে Data রূপান্তরের ধারা তৈরি হয়।

পরের অধ্যায়ে Object: Property, Method, Destructuring এবং বাস্তব Data-র গঠন—যা এই অধ্যায়ের Object-এর তালিকাগুলোকে আরও শক্তিশালী করবে।

৬.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Array Literal	`[ ]` দিয়ে লেখা Array।
Element / Index	Array-র একটি মান / তার অবস্থানের নম্বর।
Method	একটি মানের সঙ্গে যুক্ত Function, যেমন `array.push()`।
Mutation	একটি বিদ্যমান Array বা Object-কে বদলে ফেলা।
Mutating / Non-mutating	যে Method মূলটি বদলায় / যেটি নতুন মান দেয়।
Spread	`...`: একটি Array বা Object-এর বিষয়বস্তু নতুনটিতে ছড়িয়ে দেওয়া।
`map`	প্রতিটি Element রূপান্তর করে নতুন Array।
`filter`	শর্ত মেলা Element নিয়ে নতুন Array।

পরিভাষা	অর্থ
`find`	শর্ত মেলা প্রথম Element।
`reduce`	পুরো Array থেকে একটি মান।
Accumulator	`reduce`-এ ধাপে ধাপে জমা হওয়া ফল।
Method Chaining	একটি Method-এর ফলের ওপর সরাসরি আরেকটি Call করা।

৬.২৩ আরও পড়ুন

- **MDN — Array:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array` — সব Array Method-এর তালিকা।
- **MDN — Indexed collections:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Indexed_collections` — Array-র নির্দেশিকা।
- **MDN — Array.prototype.map():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map` — `map`-এর বিবরণ।
- **MDN — Array.prototype.reduce():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce` — `reduce`-এর বিবরণ ও উদাহরণ।
- **MDN — for...of:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...of` — `for...of` Statement।

অধ্যায় ৭

০৭

Objects: Properties, Methods, Destructuring এবং Data Modeling

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Object Literal দিয়ে Object তৈরি করতে এবং সংক্ষিপ্ত লেখার রীতিগুলো ব্যবহার করতে।
- Dot ও Bracket Notation-এর পার্থক্য বুঝে সঠিকটি বেছে নিতে।
- Property পড়তে, যোগ করতে, বদলাতে, মুছতে এবং তার অস্তিত্ব নির্ভরযোগ্যভাবে যাচাই করতে।
- Method লিখতে এবং `this` কীভাবে কাজ করে তা ব্যাখ্যা করতে।
- Nested Object নিরাপদে পড়তে Optional Chaining ব্যবহার করতে।
- Destructuring দিয়ে Object ও Array থেকে মান বের করতে।
- `Object.keys()`, `Object.values()` ও `Object.entries()` দিয়ে Object-এর ওপর Loop চালাতে।
- Spread দিয়ে Shallow Copy তৈরি করতে এবং তার সীমা বুঝতে।
- JSON-এ রূপান্তর করে তথ্য সংরক্ষণ ও আদান-প্রদান করতে।
- বাস্তব তথ্যের জন্য একটি Object-এর গঠন (Data Model) নকশা করতে।
- একটি সম্পূর্ণ **Student Management System** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৬ সম্পন্ন করা: Array Method, Spread এবং Mutation বনাম নতুন মান।
- অধ্যায় ২-এর Reference: Object-এ ঠিকানা Copy হয়, মান নয়।
- অধ্যায় ৫-এর Function, Arrow Function এবং Scope।
- `chapter-07` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একজন শিক্ষার্থীর নাম, রোল, নম্বর, ফোন নম্বর এবং ঠিকানা আছে। এগুলো পাঁচটি আলাদা Variable-এ রাখা যায়, কিন্তু তখন কোন নম্বরটি কার, তা মনে রাখার দায় পুরোপুরি আপনার। একশ জন শিক্ষার্থী হলে পাঁচশ Variable। **Object** এই সমস্যার সমাধান: একটি জিনিসের সব তথ্য একটি নামে, প্রতিটি তথ্যের নিজস্ব লেবেলসহ।

আগের অধ্যায়গুলোতে আপনি Object ব্যবহার করেছেন, কিন্তু শুধু তার সবচেয়ে সরল রূপে। বাস্তব Program-এর তথ্য প্রায় সব সময় Object আকারে আসে: Server থেকে আসা উত্তর, Form-এর ডেটা, Database-এর একটি সারি, Settings-এর তালিকা। Object ভালোভাবে না জানলে এসব তথ্য নিয়ে কাজ করা যায় না।

এই অধ্যায়ের শেষ ভাগে একটি গুরুত্বপূর্ণ দক্ষতা আসবে: তথ্যকে কীভাবে সাজাবেন, তা আগে থেকে ঠিক করা। ভালো Data Model থাকলে পরের সব Code সহজ হয়; দুর্বল Model থাকলে প্রতিটি Function-এ বাড়তি যাচাই লিখতে হয়।

৭.১ Object কেন প্রয়োজন

Array তথ্য রাখে ক্রম অনুযায়ী: প্রথম, দ্বিতীয়, তৃতীয়। Object তথ্য রাখে নাম অনুযায়ী।

why-objects.js · JavaScript — চলবে: Node.js

```
const student = {
  name: 'Sara',
  age: 20,
  marks: 88,
};

console.log(student.name);
console.log(student.marks);
console.log(student);
```

Output

```
Sara
88
{ name: 'Sara', age: 20, marks: 88 }
```

Object-এর প্রতিটি নাম-মানের জোড়াকে **Property** বলে। নামটিকে **Key**, আর তার সঙ্গে যুক্ত মানটিকে **Value**। `name`, `age` ও `marks` এখানে তিনটি Key।

	Array	Object
তথ্য খোঁজা হয়	অবস্থান (Index) দিয়ে	নাম (Key) দিয়ে
উপযুক্ত	একই ধরনের অনেক জিনিসের তালিকা	একটি জিনিসের বিভিন্ন বৈশিষ্ট্য
উদাহরণ	সব শিক্ষার্থী	একজন শিক্ষার্থী

বাস্তবে দুটো একসঙ্গে ব্যবহৃত হয়: শিক্ষার্থীদের একটি Array, যার প্রতিটি Element একটি Object। অধ্যায় ৪ ও ৬-এ আপনি এই কাঠামোই ব্যবহার করেছেন।

> **নোট:** Object সরাসরি `console.log()` করলে Node.js ও Browser ভিন্ন সাজে দেখায়। এই অধ্যায়ে পুরো Object দেখানো উদাহরণগুলো Node.js-এর Output; Browser Console একই তথ্য দেখায়, তবে আলাদা চেহারায়।

৭.২ Object তৈরি করা

সবচেয়ে প্রচলিত উপায় **Object Literal**: দ্বিতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করা `key: value` জোড়া। শেষ Property-এর পরের কমাটি ঐচ্ছিক, কিন্তু রাখলে নতুন Property যোগ করার সময় ভুল কম হয়।

দুটি সংক্ষিপ্ত রীতি

shorthand.js · JavaScript — চলবে: Node.js

```
const name = 'Rafi';
const marks = 70;

const student = { name, marks };
console.log(student);

const field = 'city';
const place = { [field]: 'Dhaka' };
console.log(place);
```

Output

```
{ name: 'Rafi', marks: 70 }
{ city: 'Dhaka' }
```

- **Shorthand Property** — Key এবং Variable-এর নাম এক হলে শুধু একবার লিখলেই চলে। `{ name, marks }` মানে `{ name: name, marks: marks }`।

- **Computed Key** — বর্গবন্ধনীর ভেতরে একটি Expression দিলে তার মান Key হয়ে যায়। Key আগে থেকে জানা না থাকলে এটি কাজে লাগে।

Object তৈরির আরও কিছু পদ্ধতি আছে, যেমন `new Object()` এবং `Object.create()`। পুরোনো Code-এ এগুলো দেখা যায়, কিন্তু সাধারণ তথ্য রাখার জন্য Literal-ই সবচেয়ে পরিষ্কার। `Object.create()`-এর আসল ব্যবহার Prototype-এর সঙ্গে জড়িত, যা অধ্যায় ১৩-তে আসবে।

৭.৩ Dot এবং Bracket Notation

Property পড়ার দুটি উপায়: ডট দিয়ে (`profile.name`), অথবা বর্গবন্ধনীতে Key-কে String হিসেবে দিয়ে (`profile['name']`)।

dot-bracket.js · JavaScript — চলবে: Browser Console ও Node.js

```
const profile = { name: 'Mina', 'home-city': 'Khulna' };

console.log(profile.name);
console.log(profile['name']);
console.log(profile['home-city']);

const key = 'name';
console.log(profile[key]);
console.log(profile.key);
```

Output

```
Mina
Mina
Khulna
Mina
undefined
```

পরিস্থিতি	ব্যবহার করুন
Key আগে থেকে জানা এবং সাধারণ নাম	Dot: `profile.name`
Key-তে হাইফেন, ফাঁকা জায়গা বা সংখ্যা দিয়ে শুরু	Bracket: `profile['home-city']`
Key একটি Variable-এ আছে	Bracket: `profile[key]`

শেষ লাইনটি সবচেয়ে প্রচলিত ভুলটি দেখায়। `profile.key` লিখলে JavaScript `key` নামের Variable-এর মান ব্যবহার করে না; সে `'key'` নামের একটি Property খোঁজে, যা নেই। Variable-এর মান দিয়ে খুঁজতে চাইলে সব সময় বর্গবন্ধনী।

৭.৪ Property পড়া, যোগ, বদল এবং মোছা

Object তৈরির পরেও তাতে Property যোগ, বদল বা মোছা যায়।

property-changes.js · JavaScript — চলবে: Node.js

```
const book = { title: 'JavaScript', price: 500 };

book.author = 'Moynul';
book.price = 450;
console.log(book);

delete book.author;
console.log(book);
console.log(book.author);
```

Output

```
{ title: 'JavaScript', price: 450, author: 'Moynul' }
{ title: 'JavaScript', price: 450 }
undefined
```

- নেই এমন Key-তে মান রাখলে নতুন Property যোগ হয়; আছে এমন Key-তে রাখলে মান বদলায়। দুটো দেখতে একই রকম।
- `delete` একটি Property সম্পূর্ণ সরিয়ে দেয়।
- নেই এমন Property পড়লে Error হয় না; `undefined` পাওয়া যায়।
- `book` একটি `const`, তবু ভেতরের তথ্য বদলানো যাচ্ছে (অধ্যায় ২): `const` শুধু নামটিকে একই Object-এর সঙ্গে বেঁধে রাখে।

Property আছে কি না

`book.author === undefined` দিয়ে যাচাই নির্ভরযোগ্য নয়, কারণ একটি Property থাকতে পারে, আর তার মানই হতে পারে `undefined`।

has-property.js · JavaScript চলবে: Browser Console ও Node.js

```
const book = { title: 'JavaScript', discount: undefined };

console.log(book.discount === undefined);
console.log(book.author === undefined);

console.log(Object.hasOwn(book, 'discount'));
console.log(Object.hasOwn(book, 'author'));

console.log('toString' in book);
console.log(Object.hasOwn(book, 'toString'));
```

Output

```
true
true
true
false
true
false
```

`=== undefined` দুটি ক্ষেত্রকে আলাদা করতে পারেনি। `Object.hasOwn(object, key)` পেরেছে: `discount` আছে (মান `undefined` হলেও), `author` নেই।

`in` Operator-ও অস্তিত্ব যাচাই করে, কিন্তু সে Object-এর নিজের Property ছাড়াও উত্তরাধিকারসূত্রে পাওয়া Property খোঁজে। প্রতিটি Object জন্ম থেকেই `toString` -এর মতো কিছু Method পায়, তাই `'toString' in book` সত্য। নিজের তথ্য যাচাই করতে `Object.hasOwn()` ব্যবহার করুন।

৭.৫ Method এবং this

Property-র মান একটি Function-ও হতে পারে। এমন Property-কে **Method** বলে। `console.log` , `array.push` , `text.trim` —এগুলো সবই Method।

methods.js · JavaScript চলবে: Browser Console ও Node.js

```
const account = {
  owner: 'Sara',
  balance: 1000,

  deposit(amount) {
    this.balance += amount;
    return this.balance;
  },

  describe() {
    return `${this.owner}: ${this.balance} টাকা`;
  },
};

console.log(account.deposit(500));
console.log(account.describe());
```

Output

```
1500
Sara: 1500 টাকা
```

`deposit(amount) { ... }` হলো Method লেখার সংক্ষিপ্ত রূপ; এর অর্থ `deposit: function (amount) { ... }`।

Method-এর ভেতরে **`this`** সেই Object-কে বোঝায়, যার মাধ্যমে Method-টি Call করা হয়েছে। `account.deposit(500)`-এ ডটের বাঁ পাশে `account`, তাই `this` হলো `account`, এবং `this.balance` মানে `account.balance`।

this হারিয়ে যাওয়া

`this` ঠিক হয় Method কীভাবে Call করা হলো তা থেকে, কোথায় লেখা হয়েছে তা থেকে নয়। Method-টিকে Object থেকে আলাদা করে Call করলে `this` আর Object-টি থাকে না।

lost-this.js · JavaScript চলবে: Browser Console ও Node.js

```
'use strict';

const account = {
  owner: 'Sara',
  describe() {
    return `${this.owner}`;
  },
};

const describe = account.describe;
console.log(describe());
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'owner')
```

`describe()`-এর বাঁ পাশে কোনো Object নেই, তাই Strict Mode-এ `this` হয় `undefined`। Function-টি একই, শুধু Call করার ধরন বদলেছে। এই সমস্যা সবচেয়ে বেশি দেখা দেয় যখন একটি Method-কে Callback হিসেবে পাঠানো হয়।

> **সতর্কতা:** Object-এর Method Arrow Function দিয়ে লিখবেন না। Arrow Function-এর নিজস্ব `this` থাকে না; সে বাইরের Scope-এর `this` ব্যবহার করে, যা Object-টি নয়। `describe: () => this.owner` লিখলে `undefined` পাওয়া যায়। Method-এর জন্য ওপরের সংক্ষিপ্ত রূপটি ব্যবহার করুন।

`this` এবং Object-এর আচরণ নিয়ে পূর্ণ আলোচনা Class-এর সঙ্গে অধ্যায় ১৩-তে।

৭.৬ Nested Object এবং Optional Chaining

একটি Property-র মান আরেকটি Object হতে পারে। এভাবে তথ্যকে স্তরে স্তরে সাজানো যায়।

nested.js · JavaScript চলবে: Browser Console ও Node.js

```
const student = {
  name: 'Rafi',
  contact: {
    phone: '01700-000002',
    address: { city: 'Bogura' },
  },
};

const newcomer = { name: 'Nila' };

console.log(student.contact.address.city);
console.log(newcomer.contact?.address?.city);
console.log(newcomer.contact?.address?.city ?? 'অজানা');
```

Output

```
Bogura
undefined
অজানা
```

`student.contact.address.city` প্রতিটি ধাপে এক স্তর ভেতরে যায়। কিন্তু `newcomer`-এর `contact` নেই। সেখানে সাধারণ ডট ব্যবহার করলে কী হয়:

missing-nested.js · JavaScript চলবে: Browser Console ও Node.js

```
const newcomer = { name: 'Nila' };

console.log(newcomer.contact.address);
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'address')
```

`newcomer.contact` হলো `undefined`, আর `undefined`-এর কোনো Property পড়া যায় না। **Optional Chaining** (`?.`) এই সমস্যার সমাধান: বাঁ পাশের মান `null` বা `undefined` হলে সে আর এগোয় না, Error-এর বদলে `undefined` দেয়। তার সঙ্গে অধ্যায় ৩-এর `??` জুড়ে দিলে একটি পূর্বনির্ধারিত মান দেখানো যায়।

`?.` ব্যবহার করুন শুধু সেখানে, যেখানে তথ্যটি সত্যিই না-ও থাকতে পারে, যেমন ঐচ্ছিক যোগাযোগের তথ্য। প্রতিটি ডটের জায়গায় `?.` বসালে আসল Bug লুকিয়ে যায়: যে তথ্য অবশ্যই থাকার কথা, সেটি না থাকলে Error দেখাই ভালো।

৭.৭ Destructuring

Object থেকে কয়েকটি Property বের করে আলাদা Variable-এ রাখার সংক্ষিপ্ত উপায় **Destructuring**।

destructuring.js · JavaScript — চলবে: Browser Console ও Node.js

```
const student = { id: 'S1', name: 'Sara', marks: 88 };

const { name, marks } = student;
console.log(name, marks);

const { marks: score, grade = 'জানা নেই' } = student;
console.log(score, grade);

function describe({ name, marks }) {
  return `${name} পেয়েছেন ${marks}`;
}

console.log(describe(student));
```

Output

```
Sara 88
88 জানা নেই
Sara পেয়েছেন 88
```

- **`const { name, marks } = student;`** — `student.name` যায় `name`-এ, `student.marks` যায় `marks`-এ। নাম মিলতে হয়; ক্রম গুরুত্বপূর্ণ নয়।
- **`marks: score`** — Property-টি অন্য নামে নেওয়া। এখানে কোলনের বাঁ পাশে Property-র নাম, ডান পাশে নতুন Variable-এর নাম।
- **`grade = 'জানা নেই'`** — Default মান। Property না থাকলে বা `undefined` হলে এটি বসে; অধ্যায় ৫-এর Default Parameter-এর মতোই, `null` হলে বসে না।
- **Parameter-এ Destructuring** — Function একটি Object পায় এবং সঙ্গে সঙ্গে তার দরকারি Property-গুলো আলাদা করে নেয়। কোন তথ্য দরকার, তা Function-এর প্রথম লাইনেই স্পষ্ট হয়।

Array-কেও Destructure করা যায়, তবে সেখানে নাম নয়, অবস্থান মেলে:

array-destructuring.js · JavaScript — চলবে: Browser Console ও Node.js

```
const [first, second] = ['আম', 'কলা', 'লিচু'];

console.log(first, second);
```

Output

```
আম কলা
```

৭.৮ Object-এর ওপর Loop

Object-এর সব Property নিয়ে কাজ করার জন্য তিনটি Method আছে। প্রতিটি একটি Array ফেরত দেয়, তাই অধ্যায় ৬-এর সব Array Method তার ওপর ব্যবহার করা যায়।

object-loops.js · JavaScript — চলবে: Node.js

```
const marks = { bangla: 80, english: 72, math: 91 };

console.log(Object.keys(marks));
console.log(Object.values(marks));

for (const [subject, mark] of Object.entries(marks)) {
  console.log(`${subject}: ${mark}`);
}

const total = Object.values(marks).reduce((sum, mark) => sum + mark, 0);
console.log(total);
```

Output

```
[ 'bangla', 'english', 'math' ]
[ 80, 72, 91 ]
bangla: 80
english: 72
math: 91
243
```

Method	ফেরত দেয়
`Object.keys(obj)`	সব Key-এর Array
`Object.values(obj)`	সব Value-এর Array
`Object.entries(obj)`	`[key, value]` জোড়ার Array

`Object.entries()`-এর প্রতিটি Element একটি দুই-Element-এর Array। `for (const [subject, mark] of ...)` লেখায় অনুচ্ছেদ ৭.৭-এর Array Destructuring প্রতিটি জোড়াকে দুটি নামে ভাগ করে নিচ্ছে।

> **নোট:** `for...in` নামে একটি পুরোনো Loop-ও Object-এর Key দেয়, কিন্তু সেটি উত্তরাধিকারসূত্রে পাওয়া Property-ও দেখাতে পারে। নিজের তথ্যের জন্য `Object.keys()` বা `Object.entries()` ব্যবহার করুন।

৭.৯ Spread, Rest এবং Shallow Copy

অধ্যায় ৬-এ Array-র জন্য যে Spread (`...`) দেখেছেন, Object-এও তা কাজ করে।

spread-copy.js · JavaScript চলবে: Node.js

```
const original = { name: 'Mina', marks: 74, contact: { city: 'Khulna' } };

const updated = { ...original, marks: 80 };
console.log(original.marks, updated.marks);

updated.contact.city = 'Dhaka';
console.log(original.contact.city);

const { contact, ...withoutContact } = original;
console.log(withoutContact);
```

Output

```
74 80
Dhaka
{ name: 'Mina', marks: 74 }
```

- `{ ...original, marks: 80 }` — `original`-এর সব Property নতুন Object-এ, তারপর `marks` নতুন মানে। একই Key দুবার থাকলে পরেরটি জেতে। অধ্যায় ৬-এর Inventory Project-এ মজুত বদলাতে ঠিক এটিই ব্যবহার করা হয়েছিল।
- **কিন্তু Copy-টি Shallow!** প্রথম স্তরের Property-গুলো নতুন Object-এ গেছে, কিন্তু `contact` একটি Object, আর তার জন্য শুধু ঠিকানা Copy হয়েছে। তাই `updated.contact.city` বদলালে `original.contact.city`-ও বদলে গেছে।
- **Rest** — Destructuring-এ `...withoutContact` "বাকি সব" Property একটি নতুন Object-এ সংগ্রহ করে। একটি Property বাদ দিয়ে Copy তৈরির পরিষ্কার উপায় এটি।

সম্পূর্ণ আলাদা Copy

ভেতরের স্তরসহ পুরোপুরি আলাদা Copy দরকার হলে `structuredClone()` ব্যবহার করুন। এটি আধুনিক Browser ও Node.js-এ পাওয়া যায়।

deep-copy.js · JavaScript চলবে: Browser Console ও Node.js

```
const original = { name: 'Mina', contact: { city: 'Khulna' } };
const deep = structuredClone(original);

deep.contact.city = 'Sylhet';

console.log(original.contact.city);
console.log(deep.contact.city);
```

Output

```
Khulna
Sylhet
```

ভেতরের একটি তথ্য বদলাতে পুরো Object-এর Deep Copy সাধারণত লাগে না। বরং যে স্তর বদলাচ্ছে, শুধু সেই স্তরে আবার Spread করুন: `{ ...student, contact: { ...student.contact, city: 'Sylhet' } }`।

৭.১০ Reference, সমতা এবং Freeze

অধ্যায় ২ থেকে: Object Variable-এ ঠিকানা রাখে, আর `===` ঠিকানা তুলনা করে।

equality.js · JavaScript — চলবে: Browser Console ও Node.js

```
const a = { id: 1 };
const b = { id: 1 };
const c = a;

console.log(a === b);
console.log(a === c);
console.log(JSON.stringify(a) === JSON.stringify(b));
```

Output

```
false
true
true
```

একই বিষয়বস্তুর দুটি আলাদা Object `===` দিয়ে সমান নয়। বিষয়বস্তু তুলনা করতে Property ধরে ধরে মেলাতে হয়। ছোট, সরল Object-এর জন্য দুটিকেই JSON String-এ রূপান্তর করে তুলনা করা যায় (অনুচ্ছেদ ৭.১১), তবে Property-র ক্রম আলাদা হলে এই পদ্ধতি ভুল উত্তর দেয়।

Object.freeze

একটি Object যেন আর বদলানো না যায়, তা নিশ্চিত করতে `Object.freeze()` ব্যবহার করা হয়।

freeze.js · JavaScript — চলবে: Browser Console ও Node.js

```
const settings = Object.freeze({ theme: 'light', size: { font: 14 } });

settings.theme = 'dark';
console.log(settings.theme);

settings.size.font = 18;
console.log(settings.size.font);
```

Output

```
light
18
```

`theme` বদলায়নি। কিন্তু `freeze` -ও Shallow: ভেতরের `size` Object জমাট নয়, তাই সেটি বদলে গেছে। আরেকটি বিষয়: এই File-এ Strict Mode নেই, তাই `settings.theme = 'dark'` নীরবে উপেক্ষিত হয়েছে। `'use strict';` থাকলে একই লাইন `TypeError` দিত, যা ভুল ধরার জন্য ভালো।

৭.১১ JSON: তথ্য সংরক্ষণ ও আদান-প্রদান

JSON (JavaScript Object Notation) হলো তথ্যকে লেখা (String) হিসেবে প্রকাশের একটি মান-নির্ধারিত রূপ। দেখতে JavaScript Object-এর মতো, কিন্তু এটি শুধু একটি String। Server-এর সঙ্গে তথ্য আদান-প্রদান, File-এ সংরক্ষণ এবং Browser Storage—সব জায়গায় JSON ব্যবহৃত হয়।

json.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
const student = { name: 'Sara', marks: 88, passed: true, contact: null };

const text = JSON.stringify(student);
console.log(text);
console.log(typeof text);

const back = JSON.parse(text);
console.log(back.name, back.marks);
console.log(back === student);
```

Output

```
{"name":"Sara","marks":88,"passed":true,"contact":null}
string
Sara 88
false
```

- **`JSON.stringify(value)`** — Object থেকে JSON String।
- **`JSON.parse(text)`** — JSON String থেকে নতুন Object। ফেরত আসা Object মূলটির একটি সম্পূর্ণ আলাদা Copy।

JSON-এর নিয়ম JavaScript-এর চেয়ে কঠোর: Key সব সময় Double Quote-এ, String-ও Double Quote-এ, শেষে বাড়তি কমা চলে না। সব ধরনের মান JSON-এ রাখাও যায় না।

json-limits.js · JavaScript চলবে: Browser Console ও Node.js

```
const record = {
  name: 'Rafi',
  note: undefined,
  greet() {
    return 'hi';
  },
  joined: new Date(0),
};

console.log(JSON.stringify(record));
console.log(JSON.stringify({ name: 'Rafi', marks: 70 }, null, 2));
```

Output

```
{"name":"Rafi","joined":"1970-01-01T00:00:00.000Z"}
{
  "name": "Rafi",
  "marks": 70
}
```

`undefined` মানের Property এবং Method বাদ পড়েছে, আর তারিখ একটি String হয়ে গেছে; `JSON.parse()` সেটিকে আবার তারিখে ফেরাবে না। `JSON.stringify()` -এর তৃতীয় Argument (`2`) Output-কে সাজিয়ে দেখায়, যা পড়ার জন্য সুবিধাজনক।

ভুল JSON পার্স করতে গেলে Program থেমে যায়:

bad-json.js · JavaScript চলবে: Browser Console ও Node.js

```
const text = "{name: 'Sara'}";

console.log(JSON.parse(text));
```

Error Message

```
SyntaxError: Expected property name or '}' in JSON at position 1
```

বাইরে থেকে আসা JSON সব সময় ঠিক থাকবে, এমন ধরে নেওয়া যায় না। এমন Error ধরে সামলানোর উপায় অধ্যায় ১১-এ শেখানো হবে।

Browser-এ সংরক্ষণ: localStorage

Browser-এ ছোট তথ্য স্থায়ীভাবে রাখার একটি সহজ উপায় `localStorage`। এটি শুধু String রাখে, তাই Object রাখতে JSON-এ রূপান্তর লাগে। Node.js-এ এটি নেই; উদাহরণটি Browser-এ চালান।

storage.js · JavaScript চলবে: Browser

```
const student = { name: 'Sara', marks: 88 };

localStorage.setItem('student', JSON.stringify(student));

const saved = JSON.parse(localStorage.getItem('student'));
console.log(saved.name, saved.marks);
console.log(localStorage.getItem('missing'));

localStorage.removeItem('student');
```

Output

```
Sara 88
null
```

Page বন্ধ করে আবার খুললেও `localStorage`-এর তথ্য থাকে। নেই এমন Key পড়লে `null` আসে। Password বা অন্য গোপন তথ্য এখানে রাখবেন না: একই Website-এর যেকোনো Script এটি পড়তে পারে।

৭.১২ Data Modeling: তথ্যের গঠন নকশা করা

Code লেখার আগে ঠিক করুন একটি জিনিসের তথ্য কোন আকারে থাকবে। এই নকশাকে **Data Model** বা Schema বলে। Project-এর একজন শিক্ষার্থীর Model:

Property	Type	নিয়ম	উদাহরণ
`id`	String	আবশ্যক; ফাঁকা নয়; প্রত্যেকের আলাদা	`'S1'`
`name`	String	আবশ্যক; কমপক্ষে ২ অক্ষর	`'Sara Ahmed'`
`marks`	Number	০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা	`88`
`contact`	Object বা `null`	ঐচ্ছিক	`{ phone: '01700-000001' }`

এই সারণি থেকে কয়েকটি সিদ্ধান্ত সরাসরি আসে:

- **একটি অনন্য পরিচয়।** দুজন শিক্ষার্থীর নাম একই হতে পারে, তাই খোঁজা, বদলানো ও মোছার জন্য `id` লাগে।
- **একটি জিনিস, এক জায়গায়।** Grade সংরক্ষণ করা হয় না, কারণ সেটি নম্বর থেকে হিসাব করা যায়। দুটোই রাখলে একটি বদলালে অন্যটি পুরোনো থেকে যেতে পারে।
- **"নেই" বোঝাতে `null`।** যোগাযোগের তথ্য না থাকলে `contact: null`, যাতে প্রতিটি শিক্ষার্থীর Object-এ একই Property থাকে।

- **ঐচ্ছিক তথ্য আলাদা স্তরে।** ফোন ও ঠিকানা `contact`-এর ভেতরে, যাতে পড়ার সময় `contact?.phone` দিয়ে নিরাপদে পাওয়া যায়।

৭.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Variable-এর মান দিয়ে খুঁজতে `obj.key` লেখা	`undefined`	`obj[key]`
Spread Copy-কে সম্পূর্ণ আলাদা ভাবা	ভেতরের Object বদলালে মূলটিও বদলায়	যে স্তর বদলাবেন সেখানে আবার Spread, অথবা `structuredClone()`
`=== undefined` দিয়ে অস্তিত্ব যাচাই	মান `undefined` হলে ভুল উত্তর	`Object.hasOwn()`
Method Arrow Function দিয়ে লেখা	`this` থেকে `undefined`	Method-এর সংক্ষিপ্ত রূপ
Method আলাদা করে Callback হিসেবে পাঠানো	`TypeError: Cannot read properties of undefined`	`() => account.describe()` পাঠান
প্রতিটি ডটের জায়গায় `?.`	আসল Bug চাপা পড়ে	শুধু ঐচ্ছিক তথ্যে `?.`
দুটি Object `===` দিয়ে তুলনা	একই বিষয়বস্তুতেও `false`	Property ধরে তুলনা করুন
`Object.freeze()`-কে সম্পূর্ণ সুরক্ষা ভাবা	ভেতরের Object বদলায়	`freeze` Shallow
JSON-এ Single Quote বা শেষে কমা	`SyntaxError`	Double Quote; শেষে কমা নয়
JSON-এ তারিখ রাখা	ফেরত আসে String হয়ে	পড়ার পর `new Date(text)` দিয়ে রূপান্তর
Argument-এর Object বদলানো	Call-কারীর তথ্য বদলে যায়	নতুন Object ফেরত দিন

৭.১৪ Debugging নির্দেশিকা

Object-এর Bug ধরতে প্রথম কাজ: Object-টির আসল চেহারা দেখা।

debug-object.js · JavaScript — চলবে: Node.js

```
const student = { id: 'S1', name: 'Sara', contact: null };

console.log(Object.keys(student));
console.log(JSON.stringify(student, null, 2));
console.log(Object.hasOwn(student, 'marks'));
```

Output

```
[ 'id', 'name', 'contact' ]
{
  "id": "S1",
  "name": "Sara",
  "contact": null
}
false
```

`Object.keys()` দেখায় আসলে কোন Property আছে; বানান ভুল (`marks` বনাম `mark`) সঙ্গে সঙ্গে ধরা পড়ে। `JSON.stringify(object, null, 2)` ভেতরের স্তরসহ পুরো Object সাজিয়ে দেখায়।

Object নিয়ে কাজ প্রত্যাশামতো না হলে:

১. পুরো Object `JSON.stringify(object, null, 2)` দিয়ে দেখুন।

২. `undefined` এলে `Object.keys()` দিয়ে Property-র আসল নাম মিলিয়ে নিন।

৩. `Cannot read properties of undefined (reading 'x')` এলে বার্তার শেষের নামটির আগের ধাপটি দেখুন: সেটিই `undefined`।

৪. একটি Object অপ্রত্যাশিতভাবে বদলালে খুঁজুন আর কোন Variable তার Reference ধরে আছে, এবং Copy-টি Shallow ছিল কি না।

৫. `this` সংক্রান্ত সমস্যায় দেখুন Method-টি কীভাবে Call হয়েছে: ডটের বাঁ পাশে Object আছে কি না।

৬. JSON পার্স ব্যর্থ হলে String-টি নিজে দেখুন; বার্তার `position` বলে দেয় কোন অক্ষরে সমস্যা।

৭.১৫ পেশাদার অভ্যাস

- **আগে Data Model, পরে Code।** প্রতিটি Property-র নাম, Type এবং নিয়ম লিখে নিন।
- **একই ধরনের সব Object-এ একই Property।** ঐচ্ছিক তথ্য না থাকলে `null` রাখুন, Property বাদ দেবেন না।
- **হিসাব করা যায় এমন তথ্য সংরক্ষণ করবেন না।** Grade, মোট, বয়স—এগুলো দরকারের সময় হিসাব করুন।
- **প্রতিটি রেকর্ডের একটি অনন্য `id` রাখুন।**
- **Argument-এর Object বদলাবেন না;** `{ ...object, key: newValue }` দিয়ে নতুন Object ফেরত দিন।
- **Parameter-এ Destructuring** ব্যবহার করুন, যাতে Function কোন তথ্য চায় তা প্রথম লাইনেই বোঝা যায়।
- **Key-এর নাম camelCase-এ,** হাইফেন বা ফাঁকা জায়গা ছাড়া; তাহলে সব জায়গায় Dot Notation চলে।
- **বাইরে থেকে আসা Object বিশ্বাস করবেন না।** JSON-এ আসা তথ্য ব্যবহারের আগে যাচাই করুন।

৭.১৬ Project: Student Management System

শিক্ষার্থীদের তথ্য সামলানোর একটি Program লিখবেন: যোগ, নম্বর বদলানো, মোছা, খোঁজা এবং সারসংক্ষেপ। অনুচ্ছেদ ৭.১২-এর Data Model এর ভিত্তি।

Project-এর চাহিদা

- প্রতিটি শিক্ষার্থী অনুচ্ছেদ ৭.১২-এর Model অনুযায়ী একটি Object।
- নতুন শিক্ষার্থী যোগ করার আগে যাচাই: ID ফাঁকা নয় এবং আগে থেকে নেই, নাম কমপক্ষে ২ অক্ষর, নম্বর ০–১০০-এর পূর্ণসংখ্যা। সমস্যা থাকলে কারণসহ বার্তা।
- যোগ, নম্বর বদলানো ও মোছার কাজ মূল Array বা Object বদলাবে না।
- নাম বা ID দিয়ে খোঁজা যাবে, বড়-ছোট হাতের অক্ষর নির্বিশেষে।
- সারসংক্ষেপ: মোট শিক্ষার্থী, গড় নম্বর (এক দশমিক ঘরে), উত্তীর্ণের সংখ্যা এবং সর্বোচ্চ নম্বরধারী।
- ফোন নম্বর না থাকলে "নেই" দেখাবে।

এখানে ব্যবহৃত ফোন নম্বরগুলো কাল্পনিক।

File-এর কাঠামো

Folder-এর কাঠামো

```
student-management/
├── students.js
├── README.md
└── tests/
    ├── load-script.js
    └── students.test.js
```

students.js

File-টি তিন অংশে দেখানো হলো; তিনটি মিলে সম্পূর্ণ File। অংশগুলো পরপর একই File-এ লিখুন। `// ...` লাইনগুলো শুধু অংশের সংযোগ বোঝায়; সেগুলো লিখতে হবে না।

অংশ ১: Grade এবং যাচাই

students.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const PASS_MARK = 33;
const GRADES = [
  { min: 80, grade: 'A+' },
  { min: 70, grade: 'A' },
  { min: 60, grade: 'A-' },
  { min: 50, grade: 'B' },
```

```
  { min: 40, grade: 'C' },
  { min: 33, grade: 'D' },
  { min: 0, grade: 'F' },
];

// নম্বর থেকে Grade।
function gradeFor(marks) {
  const { grade } = GRADES.find((row) => marks >= row.min);
  return grade;
}

// নতুন শিক্ষার্থীর তথ্য যাচাই করে; সমস্যা থাকলে বার্তা, না থাকলে null।
function validateStudent(input, students) {
  const id = String(input.id ?? '').trim();
  const name = String(input.name ?? '').trim();
  const marks = input.marks;

  if (id === '') {
    return 'ID দিতে হবে।';
  }

  if (name.length < 2) {
    return 'নাম কমপক্ষে ২ অক্ষরের হতে হবে।';
  }

  if (!Number.isInteger(marks) || marks < 0 || marks > 100) {
    return 'নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে।';
  }

  if (students.find((student) => student.id === id) !== undefined) {
    return `ID ${id} আগে থেকেই আছে।`;
  }

  return null;
}

// ...
```

Grade-এর সীমাগুলো Code-এর মধ্যে `if` হিসেবে না লিখে একটি Object-এর Array-তে রাখা হয়েছে। এটি তথ্য, তাই তথ্যের মতো করেই রাখা ভালো: সীমা বদলাতে শুধু টেবিলটি বদলালেই চলে। `GRADES` বড় থেকে ছোট ক্রমে সাজানো, তাই `find` প্রথম যে সারিতে নম্বর সীমা ছোঁয়, সেটিই দেয়; শেষ সারির সীমা `0`, তাই কোনো বৈধ নম্বরে `find` কখনো `undefined` দেয় না। তারপর Destructuring দিয়ে শুধু `grade` নেওয়া হয়েছে।

`validateStudent`-এ `String(input.id ?? '')` একসঙ্গে দুটি সমস্যা সামলায়: `id` না থাকলে ফাঁকা String, আর সংখ্যা হিসেবে এলে String-এ রূপান্তর।

অংশ ২: তালিকা বদলানো ও খোঁজা

students.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// যাচাই করে যোগ করে। ফেরত দেয় { students, error }।
function addStudent(students, input) {
  const error = validateStudent(input, students);

  if (error !== null) {
    return { students, error };
  }

  const student = {
    id: input.id.trim(),
    name: input.name.trim(),
    marks: input.marks,
    contact: input.contact ?? null,
  };

  return { students: [...students, student], error: null };
}

// একজনের নম্বর বদলে নতুন Array; বাকিরা অপরিবর্তিত।
function updateMarks(students, id, marks) {
  return students.map((student) =>
    student.id === id ? { ...student, marks } : student,
  );
}

// নির্দিষ্ট ID বাদ দিয়ে নতুন Array।
const removeStudent = (students, id) =>
  students.filter((student) => student.id !== id);

// নাম বা ID-তে খোঁজে; বড়-ছোট হাতের অক্ষর নির্বিশেষে।
function searchStudents(students, query) {
  const text = query.trim().toLowerCase();

  return students.filter(({ id, name }) =>
    id.toLowerCase().includes(text) || name.toLowerCase().includes(text),
  );
}

// ...
```

`addStudent` দুটি তথ্য ফেরত দিতে চায়: নতুন তালিকা এবং সম্ভাব্য সমস্যা। একটি Function একটিই মান ফেরত দিতে পারে, তাই দুটি একটি Object-এ রাখা হয়েছে; Shorthand Property-র কারণে `{ students, error }` লেখা গেছে। সমস্যা থাকলে মূল তালিকাটিই অপরিবর্তিত ফেরত যায়।

`updateMarks`-এ `{ ...student, marks }` হলো অনুচ্ছেদ ৭.৯-এর Spread এবং অনুচ্ছেদ ৭.২-এর Shorthand একসঙ্গে: পুরোনো সব তথ্য, শুধু `marks` নতুন। `searchStudents`-এর Callback Parameter-এ Destructuring করে শুধু `id` ও `name` নিয়েছে। `toLowerCase()` সব অক্ষর ছোট হাতের করে, আর String-এর `includes()` জানায় একটি লেখা আরেকটির ভেতরে আছে কি না; দুটিই অধ্যায় ৮-এ বিস্তারিত আসবে।

অংশ ৩: সারসংক্ষেপ, দেখানো এবং উদাহরণ

students.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// মোট, গড়, উত্তীর্ণ এবং সর্বোচ্চ নম্বরধারীর নাম।
function summarize(students) {
  const total = students.length;

  if (total === 0) {
    return { total, average: 0, passed: 0, topper: null };
  }

  const sum = students.reduce((acc, { marks }) => acc + marks, 0);
  const passed = students.filter(({ marks }) => marks >= PASS_MARK).length;
  const best = students.reduce(
    (top, student) => (student.marks > top.marks ? student : top),
    students[0],
  );
  const average = Math.round((sum / total) * 10) / 10;

  return { total, average, passed, topper: best.name };
}

// একজন শিক্ষার্থীর এক লাইনের বিবরণ।
function formatStudent({ id, name, marks, contact }) {
  const phone = contact?.phone ?? 'নেই';
  return `${id} ${name}: ${marks} (${gradeFor(marks)}), ফোন: ${phone}`;
}

let students = [];
const inputs = [
  {
    id: 'S1',
    name: 'Sara Ahmed',
    marks: 88,
    contact: { phone: '01700-000001' },
  },
  { id: 'S2', name: 'Rafi', marks: 28 },
  {
    id: 'S3',
    name: 'Mina Das',
    marks: 74,
```

```
    contact: { phone: '01700-000003' },
  },
  { id: 'S2', name: 'Another Rafi', marks: 50 },
  { id: 'S4', name: 'N', marks: 60 },
];

for (const input of inputs) {
  const result = addStudent(students, input);
  students = result.students;

  if (result.error !== null) {
    console.log(`বাতিল: ${result.error}`);
  }
}

students = updateMarks(students, 'S2', 35);

for (const student of students) {
  console.log(formatStudent(student));
}

const { total, average, passed, topper } = summarize(students);
console.log(`মোট ${total} জন, গড় ${average}, উত্তীর্ণ ${passed} জন, সর্বোচ্চ: ${topper}`);
console.log(searchStudents(students, 'MINA').map(({ name }) => name).join(', '));
```

Output

```
বাতিল: ID S2 আগে থেকেই আছে।
বাতিল: নাম কমপক্ষে ২ অক্ষরের হতে হবে।
S1 Sara Ahmed: 88 (A+), ফোন: 01700-000001
S2 Rafi: 35 (D), ফোন: নেই
S3 Mina Das: 74 (A), ফোন: 01700-000003
মোট 3 জন, গড় 65.7, উত্তীর্ণ 3 জন, সর্বোচ্চ: Sara Ahmed
Mina Das
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **যাচাই আগে, যোগ পরে।** পাঁচটি ইনপুটের মধ্যে দুটি বাতিল হয়েছে: একই ID দ্বিতীয়বার, এবং এক অক্ষরের নাম। প্রতিটির কারণ আলাদা বার্তায়।
- **`students = result.students`** — `students` একটি `let`, কারণ প্রতিবার নামটিকে নতুন Array-র দিকে সরানো হচ্ছে। কোনো Array নিজে বদলাচ্ছে না।
- **`updateMarks`-এর পর।** Rafi-র নম্বর ২৮ থেকে ৩৫ হয়েছে, তাই Grade `D` এবং তিনি এখন উত্তীর্ণ।
- **`summarize`-এ `reduce` দুবার।** প্রথমবার যোগফল; দ্বিতীয়বার সর্বোচ্চ নম্বরধারী, যেখানে শুরুর মান হিসেবে প্রথম শিক্ষার্থীকে দেওয়া হয়েছে (অধ্যায় ৬-এর নিয়ম)। গড় ১৯৭ ÷ ৩ = ৬৫.৬৬..., এক দশমিক ঘরে ৬৫.৭।

- **ফাঁকা তালিকা।** `summarize` শুরুতেই ফাঁকা তালিকা আলাদা করে সামলায়; নইলে শূন্য দিয়ে ভাগ হতো এবং `students[0]` হতো `undefined`।
- **`formatStudent`-এ Destructuring ও Optional Chaining।** Rafi-র `contact` হলো `null`, তাই `contact?.phone` `undefined` দেয় এবং `??` "নেই" বসায়।
- **শেষে Destructuring।** `summarize`-এর ফেরত দেওয়া Object থেকে চারটি মান একবারে আলাদা করা হয়েছে।
- **খোঁজা।** `'MINA'` বড় হাতের অক্ষরে দেওয়া হলেও Mina Das পাওয়া গেছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node students.js
```

অবস্থা	যা হওয়ার কথা
একই ID দ্বিতীয়বার	বাতিল: ID S2 আগে থেকেই আছে।
এক অক্ষরের নাম	বাতিল: নাম কমপক্ষে ২ অক্ষরের হতে হবে।
নম্বর `101`, `-1`, `88.5` বা `'88'`	বাতিল: নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে।
ফোন নম্বর ছাড়া	ফোন: নেই
`updateMarks`	শুধু সেই শিক্ষার্থীর নম্বর বদলায়; মূল তালিকা অক্ষত
ফাঁকা তালিকার সারসংক্ষেপ	মোট 0, গড় 0, উত্তীর্ণ 0, সর্বোচ্চ `null`
খোঁজা `'mina'` বা `'MINA'`	Mina Das

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই ID দুবার যোগ হয়ে যায়	`validateStudent`-এ পুরোনো তালিকা পাঠানো হয়নি	`validateStudent(input, students)`
নম্বর বদলানোর পর মূল তালিকাও বদলে যায়	`student.marks = marks` লেখা হয়েছে	`{ ...student, marks }`
`TypeError: Cannot read properties of null (reading 'phone')`	`contact.phone` লেখা হয়েছে	`contact?.phone`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Grade সব সময় `F`	`GRADES` ছোট থেকে বড় ক্রমে সাজানো	বড় থেকে ছোট ক্রমে রাখুন
`TypeError: Cannot destructure property 'grade' of 'undefined'`	কোনো সীমা মেলেনি (যেমন ঋণাত্মক নম্বর)	যাচাই আগে করুন; শেষ সারির সীমা `0` রাখুন
গড় `NaN`	ফাঁকা তালিকার যাচাই নেই	`total === 0` আগে যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Program-এর ভেতরে লেখা এবং কোথাও সংরক্ষিত হয় না; অনুশীলনী ১২-তে JSON দিয়ে সংরক্ষণের প্রথম ধাপ আছে। ফোন নম্বরের গঠন যাচাই করা হয় না। নাম ও ID-র খোঁজা খুব সরল; বাংলা নামের বানানভেদ সামলানো হয় না (অধ্যায় ৮ দেখুন)। নম্বর বদলানোর সময় নতুন নম্বর যাচাই করা হয় না।

৭.১৭ অনুশীলনী

প্রাথমিক স্তর

১. একটি `book` Object তৈরি করুন, যাতে `title`, `author` ও `price` আছে। তারপর Spread দিয়ে দাম ১০ শতাংশ কমিয়ে একটি নতুন Object তৈরি করুন; মূল Object যেন না বদলায়।

২. `getFullName` নামে একটি Function লিখুন, যা `{ firstName, lastName }` আকারের Object নিয়ে পুরো নাম ফেরত দেয়। Parameter-এ Destructuring ব্যবহার করুন।

৩. `{ bangla: 80, english: 72, math: 91 }` থেকে `Object.entries()` দিয়ে প্রতিটি বিষয় ও নম্বর দেখান।

৪. না চালিয়ে বলুন: `const user = { profile: null }; user.profile?.email ?? 'নেই'`-এর মান কী, এবং কেন।

৫. নিচের Code-এ `price` দেখানোর কথা, কিন্তু দেখাচ্ছে `undefined`। ভুলটি ঠিক করুন।

key-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const product = { name: 'Pen', price: 15 };
const field = 'price';

console.log(product.field);
```

মধ্যম স্তর

৬. `Object.hasOwn()` এবং `in`-এর পার্থক্য একটি উদাহরণ দিয়ে দেখান।

৭. শিক্ষার্থীদের একটি Array থেকে যাঁদের নম্বর ৮০ বা বেশি, তাঁদের নাম `filter`, `map` এবং Destructuring দিয়ে বের করুন।

৮. `counter` নামে একটি Object লিখুন, যাতে `count` Property এবং `increment()` ও `reset()` Method আছে। `this` ব্যবহার করুন।

৯. `moveToCity` নামে একটি Function লিখুন, যা শিক্ষার্থীর Object এবং একটি শহরের নাম নিয়ে নতুন Object ফেরত দেয়, যেখানে `contact.city` বদলেছে, কিন্তু মূল Object এবং তার `contact` অপরিবর্তিত।

চ্যালেঞ্জ স্তর

১০. **বিষয়ভিত্তিক গড়:** কয়েকজন শিক্ষার্থীর বিষয়ভিত্তিক নম্বর (`{ name, marks: { bangla, english } }`) থেকে প্রতিটি বিষয়ের গড় বের করুন।

১১. **সমতা যাচাই:** `sameStudent` নামে একটি Function লিখুন, যা দুটি শিক্ষার্থী-Object-এর `id`, `name` ও `marks` মিলিয়ে বিষয়বস্তু একই কি না বলে।

১২. **সংরক্ষণ ও পুনরুদ্ধার:** শিক্ষার্থীদের একটি Array JSON String-এ রূপান্তর করুন, তারপর সেটি থেকে আবার Array তৈরি করুন এবং দেখান যে দুটির বিষয়বস্তু একই, কিন্তু তারা আলাদা Array।

৭.১৮ অনুশীলনীর সমাধান

সমাধান ১

book.js · JavaScript — চলবে: Browser Console ও Node.js

```
const book = { title: 'JavaScript', author: 'Moynul', price: 500 };
const discounted = { ...book, price: book.price - book.price * 10 / 100 };

console.log(discounted.price);
console.log(book.price);
```

Output

```
450
500
```

সমাধান ২

full-name.js · JavaScript — চলবে: Browser Console ও Node.js

```
function getFullName({ firstName, lastName }) {
  return `${firstName} ${lastName}`;
}

console.log(getFullName({ firstName: 'Sadia', lastName: 'Rahman' }));
```

Output

```
Sadia Rahman
```

সমাধান ৩

subjects.js · JavaScript — চলবে: Browser Console ও Node.js

```
const marks = { bangla: 80, english: 72, math: 91 };

for (const [subject, mark] of Object.entries(marks)) {
  console.log(`${subject}: ${mark}`);
}
```

Output

```
bangla: 80
english: 72
math: 91
```

সমাধান ৪ — `'নেই'`। `user.profile` হলো `null`, তাই `?.` সেখানেই থেমে `undefined` দেয়; তারপর `??` বিকল্প মান `'নেই'` বসায়। `?.` না থাকলে `TypeError` হতো।

সমাধান ৫ — `product.field` Variable-এর মান ব্যবহার করে না; `'field'` নামের Property খোঁজে। Variable-এর মান দিয়ে খুঁজতে বর্গবন্ধনী লাগে।

key-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const product = { name: 'Pen', price: 15 };
const field = 'price';

console.log(product[field]);
```

Output

```
15
```

সমাধান ৬

has-own-vs-in.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const item = { name: 'Pen' };

console.log('name' in item, Object.hasOwn(item, 'name'));
console.log('toString' in item, Object.hasOwn(item, 'toString'));
```

Output

```
true true
true false
```

`toString` Object-টির নিজের Property নয়, উত্তরাধিকারসূত্রে পাওয়া। `in` সেটিও গোনে; `Object.hasOwn()` গোনে না।

সমাধান ৭

top-students.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 35 },
  { name: 'Mina', marks: 80 },
];

const toppers = students
  .filter(({ marks }) => marks >= 80)
  .map(({ name }) => name);

console.log(toppers.join(', '));
```

Output

```
Sara, Mina
```

সমাধান ৮

counter.js · JavaScript — চলবে: Browser Console ও Node.js

```
const counter = {
  count: 0,

  increment() {
    this.count++;
    return this.count;
  },

  reset() {
    this.count = 0;
  },
};

counter.increment();
console.log(counter.increment());
counter.reset();
console.log(counter.count);
```

Output

```
2
0
```

সমাধান ৯

move-city.js · JavaScript — চলবে: Browser Console ও Node.js

```
function moveToCity(student, city) {
  return { ...student, contact: { ...student.contact, city } };
}

const student = {
  name: 'Mina',
  contact: { phone: '01700-000003', city: 'Khulna' },
};
const moved = moveToCity(student, 'Sylhet');

console.log(moved.contact.city, moved.contact.phone);
console.log(student.contact.city);
```

Output

```
Sylhet 01700-000003
Khulna
```

যে স্তরটি বদলাচ্ছে, শুধু সেই স্তরে আবার Spread করা হয়েছে; ফোন নম্বর নতুন Object-এও আছে, আর মূল `contact` অক্ষত।

সমাধান ১০

subject-average.js · JavaScript চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', marks: { bangla: 80, english: 90 } },
  { name: 'Rafi', marks: { bangla: 60, english: 70 } },
];

const totals = {};

for (const { marks } of students) {
  for (const [subject, mark] of Object.entries(marks)) {
    totals[subject] = (totals[subject] ?? 0) + mark;
  }
}

for (const [subject, sum] of Object.entries(totals)) {
  console.log(`${subject}: ${sum / students.length}`);
}
```

Output

```
bangla: 70
english: 80
```

`totals[subject]`-এ বর্গবন্ধনী লাগছে, কারণ Key-টি Variable-এ আছে। প্রথমবার Key-টি থাকে না, তাই `?? 0` দিয়ে শূন্য থেকে শুরু করা হয়েছে। এখানে `totals` Function-এর বাইরে নয়, নিজের তৈরি একটি স্থানীয় Object, তাই তাতে সরাসরি মান যোগ করা নিরাপদ।

সমাধান ১১

same-student.js · JavaScript — চলবে: Browser Console ও Node.js

```
function sameStudent(a, b) {
  return a.id === b.id && a.name === b.name && a.marks === b.marks;
}

const first = { id: 'S1', name: 'Sara', marks: 88 };
const copy = { ...first };
const changed = { ...first, marks: 90 };

console.log(first === copy);
console.log(sameStudent(first, copy));
console.log(sameStudent(first, changed));
```

Output

```
false
true
false
```

সমাধান ১২

save-restore.js · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { id: 'S1', name: 'Sara', marks: 88 },
  { id: 'S2', name: 'Rafi', marks: 35 },
];

const saved = JSON.stringify(students);
const restored = JSON.parse(saved);

console.log(restored.length, restored[1].name);
console.log(JSON.stringify(restored) === saved);
console.log(restored === students);
```

Output

```
2 Rafi
true
false
```

বাস্তবে `saved` String-টি একটি File বা `localStorage`-এ রাখা হতো। পুনরুদ্ধারের পর তথ্য যাচাই করা উচিত, কারণ সংরক্ষিত লেখাটি বদলে গিয়ে থাকতে পারে।

৭.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `const key = 'name';` থাকলে `user.name`-এর সমান কোনটি?

ক) `user.key`

খ) `user['key']`

গ) `user[key]`

ঘ) `user.[key]`

প্রশ্ন ২. `const name = 'Rafi'; const s = { name };` লেখার পর `s` কী?

ক) `{ name: 'Rafi' }`

খ) `{ 'Rafi': name }`

গ) একটি Error

ঘ) `{ name: undefined }`

প্রশ্ন ৩. একটি Property আছে কিন্তু তার মান `undefined`। কোনটি নির্ভরযোগ্যভাবে বলে Property-টি আছে?

ক) `obj.key === undefined`

খ) `Object.hasOwn(obj, 'key')`

গ) `obj.key !== null`

ঘ) `typeof obj.key`

প্রশ্ন ৪. `account.deposit(100)` Call-এ `deposit`-এর ভেতরে `this` কী?

ক) `deposit` Function

খ) Global Object

গ) `undefined`

ঘ) `account`

প্রশ্ন ৫. `const a = { contact: null }; a.contact?.phone`-এর মান কী?

ক) `undefined`

খ) `null`

গ) একটি `TypeError`

ঘ) ফাঁকা String

প্রশ্ন ৬. `const { marks: score } = { marks: 88 };` লেখার পর কোন Variable তৈরি হয়?

ক) `marks`, মান `88`

খ) `score`, মান `88`

গ) দুটোই

ঘ) কোনোটিই নয়; এটি একটি Error

প্রশ্ন ৭. `const copy = { ...original };`-এর পর `copy.contact.city` বদলালে কী হয় (`contact` একটি Object)?

ক) শুধু `copy` বদলায়

খ) একটি Error হয়

গ) `original.contact.city`-ও বদলায়

ঘ) কিছুই বদলায় না

প্রশ্ন ৮. `Object.entries({ a: 1, b: 2 })` কী ফেরত দেয়?

ক) `[ [ 'a', 1 ], [ 'b', 2 ] ]`

খ) `[ 'a', 'b' ]`

গ) `[ 1, 2 ]`

ঘ) `{ a: 1, b: 2 }`

প্রশ্ন ৯. `JSON.stringify({ a: undefined, b: 2 })`-এর ফল কী?

ক) `{"a":undefined,"b":2}`

খ) `{"a":null,"b":2}`

গ) একটি Error

ঘ) `{"b":2}`

প্রশ্ন ১০. `{ id: 1 } === { id: 1 }`-এর ফল কী?

ক) `true`, কারণ বিষয়বস্তু একই

খ) `false`, কারণ এরা দুটি আলাদা Object

গ) একটি Error

ঘ) `undefined`

৭.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–খ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ক · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	বর্গবন্ধনী Variable-এর মান ব্যবহার করে। `user.key` খোঁজে `'key'` নামের Property।
২	ক	Shorthand Property: Key এবং Variable-এর নাম এক হলে একবার লিখলেই চলে।
৩	খ	`=== undefined` "নেই" এবং "আছে কিন্তু `undefined`"-এর পার্থক্য করতে পারে না।
৪	ঘ	Call-এর সময় ডটের বাঁ পাশের Object-ই `this`।
৫	ক	`?.` বাঁ পাশ `null` হলে থেমে `undefined` দেয়; Error হয় না।
৬	খ	কোলনের বাঁ পাশে Property-র নাম, ডান পাশে নতুন Variable-এর নাম।
৭	গ	Spread Copy Shallow; ভেতরের Object-এর শুধু ঠিকানা Copy হয়।
৮	ক	প্রতিটি Property একটি `[key, value]` জোড়া হয়ে আসে।
৯	ঘ	JSON-এ `undefined` রাখা যায় না, তাই Property-টি বাদ পড়ে।
১০	খ	`===` Object-এর ঠিকানা তুলনা করে, বিষয়বস্তু নয়।

৭.২১ অধ্যায়ের সারসংক্ষেপ

- Object তথ্য রাখে নাম (Key) অনুযায়ী; Array রাখে অবস্থান অনুযায়ী।
- Shorthand Property (`{ name }`) এবং Computed Key (`{ [field]: value }`) Object লেখা সংক্ষিপ্ত করে।
- Key জানা থাকলে Dot; Key Variable-এ থাকলে বা বিশেষ অক্ষর থাকলে Bracket।
- Property যোগ ও বদল একইভাবে লেখা হয়; `delete` সরায়; অস্তিত্ব যাচাইয়ে `Object.hasOwn()`।
- Method হলো Function-মানের Property; ভেতরে `this` হলো যে Object দিয়ে Call করা হয়েছে। Method Arrow Function দিয়ে লিখবেন না।
- `?.` ঐচ্ছিক Nested তথ্য নিরাপদে পড়ে; `??` বিকল্প মান দেয়।
- Destructuring Object ও Array থেকে মান আলাদা Variable-এ নেয়; Parameter-এও ব্যবহার করা যায়।
- `Object.keys`, `Object.values`, `Object.entries` Object থেকে Array তৈরি করে।
- Spread Copy Shallow; সম্পূর্ণ আলাদা Copy-র জন্য `structuredClone()`।
- `===` ঠিকানা তুলনা করে; `Object.freeze()`-ও Shallow।
- JSON হলো তথ্যের String-রূপ: `stringify` এবং `parse`। `undefined`, Function বাদ পড়ে; তারিখ String হয়ে যায়।
- আগে Data Model, পরে Code।

পরের অধ্যায়ে String এবং লেখা প্রক্রিয়াকরণ: খোঁজা, বদলানো, ভাগ করা, Unicode এবং বাংলা লেখার বিশেষ সমস্যা।

৭.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Property	Object-এর একটি নাম-মানের জোড়া।
Key / Value	Property-র নাম / তার মান।
Object Literal	`{ }` দিয়ে লেখা Object।
Shorthand Property	Key ও Variable একই নামে হলে একবার লেখা।
Computed Key	`[ ]` -এর Expression থেকে তৈরি Key।
Method	Function-মানের Property।
`this`	Method Call-এর সময় যে Object দিয়ে Call করা হয়েছে।
Optional Chaining	`?.` : মান না থাকলে থেমে `undefined` দেওয়া।
Destructuring	Object বা Array থেকে মান আলাদা Variable-এ নেওয়া।
Shallow Copy	শুধু প্রথম স্তর Copy; ভেতরের Object-এর ঠিকানা ভাগাভাগি হয়।
Deep Copy	সব স্তরসহ সম্পূর্ণ আলাদা Copy।
JSON	তথ্যকে String হিসেবে প্রকাশের মান-নির্ধারিত রূপ।
Data Model	একটি জিনিসের তথ্যের গঠন ও নিয়মের নকশা।

৭.২৩ আরও পড়ুন

- **MDN — Working with objects:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects` — Object-এর পূর্ণ নির্দেশিকা।
- **MDN — Destructuring:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring` — Destructuring-এর সব রূপ।
- **MDN — Optional chaining:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining` — `?.` -এর নিয়ম।
- **MDN — this:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this` — `this` -এর আচরণ।
- **MDN — JSON:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON` — `JSON.stringify` ও `JSON.parse`।

অধ্যায় ৮

০৮

Strings এবং Modern Text Processing

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- String লেখার তিনটি পদ্ধতি এবং Escape Sequence ব্যবহার করতে।
- `length` আসলে কী গোনে, এবং বাংলা লেখা ও Emoji-তে কেন তা চোখে দেখা অক্ষরের সংখ্যা নয়, তা ব্যাখ্যা করতে।
- `at()`, `slice()`, `includes()`, `indexOf()`, `startsWith()` ও `endsWith()` দিয়ে লেখার অংশ পড়তে ও খুঁজতে।
- `trim()`, `replace()`, `replaceAll()`, `padStart()` দিয়ে লেখা পরিষ্কার ও সাজাতে।
- `split()` ও `join()` দিয়ে লেখাকে শব্দে ভাগ করতে এবং আবার জোড়া লাগাতে।
- Unicode Normalization কেন প্রয়োজন, বিশেষত বাংলা লেখার তুলনায়, তা বুঝতে।
- Regular Expression দিয়ে সরল Pattern খুঁজতে, বদলাতে এবং যাচাই করতে।
- ব্যবহারকারীর লেখা Page-এ নিরাপদে দেখাতে এবং XSS-এর ঝুঁকি চিনতে।
- একটি সম্পূর্ণ **Content Search Utility** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭ সম্পন্ন করা: Object, `Object.entries()` এবং Destructuring।
- অধ্যায় ৬-এর Array Method: `map`, `filter`, `join`।
- অধ্যায় ১-এর `textContent` বনাম `innerHTML`-এর প্রাথমিক ধারণা।
- `chapter-08` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

ব্যবহারকারীর নাম, Email, ঠিকানা, খোঁজার শব্দ, একটি লেখার শিরোনাম, Server-এর বার্তা—Program-এ আসা তথ্যের বড় অংশই লেখা। প্রায় প্রতিটি Program-কে লেখা পরিষ্কার করতে, ভাগ করতে, খুঁজতে এবং যাচাই করতে হয়।

এর অনেকটাই সহজ, কিন্তু কিছু ফাঁদ আছে যা ইংরেজি-ভাষী প্রোগ্রামাররা প্রায়ই চোখে পান না। `'ক্ষমা'`-র `length` কত? দেখতে একই রকম দুটি বাংলা শব্দ কেন `===` দিয়ে সমান হয় না? কেন খোঁজার শব্দ লেখায় থাকা সত্ত্বেও Program তা খুঁজে পায় না? এই অধ্যায়ে এসব প্রশ্নের উত্তর আছে, কারণ বাংলা লেখা নিয়ে কাজ করা Program-এর জন্য এগুলো প্রান্তিক বিষয় নয়।

শেষে একটি গুরুত্বপূর্ণ নিরাপত্তার বিষয়: ব্যবহারকারীর লেখা যদি অসাবধানে Page-এ বসানো হয়, তার ভেতরের Code চলে যেতে পারে। ওয়েবের সবচেয়ে প্রচলিত আক্রমণগুলোর একটি এভাবেই ঘটে।

৮.১ String কী

String হলো অক্ষরের একটি ক্রম। এটি একটি Primitive (অধ্যায় ২), এবং এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য আছে: **String বদলানো যায় না** (Immutable)।

immutable.js · JavaScript — চলবে: Browser Console ও Node.js

```
let city = 'dhaka';

city[0] = 'D';
console.log(city);

const upper = city.toUpperCase();
console.log(upper);
console.log(city);

city = upper;
console.log(city);
```

Output

```
dhaka
DHAKA
dhaka
DHAKA
```

`city[0] = 'D'` কোনো কাজ করেনি, এবং Error-ও দেয়নি। `toUpperCase()` মূল String বদলায়নি; একটি নতুন String ফেরত দিয়েছে। এই অধ্যায়ের সব String Method এভাবেই কাজ করে: ফল চাইলে নতুন String-টি একটি Variable-এ রাখতে হয়। শেষ দুই লাইনে `city` নামটিকে নতুন String-এর দিকে সরানো হয়েছে; String নিজে বদলায়নি।

৮.২ String লেখা

String তিনভাবে লেখা যায়: Single Quote, Double Quote এবং Backtick (অধ্যায় ১)। Backtick-এর String-কে **Template Literal** বলে। এর দুটি বিশেষ সুবিধা: `${ }` দিয়ে মান বসানো, এবং একাধিক লাইনে লেখা।

template-literal.js · JavaScript চলবে: Browser Console ও Node.js

```
const name = 'Sara';
const marks = 88;

console.log(`${name} পেয়েছেন ${marks}, Grade ${marks >= 80 ? 'A+' : 'A'}`);

const receipt = `দোকান: Boi Ghar
মোট: ${120 * 3} টাকা`;
console.log(receipt);
```

Output

```
Sara পেয়েছেন 88, Grade A+
দোকান: Boi Ghar
মোট: 360 টাকা
```

`${ }`-এর ভেতরে যেকোনো Expression বসানো যায়: Variable, হিসাব, এমনকি Ternary।

Escape Sequence

কিছু অক্ষর সরাসরি টাইপ করা যায় না, অথবা String-এর নিয়ম ভেঙে দেয়। সেগুলো লেখা হয় Backslash (`\`) দিয়ে শুরু হওয়া **Escape Sequence** দিয়ে।

escapes.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('প্রথম লাইন\nদ্বিতীয় লাইন');
console.log('It\'s a book');
console.log('C:\\Users\\Sara');
console.log('\u0995\u09BE');
```

Output

```
প্রথম লাইন
দ্বিতীয় লাইন
It's a book
C:\Users\Sara
কা
```

Escape	অর্থ
`\n`	নতুন লাইন
`\t`	Tab
`\'` এবং `\"`	String-এর ভেতরে Quotation Mark
`\\`	একটি Backslash
`\uXXXX`	Unicode নম্বর দিয়ে একটি অক্ষর

শেষ লাইনে `\u0995` হলো `ক` এবং `\u09BE` হলো `া`; একসঙ্গে `কা`। প্রতিটি অক্ষরের এমন একটি নম্বর আছে, এবং পরের অনুচ্ছেদে এই নম্বরগুলোই গুরুত্বপূর্ণ হয়ে উঠবে।

৮.৩ Length, Index এবং Unicode

`length` মনে হয় অক্ষরের সংখ্যা দেয়। ইংরেজিতে সাধারণত তা-ই। বাংলায় এবং Emoji-তে নয়।

length.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('Dhaka'.length);
console.log('ঢাকা'.length);
console.log('ক্ষমা'.length);

const smile = '\u{1F600}';

console.log(smile.length);
console.log([...smile].length);
```

Output

```
5
4
5
2
1
```

কেন এমন হয়, তা বুঝতে তিনটি স্তর আলাদা করতে হবে:

স্তর	কী	'ক্ষমা' -তে
Code Unit	JavaScript যা দিয়ে String রাখে (UTF-16)	৫
Code Point	Unicode-এর একটি অক্ষর-নম্বর	৫: ক, ◌্, ষ, ম, ◌া
Grapheme	পাঠকের চোখে একটি অক্ষর	২: ক্ষ, মা

- **`length` গোনে Code Unit।** বেশির ভাগ অক্ষর একটি Code Unit, কিন্তু Emoji-র মতো কিছু অক্ষর দুটি। তাই হাসিমুখ Emoji `smile`-এর `length` হলো `2`। (Emoji-টি এখানে Unicode নম্বর `\u{1F600}` দিয়ে লেখা; পাঁচ অঙ্কের নম্বরের জন্য `{ }` লাগে।)
- **`[...text]` গোনে Code Point।** Spread একটি String-কে Code Point ধরে ভাগ করে, তাই Emoji একটিই।
- **বাংলায় Code Point আর চোখে দেখা অক্ষর এক নয়।** `ক্ষ` তিনটি Code Point দিয়ে তৈরি: `ক`, হসন্ত (`◌্`) এবং `ষ`। `মা` দুটি: `ম` এবং আ-কার। কার-চিহ্ন, হসন্ত, চন্দ্রবিন্দু—সবই আলাদা Code Point।

চোখে দেখা অক্ষর গুনতে `Intl.Segmenter` ব্যবহার করুন। এটি আধুনিক Browser ও Node.js-এ আছে।

graphemes.js · JavaScript — চলবে: Browser Console ও Node.js

```
const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
const text = 'ক্ষমা';
const letters = [...segmenter.segment(text)].map((part) => part.segment);

console.log(letters.length);
console.log(letters.join(' | '));
```

Output

```
2
ক্ষ | মা
```

কখন কোনটি দরকার? Database-এর একটি ঘরে কত জায়গা লাগবে—Code Unit (`length`)। "নাম কমপক্ষে ২ অক্ষর" জাতীয় নিয়ম, যা মানুষ পড়ে বিচার করবে—Grapheme। কোনো লেখা অক্ষর ধরে কাটতে হলে Grapheme-এর সীমায় কাটুন, নইলে একটি যুক্তাক্ষরের অর্ধেক বা একটি Emoji-র অর্ধেক কেটে অর্থহীন চিহ্ন তৈরি হয়।

৮.৪ অংশ পড়া: at এবং slice

String-এর Index ঠিক Array-র মতো শূন্য থেকে শুরু (অধ্যায় ৬)।

access.js · JavaScript — চলবে: Browser Console ও Node.js

```
const code = 'JS-2026-BD';

console.log(code[0]);
console.log(code.at(-1));
console.log(code.slice(3, 7));
console.log(code.slice(-2));
console.log(code.slice(0, 2));
```

Output

```
J
D
2026
BD
JS
```

`slice(start, end)` Array-র `slice`-এর মতোই: `start` থেকে, `end`-এর আগে পর্যন্ত। ঋণাত্মক সংখ্যা শেষ থেকে গোনে। পুরোনো Code-এ `substring()` ও `substr()` দেখা যায়; নতুন Code-এ `slice()` ব্যবহার করুন, কারণ এর আচরণ সবচেয়ে সামঞ্জস্যপূর্ণ।

> **সতর্কতা:** Index এবং `slice` Code Unit ধরে কাজ করে। `'ক্ষমা'.slice(0, 2)` দেয় `'ক্'`—একটি অসম্পূর্ণ যুক্তাক্ষর। বাংলা লেখা অক্ষর ধরে কাটতে হলে আগে `Intl.Segmenter` দিয়ে ভাগ করুন।

৮.৫ খোঁজা

search.js · JavaScript — চলবে: Browser Console ও Node.js

```
const title = 'JavaScript শেখার সহজ উপায়';

console.log(title.includes('শেখার'));
console.log(title.startsWith('Java'));
console.log(title.endsWith('উপায়'));
console.log(title.indexOf('Script'));
console.log(title.indexOf('Python'));
console.log(title.includes('javascript'));
```

Output

```
true
true
true
4
-1
false
```

Method	ফেরত দেয়
`includes(text)`	আছে কি না (`true` / `false`)
`startsWith(text)` / `endsWith(text)`	এ দিয়ে শুরু / শেষ কি না
`indexOf(text)`	প্রথম অবস্থানের Index; না পেলে `-1`
`lastIndexOf(text)`	শেষ অবস্থানের Index; না পেলে `-1`

শেষ লাইনটি লক্ষ করুন: সব খোঁজা **Case-sensitive**। `'javascript'` আর `'JavaScript'` আলাদা। ব্যবহারকারীর খোঁজার শব্দ সাধারণত বড়-ছোট হাতের অক্ষর নির্বিশেষে মেলানো উচিত; তার উপায় অনুচ্ছেদ ৮.৭-এ।

`indexOf()` -এর দ্বিতীয় Argument দিয়ে বলা যায় কোন Index থেকে খোঁজা শুরু হবে। এটি দিয়ে একটি শব্দের সব অবস্থান খুঁজে বের করা যায়, যা Project-এ কাজে লাগবে।

৮.৬ পরিষ্কার করা ও সাজানো

clean.js · JavaScript — চলবে: Browser Console ও Node.js

```
const raw = '   Sara    Ahmed   ';

console.log(`[${raw.trim()}]`);
console.log(`[${raw.trimStart()}]`);

const price = '1,250,000';
console.log(price.replace(',', ''));
console.log(price.replaceAll(',', ''));

console.log('7'.padStart(3, '0'));
console.log('-'.repeat(10));
```

Output

```
[Sara    Ahmed]
[Sara    Ahmed   ]
1250,000
1250000
007
----------
```

- `trim()` শুধু শুরু ও শেষের ফাঁকা জায়গা সরায়; মাঝের বাড়তি ফাঁকা জায়গা থেকে যায়। `trimStart()` ও `trimEnd()` এক প্রান্তের জন্য।
- `replace(a, b)` শুধু **প্রথম** মিলটি বদলায়। এটি একটি প্রচলিত ভুলের উৎস।
- `replaceAll(a, b)` সব মিল বদলায়।
- `padStart(length, fill)` বাঁ দিকে অক্ষর বসিয়ে নির্দিষ্ট দৈর্ঘ্যে আনে: রোল নম্বর, রসিদ নম্বর বা সময় (`'07'`) দেখাতে কাজে লাগে। `padEnd()` ডান দিকে।
- `repeat(n)` String-কে `n` বার পুনরাবৃত্তি করে।

মাঝের বাড়তি ফাঁকা জায়গা সরাতে Regular Expression লাগে (অনুচ্ছেদ ৮.১০)।

৮.৭ Case Conversion এবং তুলনা

case.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('Dhaka'.toUpperCase());
console.log('DHAKA'.toLowerCase());
console.log('ঢাকা'.toUpperCase());

const query = 'DHAKA';
const city = 'Dhaka';
console.log(city.toLowerCase() === query.toLowerCase());

console.log('I'.toLowerCase());
console.log('I'.toLocaleLowerCase('tr'));
```

Output

```
DHAKA
dhaka
ঢাকা
true
i
ı
```

বড়-ছোট হাতের অক্ষর নির্বিশেষে দুটি লেখা মেলাতে **দুই পাশকেই** একই রূপে আনুন, তারপর তুলনা করুন। শুধু এক পাশ বদলালে কাজ হয় না।

বাংলা লিপিতে বড়-ছোট হাতের অক্ষরের ধারণা নেই, তাই `toUpperCase()` বাংলা লেখা অপরিবর্তিত রাখে। শেষ দুই লাইন দেখায় যে সব ভাষায় নিয়ম এক নয়: তুর্কি ভাষায় `I`-এর ছোট হাতের রূপ বিন্দুবিহীন `ı`। ব্যবহারকারীর ভাষা জানা থাকলে `toLocaleLowerCase()` ব্যবহার করুন।

ক্রম অনুযায়ী সাজানো

অধ্যায় ৩-এ দেখেছেন, `<` দিয়ে String-এর তুলনা হয় Unicode নম্বর ধরে, যা মানুষের অভিধানের ক্রম নয়। ভাষার নিয়ম মেনে সাজাতে `localeCompare()` ব্যবহার করুন।

locale-compare.js · JavaScript — চলবে: Node.js

```
const words = ['cherry', 'Banana', 'apple'];

console.log([...words].sort());
console.log([...words].sort((a, b) => a.localeCompare(b)));
```

Output

```
[ 'Banana', 'apple', 'cherry' ]
[ 'apple', 'Banana', 'cherry' ]
```

সাধারণ `sort()` Unicode নম্বর ধরে সাজায়, আর সেখানে সব বড় হাতের অক্ষর ছোট হাতের অক্ষরের আগে; তাই `'Banana'` প্রথমে এসেছে। `localeCompare()` মানুষের অভিধানের মতো সাজিয়েছে। বাংলা নাম সাজাতে `a.localeCompare(b, 'bn')` লিখে ভাষাটিও বলে দিতে পারেন।

৮.৮ split এবং join: শব্দে ভাগ করা

`split()` একটি String-কে নির্দিষ্ট বিভাজক ধরে ভেঙে Array তৈরি করে; `join()` (অধ্যায় ৬) উল্টো কাজ করে।

split-join.js · JavaScript — চলবে: Node.js

```
console.log('আম,কলা,লিচু'.split(','));
console.log('a,b,,c'.split(','));
console.log('Sara  Ahmed'.split(' '));
console.log('2026-10-15'.split('-').map(Number));
console.log(['JavaScript', 'শিখুন'].join(' '));
```

Output

```
[ 'আম', 'কলা', 'লিচু' ]
[ 'a', 'b', '', 'c' ]
[ 'Sara', '', 'Ahmed' ]
[ 2026, 10, 15 ]
JavaScript শিখুন
```

- দুটি বিভাজক পাশাপাশি থাকলে মাঝখানে একটি ফাঁকা String তৈরি হয় (দ্বিতীয় ও তৃতীয় লাইন)।
- `split('-').map(Number)` একটি তারিখ-String থেকে সংখ্যার Array তৈরি করেছে। `Number` এখানে একটি Function হিসেবে `map`-এ পাঠানো হয়েছে (অধ্যায় ৫)।

> **সতর্কতা:** `split(',')` একটি পূর্ণাঙ্গ CSV Reader নয়। একটি ঘরের মানের ভেতরেই কমা থাকলে (`"Dhaka, Bangladesh"`) ভুল ভাগ হবে। বাস্তব CSV File-এর জন্য উপযুক্ত Library ব্যবহার করুন।

একটি লেখাকে শব্দে ভাগ করাকে **Tokenization** বলে। শুধু ফাঁকা জায়গা ধরে ভাগ করলে বিরামচিহ্ন শব্দের সঙ্গে লেগে থাকে (`'করুন।'`); নির্ভরযোগ্য Tokenization-এর জন্য Regular Expression লাগে, যা পরের দুই অনুচ্ছেদে আসছে।

৮.৯ Unicode Normalization

একই রকম দেখতে লেখা Unicode-এ একাধিক উপায়ে লেখা যায়। `é` একটি একক অক্ষর হতে পারে, আবার `e` এবং তার ওপরের একটি আলাদা চিহ্নের সমন্বয়ও হতে পারে। বাংলায়ও এমন আছে: `য়` একটি একক অক্ষর হিসেবে লেখা যায়, আবার `য` এবং নিচের বিন্দু (নুক্তা) হিসেবেও।

normalize.js · JavaScript — চলবে: Browser Console ও Node.js

```
const composed = '\u00E9';
const decomposed = 'e\u0301';

console.log(composed.length, decomposed.length);
console.log(composed === decomposed);
console.log(composed.normalize('NFC') === decomposed.normalize('NFC'));

const ya1 = '\u09DF';
const ya2 = '\u09AF\u09BC';

console.log(ya1 === ya2, ya1.length, ya2.length);
console.log(ya1.normalize('NFC') === ya2.normalize('NFC'));
```

Output

```
1 2
false
true
false 1 2
true
```

পর্দায় দুটি একই দেখায় (`é`), কিন্তু প্রথমটি একটি Code Point এবং দ্বিতীয়টি দুটি; ভেতরের Code Point আলাদা, তাই `===` মিথ্যা। একজন ব্যবহারকারী একটি Keyboard দিয়ে নাম লিখে নিবন্ধন করলেন, পরে অন্য Keyboard দিয়ে সেই নাম লিখে খুঁজলেন—Program বলবে "পাওয়া যায়নি"। বাংলা লেখা নিয়ে কাজ করা Program-এ এটি বাস্তব সমস্যা।

Normalization লেখাকে একটি নির্দিষ্ট আদর্শ রূপে আনে। `normalize('NFC')` সবচেয়ে প্রচলিত রূপ। নিয়ম: **তুলনা, খোঁজা বা সংরক্ষণের আগে দুই পাশকেই একই রূপে Normalize করুন।**

রূপ	অর্থ
NFC	যেখানে সম্ভব, অক্ষর ও চিহ্ন জোড়া লাগানো; সাধারণ কাজে এটিই
NFD	অক্ষর ও চিহ্ন আলাদা করা
NFKC / NFKD	এর সঙ্গে কিছু প্রতীককে সাধারণ অক্ষরে রূপান্তর; অর্থ বদলে যেতে পারে, তাই সাবধানে

একটি মজার ব্যতিক্রম: `য়` -এর ক্ষেত্রে Unicode-এর নিয়মে NFC জোড়া লাগানো একক রূপটি নয়, বরং `য` ও নুক্তার দুই-অংশের রূপটিকেই আদর্শ ধরে। তাতে আপনার কিছু করার নেই: দুই পাশ একই Function দিয়ে Normalize করলে তুলনা ঠিকই মেলে। কোন রূপটি আদর্শ, তা মুখস্থ রাখার দরকার নেই।

৮.১০ Regular Expression-এর পরিচয়

Regular Expression (সংক্ষেপে Regex) হলো লেখার একটি Pattern বা ছাঁচ বর্ণনা করার ভাষা। "যেকোনো সংখ্যা", "এক বা একাধিক ফাঁকা জায়গা", "শুরুতে 01, তারপর ৯টি সংখ্যা"—এমন বর্ণনা Regex দিয়ে লেখা যায়। দুটি Slash-এর মাঝে Pattern লেখা হয়, শেষে ঐচ্ছিক **Flag**।

regex-intro.js · JavaScript চলবে: Node.js

```
const text = 'Order 12 এবং Order 345';

console.log(/\d+/.test(text));
console.log(text.match(/\d+/g));
console.log(text.replace(/\d+/g, '#'));
console.log('Sara    Ahmed'.replace(/\s+/g, ' '));
console.log(/^order/i.test(text));
```

Output

```
true
[ '12', '345' ]
Order # এবং Order #
Sara Ahmed
true
```

অংশ	অর্থ
`\d`	একটি সংখ্যা (০–৯, ইংরেজি)
`\s`	একটি ফাঁকা জায়গা (Space, Tab, নতুন লাইন)
`+`	আগের জিনিসটি এক বা একাধিকবার
`{n}`	আগের জিনিসটি ঠিক `n` বার
`^` / `$`	লেখার শুরু / শেষ
`[abc]`	এদের যেকোনো একটি
Flag `g`	সব মিল খোঁজা (Global)
Flag `i`	বড়-ছোট হাতের অক্ষর নির্বিশেষে
Flag `u`	Unicode-সচেতন Pattern

- **`regex.test(text)`** — মিল আছে কি না।
- **`text.match(regex)`** — `g` Flag-সহ সব মিলের Array; কিছু না মিললে `null`।
- **`text.replace(regex, replacement)`** — `g` থাকলে সব মিল বদলায়। অনুচ্ছেদ ৮.৬-এর মাঝের বাড়তি ফাঁকা জায়গা `/\s+/g` দিয়ে একটিতে আনা হয়েছে।

বাংলা শব্দ খোঁজা

`\w` শুধু ইংরেজি অক্ষর, সংখ্যা ও `_` চেনে; বাংলা চেনে না। Unicode-সচেতন Pattern-এ `\p{L}` (যেকোনো ভাষার অক্ষর), `\p{M}` (কার-চিহ্ন, হসন্তের মতো যুক্ত-চিহ্ন) এবং `\p{N}` (সংখ্যা) ব্যবহার করা যায়। এগুলোর জন্য `u` Flag বাধ্যতামূলক।

unicode-words.js · JavaScript চলবে: Node.js

```
const sentence = 'JavaScript শিখুন, প্রতিদিন ১০ মিনিট!';

console.log(sentence.match(/\w+/g));
console.log(sentence.match(/[\p{L}\p{M}\p{N}]+/gu));
```

Output

```
[ 'JavaScript' ]
[ 'JavaScript', 'শিখুন', 'প্রতিদিন', '১০', 'মিনিট' ]
```

`\w` বাংলা শব্দগুলো পুরোপুরি বাদ দিয়েছে। Unicode Pattern সব শব্দ এবং বাংলা সংখ্যাও পেয়েছে, আর বিরামচিহ্ন বাদ দিয়েছে। `\p{M}` বাদ দিলে কার-চিহ্নে এসে শব্দ ভেঙে যেত (`'শ'`, `'খ'`...)।

Regular Expression একটি বিস্তৃত বিষয়; বিস্তারিত আলোচনা অধ্যায় ১৭-এ। আপাতত জেনে রাখুন: Regex শক্তিশালী, কিন্তু জটিল Pattern পড়া ও ঠিক রাখা কঠিন। সহজ String Method দিয়ে কাজ হলে সেটিই ব্যবহার করুন।

৮.১১ Text যাচাই

ব্যবহারকারীর লেখা যাচাইয়ের সাধারণ ধাপ: Normalize ও `trim` করা, তারপর দৈর্ঘ্য, তারপর গঠন।

validation.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isBdMobile = (text) => /^01[3-9]\d{8}$/.test(text.trim());
const looksLikeEmail = (text) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(text.trim());

console.log(isBdMobile('01712345678'));
console.log(isBdMobile('0171234567'));
console.log(isBdMobile('02712345678'));
console.log(looksLikeEmail('sara@example.com'));
console.log(looksLikeEmail('sara@example'));
console.log(looksLikeEmail('sara @example.com'));
```

Output

```
true
false
false
true
false
false
```

- **মোবাইল নম্বর:** শুরুতে `01`, তারপর ৩ থেকে ৯-এর একটি সংখ্যা, তারপর ঠিক ৮টি সংখ্যা; `^` ও `$` নিশ্চিত করে যে পুরো লেখাটিই এই গঠনের, আগে-পরে বাড়তি কিছু নেই।
- **Email:** এই Pattern শুধু মোটা দাগের গঠন দেখে: `@`-এর আগে ও পরে কিছু আছে, এবং পরে একটি ডট। Email-এর পূর্ণ নিয়ম অত্যন্ত জটিল; কোনো Regex দিয়েই প্রমাণ হয় না যে ঠিকানাটি আসলে আছে। সেটি জানার একমাত্র উপায় সেখানে একটি যাচাই-বার্তা পাঠানো।

> **নোট:** উদাহরণের মোবাইল নম্বরের নিয়ম শেখানোর জন্য সরলীকৃত। বাস্তব Program-এ দেশের Code (`+880`) এবং মাঝের হাইফেন বা ফাঁকা জায়গা আগে সরিয়ে নিয়ে তবেই যাচাই করুন।

৮.১২ নিরাপদ Rendering: XSS

অধ্যায় ১-এ বলা হয়েছিল, ব্যবহারকারীর লেখা Page-এ দেখাতে `textContent` ব্যবহার করুন, `innerHTML` নয়। এবার কারণটি নিজের চোখে দেখুন। উদাহরণটি Browser-এ চালান।

xss-demo.js · JavaScript — চলবে: Browser

```
const userText = 'Hello <img src="" onerror="console.log(\'XSS: Code চলে গেছে!\')">';

const unsafe = document.createElement('p');
unsafe.innerHTML = userText;

const safe = document.createElement('p');
safe.textContent = userText;

console.log(safe.innerHTML);
console.log(safe.children.length, unsafe.children.length);
```

Output

```
Hello &lt;img src="" onerror="console.log('XSS: Code চলে গেছে!')"&gt;
0 1
XSS: Code চলে গেছে!
```

ব্যবহারকারী একটি "নাম" লিখেছেন, যার ভেতরে একটি `<img>` Tag লুকানো। `innerHTML` লেখাটিকে HTML হিসেবে পড়েছে: একটি সত্যিকারের ছবি-Element তৈরি হয়েছে, ছবিটি Load হতে ব্যর্থ হয়েছে, এবং `onerror` -এর Code চলে গেছে। এখানে Code শুধু একটি বার্তা দেখিয়েছে; আক্রমণকারী একই জায়গায় ব্যবহারকারীর Session চুরি বা তাঁর পক্ষে কাজ করার Code বসাতে পারেন। এই আক্রমণের নাম **Cross-Site Scripting (XSS)**।

`textContent` একই লেখাকে সাধারণ লেখা হিসেবে রেখেছে: Element-এর ভেতরে কোনো Tag তৈরি হয়নি (`0`), এবং `<` ও `>` নিরাপদ রূপ `<` ও `>` -এ পরিণত হয়েছে।

খোঁজার শব্দ চিহ্নিত করা

Search Result-এ খোঁজার শব্দটি গাঢ় করে দেখানো একটি সাধারণ কাজ, এবং XSS-এর একটি সাধারণ উৎস: লেখার মাঝে `<mark>` জুড়ে দিয়ে `innerHTML` -এ বসানো। নিরাপদ উপায় হলো লেখাকে টুকরো করে প্রতিটি টুকরো `textContent` দিয়ে বসানো, এবং শুধু মিলে যাওয়া টুকরোর জন্য `<mark>` Element তৈরি করা। Project-এর `highlight` Function লেখাকে এমন টুকরোয় ভাগ করার কাজটিই করে; Page-এ বসানোর Code অধ্যায় ৯-এ আসবে।

নিরাপত্তা প্রেক্ষাপট-নির্ভর: HTML-এর ভেতরে যা নিরাপদ, URL, CSS বা Database Query-র ভেতরে তা নিরাপদ নয়। প্রতিটি প্রেক্ষাপটের নিয়ম অধ্যায় ২০-এ আলোচনা করা হয়েছে।

৮.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Method-এর ফল কোথাও না রাখা	String বদলায় না	`text = text.trim()`
`replace()` দিয়ে সব বদলানোর চেষ্টা	শুধু প্রথমটি বদলায়	`replaceAll()` বা Regex-এ `g`
শুধু এক পাশ ছোট হাতের অক্ষরে এনে তুলনা	মিল পাওয়া যায় না	দুই পাশই একই রূপে আনুন
Normalization ছাড়া বাংলা লেখা তুলনা	একই দেখতে শব্দ "আলাদা"	দুই পাশেই `normalize('NFC')`
`length` দিয়ে বাংলা অক্ষর গোনা	প্রত্যাশার চেয়ে বেশি সংখ্যা	`Intl.Segmenter`
`slice` দিয়ে বাংলা লেখা কাটা	অসম্পূর্ণ যুক্তাক্ষর	Grapheme ধরে কাটুন
`\w` দিয়ে বাংলা শব্দ খোঁজা	বাংলা বাদ পড়ে	`/[\p{L}\p{M}\p{N}]+/gu`
Regex-এ `\p{...}` কিন্তু `u` Flag নেই	`SyntaxError` বা ভুল মিল	`u` Flag দিন
`match()`-এর ফল যাচাই ছাড়া ব্যবহার	মিল না থাকলে `null`-এর ওপর Error	`?? []`
ব্যবহারকারীর লেখা `innerHTML`-এ	XSS	`textContent`
`split(',')`-কে CSV Reader ভাবা	উদ্ধৃত মানের ভেতরের কমায় ভুল ভাগ	উপযুক্ত CSV Library

৮.১৪ Debugging নির্দেশিকা

লেখা-সংক্রান্ত সবচেয়ে বিভ্রান্তিকর Bug হলো দুটি লেখা দেখতে একই, কিন্তু Program বলে আলাদা। সমাধান: অক্ষরগুলোর Unicode নম্বর দেখা।

debug-text.js · JavaScript — চলবে: Browser Console ও Node.js

```
const showCodes = (text) =>
  [...text].map((c) => c.codePointAt(0).toString(16).padStart(4, '0')).join(' ');

console.log(showCodes('\u09DF'));
console.log(showCodes('\u09AF\u09BC'));
const typed = 'Sara\u00A0Ahmed ';

console.log(JSON.stringify('Sara '));
console.log(typed.split(' ').length);
console.log(showCodes(typed.slice(4, 5)));
```

Output

```
09df
09af 09bc
"Sara "
2
00a0
```

`codePointAt(0)` একটি অক্ষরের নম্বর দেয়, আর `toString(16)` সেটি Unicode-এর প্রচলিত ষোলোভিত্তিক রূপে দেখায়। এখন স্পষ্ট যে দুটি "য়" আলাদা। তৃতীয় লাইনে `JSON.stringify` লেখাটিকে Quotation Mark-এ ঘিরে দেখায়, ফলে শেষের ফাঁকা জায়গাটি চোখে পড়ে। কিন্তু `typed`-এর মাঝের ফাঁকা জায়গাটি আসলে একটি Non-breaking Space (`\u00A0`), যা সাধারণ Space-এর মতো দেখায়, এমনকি `JSON.stringify`-তেও। চতুর্থ লাইনে তা ধরা পড়েছে: দুটি Space থাকলে `split(' ')` তিনটি অংশ দিত, দিয়েছে দুটি। শেষ লাইনে `showCodes` অক্ষরটির আসল নম্বর `00a0` দেখিয়েছে। লুকানো অক্ষর ধরার নিশ্চিত উপায় এটিই।

লেখা নিয়ে কাজ প্রত্যাশামতো না হলে:

১. লেখাটি `JSON.stringify()` দিয়ে দেখুন: আগে-পরের ফাঁকা জায়গা এবং নতুন লাইনের মতো লুকানো অক্ষর ধরা পড়ে।
২. দেখতে একই দুটি লেখা না মিললে দুটির Code Point নম্বর মিলিয়ে দেখুন।
৩. তুলনা বা খোঁজার আগে দুই পাশে একই ক্রমে `normalize('NFC')`, `trim()` এবং `toLowerCase()` হয়েছে কি না দেখুন।
৪. সংখ্যা ভুল হলে দেখুন কী গুনছেন: Code Unit, Code Point, না Grapheme।
৫. Regex না মিললে একটি সরল লেখা দিয়ে আলাদাভাবে পরীক্ষা করুন, এবং Flag (`g`, `i`, `u`) মিলিয়ে নিন।
৬. Page-এ লেখার জায়গায় অপ্রত্যাশিত Element তৈরি হলে খুঁজুন কোথায় `innerHTML` ব্যবহার হয়েছে।

৮.১৫ পেশাদার অভ্যাস

- **বাইরের লেখা প্রথমেই পরিষ্কার করুন:** `normalize('NFC')`, তারপর `trim()`। সংরক্ষণের আগে একবার, যাতে পরে প্রতিবার না করতে হয়।
- **তুলনার জন্য একটি Function রাখুন,** যা Normalize, `trim` ও ছোট হাতের অক্ষরে আনার কাজ একসঙ্গে করে, এবং সব জায়গায় সেটিই ব্যবহার করুন।
- **গণনার ধরন সচেতনভাবে বেছে নিন:** সংরক্ষণের সীমা Code Unit-এ, মানুষের জন্য সীমা Grapheme-এ।
- **বাংলা লেখার জন্য Unicode-সচেতন Regex** (`u` Flag এবং `\p{...}`)।
- **সহজ Method আগে, Regex পরে।** `startsWith('01')` পড়তে `/^01/`-এর চেয়ে সহজ।
- **ব্যবহারকারীর লেখা সব সময় `textContent` দিয়ে।**
- **Template Literal দিয়ে লেখা গঠন করুন;** `+` দিয়ে লম্বা জোড়া লাগানোর চেয়ে পড়া সহজ।
- **Regex-কে নাম দিন বা Comment দিন;** `/^01[3-9]\d{8}$/` এক মাস পরে নিজেও বুঝতে কষ্ট হবে।

৮.১৬ Project: Content Search Utility

একজন লেখক বাংলা ও ইংরেজি মেশানো একটি লেখা তৈরি করছেন। তাঁর জন্য একটি সহায়ক Program লিখবেন: শব্দ ও অক্ষর গণনা, Web Address-এর জন্য Slug, একটি ছোট সারাংশ, সবচেয়ে বেশি ব্যবহৃত শব্দ, এবং লেখার ভেতরে একটি শব্দ খুঁজে চিহ্নিত করা।

Project-এর চাহিদা

- বাংলা, ইংরেজি এবং মেশানো লেখা সঠিকভাবে সামলাতে হবে।
- সব তুলনার আগে লেখা Unicode NFC-তে আনা হবে, এবং খোঁজা হবে বড়-ছোট হাতের অক্ষর নির্বিশেষে।
- অক্ষর গণনায় Code Point এবং চোখে দেখা অক্ষর (Grapheme)—দুটোই দেখাতে হবে।
- Slug: ছোট হাতের অক্ষর; শব্দগুলো হাইফেন দিয়ে জোড়া; বিরামচিহ্ন বাদ; বাংলা অক্ষর থাকবে।
- সারাংশ: নির্দিষ্ট দৈর্ঘ্যের মধ্যে, কোনো শব্দ মাঝখানে না কেটে, শেষে `...`।
- সবচেয়ে বেশি ব্যবহৃত শব্দ: সংখ্যাসহ, সমান সংখ্যার ক্ষেত্রে বর্ণানুক্রমে; খুব ছোট শব্দ বাদ।
- খোঁজার শব্দের সব অবস্থান, এবং লেখায় সেগুলো `[ ]` দিয়ে চিহ্নিত করা। কোনো HTML তৈরি হবে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
content-search-utility/
├── text-tools.js
├── README.md
└── tests/
    ├── load-script.js
    └── text-tools.test.js
```

text-tools.js

File-টি তিন অংশে দেখানো হলো; তিনটি মিলে সম্পূর্ণ File। অংশগুলো পরপর একই File-এ লিখুন। `// ...` লাইনগুলো শুধু অংশের সংযোগ বোঝায়; সেগুলো লিখতে হবে না।

অংশ ১: Normalization, শব্দ ও অক্ষর গণনা

text-tools.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const WORD = /[\p{L}\p{M}\p{N}]+/gu;

// তুলনার জন্য একই রূপ: NFC, ছোট হাতের অক্ষর, একটিমাত্র ফাঁকা জায়গা।
function normalizeText(text) {
  return String(text ?? '')
    .normalize('NFC')
```

```
    .toLowerCase()
    .replace(/\s+/g, ' ')
    .trim();
}

// লেখার শব্দগুলোর Array (বাংলা ও ইংরেজি)।
const toWords = (text) => normalizeText(text).match(WORD) ?? [];

// অক্ষর গণনা: Code Point এবং চোখে দেখা অক্ষর।
function countCharacters(text) {
  const clean = String(text ?? '').normalize('NFC');
  const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });

  return {
    codePoints: [...clean].length,
    graphemes: [...segmenter.segment(clean)].length,
  };
}

// ...
```

`normalizeText` অনুচ্ছেদ ৮.১৫-এর "তুলনার জন্য একটি Function"। চারটি ধাপ একটি Chain-এ: প্রতিটি Method নতুন String ফেরত দেয়, তাই পরের Method সরাসরি তার ওপর Call করা যায় (অধ্যায় ৬-এর Method Chaining)। `String(text ?? '')` নিশ্চিত করে যে `null` বা সংখ্যা এলেও কাজ চলবে।

`toWords`-এ `match` মিল না পেলে `null` দেয়, তাই `?? []`। ফলে ফাঁকা লেখায়ও সব সময় একটি Array পাওয়া যায়।

অংশ ২: Slug, সারাংশ এবং শব্দের ঘনত্ব

text-tools.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// URL-এর উপযোগী Slug: শব্দগুলো হাইফেন দিয়ে জোড়া।
const makeSlug = (title) => toWords(title).join('-');

// শব্দ না কেটে নির্দিষ্ট দৈর্ঘ্যের সারাংশ।
function makeExcerpt(text, maxLength = 60) {
  const clean = String(text ?? '').normalize('NFC').replace(/\s+/g, ' ').trim();

  if (clean.length <= maxLength) {
    return clean;
  }

  const cut = clean.slice(0, maxLength);
  const lastSpace = cut.lastIndexOf(' ');
  const base = lastSpace > 0 ? cut.slice(0, lastSpace) : cut;

  return `${base}...`;
```

```
}

// সবচেয়ে বেশি ব্যবহৃত শব্দ, কমপক্ষে minLength অক্ষরের।
function topWords(text, limit = 5, minLength = 3) {
  const counts = {};

  for (const word of toWords(text)) {
    if ([...word].length >= minLength) {
      counts[word] = (counts[word] ?? 0) + 1;
    }
  }

  return Object.entries(counts)
    .sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]))
    .slice(0, limit)
    .map(([word, count]) => `${word} (${count})`);
}

// ...
```

`makeSlug` আলাদা কোনো নিয়ম লেখেনি: `toWords` ইতিমধ্যে ছোট হাতের অক্ষরে আনে এবং বিরামচিহ্ন বাদ দেয়, তাই শুধু হাইফেন দিয়ে জোড়া লাগানোই যথেষ্ট। পুনর্ব্যবহারের সুফল এটিই।

`makeExcerpt` প্রথমে নির্দিষ্ট দৈর্ঘ্যে কাটে, তারপর শেষ ফাঁকা জায়গা পর্যন্ত পিছিয়ে আসে, যাতে শেষ শব্দটি অসম্পূর্ণ না থাকে। শব্দের মাঝে কাটলে বাংলায় যুক্তাক্ষর ভেঙে যেতে পারত (অনুচ্ছেদ ৮.৪); শব্দের সীমায় কাটলে সেই ঝুঁকি থাকে না।

`topWords` অধ্যায় ৭-এর গণনা-Object ধরন ব্যবহার করে: শব্দটি Key, সংখ্যা Value। তারপর `Object.entries` থেকে Array, এবং দুই স্তরের সাজানো: `b[1] - a[1]` বেশি সংখ্যা আগে আনে; সংখ্যা সমান হলে বিয়োগফল `0`, যা Falsy, তাই `||`-এর ডান পাশের `localeCompare` বর্ণানুক্রমে সাজায় (অধ্যায় ৩-এর Short-Circuit)।

অংশ ৩: খোঁজা, চিহ্নিত করা এবং উদাহরণ

text-tools.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// খোঁজার শব্দের সব অবস্থান (NFC লেখায় Index), বড়-ছোট হাতের অক্ষর নির্বিশেষে।
function findAll(text, query) {
  const haystack = String(text).normalize('NFC').toLowerCase();
  const needle = normalizeText(query);
  const positions = [];

  if (needle === '') {
    return positions;
  }

  let index = haystack.indexOf(needle);
```

```
  while (index !== -1) {
    positions.push(index);
    index = haystack.indexOf(needle, index + needle.length);
  }

  return positions;
}

// প্রতিটি মিল [ ] দিয়ে চিহ্নিত করে; HTML তৈরি করে না।
function highlight(text, query) {
  const clean = String(text).normalize('NFC');
  const size = normalizeText(query).length;
  let result = '';
  let last = 0;

  for (const index of findAll(clean, query)) {
    result += `${clean.slice(last, index)}[${clean.slice(index, index + size)}]`;
    last = index + size;
  }

  return result + clean.slice(last);
}

const article = `JavaScript শেখা শুরু করুন। JavaScript দিয়ে Browser ও Server
দুই জায়গাতেই কাজ করা যায়। প্রতিদিন একটু করে JavaScript অনুশীলন করুন।`;

const { codePoints, graphemes } = countCharacters('ক্ষমা');

console.log(`শব্দ: ${toWords(article).length}`);
console.log(`ক্ষমা: ${codePoints} Code Point, ${graphemes} অক্ষর`);
console.log(makeSlug('JavaScript শেখার প্রথম ধাপ!'));
console.log(makeExcerpt(article, 40));
console.log(topWords(article, 3).join(', '));
console.log(findAll(article, 'JAVASCRIPT').join(', '));
console.log(highlight('প্রতিদিন অনুশীলন করুন, নিয়মিত অনুশীলন করুন।', 'অনুশীলন'));
```

Output

```
শব্দ: 20
ক্ষমা: 5 Code Point, 2 অক্ষর
javascript-শেখার-প্রথম-ধাপ
JavaScript শেখা শুরু করুন। JavaScript...
javascript (3), করুন (2), browser (1)
0, 27, 107
প্রতিদিন [অনুশীলন] করুন, নিয়মিত [অনুশীলন] করুন।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **`findAll`-এর Loop।** `indexOf`-এর দ্বিতীয় Argument দিয়ে প্রতিবার আগের মিলের পর থেকে খোঁজা শুরু হয়; কিছু না পেলে `-1`, এবং `while` থামে। এটি অধ্যায় ৪-এর সেই `while`, যার শেষ আগে থেকে জানা নেই। প্রতিটি ধাপে `index` সামনে এগোয়, তাই Loop অবশ্যই শেষ হয়।
- **দুই পাশ একই রূপে।** লেখাটিকে NFC ও ছোট হাতের অক্ষরে আনা হয়েছে, খোঁজার শব্দটিকেও; তাই `'JAVASCRIPT'` তিনটি মিলই পেয়েছে।
- **`highlight` লেখাকে টুকরো করে।** প্রতিটি মিলের আগের অংশ, তারপর `[মিল]`, শেষে বাকি অংশ। মূল লেখার অক্ষরগুলোই বসানো হয়েছে (খোঁজার শব্দ নয়), তাই বড়-ছোট হাতের অক্ষর লেখায় যেমন ছিল তেমনই থাকে। Page-এ একই টুকরোগুলো `textContent` এবং `<mark>` Element দিয়ে বসালে XSS-এর ঝুঁকি থাকে না।
- **গণনা।** ২০টি শব্দ, কারণ বিরামচিহ্ন (`।`) শব্দ হিসেবে গোনা হয়নি। `ক্ষমা`-য় ৫টি Code Point কিন্তু চোখে ২টি অক্ষর।
- **সারাংশ।** ৪০ অক্ষরের সীমায় শেষ শব্দটি না কেটে পিছিয়ে এসে `...` যোগ হয়েছে।
- **শব্দের ঘনত্ব।** `javascript` তিনবার, `করুন` দুবার; বাকি সব একবার, তাই বর্ণানুক্রমে প্রথমটি, `browser`, এসেছে। তিন অক্ষরের কম শব্দ (`ও`) বাদ পড়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node text-tools.js
```

Call	যা পাওয়ার কথা
`makeSlug('JavaScript শেখার প্রথম ধাপ!')`	`javascript-শেখার-প্রথম-ধাপ`
`makeSlug('  Hello,  World  ')`	`hello-world`
`makeExcerpt('ছোট লেখা', 60)`	পুরো লেখাটিই, `...` ছাড়া
`countCharacters('ক্ষমা')`	Code Point 5, অক্ষর 2
`findAll(text, '')`	ফাঁকা তালিকা; Loop চলে না
`findAll('\u09DF', '\u09AF\u09BC')`	`[0]`; দুই রূপের য় মেলে
`highlight('Hello <b>', 'b')`	`Hello <[b]>`; কোনো HTML Element তৈরি হয় না

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে, যার কয়েকটি বিশেষভাবে বাংলা লেখা ও Normalization পরীক্ষা করে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
বাংলা শব্দ গোনা হয় না	Regex-এ `\w`	`/[\p{L}\p{M}\p{N}]+/gu`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
শব্দগুলো কার-চিহ্নে ভেঙে যায়	Pattern-এ `\p{M}` নেই	`\p{M}` যোগ করুন
`SyntaxError: Invalid regular expression`	`\p{...}` আছে কিন্তু `u` Flag নেই	`u` Flag
খোঁজার শব্দ লেখায় থাকা সত্ত্বেও পাওয়া যায় না	এক পাশ Normalize বা ছোট হাতের অক্ষরে আনা হয়নি	দুই পাশে একই Function
Program আটকে যায়	`while`-এ `index` এগোচ্ছে না, বা ফাঁকা শব্দে খোঁজা	`index + needle.length`; ফাঁকা শব্দে আগেই `return`
`TypeError: ... reading 'length'`	`match` থেকে `null`	`?? []`
সমান সংখ্যার শব্দের ক্রম প্রতিবার আলাদা	দ্বিতীয় স্তরের সাজানো নেই	`|| a[0].localeCompare(b[0])`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। খোঁজা আক্ষরিক: বানানভেদ (যেমন "শেখা" ও "শিখা"), শব্দের রূপভেদ বা সমার্থক শব্দ ধরে না। কিছু ভাষায় ছোট হাতের অক্ষরে আনলে লেখার দৈর্ঘ্য বদলে যায়; তখন চিহ্নিত করার অবস্থান সামান্য সরে যেতে পারে (বাংলা ও ইংরেজিতে এই সমস্যা হয় না)। Slug-এ বাংলা অক্ষর থাকে; কিছু পুরোনো Web System শুধু ইংরেজি Slug গ্রহণ করে। লেখা Program-এর ভেতরে দেওয়া; File থেকে পড়া হয় না।

৮.১৭ অনুশীলনী

প্রাথমিক স্তর

১. `'  JavaScript Mastery  '` থেকে আগে-পরের ফাঁকা জায়গা বাদ দিয়ে বড় হাতের অক্ষরে দেখান। মূল String-টি যে অপরিবর্তিত, তাও দেখান।

২. `'2026-10-15'` থেকে `slice` দিয়ে বছর, মাস ও দিন আলাদা করে দেখান।

৩. `'01712345678'`-এর শেষ তিনটি সংখ্যা ছাড়া বাকিগুলো `*` দিয়ে ঢেকে দেখান (`********678`)। `slice`, `repeat` বা `padStart` ব্যবহার করুন।

৪. না চালিয়ে বলুন: `'a-b-c'.replace('-', '+')` এবং `'a-b-c'.replaceAll('-', '+')`-এর ফল কী।

৫. নিচের Code-এ নাম মেলার কথা, কিন্তু মিলছে না। ভুলটি ঠিক করুন।

compare-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const saved = 'Sara Ahmed';
const typed = '  sara ahmed ';

console.log(saved.toLowerCase() === typed);
```

মধ্যম স্তর

৬. `countVowelSigns` নামে একটি Function লিখুন, যা একটি বাংলা লেখায় কয়টি আ-কার (া) আছে তা গোনে।

৭. `initials` নামে একটি Function লিখুন, যা পুরো নাম থেকে প্রতিটি অংশের প্রথম অক্ষর বড় হাতের করে ফেরত দেয় (`'sara ahmed khan'` থেকে `'SAK'`)। নামের মাঝে একাধিক ফাঁকা জায়গা থাকলেও যেন কাজ করে।

৮. `sameText` নামে একটি Function লিখুন, যা দুটি লেখাকে NFC, `trim` ও ছোট হাতের অক্ষরে এনে তুলনা করে। `'\u09DF'` ও `'\u09AF\u09BC'` দিয়ে পরীক্ষা করুন।

৯. `isValidUsername` নামে একটি Function লিখুন: ৩ থেকে ১৫ অক্ষর, শুধু ইংরেজি ছোট হাতের অক্ষর, সংখ্যা বা `_`, এবং অক্ষর দিয়ে শুরু।

চ্যালেঞ্জ স্তর

১০. **অক্ষর ধরে সারাংশ:** `shortenByLetters` নামে একটি Function লিখুন, যা `Intl.Segmenter` দিয়ে লেখাকে নির্দিষ্ট সংখ্যক চোখে-দেখা অক্ষরে কাটে, যাতে কোনো যুক্তাক্ষর না ভাঙে।

১১. **বাক্য গণনা:** `countSentences` নামে একটি Function লিখুন, যা দাঁড়ি (`।`), প্রশ্নবোধক (`?`) ও বিস্ময়সূচক (`!`) চিহ্ন ধরে বাক্য গোনে; ফাঁকা অংশ বাদ দেবে।

১২. **খোঁজার সারসংক্ষেপ:** Project-এর `findAll` ব্যবহার করে একটি Function লিখুন, যা খোঁজার শব্দ কতবার পাওয়া গেছে এবং প্রথম মিলের আশেপাশের ১০টি করে অক্ষর দেখায়।

৮.১৮ অনুশীলনীর সমাধান

সমাধান ১

trim-upper.js · JavaScript — চলবে: Browser Console ও Node.js

```
const title = '  JavaScript Mastery  ';
const clean = title.trim().toUpperCase();

console.log(`[${clean}]`);
console.log(`[${title}]`);
```

Output

```
[JAVASCRIPT MASTERY]
[  JavaScript Mastery  ]
```

সমাধান ২

date-parts.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const date = '2026-10-15';

console.log(date.slice(0, 4));
console.log(date.slice(5, 7));
console.log(date.slice(8));
```

Output

```
2026
10
15
```

সমাধান ৩

mask.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const phone = '01712345678';
const visible = phone.slice(-3);

console.log('*'.repeat(phone.length - 3) + visible);
console.log(visible.padStart(phone.length, '*'));
```

Output

```
********678
********678
```

দুটি পদ্ধতিই একই ফল দেয়। দ্বিতীয়টি ছোট: শেষ তিনটি অক্ষরের বাঁ দিকে `*` বসিয়ে মূল দৈর্ঘ্যে আনা।

সমাধান ৪ — `'a+b-c'` এবং `'a+b+c'`। `replace` শুধু প্রথম মিল বদলায়; `replaceAll` সব।

সমাধান ৫ — শুধু এক পাশ ছোট হাতের অক্ষরে আনা হয়েছে, আর `typed`-এর আগে-পরে ফাঁকা জায়গা আছে। দুই পাশেই একই কাজ করতে হবে।

compare-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const saved = 'Sara Ahmed';
const typed = '  sara ahmed ';
const clean = (text) => text.trim().toLowerCase();

console.log(clean(saved) === clean(typed));
```

Output

```
true
```

সমাধান ৬

vowel-signs.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countVowelSigns(text) {
  return [...text.normalize('NFC')].filter((c) => c === 'া').length;
}

console.log(countVowelSigns('আমার নাম সারা'));
```

Output

```
4
```

`আ` (স্বরবর্ণ) এবং `া` (আ-কার) আলাদা Code Point; এখানে শুধু কার-চিহ্নগুলো গোনা হয়েছে: মা, না, সা, রা।

সমাধান ৭

initials.js · JavaScript — চলবে: Browser Console ও Node.js

```
function initials(fullName) {
  return fullName
    .trim()
    .split(/\s+/)
    .map((part) => part[0].toUpperCase())
    .join('');
}

console.log(initials('sara ahmed khan'));
console.log(initials('  mina    das '));
```

Output

```
SAK
MD
```

`split(' ')`-এর বদলে `split(/\s+/)` ব্যবহার করায় একাধিক ফাঁকা জায়গায় ফাঁকা অংশ তৈরি হয়নি।

সমাধান ৮

same-text.js · JavaScript চলবে: Browser Console ও Node.js

```
function sameText(a, b) {
  const clean = (text) => text.normalize('NFC').trim().toLowerCase();
  return clean(a) === clean(b);
}

console.log('\u09DF' === '\u09AF\u09BC');
console.log(sameText('\u09DF', '\u09AF\u09BC'));
console.log(sameText(' Dhaka ', 'DHAKA'));
```

Output

```
false
true
true
```

সমাধান ৯

username.js · JavaScript চলবে: Browser Console ও Node.js

```
const isValidUsername = (name) => /^[a-z][a-z0-9_]{2,14}$/.test(name);

console.log(isValidUsername('sara_88'));
console.log(isValidUsername('8sara'));
console.log(isValidUsername('sa'));
console.log(isValidUsername('Sara'));
```

Output

```
true
false
false
false
```

প্রথম অক্ষর `[a-z]`, তারপর আরও ২ থেকে ১৪টি অনুমোদিত অক্ষর; মোট ৩ থেকে ১৫। `^` ও `$` পুরো নামটিকে নিয়মের মধ্যে রাখে।

সমাধান ১০

shorten-letters.js · JavaScript — চলবে: Browser Console ও Node.js

```
function shortenByLetters(text, count) {
  const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
  const letters = [...segmenter.segment(text)].map((part) => part.segment);

  if (letters.length <= count) {
    return text;
  }

  return `${letters.slice(0, count).join('')}...`;
}

console.log(shortenByLetters('ক্ষমা চাই', 2));
console.log('ক্ষমা চাই'.slice(0, 2));
```

Output

```
ক্ষমা...
ক্
```

দ্বিতীয় লাইন দেখায় সাধারণ `slice` কী করত: যুক্তাক্ষরের অর্ধেক।

সমাধান ১১

sentences.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countSentences(text) {
  return text
    .split(/[।?!]/)
    .filter((part) => part.trim() !== '').length;
}

console.log(countSentences('আমি শিখছি। আপনি? খুব ভালো!'));
console.log(countSentences(''));
```

Output

```
3
0
```

সমাধান ৯২

search-summary.js · JavaScript — চলবে: Browser Console ও Node.js

```
function findAll(text, query) {
  const haystack = text.normalize('NFC').toLowerCase();
  const needle = query.normalize('NFC').toLowerCase().trim();
  const positions = [];
  let index = needle === '' ? -1 : haystack.indexOf(needle);

  while (index !== -1) {
    positions.push(index);
    index = haystack.indexOf(needle, index + needle.length);
  }

  return positions;
}

function searchSummary(text, query) {
  const positions = findAll(text, query);

  if (positions.length === 0) {
    return 'পাওয়া যায়নি';
  }

  const start = Math.max(0, positions[0] - 10);
  const around = text.slice(start, positions[0] + query.length + 10);

  return `${positions.length} বার পাওয়া গেছে: "...${around}..."`;
}

console.log(searchSummary('Node.js দিয়ে Server চালান, Node.js শিখুন', 'node'));
console.log(searchSummary('কিছু লেখা', 'JavaScript'));
```

Output

```
2 বার পাওয়া গেছে: "...Node.js দিয়ে ..."
পাওয়া যায়নি
```

`Math.max(0, ...)` নিশ্চিত করে যে মিলটি লেখার শুরুতে থাকলে Index ঋণাত্মক হয়ে শেষ থেকে গোনা শুরু না হয়।

৮.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `let s = 'abc'; s[0] = 'X';`-এর পর `s` কী?

ক) `'Xbc'`

খ) একটি Error

গ) `'abc'`

ঘ) `'X'`

প্রশ্ন ২. `'\u{1F600}'.length`-এর মান কত? (এটি একটি হাসিমুখ Emoji!)

ক) `2`

খ) `1`

গ) `4`

ঘ) `0`

প্রশ্ন ৩. একটি বাংলা শব্দে চোখে দেখা অক্ষর গুনতে কোনটি উপযুক্ত?

ক) `text.length`

খ) `[...text].length`

গ) `text.split('').length`

ঘ) `Intl.Segmenter`

প্রশ্ন ৪. `'a-b-c'.replace('-', '+')`-এর ফল কী?

ক) `'a+b+c'`

খ) `'a+b-c'`

গ) `'a-b+c'`

ঘ) `'abc'`

প্রশ্ন ৫. `'JavaScript'.includes('script')`-এর ফল কী?

ক) `false`, কারণ খোঁজা Case-sensitive

খ) `true`

গ) `4`

ঘ) `-1`

প্রশ্ন ৬. দেখতে একই দুটি বাংলা শব্দ `===` দিয়ে মিলছে না। প্রথমে কী করা উচিত?

ক) দুই পাশেই `normalize('NFC')` করা

খ) দুই পাশেই `toUpperCase()` করা

গ) `==` ব্যবহার করা

ঘ) `length` তুলনা করা

প্রশ্ন ৭. `'a,b,,c'.split(',')` কয়টি Element-এর Array দেয়?

ক) ৩

খ) ২

গ) ৪

ঘ) ৫

প্রশ্ন ৮. বাংলা শব্দ খুঁজতে কোন Pattern ঠিক?

ক) `/\w+/g`

খ) `/[a-z]+/gi`

গ) `/\d+/g`

ঘ) `/[\p{L}\p{M}\p{N}]+/gu`

প্রশ্ন ৯. `/^01[3-9]\d{8}$/` কোনটিকে গ্রহণ করে?

ক) `'01712345678'`

খ) `'0171234567'`

গ) `'+8801712345678'`

ঘ) `'01212345678'`

প্রশ্ন ১০. ব্যবহারকারীর লেখা Page-এ নিরাপদে দেখানোর উপায় কোনটি?

ক) `element.innerHTML = text`

খ) `element.textContent = text`

গ) `document.write(text)`

ঘ) `element.outerHTML = text`

৮.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–ক · ৬–ক · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	String Immutable; Index-এ Assignment নীরবে উপেক্ষিত হয়।
২	ক	`length` UTF-16 Code Unit গোনে; এই Emoji দুটি Code Unit।
৩	ঘ	বাংলায় একটি চোখে-দেখা অক্ষর একাধিক Code Point হতে পারে; Grapheme গোনে শুধু `Intl.Segmenter`।
৪	খ	`replace` শুধু প্রথম মিলটি বদলায়।
৫	ক	`'Script'` আর `'script'` আলাদা; দুই পাশ ছোট হাতের অক্ষরে এনে খুঁজতে হয়।
৬	ক	একই অক্ষর ভিন্ন Code Point-এ লেখা থাকতে পারে; Normalization দুই পাশকে একই রূপে আনে।
৭	গ	দুটি পাশাপাশি কমার মাঝে একটি ফাঁকা String: `'a'`, `'b'`, `''`, `'c'`।
৮	ঘ	`\w` বাংলা চেনে না; `\p{M}` কার-চিহ্ন রাখে; `u` Flag বাধ্যতামূলক।
৯	ক	শুরুতে `01`, তারপর ৩–৯-এর একটি সংখ্যা এবং ঠিক ৮টি সংখ্যা; মোট ১১টি।
১০	খ	`textContent` লেখাকে HTML হিসেবে পড়ে না, তাই ভেতরের Tag বা Code চলে না।

৮.২১ অধ্যায়ের সারসংক্ষেপ

- String Immutable; প্রতিটি Method নতুন String ফেরত দেয়।
- Template Literal মান বসাতে ও একাধিক লাইনে লিখতে; Escape Sequence বিশেষ অক্ষরের জন্য।
- `length` Code Unit গোনে; `[...text]` Code Point; চোখে দেখা অক্ষর `Intl.Segmenter`। বাংলায় তিনটি সংখ্যা আলাদা হতে পারে।
- পড়া: `at`, `slice`। খোঁজা: `includes`, `startsWith`, `endsWith`, `indexOf`; সবই Case-sensitive।
- পরিষ্কার: `trim`; বদলানো: `replace` (প্রথমটি), `replaceAll` (সব); সাজানো: `padStart`, `repeat`।
- তুলনার আগে দুই পাশ একই রূপে: `normalize('NFC')`, `trim()`, `toLowerCase()`।
- `split` String থেকে Array, `join` Array থেকে String।
- Regular Expression Pattern খোঁজে, বদলায় ও যাচাই করে; বাংলার জন্য `\p{L}\p{M}\p{N}` এবং `u` Flag।
- Regex দিয়ে গঠন যাচাই করা যায়, কিন্তু Email-এর মতো তথ্যের সত্যতা নয়।
- ব্যবহারকারীর লেখা `innerHTML`-এ বসালে XSS; সব সময় `textContent`।

পরের অধ্যায়ে DOM এবং Event: Page-এর Element তৈরি, বদলানো ও সরানো, ব্যবহারকারীর কাজে সাড়া দেওয়া, এবং এই অধ্যায়ের নিরাপদ Rendering-এর নিয়ম বাস্তবে প্রয়োগ।

৮.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Immutable	তৈরির পর বদলানো যায় না।
Template Literal	Backtick-এর String, যাতে `${ }` দিয়ে মান বসানো যায়।
Escape Sequence	`\` দিয়ে শুরু হওয়া বিশেষ অক্ষরের লিখিত রূপ।
Code Unit	JavaScript String-এর সংরক্ষণের একক (UTF-16)।
Code Point	Unicode-এর একটি অক্ষর-নম্বর।
Grapheme	পাঠকের চোখে একটি অক্ষর।
Unicode Normalization	একই লেখার ভিন্ন ভিন্ন Code Point-রূপকে একটি আদর্শ রূপে আনা।
Tokenization	লেখাকে শব্দে ভাগ করা।
Slug	Web Address-এর উপযোগী, হাইফেন দিয়ে জোড়া শব্দ।
Regular Expression	লেখার Pattern বর্ণনার ভাষা।
Flag	Regex-এর আচরণ বদলানোর চিহ্ন: `g`, `i`, `u`।
XSS	ব্যবহারকারীর লেখার মাধ্যমে Page-এ ক্ষতিকর Code চালানোর আক্রমণ।

৮.২৩ আরও পড়ুন

- **MDN — String:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String` — সব String Method-এর তালিকা।
- **MDN — String.prototype.normalize():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/normalize` — Unicode Normalization।
- **MDN — Intl.Segmenter:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter` — চোখে দেখা অক্ষর ও শব্দে ভাগ করা।
- **MDN — Regular expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions` — Regex-এর নির্দেশিকা।
- **OWASP — Cross Site Scripting (XSS):** `https://owasp.org/www-community/attacks/xss/` — XSS আক্রমণের ব্যাখ্যা।

অধ্যায় ৯

০৯

DOM Manipulation, Events এবং Interactive Web Applications

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- DOM কী, এবং Browser কীভাবে HTML থেকে Element-এর একটি গাছ তৈরি করে, তা ব্যাখ্যা করতে।
- Script কখন চলে তা নিয়ন্ত্রণ করতে, এবং `null` Element-এর Error এড়াতে।
- `querySelector` ও `querySelectorAll` দিয়ে Element খুঁজতে, পড়তে এবং বদলাতে।
- `createElement`, `append`, `remove` ও `replaceChildren` দিয়ে Page-এর অংশ তৈরি ও সরাতে।
- Event Listener যোগ করতে, Event Object পড়তে এবং Event Propagation বুঝতে।
- Event Delegation দিয়ে একটি Listener-এ অনেক Element, এমনকি পরে তৈরি হওয়া Element-ও সামলাতে।
- Form-এর Submit সামলাতে এবং ব্যবহারকারীকে বোধগম্যভাবে ভুল জানাতে।
- State ও UI আলাদা রেখে একটি `render` Function-এর মাধ্যমে Page আঁকতে।
- `localStorage`-এ তথ্য নিরাপদে রাখতে ও ফিরিয়ে আনতে।
- একটি সম্পূর্ণ **Task Manager** Web Application তৈরি করতে এবং Browser-এ Automated Test দিয়ে পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭: Object, Spread এবং `JSON.stringify` / `JSON.parse`।
- অধ্যায় ৮: `textContent` বনাম `innerHTML` এবং XSS।
- HTML-এর প্রাথমিক ধারণা: Tag, `id`, `class`, `<form>`, `<input>`, `<button>`।
- একটি আধুনিক Browser (Chrome, Edge বা Firefox) এবং তার DevTools।
- `chapter-09` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আপনার Program-গুলো Console-এ লেখা দেখিয়েছে। কিন্তু ব্যবহারকারী Console দেখেন না; তাঁরা দেখেন একটি Page, যেখানে Button চাপেন, লেখেন, তালিকা থেকে বাছেন। JavaScript-কে Browser-এ তৈরি করার মূল কারণ ছিল এটিই: Page-কে জীবন্ত করা।

এই অধ্যায়ে আপনি শিখবেন Page পড়তে ও বদলাতে, এবং ব্যবহারকারীর প্রতিটি কাজে সাড়া দিতে। সঙ্গে শিখবেন এমন একটি গঠন, যা Application বড় হলেও Code-কে গোছানো রাখে: তথ্য এক জায়গায়, আঁকার Code আরেক জায়গায়। React, Vue বা Svelte-এর মতো আধুনিক Framework এই ধারণার ওপরই তৈরি। ধারণাটি এখানে খালি হাতে বুঝে নিলে পরে সেগুলো শেখা অনেক সহজ হবে।

৯.১ DOM: Page-এর গাছ

Browser একটি HTML File পড়ে তার প্রতিটি Tag থেকে একটি করে **Element Object** তৈরি করে, এবং সেগুলোকে মূল HTML-এর মতোই একটির ভেতরে আরেকটি সাজিয়ে রাখে। এই Object-এর গাছকে বলে **DOM** (Document Object Model)। JavaScript এই গাছ পড়তে ও বদলাতে পারে, আর গাছ বদলালে Browser সঙ্গে সঙ্গে পর্দার ছবি বদলে দেয়।

নিচের Page-টিতে একটি শিরোনাম এবং একটি তালিকা আছে।

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>DOM পড়া</title>
</head>
<body>
  <h1 id="title">আজকের কাজ</h1>
  <ul class="tasks">
    <li data-priority="high">বাজার করা</li>
    <li>বই পড়া</li>
    <li class="done">হাঁটা</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

এর DOM গাছটি এমন: সবার ওপরে `document`, তার নিচে `html`, তার দুটি শাখা `head` ও `body`, এবং `body`-র ভেতরে `h1`, `ul` ও `script`। `ul`-এর তিনটি সন্তান (Child) `li`। গাছের ভাষায় `ul` হলো `li`-গুলোর **Parent**, আর `li`-গুলো পরস্পরের **Sibling**।

Element খোঁজা ও পড়া

script.js · JavaScript চলবে: Browser

```
const title = document.querySelector('#title');
const items = document.querySelectorAll('.tasks li');

console.log(title.textContent);
console.log(items.length);
console.log(items[0].dataset.priority);
console.log(document.querySelector('.done').textContent);
console.log(document.querySelector('#missing'));
console.log([...items].map((item) => item.textContent).join(', '));
```

Output

```
আজকের কাজ
3
high
হাঁটা
null
বাজার করা, বই পড়া, হাঁটা
```

- **`document.querySelector(selector)`** CSS-এর Selector লিখে প্রথম মিলে যাওয়া Element দেয়: `#title` মানে `id="title"`, `.done` মানে `class="done"`, `.tasks li` মানে `.tasks`-এর ভেতরের সব `li`। কিছু না মিললে `null`।
- **`document.querySelectorAll(selector)`** সব মিল একটি তালিকায় দেয় (`NodeList`)। এর `length` আছে, Index দিয়ে পড়া যায়, `for...of` চলে; কিন্তু `map`-এর মতো Array Method নেই। তাই শেষ লাইনে Spread দিয়ে আগে Array বানানো হয়েছে (অধ্যায় ৭)।
- **`dataset`** দিয়ে `data-` দিয়ে শুরু হওয়া Attribute পড়া যায়: `data-priority` হয় `dataset.priority`। নিজের তথ্য Element-এর সঙ্গে জুড়ে রাখার এটিই মানসম্মত উপায়।

পুরোনো Code-এ `getElementById`, `getElementsByClassName` দেখবেন। সেগুলো এখনো কাজ করে, তবে `querySelector` একাই সব ধরনের Selector বোঝে, তাই এই বইয়ে সেটিই ব্যবহার করা হয়েছে।

৯.২ Script কখন চলে

Browser HTML ওপর থেকে নিচে পড়ে। পথে `<script>` পেলে সেখানেই থেমে Script চালায়, তারপর বাকি HTML পড়ে। ফলে Script যদি এমন Element খোঁজে যা তখনো পড়াই হয়নি, তা পাবে না।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>খুব আগে</title>
  <script src="script.js"></script>
</head>
<body>
  <h1 id="title">স্বাগতম</h1>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```
const title = document.querySelector('#title');

console.log(title);
title.textContent = 'নতুন শিরোনাম';
```

Error Message

```
null
TypeError: Cannot set properties of null (setting 'textContent')
```

Script চলার সময় `<h1>` তখনো তৈরি হয়নি, তাই `querySelector` দিয়েছে `null`, এবং `null`-এর ওপর `textContent` বসাতে গিয়ে Error। DOM নিয়ে কাজের সবচেয়ে প্রচলিত Error এটিই। সমাধান দুটি:

১. **`<script>`-কে `<body>`-র একেবারে শেষে রাখুন,** যেমন অনুচ্ছেদ ৯.১-এর Page-এ।

২. **`<head>`-এ রাখলে `defer` লিখুন:** `<script src="script.js" defer></script>`। `defer` থাকলে Browser File-টি সঙ্গে সঙ্গে আনা শুরু করে, কিন্তু চালায় পুরো HTML পড়া শেষ হওয়ার পর।

একাধিক `defer` Script লেখা ক্রমেই চলে। এই অধ্যায়ের Project প্রথম পদ্ধতি ব্যবহার করেছে।

৯.৩ Element বদলানো

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>বদলানো</title>
</head>
<body>
  <p id="message" class="note">অপেক্ষা করছে</p>
  <a id="help" href="https://example.com">সহায়তা</a>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```
const message = document.querySelector('#message');
const help = document.querySelector('#help');

message.textContent = 'সংরক্ষিত হয়েছে';
message.classList.add('success');
message.classList.remove('note');
message.classList.toggle('bold');

help.setAttribute('href', 'https://developer.mozilla.org');
help.hidden = true;

console.log(message.textContent);
console.log(message.className);
console.log(message.classList.contains('note'));
console.log(help.getAttribute('href'));
console.log(help.hidden);
```

Output

```
সংরক্ষিত হয়েছে
success bold
false
https://developer.mozilla.org
true
```

কাজ	উপায়
লেখা বদলানো	`element.textContent = text`
Class যোগ / সরানো / উল্টানো	`classList.add()`, `remove()`, `toggle()`
Class আছে কি না	`classList.contains()`
Attribute পড়া / বসানো / সরানো	`getAttribute()`, `setAttribute()`, `removeAttribute()`
লুকানো / দেখানো	`element.hidden = true` বা `false`
Form Field-এর মান	`input.value`, `checkbox.checked`

চেহারা বদলাতে সরাসরি `element.style.color = 'red'` লেখা যায়, কিন্তু ভালো অভ্যাস হলো CSS-এ একটি Class-এর চেহারা লিখে রাখা এবং JavaScript থেকে শুধু Class যোগ বা সরানো। তাতে চেহারার সব নিয়ম CSS-এ এক জায়গায় থাকে।

> **সতর্কতা:** লেখা বসাতে সব সময় `textContent`। `innerHTML` লেখাকে HTML হিসেবে পড়ে; ব্যবহারকারীর লেখা সেখানে গেলে XSS-এর পথ খুলে যায় (অধ্যায় ৮)।

৯.৪ Element তৈরি, যোগ ও সরানো

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>তৈরি ও সরানো</title>
</head>
<body>
  <ul id="list">
    <li>প্রথম</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```
const list = document.querySelector('#list');

const second = document.createElement('li');
second.textContent = 'দ্বিতীয়';
list.append(second);

const first = document.createElement('li');
first.textContent = 'শুরুতে';
list.prepend(first);

console.log(list.children.length);

list.firstElementChild.remove();
console.log(list.children.length);
console.log(list.lastElementChild.textContent);

list.replaceChildren();
console.log(list.children.length);
```

Output

```
3
2
দ্বিতীয়
0
```

নতুন Element তৈরির তিন ধাপ: `document.createElement('tag')` দিয়ে তৈরি, Property বসানো, তারপর Page-এর কোথাও যোগ। তৈরি করা Element যোগ না করা পর্যন্ত Page-এ দেখা যায় না।

Method	কাজ
`parent.append(a, b)`	শেষে যোগ; একসঙ্গে একাধিক
`parent.prepend(a)`	শুরুতে যোগ
`element.remove()`	নিজেকে Page থেকে সরানো
`parent.replaceChildren(a, b)`	সব পুরোনো সন্তান সরিয়ে নতুনগুলো বসানো; কিছু না দিলে খালি করা
`parent.children`	সন্তান Element-গুলোর তালিকা

অনেক Element একসঙ্গে বসাতে হলে প্রথমে `map` দিয়ে সবগুলো তৈরি করুন, তারপর একবারে `replaceChildren(...elements)` বা `append(...elements)`। এতে Browser-কে একবারই পর্দা নতুন করে আঁকতে হয়, এবং Code-ও পরিষ্কার থাকে।

৯.৫ Event: ব্যবহারকারীর কাজে সাড়া

ব্যবহারকারী Click করলে, লিখলে, Key চাপলে বা Form পাঠালে Browser একটি **Event** তৈরি করে। আপনি আগে থেকে বলে রাখতে পারেন, অমুক Element-এ অমুক Event ঘটলে এই Function চালাতে হবে। এই Function-কে বলে **Event Listener** বা Handler।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>গণনা</title>
</head>
<body>
  <p>গণনা: <output id="count">0</output></p>
  <button id="add" type="button">যোগ</button>
  <button id="reset" type="button">শূন্য</button>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const countView = document.querySelector('#count');
let count = 0;

function show() {
  countView.textContent = count;
  console.log(`গণনা: ${count}`);
}
```

```
document.querySelector('#add').addEventListener('click', () => {
  count += 1;
  show();
});

document.querySelector('#reset').addEventListener('click', () => {
  count = 0;
  show();
});
```

Browser-এ Page খুলে "যোগ", "যোগ", "শূন্য", "যোগ" চাপলে:

Output

```
গণনা: 1
গণনা: 2
গণনা: 0
গণনা: 1
```

`addEventListener('click', function)`-এ দ্বিতীয় Argument হলো Function নিজে, যা Browser পরে Call করবে। Page Load-এর সময় কিছুই ছাপা হয়নি; প্রতিটি লাইন একটি Click-এর ফল।

Event	কখন ঘটে
`click`	Click, অথবা Button-এ Focus রেখে Enter বা Space
`input`	Field-এর লেখা প্রতিবার বদলালে
`change`	Checkbox, Select বদলালে; Text Field-এ লেখা শেষ করে বেরোলে
`submit`	Form পাঠানো হলে (Button বা Enter)
`keydown`	Key চাপলে; কোন Key তা `event.key`-এ
`DOMContentLoaded`	পুরো HTML পড়া শেষ হলে

> **মনে রাখুন:** `button.addEventListener('click', show())` লিখলে `show` সঙ্গে সঙ্গে একবার চলে, এবং তার ফেরত মান (`undefined`) Listener হিসেবে যায়। ফলে Click-এ কিছুই হয় না। Function-এর নাম দিন, Call নয়: `show`।

৯.৬ Event Object এবং Propagation

Listener Function একটি Argument পায়: **Event Object**। এতে Event-এর সব তথ্য থাকে: কোন ধরনের Event (`event.type`), কোন Element-এ ঘটেছে (`event.target`), কোন Key (`event.key`)।

একটি গুরুত্বপূর্ণ আচরণ: একটি Element-এ Event ঘটলে তা ওই Element-এ শেষ হয় না; DOM গাছ বেয়ে Parent, তার Parent, এভাবে `document` পর্যন্ত ওপরে ওঠে। একে বলে **Bubbling**।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Bubbling</title>
</head>
<body>
  <div id="card">
    <h2>কার্ড</h2>
    <button id="like" type="button">পছন্দ</button>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```
const card = document.querySelector('#card');
const like = document.querySelector('#like');

like.addEventListener('click', (event) => {
  console.log(`like: ${event.type}`);
});

card.addEventListener('click', (event) => {
  console.log(`card: target ${event.target.tagName}`);
});

document.addEventListener('click', () => {
  console.log('document');
});
```

প্রথমে "পছন্দ" Button, তারপর "কার্ড" শিরোনামে Click করলে:

Output

```
like: click
card: target BUTTON
document
card: target H2
document
```

Button-এ একটি Click তিনটি Listener চালিয়েছে: Button-এর, তার Parent `card`-এর, এবং `document`-এর। `card`-এর Listener-এ `event.target` হলো আসল Click যেখানে হয়েছে সেই Element: প্রথমবার Button, দ্বিতীয়বার শিরোনাম।

- **`event.stopPropagation()`** Event-কে আর ওপরে উঠতে দেয় না। এটি খুব কম প্রয়োজন হয়, এবং অন্য Code-এর প্রত্যাশা ভেঙে দিতে পারে; প্রয়োজন না হলে ব্যবহার করবেন না।
- **`event.preventDefault()`** Browser-এর নিজস্ব আচরণ থামায়: Link-এ গিয়ে Page বদলানো, বা Form পাঠিয়ে Page নতুন করে Load। এটি Propagation থামায় না; দুটি আলাদা জিনিস।

৯.৭ Event Delegation

Bubbling-এর সবচেয়ে বড় ব্যবহার **Event Delegation**: প্রতিটি সন্তানে আলাদা Listener না দিয়ে Parent-এ একটি Listener দেওয়া, এবং `event.target` দেখে বোঝা কোন সন্তানে ঘটেছে।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Delegation</title>
</head>
<body>
  <ul id="menu">
    <li><button type="button" data-item="চা">চা</button></li>
    <li><button type="button" data-item="কফি">কফি</button></li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const menu = document.querySelector('#menu');

menu.addEventListener('click', (event) => {
  const button = event.target.closest('button');

  if (!button) {
    return;
  }

  console.log(`অর্ডার: ${button.dataset.item}`);
});

const row = document.createElement('li');
```

```
const juice = document.createElement('button');
juice.type = 'button';
juice.dataset.item = 'জুস';
juice.textContent = 'জুস';
row.append(juice);
menu.append(row);
```

"চা" এবং পরে যোগ হওয়া "জুস" Button-এ Click করলে:

Output

```
অর্ডার: চা
অর্ডার: জুস
```

"জুস" Button Listener যোগ হওয়ার **পরে** তৈরি হয়েছে, তবু কাজ করেছে, কারণ Listener আছে Parent-এ। তালিকা যেখানে বারবার নতুন করে আঁকা হয়, সেখানে এটি অপরিহার্য: প্রতিবার আঁকার পর Listener নতুন করে যোগ করতে হয় না।

`event.target.closest('button')` Click-এর জায়গা থেকে ওপরের দিকে প্রথম `button` খোঁজে, নিজেকেসহ। Button-এর ভেতরে একটি Icon বা `<span>` থাকলে `event.target` সেই ভেতরের Element হতে পারে; `closest` তখনো আসল Button-টিই দেয়। তালিকার ফাঁকা জায়গায় Click হলে `null`, এবং Guard Clause (অধ্যায় ৫) চুপচাপ ফিরে যায়।

৯.৮ Form এবং Validation

Form-এর তথ্য পড়ার সবচেয়ে নির্ভরযোগ্য জায়গা `submit` Event। এটি Button-এ Click এবং Field-এ Enter চাপা—দুই ক্ষেত্রেই ঘটে।

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Form</title>
</head>
<body>
  <form id="signup" novalidate>
    <label for="email">Email</label>
    <input id="email" type="email" aria-describedby="error">
    <button type="submit">নিবন্ধন</button>
  </form>
  <p id="error" role="alert"></p>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```
const form = document.querySelector('#signup');
const email = document.querySelector('#email');
const error = document.querySelector('#error');

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const value = email.value.trim();

  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)) {
    error.textContent = 'সঠিক একটি Email লিখুন।';
    email.setAttribute('aria-invalid', 'true');
    console.log(`ত্রুটি: "${value}"`);
    return;
  }

  error.textContent = '';
  email.removeAttribute('aria-invalid');
  console.log(`পাঠানো হলো: ${value}`);
  form.reset();
});
```

ফাঁকা রেখে "নিবন্ধন" চাপলে, তারপর `sara@` লিখে Enter, শেষে `sara@example.com` লিখে Enter:

Output

```
ত্রুটি: ""
ত্রুটি: "sara@"
পাঠানো হলো: sara@example.com
```

- **`event.preventDefault()`** প্রথম লাইনেই। এটি না থাকলে Browser Form পাঠিয়ে Page নতুন করে Load করত, এবং আপনার Code-এর সব কাজ ও Console মুছে যেত।
- **`novalidate`** Browser-এর নিজস্ব যাচাই-বার্তা বন্ধ করেছে, যাতে বার্তা ও তার ভাষা আপনার নিয়ন্ত্রণে থাকে। Browser-এর যাচাই চালু রাখাও একটি বৈধ পছন্দ; দুটি একসঙ্গে মেশাবেন না।
- **যাচাই Browser-এ, আবার Server-এও।** Browser-এর যাচাই ব্যবহারকারীর সুবিধার জন্য; যেকোনো ব্যবহারকারী তা এড়িয়ে যেতে পারেন। আসল নিরাপত্তা Server-এ।

সবার জন্য ব্যবহারযোগ্য Form

Screen Reader ব্যবহারকারী পর্দা দেখেন না; তাঁদের Software পড়ে শোনায়। কয়েকটি ছোট অভ্যাস তাঁদের জন্য Form ব্যবহারযোগ্য করে:

- প্রতিটি Field-এর একটি `<label>`, যার `for` Field-এর `id`-এর সঙ্গে মেলে।
- ভুলের বার্তা `role="alert"` বা `aria-live="polite"` আছে এমন Element-এ; লেখা বদলালে Software তা পড়ে শোনায়।

- ভুল Field-এ `aria-invalid="true"`, এবং `aria-describedby` দিয়ে বার্তার সঙ্গে সংযোগ।
- সব কাজ Keyboard দিয়ে করা যায় কি না যাচাই: Tab দিয়ে ঘোরা, Enter দিয়ে পাঠানো। আসল `<button>` ও `<form>` ব্যবহার করলে এটি বিনা খরচে পাওয়া যায়।

৯.৯ State এবং UI আলাদা রাখা

ছোট Page-এ সরাসরি DOM বদলানো চলে। কিন্তু Page বড় হলে প্রশ্ন ওঠে: আসল তথ্য কোথায়? Page-এর লেখায়, নাকি কোনো Variable-এ? দুই জায়গায় থাকলে একসময় দুটি আলাদা হয়ে যায়।

নিয়মটি সরল: **তথ্য থাকবে একটি জায়গায়, যাকে বলে State। Page হবে সেই State-এর ছবি।** তথ্য বদলাতে হলে State বদলান, তারপর একটি `render` Function পুরো অংশটি State থেকে আবার আঁকে।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>State</title>
</head>
<body>
  <ul id="cart"></ul>
  <p id="total"></p>
  <button id="add-tea" type="button">চা যোগ</button>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const state = { items: [{ name: 'বই', price: 250 }] };

const cartView = document.querySelector('#cart');
const totalView = document.querySelector('#total');

function render() {
  const rows = state.items.map((item) => {
    const row = document.createElement('li');
    row.textContent = `${item.name}: ${item.price} টাকা`;
    return row;
  });
  const total = state.items.reduce((sum, item) => sum + item.price, 0);

  cartView.replaceChildren(...rows);
  totalView.textContent = `মোট ${total} টাকা`;
  console.log(`${rows.length}টি জিনিস, ${totalView.textContent}`);
```

```
}

document.querySelector('#add-tea').addEventListener('click', () => {
  state.items = [...state.items, { name: 'চা', price: 20 }];
  render();
});

render();
```

Page খোলার পর "চা যোগ" দুবার চাপলে:

Output

```
1টি জিনিস, মোট 250 টাকা
2টি জিনিস, মোট 270 টাকা
3টি জিনিস, মোট 290 টাকা
```

Click Handler পর্দার কিছু ছোঁয়নি; শুধু State বদলে `render()` ডেকেছে। মোট দাম কোথাও আলাদাভাবে যোগ-বিয়োগ করে রাখা হয়নি; প্রতিবার State থেকে নতুন করে হিসাব হয়েছে, তাই ভুল হওয়ার সুযোগ নেই।

এর সঙ্গে আরেকটি বিভাজন যোগ করুন: তথ্য বদলানোর নিয়ম (Task যোগ, মোছা, যাচাই) রাখুন DOM-মুক্ত Pure Function-এ (অধ্যায় ৫)। তখন সেই নিয়মগুলো Browser ছাড়াই Node.js-এ Test করা যায়। Project ঠিক এভাবেই গঠিত।

৯.১০ localStorage: নিরাপদে সংরক্ষণ

অধ্যায় ৭-এ দেখেছেন, `localStorage` Browser-এ লেখা আকারে তথ্য রাখে, এবং Object রাখতে `JSON.stringify` লাগে। এখানে তিনটি বাড়তি বিষয় দেখা যাক।

storage.js · JavaScript — চলবে: Browser

```
const KEY = 'jsm-ch09-settings';

localStorage.setItem(KEY, JSON.stringify({ theme: 'dark', fontSize: 16 }));

const raw = localStorage.getItem(KEY);
console.log(typeof raw);
console.log(JSON.parse(raw).fontSize + 2);

localStorage.setItem('count', 5);
console.log(typeof localStorage.getItem('count'));
console.log(localStorage.getItem('missing'));

localStorage.removeItem(KEY);
localStorage.removeItem('count');
console.log(localStorage.length);
```

Output

```
string
18
string
null
0
```

১. **সব কিছু লেখা হয়ে যায়।** `5` রাখলে ফেরত আসে `'5'`। সংখ্যা, Boolean, Object—সবই `JSON.stringify` দিয়ে রাখুন এবং `JSON.parse` দিয়ে ফিরিয়ে আনুন।

২. **না থাকলে `null`।** প্রথমবার Page খোলার সময় এটিই স্বাভাবিক অবস্থা।

৩. **সংরক্ষিত তথ্য বিশ্বাসযোগ্য নয়।** পুরোনো Version-এর Program অন্য গঠনে রেখে থাকতে পারে, ব্যবহারকারী DevTools-এ বদলে থাকতে পারেন, অথবা Browser Storage বন্ধ রাখতে পারে। তাই পড়ার Code কখনো ভেঙে পড়া উচিত নয়।

safe-storage.js · JavaScript চলবে: Browser

```js
function loadList(key) {
  try {
    const data = JSON.parse(localStorage.getItem(key) ?? '[]');
    return Array.isArray(data) ? data : [];
  } catch {
    return [];
  }
}

localStorage.setItem('broken', '{not json');
localStorage.setItem('wrong-shape', '{"a":1}');

console.log(loadList('broken').length);
console.log(loadList('wrong-shape').length);
console.log(loadList('never-saved').length);

localStorage.removeItem('broken');
localStorage.removeItem('wrong-shape');
```

Output

```
0
0
0
```

ভাঙা JSON, ভুল গঠন, বা কিছুই না থাকা—তিন ক্ষেত্রেই Program একটি ফাঁকা তালিকা নিয়ে স্বাভাবিকভাবে শুরু করেছে। `try...catch` Error ধরে Program-কে চালু রাখে; বিস্তারিত অধ্যায় ১১-এ। `setItem`-ও ব্যর্থ হতে পারে (জায়গা শেষ, বা Storage বন্ধ), তাই সেটিও `try...catch`-এ রাখুন।

> **সতর্কতা:** `localStorage` একই Website-এর যেকোনো Script পড়তে পারে, এবং তা Encrypt করা থাকে না। Password, Token বা ব্যক্তিগত সংবেদনশীল তথ্য সেখানে রাখবেন না।

৯.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Element তৈরির আগে Script চলা	`Cannot set properties of null`	Script `<body>` -র শেষে, বা `defer`
ভুল Selector (`title` বনাম `#title`)	`null`	DevTools Console-এ Selector পরীক্ষা
`addEventListener('click', show())`	Load-এ একবার চলে, Click-এ কিছু না	`show` (Call ছাড়া)
Submit Handler-এ `preventDefault` নেই	Page Reload, তথ্য হারায়	প্রথম লাইনেই `event.preventDefault()`
পরে তৈরি Element-এ Listener নেই	নতুন Button কাজ করে না	Event Delegation
`event.target` সরাসরি Button ধরা	ভেতরের `<span>` -এ Click হলে কাজ করে না	`event.target.closest('button')`
`querySelectorAll` -এর ওপর `map`	`map is not a function`	`[...nodeList].map()`
ব্যবহারকারীর লেখা `innerHTML` -এ	XSS	`textContent`
`localStorage` থেকে পড়া তথ্য যাচাই না করা	ভাঙা তথ্যে পুরো Page বন্ধ	`try...catch` এবং গঠন যাচাই
State ও DOM দুই জায়গায় তথ্য	পর্দা ও তথ্য মেলে না	এক State, এক `render`

৯.১২ Debugging নির্দেশিকা

Browser-এর DevTools (F12, অথবা Page-এ Right Click করে Inspect) DOM নিয়ে কাজের প্রধান যন্ত্র।

- **Console:** Error-এর লাল বার্তা এবং তার পাশে File ও লাইন নম্বর। লাইনে Click করলে সেই Code খোলে। `document.querySelector('#title')` সরাসরি লিখে দেখুন Selector কিছু পায় কি না।
- **Elements:** বর্তমান DOM গাছ, মূল HTML নয়। JavaScript যা বদলেছে, এখানে তা দেখা যায়। একটি Element বেছে নিয়ে Console-এ `$0` লিখলে সেটি পাওয়া যায়।
- **Event Listeners (Elements-এর পাশের Panel):** কোন Element-এ কোন Listener লাগানো আছে। Listener না থাকলে, বা ভুল Element-এ থাকলে এখানে ধরা পড়ে।

- **Sources:** Code-এর লাইনে Breakpoint দিয়ে Program থামানো, এবং সেই মুহূর্তের Variable দেখা। Event Listener-এর ভেতরে Breakpoint দিলে Click-এর সময় থামে।
- **Application:** Local Storage-এ কী সংরক্ষিত আছে দেখা, বদলানো ও মোছা। "সংরক্ষণ কাজ করছে না" ধরনের সমস্যায় প্রথম জায়গা।

একটি Click কাজ না করলে ক্রমে দেখুন:

১. Console-এ কোনো Error আছে কি না; থাকলে প্রথম Error থেকে শুরু করুন।
২. Selector সঠিক Element পায় কি না (`null` নয় তো)।
৩. Listener সঠিক Element-এ লাগানো আছে কি না (Event Listeners Panel)।
৪. Listener-এর প্রথম লাইনে `console.log('click')` দিয়ে দেখুন আদৌ চলে কি না।
৫. চললে State ঠিকমতো বদলেছে কি না, তারপর `render` ডাকা হয়েছে কি না।

৯.১৩ পেশাদার অভ্যাস

- **তথ্যের নিয়ম DOM থেকে আলাদা File-এ,** এবং সেগুলো Pure Function। তাতে Test সহজ হয়।
- **একটি State, একটি `render`।** Event Handler State বদলায়; পর্দা আঁকে শুধু `render`।
- **তালিকায় Event Delegation;** Element-এ নিজের তথ্য রাখতে `data-` Attribute।
- **লেখা সব সময় `textContent` দিয়ে।**
- **আসল HTML Element ব্যবহার করুন:** Click-যোগ্য জিনিস `<button>`, Form `<form>`, প্রতিটি Field-এর `<label>`। `<div>`-এ Click Listener দিলে Keyboard ও Screen Reader ব্যবহারকারী বাদ পড়েন।
- **`<button type="button">`** লিখুন যদি Button Form পাঠানোর জন্য না হয়; Form-এর ভেতরে Button-এর স্বাভাবিক ধরন `submit`।
- **ব্যবহারকারীকে সব সময় জানান** কী হলো: সফল হলে ছোট বার্তা, ভুল হলে কী ঠিক করতে হবে।
- **সংরক্ষিত তথ্যের Key-তে নাম ও Version** (`'jsm-ch09-tasks'`), যাতে অন্য Program-এর সঙ্গে মিশে না যায়।

৯.১৪ Project: Task Manager

একটি সম্পূর্ণ Web Application: কাজের তালিকা, যেখানে Task যোগ, সম্পন্ন চিহ্নিত, নাম বদলানো, মোছা, খোঁজা ও ছাঁকা যায়, এবং Page বন্ধ করে খুললেও তালিকা থাকে।

Project-এর চাহিদা

- Task যোগ: ফাঁকা নাম গ্রহণযোগ্য নয়; সর্বোচ্চ ১২০ অক্ষর; Enter দিয়েও যোগ করা যাবে।
- Checkbox দিয়ে সম্পন্ন বা বাকি চিহ্নিত করা।
- প্রতিটি Task-এর নাম বদলানো (Enter-এ সংরক্ষণ, Escape-এ বাতিল) এবং মোছা।

- নাম দিয়ে খোঁজা এবং "সব / বাকি / সম্পন্ন" ছাঁকা; সম্পন্নগুলো একসঙ্গে সরানো।
- মোট, বাকি ও সম্পন্ন Task-এর সংখ্যা।
- `localStorage`-এ সংরক্ষণ; ভাঙা সংরক্ষিত তথ্যেও Page চালু থাকবে।
- নিরাপত্তা: Task-এর নাম শুধু `textContent` দিয়ে দেখানো।
- সবার জন্য ব্যবহারযোগ্য: Label, Keyboard দিয়ে সব কাজ, বার্তা `aria-live`-এ।

File-এর কাঠামো

Folder-এর কাঠামো

```
task-manager/
├── index.html          Page-এর গঠন
├── style.css           চেহারা
├── task-core.js        তথ্যের নিয়ম; DOM নেই (Browser ও Node.js)
├── app.js              DOM, Event ও Storage
├── README.md
└── tests/
    ├── load-script.js
    ├── task-core.test.js   Node.js-এ Unit Test
    ├── cdp.mjs
    └── browser-check.mjs   আসল Chrome-এ Page-এর Test
```

কাজের প্রবাহ: ব্যবহারকারীর কাজ থেকে একটি Event; Listener `task-core.js`-এর একটি Function দিয়ে নতুন Task-তালিকা তৈরি করে; সেই তালিকা State-এ বসে, `localStorage`-এ সংরক্ষিত হয়, এবং `render()` পর্দা নতুন করে আঁকে।

index.html

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Task Manager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>Task Manager</h1>

    <form id="task-form" novalidate>
      <label for="task-input">নতুন Task</label>
      <div class="row">
        <input id="task-input" autocomplete="off">
```

```
        <button type="submit">যোগ করুন</button>
      </div>
    </form>
    <p id="notice" role="status" aria-live="polite"></p>

    <div class="tools">
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>দেখান
        <select id="filter">
          <option value="all">সব</option>
          <option value="active">বাকি</option>
          <option value="done">সম্পন্ন</option>
        </select>
      </label>
      <button id="clear-done" type="button">সম্পন্নগুলো সরান</button>
    </div>

    <p id="summary"></p>
    <ul id="task-list"></ul>
    <p id="empty">কোনো Task নেই।</p>
  </main>

  <script src="task-core.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

দুটি Script `<body>`-র শেষে, এবং ক্রম গুরুত্বপূর্ণ: `app.js` যে Function ব্যবহার করে, সেগুলো `task-core.js`-এ, তাই সেটি আগে। Field-এ `maxlength` দেওয়া হয়নি ইচ্ছা করে: সীমা পার হলে ব্যবহারকারী চুপচাপ থেমে যাওয়ার বদলে একটি ব্যাখ্যা পান।

style.css

চেহারার File দুই অংশে; দুটি মিলে সম্পূর্ণ File।

style.css · CSS অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
```

```
  max-width: 640px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

.row,
.tools {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

/* ... */
```

style.css · CSS অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
/* ... */

input,
select,
button {
  font: inherit;
  padding: 6px 10px;
}

#task-input,
#task-list .title,
#task-list .edit-input {
  flex: 1;
}

#task-list li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 0;
  border-bottom: 1px solid #e3e7ef;
}

#task-list li.done .title {
  text-decoration: line-through;
  color: #6b7585;
}

#notice {
  min-height: 1.5em;
```

```
  color: #12305f;
}
```

সম্পন্ন Task-এর চেহারা CSS-এ `li.done .title` নিয়মে লেখা; JavaScript শুধু `done` Class যোগ বা সরায় (অনুচ্ছেদ ৯.৩)।

task-core.js

তথ্যের সব নিয়ম এখানে। এই File-এ `document` বা `localStorage`-এর কোনো উল্লেখ নেই, তাই একই File Browser-এ চলে, আবার Node.js-এর Test-এও। দুই অংশ মিলে সম্পূর্ণ File।

task-core.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
'use strict';

const TITLE_LIMIT = 120;

// নাম যাচাই: সমস্যা থাকলে বার্তা, না থাকলে null।
function checkTitle(text) {
  const title = String(text ?? '').trim();

  if (title === '') {
    return 'Task-এর একটি নাম লিখুন।';
  }
  if (title.length > TITLE_LIMIT) {
    return `নাম ${TITLE_LIMIT} অক্ষরের বেশি হতে পারবে না।`;
  }
  return null;
}

function addTask(tasks, text, id) {
  const error = checkTitle(text);

  if (error) {
    return { tasks, error };
  }

  const task = { id: String(id), title: text.trim(), done: false };
  return { tasks: [...tasks, task], error: null };
}

function renameTask(tasks, id, text) {
  const error = checkTitle(text);

  if (error) {
    return { tasks, error };
  }

  const next = tasks.map((task) =>
```

```
    task.id === id ? { ...task, title: text.trim() } : task);
  return { tasks: next, error: null };
}

// ...
```

task-core.js · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

const toggleTask = (tasks, id) =>
  tasks.map((task) => (task.id === id ? { ...task, done: !task.done } : task));

const deleteTask = (tasks, id) => tasks.filter((task) => task.id !== id);

const clearDone = (tasks) => tasks.filter((task) => !task.done);

function visibleTasks(tasks, filter = 'all', query = '') {
  const needle = query.trim().toLowerCase();

  return tasks.filter((task) => {
    const statusOk = filter === 'all' || (filter === 'done') === task.done;
    return statusOk && task.title.toLowerCase().includes(needle);
  });
}

function countTasks(tasks) {
  const done = tasks.filter((task) => task.done).length;
  return { total: tasks.length, active: tasks.length - done, done };
}

// localStorage-এর লেখা থেকে নিরাপদে Task-এর তালিকা; ভাঙা হলে ফাঁকা তালিকা।
function parseStored(raw) {
  try {
    const data = JSON.parse(raw ?? '[]');
    return Array.isArray(data) ? data.filter(isTask) : [];
  } catch {
    return [];
  }
}

function isTask(value) {
  return typeof value?.id === 'string' &&
    typeof value.title === 'string' &&
    checkTitle(value.title) === null &&
```

```
    typeof value.done === 'boolean';
}
```

- **অধ্যায় ৭-এর ধরন:** প্রতিটি Function মূল তালিকা না বদলে নতুন তালিকা ফেরত দেয়; ব্যর্থ হলে মূল তালিকা এবং একটি বার্তা।
- **`visibleTasks`-এর ছাঁকনি:** `filter === 'all'` হলে সব; নইলে `(filter === 'done') === task.done`। "সম্পন্ন" বাছলে বাম পাশ `true`, তাই শুধু সম্পন্ন Task মেলে; "বাকি" বাছলে বাম পাশ `false`, তাই শুধু বাকিগুলো। দুটি `if`-এর কাজ একটি তুলনায়।
- **`parseStored` ও `isTask`:** অনুচ্ছেদ ৯.১০-এর নিরাপদ পড়া, এক ধাপ এগিয়ে। শুধু Array কি না নয়, প্রতিটি Task-এর গঠনও যাচাই করা হয়; অচেনা বা ভাঙা Task বাদ পড়ে, বাকিগুলো থাকে।

app.js

Page-এর সঙ্গে সব যোগাযোগ এখানে। চার অংশ মিলে সম্পূর্ণ File।

অংশ ১: Element, State এবং সংরক্ষণ

app.js · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Browser

```
'use strict';

const STORAGE_KEY = 'jsm-ch09-tasks';

const form = document.querySelector('#task-form');
const input = document.querySelector('#task-input');
const list = document.querySelector('#task-list');
const notice = document.querySelector('#notice');
const summary = document.querySelector('#summary');
const search = document.querySelector('#search');
const filter = document.querySelector('#filter');
const empty = document.querySelector('#empty');

let tasks = parseStored(readStorage());
let editingId = null;

function readStorage() {
  try {
    return localStorage.getItem(STORAGE_KEY);
  } catch {
    return null;
  }
}

function save() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
  } catch {
```

```
    notice.textContent = 'সংরক্ষণ করা যায়নি; Page বন্ধ করলে পরিবর্তন হারাবে।';
  }
}

// প্রতিটি পরিবর্তন এই পথে: State বদলানো, সংরক্ষণ, নতুন করে আঁকা।
function update(next, message = '') {
  tasks = next;
  notice.textContent = message;
  save();
  render();
}

// ...
```

State দুটি Variable: `tasks` (তালিকা) এবং `editingId` (কোন Task-এর নাম এখন বদলানো হচ্ছে)। সব পরিবর্তন `update` দিয়ে যায়, তাই সংরক্ষণ বা আঁকা কোথাও ভুলে বাদ পড়ার সুযোগ নেই।

অংশ ২: আঁকা

app.js · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Browser

```
// ...

function render() {
  const shown = visibleTasks(tasks, filter.value, search.value);
  const count = countTasks(tasks);

  summary.textContent =
    `মোট ${count.total} · বাকি ${count.active} · সম্পন্ন ${count.done}`;
  empty.hidden = shown.length > 0;
  list.replaceChildren(...shown.map(taskRow));
}

function taskRow(task) {
  const row = document.createElement('li');
  row.dataset.id = task.id;
  row.classList.toggle('done', task.done);

  if (task.id === editingId) {
    const field = document.createElement('input');
    field.className = 'edit-input';
    field.value = task.title;
    field.setAttribute('aria-label', 'Task-এর নতুন নাম');
    row.append(field, button('সংরক্ষণ', 'save'), button('বাতিল', 'cancel'));
    return row;
  }

  const check = document.createElement('input');
  check.type = 'checkbox';
```

```
  check.checked = task.done;
  check.dataset.action = 'toggle';
  check.setAttribute('aria-label', `সম্পন্ন: ${task.title}`);

  const title = document.createElement('span');
  title.className = 'title';
  title.textContent = task.title;

  row.append(check, title, button('বদলান', 'edit'), button('মুছুন', 'delete'));
  return row;
}

function button(label, action) {
  const element = document.createElement('button');
  element.type = 'button';
  element.textContent = label;
  element.dataset.action = action;
  return element;
}

// ...
```

`render` প্রতিবার পুরো তালিকা নতুন করে তৈরি করে। কয়েকশো Task পর্যন্ত এটি চোখে পড়ার মতো ধীর নয়, এবং Code সরল থাকে। প্রতিটি সারিতে Task-এর `id` আছে `data-id`-তে, আর প্রতিটি Button-এ তার কাজের নাম `data-action`-এ; পরের অংশের একটি Listener এই দুটি দেখেই সব কাজ চেনে। Checkbox-এর `aria-label` Screen Reader-কে বলে কোন Task-এর Checkbox।

অংশ ৩: যোগ করা এবং নাম বদলানো

app.js · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Browser

```
// ...

const newId = () => `${Date.now()}-${Math.random().toString(16).slice(2, 8)}`;

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const result = addTask(tasks, input.value, newId());

  if (result.error) {
    notice.textContent = result.error;
    input.focus();
    return;
  }

  form.reset();
  update(result.tasks, 'Task যোগ হয়েছে।');
});
```

```
function saveEdit(id) {
  const field = list.querySelector('.edit-input');
  const result = renameTask(tasks, id, field.value);

  if (result.error) {
    notice.textContent = result.error;
    field.focus();
    return;
  }

  editingId = null;
  update(result.tasks, 'নাম বদলানো হয়েছে।');
}

function startEdit(id) {
  editingId = id;
  render();
  list.querySelector('.edit-input').focus();
}

function cancelEdit() {
  editingId = null;
  render();
}

// ...
```

অংশ ৪: তালিকার Event এবং শুরু

app.js · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Browser

```
// ...

// Event Delegation: তালিকার একটি Listener সব সারির সব Button সামলায়।
list.addEventListener('click', (event) => {
  const target = event.target.closest('[data-action]');

  if (!target) {
    return;
  }

  const id = target.closest('li').dataset.id;
  const action = target.dataset.action;

  if (action === 'toggle') update(toggleTask(tasks, id));
  if (action === 'delete') update(deleteTask(tasks, id), 'Task মুছে ফেলা হয়েছে।');
  if (action === 'edit') startEdit(id);
  if (action === 'save') saveEdit(id);
```

```
  if (action === 'cancel') cancelEdit();
});

list.addEventListener('keydown', (event) => {
  if (!event.target.matches('.edit-input')) {
    return;
  }
  if (event.key === 'Enter') saveEdit(event.target.closest('li').dataset.id);
  if (event.key === 'Escape') cancelEdit();
});

search.addEventListener('input', render);
filter.addEventListener('change', render);
document.querySelector('#clear-done').addEventListener('click', () =>
  update(clearDone(tasks), 'সম্পন্ন Task সরানো হয়েছে।'));

render();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **যোগ করা।** `submit` Event Button-এ Click ও Enter—দুটোই ধরে। `addTask` ব্যর্থ হলে বার্তা দেখিয়ে Focus Field-এ ফেরত যায়, যাতে ব্যবহারকারী সঙ্গে সঙ্গে ঠিক করতে পারেন। সফল হলে Form খালি হয় এবং `update` বাকিটা করে।
- **একটি Listener, পাঁচটি কাজ।** তালিকার `click` Listener `closest('[data-action]')` দিয়ে কাজের Element খোঁজে, `closest('li')` দিয়ে সারি, আর `dataset.id` দিয়ে Task। Checkbox-এর Click-ও এখানে আসে (`toggle`)। `render` প্রতিবার সারিগুলো নতুন তৈরি করলেও Listener একবারই লাগানো হয়েছে।
- **নাম বদলানো।** `startEdit` শুধু State-এ `editingId` বসিয়ে `render` ডাকে; `taskRow` সেই সারির জন্য Checkbox-এর বদলে একটি Field আঁকে। এরপর Enter বা "সংরক্ষণ" দিয়ে `saveEdit`, Escape বা "বাতিল" দিয়ে `cancelEdit`। নতুন নাম ফাঁকা হলে `renameTask` বার্তা দেয় এবং Field খোলা থাকে।
- **খোঁজা ও ছাঁকা।** এগুলো State বদলায় না, শুধু কোন অংশ দেখানো হবে তা বদলায়, তাই সরাসরি `render`।
- **সংরক্ষণ।** `update` প্রতিটি পরিবর্তনের পর পুরো তালিকা সংরক্ষণ করে; Page খোলার সময় `parseStored(readStorage())` তা ফিরিয়ে আনে।
- **নিরাপত্তা।** Task-এর নাম শুধু `textContent` এবং Attribute হিসেবে বসানো হয়; কোথাও `innerHTML` নেই।

চালানো এবং প্রত্যাশিত আচরণ

`index.html` Browser-এ খুলুন (File-টিতে Double Click করলেই হবে)। Unit Test চালাতে Project Folder থেকে:

Terminal

```
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
ফাঁকা রেখে "যোগ করুন"	বার্তা "Task-এর একটি নাম লিখুন।"; কিছু যোগ হয় না
১২১ অক্ষরের নাম	বার্তা "নাম 120 অক্ষরের বেশি হতে পারবে না।"
নাম লিখে Enter	Task যোগ; Field খালি; সংখ্যা হালনাগাদ
Checkbox	নাম কেটে দেওয়া চেহারা; "বাকি" ও "সম্পন্ন" সংখ্যা বদলায়
"বদলান", নতুন নাম, Enter	নাম বদলায়; Escape দিলে আগের নাম থাকে
নামে `<img src="" onerror="...">`	লেখাটি হুবহু দেখা যায়; কোনো Code চলে না
Page Reload	সব Task ও তাদের অবস্থা ফিরে আসে

Companion Source Code-এ দুই ধরনের Automated Test আছে। `task-core.test.js` Node.js-এ তথ্যের নিয়মগুলোর ১৬টি Test চালায়। `browser-check.mjs` আসল Chrome খুলে Page-এ লিখে, Click করে ও Key চেপে ওপরের সারণির সব আচরণসহ ১৭টি আচরণ যাচাই করে; এর জন্য কম্পিউটারে Chrome বা Edge থাকতে হবে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`addTask is not defined`	`task-core.js` লোড হয়নি বা `app.js` -এর পরে	Script-এর ক্রম ও File-এর নাম
যোগ করলে Page Reload হয়	`preventDefault` নেই	Submit Handler-এর প্রথম লাইন
নতুন সারির Button কাজ করে না	Listener সারিতে, তালিকায় নয়	Event Delegation
Reload-এ তালিকা হারায়	`save` ডাকা হয়নি, বা Key বদলেছে	সব পরিবর্তন `update` দিয়ে; DevTools-এর Application
Page ফাঁকা, Console-এ JSON Error	সংরক্ষিত তথ্য ভাঙা এবং যাচাই নেই	`parseStored`
খুঁজলে কিছুই দেখায় না	ছাঁকনি ও খোঁজার শব্দ একসঙ্গে প্রযোজ্য	"সব" বেছে আবার দেখুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য শুধু এই Browser-এ থাকে; অন্য Device-এ দেখা যায় না, এবং Browser-এর তথ্য মুছলে হারায়। একাধিক Tab-এ একসঙ্গে খুললে শেষ সংরক্ষণটি টেকে। `file://` থেকে খুললে কিছু Browser Storage-এর আচরণ বদলায়; নির্ভরযোগ্য পরীক্ষার জন্য একটি Local Server ব্যবহার করুন (README দেখুন)। Server-এর সঙ্গে তথ্য আদান-প্রদান অধ্যায় ১০-এর বিষয়।

৯.১৫ অনুশীলনী

অনুশীলনগুলো একটি ফাঁকা Page-এ চলে: যে Element দরকার, Script নিজে তৈরি করে নেয়। একটি HTML File-এ শুধু `<script src="solution.js"></script>` রেখে `<body>`-র ভেতরে চালান।

প্রাথমিক স্তর

১. একটি `h1` তৈরি করে Page-এ যোগ করুন, তারপর `querySelector` দিয়ে খুঁজে তার লেখা বদলান এবং নতুন লেখাটি Console-এ দেখান।

২. "থিম বদলান" নামে একটি Button তৈরি করুন, যা চাপলে `body`-তে `dark` Class যোগ বা সরায়, এবং প্রতিবার Console-এ `Dark` বা `Light` দেখায়।

৩. `button.addEventListener('click', sayHello())` কেন কাজ করে না? ঠিক করুন।

৪. অনুচ্ছেদ ৯.৬-এর Code-এ `like`-এর Listener-এ `event.stopPropagation()` যোগ করলে "পছন্দ" Button-এ Click-এ কী কী ছাপা হবে?

৫. `['আম', 'কাঁঠাল', 'লিচু']` থেকে `createElement` দিয়ে একটি তালিকা তৈরি করুন; `innerHTML` ব্যবহার করবেন না।

মধ্যম স্তর

৬. একটি লেখার Box এবং তার নিচে "ব্যবহৃত/১২০" গণনা তৈরি করুন, যা প্রতিবার লেখার সময় হালনাগাদ হয়; সীমা পার হলে `over` Class।

৭. তিনটি সারির একটি তালিকা তৈরি করুন, প্রতিটিতে একটি "মুছুন" Button। একটিমাত্র Listener দিয়ে Button-এর সারিটি মুছুন এবং বাকি সংখ্যা দেখান।

৮. একটি লেখার Field-এ Enter চাপলে লেখাটি একটি তালিকায় যোগ হবে এবং Field খালি হবে; ফাঁকা হলে যোগ হবে না।

৯. `loadTheme` ও `saveTheme` লিখুন, যা `localStorage`-এ থিম রাখে। সংরক্ষিত মান `'dark'` ছাড়া অন্য কিছু হলে, বা Storage কাজ না করলে, `'light'` ধরুন।

চ্যালেঞ্জ স্তর

১০. একটি খোঁজার Field তৈরি করুন: লেখার সময় Console-এ খোঁজার শব্দ দেখাবে, আর Escape চাপলে Field খালি করবে।

১১. Task Manager-এ "সব সম্পন্ন" Button যোগ করতে প্রথমে `task-core.js`-এর জন্য একটি Pure Function `completeAll(tasks)` লিখুন, এবং দেখান যে মূল তালিকা বদলায়নি।

১২. মোছার পর "ফিরিয়ে আনুন" সুবিধার জন্য `deleteWithUndo(tasks, id)` লিখুন, যা নতুন তালিকা এবং একটি `undo` Function ফেরত দেয়; `undo` Task-টিকে তার আগের জায়গায় ফিরিয়ে আনবে।

৯.১৬ অনুশীলনীর সমাধান

সমাধান ১

solution.js · JavaScript — চলবে: Browser

```
const heading = document.createElement('h1');
heading.textContent = 'পুরোনো শিরোনাম';
document.body.append(heading);

document.querySelector('h1').textContent = 'নতুন শিরোনাম';
console.log(document.querySelector('h1').textContent);
```

Output

```
নতুন শিরোনাম
```

সমাধান ২

solution.js · JavaScript — চলবে: Browser

```
const toggle = document.createElement('button');
toggle.id = 'theme';
toggle.type = 'button';
toggle.textContent = 'থিম বদলান';
document.body.append(toggle);

toggle.addEventListener('click', () => {
  const isDark = document.body.classList.toggle('dark');
  console.log(isDark ? 'Dark' : 'Light');
});
```

দুবার চাপলে:

Output

```
Dark
Light
```

`classList.toggle` জানায় Class এখন আছে কি না, তাই আলাদা করে `contains` লাগেনি।

সমাধান ৩ — `sayHello()` লেখার সঙ্গে সঙ্গে Function চলে যায়, এবং তার ফেরত মান `undefined` Listener হিসেবে দেওয়া হয়। Function-টি নিজে দিতে হবে, Call করে নয়: `button.addEventListener('click', sayHello)`।

সমাধান ৪ — শুধু `like: click`। Event Button-এ থেমে যায়, তাই `card` ও `document`-এর Listener চলে না। শিরোনামে Click করলে আগের মতোই `card: target H2` এবং `document`।

সমাধান ৫

solution.js · JavaScript চলবে: Browser

```
const fruits = ['আম', 'কাঁঠাল', 'লিচু'];
const list = document.createElement('ul');

const items = fruits.map((fruit) => {
  const item = document.createElement('li');
  item.textContent = fruit;
  return item;
});

list.append(...items);
document.body.append(list);

console.log(list.children.length);
console.log(list.lastElementChild.textContent);
```

Output

```
3
লিচু
```

সমাধান ৬

solution.js · JavaScript চলবে: Browser

```
const LIMIT = 120;
const field = document.createElement('textarea');
const counter = document.createElement('p');
field.id = 'note';
document.body.append(field, counter);

function updateCounter() {
  const used = field.value.length;
  counter.textContent = `${used}/${LIMIT}`;
  counter.classList.toggle('over', used > LIMIT);
  console.log(counter.textContent);
}

field.addEventListener('input', updateCounter);
updateCounter();
```

Page খোলার পর "হ্যালো JavaScript" লিখলে:

Output

```
0/120
17/120
```

"হ্যালো" চোখে দুটি অক্ষর (হ্যা, লো), কিন্তু `length` ছয় (অধ্যায় ৮)। Task Manager-এর সীমাও `length` দিয়ে, যা সংরক্ষণের সীমার জন্য উপযুক্ত।

সমাধান ৭

solution.js · JavaScript চলবে: Browser

```
const list = document.createElement('ul');

for (const name of ['ক', 'খ', 'গ']) {
  const row = document.createElement('li');
  const removeButton = document.createElement('button');
  removeButton.type = 'button';
  removeButton.dataset.name = name;
  removeButton.textContent = `মুছুন ${name}`;
  row.append(removeButton);
  list.append(row);
}
document.body.append(list);

list.addEventListener('click', (event) => {
  const button = event.target.closest('button');

  if (!button) {
    return;
  }

  button.closest('li').remove();
  console.log(`${button.dataset.name} মুছে গেছে; বাকি ${list.children.length}`);
});
```

"মুছুন খ" চাপলে:

Output

```
খ মুছে গেছে; বাকি 2
```

সমাধান ৮

solution.js · JavaScript চলবে: Browser

```
const input = document.createElement('input');
const list = document.createElement('ul');
input.id = 'item';
document.body.append(input, list);

input.addEventListener('keydown', (event) => {
  if (event.key !== 'Enter') {
    return;
  }

  const text = input.value.trim();

  if (text === '') {
    console.log('ফাঁকা; যোগ হলো না');
    return;
  }

  const item = document.createElement('li');
  item.textContent = text;
  list.append(item);
  input.value = '';
  console.log(`যোগ হলো: ${text}; মোট ${list.children.length}`);
});
```

শুধু ফাঁকা জায়গা লিখে Enter, তারপর "দুধ কেনা" লিখে Enter:

Output

```
ফাঁকা; যোগ হলো না
যোগ হলো: দুধ কেনা; মোট 1
```

এখানে `<form>` ছাড়াই `keydown` দিয়ে কাজ হয়েছে। Project-এ `<form>` ও `submit` ব্যবহার করা হয়েছে, কারণ তাতে Button ও Enter দুটোই বিনা বাড়তি Code-এ কাজ করে।

সমাধান ৯

solution.js · JavaScript চলবে: Browser

```
const THEME_KEY = 'jsm-theme';

function loadTheme() {
  try {
    return localStorage.getItem(THEME_KEY) === 'dark' ? 'dark' : 'light';
```

```
  } catch {
    return 'light';
  }
}

function saveTheme(theme) {
  try {
    localStorage.setItem(THEME_KEY, theme);
  } catch {
    console.log('থিম সংরক্ষণ করা গেল না');
  }
}

console.log(loadTheme());
saveTheme('dark');
console.log(loadTheme());
localStorage.setItem(THEME_KEY, 'purple');
console.log(loadTheme());
localStorage.removeItem(THEME_KEY);
```

Output

```
light
dark
light
```

Page খোলার সময় `document.body.classList.toggle('dark', loadTheme() === 'dark')` লিখলে সংরক্ষিত থিম প্রয়োগ হয়।

সমাধান ১০

solution.js · JavaScript চলবে: Browser

```
const search = document.createElement('input');
search.id = 'search';
document.body.append(search);

search.addEventListener('input', () => {
  console.log(`খুঁজছেন: ${search.value}`);
});

search.addEventListener('keydown', (event) => {
  if (event.key === 'Escape') {
    search.value = '';
    console.log('খোঁজা মুছে ফেলা হলো');
  }
});
```

"JavaScript" লিখে Escape চাপলে:

Output

```
খুঁজছেন: JavaScript
খোঁজা মুছে ফেলা হলো
```

Code দিয়ে `value` বদলালে `input` Event ঘটে না; তাই Escape-এর পর "খুঁজছেন" লাইনটি আবার আসেনি। Task Manager-এ এমন ক্ষেত্রে মান বদলানোর পর নিজে `render()` ডাকতে হবে।

সমাধান ১১

complete-all.js · JavaScript — চলবে: Browser Console ও Node.js

```
const completeAll = (tasks) => tasks.map((task) => ({ ...task, done: true }));

const tasks = [
  { id: '1', title: 'বাজার', done: false },
  { id: '2', title: 'বই', done: true },
];
const next = completeAll(tasks);

console.log(next.map((task) => task.done).join(', '));
console.log(tasks[0].done);
console.log(next[0] === tasks[0]);
```

Output

```
true, true
false
false
```

Pure Function বলে এটি Browser ছাড়াই পরীক্ষা করা গেল। Page-এ যোগ করতে একটি Button এবং তার Listener-এ শুধু `update(completeAll(tasks), 'সব সম্পন্ন।')`।

সমাধান ১২

undo.js · JavaScript — চলবে: Browser Console ও Node.js

```
function deleteWithUndo(tasks, id) {
  const index = tasks.findIndex((task) => task.id === id);

  if (index === -1) {
    return { tasks, undo: null };
  }
```

```
  const removed = tasks[index];
  const undo = (current) => [
    ...current.slice(0, index),
    removed,
    ...current.slice(index),
  ];

  return { tasks: tasks.filter((task) => task.id !== id), undo };
}

const start = [
  { id: 'a', title: 'ক' },
  { id: 'b', title: 'খ' },
  { id: 'c', title: 'গ' },
];
const { tasks, undo } = deleteWithUndo(start, 'b');

console.log(tasks.map((task) => task.title).join(''));
console.log(undo(tasks).map((task) => task.title).join(''));
console.log(deleteWithUndo(start, 'x').undo);
```

Output

```
কগ
কখগ
null
```

`undo` একটি Closure (অধ্যায় ৫): মুছে ফেলা Task এবং তার অবস্থান মনে রাখে। এটি বর্তমান তালিকা নেয়, তাই মোছার পর অন্য পরিবর্তন হয়ে থাকলেও সেগুলো হারায় না।

৯.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কিছু না মিললে `document.querySelector('#missing')` কী দেয়?

ক) `undefined`

খ) একটি Error

গ) `null`

ঘ) একটি ফাঁকা `NodeList`

প্রশ্ন ২. `<head>`-এ রাখা Script যেন পুরো HTML পড়ার পরে চলে, সে জন্য কী লিখবেন?

ক) `defer`

খ) `async`

গ) `type="module"` ছাড়া উপায় নেই

ঘ) `onload`

প্রশ্ন ৩. ব্যবহারকারীর লেখা একটি Element-এ নিরাপদে বসানোর উপায়?

ক) `innerHTML`

খ) `outerHTML`

গ) `insertAdjacentHTML`

ঘ) `textContent`

প্রশ্ন ৪. `button.addEventListener('click', save())`-এর সমস্যা কী?

ক) `click` ভুল Event-এর নাম

খ) `save` এখনই চলে; Listener হিসেবে যায় তার ফেরত মান

গ) Arrow Function বাধ্যতামূলক

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৫. Child Element-এর Click Parent-এর Listener-এও পৌঁছায়। এই আচরণের নাম?

ক) Bubbling

খ) Hoisting

গ) Delegation

ঘ) Rendering

প্রশ্ন ৬. Event Delegation-এর প্রধান সুবিধা কোনটি?

ক) Event দ্রুত ঘটে

খ) `preventDefault` লাগে না

গ) একটি Listener পরে তৈরি হওয়া Element-ও সামলায়

ঘ) `event.target` লাগে না

প্রশ্ন ৭. Submit Handler-এ `event.preventDefault()` কী থামায়?

ক) Event-এর ওপরে ওঠা

খ) Form পাঠিয়ে Page নতুন করে Load হওয়া

গ) অন্য সব Listener

ঘ) `input` Event

প্রশ্ন ৮. `localStorage.setItem('n', 5)`-এর পর `typeof localStorage.getItem('n')`?

ক) `'number'`

খ) `'object'`

গ) `'undefined'`

ঘ) `'string'`

প্রশ্ন ৯. Task Manager-এ তথ্যের নিয়ম আলাদা `task-core.js`-এ রাখার মূল কারণ?

ক) Browser দ্রুত চলে

খ) DOM ছাড়া Node.js-এ Unit Test করা যায়

গ) `localStorage` শুধু আলাদা File থেকে কাজ করে

ঘ) HTML-এ একটিমাত্র Script রাখা যায় না

প্রশ্ন ১০. `document.querySelectorAll('li').map(...)` কেন Error দেয়?

ক) `NodeList`-এ `map` নেই

খ) `li` ভুল Selector

গ) `querySelectorAll` সব সময় `null` দেয়

ঘ) Arrow Function চলে না

৯.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–ক · ৬–গ · ৭–খ · ৮–ঘ · ৯–খ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	একক Element না পেলে `null`; `querySelectorAll` দেয় ফাঁকা `NodeList`।
২	ক	`defer` File আগে আনে, কিন্তু চালায় HTML পড়া শেষে, লেখার ক্রমে।
৩	ঘ	`textContent` লেখাকে HTML হিসেবে পড়ে না; বাকিগুলো পড়ে।
৪	খ	Parenthesis Function Call করে; Listener-এ Function নিজে দিতে হয়।
৫	ক	Event DOM গাছ বেয়ে `document` পর্যন্ত ওপরে ওঠে।
৬	গ	Listener Parent-এ থাকে, তাই পরে যোগ হওয়া সন্তানের Event-ও পায়।
৭	খ	Browser-এর নিজস্ব আচরণ থামায়; Propagation থামায় `stopPropagation`।
৮	ঘ	`localStorage` সব কিছু লেখা হিসেবে রাখে।
৯	খ	Pure Function-এ DOM নেই, তাই Browser ছাড়াই পরীক্ষা করা যায়।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	ক	`NodeList` Array নয়; `[...list]` বা `Array.from(list)` দিয়ে আগে Array বানাতে হয়।

৯.১৯ অধ্যায়ের সারসংক্ষেপ

- DOM হলো Page-এর Element Object-এর গাছ; JavaScript এটি পড়ে ও বদলায়, Browser পর্দা হালনাগাদ করে।
- Script Element পড়ার আগে চললে `null`; Script `<body>`-র শেষে বা `defer`।
- খোঁজা `querySelector` / `querySelectorAll`; নিজের তথ্য `data-` Attribute ও `dataset`।
- বদলানো: `textContent`, `classList`, `setAttribute`, `hidden`; চেহারা CSS Class-এ।
- তৈরি `createElement`, যোগ `append` / `prepend`, সরানো `remove`, সব বদলানো `replaceChildren`।
- `addEventListener`-এ Function নিজে দিন; Event Object-এ `type`, `target`, `key`।
- Event Bubbling-এর কারণে Parent-এ একটি Listener দিয়ে Event Delegation; `closest` দিয়ে আসল Element।
- Form-এ `submit` এবং প্রথমেই `preventDefault`; Label, `aria-live`, Keyboard-এর ব্যবহারযোগ্যতা।
- এক State, এক `render`; তথ্যের নিয়ম DOM-মুক্ত Pure Function-এ।
- `localStorage` শুধু লেখা রাখে; পড়ার সময় যাচাই, ভাঙলে নিরাপদ মান; সংবেদনশীল তথ্য নয়।

পরের অধ্যায়ে Asynchronous JavaScript: অপেক্ষা করতে হয় এমন কাজ, যেমন Server থেকে তথ্য আনা, Page আটকে না দিয়ে কীভাবে করা যায়।

৯.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
DOM	Page-এর Element Object-গুলোর গাছ।
Selector	CSS-এর নিয়মে Element চেনার লেখা: `#id`, `.class`, `tag`।
NodeList	`querySelectorAll`-এর ফল; Array-র মতো, কিন্তু Array নয়।
`defer`	HTML পড়া শেষে Script চালানোর নির্দেশ।
Event	Browser-এ ঘটা একটি ঘটনা: Click, লেখা, Submit।
Event Listener	Event ঘটলে চলার জন্য দেওয়া Function।
Event Object	Listener-এ আসা Event-এর তথ্য।
Bubbling	Event-এর DOM গাছ বেয়ে ওপরে ওঠা।

পরিভাষা	অর্থ
Event Delegation	Parent-এ একটি Listener দিয়ে সন্তানদের Event সামলানো।
State	Application-এর বর্তমান তথ্য, যা থেকে Page আঁকা হয়।
Render	State থেকে Page-এর অংশ তৈরি করা।
`aria-live`	লেখা বদলালে Screen Reader-কে তা পড়তে বলার Attribute।

৯.২১ আরও পড়ুন

- **MDN — Introduction to the DOM:** `https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction` — DOM-এর মূল ধারণা।
- **MDN — Introduction to events:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events` — Event, Bubbling এবং Delegation।
- **MDN — Event bubbling:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling` — Propagation-এর বিস্তারিত।
- **MDN — Window.localStorage:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage` — Storage-এর নিয়ম ও সীমা।
- **web.dev — Learn Accessibility:** `https://web.dev/learn/accessibility` — সবার জন্য ব্যবহারযোগ্য Web Page।

অধ্যায় ১০

১০

Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Synchronous ও Asynchronous কাজের পার্থক্য, এবং দীর্ঘ কাজ কেন Page আটকে দেয়, তা ব্যাখ্যা করতে।
- Call Stack, Task Queue, Microtask Queue এবং Event Loop দিয়ে কোড কোন ক্রমে চলবে তা আগে থেকে বলতে।
- Callback, Promise এবং `async` / `await` দিয়ে অপেক্ষার কাজ লিখতে, এবং তিনটির সম্পর্ক বুঝতে।
- `try` / `catch` / `finally` দিয়ে Asynchronous ত্রুটি সামলাতে।
- `fetch` দিয়ে তথ্য আনতে, এবং HTTP ত্রুটি আলাদাভাবে ধরতে।
- `Promise.all`, `allSettled`, `race` ও `any` দিয়ে একাধিক কাজ একসঙ্গে চালাতে।
- `AbortController` দিয়ে কাজ বাতিল করতে, সময়সীমা বসাতে এবং Race Condition এড়াতে।
- সাময়িক ত্রুটিতে বিরতি বাড়িয়ে আবার চেষ্টা (Retry ও Backoff) করতে।
- একটি সম্পূর্ণ **User Directory API Explorer** তৈরি করতে, এবং কৃত্রিম Server দিয়ে তার Loading, Error ও Race-এর আচরণ পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function এবং Closure।
- অধ্যায় ৭: Object, Destructuring এবং JSON।
- অধ্যায় ৯: DOM, Event Listener এবং `render` -ভিত্তিক গঠন।
- Node.js 22 বা নতুন সংস্করণ, এবং একটি আধুনিক Browser।
- `chapter-10` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত প্রতিটি লাইন আগের লাইন শেষ হওয়ার সঙ্গে সঙ্গে চলেছে। কিন্তু বাস্তব Program-এর অনেক কাজে অপেক্ষা করতে হয়: Server থেকে তথ্য আসতে কয়েকশো মিলিসেকেন্ড, একটি File পড়তে কিছুক্ষণ, ব্যবহারকারী কখন Click করবেন তার ঠিক নেই। এই অপেক্ষার সময়ে Program থেমে থাকলে Page জমে যায়: Button কাজ করে না, লেখা যায় না, Scroll হয় না।

JavaScript এর সমাধান করে অপেক্ষার কাজকে পেছনে পাঠিয়ে, এবং কাজ শেষ হলে ফল নিয়ে ফিরে এসে। এই ধারণাকে বলে **Asynchronous Programming**। আধুনিক Web Application-এর প্রায় প্রতিটি অংশ এর ওপর নির্ভর করে, এবং এর ভুলগুলো সবচেয়ে কঠিন Bug-গুলোর মধ্যে পড়ে: মাঝে মাঝে ঘটে, পরীক্ষার সময় ঘটে না। এই অধ্যায়ে আপনি শুধু Syntax নয়, ভেতরের নিয়মটি শিখবেন, যাতে Program কোন ক্রমে চলবে তা আপনি নিজেই বলতে পারেন।

১০.১ Synchronous এবং Asynchronous

একটি রেস্তোরাঁর কথা ভাবুন। একজন পরিবেশনকর্মী অর্ডার নিয়ে রান্নাঘরে দেন, তারপর রান্না শেষ হওয়া পর্যন্ত দাঁড়িয়ে না থেকে পরের টেবিলে যান। রান্না হলে রান্নাঘর তাঁকে ডাকে। JavaScript ঠিক এভাবে কাজ করে: একজন কর্মী (একটি Thread), কিন্তু অপেক্ষার কাজ অন্যকে দিয়ে নিজে এগিয়ে যান।

restaurant.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('১. অর্ডার নেওয়া হলো');

setTimeout(() => {
  console.log('৩. খাবার তৈরি');
}, 0);

console.log('২. পরের অর্ডার');
```

Output

```
১. অর্ডার নেওয়া হলো
২. পরের অর্ডার
৩. খাবার তৈরি
```

`setTimeout(callback, 0)` মানে "এখনই" নয়; মানে "যত তাড়াতাড়ি সম্ভব, তবে এখনকার কাজ শেষ হওয়ার পরে"। এখনকার কাজ, অর্থাৎ পুরো File-টি চলা শেষ না হওয়া পর্যন্ত Callback অপেক্ষা করে।

JavaScript একসময়ে একটিই কাজ করে, তাই একটি দীর্ঘ হিসাব সব কিছু আটকে দেয়:

blocking.js · JavaScript — চলবে: Browser Console ও Node.js

```
const start = Date.now();

setTimeout(() => {
  const late = Date.now() - start >= 200 ? 'কমপক্ষে 200 ms' : 'কম';
  console.log(`Timer চলল, দেরি ${late}`);
}, 0);

while (Date.now() - start < 200) {
  // 200 ms ধরে শুধু ঘুরছে
}

console.log('Loop শেষ');
```

Output

```
Loop শেষ
Timer চলল, দেরি কমপক্ষে 200 ms
```

Timer-এর সময় `0` হলেও সেটি চলেছে Loop শেষ হওয়ার পরে। Browser-এ এমন একটি Loop চলার সময় Page-এর কোনো Click বা Scroll-ও কাজ করে না। শিক্ষা: Network বা Timer-এর মতো অপেক্ষা JavaScript-কে আটকায় না, কিন্তু JavaScript-এর নিজের দীর্ঘ হিসাব আটকায়।

১০.২ Event Loop: ভেতরের নিয়ম

Program কোন ক্রমে চলবে তা বুঝতে চারটি অংশ জানা দরকার:

অংশ	কাজ
Call Stack	এই মুহূর্তে যে Function-গুলো চলছে; একটির ভেতরে আরেকটি ডাকলে ওপরে জমে
Host-এর সুবিধা	Timer, Network, Event; Browser বা Node.js নিজে এগুলো সামলায়, JavaScript-এর বাইরে

অংশ	কাজ
Task Queue	Timer শেষ, Click, Network-এর সাড়া—চলার জন্য অপেক্ষমাণ Callback
Microtask Queue	Promise-এর ফল ধরার Callback (`then` , `await` -এর পরের অংশ)

Event Loop-এর নিয়ম মোটামুটি এমন:

১. Call Stack-এর বর্তমান কাজ শেষ পর্যন্ত চলে।

২. Stack খালি হলে Microtask Queue-এর **সব** Microtask চলে; চলার সময় নতুন Microtask এলে সেটিও।

৩. তারপর Task Queue থেকে **একটি** Task নিয়ে আবার ধাপ ১। Browser-এ এর ফাঁকে পর্দা নতুন করে আঁকা হতে পারে।

মূল কথা: **Promise-এর Callback সব সময় Timer-এর Callback-এর আগে চলে,** যদিও দুটিই "পরে" চলার কথা। নিচের Code-এর Output আগে নিজে ভেবে বলুন, তারপর মিলিয়ে দেখুন।

event-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('A: শুরু');

setTimeout(() => console.log('F: setTimeout'), 0);

Promise.resolve().then(() => console.log('D: Promise'));

async function task() {
  console.log('B: async Function, await-এর আগে');
  await null;
  console.log('E: await-এর পরে');
}

task();
console.log('C: শেষ লাইন');
```

Output

```
A: শুরু
B: async Function, await-এর আগে
C: শেষ লাইন
D: Promise
E: await-এর পরে
F: setTimeout
```

- **A, B, C** Synchronous: `task()` ডাকার সঙ্গে সঙ্গে তার প্রথম অংশ চলে; `async` Function `await` পর্যন্ত সাধারণ Function-এর মতোই।
- **D, E** Microtask: `then` -এর Callback এবং `await` -এর পরের অংশ। যে আগে Queue-তে ঢুকেছে, সে আগে।
- **F** Task: সব Microtask শেষ হওয়ার পর।

> **নোট:** এটি একটি সরলীকৃত মডেল। Browser ও Node.js-এর আসল Scheduler-এ আরও কয়েকটি Queue আছে, কিন্তু দৈনন্দিন Code-এর ক্রম বোঝার জন্য এই মডেলই যথেষ্ট।

১০.৩ Callback: প্রথম পদ্ধতি

অপেক্ষার কাজ শেষ হলে কী করতে হবে, তা একটি Function হিসেবে দিয়ে দেওয়া—এটিই **Callback**। Node.js-এর পুরোনো রীতিতে Callback-এর প্রথম Argument ত্রুটি, দ্বিতীয়টি ফল (Error-first Callback)।

callbacks.js · JavaScript — চলবে: Browser Console ও Node.js

```
function loadProfile(id, callback) {
  setTimeout(() => {
    if (id <= 0) {
      callback(new Error('ভুল ID'));
      return;
    }
    callback(null, { id, name: 'Sara' });
  }, 100);
}

loadProfile(7, (error, profile) => {
  if (error) {
    console.log(`ব্যর্থ: ${error.message}`);
    return;
  }
  console.log(`পাওয়া গেল: ${profile.name}`);
});

loadProfile(0, (error) => {
  console.log(`ব্যর্থ: ${error.message}`);
});

console.log('অপেক্ষা চলছে...');
```

Output

```
অপেক্ষা চলছে...
পাওয়া গেল: Sara
ব্যর্থ: ভুল ID
```

সব Callback Asynchronous নয়: `array.map(callback)` সঙ্গে সঙ্গে চলে। Callback শুধু "পরে ডাকার জন্য দেওয়া Function"; কখন ডাকা হবে তা নির্ভর করে যে Function সেটি নিচ্ছে তার ওপর।

সমস্যা দেখা দেয় যখন একটির পর একটি অপেক্ষার কাজ লাগে:

nesting.js · JavaScript চলবে: Browser Console ও Node.js

```
function step(name, callback) {
  setTimeout(() => callback(null, name), 50);
}

step('User', (error1, user) => {
  step(`${user}-এর Order`, (error2, orders) => {
    step(`${orders}-এর Invoice`, (error3, invoice) => {
      console.log(invoice);
    });
  });
});
```

Output

```
User-এর Order-এর Invoice
```

প্রতিটি ধাপ আগের ধাপের ভেতরে ঢুকছে, Code ডান দিকে সরছে, এবং তিনটি `error`-এর একটিও যাচাই হয়নি। প্রতিটি স্তরে যাচাই যোগ করলে Code দ্বিগুণ হতো। এই অবস্থার প্রচলিত নাম **Callback Hell**। Promise এর সমাধান হিসেবে এসেছে।

১০.৪ Promise: ভবিষ্যতের ফলের প্রতিশ্রুতি

Promise একটি Object, যা একটি অপেক্ষার কাজের ফলের প্রতিনিধিত্ব করে। এর তিনটি অবস্থা:

অবস্থা	অর্থ
pending	কাজ চলছে
fulfilled	সফলভাবে শেষ, একটি মান পাওয়া গেছে
rejected	ব্যর্থ, একটি ত্রুটি পাওয়া গেছে

একবার fulfilled বা rejected হলে Promise আর কখনো অবস্থা বদলায় না। ফল ধরতে `then`, ত্রুটি ধরতে `catch`, আর দুই ক্ষেত্রেই শেষে কিছু করতে `finally`।

promise-basics.js · JavaScript চলবে: Browser Console ও Node.js

```
function wait(ms, value) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(value), ms);
  });
}

function failAfter(ms, message) {
  return new Promise((resolve, reject) => {
```

```
    setTimeout(() => reject(new Error(message)), ms);
  });
}

wait(100, 'Sara')
  .then((name) => name.toUpperCase())
  .then((upper) => console.log(`ফল: ${upper}`))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`))
  .finally(() => console.log('প্রথম কাজ শেষ'));

failAfter(200, 'Server বন্ধ')
  .then(() => console.log('এটি চলবে না'))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`))
  .finally(() => console.log('দ্বিতীয় কাজ শেষ'));
```

Output

```
ফল: SARA
প্রথম কাজ শেষ
ত্রুটি: Server বন্ধ
দ্বিতীয় কাজ শেষ
```

- `new Promise((resolve, reject) => { ... })` একটি Promise তৈরি করে। ভেতরের Function সঙ্গে সঙ্গে চলে; কাজ শেষে `resolve(value)` বা `reject(error)` ডাকে।
- **প্রতিটি `then` একটি নতুন Promise ফেরত দেয়,** তাই Chain করা যায়। একটি `then` যা ফেরত দেয়, পরের `then` তা পায়। ফেরত মান নিজে Promise হলে পরের `then` সেটি শেষ হওয়া পর্যন্ত অপেক্ষা করে।
- **একটি `catch` আগের সব ধাপের ত্রুটি ধরে।** দ্বিতীয় Chain-এ প্রথম `then` বাদ পড়েছে, সরাসরি `catch` চলেছে।

আগের Callback Hell-এর উদাহরণ Promise দিয়ে লিখলে ধাপগুলো সমান্তরালে নিচে নামে, এবং একটি `catch` সব ধাপের ত্রুটি ধরে:

promise-chain.js · JavaScript — চলবে: Browser Console ও Node.js

```
const step = (text) =>
  new Promise((resolve) => setTimeout(() => resolve(text), 50));

step('User')
  .then((user) => step(`${user}-এর Order`))
  .then((orders) => step(`${orders}-এর Invoice`))
  .then((invoice) => console.log(invoice))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`));
```

Output

```
User-এর Order-এর Invoice
```

১০.৫ async এবং await

`async` / `await` Promise-এর ওপর তৈরি একটি লেখার ধরন, যাতে Asynchronous Code দেখতে Synchronous Code-এর মতো হয়।

async-await.js · JavaScript — চলবে: Browser Console ও Node.js

```
const step = (text) =>
  new Promise((resolve) => setTimeout(() => resolve(text), 50));

async function buildInvoice() {
  const user = await step('User');
  const orders = await step(`${user}-এর Order`);
  return step(`${orders}-এর Invoice`);
}

async function main() {
  try {
    const invoice = await buildInvoice();
    console.log(invoice);
  } catch (error) {
    console.log(`ত্রুটি: ${error.message}`);
  }
}

main();
console.log('main() ডাকা হলো; ফল পরে আসবে');
```

Output

```
main() ডাকা হলো; ফল পরে আসবে
User-এর Order-এর Invoice
```

- `await promise` Promise শেষ হওয়া পর্যন্ত **এই Function-কে** থামায়, এবং ফল দেয়। Promise rejected হলে `await` সেখানে Error ছোড়ে, যা সাধারণ `try` / `catch` ধরে।
- **পুরো Program থামে না।** শেষ লাইনটি আগে ছাপা হয়েছে: `main` যখন অপেক্ষা করছিল, বাকি Code এগিয়ে গেছে।
- `await` **শুধু** `async` **Function-এর ভেতরে** (এবং Module-এর সবচেয়ে ওপরের স্তরে, যা অধ্যায় ১২-এ আসবে)।

একটি `async` Function সব সময় Promise ফেরত দেয়, `return`-এ যা-ই লেখা থাকুক:

async-returns.js · JavaScript — চলবে: Node.js

```
async function getAnswer() {
  return 42;
}

const result = getAnswer();
console.log(result);
console.log(result instanceof Promise);
result.then((value) => console.log(value));
```

Output

```
Promise { 42 }
true
42
```

তাই `async` Function-এর ফল চাইলে, ডাকার জায়গায় আবার `await` বা `then` লাগে। (`Promise { 42 }` লেখাটি Node.js-এর দেখানোর ধরন; Browser Console Promise-কে অন্যভাবে দেখায়।)

১০.৬ fetch: Server থেকে তথ্য আনা

`fetch(url)` Browser ও Node.js-এ Network থেকে তথ্য আনে, এবং একটি Promise ফেরত দেয়, যা একটি **Response** Object-এ শেষ হয়। Response-এর মূল অংশ (Body) পড়তে আরেকটি অপেক্ষা লাগে: `response.json()` বা `response.text()`।

fetch-basics.js · JavaScript — চলবে: Browser Console ও Node.js

```
async function readJson(url) {
  const response = await fetch(url);
  console.log(`Status: ${response.status}, ok: ${response.ok}`);
  return response.json();
}

const url = 'data:application/json,{"name":"Sara","city":"Dhaka"}';

readJson(url).then((user) => console.log(`${user.name}, ${user.city}`));
```

Output

```
Status: 200, ok: true
Sara, Dhaka
```

উদাহরণটি Internet ছাড়াই চলে, কারণ `data:` ঠিকানায় তথ্য ঠিকানার ভেতরেই লেখা থাকে। একটি আসল Server-এর ক্ষেত্রে শুধু ঠিকানা বদলায়। নিচের Code-টি Browser-এর Console-এ চালিয়ে দেখতে পারেন; এর জন্য Internet সংযোগ লাগবে, এবং ফল Server-এর ওপর নির্ভর করে বলে বইয়ে তা ছাপা হয়নি:

real-api.js · JavaScript — চলবে: Browser

```
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then((response) => response.json())
  .then((user) => console.log(user.name, user.email));
```

HTTP ত্রুটি আলাদা

একটি গুরুত্বপূর্ণ ফাঁদ: Server `404` (পাওয়া যায়নি) বা `500` (Server-এ সমস্যা) পাঠালেও `fetch`-এর Promise **fulfilled** হয়। `fetch` শুধু তখনই rejected হয় যখন সাড়াই আসেনি: Network নেই, ঠিকানা ভুল, বা কাজ বাতিল। HTTP ত্রুটি নিজে যাচাই করতে হয় `response.ok` দিয়ে, যা Status 200 থেকে 299-এর মধ্যে হলে `true`।

http-errors.js · JavaScript — চলবে: Browser Console ও Node.js

```
// পরীক্ষার জন্য একটি কৃত্রিম fetch: আসল Network ছাড়াই নির্দিষ্ট সাড়া দেয়।
const fakeFetch = async (url) =>
  url.endsWith('/users')
    ? new Response('[{"id":1},{"id":2}]', { status: 200 })
    : new Response('Not Found', { status: 404 });

async function getJson(url) {
  const response = await fakeFetch(url);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

async function main() {
  for (const url of ['/api/users', '/api/userz']) {
    try {
      const data = await getJson(url);
      console.log(`${url}: ${data.length}টি User`);
    } catch (error) {
      console.log(`${url}: ব্যর্থ, ${error.message}`);
    }
  }
}

main();
```

Output

```
/api/users: 2টি User
/api/userz: ব্যর্থ, HTTP 404
```

এখানে একটি পেশাদার কৌশলও দেখা গেল: `fetch`-এর জায়গায় একটি **কৃত্রিম fetch**, যা সব সময় একই সাড়া দেয়। আসল Server-এর সাড়া প্রতিবার আলাদা হতে পারে, Server বন্ধও থাকতে পারে; কৃত্রিম সাড়া দিয়ে Test সব সময় একই ফল দেয়। Project-এর Test এই কৌশলেই লেখা।

১০.৭ একাধিক কাজ: একটির পর একটি, নাকি একসঙ্গে

দুটি কাজ যদি একটি অন্যটির ফলের ওপর নির্ভর না করে, তাহলে একটির পর একটি `await` করা সময়ের অপচয়।

sequential-vs-parallel.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));
const rounded = (start) => Math.round((Date.now() - start) / 100) * 100;

async function main() {
  let start = Date.now();
  const a = await wait(200, 'User');
  const b = await wait(200, 'Order');
  console.log(`একটির পর একটি: ${a}, ${b}; প্রায় ${rounded(start)} ms`);

  start = Date.now();
  const [c, d] = await Promise.all([wait(200, 'User'), wait(200, 'Order')]);
  console.log(`একসঙ্গে: ${c}, ${d}; প্রায় ${rounded(start)} ms`);
}

main();
```

Output

```
একটির পর একটি: User, Order; প্রায় 400 ms
একসঙ্গে: User, Order; প্রায় 200 ms
```

`Promise.all` দুটি কাজই একসঙ্গে শুরু করে এবং দুটির শেষ হওয়ার জন্য অপেক্ষা করে; ফলগুলো আসে দেওয়া ক্রমে, শেষ হওয়ার ক্রমে নয়। কিন্তু কাজ যদি নির্ভরশীল হয় (Order আনতে User-এর ID লাগে), তাহলে একটির পর একটিই একমাত্র উপায়।

একসঙ্গে চালানোর চারটি উপায়, ব্যর্থতার ক্ষেত্রে আলাদা আচরণ:

combinators.js · JavaScript চলবে: Browser Console ও Node.js

```
const ok = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));
const fail = (ms, message) =>
  new Promise((_, reject) => setTimeout(() => reject(new Error(message)), ms));

async function main() {
  try {
    await Promise.all([ok(50, 'ক'), fail(100, 'খ ব্যর্থ'), ok(150, 'গ')]);
  } catch (error) {
    console.log(`all: ${error.message}`);
  }

  const settled = await Promise.allSettled([ok(50, 'ক'), fail(100, 'খ ব্যর্থ')]);
  const report = settled.map((result) =>
    result.status === 'fulfilled' ? result.value : result.reason.message);
  console.log(`allSettled: ${report.join(' | ')}`);

  const fastest = await Promise.race([ok(150, 'ধীর'), ok(50, 'দ্রুত')]);
  console.log(`race: ${fastest}`);

  const firstOk = await Promise.any([fail(50, 'x'), ok(100, 'প্রথম সফল')]);
  console.log(`any: ${firstOk}`);
}

main();
```

Output

```
all: খ ব্যর্থ
allSettled: ক | খ ব্যর্থ
race: দ্রুত
any: প্রথম সফল
```

পদ্ধতি	কখন শেষ	কখন ব্যবহার
`Promise.all`	সব সফল হলে; একটি ব্যর্থ হলেই ব্যর্থ	সবগুলো ছাড়া কাজ এগোয় না
`Promise.allSettled`	সব শেষ হলে, সফল বা ব্যর্থ	প্রতিটির ফলাফল আলাদাভাবে জানাতে
`Promise.race`	প্রথম যেটি শেষ হয়, সফল বা ব্যর্থ	সময়সীমা বসাতে
`Promise.any`	প্রথম সফলটি; সব ব্যর্থ হলে ব্যর্থ	একাধিক উৎসের যেকোনো একটি যথেষ্ট

১০.৮ বাতিল করা, সময়সীমা এবং Race Condition

ব্যবহারকারী একটি Page ছেড়ে চলে গেলেন, বা নতুন কিছু খুঁজলেন—পুরোনো Request-এর আর দরকার নেই। `AbortController` দিয়ে চলমান কাজ বাতিল করা যায়। Controller-এর `signal` কাজটিকে দেওয়া হয়; `controller.abort()` ডাকলে কাজটি একটি `AbortError` দিয়ে rejected হয়। `fetch(url, { signal })` এটি সরাসরি বোঝে।

abort.js · JavaScript — চলবে: Browser Console ও Node.js

```
function wait(ms, signal) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => resolve(`${ms} ms পরে শেষ`), ms);
    signal.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(signal.reason);
    });
  });
}

const controller = new AbortController();

wait(500, controller.signal)
  .then((text) => console.log(text))
  .catch((error) => console.log(`বাতিল: ${error.name}`));

setTimeout(() => controller.abort(), 100);

wait(500, AbortSignal.timeout(150))
  .then((text) => console.log(text))
  .catch((error) => console.log(`সময় শেষ: ${error.name}`));
```

Output

```
বাতিল: AbortError
সময় শেষ: TimeoutError
```

`AbortSignal.timeout(ms)` নির্দিষ্ট সময় পরে নিজে নিজেই বাতিল হয়, এবং ত্রুটির নাম হয় `TimeoutError`। একটি Server সাড়া না দিলে ব্যবহারকারীকে অনির্দিষ্টকাল অপেক্ষা করানো উচিত নয়; প্রতিটি Network Request-এ একটি সময়সীমা থাকা উচিত।

Race Condition

ব্যবহারকারী Search Box-এ "Ja" লিখলেন, একটি Request গেল। তারপর "JavaScript" লিখলেন, আরেকটি Request গেল। কোনো কারণে প্রথম Request-এর সাড়া দেরিতে এলো। কোন ফল পর্দায় থাকবে?

race-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

// পুরোনো খোঁজার সাড়া ধীরে আসে, নতুনটির দ্রুত।
const searchServer = (query) =>
  wait(query === 'Ja' ? 300 : 100, `"${query}"-এর ফল`);

let shown = '';
let latest = 0;

async function onType(query, guard) {
  const ticket = ++latest;
  const result = await searchServer(query);

  if (guard && ticket !== latest) {
    console.log(`পুরোনো ফল বাদ: ${result}`);
    return;
  }
  shown = result;
}

async function demo(guard) {
  await Promise.all([onType('Ja', guard), onType('JavaScript', guard)]);
  console.log(`${guard ? 'Guard সহ' : 'Guard ছাড়া'}: পর্দায় ${shown}`);
}

demo(false).then(() => demo(true));
```

Output

```
Guard ছাড়া: পর্দায় "Ja"-এর ফল
পুরোনো ফল বাদ: "Ja"-এর ফল
Guard সহ: পর্দায় "JavaScript"-এর ফল
```

Guard ছাড়া, যে সাড়া **শেষে এসেছে** সেটি পর্দায় থেকে গেছে, যদিও তা পুরোনো খোঁজার। দুটি কাজ কোন ক্রমে শেষ হবে তার ওপর ফল নির্ভর করছে—এটিই **Race Condition**। এই Bug সাধারণত দ্রুত Network-এ ধরা পড়ে না, আর ধীর Mobile Network-এ ব্যবহারকারীর কাছে ধরা পড়ে।

সমাধান দুটি, এবং Project দুটোই ব্যবহার করে: প্রতিটি Request-এ একটি ক্রমিক নম্বর দিয়ে শুধু সর্বশেষটির ফল গ্রহণ করা (ওপরের `ticket`), এবং নতুন Request শুরুর আগে পুরোনোটি `AbortController` দিয়ে বাতিল করা, যাতে অপ্রয়োজনীয় Network-এর কাজও থামে।

১০.৯ Retry, Timeout এবং Backoff

কিছু ত্রুটি সাময়িক: Server এক মুহূর্তের জন্য ব্যস্ত, Network ক্ষণিকের জন্য বিচ্ছিন্ন। এসব ক্ষেত্রে একটু থেমে আবার চেষ্টা করলে প্রায়ই সফল হয়। প্রতিবার বিরতি দ্বিগুণ করাকে বলে **Exponential Backoff**: এতে সমস্যায় থাকা Server-এ চাপ কমে।

retry.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

let calls = 0;
async function flakyServer() {
  calls += 1;
  if (calls < 3) {
    throw new Error(`চেষ্টা ${calls}: Server ব্যস্ত`);
  }
  return 'তথ্য পাওয়া গেছে';
}

async function withRetry(task, retries = 3, baseDelay = 100) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      const delay = baseDelay * 2 ** attempt;
      console.log(`${error.message}; ${delay} ms পরে আবার`);
      await wait(delay);
    }
  }
}

withRetry(flakyServer).then((data) => console.log(data));
```

Output

```
চেষ্টা 1: Server ব্যস্ত; 100 ms পরে আবার
চেষ্টা 2: Server ব্যস্ত; 200 ms পরে আবার
তথ্য পাওয়া গেছে
```

- `return await task()` : এখানে `await` অপরিহার্য। শুধু `return task()` লিখলে Promise-টি `try` থেকে বেরিয়ে যেত, এবং তার ব্যর্থতা এই `catch` ধরত না।
- **সব ত্রুটিতে আবার চেষ্টা নয়।** `404` (ঠিকানা নেই) বা `400` (ভুল Request) আবার পাঠালেও একই ফল আসবে। Retry শুধু সাময়িক ত্রুটিতে: Network বিচ্ছিন্ন, সময়সীমা পার, বা `5xx` Status। ব্যবহারকারী নিজে বাতিল করলে

কখনোই নয়।

- **সীমা রাখুন:** চেষ্টার সংখ্যা সীমিত, এবং মোট অপেক্ষা যুক্তিসঙ্গত।

১০.১০ সাধারণ ভুল

সবচেয়ে প্রচলিত ভুল `await` ভুলে যাওয়া:

forgot-await.js · JavaScript — চলবে: Node.js

```
async function getUser() {
  return { name: 'Sara' };
}

const user = getUser();
console.log(user.name);
console.log(user);
```

Output

```
undefined
Promise { { name: 'Sara' } }
```

`user` আসলে একটি Promise, User নয়; তাই `user.name` হলো `undefined`। কোনো Error হয় না, যা ভুলটিকে খুঁজে পাওয়া কঠিন করে।

দ্বিতীয়টি: `forEach`-এর ভেতরে `await`। `forEach` Callback-এর Promise-এর জন্য অপেক্ষা করে না।

foreach-await.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

async function withForEach(ids) {
  const names = [];
  ids.forEach(async (id) => {
    names.push(await wait(50, `User ${id}`));
  });
  return names.length;
}

async function withForOf(ids) {
  const names = [];
  for (const id of ids) {
    names.push(await wait(50, `User ${id}`));
  }
  return names.length;
```

```
}

withForEach([1, 2, 3]).then((count) => console.log(`forEach: ${count}টি`));
withForOf([1, 2, 3]).then((count) => console.log(`for...of: ${count}টি`));
```

Output

```
forEach: 0টি
for...of: 3টি
```

তৃতীয়টি: ধরা না পড়া Rejection। একটি rejected Promise-এর কোনো `catch` না থাকলে Browser Console-এ "Uncaught (in promise)" দেখায়, আর Node.js পুরো Program বন্ধ করে দেয়।

unhandled.js · JavaScript — চলবে: Node.js

```
async function save() {
  throw new Error('Disk ভর্তি');
}

save();
console.log('এই লাইন চলে');
```

Error Message

```
এই লাইন চলে
Error: Disk ভর্তি
```

ভুল	কী দেখা যায়	সমাধান
`await` ভুলে যাওয়া	`undefined`, বা `[object Promise]` লেখা	`async` Function-এর ফলে `await`
`async` ছাড়া Function-এ `await`	`SyntaxError`	Function-কে `async` করুন
`forEach`-এ `await`	কাজ শেষের আগেই পরের লাইন	`for...of`, অথবা `Promise.all` ও `map`
স্বাধীন কাজ একটির পর একটি `await`	অকারণে ধীর	`Promise.all`
`fetch`-এর পর `response.ok` যাচাই না করা	`404`-এর লেখা JSON হিসেবে পড়তে গিয়ে বিভ্রান্তিকর Error	`if (!response.ok) throw ...`
`catch` নেই	Uncaught (in promise); Node.js বন্ধ	প্রতিটি Chain-এর শেষে `catch`, বা `try` / `catch`

ভুল	কী দেখা যায়	সমাধান
`try`-এর ভেতরে `return promise`	ত্রুটি `catch`-এ ধরা পড়ে না	`return await promise`
সময়সীমা নেই	Loading অনির্দিষ্টকাল ঘোরে	`AbortSignal.timeout()`
পুরোনো সাড়া নতুনটির ওপরে	ভুল ফল পর্দায়	ক্রমিক নম্বর, `AbortController`
সব ত্রুটিতে Retry	`404`-এ অকারণে বারবার চেষ্টা	শুধু সাময়িক ত্রুটিতে

১০.১১ Debugging নির্দেশিকা

- **Network Tab (DevTools):** প্রতিটি Request-এর ঠিকানা, Status, সময় এবং সাড়ার লেখা। "তথ্য আসছে না" সমস্যায় প্রথম প্রশ্ন: Request আদৌ গেছে কি, এবং Status কত।
- **Throttling:** Network Tab-এর "No throttling" তালিকা থেকে "Slow 4G" বা "3G" বেছে নিলে ধীর Network-এর অভিজ্ঞতা পাওয়া যায়। Loading অবস্থা এবং Race Condition এভাবেই চোখে পড়ে।
- **Offline:** একই তালিকার "Offline" দিয়ে Network বিচ্ছিন্নতার আচরণ দেখুন।
- **Async Stack Trace:** DevTools এবং Node.js দুটিই Error-এর Stack-এ `await`-এর আগের Function-গুলোও দেখায়; Stack-এর প্রতিটি লাইন পড়ুন।
- **Breakpoint `await`-এর পরের লাইনে** দিলে ফল আসার মুহূর্তে থামে, এবং পাওয়া মান দেখা যায়।

তথ্য প্রত্যাশামতো না এলে ক্রমে দেখুন:

১. Console-এ লাল Error বা "Uncaught (in promise)" আছে কি না।
২. Network Tab-এ Request গেছে কি না, Status এবং সাড়ার লেখা কী।
৩. যে Function Promise ফেরত দেয়, তার আগে `await` আছে কি না।
৪. `response.ok` যাচাই এবং `response.json()`-এর আগে `await` আছে কি না।
৫. ফল একাধিকবার আসছে, বা পুরোনো ফল নতুনটির ওপরে বসছে কি না।

১০.১২ পেশাদার অভ্যাস

- **নতুন Code-এ `async` / `await`,** এবং প্রতিটি `await`-এর জন্য ত্রুটি কোথায় ধরা হবে তা ঠিক করে রাখুন।
- **প্রতিটি Network Request-এ সময়সীমা,** এবং ব্যবহারকারী-চালিত Request-এ বাতিলের ব্যবস্থা।
- **তিনটি অবস্থা দেখান:** Loading, ফলাফল, এবং ত্রুটি—এবং ত্রুটি থেকে বেরোনোর উপায় (আবার চেষ্টার Button)।
- **ব্যবহারকারীর ভাষায় বার্তা:** "HTTP 503" নয়, "Server-এ সমস্যা হয়েছে। একটু পরে আবার চেষ্টা করুন।" প্রযুক্তিগত বিবরণ Console-এ।

- **Server-এর তথ্যকে অবিশ্বাস করুন:** গঠন যাচাই করুন, অপ্রত্যাশিত তথ্যে ভেঙে পড়বেন না (অধ্যায় ৯-এর `parseStored`-এর মতো)।
- **Network-কে Inject করুন:** Function-কে `fetch`-এর বদলে একটি `fetchImpl` দেওয়া যায় এমনভাবে লিখুন, যাতে Test-এ কৃত্রিম সাড়া দেওয়া যায়।
- **স্বাধীন কাজ একসঙ্গে,** নির্ভরশীল কাজ ক্রমে।

১০.১৩ Project: User Directory API Explorer

একটি Web Page, যা একটি Server থেকে User-দের তালিকা আনে, এবং নাম, Email বা শহর দিয়ে খোঁজা ও সাজানোর সুযোগ দেয়। মূল লক্ষ্য Network-এর সব বাস্তব পরিস্থিতি সঠিকভাবে সামলানো: ধীর Server, ত্রুটি, সাময়িক বিভ্রাট, ভাঙা তথ্য এবং দ্রুত পরপর Request।

Project-এর চাহিদা

- Page খুললে User-দের তালিকা আনা; চলাকালীন "লোড হচ্ছে..." দেখানো।
- খোঁজা (নাম, Email, শহর) এবং সাজানো (নাম, Email, শহর) Server-এ নতুন Request ছাড়াই।
- প্রতিটি Request-এ সময়সীমা; সাময়িক ত্রুটিতে (Network, সময়সীমা, `5xx`) দুবার পর্যন্ত Backoff-সহ আবার চেষ্টা; `4xx`-এ নয়।
- ত্রুটিতে ব্যবহারকারীর ভাষায় বার্তা, এবং "আবার লোড করুন" দিয়ে নতুন চেষ্টা।
- "আবার লোড করুন" চলমান Request বাতিল করে নতুনটি শুরু করবে; পুরোনো সাড়া কখনো নতুনটির ওপরে বসবে না।
- Server-এর তথ্য যাচাই; ভুল গঠনের User বাদ।
- সব লেখা `textContent` দিয়ে; বার্তা `aria-live`-এ; Loading-এর সময় তালিকায় `aria-busy`।
- Test-এ কোনো Public API নয়: Unit Test কৃত্রিম `fetch` দিয়ে, Integration ও Browser Test একটি Local Server দিয়ে।

File-এর কাঠামো

Folder-এর কাঠামো

```
user-directory-explorer/
├── index.html
├── style.css
├── directory.js            তথ্য আনা ও সাজানোর নিয়ম; DOM নেই
├── app.js                  Page, Event ও অবস্থা
├── README.md
└── tests/
    ├── load-script.js
    ├── directory.test.js   কৃত্রিম fetch এবং Local Server দিয়ে Test
    ├── mock-server.js      পরীক্ষার জন্য নিয়ন্ত্রিত Local Server
    ├── cdp.mjs
    └── browser-check.mjs   আসল Chrome-এ Page-এর Test
```

index.html ও style.css

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>User Directory</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>User Directory</h1>

    <div class="controls">
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>সাজান
        <select id="sort">
          <option value="name">নাম</option>
          <option value="email">Email</option>
          <option value="city">শহর</option>
        </select>
      </label>
      <button id="reload" type="button">আবার লোড করুন</button>
    </div>

    <p id="status" role="status" aria-live="polite"></p>
    <ul id="users" aria-busy="false"></ul>
  </main>

  <script src="directory.js"></script>
```

```
  <script src="app.js"></script>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 720px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

.controls {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

input,
select,
button {
  font: inherit;
  padding: 6px 10px;
}

#users {
  padding: 0;
  list-style: none;
}

#users li {
  padding: 10px 0;
  border-bottom: 1px solid #e3e7ef;
}

#users[aria-busy='true'] {
  opacity: 0.5;
}
```

শেষ নিয়মটি লক্ষ করুন: Loading-এর সময় তালিকা ঝাপসা হয়, এবং তা নির্ধারিত হয় `aria-busy` Attribute দিয়ে, যা Screen Reader-কেও একই তথ্য দেয়। চেহারা ও Accessibility একই উৎস থেকে।

directory.js

তথ্য আনা, যাচাই, খোঁজা ও সাজানোর সব নিয়ম; DOM-এর কোনো উল্লেখ নেই। দুই অংশ মিলে সম্পূর্ণ File।

directory.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
'use strict';

const USERS_URL = 'https://jsonplaceholder.typicode.com/users';
const SORT_KEYS = ['name', 'email', 'city'];

// API-র একটি User থেকে Page-এর প্রয়োজনীয় অংশ; ভুল গঠনে null।
function normalizeUser(raw) {
  if (!Number.isInteger(raw?.id) || typeof raw.name !== 'string') {
    return null;
  }

  return {
    id: raw.id,
    name: raw.name.trim(),
    email: typeof raw.email === 'string' ? raw.email.trim() : '',
    city: typeof raw.address?.city === 'string' ? raw.address.city : 'অজানা',
  };
}

function filterUsers(users, query = '') {
  const needle = query.trim().toLowerCase();

  return users.filter((user) =>
    [user.name, user.email, user.city].some((text) =>
      text.toLowerCase().includes(needle)));
}

function sortUsers(users, key = 'name') {
  const field = SORT_KEYS.includes(key) ? key : 'name';
  return [...users].sort((a, b) => a[field].localeCompare(b[field]));
}

// ms মিলিসেকেন্ড অপেক্ষা; signal বাতিল হলে সঙ্গে সঙ্গে rejected।
function wait(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) {
      reject(signal.reason);
      return;
    }
```

```
    const timer = setTimeout(resolve, ms);
    signal?.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(signal.reason);
    }, { once: true });
  });
}

// ...
```

অংশ ২: Network, Retry এবং বার্তা

directory.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

// একবার চেষ্টা: সময়সীমা, বাতিল এবং HTTP ত্রুটি সামলায়।
async function fetchJson(url, options = {}) {
  const { fetchImpl = fetch, signal, timeoutMs = 5000 } = options;
  const timeout = AbortSignal.timeout(timeoutMs);
  const combined = signal ? AbortSignal.any([signal, timeout]) : timeout;
  const headers = { Accept: 'application/json' };
  const response = await fetchImpl(url, { signal: combined, headers });

  if (!response.ok) {
    const error = new Error(`HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return response.json();
}

// সাময়িক ত্রুটি: Network বিচ্ছিন্ন, সময়সীমা পার, অথবা Server-এর 5xx।
const isTemporary = (error) =>
  error.name === 'TypeError' || error.name === 'TimeoutError' ||
  error.status >= 500;

async function fetchWithRetry(url, options = {}) {
  const { retries = 2, backoffMs = 300, signal } = options;

  for (let attempt = 0; ; attempt += 1) {
    try {
      return await fetchJson(url, options);
    } catch (error) {
      if (attempt >= retries || signal?.aborted || !isTemporary(error)) {
        throw error;
      }
      await wait(backoffMs * 2 ** attempt, signal);
    }
```

```
    }
  }

  async function loadUsers(options = {}) {
    const data = await fetchWithRetry(options.url ?? USERS_URL, options);

    if (!Array.isArray(data)) {
      throw new Error('Server থেকে অপ্রত্যাশিত তথ্য এসেছে।');
    }
    return data.map(normalizeUser).filter(Boolean);
  }

  // প্রযুক্তিগত ত্রুটি থেকে ব্যবহারকারীর ভাষায় বার্তা।
  function errorMessage(error) {
    if (error.name === 'TimeoutError') {
      return 'Server সাড়া দিতে দেরি করছে। একটু পরে আবার চেষ্টা করুন।';
    }
    if (error.status >= 500) {
      return 'Server-এ সমস্যা হয়েছে। একটু পরে আবার চেষ্টা করুন।';
    }
    if (error.status === 404) {
      return 'তথ্যের ঠিকানা পাওয়া যায়নি।';
    }
    if (error.name === 'TypeError') {
      return 'Network-এ সংযোগ নেই। সংযোগ দেখে আবার চেষ্টা করুন।';
    }
    return 'তথ্য আনা যায়নি। আবার চেষ্টা করুন।';
  }
```

- **`fetchImpl = fetch`**: স্বাভাবিক অবস্থায় আসল `fetch`, Test-এ কৃত্রিম একটি (পেশাদার অভ্যাস: Network-কে Inject করা)।
- **`AbortSignal.any([signal, timeout])`** দুটি Signal জোড়া দেয়: ব্যবহারকারী বাতিল করলে, অথবা সময় শেষ হলে—যেটি আগে ঘটে।
- **`fetchWithRetry`** অনুচ্ছেদ ১০.৯-এর Retry, তিনটি শর্তসহ: চেষ্টা শেষ, ব্যবহারকারী বাতিল করেছেন, অথবা ত্রুটি সাময়িক নয়—যেকোনোটি হলে আর চেষ্টা নয়। Backoff-এর অপেক্ষাও `signal` মানে, তাই বাতিল করলে অপেক্ষার মাঝেই থামে।
- **`loadUsers`** Server-এর তথ্য যাচাই করে: Array না হলে ত্রুটি, আর `normalizeUser` থেকে `null` আসা User-গুলো `filter(Boolean)` দিয়ে বাদ।
- **`errorMessage`** ক্রম গুরুত্বপূর্ণ: Network ত্রুটির নাম `TypeError`, কিন্তু Program-এর ভুলও `TypeError` হতে পারে। তাই নির্দিষ্ট ক্ষেত্রগুলো আগে, সাধারণ বার্তা শেষে।

app.js

Page-এর সঙ্গে সব যোগাযোগ এবং তিনটি অবস্থা (Loading, ফলাফল, ত্রুটি)। দুই অংশ মিলে সম্পূর্ণ File।

app.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
'use strict';

// ?api= ও ?timeout= শুধু পরীক্ষার জন্য; স্বাভাবিক ব্যবহারে আসল API।
const params = new URLSearchParams(location.search);
const API_URL = params.get('api') ?? USERS_URL;
const TIMEOUT_MS = Number(params.get('timeout') ?? 5000);

const search = document.querySelector('#search');
const sortSelect = document.querySelector('#sort');
const reloadButton = document.querySelector('#reload');
const statusView = document.querySelector('#status');
const list = document.querySelector('#users');

let users = [];
let controller = null;

function userCard(user) {
  const item = document.createElement('li');
  const name = document.createElement('h2');
  const details = document.createElement('p');

  name.textContent = user.name;
  details.textContent = `${user.email} · ${user.city}`;
  item.append(name, details);
  return item;
}

function render() {
  const shown = sortUsers(filterUsers(users, search.value), sortSelect.value);
  list.replaceChildren(...shown.map(userCard));
  statusView.textContent = users.length === 0
    ? 'কোনো User পাওয়া যায়নি।'
    : `${users.length} জনের মধ্যে ${shown.length} জন দেখানো হচ্ছে।`;
}

// ...
```

app.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

function showLoading(isLoading) {
  list.setAttribute('aria-busy', String(isLoading));
  if (isLoading) {
    statusView.textContent = 'লোড হচ্ছে...';
  }
}
```

```
async function refresh() {
  controller?.abort();
  const current = new AbortController();
  controller = current;
  showLoading(true);

  try {
    const loaded = await loadUsers({
      url: API_URL,
      signal: current.signal,
      timeoutMs: TIMEOUT_MS,
    });

    if (controller !== current) {
      return;
    }
    users = loaded;
    render();
  } catch (error) {
    if (controller !== current) {
      return;
    }
    users = [];
    list.replaceChildren();
    statusView.textContent = errorMessage(error);
    console.warn('User আনা যায়নি:', error);
  } finally {
    if (controller === current) {
      showLoading(false);
    }
  }
}

search.addEventListener('input', render);
sortSelect.addEventListener('change', render);
reloadButton.addEventListener('click', refresh);

refresh();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **`refresh`-এর শুরু।** আগের Request চললে `controller?.abort()` সেটি বাতিল করে। তারপর এই Request-এর নিজস্ব Controller তৈরি হয় এবং `controller`-এ বসে।
- **"আমি কি এখনো সর্বশেষ?"** `await`-এর পরে `controller !== current` মানে এর মধ্যে নতুন Request শুরু হয়েছে; তখন এই ফল বা ত্রুটি কিছুই পর্দায় যায় না। এটি অনুচ্ছেদ ১০.৮-এর `ticket`-এর কাজটিই করে, আলাদা Counter ছাড়া। বাতিল হওয়া Request-এর `AbortError`-ও এই পথেই নিঃশব্দে বাদ পড়ে।

- `finally` সফল হোক বা ব্যর্থ, Loading বন্ধ করে; তবে শুধু সর্বশেষ Request-এর জন্য, যাতে পুরোনো Request নতুনটির Loading অবস্থা মুছে না দেয়।
- **ত্রুটি।** ব্যবহারকারী পান `errorMessage`-এর বোধগম্য বার্তা, ডেভেলপার পান Console-এ পুরো ত্রুটি। পুরোনো তালিকা মুছে দেওয়া হয়, যাতে ত্রুটির বার্তার পাশে বাসি তথ্য না থাকে।
- **খোঁজা ও সাজানো** Network ছোঁয় না; আগে আনা `users` থেকে শুধু `render`।
- `?api=` **এবং** `?timeout=` স্বাভাবিক ব্যবহারে থাকে না। Browser Test এগুলো দিয়ে Page-কে আসল API-র বদলে Local Server-এর দিকে পাঠায়।

চালানো এবং প্রত্যাশিত আচরণ

Browser-এর নিরাপত্তা-নিয়মের কারণে `file://` থেকে খোলা Page কিছু Server-এ Request পাঠাতে পারে না। তাই Project Folder থেকে একটি Local Server চালান, তারপর Browser-এ `http://localhost:8000` খুলুন (Internet সংযোগ লাগবে):

Terminal

```
python -m http.server 8000
```

Test চালাতে (Internet লাগে না):

Terminal

```
node --test
node tests/browser-check.mjs
```

পরিস্থিতি	যা হওয়ার কথা
Page খোলা	"লোড হচ্ছে...", তারপর তালিকা এবং "10 জনের মধ্যে 10 জন দেখানো হচ্ছে।"
খুঁজুন বা সাজান	সঙ্গে সঙ্গে তালিকা বদলায়; Network-এ নতুন Request যায় না
DevTools-এ Offline, তারপর "আবার লোড করুন"	কয়েক মুহূর্ত পর Network-এর বার্তা
Server `503` দুবার, তারপর সফল	Backoff-সহ আবার চেষ্টা; শেষে তালিকা
Server `404`	সঙ্গে সঙ্গে বার্তা; আবার চেষ্টা হয় না
Server সাড়া দেয় না	সময়সীমা পার হলে দেরির বার্তা
Loading চলাকালীন আবার "আবার লোড করুন"	পুরোনো Request বাতিল; শুধু নতুনটির ফল

Companion Source Code-এর `directory.test.js` কৃত্রিম `fetch` এবং একটি Local Server দিয়ে ১৮টি Test চালায়; `browser-check.mjs` আসল Chrome-এ Page খুলে ওপরের সারণির Local Server-এর পরিস্থিতিগুলোসহ ১২টি আচরণ যাচাই করে। কোনো Test Public API ব্যবহার করে না, তাই ফল প্রতিবার একই।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`loadUsers is not defined`	`directory.js` লোড হয়নি বা ক্রম উল্টো	Script-এর ক্রম
"লোড হচ্ছে..." থেকে নড়ে না	`finally` নেই, বা Promise কখনো শেষ হয় না	`finally`; সময়সীমা
CORS ত্রুটি Console-এ	`file://` থেকে খোলা, বা Server অনুমতি দেয় না	Local Server দিয়ে চালান
`404`-এ বারবার চেষ্টা	Retry সব ত্রুটিতে	`isTemporary`
দ্রুত দুবার Click-এ পুরোনো তালিকা ফেরে	সর্বশেষ Request যাচাই নেই	`controller !== current`
তালিকায় `undefined`	API-র গঠন বদলেছে	`normalizeUser` এবং যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। JSONPlaceholder একটি Demo API: তথ্য কাল্পনিক, এবং তার প্রাপ্যতার নিশ্চয়তা নেই। Page-এ Pagination নেই; হাজারো User-এর জন্য Server-এ খোঁজা ও ভাগে ভাগে আনা লাগত। একবার আনা তথ্য Cache করা হয় না।

১০.১৪ অনুশীলনী

প্রাথমিক স্তর

১. না চালিয়ে বলুন Output-এর ক্রম কী হবে: `console.log('ক')`, `setTimeout` দিয়ে `'খ'`, `Promise.resolve().then` দিয়ে `'গ'`, `console.log('ঘ')`।

২. `wait(ms)` নামে একটি Function লিখুন, যা `ms` মিলিসেকেন্ড পরে fulfilled হয়। এটি দিয়ে `'১'` দেখিয়ে ৩০০ মিলিসেকেন্ড পরে `'২'` দেখান।

৩. `getUser().then((user) => getOrders(user.id)).then((orders) => console.log(orders.length))` Chain-টি `async` / `await` দিয়ে লিখুন।

৪. `const data = fetchData(); console.log(data.length);` এখানে `fetchData` একটি `async` Function। কেন `undefined` দেখায়? ঠিক করুন।

৫. একটি `async` Function লিখুন যা ব্যর্থ হয়, এবং `try` / `catch` / `finally` দিয়ে ত্রুটি ও "Loading বন্ধ" দেখান।

মধ্যম স্তর

৬. তিনটি পণ্য একসঙ্গে আনুন (প্রতিটি ১০০ মিলিসেকেন্ডের কৃত্রিম অপেক্ষা), তারপর মোট দাম দেখান।

৭. তিনটি Request-এর একটি ব্যর্থ হলেও বাকিদের ফল দেখান; কোনটি ব্যর্থ তাও জানান।

৮. `withTimeout(promise, ms)` লিখুন, যা `ms`-এর মধ্যে শেষ না হলে একটি ত্রুটি দিয়ে rejected হয়।

৯. কৃত্রিম `fetch` দিয়ে `getJson(url)` পরীক্ষা করুন তিনটি ক্ষেত্রে: সফল, `404`, এবং ভাঙা JSON।

চ্যালেঞ্জ স্তর

১০. `retryServerErrors(task, retries)` লিখুন, যা শুধু `5xx` Status-এর ত্রুটিতে আবার চেষ্টা করে, `4xx`-এ নয়।

১১. `runLimited(tasks, limit)` লিখুন, যা অনেকগুলো কাজ চালায় কিন্তু একসঙ্গে `limit`-এর বেশি নয়।

১২. `debounce(fn, ms)` লিখুন: দ্রুত পরপর ডাকলে শুধু শেষ ডাকের `ms` মিলিসেকেন্ড পরে একবার `fn` চলবে। খোঁজার Box-এ প্রতিটি অক্ষরে Request না পাঠাতে এটি ব্যবহৃত হয়।

১০.১৫ অনুশীলনীর সমাধান

সমাধান ১

order.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('ক');
setTimeout(() => console.log('খ'), 0);
Promise.resolve().then(() => console.log('গ'));
console.log('ঘ');
```

Output

```
ক
ঘ
গ
খ
```

Synchronous লাইন দুটি আগে, তারপর Microtask (`গ`), শেষে Task (`খ`)।

সমাধান ২

wait.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

console.log('১');
wait(300).then(() => console.log('২'));
```

Output

```
১
২
```

সমাধান ৩

async-rewrite.js · JavaScript — চলবে: Browser Console ও Node.js

```
const getUser = async () => ({ id: 7, name: 'Sara' });
const getOrders = async (id) => [`${id}-A`, `${id}-B`];

async function showOrderCount() {
  const user = await getUser();
  const orders = await getOrders(user.id);
  console.log(`${user.name}: ${orders.length}টি Order`);
}

showOrderCount();
```

Output

```
Sara: 2টি Order
```

সমাধান ৪ — `fetchData()` একটি Promise ফেরত দেয়, Array নয়; Promise-এর `length` নেই। `await` দিতে হবে, এবং তার জন্য Code-কে একটি `async` Function-এ রাখতে হবে।

missing-await.js · JavaScript — চলবে: Browser Console ও Node.js

```
const fetchData = async () => [10, 20, 30];

async function main() {
  const data = await fetchData();
  console.log(data.length);
}

main();
```

Output

```
3
```

সমাধান ৫

try-finally.js · JavaScript — চলবে: Browser Console ও Node.js

```
async function loadReport() {
  throw new Error('Report পাওয়া যায়নি');
}

async function main() {
  try {
    await loadReport();
    console.log('এটি চলবে না');
  } catch (error) {
    console.log(`ধরা পড়েছে: ${error.message}`);
  } finally {
    console.log('Loading বন্ধ');
  }
}

main();
```

Output

```
ধরা পড়েছে: Report পাওয়া যায়নি
Loading বন্ধ
```

সমাধান ৬

all-products.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

const getProduct = (id, price) => wait(100, { id, price });

async function main() {
  const products = await Promise.all([
    getProduct('P1', 250),
    getProduct('P2', 120),
    getProduct('P3', 80),
  ]);
  const total = products.reduce((sum, product) => sum + product.price, 0);
  console.log(`${products.length}টি পণ্য, মোট ${total} টাকা`);
}

main();
```

Output

```
3টি পণ্য, মোট 450 টাকা
```

তিনটি অপেক্ষা একসঙ্গে চলেছে, তাই মোট সময় প্রায় ১০০ মিলিসেকেন্ড, ৩০০ নয়।

সমাধান ৭

all-settled.js · JavaScript — চলবে: Browser Console ও Node.js

```
function request(name, succeeds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (succeeds) {
        resolve(`${name} সফল`);
      } else {
        reject(new Error(`${name} ব্যর্থ`));
      }
    }, 50);
  });
}

async function main() {
  const results = await Promise.allSettled([
    request('Profile', true),
    request('Orders', false),
    request('Settings', true),
  ]);

  for (const result of results) {
    const ok = result.status === 'fulfilled';
    console.log(ok ? result.value : result.reason.message);
  }
}

main();
```

Output

```
Profile সফল
Orders ব্যর্থ
Settings সফল
```

সমাধান ৮

with-timeout.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new Error(`${ms} ms পার হয়েছে`)), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

async function main() {
  console.log(await withTimeout(wait(50, 'সময়মতো এসেছে'), 200));

  try {
    await withTimeout(wait(500, 'দেরিতে'), 200);
  } catch (error) {
    console.log(error.message);
  }
}

main();
```

Output

```
সময়মতো এসেছে
200 ms পার হয়েছে
```

`Promise.race` ধীর কাজটিকে থামায় না, শুধু তার জন্য অপেক্ষা বন্ধ করে। কাজটিই থামাতে হলে `AbortController` লাগে, যেমন Project-এ।

সমাধান ৯

get-json.js · JavaScript — চলবে: Browser Console ও Node.js

```
const responses = {
  '/ok': () => new Response('{"total":5}', { status: 200 }),
  '/missing': () => new Response('', { status: 404 }),
  '/broken': () => new Response('{oops', { status: 200 }),
};
const fakeFetch = async (url) => responses[url]();

async function getJson(url) {
```

```
  const response = await fakeFetch(url);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

async function main() {
  for (const url of Object.keys(responses)) {
    try {
      const data = await getJson(url);
      console.log(`${url}: total ${data.total}`);
    } catch (error) {
      const reason = error instanceof SyntaxError ? 'ভাঙা JSON' : error.message;
      console.log(`${url}: ${reason}`);
    }
  }
}

main();
```

Output

```
/ok: total 5
/missing: HTTP 404
/broken: ভাঙা JSON
```

ভাঙা JSON-এ `response.json()` একটি `SyntaxError` দিয়ে rejected হয়; Status `200` হলেও তথ্য ব্যবহারযোগ্য নয়।

সমাধান ১০

retry-5xx.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function retryServerErrors(task, retries = 2) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (!(error.status >= 500) || attempt >= retries) {
        throw error;
      }
      await wait(100 * 2 ** attempt);
    }
  }
```

```
}

function serverReplying(...statuses) {
  let call = 0;
  return async () => {
    const status = statuses[call];
    call += 1;
    if (status === 200) {
      return `সফল (${call} বারের চেষ্টায়)`;
    }
    const error = new Error(`HTTP ${status}`);
    error.status = status;
    throw error;
  };
}

async function main() {
  console.log(await retryServerErrors(serverReplying(503, 500, 200)));

  try {
    await retryServerErrors(serverReplying(404, 200));
  } catch (error) {
    console.log(`আবার চেষ্টা হয়নি: ${error.message}`);
  }
}

main();
```

Output

```
সফল (3 বারের চেষ্টায়)
আবার চেষ্টা হয়নি: HTTP 404
```

সমাধান ১১

run-limited.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function runLimited(tasks, limit) {
  const results = [];
  let next = 0;

  async function worker() {
    while (next < tasks.length) {
      const index = next;
      next += 1;
      results[index] = await tasks[index]();
```

```
    }
  }

  await Promise.all(Array.from({ length: limit }, worker));
  return results;
}

let running = 0;
const job = (name, ms) => async () => {
  running += 1;
  console.log(`শুরু ${name} (একসঙ্গে ${running})`);
  await wait(ms);
  running -= 1;
  return name;
};

const jobs = [job('ক', 120), job('খ', 50), job('গ', 50), job('ঘ', 50)];
runLimited(jobs, 2).then((names) => console.log(`শেষ: ${names.join(', ')}`));
```

Output

```
শুরু ক (একসঙ্গে 1)
শুরু খ (একসঙ্গে 2)
শুরু গ (একসঙ্গে 2)
শুরু ঘ (একসঙ্গে 2)
শেষ: ক, খ, গ, ঘ
```

দুটি `worker` একই তালিকা থেকে পালা করে কাজ নেয়। একটি কাজ শেষ হলেই সেই `worker` পরেরটি ধরে, তাই একসঙ্গে কখনো দুটির বেশি চলে না। ফলগুলো `results[index]`-এ রাখা হয়েছে, তাই শেষ হওয়ার ক্রম যা-ই হোক, ফল আসে মূল ক্রমে।

সমাধান ১২

debounce.js · JavaScript চলবে: Browser Console ও Node.js

```
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

const search = debounce((text) => console.log(`খোঁজা হলো: ${text}`), 300);

// ব্যবহারকারী দ্রুত লিখছেন: প্রতি 100 ms-এ একটি নতুন অক্ষর।
['J', 'Ja', 'Jav', 'Java'].forEach((text, index) => {
  setTimeout(() => search(text), index * 100);
});
setTimeout(() => search('JavaScript'), 900);
```

Output

```
খোঁজা হলো: Java
খোঁজা হলো: JavaScript
```

চারটি দ্রুত ডাকের মধ্যে শুধু শেষটি (`Java`) চলেছে; প্রতিটি নতুন ডাক আগের Timer বাতিল করেছে। `timer` Variable-টি Closure-এ থেকে যায় (অধ্যায় ৫)।

১০.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `setTimeout(fn, 0)` সম্পর্কে কোনটি সঠিক?

ক) `fn` সঙ্গে সঙ্গে চলে

খ) `fn` চলে বর্তমান কাজ এবং সব Microtask শেষ হওয়ার পর

গ) `fn` কখনো চলে না

ঘ) `fn` সব Promise-এর আগে চলে

প্রশ্ন ২. `console.log(1); Promise.resolve().then(() => console.log(2)); setTimeout(() => console.log(3)); console.log(4);`-এর Output?

ক) 1 2 3 4

খ) 1 4 3 2

গ) 1 4 2 3

ঘ) 1 2 4 3

প্রশ্ন ৩. একটি Promise-এর অবস্থা কোনটি নয়?

ক) pending

খ) fulfilled

গ) rejected

ঘ) cancelled

প্রশ্ন ৪. একটি `async` Function সব সময় কী ফেরত দেয়?

ক) Promise

খ) `undefined`

গ) `return`-এ লেখা মানটি

ঘ) একটি Callback

প্রশ্ন ৫. Server `404` পাঠালে `fetch`-এর Promise কী হয়?

ক) rejected হয়

খ) fulfilled হয়; `response.ok` হয় `false`

গ) pending থেকে যায়

ঘ) `null` দিয়ে fulfilled হয়

প্রশ্ন ৬. তিনটি স্বাধীন Request-এর সবগুলোর ফল একসঙ্গে দরকার, এবং একটি ব্যর্থ হলে পুরো কাজ ব্যর্থ। কোনটি?

ক) `Promise.race`

খ) `Promise.any`

গ) `Promise.all`

ঘ) পরপর তিনটি `await`

প্রশ্ন ৭. একটি চলমান `fetch` বাতিল করতে কী লাগে?

ক) `clearTimeout`

খ) `AbortController`

গ) `Promise.reject`

ঘ) `break`

প্রশ্ন ৮. `ids.forEach(async (id) => { await save(id); }); console.log('শেষ');` — `'শেষ'` কখন ছাপা হয়?

ক) সব `save` শেষ হওয়ার পর

খ) প্রথম `save` শেষ হওয়ার পর

গ) কখনোই না

ঘ) কোনো `save` শেষ হওয়ার আগেই

প্রশ্ন ৯. কোন ত্রুটিতে আবার চেষ্টা (Retry) করা উচিত নয়?

ক) `404 Not Found`

খ) `503 Service Unavailable`

গ) সময়সীমা পার

ঘ) ক্ষণিকের Network বিচ্ছিন্নতা

প্রশ্ন ১০. দ্রুত পরপর দুটি খোঁজায় পুরোনো সাড়া দেরিতে এসে নতুন ফলের ওপরে বসল। এর নাম?

ক) Callback Hell

খ) Race Condition

গ) Memory Leak

ঘ) Hoisting

১০.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Timer-এর Callback একটি Task; Stack খালি হওয়া এবং সব Microtask-এর পরে চলে।
২	গ	Synchronous 1 ও 4, তারপর Microtask 2, শেষে Task 3।
৩	ঘ	Promise-এর অবস্থা তিনটি; বাতিল করলে সেটি `AbortError` দিয়ে rejected হয়।
৪	ক	`return 42` লিখলেও ফল `42` দিয়ে fulfilled একটি Promise।

প্রশ্ন	উত্তর	ব্যাখ্যা
৫	খ	সাড়া এসেছে, তাই fulfilled; HTTP ত্রুটি `response.ok` দিয়ে যাচাই করতে হয়।
৬	গ	`Promise.all` একসঙ্গে চালায়, এবং একটি ব্যর্থ হলেই ব্যর্থ হয়।
৭	খ	Controller-এর `signal` `fetch`-কে দিয়ে `abort()` ডাকতে হয়।
৮	ঘ	`forEach` Callback-এর Promise-এর জন্য অপেক্ষা করে না।
৯	ক	ঠিকানা নেই—আবার পাঠালেও একই ফল; Retry শুধু সাময়িক ত্রুটিতে।
১০	খ	ফল নির্ভর করছে কোন কাজ আগে শেষ হয় তার ওপর।

১০.১৮ অধ্যায়ের সারসংক্ষেপ

- JavaScript একসময়ে একটি কাজ করে; অপেক্ষার কাজ Host সামলায়, ফল আসে Queue হয়ে।
- Event Loop: বর্তমান কাজ, তারপর সব Microtask (Promise), তারপর একটি Task (Timer, Event)।
- Callback প্রথম পদ্ধতি; গভীর Nesting-এ পড়া ও ত্রুটি সামলানো কঠিন।
- Promise-এর তিন অবস্থা; `then` Chain করে, একটি `catch` সব ধাপের ত্রুটি ধরে, `finally` সব ক্ষেত্রে চলে।
- `async` Function সব সময় Promise ফেরত দেয়; `await` শুধু সেই Function থামায়; ত্রুটি `try` / `catch`-এ।
- `fetch` HTTP ত্রুটিতে fulfilled হয়; `response.ok` যাচাই বাধ্যতামূলক; Body পড়তে আরেকটি `await`।
- স্বাধীন কাজ `Promise.all` দিয়ে একসঙ্গে; ব্যর্থতার প্রয়োজন অনুযায়ী `allSettled`, `race`, `any`।
- `AbortController` বাতিল করে, `AbortSignal.timeout` সময়সীমা দেয়; সর্বশেষ Request-এর ফলই গ্রহণ করে Race Condition এড়ান।
- Retry শুধু সাময়িক ত্রুটিতে, সীমিত সংখ্যায়, Exponential Backoff-সহ।
- Network-কে Inject করে কৃত্রিম সাড়া দিয়ে Test করুন; Public API-র ওপর Test নির্ভর করবেন না।

পরের অধ্যায়ে Error Handling: ত্রুটির ধরন, নিজের Error তৈরি, প্রত্যাশিত ও অপ্রত্যাশিত ত্রুটির পার্থক্য, এবং Program-কে ত্রুটির মধ্যেও নির্ভরযোগ্য রাখার কৌশল।

১০.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Asynchronous	অপেক্ষার কাজ পেছনে চলে, Program এগিয়ে যায়, ফল পরে আসে।
Call Stack	এই মুহূর্তে চলমান Function-গুলোর স্তূপ।
Event Loop	Stack খালি হলে Queue থেকে পরের কাজ এনে চালানোর ব্যবস্থা।
Task	Timer, Event-এর মতো Callback; একবারে একটি চলে।

পরিভাষা	অর্থ
Microtask	Promise-এর Callback; প্রতিটি Task-এর পরে সবগুলো চলে।
Callback	পরে ডাকার জন্য অন্য Function-কে দেওয়া Function।
Promise	একটি অপেক্ষার কাজের ভবিষ্যৎ ফলের প্রতিনিধি Object।
`async` / `await`	Promise-এর ওপর Synchronous-এর মতো দেখতে লেখার ধরন।
Response	`fetch`-এর ফল: Status, Header এবং Body।
AbortController	চলমান কাজ বাতিলের নিয়ন্ত্রক।
Race Condition	কাজ শেষের ক্রমের ওপর নির্ভর করে ভুল ফল।
Exponential Backoff	প্রতিবার ব্যর্থতার পর অপেক্ষা দ্বিগুণ করে আবার চেষ্টা।

১০.২০ আরও পড়ুন

- **MDN — Introducing asynchronous JavaScript:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing` — Asynchronous ধারণার ভূমিকা।
- **MDN — Using promises:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises` — Promise-এর বিস্তারিত নির্দেশিকা।
- **MDN — Using the Fetch API:** `https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch` — `fetch`, Response এবং ত্রুটি।
- **MDN — AbortController:** `https://developer.mozilla.org/en-US/docs/Web/API/AbortController` — বাতিল ও সময়সীমা।
- **Node.js — The Node.js Event Loop:** `https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick` — Node.js-এর Event Loop-এর বিবরণ।

অধ্যায় ১১

Error Handling, Debugging এবং Defensive Programming

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Syntax, Runtime ও Logical Error-এর পার্থক্য করতে, এবং JavaScript-এর Built-in Error-এর ধরন চিনতে।
- `throw`, `try`, `catch` ও `finally` দিয়ে ত্রুটি তৈরি, ধরা ও পরিষ্কারের কাজ নিশ্চিত করতে।
- নিজের Error তৈরি করতে, এবং `cause` দিয়ে মূল কারণ ধরে রাখতে।
- প্রত্যাশিত (ব্যবহারকারী ঠিক করতে পারেন) ও অপ্রত্যাশিত (Program-এর ভুল) ত্রুটি আলাদাভাবে সামলাতে।
- Trust Boundary-তে বাইরের তথ্য যাচাই করে Defensive Program লিখতে।
- Asynchronous ত্রুটি এবং Global Error Handler ব্যবহার করতে।
- Stack Trace পড়তে, এবং DevTools-এর Breakpoint দিয়ে Bug খুঁজতে।
- ত্রুটি সামলানোর প্রচলিত ভুলগুলো (Anti-pattern) চিনে এড়াতে।
- একটি সম্পূর্ণ **OrderGuard: Order Validation** Web Application তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৮: Regular Expression দিয়ে সরল যাচাই।
- অধ্যায় ৯: Form, `submit` Event এবং Accessibility-র অভ্যাস।
- অধ্যায় ১০: `async` / `await`, Promise-এর rejection এবং কৃত্রিম Server দিয়ে Test।
- `chapter-11` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

প্রতিটি বাস্তব Program-এ কিছু না কিছু ভুল হবে। ব্যবহারকারী Email-এর ঘরে নাম লিখবেন, Network হঠাৎ বিচ্ছিন্ন হবে, Server অপ্রত্যাশিত তথ্য পাঠাবে, আর আপনার নিজের Code-এ একটি Bug লুকিয়ে থাকবে। প্রশ্ন এটি নয় যে ত্রুটি ঘটবে কি না, প্রশ্ন হলো ঘটলে কী হবে।

একটি দুর্বল Program তখন সাদা পর্দা দেখায়, অথবা আরও খারাপ: চুপচাপ ভুল তথ্য সংরক্ষণ করে। একটি নির্ভরযোগ্য Program ব্যবহারকারীকে জানায় কী ঠিক করতে হবে, যা ঠিক করা সম্ভব নয় তা নিরাপদে থামায়, এবং ডেভেলপারের জন্য যথেষ্ট তথ্য রেখে যায় যাতে Bug খুঁজে পাওয়া যায়। এই অধ্যায়ে আপনি শিখবেন কোন ত্রুটি ধরবেন, কোনটি ধরবেন না, এবং ধরার পর কী করবেন।

১১.১ তিন ধরনের ত্রুটি

Syntax Error ঘটে যখন Code JavaScript-এর ব্যাকরণ মানে না। Program চলা শুরুই করে না।

syntax.js · JavaScript — চলবে: Node.js

```
console.log('এই লাইনও চলবে না');
const = 42;
```

Error Message

```
SyntaxError: Unexpected token '='
```

প্রথম লাইনটিও চলেনি, কারণ JavaScript পুরো File আগে পড়ে নেয়, এবং ব্যাকরণের ভুল পেলে কিছুই চালায় না।

Runtime Error ঘটে Program চলার সময়, যখন একটি কাজ করা সম্ভব হয় না।

runtime.js · JavaScript চলবে: Node.js

```
const user = null;

console.log('এই লাইন চলে');
console.log(user.name);
console.log('এই লাইন চলে না');
```

Error Message

```
এই লাইন চলে
TypeError: Cannot read properties of null (reading 'name')
```

Logical Error সবচেয়ে বিপজ্জনক: Program চলে, কোনো বার্তা দেয় না, কিন্তু ফল ভুল।

logical.js · JavaScript চলবে: Browser Console ও Node.js

```
function priceAfterDiscount(price, percent) {
  return price - percent;
}

console.log(priceAfterDiscount(100, 10));
console.log(priceAfterDiscount(2000, 10));
```

Output

```
90
1990
```

১০০ টাকায় ফল ঘটনাক্রমে ঠিক (১০%-এ ৯০), কিন্তু ২০০০ টাকায় ১০% ছাড়ের পর ১৮০০ হওয়ার কথা। সূত্রে `price * (1 - percent / 100)` লেখা উচিত ছিল। এমন ভুল কোনো Error-এর বার্তা দেয় না; এগুলো ধরে Automated Test, যেখানে একাধিক মান দিয়ে প্রত্যাশিত ফল মেলানো হয়।

১১.২ Error Object

Runtime-এ কিছু ভুল হলে JavaScript একটি **Error Object** তৈরি করে **ছোড়ে** (throw)। কেউ সেটি না ধরলে Program থামে। প্রতিটি Error Object-এ তিনটি প্রধান Property:

Property	কী থাকে	কার জন্য
`name`	ধরন: `TypeError`, `RangeError` ইত্যাদি	Code, ধরন অনুযায়ী সিদ্ধান্তের জন্য
`message`	কী ভুল হয়েছে তার বিবরণ	ডেভেলপার
`stack`	কোন Function-এর কোন লাইনে ঘটেছে	ডেভেলপার, Debugging-এর জন্য

error-types.js · JavaScript চলবে: Browser Console ও Node.js

```
const attempts = [
  () => null.length,
  () => notDeclared + 1,
  () => (12.5).toFixed(500),
  () => JSON.parse('{bad'),
  () => decodeURIComponent('%'),
];

for (const attempt of attempts) {
  try {
    attempt();
  } catch (error) {
    console.log(`${error.name}, Error-ও? ${error instanceof Error}`);
  }
}
```

Output

```
TypeError, Error-ও? true
ReferenceError, Error-ও? true
RangeError, Error-ও? true
SyntaxError, Error-ও? true
URIError, Error-ও? true
```

ধরন	কখন
`TypeError`	ভুল ধরনের মানের ওপর কাজ: `null`-এর Property, Function নয় এমন কিছু Call
`ReferenceError`	ঘোষণা করা হয়নি এমন Variable
`RangeError`	গ্রহণযোগ্য সীমার বাইরের মান
`SyntaxError`	Code-এর ব্যাকরণ ভুল, অথবা `JSON.parse`-এ ভাঙা JSON
`URIError`	ভুল গঠনের URL-অংশ Decode করা

সবগুলোই `Error` থেকে তৈরি, তাই `instanceof Error` সবার জন্য `true`। `message`-এর লেখা Browser ও Node.js-এর সংস্করণভেদে বদলাতে পারে; Code-এ সিদ্ধান্ত নেওয়ার জন্য `message`-এর লেখা নয়, `name` বা `instanceof` ব্যবহার করুন।

১১.৩ throw, try, catch এবং finally

নিজের Code-এ ভুল ধরা পড়লে আপনিও `throw` করতে পারেন। সব সময় একটি Error Object ছুড়ুন, সাধারণ লেখা নয়: Error Object-এ `stack` থাকে, লেখায় থাকে না।

throw.js · JavaScript চলবে: Browser Console ও Node.js

```
function readPrice(item) {
  if (typeof item?.price !== 'number') {
    throw new TypeError('পণ্যের দাম একটি সংখ্যা হতে হবে');
  }
  if (item.price < 0) {
    throw new RangeError(`দাম ঋণাত্মক হতে পারে না: ${item.price}`);
  }
  return item.price;
}

for (const item of [{ price: 250 }, { price: '250' }, { price: -5 }]) {
  try {
    console.log(`দাম: ${readPrice(item)}`);
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
দাম: 250
TypeError: পণ্যের দাম একটি সংখ্যা হতে হবে
RangeError: দাম ঋণাত্মক হতে পারে না: -5
```

`throw` হওয়ার সঙ্গে সঙ্গে Function থামে, এবং নিয়ন্ত্রণ চলে যায় সবচেয়ে কাছের `catch`-এ, প্রয়োজনে একাধিক Function পেরিয়ে। একটি ত্রুটিতে পুরো Loop থামেনি: প্রতিটি পণ্য আলাদা `try`-এর ভেতরে।

finally সব সময় চলে

finally.js · JavaScript চলবে: Browser Console ও Node.js

```
function runStep(input) {
  try {
    console.log('১. চেষ্টা');
    if (input < 0) {
      throw new RangeError('ঋণাত্মক');
    }
    return 'সফল';
  } catch (error) {
    console.log(`২. ধরা পড়েছে: ${error.message}`);
    return 'ব্যর্থ';
  } finally {
    console.log('৩. finally সব সময়');
  }
}
```

```
console.log(runStep(5));
console.log(runStep(-1));
```

Output

```
১. চেষ্টা
৩. finally সব সময়
সফল
১. চেষ্টা
২. ধরা পড়েছে: ঋণাত্মক
৩. finally সব সময়
ব্যর্থ
```

`try`-এর ভেতরে `return` থাকলেও Function ফেরত যাওয়ার আগে `finally` চলেছে। তাই পরিষ্কারের কাজ—Loading বন্ধ, Button আবার চালু, খোলা File বন্ধ—`finally`-তে রাখুন: সফল, ব্যর্থ বা আগেভাগে ফেরত, সব পথেই তা ঘটবে।

> **মনে রাখুন:** `catch` শুধু তার নিজের `try`-এর ভেতরের ত্রুটি ধরে, এবং শুধু সেই মুহূর্তে চলমান Code-এর। একটি `setTimeout`-এর Callback পরে চলে, তাই বাইরের `try` তার ত্রুটি ধরতে পারে না। Promise-এর ত্রুটি ধরতে `await` অপরিহার্য (অধ্যায় ১০)।

১১.৪ নিজের Error তৈরি

Built-in ধরনগুলো বলে কী ধরনের ভুল, কিন্তু আপনার Program-এর ভাষায় নয়। "ব্যবহারকারী ভুল তথ্য দিয়েছেন" এবং "Program-এ Bug আছে"—দুটিই `TypeError` হতে পারে, অথচ সামলাতে হয় সম্পূর্ণ আলাদাভাবে। নিজের Error-এর ধরন তৈরি করলে এই পার্থক্য স্পষ্ট হয়।

custom-error.js · JavaScript — চলবে: Browser Console ও Node.js

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

function parseAge(text) {
  const age = Number(text);

  if (!Number.isInteger(age) || age < 0 || age > 150) {
    throw new ValidationError('age', `বয়স সঠিক নয়: "${text}"`);
  }
  return age;
```

```
}

for (const input of ['32', 'abc', '-4']) {
  try {
    console.log(`বয়স: ${parseAge(input)}`);
  } catch (error) {
    if (!(error instanceof ValidationError)) {
      throw error;
    }
    console.log(`${error.field} ঘরে: ${error.message}`);
  }
}
```

Output

```
বয়স: 32
age ঘরে: বয়স সঠিক নয়: "abc"
age ঘরে: বয়স সঠিক নয়: "-4"
```

- `class ValidationError extends Error` `Error`-এর সব বৈশিষ্ট্যসহ একটি নতুন ধরন তৈরি করে। `super(message)` মূল `Error`-কে বার্তা দেয়; তারপর নিজের Property যোগ হয়। Class-এর পূর্ণ আলোচনা অধ্যায় ১৩-এ; এখানে এই ছকটুকুই যথেষ্ট।
- `this.name` বসানো জরুরি; নইলে Stack ও Console-এ শুধু `Error` দেখাবে।
- **`catch`-এ ধরন যাচাই, এবং অচেনা হলে আবার `throw`।** এই `catch` শুধু `ValidationError` সামলাতে জানে। অন্য কিছু এলে (যেমন একটি Bug-এর `TypeError`) তা চাপা না দিয়ে আবার ছুড়ে দেওয়া হয়েছে, যাতে ওপরের স্তর বা Console তা দেখতে পায়। এটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ অভ্যাস।

cause: মূল কারণ ধরে রাখা

নিচের স্তরের একটি ত্রুটিকে ওপরের স্তরের ভাষায় নতুন করে প্রকাশ করার সময় মূল ত্রুটি হারানো উচিত নয়।

cause.js · JavaScript চলবে: Browser Console ও Node.js

```
function loadSettings(text) {
  try {
    return JSON.parse(text);
  } catch (error) {
    throw new Error('Settings পড়া যায়নি', { cause: error });
  }
}

try {
  loadSettings('{oops');
} catch (error) {
  console.log(error.message);
  console.log(`মূল কারণ: ${error.cause.name}`);
}
```

Output

```
Settings পড়া যায়নি
মূল কারণ: SyntaxError
```

ওপরের স্তর পেল বোধগম্য বার্তা, আর Debugging-এর জন্য `cause`-এ রইল আসল `SyntaxError`, তার নিজের `stack`-সহ। Browser ও Node.js-এর Console দুটিই `cause` দেখায়।

১১.৫ প্রত্যাশিত বনাম অপ্রত্যাশিত ত্রুটি

ত্রুটি সামলানোর সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত হলো ত্রুটিটি কোন শ্রেণির।

	প্রত্যাশিত ত্রুটি	অপ্রত্যাশিত ত্রুটি
উদাহরণ	ভুল Email, শূন্য পরিমাণ, স্টক শেষ, Network বিচ্ছিন্ন	`undefined`-এর Property পড়া, নামের বানান ভুল, ভুল হিসাব
কার দায়	পরিবেশ বা ব্যবহারকারী	Program-এর Code
কে ঠিক করবেন	ব্যবহারকারী, বা কিছুক্ষণ পর আপনা থেকে	ডেভেলপার
ব্যবহারকারী দেখবেন	নির্দিষ্ট, কী করতে হবে সহ বার্তা	সাধারণ বার্তা: "অপ্রত্যাশিত সমস্যা"
ডেভেলপার পাবেন	প্রয়োজনে সংক্ষিপ্ত Log	পুরো Error ও Stack, `console.error` বা Error Reporting-এ
Program	স্বাভাবিকভাবে চলে	নিরাপদে থামে; ভুল অবস্থায় এগোয় না

প্রত্যাশিত ত্রুটি ধরুন যত কাছে সম্ভব, যেখানে আপনি জানেন কী করতে হবে। অপ্রত্যাশিত ত্রুটি ধরবেন না; সেগুলো একেবারে ওপরের স্তরে একটি জায়গায় পৌঁছাতে দিন, যেখানে সাধারণ বার্তা দেখানো এবং পুরো বিবরণ Log করা হয়।

সবচেয়ে প্রচলিত ভুল হলো দুটিকে একসঙ্গে ধরে ফেলা:

swallowed.js · JavaScript চলবে: Browser Console ও Node.js

```
const DISCOUNT_RATES = { EID10: 10, NEW5: 5 };

// ভুল: সব ত্রুটি চাপা পড়ে, এমনকি Code-এর বানান ভুলও।
function discountFor(code) {
  try {
    return DISCOUNT_RATE[code.toUpperCase()] ?? 0;
  } catch {
    return 0;
  }
}

// সঠিক: প্রত্যাশিত ক্ষেত্র নিজে যাচাই; বাকি সব ত্রুটি দৃশ্যমান থাকে।
function discountForFixed(code) {
  if (typeof code !== 'string') {
    return 0;
  }
  return DISCOUNT_RATES[code.toUpperCase()] ?? 0;
}

console.log(discountFor('eid10'));
console.log(discountForFixed('eid10'));
console.log(discountForFixed(undefined));
```

Output

```
0
10
0
```

প্রথম Function-এ `DISCOUNT_RATES`-এর বানান ভুল (`DISCOUNT_RATE`)। এটি একটি `ReferenceError`, কিন্তু খালি `catch` তা গিলে ফেলেছে, এবং প্রতিটি গ্রাহক চুপচাপ ছাড় হারাচ্ছেন। Test ছাড়া এই Bug মাসের পর মাস টিকে থাকতে পারে। লেখকের উদ্দেশ্য ছিল "`code` ভুল হলে ০", এবং দ্বিতীয় Function ঠিক সেটিই সরাসরি যাচাই করেছে, `try` ছাড়া।

১১.৬ Defensive Programming: Trust Boundary

Trust Boundary হলো সেই সীমানা যেখানে বাইরের তথ্য আপনার Program-এ ঢোকে: Form, URL, `localStorage`, Server-এর সাড়া, অন্য Team-এর Library। সীমানার বাইরের তথ্যকে বিশ্বাস করবেন না; সীমানায় একবার ভালোভাবে যাচাই করুন, তারপর ভেতরের Code পরিষ্কার তথ্য নিয়ে নিশ্চিন্তে কাজ করুক।

সংখ্যা রূপান্তরের কয়েকটি ফাঁদ:

number-traps.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(Number(''), Number('  '), Number('1e3'), Number('0x10'));
console.log(Number('12abc'), parseInt('12abc', 10), parseFloat('2.5kg'));

const strictInteger = (text) =>
  (/^[1-9]\d*$/.test(text.trim()) ? Number(text) : null);

console.log(strictInteger('12'), strictInteger(''), strictInteger('1e3'));
```

Output

```
0 0 1000 16
NaN 12 2.5
12 null null
```

ফাঁকা লেখা `0` হয়ে যায়, `'1e3'` হয় ১০০০, `parseInt` সংখ্যার পরের বাড়তি অংশ নীরবে ফেলে দেয়। পরিমাণের মতো ঘরে এর কোনোটিই গ্রহণযোগ্য নয়। তাই Project আগে লেখার গঠন যাচাই করে, তারপর রূপান্তর করে।

আরেকটি সূক্ষ্ম ফাঁদ: একটি Object-এ Key দিয়ে খোঁজা।

own-keys.js · JavaScript চলবে: Browser Console ও Node.js

```
const catalog = { keyboard: { name: 'Keyboard', price: 1250 } };

const unsafeFind = (id) => catalog[id];
const safeFind = (id) => (Object.hasOwn(catalog, id) ? catalog[id] : null);

console.log(typeof unsafeFind('toString'));
console.log(safeFind('toString'));
console.log(safeFind('keyboard').price);
```

Output

```
function
null
1250
```

প্রতিটি Object উত্তরাধিকারসূত্রে `toString`, `constructor`-এর মতো কিছু Property পায় (অধ্যায় ১৩)। ব্যবহারকারী Form-এর পণ্য-তালিকায় হাত দিয়ে `toString` পাঠালে `catalog[id]` কিছু একটা পায়, এবং Program বিভ্রান্ত হয়। `Object.hasOwn` (অধ্যায় ৭) শুধু Object-এর নিজের Key গ্রহণ করে।

Defensive Program-এর মূল নিয়মগুলো:

- **শুরুতেই যাচাই (Guard Clause),** এবং ভুল হলে সঙ্গে সঙ্গে থামা (Fail Fast)। ভুল তথ্য নিয়ে অনেক দূর এগোলে ত্রুটি দেখা দেয় এমন জায়গায়, যেখানে কারণ আর খুঁজে পাওয়া যায় না।
- **গঠন ও সীমা দুটোই:** ধরন ঠিক কি না, এবং মান গ্রহণযোগ্য সীমার মধ্যে কি না।

- **অনুমোদিত তালিকা (Allow-list):** "এগুলো বাদে সব চলবে" নয়, "শুধু এগুলোই চলবে"।
- **হিসাব পূর্ণসংখ্যায়:** টাকার হিসাব পয়সায় বা পূর্ণ টাকায় রাখুন; দশমিকের হিসাবে `0.1 + 0.2` সমস্যা (অধ্যায় ২)।
- **Browser-এর যাচাই সুবিধা, নিরাপত্তা নয়:** Server-কে সব সময় নিজে আবার যাচাই করতে হবে।

১১.৭ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা

`async` Function-এর ত্রুটি `await` এবং `try` / `catch` দিয়ে ধরা হয় (অধ্যায় ১০)। কিন্তু কোথাও `catch` ভুলে গেলে কী হয়? Browser-এ দুটি Global Event শেষ প্রতিরক্ষা হিসেবে কাজ করে: ধরা না পড়া সাধারণ ত্রুটির জন্য `error`, আর ধরা না পড়া Promise-এর ত্রুটির জন্য `unhandledrejection`।

global-handlers.js · JavaScript — চলবে: Browser

```
window.addEventListener('unhandledrejection', (event) => {
  console.log(`ধরা পড়েনি এমন Promise: ${event.reason.message}`);
  event.preventDefault();
});

async function saveDraft() {
  throw new Error('Server বন্ধ');
}

saveDraft();
console.log('saveDraft() ডাকা হয়েছে, কিন্তু await বা catch নেই');
```

Output

```
saveDraft() ডাকা হয়েছে, কিন্তু await বা catch নেই
ধরা পড়েনি এমন Promise: Server বন্ধ
```

এই Handler-গুলো ত্রুটি **ঠিক** করে না; তারা শুধু নিশ্চিত করে যে কোনো ত্রুটি নিঃশব্দে হারিয়ে না যায়। বাস্তব Application-এ এখানে ত্রুটিটি একটি Error Reporting Service-এ পাঠানো হয়, যাতে ব্যবহারকারীর Browser-এ ঘটা Bug-ও ডেভেলপার জানতে পারেন। `event.preventDefault()` Browser-এর নিজস্ব লাল বার্তা বন্ধ করে, কারণ এখানে আমরা নিজেরাই তা সামলেছি। আসল সমাধান অবশ্য একটিই: `saveDraft()`-এর আগে `await` এবং চারপাশে `try` / `catch`।

১১.৮ Stack Trace পড়া এবং DevTools

ধরা না পড়া একটি ত্রুটি Console-এ একটি **Stack Trace** দেখায়: ত্রুটির বার্তা, তারপর কোন Function কোন Function-কে ডেকেছিল তার তালিকা, সবচেয়ে ভেতরেরটি সবার ওপরে।

stack.js · JavaScript চলবে: Node.js

```
function checkStock(product) {
  return product.stock > 0;
}

function addToCart(cart, product) {
  if (checkStock(product)) {
    cart.push(product);
  }
}

addToCart([], undefined);
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'stock')
at checkStock
at addToCart
```

পড়ার নিয়ম: প্রথম লাইন বলে **কী** ঘটেছে; পরের লাইনগুলো বলে **কোথায়**, প্রতিটির সঙ্গে File-এর নাম ও লাইন নম্বর। ত্রুটি ঘটেছে `checkStock`-এ, কিন্তু ভুলটি সেখানে নয়: `checkStock` ঠিক কাজ করছে। ভুল হলো `addToCart`-কে `undefined` পণ্য দিয়ে ডাকা। তাই Stack-এর ওপর থেকে নিচে নামুন, এবং প্রতিটি স্তরে প্রশ্ন করুন: এই মানটি কোথা থেকে এলো?

DevTools-এর যেসব সুবিধা এ কাজে লাগে:

- **Breakpoint:** Sources Panel-এ লাইন নম্বরে Click করলে Program সেখানে থামে; Scope অংশে সব Variable-এর তখনকার মান দেখা যায়। Code-এ `debugger;` লিখলেও DevTools খোলা থাকলে সেখানে থামে; কাজ শেষে সেটি মুছে ফেলুন।
- **Pause on exceptions:** Sources Panel-এর এই বিকল্প চালু করলে ত্রুটি ঘটার মুহূর্তে Program থামে, Stack ও Variable অক্ষত অবস্থায়।
- **Step Over / Step Into:** থামার পর এক লাইন করে এগোনো, অথবা একটি Function-এর ভেতরে ঢোকা।
- **Conditional Breakpoint:** শুধু একটি শর্ত সত্য হলে থামা, যেমন `product === undefined`; বড় Loop-এ একটি নির্দিষ্ট ঘটনা ধরতে কাজে লাগে।
- **Console:** `console.error` লাল বার্তা ও Stack দেখায়; `console.table(array)` Object-এর তালিকা সারণি আকারে দেখায়।

Debugging-এর পদ্ধতি সব সময় একই: সমস্যাটি নিশ্চিতভাবে আবার ঘটাতে পারা, ধারণা তৈরি করা ("পণ্যটি `undefined`"), প্রমাণ দেখা (Breakpoint বা `console.log`), ঠিক করা, এবং একটি Test যোগ করা যাতে একই Bug আর না ফেরে।

১১.৯ সাধারণ ভুল

ভুল	ফল	সমাধান
খালি `catch {}`	Bug নিঃশব্দে লুকায়	প্রত্যাশিত ধরন যাচাই; বাকিগুলো আবার `throw`
সব কিছু `try`-এর ভেতরে	কোন লাইনে ভুল বোঝা যায় না; অপ্রত্যাশিত ত্রুটিও ধরা পড়ে	`try` শুধু যে লাইন ব্যর্থ হতে পারে তার চারপাশে
লেখা `throw` করা (`throw 'ভুল'`)	`stack` থাকে না, `instanceof Error` মিথ্যা	`throw new Error('ভুল')`
`message`-এর লেখা দেখে সিদ্ধান্ত	সংস্করণ বদলালে Code ভাঙে	`name`, `instanceof` বা নিজের Property
ব্যবহারকারীকে `error.message` বা Stack দেখানো	বিভ্রান্তি; ভেতরের তথ্য ফাঁস	ব্যবহারকারীর ভাষায় বার্তা; বিবরণ Console-এ
মূল ত্রুটি হারিয়ে নতুন ত্রুটি	আসল কারণ খুঁজে পাওয়া যায় না	`{ cause: error }`
পরিষ্কারের কাজ শুধু সফল পথে	ত্রুটির পর Button চিরতরে বন্ধ	`finally`
`await` ছাড়া Promise-এর চারপাশে `try`	ত্রুটি ধরা পড়ে না	`await`
নিজের Class-এ `this.name` না বসানো	Console-এ শুধু `Error`	`this.name = 'ValidationError'`
ত্রুটির পর ভুল অবস্থায় চলতে থাকা	ভুল তথ্য সংরক্ষিত হয়	অপ্রত্যাশিত ত্রুটিতে কাজ থামান

১১.১০ Debugging নির্দেশিকা

একটি ত্রুটির বার্তা পেলে:

১. **প্রথম ত্রুটিটি পড়ুন,** পরেরগুলো প্রায়ই তারই ফল। নাম ও বার্তা দুটোই পড়ুন: `Cannot read properties of undefined (reading 'stock')` মানে যার `stock` পড়া হচ্ছিল, সেটি `undefined`।

২. **Stack-এর প্রথম লাইনে আপনার নিজের File খুঁজুন,** Library-র ভেতরের লাইন নয়।

৩. **সেই লাইনে Breakpoint দিন,** অথবা ঠিক আগে মানটি `console.log` করুন।

৪. **মানটি কোথা থেকে এলো,** Stack ধরে পেছনে যান।

৫. **কোনো ত্রুটির বার্তা না থাকলে** কিন্তু ফল ভুল, তাহলে Logical Error: কয়েকটি জানা ইনপুট দিয়ে প্রত্যাশিত ফল লিখুন এবং কোথায় পার্থক্য শুরু হয় তা খুঁজুন।

৬. **কোনো বার্তা নেই, কিছু ঘটেও না?** একটি খালি `catch` খুঁজুন।

৭. **ঠিক করার পর একটি Test লিখুন,** যা আগের Code-এ ব্যর্থ হতো।

১১.১১ পেশাদার অভ্যাস

- **প্রত্যাশিত ত্রুটি নিজের ধরনে** (`ValidationError` , `ServiceError`); অপ্রত্যাশিত ত্রুটি একেবারে ওপরের এক জায়গায়।
- **`catch` -এ সব সময় ধরন যাচাই;** অচেনা ত্রুটি আবার `throw` ।
- **ব্যবহারকারীর জন্য বার্তা, ডেভেলপারের জন্য বিবরণ:** দুটি আলাদা জায়গায়।
- **প্রতিটি ব্যবহারকারী-বার্তা বলবে কী করতে হবে:** "সঠিক একটি Email লিখুন", "একটু পরে আবার চেষ্টা করুন"।
- **একসঙ্গে সব ভুল দেখান,** একটি একটি করে নয়; ব্যবহারকারীকে বারবার পাঠাতে বাধ্য করবেন না।
- **`finally` -তে পরিষ্কার।**
- **Trust Boundary-তে একবার পূর্ণ যাচাই;** ভেতরের Code-এ বারবার নয়।
- **ত্রুটির পথেরও Test লিখুন,** শুধু সফল পথের নয়।

১১.১২ Project: OrderGuard — Order Validation

একটি ছোট দোকানের Order Form: গ্রাহকের নাম, Email, পণ্য ও পরিমাণ যাচাই করে Order পাঠানো হয়। লক্ষ্য তিন শ্রেণির ত্রুটি আলাদাভাবে সামলানো: ব্যবহারকারীর ভুল তথ্য (প্রতিটি ঘরের পাশে বার্তা), Server-এর প্রত্যাখ্যান বা Network বিচ্ছিন্নতা (সাধারণ বার্তা, আবার চেষ্টার সুযোগ), এবং Program-এর অপ্রত্যাশিত ভুল (নিরাপদ বার্তা, Console-এ পুরো বিবরণ)।

> **নোট:** এটি একটি শিক্ষামূলক Project। কোনো আসল Server বা টাকা লেনদেন নেই; `createDemoServer` একটি কৃত্রিম Server, যা ৩০০ মিলিসেকেন্ড পরে সাড়া দেয় এবং স্টক হিসাব রাখে। Order-এর ইতিহাস শুধু Page খোলা থাকা পর্যন্ত থাকে।

Project-এর চাহিদা

- নাম ২ থেকে ৬০ অক্ষর; Email-এর মোটামুটি গঠন; পণ্য শুধু তালিকা থেকে; পরিমাণ ১ থেকে ১০০-এর পূর্ণসংখ্যা।
- সব ভুল একসঙ্গে, প্রতিটি ঘরের নিচে; প্রথম ভুল ঘরে Focus; `aria-invalid` ও `aria-describedby` ।
- পাঠানোর সময় Button বন্ধ এবং "পাঠানো হচ্ছে…"; সফল, ব্যর্থ বা অপ্রত্যাশিত—সব ক্ষেত্রে পরে Button আবার চালু।
- স্টকে যথেষ্ট পণ্য না থাকলে Server-এর বার্তা; Network বিচ্ছিন্ন হলে আবার চেষ্টার বার্তা।
- সফল হলে রসিদ নম্বর, টাকার অঙ্কে মোট দাম এবং ইতিহাসে একটি সারি।
- সব লেখা `textContent` দিয়ে।

File-এর কাঠামো

Folder-এর কাঠামো

```
order-validation/
├── index.html
├── style.css
├── order-validation.js        যাচাই, Error-এর ধরন, কৃত্রিম Server
├── app.js                     Form, বার্তা ও ইতিহাস
├── README.md
└── tests/
    ├── load-script.js
    ├── order-validation.test.js
    ├── cdp.mjs
    └── browser-check.mjs
```

order-validation.js

দুই অংশ মিলে সম্পূর্ণ File।

order-validation.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```js
'use strict';

const CATALOG = {
  keyboard: { name: 'Keyboard', price: 1250 },
  mouse: { name: 'Mouse', price: 650 },
  headset: { name: 'Headset', price: 2400 },
};
const STOCK = { keyboard: 10, mouse: 5, headset: 2 };
const MAX_QUANTITY = 100;

// প্রত্যাশিত: ব্যবহারকারী ঠিক করতে পারেন। fields-এ কোন ঘরে কী সমস্যা।
class ValidationError extends Error {
  constructor(fields) {
    super('কয়েকটি ঘর ঠিক করতে হবে।');
    this.name = 'ValidationError';
    this.fields = fields;
  }
}

// প্রত্যাশিত: Server অনুরোধটি গ্রহণ করেনি (যেমন স্টক শেষ)।
class ServiceError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ServiceError';
  }
}

function parseQuantity(value) {
```

```
  const text = String(value ?? '').trim();

  if (!/^[1-9]\d*$/.test(text)) {
    return null;
  }
  const quantity = Number(text);
  return quantity <= MAX_QUANTITY ? quantity : null;
}

function formatTaka(amount) {
  if (!Number.isSafeInteger(amount) || amount < 0) {
    throw new RangeError(`অবৈধ টাকার অঙ্ক: ${amount}`);
  }
  return `${amount.toLocaleString('en-US')} টাকা`;
}

// ...
```

order-validation.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

// Trust Boundary: Form-এর তথ্য যাচাই করে পরিষ্কার Order, নয়তো ValidationError।
function validateOrder(raw, catalog = CATALOG) {
  if (raw === null || typeof raw !== 'object') {
    throw new TypeError('validateOrder-কে একটি Object দিতে হবে');
  }

  const customer = String(raw.customer ?? '').trim();
  const email = String(raw.email ?? '').trim();
  const productId = String(raw.productId ?? '');
  const product = Object.hasOwn(catalog, productId) ? catalog[productId] : null;
  const quantity = parseQuantity(raw.quantity);
  const fields = {};

  if (customer.length < 2 || customer.length > 60) {
    fields.customer = 'নাম 2 থেকে 60 অক্ষরের হতে হবে।';
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    fields.email = 'সঠিক একটি Email লিখুন।';
  }
  if (product === null) {
    fields.productId = 'তালিকা থেকে একটি পণ্য বাছুন।';
  }
  if (quantity === null) {
    fields.quantity = `পরিমাণ 1 থেকে ${MAX_QUANTITY}-এর মধ্যে পূর্ণসংখ্যা হতে হবে।`;
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
```

```
  }

  const total = product.price * quantity;
  const productName = product.name;
  return { customer, email, productId, productName, quantity, total };
}

// কৃত্রিম Server: অপেক্ষার পর রসিদ নম্বর, অথবা স্টক না থাকলে ServiceError।
function createDemoServer(stock = STOCK, delayMs = 300) {
  const left = { ...stock };
  let next = 1;

  return async function send(order) {
    await new Promise((resolve) => setTimeout(resolve, delayMs));
    const available = left[order.productId] ?? 0;

    if (order.quantity > available) {
      const name = order.productName;
      throw new ServiceError(`দুঃখিত, ${name} স্টকে আছে ${available}টি।`);
    }
    left[order.productId] = available - order.quantity;
    return `ORD-${String(next++).padStart(4, '0')}`;
  };
}

// প্রত্যাশিত ত্রুটি ফলাফল হিসেবে ফেরত; অপ্রত্যাশিত ত্রুটি ওপরের স্তরে যায়।
async function placeOrder(raw, send) {
  let order;
  try {
    order = validateOrder(raw);
  } catch (error) {
    if (error instanceof ValidationError) {
      const { message, fields } = error;
      return { ok: false, kind: 'validation', message, fields };
    }
    throw error;
  }

  try {
    return { ok: true, order, receipt: await send(order) };
  } catch (error) {
    if (error instanceof ServiceError) {
      return { ok: false, kind: 'service', message: error.message };
    }
    const message =
      'Server-এর সঙ্গে যোগাযোগ করা যায়নি। একটু পরে আবার চেষ্টা করুন।';
    return { ok: false, kind: 'network', message, cause: error };
  }
}
```

- **দুটি Error-এর ধরন, দুটিই প্রত্যাশিত।** `ValidationError` বহন করে প্রতিটি ঘরের বার্তা; `ServiceError` বহন করে Server-এর ব্যাখ্যা।
- `validateOrder` **সব ঘর যাচাই করে, তারপর একবারে** `throw`**।** প্রথম ভুলেই থামলে ব্যবহারকারীকে একটি করে ভুল ঠিক করে বারবার পাঠাতে হতো।
- `validateOrder`**-এর প্রথম** `throw` **ভিন্ন জাতের।** `raw` একটি Object না হওয়া ব্যবহারকারীর ভুল নয়; এর মানে Code ভুলভাবে Function-টি ডেকেছে। তাই সেটি `TypeError`, এবং `placeOrder` তা ধরে না, ওপরে পাঠিয়ে দেয়।
- `placeOrder`**-এর দুটি আলাদা** `try`**।** প্রথমটি যাচাইয়ের, দ্বিতীয়টি পাঠানোর। একটি বড় `try` হলে যাচাইয়ের একটি Bug ভুল করে "Network বিচ্ছিন্ন" বার্তা দেখাত।
- **প্রত্যাশিত ত্রুটি ফলাফল হয়ে ফেরে** (`{ ok: false, kind, message }`), তাই Page-এর Code-কে প্রতিটি ধরন চিনতে হয় না। Network-এর আসল ত্রুটি `cause`-এ থাকে, ডেভেলপারের জন্য।
- `formatTaka` **নিজেও যাচাই করে।** অবৈধ অঙ্ক এলে তা Program-এর ভুল, তাই `RangeError`।

index.html ও style.css

index.html · HTML অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>OrderGuard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>OrderGuard</h1>

    <form id="order-form" novalidate>
      <label for="customer">গ্রাহকের নাম</label>
      <input id="customer" name="customer" autocomplete="name"
             aria-describedby="customer-error">
      <p id="customer-error" class="field-error"></p>

      <label for="email">Email</label>
      <input id="email" name="email" type="email" autocomplete="email"
             aria-describedby="email-error">
      <p id="email-error" class="field-error"></p>
<!-- ... -->
```

index.html · HTML অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
<!-- ... -->
      <label for="productId">পণ্য</label>
      <select id="productId" name="productId" aria-describedby="productId-error">
```

```
        <option value="">বাছুন</option>
        <option value="keyboard">Keyboard (1,250 টাকা)</option>
        <option value="mouse">Mouse (650 টাকা)</option>
        <option value="headset">Headset (2,400 টাকা)</option>
      </select>
      <p id="productId-error" class="field-error"></p>

      <label for="quantity">পরিমাণ</label>
      <input id="quantity" name="quantity" inputmode="numeric"
             aria-describedby="quantity-error">
      <p id="quantity-error" class="field-error"></p>

      <button type="submit">Order পাঠান</button>
    </form>

    <p id="status" role="status" aria-live="polite"></p>

    <h2>গৃহীত Order</h2>
    <ul id="history"></ul>
  </main>

  <script src="order-validation.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 560px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

#order-form {
  display: grid;
  gap: 4px;
}

input,
```

```
select,
button {
  font: inherit;
  padding: 6px 10px;
}

[aria-invalid='true'] {
  border: 2px solid #b3261e;
}

.field-error {
  min-height: 1.2em;
  margin: 0 0 8px;
  color: #b3261e;
}
```

ভুল ঘরের লাল সীমানা নির্ধারিত হয় `aria-invalid` দিয়ে, তাই চোখে দেখা চিহ্ন এবং Screen Reader-এর তথ্য সব সময় একসঙ্গে বদলায়।

app.js

দুই অংশ মিলে সম্পূর্ণ File।

app.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
'use strict';

// ?offline=1 দিলে Network বিচ্ছিন্নতার অভিনয়; ?delay= কৃত্রিম Server-এর দেরি।
const params = new URLSearchParams(location.search);
const send = params.has('offline')
  ? async () => { throw new TypeError('Failed to fetch'); }
  : createDemoServer(STOCK, Number(params.get('delay') ?? 300));

const FIELDS = ['customer', 'email', 'productId', 'quantity'];
const form = document.querySelector('#order-form');
const submitButton = form.querySelector('button[type="submit"]');
const statusView = document.querySelector('#status');
const historyList = document.querySelector('#history');

function showFieldErrors(fields = {}) {
  for (const name of FIELDS) {
    const message = fields[name] ?? '';
    document.querySelector(`#${name}-error`).textContent = message;
    form.elements[name].setAttribute('aria-invalid', String(message !== ''));
  }

  const firstInvalid = FIELDS.find((name) => fields[name]);
  if (firstInvalid) {
    form.elements[firstInvalid].focus();
```

```
  }
}

function addToHistory(order, receipt) {
  const item = document.createElement('li');
  item.textContent =
    `${receipt}: ${order.customer}, ${order.productName} ${order.quantity}টি, ` +
    formatTaka(order.total);
  historyList.prepend(item);
}

// ...
```

app.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

form.addEventListener('submit', async (event) => {
  event.preventDefault();
  submitButton.disabled = true;
  statusView.textContent = 'পাঠানো হচ্ছে...';

  try {
    const result = await placeOrder(Object.fromEntries(new FormData(form)), send);

    if (result.ok) {
      showFieldErrors();
      addToHistory(result.order, result.receipt);
      statusView.textContent =
        `Order গৃহীত: ${result.receipt}, মোট ${formatTaka(result.order.total)}।`;
      form.reset();
      return;
    }

    showFieldErrors(result.fields);
    statusView.textContent = result.message;
    if (result.cause) {
      console.warn('Order পাঠানো যায়নি:', result.cause);
    }
  } catch (error) {
    // অপ্রত্যাশিত: Program-এর ভুল। ব্যবহারকারী পান নিরাপদ বার্তা,
    // ডেভেলপার পান পুরো বিবরণ।
    statusView.textContent =
      'অপ্রত্যাশিত সমস্যা হয়েছে। Page Reload করে আবার চেষ্টা করুন।';
    console.error('OrderGuard-এ অপ্রত্যাশিত ত্রুটি:', error);
  } finally {
    submitButton.disabled = false;
  }
});
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Button বন্ধ করা প্রথমেই,** যাতে পাঠানোর সময় দুবার Click-এ দুটি Order না যায়। আবার চালু হয় `finally`-তে: সফল হলে `return`-এর পরেও, প্রত্যাশিত ব্যর্থতায়, এমনকি অপ্রত্যাশিত ত্রুটিতেও।
- **`Object.fromEntries(new FormData(form))`** Form-এর সব ঘর একবারে একটি Object-এ আনে: Key হলো প্রতিটি ঘরের `name`। Trust Boundary-র ভেতরে ঢোকার আগে এটিই কাঁচা তথ্য।
- **তিনটি পথ।** সফল: ঘরের বার্তা মুছে ইতিহাস ও রসিদ। প্রত্যাশিত ব্যর্থতা: `result.fields` থাকলে প্রতিটি ঘরের নিচে বার্তা, এবং ওপরে সাধারণ বার্তা। অপ্রত্যাশিত: `catch`-এ নিরাপদ বার্তা এবং `console.error`।
- **`showFieldErrors()` Argument ছাড়া** ডাকলে Default Parameter `{}` সব বার্তা মুছে দেয়, এবং সব ঘর `aria-invalid="false"` হয়।
- **`console.warn` বনাম `console.error`।** Network বিচ্ছিন্নতা প্রত্যাশিত, তাই সতর্কতা; শুধু অপ্রত্যাশিত ত্রুটি লাল `error`। Console দেখে এক নজরে বোঝা যায় কোনটি Bug।
- **কৃত্রিম Network বিচ্ছিন্নতা।** `?offline=1` দিলে `send` সব সময় `TypeError` ছোড়ে, ঠিক যেমন বিচ্ছিন্ন অবস্থায় `fetch` করে (অধ্যায় ১০)।

চালানো এবং প্রত্যাশিত আচরণ

`index.html` Browser-এ খুলুন (Double Click)। Test চালাতে Project Folder থেকে:

Terminal

```
node --test
node tests/browser-check.mjs
```

ইনপুট	যা হওয়ার কথা
সব ঘর ফাঁকা	চারটি ঘরের নিচেই বার্তা; নামের ঘরে Focus
Sara, `sara@example.com`, Keyboard, 2	"Order গৃহীত: ORD-0001, মোট 2,500 টাকা।"; ইতিহাসে সারি; Form খালি
পরিমাণ `0`, `2.5`, `1e3` বা `101`	পরিমাণের ঘরে বার্তা
Headset, পরিমাণ 3	"দুঃখিত, Headset স্টকে আছে 2টি।"
`index.html?offline=1` থেকে বৈধ Order	Network-এর বার্তা; Button আবার চালু
নামে `<img src="" onerror="...">`	লেখাটি হুবহু ইতিহাসে; কোনো Code চলে না

Companion Source Code-এর `order-validation.test.js` Node.js-এ ১৬টি Test চালায়, এবং `browser-check.mjs` আসল Chrome-এ Page খুলে ওপরের সব পরিস্থিতিসহ ১২টি আচরণ যাচাই করে, যার মধ্যে Button-এর বন্ধ ও চালু হওয়া এবং ইচ্ছা করে ঢোকানো একটি Bug-এর অপ্রত্যাশিত ত্রুটির পথও আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ত্রুটির পর Button আর চালু হয় না	চালু করার Code `finally`-র বাইরে	`finally`
সব ভুলে একই "Network" বার্তা	যাচাই ও পাঠানো একই `try`-তে	দুটি আলাদা `try`
Bug-এর ত্রুটি Console-এ দেখা যায় না	খালি বা অতি সাধারণ `catch`	ধরুন যাচাই, অচেনা হলে `throw`
ঘরের লাল সীমানা থেকে যায়	`aria-invalid` মোছা হয়নি	সফল হলে `showFieldErrors()`
পণ্যের ঘরে `toString` পাঠালে অদ্ভুত আচরণ	`catalog[id]` সরাসরি	`Object.hasOwn`
`1e3` বা `2.5` পরিমাণ গৃহীত	`Number()` দিয়ে সরাসরি রূপান্তর	`parseQuantity`-এর গঠন যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Server কৃত্রিম, তাই স্টক Page Reload করলে আবার শুরু থেকে গোনে। Email যাচাই শুধু মোটা দাগের গঠনের (অধ্যায় ৮)। আসল দোকানে একই যাচাই Server-এও হতো, কারণ Browser-এর Code ব্যবহারকারীর নিয়ন্ত্রণে।

১১.১৩ অনুশীলনী

প্রাথমিক স্তর

১. নিচের প্রতিটি কোন ধরনের ত্রুটি দেয়, না চালিয়ে বলুন: `undefined.x`, `missingName`, `new Array(-1)`, `JSON.parse('')`। তারপর Code দিয়ে যাচাই করুন।

২. `safeDivide(a, b)` লিখুন, যা `b` শূন্য হলে `RangeError` ছোড়ে। `try` / `catch` দিয়ে দুটি ক্ষেত্র দেখান।

৩. একটি Function-এর `try`-তে `return 'ক'` এবং `finally`-তে `console.log('খ')` আছে। Function ডেকে ফল ছাপলে কোন ক্রমে কী দেখা যাবে?

৪. `try { saveOrder(); } catch (error) {}` Code-টির সমস্যা কী? উন্নত করুন।

৫. `requireText(value, field)` লিখুন: `value` লেখা না হলে `TypeError`, ফাঁকা হলে `ValidationError`।

মধ্যম স্তর

৬. `parseJsonOr(text, fallback)` লিখুন: ভাঙা JSON-এ `fallback` ফেরত দেয়, কিন্তু অন্য যেকোনো ত্রুটি আবার ছোড়ে।

৭. `NotFoundError` তৈরি করুন, যাতে খোঁজা `id` থাকে; একটি `findProduct` Function-এ ব্যবহার করুন।

৮. `readConfig(text)` লিখুন, যা ভাঙা JSON-এ `'Config পড়া যায়নি'` বার্তার নতুন Error ছোড়ে এবং মূল ত্রুটি `cause`-এ রাখে।

৯. একটি `async` Function `loadProfile` লিখুন যা কৃত্রিমভাবে ব্যর্থ হয়, এবং `try` / `catch` / `finally` দিয়ে ত্রুটির বার্তা ও "Loading বন্ধ" দেখান।

চ্যালেঞ্জ স্তর

১০. `validateSignup(form)` লিখুন, যা নাম, Email ও Password-এর সব ভুল একসঙ্গে একটি Object-এ জমা করে `ValidationError` ছোড়ে।

১১. `retryIfAllowed(task, times)` লিখুন, যা শুধু `error.retryable === true` ত্রুটিতে আবার চেষ্টা করে।

১২. `attempt(fn)` লিখুন, যা `{ ok: true, value }` বা `{ ok: false, error }` ফেরত দেয়, কখনো ছোড়ে না। কখন এই ধরন `throw`-এর চেয়ে ভালো?

১১.১৪ অনুশীলনীর সমাধান

সমাধান ১

which-error.js · JavaScript — চলবে: Browser Console ও Node.js

```
const cases = [
  () => undefined.x,
  () => missingName,
  () => new Array(-1),
  () => JSON.parse(''),
];

for (const run of cases) {
  try {
    run();
  } catch (error) {
    console.log(error.name);
  }
}
```

Output

```
TypeError
ReferenceError
RangeError
SyntaxError
```

সমাধান ২

safe-divide.js · JavaScript — চলবে: Browser Console ও Node.js

```
function safeDivide(a, b) {
  if (b === 0) {
    throw new RangeError('শূন্য দিয়ে ভাগ করা যায় না');
  }
  return a / b;
}

for (const [a, b] of [[10, 4], [10, 0]]) {
  try {
    console.log(safeDivide(a, b));
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
2.5
RangeError: শূন্য দিয়ে ভাগ করা যায় না
```

JavaScript নিজে `10 / 0`-এ ত্রুটি দেয় না, `Infinity` দেয়; তাই নিয়মটি নিজে যাচাই করতে হয়েছে।

সমাধান ৩

finally-order.js · JavaScript — চলবে: Browser Console ও Node.js

```
function demo() {
  try {
    return 'ক';
  } finally {
    console.log('খ');
  }
}

console.log(demo());
```

Output

```
খ
ক
```

ফেরত মান তৈরি হয়েছে, কিন্তু Function ফিরে যাওয়ার আগে `finally` চলেছে; তারপর ডাকার জায়গায় `'ক'` ছাপা হয়েছে।

সমাধান ৪ — খালি `catch` সব ত্রুটি চাপা দেয়: Order আদৌ সংরক্ষিত হলো কি না, কেউ জানতে পারেন না, এবং Code-এর Bug-ও লুকিয়ে যায়। প্রত্যাশিত ত্রুটি চিনে ব্যবহারকারীকে জানান, বাকিগুলো আবার ছুড়ুন:

no-empty-catch.js · JavaScript — চলবে: Browser Console ও Node.js

```js
class ServiceError extends Error {}

function saveOrder() {
  throw new ServiceError('স্টক শেষ');
}

try {
  saveOrder();
} catch (error) {
  if (!(error instanceof ServiceError)) {
    throw error;
  }
  console.log(`Order সংরক্ষণ হয়নি: ${error.message}`);
}
```

Output

```
Order সংরক্ষণ হয়নি: স্টক শেষ
```

সমাধান ৫

require-text.js · JavaScript — চলবে: Browser Console ও Node.js

```js
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ValidationError';
  }
}

function requireText(value, field) {
  if (typeof value !== 'string') {
    throw new TypeError(`${field}: লেখা আশা করা হয়েছিল`);
```

```
  }
  if (value.trim() === '') {
    throw new ValidationError(`${field} ঘরটি পূরণ করুন`);
  }
  return value.trim();
}

for (const value of [' Sara ', '   ', 42]) {
  try {
    console.log(`[${requireText(value, 'নাম')}]`);
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
[Sara]
ValidationError: নাম ঘরটি পূরণ করুন
TypeError: নাম: লেখা আশা করা হয়েছিল
```

ফাঁকা ঘর ব্যবহারকারীর ভুল (`ValidationError`); সংখ্যা আসা মানে Code ভুলভাবে ডেকেছে (`TypeError`)।

সমাধান ৬

parse-json-or.js · JavaScript চলবে: Browser Console ও Node.js

```
function parseJsonOr(text, fallback) {
  try {
    return JSON.parse(text);
  } catch (error) {
    if (error instanceof SyntaxError) {
      return fallback;
    }
    throw error;
  }
}

console.log(parseJsonOr('{"theme":"dark"}', {}).theme);
console.log(parseJsonOr('{broken', { theme: 'light' }).theme);
```

Output

```
dark
light
```

সমাধান ৭

not-found.js · JavaScript চলবে: Browser Console ও Node.js

```
class NotFoundError extends Error {
  constructor(id) {
    super(`পণ্য পাওয়া যায়নি: ${id}`);
    this.name = 'NotFoundError';
    this.id = id;
  }
}

const products = [{ id: 'P1', name: 'Keyboard' }];

function findProduct(id) {
  const product = products.find((item) => item.id === id);
  if (!product) {
    throw new NotFoundError(id);
  }
  return product;
}

try {
  console.log(findProduct('P1').name);
  findProduct('P9');
} catch (error) {
  console.log(error instanceof NotFoundError, error.id, error.message);
}
```

Output

```
Keyboard
true P9 পণ্য পাওয়া যায়নি: P9
```

সমাধান ৮

read-config.js · JavaScript — চলবে: Browser Console ও Node.js

```
function readConfig(text) {
  try {
    return JSON.parse(text);
  } catch (error) {
    throw new Error('Config পড়া যায়নি', { cause: error });
  }
}

try {
  readConfig('port=8080');
} catch (error) {
  console.log(error.message);
  console.log(error.cause instanceof SyntaxError);
}
```

Output

```
Config পড়া যায়নি
true
```

সমাধান ৯

load-profile.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function loadProfile() {
  await wait(50);
  throw new Error('Profile Server সাড়া দেয়নি');
}

async function main() {
  console.log('Loading শুরু');
  try {
    await loadProfile();
  } catch (error) {
    console.log(`ব্যর্থ: ${error.message}`);
  } finally {
    console.log('Loading বন্ধ');
  }
}

main();
```

Output

```
Loading শুরু
ব্যর্থ: Profile Server সাড়া দেয়নি
Loading বন্ধ
```

সমাধান ১০

validate-signup.js · JavaScript — চলবে: Browser Console ও Node.js

```
class ValidationError extends Error {
  constructor(fields) {
    super('কয়েকটি ঘর ঠিক করতে হবে');
    this.name = 'ValidationError';
    this.fields = fields;
  }
}

function validateSignup({ name = '', email = '', password = '' }) {
  const fields = {};

  if (name.trim().length < 2) {
    fields.name = 'নাম কমপক্ষে 2 অক্ষর';
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    fields.email = 'সঠিক Email';
  }
  if (password.length < 8) {
    fields.password = 'Password কমপক্ষে 8 অক্ষর';
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
  }
  return { name: name.trim(), email };
}

try {
  validateSignup({ name: 'S', email: 'sara@', password: '123' });
} catch (error) {
  for (const [field, message] of Object.entries(error.fields)) {
    console.log(`${field}: ${message}`);
  }
}
```

Output

```
name: নাম কমপক্ষে 2 অক্ষর
email: সঠিক Email
password: Password কমপক্ষে 8 অক্ষর
```

সমাধান ১১

retry-if-allowed.js · JavaScript — চলবে: Browser Console ও Node.js

```
async function retryIfAllowed(task, times = 3) {
  for (let attempt = 1; ; attempt += 1) {
    try {
      return await task(attempt);
    } catch (error) {
      if (error.retryable !== true || attempt >= times) {
        throw error;
      }
      console.log(`চেষ্টা ${attempt} ব্যর্থ, আবার`);
    }
  }
}

const busyTwice = async (attempt) => {
  if (attempt < 3) {
    throw Object.assign(new Error('ব্যস্ত'), { retryable: true });
  }
  return 'সফল';
};

const forbidden = async () => {
  throw Object.assign(new Error('অনুমতি নেই'), { retryable: false });
};

async function main() {
  console.log(await retryIfAllowed(busyTwice));
  await retryIfAllowed(forbidden).catch((error) => console.log(error.message));
}

main();
```

Output

```
চেষ্টা 1 ব্যর্থ, আবার
চেষ্টা 2 ব্যর্থ, আবার
সফল
অনুমতি নেই
```

সমাধান ১২

attempt.js · JavaScript চলবে: Browser Console ও Node.js

```
function attempt(fn) {
  try {
    return { ok: true, value: fn() };
  } catch (error) {
    return { ok: false, error };
  }
}

const good = attempt(() => JSON.parse('[1, 2]'));
const bad = attempt(() => JSON.parse('[1, 2'));

console.log(good.ok, good.value.length);
console.log(bad.ok, bad.error.name);
```

Output

```
true 2
false SyntaxError
```

এই ধরন ভালো যেখানে ব্যর্থতা স্বাভাবিক ও ঘনঘন, এবং ডাকার জায়গাতেই তা সামলাতে হবে: যেমন অনেকগুলো সারি পড়া, যার কিছু ভাঙা হতে পারে। Project-এর `placeOrder` প্রত্যাশিত ত্রুটির জন্য এটিই ব্যবহার করেছে। কিন্তু অপ্রত্যাশিত ত্রুটির জন্য `throw` ভালো: সেগুলো নিজে থেকেই ওপরে উঠে যায়, এবং কেউ ভুলে "যাচাই না করা" ফল ব্যবহার করে ফেলতে পারেন না।

১১.১৫ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোন ত্রুটি Program চলা শুরুর আগেই ধরা পড়ে?

ক) TypeError

খ) Logical Error

গ) SyntaxError

ঘ) RangeError

প্রশ্ন ২. `null.length` কোন ধরনের ত্রুটি দেয়?

ক) TypeError

খ) ReferenceError

গ) RangeError

ঘ) SyntaxError

প্রশ্ন ৩. `finally` কখন চলে?

ক) শুধু ত্রুটি হলে

খ) শুধু সফল হলে

গ) `try`-তে `return` না থাকলে

ঘ) সব ক্ষেত্রে, `return` থাকলেও

প্রশ্ন ৪. কেন `throw 'ভুল'`-এর বদলে `throw new Error('ভুল')`?

ক) Error Object-এ `stack` এবং `name` থাকে

খ) লেখা `throw` করা যায় না

গ) দ্রুত চলে

ঘ) `catch` শুধু Error ধরে

প্রশ্ন ৫. একটি `catch`-এ অচেনা ধরনের ত্রুটি এলে সাধারণত কী করা উচিত?

ক) চুপচাপ উপেক্ষা

খ) আবার `throw` করা

গ) `return null`

ঘ) Page Reload

প্রশ্ন ৬. কোনটি প্রত্যাশিত ত্রুটি?

ক) নামের বানান ভুলের কারণে `ReferenceError`

খ) `undefined`-এর Property পড়া

গ) ব্যবহারকারী পরিমাণে `0` লিখেছেন

ঘ) ভুল সূত্রে হিসাব

প্রশ্ন ৭. `new Error('Settings পড়া যায়নি', { cause: error })`-এ `cause` কী কাজ করে?

ক) নতুন ত্রুটি চাপা দেয়

খ) মূল ত্রুটি ধরে রাখে, যাতে আসল কারণ খুঁজে পাওয়া যায়

গ) ত্রুটিকে প্রত্যাশিত করে

ঘ) `stack` মুছে দেয়

প্রশ্ন ৮. `Number('')`-এর মান কত?

ক) `NaN`

খ) `0`

গ) `null`

ঘ) `undefined`

> **প্রশ্ন ৯.** `catalog[id]`-এর বদলে `Object.hasOwn(catalog, id)` যাচাই কেন?
>
> ক) দ্রুত চলে
>
> খ) `toString`-এর মতো উত্তরাধিকারসূত্রে পাওয়া Key গ্রহণ না করতে
>
> গ) `id` সংখ্যা হলে কাজ করতে
>
> ঘ) Catalog বদলাতে

> **প্রশ্ন ১০.** Browser-এ ধরা না পড়া Promise-এর ত্রুটি কোন Event-এ জানা যায়?
>
> ক) `error`
>
> খ) `rejected`
>
> গ) `catch`
>
> ঘ) `unhandledrejection`

১১.১৬ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–খ · ৯–খ · ১০–ঘ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	JavaScript পুরো File আগে পড়ে; ব্যাকরণের ভুলে কিছুই চলে না।
২	ক	ভুল ধরনের মান (`null`) থেকে Property পড়া।
৩	ঘ	সফল, ব্যর্থ বা আগেভাগে `return`—সব পথে `finally`।
৪	ক	Stack থেকে ত্রুটির জায়গা পাওয়া যায়, `name` ও `instanceof` দিয়ে ধরন চেনা যায়।
৫	খ	না চিনে সামলালে Bug লুকায়; ওপরের স্তর বা Console দেখুক।
৬	গ	ব্যবহারকারী নিজে ঠিক করতে পারেন; বাকিগুলো Code-এর ভুল।
৭	খ	ওপরের স্তরের বার্তার সঙ্গে নিচের স্তরের আসল ত্রুটিও থাকে।
৮	খ	ফাঁকা লেখা `0`-তে রূপান্তরিত হয়, যা যাচাইয়ের একটি ফাঁদ।
৯	খ	প্রতিটি Object উত্তরাধিকারসূত্রে কিছু Property পায়।
১০	ঘ	`error` সাধারণ ত্রুটির জন্য; Promise-এর জন্য `unhandledrejection`।

১১.১৭ অধ্যায়ের সারসংক্ষেপ

- Syntax Error-এ Program চলেই না; Runtime Error চলার সময় থামায়; Logical Error নিঃশব্দে ভুল ফল দেয়, যা Test ধরে।

- Error Object-এ `name`, `message`, `stack`; সিদ্ধান্ত নিন `name` বা `instanceof` দিয়ে, `message`-এর লেখা দিয়ে নয়।
- `throw new Error(...)`; `catch` কাছের ত্রুটি ধরে; `finally` সব পথে চলে, তাই পরিষ্কারের কাজ সেখানে।
- নিজের Error-এর ধরন `class ... extends Error` দিয়ে, `this.name` বসিয়ে; মূল কারণ `cause`-এ।
- প্রত্যাশিত ত্রুটি কাছেই ধরে ব্যবহারকারীকে কী করতে হবে তা জানান; অপ্রত্যাশিত ত্রুটি ওপরের এক জায়গায়, নিরাপদ বার্তা ও পুরো Log।
- `catch`-এ ধরন যাচাই; অচেনা হলে আবার `throw`; খালি `catch` নয়।
- Trust Boundary-তে কঠোর যাচাই: গঠন, সীমা, অনুমোদিত তালিকা, `Object.hasOwn`।
- Asynchronous ত্রুটিতে `await` ও `try`; শেষ প্রতিরক্ষা `error` ও `unhandledrejection`।
- Stack Trace ওপর থেকে পড়ুন, তারপর মান কোথা থেকে এলো তা পেছনে খুঁজুন; Breakpoint ও Pause on exceptions।

পরের অধ্যায়ে Modern JavaScript ও Module: Code-কে আলাদা File-এ ভাগ করা, `import` ও `export`, এবং পরিষ্কার Code লেখার অভ্যাস। এখন পর্যন্ত Project-গুলোতে যে Script-এর ক্রম মেনে চলতে হয়েছে, Module সেই সমস্যার সমাধান করে।

১১.১৮ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Syntax Error	ব্যাকরণের ভুল; Program চলে না।
Runtime Error	চলার সময় ঘটা ত্রুটি।
Logical Error	Program চলে, কিন্তু ফল ভুল।
`throw`	ত্রুটি ছোড়া; স্বাভাবিক প্রবাহ থামে।
`finally`	সফল বা ব্যর্থ, সব ক্ষেত্রে চলা অংশ।
Custom Error	নিজের তৈরি Error-এর ধরন।
`cause`	নতুন ত্রুটির ভেতরে রাখা মূল ত্রুটি।
Expected Error	পরিবেশ বা ব্যবহারকারীর কারণে, যা স্বাভাবিকভাবে সামলানো যায়।
Unexpected Error	Program-এর ভুল; ডেভেলপারকে ঠিক করতে হয়।
Trust Boundary	যেখানে বাইরের তথ্য Program-এ ঢোকে।
Stack Trace	ত্রুটির মুহূর্তে Function-ডাকার ধারাবাহিক তালিকা।
Breakpoint	DevTools-এ Program থামানোর জায়গা।

১১.১৯ আরও পড়ুন

- **MDN — Control flow and error handling:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling` — `try`, `catch`, `throw` ও `finally`।
- **MDN — Error:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error` — Error-এর ধরন, `cause` এবং নিজের Error।
- **MDN — Window: unhandledrejection event:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/unhandledrejection_event` — ধরা না পড়া Promise-এর ত্রুটি।
- **Chrome DevTools — Debug JavaScript:** `https://developer.chrome.com/docs/devtools/javascript` — Breakpoint ও Step-এর ব্যবহার।
- **OWASP — Input Validation Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html` — Trust Boundary-তে যাচাইয়ের নীতি।

অধ্যায় ১২

Modern JavaScript (ES6+), Modules এবং Clean Code Practices

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- ES2015 থেকে আজ পর্যন্ত JavaScript-এ যুক্ত হওয়া প্রধান সুবিধাগুলো চিনতে, এবং কোনগুলো আগের অধ্যায়ে শিখেছেন তা মেলাতে।
- `toSorted`, `with`, Logical Assignment, `structuredClone` ও `Object.groupBy`-এর মতো নতুন সুবিধা ব্যবহার করতে।
- ES Module দিয়ে Code-কে আলাদা File-এ ভাগ করতে: Named ও Default `export`, `import`, নাম বদলানো এবং Namespace Import।
- Module-এর নিয়ম বুঝতে: নিজস্ব Scope, স্বয়ংক্রিয় Strict Mode, একবার চলা, Live Binding এবং Dynamic `import()`।
- Browser ও Node.js-এ Module চালাতে, এবং দুই পরিবেশের পার্থক্য সামলাতে।
- পাঠযোগ্য Code লিখতে: অর্থবহ নাম, ছোট Function, Guard Clause, নামযুক্ত Constant।
- Pure Core ও Side Effect-এর সীমানা মেনে Module সাজাতে।
- একটি সম্পূর্ণ **Modular Shopping Cart** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function, Default Parameter, Pure Function।
- অধ্যায় ৭: Destructuring, Spread, Rest, Optional Chaining এবং `??`।
- অধ্যায় ৯: DOM, Event Delegation এবং `localStorage`।
- অধ্যায় ১১: নিজের Error-এর ধরন (`class ... extends Error`)।
- Node.js 22 বা নতুন সংস্করণ এবং একটি আধুনিক Browser।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৯ থেকে ১১-এর Project-গুলোতে একটি অস্বস্তিকর নিয়ম মানতে হয়েছে: HTML-এ Script-গুলো সঠিক ক্রমে রাখতে হতো, আর এক File-এর সব Function ও Variable অন্য File-এ নিজে থেকেই দেখা যেত। দুটি File-এ একই নামের Variable থাকলে একটি অন্যটিকে নীরবে মুছে দিত। তিনটি File-এ এটি সামলানো যায়; তিনশো File-এ যায় না।

ES Module এই সমস্যার মানসম্মত সমাধান। প্রতিটি File নিজের একটি ঘর: বাইরে থেকে শুধু তা-ই দেখা যায় যা সে স্পষ্টভাবে `export` করে, এবং প্রতিটি File নিজেই বলে দেয় কোন File থেকে কী লাগবে। আধুনিক প্রতিটি JavaScript Project—Browser-এ হোক বা Node.js-এ, React হোক বা Server—Module দিয়ে গঠিত।

Module-এর সঙ্গে আসে আরেকটি দক্ষতা: Code এমনভাবে লেখা যাতে অন্য কেউ (এবং ছয় মাস পরের আপনি) সহজে পড়তে ও বদলাতে পারেন। পেশাদার জীবনে আপনি Code লেখার চেয়ে অনেক বেশি সময় Code পড়বেন।

১২.১ Modern JavaScript: কী বদলেছে

২০১৫ সালের ES2015 (ES6 নামে পরিচিত) সংস্করণ JavaScript-কে আমূল বদলে দেয়, এবং তারপর থেকে প্রতি বছর একটি নতুন সংস্করণ আসে। এই বইয়ে আপনি এর বেশির ভাগ ইতিমধ্যে ব্যবহার করেছেন:

সুবিধা	কোন সংস্করণে	এই বইয়ে
`let`, `const`	ES2015	অধ্যায় ২
Template Literal	ES2015	অধ্যায় ২, ৮
Arrow Function, Default Parameter	ES2015	অধ্যায় ৫
Destructuring, Spread, Rest	ES2015, ES2018	অধ্যায় ৭
Promise; `async` / `await`	ES2015; ES2017	অধ্যায় ১০
Optional Chaining, `??`	ES2020	অধ্যায় ৭
`class`	ES2015	অধ্যায় ১১ (সংক্ষেপে), ১৩

সুবিধা	কোন সংস্করণে	এই বইয়ে
Module (`import` / `export`)	ES2015	এই অধ্যায়

এগুলো আবার শেখানো হবে না। এই অধ্যায়ের মূল বিষয় Module এবং পরিষ্কার Code; তার আগে কয়েকটি নতুন সুবিধা, যেগুলো প্রতিদিনের Code-কে আরও নিরাপদ করে।

> **নোট:** নতুন কোনো সুবিধা ব্যবহারের আগে দেখে নিন আপনার লক্ষ্য-পরিবেশ (ব্যবহারকারীদের Browser, Server-এর Node.js) তা সমর্থন করে কি না। MDN-এর প্রতিটি পাতার শেষে "Browser compatibility" সারণি থাকে। এই অধ্যায়ের সব উদাহরণ Node.js 22 এবং Chrome 155-এ পরীক্ষিত।

১২.২ কয়েকটি নতুন সুবিধা

অধ্যায় ৬-এ দেখেছেন, `sort()` মূল Array বদলে দেয়। ES2023-এ এর বদল-না-করা সংস্করণ এসেছে।

new-features.js · JavaScript — চলবে: Node.js

```
const scores = [72, 95, 58];

console.log(scores.toSorted((a, b) => b - a));
console.log(scores.toReversed());
console.log(scores.with(0, 80));
console.log(scores);

const settings = { theme: '', fontSize: 0 };
settings.theme ||= 'light';
settings.fontSize ??= 16;
settings.language ??= 'bn';
console.log(settings);

console.log(1_500_000 + 250_000);
```

Output

```
[ 95, 72, 58 ]
[ 58, 95, 72 ]
[ 80, 95, 58 ]
[ 72, 95, 58 ]
{ theme: 'light', fontSize: 0, language: 'bn' }
1750000
```

- **`toSorted`**, **`toReversed`**, **`with(index, value)`** নতুন Array ফেরত দেয়; শেষ লাইনে মূল `scores` অপরিবর্তিত। অধ্যায় ৭-এর "নতুন মান তৈরি করুন, পুরোনোটি বদলাবেন না" নীতি এখন এক Method-এই।

- `a ||= b` মানে `a` Falsy হলে `b` বসাও; `a ??= b` মানে শুধু `null` বা `undefined` হলে। পার্থক্যটি দেখুন: ফাঁকা `theme` বদলেছে, কিন্তু `fontSize`-এর `0` থেকে গেছে, কারণ `0` Falsy হলেও `null` নয় (অধ্যায় ৩)।
- `1_500_000`: সংখ্যার মাঝের `_` শুধু পড়ার সুবিধার জন্য; মানে কোনো প্রভাব নেই।

clone-group.js · JavaScript — চলবে: Node.js

```
const order = { id: 7, items: [{ name: 'Pen', qty: 2 }] };

const shallow = { ...order };
const deep = structuredClone(order);
deep.items[0].qty = 5;
shallow.items[0].name = 'Pencil';

console.log(order.items[0]);
console.log(deep.items[0]);

const products = [
  { name: 'Pen', type: 'stationery' },
  { name: 'Mouse', type: 'electronics' },
  { name: 'Notebook', type: 'stationery' },
];
const byType = Object.groupBy(products, (product) => product.type);

console.log(Object.keys(byType));
console.log(byType.stationery.map((product) => product.name));
```

Output

```
{ name: 'Pencil', qty: 2 }
{ name: 'Pen', qty: 5 }
[ 'stationery', 'electronics' ]
[ 'Pen', 'Notebook' ]
```

- `structuredClone(value)` একটি সম্পূর্ণ Deep Copy তৈরি করে, অধ্যায় ৭-এর Shallow Copy-র সীমাবদ্ধতা ছাড়াই। Spread-এর Copy-তে ভেতরের Object ভাগাভাগি হয়েছে (`order`-এর নামও বদলে গেছে), কিন্তু `structuredClone`-এর Copy সম্পূর্ণ আলাদা। এটি Function Copy করতে পারে না।
- `Object.groupBy(items, fn)` প্রতিটি জিনিসকে `fn`-এর ফেরত দেওয়া Key অনুযায়ী দলে ভাগ করে; অধ্যায় ৭-এর হাতে লেখা গণনা-Object-এর কাজ এক লাইনে।

১২.৩ Module কেন: Global Scope-এর সমস্যা

সাধারণ `<script>`-এ লেখা প্রতিটি সবচেয়ে বাইরের Function একই Global Scope-এ থাকে। দুটি File-এ একই নাম থাকলে কী হয়, Browser-এ দেখুন:

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Global Scope</title>
</head>
<body>
  <script src="tax.js"></script>
  <script src="discount.js"></script>
  <script src="main.js"></script>
</body>
</html>
```

tax.js · JavaScript চলবে: Browser

```javascript
function calculate(price) {
  return price * 1.15;
}
```

discount.js · JavaScript চলবে: Browser

```javascript
function calculate(price) {
  return price * 0.9;
}
```

main.js · JavaScript চলবে: Browser

```javascript
console.log(`VAT-সহ দাম: ${calculate(1000)}`);
```

Output

```
VAT-সহ দাম: 900
```

`main.js`-এর লেখক VAT-এর হিসাব চেয়েছিলেন, পেয়েছেন ছাড়ের হিসাব। পরে লোড হওয়া `discount.js` আগের `calculate`-কে নীরবে প্রতিস্থাপন করেছে; কোনো Error নেই, কোনো সতর্কতা নেই। তা ছাড়া `main.js` পড়ে বোঝার উপায় নেই `calculate` কোথা থেকে আসছে। Module দুটি সমস্যারই সমাধান করে।

১২.৪ export এবং import

একটি Module হলো একটি File, যা নিজের যা অন্যকে দিতে চায় তা **`export`** করে, এবং অন্যের যা লাগে তা **`import`** করে। Node.js-এ `.mjs` Extension বলে দেয় File-টি একটি ES Module।

tax.mjs · JavaScript — চলবে: Node.js

```
export const VAT_RATE = 0.15;

export function withVat(price) {
  return Math.round(price * (1 + VAT_RATE));
}
```

discount.mjs · JavaScript — চলবে: Node.js

```
export function withDiscount(price, percent) {
  return Math.round(price * (1 - percent / 100));
}

export default function describe(price) {
  return `${price.toLocaleString('en-US')} টাকা`;
}
```

main.mjs · JavaScript — চলবে: Node.js

```
import { withVat, VAT_RATE } from './tax.mjs';
import describe, { withDiscount as applyDiscount } from './discount.mjs';
import * as tax from './tax.mjs';

console.log(`VAT ${VAT_RATE * 100}%: ${describe(withVat(1000))}`);
console.log(`ছাড়ের পর: ${describe(applyDiscount(1000, 10))}`);
console.log(tax.withVat(200), Object.keys(tax));
```

Output

```
VAT 15%: 1,150 টাকা
ছাড়ের পর: 900 টাকা
230 [ 'VAT_RATE', 'withVat' ]
```

লেখার ধরন	অর্থ
`export const X` / `export function f`	Named Export: নির্দিষ্ট নামে দেওয়া
`export default ...`	Default Export: File-এর প্রধান জিনিস; একটি File-এ সর্বোচ্চ একটি
`import { a, b } from './file.mjs'`	Named Import; নাম হুবহু মিলতে হবে
`import { a as b }`	Import-এর সময় নাম বদলানো
`import name from './file.mjs'`	Default Import; যেকোনো নাম দেওয়া যায়
`import * as ns from './file.mjs'`	সব Named Export একটি Object-এ (Namespace)

এখন `main.mjs` পড়লেই বোঝা যায় প্রতিটি নাম কোথা থেকে এসেছে, এবং দুটি File-এ একই নাম থাকলেও সংঘর্ষ হয় না: প্রতিটি File-এর Variable শুধু সেই File-এর।

> **পরামর্শ:** Named Export পছন্দ করুন। এতে Import-এর নাম Export-এর নামের সঙ্গে মেলে, তাই Code-এ খোঁজা সহজ, এবং বানান ভুল হলে সঙ্গে সঙ্গে Error আসে। Default Export-এ প্রতিটি File নিজের মতো নাম দিতে পারে, যা বড় Project-এ বিভ্রান্তি তৈরি করে।

ভুল নামে Import করলে কী হয়:

wrong-import.mjs · JavaScript — চলবে: Node.js

```
import { withVAT } from '../../04-modules/tax.mjs';

console.log(withVAT(100));
```

Error Message

```
SyntaxError: The requested module '../../04-modules/tax.mjs'
does not provide an export named 'withVAT'
```

Program-এর কোনো লাইন চলার **আগেই** ত্রুটি ধরা পড়েছে। কারণ `import` স্থির (Static): JavaScript Code চালানোর আগে সব File-এর `import` ও `export` মিলিয়ে দেখে। সাধারণ Script-এ একই ভুল ধরা পড়ত শুধু সেই লাইন চলার সময়, হয়তো কোনো ব্যবহারকারীর কাছে।

১২.৫ Module-এর নিয়ম

counter.mjs · JavaScript — চলবে: Node.js

```
console.log('counter.mjs চলছে');

export let count = 0;

export function increment() {
  count += 1;
}
```

report.mjs · JavaScript — চলবে: Node.js

```
import { count } from './counter.mjs';

export const showCount = () => `report দেখছে: ${count}`;
```

main.mjs · JavaScript চলবে: Node.js

```
import { count, increment } from './counter.mjs';
import { showCount } from './report.mjs';

increment();
increment();
console.log(count, showCount());
console.log(typeof this);

try {
  total = 5;
} catch (error) {
  console.log(error.name);
}

const { withVat } = await import('../04-modules/tax.mjs');
console.log(withVat(100));
```

Output

```
counter.mjs চলছে
2 report দেখছে: 2
undefined
ReferenceError
115
```

পাঁচটি নিয়ম এই Output-এ দেখা যাচ্ছে:

১. **একবার চলে।** `counter.mjs` দুটি File থেকে Import হয়েছে, কিন্তু "চলছে" ছাপা হয়েছে একবার। প্রথম Import-এ File চলে; পরের সবাই একই Module পায়।

২. **Live Binding।** `main.mjs` ও `report.mjs` দুজনেই `count`-এর নতুন মান `2` দেখেছে। Import করা নাম মূল Variable-এর দিকে একটি জানালা, তার Copy নয়। তবে Import করা নামে সরাসরি মান বসানো যায় না (`count = 5` Error দেয়); বদলাতে হয় Module-এর নিজের Function দিয়ে।

৩. **স্বয়ংক্রিয় Strict Mode।** ঘোষণা ছাড়া `total = 5` সাধারণ Script-এ নীরবে একটি Global Variable তৈরি করত; Module-এ এটি `ReferenceError`। আর সবচেয়ে বাইরের `this` হলো `undefined`।

৪. **নিজস্ব Scope।** `main.mjs`-এর Variable অন্য কোনো File দেখে না, যতক্ষণ না সেগুলো `export` করা হয়।

৫. **Top-level `await` এবং Dynamic `import()`।** Module-এর সবচেয়ে বাইরের স্তরে `await` লেখা যায় (অধ্যায় ১০-এর প্রতিশ্রুতি)। `import()` একটি Function-এর মতো, Program চলার সময় Module আনে এবং একটি Promise ফেরত দেয়। যে অংশ সব সময় লাগে না (যেমন একটি বড় Chart Library), তা প্রয়োজনের মুহূর্তে আনতে এটি কাজে লাগে।

১২.৬ Browser-এ Module

Browser-এ Module চালাতে `<script>`-এ `type="module"` লিখুন:

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Browser Module</title>
  <script type="module" src="app.mjs"></script>
</head>
<body>
  <h1 id="title">লোড হচ্ছে</h1>
</body>
</html>
```

greeting.mjs · JavaScript — চলবে: Browser

```
export const greet = (name) => `স্বাগতম, ${name}`;
```

app.mjs · JavaScript — চলবে: Browser

```
import { greet } from './greeting.mjs';

const title = document.querySelector('#title');
title.textContent = greet('Sara');

console.log(title.textContent);
console.log(typeof greet, typeof window.greet);
```

Output

```
স্বাগতম, Sara
function undefined
```

- **Module Script নিজে থেকেই `defer`।** এখানে Script `<head>`-এ, তবু `<h1>` পাওয়া গেছে (অধ্যায় ৯-এর `null` সমস্যা নেই)।
- **Global-এ কিছু যায় না।** `greet` `window`-এর Property হয়নি।
- **ঠিকানায় `./` এবং Extension বাধ্যতামূলক।** Browser `'./greeting.mjs'` বোঝে, `'greeting'` বোঝে না।
- **Server লাগে।** নিরাপত্তার কারণে Browser `file://` থেকে খোলা Page-এর Module লোড করে না; Console-এ CORS-এর ত্রুটি দেখায়। Project Folder-এ `python -m http.server 8000` বা VS Code-এর Live Server চালিয়ে `http://localhost:8000` খুলুন। (এই অধ্যায়ের সব Browser উদাহরণ একটি Local Server থেকে চালিয়ে পরীক্ষা করা হয়েছে।)

১২.৭ Node.js-এ Module

Node.js দুই ধরনের Module বোঝে। পুরোনো **CommonJS** (`require` ও `module.exports`) এখনো অনেক Package-এ আছে; নতুন Code-এ **ES Module** ব্যবহার করুন। Node.js একটি File-কে ES Module ধরে যদি:

- Extension `.mjs` হয়, অথবা
- নিকটতম `package.json`-এ `"type": "module"` লেখা থাকে (তখন `.js` File-ও ES Module)।

paths.mjs · JavaScript — চলবে: Node.js

```
import path from 'node:path';

console.log(path.basename(import.meta.dirname));
console.log(path.basename(import.meta.filename));
console.log(typeof require, typeof __dirname);
```

Output

```
08-node-paths
paths.mjs
undefined undefined
```

ES Module-এ CommonJS-এর `require`, `__dirname` ও `__filename` নেই। তাদের জায়গায় `import` এবং `import.meta.dirname` ও `import.meta.filename`। Node.js-এর নিজস্ব Module আনার সময় `node:` উপসর্গ লিখুন (`'node:path'`, `'node:fs'`), যাতে বোঝা যায় এটি Built-in, কোনো Package নয়।

পুরোনো CommonJS Code-এর সঙ্গে কাজ

বাস্তব Project-এ প্রায়ই এমন পুরোনো Code বা Package পাবেন, যা CommonJS-এ লেখা। ধরুন একটি পুরোনো Folder, যার `package.json` বলে দেয় সেখানকার `.js` File-গুলো CommonJS:

package.json · JSON — চলবে: Node.js

```
{ "type": "commonjs" }
```

tax.js · JavaScript — চলবে: Node.js

```
// পুরোনো CommonJS File: module.exports দিয়ে দেয়।
const RATE = 0.05;

function sourceTax(amount) {
  return amount * RATE;
}

module.exports = { RATE, sourceTax };
```

use-legacy.mjs · JavaScript চলবে: Node.js

```
import legacy from './legacy/tax.js';
import { sourceTax } from './legacy/tax.js';

console.log(legacy.RATE, sourceTax(2000));
```

Output

```
0.05 100
```

CommonJS File-এর `module.exports` Object-টি Default Import হিসেবে আসে, এবং Node.js যতটা পারে Named Import-ও দেয়। উল্টো পথ, অর্থাৎ CommonJS থেকে `require()` দিয়ে ES Module আনা, পুরোনো Node.js সংস্করণে কাজ করে না, এবং নতুন সংস্করণেও Top-level `await` থাকা Module-এ কাজ করে না। তাই নতুন Code সব সময় ES Module-এ লিখুন; পুরোনো Code ধীরে ধীরে রূপান্তর করুন। `package.json` ছাড়াও একটি একক File-কে CommonJS করা যায় Extension `.cjs` দিয়ে, ঠিক যেমন `.mjs` একটি File-কে ES Module করে।

১২.৮ Clean Code: পাঠযোগ্যতা আগে

Code একবার লেখা হয়, কিন্তু পড়া হয় বহুবার। নিচের Function কাজ করে, কিন্তু পড়ে বোঝা কঠিন:

before.js · JavaScript চলবে: Browser Console ও Node.js

```
function calc(o) {
  let r = 0;
  if (o) {
    if (o.items.length > 0) {
      for (const i of o.items) {
        r += i.p * i.q;
      }
      if (r > 2000) {
        r = r - r * 0.05;
      } else {
        r = r + 60;
      }
    }
  }
  return r;
}

console.log(calc({ items: [{ p: 250, q: 4 }, { p: 1850, q: 1 }] }));
console.log(calc({ items: [{ p: 120, q: 2 }] }));
```

Output

```
2707.5
300
```

একই কাজ, পরিষ্কার করে:

after.js · JavaScript চলবে: Browser Console ও Node.js

```
const BULK_THRESHOLD = 2000;
const BULK_DISCOUNT_RATE = 0.05;
const DELIVERY_FEE = 60;

const lineTotal = (item) => item.price * item.quantity;

function orderTotal(order) {
  if (!order || order.items.length === 0) {
    return 0;
  }

  const subtotal = order.items.reduce((sum, item) => sum + lineTotal(item), 0);

  if (subtotal > BULK_THRESHOLD) {
    return subtotal * (1 - BULK_DISCOUNT_RATE);
  }
  return subtotal + DELIVERY_FEE;
}

const bigOrder = {
  items: [{ price: 250, quantity: 4 }, { price: 1850, quantity: 1 }],
};
console.log(orderTotal(bigOrder));
console.log(orderTotal({ items: [{ price: 120, quantity: 2 }] }));
```

Output

```
2707.5
300
```

ফল হুবহু এক, কিন্তু এখন Code নিজেই নিজের ব্যাখ্যা:

- **অর্থবহ নাম।** `calc`, `o`, `r`, `p`, `q` থেকে `orderTotal`, `order`, `subtotal`, `price`, `quantity`। নাম বলে দেয় **কী**, Comment ছাড়াই।
- **জাদুকরী সংখ্যার নাম।** `2000`, `0.05`, `60`-এর অর্থ এখন স্পষ্ট, এবং নিয়ম বদলালে একটি জায়গায় বদলাতে হয়।
- **Guard Clause।** তিন স্তরের `if`-এর বদলে শুরুতেই অস্বাভাবিক ক্ষেত্রটি ফেরত (অধ্যায় ৫)। মূল যুক্তি বাঁ দিকে সোজা নামে।

- **ছোট, একটি কাজের Function।** `lineTotal`-এর একটি দায়িত্ব; তার নাম পড়লেই বোঝা যায়।
- **একই ফল যাচাই।** Refactoring মানে আচরণ না বদলে গঠন বদলানো। ঠিক এ কারণেই আগে Test, তারপর Refactoring: Test প্রমাণ করে আচরণ বদলায়নি।

Comment-এর নিয়ম: Code বলে **কী** করছে; Comment বলুক **কেন**। `// subtotal-এ যোগ` অপ্রয়োজনীয়; `// সরকারি নিয়মে 2000 টাকার বেশি Order-এ ছাড় বাধ্যতামূলক` মূল্যবান, কারণ সেটি Code থেকে জানা যায় না।

১২.৯ Module সাজানো: Pure Core, Side Effect সীমানায়

একটি Application-কে Module-এ ভাগ করার ভালো নিয়ম হলো দায়িত্ব অনুযায়ী ভাগ, এবং নির্ভরতার একটি দিক:

স্তর	কাজ	নির্ভর করে
Domain (Core)	ব্যবসার নিয়ম: হিসাব, যাচাই	কিছুর ওপর নয়; Pure Function
Adapter (সীমানা)	বাইরের জগৎ: Storage, Network	Domain
UI	DOM, Event	Domain ও Adapter

Domain কখনো DOM বা `localStorage` Import করে না। এর ফলে Domain-এর প্রতিটি Function Browser ছাড়াই Node.js-এ Test করা যায়, এবং একই Domain কাল একটি Mobile App বা Server-এও কাজ করবে। অধ্যায় ৯-১১-এর Project-এ আপনি এই বিভাজন হাতে মেনেছেন; Module এটিকে File-এর গঠনে স্পষ্ট করে। Adapter-এ বাইরের জিনিসটি (যেমন `localStorage`) Parameter হিসেবে নেওয়ার কৌশলকে বলে **Dependency Injection**: Test-এ তার জায়গায় একটি নকল দেওয়া যায়, যেমন অধ্যায় ১০-এ `fetchImpl`।

Circular Import: দুটি Module পরস্পরকে চাইলে

ধরুন `cart.mjs` Import করে `storage.mjs`-কে, আর `storage.mjs`-ও Import করে `cart.mjs`-কে। JavaScript এটি অনুমতি দেয়, কিন্তু একটি Module যখন চলা শুরু করে, অন্যটি তখনো অর্ধেক চলেছে; তাই একটি Module অন্যটির `const` বা `class` তার ঘোষণার আগেই পড়তে গিয়ে `ReferenceError` পেতে পারে। কোন Module আগে চলবে তা নির্ভর করে কোন File থেকে Program শুরু হলো তার ওপর, তাই এই Bug একটি জায়গায় দেখা দেয়, অন্য জায়গায় নয়।

সমাধান গঠনগত: নির্ভরতা একমুখী রাখুন। ওপরের সারণির নিয়ম মানলে Circular Import হয় না, কারণ Domain কখনো Adapter বা UI-কে Import করে না। দুটি Module-এর যদি একই জিনিস লাগে, সেটিকে তৃতীয় একটি ছোট Module-এ সরান, যেমন Project-এ `catalog.mjs`: `cart.mjs` ও `storage.mjs` দুজনেই পণ্যের তথ্য চায়, তাই দুজনেই `catalog.mjs`-কে Import করে, পরস্পরকে নয়।

১২.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`file://` থেকে Module Page খোলা	Console-এ CORS ত্রুটি, Page কাজ করে না	Local Server
`<script>`-এ `type="module"` নেই	`SyntaxError: Cannot use import statement outside a module`	`type="module"`
Import-এর ঠিকানায় `./` বা Extension নেই	Browser-এ ঠিকানা খুঁজে পাওয়া যায় না	`'./cart.mjs'`
Named Export-কে Default হিসেবে Import	`does not provide an export named 'default'`	`{ }`-সহ Import
Import-এর নামে বানান ভুল	`does not provide an export named ...`	Export-এর নাম হুবহু
Import করা নামে মান বসানো	`TypeError: Assignment to constant variable`	Module-এর Function দিয়ে বদলান
`.mjs`-এ `require` বা `__dirname`	`ReferenceError`	`import`, `import.meta.dirname`
দুটি Module পরস্পরকে Import করে	Import-এর সময় `undefined` বা `ReferenceError`	নির্ভরতা একমুখী রাখুন
Domain Module-এ DOM বা Storage	Node.js-এ Test করা যায় না	Adapter-এ সরান
`toSorted` ভেবে `sort`	মূল Array বদলে যায়	বদল-না-করা Method

১২.১১ Debugging নির্দেশিকা

- **Module লোড হয় না:** DevTools-এর Network Tab-এ প্রতিটি `.mjs`-এর Status দেখুন। `404` মানে ঠিকানা ভুল; File আসেনি মানে আগের কোনো Module-এ ত্রুটি।
- **Console-এর প্রথম ত্রুটি পড়ুন।** Module-এর Import-ত্রুটি Program শুরুর আগে আসে, এবং বার্তায় Module-এর নাম ও হারানো Export-এর নাম থাকে।
- **Sources Panel-এ** প্রতিটি Module আলাদা File হিসেবে দেখা যায়, এবং Breakpoint দেওয়া যায়।
- **Console-এ Module-এর Variable পাওয়া যায় না,** কারণ সেগুলো Global নয়। Breakpoint দিয়ে থামিয়ে দেখুন, অথবা সাময়িকভাবে `console.log` দিন। শুধু Debugging-এর জন্য `window.cart = cart` লেখা চলে, কিন্তু কাজ শেষে মুছে দিন।

- **Node.js-এ "Cannot use import statement outside a module"** মানে File-টি CommonJS হিসেবে চলছে: Extension `.mjs` করুন, অথবা `package.json`-এ `"type": "module"`।
- **Refactoring-এর পর আচরণ বদলেছে?** আগের ও পরের Code একই ইনপুটে চালিয়ে ফল মিলিয়ে দেখুন; এজন্যই Refactoring-এর আগে Test লিখুন।

১২.১২ পেশাদার অভ্যাস

- **একটি File, একটি দায়িত্ব;** File-এর নাম সেই দায়িত্বের নাম (`cart.mjs`, `storage.mjs`)।
- **Named Export;** Default Export শুধু যখন File-এর একটিই প্রধান জিনিস।
- **নির্ভরতা একমুখী:** UI থেকে Adapter, Adapter থেকে Domain; উল্টো দিকে কখনো নয়।
- **Domain Pure রাখুন,** বাইরের জিনিস Inject করুন।
- **নাম দিয়ে ব্যাখ্যা,** Comment দিয়ে কারণ।
- **ছোট Function, অগভীর Nesting,** Guard Clause।
- **আচরণ না বদলে ছোট ছোট ধাপে Refactoring,** প্রতিটি ধাপের পর Test।
- **একটি Code-এর ধরন (Style) পুরো Project-এ;** Prettier-এর মতো Formatter এবং ESLint-এর মতো Linter এ কাজ স্বয়ংক্রিয় করে (Tooling-এর বিস্তারিত পরের অধ্যায়গুলোতে)।

১২.১৩ Project: Modular Shopping Cart

একটি ছোট Stationery দোকানের Web Page: পণ্যের তালিকা, কার্টে যোগ, পরিমাণ বদলানো, Coupon, Delivery-র হিসাব, এবং Page বন্ধ করে খুললেও কার্ট থাকে। মূল লক্ষ্য Module-এর গঠন: প্রতিটি দায়িত্ব নিজের File-এ, এবং ব্যবসার নিয়মগুলো Browser ছাড়াই পরীক্ষাযোগ্য।

Project-এর চাহিদা

- চারটি পণ্য; একই পণ্য আবার যোগ করলে পরিমাণ বাড়ে; প্রতিটি পণ্যের পরিমাণ ০ থেকে ১০; ০ মানে সরানো।
- Coupon `STUDENT10` (১০%) ও `EID15` (১৫%); বড়-ছোট হাতের অক্ষর নির্বিশেষে; ভুল Coupon-এ বার্তা।
- ছাড়ের পর মূল্য ২০০০ টাকা বা বেশি হলে Delivery বিনামূল্যে, নইলে ৬০ টাকা; কার্ট ফাঁকা হলে Delivery নেই।
- সব টাকা পূর্ণসংখ্যায়; ছাড় নিকটতম টাকায় রাউন্ড।
- `localStorage`-এ সংরক্ষণ; ভাঙা বা অচেনা তথ্যে ফাঁকা কার্ট।
- কার্টের নিয়ম (`src/`) DOM বা Storage-এর কিছু জানে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
modular-shopping-cart/
├── index.html
├── style.css
├── app.mjs                 UI: DOM ও Event
├── demo.mjs                Node.js-এ কার্টের নিয়মের প্রদর্শনী
├── src/
│   ├── catalog.mjs         পণ্যের তালিকা
│   ├── money.mjs           টাকার লেখা
│   ├── cart.mjs            কার্টের নিয়ম (Domain)
│   └── storage.mjs         localStorage Adapter
├── README.md
└── tests/
    ├── cart.test.mjs
    ├── storage.test.mjs
    ├── static-server.mjs
    ├── cdp.mjs
    └── browser-check.mjs
```

src/catalog.mjs ও src/money.mjs

catalog.mjs · JavaScript — চলবে: Browser Console ও Node.js

```javascript
// পণ্যের তালিকা; দাম পূর্ণ টাকায়।
export const PRODUCTS = Object.freeze([
  Object.freeze({ id: 'notebook', name: 'Notebook', price: 120 }),
  Object.freeze({ id: 'pen-set', name: 'Pen Set', price: 250 }),
  Object.freeze({ id: 'calculator', name: 'Calculator', price: 950 }),
  Object.freeze({ id: 'backpack', name: 'Backpack', price: 1850 }),
]);

export function findProduct(id) {
  return PRODUCTS.find((product) => product.id === id) ?? null;
}
```

money.mjs · JavaScript — চলবে: Browser Console ও Node.js

```javascript
export const formatTaka = (amount) => `${amount.toLocaleString('en-US')} টাকা`;
```

পণ্যের তালিকা `Object.freeze` দিয়ে অপরিবর্তনীয় (অধ্যায় ৭): কোনো Module ভুল করে দাম বদলাতে পারবে না। `money.mjs` একটি লাইনের File হলেও আলাদা, কারণ টাকা দেখানোর নিয়ম (যেমন পয়সা যোগ করা) বদলালে শুধু এখানেই বদলাতে হবে।

src/cart.mjs

কার্টের সব নিয়ম; দুই অংশ মিলে সম্পূর্ণ File।

cart.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) — চলবে: Browser Console ও Node.js

```
import { findProduct } from './catalog.mjs';

export const MAX_QUANTITY = 10;
export const DELIVERY_FEE = 60;
export const FREE_DELIVERY_FROM = 2000;
const COUPONS = Object.freeze({ STUDENT10: 10, EID15: 15 });

export class CartError extends Error {
  constructor(message) {
    super(message);
    this.name = 'CartError';
  }
}

// কার্ট: { items: [{ id, quantity }], coupon }; প্রতিটি Function নতুন কার্ট ফেরত দেয়।
export const emptyCart = () => ({ items: [], coupon: null });

export const isCoupon = (code) => Object.hasOwn(COUPONS, code);

export function setQuantity(cart, productId, quantity) {
  if (!findProduct(productId)) {
    throw new CartError(`অজানা পণ্য: ${productId}`);
  }
  if (!Number.isInteger(quantity) || quantity < 0 || quantity > MAX_QUANTITY) {
    throw new CartError(`পরিমাণ 0 থেকে ${MAX_QUANTITY}-এর মধ্যে পূর্ণসংখ্যা হতে হবে।`);
  }

  const others = cart.items.filter((item) => item.id !== productId);
  if (quantity === 0) {
    return { ...cart, items: others };
  }

  const exists = others.length < cart.items.length;
  const items = exists
    ? cart.items.map((item) =>
      (item.id === productId ? { ...item, quantity } : item))
    : [...cart.items, { id: productId, quantity }];
  return { ...cart, items };
}

// ...
```

cart.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

export function addItem(cart, productId, quantity = 1) {
  const current = cart.items.find((item) => item.id === productId)?.quantity ?? 0;
  return setQuantity(cart, productId, current + quantity);
}

export const removeItem = (cart, productId) => setQuantity(cart, productId, 0);

export function applyCoupon(cart, code) {
  const normalized = String(code).trim().toUpperCase();

  if (!isCoupon(normalized)) {
    throw new CartError(`Coupon "${String(code).trim()}" সঠিক নয়।`);
  }
  return { ...cart, coupon: normalized };
}

// কার্টের প্রতিটি সারি, পণ্যের নাম ও দামসহ।
export function cartLines(cart) {
  return cart.items.map(({ id, quantity }) => {
    const { name, price } = findProduct(id);
    return { id, name, price, quantity, lineTotal: price * quantity };
  });
}

export function cartTotals(cart) {
  const subtotal = cartLines(cart).reduce((sum, line) => sum + line.lineTotal, 0);
  const percent = isCoupon(cart.coupon) ? COUPONS[cart.coupon] : 0;
  const discount = Math.round((subtotal * percent) / 100);
  const afterDiscount = subtotal - discount;
  const isFree = subtotal === 0 || afterDiscount >= FREE_DELIVERY_FROM;
  const delivery = isFree ? 0 : DELIVERY_FEE;

  return { subtotal, discount, delivery, total: afterDiscount + delivery };
}
```

- **কোন নাম `export`, কোনটি নয়।** `COUPONS` বাইরে দেওয়া হয়নি; বাইরের Code জানে শুধু `isCoupon`। ভবিষ্যতে Coupon Server থেকে এলে শুধু এই File বদলাবে।
- **`setQuantity` একমাত্র দরজা।** `addItem` ও `removeItem` দুটিই এটিকে ডাকে, তাই যাচাইয়ের নিয়ম একটি জায়গায়।
- **`cartTotals` ধাপে ধাপে, প্রতিটি ধাপের একটি নাম:** `subtotal`, `discount`, `afterDiscount`, `isFree`। নিয়ম বদলালে কোন লাইন বদলাতে হবে তা নাম দেখেই বোঝা যায়।
- **`CartError`** প্রত্যাশিত ত্রুটি (অধ্যায় ১১): UI এটি ধরে ব্যবহারকারীকে বার্তা দেখাবে; অন্য সব ত্রুটি ওপরে যাবে।

src/storage.mjs

storage.mjs · JavaScript চলবে: Browser Console ও Node.js

```
import { findProduct } from './catalog.mjs';
import { emptyCart, isCoupon, MAX_QUANTITY } from './cart.mjs';

const STORAGE_KEY = 'jsm-ch12-cart';

const isItem = (item) =>
  findProduct(item?.id) !== null &&
  Number.isInteger(item.quantity) &&
  item.quantity >= 1 && item.quantity <= MAX_QUANTITY;

const isCart = (value) =>
  Array.isArray(value?.items) && value.items.every(isItem) &&
  (value.coupon === null || isCoupon(value.coupon));

// storage Inject করা যায়: Browser-এ localStorage, Test-এ একটি নকল Object।
export function createCartStore(
  storage = globalThis.localStorage,
  key = STORAGE_KEY,
) {
  return {
    load() {
      try {
        const data = JSON.parse(storage.getItem(key) ?? 'null');
        return isCart(data) ? data : emptyCart();
      } catch {
        return emptyCart();
      }
    },
    save(cart) {
      try {
        storage.setItem(key, JSON.stringify(cart));
        return true;
      } catch {
        return false;
      }
    },
  };
}
```

`createCartStore` নিজে `localStorage` ব্যবহার করে না; যা দেওয়া হয় তা-ই ব্যবহার করে। Default Parameter-এর কারণে Browser-এ কিছু না দিলেই আসল `localStorage`, আর Test-এ `getItem` ও `setItem` আছে এমন যেকোনো Object। যাচাইয়ের নিয়ম অধ্যায় ৯-এর `parseStored`-এর মতোই, তবে এখানে পণ্য ও Coupon-এর তথ্য আসে Domain Module থেকে।

Node.js-এ প্রদর্শনী

Domain Module DOM ছাড়া চলে, তাই Node.js-এ সরাসরি ব্যবহার করা যায়:

demo.mjs · JavaScript — চলবে: Node.js

```
import {
  emptyCart, addItem, applyCoupon, cartTotals, setQuantity,
} from './src/cart.mjs';
import { formatTaka } from './src/money.mjs';

let cart = emptyCart();
cart = addItem(cart, 'notebook', 2);
cart = addItem(cart, 'backpack');
console.log(cartTotals(cart));

cart = applyCoupon(cart, ' student10 ');
console.log(cartTotals(cart));

try {
  setQuantity(cart, 'notebook', 11);
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
console.log(`পরিশোধ: ${formatTaka(cartTotals(cart).total)}`);
```

Output

```
{ subtotal: 2090, discount: 0, delivery: 0, total: 2090 }
{ subtotal: 2090, discount: 209, delivery: 60, total: 1941 }
CartError: পরিমাণ 0 থেকে 10-এর মধ্যে পূর্ণসংখ্যা হতে হবে।
পরিশোধ: 1,941 টাকা
```

দ্বিতীয় লাইনটি লক্ষ করুন: ছাড় দেওয়ার পর মূল্য ২০০০-এর নিচে নেমেছে, তাই Delivery আবার যোগ হয়েছে। নিয়মটি ("ছাড়ের পর মূল্য") ব্যবসার সিদ্ধান্ত; Code-এ তা `afterDiscount` নামে স্পষ্ট।

index.html, style.css ও app.mjs

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Modular Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.mjs"></script>
</head>
```

```
<body>
  <main class="app">
    <h1>Stationery Shop</h1>
    <h2>পণ্য</h2>
    <ul id="products"></ul>

    <h2>কার্ট</h2>
    <ul id="cart-lines"></ul>
    <p id="empty">কার্ট ফাঁকা।</p>

    <form id="coupon-form">
      <label for="coupon">Coupon</label>
      <input id="coupon" autocomplete="off">
      <button type="submit">প্রয়োগ করুন</button>
    </form>

    <div id="totals"></div>
    <p id="message" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 640px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 6px 0;
}

input,
button {
  font: inherit;
```

```
  padding: 4px 8px;
}

.quantity {
  width: 4em;
}

#totals p {
  margin: 4px 0;
}
```

UI Module দুই অংশে; দুটি মিলে সম্পূর্ণ File।

app.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
import { PRODUCTS } from './src/catalog.mjs';
import { formatTaka } from './src/money.mjs';
import { createCartStore } from './src/storage.mjs';
import {
  addItem, applyCoupon, cartLines, cartTotals, CartError, setQuantity,
} from './src/cart.mjs';

const store = createCartStore();
let cart = store.load();

const productList = document.querySelector('#products');
const cartList = document.querySelector('#cart-lines');
const emptyView = document.querySelector('#empty');
const totalsView = document.querySelector('#totals');
const message = document.querySelector('#message');
const couponForm = document.querySelector('#coupon-form');

// প্রত্যাশিত ত্রুটি (CartError) বার্তা হয়; অন্য সব ত্রুটি ওপরে যায়।
function change(makeNextCart, successText) {
  try {
    cart = makeNextCart();
  } catch (error) {
    if (!(error instanceof CartError)) {
      throw error;
    }
    message.textContent = error.message;
    renderCart();
    return;
  }
  store.save(cart);
  message.textContent = successText;
  renderCart();
}
```

```
function textRow(tag, text) {
  const element = document.createElement(tag);
  element.textContent = text;
  return element;
}

// ...
```

app.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

function productRow(product) {
  const button = textRow('button', 'কার্টে যোগ');
  button.type = 'button';
  button.dataset.add = product.id;

  const row = textRow('li', `${product.name}: ${formatTaka(product.price)}`);
  row.append(button);
  return row;
}

function cartRow(line) {
  const quantity = document.createElement('input');
  quantity.type = 'number';
  quantity.className = 'quantity';
  quantity.value = line.quantity;
  quantity.dataset.id = line.id;
  quantity.setAttribute('aria-label', `${line.name}-এর পরিমাণ`);

  const row = textRow('li', line.name);
  row.append(quantity, ` = ${formatTaka(line.lineTotal)}`);
  return row;
}

function renderCart() {
  const lines = cartLines(cart);
  const { subtotal, discount, delivery, total } = cartTotals(cart);

  cartList.replaceChildren(...lines.map(cartRow));
  emptyView.hidden = lines.length > 0;
  totalsView.replaceChildren(
    textRow('p', `মোট দাম: ${formatTaka(subtotal)}`),
    textRow('p', `ছাড় (${cart.coupon ?? 'নেই'}): ${formatTaka(discount)}`),
    textRow('p', `Delivery: ${formatTaka(delivery)}`),
    textRow('p', `পরিশোধ: ${formatTaka(total)}`),
  );
}
```

```
productList.addEventListener('click', (event) => {
  const id = event.target.closest('[data-add]')?.dataset.add;
  if (id) {
    change(() => addItem(cart, id), 'কার্টে যোগ হয়েছে।');
  }
});

cartList.addEventListener('change', (event) => {
  const { id } = event.target.dataset;
  change(() => setQuantity(cart, id, Number(event.target.value)), 'পরিমাণ বদলেছে।');
});

couponForm.addEventListener('submit', (event) => {
  event.preventDefault();
  const code = couponForm.elements.coupon.value;
  change(() => applyCoupon(cart, code), 'Coupon প্রয়োগ হয়েছে।');
});

productList.replaceChildren(...PRODUCTS.map(productRow));
renderCart();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Import-এর তালিকা একটি মানচিত্র।** `app.mjs`-এর প্রথম ছয় লাইন পড়লেই বোঝা যায় এই File কোন কোন Module-এর ওপর নির্ভর করে। লম্বা Import তালিকা একাধিক লাইনে ভাঙা যায়।
- **নির্ভরতার দিক।** `app.mjs` সবাইকে Import করে; `storage.mjs` শুধু Domain-কে; `cart.mjs` শুধু `catalog.mjs`-কে; `catalog.mjs` কাউকে নয়। কোনো Domain File DOM বা Storage জানে না।
- **`change` একটি ছক:** নতুন কার্ট তৈরির চেষ্টা; `CartError` হলে বার্তা এবং কার্ট অপরিবর্তিত (তাই পরিমাণের ঘরে ভুল মান লিখলে আগের মান ফিরে আসে); সফল হলে সংরক্ষণ, বার্তা, আঁকা। নতুন কার্ট তৈরির কাজটি একটি Function হিসেবে পাঠানো হয়েছে, যাতে `try` তার ত্রুটি ধরতে পারে।
- **Event Delegation** (অধ্যায় ৯): পণ্যের তালিকায় একটি `click`, কার্টের তালিকায় একটি `change`।
- **Module Script নিজে থেকেই `defer`,** তাই `<head>`-এ থাকা সত্ত্বেও সব Element পাওয়া যায়।

চালানো এবং প্রত্যাশিত আচরণ

Project Folder থেকে:

Terminal

```
python -m http.server 8000
```

Browser-এ `http://localhost:8000` খুলুন। Test এবং Node.js-এর প্রদর্শনী:

Terminal

```
node demo.mjs
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
Notebook দুবার যোগ	পরিমাণ 2; মোট 240, Delivery 60, পরিশোধ 300
Backpack যোগ	মোট 2,090; Delivery 0
Coupon `student10`	ছাড় 209; ছাড়ের পর 1,881, তাই Delivery 60; পরিশোধ 1,941
Coupon `FREE100`	"Coupon "FREE100" সঠিক নয়।"; কার্ট অপরিবর্তিত
পরিমাণের ঘরে 11	সীমার বার্তা; ঘরে আগের মান ফিরে আসে
পরিমাণের ঘরে 0	সারিটি সরে যায়
Page Reload	কার্ট ও Coupon ফিরে আসে

Companion Source Code-এ ২৪টি Node.js Test আছে (কার্টের নিয়ম ও Storage Adapter), এবং `browser-check.mjs` একটি Local Server চালিয়ে আসল Chrome-এ ওপরের সব আচরণ যাচাই করে, সঙ্গে প্রমাণ করে যে `file://` থেকে খুললে Module লোড হয় না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
পণ্যের তালিকা ফাঁকা, Console-এ CORS	`file://` থেকে খোলা	Local Server
`does not provide an export named`	Import ও Export-এর নাম মেলে না	নাম মিলিয়ে নিন
`Failed to load module script` বা `404`	Import-এর ঠিকানা ভুল	`./src/...` এবং `.mjs`
Reload-এ কার্ট হারায়	`store.save` ডাকা হয়নি	সব পরিবর্তন `change` দিয়ে
ভুল পরিমাণের পর Page থেমে যায়	`CartError` ধরা হয়নি	`change`-এর `try`
Delivery-র হিসাব অপ্রত্যাশিত	ছাড়ের আগের মূল্য ধরে হিসাব	`afterDiscount`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। দাম ও Coupon Browser-এর Code-এ আছে, তাই যেকোনো ব্যবহারকারী তা দেখতে ও বদলাতে পারেন; আসল দোকানে দাম, ছাড় ও মোট হিসাব Server আবার যাচাই করে। কোনো Checkout বা লেনদেন নেই।

১২.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `[5, 1, 4]` থেকে `toSorted` দিয়ে সাজানো Copy তৈরি করুন, এবং দেখান মূল Array অপরিবর্তিত।

২. একটি Settings Object-এ `??=` দিয়ে `pageSize`-এর Default মান ২০ বসান, কিন্তু আগে থেকে `0` থাকলে তা যেন না বদলায়।

৩. `greet.mjs` থেকে Named Export `greet` এবং `main.mjs`-এ Import করে চালান।

৪. `import greet from './greet.mjs'` কেন ব্যর্থ হয়, যখন `greet.mjs`-এ শুধু `export function greet` আছে? ঠিক করুন।

৫. নিচের Function-এর নাম ও Variable-এর নাম পরিষ্কার করুন: `function f(a) { return a * 0.15; }`।

মধ্যম স্তর

৬. `structuredClone` দিয়ে একটি Nested Order-এর Copy করে Copy বদলান, এবং দেখান মূলটি অপরিবর্তিত।

৭. `Object.groupBy` দিয়ে শিক্ষার্থীদের পাস ও ফেলে ভাগ করুন (পাস নম্বর ৩৩)।

৮. একটি Module-এ `let visits = 0` এবং `recordVisit()` Export করুন; অন্য Module থেকে তিনবার ডেকে Live Binding দেখান।

৯. তিন স্তরের Nested `if` থাকা একটি Function-কে Guard Clause দিয়ে লিখুন।

চ্যালেঞ্জ স্তর

১০. Project-এর `cart.mjs`-এ একটি নতুন নিয়ম যোগ করুন: একই Order-এ ৫টির বেশি Notebook হলে প্রতিটি Notebook-এ ১০ টাকা ছাড়। শুধু Domain বদলান, এবং Node.js-এ পরীক্ষা করুন।

১১. Dynamic `import()` দিয়ে একটি "Report" Module শুধু তখনই আনুন, যখন ব্যবহারকারী চান।

১২. Project-এর `createCartStore`-এর জন্য একটি নকল Storage লিখুন, যার `setItem` সব সময় ত্রুটি ছোড়ে (জায়গা শেষ), এবং দেখান `save` তখন `false` ফেরত দেয়।

১২.১৫ অনুশীলনীর সমাধান

সমাধান ১

to-sorted.mjs · JavaScript — চলবে: Node.js

```
const numbers = [5, 1, 4];
const sorted = numbers.toSorted((a, b) => a - b);

console.log(sorted, numbers);
```

Output

```
[ 1, 4, 5 ] [ 5, 1, 4 ]
```

সমাধান ২

nullish-assign.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const fresh = {};
const custom = { pageSize: 0 };

fresh.pageSize ??= 20;
custom.pageSize ??= 20;

console.log(fresh.pageSize, custom.pageSize);
```

Output

```
20 0
```

`||=` লিখলে `custom`-এর `0`-ও ২০ হয়ে যেত।

সমাধান ৩

greet.mjs · JavaScript — চলবে: Node.js

```
export function greet(name) {
  return `আসসালামু আলাইকুম, ${name}`;
}
```

main.mjs · JavaScript — চলবে: Node.js

```
import { greet } from './greet.mjs';

console.log(greet('Rafi'));
```

Output

```
আসসালামু আলাইকুম, Rafi
```

সমাধান ৪ — `import greet from` একটি **Default** Export চায়, কিন্তু File-এ আছে Named Export। Node.js বলে `does not provide an export named 'default'`। দুটি সমাধান: Import-এ বন্ধনী দিন (`import { greet } from './greet.mjs'`), অথবা File-এ `export default function greet`। Named Export-ই ভালো (অনুচ্ছেদ ১২.৪-এর পরামর্শ)।

সমাধান ৫

vat-name.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const VAT_RATE = 0.15;

function vatFor(price) {
  return price * VAT_RATE;
}

console.log(vatFor(1000));
```

Output

```
150
```

সমাধান ৬

clone.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const order = { id: 1, customer: { name: 'Mina', city: 'Sylhet' } };
const copy = structuredClone(order);

copy.customer.city = 'Dhaka';

console.log(order.customer.city, copy.customer.city);
```

Output

```
Sylhet Dhaka
```

সমাধান ৭

group-pass.mjs · JavaScript — চলবে: Node.js

```js
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 28 },
  { name: 'Mina', marks: 61 },
];
const PASS_MARK = 33;

const groups = Object.groupBy(students, ({ marks }) =>
  (marks >= PASS_MARK ? 'pass' : 'fail'));

console.log(groups.pass.map((student) => student.name));
console.log(groups.fail.map((student) => student.name));
```

Output

```
[ 'Sara', 'Mina' ]
[ 'Rafi' ]
```

সমাধান ৮

visits.mjs · JavaScript — চলবে: Node.js

```js
export let visits = 0;

export function recordVisit() {
  visits += 1;
}
```

main.mjs · JavaScript — চলবে: Node.js

```js
import { visits, recordVisit } from './visits.mjs';

console.log(visits);
recordVisit();
recordVisit();
recordVisit();
console.log(visits);
```

Output

```
0
3
```

`visits` Import-এর সময়ের মান নয়, মূল Variable-এর বর্তমান মান দেখায়।

সমাধান ৯

guard.mjs · JavaScript চলবে: Browser Console ও Node.js

```
function shippingLabel(order) {
  if (!order) {
    return 'Order নেই';
  }
  if (!order.paid) {
    return 'পরিশোধের অপেক্ষায়';
  }
  if (!order.address) {
    return 'ঠিকানা নেই';
  }
  return `পাঠানো হবে: ${order.address}`;
}

console.log(shippingLabel({ paid: true, address: 'Bogura' }));
console.log(shippingLabel({ paid: false }));
```

Output

```
পাঠানো হবে: Bogura
পরিশোধের অপেক্ষায়
```

প্রতিটি অস্বাভাবিক ক্ষেত্র আগেভাগে ফেরত যায়; শেষ লাইনে থাকে সফল পথ, কোনো Nesting ছাড়া।

সমাধান ১০ — Domain-এ একটি নতুন Function, এবং `cartTotals`-এর হিসাবে তার ব্যবহার। পরীক্ষার জন্য নিয়মটি আলাদা Module-এ লেখা হলো, যাতে মূল Project অপরিবর্তিত থাকে:

bulk-rule.mjs · JavaScript চলবে: Node.js

```
import {
  cartLines, addItem, emptyCart,
} from '../../project/modular-shopping-cart/src/cart.mjs';

const BULK_ITEM = 'notebook';
const BULK_MIN = 6;
const BULK_OFF_EACH = 10;

export function bulkDiscount(cart) {
  const line = cartLines(cart).find((item) => item.id === BULK_ITEM);
  return line && line.quantity >= BULK_MIN ? line.quantity * BULK_OFF_EACH : 0;
}

console.log(bulkDiscount(addItem(emptyCart(), 'notebook', 5)));
console.log(bulkDiscount(addItem(emptyCart(), 'notebook', 7)));
```

Output

```
0
70
```

"৫টির বেশি" মানে কমপক্ষে ৬টি; তাই Constant-এর নাম `BULK_MIN` এবং মান `6`। `cartTotals`-এ `discount`-এর সঙ্গে এটি যোগ করলেই নিয়মটি পুরো Application-এ কার্যকর, UI-এর একটি লাইনও না বদলে।

সমাধান ১১

report.mjs · JavaScript চলবে: Node.js

```
console.log('report.mjs লোড হলো');

export const buildReport = (sales) => `মোট বিক্রি: ${sales.reduce((a, b) => a + b,
    0)}`;
```

main.mjs · JavaScript — চলবে: Node.js

```
async function showReport(wanted) {
  if (!wanted) {
    return 'Report চাওয়া হয়নি';
  }
  const { buildReport } = await import('./report.mjs');
  return buildReport([1200, 800, 450]);
}

console.log(await showReport(false));
console.log(await showReport(true));
```

Output

```
Report চাওয়া হয়নি
report.mjs লোড হলো
মোট বিক্রি: 2450
```

প্রথম ডাকে Module লোডই হয়নি; শুধু প্রয়োজনের মুহূর্তে এসেছে।

সমাধান ১২

full-storage.mjs · JavaScript — চলবে: Node.js

```
const PROJECT = '../../project/modular-shopping-cart/src';
const { createCartStore } = await import(`${PROJECT}/storage.mjs`);
const { addItem, emptyCart } = await import(`${PROJECT}/cart.mjs`);

const fullStorage = {
  getItem: () => null,
  setItem: () => {
    throw new Error('QuotaExceededError');
  },
};

const store = createCartStore(fullStorage);

console.log(store.load().items.length);
console.log(store.save(addItem(emptyCart(), 'pen-set')));
```

Output

```
0
false
```

নকল Storage মাত্র দুটি Function; Browser ছাড়াই "জায়গা শেষ" পরিস্থিতি পরীক্ষা করা গেল। এটিই Dependency Injection-এর সুবিধা।

১২.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `[3, 1, 2].toSorted()`-এর পর মূল Array কী?

ক) `[1, 2, 3]`

খ) `[3, 1, 2]`

গ) `[]`

ঘ) `undefined`

প্রশ্ন ২. `x = 0; x ??= 5;`-এর পর `x` কত?

ক) `5`

খ) `null`

গ) `0`

ঘ) `undefined`

প্রশ্ন ৩. একটি File-এ সর্বোচ্চ কয়টি Default Export থাকতে পারে?

ক) একটি

খ) দুটি

গ) যতগুলো খুশি

ঘ) একটিও না

প্রশ্ন ৪. একটি Module তিনটি File থেকে Import হলে সেটি কতবার চলে?

ক) তিনবার

খ) একবার

গ) প্রতিটি Function ডাকার সময়

ঘ) কখনো না

প্রশ্ন ৫. Module-এর সবচেয়ে বাইরের স্তরে `this`-এর মান কী?

ক) `window`

খ) Module Object

গ) `globalThis`

ঘ) `undefined`

প্রশ্ন ৬. Browser-এ `file://` থেকে খোলা Page-এর Module লোড না হওয়ার কারণ?

ক) Browser Module সমর্থন করে না

খ) নিরাপত্তার নিয়ম (CORS); Local Server লাগে

গ) `.mjs` Extension নিষিদ্ধ

ঘ) `defer` দরকার

প্রশ্ন ৭. Node.js-এর ES Module-এ `__dirname`-এর বদলে কী?

ক) `process.cwd()`

খ) `require.main`

গ) `import.meta.dirname`

ঘ) `module.dirname`

প্রশ্ন ৮. ভুল নামে Named Import করলে ত্রুটি কখন ধরা পড়ে?

ক) Program-এর কোনো লাইন চলার আগে

খ) সেই নাম ব্যবহারের সময়

গ) কখনো না; `undefined` আসে

ঘ) Program শেষে

প্রশ্ন ৯. Clean Code-এর নিয়মে Comment কী ব্যাখ্যা করা উচিত?

ক) প্রতিটি লাইন কী করে

খ) Variable-এর ধরন

গ) Code কেন এভাবে লেখা, যা Code থেকে বোঝা যায় না

ঘ) কিছুই না; Comment নিষিদ্ধ

প্রশ্ন ১০. Domain Module-এ `localStorage` ব্যবহার না করার প্রধান কারণ?

ক) `localStorage` ধীর

খ) Domain Browser ছাড়া Test ও পুনর্ব্যবহারযোগ্য থাকে

গ) Module-এ `localStorage` নিষিদ্ধ

ঘ) Storage-এ সংখ্যা রাখা যায় না

১২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–খ · ৫–ঘ · ৬–খ · ৭–গ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	`toSorted` নতুন Array ফেরত দেয়; মূলটি বদলায় না।
২	গ	`??=` শুধু `null` বা `undefined` -এ বসায়; `0` থেকে যায়।
৩	ক	Default Export File-এর একমাত্র প্রধান জিনিস।
৪	খ	প্রথম Import-এ চলে; পরের সবাই একই Module পায়।
৫	ঘ	Module স্বয়ংক্রিয়ভাবে Strict Mode-এ চলে।
৬	খ	Browser `file://` থেকে Module আনতে দেয় না; `http://` লাগে।
৭	গ	ES Module-এ `import.meta.dirname` ও `import.meta.filename`।
৮	ক	`import` স্থির; চালানোর আগে সব Import ও Export মেলানো হয়।
৯	গ	কী করছে তা নাম বলে; Comment বলে কেন।
১০	খ	Pure Domain Node.js-এ Test করা যায় এবং অন্য পরিবেশে চলে।

১২.১৮ অধ্যায়ের সারসংক্ষেপ

- Modern JavaScript-এর বেশির ভাগ সুবিধা আপনি আগের অধ্যায়গুলোতে শিখেছেন; নতুন: `toSorted`, `toReversed`, `with`, `||=`, `??=`, `structuredClone`, `Object.groupBy`।
- সাধারণ Script-এর Global Scope-এ নাম নীরবে সংঘর্ষে জড়ায়; Module প্রতিটি File-কে নিজস্ব Scope দেয়।
- `export` দেয়, `import` নেয়; Named Export পছন্দনীয়; Default একটিই।
- Module একবার চলে, Live Binding দেয়, স্বয়ংক্রিয় Strict Mode-এ চলে, Top-level `await` ও Dynamic `import()` সমর্থন করে।
- Import-এর ত্রুটি Program চলার আগেই ধরা পড়ে।
- Browser: `type="module"`, নিজে থেকেই `defer`, ঠিকানায় `./` ও Extension, এবং একটি Server।
- Node.js: `.mjs` বা `"type": "module"`; `require` ও `__dirname` -এর বদলে `import` ও `import.meta.dirname`।
- Clean Code: অর্থবহ নাম, নামযুক্ত Constant, Guard Clause, ছোট Function; Comment বলে কেন।
- Module সাজান স্তরে: Pure Domain, Adapter, UI; নির্ভরতা একমুখী; বাইরের জিনিস Inject করুন।

পরের অধ্যায়ে Object-Oriented Programming: Class, Prototype, Inheritance এবং Encapsulation, যা দিয়ে তথ্য ও তার আচরণ একসঙ্গে রাখা যায়।

১২.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
ES2015 (ES6)	JavaScript-এর বড় পরিবর্তনের সংস্করণ; তারপর প্রতি বছর একটি।
ES Module	`import` ও `export` ব্যবহারকারী নিজস্ব Scope-এর File।
Named Export	নির্দিষ্ট নামে Export।
Default Export	File-এর প্রধান Export; একটিই।
Namespace Import	`import * as` দিয়ে সব Export একটি Object-এ।
Live Binding	Import করা নাম মূল Variable-এর বর্তমান মান দেখায়।
Dynamic Import	`import()` দিয়ে চলার সময় Module আনা।
Top-level `await`	Module-এর বাইরের স্তরে `await`।
CommonJS	Node.js-এর পুরোনো Module-ব্যবস্থা (`require`)।
Refactoring	আচরণ না বদলে Code-এর গঠন উন্নত করা।
Guard Clause	শুরুতেই অস্বাভাবিক ক্ষেত্র থেকে ফেরত।
Dependency Injection	বাইরের নির্ভরতা Parameter হিসেবে দেওয়া।

১২.২০ আরও পড়ুন

- **MDN — JavaScript modules:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules` — Browser-এ Module-এর পূর্ণ নির্দেশিকা।
- **MDN — import:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import` — Import-এর সব রূপ।
- **Node.js — ECMAScript modules:** `https://nodejs.org/api/esm.html` — Node.js-এ ES Module, `import.meta`।
- **MDN — structuredClone():** `https://developer.mozilla.org/en-US/docs/Web/API/Window/structuredClone` — Deep Copy।
- **MDN — Object.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy` — দলে ভাগ করা।

অধ্যায় ১৩

১৩

Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Object-Oriented Programming-এর মূল ধারণা—তথ্য ও আচরণ একসঙ্গে—ব্যাখ্যা করতে।
- `class`, `constructor`, Field ও Method দিয়ে Object-এর ছাঁচ তৈরি করতে।
- Prototype Chain কীভাবে কাজ করে, এবং Class যে তারই ওপর একটি লেখার ধরন, তা বুঝতে।
- Private Field (`#`) দিয়ে Object-এর ভেতরের অবস্থা সুরক্ষিত রাখতে।
- Getter, Setter এবং Static সদস্য ব্যবহার করতে।
- `extends` ও `super` দিয়ে Inheritance এবং Method Override করতে।
- Polymorphism দিয়ে বিভিন্ন ধরনের Object একই ভাবে ব্যবহার করতে।
- Method আলাদা করে পাঠালে `this` হারানোর সমস্যা চিনে সমাধান করতে।
- কখন Inheritance আর কখন Composition, তা বিচার করতে।
- একটি সম্পূর্ণ **Booking Management Lab** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭: Object, Method এবং `this`।
- অধ্যায় ১১: `class ... extends Error` দিয়ে নিজের Error (এখানে তার পূর্ণ ব্যাখ্যা)।
- অধ্যায় ১২: ES Module (`import` / `export`)।
- Node.js 22 বা নতুন সংস্করণ।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আমরা তথ্য রেখেছি সাধারণ Object-এ, আর তার ওপর কাজ করেছি আলাদা Function দিয়ে: অধ্যায় ৭-এর শিক্ষার্থী, অধ্যায় ১২-এর কার্ট। এটি অনেক ক্ষেত্রেই সবচেয়ে ভালো পদ্ধতি। কিন্তু কিছু জিনিসের নিজস্ব নিয়ম থাকে, যা তথ্যের সঙ্গে অবিচ্ছেদ্য: একটি Bank Account-এর Balance কখনো ঋণাত্মক হবে না, একটি বাতিল Booking আবার বাতিল করা যাবে না। এমন ক্ষেত্রে তথ্য ও নিয়ম একসঙ্গে, একটি সুরক্ষিত একক হিসেবে রাখা সুবিধাজনক। এই ধারণার নাম **Object-Oriented Programming (OOP)**।

JavaScript-এর নিজের প্রায় সবকিছুই এভাবে তৈরি: প্রতিটি Array তার Method (`map`, `filter`) পায় একই উৎস থেকে, প্রতিটি Error-এর `name` ও `message` থাকে। Browser-এর DOM, Node.js-এর Stream—সবই Class। এই অধ্যায় শেষে আপনি শুধু Class লিখতেই পারবেন না, বুঝবেন ভেতরে কী ঘটে, এবং কখন Class ব্যবহার **না** করাই ভালো।

১৩.১ Factory Function থেকে Class

একটি Account তৈরির দুটি উপায় পাশাপাশি দেখুন:

factory-vs-class.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createAccount(owner, balance) {
  return {
    owner,
    balance,
    deposit(amount) {
      this.balance += amount;
    },
  };
}

class Account {
  constructor(owner, balance) {
    this.owner = owner;
    this.balance = balance;
  }
```

```
  deposit(amount) {
    this.balance += amount;
  }
}

const first = createAccount('Sara', 100);
const second = new Account('Rafi', 100);
first.deposit(50);
second.deposit(50);

console.log(first.balance, second.balance);
console.log(second instanceof Account, first instanceof Account);

const third = createAccount('Mina', 0);
const fourth = new Account('Nila', 0);
console.log(first.deposit === third.deposit, second.deposit === fourth.deposit);
```

Output

```
150 150
true false
false true
```

দুটিই কাজ করে, কিন্তু তিনটি পার্থক্য আছে:

- `new`: Class থেকে Object (Instance) তৈরি হয় `new` দিয়ে। `constructor` প্রতিটি নতুন Object-এর প্রাথমিক মান বসায়, আর ভেতরের `this` সেই নতুন Object।
- **পরিচয়**: `instanceof` বলতে পারে Object-টি কোন Class থেকে তৈরি। Factory-র Object শুধু একটি সাধারণ Object।
- **শেষ লাইনটি সবচেয়ে গুরুত্বপূর্ণ।** Factory প্রতিটি Object-এর জন্য `deposit`-এর একটি নতুন Copy তৈরি করেছে; Class-এর সব Instance একটিই `deposit` ভাগ করে নিয়েছে। কীভাবে, তা অনুচ্ছেদ ১৩.৩-এ।

১৩.২ Class-এর গঠন

class-basics.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Product {
  discount = 0;

  constructor(name, price) {
    this.name = name;
    this.price = price;
  }

  finalPrice() {
```

```
    return this.price * (1 - this.discount / 100);
  }

  label() {
    return `${this.name}: ${this.finalPrice()} টাকা`;
  }
}

const pen = new Product('Pen', 50);
pen.discount = 10;

console.log(pen.label());
console.log(Object.keys(pen).join(', '));
console.log(typeof Product);
```

Output

```
Pen: 45 টাকা
discount, name, price
function
```

অংশ	কী
`discount = 0;`	Public Field: প্রতিটি Instance-এ এই Property, এই প্রাথমিক মান নিয়ে
`constructor(...)`	`new`-এর সময় একবার চলে; Argument থেকে Property বসায়
`finalPrice() { }`	Method: সব Instance ভাগ করে নেয়; ভেতরে `this` মানে যে Instance-এর ওপর ডাকা হয়েছে

- Field-গুলো `constructor`-এর আগে বসে, তাই `Object.keys`-এ `discount` প্রথমে।
- একটি Method আরেকটিকে `this.finalPrice()` দিয়ে ডাকে।
- Class-এর নামের প্রথম অক্ষর বড় হাতের (`Product`), যাতে সাধারণ Function থেকে আলাদা করে চেনা যায়।
- `typeof Product` হলো `function`। Class আসলে একটি বিশেষ Function, তবে `new` ছাড়া ডাকলে `TypeError: Class constructor Product cannot be invoked without 'new'`।
- Class-এর ভেতরের সব Code নিজে থেকেই Strict Mode-এ চলে।

১৩.৩ Prototype Chain: Class-এর ভেতরে কী ঘটে

প্রতিটি JavaScript Object-এর একটি গোপন সংযোগ থাকে আরেকটি Object-এর সঙ্গে, যাকে বলে তার **Prototype**। কোনো Property Object-এ না পেলে JavaScript তার Prototype-এ খোঁজে, সেখানে না পেলে Prototype-এর Prototype-এ—এভাবে একটি শিকল ধরে, যতক্ষণ না `null`-এ পৌঁছায়। এটিই **Prototype Chain**।

prototype.js · JavaScript চলবে: Browser Console ও Node.js

```
class Product {
  constructor(name) {
    this.name = name;
  }

  label() {
    return `পণ্য: ${this.name}`;
  }
}

const pen = new Product('Pen');

console.log(Object.getPrototypeOf(pen) === Product.prototype);
console.log(Object.hasOwn(pen, 'name'), Object.hasOwn(pen, 'label'));
console.log('label' in pen);
console.log(Object.getPrototypeOf(Product.prototype) === Object.prototype);
console.log(typeof pen.toString);

Product.prototype.shout = function () {
  return this.label().toUpperCase();
};
console.log(pen.shout());
```

Output

```
true
true false
true
true
function
পণ্য: PEN
```

`new Product('Pen')` একটি Object তৈরি করেছে, যার নিজের Property শুধু `name`; তার Prototype হলো `Product.prototype`, যেখানে Class-এর সব Method থাকে। তাই `pen.label()` ডাকলে JavaScript প্রথমে `pen`-এ খোঁজে, না পেয়ে `Product.prototype`-এ পায়। শিকলটি:

ধাপ	Object	এখানে যা আছে
১	`pen`	`name` (নিজের)
২	`Product.prototype`	`constructor`, `label` (পরে `shout`)
৩	`Object.prototype`	`toString`, `hasOwnProperty` ইত্যাদি
৪	`null`	শিকলের শেষ

এ থেকে তিনটি জিনিস ব্যাখ্যা হয়: সব Instance একটিই Method ভাগ করে নেয় (অনুচ্ছেদ ১৩.১-এর শেষ লাইন); Prototype-এ পরে যোগ করা `shout` পুরোনো `pen` -ও পেয়েছে; আর অধ্যায় ১১-এর `catalog['toString']` সমস্যার উৎস এই শিকলের তৃতীয় ধাপ। `class` মূলত এই ব্যবস্থার ওপর একটি পরিষ্কার লেখার ধরন।

Class-এর আগের যুগ: Constructor Function

২০১৫-র আগে `class` ছিল না, এবং পুরোনো Library ও Project-এ আজও এই ধরন দেখবেন। ব্যবস্থাটি একই, শুধু হাতে লেখা:

constructor-function.js · JavaScript — চলবে: Browser Console ও Node.js

```
function Product(name) {
  this.name = name;
}

Product.prototype.label = function () {
  return `পণ্য: ${this.name}`;
};

const pen = new Product('Pen');
console.log(pen.label(), pen instanceof Product);

const base = { greet() { return `হ্যালো, ${this.name}`; } };
const user = Object.create(base);
user.name = 'Sara';
console.log(user.greet(), Object.getPrototypeOf(user) === base);
```

Output

```
পণ্য: Pen true
হ্যালো, Sara true
```

একটি সাধারণ Function-কে `new` দিয়ে ডাকলে সেটি Constructor-এর কাজ করে, আর Method হাতে বসানো হয় `Product.prototype` -এ। `Object.create(base)` একটি নতুন Object তৈরি করে যার Prototype সরাসরি `base` , কোনো Class বা Constructor ছাড়াই। পুরোনো Code পড়ার সময় এগুলো চিনতে পারলেই যথেষ্ট; নতুন Code-এ `class` লিখুন, যা একই কাজ নিরাপদে করে (যেমন `new` ভুলে গেলে Error দেয়)।

> **সতর্কতা:** উদাহরণের মতো পরে Prototype-এ Method যোগ করা শুধু বোঝার জন্য। নিজের Code-এ Method Class-এর ভেতরেই লিখুন, আর `Array.prototype` -এর মতো Built-in Prototype কখনো বদলাবেন না: অন্য Code ও ভবিষ্যতের JavaScript সংস্করণের সঙ্গে সংঘর্ষ হয়।

১৩.8 Encapsulation: Private Field

`account.balance = 1000000` লিখে যে-কেউ একটি সাধারণ Property বদলে দিতে পারে, নিয়ম না মেনেই। **Encapsulation** মানে Object-এর ভেতরের অবস্থা শুধু তার নিজের Method দিয়ে বদলানো যাবে। নামের আগে `#` দিলে Field বা Method **Private** হয়: Class-এর বাইরে থেকে তা পড়া বা লেখা যায় না।

private.js · JavaScript — চলবে: Browser Console ও Node.js

```
class BankAccount {
  #balance = 0;

  deposit(amount) {
    this.#validate(amount);
    this.#balance += amount;
  }

  withdraw(amount) {
    this.#validate(amount);
    if (amount > this.#balance) {
      throw new RangeError('যথেষ্ট টাকা নেই');
    }
    this.#balance -= amount;
  }

  get balance() {
    return this.#balance;
  }

  #validate(amount) {
    if (!Number.isFinite(amount) || amount <= 0) {
      throw new RangeError('পরিমাণ ধনাত্মক হতে হবে');
    }
  }
}

const account = new BankAccount();
account.deposit(500);
account.withdraw(200);
console.log(account.balance, Object.keys(account).length);

try {
  account.withdraw(1000);
} catch (error) {
  console.log(error.message);
}
console.log(account.balance);
```

Output

```
300 0
যথেষ্ট টাকা নেই
300
```

- `#balance` শুধু Class-এর ভেতর থেকে পাওয়া যায়; `Object.keys`-এও দেখা যায় না (সংখ্যা `0`)। Balance বদলানোর একমাত্র পথ `deposit` ও `withdraw`, এবং দুটিই নিয়ম যাচাই করে।
- `#validate` একটি Private Method: Class-এর ভেতরের সহায়ক, বাইরের Code-এর জন্য নয়।
- `get balance()` বাইরের Code-কে মান পড়তে দেয়, লিখতে নয় (পরের অনুচ্ছেদ)।
- **ব্যর্থ `withdraw` অবস্থা বদলায়নি।** যাচাই আগে, বদল পরে: Object কখনো অর্ধেক-বদলানো অবস্থায় থাকে না।

বাইরে থেকে Private Field ছোঁয়ার চেষ্টা Program চলার আগেই ধরা পড়ে:

private-outside.js · JavaScript — চলবে: Node.js

```
class Safe {
  #code = 1234;
}

const safe = new Safe();
console.log(safe.#code);
```

Error Message

```
SyntaxError: Private field '#code' must be declared in an enclosing class
```

পুরোনো Code-এ Private বোঝাতে নামের আগে `_` দেওয়ার রীতি দেখবেন (`this._balance`)। সেটি শুধু একটি অনুরোধ; JavaScript তা মানতে বাধ্য করে না। `#` সত্যিকারের সুরক্ষা দেয়।

১৩.৫ Getter, Setter এবং Static

getter-static.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Temperature {
  static ABSOLUTE_ZERO = -273.15;

  #celsius = 0;

  constructor(celsius) {
    this.celsius = celsius;
  }

  get celsius() {
    return this.#celsius;
```

```
  }

  set celsius(value) {
    if (value < Temperature.ABSOLUTE_ZERO) {
      throw new RangeError(`${value} ডিগ্রি সম্ভব নয়`);
    }
    this.#celsius = value;
  }

  get fahrenheit() {
    return (this.#celsius * 9) / 5 + 32;
  }

  static fromFahrenheit(fahrenheit) {
    return new Temperature(((fahrenheit - 32) * 5) / 9);
  }
}

const today = new Temperature(30);
console.log(today.fahrenheit);
today.celsius = 35;
console.log(today.celsius, today.fahrenheit);

const boiling = Temperature.fromFahrenheit(212);
console.log(boiling.celsius);

try {
  today.celsius = -300;
} catch (error) {
  console.log(error.message);
}
console.log(typeof today.fromFahrenheit);
```

Output

```
86
35 95
100
-300 ডিগ্রি সম্ভব নয়
undefined
```

- **Getter (`get fahrenheit()`)** Property-র মতো পড়া যায় (Parenthesis ছাড়া), কিন্তু প্রতিবার হিসাব করে মান দেয়। তাই `celsius` বদলালে `fahrenheit` নিজে থেকেই ঠিক থাকে; দুটি আলাদা করে রাখলে একসময় অমিল হতো।
- **Setter (`set celsius(value)`)** সাধারণ Assignment-এর মতো লেখা যায়, কিন্তু ভেতরে যাচাই করে। `constructor`-ও `this.celsius = ...` লিখে Setter ব্যবহার করেছে, তাই তৈরির সময়ও একই নিয়ম।

- **Static** সদস্য Class-এর নিজের, Instance-এর নয়। `Temperature.ABSOLUTE_ZERO` একটি Class-স্তরের Constant; `Temperature.fromFahrenheit` একটি **Static Factory**, যা ভিন্ন ধরনের ইনপুট থেকে Instance তৈরি করে। Instance থেকে এগুলো পাওয়া যায় না (`undefined`)। `Array.isArray`, `Object.keys`, `Number.isInteger`—আপনার পরিচিত এগুলোও Static Method।

১৩.৬ Inheritance: extends এবং super

কিছু Class একই মূল ধারণার বিশেষ রূপ: রিকশা ও বাস দুটিই যানবাহন। **Inheritance** দিয়ে একটি Class আরেকটির সব বৈশিষ্ট্য নিয়ে নিজের পার্থক্যটুকু যোগ বা বদল করে।

inheritance.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Vehicle {
  constructor(name, wheels) {
    this.name = name;
    this.wheels = wheels;
  }

  describe() {
    return `${this.name}: ${this.wheels} চাকা`;
  }

  fare(km) {
    return km * 10;
  }
}

class Rickshaw extends Vehicle {
  constructor(name) {
    super(name, 3);
  }

  fare(km) {
    return Math.max(30, super.fare(km) * 2);
  }
}

class Bus extends Vehicle {
  constructor(name, seats) {
    super(name, 6);
    this.seats = seats;
  }

  describe() {
    return `${super.describe()}, ${this.seats} আসন`;
  }
}
```

```
const rickshaw = new Rickshaw('রিকশা');
const bus = new Bus('বাস', 40);

console.log(rickshaw.describe());
console.log(rickshaw.fare(1), rickshaw.fare(5));
console.log(bus.describe(), bus.fare(5));
console.log(bus instanceof Bus, bus instanceof Vehicle, bus instanceof Rickshaw);
```

Output

```
রিকশা: 3 চাকা
30 100
বাস: 6 চাকা, 40 আসন 50
true true false
```

- `class Rickshaw extends Vehicle` : `Rickshaw.prototype` -এর Prototype হলো `Vehicle.prototype` ; Prototype Chain-এ একটি ধাপ বাড়ল। তাই `rickshaw.describe()` নিজের Class-এ না পেয়ে `Vehicle` -এ পেয়েছে।
- `super(...)` Parent Class-এর `constructor` ডাকে। Child-এর `constructor` -এ `this` ব্যবহারের **আগে** এটি ডাকতে হয়।
- **Override**: Child একই নামের Method লিখলে সেটি Parent-এরটিকে ঢেকে দেয় (`Rickshaw` -এর `fare`)।
- `super.method()` Override করা Method-এর ভেতর থেকে Parent-এর সংস্করণ ডাকে, যাতে নিয়মটি নতুন করে লিখতে না হয়।
- `bus` একই সঙ্গে `Bus` এবং `Vehicle` , কিন্তু `Rickshaw` নয়।

`super()` -এর আগে `this` ব্যবহারের ফল:

this-before-super.js · JavaScript চলবে: Node.js

```
class Vehicle {
  constructor(name) {
    this.name = name;
  }
}

class Truck extends Vehicle {
  constructor(name, load) {
    this.load = load;
    super(name);
  }
}

new Truck('Truck', 5);
```

Error Message

```
ReferenceError: Must call super constructor in derived class
before accessing 'this'
```

১৩.৭ Polymorphism: একই প্রশ্ন, ভিন্ন উত্তর

Polymorphism মানে বিভিন্ন Class-এর Object একই Method-এর নামে সাড়া দেয়, প্রত্যেকে নিজের মতো করে। ব্যবহারকারী Code-কে জানতে হয় না কোনটি কোন Class।

polymorphism.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
class Payment {
  constructor(amount) {
    this.amount = amount;
  }

  fee() {
    return 0;
  }

  total() {
    return this.amount + this.fee();
  }

  label() {
    return `${this.constructor.name}: ${this.total()} টাকা`;
  }
}

class CardPayment extends Payment {
  fee() {
    return Math.round(this.amount * 0.02);
  }
}

class MobilePayment extends Payment {
  fee() {
    return Math.min(25, Math.round(this.amount * 0.015));
  }
}

const payments = [
  new Payment(500),
  new CardPayment(1000),
  new MobilePayment(3000),
];
```

```
for (const payment of payments) {
  console.log(payment.label());
}
console.log(`মোট Fee: ${payments.reduce((sum, payment) => sum + payment.fee(),
    0)}`);
```

Output

```
Payment: 500 টাকা
CardPayment: 1020 টাকা
MobilePayment: 3025 টাকা
মোট Fee: 45
```

`label` ও `total` শুধু `Payment`-এ লেখা, কিন্তু ভেতরের `this.fee()` প্রতিটি Object-এর নিজের Class-এর `fee` ডেকেছে। Loop জানে না কোনটি Card, কোনটি Mobile; জানার দরকারও নেই। নতুন একটি পরিশোধের ধরন যোগ করতে শুধু একটি নতুন Class লিখতে হবে, পুরোনো কোনো Code না বদলে। এই নীতির নাম **Open/Closed**: নতুন সংযোজনের জন্য খোলা, পুরোনো Code বদলের জন্য বন্ধ। `if (type === 'card') ... else if (type === 'mobile') ...` শৃঙ্খলের তুলনায় এটি অনেক নিরাপদ, কারণ নতুন ধরন যোগের সময় কোনো `if` ভুলে বাদ পড়ার সুযোগ নেই।

বাধ্যতামূলক Override: Abstract Method-এর ধরন

এখানে `Payment`-এর `fee()` একটি যুক্তিসঙ্গত Default (`0`) দেয়। কিন্তু কখনো Parent-এর পক্ষে কোনো যুক্তিসঙ্গত উত্তর থাকে না: একটি সাধারণ "আকৃতি"-র ক্ষেত্রফল কত? কিছু ভাষায় এমন Method-কে **Abstract** ঘোষণা করা যায়, যা প্রতিটি Child-কে লিখতেই হয়। JavaScript-এ সেই ব্যবস্থা নেই; প্রচলিত বিকল্প হলো Parent-এর Method-এ একটি স্পষ্ট ত্রুটি ছোড়া:

abstract.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Shape {
  area() {
    throw new Error(`${this.constructor.name} area() লেখেনি`);
  }
}

class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }

  area() {
    return this.side ** 2;
  }
}

class Triangle extends Shape {}
```

```
console.log(new Square(3).area());

try {
  new Triangle().area();
} catch (error) {
  console.log(error.message);
}
```

Output

```
9
Triangle area() লেখেনি
```

`Triangle`-এর লেখক `area()` লিখতে ভুলে গেছেন; ভুলটি নীরব `undefined` বা `NaN` না হয়ে প্রথম ব্যবহারেই একটি স্পষ্ট বার্তা দিয়েছে। বার্তায় `this.constructor.name` থাকায় বোঝা যায় কোন Class-এ ভুল।

১৩.৮ this হারানো: Method আলাদা করে পাঠালে

অধ্যায় ৭-এ দেখেছেন, `this` নির্ভর করে Method কীভাবে ডাকা হয়েছে তার ওপর। Class-এ এর একটি প্রচলিত ফাঁদ আছে:

lost-this.js · JavaScript চলবে: Browser Console ও Node.js

```
class Counter {
  count = 0;

  increment() {
    this.count += 1;
    return this.count;
  }

  incrementArrow = () => {
    this.count += 1;
    return this.count;
  };
}

const counter = new Counter();
const loose = counter.increment;

try {
  loose();
} catch (error) {
  console.log(error.name);
}

const bound = counter.increment.bind(counter);
```

```
console.log(bound());

const arrow = counter.incrementArrow;
console.log(arrow());

console.log([1, 2].map(() => counter.increment()).join(', '));
```

Output

```
TypeError
1
2
3, 4
```

`counter.increment`-কে একটি Variable-এ রেখে আলাদাভাবে ডাকলে সেটি আর `counter`-এর সঙ্গে যুক্ত নয়; Class-এর Strict Mode-এ `this` হয় `undefined`, এবং `this.count` পড়তে `TypeError`। একই ঘটনা ঘটে `setTimeout(counter.increment, 100)` বা `button.addEventListener('click', counter.increment)` লিখলে। তিনটি সমাধান:

১. **`bind(counter)`** একটি নতুন Function দেয়, যার `this` স্থায়ীভাবে `counter`।

২. **Arrow Function Field** (`incrementArrow = () => {}`): Arrow Function-এর নিজস্ব `this` নেই, তৈরির জায়গার `this`, অর্থাৎ Instance, ব্যবহার করে। তবে এটি প্রতিটি Instance-এ আলাদা Copy তৈরি করে, Prototype-এ থাকে না।

৩. **ডাকার জায়গায় Arrow Function**: `() => counter.increment()`। সবচেয়ে স্পষ্ট; সাধারণত এটিই ভালো।

১৩.৯ Composition বনাম Inheritance

Inheritance শক্তিশালী, কিন্তু তার একটি বড় দুর্বলতা আছে। ধরুন Order Service-কে গ্রাহককে বার্তা পাঠাতে হবে: Email-এ, SMS-এ, অথবা দুটোতেই। Inheritance দিয়ে লিখলে `EmailOrderService`, `SmsOrderService`, `EmailAndSmsOrderService`... প্রতিটি নতুন মাধ্যমে Class-এর সংখ্যা গুণ হারে বাড়ে। বিকল্প হলো **Composition**: Order Service-এর **একটি** বার্তা-প্রেরক **আছে**, যা বাইরে থেকে দেওয়া হয়।

composition.js · JavaScript চলবে: Browser Console ও Node.js

```
class EmailNotifier {
  send(to, text) {
    return `Email (${to}): ${text}`;
  }
}

class SmsNotifier {
  send(to, text) {
    return `SMS (${to}): ${text}`;
  }
```

```
}

class OrderService {
  #notifier;

  constructor(notifier) {
    this.#notifier = notifier;
  }

  place(order) {
    return this.#notifier.send(order.contact, `Order ${order.id} গৃহীত`);
  }
}

const byEmail = new OrderService(new EmailNotifier());
const bySms = new OrderService(new SmsNotifier());

console.log(byEmail.place({ id: 7, contact: 'sara@example.com' }));
console.log(bySms.place({ id: 8, contact: '01700-000000' }));
```

Output

```
Email (sara@example.com): Order 7 গৃহীত
SMS (01700-000000): Order 8 গৃহীত
```

`OrderService` জানে না বার্তা কীভাবে যায়; সে শুধু জানে তার প্রেরকের একটি `send` আছে। নতুন মাধ্যম (WhatsApp) যোগ করতে একটি নতুন Notifier Class, Order Service অপরিবর্তিত। Test-এ একটি নকল Notifier দেওয়া যায় (অধ্যায় ১২-এর Dependency Injection)। এমনকি `EmailNotifier` ও `SmsNotifier`-এর কোনো সাধারণ Parent Class নেই; শুধু একই আকারের Method থাকাই যথেষ্ট, এবং JavaScript-এ একেই বলে **Duck Typing**: "যদি হাঁসের মতো হাঁটে আর ডাকে, তবে সেটি হাঁস।"

প্রশ্ন	Inheritance (is-a)	Composition (has-a)
সম্পর্ক	রিকশা **একটি** যানবাহন	Order Service-এর **একটি** প্রেরক **আছে**
পরিবর্তনের নমনীয়তা	গঠন তৈরির সময় স্থির	চলার সময় বদলানো যায়
ঝুঁকি	গভীর শিকল; Parent বদলালে সব Child-এ প্রভাব	কম জড়ানো
কখন	স্পষ্ট "একটি বিশেষ ধরন" সম্পর্ক, অগভীর (এক-দুই স্তর)	আচরণ যোগ বা বদলাতে, প্রায় সব ক্ষেত্রে

প্রচলিত পরামর্শ: **Inheritance-এর আগে Composition বিবেচনা করুন।** Project-এ দুটিই আছে, প্রত্যেকটি তার উপযুক্ত জায়গায়।

১৩.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`new` ছাড়া Class ডাকা	`Class constructor ... cannot be invoked without 'new'`	`new Product()`
Child-এ `super()`-এর আগে `this`	`Must call super constructor ...`	প্রথমে `super(...)`
Child-এর `constructor`-এ `super()` নেই	একই `ReferenceError`	`super(...)` ডাকুন
Method-এর ভেতরে `this` ছাড়া Property	`ReferenceError` বা ভুল মান	`this.price`
Method আলাদা করে পাঠানো	`this` `undefined`, `TypeError`	`bind`, Arrow, বা `() => obj.method()`
Private Field বাইরে থেকে	`SyntaxError`	Getter বা Method
Getter-এর নামে Field-ও	অসীম পুনরাবৃত্তি (`Maximum call stack size exceeded`)	ভেতরের মান `#` নামে
Override করে Parent-এর নিয়ম হারানো	যাচাই বাদ পড়ে	`super.method()`
গভীর Inheritance শিকল	একটি বদলে অনেক Class ভাঙে	Composition
সব কিছুর জন্য Class	অকারণ জটিলতা	সাধারণ তথ্যে Object ও Function

১৩.১১ Debugging নির্দেশিকা

- **Console-এ Instance দেখুন।** Chrome ও Node.js Class-এর নাম দেখায় (`BankAccount {owner: 'Sara'}`); Chrome-এ `[[Prototype]]` খুলে Prototype Chain ধরে Method-গুলো দেখা যায়। Private Field Chrome-এর Console-এ দেখা যায়, কিন্তু Code থেকে পড়া যায় না।
- **"is not a function"** পেলে দেখুন Method-টি কোন Class-এ লেখা এবং Object-টি আসলে কোন Class-এর: `obj.constructor.name` এবং `obj instanceof X`।
- **`Cannot read properties of undefined (reading ...)` একটি Method-এর ভেতরে** প্রায় সব সময় হারানো `this`। Stack দেখুন Method-টি কে ডেকেছে: `setTimeout`, Event, নাকি `map`।
- **Override কাজ করছে না?** বানান মিলিয়ে দেখুন; ভুল বানানে নতুন একটি Method তৈরি হয়, Parent-এরটি চলতেই থাকে।
- **অবস্থা অপ্রত্যাশিত?** যে Method-গুলো Private Field বদলায়, শুধু সেগুলোতে Breakpoint দিন; Encapsulation-এর সুবিধা হলো খোঁজার জায়গা সীমিত।

১৩.১২ পেশাদার অভ্যাস

- **Class তখনই, যখন তথ্যের সঙ্গে নিয়ম ও অবস্থা আছে।** শুধু তথ্যের জন্য সাধারণ Object, শুধু হিসাবের জন্য Function।
- **Constructor-এ যাচাই:** কোনো Instance কখনো অবৈধ অবস্থায় তৈরি হবে না।
- **অবস্থা Private (`#`),** বাইরে শুধু প্রয়োজনীয় Getter ও Method।
- **হিসাব করা মান Getter-এ,** আলাদা Property-তে নয়।
- **Inheritance অগভীর;** আচরণ যোগে Composition।
- **Polymorphism দিয়ে `if (type === ...)` শৃঙ্খল এড়ান।**
- **Static Factory** ভিন্ন উৎস থেকে Instance তৈরিতে (`fromJSON`, `fromFahrenheit`)।
- **`toJSON`** দিয়ে ঠিক করুন Object কীভাবে সংরক্ষিত হবে; Private Field নিজে থেকে JSON-এ যায় না।

১৩.১৩ Project: Booking Management Lab

একটি ভ্রমণ সংস্থার Booking ব্যবস্থার মূল অংশ: Flight ও Hotel Booking, বাতিল করা, নিশ্চিত আয়ের হিসাব, এবং সংরক্ষণ ও পুনরুদ্ধার। Project-টি Node.js-এ চলে; এর মূল বিষয় Class-এর নকশা, তাই UI রাখা হয়নি। অধ্যায় ১২-এর মতো এটিও একটি Domain স্তর, যা পরে যেকোনো UI-এর সঙ্গে জোড়া যায়।

Project-এর চাহিদা

- প্রতিটি Booking-এর ID ও গ্রাহকের নাম খালি হবে না; দাম ধনাত্মক সংখ্যা।
- Flight-এর মোট = ভাড়া + Baggage Fee (ঋণাত্মক নয়)।
- Hotel-এর মোট = রাতপ্রতি দাম × রাতের সংখ্যা (১ বা বেশি পূর্ণসংখ্যা)।
- একই ID দুবার নয়; বাতিল Booking আবার বাতিল নয়; বাতিল Booking আয়ে ধরা হবে না।
- Booking-এর অবস্থা বাইরে থেকে সরাসরি বদলানো যাবে না।
- JSON-এ সংরক্ষণ এবং সঠিক Class-এ পুনরুদ্ধার, বাতিল অবস্থাসহ।
- প্রতিটি নতুন ও বাতিল Booking-এর খবর একটি Notifier-কে জানানো হবে (Composition)।

চাহিদা থেকে Class: নকশার ধাপ

Code লেখার আগে চাহিদার তালিকা থেকে নকশা বের করুন। একটি সহজ পদ্ধতি:

১. **বিশেষ্য খুঁজুন।** "Booking", "Flight Booking", "Hotel Booking", "Booking-এর তালিকা"। প্রতিটি সম্ভাব্য Class, কিন্তু সবগুলো নয়: গ্রাহকের নাম শুধু একটি লেখা, তার নিজের কোনো নিয়ম নেই, তাই সেটি Property।

২. **নিয়ম খুঁজুন।** "ID খালি হবে না", "দাম ধনাত্মক"—এগুলো Constructor-এর যাচাই, যাতে কোনো অবৈধ Booking তৈরিই না হয়। "বাতিল Booking আবার বাতিল নয়"—এটি একটি অবস্থার নিয়ম, তাই অবস্থাটি Private এবং বদলায় শুধু `cancel()` দিয়ে।

৩. **কী ধরন অনুযায়ী বদলায়, তা খুঁজুন।** মোট দামের সূত্র Flight ও Hotel-এ আলাদা, বাকি সব এক। এটিই Inheritance ও Polymorphism-এর সংকেত: সাধারণ অংশ Parent-এ, পার্থক্য (`total`, `type`) Child-এ।

৪. **কোনটি "is-a", কোনটি "has-a"।** Flight Booking **একটি** Booking: Inheritance। Manager-এর **একটি** Notifier **আছে**: Composition। Manager নিজে কোনো Booking নয়, তাই সে Booking থেকে Extend করে না; সে Booking-এর একটি তালিকা **রাখে**।

৫. **সীমানা খুঁজুন।** JSON-এ রূপান্তর একটি সীমানা (অধ্যায় ১১-এর Trust Boundary): বাইরে থেকে আসা তথ্য `restoreBooking`-এর অনুমোদিত তালিকা পেরিয়ে তবেই Class-এ ঢোকে।

এই পাঁচটি প্রশ্নের উত্তর থেকেই নিচের File-গুলোর গঠন এসেছে।

File-এর কাঠামো

Folder-এর কাঠামো

```
booking-lab/
├── src/
│   ├── booking.mjs       Booking, FlightBooking, HotelBooking, restoreBooking
│   └── manager.mjs       BookingManager
├── demo.mjs
├── README.md
└── tests/
    ├── booking.test.mjs
    └── manager.test.mjs
```

src/booking.mjs

দুই অংশ মিলে সম্পূর্ণ File।

booking.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
export class BookingError extends Error {
  constructor(message) {
    super(message);
    this.name = 'BookingError';
  }
}

function requireText(value, field) {
  if (typeof value !== 'string' || value.trim() === '') {
    throw new BookingError(`${field} খালি হতে পারবে না।`);
  }
  return value.trim();
}

function requirePositive(value, field) {
  if (!Number.isFinite(value) || value <= 0) {
    throw new BookingError(`${field} একটি ধনাত্মক সংখ্যা হতে হবে।`);
```

```
  }
  return value;
}

export class Booking {
  #status = 'confirmed';

  constructor({ id, customer, price }) {
    this.id = requireText(id, 'Booking ID');
    this.customer = requireText(customer, 'গ্রাহকের নাম');
    this.price = requirePositive(price, 'দাম');
  }

  get type() {
    return 'booking';
  }

  get status() {
    return this.#status;
  }

  get isActive() {
    return this.#status === 'confirmed';
  }

  get total() {
    return this.price;
  }

  cancel() {
    if (!this.isActive) {
      throw new BookingError(`${this.id} আগেই বাতিল হয়েছে।`);
    }
    this.#status = 'cancelled';
  }

  describe() {
    return `${this.id} (${this.type}) ${this.customer}: ${this.total} টাকা,
        ${this.#status}`;
  }

  toJSON() {
    const { id, customer, price } = this;
    return { type: this.type, id, customer, price, status: this.#status };
  }
}

// ...
```

booking.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export class FlightBooking extends Booking {
  constructor({ baggageFee = 0, ...details }) {
    super(details);
    if (!Number.isFinite(baggageFee) || baggageFee < 0) {
      throw new BookingError('Baggage Fee ঋণাত্মক হতে পারবে না।');
    }
    this.baggageFee = baggageFee;
  }

  get type() {
    return 'flight';
  }

  get total() {
    return super.total + this.baggageFee;
  }

  toJSON() {
    return { ...super.toJSON(), baggageFee: this.baggageFee };
  }
}

export class HotelBooking extends Booking {
  constructor({ nights, ...details }) {
    super(details);
    if (!Number.isInteger(nights) || nights < 1) {
      throw new BookingError('রাতের সংখ্যা 1 বা বেশি পূর্ণসংখ্যা হতে হবে।');
    }
    this.nights = nights;
  }

  get type() {
    return 'hotel';
  }

  get total() {
    return this.price * this.nights;
  }

  toJSON() {
    return { ...super.toJSON(), nights: this.nights };
  }
}

const KINDS = { booking: Booking, flight: FlightBooking, hotel: HotelBooking };
```

```
// সংরক্ষিত JSON থেকে সঠিক Class-এর Booking, বাতিল অবস্থাসহ।
export function restoreBooking(data) {
  if (!Object.hasOwn(KINDS, data?.type)) {
    throw new BookingError(`অচেনা Booking-এর ধরন: ${data?.type}`);
  }
  const booking = new KINDS[data.type](data);
  if (data.status === 'cancelled') {
    booking.cancel();
  }
  return booking;
}
```

- `#status` **শুধু** `cancel()` **দিয়ে বদলায়,** এবং `cancel()` নিয়ম যাচাই করে। বাইরে থেকে `booking.status = 'confirmed'` লিখলে কিছুই হয় না, কারণ শুধু Getter আছে।
- `describe()` **একবার লেখা,** কিন্তু `this.type` ও `this.total` Getter প্রতিটি Child Class-এ Override করা। Flight-এর `describe()` তাই Flight-এর মোট দেখায়, Hotel-এরটি Hotel-এর (Polymorphism)।
- `super.total` : Flight-এর মোট Parent-এর মোটের ওপর ভিত্তি করে; ভাড়ার নিয়ম বদলালে একটি জায়গায় বদলাবে।
- `constructor({ baggageFee = 0, ...details })` : Destructuring ও Rest (অধ্যায় ৭) দিয়ে নিজের Field আলাদা করে বাকিটা `super` -কে। Parent-এর যাচাই (ID, নাম, দাম) Child-এর জন্যও প্রযোজ্য।
- `toJSON()` : `JSON.stringify` এটি নিজে থেকে ডাকে। Private `#status` নিজে থেকে JSON-এ যায় না, তাই এখানে স্পষ্টভাবে যোগ করা হয়েছে; Child Class `super.toJSON()` -এর সঙ্গে নিজের Field যোগ করে।
- `restoreBooking` : `type` দেখে সঠিক Class বেছে নেয়; `KINDS` একটি অনুমোদিত তালিকা, এবং `Object.hasOwn` অধ্যায় ১১-এর `toString` ফাঁদ এড়ায়।

src/manager.mjs

দুই অংশ মিলে সম্পূর্ণ File।

manager.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { Booking, BookingError, restoreBooking } from './booking.mjs';

export class BookingManager {
  #bookings = [];
  #notify;

  // Composition: Manager-এর একটি Notifier আছে, যা বাইরে থেকে দেওয়া হয়।
  constructor({ notify = () => {} } = {}) {
    this.#notify = notify;
  }

  add(booking) {
    if (!(booking instanceof Booking)) {
      throw new TypeError('শুধু Booking যোগ করা যায়');
```

```
    }
    if (this.find(booking.id)) {
      throw new BookingError(`${booking.id} আগে থেকেই আছে।`);
    }
    this.#bookings.push(booking);
    this.#notify(`নতুন: ${booking.describe()}`);
    return booking;
  }

  find(id) {
    return this.#bookings.find((booking) => booking.id === id) ?? null;
  }

  cancel(id) {
    const booking = this.find(id);
    if (!booking) {
      throw new BookingError(`${id} পাওয়া যায়নি।`);
    }
    booking.cancel();
    this.#notify(`বাতিল: ${id}`);
  }

  // ...
```

manager.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
  // ...

  get count() {
    return this.#bookings.length;
  }

  get confirmedRevenue() {
    return this.#bookings
      .filter((booking) => booking.isActive)
      .reduce((sum, booking) => sum + booking.total, 0);
  }

  revenueByType() {
    const revenue = {};
    for (const booking of this.#bookings.filter((item) => item.isActive)) {
      revenue[booking.type] = (revenue[booking.type] ?? 0) + booking.total;
    }
    return revenue;
  }

  list() {
    return [...this.#bookings];
  }
```

```
  toJSON() {
    return this.#bookings.map((booking) => booking.toJSON());
  }

  static fromJSON(items, options = {}) {
    const manager = new BookingManager();
    for (const item of items) {
      manager.add(restoreBooking(item));
    }
    manager.#notify = options.notify ?? (() => {});
    return manager;
  }
}
```

- `#bookings` **Private,** এবং `list()` একটি Copy ফেরত দেয়। বাইরের Code তালিকায় সরাসরি কিছু ঢোকাতে বা মুছতে পারে না; সব কিছু `add` ও `cancel`-এর নিয়ম মেনে।
- `confirmedRevenue` প্রতিটি Booking-এর `total` যোগ করে, না জেনে সেটি Flight না Hotel।
- `static fromJSON` একটি Static Factory। লক্ষ করুন, Static Method-এর ভেতরে `manager.#notify` লেখা গেছে: Private Field Class-এর **ভেতরের যেকোনো Code**-এর জন্য খোলা, এমনকি অন্য Instance-এর জন্যও। Notifier পুনরুদ্ধারের **পরে** বসানো হয়েছে, যাতে পুরোনো Booking ফিরিয়ে আনার সময় নতুন করে খবর না যায়।
- `instanceof Booking` যাচাই Polymorphism-এর সঙ্গে মেলে: যেকোনো Child Class-এর Booking গ্রহণযোগ্য।

demo.mjs

demo.mjs · JavaScript — চলবে: Node.js

```
import { FlightBooking, HotelBooking } from './src/booking.mjs';
import { BookingManager } from './src/manager.mjs';

const messages = [];
const manager = new BookingManager({ notify: (text) => messages.push(text) });

manager.add(new FlightBooking({
  id: 'F-101', customer: 'Sara', price: 5200, baggageFee: 800,
}));
manager.add(new HotelBooking({
  id: 'H-201', customer: 'Rafi', price: 3500, nights: 3,
}));
manager.add(new FlightBooking({ id: 'F-102', customer: 'Mina', price: 4800 }));
manager.cancel('F-102');

for (const booking of manager.list()) {
  console.log(booking.describe());
}
console.log(`নিশ্চিত আয়: ${manager.confirmedRevenue} টাকা`);
```

```
console.log(manager.revenueByType());

try {
  manager.cancel('F-102');
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}

const saved = JSON.stringify(manager);
const restored = BookingManager.fromJSON(JSON.parse(saved));
console.log(`পুনরুদ্ধার: ${restored.count}টি, আয় ${restored.confirmedRevenue}`);
console.log(restored.find('H-201') instanceof HotelBooking, messages.length);
```

Output

```
F-101 (flight) Sara: 6000 টাকা, confirmed
H-201 (hotel) Rafi: 10500 টাকা, confirmed
F-102 (flight) Mina: 4800 টাকা, cancelled
নিশ্চিত আয়: 16500 টাকা
{ flight: 6000, hotel: 10500 }
BookingError: F-102 আগেই বাতিল হয়েছে।
পুনরুদ্ধার: 3টি, আয় 16500
true 4
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Notifier একটি সাধারণ Function** (`(text) => messages.push(text)`); Manager শুধু জানে এটিকে ডাকা যায়। বাস্তবে এখানে Email বা Log পাঠানো যেত, Manager না বদলে। চারটি বার্তা: তিনটি নতুন, একটি বাতিল।
- **`describe()` -এর তিনটি লাইন** তিনটি ভিন্ন হিসাব দেখায়: ভাড়া + Baggage, রাতপ্রতি × রাত, এবং বাতিল অবস্থা।
- **আয় ১৬,৫০০:** বাতিল F-102 বাদ। `revenueByType` প্রতিটি ধরনের আলাদা মোট দেয়।
- **দ্বিতীয়বার বাতিল** `Booking` -এর নিজের নিয়মে আটকেছে; Manager-কে এ নিয়ম জানতে হয়নি।
- **সংরক্ষণ ও পুনরুদ্ধার:** `JSON.stringify(manager)` Manager-এর `toJSON` , এবং তার ভেতরে প্রতিটি Booking-এর `toJSON` ডেকেছে। পুনরুদ্ধারে H-201 আবার একটি সত্যিকারের `HotelBooking` , এবং F-102 আবার বাতিল; আয় একই।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node demo.mjs
node --test
```

Call	যা হওয়ার কথা
`new Booking({ id: ' ', customer: 'A', price: 10 })`	`BookingError` : "Booking ID খালি হতে পারবে না।"
`new HotelBooking({ ..., nights: 1.5 })`	`BookingError` : রাতের সংখ্যার বার্তা
`new FlightBooking({ ..., baggageFee: -1 })`	`BookingError` : Baggage Fee-র বার্তা
একই ID দুবার `add`	`BookingError` : "... আগে থেকেই আছে।"
`manager.add({ id: 'X' })`	`TypeError` : সাধারণ Object গ্রহণযোগ্য নয়
`booking.status = 'confirmed'`	কিছুই বদলায় না (Module-এ `TypeError`)
`restoreBooking({ type: 'toString' })`	`BookingError` : অচেনা ধরন

Companion Source Code-এ ২০টি Automated Test আছে, যার মধ্যে Prototype Chain, Private Field-এর সুরক্ষা এবং JSON-এর মধ্য দিয়ে Class ও অবস্থা টিকে থাকার Test আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Hotel-এর মোট ভুল	`total` Override হয়নি বা বানান ভুল	Getter-এর নাম
`describe()` Child-এর মোট দেখায় না	`describe` -এ `this.total` -এর বদলে `this.price`	Getter ব্যবহার করুন
পুনরুদ্ধারের পর `instanceof` মিথ্যা	সাধারণ Object হিসেবে রাখা	`restoreBooking`
পুনরুদ্ধারের পর বাতিল Booking আবার নিশ্চিত	`toJSON` -এ `status` নেই	Private অবস্থা `toJSON` -এ
পুনরুদ্ধারের সময় অপ্রত্যাশিত বার্তা	Notifier আগে বসানো	পুনরুদ্ধারের পরে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। টাকা পূর্ণসংখ্যা হিসেবে ধরা হয়েছে; তারিখ, আসন বা ঘরের প্রাপ্যতা নেই; তথ্য শুধু Memory-তে বা JSON লেখায়। Browser-এর UI অধ্যায় ১২-এর মতো একটি আলাদা Module হিসেবে এর ওপর তৈরি করা যায়।

১৩.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `Book` Class লিখুন: `title`, `author`, `pages` এবং `summary()` Method, যা `"বইয়ের নাম — লেখক (পৃষ্ঠা)"` ধরনের লেখা দেয়।

২. `Rectangle` Class লিখুন, যার `area` ও `perimeter` Getter।

৩. না চালিয়ে বলুন: `Object.hasOwn(pen, 'label')` কেন `false`, যদিও `pen.label()` কাজ করে?

৪. `const p = Product('Pen', 50);` কেন ব্যর্থ হয়? ঠিক করুন।

৫. `Wallet` Class লিখুন, যার Private `#balance`; `spend(amount)` বেশি খরচে `RangeError` দেয়।

মধ্যম স্তর

৬. `Shape` থেকে `Circle` ও `Square` তৈরি করুন, প্রতিটির নিজস্ব `area()`; একটি তালিকার মোট ক্ষেত্রফল দুই দশমিক পর্যন্ত দেখান।

৭. `Student` Class-এ `marks`-এর Setter লিখুন, যা ০ থেকে ১০০-এর বাইরে `RangeError` দেয়।

৮. নিচের `Clock`-এর `this` সমস্যা ঠিক করুন: `start() { setTimeout(this.tick, 10); }`।

৯. `Money.fromText('1,250 টাকা')` Static Factory লিখুন, যা লেখা থেকে সংখ্যা বের করে `Money` তৈরি করে।

চ্যালেঞ্জ স্তর

১০. Project-এ `BusBooking` যোগ করুন: `seats` (১ থেকে ৪০), মোট = ভাড়া × আসন। পুনরুদ্ধারও যেন কাজ করে।

১১. Inheritance ছাড়া ছাড়ের নিয়ম লিখুন: `NoDiscount`, `PercentDiscount`, `FlatDiscount`—প্রতিটির `apply(amount)`; একটি `PriceCalculator` যেকোনোটি নিয়ে কাজ করে।

১২. `Stack` Class লিখুন: Private Array, `push`, `pop`, `peek`, `size`; ফাঁকা Stack-এ `pop` ত্রুটি দেয়।

১৩.১৫ অনুশীলনীর সমাধান

সমাধান ১

book.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Book {
  constructor(title, author, pages) {
    this.title = title;
    this.author = author;
    this.pages = pages;
  }

  summary() {
    return `${this.title} — ${this.author} (${this.pages} পৃষ্ঠা)`;
  }
}

console.log(new Book('পথের পাঁচালী', 'বিভূতিভূষণ', 320).summary());
```

Output

```
পথের পাঁচালী — বিভূতিভূষণ (320 পৃষ্ঠা)
```

সমাধান ২

rectangle.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  get area() {
    return this.width * this.height;
  }

  get perimeter() {
    return 2 * (this.width + this.height);
  }
}

const room = new Rectangle(4, 3);
console.log(room.area, room.perimeter);
```

```
room.width = 5;
console.log(room.area);
```

Output

```
12 14
15
```

Getter প্রতিবার হিসাব করে, তাই প্রস্থ বদলানোর পর ক্ষেত্রফল নিজে থেকেই ঠিক।

সমাধান ৩ — `label` থাকে `Product.prototype`-এ, `pen`-এর নিজের Property হিসেবে নয়। `Object.hasOwn` শুধু নিজের Property দেখে; `pen.label()` কাজ করে Prototype Chain ধরে খোঁজার কারণে। `'label' in pen` হলে `true` হতো, কারণ `in` পুরো শিকল দেখে।

সমাধান ৪ — Class `new` ছাড়া ডাকা যায় না: `TypeError: Class constructor Product cannot be invoked without 'new'`। লিখুন `const p = new Product('Pen', 50);`।

সমাধান ৫

wallet.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Wallet {
  #balance;

  constructor(balance) {
    this.#balance = balance;
  }

  spend(amount) {
    if (amount > this.#balance) {
      throw new RangeError(`${amount} টাকা খরচ সম্ভব নয়; আছে ${this.#balance}`);
    }
    this.#balance -= amount;
    return this.#balance;
  }
}

const wallet = new Wallet(300);
console.log(wallet.spend(120));

try {
  wallet.spend(500);
} catch (error) {
```

```
  console.log(error.message);
}
```

Output

```
180
500 টাকা খরচ সম্ভব নয়; আছে 180
```

সমাধান ৬

shapes.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Shape {
  area() {
    return 0;
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }

  area() {
    return Math.PI * this.radius ** 2;
  }
}

class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }

  area() {
    return this.side ** 2;
  }
}

const shapes = [new Circle(1), new Square(2), new Circle(2)];
const total = shapes.reduce((sum, shape) => sum + shape.area(), 0);

console.log(total.toFixed(2));
```

Output

```
19.71
```

π + ৪ + ৪π = ৫π + ৪ ≈ ১৯.৭১। Loop জানে না কোনটি বৃত্ত, কোনটি বর্গ।

সমাধান ৭

student-marks.js · JavaScript চলবে: Browser Console ও Node.js

```
class Student {
  #marks = 0;

  constructor(name, marks) {
    this.name = name;
    this.marks = marks;
  }

  get marks() {
    return this.#marks;
  }

  set marks(value) {
    if (!Number.isFinite(value) || value < 0 || value > 100) {
      throw new RangeError(`নম্বর 0 থেকে 100-এর মধ্যে হতে হবে: ${value}`);
    }
    this.#marks = value;
  }
}

const student = new Student('Sara', 88);
console.log(student.marks);

try {
  student.marks = 120;
} catch (error) {
  console.log(error.message);
}
console.log(student.marks);
```

Output

```
88
নম্বর 0 থেকে 100-এর মধ্যে হতে হবে: 120
88
```

সমাধান ৮

clock.js · JavaScript চলবে: Browser Console ও Node.js

```
class Clock {
  ticks = 0;

  tick() {
    this.ticks += 1;
    console.log(`টিক ${this.ticks}`);
  }

  start() {
    setTimeout(() => this.tick(), 10);
  }
}

new Clock().start();
```

Output

```
টিক 1
```

`setTimeout(this.tick, 10)` শুধু Function-টি পাঠায়, Object নয়; পরে চলার সময় `this` হারিয়ে যায়। Arrow Function `this` ধরে রাখে, এবং ভেতরে `this.tick()` আবার Object-এর ওপর ডাকে।

সমাধান ৯

money.js · JavaScript চলবে: Browser Console ও Node.js

```
class Money {
  constructor(amount) {
    if (!Number.isInteger(amount) || amount < 0) {
      throw new RangeError(`অবৈধ অঙ্ক: ${amount}`);
    }
    this.amount = amount;
  }

  static fromText(text) {
    const digits = text.replace(/[^\d]/g, '');
    return new Money(Number(digits));
  }

  toString() {
    return `${this.amount.toLocaleString('en-US')} টাকা`;
  }
}
```

```
const price = Money.fromText('1,250 টাকা');
console.log(price.amount, `${price}`);
```

Output

```
1250 1,250 টাকা
```

Template Literal-এ Object বসালে JavaScript তার `toString()` ডাকে; Override করা `toString` তাই সুন্দর লেখা দেয়।

সমাধান ১০

bus-booking.mjs · JavaScript — চলবে: Node.js

```
import { Booking, BookingError } from '../../project/booking-lab/src/booking.mjs';

export class BusBooking extends Booking {
  constructor({ seats, ...details }) {
    super(details);
    if (!Number.isInteger(seats) || seats < 1 || seats > 40) {
      throw new BookingError('আসন 1 থেকে 40-এর মধ্যে হতে হবে।');
    }
    this.seats = seats;
  }

  get type() {
    return 'bus';
  }

  get total() {
    return this.price * this.seats;
  }

  toJSON() {
    return { ...super.toJSON(), seats: this.seats };
  }
}

const trip = new BusBooking({
  id: 'B-1', customer: 'Nila', price: 650, seats: 3,
});
const saved = JSON.parse(JSON.stringify(trip));

console.log(trip.describe());
console.log(saved.type, saved.seats, saved.status);
```

Output

```
B-1 (bus) Nila: 1950 টাকা, confirmed
bus 3 confirmed
```

`describe`, `cancel`, যাচাই—সব Parent থেকে এসেছে; নতুন Class-এ শুধু পার্থক্যটুকু। পুনরুদ্ধারের জন্য `booking.mjs`-এর `KINDS` তালিকায় `bus: BusBooking` যোগ করতে হবে। `BookingManager` একটুও বদলাতে হয় না।

সমাধান ১১

discounts.js · JavaScript — চলবে: Browser Console ও Node.js

```
class NoDiscount {
  apply(amount) {
    return amount;
  }
}

class PercentDiscount {
  constructor(percent) {
    this.percent = percent;
  }

  apply(amount) {
    return Math.round(amount * (1 - this.percent / 100));
  }
}

class FlatDiscount {
  constructor(off) {
    this.off = off;
  }

  apply(amount) {
    return Math.max(0, amount - this.off);
  }
}

class PriceCalculator {
  #discount;

  constructor(discount = new NoDiscount()) {
    this.#discount = discount;
  }

  final(amount) {
    return this.#discount.apply(amount);
```

```
  }
}

const rules = [new NoDiscount(), new PercentDiscount(10), new FlatDiscount(200)];

for (const rule of rules) {
  const price = new PriceCalculator(rule).final(1500);
  console.log(`${rule.constructor.name}: ${price}`);
}
```

Output

```
NoDiscount: 1500
PercentDiscount: 1350
FlatDiscount: 1300
```

তিনটি ছাড়-Class-এর কোনো সাধারণ Parent নেই; `PriceCalculator` শুধু জানে প্রতিটির `apply` আছে (Duck Typing ও Composition)।

সমাধান ১২

stack.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Stack {
  #items = [];

  push(item) {
    this.#items.push(item);
    return this;
  }

  pop() {
    if (this.#items.length === 0) {
      throw new RangeError('Stack ফাঁকা');
    }
    return this.#items.pop();
  }

  peek() {
    return this.#items.at(-1);
  }

  get size() {
    return this.#items.length;
  }
}

const stack = new Stack().push('ক').push('খ').push('গ');
```

```
console.log(stack.pop(), stack.peek(), stack.size);
stack.pop();
stack.pop();

try {
  stack.pop();
} catch (error) {
  console.log(error.message);
}
```

Output

```
গ খ 2
Stack ফাঁকা
```

`push` `this` ফেরত দেয়, তাই পরপর ডাকা (Method Chaining) সম্ভব হয়েছে।

১৩.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `typeof` একটি Class-এর জন্য কী দেয়?

ক) `'class'`

খ) `'object'`

গ) `'function'`

ঘ) `'prototype'`

প্রশ্ন ২. একটি Class-এর Method কোথায় থাকে?

ক) Class-এর `prototype` Object-এ

খ) প্রতিটি Instance-এর নিজের Property হিসেবে

গ) Global Scope-এ

ঘ) `constructor`-এর ভেতরে

প্রশ্ন ৩. `#balance` Field বাইরে থেকে পড়ার চেষ্টা করলে কী হয়?

ক) `undefined`

খ) মানটি পাওয়া যায়

গ) `RangeError`

ঘ) `SyntaxError`, Program চলার আগেই

প্রশ্ন ৪. Child Class-এর `constructor`-এ `this` ব্যবহারের আগে কী ডাকতে হয়?

ক) `new`

খ) `super()`

গ) `this.constructor()`

ঘ) কিছুই না

প্রশ্ন ৫. `Temperature.fromFahrenheit()` কোন ধরনের Method?

ক) Private

খ) Getter

গ) Static

ঘ) Setter

প্রশ্ন ৬. বিভিন্ন Class-এর Object একই নামের Method-এ নিজের মতো সাড়া দেয়। এর নাম?

ক) Encapsulation

খ) Polymorphism

গ) Hoisting

ঘ) Composition

প্রশ্ন ৭. `setTimeout(counter.increment, 100)`-এ সমস্যা কী?

ক) Method-টি `counter`-এর সঙ্গে যুক্ত থাকে না; `this` হারায়

খ) `setTimeout` Method নিতে পারে না

গ) `increment` দুবার চলে

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৮. "Order Service-এর একটি Notifier আছে"—এটি কোন সম্পর্ক?

ক) Inheritance

খ) Prototype

গ) Static

ঘ) Composition

প্রশ্ন ৯. `get total()` ব্যবহারের সুবিধা কোনটি?

ক) মান কখনো বদলায় না

খ) দ্রুত চলে

গ) Property-র মতো পড়া যায়, কিন্তু প্রতিবার সর্বশেষ তথ্য থেকে হিসাব হয়

ঘ) মান Private হয়ে যায়

> **প্রশ্ন ১০.** `JSON.stringify(booking)` Private Field কীভাবে সংরক্ষণ করবে?
>
> ক) নিজে থেকেই সব Private Field যায়
>
> খ) Class-এর `toJSON()` Method-এ স্পষ্টভাবে যোগ করতে হয়
>
> গ) Private Field কখনো সংরক্ষণ করা যায় না
>
> ঘ) `#` চিহ্নসহ সংরক্ষিত হয়

১৩.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–খ · ৭–ক · ৮–ঘ · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	Class একটি বিশেষ Function।
২	ক	সব Instance Prototype Chain ধরে একই Method পায়।
৩	ঘ	Private নাম Class-এর বাইরে ব্যাকরণগতভাবেই অবৈধ।
৪	খ	Parent-এর `constructor` না চলা পর্যন্ত `this` তৈরি হয় না।
৫	গ	Class-এর ওপর ডাকা হয়, Instance-এর ওপর নয়।
৬	খ	একই প্রশ্ন (`fee()`), প্রতিটি Class-এর নিজের উত্তর।
৭	ক	Function আলাদা করে পাঠালে `this` আর Object নয়; `() => counter.increment()` লিখুন।
৮	ঘ	"আছে" (has-a) মানে Composition; "একটি ধরন" (is-a) মানে Inheritance।
৯	গ	হিসাব করা মান আলাদা করে রাখলে অমিলের ঝুঁকি থাকে।
১০	খ	`JSON.stringify` শুধু নিজের Public Property দেখে; `toJSON` তা নিয়ন্ত্রণ করে।

১৩.১৮ অধ্যায়ের সারসংক্ষেপ

- OOP তথ্য ও তার নিয়ম একসঙ্গে রাখে; Class একটি ছাঁচ, `new` দিয়ে Instance।
- Field ও `constructor` প্রতিটি Instance-এর অবস্থা বসায়; Method সবাই ভাগ করে নেয়।
- প্রতিটি Object-এর একটি Prototype; Property না পেলে JavaScript Prototype Chain ধরে খোঁজে; `class` এই ব্যবস্থার ওপর লেখার ধরন।
- Private (`#`) Field ও Method বাইরে থেকে অগম্য; অবস্থা বদলায় শুধু নিয়ম-মানা Method দিয়ে।
- Getter হিসাব করা মান দেয়, Setter যাচাই করে; Static সদস্য Class-এর নিজের, যেমন Static Factory।

- `extends` Prototype Chain-এ একটি ধাপ যোগ করে; `super()` আগে, `super.method()` দিয়ে Parent-এর নিয়ম পুনর্ব্যবহার।
- Polymorphism `if (type === ...)` শৃঙ্খলের বদলে; নতুন ধরন মানে নতুন Class, পুরোনো Code অপরিবর্তিত।
- Method আলাদা করে পাঠালে `this` হারায়; `bind`, Arrow Field, বা `() => obj.method()`।
- Inheritance (is-a) অগভীর রাখুন; আচরণ যোগে Composition (has-a) ও Duck Typing।
- `toJSON` ও Static `fromJSON` দিয়ে Class-এর Object সংরক্ষণ ও পুনরুদ্ধার।

পরের অধ্যায়ে Function-এর উন্নত ব্যবহার: Higher-Order Function, Composition, Currying, Memoization এবং Functional Programming-এর ধরন—Class-এর বিকল্প আরেকটি শক্তিশালী উপায়।

১৩.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Class	একই ধরনের Object তৈরির ছাঁচ।
Instance	Class থেকে `new` দিয়ে তৈরি Object।
Constructor	Instance তৈরির সময় চলা প্রাথমিক Code।
Prototype	যে Object থেকে আরেকটি Object Property উত্তরাধিকারসূত্রে পায়।
Prototype Chain	Property খোঁজার জন্য Prototype থেকে Prototype-এ শিকল।
Encapsulation	অবস্থা লুকিয়ে শুধু Method দিয়ে বদলানোর নীতি।
Private Field	`#` দিয়ে শুরু, Class-এর বাইরে অগম্য Field।
Getter / Setter	Property-র মতো পড়া ও লেখা, ভেতরে Code চলে।
Static	Class-এর নিজের সদস্য, Instance-এর নয়।
Inheritance	`extends` দিয়ে আরেকটি Class-এর বৈশিষ্ট্য নেওয়া (is-a)।
Polymorphism	একই Method-এ বিভিন্ন Class-এর ভিন্ন আচরণ।
Composition	অন্য Object-কে অংশ হিসেবে রেখে আচরণ গঠন (has-a)।

১৩.২০ আরও পড়ুন

- **MDN — Classes:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes` — Class-এর সম্পূর্ণ রেফারেন্স।

- **MDN — Private properties:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_properties` — `#` Field ও Method।
- **MDN — Inheritance and the prototype chain:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain` — Prototype Chain-এর গভীর ব্যাখ্যা।
- **MDN — extends:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends` — Inheritance ও `super`।
- **MDN — Function.prototype.bind():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind` — `this` স্থির করা।

অধ্যায় ১৪

১৪

Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Function-কে মান হিসেবে ব্যবহার করতে: রাখা, পাঠানো, ফেরত দেওয়া।
- Closure দিয়ে সুরক্ষিত অবস্থা তৈরি করতে, এবং Loop-এর Closure ফাঁদ চিনতে।
- নিজের Higher-Order Function লিখতে: অন্য Function-কে মুড়ে নতুন আচরণ যোগ করা।
- `call`, `apply` ও `bind` দিয়ে `this` এবং Argument নিয়ন্ত্রণ করতে।
- ছোট Function জুড়ে Pipeline তৈরি করতে (`pipe`, `compose`)।
- Currying ও Partial Application দিয়ে পুনর্ব্যবহারযোগ্য Function তৈরি করতে।
- Pure Function ও Immutability-র সুবিধা ব্যাখ্যা করতে, এবং Side Effect সীমানায় রাখতে।
- Memoization, Debounce ও Throttle কখন উপকারী আর কখন ক্ষতিকর, তা বিচার করতে।
- একটি সম্পূর্ণ **Sales Function Lab** তৈরি ও পরীক্ষা করতে, যার Timer-নির্ভর অংশও নিয়ন্ত্রিতভাবে Test করা।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function, Scope, Closure-এর প্রাথমিক ধারণা।
- অধ্যায় ৬: `map`, `filter`, `reduce`।
- অধ্যায় ১০: Timer এবং Debounce (অনুশীলনী ১২)।
- অধ্যায় ১২–১৩: ES Module এবং `this`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৩-এ তথ্য ও আচরণ একসঙ্গে রাখার একটি উপায় দেখেছেন: Class। এই অধ্যায় দেখায় আরেকটি, সমান শক্তিশালী উপায়: ছোট, নির্ভরযোগ্য Function তৈরি করে সেগুলো জুড়ে বড় কাজ গঠন। এই ধারার নাম **Functional Programming**। JavaScript দুটি ধারাই সমর্থন করে, এবং আধুনিক Code-এ দুটিই পাশাপাশি দেখা যায়: React-এর Component ও Hook, Redux-এর Reducer, Data Processing-এর Pipeline—সবই Functional ধারণার ওপর।

এর মূল সুবিধা নির্ভরযোগ্যতা। যে Function একই ইনপুটে সব সময় একই ফল দেয় এবং বাইরের কিছু বদলায় না, তাকে বোঝা, পরীক্ষা করা এবং পুনর্ব্যবহার করা সহজ। আর Closure ও Higher-Order Function-এর মতো কৌশল দিয়ে এমন সাধারণ সরঞ্জাম লেখা যায়—যেমন `once`, `memoize`, `throttle`—যা যেকোনো Function-এর ওপর প্রয়োগ করা যায়।

১৪.১ Function একটি মান

JavaScript-এ Function একটি **First-Class Value**: সংখ্যা বা লেখার মতোই Variable-এ রাখা যায়, Object-এর ভেতরে রাখা যায়, Argument হিসেবে পাঠানো যায়, এবং অন্য Function থেকে ফেরত দেওয়া যায়।

values.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greet(name) {
  return `স্বাগতম, ${name}`;
}

const actions = { greet, shout: (text) => text.toUpperCase() };
console.log(actions.greet('Sara'), actions.shout('ok'));
console.log(greet.name, greet.length, typeof greet);

function multiplier(factor) {
  return (value) => value * factor;
}

const double = multiplier(2);
const triple = multiplier(3);
console.log(double(21), triple(7));
```

Output

```
স্বাগতম, Sara OK
greet 1 function
42 21
```

`multiplier` একটি **Function Factory**: প্রতিবার ডাকলে একটি নতুন Function তৈরি করে ফেরত দেয়, যা তৈরির সময়ের `factor` মনে রাখে। Function-এর নিজেরও Property আছে: `name` তার নাম, `length` কয়টি Parameter আশা করে (Default ও Rest Parameter বাদে)।

১৪.২ Closure: মনে রাখা অবস্থা

অধ্যায় ৫-এ দেখেছেন, একটি Function তার তৈরির জায়গার Variable মনে রাখে, এমনকি বাইরের Function শেষ হয়ে যাওয়ার পরও। এটিই **Closure**। এর সবচেয়ে কাজের ব্যবহার হলো এমন অবস্থা তৈরি, যা বাইরে থেকে সরাসরি ছোঁয়া যায় না:

closure-state.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createCounter(start = 0) {
  let count = start;

  return {
    next: () => ++count,
    reset: () => {
      count = start;
    },
    get value() {
      return count;
    },
  };
}

const tickets = createCounter(100);
const orders = createCounter();

tickets.next();
tickets.next();
orders.next();
console.log(tickets.value, orders.value);

tickets.reset();
console.log(tickets.value, 'count' in tickets);
```

Output

```
102 1
100 false
```

প্রতিটি `createCounter()` ডাকে একটি নতুন `count` তৈরি হয়, তাই `tickets` ও `orders` পরস্পরকে প্রভাবিত করে না। `count` Object-এর Property নয় (`'count' in tickets` মিথ্যা); এটিকে বদলানোর একমাত্র পথ ফেরত দেওয়া তিনটি Function। অধ্যায় ১৩-এর Private Field (`#`) যে সুরক্ষা দেয়, Closure তা দেয় Class ছাড়াই।

Loop-এর Closure ফাঁদ

loop-trap.js · JavaScript — চলবে: Browser Console ও Node.js

```
const withVar = [];
for (var i = 0; i < 3; i++) {
  withVar.push(() => i);
}

const withLet = [];
for (let j = 0; j < 3; j++) {
  withLet.push(() => j);
}

console.log(withVar.map((fn) => fn()).join(', '));
console.log(withLet.map((fn) => fn()).join(', '));
```

Output

```
3, 3, 3
0, 1, 2
```

`var`-এর একটিই Variable পুরো Loop জুড়ে; তিনটি Function একই `i` মনে রাখে, এবং ডাকার সময় তার মান Loop শেষের `3`। `let` প্রতিটি পুনরাবৃত্তিতে একটি নতুন Variable তৈরি করে, তাই প্রতিটি Function নিজের মান পায়। পুরোনো Code-এ Timer বা Event Listener-এ এই Bug প্রায়ই দেখা যায়; সমাধান সহজ: `var`-এর বদলে `let`।

> **নোট:** Closure যা মনে রাখে, তা Memory-তে থেকে যায় যতক্ষণ Function-টি বেঁচে থাকে। একটি বড় তালিকা মনে রাখা Event Listener সরানো না হলে সেই তালিকাও কখনো মুক্ত হয় না। দীর্ঘজীবী Closure-এ শুধু প্রয়োজনীয় তথ্যটুকু রাখুন।

১৪.৩ Higher-Order Function: Function-কে মোড়ানো

যে Function অন্য Function নেয় বা ফেরত দেয়, তা **Higher-Order Function**। `map` ও `filter` আপনি ব্যবহার করেছেন; এবার নিজে লিখুন। সবচেয়ে কাজের ধরন হলো একটি Function-কে "মোড়ানো": মূল কাজ অপরিবর্তিত রেখে তার চারপাশে নতুন আচরণ যোগ।

wrappers.js · JavaScript — চলবে: Browser Console ও Node.js

```
function once(fn) {
  let done = false;
  let result;
```

```
  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
    }
    return result;
  };
}

function logged(fn) {
  return (...args) => {
    const result = fn(...args);
    console.log(`${fn.name}(${args.join(', ')}) = ${result}`);
    return result;
  };
}

const connect = once(() => {
  console.log('সংযোগ তৈরি');
  return 'connection-1';
});
console.log(connect(), connect());

const add = (a, b) => a + b;
const loggedAdd = logged(add);
console.log(loggedAdd(2, 3) * 10);
```

Output

```
সংযোগ তৈরি
connection-1 connection-1
add(2, 3) = 5
50
```

- `once` নিশ্চিত করে কাজটি একবারই হবে—Database সংযোগ, Payment-এর অনুরোধ, স্বাগত-বার্তা। দ্বিতীয় ডাকে আগের ফল ফেরত আসে, কাজ আবার হয় না। `done` ও `result` Closure-এ।
- `logged` যেকোনো Function-কে না বদলে তার প্রতিটি ডাক লিখে রাখে। মোড়ানো Function আসলটির মতোই আচরণ করে (ফল ফেরত দেয়), তাই যেকোনো জায়গায় বসানো যায়।
- `...args` (Rest) সব Argument সংগ্রহ করে, `fn(...args)` (Spread) সেগুলো হুবহু পাঠায়; ফলে Wrapper জানে না, জানার দরকারও নেই, মূল Function কয়টি Argument নেয়।

১৪.৪ call, apply এবং bind

অধ্যায় ১৩-এ `bind` দিয়ে `this` স্থির করেছেন। প্রতিটি Function-এর তিনটি Method আছে, যা দিয়ে `this` নিজে বেছে দেওয়া যায়:

call-apply-bind.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describe(prefix, suffix) {
  return `${prefix} ${this.name}${suffix}`;
}

const user = { name: 'Sara' };

console.log(describe.call(user, 'নাম:', '।'));
console.log(describe.apply(user, ['নাম:', '!']));

const hello = describe.bind(user, 'হ্যালো,');
console.log(hello('!'));

console.log(Math.max.apply(null, [4, 9, 2]), Math.max(...[4, 9, 2]));
```

Output

```
নাম: Sara।
নাম: Sara!
হ্যালো, Sara!
9 9
```

Method	কী করে	Argument
`fn.call(thisValue, a, b)`	এখনই ডাকে	একটি একটি করে
`fn.apply(thisValue, [a, b])`	এখনই ডাকে	একটি Array-তে
`fn.bind(thisValue, a)`	ডাকে না; নতুন Function ফেরত দেয়	আগে থেকে কিছু Argument বসানো যায়

আধুনিক Code-এ `apply`-এর বেশির ভাগ কাজ Spread (`...`) করে, যেমন শেষ লাইন। কিন্তু `bind`-এর একটি দ্বিতীয় ক্ষমতা লক্ষ করুন: `hello`-তে প্রথম Argument `'হ্যালো,'` আগে থেকেই বসানো। একে বলে **Partial Application**, যা অনুচ্ছেদ ১৪.৬-এ আরও দেখবেন।

১৪.৫ Function Composition: pipe এবং compose

ছোট, একটি কাজের Function জুড়ে বড় কাজ গঠনকে বলে **Composition**। একটির ফল পরেরটির ইনপুট—কারখানার Assembly Line-এর মতো।

pipe.js · JavaScript চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);

const trim = (text) => text.trim();
const lower = (text) => text.toLowerCase();
const words = (text) => text.split(/\s+/);
const count = (list) => list.length;

const wordCount = pipe(trim, lower, words, count);
const slug = pipe(trim, lower, words, (list) => list.join('-'));

console.log(wordCount('  JavaScript শিখুন প্রতিদিন  '));
console.log(slug(' Modern JavaScript Guide '));
```

Output

```
3
modern-javascript-guide
```

`pipe(f, g, h)(x)` মানে `h(g(f(x)))`, কিন্তু পড়া যায় বাঁ থেকে ডানে, কাজের ক্রমে। `pipe`-এর ভেতরটা একটি `reduce`: শুরুর মান `input`, এবং প্রতিটি Function আগের ফলকে নতুন ফলে রূপান্তর করে। লক্ষ করুন, `trim`, `lower`, `words` দুটি Pipeline-এ পুনর্ব্যবহৃত; প্রতিটি আলাদাভাবে পরীক্ষাযোগ্য।

`compose` একই কাজ করে উল্টো ক্রমে, গণিতের `f(g(x))`-এর মতো: `compose(h, g, f)` = `pipe(f, g, h)`। দুটির কোনটি ব্যবহার করবেন তা রুচির বিষয়; একটি Project-এ একটিই বেছে নিন।

Pipeline-এর শর্ত: প্রতিটি ধাপ একটি ইনপুট নেয় এবং একটি ফল দেয়। দুটি Argument লাগে এমন Function (যেমন `join(list, '-')`) সরাসরি বসানো যায় না; তখন পরের অনুচ্ছেদের কৌশল লাগে।

১৪.৬ Currying এবং Partial Application

Partial Application মানে একটি Function-এর কিছু Argument আগে বসিয়ে কম Argument-এর নতুন Function তৈরি। **Currying** মানে বহু-Argument-এর Function-কে এমনভাবে রূপান্তর, যাতে Argument এক এক করে দেওয়া যায়, এবং যতক্ষণ সব না আসে ততক্ষণ একটি নতুন Function ফেরত আসে।

curry.js · JavaScript চলবে: Browser Console ও Node.js

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...more) => curried(...args, ...more);
  };
```

```
}

const priceWithVat = curry((ratePercent, price) =>
  Math.round((price * (100 + ratePercent)) / 100));

const bdVat = priceWithVat(15);
console.log(bdVat(1000), bdVat(200));
console.log(priceWithVat(5, 1000));

const isAbove = curry((limit, value) => value > limit);
console.log([120, 2500, 950, 4000].filter(isAbove(1000)).join(', '));

const discount = (percent, price) => price * (1 - percent / 100);
const tenPercentOff = discount.bind(null, 10);
console.log(tenPercentOff(500));
```

Output

```
1150 230
1050
2500, 4000
450
```

- `bdVat` একটি Partial Application: হার আগে বসানো, দাম পরে। একটি সাধারণ Function থেকে নির্দিষ্ট কাজের Function তৈরি হলো।
- `isAbove(1000)` সরাসরি `filter`-এ বসেছে। `filter` প্রতিটি মানের জন্য একটি Argument দেয়; Currying দুই-Argument-এর তুলনাকে এক-Argument-এর প্রশ্নে পরিণত করেছে। এটিই Pipeline-এ Currying-এর আসল উপযোগিতা।
- `curry` `fn.length` দেখে বোঝে কতগুলো Argument দরকার। তাই Default বা Rest Parameter-এর Function-এ এটি নির্ভরযোগ্য নয় (তাদের `length` কম দেখায়)।
- `bind(null, 10)` একই Partial Application, Built-in উপায়ে, যখন `this` লাগে না।
- টাকার হিসাবে দশমিকের সমস্যা এড়াতে হার শতাংশে (`15`) রেখে পূর্ণসংখ্যায় হিসাব করা হয়েছে (অধ্যায় ২)।

১৪.৭ Pure Function, Immutability এবং Side Effect

একটি **Pure Function**-এর দুটি শর্ত: একই ইনপুটে সব সময় একই ফল, এবং বাইরের কিছু বদলায় না (কোনো **Side Effect** নেই)।

purity.js · JavaScript চলবে: Browser Console ও Node.js

```
let taxPercent = 15;
const impureTotal = (price) => (price * (100 + taxPercent)) / 100;
const pureTotal = (price, percent) => (price * (100 + percent)) / 100;

console.log(impureTotal(100));
taxPercent = 20;
console.log(impureTotal(100), pureTotal(100, 15));

const addTag = (product, tag) => ({ ...product, tags: [...product.tags, tag] });

const pen = Object.freeze({ name: 'Pen', tags: Object.freeze(['stationery']) });
const tagged = addTag(pen, 'sale');
console.log(pen.tags.join(','), '|', tagged.tags.join(','));
```

Output

```
115
120 115
stationery | stationery,sale
```

- **`impureTotal`** একই ইনপুট `100`-এ দুবার দুই রকম ফল দিয়েছে, কারণ এটি বাইরের `taxPercent` পড়ে। Test-এ এর ফল নির্ভর করে Test-এর আগে কোন Code চলেছে তার ওপর। **`pureTotal`** যা লাগে সব Argument হিসেবে নেয়।
- **`addTag`** মূল Product বদলায় না, নতুন একটি ফেরত দেয় (অধ্যায় ৭-এর Spread)। `Object.freeze` এর প্রমাণ: মূল Object বদলানোর চেষ্টা হলে সেটি নীরবে ব্যর্থ হতো, আর Strict Mode-এ Error দিত।

Side Effect	উদাহরণ
বাইরের Variable বদলানো	`total += x` (Function-এর বাইরের `total`)
Argument বদলানো	`items.push(x)`, `user.name = ...`
বাইরের জগতের সঙ্গে যোগাযোগ	`console.log`, `fetch`, `localStorage`, DOM
অনির্দিষ্ট ইনপুট	`Date.now()`, `Math.random()`

Immutability-র একটি পারফরম্যান্স ফাঁদ জানা দরকার। `rows.reduce((acc, row) => ({ ...acc, [row.id]: row }), {})` দেখতে সুন্দর, কিন্তু প্রতিটি ধাপে আগের সব Key-সহ একটি নতুন Object তৈরি হয়: ১০০০ সারিতে প্রায় পাঁচ লক্ষ Key Copy। ১০ সারিতে পার্থক্য চোখে পড়ে না, ১০,০০০ সারিতে Program লক্ষণীয়ভাবে ধীর। সমাধান: Function-এর ভেতরে একটি নতুন, স্থানীয় Object তৈরি করে তাতে সরাসরি লিখুন, অথবা `Object.fromEntries(rows.map((row) => [row.id, row]))`। বাইরে থেকে Function তখনও Pure, কারণ স্থানীয় Object বাইরে থেকে কেউ দেখে না যতক্ষণ না সেটি ফেরত যায়।

Side Effect ছাড়া কোনো Program কাজের নয়: শেষ পর্যন্ত কিছু দেখাতে বা সংরক্ষণ করতেই হয়। লক্ষ্য Side Effect দূর করা নয়, **সীমানায় রাখা**: হিসাবের কেন্দ্র Pure, আর Console, Network, Storage প্রান্তে, অল্প কয়েকটি জায়গায় (অধ্যায় ১২-এর Domain ও Adapter)। একটি Function-এর ভেতরে স্থানীয় Variable বদলানো Side Effect নয়, যতক্ষণ তা বাইরে থেকে দেখা যায় না।

১৪.৮ Memoization

Pure Function একই ইনপুটে একই ফল দেয়, তাই একবার হিসাব করা ফল মনে রেখে পরে সরাসরি ফেরত দেওয়া যায়। একে বলে **Memoization**। ফল মনে রাখতে এখানে `Map` ব্যবহার করা হয়েছে: Object-এর মতোই Key-Value জোড়া রাখে, তবে যেকোনো ধরনের মান Key হতে পারে, এবং `has`, `get`, `set` Method আছে।

memoize.js · JavaScript — চলবে: Browser Console ও Node.js

```
function memoize(fn) {
  const cache = new Map();

  return (n) => {
    if (!cache.has(n)) {
      cache.set(n, fn(n));
    }
    return cache.get(n);
  };
}

let calls = 0;
const slowSquare = (n) => {
  calls += 1;
  return n * n;
};

const fastSquare = memoize(slowSquare);
console.log(fastSquare(9), fastSquare(9), fastSquare(4), calls);

const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
console.log(fib(70));
```

Output

```
81 81 16 2
190392490709135
```

`fastSquare(9)` দ্বিতীয়বার হিসাব ছাড়াই এসেছে (`calls` মাত্র ২)। দ্বিতীয় উদাহরণ নাটকীয়: Memoization ছাড়া `fib(70)` সরল Recursive পদ্ধতিতে কয়েক হাজার কোটি Function Call করত; এখানে প্রতিটি `n` একবারই হিসাব হয়েছে।

Memoization বিনামূল্যে নয়। সতর্কতা:

- **শুধু Pure Function।** `Date.now()` বা Network-নির্ভর ফল মনে রাখলে বাসি ফল আসবে।
- **Memory বাড়ে।** প্রতিটি নতুন ইনপুট Cache-এ জমে; সীমাহীন ইনপুটে সীমাহীন Memory। প্রয়োজনে Cache-এর আকার সীমিত করুন বা মুছে ফেলার ব্যবস্থা রাখুন।
- **Key সাবধানে।** Object-কে Key বানালে `Map` তার পরিচয় (Reference) দেখে, বিষয়বস্তু নয়; একই বিষয়ের দুটি আলাদা Object দুটি আলাদা Key।
- **সস্তা হিসাবে লাভ নেই।** Cache-এ খোঁজাও সময় নেয়; শুধু সত্যিকারের ব্যয়বহুল কাজে Memoize করুন, এবং আগে মেপে দেখুন।

১৪.৯ Debounce এবং Throttle

অধ্যায় ১০-এর অনুশীলনীতে **Debounce** দেখেছেন: দ্রুত পরপর ঘটনায় শুধু শেষটির পর একবার কাজ। এর সঙ্গী **Throttle**: নির্দিষ্ট সময়ের মধ্যে সর্বোচ্চ একবার কাজ, ঘটনা যত ঘনই হোক।

throttle.js · JavaScript — চলবে: Browser Console ও Node.js

```
function throttle(fn, ms) {
  let last = -Infinity;

  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}

let runs = 0;
const onScroll = throttle(() => {
  runs += 1;
}, 180);

for (let t = 0; t < 500; t += 50) {
  setTimeout(onScroll, t);
}
setTimeout(() => console.log(`10টি ঘটনা, কাজ হলো ${runs} বার`), 600);
```

Output

```
10টি ঘটনা, কাজ হলো 3 বার
```

	Debounce	Throttle
কখন চলে	ঘটনা থামার পর একবার	চলাকালীন নিয়মিত বিরতিতে
উদাহরণ	খোঁজার Box: লেখা থামলে Request	Scroll বা Resize: চলার সময় অবস্থান হালনাগাদ
ঝুঁকি	ঘটনা না থামলে কখনো চলে না	শেষ ঘটনাটি বাদ পড়তে পারে

দুটিই Closure-এ অবস্থা রাখে (`timer` বা `last`) এবং দুটিই Higher-Order Function: যেকোনো Function-কে মুড়ে দেয়।

Functional না Object-Oriented?

একই সমস্যা দুই ধারায় লিখে তুলনা করুন। একটি Shopping Cart-এর মোট:

two-styles.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Cart {
  #items = [];

  add(name, price) {
    this.#items.push({ name, price });
    return this;
  }

  get total() {
    return this.#items.reduce((sum, item) => sum + item.price, 0);
  }
}

const addItem = (items, name, price) => [...items, { name, price }];
const totalOf = (items) => items.reduce((sum, item) => sum + item.price, 0);

const cart = new Cart().add('Pen', 50).add('Notebook', 120);
const before = addItem([], 'Pen', 50);
const after = addItem(before, 'Notebook', 120);

console.log(cart.total, totalOf(after), totalOf(before));
```

Output

```
170 170 50
```

Class-এর ধারায় একটি Object নিজের অবস্থা রাখে এবং তা বদলায়; পুরোনো অবস্থা আর পাওয়া যায় না। Functional ধারায় প্রতিটি ধাপ একটি নতুন মান, তাই আগের অবস্থা (`before`) অক্ষত, যা Undo, ইতিহাস বা তুলনার জন্য স্বাভাবিক সুবিধা, এবং Test-এ প্রতিটি ধাপ আলাদাভাবে যাচাই করা যায়। অন্যদিকে Class নিয়ম ও অবস্থাকে একটি নামযুক্ত এককে আবদ্ধ রাখে, এবং দীর্ঘজীবী, অনেক নিয়মের জিনিসের (Bank Account, Booking) জন্য প্রায়ই বেশি স্বাভাবিক।

ব্যবহারিক নিয়ম: তথ্যের রূপান্তর (Report, ছাঁকা, হিসাব) Functional ধারায়; নিজস্ব জীবনচক্র ও নিয়মের সত্তা Class-এ। অধ্যায় ১২-এর কার্ট Functional ছিল, অধ্যায় ১৩-এর Booking Class; দুটি সিদ্ধান্তই তাদের সমস্যার জন্য উপযুক্ত। Timer-নির্ভর Code পরীক্ষা করতে আসল সময়ের অপেক্ষা ধীর ও অনিশ্চিত; Project-এর Test Node.js-এর **Fake Timer** দিয়ে সময়কে হাতে এগিয়ে নেয়, তাই ফল প্রতিবার নিশ্চিত ও তাৎক্ষণিক।

১৪.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Loop-এ `var` এবং Closure	সব Function শেষ মান দেখে	`let`
`fn` পাঠানোর বদলে `fn()`	ফল পাঠানো হয়, Function নয়	Parenthesis ছাড়া
Wrapper ফল ফেরত দেয় না	মোড়ানো Function `undefined` দেয়	`return fn(...args)`
Wrapper Argument বাদ দেয়	কিছু Argument হারায়	`...args`
Impure Function Memoize	বাসি বা ভুল ফল	শুধু Pure Function
সীমাহীন Cache	Memory ধীরে ধীরে বাড়ে	সীমা বা মুছে ফেলা
Default Parameter-এর Function Curry	আগেই চলে যায়	Argument-এর সংখ্যা নিজে দিন
Argument বদলানো "Pure" Function	অন্য জায়গায় অপ্রত্যাশিত পরিবর্তন	নতুন মান ফেরত দিন
অতিরিক্ত দীর্ঘ Pipeline	ত্রুটির ধাপ খুঁজে পাওয়া কঠিন	নামযুক্ত মধ্যবর্তী ধাপ
সব কিছু Functional করার জেদ	অস্পষ্ট Code	সরল `for` Loop-ও চলে, যদি পরিষ্কার হয়

১৪.১১ Debugging নির্দেশিকা

- **Pipeline-এর মাঝে দেখুন।** একটি ছোট `tap` Function বসান: `const tap = (label) => (value) => { console.log(label, value); return value; };` তারপর `pipe(trim, tap('trim-এর পর'), lower, ...)`। এটি মান না বদলে শুধু দেখায়।
- **Closure-এর অবস্থা দেখতে** DevTools-এ Function-এর ভেতরে Breakpoint দিন; Scope Panel-এ "Closure" অংশে মনে রাখা Variable দেখা যায়।
- **Stack-এ Anonymous Function** পড়া কঠিন; Wrapper-এর ভেতরের Function-কে নাম দিন, বা মূল Function-এর নাম (`fn.name`) Log করুন।
- **Memoized Function ভুল ফল দিচ্ছে?** Cache মুছে আবার চালান; ঠিক হলে সমস্যা Cache-এর Key বা Impure Function-এ।

- **Currying-এ "is not a function"** মানে সব Argument আসার আগেই আসল Function চলে গেছে, বা উল্টো; `fn.length` দেখুন।
- **Timer-নির্ভর Test মাঝে মাঝে ব্যর্থ হয়?** আসল সময়ের ওপর নির্ভর না করে Fake Timer ব্যবহার করুন।

১৪.১২ পেশাদার অভ্যাস

- **ছোট, এক কাজের, নামযুক্ত Function;** নাম পড়েই Pipeline বোঝা যায়।
- **Pure কেন্দ্র, Side Effect প্রান্তে।**
- **ইনপুট কখনো বদলাবেন না;** নতুন মান ফেরত দিন, প্রয়োজনে `Object.freeze`।
- **Wrapper সব সময় `...args` নেবে এবং ফল ফেরত দেবে।**
- **Memoize শুধু মাপা ব্যয়বহুল Pure কাজে।**
- **Debounce ব্যবহারকারীর লেখায়, Throttle ঘন ঘটনায়।**
- **Functional ও Object-Oriented মেশাতে দ্বিধা করবেন না;** যেটি সমস্যাকে স্পষ্ট করে সেটিই।
- **পরিচিত Library-র নাম ও আচরণ মানুন** (`pipe`, `compose`, `debounce`), যাতে অন্য প্রোগ্রামাররা সহজে বোঝেন।

১৪.১৩ Project: Sales Function Lab

একটি দোকানের বিক্রির তালিকা থেকে Report তৈরি: অবৈধ সারি বাদ দিয়ে কারণসহ তালিকা, মোট আয়, বিভাগভিত্তিক আয়, এবং শীর্ষ বিক্রি। পুরো হিসাব Pure Function-এর একটি Pipeline; Side Effect শুধু Report ছাপার File-এ। সঙ্গে একটি ছোট Functional সরঞ্জামের Module, যার Timer-নির্ভর অংশ Fake Timer দিয়ে পরীক্ষিত।

Project-এর চাহিদা

- প্রতিটি বিক্রি: `{ id, category, quantity, unitPrice }`; `quantity` ধনাত্মক পূর্ণসংখ্যা, `unitPrice` অঋণাত্মক সংখ্যা, `id` ও `category` খালি নয়।
- অবৈধ সারি Report থামাবে না; বাদ পড়া সারির নম্বর ও কারণ জানাবে।
- মোট আয়; বিভাগভিত্তিক সংখ্যা ও আয়, বেশি আয় আগে; শীর্ষ ৩ বিক্রি।
- ইনপুট তালিকা কখনো বদলাবে না; তৈরি হওয়া প্রতিটি সারি অপরিবর্তনীয়।
- টাকা দেখানোর Formatter একবারই তৈরি হবে, পরে পুনর্ব্যবহৃত (Memoization)।

File-এর কাঠামো

Folder-এর কাঠামো

```
sales-function-lab/
├── src/
│   ├── fp.mjs          pipe, compose, curry, memoize, once, debounce, throttle
│   ├── sales.mjs       বিক্রির নিয়ম ও Report-এর Pipeline
│   └── format.mjs      টাকা দেখানো (Memoized Formatter)
├── data/
│   └── sales.mjs       নমুনা তথ্য
├── report.mjs          Report ছাপা (Side Effect এখানে)
├── README.md
└── tests/
    ├── fp.test.mjs
    └── sales.test.mjs
```

src/fp.mjs

দুই অংশ মিলে সম্পূর্ণ File।

fp.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```js
export const pipe = (...fns) => (input) =>
  fns.reduce((value, fn) => fn(value), input);

export const compose = (...fns) => pipe(...fns.toReversed());

export function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...more) => curried(...args, ...more);
  };
}

export function memoize(fn, keyOf = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  const memoized = (...args) => {
    const key = keyOf(...args);
    if (!cache.has(key)) {
      cache.set(key, fn(...args));
    }
    return cache.get(key);
  };
  memoized.cacheSize = () => cache.size;
  return memoized;
}
```

```
// ...
```

fp.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export function once(fn) {
  let done = false;
  let result;
  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
    }
    return result;
  };
}

export function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

export function throttle(fn, ms) {
  let last = -Infinity;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}
```

এই Module-এর প্রতিটি Function অধ্যায়ের কোনো না কোনো অনুচ্ছেদ থেকে এসেছে, একটি সাধারণ রূপে। `memoize` এখন যেকোনো সংখ্যক Argument নেয়, এবং Key তৈরির নিয়ম (`keyOf`) বাইরে থেকে বদলানো যায়; Default নিয়ম সব Argument-কে JSON লেখায় রূপান্তর করে। `cacheSize` Function-এর একটি Property: Function-ও Object, তাই তাতে Property যোগ করা যায়, এবং Test এটি দিয়ে যাচাই করে Cache কাজ করছে কি না।

src/sales.mjs

দুই অংশ মিলে সম্পূর্ণ File।

sales.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Browser Console ও Node.js

```
import { curry, pipe } from './fp.mjs';

export class SaleError extends Error {
  constructor(message) {
    super(message);
    this.name = 'SaleError';
  }
}

// একটি সারি যাচাই: ঠিক থাকলে অপরিবর্তনীয় নতুন Object, নইলে SaleError।
export function validateSale(row) {
  const { id, category, quantity, unitPrice } = row ?? {};

  if (typeof id !== 'string' || id.trim() === '') {
    throw new SaleError('id নেই');
  }
  if (typeof category !== 'string' || category.trim() === '') {
    throw new SaleError('বিভাগ নেই');
  }
  if (!Number.isInteger(quantity) || quantity < 1) {
    throw new SaleError('পরিমাণ 1 বা বেশি পূর্ণসংখ্যা হতে হবে');
  }
  if (!Number.isFinite(unitPrice) || unitPrice < 0) {
    throw new SaleError('দাম অঋণাত্মক সংখ্যা হতে হবে');
  }
  return Object.freeze({
    id: id.trim(), category: category.trim(), quantity, unitPrice,
  });
}

export const withTotal = (sale) =>
  Object.freeze({ ...sale, total: sale.quantity * sale.unitPrice });

export const inCategory = curry((category, sale) => sale.category === category);
export const atLeast = curry((minimum, sale) => sale.total >= minimum);
export const sumBy = curry((field, rows) =>
  rows.reduce((sum, row) => sum + row[field], 0));

// ...
```

sales.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

// বৈধ সারি রাখে; অবৈধগুলোর কারণ আলাদা তালিকায়। ইনপুট বদলায় না।
export function partitionValid(rows) {
  const valid = [];
```

```
  const rejected = [];

  for (const [index, row] of rows.entries()) {
    try {
      valid.push(withTotal(validateSale(row)));
    } catch (error) {
      if (!(error instanceof SaleError)) {
        throw error;
      }
      rejected.push(`সারি ${index + 1}: ${error.message}`);
    }
  }
  return { valid, rejected };
}

export function totalsByCategory(sales) {
  return Object.entries(Object.groupBy(sales, (sale) => sale.category))
    .map(([category, rows]) => ({
      category, count: rows.length, revenue: sumBy('total', rows),
    }))
    .toSorted((a, b) =>
      b.revenue - a.revenue || a.category.localeCompare(b.category));
}

const byTotalDesc = (a, b) => b.total - a.total || a.id.localeCompare(b.id);

export const topSales = curry((count, sales) =>
  sales.toSorted(byTotalDesc).slice(0, count));

export const salesReport = pipe(
  partitionValid,
  ({ valid, rejected }) => ({
    revenue: sumBy('total', valid),
    byCategory: totalsByCategory(valid),
    top: topSales(3, valid).map((sale) => sale.id),
    rejected,
  }),
);
```

- **`validateSale` সীমানার পাহারাদার** (Trust Boundary): বাইরের সারি যাচাই করে একটি নতুন, `Object.freeze` করা Object দেয়। এরপরের প্রতিটি Function নিশ্চিন্তে ধরে নিতে পারে তথ্য বৈধ।
- **`withTotal`** নতুন Object তৈরি করে `total` যোগ করে; মূলটি অপরিবর্তিত।
- **Curried সাহায্যকারী:** `inCategory('Books')`, `atLeast(1000)`, `sumBy('total')` —প্রতিটি `filter`, `reduce` বা Pipeline-এ সরাসরি বসানো যায়।
- **`partitionValid`-এর ভেতরে `push`** Side Effect নয়: `valid` ও `rejected` এই Function-এর নিজের নতুন Array, বাইরে থেকে দেখা যায় না। একটি অবৈধ সারি পুরো Report থামায় না, কিন্তু অপ্রত্যাশিত ত্রুটি (Bug) চাপা পড়ে

না (অধ্যায় ১১)।

- `toSorted` (অধ্যায় ১২) ইনপুট না বদলে সাজায়; সমান আয়ে দ্বিতীয় স্তরে নাম ধরে সাজানো, যাতে ফল সব সময় নিশ্চিত ক্রমে আসে।
- `salesReport` **একটি Pipeline:** প্রথম ধাপ সারি পরিষ্কার করে, দ্বিতীয় ধাপ Report গঠন করে। নতুন একটি হিসাব যোগ করতে শুধু দ্বিতীয় ধাপে একটি লাইন।

src/format.mjs, নমুনা তথ্য এবং report.mjs

format.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { memoize } from './fp.mjs';

// Intl.NumberFormat তৈরি তুলনামূলক ব্যয়বহুল; প্রতিটি দশমিক-সংখ্যার জন্য একবারই।
export const formatterFor = memoize((decimals) =>
  new Intl.NumberFormat('en-US', {
    minimumFractionDigits: decimals,
    maximumFractionDigits: decimals,
  }));

export const formatTaka = (amount, decimals = 0) =>
  `${formatterFor(decimals).format(amount)} টাকা`;
```

sales.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export const SALES = [
  { id: 'S1', category: 'Books', quantity: 3, unitPrice: 450 },
  { id: 'S2', category: 'Electronics', quantity: 1, unitPrice: 2800 },
  { id: 'S3', category: 'Books', quantity: 1, unitPrice: 620 },
  { id: 'S4', category: 'Stationery', quantity: 10, unitPrice: 25 },
  { id: 'S5', category: 'Electronics', quantity: 2, unitPrice: 950 },
  { id: 'S6', category: 'Books', quantity: 0, unitPrice: 300 },
  { id: 'S7', category: ' ', quantity: 1, unitPrice: 100 },
  { id: 'S8', category: 'Stationery', quantity: 4, unitPrice: 120 },
];
```

report.mjs · JavaScript — চলবে: Node.js

```
import { SALES } from './data/sales.mjs';
import {
  atLeast, inCategory, partitionValid, salesReport,
} from './src/sales.mjs';
import { formatTaka, formatterFor } from './src/format.mjs';

const report = salesReport(SALES);

console.log(`মোট আয়: ${formatTaka(report.revenue)}`);
for (const { category, count, revenue } of report.byCategory) {
```

```
  console.log(`${category}: ${count}টি বিক্রি, ${formatTaka(revenue)}`);
}
console.log(`শীর্ষ ৩: ${report.top.join(', ')}`);
for (const reason of report.rejected) {
  console.log(`বাদ: ${reason}`);
}

const { valid } = partitionValid(SALES);
const bigBooks = valid.filter(inCategory('Books')).filter(atLeast(1000));
console.log(`১০০০+ টাকার বই: ${bigBooks.map((sale) => sale.id).join(', ')}`);
console.log(formatTaka(1234.5, 2), formatterFor.cacheSize(), SALES[5].quantity);
```

Output

```
মোট আয়: 7,400 টাকা
Electronics: 2টি বিক্রি, 4,700 টাকা
Books: 2টি বিক্রি, 1,970 টাকা
Stationery: 2টি বিক্রি, 730 টাকা
শীর্ষ ৩: S2, S5, S1
বাদ: সারি 6: পরিমাণ 1 বা বেশি পূর্ণসংখ্যা হতে হবে
বাদ: সারি 7: বিভাগ নেই
১০০০+ টাকার বই: S1
1,234.50 টাকা 2 0
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Side Effect একটি File-এ।** `report.mjs`-ই একমাত্র জায়গা যেখানে `console.log`; `src/`-এর সব Function Pure (Formatter-এর Cache ছাড়া, যা বাইরে থেকে অদৃশ্য এবং ফলে প্রভাব ফেলে না)। তাই Report-এর প্রতিটি সংখ্যা Test-এ সরাসরি যাচাই করা যায়।
- **ছয়টি বৈধ বিক্রি থেকে ৭,৪০০ টাকা;** দুটি সারি কারণসহ বাদ। সারির নম্বর মানুষের গণনায় (১ থেকে), Index-এ নয়।
- **বিভাগের তিনটিরই ২টি করে বিক্রি,** তাই আয়ই ক্রম ঠিক করেছে।
- **Curried Predicate-এর পুনর্ব্যবহার:** `inCategory('Books')` ও `atLeast(1000)` একই `filter` শৃঙ্খলে, পড়তে প্রায় সাধারণ বাক্যের মতো।
- **শেষ লাইন তিনটি প্রমাণ দেয়:** দশমিকসহ টাকা দেখানো; Formatter Cache-এ মাত্র দুটি (০ ও ২ দশমিক), যদিও `formatTaka` পাঁচবার ডাকা হয়েছে; এবং মূল তথ্যের অবৈধ সারির পরিমাণ এখনো `0`, অর্থাৎ ইনপুট কেউ বদলায়নি।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node report.mjs
node --test
```

Call	যা হওয়ার কথা
`validateSale({ id: 'X', category: 'A', quantity: 1.5, unitPrice: 10 })`	`SaleError` : পরিমাণের বার্তা
`salesReport([])`	আয় 0; ফাঁকা বিভাগ, শীর্ষ ও বাদের তালিকা
`topSales(2)(sales)` এবং `topSales(2, sales)`	একই ফল (Currying)
`compose(f, g)(x)`	`pipe(g, f)(x)` -এর সমান
`debounce(fn, 300)` চারবার দ্রুত	৩০০ মিলিসেকেন্ড পরে একবার, শেষ Argument দিয়ে
`throttle(fn, 100)` প্রতি ১০ মিলিসেকেন্ডে ৫০ বার	প্রতি ১০০ মিলিসেকেন্ডে একবার

Companion Source Code-এ ২২টি Test আছে। Debounce ও Throttle-এর Test Node.js-এর `mock.timers` দিয়ে সময়কে হাতে এগিয়ে নেয়, তাই এক সেকেন্ডের পরিস্থিতিও তাৎক্ষণিক ও নিশ্চিতভাবে পরীক্ষিত। Purity-র Test ইনপুট `Object.freeze` করে দেখে কোনো Function তা বদলানোর চেষ্টা করে কি না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একটি অবৈধ সারিতে পুরো Report থেমে যায়	যাচাইয়ের ত্রুটি ধরা হয়নি	`partitionValid`
Report চালানোর পর মূল তথ্য বদলে গেছে	কোথাও `sort` বা সরাসরি Assignment	`toSorted` , নতুন Object
সমান আয়ের বিভাগের ক্রম অনিশ্চিত	দ্বিতীয় স্তরের সাজানো নেই	`localeCompare`
`sumBy(...) is not a function`	সব Argument একবারে দেওয়ায় ফল পাওয়া গেছে, তারপর আবার ডাকা	Curried Function-এর ব্যবহার
Formatter-এর Cache বড় হয়ে যায়	Key-তে অপ্রয়োজনীয় মান	`keyOf`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Code-এর ভেতরে, File বা Network থেকে আসে না; টাকা সাধারণ সংখ্যায়, পয়সার হিসাব নেই; Memoized Formatter-এর Cache কখনো খালি হয় না, যা এখানে সমস্যা নয় কারণ দশমিক-সংখ্যা মাত্র কয়েকটি।

১৪.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `makeGreeting(greeting)` লিখুন, যা একটি Function ফেরত দেয়; সেটি নাম নিয়ে পূর্ণ শুভেচ্ছা দেয়।

২. `var` দিয়ে লেখা Loop-এ তৈরি Function-গুলোর Output না চালিয়ে বলুন, তারপর `let` দিয়ে ঠিক করুন।

৩. `twice(fn)` লিখুন, যা একটি নতুন Function দেয়, যেটি `fn`-কে পরপর দুবার প্রয়োগ করে: `twice(double)(3)` হবে `12`।

৪. নিচের কোনটি Pure? `(a) => a * 2`, `(a) => a * Math.random()`, `(list) => list.push(1)`। কারণ বলুন।

৫. `bind` দিয়ে `calcTax(percent, amount)` থেকে `vat15` তৈরি করুন।

মধ্যম স্তর

৬. `pipe` দিয়ে নামের তালিকা পরিষ্কার করুন: আগে-পরের ফাঁকা জায়গা বাদ, প্রথম অক্ষর বড়, তারপর কমা দিয়ে জোড়া।

৭. `between(min, max, value)` Curry করে `filter`-এ ব্যবহার করুন।

৮. `memoize` দিয়ে একটি ধীর Function-এর কতবার আসল হিসাব হলো তা গুনে দেখান।

৯. `once` দিয়ে "স্বাগত Email" পাঠানোর Function লিখুন, যা তিনবার ডাকলেও একবারই পাঠায়।

চ্যালেঞ্জ স্তর

১০. `pipe` ব্যবহার করে `compose` লিখুন, এবং একই তিনটি Function-এ দুটির ক্রমের পার্থক্য দেখান।

১১. Closure দিয়ে একটি Rate Limiter লিখুন: একটি সময়-সীমার মধ্যে সর্বোচ্চ `n` বার অনুমতি দেয়। সময় একটি `clock` Function থেকে নিন, যাতে আসল সময় ছাড়াই পরীক্ষা করা যায়।

১২. Project-এ বিভাগপ্রতি গড় বিক্রি যোগ করুন: একটি Curried `averageBy(field)`, এবং প্রতিটি বিভাগের গড় `total`।

১৪.১৫ অনুশীলনীর সমাধান

সমাধান ১

greeting.js · JavaScript — চলবে: Browser Console ও Node.js

```
const makeGreeting = (greeting) => (name) => `${greeting}, ${name}!`;

const welcome = makeGreeting('স্বাগতম');
const morning = makeGreeting('শুভ সকাল');

console.log(welcome('Sara'));
console.log(morning('Rafi'));
```

Output

```
স্বাগতম, Sara!
শুভ সকাল, Rafi!
```

সমাধান ২ — `var`-এর Output `3, 3, 3`: সব Function একই `i` দেখে, যার মান Loop শেষে `3`। `let` দিলে প্রতিটি পুনরাবৃত্তির নিজস্ব Variable, ফল `0, 1, 2` (অনুচ্ছেদ ১৪.২-এর উদাহরণ)।

সমাধান ৩

twice.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const twice = (fn) => (value) => fn(fn(value));

const double = (n) => n * 2;
const exclaim = (text) => `${text}!`;

console.log(twice(double)(3));
console.log(twice(exclaim)('বাহ'));
```

Output

```
12
বাহ!!
```

সমাধান ৪ — শুধু `(a) => a * 2` Pure। `Math.random()` থাকায় দ্বিতীয়টি একই ইনপুটে ভিন্ন ফল দেয়; তৃতীয়টি তার Argument বদলায় (Side Effect), এবং ফেরত দেয় নতুন দৈর্ঘ্য, তালিকা নয়। Pure রূপ: `(list) => [...list, 1]`।

সমাধান ৫

vat-bind.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const calcTax = (percent, amount) => (amount * percent) / 100;
const vat15 = calcTax.bind(null, 15);

console.log(vat15(1000), vat15(240));
```

Output

```
150 36
```

সমাধান ৬

clean-names.js · JavaScript — চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);

const trimAll = (names) => names.map((name) => name.trim());
const capitalize = (names) =>
  names.map((name) => name[0].toUpperCase() + name.slice(1));
const joinWithComma = (names) => names.join(', ');

const cleanNames = pipe(trimAll, capitalize, joinWithComma);
console.log(cleanNames(['  sara ', 'rafi', ' mina']));
```

Output

```
Sara, Rafi, Mina
```

সমাধান ৭

between.js · JavaScript — চলবে: Browser Console ও Node.js

```
const curry = (fn) => function curried(...args) {
  if (args.length >= fn.length) {
    return fn(...args);
  }
  return (...more) => curried(...args, ...more);
};

const between = curry((min, max, value) => value >= min && value <= max);
const isPassMark = between(33, 100);

console.log([12, 45, 88, 101, 33].filter(isPassMark).join(', '));
console.log(between(1)(10)(5), between(1, 10)(50));
```

Output

```
45, 88, 33
true false
```

Argument একসঙ্গে বা এক এক করে—দুভাবেই চলে।

সমাধান ৮

count-calls.js · JavaScript চলবে: Browser Console ও Node.js

```
function memoize(fn) {
  const cache = new Map();
  return (key) => {
    if (!cache.has(key)) {
      cache.set(key, fn(key));
    }
    return cache.get(key);
  };
}

let realCalls = 0;
const shippingCost = memoize((city) => {
  realCalls += 1;
  return city === 'Dhaka' ? 60 : 120;
});

const orders = ['Dhaka', 'Bogura', 'Dhaka', 'Dhaka', 'Bogura'];
console.log(orders.map(shippingCost).join(', '));
console.log(`আসল হিসাব: ${realCalls} বার`);
```

Output

```
60, 120, 60, 60, 120
আসল হিসাব: 2 বার
```

পাঁচটি Order, কিন্তু দুটি শহর; তাই আসল হিসাব দুবার।

সমাধান ৯

welcome-once.js · JavaScript চলবে: Browser Console ও Node.js

```
function once(fn) {
  let done = false;
  let result;
  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
    }
    return result;
  };
}

const sendWelcome = once((email) => {
```

```
  console.log(`স্বাগত Email পাঠানো হলো: ${email}`);
  return true;
});

sendWelcome('sara@example.com');
sendWelcome('sara@example.com');
console.log(sendWelcome('other@example.com'));
```

Output

```
স্বাগত Email পাঠানো হলো: sara@example.com
true
```

তৃতীয় ডাকে নতুন ঠিকানা দিলেও কিছু পাঠানো হয়নি; প্রথম ফল ফেরত এসেছে। বাস্তবে "প্রতি ব্যবহারকারী একবার" চাইলে ঠিকানা ধরে Memoize করাই উপযুক্ত।

সমাধান ১০

compose.js · JavaScript — চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);
const compose = (...fns) => pipe(...fns.toReversed());

const add10 = (n) => n + 10;
const double = (n) => n * 2;
const square = (n) => n * n;

console.log(pipe(add10, double, square)(1));
console.log(compose(add10, double, square)(1));
```

Output

```
484
12
```

`pipe`: (১ + ১০) × ২ = ২২, তারপর বর্গ ৪৮৪। `compose` ডান থেকে: ১-এর বর্গ ১, × ২ = ২, + ১০ = ১২।

সমাধান ১১

rate-limiter.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createRateLimiter(limit, windowMs, clock = Date.now) {
  let times = [];

  return () => {
    const now = clock();
```

```
    times = times.filter((time) => now - time < windowMs);
    if (times.length >= limit) {
      return false;
    }
    times = [...times, now];
    return true;
  };
}

let fakeNow = 0;
const allow = createRateLimiter(2, 1000, () => fakeNow);

const results = [allow(), allow(), allow()];
fakeNow = 1000;
results.push(allow());
console.log(results.join(', '));
```

Output

```
true, true, false, true
```

প্রথম সেকেন্ডে দুবার অনুমতি, তৃতীয়টি প্রত্যাখ্যাত; ১০০০ মিলিসেকেন্ড পরে পুরোনো দুটি সময়-সীমার বাইরে চলে যায়, তাই আবার অনুমতি। `clock` Inject করায় এক সেকেন্ড অপেক্ষা না করেই পরীক্ষা হয়েছে, Project-এর Fake Timer-এর মতোই।

সমাধান ১২

average.mjs · JavaScript — চলবে: Node.js

```
import { curry } from '../../project/sales-function-lab/src/fp.mjs';
import {
  partitionValid, sumBy,
} from '../../project/sales-function-lab/src/sales.mjs';
import { SALES } from '../../project/sales-function-lab/data/sales.mjs';

const averageBy = curry((field, rows) =>
  (rows.length === 0 ? 0 : Math.round(sumBy(field, rows) / rows.length)));

const { valid } = partitionValid(SALES);
const groups = Object.groupBy(valid, (sale) => sale.category);

for (const [category, rows] of Object.entries(groups)) {
  console.log(`${category}: গড় ${averageBy('total', rows)}`);
}
```

Output

```
Books: গড় 985
Electronics: গড় 2350
Stationery: গড় 365
```

ফাঁকা তালিকায় শূন্য দিয়ে ভাগ এড়াতে Guard রাখা হয়েছে; পুরোনো কোনো Function বদলাতে হয়নি।

১৪.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. "Function একটি First-Class Value"—এর অর্থ কোনটি?

ক) Function সবার আগে চলে

খ) Function-কে অন্য মানের মতো রাখা, পাঠানো ও ফেরত দেওয়া যায়

গ) Function শুধু Class-এর ভেতরে থাকে

ঘ) Function-এর নাম বাধ্যতামূলক

প্রশ্ন ২. `var` দিয়ে `for` Loop-এ তিনটি Function তৈরি করে পরে ডাকলে, প্রতিটি `i` হিসেবে কী দেখে?

ক) Loop শেষের মান

খ) নিজের পুনরাবৃত্তির মান

গ) `undefined`

ঘ) Error

প্রশ্ন ৩. `fn.bind(obj, 5)` কী ফেরত দেয়?

ক) `fn`-এর ফল

খ) `obj`

গ) একটি নতুন Function, যার `this` `obj` এবং প্রথম Argument `5`

ঘ) `undefined`

প্রশ্ন ৪. `pipe(f, g, h)(x)` কীসের সমান?

ক) `f(g(h(x)))`

খ) `h(g(f(x)))`

গ) `[f(x), g(x), h(x)]`

ঘ) `f(x) + g(x) + h(x)`

প্রশ্ন ৫. `add(1)(2)` এবং `add(1, 2)` দুটিই কাজ করে। `add` কোন কৌশলে তৈরি?

ক) Memoization

খ) Debounce

গ) Currying

ঘ) Closure ছাড়া সাধারণ Function

প্রশ্ন ৬. কোনটি Pure Function?

ক) `(price) => price * (1 + taxRate)`, যেখানে `taxRate` বাইরের Variable

খ) `(list) => list.sort()`

গ) `() => Date.now()`

ঘ) `(price, rate) => price * (1 + rate)`

প্রশ্ন ৭. কোন Function Memoize করা নিরাপদ নয়?

ক) Network থেকে সর্বশেষ দাম আনা

খ) একটি সংখ্যার বর্গ

গ) Fibonacci সংখ্যা

ঘ) একটি শহরের নির্দিষ্ট Delivery Fee

প্রশ্ন ৮. Scroll চলাকালীন প্রতি ২০০ মিলিসেকেন্ডে সর্বোচ্চ একবার অবস্থান হালনাগাদ করতে কোনটি?

ক) Debounce

খ) Throttle

গ) Memoize

ঘ) Curry

প্রশ্ন ৯. `once(fn)`-এর তৈরি Function দ্বিতীয়বার ডাকলে কী হয়?

ক) `fn` আবার চলে

খ) Error

গ) `undefined`

ঘ) `fn` চলে না; প্রথম ফল ফেরত আসে

প্রশ্ন ১০. Debounce-এর Test আসল সময়ের অপেক্ষা ছাড়া নিশ্চিতভাবে করার উপায়?

ক) `setTimeout`-এর সময় বাড়ানো

খ) Test একাধিকবার চালানো

গ) Fake Timer দিয়ে সময় হাতে এগিয়ে নেওয়া

ঘ) Debounce-এর Test সম্ভব নয়

১৪.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–খ · ৫–গ · ৬–ঘ · ৭–ক · ৮–খ · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Function সংখ্যা বা লেখার মতোই একটি মান।
২	ক	`var`-এর একটিই Variable; সব Function তার শেষ মান দেখে।
৩	গ	`bind` ডাকে না; `this` ও কিছু Argument স্থির করা নতুন Function দেয়।
৪	খ	`pipe` বাঁ থেকে ডানে: প্রথমে `f`, শেষে `h`।
৫	গ	Argument এক এক করে বা একসঙ্গে দেওয়া যায়।
৬	ঘ	সব ইনপুট Argument-এ, বাইরের কিছু পড়ে বা বদলায় না; `sort` তালিকা বদলায়।
৭	ক	ফল সময়ের সঙ্গে বদলায়; Cache বাসি দাম দেবে।
৮	খ	ঘটনা চলাকালীন নিয়মিত বিরতিতে কাজ: Throttle।
৯	ঘ	`done` Closure-এ মনে রাখে কাজ হয়ে গেছে।
১০	গ	Fake Timer সময়কে নিয়ন্ত্রিত ও তাৎক্ষণিক করে।

১৪.১৮ অধ্যায়ের সারসংক্ষেপ

- Function একটি মান: রাখা, পাঠানো, ফেরত দেওয়া যায়; Function Factory নির্দিষ্ট কাজের Function তৈরি করে।
- Closure অবস্থা মনে রাখে এবং বাইরে থেকে সুরক্ষিত রাখে; Loop-এ `var` ফাঁদ, `let` সমাধান।
- Higher-Order Function দিয়ে Function মোড়ানো: `once`, `logged`; Wrapper `...args` নেয় ও ফল ফেরত দেয়।
- `call` ও `apply` এখনই ডাকে, `bind` নতুন Function দেয়; `bind` দিয়ে Partial Application।
- `pipe` বাঁ থেকে ডানে, `compose` ডান থেকে বাঁয়ে ছোট Function জুড়ে বড় কাজ।
- Currying বহু-Argument-এর Function-কে Pipeline ও `filter`-এর উপযোগী করে।
- Pure Function: একই ইনপুটে একই ফল, কোনো Side Effect নয়; Side Effect প্রান্তে।
- Memoization Pure ও ব্যয়বহুল কাজে; Cache-এর Memory ও Key-র দিকে নজর।
- Debounce ঘটনা থামার পর, Throttle চলাকালীন নির্দিষ্ট বিরতিতে; Timer-নির্ভর Code Fake Timer দিয়ে Test।

পরের অধ্যায়ে Browser API: URL, Web Storage, Fetch, File ও Blob—Browser যে সুবিধাগুলো দেয়, এবং বাস্তব Web Application-এ সেগুলোর নিরাপদ ব্যবহার।

১৪.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
First-Class Function	মান হিসেবে ব্যবহারযোগ্য Function।
Function Factory	নতুন Function তৈরি করে ফেরত দেওয়া Function।
Closure	তৈরির জায়গার Variable মনে রাখা Function।
Higher-Order Function	Function নেয় বা ফেরত দেয় এমন Function।
Wrapper	মূল Function-কে মুড়ে নতুন আচরণ যোগ করা Function।
Partial Application	কিছু Argument আগে বসিয়ে নতুন Function।
Currying	Argument এক এক করে নেওয়ার রূপান্তর।
Composition	ছোট Function জুড়ে বড় কাজ (`pipe`, `compose`)।
Pure Function	একই ইনপুটে একই ফল, কোনো Side Effect নেই।
Side Effect	Function-এর বাইরের জগতে পরিবর্তন বা যোগাযোগ।
Memoization	ফল মনে রেখে পুনরায় হিসাব এড়ানো।
Throttle	নির্দিষ্ট সময়ে সর্বোচ্চ একবার চালানো।

১৪.২০ আরও পড়ুন

- **MDN — Closures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures` — Closure-এর বিস্তারিত ও প্রচলিত ভুল।
- **MDN — Function.prototype.call():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call` — `call`, `apply`, `bind`।
- **MDN — Map:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map` — Key-Value সংগ্রহ।
- **Wikipedia — Pure function:** `https://en.wikipedia.org/wiki/Pure_function` — Pure Function-এর সংজ্ঞা ও উদাহরণ।
- **Node.js — Mocking timers in tests:** `https://nodejs.org/api/test.html#class-mocktimers` — `mock.timers` দিয়ে Timer-নির্ভর Test।

অধ্যায় ১৫

JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- JavaScript ভাষা এবং Browser-এর দেওয়া API-র পার্থক্য করতে, এবং Feature Detection করতে।
- `URL` ও `URLSearchParams` দিয়ে ঠিকানা নিরাপদে পড়তে, গড়তে ও যাচাই করতে।
- `localStorage` ও `sessionStorage`-এর পার্থক্য, সীমা ও ত্রুটি সামলাতে, এবং `storage` Event দিয়ে একাধিক Tab সমন্বয় করতে।
- সংরক্ষিত তথ্যে Version রেখে পুরোনো তথ্য নতুন গঠনে রূপান্তর (Migration) করতে।
- `Request`, `Response` ও `fetch` দিয়ে JSON পাঠাতে ও আনতে।
- History API দিয়ে Page-এর অবস্থা URL-এ রাখতে, যাতে Link শেয়ার ও Reload-এ অবস্থা টিকে থাকে।
- `Blob` ও `File` দিয়ে File তৈরি, Download এবং ব্যবহারকারীর দেওয়া File পড়তে।
- Browser-এ তথ্য রাখার নিরাপত্তা ও গোপনীয়তার সীমা বুঝতে।
- একটি সম্পূর্ণ **Bookmark Vault** তৈরি করতে এবং আসল Browser-এ তার প্রতিটি API পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৯: DOM, Event Delegation, `localStorage`-এর প্রাথমিক ব্যবহার।
- অধ্যায় ১০: `fetch`, Promise, `AbortController`।
- অধ্যায় ১১: প্রত্যাশিত ত্রুটি ও Trust Boundary।
- অধ্যায় ১২: ES Module ও Local Server।

এই অধ্যায় কেন গুরুত্বপূর্ণ

JavaScript ভাষাটি নিজে শুধু হিসাব, তথ্য ও নিয়ন্ত্রণ জানে; Page দেখানো, ঠিকানা পড়া, তথ্য সংরক্ষণ বা File নামানো—এর কোনোটিই ভাষার অংশ নয়। এগুলো দেয় Browser, তার API-র মাধ্যমে। একটি বাস্তব Web Application-এর বেশির ভাগ কাজই এই API-গুলোর ব্যবহার: খোঁজার শব্দ URL-এ রাখা যাতে Link শেয়ার করা যায়, ব্যবহারকারীর পছন্দ মনে রাখা, অর্ধেক লেখা Form হারিয়ে না যাওয়া, তথ্য File হিসেবে নামানো ও আবার তোলা।

প্রতিটি API-র নিজস্ব সীমা ও ফাঁদ আছে: Storage ভর্তি হয়ে যায়, URL-এ বিপজ্জনক Protocol লুকিয়ে থাকে, ব্যবহারকারী যেকোনো File দিতে পারেন। এই অধ্যায়ে আপনি API-গুলো শিখবেন তাদের ত্রুটির পথসহ, এবং Project-এ প্রতিটি API আসল Browser-এ পরীক্ষিত হবে।

১৫.১ ভাষা এবং পরিবেশ

একই JavaScript ভাষা Browser ও Node.js দুই জায়গায় চলে, কিন্তু প্রতিটি পরিবেশ নিজের API যোগ করে। Browser-এ দেখুন:

browser-env.js · JavaScript — চলবে: Browser

```
console.log(typeof window, typeof document, location.protocol);
console.log('localStorage' in window, isSecureContext);
console.log(typeof URL, typeof fetch);
```

Output

```
object object http:
true true
function function
```

একই প্রশ্ন Node.js-এ:

node-env.js · JavaScript চলবে: Node.js

```
console.log(typeof window, typeof document, typeof process);
console.log(typeof URL, typeof fetch);
```

Output

```
undefined undefined object
function function
```

`window`, `document`, `location`, `localStorage` শুধু Browser-এর; `process` শুধু Node.js-এর। `URL` ও `fetch` দুই জায়গাতেই আছে, কারণ উভয় পরিবেশ একই মানদণ্ড অনুসরণ করে। তাই একটি Module-এ শুধু `URL`, `fetch`, `JSON`-এর মতো সাধারণ API ব্যবহার করলে সেটি দুই জায়গাতেই চলে এবং Node.js-এ Test করা যায় (অধ্যায় ১২-এর Domain স্তর)।

Feature Detection: একটি API ব্যবহারের আগে দেখে নিন সেটি আছে কি না, যেমন `if ('share' in navigator)`। Browser-এর নাম বা সংস্করণ দেখে অনুমান করবেন না; একই Browser-এর ভিন্ন সংস্করণে, বা নিরাপত্তা-সেটিংসে, API থাকতে বা না থাকতে পারে। কিছু API শুধু **Secure Context**-এ পাওয়া যায়, অর্থাৎ `https://` বা `localhost`-এ; `isSecureContext` তা জানায়।

১৫.২ URL এবং URLSearchParams

URL হাতে লেখা String জোড়া দিয়ে বানানো বা ভাঙা ভুলের উৎস: `?` আর `&` গুলিয়ে যায়, বিশেষ অক্ষর Encode হয় না। `URL` Object এ কাজ নির্ভুলভাবে করে।

url.js · JavaScript চলবে: Browser Console ও Node.js

```
const address =
  'https://shop.example.com:8080/products/list?category=books&page=2#top';
const url = new URL(address);

console.log(url.hostname, url.port, url.pathname);
const { searchParams } = url;
console.log(searchParams.get('category'), searchParams.get('page'), url.hash);

url.searchParams.set('page', '3');
url.searchParams.append('tag', 'বই');
console.log(url.search);

const relative = new URL('../about', 'https://example.com/docs/intro/');
console.log(relative.href);

try {
  new URL('not a url');
} catch (error) {
```

```
  console.log(error.name);
}
```

Output

```
shop.example.com 8080 /products/list
books 2 #top
?category=books&page=3&tag=%E0%A6%AC%E0%A6%87
https://example.com/docs/about
TypeError
```

- **অংশগুলো আলাদা Property-তে:** `protocol`, `hostname`, `port`, `pathname`, `search`, `hash`।
- `searchParams` Query অংশ পড়ে ও লেখে; `set` বদলায়, `append` যোগ করে। বাংলা `বই` নিজে থেকেই Encode হয়েছে (`%E0%A6%AC...`)। `get` করলে আবার `বই` পাওয়া যায়।
- **আপেক্ষিক ঠিকানা:** দ্বিতীয় Argument একটি ভিত্তি; `../about` তার সঙ্গে মিলিয়ে পূর্ণ ঠিকানা হয়েছে।
- **অবৈধ লেখা `TypeError` দেয়,** তাই `new URL()` নিজেই একটি যাচাইয়ের উপায়, যা Project ব্যবহার করবে।

`URLSearchParams` আলাদাভাবেও ব্যবহার করা যায়, যেমন একটি Object থেকে Query তৈরি:

search-params.js · JavaScript — চলবে: Browser Console ও Node.js

```
const params = new URLSearchParams({ q: 'js & dom', sort: 'new' });
console.log(params.toString());

for (const [key, value] of params) {
  console.log(`${key} = ${value}`);
}

const fromAddress = new URLSearchParams('?page=abc&size=20');
const page = Number(fromAddress.get('page'));
console.log(Number.isInteger(page) ? page : 1, fromAddress.get('missing'));
```

Output

```
q=js+%26+dom&sort=new
q = js & dom
sort = new
1 null
```

`&` চিহ্নটি Encode হয়েছে, নইলে Server সেটিকে একটি নতুন Parameter-এর শুরু ভাবত। শেষ লাইনে দেখুন: URL-এর মান সব সময় লেখা, এবং ব্যবহারকারী যেকোনো কিছু লিখতে পারেন (`page=abc`), তাই URL-ও একটি Trust Boundary: রূপান্তরের পর যাচাই, অবৈধ হলে নিরাপদ Default।

> **সতর্কতা:** ব্যবহারকারীর দেওয়া URL Link হিসেবে বসানোর আগে Protocol যাচাই করুন। `javascript:alert(1)` একটি বৈধ URL; `<a href>`-এ বসালে Click-এ Code চলে। শুধু `http:` ও `https:` অনুমোদন করুন।

১৫.৩ Web Storage: localStorage এবং sessionStorage

অধ্যায় ৯-এ `localStorage` ব্যবহার করেছেন। এর একটি সঙ্গী আছে: `sessionStorage`। দুটির ব্যবহারের নিয়ম একই, পার্থক্য তাদের আয়ুতে।

storage-kinds.js · JavaScript — চলবে: Browser

```
localStorage.setItem('theme', 'dark');
sessionStorage.setItem('draft', 'অর্ধেক লেখা');

console.log(localStorage.getItem('theme'), sessionStorage.getItem('draft'));
console.log(sessionStorage.length, typeof sessionStorage.key(0));

localStorage.removeItem('theme');
sessionStorage.clear();
console.log(localStorage.getItem('theme'), sessionStorage.length);
```

Output

```
dark অর্ধেক লেখা
1 string
null 0
```

	localStorage	sessionStorage
কতদিন থাকে	মুছে না ফেলা পর্যন্ত	Tab বন্ধ হওয়া পর্যন্ত (Reload-এ থাকে)
কে দেখে	একই Website-এর সব Tab	শুধু সেই Tab
উপযুক্ত ব্যবহার	পছন্দ, সংরক্ষিত তালিকা	অর্ধেক পূরণ করা Form, Tab-এর সাময়িক অবস্থা
সীমা	সাধারণত প্রতি Website প্রায় ৫ MB	একই

দুটিই Synchronous (কাজ শেষ না হওয়া পর্যন্ত Program থামে), শুধু লেখা রাখে, এবং ব্যর্থ হতে পারে। জায়গা শেষ হলে কী হয়, আসল Browser-এ দেখুন:

quota.js · JavaScript চলবে: Browser

```
const huge = 'x'.repeat(11 * 1024 * 1024);

try {
  localStorage.setItem('huge', huge);
  console.log('সংরক্ষিত');
} catch (error) {
  console.log(`ব্যর্থ: ${error.name}`);
}
console.log(localStorage.getItem('huge'));
```

Output

```
ব্যর্থ: QuotaExceededError
null
```

প্রায় এক কোটি ১৫ লক্ষ অক্ষরের লেখা সীমা ছাড়িয়েছে; `setItem` একটি `QuotaExceededError` ছুড়েছে এবং কিছুই সংরক্ষিত হয়নি। Private Browsing-এ বা ব্যবহারকারী Storage বন্ধ রাখলেও একই ধরনের ত্রুটি আসতে পারে। তাই প্রতিটি `setItem` একটি `try`-এর ভেতরে, এবং ব্যর্থ হলে ব্যবহারকারীকে জানানো (অধ্যায় ৯-এর Task Manager)।

কখন Web Storage যথেষ্ট নয়

Web Storage ছোট, সরল তথ্যের জন্য: পছন্দ, কয়েকশো সারির তালিকা। তিনটি পরিস্থিতিতে অন্য ব্যবস্থা লাগে। **অনেক তথ্য** (হাজারো সারি, ছবি, File): সীমা কয়েক MB, আর বড় JSON প্রতিবার পুরোটা পড়া ও লেখা ধীর। **খোঁজা ও সূচি** (Index) দরকার হলে: Web Storage শুধু Key দিয়ে খোঁজে। **Page আটকানো চলবে না**: Web Storage Synchronous, তাই বড় লেখার সময় Page থেমে থাকে। এসব ক্ষেত্রে Browser-এর **IndexedDB** (Asynchronous, বড়, সূচিযুক্ত Database) এবং Offline-এ ব্যবহারের জন্য Network-এর সাড়া রাখতে **Cache API** ব্যবহৃত হয়। দুটিরই API তুলনামূলক জটিল; বাস্তবে অনেকেই এগুলোর ওপর তৈরি ছোট Library ব্যবহার করেন। এই বইয়ের Project-গুলোর তথ্য ছোট, তাই Web Storage-ই উপযুক্ত।

storage Event: একাধিক Tab-এর সমন্বয়

একই Website দুটি Tab-এ খোলা থাকলে, একটিতে `localStorage` বদলালে **অন্যটি** একটি `storage` Event পায়। নিচের Page-এ একটি `<iframe>`-এ একই Website-এর আরেকটি Page আছে, যা দ্বিতীয় Tab-এর মতো আচরণ করে:

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>storage Event</title>
</head>
<body>
  <iframe id="other" src="other.html" title="অন্য Tab-এর মতো"></iframe>
  <script src="main.js"></script>
</body>
</html>
```

other.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>অন্য Page</title>
</head>
<body>
  <script>
    window.addEventListener('storage', (event) => {
      console.log(`অন্য Page দেখল: ${event.key} = ${event.newValue}`);
    });
  </script>
</body>
</html>
```

main.js · JavaScript চলবে: Browser

```javascript
let ownEvents = 0;
window.addEventListener('storage', () => {
  ownEvents += 1;
});

document.querySelector('#other').addEventListener('load', () => {
  localStorage.setItem('last-update', '10:30');
  setTimeout(() => {
    console.log(`এই Page নিজে Event পেয়েছে ${ownEvents} বার`);
    localStorage.removeItem('last-update');
  }, 200);
});
```

Output

```
অন্য Page দেখল: last-update = 10:30
এই Page নিজে Event পেয়েছে 0 বার
অন্য Page দেখল: last-update = null
```

যে Page পরিবর্তন করেছে সে নিজে Event পায় না; পায় একই Website-এর অন্য সব Page। শেষ লাইনটিও লক্ষ করুন: `removeItem`-ও একটি Event পাঠায়, যার `newValue` `null`। তাই Listener-কে মুছে ফেলার ক্ষেত্রটিও সামলাতে হয়। Event Object-এ থাকে কোন Key (`key`), আগের মান (`oldValue`) এবং নতুন মান (`newValue`)। Project এই Event দিয়ে দুটি Tab-এর তালিকা একই রাখে। `sessionStorage`-এর জন্য এই Event অন্য Tab-এ যায় না, কারণ প্রতিটি Tab-এর `sessionStorage` আলাদা।

১৫.৪ সংরক্ষিত তথ্যের Version এবং Migration

আজ আপনি কাজের তালিকা রাখছেন লেখার Array হিসেবে। পরের মাসে প্রতিটি কাজের সঙ্গে "সম্পন্ন" অবস্থা রাখতে চাইলেন। ব্যবহারকারীদের Browser-এ পুরোনো গঠনের তথ্য রয়ে গেছে; নতুন Code সেটি পড়তে না পারলে তাদের তালিকা হারাবে। সমাধান: সংরক্ষিত তথ্যে একটি `version` রাখা, এবং পড়ার সময় পুরোনো Version-কে নতুনে রূপান্তর করা।

migrate.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function migrate(data) {
  if (data?.version === 1) {
    const items = data.items.map((name) => ({ name, done: false }));
    return { version: 2, items };
  }
  if (data?.version === 2) {
    return data;
  }
  throw new Error(`অচেনা Version: ${data?.version}`);
}

const saved = { version: 1, items: ['বাজার', 'বই পড়া'] };
const current = migrate(saved);

console.log(current.version, current.items[0].name, current.items[0].done);
console.log(migrate(current) === current);

try {
  migrate({ version: 9 });
} catch (error) {
  console.log(error.message);
}
```

Output

```
2 বাজার false
true
অচেনা Version: 9
```

প্রতিটি পুরোনো Version-এর জন্য একটি ধাপ, এবং ধাপগুলো ক্রমে চলে (১ থেকে ২, ২ থেকে ৩...)। অচেনা Version (হয়তো নতুন সংস্করণের App থেকে, বা ভাঙা) চুপচাপ গ্রহণ না করে স্পষ্ট ত্রুটি। Project-এর Export File-এও `version` আছে, একই কারণে।

১৫.৫ fetch, Request এবং Response

অধ্যায় ১০-এ `fetch` দিয়ে তথ্য এনেছেন। Server-এ তথ্য **পাঠাতে** Method, Header ও Body দিতে হয়। `fetch`-এর ভেতরে এগুলো একটি `Request` Object-এ জমা হয়; সরাসরি `Request` তৈরি করে দেখা যায় কী পাঠানো হবে, Network ছাড়াই:

request.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const request = new Request('https://api.example.com/bookmarks', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: 'MDN', url: 'https://developer.mozilla.org/' }),
});

console.log(request.method, request.headers.get('content-type'));
console.log((await request.json()).title);

const response = new Response(JSON.stringify({ id: 7 }), {
  status: 201,
  headers: { 'Content-Type': 'application/json' },
});
console.log(response.status, response.ok, (await response.json()).id);
```

Output

```
POST application/json
MDN
201 true 7
```

- `method: 'POST'` তথ্য পাঠানোর জন্য; `GET` (Default) আনার জন্য।
- **Body লেখা হিসেবে যায়;** Object পাঠাতে `JSON.stringify`, এবং `Content-Type` Header দিয়ে Server-কে জানাতে হয় এটি JSON।
- **Header-এর নাম বড়-ছোট হাতের অক্ষর নির্বিশেষে;** `get('content-type')` কাজ করেছে।

- `fetch(url, options)` লিখলে ঠিক এই `Request`-টিই পাঠানো হয়। অধ্যায় ১০-এর কৃত্রিম `Response` দিয়ে Server-এর সাড়াও একইভাবে তৈরি করা যায়।

একই Website-এর একটি JSON File আনা, আসল Browser-এ:

links.json · JSON — চলবে: Browser

```
[
  { "title": "MDN", "url": "https://developer.mozilla.org/" },
  { "title": "Node.js", "url": "https://nodejs.org/" }
]
```

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>fetch</title>
  <script type="module" src="app.mjs"></script>
</head>
<body></body>
</html>
```

app.mjs · JavaScript — চলবে: Browser

```
const response = await fetch('links.json');
console.log(response.status, response.headers.get('content-type'));

const links = await response.json();
console.log(links.map((link) => link.title).join(', '));
console.log(new URL('links.json', location.href).pathname);
```

Output

```
200 application/json; charset=utf-8
MDN, Node.js
/examples/10-fetch-local/links.json
```

আপেক্ষিক ঠিকানা `'links.json'` Page-এর ঠিকানার সঙ্গে মিলিয়ে পূর্ণ হয়েছে (শেষ লাইন), ঠিক `new URL(relative, base)`-এর মতো। চলমান Request বাতিল এবং সময়সীমার জন্য অধ্যায় ১০-এর `AbortController` ও `AbortSignal.timeout` একইভাবে প্রযোজ্য।

১৫.৬ History API: অবস্থা URL-এ

একটি খোঁজার Page-এ ব্যবহারকারী "javascript" খুঁজলেন, তারপর Link-টি সহকর্মীকে পাঠালেন। সহকর্মী Link খুলে খোঁজার ফল না পেয়ে ফাঁকা Page পেলে অভিজ্ঞতা খারাপ। সমাধান: Page-এর গুরুত্বপূর্ণ অবস্থা URL-এ রাখা। **History API** Page Reload না করেই Address Bar-এর URL বদলায়।

history.js · JavaScript চলবে: Browser

```
const params = new URLSearchParams(location.search);
params.set('q', 'javascript');
history.replaceState(null, '', `${location.pathname}?${params}`);
console.log(location.search);

history.pushState({ page: 2 }, '', '?q=javascript&page=2');
console.log(location.search, history.state.page);

window.addEventListener('popstate', (event) => {
  console.log(`Back: ${location.search}, state ${JSON.stringify(event.state)}`);
});
history.back();
```

Output

```
?q=javascript
?q=javascript&page=2 2
Back: ?q=javascript, state null
```

Method	কাজ	Back Button
`history.replaceState(state, '', url)`	বর্তমান URL বদলায়	নতুন ধাপ তৈরি হয় না
`history.pushState(state, '', url)`	নতুন URL যোগ করে	Back চাপলে আগেরটিতে ফেরে
`popstate` Event	Back বা Forward চাপলে	URL ও `state` থেকে Page আবার আঁকুন

খোঁজার প্রতিটি অক্ষরে `pushState` করলে ব্যবহারকারীকে Back বিশবার চাপতে হবে; তাই লেখার সময় `replaceState`, আর "নতুন Page"-এর মতো বড় পরিবর্তনে `pushState`। Project খোঁজা ও ছাঁকনি `replaceState` দিয়ে URL-এ রাখে।

১৫.৭ Blob, File এবং Download

Blob হলো কাঁচা তথ্যের একটি টুকরো (লেখা বা বাইনারি), একটি ধরনসহ। **File** একটি Blob, যার সঙ্গে নাম ও তারিখ আছে; ব্যবহারকারী `<input type="file">` দিয়ে যে File দেন, তা এই ধরনের।

blob.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const blob = new Blob(['নাম,শহর\n', 'Sara,Dhaka\n'], { type: 'text/csv' });
console.log(blob.type, blob.size);
console.log((await blob.text()).split('\n')[1]);

const file = new File(['{"ok":true}'], 'result.json', {
  type: 'application/json',
});
console.log(file.name, file.size, JSON.parse(await file.text()).ok);
```

Output

```
text/csv 31
Sara,Dhaka
result.json 11 true
```

`size` গোনে Byte, অক্ষর নয়: প্রতিটি বাংলা অক্ষর UTF-8-এ তিন Byte, তাই "নাম,শহর" অংশ ১৯ Byte (অধ্যায় ৮)। `text()` একটি Promise ফেরত দেয়, কারণ বড় File পড়তে সময় লাগে।

Browser-এ একটি Blob ব্যবহারকারীর কম্পিউটারে নামাতে:

download.js · JavaScript — চলবে: Browser

```
const text = JSON.stringify({ hello: 'world' });
const blob = new Blob([text], { type: 'application/json' });
const link = document.createElement('a');

link.href = URL.createObjectURL(blob);
link.download = 'data.json';
link.click();

setTimeout(() => URL.revokeObjectURL(link.href), 1000);
```

`URL.createObjectURL(blob)` Memory-তে থাকা Blob-এর জন্য একটি সাময়িক `blob:` ঠিকানা তৈরি করে; `download` Attribute-সহ একটি Link-এ Click করলে Browser File হিসেবে নামায়। কাজ শেষে `revokeObjectURL` সেই Memory মুক্ত করে। এই উদাহরণটি Automated Run-এ চালানো হয়নি, কারণ এটি একটি File নামায়; Project-এর Browser Test একই Code-এর তৈরি Blob-এর বিষয়বস্তু আসল Chrome-এ যাচাই করে।

ব্যবহারকারীর File পড়া: `<input type="file">`-এর `change` Event-এ `input.files[0]` একটি File, এবং `await file.text()` তার বিষয়বস্তু। পুরোনো Code-এ একই কাজ `FileReader` দিয়ে দেখবেন। ব্যবহারকারীর File সম্পূর্ণ অবিশ্বস্ত: যেকোনো আকার, যেকোনো বিষয়বস্তু। আকার সীমা, JSON পার্সের ত্রুটি এবং গঠন—তিনটিই যাচাই করুন।

১৫.৮ নিরাপত্তা ও গোপনীয়তা

- **Web Storage Encrypt করা নয়,** এবং একই Website-এর যেকোনো Script পড়তে পারে। একটি XSS দুর্বলতা (অধ্যায় ৮) থাকলে আক্রমণকারীর Script সব পড়ে নেয়। Password, Login Token বা ব্যক্তিগত সংবেদনশীল তথ্য সেখানে রাখবেন না; Login-এর জন্য Server-এর `HttpOnly` Cookie ব্যবহৃত হয় (অধ্যায় ২০)।
- **ব্যবহারকারীর দেওয়া URL** Protocol যাচাই ছাড়া Link বা `src`-এ নয়। বাইরের Link-এ `rel="noopener noreferrer"`, যাতে নতুন Tab আপনার Page নিয়ন্ত্রণ করতে না পারে।
- **URL-এ গোপন তথ্য নয়।** URL Browser History, Server Log এবং শেয়ার করা Link-এ থেকে যায়।
- **Storage যেকোনো সময় মুছে যেতে পারে:** ব্যবহারকারী মুছতে পারেন, Browser জায়গা বাঁচাতে মুছতে পারে। গুরুত্বপূর্ণ তথ্যের জন্য Export-এর সুবিধা দিন, বা Server-এ রাখুন।
- **ব্যবহারকারীর তথ্য তাঁর নিয়ন্ত্রণে:** কী রাখা হচ্ছে স্পষ্ট করুন, এবং মোছার উপায় দিন।

Browser-এ তথ্য রাখার আরেকটি পুরোনো ব্যবস্থা **Cookie**। পার্থক্যটি জানা জরুরি, কারণ ভুল জায়গায় ভুল তথ্য রাখা নিরাপত্তার সাধারণ উৎস:

	Cookie	Web Storage
Server-এ যায় কি	প্রতিটি Request-এর সঙ্গে নিজে থেকে	কখনো না; JavaScript নিজে পাঠালে তবেই
আকার	প্রতিটি প্রায় ৪ KB	প্রায় ৫ MB
JavaScript থেকে লুকানো যায় কি	হ্যাঁ, `HttpOnly` দিলে	না
সাধারণ ব্যবহার	Login Session (Server নির্ধারণ করে)	Client-এর পছন্দ ও সাময়িক তথ্য

Login-এর পরিচয় তাই Server-এর দেওয়া `HttpOnly` Cookie-তে থাকে, যা কোনো Script—আক্রমণকারীর Script-সহ—পড়তে পারে না; আর অ-সংবেদনশীল Client-তথ্য থাকে Web Storage-এ।

১৫.৯ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
String জুড়ে URL বানানো	`&`, `#`, বাংলা অক্ষরে ভাঙা URL	`URL`, `URLSearchParams`
URL-এর মান যাচাই না করা	`NaN` Page, ভুল অবস্থা	রূপান্তর, যাচাই, Default
`javascript:` URL Link-এ	Click-এ Code চলে	শুধু `http:` / `https:`

ভুল	কী দেখা যায়	সমাধান
`setItem`-এ `try` নেই	Storage ভর্তি হলে App থামে	`try` / `catch` ও বার্তা
Object সরাসরি Storage-এ	`"[object Object]"`	`JSON.stringify`
সংরক্ষিত তথ্যে Version নেই	App হালনাগাদের পর পুরোনো তথ্য অপাঠ্য	`version` ও Migration
নিজের Tab-এ `storage` Event আশা করা	কিছুই ঘটে না	নিজের পরিবর্তনে নিজে আঁকুন
প্রতিটি অক্ষরে `pushState`	Back Button বিরক্তিকর	`replaceState`
`createObjectURL`-এর পর `revoke` নেই	Memory জমে	`revokeObjectURL`
ব্যবহারকারীর File বিশ্বাস	ভাঙা JSON-এ App থামে	পার্স ও গঠন যাচাই

১৫.১০ Debugging নির্দেশিকা

- **Application Panel (DevTools):** Local Storage ও Session Storage-এর প্রতিটি Key-Value দেখা, বদলানো ও মোছা যায়। "Reload-এ তথ্য থাকে না" সমস্যায় প্রথম জায়গা। Storage ভর্তির পরিস্থিতি পরীক্ষা করতে এখানে "Storage" অংশে সীমা অনুকরণ করা যায়।
- **Network Panel:** প্রতিটি Request-এর Method, Header, পাঠানো Body (Payload) ও সাড়া। POST-এ `Content-Type` আছে কি না এখানে দেখুন।
- **দুটি Tab পাশাপাশি খুলে** `storage` Event পরীক্ষা করুন; একই Tab-এ পরীক্ষা করলে Event আসে না।
- **URL-এর অবস্থা:** Address Bar-এর URL নিজে বদলে Reload করুন; Page সেই অবস্থায় খোলে কি না দেখুন। Back ও Forward চেপে দেখুন।
- **Console-এ** `new URL(text)` লিখে ব্যবহারকারীর দেওয়া ঠিকানা কীভাবে পড়া হচ্ছে দেখুন।
- **Download কাজ না করলে** Console-এ ত্রুটি দেখুন, এবং `blob:` ঠিকানা আগেভাগে `revoke` হচ্ছে কি না।

১৫.১১ পেশাদার অভ্যাস

- **Browser API সীমানায়, নিয়ম Pure Module-এ;** তাহলে নিয়মগুলো Node.js-এ Test করা যায়।
- **প্রতিটি বাইরের উৎস Trust Boundary:** URL, Storage, File, Server—সবকিছু যাচাই।
- **Feature Detection,** Browser-এর নাম নয়।
- **Storage-এর তথ্যে Version,** এবং প্রতিটি লেখা `try`-এ।
- **শেয়ারযোগ্য অবস্থা URL-এ,** Tab-এর সাময়িক অবস্থা `sessionStorage`-এ, দীর্ঘস্থায়ী পছন্দ `localStorage`-এ।
- **ব্যবহারকারীর তথ্য তাঁর হাতে:** Export ও Import।

- **Test-এ আসল Browser** যেখানে API শুধু Browser-এর; Node.js-এর Test-কে Browser-এর যাচাই বলবেন না।

১৫.১২ Project: Bookmark Vault

ডেভেলপারদের প্রয়োজনীয় Link-এর একটি ব্যক্তিগত ভান্ডার: Link যোগ, Tag, প্রিয় চিহ্ন, খোঁজা ও ছাঁকা, এবং File-এ Export ও Import। এই অধ্যায়ের প্রতিটি API এখানে একটি বাস্তব কাজে ব্যবহৃত।

Project-এর চাহিদা

- শিরোনাম ১ থেকে ১০০ অক্ষর; URL শুধু `http:` বা `https:`, `#` অংশ বাদ; একই URL দুবার নয়।
- Tag কমা দিয়ে; ছোট হাতের অক্ষরে, পুনরাবৃত্তি বাদ।
- প্রিয় চিহ্ন, মোছা; খোঁজা (শিরোনাম ও URL), Tag এবং "শুধু প্রিয়" ছাঁকনি।
- খোঁজা ও ছাঁকনি URL-এ (`?q=&tag=&fav=1`); Link শেয়ার বা Reload-এ একই অবস্থা।
- তালিকা `localStorage` -এ; অন্য Tab-এ পরিবর্তন হলে এখানেও হালনাগাদ (`storage` Event)।
- অর্ধেক পূরণ করা Form `sessionStorage` -এ; Reload-এ ফিরে আসে, যোগ করার পর মুছে যায়।
- Export: `version` -সহ JSON File নামানো (Blob)। Import: JSON File পড়া; অবৈধ ও পুনরাবৃত্ত Bookmark বাদ, সংখ্যা জানানো; ভাঙা File-এ বার্তা।
- সব লেখা `textContent` ; বাইরের Link-এ `rel="noopener noreferrer"` ।

File-এর কাঠামো

Folder-এর কাঠামো

```
bookmark-vault/
├── index.html
├── style.css
├── app.mjs                 UI ও Browser API
├── src/
│   ├── bookmarks.mjs       নিয়ম, যাচাই, Import ও Export (Pure)
│   ├── url-state.mjs       ছাঁকনি থেকে URL এবং URL থেকে ছাঁকনি
│   └── storage.mjs         localStorage Adapter
├── README.md
└── tests/
    ├── bookmarks.test.mjs
    ├── url-state.test.mjs
    ├── static-server.mjs
    ├── cdp.mjs
    └── browser-check.mjs
```

src/bookmarks.mjs

দুই অংশ মিলে সম্পূর্ণ File।

bookmarks.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
export const SCHEMA_VERSION = 1;
const ALLOWED_PROTOCOLS = ['http:', 'https:'];
const MAX_TITLE = 100;

export class BookmarkError extends Error {
  constructor(message) {
    super(message);
    this.name = 'BookmarkError';
  }
}

// শুধু http/https; # অংশ বাদ; Host ছোট হাতের অক্ষরে (URL নিজে করে)।
export function normalizeUrl(text) {
  let url;
  try {
    url = new URL(String(text ?? '').trim());
  } catch {
    throw new BookmarkError('সঠিক একটি URL লিখুন (https://...)।');
  }
  if (!ALLOWED_PROTOCOLS.includes(url.protocol)) {
    throw new BookmarkError(`"${url.protocol}" ধরনের URL গ্রহণযোগ্য নয়।`);
  }
  url.hash = '';
  return url.href;
}

export function parseTags(input) {
  const parts = Array.isArray(input) ? input : String(input ?? '').split(',');
  const tags = parts
    .map((tag) => String(tag).trim().toLowerCase())
    .filter(Boolean);
  return [...new Set(tags)];
}

const defaults = { now: () => new Date(), makeId: () => crypto.randomUUID() };

export function createBookmark(input, { now, makeId } = defaults) {
  const title = String(input?.title ?? '').trim();
  if (title === '' || title.length > MAX_TITLE) {
    throw new BookmarkError(`শিরোনাম 1 থেকে ${MAX_TITLE} অক্ষরের হতে হবে।`);
  }
  return {
    id: makeId(),
    title,
    url: normalizeUrl(input.url),
    tags: parseTags(input.tags),
    favorite: false,
    createdAt: now().toISOString(),
```

```
  };
}

// ...
```

bookmarks.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export function addBookmark(list, bookmark) {
  if (list.some((item) => item.url === bookmark.url)) {
    throw new BookmarkError('এই URL আগে থেকেই আছে।');
  }
  return [bookmark, ...list];
}

export const toggleFavorite = (list, id) =>
  list.map((item) =>
    (item.id === id ? { ...item, favorite: !item.favorite } : item));

export const removeBookmark = (list, id) => list.filter((item) => item.id !== id);

export function filterBookmarks(list, filters = {}) {
  const { query = '', tag = '', favoritesOnly = false } = filters;
  const needle = query.trim().toLowerCase();
  return list.filter((item) =>
    (!favoritesOnly || item.favorite) &&
    (tag === '' || item.tags.includes(tag)) &&
    `${item.title} ${item.url}`.toLowerCase().includes(needle));
}

export const allTags = (list) =>
  [...new Set(list.flatMap((item) => item.tags))].sort();

// বাইরে থেকে আসা Bookmark (Storage বা Import) যাচাই; ঠিক না হলে null।
export function reviveBookmark(raw) {
  const validShape = typeof raw?.id === 'string' && raw.id !== '' &&
    typeof raw.favorite === 'boolean' && !Number.isNaN(Date.parse(raw.createdAt));
  if (!validShape) {
    return null;
  }
  try {
    const clock = { now: () => new Date(raw.createdAt), makeId: () => raw.id };
    return { ...createBookmark(raw, clock), favorite: raw.favorite };
  } catch (error) {
    if (error instanceof BookmarkError) {
      return null;
    }
    throw error;
```

```
  }
}

export function parseStored(text) {
  try {
    const data = JSON.parse(text ?? '[]');
    return Array.isArray(data) ? data.map(reviveBookmark).filter(Boolean) : [];
  } catch {
    return [];
  }
}

export function exportData(list, now = new Date()) {
  const data = {
    version: SCHEMA_VERSION,
    exportedAt: now.toISOString(),
    bookmarks: list,
  };
  return JSON.stringify(data, null, 2);
}

export function importData(text, existing) {
  let data;
  try {
    data = JSON.parse(text);
  } catch {
    throw new BookmarkError('File-টি সঠিক JSON নয়।');
  }
  if (data?.version !== SCHEMA_VERSION || !Array.isArray(data.bookmarks)) {
    throw new BookmarkError('অচেনা File-এর গঠন বা Version।');
  }

  let list = existing;
  let added = 0;
  for (const raw of data.bookmarks) {
    const bookmark = reviveBookmark(raw);
    const duplicate = bookmark && list.some((item) =>
      item.url === bookmark.url || item.id === bookmark.id);
    if (bookmark && !duplicate) {
      list = [...list, bookmark];
      added += 1;
    }
  }
  return { list, added, skipped: data.bookmarks.length - added };
}
```

- **`normalizeUrl`** `URL`-কে যাচাইকারী হিসেবে ব্যবহার করে: অবৈধ লেখায় তার `TypeError`-কে ব্যবহারকারীর ভাষার `BookmarkError`-এ রূপান্তর, তারপর Protocol-এর অনুমোদিত তালিকা। `javascript:` বা `data:`

এখানেই আটকায়।

- `new Set(tags)` : `Set` এমন একটি সংগ্রহ, যেখানে প্রতিটি মান একবারই থাকে; Array থেকে Set, আবার Spread দিয়ে Array—পুনরাবৃত্তি বাদের প্রচলিত কৌশল।
- `createBookmark` **-এর সময় ও ID Inject করা** (অধ্যায় ১২, ১৪): Test-এ নির্দিষ্ট তারিখ ও ID দেওয়া যায়, আর `reviveBookmark` একই Function দিয়ে সংরক্ষিত Bookmark-কে তার নিজের তারিখ ও ID-সহ আবার যাচাই করে।
- `reviveBookmark` **সব বাইরের তথ্যের দরজা:** Storage ও Import দুটিই এখান দিয়ে যায়, তাই একটি `javascript:` URL হাতে বসানো Storage বা File দিয়েও ঢুকতে পারে না।
- `importData` **কখনো বিদ্যমান তালিকা বদলায় না;** নতুন তালিকা, যোগ ও বাদের সংখ্যা ফেরত দেয়।

src/url-state.mjs এবং src/storage.mjs

url-state.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export function readFilters(search) {
  const params = new URLSearchParams(search);
  return {
    query: params.get('q') ?? '',
    tag: params.get('tag') ?? '',
    favoritesOnly: params.get('fav') === '1',
  };
}

export function filtersToSearch({ query, tag, favoritesOnly }) {
  const params = new URLSearchParams();
  if (query.trim() !== '') {
    params.set('q', query.trim());
  }
  if (tag !== '') {
    params.set('tag', tag);
  }
  if (favoritesOnly) {
    params.set('fav', '1');
  }
  const text = params.toString();
  return text === '' ? '' : `?${text}`;
}
```

storage.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { parseStored } from './bookmarks.mjs';

export const STORAGE_KEY = 'jsm-ch15-bookmarks';

export function createStore(
  storage = globalThis.localStorage,
```

```
  key = STORAGE_KEY,
) {
  return {
    load() {
      try {
        return parseStored(storage.getItem(key));
      } catch {
        return [];
      }
    },
    save(list) {
      try {
        storage.setItem(key, JSON.stringify(list));
        return true;
      } catch {
        return false;
      }
    },
  };
}
```

`url-state.mjs` URL-এর সঙ্গে কাজ করে কিন্তু `location` বা `history` ছোঁয় না: লেখা নেয়, লেখা দেয়। তাই Node.js-এ Test করা যায়; `history.replaceState` ডাকে শুধু `app.mjs`।

index.html ও style.css

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bookmark Vault</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.mjs"></script>
</head>
<body>
  <main class="app">
    <h1>Bookmark Vault</h1>

    <form id="bookmark-form">
      <label>শিরোনাম <input name="title" autocomplete="off"></label>
      <label>URL <input name="url" type="url" placeholder="https://"></label>
      <label>Tag (কমা দিয়ে) <input name="tags" autocomplete="off"></label>
      <button type="submit">যোগ করুন</button>
    </form>

    <div class="filters">
```

```
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>Tag <select id="tag"></select></label>
      <label><input id="favorites-only" type="checkbox"> শুধু প্রিয়</label>
    </div>

    <p id="count"></p>
    <ul id="bookmarks"></ul>

    <div class="files">
      <button id="export" type="button">Export</button>
      <label>Import
        <input id="import" type="file" accept="application/json,.json">
      </label>
    </div>
    <p id="status" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 720px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

form,
.filters,
.files,
#bookmarks li {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
}

#bookmarks li {
  padding: 8px 0;
  border-bottom: 1px solid #e3e7ef;
```

```
}

.tags {
  color: #4a5668;
  font-size: 0.9em;
}

[aria-pressed='true'] {
  background: #ffe9a8;
}
```

app.mjs

তিন অংশ মিলে সম্পূর্ণ File।

app.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Browser

```
import {
  addBookmark, allTags, BookmarkError, createBookmark, exportData,
  filterBookmarks, importData, removeBookmark, toggleFavorite,
} from './src/bookmarks.mjs';
import { createStore, STORAGE_KEY } from './src/storage.mjs';
import { filtersToSearch, readFilters } from './src/url-state.mjs';

const FORM_STORE_KEY = 'jsm-ch15-draft';
const store = createStore();
let bookmarks = store.load();
let filters = readFilters(location.search);

const $ = (selector) => document.querySelector(selector);
const form = $('#bookmark-form');
const list = $('#bookmarks');
const statusView = $('#status');
const search = $('#search');
const tagSelect = $('#tag');
const favoritesOnly = $('#favorites-only');

function setBookmarks(next, message) {
  bookmarks = next;
  const saved = store.save(bookmarks);
  statusView.textContent = saved
    ? message
    : 'সংরক্ষণ করা যায়নি; Page বন্ধ করলে পরিবর্তন হারাবে।';
  render();
}

// প্রত্যাশিত ত্রুটি (BookmarkError) বার্তা হয়; অন্য সব ত্রুটি ওপরে যায়।
function attempt(action) {
  try {
```

```
    action();
  } catch (error) {
    if (!(error instanceof BookmarkError)) {
      throw error;
    }
    statusView.textContent = error.message;
  }
}

// ...
```

app.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Browser

```
// ...

function button(label, action) {
  const element = document.createElement('button');
  element.type = 'button';
  element.textContent = label;
  element.dataset.action = action;
  return element;
}

function bookmarkRow(item) {
  const link = document.createElement('a');
  link.href = item.url;
  link.textContent = item.title;
  link.target = '_blank';
  link.rel = 'noopener noreferrer';

  const tags = document.createElement('span');
  tags.className = 'tags';
  tags.textContent = item.tags.join(', ');

  const star = button('প্রিয়', 'favorite');
  star.setAttribute('aria-pressed', String(item.favorite));

  const row = document.createElement('li');
  row.dataset.id = item.id;
  row.append(link, tags, star, button('মুছুন', 'remove'));
  return row;
}

function renderTagOptions() {
  const tags = allTags(bookmarks);
  if (!tags.includes(filters.tag)) {
    filters = { ...filters, tag: '' };
  }
  tagSelect.replaceChildren(...['', ...tags].map((tag) => {
```

```
    const option = document.createElement('option');
    option.value = tag;
    option.textContent = tag === '' ? 'সব Tag' : tag;
    return option;
  }));
  tagSelect.value = filters.tag;
}

function render() {
  renderTagOptions();
  if (search.value !== filters.query) {
    search.value = filters.query;
  }
  favoritesOnly.checked = filters.favoritesOnly;

  const shown = filterBookmarks(bookmarks, filters);
  list.replaceChildren(...shown.map(bookmarkRow));
  $('#count').textContent = `${bookmarks.length}টির মধ্যে ${shown.length}টি দেখানো হচ্ছে`;
  const address = `${location.pathname}${filtersToSearch(filters)}`;
  history.replaceState(null, '', address);
}

// ...
```

app.mjs · JavaScript (অংশ ৩ / ৩ (Part 3 of 3)) চলবে: Browser

```
// ...

form.addEventListener('submit', (event) => {
  event.preventDefault();
  attempt(() => {
    const bookmark = createBookmark(Object.fromEntries(new FormData(form)));
    setBookmarks(addBookmark(bookmarks, bookmark), `যোগ হয়েছে: ${bookmark.title}`);
    form.reset();
    sessionStorage.removeItem(FORM_STORE_KEY);
  });
});

// খসড়া শুধু এই Tab-এর; Reload-এর পরও থাকে।
form.addEventListener('input', () => {
  const draft = Object.fromEntries(new FormData(form));
  sessionStorage.setItem(FORM_STORE_KEY, JSON.stringify(draft));
});

list.addEventListener('click', (event) => {
  const action = event.target.closest('[data-action]')?.dataset.action;
  const id = event.target.closest('li')?.dataset.id;
  if (action === 'favorite') {
    setBookmarks(toggleFavorite(bookmarks, id), 'প্রিয় তালিকা বদলেছে।');
```

```
  }
  if (action === 'remove') {
    setBookmarks(removeBookmark(bookmarks, id), 'মুছে ফেলা হয়েছে।');
  }
});

const updateFilters = (change) => {
  filters = { ...filters, ...change };
  render();
};
search.addEventListener('input', () => updateFilters({ query: search.value }));
tagSelect.addEventListener('change', () =>
  updateFilters({ tag: tagSelect.value }));
favoritesOnly.addEventListener('change', () =>
  updateFilters({ favoritesOnly: favoritesOnly.checked }));

$('#export').addEventListener('click', () => {
  const blob = new Blob([exportData(bookmarks)], { type: 'application/json' });
  const link = document.createElement('a');
  link.href = URL.createObjectURL(blob);
  link.download = 'bookmarks.json';
  link.click();
  setTimeout(() => URL.revokeObjectURL(link.href), 1000);
  statusView.textContent = `${bookmarks.length}টি Bookmark Export হয়েছে।`;
});

$('#import').addEventListener('change', async (event) => {
  const [file] = event.target.files;
  if (!file) {
    return;
  }
  statusView.textContent = 'পড়া হচ্ছে...';
  try {
    const text = await file.text();
    const { list: next, added, skipped } = importData(text, bookmarks);
    setBookmarks(next, `Import: ${added}টি যোগ, ${skipped}টি বাদ।`);
  } catch (error) {
    if (!(error instanceof BookmarkError)) {
      throw error;
    }
    statusView.textContent = error.message;
  } finally {
    event.target.value = '';
  }
});

// অন্য Tab-এ তালিকা বদলালে এখানেও।
window.addEventListener('storage', (event) => {
  if (event.key === STORAGE_KEY) {
    bookmarks = store.load();
```

```
    render();
    statusView.textContent = 'অন্য Tab-এ তালিকা বদলেছে।';
  }
});

try {
  const draft = JSON.parse(sessionStorage.getItem(FORM_STORE_KEY) ?? '{}');
  for (const [name, value] of Object.entries(draft)) {
    if (form.elements[name]) {
      form.elements[name].value = value;
    }
  }
} catch {
  sessionStorage.removeItem(FORM_STORE_KEY);
}
render();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **শুরু:** তালিকা `localStorage` থেকে (যাচাইসহ), ছাঁকনি URL থেকে (`readFilters(location.search)`), এবং অর্ধেক লেখা Form `sessionStorage` থেকে। ভাঙা খসড়া হলে চুপচাপ মুছে ফেলা হয়।
- `render` **URL হালনাগাদ করে** `history.replaceState` দিয়ে; তাই প্রতিটি খোঁজা বা ছাঁকনি URL-এ প্রতিফলিত হয়, কিন্তু Back Button-এর ইতিহাস ভরে না। Tag-টি তালিকায় আর না থাকলে (যেমন শেষ Bookmark মোছার পর) ছাঁকনি নিজে থেকে "সব Tag"-এ ফেরে।
- **Export:** Pure `exportData` লেখা তৈরি করে; Blob, `createObjectURL` , Link-এ Click এবং পরে `revokeObjectURL` —অনুচ্ছেদ ১৫.৭-এর ধাপগুলো।
- **Import:** `await file.text()` , তারপর Pure `importData` , যা ভাঙা JSON বা অচেনা Version-এ `BookmarkError` দেয়। `finally` -তে File-এর ঘর খালি করা হয়, যাতে একই File আবার বাছলেও `change` Event ঘটে। পড়ার আগে "পড়া হচ্ছে…" বার্তা বড় File-এ ব্যবহারকারীকে আশ্বস্ত করে।
- `storage` **Event:** অন্য Tab তালিকা বদলালে এই Tab আবার `store.load()` করে আঁকে। নিজের পরিবর্তনে এই Event আসে না, তাই `setBookmarks` নিজেই আঁকে।
- **নিরাপত্তা:** সব লেখা `textContent` ; `href` -এ বসার আগে প্রতিটি URL `normalizeUrl` -এর Protocol যাচাই পেরিয়েছে; বাইরের Link নতুন Tab-এ `noopener` -সহ।

চালানো এবং প্রত্যাশিত আচরণ

Project Folder থেকে Local Server (Module ও `crypto.randomUUID` -এর জন্য):

Terminal

```
python -m http.server 8000
```

Browser-এ `http://localhost:8000` । Test:

Terminal

```
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
MDN, `https://developer.mozilla.org/#top`, Tag `JS, Docs, js`	URL-এ `#top` নেই; Tag `js, docs`
URL `javascript:alert(1)`	"\"javascript:\" ধরনের URL গ্রহণযোগ্য নয়।"
একই URL আবার	"এই URL আগে থেকেই আছে।"
খুঁজুন `node`, Tag `docs`, শুধু প্রিয়	URL হয় `?q=node&tag=docs&fav=1`; Reload-এ একই ছাঁকনি
Form অর্ধেক লিখে Reload	লেখা ফিরে আসে; যোগ করার পর মুছে যায়
Export, তারপর অন্য Browser-এ Import	একই Bookmark; পুনরাবৃত্তগুলো "বাদ"
ভাঙা JSON Import	"File-টি সঠিক JSON নয়।"
দুটি Tab খোলা, একটিতে যোগ	অন্যটিতে তালিকা ও "অন্য Tab-এ তালিকা বদলেছে।"

Companion Source Code-এর Node.js Test (২১টি) নিয়ম, Import-Export ও URL-এর রূপান্তর পরীক্ষা করে। `browser-check.mjs` একটি Local Server চালিয়ে আসল Chrome-এ ওপরের প্রতিটি সারি যাচাই করে: Export-এর Blob-এর বিষয়বস্তু পড়ে দেখে, Import-এ DevTools Protocol দিয়ে আসল File বাছাই করে, এবং একটি `<iframe>`-কে দ্বিতীয় Tab হিসেবে ব্যবহার করে `storage` Event পরীক্ষা করে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Page ফাঁকা, Console-এ CORS	`file://` থেকে খোলা	Local Server
`crypto.randomUUID is not a function`	Secure Context নয় (যেমন LAN-এর IP)	`localhost` বা `https://`
Reload-এ ছাঁকনি হারায়	`replaceState` ডাকা হয়নি	`render`-এর শেষ লাইন
অন্য Tab হালনাগাদ হয় না	`storage` Listener নেই, বা Key ভিন্ন	`STORAGE_KEY`
একই File দ্বিতীয়বার Import হয় না	File-এর ঘর খালি করা হয়নি	`event.target.value = ''`
Import-এ সব বাদ	Export-এর Version ভিন্ন	`SCHEMA_VERSION` ও Migration

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য শুধু এই Browser-এ; দুটি Tab একসঙ্গে লিখলে শেষ লেখাটি টেকে। Import-এর File-এর আকার সীমিত করা হয়নি; বড় File-এর আগে `file.size` যাচাই যোগ করা উচিত। শুধু একটি Schema Version আছে, তাই Migration-এর ধাপ এখনো দরকার হয়নি।

১৫.১৩ অনুশীলনী

প্রাথমিক স্তর

১. `https://example.com/search?q=dom&page=4` থেকে `hostname`, `q` এবং `page` (সংখ্যা হিসেবে) বের করুন।

২. `{ city: 'Dhaka', type: 'hotel & resort' }` থেকে `URLSearchParams` দিয়ে Query String তৈরি করুন।

৩. `localStorage`-এ একটি সংখ্যা রেখে আবার পড়ুন, এবং দেখান যে সেটি লেখা হয়ে গেছে; সঠিকভাবে সংখ্যায় ফেরান।

৪. নিচের প্রতিটির জন্য `localStorage`, `sessionStorage` না URL—কোনটি উপযুক্ত? (ক) Dark Mode পছন্দ (খ) এই Tab-এর অর্ধেক লেখা মন্তব্য (গ) খোঁজার শব্দ, যা Link-এ শেয়ার হবে।

৫. `isSafeLink(text)` লিখুন: শুধু `https:` URL-এ `true`; অবৈধ লেখায় `false`, ত্রুটি নয়।

মধ্যম স্তর

৬. অনুচ্ছেদ ১৫.৪-এর `migrate`-এ Version ৩ যোগ করুন: প্রতিটি কাজে `createdAt: null`।

৭. `readPage(search)` লিখুন: `?page=` থেকে ১ বা বেশি পূর্ণসংখ্যা; না থাকলে বা অবৈধ হলে ১।

৮. শিক্ষার্থীদের তালিকা থেকে CSV লেখার একটি Blob তৈরি করুন এবং তার আকার ও প্রথম সারি দেখান।

৯. একটি নকল Storage দিয়ে দেখান Project-এর `createStore`-এর `load` ভাঙা তথ্যে ফাঁকা তালিকা দেয়।

চ্যালেঞ্জ স্তর

১০. `filtersToSearch` ও `readFilters` একে অপরের উল্টো কি না পরীক্ষা করুন: একটি ছাঁকনি URL-এ নিয়ে আবার পড়লে একই ছাঁকনি পাওয়া যায় কি?

১১. Project-এর `exportData` ও `importData` দিয়ে Round-Trip দেখান: Export করে খালি তালিকায় Import, তারপর একই File আবার Import।

১২. `safeSave(key, value)` লিখুন, যা `QuotaExceededError` ধরে ব্যবহারকারীর ভাষায় বার্তা দেয়; আসল Browser-এ পরীক্ষা করুন।

১৫.১৪ অনুশীলনীর সমাধান

সমাধান ১

read-url.js · JavaScript — চলবে: Browser Console ও Node.js

```
const url = new URL('https://example.com/search?q=dom&page=4');
const page = Number(url.searchParams.get('page'));

console.log(url.hostname, url.searchParams.get('q'), page + 1);
```

Output

```
example.com dom 5
```

`page + 1` দেখায় সংখ্যায় রূপান্তর কাজ করেছে; রূপান্তর ছাড়া ফল হতো `'41'`।

সমাধান ২

build-query.js · JavaScript — চলবে: Browser Console ও Node.js

```
const params = new URLSearchParams({ city: 'Dhaka', type: 'hotel & resort' });

console.log(`https://example.com/search?${params}`);
```

Output

```
https://example.com/search?city=Dhaka&type=hotel+%26+resort
```

সমাধান ৩

number-storage.js · JavaScript — চলবে: Browser

```
localStorage.setItem('visits', 41);

const raw = localStorage.getItem('visits');
const visits = Number(raw) + 1;

console.log(typeof raw, raw + 1, visits);
localStorage.removeItem('visits');
```

Output

```
string 411 42
```

সমাধান ৪ — (ক) `localStorage`: দীর্ঘস্থায়ী পছন্দ, সব Tab-এ। (খ) `sessionStorage`: শুধু এই Tab, Reload-এ টেকে, Tab বন্ধ হলে যায়। (গ) URL: শেয়ার করা Link-এ অবস্থা থাকতে হবে; Storage অন্যের Browser-এ পৌঁছায় না।

সমাধান ৫

safe-link.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isSafeLink(text) {
  try {
    return new URL(text).protocol === 'https:';
  } catch {
    return false;
  }
}

console.log(isSafeLink('https://example.com'), isSafeLink('http://example.com'));
console.log(isSafeLink('javascript:alert(1)'), isSafeLink('example.com'));
```

Output

```
true false
false false
```

`example.com`-এ Protocol নেই, তাই `new URL` ত্রুটি দেয়, এবং Function শান্তভাবে `false` ফেরত দেয়।

সমাধান ৬

migrate-v3.js · JavaScript — চলবে: Browser Console ও Node.js

```
function migrate(data) {
  let current = data;
  if (current?.version === 1) {
    const items = current.items.map((name) => ({ name, done: false }));
    current = { version: 2, items };
  }
  if (current?.version === 2) {
    const items = current.items.map((item) => ({ ...item, createdAt: null }));
    current = { version: 3, items };
  }
  if (current?.version !== 3) {
    throw new Error(`অচেনা Version: ${data?.version}`);
  }
  return current;
}
```

```
const fromV1 = migrate({ version: 1, items: ['বাজার'] });
console.log(fromV1.version, JSON.stringify(fromV1.items[0]));
```

Output

```
3 {"name":"বাজার","done":false,"createdAt":null}
```

ধাপগুলো পরপর চলে, তাই Version ১-এর তথ্যও দুটি ধাপ পেরিয়ে ৩-এ পৌঁছায়। নতুন Version মানে শুধু একটি নতুন ধাপ।

সমাধান ৭

read-page.js · JavaScript — চলবে: Browser Console ও Node.js

```
function readPage(search) {
  const page = Number(new URLSearchParams(search).get('page'));
  return Number.isInteger(page) && page >= 1 ? page : 1;
}

console.log(readPage('?page=3'), readPage('?page=0'), readPage('?page=2.5'));
console.log(readPage('?page=abc'), readPage(''));
```

Output

```
3 1 1
1 1
```

সমাধান ৮

csv-blob.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 72 },
];

const lines = ['name,marks', ...students.map((s) => `${s.name},${s.marks}`)];
const blob = new Blob([lines.join('\n')], { type: 'text/csv' });

console.log(blob.size, (await blob.text()).split('\n')[1]);
```

Output

```
26 Sara,88
```

সমাধান ৯

broken-store.mjs · JavaScript — চলবে: Node.js

```
import { createStore } from '../../project/bookmark-vault/src/storage.mjs';

const fakeStorage = {
  getItem: () => '{this is not json',
  setItem: () => {},
};

const store = createStore(fakeStorage);
console.log(store.load().length, store.save([]));
```

Output

```
0 true
```

সমাধান ১০

round-trip-url.mjs · JavaScript — চলবে: Node.js

```
import {
  filtersToSearch, readFilters,
} from '../../project/bookmark-vault/src/url-state.mjs';

const cases = [
  { query: 'node & npm', tag: 'docs', favoritesOnly: true },
  { query: '', tag: '', favoritesOnly: false },
  { query: 'বাংলা', tag: '', favoritesOnly: false },
];

for (const filters of cases) {
  const search = filtersToSearch(filters);
  const back = readFilters(search);
  console.log(JSON.stringify(back) === JSON.stringify(filters), search || '(ফাঁকা)');
}
```

Output

```
true ?q=node+%26+npm&tag=docs&fav=1
true (ফাঁকা)
true ?q=%E0%A6%AC%E0%A6%BE%E0%A6%82%E0%A6%B2%E0%A6%BE
```

তিনটি ক্ষেত্রেই ফিরে পাওয়া ছাঁকনি মূলটির সমান। একটি সূক্ষ্মতা: আগে-পরের ফাঁকা জায়গাসহ খোঁজার শব্দ (`' node '`) ফিরে আসবে `'node'` হয়ে, কারণ `filtersToSearch` `trim` করে। Project-এর Test এই আচরণও যাচাই করে।

সমাধান ১১

round-trip-file.mjs · JavaScript — চলবে: Node.js

```
import {
  createBookmark, exportData, importData,
} from '../../project/bookmark-vault/src/bookmarks.mjs';

const clock = { now: () => new Date('2026-10-15T09:00:00Z'), makeId: () => 'b1' };
const input = { title: 'MDN', url: 'https://developer.mozilla.org/' };
const bookmark = createBookmark(input, clock);

const file = exportData([bookmark], new Date('2026-10-15T10:00:00Z'));
const first = importData(file, []);
const second = importData(file, first.list);

console.log(first.added, first.skipped, first.list[0].title);
console.log(second.added, second.skipped, second.list.length);
```

Output

```
1 0 MDN
0 1 1
```

দ্বিতীয়বার একই File: পুনরাবৃত্ত, তাই বাদ, তালিকা অপরিবর্তিত।

সমাধান ১২

safe-save.js · JavaScript — চলবে: Browser

```
function safeSave(key, value) {
  try {
    localStorage.setItem(key, value);
    return 'সংরক্ষিত';
  } catch (error) {
    if (error.name === 'QuotaExceededError') {
      return 'জায়গা শেষ; পুরোনো তথ্য মুছে আবার চেষ্টা করুন।';
    }
    return 'সংরক্ষণ করা যায়নি।';
  }
}

console.log(safeSave('small', 'ok'));
console.log(safeSave('big', 'x'.repeat(11 * 1024 * 1024)));
localStorage.removeItem('small');
```

Output

```
সংরক্ষিত
জায়গা শেষ; পুরোনো তথ্য মুছে আবার চেষ্টা করুন।
```

১৫.১৫ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোনটি Browser-এর API, JavaScript ভাষার অংশ নয়?

ক) `Array.prototype.map`

খ) `JSON.parse`

গ) `localStorage`

ঘ) `Promise`

প্রশ্ন ২. `new URL('not a url')` কী করে?

ক) `null` ফেরত দেয়

খ) `TypeError` ছোড়ে

গ) একটি খালি URL দেয়

ঘ) `about:blank` দেয়

প্রশ্ন ৩. `sessionStorage`-এর তথ্য কখন মুছে যায়?

ক) Page Reload করলে

খ) কখনো না

গ) Tab বন্ধ করলে

ঘ) প্রতি ২৪ ঘণ্টায়

প্রশ্ন ৪. `localStorage.setItem` জায়গা শেষ হলে কী করে?

ক) `QuotaExceededError` ছোড়ে

খ) পুরোনো তথ্য মুছে জায়গা করে

গ) নীরবে কিছু না করে

ঘ) `false` ফেরত দেয়

প্রশ্ন ৫. `storage` Event কোন Page পায়?

ক) যে Page পরিবর্তন করেছে

খ) একই Website-এর অন্য খোলা Page

গ) সব Website-এর সব Page

ঘ) শুধু Server

প্রশ্ন ৬. খোঁজার শব্দ লেখার সময় URL-এ রাখতে কোনটি উপযুক্ত, যাতে Back Button-এর ইতিহাস না ভরে?

ক) `history.pushState`

খ) `location.href = ...`

গ) `history.replaceState`

ঘ) `location.reload()`

প্রশ্ন ৭. ব্যবহারকারীর দেওয়া URL Link-এ বসানোর আগে কী যাচাই সবচেয়ে জরুরি?

ক) URL-এর দৈর্ঘ্য

খ) Protocol (`http:` / `https:`)

গ) Domain-এর বানান

ঘ) Port নম্বর

প্রশ্ন ৮. `new Blob(['বই']).size` কত?

ক) 6

খ) 2

গ) 1

ঘ) 4

প্রশ্ন ৯. সংরক্ষিত তথ্যে `version` রাখার উদ্দেশ্য?

ক) তথ্য Encrypt করা

খ) Storage-এর সীমা বাড়ানো

গ) App হালনাগাদের পর পুরোনো গঠনের তথ্য চিনে রূপান্তর করা

ঘ) Tab-এর সংখ্যা গোনা

প্রশ্ন ১০. Login Token রাখার জন্য `localStorage` কেন ঝুঁকিপূর্ণ?

ক) এটি ধীর

খ) এটি শুধু সংখ্যা রাখে

গ) Tab বন্ধ হলে মুছে যায়

ঘ) একই Website-এর যেকোনো Script, XSS-এর Script-সহ, পড়তে পারে

১৫.১৬ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–খ · ৩–গ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–ক · ৯–গ · ১০–ঘ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	Storage Browser দেয়; Node.js-এ নেই। বাকিগুলো ভাষার অংশ।
২	খ	অবৈধ লেখায় `URL` Constructor `TypeError` দেয়।
৩	গ	Reload-এ টেকে; Tab বন্ধ হলে যায়।
৪	ক	ত্রুটি ছোড়ে, কিছুই সংরক্ষিত হয় না; তাই `try` বাধ্যতামূলক।
৫	খ	পরিবর্তনকারী Page নিজে পায় না।
৬	গ	বর্তমান ইতিহাসের ধাপ বদলায়, নতুন ধাপ যোগ করে না।
৭	খ	`javascript:` URL Click-এ Code চালায়।
৮	ক	UTF-8-এ প্রতিটি বাংলা অক্ষর ৩ Byte।
৯	গ	পুরোনো Version চিনে Migration-এর ধাপ চালানো যায়।
১০	ঘ	Storage Encrypt করা নয় এবং Script-এর নাগালে।

১৫.১৭ অধ্যায়ের সারসংক্ষেপ

- Browser API (DOM, Storage, Location, History) ভাষার বাইরে; `URL`, `fetch`, `Blob` Browser ও Node.js দুটিতেই আছে। Feature Detection করুন।
- `URL` ও `URLSearchParams` ঠিকানা পড়ে, গড়ে, Encode করে ও যাচাই করে; URL-এর মানও Trust Boundary।
- `localStorage` দীর্ঘস্থায়ী ও সব Tab-এ, `sessionStorage` Tab-এর আয়ু পর্যন্ত; দুটিই লেখা রাখে, সীমিত, এবং ব্যর্থ হতে পারে।
- `storage` Event অন্য Tab-কে পরিবর্তন জানায়।
- সংরক্ষিত ও Export-এর তথ্যে `version` এবং পর্যায়ক্রমিক Migration।
- `Request` Method, Header ও Body জমা রাখে; JSON পাঠাতে `Content-Type`।
- History API অবস্থা URL-এ রাখে: লেখায় `replaceState`, নতুন ধাপে `pushState`, Back-এ `popstate`।
- `Blob` ও `File` তথ্যের টুকরো; `createObjectURL` দিয়ে Download, `file.text()` দিয়ে পড়া; ব্যবহারকারীর File যাচাই।
- Web Storage-এ গোপন তথ্য নয়; ব্যবহারকারীর URL-এ Protocol যাচাই।

পরের অধ্যায়ে Software Reliability: Testing-এর ধরন, Node.js Test Runner-এর গভীর ব্যবহার, Test Double, এবং নির্ভরযোগ্য Program তৈরির পদ্ধতি।

১৫.১৮ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Browser API	Browser-এর দেওয়া সুবিধা, ভাষার অংশ নয়।
Feature Detection	ব্যবহারের আগে API আছে কি না যাচাই।
Secure Context	`https://` বা `localhost`; কিছু API শুধু এখানে।
URLSearchParams	URL-এর Query অংশ পড়া ও লেখার Object।
`sessionStorage`	Tab-এর আয়ু পর্যন্ত টেকা Storage।
Quota	প্রতি Website-এর Storage-সীমা।
`storage` Event	অন্য Tab-এ Storage বদলালে আসা Event।
Migration	পুরোনো গঠনের তথ্য নতুন গঠনে রূপান্তর।
History API	Reload ছাড়া URL বদলানো ও Back সামলানো।
Blob	কাঁচা তথ্যের টুকরো, একটি ধরনসহ।
Object URL	Memory-র Blob-এর জন্য সাময়িক `blob:` ঠিকানা।
`Set`	প্রতিটি মান একবারই থাকে এমন সংগ্রহ।

১৫.১৯ আরও পড়ুন

- **MDN — URL:** `https://developer.mozilla.org/en-US/docs/Web/API/URL` — `URL` ও `URLSearchParams`।
- **MDN — Web Storage API:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API` — `localStorage`, `sessionStorage` ও `storage` Event।
- **MDN — Storage quotas and eviction criteria:** `https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria` — Storage-এর সীমা ও মুছে যাওয়া।
- **MDN — History API:** `https://developer.mozilla.org/en-US/docs/Web/API/History_API` — `pushState`, `replaceState`, `popstate`।
- **MDN — Blob:** `https://developer.mozilla.org/en-US/docs/Web/API/Blob` — Blob, File ও Object URL।

অধ্যায় ১৬

JavaScript Error Handling, Debugging, Testing এবং Software Reliability

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Software Reliability-র চারটি দিক—সঠিকতা, সহনশীলতা, পুনরুদ্ধার এবং পর্যবেক্ষণযোগ্যতা—ব্যাখ্যা করতে।
- `code`, `retryable` ও `cause`-সহ একটি Error-এর শ্রেণিবিন্যাস তৈরি করতে, এবং `AggregateError` সামলাতে।
- Invariant ও Assertion দিয়ে Program-এর ভেতরের নিয়ম পাহারা দিতে।
- Structured Log লিখতে, এবং একটি পুনরাবৃত্তিযোগ্য Debugging পদ্ধতি অনুসরণ করতে।
- Test-এর ধরন (Unit, Integration, End-to-End, Regression, Boundary) এবং Testing Pyramid বুঝতে।
- Node.js-এর `node:test` গভীরভাবে ব্যবহার করতে: `describe`, Hook, Subtest, Mock, Coverage।
- Stub, Spy, Mock ও Fake দিয়ে বাইরের নির্ভরতা নিয়ন্ত্রণ করে নিশ্চিত Test লিখতে।
- সাময়িক ত্রুটিতে Retry, ব্যর্থতায় Compensation, এবং দুবার পাঠানো অনুরোধে Idempotency নিশ্চিত করতে।
- একটি সম্পূর্ণ **BookingGuard** Booking Service তৈরি ও বহুস্তরে পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ১০: Promise, `async` / `await`, Retry ও Backoff।
- অধ্যায় ১১: `try` / `catch` / `finally`, নিজের Error, `cause`, প্রত্যাশিত বনাম অপ্রত্যাশিত ত্রুটি।
- অধ্যায় ১২–১৪: ES Module, Class, Dependency Injection, Fake Timer।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১১-এ আপনি একটি ত্রুটি ঘটলে কী করবেন তা শিখেছেন। এই অধ্যায়ের প্রশ্ন বড়: একটি Program কীভাবে **নির্ভরযোগ্য** হয়? একটি Booking Service-এর কথা ভাবুন। Payment Gateway মাঝে মাঝে সাময়িক ত্রুটি দেয়; ব্যবহারকারী অধৈর্য হয়ে Button দুবার চাপেন; Email Server বন্ধ থাকে; শেষ আসনটির জন্য দুজন একসঙ্গে চেষ্টা করেন। প্রতিটি পরিস্থিতিতে Program-কে সঠিক থাকতে হবে: দুবার টাকা কাটা যাবে না, আসন হারাবে না, এবং Email না গেলেও Booking ব্যর্থ হবে না।

এই ধরনের আচরণ শুধু সতর্কভাবে Code লিখে নিশ্চিত হয় না; প্রমাণ করতে হয়। প্রমাণের হাতিয়ার Automated Test, এবং এই অধ্যায়ের অর্ধেক Test নিয়ে: কী Test করবেন, কীভাবে বাইরের জগৎকে (Payment, Database, সময়) Test-এ নিয়ন্ত্রণে আনবেন, এবং কীভাবে একটি Bug আর কখনো ফিরে না আসে তা নিশ্চিত করবেন।

১৬.১ Software Reliability কী

দিক	প্রশ্ন	উদাহরণ
সঠিকতা (Correctness)	স্বাভাবিক অবস্থায় ফল কি ঠিক?	পাঁচ আসনে গ্রুপ ছাড় ঠিকমতো হিসাব
সহনশীলতা (Robustness)	অস্বাভাবিক ইনপুট বা পরিবেশে কি ভেঙে পড়ে?	ভুল Email, Payment-এর সাময়িক ত্রুটি
পুনরুদ্ধার (Recoverability)	ব্যর্থতার পর অবস্থা কি সঠিক থাকে?	Payment ব্যর্থ হলে সংরক্ষিত আসন ফেরত
পর্যবেক্ষণযোগ্যতা (Observability)	সমস্যা হলে কি জানা যায় কী হয়েছে?	প্রতিটি Retry ও ব্যর্থতার Log

অধ্যায় ১১-এর মূল নিয়মগুলো এখানে ভিত্তি: প্রত্যাশিত ত্রুটি নিজের ধরনে এবং কাছে ধরা; অপ্রত্যাশিত ত্রুটি ওপরে যেতে দেওয়া; `finally`-তে পরিষ্কার; খালি `catch` কখনো নয়। এই অধ্যায় সেগুলো আবার শেখাবে না; বরং বড় ব্যবস্থায় সেগুলো কীভাবে একসঙ্গে কাজ করে তা দেখাবে।

১৬.২ Error-এর শ্রেণিবিন্যাস: code, retryable এবং cause

বড় Application-এ ত্রুটির একটি পরিবার থাকে। প্রতিটি সদস্যের নামের সঙ্গে একটি স্থির `code` (যা Log ও Server-এর সাড়ায় যায়, এবং ভাষা বদলালেও বদলায় না) এবং প্রয়োজনে `retryable` (আবার চেষ্টা করলে সফল হতে পারে কি না) রাখলে সামলানোর সিদ্ধান্ত সহজ হয়।

error-family.js · JavaScript — চলবে: Browser Console ও Node.js

```
class AppError extends Error {
  constructor(message, { code, retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = new.target.name;
    this.code = code;
    this.retryable = retryable;
  }
}

class DatabaseError extends AppError {}
class SaveFailedError extends AppError {}

function causeChain(error) {
  const chain = [];
  for (let current = error; current; current = current.cause) {
    chain.push(`${current.name}: ${current.message}`);
  }
  return chain;
}

const low = new Error('connection reset');
const middle = new DatabaseError('INSERT ব্যর্থ', {
  code: 'DB_WRITE', retryable: true, cause: low,
});
const outer = new SaveFailedError('Booking সংরক্ষণ হয়নি', {
  code: 'SAVE', cause: middle,
});

console.log(causeChain(outer).join('\n  <- '));
console.log(outer instanceof AppError, outer.code, middle.retryable);
```

Output

```
SaveFailedError: Booking সংরক্ষণ হয়নি
  <- DatabaseError: INSERT ব্যর্থ
  <- Error: connection reset
true SAVE true
```

- `new.target.name` Constructor-কে যে Class দিয়ে ডাকা হয়েছে তার নাম দেয়। তাই প্রতিটি Child Class-এ আলাদা করে `this.name` লিখতে হয়নি (অধ্যায় ১৩-এর `this.name` সমস্যার স্থায়ী সমাধান)।
- **`cause`-এর শিকল** ধরে ওপর থেকে মূল কারণ পর্যন্ত যাওয়া যায়। ব্যবহারকারী দেখবেন শুধু ওপরের বার্তা; Log-এ যাবে পুরো শিকল।
- **ধরন দিয়ে পরিবার, `code` দিয়ে নির্দিষ্টতা:** `instanceof AppError` বলে এটি আমাদের প্রত্যাশিত ত্রুটি; `code` বলে ঠিক কোনটি।

একাধিক ত্রুটি একসঙ্গে ঘটলে JavaScript একটি **`AggregateError`** দেয়, যার `errors` Property-তে সবগুলো থাকে। যেমন `Promise.any`-এর সব Promise ব্যর্থ হলে:

aggregate.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const fail = (ms, message) =>
  new Promise((_, reject) => setTimeout(() => reject(new Error(message)), ms));

try {
  await Promise.any([fail(10, 'Server A বন্ধ'), fail(20, 'Server B বন্ধ')]);
} catch (error) {
  console.log(error.name, error.errors.length);
  console.log(error.errors.map((item) => item.message).join(' | '));
}
```

Output

```
AggregateError 2
Server A বন্ধ | Server B বন্ধ
```

১৬.৩ Invariant: ভেতরের নিয়মের পাহারা

ইনপুট যাচাই বাইরের তথ্য পাহারা দেয় (Trust Boundary, অধ্যায় ১১)। কিন্তু Program-এর ভেতরেও কিছু নিয়ম আছে, যা কখনো ভাঙার কথা নয়: টাকা স্থানান্তরের পর মোট টাকা অপরিবর্তিত; আসনের সংখ্যা কখনো ঋণাত্মক নয়। এমন নিয়মকে বলে **Invariant**। ভাঙলে তা অবশ্যই একটি Bug, তাই সঙ্গে সঙ্গে জোরে ব্যর্থ হওয়াই ভালো (Fail Fast), ভুল অবস্থায় এগোনোর চেয়ে।

invariant.js · JavaScript চলবে: Browser Console ও Node.js

```
function invariant(condition, message) {
  if (!condition) {
    throw new Error(`Invariant ভঙ্গ: ${message}`);
  }
}

function transfer(accounts, from, to, amount) {
  invariant(Number.isInteger(amount) && amount > 0, 'পরিমাণ ধনাত্মক পূর্ণসংখ্যা');
  const totalBefore = accounts[from] + accounts[to];
  const next = {
    ...accounts,
    [from]: accounts[from] - amount,
    [to]: accounts[to] + amount,
  };

  invariant(next[from] >= 0, `${from}-এর Balance ঋণাত্মক হবে`);
  invariant(next[from] + next[to] === totalBefore, 'মোট টাকা অপরিবর্তিত');
  return next;
}

console.log(JSON.stringify(transfer({ a: 100, b: 50 }, 'a', 'b', 30)));

try {
  transfer({ a: 100, b: 50 }, 'a', 'b', 500);
} catch (error) {
  console.log(error.message);
}
```

Output

```
{"a":70,"b":80}
Invariant ভঙ্গ: a-এর Balance ঋণাত্মক হবে
```

শেষ Invariant ("মোট অপরিবর্তিত") এখন অপ্রয়োজনীয় মনে হতে পারে; কিন্তু ছয় মাস পরে কেউ Fee কাটার নিয়ম যোগ করলে এবং ভুল করে টাকা "হারিয়ে" ফেললে, এটিই প্রথম ধরবে। Node.js-এ একই কাজের জন্য Built-in `assert` Module আছে; Browser-এ `console.assert` শুধু বার্তা দেখায়, থামায় না।

১৬.৪ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা

অধ্যায় ১০ ও ১১-এ Asynchronous ত্রুটি এবং Browser-এর `unhandledrejection` দেখেছেন। Node.js-এ একই শেষ প্রতিরক্ষা:

last-defense.js · JavaScript চলবে: Node.js

```
process.on('unhandledRejection', (reason) => {
  console.log(`ধরা পড়েনি: ${reason.message}`);
});

async function saveReport() {
  throw new Error('Disk ভর্তি');
}

saveReport();
console.log('Program চলছে');
```

Output

```
Program চলছে
ধরা পড়েনি: Disk ভর্তি
```

Server-এ এখানে সাধারণত ত্রুটিটি Log করে Process-কে ব্যর্থ অবস্থায় বন্ধ করা হয় (`process.exitCode = 1`), যাতে একটি Process Manager সেটিকে পরিষ্কার অবস্থায় আবার চালু করে। অজানা অবস্থায় চলতে থাকা একটি Server বন্ধ হয়ে যাওয়া Server-এর চেয়ে বেশি বিপজ্জনক: সে ভুল তথ্য সংরক্ষণ করতে পারে।

১৬.৫ Debugging: একটি পুনরাবৃত্তিযোগ্য পদ্ধতি

অধ্যায় ১১-এ DevTools দেখেছেন। বড় Bug-এর জন্য হাতিয়ারের চেয়ে পদ্ধতি বেশি গুরুত্বপূর্ণ:

১. **নিশ্চিতভাবে আবার ঘটান।** কোন ইনপুট, কোন ক্রম, কোন পরিবেশ। যে Bug নিজের ইচ্ছায় ঘটানো যায় না, তা ঠিক হয়েছে কি না প্রমাণও করা যায় না।

২. **সবচেয়ে ছোট উদাহরণে নামান।** একটি একটি অংশ সরিয়ে দেখুন Bug থাকে কি না।

৩. **ধারণা লিখুন, তারপর প্রমাণ খুঁজুন।** "Payment দুবার কাটছে, কারণ Retry-র সময় আগের অনুরোধ আসলে সফল ছিল"—তারপর Log বা Breakpoint দিয়ে যাচাই।

৪. **অর্ধেক করে খুঁজুন (Binary Search)।** Program-এর মাঝখানে মান দেখুন: ঠিক থাকলে Bug পরের অর্ধেকে, নইলে আগের অর্ধেকে। কোন পরিবর্তনে Bug এসেছে তা খুঁজতে Git-এর `git bisect` একই কৌশল Commit-এর ওপর চালায়।

৫. **ঠিক করার আগে একটি ব্যর্থ Test লিখুন** (Regression Test), তারপর ঠিক করুন, তারপর Test পাস করছে দেখুন।

পর্যবেক্ষণের জন্য Log অপরিহার্য, বিশেষত যেখানে Debugger লাগানো যায় না (Server, ব্যবহারকারীর Browser)। মানুষের পড়ার বাক্যের চেয়ে **Structured Log**—প্রতিটি লাইন একটি JSON Object—অনেক বেশি কাজের, কারণ যন্ত্র দিয়ে খোঁজা ও গোনা যায়:

logger.js · JavaScript চলবে: Browser Console ও Node.js

```
function createLogger(context, write = console.log) {
  return (level, event, data = {}) =>
    write(JSON.stringify({ level, event, ...context, ...data }));
}

const log = createLogger({ service: 'booking' });

log('info', 'created', { id: 'B-1', seats: 2 });
log('warn', 'payment-retry', { attempt: 1 });
```

Output

```
{"level":"info","event":"created","service":"booking","id":"B-1","seats":2}
{"level":"warn","event":"payment-retry","service":"booking","attempt":1}
```

বাস্তব Log-এ সময় (`time`) এবং অনুরোধের ID-ও থাকে, যাতে একটি অনুরোধের সব লাইন একসঙ্গে খোঁজা যায়। **Log-এ কখনো গোপন তথ্য নয়:** Password, Card নম্বর, Token। `write` Inject করা যায়, তাই Test-এ Log-কে একটি Array-তে জমিয়ে যাচাই করা যায় (Project দেখুন)।

১৬.৬ Test-এর ধরন এবং Testing Pyramid

ধরন	কী যাচাই করে	গতি	উদাহরণ
Unit	একটি Function বা Class, বিচ্ছিন্নভাবে	খুব দ্রুত	`priceFor(5)`
Integration	কয়েকটি অংশ একসঙ্গে	দ্রুত	Service + Repository
End-to-End (E2E)	আসল Browser-এ ব্যবহারকারীর পুরো যাত্রা	ধীর	Form পূরণ থেকে নিশ্চিতকরণ
Regression	একটি পুরোনো Bug আর ফেরেনি	ধরনভেদে	দুবার Click-এ দুবার Payment নয়
Boundary	সীমার দুই পাশ	দ্রুত	০, ১, ১০, ১১ আসন

Testing Pyramid-এর পরামর্শ: নিচে অনেক Unit Test, মাঝে কিছু Integration Test, শীর্ষে অল্প কয়েকটি E2E Test। নিচের Test দ্রুত ও নিশ্চিত, এবং ব্যর্থ হলে ঠিক কোথায় সমস্যা তা দেখায়; ওপরের Test বাস্তবের সবচেয়ে কাছাকাছি, কিন্তু ধীর ও ভঙ্গুর। এই বইয়ের অধ্যায় ৯–১২ ও ১৫-এর `browser-check.mjs` E2E ধরনের; প্রতিটি Project-এর `node --test` Unit ও Integration।

Boundary Test সবচেয়ে বেশি Bug ধরে, কারণ ভুল প্রায়ই সীমায় থাকে: `<` না `<=`। "আসন ১ থেকে ১০" নিয়মের জন্য ০, ১, ১০ ও ১১—চারটি মান যাচাই যথেষ্ট; ৫ বা ৭-এর মতো মাঝের মান নতুন কিছু প্রমাণ করে না।

১৬.৭ node:test গভীরভাবে

এ পর্যন্ত আপনি `test()` ও `assert` ব্যবহার করেছেন। `node:test`-এ আরও আছে:

price.test.mjs · JavaScript — চলবে: Node.js

```
import { describe, it, beforeEach } from 'node:test';
import assert from 'node:assert/strict';

const SEAT_PRICE = 350;
const priceFor = (seats) => {
  const subtotal = seats * SEAT_PRICE;
  return seats >= 5 ? Math.round(subtotal * 0.9) : subtotal;
};

describe('priceFor', () => {
  let seatCounts;

  beforeEach(() => {
    seatCounts = [];
  });

  it('৪ আসনে ছাড় নেই', () => {
    assert.equal(priceFor(4), 1400);
  });

  it('৫ আসন থেকে ১০% গ্রুপ ছাড়', () => {
    assert.equal(priceFor(5), 1575);
  });

  it('প্রতিটি Test নতুন অবস্থায় শুরু হয়', () => {
    seatCounts.push(1);
    assert.deepEqual(seatCounts, [1]);
  });

  it.todo('আসনের সর্বোচ্চ সীমা');
});
```

Terminal

```
node --test "examples/tests/*.test.mjs"
```

- **`describe`** ও **`it`** Test-গুলোকে দলে সাজায়; Report-এ দলের নাম দেখা যায়। `test()` ও `it()` একই জিনিস।
- **Hook**: `beforeEach` প্রতিটি Test-এর আগে চলে, তাই প্রতিটি Test নতুন, পরিষ্কার অবস্থায় শুরু হয়। Test পরস্পরের ওপর নির্ভর করলে একটির ব্যর্থতা অন্যগুলোকে বিভ্রান্তভাবে ব্যর্থ করে। `before`, `after`, `afterEach`-ও আছে।

- `it.todo` লেখা বাকি Test-এর জায়গা রাখে; Report-এ "todo" দেখায়, ব্যর্থ হয় না। `it.skip` একটি Test সাময়িকভাবে বন্ধ রাখে, কিন্তু দীর্ঘদিন Skip রাখা একটি সতর্কসংকেত।
- **চালানো**: Argument ছাড়া `node --test` বর্তমান Folder ও তার ভেতরের সব `*.test.mjs` খুঁজে চালায়; নির্দিষ্ট File-গুলো চাইলে উদ্ধৃতিচিহ্নের ভেতরে একটি Glob Pattern দিন, যেমন ওপরের Command। (Node.js 22-এ Folder-এর নাম সরাসরি দিলে কাজ করে না।) একটি নির্দিষ্ট Test: `node --test --test-name-pattern="গ্রুপ"`। লেখার সময় প্রতিবার সংরক্ষণে আবার চালাতে: `node --test --watch`।

এই অধ্যায়ের সব উদাহরণ-Test (`examples/tests/` এবং সমাধানের Test) Companion Source Code-এ একসঙ্গে চালিয়ে যাচাই করা হয়েছে।

ব্যর্থ Assertion কী তথ্য দেয়, দেখুন:

assert-message.mjs · JavaScript — চলবে: Node.js

```
import assert from 'node:assert/strict';

const priceFor = (seats) => seats * 350;

try {
  assert.equal(priceFor(5), 1575, 'পাঁচ আসনে গ্রুপ ছাড় প্রত্যাশিত');
} catch (error) {
  console.log(error.code, error.actual, error.expected);
  console.log(error.message);
}
```

Output

```
ERR_ASSERTION 1750 1575
পাঁচ আসনে গ্রুপ ছাড় প্রত্যাশিত

1750 !== 1575
```

`actual` ও `expected` দেখায় কী পাওয়া গেল আর কী আশা করা হয়েছিল, এবং তৃতীয় Argument-এর বার্তা বলে **কেন** তা আশা করা হয়েছিল। Node.js 22 নিজের বার্তার নিচে তুলনাটিও (`1750 !== 1575`) যোগ করে দেয়। ভালো Test-এর ব্যর্থতা নিজেই একটি Bug Report।

Coverage: কোন Code কোনো Test ছোঁয়নি

Terminal

```
node --test --experimental-test-coverage
```

এই Command প্রতিটি File-এর কত শতাংশ লাইন, শাখা (Branch) ও Function Test চলার সময় ব্যবহৃত হয়েছে তার একটি সারণি দেখায়, এবং কোন লাইনগুলো কখনো চলেনি তা উল্লেখ করে। Project-এ এটি চালিয়ে অচলিত লাইনগুলো দেখলে প্রায়ই একটি অপরীক্ষিত ত্রুটির পথ পাওয়া যায়। তবে ১০০% Coverage মানে Bug-মুক্ত নয়: একটি লাইন চলেছে,

কিন্তু তার ফল যাচাই হয়েছে কি না Coverage জানে না। Coverage দেখায় কোথায় Test **নেই**, কোথায় যথেষ্ট আছে তা নয়।

১৬.৮ Test Double: Stub, Spy, Mock এবং Fake

একটি Booking Service আসল Payment Gateway-তে টাকা কাটে, আসল Database-এ লেখে, আসল Email পাঠায়। Test-এ এগুলো ব্যবহার করা ধীর, ব্যয়বহুল, এবং অনিশ্চিত। তাদের জায়গায় নিয়ন্ত্রিত বিকল্প দেওয়া হয়, যাদের সাধারণ নাম **Test Double** (সিনেমার Stunt Double-এর মতো)।

ধরন	কী করে	কখন
Stub	নির্দিষ্ট উত্তর দেয়	"Payment সফল" বা "Payment ব্যর্থ" পরিস্থিতি তৈরি
Spy	আসলটির মতো কাজ করে, এবং প্রতিটি ডাক লিখে রাখে	কতবার, কী Argument-এ ডাকা হলো
Mock	আগে থেকে ঠিক করা প্রত্যাশা যাচাই করে	"Email ঠিক একবার পাঠানো হয়েছে"
Fake	সরল কিন্তু কার্যকর বিকল্প বাস্তবায়ন	Database-এর বদলে Memory-র Repository

doubles.test.mjs · JavaScript — চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

async function placeOrder(order, { payment, mailer }) {
  const receipt = await payment.charge(order.amount);
  await mailer.send(order.email, `রসিদ ${receipt}`);
  return receipt;
}

test('Stub ও Mock: Payment-এর নির্দিষ্ট উত্তর, Email ঠিক একবার', async () => {
  const payment = { charge: async () => 'PAY-1' };
  const mailer = { send: mock.fn(async () => {}) };

  const order = { amount: 700, email: 'a@b.co' };
  const receipt = await placeOrder(order, { payment, mailer });

  assert.equal(receipt, 'PAY-1');
  assert.equal(mailer.send.mock.callCount(), 1);
  assert.deepEqual(mailer.send.mock.calls[0].arguments, ['a@b.co', 'রসিদ PAY-1']);
});

test('Stub-এর ব্যর্থতা: Payment ব্যর্থ হলে Email যায় না', async () => {
  const payment = { charge: async () => { throw new Error('declined'); } };
  const mailer = { send: mock.fn() };
```

```
  const order = { amount: 700, email: 'a@b.co' };
  await assert.rejects(placeOrder(order, { payment, mailer }), /declined/);
  assert.equal(mailer.send.mock.callCount(), 0);
});

test('Spy: একটি বিদ্যমান Object-এর Method-এর ডাক গোনা', () => {
  const calculator = { add: (a, b) => a + b };
  const spy = mock.method(calculator, 'add');

  assert.equal(calculator.add(2, 3), 5);
  assert.equal(spy.mock.callCount(), 1);
  spy.mock.restore();
});
```

- `mock.fn(impl)` একটি Function দেয় যা `impl`-এর মতো কাজ করে এবং প্রতিটি ডাক `mock.calls`-এ রাখে।
- `mock.method(object, 'name')` একটি বিদ্যমান Method-কে Spy-তে মোড়ায়; কাজ শেষে `restore()` আসলটি ফিরিয়ে দেয়।
- **নির্ভরতা Parameter হিসেবে** (`{ payment, mailer }`): অধ্যায় ১২-এর Dependency Injection ছাড়া এর কোনোটিই সম্ভব হতো না। Test-যোগ্য Code লেখার প্রথম শর্ত বাইরের জিনিস ভেতরে তৈরি না করে বাইরে থেকে নেওয়া।
- **সময়ও একটি নির্ভরতা।** `new Date()` সরাসরি ব্যবহার করলে Test-এর ফল তারিখের ওপর নির্ভর করে; একটি `clock` Function Inject করুন, অথবা অধ্যায় ১৪-এর Fake Timer।

সতর্কতা: Test Double যত বেশি, Test তত কম বাস্তবের সঙ্গে মেলে। প্রতিটি Mock একটি অনুমান: "আসল Payment Gateway এভাবেই আচরণ করে।" অনুমান ভুল হলে Test পাস করবে, Production ব্যর্থ হবে। তাই Fake-কে আসলের মতো আচরণ করান, এবং অন্তত কয়েকটি Integration বা E2E Test আসল অংশ দিয়ে চালান।

১৬.৯ ভালো Test-এর বৈশিষ্ট্য

- **একটি Test, একটি আচরণ;** নামেই সেই আচরণ: "Payment ব্যর্থ হলে সংরক্ষিত আসন ফেরত যায়"।
- **Arrange, Act, Assert:** প্রস্তুতি, একটি কাজ, যাচাই—তিনটি স্পষ্ট অংশ।
- **নিশ্চিত (Deterministic):** একই Code-এ সব সময় একই ফল। সময়, Random, Network—সব নিয়ন্ত্রিত। মাঝে মাঝে ব্যর্থ হওয়া (Flaky) Test-কে সবাই উপেক্ষা করতে শেখে, এবং তখন সেটি আসল Bug-ও ধরে না।
- **স্বাধীন:** কোনো Test অন্য Test-এর রেখে যাওয়া অবস্থার ওপর নির্ভর করে না; যেকোনো ক্রমে চলে।
- **আচরণ যাচাই করে, ভেতরের গঠন নয়:** Private Function-এর নাম বদলালে Test ভাঙা উচিত নয়, আচরণ বদলালে উচিত।
- **ব্যর্থতার পথও:** প্রতিটি `throw` এবং প্রতিটি `catch`-এর জন্য অন্তত একটি Test।
- **প্রতিটি Bug-এর জন্য একটি Regression Test,** যা আগের Code-এ ব্যর্থ হতো।

১৬.১০ সাধারণ ভুল

ভুল	সমস্যা	সমাধান
খালি `catch`	ত্রুটি অদৃশ্য	ধরুন যাচাই, অচেনা হলে আবার `throw`
সব ত্রুটিতে Retry	`400` -এ অকারণ চেষ্টা; দ্বিগুণ কাজ	শুধু `retryable`
Retry-তে Idempotency নেই	দুবার Payment	Idempotency Key
আংশিক ব্যর্থতায় ফেরত নেই	আসন বা টাকা "হারিয়ে" যায়	Compensation (ফেরত দেওয়া)
অপ্রধান কাজের ব্যর্থতায় পুরো কাজ ব্যর্থ	Email না যাওয়ায় Booking বাতিল	অপ্রধান কাজ আলাদা `try` , Log
Log-এ গোপন তথ্য	তথ্য ফাঁস	শুধু প্রয়োজনীয়, নিরাপদ Field
`new Date()` , `Math.random()` সরাসরি	Flaky Test	Inject করুন
Test-এ আসল Network	ধীর, অনিশ্চিত	Stub, Fake, Local Server
একটি Test-এ অনেক আচরণ	কোনটি ভাঙল বোঝা যায় না	একটি Test, একটি আচরণ
শুধু সফল পথের Test	ত্রুটির পথ অপরীক্ষিত	প্রতিটি `throw` -এর Test

১৬.১১ Debugging নির্দেশিকা

একটি Test ব্যর্থ হলে:

- **প্রথমে ব্যর্থতার বার্তা পড়ুন:** কোন Test, `actual` , `expected` , এবং Stack-এর প্রথম নিজস্ব লাইন।
- **শুধু সেই Test চালান:** `node --test --test-name-pattern="..."` ; দ্রুত পুনরাবৃত্তি।
- **Test ভুল, না Code ভুল?** প্রত্যাশিত মানটি চাহিদা থেকে আবার হিসাব করুন। প্রায়ই Test-এর প্রত্যাশাই ভুল।
- **মাঝে মাঝে ব্যর্থ হয়?** সময়, Random, ক্রম, বা Test-এর মধ্যে ভাগ করা অবস্থা খুঁজুন; `beforeEach` -এ নতুন অবস্থা তৈরি করুন।
- **Mock-এর ডাক মিলছে না?** `mock.calls` -এর পুরো তালিকা Log করুন; প্রায়ই Argument-এর ক্রম বা ধরন আলাদা।
- **Test Debugger-এ:** `node --inspect-brk --test file.test.mjs` চালিয়ে Chrome-এর `chrome://inspect` থেকে Breakpoint দেওয়া যায়।

১৬.১২ পেশাদার অভ্যাস

- **প্রতিটি বাইরের নির্ভরতা Inject করুন:** Payment, Storage, Network, Clock, Logger।

- **ত্রুটির পরিবার, স্থির `code` এবং `retryable`।**
- **Retry শুধু সাময়িক ত্রুটিতে, সীমিত, Backoff-সহ, এবং Idempotency Key দিয়ে।**
- **বহু-ধাপের কাজে প্রতিটি ধাপের ব্যর্থতার জন্য ফেরত দেওয়ার পথ** (Compensation)।
- **প্রধান ও অপ্রধান কাজ আলাদা:** Booking সফল হলে Email-এর ব্যর্থতা শুধু Log।
- **Structured Log, গোপন তথ্য ছাড়া।**
- **Testing Pyramid:** অনেক দ্রুত Unit Test, কিছু Integration, অল্প E2E।
- **Bug মানে প্রথমে একটি ব্যর্থ Test, তারপর সংশোধন।**
- **Continuous Integration:** প্রতিটি পরিবর্তনে সব Test স্বয়ংক্রিয়ভাবে চালানো (GitHub Actions-এর মতো সেবা); লাল হলে Merge নয়।

১৬.১৩ Project: BookingGuard

একটি অনুষ্ঠানের আসন Booking Service। আসন সীমিত; প্রতিটি Booking-এ Payment কাটা হয় এবং Email-এ নিশ্চিতকরণ যায়। লক্ষ্য নির্ভরযোগ্যতা: Payment-এর সাময়িক ত্রুটিতে Retry, স্থায়ী ব্যর্থতায় সংরক্ষিত আসন ফেরত, দুবার পাঠানো অনুরোধে একবারই Payment, Email ব্যর্থ হলেও Booking অক্ষত, এবং সব ঘটনার Structured Log। প্রতিটি আচরণ Test Double দিয়ে প্রমাণিত।

Project-এর চাহিদা

- নাম কমপক্ষে ২ অক্ষর, Email-এর গঠন, আসন ১ থেকে ১০; সব ভুল একসঙ্গে (`ValidationError`)।
- আসনপ্রতি ৩৫০ টাকা; ৫ বা বেশি আসনে ১০% গ্রুপ ছাড়।
- আসন না থাকলে `SoldOutError`, এবং Payment-এর চেষ্টাই হবে না।
- Payment-এর সাময়িক ত্রুটিতে ২ বার পর্যন্ত Retry (১০০ ও ২০০ মিলিসেকেন্ড বিরতি); স্থায়ী বা শেষ ব্যর্থতায় আসন ফেরত।
- একই অনুরোধ-ID আবার এলে আগের Booking ফেরত; Payment আবার নয়।
- Email ব্যর্থ হলে Booking সফল থাকে; একটি `warn` Log।
- বাতিলে টাকা ফেরত (Refund) এবং আসন ফেরত; দুবার বাতিল নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
booking-guard/
├── src/
│   ├── errors.mjs              AppError পরিবার
│   ├── booking-core.mjs        যাচাই ও দাম (Pure)
│   ├── booking-service.mjs     Booking-এর প্রবাহ, Retry, Compensation
│   └── memory-repository.mjs   Database-এর Fake
├── demo.mjs
├── README.md
└── tests/
    ├── core.test.mjs
    ├── errors.test.mjs
    └── service.test.mjs
```

src/errors.mjs ও src/booking-core.mjs

errors.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export class AppError extends Error {
  constructor(message, { code, retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = new.target.name;
    this.code = code;
    this.retryable = retryable;
  }
}

export class ValidationError extends AppError {
  constructor(fields) {
    super('Booking-এর তথ্য ঠিক নেই।', { code: 'VALIDATION' });
    this.fields = fields;
  }
}

export class SoldOutError extends AppError {
  constructor(left) {
    super(`যথেষ্ট আসন নেই; বাকি ${left}টি।`, { code: 'SOLD_OUT' });
    this.left = left;
  }
}

export class PaymentError extends AppError {
  constructor(message, { retryable = false, cause } = {}) {
    super(message, { code: 'PAYMENT', retryable, cause });
  }
}
```

booking-core.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { ValidationError } from './errors.mjs';

export const MAX_SEATS = 10;
export const SEAT_PRICE = 350;
export const GROUP_SIZE = 5;
const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export function validateRequest(raw) {
  const customer = String(raw?.customer ?? '').trim();
  const email = String(raw?.email ?? '').trim().toLowerCase();
  const seats = raw?.seats;
  const fields = {};

  if (customer.length < 2) {
    fields.customer = 'নাম কমপক্ষে 2 অক্ষরের হতে হবে।';
  }
  if (!EMAIL.test(email)) {
    fields.email = 'সঠিক একটি Email লিখুন।';
  }
  if (!Number.isInteger(seats) || seats < 1 || seats > MAX_SEATS) {
    fields.seats = `আসন 1 থেকে ${MAX_SEATS}-এর মধ্যে হতে হবে।`;
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
  }
  return { customer, email, seats };
}

export function priceFor(seats) {
  const subtotal = seats * SEAT_PRICE;
  return seats >= GROUP_SIZE ? Math.round(subtotal * 0.9) : subtotal;
}
```

src/memory-repository.mjs

memory-repository.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { SoldOutError } from './errors.mjs';

// Database-এর Fake: একই আচরণ, Memory-তে।
// আসল Database-এর Adapter-কেও এই চারটি Method দিতে হবে।
export function createMemoryRepository(capacity) {
  let available = capacity;
  const bookings = new Map();

  return {
    async reserve(seats) {
```

```
      if (seats > available) {
        throw new SoldOutError(available);
      }
      available -= seats;
    },
    async release(seats) {
      available += seats;
    },
    async save(booking) {
      bookings.set(booking.id, structuredClone(booking));
    },
    async find(id) {
      return bookings.has(id) ? structuredClone(bookings.get(id)) : null;
    },
    get available() {
      return available;
    },
  };
}
```

`reserve`-এ যাচাই ও বিয়োগের মাঝে কোনো `await` নেই, তাই একসঙ্গে দুটি অনুরোধ এলেও (অধ্যায় ১০) একটির যাচাই ও বিয়োগ শেষ হওয়ার আগে অন্যটি ঢুকতে পারে না: শেষ আসন দুজনকে দেওয়া হবে না। `structuredClone` (অধ্যায় ১২) নিশ্চিত করে যে বাইরের Code সংরক্ষিত Booking-কে সরাসরি বদলাতে পারে না, ঠিক আসল Database-এর মতো। Fake-এর এই আসলের-মতো আচরণই Test-কে বিশ্বাসযোগ্য করে।

src/booking-service.mjs

দুই অংশ মিলে সম্পূর্ণ File।

booking-service.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { priceFor, validateRequest } from './booking-core.mjs';
import { AppError, PaymentError } from './errors.mjs';

const realSleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

export function createBookingService({
  repository,
  payment,
  notifier,
  clock = () => new Date(),
  sleep = realSleep,
  log = () => {},
  retries = 2,
}) {
  async function chargeWithRetry(request, amount, requestId) {
    for (let attempt = 0; ; attempt += 1) {
      try {
```

```
      const order = { amount, email: request.email, key: requestId };
      return await payment.charge(order);
    } catch (error) {
      const retryable = error instanceof PaymentError && error.retryable;
      if (!retryable || attempt >= retries) {
        throw error;
      }
      log('warn', 'payment-retry', { requestId, attempt: attempt + 1 });
      await sleep(100 * 2 ** attempt);
    }
  }
}

async function notify(booking) {
  try {
    await notifier.send(booking.email, `Booking ${booking.id} নিশ্চিত`);
  } catch (error) {
    log('warn', 'notify-failed', { id: booking.id, reason: error.message });
  }
}

// ...
```

booking-service.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) — চলবে: Browser Console ও Node.js

```
// ...

async function book(raw, requestId) {
  const existing = await repository.find(requestId);
  if (existing) {
    return existing;
  }

  const request = validateRequest(raw);
  const amount = priceFor(request.seats);
  await repository.reserve(request.seats);

  let receipt;
  try {
    receipt = await chargeWithRetry(request, amount, requestId);
  } catch (error) {
    await repository.release(request.seats);
    const code = error.code ?? 'UNKNOWN';
    log('error', 'payment-failed', { requestId, code });
    throw error;
  }

  const booking = {
    id: requestId, ...request, amount, receipt,
```

```
      status: 'confirmed', createdAt: clock().toISOString(),
    };
    await repository.save(booking);
    const { seats } = request;
    log('info', 'booking-confirmed', { id: requestId, seats, amount });
    await notify(booking);
    return booking;
  }

  async function cancel(id) {
    const booking = await repository.find(id);
    if (!booking) {
      throw new AppError(`${id} পাওয়া যায়নি।`, { code: 'NOT_FOUND' });
    }
    if (booking.status === 'cancelled') {
      throw new AppError(`${id} আগেই বাতিল।`, { code: 'ALREADY_CANCELLED' });
    }
    await payment.refund(booking.receipt);
    await repository.release(booking.seats);
    const cancelled = { ...booking, status: 'cancelled' };
    await repository.save(cancelled);
    log('info', 'booking-cancelled', { id });
    return cancelled;
  }

  return { book, cancel };
}
```

- **Factory ও Closure (অধ্যায় ১৪):** `createBookingService` সব নির্ভরতা একবার নেয়; ফেরত দেওয়া `book` ও `cancel` সেগুলো মনে রাখে। প্রতিটি নির্ভরতা Inject করা, এমনকি `sleep` ও `clock`।
- **ক্রম গুরুত্বপূর্ণ।** প্রথমে Idempotency যাচাই, তারপর ইনপুট যাচাই (সস্তা, কোনো Side Effect নেই), তারপর আসন সংরক্ষণ, তারপর Payment। আসন না থাকলে Payment-এর চেষ্টাই হয় না।
- **Retry** শুধু `retryable` `PaymentError`-এ, Backoff ১০০, ২০০; প্রতিটি Retry একটি `warn` Log। Payment-কে `key: requestId` পাঠানো হয়: আসল Payment Gateway এই **Idempotency Key** দেখে একই অনুরোধ দুবার এলে দ্বিতীয়বার টাকা কাটে না, যা Timeout-এর পর Retry-তে জরুরি (অধ্যায় ১০: সময়সীমা পার হওয়া মানে অনুরোধ ব্যর্থ নয়, শুধু উত্তর আসেনি)।
- **Compensation:** Payment চূড়ান্তভাবে ব্যর্থ হলে সংরক্ষিত আসন `release` হয়, তারপর ত্রুটি আবার ছোড়া হয়। কোনো আসন "আটকে" থাকে না।
- **প্রধান বনাম অপ্রধান:** Booking সংরক্ষণের পরে `notify` তার নিজের `try`-তে; Email ব্যর্থ হলে শুধু Log, Booking ফেরত যায়।
- `cancel` একই নীতি: প্রত্যাশিত ত্রুটির `code` (`NOT_FOUND`, `ALREADY_CANCELLED`), Refund, আসন ফেরত, নতুন অবস্থা সংরক্ষণ।

demo.mjs

demo.mjs · JavaScript — চলবে: Node.js

```
import { createBookingService } from './src/booking-service.mjs';
import { createMemoryRepository } from './src/memory-repository.mjs';
import { PaymentError } from './src/errors.mjs';

let charges = 0;
const payment = {
  async charge() {
    charges += 1;
    if (charges === 1) {
      throw new PaymentError('Gateway সাময়িক ব্যস্ত', { retryable: true });
    }
    return `PAY-${charges}`;
  },
  async refund() {},
};

const repository = createMemoryRepository(10);
const service = createBookingService({
  repository,
  payment,
  notifier: { send: async () => { throw new Error('SMTP বন্ধ'); } },
  clock: () => new Date('2026-12-01T10:00:00Z'),
  sleep: async () => {},
  log: (level, event, data) =>
    console.log(JSON.stringify({ level, event, ...data })),
});

const request = { customer: 'Sara', email: 'Sara@Example.com', seats: 5 };
const first = await service.book(request, 'REQ-1');
const again = await service.book(request, 'REQ-1');
const { available } = repository;
console.log(first.amount, first.receipt, again.receipt, charges, available);

const failing = [[{ ...request, seats: 6 }, 'REQ-2'], [{ seats: 0 }, 'REQ-3']];
for (const [raw, id] of failing) {
  try {
    await service.book(raw, id);
  } catch (error) {
    const detail = error.left ?? Object.keys(error.fields).join(',');
    console.log(error.name, error.code, detail);
  }
}
```

Output

```
{"level":"warn","event":"payment-retry","requestId":"REQ-1","attempt":1}
{"level":"info","event":"booking-confirmed","id":"REQ-1","seats":5,"amount":1575}
{"level":"warn","event":"notify-failed","id":"REQ-1","reason":"SMTP বন্ধ"}
1575 PAY-2 PAY-2 2 5
SoldOutError SOLD_OUT 5
ValidationError VALIDATION customer,email,seats
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **প্রথম Booking-এর তিনটি Log** পুরো গল্প বলে: Payment প্রথমবার ব্যর্থ, Retry-তে সফল; Booking নিশ্চিত; Email ব্যর্থ, কিন্তু Booking অক্ষত। ৫ আসনে গ্রুপ ছাড়: ১৭৫০-এর ৯০% = ১৫৭৫।
- **দ্বিতীয়বার একই** `REQ-1`: একই রসিদ `PAY-2`, Payment-এর মোট ডাক এখনো ২ (একটি ব্যর্থ, একটি সফল), নতুন কোনো Log নেই। ব্যবহারকারী দুবার Click করলেও টাকা একবার।
- **বাকি ৫ আসন:** ৬ আসনের অনুরোধ `SoldOutError`, `left` ৫; Payment ডাকাই হয়নি।
- **অবৈধ অনুরোধ:** তিনটি ঘরের ভুল একসঙ্গে।
- `sleep: async () => {}`: প্রদর্শনীতে Backoff-এর আসল অপেক্ষা বাদ; Test এই `sleep`-কে Spy বানিয়ে যাচাই করে ঠিক ১০০ ও ২০০ মিলিসেকেন্ড চাওয়া হয়েছে কি না।
- **Email** `Sara@Example.com` **ছোট হাতের অক্ষরে** সংরক্ষিত, যাতে পরে খোঁজা ও তুলনা নির্ভরযোগ্য হয়।

চালানো এবং Test

Terminal

```
node demo.mjs
node --test
node --test --experimental-test-coverage
```

Companion Source Code-এর ২৪টি Test তিন স্তরে সাজানো:

File	স্তর	যা প্রমাণ করে
`core.test.mjs`	Unit, Boundary	০, ১, ১০, ১১ আসন; ৪ ও ৫ আসনের দাম; সব ভুল একসঙ্গে
`errors.test.mjs`	Unit	নাম, `code`, `retryable`, `cause`, `instanceof` শিকল
`service.test.mjs`	Integration (Fake Repository, Stub Payment, Mock Notifier, Spy Sleep)	Retry ও Backoff, Compensation, Sold Out-এ Payment নয়, Idempotency (Regression), Email ব্যর্থতা, বাতিল ও Refund, একসঙ্গে অনেক অনুরোধে আসনের সঠিক হিসাব

`--experimental-test-coverage` দিয়ে চালানোর আসল ফলের `src` অংশ (Node.js 22.16):

Terminal

```
# file                  | line % | branch % | funcs % | uncovered lines
#  booking-core.mjs     | 100.00 |   100.00 |  100.00 |
#  booking-service.mjs  | 100.00 |    96.15 |   62.50 |
#  errors.mjs           | 100.00 |   100.00 |  100.00 |
#  memory-repository.mjs | 100.00 |   100.00 |  100.00 |
```

`booking-service.mjs`-এ Function-এর Coverage ৬২.৫%, যদিও সব লাইন চলেছে। কারণটি শিক্ষণীয়: Default `sleep`, `clock` ও `log` Function কখনো চলেনি, কারণ প্রতিটি Test নিজের নিয়ন্ত্রিত বিকল্প দিয়েছে। এটি কোনো ফাঁক নয়, বরং Dependency Injection-এর প্রমাণ; আসল `sleep` ও `clock` পরীক্ষিত হয় শুধু Integration বা E2E স্তরে। Coverage-এর সংখ্যা তাই সব সময় পড়তে হয় তার কারণসহ।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Payment ব্যর্থতার পর আসন কমে থাকে	Compensation নেই	`catch`-এ `release`
দুবার Click-এ দুটি Booking	Idempotency যাচাই নেই	`find(requestId)` আগে
Email না যাওয়ায় Booking ব্যর্থ	`notify` আলাদা `try`-তে নেই	অপ্রধান কাজ আলাদা
Test ধীর (Backoff-এর অপেক্ষা)	আসল `sleep`	`sleep` Inject করুন
তারিখের ওপর নির্ভর করে Test ব্যর্থ	`new Date()` সরাসরি	`clock` Inject করুন
Retry হয় না	`retryable` মিথ্যা, বা `PaymentError` নয়	ত্রুটির ধরন ও Property

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Repository Memory-তে, তাই Process বন্ধ হলে তথ্য যায়; আসল Database-এ "আসন সংরক্ষণ" একটি Transaction হতো, এবং একাধিক Server-এ চললে Memory-র `reserve` যথেষ্ট নয়। Payment ও Email কৃত্রিম; কোনো UI নেই। Refund ব্যর্থ হলে কী হবে (পুনরায় চেষ্টার একটি তালিকা) তা বাস্তবায়িত হয়নি।

১৬.১৪ অনুশীলনী

প্রাথমিক স্তর

১. প্রতিটি কোন ধরনের Test? (ক) `priceFor(5)` (খ) আসল Browser-এ Form পূরণ করে নিশ্চিতকরণ দেখা (গ) Service ও Fake Repository একসঙ্গে (ঘ) গত মাসের "দুবার Payment" Bug আর না ফেরা।

২. "বয়স ১৮ থেকে ৬০" নিয়মের Boundary মানগুলো লিখুন, এবং একটি Function দিয়ে যাচাই করুন।

৩. `isLeapYear(year)`-এর জন্য `node:test`-এ Test লিখুন: ২০২৪, ২০২৩, ১৯০০, ২০০০।

৪. একটি Test মাঝে মাঝে ব্যর্থ হয়: `assert.equal(greeting(), 'শুভ সকাল')`, যেখানে `greeting` `new Date().getHours()` দেখে। কেন? ঠিক করুন।

৫. `AppError` থেকে `NotFoundError` তৈরি করুন, যার `code` `NOT_FOUND`, এবং `name` আলাদা করে না লিখে।

মধ্যম স্তর

৬. `getTemperature(city, fetchImpl)`-এর Test লিখুন একটি Stub `fetch` দিয়ে: সফল ও `404`।

৭. `mock.fn` দিয়ে যাচাই করুন একটি `logger` ঠিক কোন Argument-এ ডাকা হয়েছে।

৮. ব্যবহারকারীদের জন্য একটি Memory-র Fake Repository লিখুন (`add`, `findByEmail`), এবং Test।

৯. একটি Bug: Coupon দুবার প্রয়োগ করলে ছাড় দুবার হয়। প্রথমে ব্যর্থ Regression Test, তারপর সংশোধন।

চ্যালেঞ্জ স্তর

১০. Inject করা `sleep` দিয়ে একটি Retry Function-এর Test লিখুন, যা প্রমাণ করে বিরতিগুলো ১০০, ২০০, ৪০০।

১১. তিনটি Service-এর Health Check `Promise.allSettled` দিয়ে চালিয়ে একটি Report দিন: কোনগুলো সুস্থ, কোনগুলো নয় এবং কেন।

১২. Project-এ একসঙ্গে ২০টি Booking অনুরোধ (প্রতিটি ১ আসন, মোট আসন ১০) পাঠিয়ে দেখান ঠিক ১০টি সফল এবং বাকি আসন ০।

১৬.১৫ অনুশীলনীর সমাধান

সমাধান ১ — (ক) Unit (খ) End-to-End (গ) Integration (ঘ) Regression। একটি Test একাধিক ধরনের হতে পারে: (ঘ) সাধারণত একটি Integration Test, যার উদ্দেশ্য Regression ধরা।

সমাধান ২

age-boundary.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isEligible = (age) => Number.isInteger(age) && age >= 18 && age <= 60;

const boundaries = [17, 18, 60, 61];
console.log(boundaries.map((age) => `${age}:${isEligible(age)}`).join(' '));
```

Output

```
17:false 18:true 60:true 61:false
```

প্রতিটি সীমার দুই পাশে একটি করে মান; `>=` ভুল করে `>` লেখা হলে ১৮-এর Test সঙ্গে সঙ্গে ধরবে।

সমাধান ৩

leap-year.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

const isLeapYear = (year) =>
  (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0;

test('৪ দিয়ে বিভাজ্য: অধিবর্ষ', () => assert.equal(isLeapYear(2024), true));
test('৪ দিয়ে অবিভাজ্য: নয়', () => assert.equal(isLeapYear(2023), false));
test('১০০ দিয়ে বিভাজ্য, ৪০০ দিয়ে নয়: নয়', () =>
  assert.equal(isLeapYear(1900), false));
test('৪০০ দিয়ে বিভাজ্য: অধিবর্ষ', () => assert.equal(isLeapYear(2000), true));
```

চারটি Test নিয়মের চারটি শাখা ঢেকেছে; ১৯০০ ও ২০০০ হলো এই নিয়মের Boundary।

সমাধান ৪ — ফল নির্ভর করে Test কখন চলছে তার ওপর: সকালে পাস, বিকেলে ব্যর্থ। সময়কে Argument বানান:

greeting-clock.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greeting(now = new Date()) {
  return now.getHours() < 12 ? 'শুভ সকাল' : 'শুভ অপরাহ্ন';
}

console.log(greeting(new Date(2026, 0, 1, 9, 0)));
console.log(greeting(new Date(2026, 0, 1, 15, 0)));
```

Output

```
শুভ সকাল
শুভ অপরাহ্ন
```

Production-এ Default আসল সময়; Test-এ নির্দিষ্ট সময়, তাই ফল সব সময় একই।

সমাধান ৫

not-found.js · JavaScript — চলবে: Browser Console ও Node.js

```
class AppError extends Error {
  constructor(message, { code } = {}) {
    super(message);
    this.name = new.target.name;
    this.code = code;
  }
}

class NotFoundError extends AppError {
  constructor(what) {
    super(`${what} পাওয়া যায়নি`, { code: 'NOT_FOUND' });
  }
}

const error = new NotFoundError('Booking B-9');
console.log(error.name, error.code, error.message, error instanceof AppError);
```

Output

```
NotFoundError NOT_FOUND Booking B-9 পাওয়া যায়নি true
```

সমাধান ৬

temperature.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

async function getTemperature(city, fetchImpl = fetch) {
  const response = await fetchImpl(`/weather?city=${encodeURIComponent(city)}`);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return (await response.json()).celsius;
}

const stubFetch = (status, body) => async () =>
  new Response(JSON.stringify(body), { status });

test('সফল সাড়ায় তাপমাত্রা', async () => {
  const celsius = await getTemperature('Dhaka', stubFetch(200, { celsius: 31 }));
  assert.equal(celsius, 31);
});
```

```
test('404-এ ত্রুটি', async () => {
  const result = getTemperature('Atlantis', stubFetch(404, {}));
  await assert.rejects(result, { message: 'HTTP 404' });
});
```

সমাধান ৭

logger-mock.test.mjs · JavaScript — চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

function register(user, logger) {
  logger('info', 'user-registered', { email: user.email });
  return { ...user, active: true };
}

test('নিবন্ধনে ঠিক একটি info Log, Password ছাড়া', () => {
  const logger = mock.fn();
  register({ email: 'a@b.co', password: 'secret' }, logger);

  assert.equal(logger.mock.callCount(), 1);
  const expected = ['info', 'user-registered', { email: 'a@b.co' }];
  assert.deepEqual(logger.mock.calls[0].arguments, expected);
  assert.doesNotMatch(JSON.stringify(logger.mock.calls), /secret/);
});
```

শেষ Assertion একটি নিরাপত্তা-Test: Log-এ Password কখনো যাবে না।

সমাধান ৮

user-repo.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

function createUserRepository() {
  const users = new Map();
  return {
    async add(user) {
      const key = user.email.toLowerCase();
      if (users.has(key)) {
        throw new Error('Email আগে থেকেই আছে');
      }
      users.set(key, structuredClone(user));
    },
    async findByEmail(email) {
```

```
      const user = users.get(email.toLowerCase());
      return user ? structuredClone(user) : null;
    },
  };
}

test('যোগ ও খোঁজা, বড়-ছোট হাতের অক্ষর নির্বিশেষে', async () => {
  const repo = createUserRepository();
  await repo.add({ email: 'Sara@Example.com', name: 'Sara' });
  assert.equal((await repo.findByEmail('sara@example.com')).name, 'Sara');
  assert.equal(await repo.findByEmail('x@y.z'), null);
});

test('একই Email দুবার নয়; সংরক্ষিত Object অক্ষত', async () => {
  const repo = createUserRepository();
  await repo.add({ email: 'a@b.co', name: 'A' });
  await assert.rejects(repo.add({ email: 'A@B.CO', name: 'B' }), /আগে থেকেই/);
  const found = await repo.findByEmail('a@b.co');
  found.name = 'বদলানো';
  assert.equal((await repo.findByEmail('a@b.co')).name, 'A');
});
```

সমাধান ৯

coupon-regression.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

// সংশোধিত সংস্করণ: Coupon আগে থেকে থাকলে আবার প্রয়োগ হয় না।
function applyCoupon(order, code) {
  if (order.coupons.includes(code)) {
    return order;
  }
  const coupons = [...order.coupons, code];
  return { ...order, coupons, total: order.total - 100 };
}

test('Regression: একই Coupon দুবার প্রয়োগে ছাড় একবার', () => {
  const order = { total: 1000, coupons: [] };
  const twice = applyCoupon(applyCoupon(order, 'SAVE100'), 'SAVE100');
  assert.equal(twice.total, 900);
});
```

পদ্ধতি: প্রথমে Test লিখুন পুরোনো Code-এর বিরুদ্ধে (সেখানে `total` হতো ৮০০, Test ব্যর্থ), তারপর `includes`-এর যাচাই যোগ করুন, Test পাস। Test-টি চিরকাল থাকবে, যাতে কেউ ভুল করে যাচাইটি মুছে দিলে সঙ্গে সঙ্গে ধরা পড়ে।

সমাধান ১০

retry-backoff.test.mjs · JavaScript — চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

async function withRetry(task, { retries = 3, sleep }) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      await sleep(100 * 2 ** attempt);
    }
  }
}

test('তিনবার ব্যর্থতার পর সফল: বিরতি 100, 200, 400', async () => {
  const sleep = mock.fn(async () => {});
  let calls = 0;
  const task = async () => {
    calls += 1;
    if (calls <= 3) {
      throw new Error('busy');
    }
    return 'ok';
  };

  assert.equal(await withRetry(task, { sleep }), 'ok');
  const waits = sleep.mock.calls.map((call) => call.arguments[0]);
  assert.deepEqual(waits, [100, 200, 400]);
});

test('সব চেষ্টা ব্যর্থ: শেষ ত্রুটি ছোড়ে, বিরতি তিনবার', async () => {
  const sleep = mock.fn(async () => {});
  const alwaysDown = async () => {
    throw new Error('down');
  };
  await assert.rejects(withRetry(alwaysDown, { sleep }), /down/);
  assert.equal(sleep.mock.callCount(), 3);
});
```

`sleep` Spy হওয়ায় Test আসলে এক মুহূর্তও অপেক্ষা করে না, অথচ প্রমাণ করে কতক্ষণ অপেক্ষা করা **হতো**।

সমাধান ১১

health.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const check = (name, ok) => (ok
  ? Promise.resolve(`${name} সুস্থ`)
  : Promise.reject(new Error(`${name} সাড়া দিচ্ছে না`)));

const results = await Promise.allSettled([
  check('Database', true),
  check('Payment', false),
  check('Email', true),
]);

const healthy = results.filter((result) => result.status === 'fulfilled');
console.log(`${healthy.length}/${results.length} সুস্থ`);

for (const result of results.filter((item) => item.status === 'rejected')) {
  console.log(`সমস্যা: ${result.reason.message}`);
}
```

Output

```
2/3 সুস্থ
সমস্যা: Payment সাড়া দিচ্ছে না
```

`Promise.all` প্রথম ব্যর্থতায় থেমে যেত এবং বাকি দুটির অবস্থা জানাত না; Health Check-এ প্রতিটির ফল দরকার।

সমাধান ১২

concurrent-booking.mjs · JavaScript চলবে: Node.js

```
const SRC = '../../project/booking-guard/src';
const { createBookingService } = await import(`${SRC}/booking-service.mjs`);
const { createMemoryRepository } = await import(`${SRC}/memory-repository.mjs`);

const repository = createMemoryRepository(10);
const service = createBookingService({
  repository,
  payment: { charge: async () => 'PAY', refund: async () => {} },
  notifier: { send: async () => {} },
});

const requests = Array.from({ length: 20 }, (_, i) =>
  service.book({ customer: `User ${i}`, email: `u${i}@x.co`, seats: 1 }, i));
const results = await Promise.allSettled(requests);

const ok = results.filter((result) => result.status === 'fulfilled').length;
```

```
const soldOut = results.filter((r) => r.reason?.code === 'SOLD_OUT').length;
console.log(ok, soldOut, repository.available);
```

Output

```
10 10 0
```

২০টি অনুরোধ একসঙ্গে, প্রতিটি `await`-এ থেমে অন্যকে সুযোগ দিয়েছে; তবু ঠিক ১০টি সফল, বাকি ১০টি `SOLD_OUT`, এবং কোনো আসন ঋণাত্মক হয়নি। `reserve`-এর যাচাই ও বিয়োগের মাঝে `await` না থাকাই এর কারণ। Project-এর Test-ও এটি যাচাই করে।

১৬.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Error-এ `code` Property রাখার প্রধান সুবিধা কী?

ক) Error দ্রুত ছোড়া যায়

খ) বার্তার ভাষা বদলালেও Code ও Log-এ ত্রুটি চেনা যায়

গ) `stack` বড় হয়

ঘ) `catch` লাগে না

প্রশ্ন ২. `Promise.any`-এর সব Promise ব্যর্থ হলে কোন ধরনের ত্রুটি আসে?

ক) `TypeError`

খ) `RangeError`

গ) `AggregateError`

ঘ) `undefined`

প্রশ্ন ৩. Invariant ভাঙলে সাধারণত কী করা উচিত?

ক) চুপচাপ ঠিক করে এগোনো

খ) সঙ্গে সঙ্গে স্পষ্ট ত্রুটি দিয়ে থামা

গ) ব্যবহারকারীকে আবার চেষ্টা করতে বলা

ঘ) Log না করে উপেক্ষা

প্রশ্ন ৪. Testing Pyramid-এর নিচের স্তরে কোন Test সবচেয়ে বেশি?

ক) End-to-End

খ) Manual

গ) Unit

ঘ) Performance

প্রশ্ন ৫. "আসন ১ থেকে ১০" নিয়মের জন্য সবচেয়ে কার্যকর Test-মানের সেট?

ক) ৫

খ) ৩, ৫, ৭

গ) ১০০

ঘ) ০, ১, ১০, ১১

প্রশ্ন ৬. নির্দিষ্ট উত্তর দেয় এমন Test Double-এর নাম?

ক) Stub

খ) Fake

গ) Regression

ঘ) Coverage

প্রশ্ন ৭. Payment চূড়ান্তভাবে ব্যর্থ হলে সংরক্ষিত আসন ফেরত দেওয়াকে কী বলে?

ক) Retry

খ) Compensation

গ) Idempotency

ঘ) Memoization

প্রশ্ন ৮. ব্যবহারকারী দুবার Click করলেও একবারই Payment নিশ্চিত করার ধারণা?

ক) Throttle

খ) Coverage

গ) Idempotency

ঘ) Polymorphism

প্রশ্ন ৯. একটি Test মাঝে মাঝে ব্যর্থ হয়, কারণ তা `new Date()` সরাসরি ব্যবহার করে। সমাধান?

ক) Test আবার চালানো

খ) Test মুছে ফেলা

গ) Timeout বাড়ানো

ঘ) সময় (`clock`) Inject করা

প্রশ্ন ১০. ১০০% Coverage সম্পর্কে কোনটি সঠিক?

ক) Program Bug-মুক্ত

খ) প্রতিটি লাইন অন্তত একবার চলেছে, কিন্তু ফল যাচাই হয়েছে কি না বলে না

গ) আর Test লাগবে না

ঘ) E2E Test অপ্রয়োজনীয়

১৬.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–খ · ৪–গ · ৫–ঘ · ৬–ক · ৭–খ · ৮–গ · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	`code` স্থির; বার্তা মানুষের জন্য, `code` যন্ত্রের জন্য।
২	গ	`AggregateError`-এর `errors`-এ সব ত্রুটি থাকে।
৩	খ	Invariant ভাঙা মানে Bug; ভুল অবস্থায় এগোনো বিপজ্জনক।
৪	গ	Unit Test দ্রুত ও নিশ্চিত, তাই সবচেয়ে বেশি।
৫	ঘ	সীমার দুই পাশ; মাঝের মান নতুন কিছু প্রমাণ করে না।
৬	ক	Stub নির্দিষ্ট উত্তর দেয়; Fake কার্যকর বিকল্প বাস্তবায়ন।
৭	খ	আগের ধাপের প্রভাব উল্টে দেওয়া।
৮	গ	একই অনুরোধ বারবার এলেও ফল একবারের মতো।
৯	ঘ	সময় নিয়ন্ত্রণে এলে Test নিশ্চিত হয়।
১০	খ	Coverage দেখায় কোথায় Test নেই, Test যথেষ্ট কি না নয়।

১৬.১৮ অধ্যায়ের সারসংক্ষেপ

- Reliability-র চার দিক: সঠিকতা, সহনশীলতা, পুনরুদ্ধার, পর্যবেক্ষণযোগ্যতা।
- ত্রুটির পরিবার: ধরন, স্থির `code`, `retryable`, `cause`-এর শিকল; `new.target.name`; একাধিক ত্রুটিতে `AggregateError`।
- Invariant ভেতরের নিয়ম পাহারা দেয়; ভাঙলে Fail Fast।
- শেষ প্রতিরক্ষা: Browser-এ `unhandledrejection`, Node.js-এ `process.on('unhandledRejection')`।
- Debugging একটি পদ্ধতি: আবার ঘটানো, ছোট করা, ধারণা ও প্রমাণ, অর্ধেক করে খোঁজা, আগে Regression Test।
- Structured Log যন্ত্রে পাঠযোগ্য; গোপন তথ্য কখনো নয়।
- Testing Pyramid: অনেক Unit, কিছু Integration, অল্প E2E; Boundary-তে Bug বেশি।
- `node:test`: `describe`, `it`, Hook, `todo`, `skip`, Name Pattern, Watch, Coverage।
- Test Double: Stub, Spy, Mock, Fake; সব নির্ভরতা Inject করুন, সময়সহ।
- নির্ভরযোগ্য প্রবাহ: Retry শুধু সাময়িক ত্রুটিতে, Idempotency Key, ব্যর্থতায় Compensation, অপ্রধান কাজ আলাদা।

পরের অধ্যায়ে Regular Expression-এর গভীর ব্যবহার, উন্নত যাচাই এবং নিরাপদ Input Processing।

১৬.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Reliability	সব পরিস্থিতিতে Program-এর সঠিক ও প্রত্যাশিত আচরণ।
Error Code	ত্রুটির স্থির, যন্ত্রপাঠ্য পরিচয়।
AggregateError	একাধিক ত্রুটি একসঙ্গে বহনকারী Error।
Invariant	যে নিয়ম Program-এ সব সময় সত্য থাকার কথা।
Structured Log	যন্ত্রপাঠ্য গঠনের (যেমন JSON) Log লাইন।
Testing Pyramid	Unit, Integration, E2E-এর অনুপাতের নির্দেশিকা।
Boundary Test	সীমার দুই পাশের মান দিয়ে Test।
Regression Test	পুরোনো Bug না ফেরার Test।
Test Double	Test-এ আসল নির্ভরতার নিয়ন্ত্রিত বিকল্প।
Stub / Spy / Mock / Fake	নির্দিষ্ট উত্তর / ডাক লেখা / প্রত্যাশা যাচাই / কার্যকর বিকল্প।
Idempotency	একই অনুরোধ বারবার এলেও ফল একবারের মতো।
Compensation	ব্যর্থতার পর আগের ধাপের প্রভাব উল্টে দেওয়া।

১৬.২০ আরও পড়ুন

- **Node.js — Test runner:** `https://nodejs.org/api/test.html` — `describe`, Hook, Mock, Coverage।
- **Node.js — Assert:** `https://nodejs.org/api/assert.html` — সব Assertion।
- **MDN — AggregateError:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError` — একাধিক ত্রুটি।
- **MDN — new.target:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new.target` — Constructor-এর ভেতরে ডাকার Class।
- **Martin Fowler — Test Double:** `https://martinfowler.com/bliki/TestDouble.html` — Stub, Mock, Fake-এর সংজ্ঞা।

অধ্যায় ১৭

১৭

JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Character Class, Quantifier (Greedy ও Lazy) এবং Anchor দিয়ে নির্ভুল Pattern লিখতে।
- Capturing, Named ও Non-capturing Group, Backreference এবং Alternation ব্যবহার করতে।
- Lookahead ও Lookbehind দিয়ে "পাশে কী আছে" শর্ত প্রকাশ করতে।
- `exec`, `matchAll`, Function-সহ `replace`, Regex দিয়ে `split` এবং Sticky Flag ব্যবহার করতে, এবং `lastIndex`-এর ফাঁদ এড়াতে।
- ব্যবহারকারীর লেখা দিয়ে নিরাপদে Regex তৈরি করতে (Escaping)।
- বাংলা লেখা ও বাংলা সংখ্যা Unicode-সচেতনভাবে যাচাই ও রূপান্তর করতে।
- "আগে Normalize, তারপর Validate, তারপর Parse"—এই ক্রমে Input প্রক্রিয়া করতে।
- Email, Phone ও Password যাচাইয়ের সীমা বুঝতে, এবং Regex-এর বাড়াবাড়ি এড়াতে।
- ReDoS (Regular Expression Denial of Service) চিনতে ও প্রতিরোধ করতে।
- একটি সম্পূর্ণ **FormShield** Schema-ভিত্তিক Validation Engine তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৮: Regex-এর পরিচয় (`test` , `match` , `replace` , `\d` , `\s` , `+` , `{n}` , `^` , `$` , Flag `g` , `i` , `u` , `\p{L}`), Unicode Normalization এবং XSS।
- অধ্যায় ১১: Trust Boundary, `ValidationError` , সব ভুল একসঙ্গে জানানো।
- অধ্যায় ১২: ES Module।
- অধ্যায় ১৫: `URL` দিয়ে ঠিকানা যাচাই।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৮-এ Regular Expression-এর সঙ্গে পরিচয় হয়েছে, এবং তারপর থেকে প্রায় প্রতিটি Project-এ ছোট ছোট Pattern দেখেছেন: Email-এর গঠন, পরিমাণের পূর্ণসংখ্যা, মোবাইল নম্বর। কিন্তু বাস্তব Input-এর চেহারা অনেক বেশি এলোমেলো: কেউ নম্বর লেখেন `+880 1712-345678` , কেউ `০১৭১২৩৪৫৬৭৮` ; কেউ তারিখ লেখেন `2026-02-30` , যা দেখতে ঠিক কিন্তু বাস্তবে নেই। এগুলো সামলাতে Regex-এর গভীরতর ক্ষমতা লাগে।

একই সঙ্গে Regex একটি নিরাপত্তা-ঝুঁকিও: ভুলভাবে লেখা একটি Pattern একটি সাধারণ দেখতে লম্বা Input-এ Server-কে কয়েক মিনিট আটকে রাখতে পারে। আর "Regex দিয়ে বিপজ্জনক HTML পরিষ্কার" করার চেষ্টা প্রায় সব সময়ই ব্যর্থ হয়। এই অধ্যায় শেখায় Regex কখন সঠিক হাতিয়ার, কখন নয়, এবং Input প্রক্রিয়ার একটি নিরাপদ, পরীক্ষাযোগ্য পদ্ধতি।

১৭.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা (অধ্যায় ৮)	এই অধ্যায়ে নতুন
`\d` , `\s` , `+` , `{n}` , `^` , `$` , `[abc]`	`\w` , `\b` , Negated Class, `{m,n}` , Lazy Quantifier
`test` , `match` , `replace`	`exec` , `matchAll` , Function-সহ `replace` , `split` , Sticky
Flag `g` , `i` , `u`	Flag `m` , `s` , `y` , `d` ; `lastIndex` -এর আচরণ
`\p{L}` , `\p{M}` , `\p{N}`	`\p{Script=Bengali}` , বাংলা সংখ্যা রূপান্তর
—	Group, Named Group, Backreference, Lookaround
—	Escaping, ReDoS, Schema-ভিত্তিক যাচাই

অধ্যায় ৮-এর জিনিসগুলো এখানে আবার শেখানো হবে না; প্রয়োজনে সেখানে ফিরে দেখুন।

১৭.২ Character Class, Quantifier এবং সীমানা

classes.js · JavaScript — চলবে: Node.js

```
const text = 'Order A-102, B-7 এবং C-3051; মোট 3টি';

console.log(text.match(/[A-Z]-\d{3,4}/g));
console.log(text.match(/[^\s,;]+/g).length);
console.log('cat category scat'.match(/\bcat\b/g));
console.log('price_2026 মূল্য'.match(/\w+/g));
```

Output

```
[ 'A-102', 'C-3051' ]
7
[ 'cat' ]
[ 'price_2026' ]
```

- `{3,4}` আগের জিনিসটি ৩ থেকে ৪ বার; `{2,}` মানে কমপক্ষে ২ বার। `B-7` বাদ পড়েছে, কারণ মাত্র একটি সংখ্যা।
- `[^...]` (Negated Class) বন্ধনীর ভেতরের অক্ষরগুলো **ছাড়া** যেকোনো একটি।
- `\b` শব্দের সীমানা: একটি শব্দ-অক্ষর (`\w`) এবং একটি অ-শব্দ-অক্ষরের মাঝখানের অদৃশ্য জায়গা। `category` বা `scat`-এর ভেতরের `cat` মেলেনি।
- `\w` মানে `[A-Za-z0-9_]`; বাংলা অক্ষর এতে নেই, তাই `মূল্য` আসেনি। `\b`-ও একই কারণে বাংলায় নির্ভরযোগ্য নয় (অনুচ্ছেদ ১৭.৬)।

Greedy এবং Lazy

greedy.js · JavaScript — চলবে: Browser Console ও Node.js

```
const html = '<b>সতর্ক</b> এবং <i>দ্রুত</i>';

console.log(html.match(/<.+>/)[0]);
console.log(html.match(/<.+?>/)[0]);
console.log(html.match(/<[^>]+>/g).join(' '));
```

Output

```
<b>সতর্ক</b> এবং <i>দ্রুত</i>
<b>
<b> </b> <i> </i>
```

Quantifier (`+`, `*`, `{m,n}`) স্বাভাবিকভাবে **Greedy**: যতটা সম্ভব বেশি নেয়, তারপর মেলাতে প্রয়োজনে পিছিয়ে আসে। তাই `<.+>` প্রথম `<` থেকে **শেষ** `>` পর্যন্ত নিয়েছে। পেছনে `?` দিলে **Lazy**: যতটা সম্ভব কম। তৃতীয় লাইনের Negated Class (`[^>]+`) প্রায়ই Lazy-এর চেয়ে ভালো: অর্থ স্পষ্ট ("`>` ছাড়া যেকোনো কিছু"), এবং পিছিয়ে আসার কাজ কম।

> **সতর্কতা:** এই উদাহরণ HTML **খোঁজার** কৌশল দেখায়, HTML **পরিষ্কার** করার নয়। Regex দিয়ে বিপজ্জনক HTML ছেঁকে ফেলা নিরাপদ নয় (অনুচ্ছেদ ১৭.১০)।

১৭.৩ Group, Backreference এবং Alternation

বন্ধনী `( )` তিনটি কাজ করে: কয়েকটি অংশকে একসঙ্গে Quantifier দেয়, মিলে যাওয়া অংশ আলাদা করে ধরে রাখে (**Capture**), এবং Alternation-এর সীমা ঠিক করে।

groups.js · JavaScript — চলবে: Node.js

```
const date = /^(\d{4})-(\d{2})-(\d{2})$/.exec('2026-10-15');
console.log(date[0], date[1], date[3]);

const named = /^(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})$/.exec('2026-10-15');
const { year, month, day } = named.groups;
console.log(`${day}/${month}/${year}`);

console.log('ha-ha-ha!'.match(/(?:ha-?)+/)[0]);
console.log(/^(\w)\w*\1$/.test('level'), /^(\w)\w*\1$/.test('lever'));

console.log(/^cat|dog$/.test('catalog'), /^(?:cat|dog)$/.test('catalog'));
```

Output

```
2026-10-15 2026 15
15/10/2026
ha-ha-ha
true false
true false
```

- **Capturing Group** ফলাফলে নম্বর অনুযায়ী আসে: `[1]`, `[2]`… `[0]` পুরো মিল।
- **Named Group** `(?<name>...)` ফলাফলের `groups` Object-এ নাম ধরে আসে; Destructuring (অধ্যায় ৭) দিয়ে পড়া সহজ, এবং Pattern বদলালে নম্বর গুলিয়ে যাওয়ার ভয় নেই।
- **Non-capturing Group** `(?:...)` শুধু দলবদ্ধ করে, কিছু ধরে রাখে না।
- **Backreference** `\1` মানে "প্রথম Group যা মিলেছিল, ঠিক সেটিই আবার"। Named Group-এর জন্য `\k<name>`।
- **Alternation `|` -এর অগ্রাধিকার সবচেয়ে কম।** `^cat|dog$` মানে "`^cat`" অথবা "`dog$`", তাই `catalog` মিলে গেছে! পুরো Alternation-কে Anchor-এর ভেতরে রাখতে Group লাগে: `^(?:cat|dog)$`। এটি Validation-এর সবচেয়ে প্রচলিত ও বিপজ্জনক ভুলগুলোর একটি।

১৭.৪ Lookahead এবং Lookbehind

কখনো শর্ত থাকে "পরে অমুক আছে" বা "আগে অমুক নেই", কিন্তু সেই অমুক মিলের অংশ নয়। এগুলো **Lookaround**: অবস্থান যাচাই করে, কোনো অক্ষর গ্রাস করে না।

লেখা	নাম	অর্থ
`X(?=Y)`	Positive Lookahead	X, যার পরে Y আছে
`X(?!Y)`	Negative Lookahead	X, যার পরে Y নেই
`(?<=Y)X`	Positive Lookbehind	X, যার আগে Y আছে
`(?<!Y)X`	Negative Lookbehind	X, যার আগে Y নেই

lookaround.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = 'Pen ৳50, Book ৳450, Bag $30';
console.log(prices.match(/(?<=৳)\d+/g).join(', '));

const withCommas = (n) => String(n).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
console.log(withCommas(1234567), withCommas(950));

const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
const samples = ['Dhaka2026', 'dhaka2026', 'Dk2026'];
console.log(samples.map((sample) => strong.test(sample)).join(' '));
```

Output

```
50, 450
1,234,567 950
true false false
```

- `(?<=৳)\d+` : শুধু টাকার চিহ্নের পরের সংখ্যা; ডলারেরটি বাদ, এবং চিহ্নটি ফলে আসেনি।
- **হাজারের কমা:** `\B` (শব্দের সীমানা নয় এমন জায়গা), যার পরে তিন-তিন অঙ্কের এক বা একাধিক দল এবং তারপর আর কোনো অঙ্ক নেই। দেখতে জটিল, তাই বাস্তবে `toLocaleString` (অধ্যায় ১১) সহজ ও পাঠযোগ্য; এটি শুধু Lookahead বোঝার জন্য।
- **Password-এর তিনটি Lookahead** একই অবস্থান থেকে তিনটি আলাদা শর্ত যাচাই করে। কাজ করে, কিন্তু ব্যর্থ হলে **কোন** শর্ত ভাঙল বলে না। অনুচ্ছেদ ১৭.৮-এ এর ভালো বিকল্প।

১৭.৫ Regex-এর Method এবং Flag-এর গভীরে

methods.js · JavaScript চলবে: Node.js

```
const log = 'ERROR 12:01 disk; WARN 12:05 cpu; ERROR 12:09 net';
const entry = /(?<level>ERROR|WARN) (?<time>\d{2}:\d{2}) (?<what>\w+)/g;

for (const match of log.matchAll(entry)) {
  console.log(match.index, match.groups.level, match.groups.what);
}

console.log(log.replace(entry, '$<time>=$<what>'));
console.log(log.replace(/\d{2}:\d{2}/g, (time) => time.replace(':', 'h')));
console.log('আম, কলা;লিচু  ,জাম'.split(/\s*[,;]\s*/));
```

Output

```
0 ERROR disk
18 WARN cpu
34 ERROR net
12:01=disk; 12:05=cpu; 12:09=net
ERROR 12h01 disk; WARN 12h05 cpu; ERROR 12h09 net
[ 'আম', 'কলা', 'লিচু', 'জাম' ]
```

- `matchAll` (`g` Flag বাধ্যতামূলক) প্রতিটি মিলের পূর্ণ তথ্য দেয়: অবস্থান (`index`), Group, Named Group। `match(/.../g)` শুধু মিলে যাওয়া লেখাগুলো দেয়।
- `replace`-এ `$<name>` Named Group-এর মান বসায় (`$1`, `$2` নম্বরের জন্য)।
- `replace`-এ **Function**: প্রতিটি মিলের জন্য Function ডাকা হয়, এবং তার ফেরত দেওয়া লেখা বসে। জটিল রূপান্তরে এটি সবচেয়ে পরিষ্কার।
- `split` **Regex নেয়:** কমা বা সেমিকোলন, চারপাশের যেকোনো পরিমাণ ফাঁকা জায়গাসহ, একবারে।

lastIndex-এর ফাঁদ

last-index.js · JavaScript চলবে: Browser Console ও Node.js

```
const digits = /\d+/g;

console.log(digits.test('abc 42'), digits.lastIndex);
console.log(digits.test('abc 42'), digits.lastIndex);
console.log(digits.test('abc 42'));

const safe = /\d+/;
console.log(safe.test('abc 42'), safe.test('abc 42'), safe.test('abc 42'));
```

Output

```
true 6
false 0
true
true true true
```

`g` বা `y` Flag-যুক্ত একটি Regex Object মনে রাখে শেষ মিল কোথায় শেষ হয়েছিল (`lastIndex`), এবং পরের `test` বা `exec` সেখান থেকে খোঁজা শুরু করে। একই লেখায় দ্বিতীয়বার `test` তাই `false` দিয়েছে, এবং শুরুতে ফিরে গেছে। এই Bug সাধারণত দেখা দেয় যখন একটি `g` Regex Module-এর একটি Constant-এ রেখে অনেক Input যাচাই করা হয়: পর্যায়ক্রমে সত্য-মিথ্যা। নিয়ম: **যাচাইয়ের (`test`) Regex-এ `g` দেবেন না।**

Flag	অর্থ
`m`	`^` ও `$` প্রতিটি লাইনের শুরু ও শেষ
`s`	`.` নতুন লাইনও মেলায়
`y` (Sticky)	ঠিক `lastIndex` থেকেই মিলতে হবে, পরে খোঁজা নয়; Tokenizer-এ কাজে লাগে
`d`	মিলের সঙ্গে প্রতিটি Group-এর শুরু-শেষ অবস্থান (`indices`)

ব্যবহারকারীর লেখা দিয়ে Regex: Escaping

খোঁজার Box-এ ব্যবহারকারী `c++` লিখলেন, আপনি লিখলেন `new RegExp(query)`। ফল: `+` একটি Quantifier, এবং `c++` অবৈধ বা অপ্রত্যাশিত Pattern।

escape.js · JavaScript — চলবে: Browser Console ও Node.js

```
const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

const query = 'c++ (basic)';
const titles = ['c++ (basic) guide', 'c (basic)', 'cc (basic)'];

try {
  new RegExp(query);
} catch (error) {
  console.log(error.name);
}

const safe = new RegExp(escapeRegExp(query), 'i');
console.log(escapeRegExp(query));
console.log(titles.filter((title) => safe.test(title)).length);
```

Output

```
SyntaxError
c\+\+ \(basic\)
1
```

`escapeRegExp` প্রতিটি বিশেষ অক্ষরের আগে একটি `\` বসায় (`$&` মানে "পুরো মিল")। ফলে ব্যবহারকারীর লেখা আক্ষরিক অর্থে খোঁজা হয়। আরও নিরাপদ বিকল্প: আক্ষরিক খোঁজার জন্য Regex-ই ব্যবহার না করা— `includes` বা `indexOf` (অধ্যায় ৮-এর `findAll`)।

১৭.৬ Unicode এবং বাংলা

bengali.js · JavaScript চলবে: Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

console.log(toAsciiDigits('০১৭১২-৩৪৫৬৭৮'));

const banglaName = /^[\p{Script=Bengali}\s.]+$/u;
console.log(banglaName.test('মোঃ রফিক'), banglaName.test('Rafiq'),
    banglaName.test('রফিক১'));

const words = 'আমি JavaScript শিখি'.match(/[\p{L}\p{M}]+/gu);
console.log(words);
const smile = '\u{1F600}';
console.log('বই'.match(/\bবই\b/u), smile.match(/^.$/u)?.[0].length);
```

Output

```
01712-345678
true false true
[ 'আমি', 'JavaScript', 'শিখি' ]
null 2
```

- **বাংলা সংখ্যা:** `০` থেকে `৯` Unicode-এ পরপর (U+09E6 থেকে U+09EF), তাই অক্ষরের নম্বর থেকে `0x09e6` বাদ দিলে অঙ্কটি পাওয়া যায়। ব্যবহারকারী বাংলা Keyboard-এ নম্বর লিখলে যাচাইয়ের আগে এভাবে রূপান্তর করুন।
- **`\p{Script=Bengali}`** শুধু বাংলা লিপির অক্ষর, কার-চিহ্ন ও সংখ্যা মেলায়; `u` Flag বাধ্যতামূলক। লক্ষ করুন, `রফিক১` মিলেছে, কারণ বাংলা অঙ্কও বাংলা লিপির অংশ। নামে সংখ্যা না চাইলে আলাদা শর্ত লাগবে।
- **`\b` বাংলায় কাজ করে না,** এমনকি `u` Flag দিয়েও: `\b` এখনো `\w`-এর ইংরেজি সংজ্ঞায় চলে। বাংলা শব্দের সীমানার জন্য Lookaround লিখুন, যেমন `(?<![\p{L}\p{M}])বই(?![\p{L}\p{M}])`।
- **`u` Flag ছাড়া `.`** একটি Emoji-কে দুটি টুকরো ভাবে (অধ্যায় ৮-এর Code Unit)। `u` দিলে Code Point ধরে মেলে; তবু `length` এখনো ২, কারণ String-এর দৈর্ঘ্য Code Unit-এ।

১৭.৭ Normalize, Validate, Parse: একটি ক্রম

ব্যবহারকারীর Input-এর একই অর্থের বহু রূপ: `+880 1712-345678`, `8801712345678`, `০১৭১২ ৩৪৫৬৭৮`। প্রতিটি রূপের জন্য আলাদা Regex লেখা অসম্ভব। পদ্ধতিটি তিন ধাপের:

১. **Normalize:** একটি আদর্শ রূপে আনুন—Unicode NFC (অধ্যায় ৮), বাংলা অঙ্ক থেকে ইংরেজি, ফাঁকা জায়গা ও হাইফেন বাদ, দেশের Code বাদ।

২. **Validate:** আদর্শ রূপটি একটি সরল, কঠোর Pattern দিয়ে যাচাই করুন।

৩. **Parse:** বৈধ লেখা থেকে অর্থপূর্ণ মান তৈরি করুন (যেমন তারিখ থেকে `Date`), এবং অর্থগত শর্ত যাচাই করুন।

pipeline.js · JavaScript — চলবে: Browser Console ও Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

function normalizePhone(raw) {
  const digits = toAsciiDigits(raw.normalize('NFC')).replace(/[\s\-().]/g, '');
  return digits.replace(/^(?:\+?880)/, '0');
}

const BD_MOBILE = /^01[3-9]\d{8}$/;

for (const raw of ['+880 1712-345678', '০১৭১২ ৩৪৫৬৭৮', '8801512345678',
    '01212345678']) {
  const phone = normalizePhone(raw);
  console.log(phone, BD_MOBILE.test(phone));
}
```

Output

```
01712345678 true
01712345678 true
01512345678 true
01212345678 false
```

যাচাইয়ের Pattern এখন অধ্যায় ৮-এর মতোই সরল; সব জটিলতা Normalization-এ, যেখানে প্রতিটি ধাপ আলাদাভাবে বোঝা ও পরীক্ষা করা যায়। সংরক্ষণও করুন আদর্শ রূপে, যাতে পরে খোঁজা ও তুলনা সহজ হয়।

Parse ধাপ কেন আলাদা, তারিখে দেখুন:

parse-date.js · JavaScript চলবে: Browser Console ও Node.js

```
const ISO_DATE = /^(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})$/;

function parseDate(text) {
  const match = ISO_DATE.exec(text);
  if (!match) {
    return null;
  }
  const [y, m, d] = [match.groups.y, match.groups.m, match.groups.d].map(Number);
  const date = new Date(Date.UTC(y, m - 1, d));
  const real = date.getUTCMonth() === m - 1 && date.getUTCDate() === d;
  return real ? date.toISOString().slice(0, 10) : null;
}

const inputs = ['2026-10-15', '2026-02-30', '15/10/2026', '2024-02-29',
  '2026-02-29'];
console.log(inputs.map((text) => String(parseDate(text))).join(' '));
```

Output

```
2026-10-15 null null 2024-02-29 null
```

`2026-02-30` Regex পাস করে, কারণ গঠন ঠিক; কিন্তু ফেব্রুয়ারিতে ৩০ তারিখ নেই। `Date` এমন তারিখকে নীরবে মার্চে সরিয়ে দেয়, তাই তৈরি হওয়া তারিখের মাস ও দিন মূলটির সঙ্গে মিলিয়ে দেখা হয়েছে। Regex গঠন যাচাই করে, অর্থ নয়।

১৭.৮ Email, Phone এবং Password: Regex-এর সীমা

- **Email:** পূর্ণ মান অনুযায়ী Email-এর ব্যাকরণ এত জটিল যে তার Regex কয়েক হাজার অক্ষরের, আর তবু একটি ঠিকানা **আছে** কি না তা প্রমাণ করে না। ব্যবহারিক নিয়ম: একটি সরল গঠন-যাচাই (`@`-এর দুই পাশে কিছু, পরে একটি ডট), দৈর্ঘ্যের সীমা (২৫৪), এবং নিশ্চিত করতে একটি যাচাই-Email। বেশি কড়া Regex বৈধ ঠিকানাও বাতিল করে, যেমন `sara+bills@example.com`।
- **Phone:** আন্তর্জাতিক নম্বরে নিয়ম দেশে দেশে আলাদা; নিজের দেশের জন্য Normalize করে সরল Pattern, অন্য দেশের জন্য একটি মানসম্মত Library।
- **Name:** নামের কোনো সর্বজনীন নিয়ম নেই। অ্যাপোস্ট্রফি, হাইফেন, ডট, এক-শব্দের নাম—সব বৈধ। দৈর্ঘ্য ও নিয়ন্ত্রণ-অক্ষর (Control Character) বাদ দেওয়া যথেষ্ট।
- **Password:** একটি বড় Lookahead Regex ব্যবহারকারীকে শুধু "হয়নি" বলে। আলাদা আলাদা নিয়ম, প্রতিটির নিজের বার্তা, অনেক বেশি সহায়ক; আর আধুনিক নির্দেশিকা (যেমন NIST) জটিলতার নিয়মের চেয়ে **দৈর্ঘ্য** ও পরিচিত ফাঁস হওয়া Password-এর তালিকা যাচাইকে বেশি গুরুত্ব দেয়।

password.js · JavaScript চলবে: Node.js

```
const PASSWORD_RULES = [
  { test: (p) => p.length >= 12, message: 'কমপক্ষে 12 অক্ষর' },
  { test: (p) => /\p{Lu}/u.test(p), message: 'একটি বড় হাতের অক্ষর' },
  { test: (p) => /\d/.test(p), message: 'একটি সংখ্যা' },
  { test: (p) => !/(.)\1\1/.test(p), message: 'একই অক্ষর পরপর তিনবার নয়' },
];

const problems = (password) =>
  PASSWORD_RULES
    .filter((rule) => !rule.test(password))
    .map((rule) => rule.message);

console.log(problems('aaab'));
console.log(problems('Bangladesh2026!'));
```

Output

```
[
  'কমপক্ষে 12 অক্ষর',
  'একটি বড় হাতের অক্ষর',
  'একটি সংখ্যা',
  'একই অক্ষর পরপর তিনবার নয়'
]
[]
```

প্রতিটি নিয়ম একটি ছোট, নামযুক্ত পরীক্ষা; ব্যবহারকারী জানেন ঠিক কী বদলাতে হবে। Backreference `(.)\1\1` "একই অক্ষর তিনবার" ধরেছে।

১৭.৯ ReDoS: যখন একটি Regex Server আটকে দেয়

JavaScript-এর Regex Engine মেলাতে না পারলে পেছনে ফিরে অন্য পথ চেষ্টা করে (**Backtracking**)। কিছু Pattern-এ পথের সংখ্যা ইনপুটের দৈর্ঘ্যের সঙ্গে সূচকীয় হারে বাড়ে। এর সবচেয়ে পরিচিত রূপ Nested Quantifier: `(a+)+$`। প্রায়-মেলা একটি ইনপুট (`aaaa...ab`) দিলে Engine প্রতিটি সম্ভাব্য ভাগ চেষ্টা করে। JavaScript Single-threaded (অধ্যায় ১০), তাই এই সময়ে Server অন্য কোনো অনুরোধের উত্তর দেয় না। ইচ্ছাকৃতভাবে এমন ইনপুট পাঠানোকে বলে **ReDoS** (Regular Expression Denial of Service)।

ক্ষতিকর নয় এমন ছোট দৈর্ঘ্যে বৃদ্ধির হার মাপুন:

redos.js · JavaScript চলবে: Node.js

```
const evil = /^(a+)+$/;
const safe = /^a+$/;

function timeMs(regex, input) {
```

```
  const start = performance.now();
  regex.test(input);
  return performance.now() - start;
}

timeMs(evil, 'aaab');
timeMs(safe, 'aaab');

for (const n of [16, 18, 20, 22, 24]) {
  const ms = timeMs(evil, `${'a'.repeat(n)}b`);
  console.log(`/^(a+)+$/  ${n} অক্ষর: ${ms.toFixed(2)} ms`);
}
const safeMs = timeMs(safe, `${'a'.repeat(24)}b`);
console.log(`/^a+$/     24 অক্ষর: ${safeMs.toFixed(2)} ms`);
```

প্রথম দুটি ডাক Warm-up: প্রথমবার Regex Compile হওয়ার খরচ যাতে মাপে না ঢোকে (Benchmark-এর এই সতর্কতা অধ্যায় ১৯-এ বিস্তারিত)। সময়ের সংখ্যা যন্ত্র ও মুহূর্তভেদে বদলায়, তাই এই Listing-এর কোনো নির্দিষ্ট "প্রত্যাশিত" Output নেই, এবং বইয়ের স্বয়ংক্রিয় যাচাই একে শুধু চালায়, সংখ্যা মেলায় না। এই বইয়ের লেখার সময় একটি যন্ত্রে (Windows 10, Node.js 22.16.0) একবার চালানোর আসল Output:

Terminal

```
/^(a+)+$/  16 অক্ষর: 0.44 ms
/^(a+)+$/  18 অক্ষর: 1.73 ms
/^(a+)+$/  20 অক্ষর: 6.52 ms
/^(a+)+$/  22 অক্ষর: 28.40 ms
/^(a+)+$/  24 অক্ষর: 110.09 ms
/^a+$/     24 অক্ষর: 0.05 ms
```

সঙ্গে সঙ্গে দ্বিতীয়বার চালালে সংখ্যাগুলো সামান্য আলাদা ছিল (যেমন ১৬ অক্ষরে 0.58 ms, ২৪ অক্ষরে 111.46 ms), কিন্তু বৃদ্ধির হার একই: প্রতি দুটি অক্ষরে সময় প্রায় চারগুণ; এই হারে ৪০ অক্ষরে কয়েক ঘণ্টা। অথচ `^a+$` ঠিক একই ভাষা মেলায় এবং প্রায় সঙ্গে সঙ্গে শেষ হয়।

প্রতিরোধ:

- **Nested বা পরস্পর-আচ্ছাদিত Quantifier এড়ান:** `(a+)+`, `(a|a)*`, `(\w+\s?)*`।
- **Regex চালানোর আগে দৈর্ঘ্যের সীমা:** ১০০ অক্ষরের সীমায় বিপজ্জনক Pattern-ও ক্ষতি করতে পারে না। Project প্রতিটি ঘরে এটি প্রথমে যাচাই করে।
- **ব্যবহারকারীর লেখা দিয়ে কখনো Pattern নয়** (Escaping ছাড়া)।
- **Server-এ অজানা Pattern চালাতে হলে** Worker Thread-এ সময়সীমাসহ, বা Backtracking-মুক্ত Engine (যেমন RE2) দিয়ে।

১৭.১০ নিরাপদ Input প্রক্রিয়া: Regex কোথায় থামবে

- **Allow-list, Deny-list নয়:** "এগুলো বাদে সব চলবে" (যেমন `<script>` খুঁজে বাদ) সব সময় ভাঙা যায়: `<SCRIPT>`, `<scr<script>ipt>`, `<img onerror>`। "শুধু এগুলোই চলবে" (যেমন শুধু অক্ষর, সংখ্যা, ফাঁকা জায়গা) নিরাপদ।
- **Regex দিয়ে HTML পরিষ্কার করবেন না।** লেখা দেখাতে `textContent` (অধ্যায় ৮); যদি সত্যিই ব্যবহারকারীর HTML দেখাতে হয়, DOMPurify-এর মতো পরীক্ষিত Library, যা আসল HTML Parser ব্যবহার করে।
- **Validation ও Output Encoding আলাদা কাজ।** বৈধ Input-ও একটি প্রেক্ষাপটে বিপজ্জনক হতে পারে: `O'Brien` একটি বৈধ নাম, কিন্তু SQL-এ হাতে জোড়া দিলে Query ভাঙে। প্রতিটি প্রেক্ষাপটের নিজস্ব নিরাপদ উপায় আছে (অধ্যায় ২০)।
- **URL-এর জন্য `URL`** (অধ্যায় ১৫), তারিখের জন্য Parse ও অর্থগত যাচাই, সংখ্যার জন্য রূপান্তর ও সীমা—Regex শুধু গঠনের জন্য।
- **Client-এর যাচাই সুবিধা, নিরাপত্তা নয়।** একই Schema Server-এও চালান; এজন্য Project-এর Engine DOM-মুক্ত, Browser ও Node.js দুই জায়গাতেই চলে।

১৭.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Anchor ছাড়া যাচাই (`/\d{11}/`)	`abc01712345678xyz` গৃহীত	`^...$`
`^a|b$`	Alternation-এর অর্ধেক Anchor-হীন	`^(?:a|b)$`
যাচাইয়ের Regex-এ `g`	পর্যায়ক্রমে সত্য-মিথ্যা	`g` বাদ দিন
`.` দিয়ে "যেকোনো কিছু"	Greedy মিল অতিরিক্ত নেয়	Negated Class বা Lazy
Escaping ছাড়া `new RegExp(userText)`	`SyntaxError` বা ভুল মিল	`escapeRegExp`, বা `includes`
বাংলায় `\w` বা `\b`	বাংলা বাদ পড়ে	`\p{...}` এবং Lookaround
Normalize-এর আগে যাচাই	বাংলা অঙ্ক বা `+880` বাতিল	আগে Normalize
Regex পাস মানেই বৈধ	`2026-02-30` গৃহীত	Parse ও অর্থগত যাচাই
Nested Quantifier	লম্বা ইনপুটে আটকে যাওয়া	Pattern সরল করুন, দৈর্ঘ্যের সীমা
Regex দিয়ে HTML পরিষ্কার	XSS	`textContent` বা পরীক্ষিত Library

১৭.১২ Debugging নির্দেশিকা

- **ছোট টুকরো করে পরীক্ষা করুন:** পুরো Pattern কাজ না করলে Anchor সরিয়ে, তারপর এক এক Group যোগ করে দেখুন কোথায় মেলা থামে।
- **ইতিবাচক ও নেতিবাচক উদাহরণের তালিকা:** যা অবশ্যই মিলবে এবং যা অবশ্যই মিলবে না, দুটি তালিকাই রাখুন; এগুলোই পরে Unit Test।
- `matchAll` ও `d` **Flag** দিয়ে দেখুন ঠিক কোন অংশ কোন Group-এ গেছে।
- **Visual Tool:** regex101.com-এর মতো Website-এ Pattern-এর প্রতিটি অংশের ব্যাখ্যা ও ধাপে ধাপে মেলানো দেখা যায় (ECMAScript Flavor বেছে নিন)। সেখানে ব্যবহারকারীর সংবেদনশীল তথ্য পেস্ট করবেন না।
- **মাঝে মাঝে ভুল ফল?** `g` Flag ও `lastIndex` সন্দেহ করুন।
- **হঠাৎ ধীর?** Nested Quantifier খুঁজুন, এবং ইনপুটের দৈর্ঘ্য Log করুন।

১৭.১৩ পেশাদার অভ্যাস

- **নামযুক্ত Constant-এ Pattern,** পাশে একটি বাক্যে ব্যাখ্যা এবং কয়েকটি উদাহরণ।
- **Named Group** নম্বরের বদলে।
- **সব সময় Anchor** যাচাইয়ের Pattern-এ, Alternation Group-এর ভেতরে।
- **Normalize, তারপর Validate, তারপর Parse,** প্রতিটি ধাপ আলাদা Function।
- **দৈর্ঘ্যের সীমা আগে, Regex পরে।**
- **প্রতিটি নিয়মের নিজস্ব বার্তা,** একটি বড় Regex-এর বদলে।
- **একই Validation Code Client ও Server-এ।**
- **Regex শুধু গঠনের জন্য;** HTML, URL, তারিখ, সংখ্যার জন্য উপযুক্ত হাতিয়ার।

১৭.১৪ Project: FormShield

FormShield একটি ছোট, পুনর্ব্যবহারযোগ্য Validation Engine: প্রতিটি Form-এর জন্য একটি **Schema** লিখুন—কোন ঘর, কীভাবে Normalize, কোন নিয়মে যাচাই, কীভাবে Parse—এবং Engine সব ঘরের ফল ও সব ভুল একসঙ্গে দেয়। অধ্যায় ১১-এর OrderGuard একটি নির্দিষ্ট Form-এর জন্য হাতে লেখা যাচাই ছিল; FormShield যেকোনো Form-এর জন্য একটি সাধারণ যন্ত্র।

Project-এর চাহিদা

- প্রতিটি ঘরের Normalization: NFC, আগে-পরে ফাঁকা জায়গা বাদ, মাঝের বাড়তি ফাঁকা জায়গা একটিতে, বাংলা অঙ্ক থেকে ইংরেজি (যেখানে প্রযোজ্য)।
- **ReDoS প্রতিরোধ:** প্রতিটি ঘরের দৈর্ঘ্য Regex-এর **আগে** যাচাই; সব লেখা `MAX_INPUT` (২৫৬) অক্ষরে সীমিত।

- নিয়মগুলো ছোট, নামযুক্ত, প্রতিটির নিজস্ব বার্তা; একটি ঘরের প্রথম ভাঙা নিয়মের বার্তা জানানো।
- বাংলাদেশি মোবাইল, Email, ব্যবহারকারী-নাম, ISO তারিখ (অর্থগত যাচাইসহ), Password নীতি।
- Schema-র বাইরের ঘর বাদ (Mass Assignment প্রতিরোধ); `__proto__` ধরনের Key কখনো ফলে যায় না।
- DOM-মুক্ত, তাই Browser ও Node.js দুই জায়গাতেই একই নিয়ম।

File-এর কাঠামো

Folder-এর কাঠামো

```
form-shield/
├── src/
│   ├── normalize.mjs     লেখা ও সংখ্যা Normalization
│   ├── patterns.mjs      নামযুক্ত Regex Constant ও Escaping
│   ├── rules.mjs         নিয়মের Factory
│   └── engine.mjs        Schema চালানোর Engine
├── signup-schema.mjs     একটি Form-এর Schema (উদাহরণ)
├── demo.mjs
├── README.md
└── tests/
    ├── normalize.test.mjs
    ├── rules.test.mjs
    └── engine.test.mjs
```

src/normalize.mjs ও src/patterns.mjs

normalize.mjs · JavaScript চলবে: Browser Console ও Node.js

```
// বাংলা অঙ্ক (০-৯) থেকে ইংরেজি অঙ্ক।
export const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

// NFC, আগে-পরে ফাঁকা জায়গা বাদ, মাঝের ফাঁকা জায়গা একটিতে।
export const cleanText = (value) =>
  String(value ?? '').normalize('NFC').replace(/\s+/g, ' ').trim();

export function normalizePhone(value) {
  const digits = toAsciiDigits(cleanText(value)).replace(/[\s\-().]/g, '');
  return digits.replace(/^\+?880/, '0');
}

export const normalizeEmail = (value) => cleanText(value).toLowerCase();
```

patterns.mjs · JavaScript চলবে: Browser Console ও Node.js

```
// প্রতিটি Pattern Anchor-সহ, কোনো g Flag নেই, কোনো Nested Quantifier নেই।
export const PATTERNS = Object.freeze({
  // 01, তারপর 3-9-এর একটি অঙ্ক, তারপর ঠিক ৪টি অঙ্ক: 01712345678
  BD_MOBILE: /^01[3-9]\d{8}$/,
  // @-এর দুই পাশে ফাঁকা জায়গা ও @ ছাড়া কিছু, পরে একটি ডট।
  EMAIL: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
  // ইংরেজি ছোট হাতের অক্ষরে শুরু, তারপর 2-19টি অক্ষর, অঙ্ক বা _
  USERNAME: /^[a-z][a-z0-9_]{2,19}$/,
  // YYYY-MM-DD, অর্থগত যাচাই আলাদা (rules.mjs)
  ISO_DATE: /^(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})$/,
  // অক্ষর, কার-চিহ্ন, ফাঁকা জায়গা, ডট, অ্যাপোস্ট্রফি, হাইফেন
  PERSON_NAME: /^[\p{L}\p{M}][\p{L}\p{M} .'-]*$/u,
});

export const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
```

src/rules.mjs

rules.mjs · JavaScript চলবে: Browser Console ও Node.js

```
import { PATTERNS } from './patterns.mjs';

// প্রতিটি নিয়ম: (value) => বার্তা, অথবা ঠিক থাকলে null।
export const required = (message = 'ঘরটি পূরণ করুন।') => (value) =>
  (value === '' ? message : null);

export const length = (min, max) => (value) =>
  ([...value].length < min || [...value].length > max
    ? `দৈর্ঘ্য ${min} থেকে ${max} অক্ষরের মধ্যে হতে হবে।`
    : null);

export const matches = (regex, message) => (value) =>
  (regex.test(value) ? null : message);

export const notIn = (blocked, message) => (value) =>
  (blocked.includes(value.toLowerCase()) ? message : null);

export function realDate(value) {
  const match = PATTERNS.ISO_DATE.exec(value);
  if (!match) {
    return 'তারিখ YYYY-MM-DD আকারে লিখুন।';
  }
  const [y, m, d] = Object.values(match.groups).map(Number);
  const date = new Date(Date.UTC(y, m - 1, d));
  const real = date.getUTCMonth() === m - 1 && date.getUTCDate() === d;
  return real ? null : 'এমন কোনো তারিখ নেই।';
}
```

```
export const passwordPolicy = [
  length(12, 64),
  matches(/\p{Lu}/u, 'একটি বড় হাতের অক্ষর দিন।'),
  matches(/\d/, 'একটি সংখ্যা দিন।'),
  (value) => (/(.)\1\1/.test(value) ? 'একই অক্ষর পরপর তিনবার নয়।' : null),
];
```

প্রতিটি নিয়ম একটি **Factory** (অধ্যায় ১৪): প্রয়োজনীয় সেটিংস নিয়ে একটি ছোট Function ফেরত দেয়, যা একটি মান নিয়ে বার্তা বা `null` দেয়। `length` গোনে Code Point (`[...value]`), যাতে একটি Emoji দুটি অক্ষর না গোনা হয় (অধ্যায় ৮)। `realDate` হলো অনুচ্ছেদ ১৭.৭-এর Parse ধাপ: Named Group-এর মান `Object.values` দিয়ে ক্রমে বের করা হয়েছে।

src/engine.mjs

engine.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { cleanText } from './normalize.mjs';

export const MAX_INPUT = 256;

// Schema: { field: { normalize, rules: [...], optional } }
// ফল: { ok, values, errors }; values-এ শুধু Schema-র ঘর।
export function validate(schema, input) {
  const values = Object.create(null);
  const errors = Object.create(null);

  for (const [field, spec] of Object.entries(schema)) {
    const raw = Object.hasOwn(input ?? {}, field) ? input[field] : '';
    const text = String(raw ?? '');

    if (text.length > MAX_INPUT) {
      errors[field] = `লেখা অনেক বড় (সর্বোচ্চ ${MAX_INPUT} অক্ষর)।`;
      continue;
    }

    const value = (spec.normalize ?? cleanText)(text);
    if (spec.optional && value === '') {
      continue;
    }

    const message = spec.rules.map((rule) => rule(value)).find(Boolean);
    if (message) {
      errors[field] = message;
    } else {
      values[field] = value;
    }
  }
```

```
  const ok = Object.keys(errors).length === 0;
  return { ok, values: { ...values }, errors: { ...errors } };
}
```

- **দৈর্ঘ্য সবার আগে:** Normalization বা কোনো Regex চলার আগেই ২৫৬ অক্ষরের সীমা। একটি ১০ লক্ষ অক্ষরের ক্ষতিকর ইনপুট সঙ্গে সঙ্গে প্রত্যাখ্যাত (ReDoS প্রতিরোধ)।
- **শুধু Schema-র ঘর:** Loop চলে Schema-র ওপর, ইনপুটের ওপর নয়। ইনপুটে `isAdmin: true` বা `__proto__` থাকলেও ফলে যায় না—**Mass Assignment** প্রতিরোধ। `Object.hasOwn` উত্তরাধিকারসূত্রে পাওয়া Key পড়ে না (অধ্যায় ১১)।
- **`Object.create(null)`** একটি Prototype-হীন Object তৈরি করে, যাতে কোনো ঘরের নাম কোনো উত্তরাধিকারসূত্রে পাওয়া Property-র সঙ্গে সংঘর্ষে না যায়; শেষে সাধারণ Object-এ Copy করে ফেরত।
- **`find(Boolean)`:** নিয়মগুলো ক্রমে চলে, প্রথম বার্তাটি নেওয়া হয়। প্রথমে "পূরণ করুন", তারপর দৈর্ঘ্য, তারপর গঠন—সবচেয়ে মৌলিক সমস্যাটি আগে জানানো।
- **Optional ঘর** ফাঁকা হলে নিয়ম চলে না; পূরণ করলে চলে।

Schema এবং প্রদর্শনী

signup-schema.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { normalizeEmail, normalizePhone, cleanText } from './src/normalize.mjs';
import { PATTERNS } from './src/patterns.mjs';
import {
  length, matches, notIn, passwordPolicy, realDate, required,
} from './src/rules.mjs';

export const signupSchema = {
  name: {
    rules: [required(), length(2, 60), matches(PATTERNS.PERSON_NAME, 'নামে শুধু অক্ষর
        দিন।')],
  },
  username: {
    normalize: (value) => cleanText(value).toLowerCase(),
    rules: [
      required(),
      matches(PATTERNS.USERNAME, '3-20 অক্ষর: ইংরেজি ছোট হাতের অক্ষর, অঙ্ক বা _'),
      notIn(['admin', 'root', 'support'], 'এই নাম সংরক্ষিত।'),
    ],
  },
  email: { normalize: normalizeEmail, rules: [required(), matches(PATTERNS.EMAIL,
      'সঠিক Email লিখুন।')] },
  phone: {
    normalize: normalizePhone,
    rules: [required(), matches(PATTERNS.BD_MOBILE, 'সঠিক বাংলাদেশি মোবাইল নম্বর লিখুন।')],
```

```
  },
  birthDate: { optional: true, rules: [realDate] },
  password: {
    normalize: (value) => value,
    rules: [required(), ...passwordPolicy],
  },
};
```

demo.mjs · JavaScript — চলবে: Node.js

```
import { validate } from './src/engine.mjs';
import { signupSchema } from './signup-schema.mjs';

const good = validate(signupSchema, {
  name: '  মোঃ  রফিক ',
  username: 'Rafiq_88',
  email: ' Rafiq@Example.COM ',
  phone: '+৮৮০ ১৭১২-৩৪৫৬৭৮',
  birthDate: '2000-02-29',
  password: 'Bogura-River-2026',
  isAdmin: true,
});
console.log(good.ok, good.values.name, good.values.phone, good.values.email);
console.log('isAdmin' in good.values, good.values.password === undefined);

const bad = validate(signupSchema, {
  name: 'R2D2',
  username: 'admin',
  email: 'rafiq@',
  phone: '01212345678',
  birthDate: '2001-02-29',
  password: 'aaa1111',
});
console.log(bad.ok, Object.keys(bad.errors).length);
for (const [field, message] of Object.entries(bad.errors)) {
  console.log(`${field}: ${message}`);
}
console.log(validate(signupSchema, { name: 'x'.repeat(1_000_000) }).errors.name);
```

Output

```
true মোঃ রফিক 01712345678 rafiq@example.com
false false
false 6
name: নামে শুধু অক্ষর দিন।
username: এই নাম সংরক্ষিত।
email: সঠিক Email লিখুন।
phone: সঠিক বাংলাদেশি মোবাইল নম্বর লিখুন।
birthDate: এমন কোনো তারিখ নেই।
password: দৈর্ঘ্য 12 থেকে 64 অক্ষরের মধ্যে হতে হবে।
লেখা অনেক বড় (সর্বোচ্চ 256 অক্ষর)।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **বৈধ ইনপুট পরিষ্কার হয়ে আসে:** নামের বাড়তি ফাঁকা জায়গা, Email-এর বড় হাতের অক্ষর, `+৮৮০` ও বাংলা অঙ্কসহ মোবাইল নম্বর—সব আদর্শ রূপে। `+৮৮০`-এর বাংলা অঙ্ক আগে ইংরেজি হয়েছে, তারপর `880` বাদ পড়েছে; Normalization-এর ক্রম গুরুত্বপূর্ণ।
- `isAdmin` **ফলে নেই:** Schema-তে নেই এমন ঘর কখনো ফলের অংশ হয় না।
- **Password ফলে আছে** (`undefined` নয়), কারণ এই Engine-এর কাজ যাচাই; Password-এর Hash তৈরি ও নিরাপদ সংরক্ষণ অধ্যায় ২০-এর বিষয়। তার `normalize` লেখাটি অপরিবর্তিত রাখে, কারণ Password-এর ফাঁকা জায়গাও অর্থবহ।
- **২০০০ সাল অধিবর্ষ,** তাই ২৯ ফেব্রুয়ারি বৈধ; ২০০১-এর নয়।
- **ছয়টি ঘরের প্রতিটির নিজস্ব বার্তা,** একসঙ্গে। `R2D2` নামে সংখ্যা থাকায় বাতিল; `admin` গঠনে ঠিক, কিন্তু সংরক্ষিত তালিকায়।
- **দশ লক্ষ অক্ষরের নাম** Normalization বা Regex পর্যন্ত পৌঁছায়নি।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node demo.mjs
node --test
```

Companion Source Code-এ ২৪টি Test তিনটি File-এ: Normalization (বাংলা অঙ্ক, `+880`, NFC), নিয়ম (Boundary, অধিবর্ষ, Lookahead-মুক্ত Password নীতি, `g` Flag না থাকা), এবং Engine (Mass Assignment, `__proto__`, Optional ঘর, ReDoS-এর দৈর্ঘ্য-সীমা এবং ১০ লক্ষ অক্ষরের ইনপুটে তাৎক্ষণিক প্রত্যাখ্যান)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
বাংলা অঙ্কের নম্বর বাতিল	Normalization-এ `toAsciiDigits` নেই	`normalize` ঘরে দিন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই ইনপুট কখনো বৈধ, কখনো নয়	Pattern-এ `g` Flag	`g` বাদ দিন
`2026-02-30` গৃহীত	শুধু Regex, Parse নেই	`realDate`
অজানা ঘর ফলে আসে	ইনপুটের ওপর Loop	Schema-র ওপর Loop
বড় ইনপুটে ধীর	দৈর্ঘ্যের সীমা পরে	`MAX_INPUT` আগে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Email ও নামের যাচাই ইচ্ছা করে উদার; ফাঁস হওয়া Password-এর তালিকা যাচাই নেই; বার্তাগুলো একটি ভাষায়। Browser-এ যাচাই শুধু ব্যবহারকারীর সুবিধা; Server-এ একই Schema আবার চালাতে হবে।

১৭.১৫ অনুশীলনী

প্রাথমিক স্তর

১. একটি লেখা থেকে সব `#hashtag` (ইংরেজি অক্ষর, অঙ্ক, `_`) বের করুন।

২. `^(?:jpg|png|gif)$` এবং `^jpg|png|gif$`-এর পার্থক্য `'xpng'` দিয়ে দেখান।

৩. Named Group দিয়ে `'14:35'` থেকে ঘণ্টা ও মিনিট আলাদা করুন।

৪. একটি Constant-এ রাখা `/\d+/g` দিয়ে একই ইনপুট তিনবার যাচাই করলে কী হয়? ঠিক করুন।

৫. বাংলা অঙ্কের একটি দাম (`'৳১,২৫০'`) থেকে সংখ্যা (`1250`) বের করুন।

মধ্যম স্তর

৬. `replace`-এ Function দিয়ে একটি লেখার প্রতিটি শব্দের প্রথম অক্ষর বড় হাতের করুন।

৭. `escapeRegExp` ব্যবহার করে ব্যবহারকারীর খোঁজার শব্দ (`'1+1=2'`) লেখায় চিহ্নিত করুন (`[1+1=2]`)।

৮. Lookbehind দিয়ে শুধু `#` চিহ্নের পরের Hex রঙের কোড (`#1d4f9c`) বের করুন, `#` ছাড়া।

৯. FormShield-এর জন্য একটি `postcode` নিয়ম লিখুন: বাংলাদেশি পোস্টকোড ৪ অঙ্ক, বাংলা অঙ্কও চলবে।

চ্যালেঞ্জ স্তর

১০. Sticky Flag (`y`) দিয়ে একটি ছোট Tokenizer লিখুন, যা `'12 + 3*4'` থেকে সংখ্যা ও চিহ্নের তালিকা দেয়, ফাঁকা জায়গা বাদ।

১১. একটি Log থেকে `matchAll` দিয়ে প্রতিটি স্তরের (ERROR, WARN) সংখ্যা গুনুন।

১২. FormShield-এ একটি `confirmPassword` ঘর যোগ করুন, যা `password`-এর সঙ্গে মিলতে হবে। Engine-এর নকশা কীভাবে বাড়াতে হবে, দেখান।

১৭.১৬ অনুশীলনীর সমাধান

সমাধান ১

hashtags.js · JavaScript — চলবে: Node.js

```
const post = 'আজ #JavaScript আর #node_js শিখলাম! #2026 #';

console.log(post.match(/#\w+/g));
```

Output

```
[ '#JavaScript', '#node_js', '#2026' ]
```

একা `#`-এর পরে কোনো `\w` নেই, তাই মেলেনি (`+` মানে কমপক্ষে একটি)।

সমাধান ২

anchor-group.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(/^(?:jpg|png|gif)$/.test('xpng'));
console.log(/^jpg|png|gif$/.test('xpng'));
```

Output

```
false
true
```

দ্বিতীয় Pattern আসলে "`^jpg` অথবা `png` অথবা `gif$`", তাই `xpng`-এর ভেতরের `png` মিলেছে।

সমাধান ৩

time-groups.js · JavaScript — চলবে: Browser Console ও Node.js

```
const { hour, minute } = /^(?<hour>\d{2}):(?<minute>\d{2})$/.exec('14:35').groups;

console.log(Number(hour) * 60 + Number(minute));
```

Output

```
875
```

সমাধান ৪ — `g`-যুক্ত Regex `lastIndex` মনে রাখে, তাই ফল হয় `true`, `false`, `true` (অনুচ্ছেদ ১৭.৫)। যাচাইয়ে `g` বাদ দিন: `/\d+/`। প্রতিটি ডাকে নতুন Regex তৈরি করাও কাজ করে, কিন্তু `g` বাদ দেওয়াই সঠিক সমাধান।

সমাধান ৫

bangla-price.js · JavaScript — চলবে: Browser Console ও Node.js

```
const toAsciiDigits = (text) =>
  text.replace(/[০-৯]/g, (digit) => String(digit.charCodeAt(0) - 0x09e6));

const price = Number(toAsciiDigits('৳১,২৫০').replace(/[^\d]/g, ''));
console.log(price + 1);
```

Output

```
1251
```

সমাধান ৬

title-case.js · JavaScript — চলবে: Browser Console ও Node.js

```
const titleCase = (text) =>
  text.toLowerCase().replace(/\b[a-z]/g, (letter) => letter.toUpperCase());

console.log(titleCase('modern JAVASCRIPT for bangla readers'));
```

Output

```
Modern Javascript For Bangla Readers
```

সমাধান ৭

highlight-escaped.js · JavaScript — চলবে: Browser Console ও Node.js

```
const escapeRegExp = (text) => text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

const query = '1+1=2';
const pattern = new RegExp(escapeRegExp(query), 'g');

console.log('প্রমাণ: 1+1=2, আর 11=2 নয়'.replace(pattern, (match) => `[${match}]`));
```

Output

```
প্রমাণ: [1+1=2], আর 11=2 নয়
```

Escaping ছাড়া `1+1` মানে "এক বা একাধিক 1, তারপর 1", এবং ভুল জায়গায় মিলত।

সমাধান ৮

hex-colors.js · JavaScript চলবে: Node.js

```javascript
const css = 'color: #1d4f9c; background: #FFF; border: 1px solid #12305f80;';

console.log(css.match(/(?<=#)(?:[0-9a-f]{6}|[0-9a-f]{3})(?![0-9a-f])/gi));
```

Output

```
[ '1d4f9c', 'FFF' ]
```

শেষের Negative Lookahead নিশ্চিত করে যে ৮ অঙ্কের `#12305f80` থেকে প্রথম ৬টি ভুল করে নেওয়া হয়নি।

সমাধান ৯

postcode.mjs · JavaScript চলবে: Node.js

```javascript
import { validate } from '../../project/form-shield/src/engine.mjs';
import {
  toAsciiDigits, cleanText,
} from '../../project/form-shield/src/normalize.mjs';
import { matches, required } from '../../project/form-shield/src/rules.mjs';

const schema = {
  postcode: {
    normalize: (value) => toAsciiDigits(cleanText(value)),
    rules: [required(), matches(/^\d{4}$/, 'পোস্টকোড 4 অঙ্কের।')],
  },
};

for (const postcode of ['৫৮০০', '5800', '580', 'abcd']) {
  const result = validate(schema, { postcode });
  const shown = result.ok ? result.values.postcode : result.errors.postcode;
  console.log(postcode, shown);
}
```

Output

```
৫৮০০ 5800
5800 5800
580 পোস্টকোড 4 অঙ্কের।
abcd পোস্টকোড 4 অঙ্কের।
```

Engine-এর কোনো পরিবর্তন লাগেনি; নতুন ঘর মানে শুধু Schema-য় একটি নতুন প্রবেশ।

সমাধান ১০

tokenizer.js · JavaScript চলবে: Node.js

```
function tokenize(text) {
  const token = /\s*(?:(?<number>\d+)|(?<op>[+\-*/]))/y;
  const source = text.trimEnd();
  const tokens = [];
  let position = 0;

  while (position < source.length) {
    token.lastIndex = position;
    const match = token.exec(source);
    if (match === null) {
      throw new SyntaxError(`অচেনা অক্ষর, অবস্থান ${position}`);
    }
    tokens.push(match.groups.number ?? match.groups.op);
    position = token.lastIndex;
  }
  return tokens;
}

console.log(tokenize('12 + 3*4'));
try {
  tokenize('12 + x');
} catch (error) {
  console.log(error.message);
}
```

Output

```
[ '12', '+', '3', '*', '4' ]
অচেনা অক্ষর, অবস্থান 4
```

Sticky Flag-এ প্রতিটি মিল ঠিক `lastIndex` থেকে শুরু হতে হয়; অচেনা অক্ষরে এসে মেলা থামে, এবং অবস্থান থেকে বোঝা যায় কোথায় সমস্যা। মেলা ব্যর্থ হলে `exec` `lastIndex`-কে ০-তে ফিরিয়ে দেয়, তাই অবস্থান আলাদা Variable-এ (`position`) রাখা হয়েছে। `g` হলে Regex অচেনা অক্ষর লাফিয়ে পরের সংখ্যা খুঁজত, এবং ভুলটি নীরবে চাপা পড়ত।

সমাধান ১১

log-levels.js · JavaScript চলবে: Node.js

```
const log = `ERROR db timeout
WARN slow query
ERROR db timeout
INFO started
WARN cache miss`;

const counts = {};
for (const { groups } of log.matchAll(/^(?<level>ERROR|WARN)\b/gm)) {
  counts[groups.level] = (counts[groups.level] ?? 0) + 1;
}
console.log(counts);
```

Output

```
{ ERROR: 2, WARN: 2 }
```

`m` Flag-এ `^` প্রতিটি লাইনের শুরু; তাই একটি লাইনের মাঝের "ERROR" শব্দ গোনা হতো না।

সমাধান ১২

confirm.mjs · JavaScript চলবে: Node.js

```
import { validate } from '../../project/form-shield/src/engine.mjs';
import { required } from '../../project/form-shield/src/rules.mjs';

// ঘরের নিয়ম একটি মান দেখে; দুটি ঘরের তুলনা Engine-এর পরে, Form-স্তরের নিয়মে।
function validateWithChecks(schema, input, checks) {
  const result = validate(schema, input);
  const errors = { ...result.errors };
  for (const check of checks) {
    const problem = check(result.values);
    if (problem && !errors[problem.field]) {
      errors[problem.field] = problem.message;
    }
  }
  return { ok: Object.keys(errors).length === 0, values: result.values, errors };
}

const schema = {
  password: { normalize: (v) => v, rules: [required()] },
  confirmPassword: { normalize: (v) => v, rules: [required()] },
};
const samePassword = (values) =>
  (values.password !== values.confirmPassword
```

```
    ? { field: 'confirmPassword', message: 'দুটি Password মেলেনি।' }
    : null);

const input = {
  password: 'Bogura-River-2026',
  confirmPassword: 'Bogura-river-2026',
};
console.log(validateWithChecks(schema, input, [samePassword]).errors);
```

Output

```
{ confirmPassword: 'দুটি Password মেলেনি।' }
```

নকশার সিদ্ধান্ত: ঘরের নিয়ম শুধু নিজের মান দেখে, তাই সেগুলো সরল ও পুনর্ব্যবহারযোগ্য থাকে। একাধিক ঘরের সম্পর্ক (মেলা, তারিখের ক্রম) আলাদা "Form-স্তরের" নিয়মে, যা সব ঘর পরিষ্কার হওয়ার পর চলে।

১৭.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `/<.+?>/`-এর `?` কী বোঝায়?

ক) আগের অক্ষরটি ঐচ্ছিক

খ) Quantifier Lazy: যতটা সম্ভব কম নেয়

গ) Lookahead

ঘ) Named Group

প্রশ্ন ২. `/^cat|dog$/.test('catalog')`-এর ফল কেন `true`?

ক) `|`-এর অগ্রাধিকার সবচেয়ে কম; `^cat` একাই মেলে

খ) `i` Flag নেই

গ) `catalog`-এ `dog` আছে

ঘ) এটি `false` দেয়

প্রশ্ন ৩. `(?<year>\d{4})`-এর মান কোথায় পাওয়া যায়?

ক) `match.year`

খ) `match[0]`

গ) `match.groups.year`

ঘ) `match.index`

প্রশ্ন ৪. `(?<=৳)\d+` কোনটি মেলায়?

ক) `৳`-সহ পুরো দাম

খ) `৳`-এর আগের সংখ্যা

গ) যেকোনো সংখ্যা

ঘ) `৳`-এর পরের সংখ্যা, `৳` ছাড়া

প্রশ্ন ৫. একটি Constant-এ রাখা `/x/g` দিয়ে একই লেখা বারবার `test` করলে কী হয়?

ক) সব সময় `true`

খ) `lastIndex`-এর কারণে ফল পর্যায়ক্রমে বদলায়

গ) `SyntaxError`

ঘ) সব সময় `false`

প্রশ্ন ৬. ব্যবহারকারীর লেখা `c++` দিয়ে নিরাপদে খুঁজতে কী করবেন?

ক) `new RegExp('c++')`

খ) `i` Flag যোগ

গ) বিশেষ অক্ষর Escape করা, বা `includes` ব্যবহার

ঘ) `g` Flag যোগ

প্রশ্ন ৭. বাংলা লেখায় `\b` কেন নির্ভরযোগ্য নয়?

ক) `\b` শুধু ইংরেজি `\w`-এর সংজ্ঞায় শব্দের সীমানা চেনে

খ) বাংলায় ফাঁকা জায়গা নেই

গ) `\b` শুধু সংখ্যা চেনে

ঘ) `u` Flag `\b` নিষিদ্ধ করে

প্রশ্ন ৮. `2026-02-30` তারিখ-Regex পাস করে। সঠিক পদক্ষেপ?

ক) আরও জটিল Regex

খ) গ্রহণ করা

গ) `g` Flag

ঘ) Parse করে মাস ও দিন মিলিয়ে অর্থগত যাচাই

প্রশ্ন ৯. কোন Pattern ReDoS-এর ঝুঁকিপূর্ণ?

ক) `^(a+)+$`

খ) `^a+$`

গ) `^\d{4}$`

ঘ) `^[a-z]{3,20}$`

> **প্রশ্ন ১০.** ReDoS প্রতিরোধে Regex চালানোর আগে কোনটি সবচেয়ে কার্যকর?
>
> ক) `i` Flag
>
> খ) ইনপুটের দৈর্ঘ্যের সীমা
>
> গ) `toUpperCase`
>
> ঘ) `JSON.stringify`

১৭.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Quantifier-এর পরে `?` তাকে Lazy করে।
২	ক	`^cat` অথবা `dog$`; পুরোটাকে Group-এ রাখতে হয়।
৩	গ	Named Group-এর মান `groups` Object-এ।
৪	ঘ	Lookbehind শর্ত যাচাই করে, অক্ষর নেয় না।
৫	খ	`g` Regex শেষ অবস্থান মনে রাখে।
৬	গ	`+` একটি Quantifier; আক্ষরিক খোঁজায় Escape বা `includes`।
৭	ক	`u` Flag দিলেও `\b` বাংলা অক্ষরকে শব্দ-অক্ষর ধরে না।
৮	ঘ	Regex গঠন দেখে, অর্থ নয়।
৯	ক	Nested Quantifier-এ পথের সংখ্যা সূচকীয় হারে বাড়ে।
১০	খ	সীমিত দৈর্ঘ্যে সবচেয়ে খারাপ Pattern-ও দ্রুত শেষ হয়।

১৭.১৯ অধ্যায়ের সারসংক্ষেপ

- Character Class, Negated Class, `{m,n}`, `\b`; Quantifier Greedy, `?` দিলে Lazy।
- Group ধরে রাখে, Named Group নাম দেয়, `(?:)` শুধু দলবদ্ধ করে, `\1` আগের মিলের পুনরাবৃত্তি; Alternation-এর অগ্রাধিকার সবচেয়ে কম।
- Lookahead ও Lookbehind অবস্থানের শর্ত, অক্ষর গ্রাস করে না।
- `matchAll` পূর্ণ মিল-তথ্য, `replace`-এ `$<name>` ও Function, `split`-এ Regex, Sticky Tokenizer।
- `g` / `y` Regex `lastIndex` মনে রাখে; যাচাইয়ে `g` নয়।
- ব্যবহারকারীর লেখা দিয়ে Regex মানে Escaping; আক্ষরিক খোঁজায় `includes`।

- বাংলায় `\p{Script=Bengali}`, বাংলা অঙ্কের রূপান্তর; `\w` ও `\b` বাংলা চেনে না।
- Normalize, Validate, Parse—তিন আলাদা ধাপ; Regex শুধু গঠন যাচাই করে।
- Email, নাম, Password-এ Regex-এর বাড়াবাড়ি নয়; Password-এ আলাদা নিয়ম ও বার্তা।
- ReDoS: Nested Quantifier এড়ান, দৈর্ঘ্যের সীমা আগে; HTML পরিষ্কারে Regex নয়।

পরের অধ্যায়ে Functional Programming-এর উন্নত প্রয়োগ: Immutable তথ্য, জটিল তথ্যের রূপান্তর, Grouping ও Aggregation।

১৭.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Character Class	বন্ধনীর ভেতরের যেকোনো একটি অক্ষর।
Quantifier	কতবার: `*`, `+`, `?`, `{m,n}`।
Greedy / Lazy	যতটা সম্ভব বেশি / কম নেওয়া।
Anchor	অবস্থান: `^`, `$`, `\b`।
Capturing Group	মিলে যাওয়া অংশ ধরে রাখা বন্ধনী।
Named Group	নামযুক্ত Capturing Group।
Backreference	আগের Group-এর মিলের পুনরাবৃত্তি।
Lookaround	পাশে কী আছে তার শর্ত, অক্ষর না নিয়ে।
Sticky Flag	ঠিক `lastIndex` থেকেই মেলানো।
Escaping	বিশেষ অক্ষরকে আক্ষরিক অর্থ দেওয়া।
ReDoS	Backtracking-এর বিস্ফোরণে Program আটকে যাওয়া।
Mass Assignment	অনাকাঙ্ক্ষিত ঘর ইনপুট থেকে ফলে ঢুকে পড়া।

১৭.২১ আরও পড়ুন

- **MDN — Regular expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions` — সম্পূর্ণ নির্দেশিকা।
- **MDN — Lookahead assertion:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Lookahead_assertion` — Lookaround।

- **MDN — Unicode character class escape:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Unicode_character_class_escape` — `\p{...}`।
- **OWASP — Regular expression Denial of Service:** `https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS` — ReDoS-এর ব্যাখ্যা।
- **OWASP — Input Validation Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html` — Allow-list ও যাচাইয়ের নীতি।

অধ্যায় ১৮

১৮

Functional Programming, Immutability এবং Advanced Data Transformation

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Referential Transparency ব্যাখ্যা করতে, এবং কেন তা Code-কে যুক্তি দিয়ে বোঝা সহজ করে তা বলতে।
- Nested তথ্য Immutable উপায়ে হালনাগাদ করতে, এবং Structural Sharing-এর সুবিধা কাজে লাগাতে।
- Path ধরে হালনাগাদের একটি সাধারণ সাহায্যকারী (`setIn` , `updateIn`) লিখতে।
- `flatMap` দিয়ে Nested তথ্য সমতল করতে, এবং `reduce` দিয়ে `Map` -এ জমাতে।
- `Map.groupBy` ও বহুস্তর Grouping দিয়ে Aggregation ও Pivot তৈরি করতে।
- একাধিক তালিকা ID ধরে জোড়া দিতে (Join), দক্ষ উপায়ে।
- Stable, বহু-Key তুলনা এবং বাংলা নামের জন্য `Intl.Collator` দিয়ে সাজাতে।
- Result Type দিয়ে ভুল সারিসহ তথ্যের প্রবাহ সামলাতে।
- Asynchronous তথ্য-প্রবাহ ক্রমে, একসঙ্গে, বা সীমিত দলে চালাতে।
- একটি সম্পূর্ণ **LedgerFlow** Order Ledger Analytics তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৬–৭: Array Method, Object, Spread, Shallow বনাম Deep Copy।
- অধ্যায় ১০: Promise, `Promise.all`, সীমিত Concurrency (অনুশীলনী ১১)।
- অধ্যায় ১২: `toSorted`, `structuredClone`, `Object.groupBy`।
- অধ্যায় ১৪: Pure Function, Side Effect, `pipe`, `compose`, Currying, Memoization।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৪-এ Functional Programming-এর মূল হাতিয়ারগুলো শিখেছেন: Pure Function, Composition, Currying। এই অধ্যায় সেগুলোকে বাস্তব তথ্যের মুখোমুখি করে। বাস্তব তথ্য সমতল তালিকা নয়: একটি Order-এর ভেতরে একাধিক পণ্য, প্রতিটি পণ্যের তথ্য আরেকটি তালিকায়, গ্রাহকের তথ্য তৃতীয়টিতে; কিছু সারি ভাঙা; কিছু তথ্য Network থেকে আসে, সময় নিয়ে। আর এই তথ্য বদলাতেও হয়: একটি Order ফেরত এলে তার একটি পণ্যের অবস্থা বদলাতে হবে, পুরো ব্যবস্থাকে অক্ষত রেখে।

এগুলো সামলানোর Functional উপায়গুলো আধুনিক Frontend (React-এর State, Redux) ও Data Processing-এর মূল ভিত্তি। এই অধ্যায় শেষে আপনি জটিল, Nested, অসম্পূর্ণ তথ্যকে নির্ভরযোগ্য, পরীক্ষাযোগ্য Pipeline-এ রূপান্তর করতে পারবেন।

১৮.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Pure Function, Side Effect (অধ্যায় ১৪)	Referential Transparency
Spread দিয়ে এক স্তরের Copy (অধ্যায় ৭)	Nested হালনাগাদ, Structural Sharing, `setIn` / `updateIn`
`map`, `filter`, `reduce` (অধ্যায় ৬)	`flatMap`, `reduce`-এ `Map`
`Object.groupBy` (অধ্যায় ১২)	`Map.groupBy`, বহুস্তর Grouping, Pivot
`localeCompare` (অধ্যায় ৮)	Stable Sort, বহু-Key Comparator, `Intl.Collator`
`partitionValid` (অধ্যায় ১৪)	Result Type
`Promise.all` (অধ্যায় ১০)	Asynchronous Pipeline, Batch প্রক্রিয়া

১৮.২ Referential Transparency

একটি Expression **Referentially Transparent** যদি তাকে তার ফল দিয়ে প্রতিস্থাপন করলে Program-এর আচরণ বদলায় না। Pure Function-এর ডাক এমনই: `add(2, 3)` সর্বত্র `5` দিয়ে বদলানো যায়।

transparency.js · JavaScript — চলবে: Browser Console ও Node.js

```
const vat = (price) => Math.round(price * 1.15);

const total1 = vat(1000) + vat(1000);
const once = vat(1000);
const total2 = once + once;
console.log(total1 === total2);

let counter = 0;
const nextId = () => {
  counter += 1;
  return counter;
};

const ids1 = [nextId(), nextId()];
const single = nextId();
const ids2 = [single, single];
console.log(ids1.join(','), ids2.join(','));
```

Output

```
true
1,2 3,3
```

`vat(1000)` দুবার ডাকা বা একবার ডেকে দুবার ব্যবহার—একই ফল। `nextId()` তেমন নয়: একবার ডেকে মান পুনর্ব্যবহার করলে আচরণ বদলে গেছে। এর ব্যবহারিক মূল্য: Referentially Transparent Code-এ আপনি নিশ্চিন্তে Refactor করতে পারেন (একটি হিসাব একটি Variable-এ তুলে নেওয়া, Memoize করা, ক্রম বদলানো), কারণ ফল বদলাবে না। Impure Code-এ এমন প্রতিটি পরিবর্তন একটি সম্ভাব্য Bug।

১৮.৩ Nested তথ্যের Immutable হালনাগাদ

অধ্যায় ৭-এ এক স্তরের Spread দেখেছেন। Nested তথ্যে প্রতিটি স্তরে নতুন Object তৈরি করতে হয়, **শুধু বদলের পথ ধরে**।

nested-update.js · JavaScript — চলবে: Browser Console ও Node.js

```
const store = {
  name: 'Boi Ghar',
  address: { city: 'Bogura', zip: '5800' },
  orders: [
```

```
    { id: 'O1', items: [{ sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 }] },
    { id: 'O2', items: [{ sku: 'B3', qty: 5 }] },
  ],
};

const updated = {
  ...store,
  orders: store.orders.map((order) =>
    (order.id !== 'O1' ? order : {
      ...order,
      items: order.items.map((item) =>
        (item.sku === 'B2' ? { ...item, qty: 3 } : item)),
    })),
};

console.log(store.orders[0].items[1].qty, updated.orders[0].items[1].qty);
console.log(updated.address === store.address);
console.log(updated.orders[1] === store.orders[1]);
console.log(updated.orders[0].items[0] === store.orders[0].items[0]);
console.log(updated.orders[0] === store.orders[0]);
```

Output

```
1 3
true
true
true
false
```

শুধু বদলের পথের Object নতুন: মূল Store, `orders` Array, `O1` Order, তার `items` Array এবং `B2` পণ্য। বাকি সব— `address`, `O2`, `B1` —**একই** Object, নতুন ও পুরোনো Store দুজনেই ভাগ করে নেয়। একে বলে **Structural Sharing**।

এর দুটি বড় সুবিধা:

- **সস্তা:** ১০,০০০ Order-এর Store-এ একটি পণ্য বদলাতে পাঁচটি Object তৈরি হয়, ১০,০০০ নয়। `structuredClone` দিয়ে পুরোটা Copy করলে সব Copy হতো।
- **দ্রুত পরিবর্তন-শনাক্তকরণ:** কোন অংশ বদলেছে জানতে গভীর তুলনা লাগে না; `===` যথেষ্ট। `updated.address === store.address` সত্য মানে ঠিকানা বদলায়নি। React এই নীতিতেই কোন অংশ নতুন করে আঁকতে হবে তা ঠিক করে।

তবে লক্ষ করুন, হাতে লেখা Nested Spread পড়া কষ্টকর এবং ভুলপ্রবণ।

Path ধরে হালনাগাদ: setIn এবং updateIn

update-in.js · JavaScript চলবে: Browser Console ও Node.js

```
function updateIn(target, [key, ...rest], change) {
  const current = target?.[key];
  const next = rest.length === 0
    ? change(current)
    : updateIn(current, rest, change);
  if (Array.isArray(target)) {
    return target.with(key, next);
  }
  return { ...target, [key]: next };
}

const setIn = (target, path, value) => updateIn(target, path, () => value);

const store = { address: { city: 'Bogura' }, orders: [{ items: [{ qty: 2 }] }] };
const a = setIn(store, ['address', 'city'], 'Dhaka');
const b = updateIn(store, ['orders', 0, 'items', 0, 'qty'], (qty) => qty + 1);

console.log(a.address.city, store.address.city, a.orders === store.orders);
console.log(b.orders[0].items[0].qty, b.address === store.address);
```

Output

```
Dhaka Bogura true
3 true
```

`updateIn` Recursive: Path-এর প্রথম Key-র মানকে বাকি Path দিয়ে হালনাগাদ করে, তারপর সেই স্তরের একটি নতুন Object (বা Array-র জন্য `with`, অধ্যায় ১২) ফেরত দেয়। Structural Sharing নিজে থেকেই রক্ষিত। Immer-এর মতো Library একই কাজ আরও সুবিধাজনক Syntax-এ করে; ভেতরের নীতিটি এটিই।

Deep Freeze: ভুল করে বদলানো ধরা

`Object.freeze` শুধু এক স্তরের (অধ্যায় ৭)। Development-এ পুরো গাছ Freeze করলে ভুল করে Mutation সঙ্গে সঙ্গে ধরা পড়ে:

deep-freeze.js · JavaScript চলবে: Browser Console ও Node.js

```
'use strict';

function deepFreeze(value) {
  if (value !== null && typeof value === 'object' && !Object.isFrozen(value)) {
    Object.freeze(value);
    Object.values(value).forEach(deepFreeze);
  }
  return value;
```

```
}

const config = deepFreeze({ tax: { rate: 15 }, regions: ['Dhaka', 'Sylhet'] });

try {
  config.tax.rate = 0;
} catch (error) {
  console.log(error.name);
}
try {
  config.regions.push('Khulna');
} catch (error) {
  console.log(error.name);
}
console.log(config.tax.rate, config.regions.length);
```

Output

```
TypeError
TypeError
15 2
```

Strict Mode-এ Frozen Object বদলানোর চেষ্টা `TypeError` দেয় (Strict Mode ছাড়া নীরবে উপেক্ষিত হয়)। Freeze Production-এ সাধারণত বাদ রাখা হয়, কারণ গভীর গাছ Freeze করতে সময় লাগে; Test ও Development-এ এটি একটি সস্তা সুরক্ষা। Project-এর Test সব ইনপুট Deep Freeze করে চালানো হয়।

কখন Mutation গ্রহণযোগ্য

Immutability একটি নীতি, ধর্ম নয়। মূল প্রশ্ন হলো **মালিকানা**: যে Object শুধু একটি Function-এর ভেতরে তৈরি হয়েছে এবং এখনো বাইরে যায়নি, সেটিকে বদলানো নিরাপদ, কারণ আর কেউ তাকে দেখছে না। অধ্যায় ১৪-এর `partitionValid` এবং এই অধ্যায়ের `reduce`-এর `Map` এর উদাহরণ। কিন্তু যে Object Argument হিসেবে এসেছে, অথবা ফেরত দেওয়ার পর অন্য কেউ রেখে দিয়েছে, তা কখনো বদলাবেন না।

একটি দ্বিতীয় ব্যতিক্রম পারফরম্যান্স: লক্ষাধিক সারির একটি Loop-এ প্রতিটি ধাপে নতুন Object তৈরি Garbage Collector-এর ওপর চাপ ফেলে। তখন একটি স্থানীয় Array-তে `push` করে শেষে একবার ফেরত দেওয়াই যুক্তিসঙ্গত। কিন্তু এই সিদ্ধান্ত নেওয়ার আগে মাপুন (অধ্যায় ১৯); সাধারণ আকারের তথ্যে পার্থক্য চোখে পড়ে না, অথচ Mutation-এর Bug খুঁজতে দিন লেগে যায়।

তথ্যের আকার: Nested বনাম Normalized

একই তথ্য দুইভাবে রাখা যায়। **Nested** আকারে প্রতিটি Order-এর ভেতরে গ্রাহকের পুরো তথ্য; **Normalized** আকারে প্রতিটি ধরনের সত্তা (Entity) আলাদা, ID ধরে, এবং সম্পর্ক শুধু ID দিয়ে—Database-এর সারণির মতো।

normalized.js · JavaScript চলবে: Node.js

```
const nested = [
```

```
  { id: 'O1', customer: { id: 'C1', name: 'Sara', city: 'Bogura' }, total: 500 },
  { id: 'O2', customer: { id: 'C1', name: 'Sara', city: 'Bogura' }, total: 300 },
];

function normalize(orders) {
  const customers = {};
  const byId = {};
  for (const { customer, ...order } of orders) {
    customers[customer.id] = customer;
    byId[order.id] = { ...order, customerId: customer.id };
  }
  return { orders: byId, customers };
}

const state = normalize(nested);
const moved = {
  ...state,
  customers: { ...state.customers, C1: { ...state.customers.C1, city: 'Dhaka' } },
};

const cityOf = (s, orderId) => s.customers[s.orders[orderId].customerId].city;
const count = Object.keys(state.customers).length;
console.log(count, cityOf(moved, 'O1'), cityOf(moved, 'O2'));
console.log(moved.orders === state.orders);
```

Output

```
1 Dhaka Dhaka
true
```

Nested আকারে Sara-র শহর দুটি জায়গায়; ঠিকানা বদলালে দুটিই বদলাতে হতো, একটি ভুলে গেলে তথ্য পরস্পরবিরোধী। Normalized আকারে তথ্য একটি জায়গায়: একটি হালনাগাদে দুটি Order-ই নতুন শহর দেখায়, এবং Order-এর তালিকা অক্ষত (`===`)। অনেক সত্তা ও পারস্পরিক সম্পর্কের Application State-এ (Redux-এর নির্দেশিকাও তাই বলে) Normalized আকার প্রায় সব সময় ভালো; দেখানোর সময় প্রয়োজনমতো জোড়া দেওয়া হয় (অনুচ্ছেদ ১৮.৬-এর Join)।

১৮.৪ flatMap এবং reduce-এর গভীরে

একটি Order-এর ভেতরে অনেক পণ্য; বিক্রির হিসাবের জন্য দরকার প্রতিটি পণ্যের একটি সমতল তালিকা।

flat-map.js · JavaScript চলবে: Node.js

```
const orders = [
  { id: 'O1', customer: 'C1', items: [
    { sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 },
  ] },
```

```
  { id: 'O2', customer: 'C2', items: [] },
  { id: 'O3', customer: 'C1', items: [{ sku: 'B1', qty: 1 }] },
];

const lines = orders.flatMap((order) =>
  order.items.map((item) =>
    ({ order: order.id, customer: order.customer, ...item })));

console.log(lines.length);
const labels = lines.map((line) => `${line.order}:${line.sku}x${line.qty}`);
console.log(labels.join(' '));

const words = ['আম কলা', 'লিচু'].flatMap((text) => text.split(' '));
console.log(words);
```

Output

```
3
O1:B1x2 O1:B2x1 O3:B1x1
[ 'আম', 'কলা', 'লিচু' ]
```

`flatMap(fn)` মানে `map(fn).flat()`: প্রতিটি উপাদান থেকে একটি Array, তারপর সব জোড়া লাগিয়ে একটি সমতল তালিকা। পণ্যহীন `O2` থেকে ফাঁকা Array, তাই সে নিঃশব্দে বাদ। একটি উপাদান থেকে শূন্য, এক বা একাধিক ফল—এটিই `flatMap`-এর শক্তি।

`reduce` দিয়ে একটি `Map`-এ জমানো, Key যেকোনো ধরনের:

reduce-map.js · JavaScript চলবে: Node.js

```
const lines = [
  { sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 }, { sku: 'B1', qty: 1 },
];

const qtyBySku = lines.reduce(
  (totals, line) => totals.set(line.sku, (totals.get(line.sku) ?? 0) + line.qty),
  new Map(),
);

console.log(qtyBySku);
console.log([...qtyBySku].toSorted((a, b) => b[1] - a[1])[0]);
```

Output

```
Map(2) { 'B1' => 3, 'B2' => 1 }
[ 'B1', 3 ]
```

`Map.set` নিজেই Map ফেরত দেয়, তাই Reducer এক লাইনে। এখানে Reducer-এর ভেতরে একটি নতুন Map বদলানো হচ্ছে, যা `reduce`-এর বাইরে কেউ দেখে না; বাইরে থেকে পুরো হিসাব Pure (অধ্যায় ১৪-এর পারফরম্যান্স-ফাঁদের সমাধানও এটিই)। `[...map]` একটি Map থেকে `[key, value]` জোড়ার Array দেয়।

১৮.৫ Grouping, Aggregation এবং Pivot

`Object.groupBy` (অধ্যায় ১২) Key-কে লেখায় রূপান্তর করে। **`Map.groupBy`** Key-কে যেমন আছে রাখে—সংখ্যা, Object, এমনকি তারিখও।

group-by.js · JavaScript চলবে: Node.js

```
const sales = [
  { region: 'Dhaka', month: '2026-01', amount: 1200 },
  { region: 'Sylhet', month: '2026-01', amount: 800 },
  { region: 'Dhaka', month: '2026-02', amount: 500 },
  { region: 'Dhaka', month: '2026-01', amount: 300 },
];

const band = (sale) => (sale.amount >= 1000 ? 1000 : 0);
const byAmountBand = Map.groupBy(sales, band);
console.log([...byAmountBand.keys()], typeof [...byAmountBand.keys()][0]);

function aggregate(rows) {
  return rows.reduce(
    (stats, row) => ({
      count: stats.count + 1,
      total: stats.total + row.amount,
      max: Math.max(stats.max, row.amount),
    }),
    { count: 0, total: 0, max: -Infinity },
  );
}

const pivot = {};
const byRegion = Object.groupBy(sales, (s) => s.region);
for (const [region, rows] of Object.entries(byRegion)) {
  pivot[region] = Object.fromEntries(
    Object.entries(Object.groupBy(rows, (s) => s.month))
      .map(([month, monthRows]) => [month, aggregate(monthRows).total]),
  );
}
console.log(pivot);
console.log(aggregate(sales));
```

Output

```
[ 1000, 0 ] number
{
  Dhaka: { '2026-01': 1500, '2026-02': 500 },
  Sylhet: { '2026-01': 800 }
}
{ count: 4, total: 2800, max: 1200 }
```

- `Map.groupBy`-এর Key `1000` একটি সংখ্যা রয়ে গেছে; `Object.groupBy` হলে তা `'1000'` লেখা হতো।
- **একবারে একাধিক মান (`aggregate`):** একটি `reduce` দিয়ে সংখ্যা, মোট ও সর্বোচ্চ। Accumulator একটি ছোট নতুন Object; ছোট বলে প্রতিবার নতুন তৈরি করা সস্তা।
- **Pivot:** বহুস্তর Grouping—প্রথমে অঞ্চল, তারপর প্রতিটি অঞ্চলের ভেতরে মাস। এটিই Spreadsheet-এর Pivot Table: সারি অঞ্চল, কলাম মাস। `Object.fromEntries` জোড়ার তালিকা থেকে Object তৈরি করে, `Object.entries`-এর উল্টো।

১৮.৬ Join: তালিকা জোড়া দেওয়া

বিক্রির সারিতে শুধু পণ্যের `sku`; দাম ও নাম আরেকটি তালিকায়। প্রতিটি সারির জন্য `products.find(...)` লিখলে প্রতিটি সারিতে পুরো পণ্য-তালিকা খোঁজা হয়: ১০,০০০ সারি ও ১,০০০ পণ্যে প্রায় এক কোটি তুলনা। আগে একটি **Index** (`Map`) তৈরি করলে প্রতিটি খোঁজা প্রায় তাৎক্ষণিক (অধ্যায় ১৯-এ এর বিশ্লেষণ)।

join.js · JavaScript চলবে: Node.js

```
const products = [
  { sku: 'B1', name: 'Notebook', price: 120 },
  { sku: 'B2', name: 'Pen Set', price: 250 },
];
const lines = [
  { order: 'O1', sku: 'B1', qty: 2 },
  { order: 'O1', sku: 'B9', qty: 1 },
  { order: 'O2', sku: 'B2', qty: 1 },
];

const indexBy = (rows, key) => new Map(rows.map((row) => [row[key], row]));
const productIndex = indexBy(products, 'sku');

const joined = lines.flatMap((line) => {
  const product = productIndex.get(line.sku);
  if (!product) {
    return [];
  }
  return [{ ...line, name: product.name, total: product.price * line.qty }];
});
const missing = lines
```

```
  .filter((line) => !productIndex.has(line.sku))
  .map((line) => line.sku);

console.log(joined.map((row) => `${row.order} ${row.name} ${row.total}`));
console.log(missing);
```

Output

```
[ 'O1 Notebook 240', 'O2 Pen Set 250' ]
[ 'B9' ]
```

এটি Database-এর **Inner Join**: দুই পাশে মেলে এমন সারিগুলোই ফলে। `flatMap`-এ মেলেনি এমন সারির জন্য ফাঁকা Array বাদ দেওয়ার কৌশল লক্ষ করুন। অচেনা `B9` নিঃশব্দে হারিয়ে যায়নি, আলাদা তালিকায় জানানো হয়েছে—তথ্যের মান যাচাইয়ে এটি অপরিহার্য।

১৮.৭ Sorting: Stability, বহু Key এবং বাংলা

JavaScript-এর `sort` ও `toSorted` **Stable** (ES2019 থেকে নিশ্চিত): দুটি উপাদান সমান হলে তাদের মূল ক্রম অক্ষত থাকে। তাই একাধিক Key-তে সাজাতে Comparator জোড়া দেওয়া যায়:

sorting.js · JavaScript — চলবে: Node.js

```
const by = (key, direction = 1) => (a, b) =>
  (a[key] < b[key] ? -direction : a[key] > b[key] ? direction : 0);
const thenBy = (...comparators) => (a, b) => {
  for (const compare of comparators) {
    const result = compare(a, b);
    if (result !== 0) {
      return result;
    }
  }
  return 0;
};

const students = [
  { name: 'Sara', class: 9, marks: 88 },
  { name: 'Rafi', class: 10, marks: 88 },
  { name: 'Mina', class: 9, marks: 92 },
  { name: 'Nila', class: 9, marks: 88 },
];

const order = thenBy(by('class'), by('marks', -1), by('name'));
const ranked = students.toSorted(order);
console.log(ranked.map((s) => `${s.class}-${s.name}`).join(' '));

const collator = new Intl.Collator('bn');
```

```
console.log(['সুমন', 'আরিফ', 'ঋতু', 'কবির'].toSorted(collator.compare));
const natural = new Intl.Collator('en', { numeric: true });
console.log(['item10', 'item2', 'item1'].toSorted(natural.compare));
```

Output

```
9-Mina 9-Nila 9-Sara 10-Rafi
[ 'আরিফ', 'ঋতু', 'কবির', 'সুমন' ]
[ 'item1', 'item2', 'item10' ]
```

- **`thenBy`** Comparator-এর একটি Composition (অধ্যায় ১৪): প্রথমটি সমান বললে পরেরটি। শ্রেণি অনুযায়ী, তারপর নম্বর (বেশি আগে), তারপর নাম।
- **`Intl.Collator`** ভাষার নিয়মে তুলনা করে, এবং বারবার তুলনায় `localeCompare`-এর চেয়ে দ্রুত, কারণ নিয়ম একবারই তৈরি হয়। বাংলা স্বরবর্ণ ও ব্যঞ্জনবর্ণ অভিধানের ক্রমে এসেছে।
- **`numeric: true`**: লেখার ভেতরের সংখ্যাকে সংখ্যা হিসেবে তুলনা— `item2` আগে `item10`-এর।

১৮.৮ Result Type: ভুলসহ তথ্যের প্রবাহ

অধ্যায় ১৪-এর `partitionValid` বৈধ ও অবৈধ সারি আলাদা করেছিল। একটি সাধারণ রূপ হলো প্রতিটি ধাপ একটি **Result** ফেরত দেবে— `{ ok: true, value }` বা `{ ok: false, error }` —এবং Pipeline-এর ধাপগুলো Result-এর ওপর কাজ করবে।

result.js · JavaScript চলবে: Node.js

```
const ok = (value) => ({ ok: true, value });
const fail = (error) => ({ ok: false, error });
const andThen = (fn) => (result) => (result.ok ? fn(result.value) : result);

const parseQty = (text) =>
  (/^\d+$/.test(text) ? ok(Number(text)) : fail(`পরিমাণ সংখ্যা নয়: "${text}"`));
const positive = (n) => (n > 0 ? ok(n) : fail('পরিমাণ শূন্য'));
const withinStock = (n) => (n <= 50 ? ok(n) : fail(`স্টকে ${n}টি নেই`));

const checkQty = (text) => [andThen(positive), andThen(withinStock)]
  .reduce((result, step) => step(result), parseQty(text));

for (const input of ['12', 'abc', '0', '80']) {
  const result = checkQty(input);
  console.log(input, result.ok ? result.value : result.error);
}
```

Output

```
12 12
abc পরিমাণ সংখ্যা নয়: "abc"
0 পরিমাণ শূন্য
৪0 স্টকে ৪0টি নেই
```

`andThen` একটি সফল Result-এর মান পরের ধাপে পাঠায়, আর ব্যর্থ Result-কে অপরিবর্তিত পেরিয়ে যেতে দেয়। ফলে প্রথম ব্যর্থতার পরের ধাপগুলো নিজে থেকেই বাদ পড়ে, `try` / `catch` বা `if` ছাড়া। Exception-এর সঙ্গে পার্থক্য (অধ্যায় ১১-এর সমাধান ১২): ব্যর্থতা এখানে একটি সাধারণ মান, তাই হাজারো সারির তালিকায় প্রতিটির ফল জমিয়ে রাখা, গোনা বা Report করা সহজ।

১৮.৯ Asynchronous Pipeline

তথ্য যখন File, Database বা Network থেকে আসে, Pipeline-এর ধাপ Asynchronous। তিনটি ধরন:

async-pipeline.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
let active = 0;
let peak = 0;

async function loadFile(name) {
  active += 1;
  peak = Math.max(peak, active);
  await wait(30);
  active -= 1;
  return `${name}: ১০০ সারি`;
}

const files = ['jan', 'feb', 'mar', 'apr', 'may'];

const sequential = [];
for (const file of files) {
  sequential.push(await loadFile(file));
}
console.log(`ক্রমে: সর্বোচ্চ একসঙ্গে ${peak}`);

peak = 0;
const all = await Promise.all(files.map(loadFile));
console.log(`একসঙ্গে: সর্বোচ্চ একসঙ্গে ${peak}, ফল ${all.length}`);

peak = 0;
const batched = [];
for (let i = 0; i < files.length; i += 2) {
  const batch = files.slice(i, i + 2);
```

```
  batched.push(...(await Promise.all(batch.map(loadFile))));
}
console.log(`দলে দলে: সর্বোচ্চ একসঙ্গে ${peak}, ফল ${batched.length}`);
console.log(batched[4]);
```

Output

```
ক্রমে: সর্বোচ্চ একসঙ্গে 1
একসঙ্গে: সর্বোচ্চ একসঙ্গে 5, ফল 5
দলে দলে: সর্বোচ্চ একসঙ্গে 2, ফল 5
may: ১০০ সারি
```

ধরন	সুবিধা	ঝুঁকি
ক্রমে (`for...of` + `await`)	সরল; প্রতিটি ধাপ আগেরটির ফল ব্যবহার করতে পারে	ধীর
একসঙ্গে (`Promise.all`)	দ্রুততম	অনেক কাজে Server বা Memory-র ওপর চাপ
দলে দলে (Batch)	গতি ও চাপের ভারসাম্য	একটি ধীর কাজ পুরো দলকে আটকায়

অধ্যায় ১০-এর `runLimited` (Worker Pool) দলে দলের চেয়েও দক্ষ: একটি কাজ শেষ হলেই পরেরটি শুরু হয়। Asynchronous Pipeline-এ একটি সীমাবদ্ধতা মনে রাখুন: `map`, `filter`, `reduce` Promise-এর জন্য অপেক্ষা করে না (অধ্যায় ১০-এর `forEach` ফাঁদ)। `filter(async ...)` সব উপাদান রেখে দেয়, কারণ প্রতিটি Promise Truthy।

১৮.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
এক স্তরের Spread দিয়ে Nested হালনাগাদ	মূল তথ্যের ভেতরের Object বদলে যায়	পথের প্রতিটি স্তরে নতুন Object, বা `updateIn`
সব কিছু `structuredClone` করে হালনাগাদ	ধীর; `===` দিয়ে পরিবর্তন বোঝা যায় না	Structural Sharing
`map` দিয়ে Nested Array, তারপর ভুলে যাওয়া `flat`	Array-র ভেতরে Array	`flatMap`
প্রতিটি সারিতে `find` দিয়ে Join	বড় তথ্যে অত্যন্ত ধীর	আগে `Map` Index
Join-এ না মেলা সারি নিঃশব্দে বাদ	তথ্য হারায়	আলাদা তালিকায় জানান
`Object.groupBy` -এর Key সংখ্যা ভাবা	Key লেখা হয়ে যায়	`Map.groupBy`
বাংলা নাম `<` দিয়ে সাজানো	অভিধানের ক্রম নয়	`Intl.Collator('bn')`

ভুল	কী দেখা যায়	সমাধান
Comparator সমান ক্ষেত্রে `0` দেয় না	অনিশ্চিত ক্রম	তিন ক্ষেত্রই সামলান
`filter(async ...)`	কিছুই বাদ পড়ে না	আগে `Promise.all`, তারপর `filter`
সীমাহীন `Promise.all`	Server-এ অনুরোধের বন্যা	Batch বা Worker Pool

১৮.১১ Debugging নির্দেশিকা

- **অপ্রত্যাশিত Mutation ধরতে** Test-এ ইনপুট Deep Freeze করুন; বদলানোর চেষ্টার লাইনেই `TypeError`।
- **কোন অংশ বদলেছে?** পুরোনো ও নতুন অবস্থার প্রতিটি শাখা `===` দিয়ে তুলনা করুন; অপ্রত্যাশিত `false` মানে অকারণে নতুন Object।
- **Pipeline-এর মাঝে দেখুন** অধ্যায় ১৪-এর `tap` দিয়ে, বিশেষত Join ও Grouping-এর পর সারির সংখ্যা।
- **সংখ্যা মেলে না?** মোট হিসাব দুইভাবে করুন (সব সারির যোগফল এবং দলগুলোর যোগফলের যোগফল); দুটি না মিললে কোনো সারি হারিয়েছে বা দুবার গোনা হয়েছে।
- `console.table(rows)` সারির তালিকা সারণি আকারে দেখায়।
- **Asynchronous ক্রম অপ্রত্যাশিত?** প্রতিটি ধাপের শুরু ও শেষ Log করুন, একসঙ্গে কতগুলো চলছে গুনুন (উদাহরণের `peak`-এর মতো)।

১৮.১২ পেশাদার অভ্যাস

- **তথ্য কখনো জায়গায় বদলাবেন না;** নতুন অবস্থা, Structural Sharing-সহ।
- **Test-এ ইনপুট Deep Freeze।**
- **Normalize আগে, তারপর Join ও Aggregation,** যাতে প্রতিটি ধাপ পরিষ্কার তথ্য পায়।
- **Join-এর আগে Index;** না মেলা সারি জানান।
- **প্রতিটি ধাপ ছোট, নামযুক্ত, আলাদাভাবে পরীক্ষাযোগ্য।**
- **মোট হিসাবের Invariant যাচাই** (অধ্যায় ১৬): দলের যোগফল = সব সারির যোগফল।
- **সাজানোয় সব সময় শেষ Key হিসেবে একটি অনন্য ID,** যাতে ক্রম নিশ্চিত।
- **Asynchronous কাজে সীমিত Concurrency।**

১৮.১৩ Project: LedgerFlow

একটি অনলাইন বইয়ের দোকানের Order Ledger থেকে Report: কাঁচা Order (প্রতিটিতে একাধিক পণ্য), পণ্যের তালিকা এবং গ্রাহকের তালিকা যাচাই ও জোড়া দিয়ে অঞ্চল ও মাসভিত্তিক Pivot, শীর্ষ গ্রাহক, এবং ফেরত (Refund) সংশোধন—সবই Immutable উপায়ে। তথ্য একাধিক মাসিক "File" থেকে Asynchronous ভাবে আসে, সীমিত দলে।

অধ্যায় ১৪-এর Sales Function Lab একটি সমতল তালিকার হিসাব ছিল। LedgerFlow-এর তথ্য Nested ও একাধিক উৎসের, এবং তাতে সংশোধন হয়।

Project-এর চাহিদা

- কাঁচা Order যাচাই (Result Type): ID, গ্রাহক, তারিখ (`YYYY-MM-DD`), কমপক্ষে একটি পণ্য, প্রতিটি পণ্যের ধনাত্মক পূর্ণসংখ্যা পরিমাণ।
- Order-কে পণ্য-সারিতে সমতল করা (`flatMap`); পণ্য ও গ্রাহকের সঙ্গে Index-ভিত্তিক Join; অচেনা পণ্য বা গ্রাহক জানানো।
- অঞ্চল × মাস Pivot; গ্রাহকভিত্তিক মোট; শীর্ষ গ্রাহক (সমান হলে নাম, বাংলা নিয়মে)।
- Refund: একটি Order-এর একটি পণ্যের ফেরত পরিমাণ চিহ্নিত করা, Immutable, Structural Sharing-সহ।
- মাসিক File Asynchronous Loader থেকে, সর্বোচ্চ ২টি একসঙ্গে; একটি File ব্যর্থ হলে বাকিগুলোর ফলসহ ব্যর্থতার তালিকা।
- Invariant: Pivot-এর সব ঘরের যোগফল = সব সারির মোট।

File-এর কাঠামো

Folder-এর কাঠামো

```
ledger-flow/
├── src/
│   ├── immutable.mjs     updateIn, setIn, deepFreeze
│   ├── validate.mjs      Result Type এবং Order যাচাই
│   ├── transform.mjs     flatten, Index, Join, Pivot, শীর্ষ গ্রাহক, Refund
│   └── load.mjs          Asynchronous Batch Loader
├── data/
│   └── sample.mjs        নমুনা তথ্য ও কৃত্রিম Loader
├── report.mjs
├── README.md
└── tests/
    ├── immutable.test.mjs
    ├── transform.test.mjs
    └── load.test.mjs
```

src/immutable.mjs ও src/validate.mjs

immutable.mjs · JavaScript চলবে: Browser Console ও Node.js

```
export function updateIn(target, [key, ...rest], change) {
  const current = target?.[key];
  const next = rest.length === 0
    ? change(current)
    : updateIn(current, rest, change);
  if (Object.is(next, current)) {
    return target;
  }
  if (Array.isArray(target)) {
    return target.with(key, next);
  }
  return { ...target, [key]: next };
}

export const setIn = (target, path, value) => updateIn(target, path, () => value);

export function deepFreeze(value) {
  if (value !== null && typeof value === 'object' && !Object.isFrozen(value)) {
    Object.freeze(value);
    Object.values(value).forEach(deepFreeze);
  }
  return value;
}
```

অধ্যায়ের সংস্করণের সঙ্গে একটি পার্থক্য: নতুন মান পুরোনোটির সমান হলে (`Object.is`) কোনো নতুন Object তৈরি হয় না, পুরোনোটিই ফেরত। ফলে "কিছু না বদলানো" হালনাগাদ `===` দিয়ে শনাক্ত করা যায়।

validate.mjs · JavaScript চলবে: Browser Console ও Node.js

```
export const ok = (value) => ({ ok: true, value });
export const fail = (error) => ({ ok: false, error });

const DATE = /^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$/;
const isText = (value) => typeof value === 'string' && value.trim() !== '';

export function validateOrder(raw) {
  if (!isText(raw?.id)) {
    return fail('Order ID নেই');
  }
  if (!isText(raw.customer)) {
    return fail(`${raw.id}: গ্রাহক নেই`);
  }
  if (!DATE.test(raw.date ?? '')) {
    return fail(`${raw.id}: তারিখ YYYY-MM-DD নয়`);
  }
```

```
  if (!Array.isArray(raw.items) || raw.items.length === 0) {
    return fail(`${raw.id}: কোনো পণ্য নেই`);
  }
  const badItem = raw.items.find((item) =>
    !isText(item?.sku) || !Number.isInteger(item.qty) || item.qty < 1);
  if (badItem) {
    return fail(`${raw.id}: পণ্যের তথ্য ভুল`);
  }
  const items = raw.items.map(({ sku, qty }) =>
    ({ sku: sku.trim(), qty, refunded: 0 }));
  const id = raw.id.trim();
  return ok({ id, customer: raw.customer.trim(), date: raw.date, items });
}
```

src/transform.mjs

দুই অংশ মিলে সম্পূর্ণ File।

transform.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { updateIn } from './immutable.mjs';

const banglaCollator = new Intl.Collator('bn');

export const indexBy = (rows, key) => new Map(rows.map((row) => [row[key], row]));

// Order থেকে পণ্য-সারি; ফেরত বাদে বিক্রি পরিমাণ।
export const flattenOrders = (orders) =>
  orders.flatMap((order) => order.items.map((item) => ({
    order: order.id,
    customer: order.customer,
    month: order.date.slice(0, 7),
    sku: item.sku,
    qty: item.qty - item.refunded,
  })));

// পণ্য ও গ্রাহকের সঙ্গে Join; না মেলা সারি আলাদা।
export function joinLines(lines, products, customers) {
  const productIndex = indexBy(products, 'sku');
  const customerIndex = indexBy(customers, 'id');
  const joined = [];
  const unmatched = [];

  for (const line of lines) {
    const product = productIndex.get(line.sku);
    const customer = customerIndex.get(line.customer);
    if (!product || !customer) {
      unmatched.push(`${line.order}:${product ? line.customer : line.sku}`);
      continue;
```

```
    }
    joined.push({
      ...line,
      region: customer.region,
      customerName: customer.name,
      amount: product.price * line.qty,
    });
  }
  return { joined, unmatched };
}

// ...
```

transform.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

const sum = (rows) => rows.reduce((total, row) => total + row.amount, 0);

// অঞ্চল × মাস; ঘরগুলো মাসের ক্রমে।
export function pivotRegionMonth(rows) {
  const pivot = {};
  for (const [region, regionRows] of Map.groupBy(rows, (row) => row.region)) {
    const months = Map.groupBy(regionRows, (row) => row.month);
    pivot[region] = Object.fromEntries(
      [...months].toSorted(([a], [b]) => a.localeCompare(b))
        .map(([month, monthRows]) => [month, sum(monthRows)]),
    );
  }
  return pivot;
}

export function topCustomers(rows, count = 3) {
  return [...Map.groupBy(rows, (row) => row.customerName)]
    .map(([name, customerRows]) => ({ name, total: sum(customerRows) }))
    .toSorted((a, b) =>
      b.total - a.total || banglaCollator.compare(a.name, b.name))
    .slice(0, count);
}

// একটি Order-এর একটি পণ্যের ফেরত পরিমাণ যোগ; বাকি সব অক্ষত।
export function applyRefund(orders, { orderId, sku, qty }) {
  const orderIndex = orders.findIndex((order) => order.id === orderId);
  const items = orders[orderIndex]?.items ?? [];
  const itemIndex = items.findIndex((item) => item.sku === sku);
  if (orderIndex === -1 || itemIndex === -1) {
    throw new RangeError(`${orderId}/${sku} পাওয়া যায়নি`);
  }
  const { qty: sold, refunded } = orders[orderIndex].items[itemIndex];
```

```
  if (!Number.isInteger(qty) || qty < 1 || refunded + qty > sold) {
    throw new RangeError(
      `ফেরত ${qty}টি সম্ভব নয়; বিক্রি ${sold}, আগে ফেরত ${refunded}`);
  }
  const path = [orderIndex, 'items', itemIndex, 'refunded'];
  return updateIn(orders, path, (n) => n + qty);
}

export const totalOf = sum;
```

- **`flattenOrders`** Nested Order থেকে সমতল পণ্য-সারি; একই সঙ্গে প্রতিটি সারিতে মাস (`date`-এর প্রথম সাত অক্ষর) এবং ফেরত বাদে নিট পরিমাণ।
- **`joinLines`** দুটি Index একবার তৈরি করে, তারপর প্রতিটি সারিতে দুটি `get`। অচেনা পণ্য বা গ্রাহক `unmatched`-এ, কারণসহ।
- **`pivotRegionMonth`** `Map.groupBy` দিয়ে দুই স্তরে ভাগ; মাসগুলো ক্রমে সাজানো, যাতে Report-এর কলাম নিশ্চিত ক্রমে আসে।
- **`topCustomers`**: মোট বেশি আগে; সমান হলে নাম বাংলা নিয়মে—নিশ্চিত ক্রম।
- **`applyRefund`** প্রথমে যাচাই (অচেনা Order বা পণ্য, অতিরিক্ত ফেরত), তারপর `updateIn`-এর একটি Path। Structural Sharing নিজে থেকে: শুধু সেই Order, তার `items` এবং সেই পণ্যটি নতুন।

src/load.mjs এবং নমুনা তথ্য

load.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// নাম ধরে File আনে, সর্বোচ্চ batchSize একসঙ্গে; ব্যর্থ File বাকিদের থামায় না।
export async function loadInBatches(names, loader, batchSize = 2) {
  const records = [];
  const failures = [];

  for (let i = 0; i < names.length; i += batchSize) {
    const batch = names.slice(i, i + batchSize);
    const results = await Promise.allSettled(batch.map((name) => loader(name)));
    results.forEach((result, j) => {
      if (result.status === 'fulfilled') {
        records.push(...result.value);
      } else {
        failures.push(`${batch[j]}: ${result.reason.message}`);
      }
    });
  }
  return { records, failures };
}
```

নমুনা তথ্যের File দুই অংশে; দুটি মিলে সম্পূর্ণ File।

sample.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Browser Console ও Node.js

```
export const PRODUCTS = [
  { sku: 'B1', name: 'Notebook', price: 120 },
  { sku: 'B2', name: 'Pen Set', price: 250 },
  { sku: 'B3', name: 'Atlas', price: 900 },
];

export const CUSTOMERS = [
  { id: 'C1', name: 'সারা', region: 'Dhaka' },
  { id: 'C2', name: 'রফিক', region: 'Bogura' },
  { id: 'C3', name: 'মিনা', region: 'Dhaka' },
];

// ...
```

sample.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

const FILES = {
  '2026-01': [
    {
      id: 'O1', customer: 'C1', date: '2026-01-05',
      items: [{ sku: 'B1', qty: 2 }, { sku: 'B3', qty: 1 }],
    },
    {
      id: 'O2', customer: 'C2', date: '2026-01-09',
      items: [{ sku: 'B2', qty: 3 }],
    },
  ],
  '2026-02': [
    {
      id: 'O3', customer: 'C3', date: '2026-02-11',
      items: [{ sku: 'B3', qty: 1 }, { sku: 'B9', qty: 1 }],
    },
    {
      id: 'O4', customer: 'C1', date: '2026-02-30x',
      items: [{ sku: 'B1', qty: 1 }],
    },
  ],
  '2026-03': [
    {
      id: 'O5', customer: 'C2', date: '2026-03-02',
      items: [{ sku: 'B1', qty: 5 }],
    },
  ],
};
```

```
// কৃত্রিম Asynchronous Loader: অজানা মাসে ব্যর্থ।
export async function sampleLoader(name) {
  await new Promise((resolve) => setTimeout(resolve, 10));
  if (!Object.hasOwn(FILES, name)) {
    throw new Error('File নেই');
  }
  return structuredClone(FILES[name]);
}
```

report.mjs

report.mjs · JavaScript চলবে: Node.js

```
import { CUSTOMERS, PRODUCTS, sampleLoader } from './data/sample.mjs';
import { loadInBatches } from './src/load.mjs';
import { validateOrder } from './src/validate.mjs';
import {
  applyRefund, flattenOrders, joinLines, pivotRegionMonth, topCustomers, totalOf,
} from './src/transform.mjs';

const months = ['2026-01', '2026-02', '2026-03', '2026-04'];
const { records, failures } = await loadInBatches(months, sampleLoader);

const results = records.map(validateOrder);
const orders = results.filter((r) => r.ok).map((r) => r.value);
const invalid = results.filter((r) => !r.ok).map((r) => r.error);

const corrected = applyRefund(orders, { orderId: 'O5', sku: 'B1', qty: 2 });
const lines = flattenOrders(corrected);
const { joined, unmatched } = joinLines(lines, PRODUCTS, CUSTOMERS);
const pivot = pivotRegionMonth(joined);

console.log(`Order: ${records.length}, বৈধ ${orders.length};`,
  `বাদ: ${invalid.join('; ')}`);
console.log(`File ব্যর্থ: ${failures.join('; ')}; অমিল: ${unmatched.join(', ')}`);
console.log(pivot);
console.log(topCustomers(joined).map((c) => `${c.name} ${c.total}`).join(', '));

const pivotTotal = Object.values(pivot)
  .flatMap((months) => Object.values(months))
  .reduce((a, b) => a + b, 0);
console.log(`Invariant: ${pivotTotal === totalOf(joined)} (${pivotTotal})`);
console.log(corrected[0] === orders[0], corrected.at(-1) === orders.at(-1));
```

Output

```
Order: 5, বৈধ 4; বাদ: O4: তারিখ YYYY-MM-DD নয়
File ব্যর্থ: 2026-04: File নেই; অমিল: O3:B9
{
  Dhaka: { '2026-01': 1140, '2026-02': 900 },
  Bogura: { '2026-01': 750, '2026-03': 360 }
}
সারা 1140, রফিক 1110, মিনা 900
Invariant: true (3150)
true false
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Load:** চারটি মাস, দুটি করে একসঙ্গে। এপ্রিলের File নেই; `allSettled` থাকায় সেটি ব্যর্থতার তালিকায়, বাকি তিনটির পাঁচটি Order হাতে।
- **Validate:** `O4`-এর তারিখ ভুল (`2026-02-30x`); Result Type তাকে কারণসহ আলাদা করে, Pipeline থামে না।
- **Refund:** `O5`-এর ৫টি Notebook থেকে ২টি ফেরত। শেষ লাইন দেখায়: প্রথম Order (`O1`) পুরোনো ও নতুন তালিকায় **একই** Object (Structural Sharing), শুধু `O5` নতুন।
- **Flatten ও Join:** `O3`-এর `B9` পণ্যটি তালিকায় নেই; বাকি সারি থাকে, অমিলটি জানানো হয়।
- **Pivot:** ঢাকার জানুয়ারি = সারার Order: ২×১২০ + ৯০০ = ১১৪০; বগুড়ার মার্চ = ফেরতের পর ৩টি Notebook = ৩৬০।
- **শীর্ষ গ্রাহক** ও **Invariant:** Pivot-এর সব ঘরের যোগফল সব সারির মোটের সমান (৩১৫০)—কোনো সারি হারায়নি বা দুবার গোনা হয়নি।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node report.mjs
node --test
```

Companion Source Code-এ ২২টি Test, প্রতিটি ইনপুট Deep Freeze করে চালানো: Structural Sharing (`===` যাচাই), `updateIn`-এর Array ও Object পথ, যাচাইয়ের প্রতিটি ব্যর্থতা, Join-এর অমিল, Pivot-এর Invariant, শীর্ষ গ্রাহকের নিশ্চিত ক্রম, অতিরিক্ত ফেরতের প্রত্যাখ্যান, এবং Batch Loader-এর সর্বোচ্চ Concurrency ও আংশিক ব্যর্থতা (Fake Timer ছাড়াই, নিয়ন্ত্রিত Loader দিয়ে)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Refund-এর পর মূল তালিকাও বদলেছে	Nested Spread-এ একটি স্তর বাদ	`updateIn`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pivot-এর যোগফল মোটের চেয়ে কম	Join-এ সারি নিঃশব্দে বাদ	`unmatched` দেখুন
শীর্ষ গ্রাহকের ক্রম মাঝে মাঝে বদলায়	সমান মোটে দ্বিতীয় Key নেই	নাম ধরে Collator
একটি File ব্যর্থ হলে সব থামে	`Promise.all`	`Promise.allSettled`
Test-এ `TypeError: Cannot assign`	কোনো Function ইনপুট বদলাচ্ছে	সেই Function Immutable করুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Memory-তে; লক্ষাধিক সারির জন্য Stream বা Database লাগবে। মাস ধরা হয়েছে তারিখের লেখা থেকে, সময়-অঞ্চল বিবেচনা ছাড়া। টাকা পূর্ণসংখ্যায়।

১৮.১৪ অনুশীলনী

প্রাথমিক স্তর

১. নিচের কোনটি Referentially Transparent? `(x) => x * 2`, `() => Date.now()`, `(list) => list.length`। কারণ বলুন।

২. `{ user: { profile: { city: 'Bogura' } } }`-এ শহর `Dhaka` করুন Spread দিয়ে, মূলটি অক্ষত রেখে।

৩. `flatMap` দিয়ে `['a,b', 'c']` থেকে `['a', 'b', 'c']` তৈরি করুন।

৪. `Map.groupBy` দিয়ে সংখ্যাগুলোকে জোড় ও বিজোড়ে ভাগ করুন, Key `true` / `false` (Boolean)।

৫. বাংলা নামের একটি তালিকা `Intl.Collator('bn')` দিয়ে সাজান।

মধ্যম স্তর

৬. `updateIn` দিয়ে একটি Nested Array-র (`matrix[1][2]`) একটি মান দ্বিগুণ করুন, এবং দেখান অন্য সারিগুলো একই Object।

৭. দুটি তালিকা (শিক্ষার্থী ও ফলাফল) ID ধরে Left Join করুন: ফল না থাকলে `marks: null`।

৮. একটি `reduce`-এ বিক্রির তালিকা থেকে সংখ্যা, মোট, গড় ও সর্বনিম্ন বের করুন।

৯. Result Type-এ একটি `map` সাহায্যকারী লিখুন, যা সফল Result-এর মান রূপান্তর করে, ব্যর্থতা অপরিবর্তিত রাখে।

চ্যালেঞ্জ স্তর

১০. `diffShallow(before, after)` লিখুন, যা দুটি Object-এর কোন Key-গুলোর মান `===` অনুযায়ী বদলেছে তা দেয়; Structural Sharing-এর সুবিধা দেখান।

১১. একটি `asyncFilter(items, predicate)` লিখুন, যা Asynchronous শর্তে সঠিকভাবে ছাঁকে।

১২. LedgerFlow-এ পণ্যভিত্তিক মোট বিক্রি (নাম ও পরিমাণ) এবং সবচেয়ে বেশি বিক্রি হওয়া পণ্য যোগ করুন।

১৮.১৫ অনুশীলনীর সমাধান

সমাধান ১ — `(x) => x * 2` হ্যাঁ: যেকোনো `x`-এ ফল দিয়ে প্রতিস্থাপন করা যায়। `() => Date.now()` না: প্রতি ডাকে আলাদা মান। `(list) => list.length` তালিকাটি বদলানো না হলে হ্যাঁ; কিন্তু একই তালিকা পরে বদলালে একই Argument-এ আলাদা ফল। এজন্যই Immutable তথ্যের সঙ্গে Referential Transparency স্বাভাবিকভাবে আসে।

সমাধান ২

nested-city.js · JavaScript — চলবে: Browser Console ও Node.js

```
const state = {
  user: { profile: { city: 'Bogura', zip: '5800' }, name: 'Sara' },
};

const next = {
  ...state,
  user: { ...state.user, profile: { ...state.user.profile, city: 'Dhaka' } },
};

const { city, zip } = next.user.profile;
console.log(state.user.profile.city, city, zip);
```

Output

```
Bogura Dhaka 5800
```

সমাধান ৩

flat-split.js · JavaScript — চলবে: Node.js

```
console.log(['a,b', 'c'].flatMap((text) => text.split(',')));
```

Output

```
[ 'a', 'b', 'c' ]
```

সমাধান ৪

even-odd.js · JavaScript — চলবে: Node.js

```js
const groups = Map.groupBy([1, 2, 3, 4, 5], (n) => n % 2 === 0);

console.log(groups.get(true), groups.get(false), [...groups.keys()]);
```

Output

```
[ 2, 4 ] [ 1, 3, 5 ] [ false, true ]
```

Key-গুলো Boolean রয়ে গেছে; ক্রম প্রথম দেখা অনুযায়ী (`1` বিজোড়, তাই `false` আগে)।

সমাধান ৫

bangla-sort.js · JavaScript — চলবে: Node.js

```js
const names = ['রফিক', 'অনিক', 'তানিয়া', 'ইমরান', 'খাদিজা'];

console.log(names.toSorted(new Intl.Collator('bn').compare));
```

Output

```
[ 'অনিক', 'ইমরান', 'খাদিজা', 'তানিয়া', 'রফিক' ]
```

সমাধান ৬

matrix.mjs · JavaScript — চলবে: Node.js

```js
import { updateIn } from '../../project/ledger-flow/src/immutable.mjs';

const matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
const doubled = updateIn(matrix, [1, 2], (value) => value * 2);

console.log(doubled[1], matrix[1]);
const same = [0, 1, 2].map((row) => doubled[row] === matrix[row]);
console.log(same.join(' '));
```

Output

```
[ 4, 5, 12 ] [ 4, 5, 6 ]
true false true
```

সমাধান ৭

left-join.js · JavaScript চলবে: Node.js

```
const students = [
  { id: 1, name: 'Sara' }, { id: 2, name: 'Rafi' }, { id: 3, name: 'Mina' },
];
const results = [{ studentId: 1, marks: 88 }, { studentId: 3, marks: 72 }];

const marksById = new Map(results.map((r) => [r.studentId, r.marks]));
const report = students.map((s) =>
  ({ ...s, marks: marksById.get(s.id) ?? null }));

console.log(report.map((r) => `${r.name}:${r.marks}`).join(' '));
```

Output

```
Sara:88 Rafi:null Mina:72
```

Left Join বাম তালিকার সব সারি রাখে; ডান পাশে না মিললে খালি মান। Inner Join হলে Rafi বাদ পড়তেন।

সমাধান ৮

stats.js · JavaScript চলবে: Node.js

```
const amounts = [1200, 800, 500, 300];

const stats = amounts.reduce(
  (s, amount) => ({
    count: s.count + 1,
    total: s.total + amount,
    min: Math.min(s.min, amount),
  }),
  { count: 0, total: 0, min: Infinity },
);

console.log({ ...stats, average: stats.count ? stats.total / stats.count : 0 });
```

Output

```
{ count: 4, total: 2800, min: 300, average: 700 }
```

গড় `reduce`-এর পরে, মোট ও সংখ্যা থেকে; Reducer-এর ভেতরে প্রতিবার গড় হিসাব অপ্রয়োজনীয় এবং দশমিকের ভুল জমায়।

সমাধান ৯

result-map.js · JavaScript চলবে: Browser Console ও Node.js

```
const ok = (value) => ({ ok: true, value });
const fail = (error) => ({ ok: false, error });
const mapResult = (fn) => (result) => (result.ok ? ok(fn(result.value)) : result);

const toTaka = mapResult((n) => `${n} টাকা`);

console.log(JSON.stringify(toTaka(ok(250))));
console.log(JSON.stringify(toTaka(fail('দাম নেই'))));
```

Output

```
{"ok":true,"value":"250 টাকা"}
{"ok":false,"error":"দাম নেই"}
```

`map` মানের রূপান্তর (যা ব্যর্থ হতে পারে না), `andThen` এমন ধাপ যা নিজেই ব্যর্থ হতে পারে—দুটি আলাদা সাহায্যকারী।

সমাধান ১০

diff.mjs · JavaScript চলবে: Node.js

```
import { setIn } from '../../project/ledger-flow/src/immutable.mjs';

const diffShallow = (before, after) =>
  [...new Set([...Object.keys(before), ...Object.keys(after)])]
    .filter((key) => before[key] !== after[key]);

const state = { cart: { items: [1, 2] }, user: { name: 'Sara' }, theme: 'light' };
const next = setIn(state, ['cart', 'items'], [1, 2, 3]);

console.log(diffShallow(state, next));
console.log(diffShallow(state, setIn(state, ['theme'], 'light')));
```

Output

```
[ 'cart' ]
[]
```

গভীর তুলনা ছাড়াই, শুধু `===`-এ, বোঝা গেল শুধু `cart` বদলেছে; একই মান বসালে `setIn` পুরোনো Object-ই ফেরত দেয়, তাই কিছুই বদলায়নি। UI Library-গুলো এভাবেই ঠিক করে কোন অংশ নতুন করে আঁকবে।

সমাধান ১১

async-filter.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const isAvailable = async (sku) => {
  await wait(10);
  return sku !== 'B2';
};

async function asyncFilter(items, predicate) {
  const keep = await Promise.all(items.map(predicate));
  return items.filter((_, index) => keep[index]);
}

const wrong = ['B1', 'B2', 'B3'].filter(isAvailable);
console.log(wrong.length);
console.log((await asyncFilter(['B1', 'B2', 'B3'], isAvailable)).join(','));
```

Output

```
3
B1,B3
```

`filter(isAvailable)` প্রতিটি উপাদানের জন্য একটি Promise পায়, যা সব সময় Truthy, তাই কিছুই বাদ পড়েনি। আগে সব শর্তের ফল (`Promise.all`), তারপর সাধারণ `filter`।

সমাধান ১২

by-product.mjs · JavaScript — চলবে: Node.js

```
const P = '../../project/ledger-flow';
const sample = await import(`${P}/data/sample.mjs`);
const { PRODUCTS, CUSTOMERS, sampleLoader } = sample;
const { loadInBatches } = await import(`${P}/src/load.mjs`);
const { validateOrder } = await import(`${P}/src/validate.mjs`);
const { flattenOrders, joinLines } = await import(`${P}/src/transform.mjs`);

const months = ['2026-01', '2026-02', '2026-03'];
const { records } = await loadInBatches(months, sampleLoader);
const orders = records.map(validateOrder).filter((r) => r.ok).map((r) => r.value);
const { joined } = joinLines(flattenOrders(orders), PRODUCTS, CUSTOMERS);
const names = new Map(PRODUCTS.map((p) => [p.sku, p.name]));

const bySku = [...Map.groupBy(joined, (row) => row.sku)]
  .map(([sku, rows]) => ({
    name: names.get(sku),
```

```
    qty: rows.reduce((a, r) => a + r.qty, 0),
  }))
  .toSorted((a, b) => b.qty - a.qty);

console.log(bySku.map((p) => `${p.name} ${p.qty}`).join(', '));
console.log(`সবচেয়ে বেশি: ${bySku[0].name}`);
```

Output

```
Notebook 7, Pen Set 3, Atlas 2
সবচেয়ে বেশি: Notebook
```

বিদ্যমান ধাপগুলো পুনর্ব্যবহার হয়েছে; শুধু শেষে একটি নতুন Grouping। এখানে Refund প্রয়োগ হয়নি, তাই `05` -এর ৫টি Notebook পুরোটাই গোনা।

১৮.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Referential Transparency মানে কী?

ক) Function দ্রুত চলে

খ) Expression-কে তার ফল দিয়ে প্রতিস্থাপন করলে আচরণ বদলায় না

গ) Function-এর নাম নেই

ঘ) Code সবার জন্য দৃশ্যমান

প্রশ্ন ২. Structural Sharing-এ Nested তথ্যের একটি মান বদলালে কী নতুন তৈরি হয়?

ক) পুরো গাছ

খ) কিছুই না, মূলটি বদলায়

গ) শুধু বদলের পথের Object-গুলো

ঘ) শুধু বদলানো মানটি

প্রশ্ন ৩. `[[1], [2, 3]].flatMap((a) => a)` -এর ফল?

ক) `[1, 2, 3]`

খ) `[[1], [2, 3]]`

গ) `[1, [2, 3]]`

ঘ) `6`

প্রশ্ন ৪. Grouping-এর Key সংখ্যা হিসেবে রাখতে কোনটি?

ক) `Object.groupBy`

খ) `JSON.stringify`

গ) `Array.from`

ঘ) `Map.groupBy`

প্রশ্ন ৫. বড় তালিকায় Join দ্রুত করতে কী করবেন?

ক) প্রতিটি সারিতে `find`

খ) আগে একটি `Map` Index

গ) দুটি তালিকা সাজানো

ঘ) `structuredClone`

প্রশ্ন ৬. `toSorted` Stable—এর অর্থ?

ক) সমান উপাদানের মূল ক্রম অক্ষত থাকে

খ) মূল Array বদলায় না

গ) সব সময় একই সময় লাগে

ঘ) শুধু সংখ্যা সাজায়

প্রশ্ন ৭. বাংলা নাম অভিধানের ক্রমে সাজাতে কোনটি উপযুক্ত?

ক) `sort()` Argument ছাড়া

খ) `<` তুলনা

গ) `Intl.Collator('bn').compare`

ঘ) `reverse()`

প্রশ্ন ৮. `andThen` একটি ব্যর্থ Result পেলে কী করে?

ক) পরের Function চালায়

খ) ব্যর্থ Result অপরিবর্তিত ফেরত দেয়

গ) Exception ছোড়ে

ঘ) `null` দেয়

প্রশ্ন ৯. `items.filter(async (x) => ...)` কেন কাজ করে না?

ক) `filter`-এ Function দেওয়া যায় না

খ) `async` নিষিদ্ধ

গ) প্রতিটি Promise Truthy, তাই কিছুই বাদ পড়ে না

ঘ) সব বাদ পড়ে

> **প্রশ্ন ১০.** অনেক File একসঙ্গে কিন্তু সীমিত সংখ্যায় আনতে, এবং একটির ব্যর্থতায় বাকিগুলো না থামাতে কোনটি?
>
> ক) Batch এবং `Promise.allSettled`
>
> খ) সব একসঙ্গে `Promise.all`
>
> গ) `Promise.race`
>
> ঘ) `forEach` ও `await`

১৮.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–খ · ৯–গ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Pure Function-এর ডাক তার ফলের সমতুল্য।
২	গ	বাকি শাখা পুরোনো ও নতুন অবস্থা ভাগ করে নেয়।
৩	ক	`flatMap` এক স্তর সমতল করে।
৪	ঘ	`Map` -এর Key যেকোনো ধরনের; `Object.groupBy` লেখায় রূপান্তর করে।
৫	খ	Index একবার তৈরি, প্রতিটি খোঁজা তাৎক্ষণিক।
৬	ক	তাই বহু-Key সাজানো নির্ভরযোগ্য।
৭	গ	ভাষার নিয়মে তুলনা।
৮	খ	প্রথম ব্যর্থতার পরের ধাপগুলো বাদ পড়ে।
৯	গ	`filter` Promise-এর জন্য অপেক্ষা করে না।
১০	ক	দল চাপ সীমিত রাখে; `allSettled` প্রতিটির ফল দেয়।

১৮.১৮ অধ্যায়ের সারসংক্ষেপ

- Referentially Transparent Code-এ Refactor ও Memoization নিরাপদ।
- Nested তথ্য হালনাগাদ শুধু বদলের পথ ধরে; বাকি শাখা ভাগ করা (Structural Sharing), পরিবর্তন `===` -এ ধরা যায়।
- `updateIn` ও `setIn` Path ধরে হালনাগাদ সহজ করে; Deep Freeze Test-এ Mutation ধরে।
- `flatMap` Nested তথ্য সমতল করে; `reduce` দিয়ে `Map` -এ জমানো।
- `Map.groupBy` Key-র ধরন রাখে; বহুস্তর Grouping-এ Pivot; একটি `reduce` -এ একাধিক Aggregate।
- Join-এর আগে `Map` Index; না মেলা সারি জানান।

- সাজানো Stable; বহু-Key Comparator; বাংলায় `Intl.Collator`।
- Result Type ব্যর্থতাকে মান হিসেবে প্রবাহিত করে।
- Asynchronous Pipeline: ক্রমে, একসঙ্গে, বা দলে দলে; `map` / `filter` Promise-এর জন্য অপেক্ষা করে না।

পরের অধ্যায়ে Performance Optimization ও Memory Management: এই অধ্যায়ের Index, Cache ও Batch-এর সিদ্ধান্তগুলো কীভাবে মেপে নিতে হয়।

১৮.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Referential Transparency	Expression-কে ফল দিয়ে প্রতিস্থাপনযোগ্যতা।
Structural Sharing	নতুন ও পুরোনো অবস্থার অপরিবর্তিত অংশ ভাগাভাগি।
Deep Freeze	সব স্তরে অপরিবর্তনীয় করা।
`flatMap`	Map করে এক স্তর সমতল করা।
Aggregation	একটি দলের সংখ্যা, মোট, সর্বোচ্চ ইত্যাদি।
Pivot	দুই মাত্রার Grouping-এর সারণি।
Index	Key থেকে সারিতে দ্রুত পৌঁছানোর `Map`।
Inner / Left Join	উভয় পাশে মেলা সারি / বাম পাশের সব সারি।
Stable Sort	সমান উপাদানের মূল ক্রম রক্ষাকারী সাজানো।
Collator	ভাষার নিয়মে লেখা তুলনাকারী।
Result Type	সফল মান বা ব্যর্থতা বহনকারী মান।
Batch	একসঙ্গে প্রক্রিয়া করা কাজের দল।

১৮.২০ আরও পড়ুন

- **MDN — Array.prototype.flatMap():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flatMap` — `flatMap`।
- **MDN — Map.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy` — Key-র ধরন রেখে Grouping।
- **MDN — Intl.Collator:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator` — ভাষাভিত্তিক তুলনা।

- **MDN — Array.prototype.sort():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort` — Stability ও Comparator।
- **Immer — Introduction:** `https://immerjs.github.io/immer/` — Immutable হালনাগাদের একটি জনপ্রিয় Library।

অধ্যায় ১৯

JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Performance-এর বিভিন্ন মাত্রা—সময়, সাড়াদানের ক্ষমতা, Memory, Network—আলাদা করতে।
- Big-O দিয়ে একটি Algorithm-এর বৃদ্ধির হার বিশ্লেষণ করতে, এবং কাজের ধাপ গুনে তা যাচাই করতে।
- নির্ভরযোগ্যভাবে মাপতে: Warm-up, একাধিক পুনরাবৃত্তি, Median, এবং পরিবেশের বিবরণ।
- Benchmark কীভাবে ভুল পথে নিয়ে যায় তা চিনতে।
- Main Thread-এর দীর্ঘ কাজ ছোট টুকরোয় ভাগ করে Page সচল রাখতে।
- DOM-এর হালনাগাদ একত্রে করতে এবং Layout Thrashing এড়াতে।
- Garbage Collection-এর মূল নীতি এবং সাধারণ Memory Leak চিনতে, এবং `WeakMap` ব্যবহার করতে।
- সীমিত আকারের LRU Cache তৈরি করতে।
- একটি সম্পূর্ণ **PerfLab** Benchmark ও Optimization Toolkit তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৯: DOM, `replaceChildren`।
- অধ্যায় ১০: Event Loop, Task ও Microtask।
- অধ্যায় ১২: ES Module, Dynamic `import()`।
- অধ্যায় ১৪: Memoization, Debounce ও Throttle।
- অধ্যায় ১৮: `Map` Index ও Join।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি Program সঠিক হলেই যথেষ্ট নয়; তাকে যথেষ্ট দ্রুতও হতে হয়। ব্যবহারকারী একটি Button চাপার পর দুই সেকেন্ড কিছু না ঘটলে ধরে নেন Page নষ্ট; একটি Mobile ফোনে পাঁচ MB-র JavaScript নামতে নামতে তিনি চলে যান; একটি Server-এর Memory ধীরে ধীরে বাড়তে বাড়তে এক সপ্তাহ পর বন্ধ হয়ে যায়।

কিন্তু Performance-এর কাজে সবচেয়ে বড় ঝুঁকি হলো ভুল জায়গায় সময় দেওয়া। Donald Knuth-এর বিখ্যাত সতর্কবাণী: "Premature optimization is the root of all evil"—না মেপে Optimize করা প্রায় সব সময় Code-কে জটিল করে, অথচ দ্রুত করে না। এই অধ্যায়ের মূল শিক্ষা তাই একটি পদ্ধতি: **আগে মাপুন, তারপর সবচেয়ে বড় সমস্যাটি ঠিক করুন, তারপর আবার মাপুন।**

১৯.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Memoization (অধ্যায় ১৪)	LRU Cache: সীমিত আকার, Eviction, মেয়াদ
Debounce, Throttle (অধ্যায় ১০, ১৪)	দীর্ঘ কাজ টুকরোয় ভাগ, `requestAnimationFrame`
`Map` Index দিয়ে Join (অধ্যায় ১৮)	Big-O বিশ্লেষণ, ধাপ গোনা
ReDoS-এর সময় মাপা (অধ্যায় ১৭)	নির্ভরযোগ্য Benchmark পদ্ধতি
Dynamic `import()` (অধ্যায় ১২)	Lazy Loading, Bundle-এর আকার
—	Garbage Collection, Memory Leak, `WeakMap`

১৯.২ Performance-এর মাত্রা

মাত্রা	প্রশ্ন	মাপার উপায়
Latency	একটি কাজ কত সময় নেয়?	`performance.now()`, DevTools Performance Panel
Responsiveness	ব্যবহারকারীর কাজে Page কত দ্রুত সাড়া দেয়?	দীর্ঘ কাজ (৫০ মিলিসেকেন্ডের বেশি), INP
Throughput	প্রতি সেকেন্ডে কত কাজ?	Benchmark-এ ops/sec
Memory	কত Memory, এবং তা কি বাড়তেই থাকে?	Heap Snapshot, `process.memoryUsage()`
Network	কত Byte, কতগুলো অনুরোধ?	Network Panel, Lighthouse

Browser-এ Main Thread একটি কাজে ৫০ মিলিসেকেন্ডের বেশি আটকে থাকলে তাকে **Long Task** বলা হয়, কারণ তখন Click বা লেখা চোখে পড়ার মতো দেরিতে সাড়া পায়। Google-এর **Core Web Vitals** এই অভিজ্ঞতাগুলো মাপে: LCP (মূল বিষয়বস্তু কত দ্রুত দেখা যায়), INP (ব্যবহারকারীর কাজে কত দ্রুত সাড়া), CLS (Layout কতটা অপ্রত্যাশিতভাবে সরে)।

১৯.৩ Big-O: বৃদ্ধির হার

Big-O বলে ইনপুট বাড়লে কাজ **কী হারে** বাড়ে, সেকেন্ডে কত সময় নয়। মাপার সবচেয়ে নিশ্চিত উপায়: কাজের ধাপ গোনা, যা যন্ত্রভেদে বদলায় না।

count-steps.js · JavaScript — চলবে: Browser Console ও Node.js

```
function commonNested(a, b) {
  let steps = 0;
  const result = a.filter((x) => b.some((y) => {
    steps += 1;
    return x === y;
  }));
  return { result: result.length, steps };
}

function commonWithSet(a, b) {
  let steps = b.length;
  const lookup = new Set(b);
  const result = a.filter((x) => {
    steps += 1;
    return lookup.has(x);
  });
  return { result: result.length, steps };
}
```

```
for (const n of [100, 1000, 10000]) {
  const a = Array.from({ length: n }, (_, i) => i * 2);
  const b = Array.from({ length: n }, (_, i) => i * 3);
  const nested = commonNested(a, b);
  const set = commonWithSet(a, b);
  console.log(n, nested.result === set.result, nested.steps, set.steps);
}
```

Output

```
100 true 7756 200
1000 true 777556 2000
10000 true 77775556 20000
```

ইনপুট ১০ গুণ হলে Nested পদ্ধতির ধাপ প্রায় ১০০ গুণ (**$O(n^2)$**), Set পদ্ধতির ঠিক ১০ গুণ (**$O(n)$**)। ফল দুটোরই এক (দ্বিতীয় কলাম)। ১০,০০০ উপাদানে পার্থক্য প্রায় চার হাজার গুণ—এমন পার্থক্য কোনো ছোটখাটো Optimization দিয়ে পূরণ হয় না। Algorithm-এর পছন্দই Performance-এর সবচেয়ে বড় লিভার।

Big-O	নাম	উদাহরণ	n = ১০,০০০-এ মোটামুটি ধাপ
O(1)	ধ্রুবক	`Map.get`, `Set.has`, Array-র Index	১
O(log n)	লগারিদমিক	সাজানো তালিকায় Binary Search	১৪
O(n)	রৈখিক	`includes`, `find`, একটি Loop	১০,০০০
O(n log n)		`sort`	১,৩০,০০০
$O(n^2)$	বর্গ	Loop-এর ভেতরে `includes` বা `find`	১০ কোটি

সতর্কতা: Big-O ধ্রুবক ও ছোট ইনপুট উপেক্ষা করে। ২০টি উপাদানে একটি সরল Array-তে `includes` প্রায়ই `Set` তৈরির চেয়ে দ্রুত, কারণ Set তৈরিরও খরচ আছে। Big-O বলে কোনটি **বড় হলে** টিকবে; ছোট ইনপুটে মাপুন।

১৯.৪ নির্ভরযোগ্যভাবে মাপা

একবার `performance.now()` দিয়ে মাপা সময় প্রায় অর্থহীন। JavaScript Engine (V8) প্রথমে Code ধীরে চালায়, তারপর ঘনঘন চলা অংশ অপ্টিমাইজড Machine Code-এ Compile করে (**JIT**); Garbage Collection মাঝে মাঝে সবকিছু থামায়; অন্য Program CPU ভাগ করে নেয়। একটি নির্ভরযোগ্য পরিমাপের পাঁচটি নিয়ম:

১. **Warm-up:** মাপার আগে কয়েকবার চালান, যাতে JIT-এর প্রাথমিক খরচ মাপে না ঢোকে (অধ্যায় ১৭-এর ReDoS উদাহরণ)।

২. **বহুবার, তারপর Median:** গড় একটি বিরল দীর্ঘ বিরতিতে (GC) বিকৃত হয়; Median ও p95 (৯৫% পরিমাপ যার নিচে) বেশি বিশ্বাসযোগ্য।

৩. **ফল ব্যবহার করুন:** যে হিসাবের ফল কেউ ব্যবহার করে না, Engine তা পুরো বাদ দিতে পারে (Dead Code Elimination), এবং আপনি "শূন্য সময়" মাপবেন।
৪. **একই অবস্থায় তুলনা:** একই ইনপুট, একই যন্ত্র, একই Node.js সংস্করণ; ল্যাপটপ চার্জে আছে কি না পর্যন্ত।
৫. **পরিবেশ লিখে রাখুন:** কোন CPU, কোন Node.js বা Browser, কোন OS। অন্য পরিবেশে ফল ভিন্ন হবে, এবং তা স্বাভাবিক।

stats.js · JavaScript — চলবে: Browser Console ও Node.js

```
function summarize(samples) {
  const sorted = samples.toSorted((a, b) => a - b);
  const last = sorted.length - 1;
  const at = (q) => sorted[Math.min(last, Math.floor(q * sorted.length))];
  const mean = samples.reduce((a, b) => a + b, 0) / samples.length;
  return { mean, median: at(0.5), p95: at(0.95), min: sorted[0] };
}

const samples = [10, 11, 10, 12, 11, 10, 95, 11, 10, 12];
console.log(JSON.stringify(summarize(samples)));
```

Output

```
{"mean":19.2,"median":11,"p95":95,"min":10}
```

দশটি পরিমাপের একটিতে Garbage Collection-এর বিরতি (৯৫)। গড় বলছে ১৯.২—বাস্তবের প্রায় দ্বিগুণ; Median বলছে ১১, যা স্বাভাবিক অবস্থা। p95 দেখায় সবচেয়ে খারাপ অভিজ্ঞতাগুলো কেমন—ব্যবহারকারীর দৃষ্টিতে সেটিও গুরুত্বপূর্ণ। Project-এর Benchmark এই পরিসংখ্যানগুলোই দেয়।

> **সতর্কতা:** Benchmark-এর সংখ্যা কখনো "সব সময় দ্রুত"-এর প্রমাণ নয়; প্রমাণ শুধু এই ইনপুটে, এই পরিবেশে। একটি Micro-benchmark-এ ১০% পার্থক্য প্রায়ই পরিমাপের গোলমালের মধ্যেই পড়ে। এবং Benchmark-কে Automated Test-এর Pass/Fail শর্ত বানাবেন না: যন্ত্র ব্যস্ত থাকলে Test এলোমেলোভাবে ব্যর্থ হবে।

১৯.৫ Main Thread: দীর্ঘ কাজ টুকরোয় ভাগ

অধ্যায় ১০-এ দেখেছেন, JavaScript-এর একটি দীর্ঘ Loop চলার সময় Timer, Click, পর্দা আঁকা—সব অপেক্ষা করে। সমাধান: কাজকে ছোট টুকরোয় ভাগ করে প্রতিটি টুকরোর পর Event Loop-কে সুযোগ দেওয়া। Browser-এ দেখুন:

chunks.js · JavaScript — চলবে: Browser

```
const items = Array.from({ length: 300000 }, (_, i) => i);
const work = (n) => Math.sqrt(n) * Math.sin(n);

function blocking() {
  let timerRan = false;
```

```
  setTimeout(() => {
    timerRan = true;
  }, 0);
  let total = 0;
  for (const n of items) {
    total += work(n);
  }
  return { timerRan, total };
}

async function chunked(size = 20000) {
  let timerRan = false;
  setTimeout(() => {
    timerRan = true;
  }, 0);
  let total = 0;
  let chunks = 0;
  for (let i = 0; i < items.length; i += size) {
    for (const n of items.slice(i, i + size)) {
      total += work(n);
    }
    chunks += 1;
    await new Promise((resolve) => setTimeout(resolve, 0));
  }
  return { timerRan, total, chunks };
}

const a = blocking();
console.log(`একটানা: কাজ চলাকালীন Timer চলেছে? ${a.timerRan}`);
const b = await chunked();
console.log(`টুকরোয়: Timer চলেছে? ${b.timerRan}, টুকরো ${b.chunks}`);
console.log(`একই ফল? ${Math.abs(a.total - b.total) < 1e-6}`);
```

Output

```
একটানা: কাজ চলাকালীন Timer চলেছে? false
টুকরোয়: Timer চলেছে? true, টুকরো 15
একই ফল? true
```

একটানা Loop-এর সময় একটি `setTimeout(..., 0)`-ও চলতে পারেনি, ঠিক যেমন একটি Click বা লেখা পারত না। টুকরোয় ভাগ করা সংস্করণে প্রতিটি টুকরোর পর `await` Event Loop-কে সুযোগ দিয়েছে, এবং Timer মাঝপথেই চলেছে; ফল হুবহু এক। (Listing-টি Module হিসেবে চলে, কারণ এতে Top-level `await` ।) নতুন Browser-এ এ কাজের জন্য বিশেষ `scheduler.yield()` আছে; পুরোনো Browser-এর জন্য `setTimeout`-ই সর্বজনীন। সত্যিকারের ভারী হিসাব Main Thread থেকেই সরিয়ে দেওয়া যায় **Web Worker**-এ, যা আলাদা Thread-এ চলে এবং Message দিয়ে কথা বলে।

১৯.৬ DOM: হালনাগাদ একত্রে, Layout Thrashing নয়

Browser প্রতিটি DOM পরিবর্তনের পর পর্দা নতুন করে হিসাব করতে পারে: কোন Element কোথায়, কত বড় (**Layout**), তারপর আঁকা (**Paint**)। সাধারণত Browser পরিবর্তনগুলো জমিয়ে একবারে হিসাব করে। কিন্তু Code যদি একটি পরিবর্তনের পরেই একটি মাপ **পড়ে** (`offsetHeight`, `getBoundingClientRect`), Browser-কে তখনই হিসাব করতে হয়। লেখা ও পড়া পর্যায়ক্রমে চললে প্রতিটি ধাপে একটি পূর্ণ Layout—একে বলে **Layout Thrashing**।

batch-dom.js · JavaScript — চলবে: Browser

```
const list = document.createElement('ul');
document.body.append(list);
const names = Array.from({ length: 500 }, (_, i) => `সারি ${i + 1}`);

// ধীর ধরন: প্রতিটি সারি যোগ, তারপর সঙ্গে সঙ্গে উচ্চতা পড়া (500টি Layout)।
for (const name of names) {
  const item = document.createElement('li');
  item.textContent = name;
  list.append(item);
  void list.offsetHeight;
}
const slowCount = list.children.length;

// দ্রুত ধরন: সব তৈরি, একবারে যোগ, শেষে একবার পড়া (1টি Layout)।
list.replaceChildren(...names.map((name) => {
  const item = document.createElement('li');
  item.textContent = name;
  return item;
}));
const height = list.offsetHeight;

console.log(slowCount, list.children.length, height > 0);
console.log(list.lastElementChild.textContent);
```

Output

```
500 500 true
সারি 500
```

দুটি পদ্ধতির ফল একই DOM; পার্থক্য Browser-এর কাজে। DevTools-এর Performance Panel-এ প্রথম পদ্ধতিতে বেগুনি "Layout" ঘটনার একটি দীর্ঘ সারি দেখা যায়; দ্বিতীয়টিতে একটি। নিয়ম: **আগে সব পড়ুন, তারপর সব লিখুন।** অধ্যায় ৯ থেকে আমরা যে `replaceChildren(...elements)` ব্যবহার করে আসছি, তা এ কারণেই ভালো অভ্যাস। ব্যবহারকারী দেখতে পান এমন Animation-এর হালনাগাদ `requestAnimationFrame`-এ রাখুন: Browser ঠিক পরের ছবি আঁকার আগে Callback চালায়, এবং লুকানো Tab-এ চালায় না।

একটি হাজারো সারির তালিকার জন্য সবচেয়ে কার্যকর Optimization হলো সারিগুলো আঁকাই না: পর্দায় যা দেখা যায় শুধু সেটুকু (**Virtualization**), অথবা Pagination।

১৯.৭ Memory এবং Garbage Collection

JavaScript-এ Memory হাতে মুক্ত করতে হয় না: **Garbage Collector (GC)** নিয়মিত খোঁজে কোন Object আর **পৌঁছানো যায় না** (Unreachable)—কোনো Variable, Closure, অন্য Object বা Global থেকে কোনো পথ নেই—এবং সেগুলো মুক্ত করে। তাই Memory Leak মানে এমন Object, যা আর দরকার নেই কিন্তু কোথাও থেকে এখনো পৌঁছানো যায়।

সাধারণ Leak	কেন মুক্ত হয় না	সমাধান
সীমাহীন Cache বা Global Array	Global সব সময় পৌঁছানো যায়	আকারের সীমা (LRU), মেয়াদ
সরানো হয়নি এমন Event Listener	Listener-এর Closure তথ্য ধরে রাখে	`removeEventListener`, বা `AbortController`-এর `signal`
চলতে থাকা `setInterval`	Callback ও তার Closure জীবিত	`clearInterval`
Page থেকে সরানো কিন্তু Variable-এ রাখা DOM	Detached Element	Reference মুছে ফেলা
বড় তথ্য ধরে রাখা Closure	Closure যতদিন, তথ্যও ততদিন	শুধু প্রয়োজনীয় অংশ রাখা

`WeakMap`-এর Key দুর্বলভাবে ধরা থাকে: Key Object অন্য কোথাও থেকে আর পৌঁছানো না গেলে Entry-টি নিজেই মুছে যায়। তাই কোনো Object-এর সঙ্গে বাড়তি তথ্য জুড়তে, Object-টিকে বাঁচিয়ে না রেখে, `WeakMap` উপযুক্ত:

weakmap.js · JavaScript — চলবে: Browser Console ও Node.js

```
const metadata = new WeakMap();

function track(element) {
  const info = metadata.get(element) ?? { clicks: 0 };
  info.clicks += 1;
  metadata.set(element, info);
  return info.clicks;
}

let button = { id: 'save' };
track(button);
console.log(track(button), metadata.has(button));

button = null;
console.log(metadata.has({ id: 'save' }));
console.log(typeof metadata.size, typeof metadata.keys);
```

Output

```
2 true
false
undefined undefined
```

`button = null`-এর পর পুরোনো Object আর কোথাও থেকে পৌঁছানো যায় না, তাই GC তার সঙ্গে `metadata`-র Entry-টিও মুক্ত করতে পারে (কখন করবে, তা GC-র সিদ্ধান্ত)। একটি সাধারণ `Map` হলে Map-ই Object-কে বাঁচিয়ে রাখত —Leak। এর মূল্য: `WeakMap`-এর `size` বা `keys` নেই, কারণ কোন Entry এই মুহূর্তে জীবিত তা অনিশ্চিত। নতুন Object দিয়ে খোঁজা (`{ id: 'save' }`) কাজ করে না, কারণ `WeakMap` পরিচয় (Reference) দেখে, বিষয়বস্তু নয়।

Memory মাপা: Chrome DevTools-এর Memory Panel-এ দুটি Heap Snapshot নিয়ে তুলনা করলে দেখা যায় কোন ধরনের Object বাড়ছে; Node.js-এ `process.memoryUsage().heapUsed`। একটি Leak-এর লক্ষণ: একই কাজ বারবার করলে Memory সমতলে স্থির না হয়ে সিঁড়ির মতো বাড়তেই থাকে।

১৯.৮ Cache: LRU, মেয়াদ এবং ঝুঁকি

অধ্যায় ১৪-এর Memoization-এর Cache সীমাহীন—দীর্ঘজীবী Server-এ এটি নিজেই একটি Leak। **LRU (Least Recently Used)** Cache একটি নির্দিষ্ট সংখ্যার বেশি Entry রাখে না; ভর্তি হলে সবচেয়ে বেশি দিন অব্যবহৃতটি বাদ দেয় (**Eviction**)। JavaScript-এর `Map` Entry যোগ করার ক্রম মনে রাখে, তাই LRU খুব সহজে লেখা যায়:

lru.js · JavaScript — চলবে: Node.js

```
class LruCache {
  #limit;
  #map = new Map();

  constructor(limit) {
    this.#limit = limit;
  }

  get(key) {
    if (!this.#map.has(key)) {
      return undefined;
    }
    const value = this.#map.get(key);
    this.#map.delete(key);
    this.#map.set(key, value);
    return value;
  }

  set(key, value) {
    this.#map.delete(key);
    this.#map.set(key, value);
    if (this.#map.size > this.#limit) {
      this.#map.delete(this.#map.keys().next().value);
    }
  }

  get keys() {
    return [...this.#map.keys()];
  }
```

```
}

const cache = new LruCache(3);
['a', 'b', 'c'].forEach((key) => cache.set(key, key.toUpperCase()));
cache.get('a');
cache.set('d', 'D');
console.log(cache.keys, cache.get('b'));
```

Output

```
[ 'c', 'a', 'd' ] undefined
```

`get` একটি Entry-কে মুছে আবার যোগ করে, ফলে সেটি "সাম্প্রতিকতম" প্রান্তে চলে যায়। ভর্তি হলে `Map`-এর প্রথম Key —সবচেয়ে পুরোনো ব্যবহৃত—বাদ। `a` সম্প্রতি পড়া হয়েছিল বলে বেঁচে গেছে, `b` বাদ পড়েছে।

Cache-এর ঝুঁকি: **বাসি তথ্য** (মূল তথ্য বদলালেও Cache পুরোনোটি দেয়—তাই মেয়াদ বা TTL), **Memory** (তাই আকারের সীমা), এবং **ভুল Key** (দুটি ভিন্ন অনুরোধ একই Key পেলে একজনের তথ্য অন্যজন পায়—ব্যবহারকারীভিত্তিক তথ্যে এটি নিরাপত্তা-ঝুঁকিও)। প্রবাদটি: Computer Science-এর দুটি কঠিন সমস্যার একটি হলো Cache কখন অবৈধ করতে হবে তা জানা।

১৯.৯ Network: Byte-ই সবচেয়ে ধীর

Browser-এ প্রায়ই সবচেয়ে বড় খরচ JavaScript চালানো নয়, আনা। প্রতিটি KB ধীর Mobile Network-এ সময় নেয়, এবং আসার পর Parse ও Compile করতেও CPU লাগে।

- **কম পাঠান:** অব্যবহৃত Code বাদ (Bundler-এর **Tree Shaking**, যা ES Module-এর স্থির `import`-এর কারণে সম্ভব—অধ্যায় ১২), এবং একটি ছোট কাজের জন্য বিশাল Library এড়ানো।
- **পরে পাঠান (Lazy Loading):** যা প্রথম পর্দায় লাগে না, তা প্রয়োজনের মুহূর্তে Dynamic `import()` দিয়ে (অধ্যায় ১২-এর সমাধান ১১)। ছবির জন্য `<img loading="lazy">`।
- **সংকুচিত করে পাঠান:** Server-এর gzip বা Brotli Compression; ছবির উপযুক্ত আকার ও Format (WebP, AVIF)।
- **একবার পাঠান:** Cache Header (`Cache-Control`), এবং File-এর নামে বিষয়বস্তুর Hash (`app.3f9a1c.js`), যাতে বদলালে নতুন নাম, না বদলালে দীর্ঘ মেয়াদি Cache।
- **মাপুন:** DevTools-এর Network Panel-এ "Slow 4G" Throttling, এবং Lighthouse Report।

১৯.১০ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
না মেপে Optimize	জটিল Code, লাভ নেই	আগে Profile

ভুল	কেন সমস্যা	সমাধান
একবারের পরিমাপে সিদ্ধান্ত	গোলমালকে পার্থক্য ভাবা	Warm-up, বহুবার, Median
গড় দিয়ে বিচার	একটি GC-বিরতি গড় বিকৃত করে	Median ও p95
Benchmark-এর ফল ব্যবহার না করা	Engine হিসাব বাদ দেয়	ফল জমিয়ে রাখুন
Loop-এর ভেতরে `includes` / `find`	$O(n^2)$	`Set` / `Map` Index
ছোট ইনপুটে Big-O-র কথা ভেবে জটিলতা	ধ্রুবক খরচ বেশি	মাপুন
লেখা-পড়া পর্যায়ক্রমে DOM-এ	Layout Thrashing	আগে পড়া, পরে লেখা
একটানা ভারী Loop Main Thread-এ	Page জমে যায়	টুকরো, বা Web Worker
সীমাহীন Cache	Memory Leak	LRU, মেয়াদ
Listener ও Interval সরানো হয় না	Memory Leak	পরিষ্কারের ব্যবস্থা

১৯.১১ Debugging নির্দেশিকা

১. **সমস্যা নির্দিষ্ট করুন:** "ধীর" নয়; "১০,০০০ সারির Report তৈরিতে ৪ সেকেন্ড, লক্ষ্য ৫০০ মিলিসেকেন্ড"।

২. **Profile করুন:** Chrome DevTools-এর Performance Panel-এ Record করে কাজটি করুন; Flame Chart-এ সবচেয়ে চওড়া Function-গুলো খুঁজুন। Node.js-এ `node --cpu-prof script.mjs` একটি `.cpuprofile` File তৈরি করে, যা DevTools-এ খোলা যায়।

৩. **সবচেয়ে বড় খরচটি আগে:** একটি Function যদি মোট সময়ের ৮০% নেয়, বাকিগুলো দ্বিগুণ দ্রুত করেও সামান্য লাভ।

৪. **একটি পরিবর্তন, তারপর আবার মাপুন,** একই ইনপুটে; ফল বা আচরণ বদলায়নি তা Test দিয়ে নিশ্চিত করুন।

৫. **Memory:** দুটি Heap Snapshot-এর তুলনা; Detached DOM Element ও সীমাহীন Collection খুঁজুন।

৬. **Long Task:** Performance Panel-এ লাল কোণযুক্ত ৫০ মিলিসেকেন্ডের বেশি কাজগুলো।

১৯.১২ পেশাদার অভ্যাস

- **আগে সঠিক, তারপর পরিষ্কার, তারপর দ্রুত**—এবং দ্রুত করার আগে মাপা।
- **Algorithm আগে, Micro-optimization পরে।**
- **প্রতিটি Optimization-এর সঙ্গে একটি Correctness Test,** যা প্রমাণ করে ফল বদলায়নি।
- **Benchmark-এর ফলের সঙ্গে পরিবেশ ও পদ্ধতি লিখে রাখুন;** একক সংখ্যা নয়।
- **Main Thread হালকা:** ৫০ মিলিসেকেন্ডের বেশি কাজ ভাগ করুন।
- **প্রতিটি Cache-এর আকার ও মেয়াদের সীমা।**
- **যা চালু করেন, তা বন্ধের ব্যবস্থা:** Listener, Interval, Subscription।
- **Performance-এর বাজেট:** যেমন "প্রথম পর্দার JavaScript ১৭০ KB-র কম"; প্রতিটি পরিবর্তনে যাচাই।

১৯.১৩ Project: PerfLab

PerfLab একটি ছোট Performance Toolkit (Node.js): একটি নির্ভরযোগ্য Benchmark Harness (Warm-up, পুনরাবৃত্তি, Median, p95, পরিবেশের বিবরণ), একই কাজের ধীর ও দ্রুত Algorithm-জোড়া, একটি LRU Cache (আকার, মেয়াদ, Hit-Miss পরিসংখ্যান), এবং দীর্ঘ কাজ টুকরোয় ভাগের একটি সাহায্যকারী। এর সবচেয়ে গুরুত্বপূর্ণ অংশ Benchmark নয়, Test: প্রতিটি দ্রুত Algorithm তার ধীর সংস্করণের সঙ্গে ফলে হুবহু মেলে কি না, তা এলোমেলো কিন্তু পুনরুৎপাদনযোগ্য ইনপুটে যাচাই।

Project-এর চাহিদা

- Harness: Warm-up, নির্দিষ্ট সংখ্যক পরিমাপ, Median, p95, সর্বনিম্ন, ops/sec; সময়ের উৎস Inject করা যায় (Test-এর জন্য)।
- Algorithm-জোড়া: পুনরাবৃত্তি বাদ (Nested বনাম `Set`), সাধারণ উপাদান (Nested বনাম `Set`), সাজানো তালিকায় খোঁজা (Linear বনাম Binary)।
- Correctness: প্রতিটি জোড়া একই ইনপুটে একই ফল, Seed-যুক্ত এলোমেলো ইনপুটে, যাতে ব্যর্থ হলে আবার একই ইনপুট পাওয়া যায়।
- LRU Cache: সীমা, মেয়াদ (TTL, ঘড়ি Inject করা), Hit/Miss গণনা।
- টুকরোয় কাজ: সাহায্যকারী প্রতিটি টুকরোর পর একটি Inject করা "বিরতি" Function ডাকে।
- Benchmark Report: Markdown, পরিবেশসহ; Benchmark কখনো Test-এর শর্ত নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
perf-lab/
├── src/
│   ├── bench.mjs          Benchmark Harness ও পরিসংখ্যান
│   ├── algorithms.mjs     ধীর ও দ্রুত Algorithm-জোড়া
│   ├── lru.mjs            LRU Cache (TTL, পরিসংখ্যান)
│   ├── chunk.mjs          টুকরোয় কাজ
│   └── random.mjs         Seed-যুক্ত এলোমেলো সংখ্যা
├── run-bench.mjs          Benchmark চালিয়ে Report লেখে
├── README.md
└── tests/
    ├── algorithms.test.mjs
    ├── lru.test.mjs
    └── bench-chunk.test.mjs
```

src/random.mjs ও src/algorithms.mjs

random.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// Seed-যুক্ত ছোট PRNG (mulberry32): একই Seed-এ সব সময় একই ক্রম।
export function createRandom(seed) {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let t = state;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export const randomInts = (random, count, max) =>
  Array.from({ length: count }, () => Math.floor(random() * max));
```

`Math.random()`-এর Seed দেওয়া যায় না, তাই একটি ব্যর্থ Test-এর ইনপুট আর ফিরে পাওয়া যায় না। ছোট এই Generator (mulberry32 নামে পরিচিত) একটি সংখ্যা (Seed) থেকে সব সময় একই "এলোমেলো" ক্রম দেয়। বিটের কারসাজিগুলো (`>>>`, `Math.imul`) এখানে বোঝার দরকার নেই; শুধু জানুন এটি নিরাপত্তার কাজের উপযুক্ত নয় (তার জন্য `crypto`, অধ্যায় ২০)।

algorithms.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// প্রতিটি কাজের দুটি সংস্করণ: সরল (ধীর) এবং দ্রুত। ফল হুবহু এক হতে হবে।

export const uniqueNested = (items) =>
  items.filter((item, index) => items.indexOf(item) === index);

export const uniqueSet = (items) => [...new Set(items)];

export const commonNested = (a, b) => a.filter((x) => b.includes(x));

export function commonSet(a, b) {
  const lookup = new Set(b);
  return a.filter((x) => lookup.has(x));
}

export const indexOfLinear = (sorted, target) => sorted.indexOf(target);

// সাজানো তালিকায় প্রথম মিলের Index, বা -1। O(log n)।
export function indexOfBinary(sorted, target) {
  let low = 0;
  let high = sorted.length - 1;
  let found = -1;
  while (low <= high) {
    const middle = (low + high) >>> 1;
```

```
    if (sorted[middle] < target) {
      low = middle + 1;
    } else {
      if (sorted[middle] === target) {
        found = middle;
      }
      high = middle - 1;
    }
  }
  return found;
}
```

- `uniqueNested` প্রতিটি উপাদানের জন্য পুরো তালিকায় `indexOf`: $O(n^2)$। `uniqueSet` O(n), এবং প্রথম দেখার ক্রম রাখে—তাই দুটির ফল হুবহু এক।
- `indexOfBinary` প্রতিবার খোঁজার পরিসর অর্ধেক করে। মিলে গেলেও বাঁ দিকে খোঁজা চালিয়ে যায়, যাতে পুনরাবৃত্ত মানে **প্রথম** Index ফেরত দেয়—ঠিক `indexOf`-এর মতো। এই সূক্ষ্মতা বাদ পড়লে দ্রুত সংস্করণ "প্রায়" সঠিক হতো; Correctness Test এমন ভুলই ধরে।
- `(low + high) >>> 1` দুটি পূর্ণসংখ্যার গড়ের পূর্ণ অংশ।

src/bench.mjs

bench.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export function summarize(samples) {
  const sorted = samples.toSorted((a, b) => a - b);
  const last = sorted.length - 1;
  const at = (q) => sorted[Math.min(last, Math.floor(q * sorted.length))];
  return { median: at(0.5), p95: at(0.95), min: sorted[0], runs: samples.length };
}

// fn বহুবার চালিয়ে সময় মাপে। now Inject করা যায় (Test-এ নিয়ন্ত্রিত ঘড়ি)।
export function bench(fn, options = {}) {
  const { warmup = 5, runs = 30, now = () => performance.now() } = options;
  let sink;
  for (let i = 0; i < warmup; i += 1) {
    sink = fn();
  }
  const samples = [];
  for (let i = 0; i < runs; i += 1) {
    const start = now();
    sink = fn();
    samples.push(now() - start);
  }
  const stats = summarize(samples);
  const opsPerSec = stats.median > 0 ? 1000 / stats.median : Infinity;
  return { ...stats, opsPerSec, sinkType: typeof sink };
}
```

```
export function environment() {
  return {
    node: process.version,
    platform: `${process.platform} ${process.arch}`,
    date: new Date().toISOString().slice(0, 10),
  };
}
```

`sink` প্রতিটি ফল ধরে রাখে, এবং শেষে তার ধরন ফলের সঙ্গে ফেরত যায়; ফলটি "ব্যবহৃত", তাই Engine হিসাব বাদ দিতে পারে না (Dead Code Elimination প্রতিরোধ)। `now` Inject করা যায়: Test-এ একটি কৃত্রিম ঘড়ি দিয়ে Median ও p95-এর হিসাব নিশ্চিতভাবে যাচাই করা হয়, আসল সময়ের ওপর নির্ভর না করে।

src/lru.mjs ও src/chunk.mjs

lru.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export class LruCache {
  #limit;
  #ttl;
  #now;
  #map = new Map();
  stats = { hits: 0, misses: 0, evictions: 0 };

  constructor({ limit, ttlMs = Infinity, now = () => Date.now() }) {
    if (!Number.isInteger(limit) || limit < 1) {
      throw new RangeError('limit একটি ধনাত্মক পূর্ণসংখ্যা হতে হবে');
    }
    this.#limit = limit;
    this.#ttl = ttlMs;
    this.#now = now;
  }

  get(key) {
    const entry = this.#map.get(key);
    if (!entry || this.#now() - entry.at >= this.#ttl) {
      this.#map.delete(key);
      this.stats.misses += 1;
      return undefined;
    }
    this.#map.delete(key);
    this.#map.set(key, entry);
    this.stats.hits += 1;
    return entry.value;
  }

  set(key, value) {
    this.#map.delete(key);
```

```
    this.#map.set(key, { value, at: this.#now() });
    if (this.#map.size > this.#limit) {
      this.#map.delete(this.#map.keys().next().value);
      this.stats.evictions += 1;
    }
  }

  get size() {
    return this.#map.size;
  }
}
```

chunk.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const nextTask = () => new Promise((resolve) => setTimeout(resolve, 0));

// items-কে size-এর টুকরোয় process করে; প্রতিটি টুকরোর পর yieldFn।
export async function processInChunks(items, fn, options = {}) {
  const { size = 1000, yieldFn = nextTask } = options;
  if (!Number.isInteger(size) || size < 1) {
    throw new RangeError('size একটি ধনাত্মক পূর্ণসংখ্যা হতে হবে');
  }
  const results = [];
  for (let i = 0; i < items.length; i += size) {
    for (const item of items.slice(i, i + size)) {
      results.push(fn(item));
    }
    if (i + size < items.length) {
      await yieldFn();
    }
  }
  return results;
}
```

LRU-তে নতুন দুটি জিনিস: প্রতিটি Entry-র সময় (`at`), যা দিয়ে মেয়াদোত্তীর্ণ Entry পড়ার সময়ই বাদ পড়ে; এবং `stats`, যাতে বোঝা যায় Cache আসলে কাজে লাগছে কি না (Hit-এর হার কম হলে Cache শুধু Memory খরচ)। `processInChunks` শেষ টুকরোর পরে আর বিরতি নেয় না, এবং `yieldFn` Inject করা যায়—Browser-এ `scheduler.yield`, Test-এ একটি গণনাকারী Function।

run-bench.mjs

run-bench.mjs · JavaScript — চলবে: Node.js

```
import { writeFileSync, mkdirSync } from 'node:fs';
import { bench, environment } from './src/bench.mjs';
import { createRandom, randomInts } from './src/random.mjs';
import * as algo from './src/algorithms.mjs';
```

```
const random = createRandom(2026);
const a = randomInts(random, 5000, 10000);
const b = randomInts(random, 5000, 10000);
const sorted = a.toSorted((x, y) => x - y);
const targets = randomInts(random, 200, 10000);
const searchAll = (find) => targets.map((t) => find(sorted, t));

const cases = [
  ['unique: nested', () => algo.uniqueNested(a)],
  ['unique: Set', () => algo.uniqueSet(a)],
  ['common: nested', () => algo.commonNested(a, b)],
  ['common: Set', () => algo.commonSet(a, b)],
  ['search x200: linear', () => searchAll(algo.indexOfLinear)],
  ['search x200: binary', () => searchAll(algo.indexOfBinary)],
];

const env = environment();
const rows = cases.map(([name, fn]) => {
  const result = bench(fn, { warmup: 5, runs: 25 });
  const { median, p95, runs } = result;
  return `| ${name} | ${median.toFixed(3)} | ${p95.toFixed(3)} | ${runs} |`;
});

const report = [
  '# PerfLab Benchmark',
  '',
  `Node.js ${env.node}, ${env.platform}, ${env.date}.`,
  'n = 5000; warm-up 5, runs 25.',
  '',
  '| Case | median ms | p95 ms | runs |',
  '| --- | --- | --- | --- |',
  ...rows,
].join('\n');

mkdirSync('results', { recursive: true });
writeFileSync('results/benchmark-report.md', `${report}\n`);
console.log(report);
```

এই Script ইচ্ছা করে Automated Test-এর অংশ নয়: সময়ের সংখ্যা যন্ত্র ও মুহূর্তভেদে বদলায়। এই বইয়ের লেখার সময় একটি যন্ত্রে (Intel Core i5-6600, 8 Core, ৮ GB RAM, Windows 10) চালানো আসল Report, Companion Code-এর `results/benchmark-report.md`-এ সংরক্ষিত:

পরিবেশ: Node.js v22.16.0, win32 x64, 2026-10-15. n = 5000; warm-up 5, runs 25.

Case	median (ms)	p95 (ms)	পরিমাপ
unique: nested	13.363	33.531	25
unique: Set	0.254	1.689	25

Case	median (ms)	p95 (ms)	পরিমাপ
common: nested	44.422	108.101	25
common: Set	0.425	1.182	25
search x200: linear	0.854	2.779	25
search x200: binary	0.023	0.173	25

একই যন্ত্রে ঠিক আগের চালানোয় "common: nested"-এর Median ছিল 33.674 ms, এবারে 44.422 ms—কোনো Code না বদলেই প্রায় ৩০% পার্থক্য। একবারের পরিমাপ কেন যথেষ্ট নয়, এটিই তার জীবন্ত প্রমাণ। অনুপাতও কিছুটা বদলেছে, কিন্তু তার মাত্রা দুবারই এক: Set সংস্করণ প্রায় ৫০ থেকে ১০০ গুণ দ্রুত। সংখ্যাগুলো পড়ার নিয়ম: নিজের যন্ত্রে আপনার সংখ্যা আলাদা হবে; গুরুত্বপূর্ণ হলো **অনুপাত** এবং ইনপুট বাড়ালে সেই অনুপাত কীভাবে বদলায়। Nested বনাম Set-এর পার্থক্য Big-O-র ভবিষ্যদ্বাণীর সঙ্গে মেলে কি না দেখুন; n দ্বিগুণ করে আবার চালালে Nested-এর সময় প্রায় চারগুণ, Set-এর প্রায় দ্বিগুণ হওয়ার কথা।

Code-এর ধাপে ধাপে ব্যাখ্যা

- **একই Seed (২০২৬)** প্রতিবার একই ইনপুট দেয়, তাই দুটি চালানোর তুলনা ন্যায্য।
- **প্রতিটি Case একটি Function,** যা ফল ফেরত দেয়; `bench` সেই ফল ধরে রাখে।
- **Warm-up ৫, পরিমাপ ২৫, Median ও p95**; Report-এ পরিবেশ ও পদ্ধতি লেখা।
- **Report একটি File-এ,** তারিখসহ; পরের Optimization-এর পর একই Script চালিয়ে তুলনা করা যায়।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node --test
node run-bench.mjs
```

Companion Source Code-এর ২০টি Test: প্রতিটি Algorithm-জোড়ার ফল ১০০টি Seed-যুক্ত এলোমেলো ইনপুটে হুবহু মেলে কি না (Binary Search-এর পুনরাবৃত্ত মান ও সীমার ক্ষেত্রসহ); LRU-র Eviction ক্রম, মেয়াদ (কৃত্রিম ঘড়ি) ও পরিসংখ্যান; Harness-এর Median ও p95 (কৃত্রিম ঘড়ি); এবং টুকরোয় কাজের ফল, ক্রম ও বিরতির সংখ্যা। কোনো Test সময়ের ওপর নির্ভর করে না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
দ্রুত সংস্করণের ফল মাঝে মাঝে আলাদা	সীমার ক্ষেত্র (পুনরাবৃত্ত মান, ফাঁকা তালিকা)	Seed দিয়ে ব্যর্থ ইনপুট আবার চালান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Benchmark-এর সংখ্যা প্রতিবার আলাদা	স্বাভাবিক গোলমাল	Median, বেশি পরিমাপ, শান্ত যন্ত্র
একটি Case "শূন্য" সময়	ফল ব্যবহৃত হয়নি	`sink`
Cache-এ Hit প্রায় নেই	Key প্রতিবার আলাদা, বা সীমা ছোট	`stats` দেখুন
টুকরোয় ভাগের পরও Page জমে	টুকরো বড়	`size` কমান

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Harness পূর্ণাঙ্গ Benchmark Library-র (যেমন Tinybench বা Benchmark.js) মতো পরিসংখ্যানগত তাৎপর্য যাচাই করে না; শুধু Node.js-এ চলে, Browser-এর Rendering মাপে না। ফলাফল এক যন্ত্রের, এক মুহূর্তের।

১৯.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `[3, 1, 2]`-এ `includes` এবং `Set.has`—দুটির Big-O কত? কোনটি ১০ উপাদানে ভালো, কেন বলা কঠিন?

২. পাঁচটি পরিমাপ `[12, 13, 12, 80, 12]`-এর গড় ও Median বের করুন; কোনটি বিশ্বাসযোগ্য?

৩. Nested Loop-এ দুটি তালিকার সাধারণ উপাদান খোঁজার ধাপ গুনুন (n = ১০০); তারপর Set দিয়ে।

৪. একটি `setInterval`-এর Leak সারান, যাতে Component সরানোর সময় Interval বন্ধ হয়।

৫. `WeakMap` ও `Map`-এর একটি পার্থক্য উদাহরণসহ লিখুন।

মধ্যম স্তর

৬. দুটি পদ্ধতিতে একটি তালিকা থেকে সবচেয়ে বেশি আসা মান বের করুন (Nested ও `Map`), এবং Seed-যুক্ত ইনপুটে দেখান ফল এক।

৭. PerfLab-এর LRU দিয়ে একটি ধীর Function Memoize করুন, এবং ১০টি অনুরোধে Hit-এর সংখ্যা দেখান।

৮. `processInChunks`-কে একটি গণনাকারী `yieldFn` দিয়ে চালিয়ে দেখান ১০,০০০ উপাদানে ১,০০০-এর টুকরোয় কয়টি বিরতি হয়।

৯. একটি Event Listener-কে `AbortController`-এর `signal` দিয়ে যোগ করুন, এবং `abort()` দিয়ে সরান (Browser)।

চ্যালেঞ্জ স্তর

১০. সাজানো দুটি তালিকার সাধারণ উপাদান O(n + m)-এ বের করুন (দুটি Pointer), এবং Set পদ্ধতির সঙ্গে ফল মেলান।

১১. PerfLab-এর Harness দিয়ে `uniqueNested` ও `uniqueSet` n = ১০০০ ও ২০০০-এ মাপুন; Median-এর অনুপাত Big-O-র সঙ্গে মেলে কি না বিচার করুন (নিজের যন্ত্রে)।

১২. LRU-তে একটি `peek(key)` যোগ করুন, যা মান দেখায় কিন্তু "সাম্প্রতিক" ক্রম বদলায় না।

১৯.১৫ অনুশীলনীর সমাধান

সমাধান ১ — `includes` O(n), `Set.has` O(1)। কিন্তু ১০টি উপাদানে Set তৈরির খরচ (প্রতিটি উপাদান Hash করা) একবারের খোঁজার চেয়ে বেশি হতে পারে; ছোট n-এ ধ্রুবক খরচই নির্ণায়ক। একই তালিকায় বহুবার খুঁজলে Set একবার তৈরি করা লাভজনক। ছোট ইনপুটে উত্তর দেয় পরিমাপ, Big-O নয়।

সমাধান ২

mean-median.js · JavaScript — চলবে: Browser Console ও Node.js

```
const samples = [12, 13, 12, 80, 12];
const mean = samples.reduce((a, b) => a + b, 0) / samples.length;
const median = samples.toSorted((a, b) => a - b)[Math.floor(samples.length / 2)];

console.log(mean, median);
```

Output

```
25.8 12
```

একটি ৮০-র বিরতি গড়কে দ্বিগুণের বেশি করেছে; Median স্বাভাবিক অবস্থা দেখায়।

সমাধান ৩

count-common.js · JavaScript — চলবে: Browser Console ও Node.js

```
const a = Array.from({ length: 100 }, (_, i) => i);
const b = Array.from({ length: 100 }, (_, i) => i + 50);

let nestedSteps = 0;
for (const x of a) {
  for (const y of b) {
    nestedSteps += 1;
    if (x === y) {
      break;
    }
  }
}
```

```
const lookup = new Set(b);
let setSteps = b.length;
for (const x of a) {
  setSteps += 1;
  lookup.has(x);
}
console.log(nestedSteps, setSteps);
```

Output

```
6275 200
```

সমাধান ৪

interval-cleanup.js · JavaScript — চলবে: Browser Console ও Node.js

```
function startClock(onTick) {
  const id = setInterval(onTick, 50);
  return () => clearInterval(id);
}

let ticks = 0;
const stop = startClock(() => {
  ticks += 1;
});

setTimeout(() => {
  stop();
  const seen = ticks;
  setTimeout(() => console.log(`বন্ধের পর নতুন টিক: ${ticks - seen}`), 150);
}, 120);
```

Output

```
বন্ধের পর নতুন টিক: 0
```

যে Function কিছু চালু করে, সে বন্ধের Function ফেরত দেয়—React-এর `useEffect`-এর Cleanup এই নীতিতেই।

সমাধান ৫ — `Map` Key-কে শক্তভাবে ধরে রাখে: Key Object আর কোথাও না থাকলেও Map-ই তাকে বাঁচিয়ে রাখে। `WeakMap` দুর্বলভাবে ধরে: Key আর পৌঁছানো না গেলে Entry মুছে যেতে পারে। তাই একটি DOM Element-এর সঙ্গে বাড়তি তথ্য রাখতে `WeakMap`: Element Page থেকে সরে গেলে তথ্যও মুক্ত। মূল্য: `WeakMap`-এর Key শুধু Object, এবং `size` বা Loop নেই (অনুচ্ছেদ ১৯.৭-এর উদাহরণ)।

সমাধান ৬

most-frequent.mjs · JavaScript — চলবে: Node.js

```
import { createRandom, randomInts } from '../../project/perf-lab/src/random.mjs';

const mostNested = (items) => items.reduce((best, item) => {
  const count = items.filter((x) => x === item).length;
  return count > best.count || (count === best.count && item < best.value)
    ? { value: item, count }
    : best;
}, { value: undefined, count: 0 });

function mostMap(items) {
  const counts = new Map();
  let best = { value: undefined, count: 0 };
  for (const item of items) {
    const count = (counts.get(item) ?? 0) + 1;
    counts.set(item, count);
    if (count > best.count || (count === best.count && item < best.value)) {
      best = { value: item, count };
    }
  }
  return best;
}

const random = createRandom(7);
let same = true;
for (let round = 0; round < 50; round += 1) {
  const items = randomInts(random, 200, 30);
  same &&= JSON.stringify(mostNested(items)) === JSON.stringify(mostMap(items));
}
console.log(same);
```

Output

```
true
```

সমান সংখ্যায় সবচেয়ে ছোট মান—এই নিয়ম দুটিতে একই থাকায় ফল নিশ্চিত; নিয়মটি না থাকলে দুটি সঠিক Algorithm-ও ভিন্ন "সঠিক" উত্তর দিত, এবং তুলনা অর্থহীন হতো।

সমাধান ৭

lru-memo.mjs · JavaScript — চলবে: Node.js

```
import { LruCache } from '../../project/perf-lab/src/lru.mjs';
```

```
const cache = new LruCache({ limit: 3 });
const slowSquare = (n) => n * n;

function cachedSquare(n) {
  const hit = cache.get(n);
  if (hit !== undefined) {
    return hit;
  }
  const value = slowSquare(n);
  cache.set(n, value);
  return value;
}

[1, 2, 1, 3, 1, 4, 2, 1, 5, 1].forEach(cachedSquare);
console.log(cache.stats, cache.size);
```

Output

```
{ hits: 4, misses: 6, evictions: 3 } 3
```

`1` বারবার ব্যবহৃত বলে কখনো বাদ পড়েনি; সীমা ৩ হওয়ায় Memory কখনো তিনটির বেশি নয়।

সমাধান ৮

count-yields.mjs · JavaScript — চলবে: Node.js

```
import { processInChunks } from '../../project/perf-lab/src/chunk.mjs';

let pauses = 0;
const items = Array.from({ length: 10000 }, (_, i) => i);
const results = await processInChunks(items, (n) => n * 2, {
  size: 1000,
  yieldFn: async () => {
    pauses += 1;
  },
});

console.log(results.length, results.at(-1), pauses);
```

Output

```
10000 19998 9
```

দশটি টুকরো, কিন্তু বিরতি নয়টি: শেষ টুকরোর পর বিরতির প্রয়োজন নেই।

সমাধান ৯

abort-listener.js · JavaScript চলবে: Browser

```
const button = document.createElement('button');
button.id = 'go';
button.textContent = 'চাপুন';
document.body.append(button);

const controller = new AbortController();
let clicks = 0;

button.addEventListener('click', () => {
  clicks += 1;
  console.log(`Click ${clicks}`);
  controller.abort();
}, { signal: controller.signal });
```

দুবার চাপলে:

Output

```
Click 1
```

প্রথম Click-এর পর `abort()` Listener সরিয়ে দিয়েছে; দ্বিতীয় Click-এ কিছুই চলেনি। একই `signal` দিয়ে অনেক Listener যোগ করলে একটি `abort()` সবগুলো একসঙ্গে সরায়—একটি Component বন্ধের সময় সব পরিষ্কারের সহজ উপায়।

সমাধান ১০

two-pointers.mjs · JavaScript চলবে: Node.js

```
import { createRandom, randomInts } from '../../project/perf-lab/src/random.mjs';
import { commonSet } from '../../project/perf-lab/src/algorithms.mjs';

function commonSorted(a, b) {
  const result = [];
  let i = 0;
  let j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] < b[j]) {
      i += 1;
    } else if (a[i] > b[j]) {
      j += 1;
    } else {
      result.push(a[i]);
      i += 1;
```

```
      j += 1;
    }
  }
  return result;
}

const random = createRandom(42);
const unique = (n) =>
  [...new Set(randomInts(random, n, 500))].toSorted((x, y) => x - y);
const a = unique(300);
const b = unique(300);

const fast = JSON.stringify(commonSorted(a, b));
const same = fast === JSON.stringify(commonSet(a, b));
console.log(same);
```

Output

```
true
```

দুটি Pointer প্রতি ধাপে অন্তত একটি এগোয়, তাই মোট ধাপ সর্বোচ্চ n + m, এবং কোনো বাড়তি Memory লাগে না। শর্ত: দুটি তালিকা সাজানো এবং পুনরাবৃত্তিহীন।

সমাধান ১১ — `bench(() => uniqueNested(a1000))` ও `bench(() => uniqueNested(a2000))`-এর Median-এর অনুপাত প্রায় ৪ এবং `uniqueSet`-এর প্রায় ২ হওয়ার কথা ($O(n^2)$ ও $O(n)$)। বাস্তবে Cache, JIT ও ছোট n-এর কারণে অনুপাত কিছুটা আলাদা হবে; কয়েকবার চালিয়ে দেখুন অনুপাত স্থির কি না। একবারের সংখ্যা থেকে সিদ্ধান্ত নেবেন না, এবং ফলের সঙ্গে নিজের যন্ত্রের বিবরণ লিখে রাখুন।

সমাধান ১২

peek.mjs · JavaScript — চলবে: Node.js

```
import { LruCache } from '../../project/perf-lab/src/lru.mjs';

class PeekableCache extends LruCache {
  #shadow = new Map();

  set(key, value) {
    super.set(key, value);
    this.#shadow.set(key, value);
  }

  peek(key) {
    return this.#shadow.get(key);
  }
}
```

```
const cache = new PeekableCache({ limit: 2 });
cache.set('a', 1);
cache.set('b', 2);
console.log(cache.peek('a'));
cache.set('c', 3);
console.log(cache.get('a'), cache.get('b'));
```

Output

```
1
undefined 2
```

`peek` `a`-কে "সাম্প্রতিক" করেনি, তাই `c` যোগে `a`-ই বাদ পড়েছে। লক্ষ করুন, এই সমাধান একটি দুর্বলতা দেখায়: Child Class Parent-এর Private `#map`-এ পৌঁছাতে পারে না, তাই আলাদা `#shadow` রাখতে হয়েছে, যা বাদ পড়া Entry মুছে না দিলে নিজেই Leak হবে। ভালো নকশা: `peek` LRU-র নিজের Class-এ যোগ করা (অধ্যায় ১৩-এর Composition বনাম Inheritance)।

১৯.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Loop-এর ভেতরে `includes` দিয়ে দুটি তালিকার সাধারণ উপাদান খোঁজার Big-O?

ক) O(n)

খ) $O(n^2)$

গ) O(log n)

ঘ) O(1)

প্রশ্ন ২. Benchmark-এ Warm-up কেন?

ক) JIT Compile-এর প্রাথমিক খরচ মাপ থেকে বাদ দিতে

খ) CPU গরম করতে

গ) ফল বদলাতে

ঘ) Memory কমাতে

প্রশ্ন ৩. একটি GC-বিরতিসহ পরিমাপে কোনটি স্বাভাবিক অবস্থা ভালো দেখায়?

ক) গড়

খ) সর্বোচ্চ

গ) Median

ঘ) যোগফল

প্রশ্ন ৪. Benchmark-এর ফল ব্যবহার না করলে কী হতে পারে?

ক) কিছুই না

খ) Program ভেঙে যায়

গ) সময় দ্বিগুণ হয়

ঘ) Engine হিসাব বাদ দিয়ে অবাস্তব কম সময় দেখাতে পারে

প্রশ্ন ৫. Browser-এ "Long Task" কী?

ক) ৫০ মিলিসেকেন্ডের বেশি Main Thread আটকে রাখা কাজ

খ) Network-এর দীর্ঘ অনুরোধ

গ) বড় File

ঘ) অনেক লাইনের Function

প্রশ্ন ৬. Layout Thrashing-এর কারণ?

ক) অনেক CSS

খ) DOM-এ লেখা ও মাপ পড়া পর্যায়ক্রমে

গ) বড় ছবি

ঘ) `textContent`

প্রশ্ন ৭. Garbage Collector কোন Object মুক্ত করে?

ক) পুরোনোগুলো

খ) বড়গুলো

গ) যেগুলো আর কোথাও থেকে পৌঁছানো যায় না

ঘ) `null` বসানো প্রতিটি Variable-এর মান, তৎক্ষণাৎ

প্রশ্ন ৮. একটি DOM Element-এর সঙ্গে বাড়তি তথ্য রাখতে, Element-কে বাঁচিয়ে না রেখে, কোনটি?

ক) `Map`

খ) Array

গ) Global Object

ঘ) `WeakMap`

প্রশ্ন ৯. LRU Cache ভর্তি হলে কোন Entry বাদ দেয়?

ক) সবচেয়ে দীর্ঘ সময় অব্যবহৃত

খ) সবচেয়ে নতুন

গ) সবচেয়ে বড়

ঘ) এলোমেলো

প্রশ্ন ১০. Benchmark-কে Automated Test-এর Pass/Fail শর্ত কেন বানানো উচিত নয়?

ক) Benchmark ধীর

খ) সময় যন্ত্রের অবস্থাভেদে বদলায়; Test এলোমেলোভাবে ব্যর্থ হবে

গ) Test Runner সময় মাপতে পারে না

ঘ) Benchmark-এর ফল সব সময় সঠিক

১৯.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	প্রতিটি উপাদানে পুরো তালিকা খোঁজা।
২	ক	প্রথম চালানো ধীর, অপ্টিমাইজড Code পরে আসে।
৩	গ	একটি বিরল দীর্ঘ মান Median-কে প্রায় বদলায় না।
৪	ঘ	Dead Code Elimination।
৫	ক	তখন ব্যবহারকারীর কাজ দেরিতে সাড়া পায়।
৬	খ	প্রতিটি পড়া Browser-কে তখনই Layout হিসাব করতে বাধ্য করে।
৭	গ	Reachability-ই মাপকাঠি; কখন মুক্ত করবে, তা GC ঠিক করে।
৮	ঘ	দুর্বল Reference; Element সরে গেলে তথ্যও মুক্ত হতে পারে।
৯	ক	Least Recently Used।
১০	খ	Correctness Test নিশ্চিত হওয়া উচিত; সময় নয়।

১৯.১৮ অধ্যায়ের সারসংক্ষেপ

- Performance মানে সময়, সাড়া, Throughput, Memory ও Network; আগে মাপুন, তারপর সবচেয়ে বড় সমস্যা।
- Big-O বৃদ্ধির হার; ধাপ গুনে যাচাই; Loop-এর ভেতরে খোঁজা $O(n^2)$, Set/Map Index $O(n)$।
- নির্ভরযোগ্য পরিমাপ: Warm-up, বহুবার, Median ও p95, ফল ব্যবহার, পরিবেশ লেখা।
- Main Thread-এ ৫০ মিলিসেকেন্ডের বেশি কাজ টুকরোয় ভাগ, বা Web Worker।
- DOM: আগে পড়া, পরে লেখা; একত্রে হালনাগাদ; বড় তালিকায় Virtualization।
- GC Unreachable Object মুক্ত করে; Leak মানে অপ্রয়োজনীয় কিন্তু পৌঁছানো যায় এমন Object; `WeakMap`।
- Cache-এর আকার (LRU) ও মেয়াদের সীমা; Hit-এর হার মাপুন।

- Network: কম, পরে, সংকুচিত, একবার পাঠান।
- প্রতিটি Optimization-এর সঙ্গে Correctness Test; Benchmark Test-এর শর্ত নয়।

পরের অধ্যায়ে JavaScript Security: Threat Modeling, XSS, Injection, Authentication ও Authorization, এবং নিরাপদ Code লেখার অভ্যাস।

১৯.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Long Task	৫০ মিলিসেকেন্ডের বেশি Main Thread আটকানো কাজ।
Big-O	ইনপুটের সঙ্গে কাজের বৃদ্ধির হার।
JIT	চলার সময় ঘনঘন Code-কে অপ্টিমাইজড Machine Code-এ রূপান্তর।
Median / p95	মাঝের মান / ৯৫% মান যার নিচে।
Dead Code Elimination	অব্যবহৃত ফলের হিসাব বাদ দেওয়া।
Layout Thrashing	লেখা-পড়া পর্যায়ক্রমে বারবার Layout।
Garbage Collection	পৌঁছানো যায় না এমন Object-এর Memory মুক্ত করা।
Memory Leak	অপ্রয়োজনীয় Object জীবিত থেকে যাওয়া।
LRU Cache	সবচেয়ে দীর্ঘ অব্যবহৃত Entry বাদ দেওয়া সীমিত Cache।
Lazy Loading	প্রয়োজনের মুহূর্তে আনা।

১৯.২০ আরও পড়ুন

- **MDN — Performance.now():** `https://developer.mozilla.org/en-US/docs/Web/API/Performance/now` — উচ্চ-নির্ভুল সময় মাপা।
- **web.dev — Optimize long tasks:** `https://web.dev/articles/optimize-long-tasks` — Main Thread-এ কাজ ভাগ করা।
- **MDN — Memory management:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_management` — Garbage Collection ও Reachability।
- **Chrome DevTools — Analyze runtime performance:** `https://developer.chrome.com/docs/devtools/performance` — Performance Panel।
- **web.dev — Web Vitals:** `https://web.dev/articles/vitals` — LCP, INP, CLS।

অধ্যায় ২০

২০

JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি Application-এর Threat Model তৈরি করতে: কী রক্ষা করছেন, কার কাছ থেকে, এবং Trust Boundary কোথায়।
- XSS কীভাবে ঘটে তা ব্যাখ্যা করতে, এবং প্রসঙ্গ অনুযায়ী Output Encoding ও নিরাপদ DOM API দিয়ে তা প্রতিরোধ করতে।
- Input Validation ও Output Encoding-এর পার্থক্য বুঝতে, এবং কেন Client-side Validation নিরাপত্তা দেয় না তা ব্যাখ্যা করতে।
- Parameterized Query দিয়ে SQL Injection প্রতিরোধ করতে।
- Authentication ও Authorization আলাদা করতে, এবং প্রতিটি Object-এর জন্য Authorization যাচাই করতে।
- Password নিরাপদে সংরক্ষণ করতে (scrypt, Salt, ধ্রুব-সময়ের তুলনা)।
- Session Cookie-র Flag, CSRF, CORS ও Content Security Policy-র ভূমিকা বুঝতে।
- Secret, Dependency ও Log-এর ঝুঁকি সামলাতে, এবং Prototype Pollution চিনতে।
- একটি স্থানীয় **SecureTrip** API ও Portal তৈরি করে প্রতিটি নিয়ন্ত্রণ Test দিয়ে যাচাই করতে।

পূর্বশর্ত

- অধ্যায় ৮ ও ১১: DOM-এ লেখা দেখানো, Form Validation।
- অধ্যায় ১৫: `fetch`, HTTP Status, JSON।
- অধ্যায় ১৭: Regular Expression দিয়ে Validation, Allowlist।
- অধ্যায় ১৯: Map, Cache।
- Node.js 22.5 বা নতুন (`node:sqlite` Module-এর জন্য)।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Website-এর কথা ভাবুন। ভুল একটি লাইনের কারণে একজন গ্রাহক আরেকজনের Booking, ফোন নম্বর ও পাসপোর্টের তথ্য দেখে ফেলতে পারেন; একটি Form-এ লুকানো দামের Field বদলে কেউ ১২,৫০০ টাকার Package ১ টাকায় বুক করতে পারেন; একটি Review-এর ভেতরে লুকানো Script প্রতিটি দর্শকের Browser-এ চলতে পারে। এর কোনোটির জন্য উন্নত Hacking দক্ষতা লাগে না—লাগে শুধু Developer-এর একটি ভুল ধারণা।

নিরাপত্তা তাই একটি শেষ ধাপের "Feature" নয়; প্রতিটি Function লেখার সময়ের একটি অভ্যাস। এই অধ্যায়ের মূল প্রশ্ন সব সময় একটি: **এই তথ্য কোথা থেকে এসেছে, এবং আমি কি একে বিশ্বাস করতে পারি?**

> **সতর্কতা:** এই অধ্যায়ের প্রতিটি দুর্বলতার প্রদর্শন শুধু আপনার নিজের কম্পিউটারে, নিরীহ ইনপুট দিয়ে (যেমন একটি Flag বসানো), চালানোর জন্য। অনুমতি ছাড়া অন্যের System-এ এ ধরনের কিছু চেষ্টা করা অনৈতিক এবং অনেক দেশে অপরাধ। Project-এর Server শুধু `127.0.0.1`-এ চলে এবং Deploy করার জন্য নয়।

এই অধ্যায়ে প্রতিটি বিষয় তিনটি স্তরে আলাদা করে দেখানো হয়েছে:

চিহ্ন	অর্থ
শিক্ষামূলক প্রদর্শন	দুর্বলতা কীভাবে কাজ করে, স্থানীয় ও নিরীহ ইনপুটে
প্রস্তাবিত প্রতিরক্ষা	Code-এ যা করা উচিত; SecureTrip-এ প্রয়োগ ও Test করা
Production-এর প্রয়োজন	বাস্তব System-এ যা লাগবে, কিন্তু এই শিক্ষামূলক Project-এ নেই

২০.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`textContent` বনাম `innerHTML` (অধ্যায় ৮)	XSS কীভাবে চলে, প্রসঙ্গভিত্তিক Encoding, CSP
Form Validation (অধ্যায় ১১)	Server-এ পুনরায় Validation, Allowlist, Trust Boundary

আগে শেখা	এই অধ্যায়ে নতুন
Regex Validation ও ReDoS (অধ্যায় ১৭)	Validation বনাম Encoding-এর আলাদা দায়িত্ব
`fetch` ও HTTP Status (অধ্যায় ১৫)	Cookie-র Flag, CSRF, CORS, Security Header
`localStorage` (অধ্যায় ১৫)	কেন Session Token সেখানে রাখা ঝুঁকিপূর্ণ
—	SQL Injection, Password Hashing, Authorization, Prototype Pollution

২০.২ Threat Model ও Trust Boundary

Threat Model চারটি প্রশ্নের উত্তর: আমরা কী তৈরি করছি? কী ভুল হতে পারে? তা নিয়ে কী করব? আমরা কি যথেষ্ট ভালো করেছি? SecureTrip-এর জন্য:

সম্পদ	কে ক্ষতি করতে পারে	কী ঝুঁকি
গ্রাহকের Trip ও ব্যক্তিগত তথ্য	অন্য গ্রাহক	অন্যের তথ্য দেখা (Authorization-এর ত্রুটি)
দাম	যেকোনো গ্রাহক	Request বদলে কম দাম পাঠানো
Account	বাইরের কেউ	Password অনুমান, Session চুরি
অন্য দর্শকের Browser	ক্ষতিকর ইনপুট দেওয়া কেউ	XSS
Database	যেকোনো ইনপুট	SQL Injection

Trust Boundary হলো যেখানে তথ্য কম বিশ্বস্ত এলাকা থেকে বেশি বিশ্বস্ত এলাকায় ঢোকে। সবচেয়ে গুরুত্বপূর্ণটি: **Browser থেকে Server।** ব্যবহারকারীর Browser পুরোপুরি তাঁর নিয়ন্ত্রণে—তিনি DevTools-এ JavaScript বদলাতে, Form-এর `maxlength` মুছে ফেলতে, অথবা Browser ছাড়াই `curl` দিয়ে যেকোনো Request পাঠাতে পারেন। তাই:

> **মনে রাখুন:** Client-side Validation ব্যবহারকারীর সুবিধার জন্য (তাৎক্ষণিক ভুল দেখানো), নিরাপত্তার জন্য নয়। Server-এ পৌঁছানো প্রতিটি তথ্য পুনরায় যাচাই করতে হবে, যেন Browser-এ কোনো যাচাই নেই।

২০.৩ XSS: অন্যের Browser-এ আপনার Page দিয়ে Script

Cross-Site Scripting (XSS) ঘটে যখন ব্যবহারকারীর দেওয়া লেখা Page-এ HTML বা Script হিসেবে ঢুকে পড়ে। তখন সেই Script আপনার Website-এর নামে, প্রতিটি দর্শকের Browser-এ চলে: তাঁর তথ্য পড়তে পারে, তাঁর নামে Request পাঠাতে পারে। নিচের **শিক্ষামূলক প্রদর্শনে** ক্ষতিকর অংশের বদলে নিরীহ একটি Flag বসানো হয়েছে:

xss-demo.js · JavaScript চলবে: Browser

```
const note = '<img src="data:," onerror="window.imgRan = true">';
const scriptNote = '<script>window.scriptRan = true</' + 'script>';

const unsafe = document.createElement('div');
const safe = document.createElement('div');
document.body.append(unsafe, safe);

// শিক্ষামূলক প্রদর্শন: ব্যবহারকারীর লেখা innerHTML-এ (এভাবে করবেন না)।
unsafe.innerHTML = note + scriptNote;
// প্রস্তাবিত প্রতিরক্ষা: লেখা হিসেবে বসানো।
safe.textContent = note;

setTimeout(() => {
  const img = unsafe.querySelector('img');
  console.log(`innerHTML: img Element তৈরি? ${img !== null}`);
  console.log(`innerHTML: onerror চলেছে? ${window.imgRan === true}`);
  console.log(`innerHTML: script চলেছে? ${window.scriptRan === true}`);
  console.log(`textContent: Element তৈরি? ${safe.children.length > 0}`);
}, 300);
```

Output

```
innerHTML: img Element তৈরি? true
innerHTML: onerror চলেছে? true
innerHTML: script চলেছে? false
textContent: Element তৈরি? false
```

দুটি গুরুত্বপূর্ণ পর্যবেক্ষণ। প্রথমত, `innerHTML`-এ ঢোকানো `<script>` চলেনি (HTML-এর নিয়মে এভাবে যোগ করা Script চলে না), কিন্তু একটি নিরীহ চেহারার `<img>`-এর `onerror` ঠিকই চলেছে—তাই "শুধু `<script>` শব্দটি আটকালেই হবে" ধারণাটি ভুল। দ্বিতীয়ত, `textContent` কোনো Element তৈরি করে না; লেখাটি হুবহু লেখা হিসেবেই দেখায়।

XSS-এর তিন ধরন: **Stored** (ক্ষতিকর লেখা Database-এ সংরক্ষিত হয়ে সবাইকে দেখানো হয়, যেমন Review), **Reflected** (URL-এর অংশ সরাসরি Page-এ ফেরত দেখানো), এবং **DOM-based** (Client-এর JavaScript নিজেই `location.hash` বা অন্য উৎস থেকে নিয়ে `innerHTML`-এ বসায়)। প্রতিরক্ষা তিনটিতেই এক: অবিশ্বস্ত তথ্য কখনো Code বা Markup হিসেবে নয়।

প্রস্তাবিত প্রতিরক্ষা:

- লেখার জন্য `textContent`, Attribute-এর জন্য `setAttribute` বা Property; `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write` অবিশ্বস্ত তথ্যে নয়।
- সত্যিই ব্যবহারকারীর HTML দেখাতে হলে (যেমন Rich Text) একটি পরীক্ষিত Sanitizer Library (যেমন DOMPurify)—নিজের লেখা Regex নয়।

- React, Vue-র মতো Framework স্বয়ংক্রিয়ভাবে Escape করে; কিন্তু `dangerouslySetInnerHTML` বা `v-html` সেই সুরক্ষা বন্ধ করে।
- দ্বিতীয় স্তর হিসেবে Content Security Policy (অনুচ্ছেদ ২০.৯)।

২০.৪ Input Validation বনাম Output Encoding

দুটি আলাদা কাজ, এবং একটি অন্যটির বিকল্প নয়।

- **Input Validation** প্রশ্ন করে: "এই তথ্য কি আমাদের নিয়মে বৈধ?" (যাত্রী ১ থেকে ৮, তারিখ ক্যালেন্ডারে আছে)। Server-এ, **Allowlist** দিয়ে: কী গ্রহণযোগ্য তা বলুন, কী নিষিদ্ধ তার তালিকা নয়।
- **Output Encoding** প্রশ্ন করে: "এই তথ্য যেখানে বসাচ্ছি, সেই প্রসঙ্গে কি এটি শুধু তথ্য হিসেবেই থাকবে?" এটি ঘটে **ব্যবহারের মুহূর্তে**, এবং প্রসঙ্গভেদে আলাদা।

একটি নোট সম্পূর্ণ বৈধ হতে পারে ("আমার বাজেট <১৫০০০ টাকা & দুটি রুম"), তবু HTML-এ Encoding ছাড়া বসালে ভেঙে যাবে। তাই Validation পাস করা তথ্যও Output-এর সময় Encode করতে হয়।

encoding.js · JavaScript — চলবে: Browser Console ও Node.js

```
const HTML_ESCAPES = {
  '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
};
const escapeHtml = (text) =>
  String(text).replace(/[&<>"']/g, (ch) => HTML_ESCAPES[ch]);

function isSafeLink(url) {
  try {
    const { protocol } = new URL(url, 'https://trip.example');
    return ['https:', 'http:'].includes(protocol);
  } catch {
    return false;
  }
}

const name = 'Rafi" onfocus="window.flag=1';
console.log(`<input value="${escapeHtml(name)}">`);
console.log(`<p>${escapeHtml('বাজেট <15000 & দুটি রুম')}</p>`);
console.log(`/search?q=${encodeURIComponent('Cox & Sylhet = সাগর')}`);
console.log(isSafeLink('javascript:alert(1)'), isSafeLink('/trips/7'));
```

Output

```
<input value="Rafi&amp;quot; onfocus=&amp;quot;window.flag=1">
<p>বাজেট &amp;lt;15000 &amp;amp; দুটি রুম</p>
/search?q=Cox%20%26%20Sylhet%20%3D%20%E0%A6%B8%E0%A6%BE%E0%A6%97%E0%A6%B0
false true
```

প্রসঙ্গ	সঠিক উপায়
HTML-এর লেখা	`textContent`; Server-এ Template-এ HTML Escape
HTML Attribute	`setAttribute`; Template-এ উদ্ধৃতিচিহ্নসহ Attribute ও Escape
URL-এর অংশ	`encodeURIComponent`, অথবা `URLSearchParams`
Link-এর `href`	Protocol যাচাই (`javascript:` URL একটি XSS-এর পথ)
SQL	Parameterized Query (অনুচ্ছেদ ২০.৫)
Shell Command	Shell নয়; Argument-এর Array (`execFile`)

`escapeHtml` উদ্ধৃতিচিহ্নসহ Attribute-এর জন্য নিরাপদ, কিন্তু `<script>` Block-এর ভেতরে, CSS-এ বা উদ্ধৃতিচিহ্নহীন Attribute-এ নয়—প্রতিটি প্রসঙ্গের নিজস্ব নিয়ম। এ কারণেই হাতে Encoding-এর চেয়ে নিরাপদ DOM API ও স্বয়ংক্রিয় Escape করা Template System ভালো।

২০.৫ SQL Injection ও Parameterized Query

ব্যবহারকারীর লেখা দিয়ে SQL জোড়া লাগালে লেখাটিই SQL-এর অংশ হয়ে যায়। নিচের **শিক্ষামূলক প্রদর্শন** একটি In-memory Database-এ চলে, যা Program শেষে মুছে যায় (Node.js-এর নিজস্ব `node:sqlite`; চালানোর সময় এটি "Experimental" বলে একটি সতর্কবার্তা দেখায়):

sql-injection.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE trips (id INTEGER, owner TEXT, place TEXT);
  INSERT INTO trips VALUES (1, 'rafi', 'Sylhet'), (2, 'mina', 'Cox'),
    (3, 'sara', 'Bogura');`);

const owner = "rafi' OR '1'='1";

// শিক্ষামূলক প্রদর্শন: লেখা জোড়া লাগানো (এভাবে করবেন না)।
const query = `SELECT id FROM trips WHERE owner = '${owner}'`;
const unsafe = db.prepare(query).all();
// প্রস্তাবিত প্রতিরক্ষা: Parameter; মান কখনো SQL-এর লেখা হয় না।
const safe = db.prepare('SELECT id FROM trips WHERE owner = ?').all(owner);

console.log(query);
console.log('জোড়া লাগানো:', unsafe.map((row) => row.id).join(', '));
console.log('Parameterized:', safe.length, 'টি সারি');
```

Output

```
SELECT id FROM trips WHERE owner = 'rafi' OR '1'='1'
জোড়া লাগানো: 1, 2, 3
Parameterized: 0 টি সারি
```

ইনপুটের `'` চিহ্ন SQL-এর লেখা বন্ধ করে দিয়েছে, এবং `OR '1'='1'` শর্তটি সব সারির জন্য সত্য—শুধু rafi-র নয়, সবার Trip বেরিয়ে এসেছে। Parameter (`?`) ব্যবহারে Database মানটিকে শুধু মান হিসেবে পায়: "rafi' OR '1'='1" নামে কোনো মালিক নেই, তাই শূন্য সারি।

Parameter দিয়ে মান পাঠানো যায়, কিন্তু Table বা Column-এর নাম নয়। ব্যবহারকারী Sort-এর Column বেছে নিলে একটি Allowlist থেকে মেলান (`{ date: 'start_date', price: 'total' }`), কখনো সরাসরি বসাবেন না। ORM বা Query Builder সাধারণত Parameter ব্যবহার করে, কিন্তু তাদের "raw" Query-তেও একই নিয়ম। একই নীতি NoSQL-এ: MongoDB-তে `{ email: req.body.email }`-এ Body থেকে `{ "$ne": null }` এলে তা একটি Operator হয়ে যায়; তাই ধরন যাচাই (`typeof email === 'string'`)।

২০.৬ Authentication বনাম Authorization

- **Authentication (AuthN):** আপনি কে? (Login, Session)
- **Authorization (AuthZ):** আপনি কি **এই নির্দিষ্ট কাজটি, এই নির্দিষ্ট Object-এ** করতে পারেন?

সবচেয়ে সাধারণ ত্রুটি দ্বিতীয়টিতে: Login যাচাই করা হলো, কিন্তু Trip-টি এই ব্যবহারকারীর কি না তা নয়। তখন `/api/trips/T2`-এর জায়গায় `/api/trips/T3` লিখলেই অন্যের তথ্য—এর নাম **IDOR** (Insecure Direct Object Reference), এবং OWASP-এর তালিকায় এক নম্বর ঝুঁকি "Broken Access Control"-এর অংশ।

authz.js · JavaScript — চলবে: Browser Console ও Node.js

```
const trips = [
  { id: 'T1', ownerId: 1, place: 'Sylhet' },
  { id: 'T2', ownerId: 2, place: 'Cox' },
];

function readTrip(user, id) {
  if (!user) {
    return '401 Login প্রয়োজন';
  }
  const trip = trips.find((item) => item.id === id);
  if (!trip || (trip.ownerId !== user.id && user.role !== 'admin')) {
    return '404 পাওয়া যায়নি';
  }
  return `200 ${trip.place}`;
}

const rafi = { id: 1, role: 'customer' };
const admin = { id: 9, role: 'admin' };
```

```
console.log(readTrip(null, 'T1'));
console.log(['T1', 'T2', 'T9'].map((id) => readTrip(rafi, id)).join(' | '));
console.log(readTrip(admin, 'T2'));
```

Output

```
401 Login প্রয়োজন
200 Sylhet | 404 পাওয়া যায়নি | 404 পাওয়া যায়নি
200 Cox
```

অন্যের Trip (`T2`) এবং অস্তিত্বহীন Trip (`T9`)—দুটিতেই একই উত্তর। 403 দিলে আক্রমণকারী জানতেন T2 আছে; 404 অস্তিত্বও লুকায়। আরেকটি নিয়ম: **মালিকের পরিচয় আসে Session থেকে, Request Body থেকে কখনো নয়।** Body-তে `ownerId: 2` পাঠালেই কেউ অন্যের নামে Trip তৈরি করতে পারলে সেটি একই ত্রুটি। UI-তে Button লুকানো Authorization নয়; প্রতিটি Server Endpoint-এ যাচাই।

২০.৭ Password: Hash, Salt এবং ধ্রুব-সময়ের তুলনা

Password কখনো সরাসরি সংরক্ষণ নয়, এবং SHA-256-এর মতো দ্রুত Hash-ও নয়: দ্রুত Hash একটি সাধারণ GPU-তে প্রতি সেকেন্ডে শত কোটি অনুমান যাচাই করতে দেয়। দরকার **ইচ্ছাকৃতভাবে ধীর ও Memory-নির্ভর** Password Hash: **scrypt** (Node.js-এ অন্তর্নির্মিত), Argon2id বা bcrypt। প্রতিটি Password-এর আলাদা এলোমেলো **Salt** নিশ্চিত করে যে দুজনের একই Password থাকলেও Hash আলাদা, এবং আগে থেকে হিসাব করা তালিকা (Rainbow Table) কাজ করে না।

password-demo.mjs · JavaScript চলবে: Node.js

```
import { hashPassword, verifyPassword }
  from '../project/secure-trip/src/passwords.mjs';

const stored = await hashPassword('Bogura-2026');
const [scheme, salt, key] = stored.split('$');
console.log(scheme, `salt ${salt.length / 2} byte`, `key ${key.length / 2} byte`);

console.log(await verifyPassword('Bogura-2026', stored));
console.log(await verifyPassword('bogura-2026', stored));
console.log(stored === await hashPassword('Bogura-2026'));
```

Output

```
scrypt salt 16 byte key 32 byte
true
false
false
```

শেষ লাইনটি লক্ষ করুন: একই Password-এর দ্বিতীয় Hash ভিন্ন, কারণ Salt নতুন। যাচাইয়ের সময় সংরক্ষিত Salt দিয়ে আবার হিসাব করে তুলনা হয়, এবং তুলনা `timingSafeEqual` দিয়ে: সাধারণ `===` প্রথম অমিল Byte-এ থেমে যায়, তাই উত্তরের সময় থেকে তথ্য ফাঁস হতে পারে। Project-এর Code অনুচ্ছেদ ২০.১২-এ।

Production-এর প্রয়োজন: scrypt বা Argon2id-এর Parameter (Memory, পুনরাবৃত্তি) সময়ের সঙ্গে বাড়ানো ও পুরোনো Hash নতুন Login-এ হালনাগাদ; ফাঁস হওয়া Password-এর তালিকার সঙ্গে মিলিয়ে দেখা; Multi-factor Authentication; নিরাপদ Password Reset (এককালীন, স্বল্পমেয়াদি Token)।

২০.৮ Session, Cookie এবং CSRF

Login-এর পর Server একটি দীর্ঘ এলোমেলো **Session ID** তৈরি করে Cookie-তে পাঠায়; Browser পরের প্রতিটি Request-এ তা ফেরত পাঠায়। Cookie-র Flag-গুলোই তার নিরাপত্তা:

Flag	কাজ
`HttpOnly`	JavaScript (`document.cookie`) পড়তে পারে না; XSS হলেও Session ID চুরি কঠিন
`Secure`	শুধু HTTPS-এ পাঠানো হয়
`SameSite=Strict` বা `Lax`	অন্য Site থেকে শুরু হওয়া Request-এ Cookie পাঠানো হয় না (Lax-এ সাধারণ Link-এ পাঠায়)
`Path=/` , মেয়াদ	কোথায় ও কতদিন বৈধ

Session Token `localStorage`-এ রাখলে যেকোনো XSS তা পড়ে নিতে পারে; `HttpOnly` Cookie-তে পারে না। (XSS থাকলে আক্রমণকারী তবু Page থেকেই Request পাঠাতে পারেন— `HttpOnly` ক্ষতি কমায়, XSS-এর সমাধান নয়।)

CSRF (Cross-Site Request Forgery): Browser Cookie স্বয়ংক্রিয়ভাবে পাঠায়, এমনকি অন্য একটি Website থেকে শুরু হওয়া Request-এও। তাই একটি ক্ষতিকর Page লুকিয়ে আপনার Site-এ একটি Form Submit করালে, Login থাকা ব্যবহারকারীর Cookie-সহ তা পৌঁছায়। প্রতিরক্ষার স্তর:

১. `SameSite` **Cookie**—আধুনিক Browser-এ সবচেয়ে বড় সুরক্ষা।
২. **CSRF Token:** Session-এর সঙ্গে যুক্ত একটি গোপন মান, যা Page-এর JavaScript একটি Header-এ পাঠায়। অন্য Site সেই মান পড়তে পারে না (Same-Origin Policy), তাই পাঠাতেও পারে না।
৩. `Origin` **Header যাচাই:** অবস্থা বদলানো Request কোন Site থেকে এসেছে।
৪. অবস্থা বদলানো কাজ কখনো `GET`-এ নয়।

SecureTrip তিনটিই করে (অনুচ্ছেদ ২০.১২)। এখানে একটি সতর্কতা: এই বইয়ের Test-গুলো Token ও Origin যাচাই প্রমাণ করে, কিন্তু Browser আসলে অন্য Site থেকে `SameSite` Cookie পাঠায় কি না, তা দুটি আলাদা Domain ছাড়া পরীক্ষা করা যায় না—এর জন্য আমরা Browser-এর নথিভুক্ত আচরণের ওপর নির্ভর করছি।

২০.৯ CORS, Same-Origin Policy এবং Content Security Policy

Same-Origin Policy: একটি Page-এর JavaScript অন্য Origin-এর (Protocol + Host + Port) Response পড়তে পারে না। **CORS** এই নিয়ম শিথিল করার উপায়: Server `Access-Control-Allow-Origin` Header দিয়ে জানায় কোন Origin তার Response পড়তে পারবে।

> **সতর্কতা:** CORS একটি নিরাপত্তা-নিয়ন্ত্রণ নয় যা আপনার API রক্ষা করে; এটি Browser-কে বলে কাদের **পড়তে দেওয়া** হবে। `curl` বা অন্য Server CORS মানে না। `Access-Control-Allow-Origin: *` একটি Public API-র জন্য ঠিক, কিন্তু Cookie-সহ API-তে কখনো Request-এর `Origin` হুবহু ফেরত পাঠানো নয়। Authentication ও Authorization সব সময় আলাদা।

SecureTrip-এর Page ও API একই Origin-এ, তাই কোনো CORS Header নেই—সবচেয়ে নিরাপদ অবস্থা।

Content Security Policy (CSP) একটি Response Header, যা Browser-কে বলে কোন উৎসের Script, Style ইত্যাদি চলতে পারবে। `script-src 'self'` মানে শুধু একই Origin-এর Script File; Page-এর ভেতরে লেখা Inline Script ও `onerror="..."`-এর মতো Attribute চলবে না। ফলে কোনোভাবে XSS ঢুকে পড়লেও তা চলার সম্ভাবনা অনেক কমে। Chrome-এ যাচাই করুন: Page-এ একটি `<meta>` CSP, একটি Inline Script, এবং একটি বাইরের Script File।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta http-equiv="Content-Security-Policy" content="script-src 'self'">
  <title>CSP পরীক্ষা</title>
</head>
<body>
  <script>window.inlineRan = true;</script>
  <script src="check.js"></script>
</body>
</html>
```

check.js · JavaScript চলবে: Browser

```
const holder = document.createElement('div');
holder.innerHTML = '<img src="data:," onerror="window.handlerRan = true">';
document.body.append(holder);

setTimeout(() => {
  console.log(`Inline script চলেছে? ${window.inlineRan === true}`);
  console.log(`onerror Attribute চলেছে? ${window.handlerRan === true}`);
  console.log('বাইরের File (check.js) চলেছে? true');
}, 200);
```

Error Message

```
Inline script চলেছে? false
onerror Attribute চলেছে? false
বাইরের File (check.js) চলেছে? true
Executing inline script violates the following Content Security Policy
Executing inline event handler violates the following Content Security Policy
```

শেষ দুটি লাইন Chrome-এর নিজের CSP বার্তার শুরু (পুরো বার্তায় কোন Directive আটকেছে এবং কীভাবে অনুমতি দেওয়া যায়, তাও লেখা থাকে)। অনুচ্ছেদ ২০.৩-এ যে `onerror` চলেছিল, CSP-র অধীনে একই Code চলেনি। বাস্তবে CSP একটি **Header** হিসেবে পাঠানো ভালো (`<meta>` কিছু Directive সমর্থন করে না); SecureTrip তাই করে। অন্যান্য দরকারি Header:

Header	কাজ
`Strict-Transport-Security`	শুধু HTTPS (Production-এ, HTTPS চালু থাকলে)
`X-Content-Type-Options: nosniff`	Browser যেন File-এর ধরন অনুমান না করে
`Referrer-Policy`	অন্য Site-এ যাওয়ার সময় URL কতটা জানানো হবে
CSP-র `frame-ancestors 'none'`	অন্য Site আপনার Page Frame-এ বসাতে পারবে না (Clickjacking)

২০.১০ Prototype Pollution

JavaScript-এ প্রায় প্রতিটি Object `Object.prototype` থেকে Property উত্তরাধিকারে পায় (অধ্যায় ১৩)। কোনো Code যদি ব্যবহারকারীর JSON-এর Key ধরে Object-এ লেখে, এবং Key হয় `__proto__`, তবে লেখাটি সব Object-এর সাধারণ Prototype-এ পড়ে—**Prototype Pollution**। নিচের **শিক্ষামূলক প্রদর্শন** একটি নিজস্ব Process-এ চলে, এবং শেষে দূষণ মুছে ফেলে:

pollution.js · JavaScript চলবে: Node.js

```
function naiveMerge(target, source) {
  for (const key of Object.keys(source)) {
```

```
    const value = source[key];
    if (typeof value === 'object' && value !== null) {
      target[key] ??= {};
      naiveMerge(target[key], value);
    } else {
      target[key] = value;
    }
  }
  return target;
}

const body = JSON.parse('{ "theme": "dark", "__proto__": { "isAdmin": true } }');
console.log(Object.keys(body));

naiveMerge({}, body);
const unrelated = {};
console.log('সম্পর্কহীন Object-এ isAdmin:', unrelated.isAdmin);
delete Object.prototype.isAdmin;

const BLOCKED = new Set(['__proto__', 'constructor', 'prototype']);
function safeMerge(target, source) {
  for (const [key, value] of Object.entries(source)) {
    if (BLOCKED.has(key)) {
      continue;
    }
    target[key] = typeof value === 'object' && value !== null
      ? safeMerge(Object.hasOwn(target, key) ? target[key] : {}, value)
      : value;
  }
  return target;
}

console.log(safeMerge({}, body), ({}).isAdmin);
```

Output

```
[ 'theme', '__proto__' ]
সম্পর্কহীন Object-এ isAdmin: true
{ theme: 'dark' } undefined
```

`JSON.parse` নিজে নিরাপদ: সে `__proto__` নামে একটি সাধারণ Key তৈরি করে। বিপদ আসে `target['__proto__']` পড়ায়, যা আসলে `Object.prototype`, এবং তার ভেতরে লেখায়। এরপর Program-এর **যেকোনো** Object—এমনকি যেটির সঙ্গে ব্যবহারকারীর কোনো সম্পর্ক নেই—`isAdmin: true` দেখায়; `if (user.isAdmin)`-এর মতো Code বিপজ্জনক হয়ে ওঠে।

প্রতিরক্ষা: বিপজ্জনক Key বাদ; Dictionary হিসেবে `Map` বা `Object.create(null)` (যার Prototype নেই); Property পড়ার সময় `Object.hasOwn`; এবং সবচেয়ে ভালো—SecureTrip-এর মতো, Body থেকে শুধু নির্দিষ্ট Field নেওয়া (Allowlist), পুরো Object মেশানো নয়। পুরোনো Deep-merge Library-গুলোতে এ ধরনের ত্রুটির ইতিহাস আছে, যা পরের অনুচ্ছেদের বিষয়।

২০.১১ Secret, Dependency এবং Log

Secret (API Key, Database Password, Session-এর Signing Key):

- Code-এ বা Git-এ কখনো নয়; Environment Variable বা Secret Manager থেকে। Git-এ একবার Commit হলে পরে মুছলেও History-তে থেকে যায়—তখন Secret বদলাতে (Rotate) হবে।
- Browser-এর JavaScript-এ পাঠানো কোনো কিছুই গোপন নয়: Bundle-এর ভেতরের API Key যে কেউ দেখতে পারে।
- Program শুরুতেই প্রয়োজনীয় Secret আছে কি না যাচাই করে স্পষ্ট বার্তায় থেমে যাওয়া, পরে হঠাৎ ব্যর্থ হওয়ার চেয়ে ভালো।

secrets.js · JavaScript — চলবে: Browser Console ও Node.js

```
function requireSecret(name, env) {
  const value = env[name];
  if (typeof value !== 'string' || value.length < 32) {
    throw new Error(`${name} সেট করা নেই বা ৩২ অক্ষরের কম`);
  }
  return value;
}

try {
  requireSecret('SESSION_SECRET', {});
} catch (error) {
  console.log(error.message);
}
const env = { SESSION_SECRET: 'k'.repeat(48) };
console.log(requireSecret('SESSION_SECRET', env).length);
```

Output

```
SESSION_SECRET সেট করা নেই বা ৩২ অক্ষরের কম
48
```

Dependency: একটি সাধারণ Project শত শত npm Package ব্যবহার করে, এবং প্রতিটির Code আপনার Server-এ আপনার অধিকারে চলে। `npm audit` জানা দুর্বলতা দেখায়; `package-lock.json` Commit করে সংস্করণ স্থির রাখুন; নতুন Package যোগের আগে তার রক্ষণাবেক্ষণ ও জনপ্রিয়তা দেখুন, এবং নামের বানান যাচাই করুন (নকল নামের ক্ষতিকর Package বা Typosquatting আছে)। SecureTrip কোনো বাইরের Package ব্যবহার করে না—শুধু Node.js-এর অন্তর্নির্মিত Module।

Log: নিরাপত্তার ঘটনা (ব্যর্থ Login, প্রত্যাখ্যাত Request) Log করুন, কিন্তু Password, Token, Cookie বা পূর্ণ Card নম্বর কখনো নয়। SecureTrip-এর `redact` Log-এ যাওয়ার আগে সংবেদনশীল Key মুছে দেয়। ব্যবহারকারীকে Error-এর বিস্তারিত (Stack Trace, SQL) দেখাবেন না; তা শুধু Server-এর Log-এ।

২০.১২ Project: SecureTrip

SecureTrip একটি স্থানীয় Trip Request Portal: Node.js-এর `node:http` দিয়ে একটি ছোট API, `node:sqlite`-এর In-memory Database, এবং একটি সাধারণ Browser UI। গ্রাহক Login করে Package, যাত্রীসংখ্যা ও তারিখ দিয়ে Trip-এর অনুরোধ পাঠান এবং নিজের অনুরোধগুলো দেখেন; Admin সব দেখেন। কোনো বাইরের Package নেই।

> **সতর্কতা:** SecureTrip একটি **শিক্ষামূলক Demo**। এটি শুধু `127.0.0.1`-এ চলে, তথ্য Memory-তে থাকে এবং বন্ধ করলে মুছে যায়, এবং এটি Production-এ ব্যবহারের উপযুক্ত নয়। প্রতিটি নিয়ন্ত্রণ Test দিয়ে যাচাই করা হয়েছে, কিন্তু কোনো স্বাধীন Security Review বা Penetration Test হয়নি। নিচের তালিকায় দেখুন বাস্তব System-এ আর কী লাগবে।

নিয়ন্ত্রণ	SecureTrip-এ	Production-এর প্রয়োজন
Password	scrypt, Salt, `timingSafeEqual`	Argon2id বা শক্তিশালী Parameter, MFA, Reset-এর ব্যবস্থা
Session	এলোমেলো ID, HttpOnly, SameSite=Strict, ৩০ মিনিট	`Secure` (HTTPS), স্থায়ী Session Store, Login-এ নতুন ID
CSRF	Token Header ও Origin যাচাই	একই, সব অবস্থা-বদলানো Route-এ
Authorization	প্রতিটি Trip-এ মালিক বা Admin	Role ও অনুমতির পূর্ণ নকশা, Audit Log
দাম	Server-এর Catalog	Database-এর দাম, Transaction, Payment Provider
Validation	Server-এ Allowlist, Body-র আকার ও ধরন	একই, প্রতিটি Endpoint-এ Schema
SQL	Parameterized Query	একই; Database ব্যবহারকারীর ন্যূনতম অধিকার
Login-এর সীমা	Email প্রতি ১৫ মিনিটে ৫ বার (Memory-তে)	IP ও Account-ভিত্তিক, ভাগ করা Store, CAPTCHA
Header	CSP, nosniff, Referrer-Policy, frame-ancestors	HSTS, HTTPS, Reverse Proxy-র নিয়ম
Log	Redaction, সাধারণ Error বার্তা	কেন্দ্রীয় Log, সতর্কতা, সংরক্ষণের নীতি
Dependency	নেই	`npm audit`, Lock File, নিয়মিত হালনাগাদ

File-এর কাঠামো

Folder-এর কাঠামো

```
secure-trip/
├── server.mjs            চালু করে (127.0.0.1:3020)
├── src/
│   ├── start.mjs         সব অংশ জুড়ে Server তৈরি
│   ├── app.mjs           Route, AuthN, AuthZ, CSRF
│   ├── http.mjs          Security Header, JSON পড়া, Cookie
│   ├── passwords.mjs     scrypt Hash ও যাচাই
│   ├── sessions.mjs      Session ও CSRF Token
│   ├── validate.mjs      Trip-এর Allowlist Validation, Catalog
│   ├── db.mjs            SQLite, Parameterized Query
│   ├── limiter.mjs       ব্যর্থ Login-এর সীমা
│   └── log.mjs           Redaction-সহ Log
├── public/               index.html, app.mjs, style.css
└── tests/                units.test.mjs, api.test.mjs
```

src/passwords.mjs ও src/sessions.mjs

passwords.mjs · JavaScript — চলবে: Node.js

```
import { scrypt, randomBytes, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';

const scryptAsync = promisify(scrypt);
const KEY_LENGTH = 32;

// সংরক্ষণের রূপ "scrypt$<salt>$<hash>"; প্রতিটি Password-এর নিজস্ব Salt।
export async function hashPassword(password) {
  const salt = randomBytes(16);
  const key = await scryptAsync(password.normalize('NFC'), salt, KEY_LENGTH);
  return `scrypt$${salt.toString('hex')}$${key.toString('hex')}`;
}

export async function verifyPassword(password, stored) {
  const [scheme, saltHex, keyHex] = String(stored).split('$');
  const expected = Buffer.from(keyHex ?? '', 'hex');
  if (scheme !== 'scrypt' || !saltHex || expected.length !== KEY_LENGTH) {
    return false;
  }
  const salt = Buffer.from(saltHex, 'hex');
  const actual = await scryptAsync(password.normalize('NFC'), salt, KEY_LENGTH);
  return timingSafeEqual(actual, expected);
}
```

sessions.mjs · JavaScript — চলবে: Node.js

```
import { randomBytes, timingSafeEqual } from 'node:crypto';

const token = () => randomBytes(32).toString('base64url');

// দৈর্ঘ্য আলাদা হলে সঙ্গে সঙ্গে false; সমান হলে ধ্রুব-সময়ের তুলনা।
export function safeEqual(a, b) {
  const left = Buffer.from(String(a));
  const right = Buffer.from(String(b));
  return left.length === right.length && timingSafeEqual(left, right);
}

export function createSessions(options = {}) {
  const { ttlMs = 30 * 60 * 1000, now = Date.now } = options;
  const sessions = new Map();
  return {
    create(user) {
      const id = token();
      const csrf = token();
      const expires = now() + ttlMs;
      const { id: userId, email, role } = user;
      sessions.set(id, { userId, email, role, csrf, expires });
      return { id, csrf };
    },
    get(id) {
      const session = sessions.get(id);
      if (!session) {
        return null;
      }
      if (now() >= session.expires) {
        sessions.delete(id);
        return null;
      }
      return session;
    },
    destroy(id) {
      sessions.delete(id);
    },
  };
}
```

- **`verifyPassword`** সংরক্ষিত মান বিকৃত হলে Exception নয়, `false` দেয়; Key-এর দৈর্ঘ্য না মিললে `timingSafeEqual` চালানোর আগেই ফেরে (এই Function সমান দৈর্ঘ্য চায়)।
- **`normalize('NFC')`** : বাংলা বা অন্য ভাষার একই দেখতে লেখা ভিন্ন Unicode ক্রমে আসতে পারে (অধ্যায় ১৭); Normalize না করলে সঠিক Password-ও না মিলতে পারে।

- **Session ID ও CSRF Token** দুটিই `randomBytes(32)` —২৫৬ Bit এলোমেলো, অনুমান অসম্ভব। `Math.random()` নিরাপত্তার কাজে কখনো নয়।
- মেয়াদোত্তীর্ণ Session পড়ার সময়ই মুছে যায়।

src/validate.mjs

validate.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// দাম আসে Server-এর Catalog থেকে; Client-এর পাঠানো দাম কখনো নয়।
export const CATALOG = Object.freeze({
  'cox-3d': Object.freeze({ name: "Cox's Bazar, ৩ দিন", price: 12500 }),
  'sylhet-2d': Object.freeze({ name: 'Sylhet, ২ দিন', price: 8900 }),
  'sundarbans-3d': Object.freeze({ name: 'Sundarbans, ৩ দিন', price: 15800 }),
});

const DATE = /^\d{4}-\d{2}-\d{2}$/;
const DAY = 24 * 60 * 60 * 1000;

function checkDate(text, today) {
  if (typeof text !== 'string' || !DATE.test(text)) {
    return 'তারিখ YYYY-MM-DD আকারে দিন';
  }
  const date = new Date(`${text}T00:00:00Z`);
  if (date.toISOString().slice(0, 10) !== text) {
    return 'তারিখটি ক্যালেন্ডারে নেই';
  }
  const days = (date - Date.parse(today.toISOString().slice(0, 10))) / DAY;
  return days >= 1 && days <= 365 ? null : 'আগামীকাল থেকে এক বছরের মধ্যে';
}

// ...
```

validate.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function validateTripRequest(body, today = new Date()) {
  if (typeof body !== 'object' || body === null || Array.isArray(body)) {
    return { ok: false, errors: { body: 'একটি JSON Object পাঠান' } };
  }
  const { packageId, travelers, startDate, note = '' } = body;
  const errors = {};
  if (typeof packageId !== 'string' || !Object.hasOwn(CATALOG, packageId)) {
    errors.packageId = 'অজানা Package';
  }
  if (!Number.isInteger(travelers) || travelers < 1 || travelers > 8) {
    errors.travelers = 'যাত্রী ১ থেকে ৮ জন';
  }
```

```
  const dateError = checkDate(startDate, today);
  if (dateError) {
    errors.startDate = dateError;
  }
  if (typeof note !== 'string' || note.length > 200) {
    errors.note = 'নোট সর্বোচ্চ ২০০ অক্ষরের লেখা';
  }
  if (Object.keys(errors).length > 0) {
    return { ok: false, errors };
  }
  const total = CATALOG[packageId].price * travelers;
  return { ok: true, value: { packageId, travelers, startDate, note, total } };
}
```

এখানে তিনটি নিরাপত্তার সিদ্ধান্ত। দাম `CATALOG` থেকে; Body-তে `total` থাকলেও তা পড়াই হয় না। Body থেকে শুধু চারটি নামের Field নেওয়া হয় (Allowlist), তাই `ownerId`, `isAdmin` বা `__proto__` কোনো প্রভাব ফেলে না। এবং `Object.hasOwn(CATALOG, packageId)`: `packageId` যদি `"toString"` হয়, সাধারণ `CATALOG[packageId]` উত্তরাধিকারের একটি Function ফেরত দিত।

src/db.mjs

db.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const TRIP = `SELECT id, owner_id AS ownerId, package_id AS packageId,
  travelers, start_date AS startDate, note, total FROM trips`;

export function createDatabase(file = ':memory:') {
  const db = new DatabaseSync(file);
  db.exec(`
    CREATE TABLE users (
      id INTEGER PRIMARY KEY, email TEXT UNIQUE NOT NULL,
      hash TEXT NOT NULL,
      role TEXT NOT NULL CHECK (role IN ('customer', 'admin')));
    CREATE TABLE trips (
      id TEXT PRIMARY KEY, owner_id INTEGER NOT NULL REFERENCES users (id),
      package_id TEXT NOT NULL, travelers INTEGER NOT NULL,
      start_date TEXT NOT NULL, note TEXT NOT NULL, total INTEGER NOT NULL);
  `);
  // প্রতিটি Query Parameterized: মান আলাদা যায়, SQL-এর লেখার অংশ হয় না।
  const sql = {
    addUser: db.prepare('INSERT INTO users (email, hash, role) VALUES (?, ?, ?)'),
    userByEmail: db.prepare('SELECT * FROM users WHERE email = ?'),
    addTrip: db.prepare('INSERT INTO trips VALUES (?, ?, ?, ?, ?, ?, ?)'),
    tripById: db.prepare(`${TRIP} WHERE id = ?`),
    tripsOf: db.prepare(`${TRIP} WHERE owner_id = ? ORDER BY rowid`),
    allTrips: db.prepare(`${TRIP} ORDER BY rowid`),
```

```
  };
  const plain = (row) => (row ? { ...row } : null);
  return {
    addUser: (email, hash, role) =>
      Number(sql.addUser.run(email, hash, role).lastInsertRowid),
    userByEmail: (email) => plain(sql.userByEmail.get(email)),
    addTrip: (t) => sql.addTrip.run(t.id, t.ownerId, t.packageId,
      t.travelers, t.startDate, t.note, t.total),
    tripById: (id) => plain(sql.tripById.get(id)),
    tripsFor: (user) => (user.role === 'admin'
      ? sql.allTrips.all()
      : sql.tripsOf.all(user.id)).map(plain),
  };
}
```

প্রতিটি Query আগেই `prepare` করা, এবং প্রতিটি মান `?`-এর মাধ্যমে। Template String (`${TRIP}`) শুধু আমাদের নিজের স্থির লেখা জোড়ে, কখনো ব্যবহারকারীর তথ্য নয়। `tripsFor` Authorization-এর নিয়ম মেনে Query বাছে: গ্রাহক শুধু নিজেরগুলো। `plain` SQLite-এর Prototype-হীন Row-কে সাধারণ Object-এ রূপান্তর করে।

src/http.mjs

http.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
export const SECURITY_HEADERS = Object.freeze({
  'content-security-policy': [
    "default-src 'self'", "script-src 'self'", "object-src 'none'",
    "base-uri 'none'", "frame-ancestors 'none'", "form-action 'self'",
  ].join('; '),
  'x-content-type-options': 'nosniff',
  'referrer-policy': 'no-referrer',
  'cross-origin-opener-policy': 'same-origin',
});

export class HttpError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

// শুধু application/json, সর্বোচ্চ limit Byte। বড় হলে বাকিটা পড়ে ফেলে দেয়
// (Demo-র সরলতার জন্য; Production-এ Connection বন্ধ করাই ভালো)।
export async function readJson(req, limit = 10_000) {
  if (!(req.headers['content-type'] ?? '').startsWith('application/json')) {
    throw new HttpError(415, 'Content-Type application/json হতে হবে');
  }
  const chunks = [];
  let size = 0;
```

```
  for await (const chunk of req) {
    size += chunk.length;
    if (size <= limit) {
      chunks.push(chunk);
    }
  }
  if (size > limit) {
    throw new HttpError(413, 'অনুরোধ খুব বড়');
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new HttpError(400, 'অবৈধ JSON');
  }
}

// ...
```

http.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function send(res, status, body, headers = {}) {
  res.writeHead(status, { ...SECURITY_HEADERS, ...headers });
  res.end(body);
}

export const sendJson = (res, status, data, headers = {}) =>
  send(res, status, JSON.stringify(data), {
    'content-type': 'application/json; charset=utf-8',
    'cache-control': 'no-store',
    ...headers,
  });

export function parseCookies(header = '') {
  const cookies = Object.create(null);
  for (const part of header.split(';')) {
    const index = part.indexOf('=');
    if (index > 0) {
      cookies[part.slice(0, index).trim()] = part.slice(index + 1).trim();
    }
  }
  return cookies;
}
```

`readJson` তিনটি জিনিস আটকায়: ভুল `Content-Type` (415), বড় Body (413; Memory ভরিয়ে Server বসিয়ে দেওয়ার চেষ্টা), ভাঙা JSON (400)। `Content-Type: application/json` বাধ্যতামূলক করাও CSRF-এর বিরুদ্ধে একটি বাড়তি স্তর: একটি সাধারণ HTML Form এই ধরন পাঠাতে পারে না। প্রতিটি Response-এ `SECURITY_HEADERS`,

এবং API-র Response-এ `no-store`, যাতে ব্যক্তিগত তথ্য কোনো Cache-এ না থাকে।

src/app.mjs

app.mjs · JavaScript অংশ ৬ / 8 (Part 1 of 4) চলবে: Node.js

```
import { readFile } from 'node:fs/promises';
import { randomUUID } from 'node:crypto';
import { HttpError, readJson, send, sendJson, parseCookies } from './http.mjs';
import { verifyPassword } from './passwords.mjs';
import { safeEqual } from './sessions.mjs';
import { validateTripRequest } from './validate.mjs';

// শুধু এই তিনটি File; URL থেকে File-এর পথ তৈরি হয় না (Path Traversal নেই)।
const STATIC = {
  '/': ['index.html', 'text/html; charset=utf-8'],
  '/app.mjs': ['app.mjs', 'text/javascript; charset=utf-8'],
  '/style.css': ['style.css', 'text/css; charset=utf-8'],
};
const PUBLIC = new URL('../public/', import.meta.url);

export function createApp(deps) {
  const { db, sessions, limiter, log, dummyHash, secureCookie = false } = deps;
  const secure = secureCookie ? '; Secure' : '';
  const flags = `HttpOnly; SameSite=Strict; Path=/${secure}`;

  function requireUser(req, { csrf = false } = {}) {
    const sid = parseCookies(req.headers.cookie).sid;
    const session = sid ? sessions.get(sid) : null;
    if (!session) {
      throw new HttpError(401, 'Login প্রয়োজন');
    }
    if (csrf) {
      const origin = req.headers.origin;
      if (origin && origin !== `http://${req.headers.host}`) {
        throw new HttpError(403, 'এই Origin থেকে অনুরোধ গ্রহণযোগ্য নয়');
      }
      if (!safeEqual(req.headers['x-csrf-token'] ?? '', session.csrf)) {
        throw new HttpError(403, 'CSRF Token মেলেনি');
      }
    }
    return { sid, session, user: { id: session.userId, role: session.role } };
  }

// ...
```

`requireUser` প্রতিটি সুরক্ষিত Route-এর প্রবেশদ্বার: Cookie থেকে Session, না পেলে 401। অবস্থা বদলানো Route-এ (`csrf: true`) আরও দুটি যাচাই: `Origin` থাকলে তা আমাদের নিজের হতে হবে, এবং Header-এর CSRF Token Session-এরটির সঙ্গে ধ্রুব-সময়ে মিলতে হবে। Static File-এর জন্য একটি স্থির তালিকা: URL-এর কোনো অংশ দিয়ে File-এর পথ তৈরি হয় না, তাই `/../server.mjs`-এর মতো Path Traversal-এর সুযোগ নেই।

app.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

  async function login(req, res) {
    const { email, password } = await readJson(req);
    if (typeof email !== 'string' || typeof password !== 'string') {
      throw new HttpError(400, 'email ও password লেখা হতে হবে');
    }
    const key = email.trim().toLowerCase();
    if (limiter.blocked(key)) {
      throw new HttpError(429, 'অনেকবার ব্যর্থ; কিছুক্ষণ পরে চেষ্টা করুন');
    }
    const user = db.userByEmail(key);
    // ব্যবহারকারী না থাকলেও একই হিসাব, যাতে সময় দেখে Email অনুমান করা না যায়।
    const valid = await verifyPassword(password, user?.hash ?? dummyHash);
    if (!user || !valid) {
      limiter.fail(key);
      log('login.failed', { email: key });
      throw new HttpError(401, 'Email বা Password ভুল');
    }
    limiter.reset(key);
    const { id, csrf } = sessions.create(user);
    log('login.ok', { userId: user.id });
    sendJson(res, 200, { email: user.email, role: user.role, csrf },
      { 'set-cookie': `sid=${id}; ${flags}` });
  }

  async function createTrip(req, res) {
    const { user } = requireUser(req, { csrf: true });
    const result = validateTripRequest(await readJson(req));
    if (!result.ok) {
      sendJson(res, 422, { errors: result.errors });
      return;
    }
    // মালিক আসে Session থেকে; Body-র ownerId বা total গ্রহণ করা হয় না।
    const trip = { id: randomUUID(), ownerId: user.id, ...result.value };
    db.addTrip(trip);
    log('trip.created', { tripId: trip.id, userId: user.id });
    sendJson(res, 201, { trip });
  }

// ...
```

Login-এ চারটি সূক্ষ্মতা। ভুল Password ও অজানা Email—দুটিতেই হুবহু একই বার্তা ও Status, যাতে কোন Email নিবন্ধিত তা জানা না যায়। অজানা Email-এও একটি Dummy Hash-এর সঙ্গে যাচাই চলে, যাতে উত্তরের সময়ও একই রকম থাকে। Limiter একটি Email-এ পাঁচবার ব্যর্থতার পর Password অনুমান থামায়। এবং Log-এ Email ও User ID যায়, Password বা Token কখনো নয়।

app.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  function readTrip(req, res, id) {
    const { user } = requireUser(req);
    const trip = db.tripById(id);
    // অন্যের Trip-এর জন্যও 404, 403 নয়: অস্তিত্বও প্রকাশ করা হয় না।
    if (!trip || (trip.ownerId !== user.id && user.role !== 'admin')) {
      throw new HttpError(404, 'পাওয়া যায়নি');
    }
    sendJson(res, 200, { trip });
  }

  async function route(req, res, pathname) {
    const key = `${req.method} ${pathname}`;
    const tripId = /^\/api\/trips\/([\w-]{1,64})$/.exec(pathname)?.[1];
    if (req.method === 'GET' && Object.hasOwn(STATIC, pathname)) {
      const [file, type] = STATIC[pathname];
      const body = await readFile(new URL(file, PUBLIC));
      send(res, 200, body, { 'content-type': type });
    } else if (key === 'POST /api/login') {
      await login(req, res);
    } else if (key === 'GET /api/session') {
      const { email, role, csrf } = requireUser(req).session;
      sendJson(res, 200, { email, role, csrf });
    } else if (key === 'POST /api/logout') {
      sessions.destroy(requireUser(req, { csrf: true }).sid);
      sendJson(res, 200, { ok: true },
        { 'set-cookie': `sid=; Max-Age=0; ${flags}` });
    } else if (key === 'GET /api/trips') {
      sendJson(res, 200, { trips: db.tripsFor(requireUser(req).user) });
    } else if (key === 'POST /api/trips') {
      await createTrip(req, res);
    } else if (req.method === 'GET' && tripId) {
      readTrip(req, res, tripId);
    } else {
      throw new HttpError(404, 'পাওয়া যায়নি');
    }
  }

// ...
```

app.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  return async function handle(req, res) {
    const { pathname } = new URL(req.url, 'http://localhost');
    try {
      await route(req, res, pathname);
    } catch (error) {
      if (error instanceof HttpError) {
        sendJson(res, error.status, { error: error.message });
        return;
      }
      // বিস্তারিত শুধু Server-এর Log-এ; ব্যবহারকারী পান সাধারণ বার্তা।
      log('server.error', { message: error.message, path: pathname });
      sendJson(res, 500, { error: 'Server-এ একটি সমস্যা হয়েছে' });
    }
  };
}
```

`/api/session` একটি `GET` Route যা CSRF Token ফেরত দেয়, যাতে Page পুনরায় লোড হলে UI আবার Token পায়। এটি নিরাপদ কারণ অন্য Origin-এর Script Same-Origin Policy-র কারণে এই Response পড়তে পারে না, এবং আমরা কোনো CORS Header পাঠাই না। শেষ অংশে অপ্রত্যাশিত Error-এর বিস্তারিত শুধু Log-এ; ব্যবহারকারী পান একটি সাধারণ বার্তা।

src/limiter.mjs, src/log.mjs ও src/start.mjs

limiter.mjs · JavaScript চলবে: Node.js

```
// নির্দিষ্ট সময়-জানালায় একটি Key-এর ব্যর্থ চেষ্টা গোনে (Login-এর জন্য)।
export function createLimiter(options = {}) {
  const { max = 5, windowMs = 15 * 60 * 1000, now = Date.now } = options;
  const failures = new Map();
  const current = (key) => {
    const entry = failures.get(key);
    if (!entry || now() - entry.start >= windowMs) {
      return { count: 0, start: now() };
    }
    return entry;
  };
  return {
    blocked: (key) => current(key).count >= max,
    fail(key) {
      const entry = current(key);
      failures.set(key, { count: entry.count + 1, start: entry.start });
    },
    reset: (key) => failures.delete(key),
```

```
  };
}
```

log.mjs · JavaScript চলবে: Node.js

```
const SENSITIVE = /pass|token|secret|cookie|authorization|csrf/i;

// Log-এ যাওয়ার আগে সংবেদনশীল Key-এর মান মুছে দেয় (Nested Object-সহ)।
export function redact(value) {
  if (Array.isArray(value)) {
    return value.map(redact);
  }
  if (typeof value !== 'object' || value === null) {
    return value;
  }
  return Object.fromEntries(Object.entries(value).map(([key, inner]) =>
    [key, SENSITIVE.test(key) ? '[REDACTED]' : redact(inner)]));
}

export const createLogger = (write = console.log) => (event, details = {}) =>
  write(JSON.stringify({ event, ...redact(details) }));
```

start.mjs · JavaScript চলবে: Node.js

```
import { createServer } from 'node:http';
import { randomBytes } from 'node:crypto';
import { createApp } from './app.mjs';
import { createDatabase } from './db.mjs';
import { createSessions } from './sessions.mjs';
import { createLimiter } from './limiter.mjs';
import { createLogger } from './log.mjs';
import { hashPassword } from './passwords.mjs';

// শুধু 127.0.0.1-এ শোনে: এই কম্পিউটারের বাইরে থেকে পৌঁছানো যায় না।
export async function startServer(options = {}) {
  const { port = 0, users = [], log = createLogger() } = options;
  const db = createDatabase();
  for (const { email, password, role } of users) {
    db.addUser(email, await hashPassword(password), role);
  }
  const dummyHash = await hashPassword(randomBytes(16).toString('hex'));
  const app = createApp({
    db, sessions: createSessions(), limiter: createLimiter(), log, dummyHash,
  });
  const server = createServer(app);
  await new Promise((resolve) => server.listen(port, '127.0.0.1', resolve));
  return { server, url: `http://127.0.0.1:${server.address().port}` };
}
```

server.mjs · JavaScript চলবে: Node.js

```
import { randomBytes } from 'node:crypto';
import { startServer } from './src/start.mjs';

// Demo Password Code-এ লেখা নেই: Environment Variable, নয়তো প্রতি চালানোয় নতুন।
const password = process.env.SECURETRIP_DEMO_PASSWORD
  ?? randomBytes(9).toString('base64url');
const users = [
  { email: 'rafi@example.com', role: 'customer' },
  { email: 'mina@example.com', role: 'customer' },
  { email: 'admin@example.com', role: 'admin' },
].map((user) => ({ ...user, password }));

const port = Number(process.env.PORT ?? 3020);
const { url } = await startServer({ port, users });
console.log(`SecureTrip (শিক্ষামূলক Demo, শুধু এই কম্পিউটারে): ${url}`);
console.log(`ব্যবহারকারী: ${users.map((user) => user.email).join(', ')}`);
if (!process.env.SECURETRIP_DEMO_PASSWORD) {
  console.log(`এই চালানোর Demo Password: ${password}`);
}
```

`listen(port, '127.0.0.1')` : Server শুধু এই কম্পিউটারের ভেতর থেকে পৌঁছানো যায়, একই Wi-Fi-র অন্য যন্ত্র থেকে নয়। Demo Password Code-এ লেখা নেই; Environment Variable থেকে, নয়তো প্রতিবার নতুন এলোমেলো, এবং শুধু নিজের Terminal-এ দেখানো হয়।

Browser UI: public/index.html ও public/app.mjs

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SecureTrip</title>
  <link rel="stylesheet" href="/style.css">
  <script type="module" src="/app.mjs"></script>
</head>
<body>
  <h1>SecureTrip <small>শিক্ষামূলক Demo</small></h1>
  <form id="login">
    <label>Email <input id="email" type="email" required></label>
    <label>Password <input id="password" type="password" required></label>
    <button id="login-btn">Login</button>
  </form>
  <form id="trip" hidden>
    <label>Package <select id="package">
```

```
      <option value="cox-3d">Cox's Bazar, ৩ দিন</option>
      <option value="sylhet-2d">Sylhet, ২ দিন</option>
      <option value="sundarbans-3d">Sundarbans, ৩ দিন</option>
    </select></label>
    <label>যাত্রী <input id="travelers" type="number" min="1" max="8"
      value="2"></label>
    <label>তারিখ <input id="date" type="date" required></label>
    <label>নোট <input id="note" maxlength="200"></label>
    <button id="add">অনুরোধ পাঠান</button>
  </form>
  <p id="status" role="status"></p>
  <ul id="trips"></ul>
</body>
</html>
```

app.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
const $ = (selector) => document.querySelector(selector);
let csrf = '';

// CSRF Token শুধু এই Variable-এ (Memory); localStorage-এ নয়।
async function api(path, { method = 'GET', body } = {}) {
  const response = await fetch(path, {
    method,
    headers: { 'content-type': 'application/json', 'x-csrf-token': csrf },
    body: body && JSON.stringify(body),
  });
  const data = await response.json();
  if (!response.ok) {
    const details = Object.values(data.errors ?? {}).join(', ');
    throw new Error(data.error ?? details);
  }
  return data;
}

// সব লেখা textContent দিয়ে: নোটে HTML থাকলেও তা লেখা হিসেবেই দেখায়।
function renderTrips(trips) {
  $('#trips').replaceChildren(...trips.map((trip) => {
    const item = document.createElement('li');
    item.textContent = `${trip.packageId} · ${trip.travelers} জন · `
      + `${trip.startDate} · ৳${trip.total} · ${trip.note}`;
    return item;
  }));
}

async function showApp(session) {
  csrf = session.csrf;
  $('#login').hidden = true;
  $('#trip').hidden = false;
```

```
  $('#status').textContent = `Login: ${session.email} (${session.role})`;
  renderTrips((await api('/api/trips')).trips);
}

// ...
```

app.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) — চলবে: Browser

```
// ...

$('#login').addEventListener('submit', async (event) => {
  event.preventDefault();
  try {
    const body = { email: $('#email').value, password: $('#password').value };
    $('#password').value = '';
    await showApp(await api('/api/login', { method: 'POST', body }));
  } catch (error) {
    $('#status').textContent = error.message;
  }
});

$('#trip').addEventListener('submit', async (event) => {
  event.preventDefault();
  try {
    const body = {
      packageId: $('#package').value,
      travelers: Number($('#travelers').value),
      startDate: $('#date').value,
      note: $('#note').value,
    };
    const { trip } = await api('/api/trips', { method: 'POST', body });
    $('#status').textContent = `অনুরোধ গৃহীত, মোট ৳${trip.total}`;
    renderTrips((await api('/api/trips')).trips);
  } catch (error) {
    $('#status').textContent = error.message;
  }
});

api('/api/session').then(showApp, () => {});
```

style.css · CSS — চলবে: Browser

```
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 40rem;
  margin: 2rem auto;
  padding: 0 1rem;
}
```

```
form {
  display: grid;
  gap: 0.5rem;
  margin-bottom: 1rem;
}

label {
  display: grid;
  gap: 0.25rem;
}

#status {
  color: #1d4ed8;
}
```

HTML-এ কোনো Inline Script বা `onclick` নেই; সব JavaScript একটি বাইরের File-এ, যা `script-src 'self'` CSP-র সঙ্গে খাপ খায়। Trip-এর প্রতিটি লেখা `textContent` দিয়ে, CSRF Token শুধু একটি Variable-এ। UI-র `maxlength` বা `min` / `max` শুধু ব্যবহারকারীর সুবিধা; আসল নিয়ম Server-এর `validateTripRequest`।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd secure-trip
node server.mjs
node --test
```

`node server.mjs` চালালে Terminal-এ ঠিকানা (`http://127.0.0.1:3020`), তিনটি Demo ব্যবহারকারী এবং সেই চালানোর এলোমেলো Password দেখায়; Browser-এ ঠিকানাটি খুলে Login করুন। `node --test` ২৪টি Test চালায়: Unit Test ১১টি (Password, Validation, Session, Limiter, Redaction) এবং আসল HTTP Server-এর বিরুদ্ধে Integration Test ১৩টি—Security Header, একই রকম Login-ব্যর্থতার বার্তা, Cookie-র Flag, CSRF Token ও Origin, Server-এর দাম ও মালিক, Object-level Authorization, 413/415/400, SQL Injection-এর চেষ্টা, Login-এর সীমা, Logout, Static File-এর সীমা, এবং Log-এ কোনো Password বা Token নেই। এছাড়া এই বইয়ের QA-তে আসল Chrome-এ একটি End-to-end যাচাই চলেছে: UI দিয়ে Login, `document.cookie`-তে Session ID দেখা যায় না, `<img onerror>`-যুক্ত নোট শুধু লেখা হিসেবে দেখায় এবং কোনো Handler চলে না, এবং Page-এ ঢোকানো Inline Script CSP আটকে দেয়।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find module 'node:sqlite'`	Node.js পুরোনো	22.5 বা নতুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Login-এর পর প্রতিটি অনুরোধে 401	Cookie যায়নি (ভিন্ন Host, যেমন localhost বনাম 127.0.0.1)	একই ঠিকানা ব্যবহার করুন
অনুরোধ পাঠাতে 403	CSRF Token নেই বা পুরোনো	আবার Login, অথবা `/api/session`
429	পাঁচবার ভুল Password	১৫ মিনিট, অথবা Server আবার চালু
Page-এ কিছুই চলে না	Inline Script যোগ করেছেন; CSP আটকাচ্ছে	Script বাইরের File-এ

২০.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Client-side Validation-কে নিরাপত্তা ভাবা	Browser ব্যবহারকারীর নিয়ন্ত্রণে	Server-এ পুনরায় Validation
ব্যবহারকারীর লেখা `innerHTML`-এ	XSS	`textContent`, Sanitizer
শুধু `<script>` আটকানো	`onerror`, `javascript:` URL	প্রসঙ্গভিত্তিক Encoding, CSP
SQL-এ লেখা জোড়া লাগানো	SQL Injection	Parameterized Query
শুধু Login যাচাই, মালিকানা নয়	IDOR	প্রতিটি Object-এ Authorization
Body থেকে দাম বা `ownerId` নেওয়া	ইচ্ছামতো দাম, অন্যের নামে কাজ	Server-এর উৎস, Session
SHA-256 বা MD5-এ Password	দ্রুত অনুমান	scrypt, Argon2id, bcrypt
Token `localStorage`-এ	XSS পড়ে নিতে পারে	HttpOnly Cookie
CORS-এ যেকোনো Origin ফেরত	অন্য Site Cookie-সহ পড়তে পারে	স্থির Allowlist
Secret Code বা Git-এ	ফাঁস	Environment, Secret Manager
Log-এ Password বা Token	Log-ও ফাঁস হয়	Redaction
ব্যবহারকারীকে Stack Trace দেখানো	ভেতরের তথ্য প্রকাশ	সাধারণ বার্তা, বিস্তারিত Log-এ

২০.১৪ Debugging নির্দেশিকা

১. **Network Panel:** প্রতিটি Request-এর Status, Request ও Response Header দেখুন; Cookie পাঠানো হয়েছে কি না (Request Header-এ `Cookie`), এবং `Set-Cookie`-র Flag।

২. **Application Panel-এর Cookies অংশ:** HttpOnly, Secure, SameSite কলাম।

৩. **Console-এ CSP বার্তা:** "...violates the following Content Security Policy directive" মানে CSP কিছু আটকেছে; Inline Code সরিয়ে বাইরের File-এ নিন, CSP শিথিল করবেন না।
৪. **401 বনাম 403 বনাম 404:** 401 = পরিচয় নেই; 403 = পরিচয় আছে কিন্তু এই কাজের অনুমতি নেই (এখানে CSRF); 404 = নেই, বা আপনার নয়।
৫. **Server Log:** `login.failed`, `server.error`—Redaction-সহ।
৬. **নিজের Code-এ নিরাপত্তার Test:** প্রতিটি ঠিক করা দুর্বলতার জন্য একটি Test, যাতে তা ফিরে না আসে।

২০.১৫ পেশাদার অভ্যাস

- **কখনো Client-কে বিশ্বাস নয়:** Server-এ Validation, দাম, মালিকানা।
- **Allowlist, Blocklist নয়:** Field, Package, Protocol, Sort Column।
- **Defense in Depth:** Encoding এবং CSP; SameSite এবং CSRF Token; একটি স্তর ব্যর্থ হলে আরেকটি।
- **Least Privilege:** প্রতিটি ব্যবহারকারী, Database Account ও Token-এর ন্যূনতম অধিকার।
- **নিজে Crypto না বানানো:** Node.js-এর `crypto`, পরীক্ষিত Library।
- **ব্যর্থতায় বন্ধ (Fail Closed):** সন্দেহ হলে প্রত্যাখ্যান।
- **নিরাপত্তার Test** স্বাভাবিক Test-এর মতোই; প্রতিটি নিয়ন্ত্রণ প্রমাণিত।
- **OWASP Top 10 ও ASVS** নিয়মিত পড়া; Dependency হালনাগাদ।

২০.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `escapeHtml` দিয়ে `Tom & "Jerry" <b>`-কে HTML-এ নিরাপদ করুন, এবং ফল দেখান।
২. একজন সহকর্মী বলছেন, "Form-এ `required` ও `maxlength` দিয়েছি, Server-এ যাচাই লাগবে না।" দুই বাক্যে উত্তর দিন।
৩. প্রতিটি Authentication না Authorization: (ক) Password যাচাই, (খ) Trip-এর মালিক যাচাই, (গ) Session-এর মেয়াদ, (ঘ) Admin-ই সব দেখতে পারেন।
৪. একটি Session Cookie-র `Set-Cookie` Header লিখুন, Production-এর জন্য (HTTPS)।
৫. নিচের Code-এর নিরাপত্তা-ত্রুটি খুঁজুন: `fetch('https://api.maps.example/geo?key=sk_live_93ab...')`, একটি Browser Bundle-এ।

মধ্যম স্তর

৬. `safeHref(url)` লিখুন, যা শুধু `https:`, `http:` ও Relative Link গ্রহণ করে, বাকিগুলোর জন্য `'#'` দেয়।

৭. `node:sqlite`-এ Place-এর নামে আংশিক খোঁজ (`LIKE`) Parameter দিয়ে করুন, যাতে ব্যবহারকারীর `%` চিহ্ন Wildcard হিসেবে কাজ না করে।

৮. একটি `corsHeaders(origin)` লিখুন: স্থির Allowlist-এর Origin হলে Header দেয়, নয়তো কিছু নয়।

৯. SecureTrip-এর `redact`-কে বাড়ান, যাতে `email` পুরো না মুছে আংশিক ঢাকে (`ra***@example.com`)।

চ্যালেঞ্জ স্তর

১০. একটি `safeMerge` লিখুন, যা Prototype-হীন Object-এ মেশায়, বিপজ্জনক Key বাদ দেয়, এবং ৫ স্তরের বেশি গভীরতায় Error দেয়।

১১. SecureTrip-এর Limiter দিয়ে Email ও IP দুটির জন্যই সীমা প্রয়োগের একটি Function লিখুন, এবং দেখান যে একটি IP বিভিন্ন Email-এ চেষ্টা করলেও আটকে যায়।

১২. নিচের Express-ধাঁচের Route-এর অন্তত পাঁচটি নিরাপত্তা-ত্রুটি চিহ্নিত করুন এবং প্রতিটির সমাধান লিখুন:
`app.get('/trip/delete', (req, res) => { db.exec("DELETE FROM trips WHERE id = " + req.query.id); res.send('<p>' + req.query.id + ' মুছে ফেলা হয়েছে</p>'); });`

২০.১৭ অনুশীলনীর সমাধান

সমাধান ১

escape.js · JavaScript — চলবে: Browser Console ও Node.js

```
const HTML_ESCAPES = {
  '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
};
const escapeHtml = (text) =>
  String(text).replace(/[&<>"']/g, (ch) => HTML_ESCAPES[ch]);

console.log(escapeHtml('Tom & "Jerry" <b>'));
```

Output

```
Tom &amp; &quot;Jerry&quot; &lt;b&gt;
```

`&` সবার আগে নয়, একবারে সব চিহ্ন বদলানো হয়েছে; ফলে আগে বদলানো `<`-এর `&` আবার বদলায় না।

সমাধান ২ — Browser ব্যবহারকারীর নিয়ন্ত্রণে: DevTools-এ `maxlength` মুছে ফেলা যায়, অথবা Browser ছাড়াই `curl` দিয়ে যেকোনো Request পাঠানো যায়। তাই Client-এর যাচাই ব্যবহারকারীর সুবিধা, আর Server-এর যাচাই নিরাপত্তা—দ্বিতীয়টি সব সময় লাগবে।

সমাধান ৩ — (ক) Authentication, (খ) Authorization (Object-level), (গ) Authentication (পরিচয় এখনো বৈধ কি না), (ঘ) Authorization (Role-ভিত্তিক)।

সমাধান ৪

Terminal

```
Set-Cookie: sid=<৩২ Byte এলোমেলো, base64url>; HttpOnly; Secure;
  SameSite=Lax; Path=/; Max-Age=1800
```

`HttpOnly` JavaScript থেকে লুকায়, `Secure` শুধু HTTPS-এ পাঠায়, `SameSite` অন্য Site-এর Request-এ বাধা, `Max-Age` ৩০ মিনিটে শেষ। অন্য Site থেকে আসা Link-এ ক্লিক করেও ব্যবহারকারী Login অবস্থায় থাকতে চাইলে `Lax`; SecureTrip-এর মতো কড়া হলে `Strict`।

সমাধান ৫ — Browser Bundle-এ থাকা যেকোনো Key Public: যে কেউ DevTools-এর Network বা Source থেকে তা পড়ে নিজের কাজে ব্যবহার করতে পারে (এবং বিল আপনার নামে)। সমাধান: Secret Key-যুক্ত Request নিজের Server থেকে (Server Proxy), অথবা Provider যদি Browser-এর জন্য বিশেষ সীমিত Key দেয় (নির্দিষ্ট Domain-এ সীমাবদ্ধ), সেটি। Key ইতিমধ্যে প্রকাশিত হলে তা বাতিল করে নতুন Key (Rotate)।

সমাধান ৬

safe-href.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function safeHref(url) {
  try {
    const parsed = new URL(url, 'https://trip.example');
    return ['https:', 'http:'].includes(parsed.protocol) ? url : '#';
  } catch {
    return '#';
  }
}

const inputs = ['https://ok.example/a', '/trips/7', 'JavaScript:alert(1)',
  ' javascript:void(0)', 'data:text/html,hi', 'http://[bad'];
console.log(inputs.map(safeHref).join(' | '));
```

Output

```
https://ok.example/a | /trips/7 | # | # | # | #
```

বড় হাতের `JavaScript:` এবং সামনে ফাঁকা স্থান—দুটিই Browser `javascript:` হিসেবে মানে; `URL` Parser একই নিয়মে Protocol বের করে, তাই নিজের Regex-এর চেয়ে নির্ভরযোগ্য।

সমাধান ৭

like.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE places (name TEXT);
  INSERT INTO places VALUES ('Sylhet'), ('Cox''s Bazar'), ('100% Sundarbans');`);

const escapeLike = (text) => text.replace(/[\\%_]/g, (ch) => `\\${ch}`);
const search = db.prepare(
  "SELECT name FROM places WHERE name LIKE ? ESCAPE '\\'");

for (const input of ['ba', '%', "'"]) {
  const rows = search.all(`%${escapeLike(input)}%`);
  console.log(JSON.stringify(input), rows.map((row) => row.name).join(', '));
}
```

Output

```
"ba" Cox's Bazar, 100% Sundarbans
"%" 100% Sundarbans
"'" Cox's Bazar
```

Parameter SQL Injection আটকায় (`'` নিরাপদ), কিন্তু `LIKE`-এর ভেতরে `%` ও `_` নিজস্ব অর্থ রাখে। Escape না করলে `%` খুঁজলে সব সারি আসত। (SQLite-এর `LIKE` ASCII-তে ছোট-বড় হাতের পার্থক্য করে না, তাই `ba` দিয়ে `Bazar` ও `Sundarbans` দুটিই মিলেছে।)

সমাধান ৮

cors.js · JavaScript — চলবে: Node.js

```
const ALLOWED = new Set([
  'https://app.trip.example',
  'https://admin.trip.example',
]);

function corsHeaders(origin) {
  if (!ALLOWED.has(origin)) {
    return {};
  }
  return {
    'access-control-allow-origin': origin,
    'access-control-allow-credentials': 'true',
    vary: 'Origin',
  };
```

```
}

console.log(corsHeaders('https://app.trip.example'));
console.log(corsHeaders('https://app.trip.example.evil.test'));
console.log(corsHeaders(undefined));
```

Output

```
{
  'access-control-allow-origin': 'https://app.trip.example',
  'access-control-allow-credentials': 'true',
  vary: 'Origin'
}
{}
{}
```

হুবহু মিল, `startsWith` বা `includes` নয়: দ্বিতীয় উদাহরণটি `startsWith` পরীক্ষা পেরিয়ে যেত। `Vary: Origin` Cache-কে বলে যে Response Origin-ভেদে আলাদা।

সমাধান ৯

mask-email.mjs · JavaScript চলবে: Node.js

```
import { redact } from '../../project/secure-trip/src/log.mjs';

const maskEmail = (email) => email.replace(/^(.{1,2})[^@]*(@.*)$/u, '$1***$2');

function redactForLog(details) {
  const clean = redact(details);
  return typeof clean.email === 'string'
    ? { ...clean, email: maskEmail(clean.email) }
    : clean;
}

console.log(redactForLog({ email: 'rafi@example.com', password: 'x' }));
console.log(redactForLog({ email: 'a@b.example' }));
```

Output

```
{ email: 'ra***@example.com', password: '[REDACTED]' }
{ email: 'a***@b.example' }
```

সমাধান ১০

safe-merge.mjs · JavaScript — চলবে: Node.js

```
const BLOCKED = new Set(['__proto__', 'constructor', 'prototype']);

function safeMerge(target, source, depth = 0) {
  if (depth > 5) {
    throw new RangeError('অনেক গভীর Object');
  }
  for (const [key, value] of Object.entries(source)) {
    if (BLOCKED.has(key)) {
      continue;
    }
    const isObject = typeof value === 'object' && value !== null;
    const existing = Object.hasOwn(target, key)
      ? target[key]
      : Object.create(null);
    target[key] = isObject ? safeMerge(existing, value, depth + 1) : value;
  }
  return target;
}

const text = '{"a":{"b":1},"__proto__":{"x":1},"constructor":{"y":2}}';
const body = JSON.parse(text);
const merged = safeMerge(Object.create(null), body);
console.log(JSON.stringify(merged), ({}).x, Object.getPrototypeOf(merged));

let deep = {};
for (let i = 0; i < 8; i += 1) {
  deep = { inner: deep };
}
try {
  safeMerge(Object.create(null), deep);
} catch (error) {
  console.log(error.name, error.message);
}
```

Output

```
{"a":{"b":1}} undefined null
RangeError অনেক গভীর Object
```

সমাধান ১১

dual-limit.mjs · JavaScript — চলবে: Node.js

```
import { createLimiter } from '../../project/secure-trip/src/limiter.mjs';

const byEmail = createLimiter({ max: 5 });
const byIp = createLimiter({ max: 20 });

function attempt(ip, email, success) {
  if (byIp.blocked(ip) || byEmail.blocked(email)) {
    return 429;
  }
  if (success) {
    byEmail.reset(email);
    return 200;
  }
  byIp.fail(ip);
  byEmail.fail(email);
  return 401;
}

const statuses = [];
for (let i = 0; i < 22; i += 1) {
  statuses.push(attempt('10.0.0.7', `user${i}@example.com`, false));
}
console.log(statuses.slice(-3).join(' '));
console.log(attempt('10.0.0.8', 'user1@example.com', true));
```

Output

```
401 429 429
200
```

একটি IP প্রতিবার ভিন্ন Email দিয়ে চেষ্টা করলে (Password Spraying) Email-ভিত্তিক সীমা কাজে আসে না, কিন্তু IP-ভিত্তিক সীমা বিশটির পর আটকায়। অন্য IP থেকে সঠিক Login তখনো চলে। বাস্তবে IP ভাগ করা (একই অফিস) হতে পারে, তাই IP-র সীমা ঢিলে রাখা হয়।

সমাধান ১২ — (১) অবস্থা বদলানো কাজ `GET`-এ: একটি `<img src="/trip/delete?id=5">` যুক্ত যেকোনো Page তা চালাতে পারে (CSRF); সমাধান `DELETE` বা `POST`, CSRF Token। (২) SQL-এ লেখা জোড়া লাগানো: `id=1 OR 1=1` সব মুছে দেবে; সমাধান Parameterized Query। (৩) Authentication নেই। (৪) Authorization নেই: যে কেউ যেকোনো Trip মুছতে পারে; সমাধান মালিক বা Admin যাচাই। (৫) Response-এ `req.query.id` সরাসরি HTML-এ: Reflected XSS; সমাধান Escape বা JSON Response। (৬) `id`-এর Validation নেই; সমাধান ধরন ও আকারের Allowlist। (৭) কোনো Log নেই, যাতে পরে বোঝা যায় কে কী মুছেছে।

২০.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Client-side Validation-এর মূল উদ্দেশ্য?

ক) নিরাপত্তা

খ) ব্যবহারকারীকে তাৎক্ষণিক ভুল দেখানো

গ) Server-এর কাজ বাদ দেওয়া

ঘ) SQL Injection প্রতিরোধ

প্রশ্ন ২. `innerHTML`-এ `<img src="data:," onerror="...">` বসালে কী ঘটে?

ক) কিছুই না, কারণ `<script>` নেই

খ) শুধু লেখা দেখায়

গ) `onerror` Handler চলে

ঘ) Browser Error দিয়ে থেমে যায়

প্রশ্ন ৩. SQL Injection-এর সঠিক প্রতিরোধ?

ক) Parameterized Query

খ) Input থেকে `'` মুছে ফেলা

গ) Input বড় হাতের অক্ষরে রূপান্তর

ঘ) Client-side Validation

প্রশ্ন ৪. Login করা গ্রাহক URL-এর ID বদলে অন্যের Trip দেখতে পাচ্ছেন। এটি কোন ধরনের ত্রুটি?

ক) XSS

খ) CSRF

গ) SQL Injection

ঘ) Broken Access Control (IDOR)

প্রশ্ন ৫. Password সংরক্ষণের জন্য উপযুক্ত?

ক) SHA-256

খ) Base64

গ) Salt-সহ scrypt বা Argon2id

ঘ) AES Encryption, Key Code-এ

প্রশ্ন ৬. `HttpOnly` Cookie Flag কী করে?

ক) JavaScript-কে Cookie পড়তে দেয় না

খ) শুধু HTTP-তে পাঠায়, HTTPS-এ নয়

গ) CSRF পুরোপুরি বন্ধ করে

ঘ) Cookie-কে Encrypt করে

প্রশ্ন ৭. CORS সম্পর্কে কোনটি সঠিক?

ক) এটি API-কে সব অননুমোদিত অনুরোধ থেকে রক্ষা করে

খ) এটি Browser-কে জানায় কোন Origin Response পড়তে পারবে

গ) `*` দিলে Cookie-সহ API নিরাপদ

ঘ) `curl`-ও CORS মানে

প্রশ্ন ৮. `script-src 'self'` CSP কী আটকায়?

ক) একই Origin-এর Script File

খ) সব Network Request

গ) CSS File

ঘ) Inline Script ও `onerror`-এর মতো Inline Handler

প্রশ্ন ৯. নিচের কোনটি Prototype Pollution ঘটাতে পারে?

ক) `JSON.parse` একা

খ) `__proto__` Key বাদ না দিয়ে Recursive Merge

গ) `Object.freeze`

ঘ) `Map`-এ `set('__proto__', 1)`

প্রশ্ন ১০. অন্যের Trip-এর অনুরোধে SecureTrip 403-এর বদলে 404 দেয় কেন?

ক) 404 দ্রুত

খ) 403 HTTP-তে নেই

গ) Trip-এর অস্তিত্বও প্রকাশ না করতে

ঘ) Browser 403 দেখাতে পারে না

২০.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–গ · ৬–ক · ৭–খ · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	নিরাপত্তা আসে Server-এর যাচাই থেকে।
২	গ	অনুচ্ছেদ ২০.৩-এ Chrome-এ প্রমাণিত; `<script>` না চললেও Handler চলে।
৩	ক	মান কখনো SQL-এর লেখা হয় না; চিহ্ন মোছা অসম্পূর্ণ ও ভঙ্গুর।
৪	ঘ	Authentication ঠিক, Object-level Authorization নেই।
৫	গ	ধীর, Salt-যুক্ত, Memory-নির্ভর; দ্রুত Hash অনুমান সহজ করে।
৬	ক	XSS হলেও Session ID পড়া যায় না; CSRF-এর জন্য SameSite ও Token।
৭	খ	CORS Browser-এর পড়ার অনুমতি; সুরক্ষা আসে AuthN ও AuthZ থেকে।
৮	ঘ	একই Origin-এর File চলে; অনুচ্ছেদ ২০.৯-এ Chrome-এ যাচাই।
৯	খ	`target['__proto__']` আসলে `Object.prototype`।
১০	গ	অস্তিত্বহীন ও অন্যের Trip—দুটিতে একই উত্তর।

২০.২০ অধ্যায়ের সারসংক্ষেপ

- Threat Model: কী রক্ষা, কার কাছ থেকে; সবচেয়ে গুরুত্বপূর্ণ Trust Boundary হলো Browser থেকে Server।
- Client-side Validation সুবিধা, নিরাপত্তা নয়; Server-এ Allowlist দিয়ে পুনরায় যাচাই।
- XSS: অবিশ্বস্ত লেখা কখনো Markup নয়; `textContent`, প্রসঙ্গভিত্তিক Encoding, CSP।
- SQL Injection: Parameterized Query; Column-এর নাম Allowlist থেকে।
- Authentication পরিচয়, Authorization প্রতিটি Object-এর অনুমতি; মালিক Session থেকে।
- Password: scrypt বা Argon2id, Salt, `timingSafeEqual`; একই রকম ব্যর্থতার বার্তা; ব্যর্থ চেষ্টার সীমা।
- Session Cookie: HttpOnly, Secure, SameSite; CSRF Token ও Origin যাচাই।
- CORS পড়ার অনুমতি, সুরক্ষা নয়; CSP দ্বিতীয় স্তরের প্রতিরক্ষা।
- Prototype Pollution: বিপজ্জনক Key বাদ, `Map`, `Object.create(null)`, Allowlist।
- Secret Environment-এ, Dependency নিয়মিত যাচাই, Log-এ Redaction।
- শিক্ষামূলক Demo-কে Production-প্রস্তুত বলা যায় না; তার জন্য স্বাধীন Review ও আরও অনেক নিয়ন্ত্রণ লাগে।

পরের অধ্যায়ে Design Pattern ও Software Architecture: বড় Application-কে কীভাবে পরিষ্কার, পরীক্ষাযোগ্য অংশে সাজানো যায়।

২০.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Threat Model	কী রক্ষা, কার কাছ থেকে, কীভাবে—তার বিশ্লেষণ।
Trust Boundary	যেখানে কম বিশ্বস্ত তথ্য বেশি বিশ্বস্ত এলাকায় ঢোকে।
XSS	ব্যবহারকারীর লেখা অন্যের Browser-এ Script হিসেবে চলা।
Output Encoding	প্রসঙ্গ অনুযায়ী লেখাকে শুধু তথ্য হিসেবে নিরাপদ করা।
SQL Injection	ইনপুট SQL-এর অংশ হয়ে যাওয়া।
Authentication	পরিচয় যাচাই।
Authorization	নির্দিষ্ট কাজের অনুমতি যাচাই।
IDOR	ID বদলে অন্যের Object-এ পৌঁছানো।
Salt	প্রতিটি Password-এর আলাদা এলোমেলো মান।
CSRF	অন্য Site থেকে ব্যবহারকারীর Cookie-সহ অনুরোধ করানো।
CORS	অন্য Origin-কে Response পড়ার অনুমতির নিয়ম।
CSP	কোন উৎসের Script চলবে তার Browser-নীতি।
Prototype Pollution	`Object.prototype`-এ অনাকাঙ্ক্ষিত Property লেখা।
Redaction	Log থেকে সংবেদনশীল তথ্য মুছে ফেলা।

২০.২২ আরও পড়ুন

- **OWASP Top 10:** `https://owasp.org/www-project-top-ten/` — Web Application-এর প্রধান ঝুঁকি।
- **OWASP Cheat Sheet Series:** `https://cheatsheetseries.owasp.org/` — XSS, Session, Password Storage, CSRF-এর ব্যবহারিক নির্দেশিকা।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP` — CSP-র Directive।
- **MDN — Set-Cookie:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie` — Cookie-র Flag।
- **Node.js — crypto.scrypt:** `https://nodejs.org/api/crypto.html#cryptoscryptpassword-salt-keylen-options-callback` — Password Hashing।

অধ্যায় ২১

২১

JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি বড় হতে থাকা Code-এর সমস্যা চিনতে: জট পাকানো দায়িত্ব, লুকানো নির্ভরতা, পরীক্ষা করতে না পারা।
- Design Pattern ও Architecture-এর পার্থক্য বুঝতে, এবং কখন একটি Pattern দরকার নেই তা বলতে।
- Factory, Strategy, Observer, Repository, Adapter ও Decorator Pattern JavaScript-এর নিজস্ব ধরনে (Function ও Object দিয়ে) লিখতে।
- Dependency Injection ও Composition Root দিয়ে অংশগুলো আলাদা রাখতে।
- Layered Architecture-এ নির্ভরতার দিক ঠিক করতে, এবং একটি Test দিয়ে সেই নিয়ম পাহারা দিতে।
- একটি Workflow-কে টেবিলভিত্তিক State Machine হিসেবে প্রকাশ করতে।
- Contract Test দিয়ে একই চুক্তির একাধিক Implementation যাচাই করতে।
- একটি সম্পূর্ণ **VisaFlow** Application তৈরি করতে, যা Node.js ও Browser—দুই পরিবেশে একই Domain ও Service ব্যবহার করে।

পূর্বশর্ত

- অধ্যায় ১২: ES Module, `import` / `export`।
- অধ্যায় ১৩: Class, Composition বনাম Inheritance।
- অধ্যায় ১৬: Error-এর `code`, Repository ও Service, Test Double।
- অধ্যায় ১৮: Immutable হালনাগাদ।
- অধ্যায় ২০: Allowlist, উত্তরাধিকারের Key-এর ঝুঁকি।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ৫০ লাইনের Script-এ সবকিছু এক জায়গায় লিখলে সমস্যা নেই। কিন্তু ছয় মাস পর সেটি ৫,০০০ লাইন, তিনজন Developer, এবং প্রতিটি নতুন Feature-এ পুরোনো কিছু ভেঙে যায়। কারণ প্রায় সব সময় একই: একটি Function একসঙ্গে Form পড়ে, দাম হিসাব করে, `localStorage`-এ লেখে এবং DOM আঁকে। তখন দামের নিয়ম বদলাতে DOM-এর Code ছুঁতে হয়, এবং দামের হিসাব Browser ছাড়া Test করা যায় না।

Architecture-এর লক্ষ্য তাই সুন্দর Diagram নয়; লক্ষ্য হলো **পরিবর্তন সস্তা রাখা**। যে অংশগুলো ভিন্ন কারণে বদলায় (ব্যবসার নিয়ম, সংরক্ষণের পদ্ধতি, পর্দার নকশা), সেগুলো আলাদা থাকলে একটি বদলালে অন্যটি অক্ষত থাকে। Design Pattern সেই আলাদা করার পরীক্ষিত, নামযুক্ত উপায়—এবং নাম থাকায় দল সহজে আলোচনা করতে পারে: "এখানে একটি Strategy লাগবে।"

২১.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
ES Module (অধ্যায় ১২)	Module-কে স্তরে সাজানো, নির্ভরতার দিক, Module-Singleton-এর ঝুঁকি
Composition বনাম Inheritance (অধ্যায় ১৩)	Strategy ও Decorator: Composition-এর নামযুক্ত রূপ
Repository ও Service (অধ্যায় ১৬, BookingGuard)	Adapter, Contract Test, Composition Root
Event Listener (অধ্যায় ৯)	নিজস্ব Event Bus, Observer, Error-এর বিচ্ছিন্নতা
`if/switch`-এ Status (আগের Project)	টেবিলভিত্তিক State Machine
—	Architecture Test, Dependency Injection-এর পূর্ণ প্রয়োগ

অধ্যায় ১৩ ও ১৬-এর Project দুটি ছিল Booking-কেন্দ্রিক। পুনরাবৃত্তি এড়াতে এই অধ্যায়ের Project ভিন্ন একটি Domain-এ: Visa আবেদনের Workflow।

২১.২ Design Pattern বনাম Architecture

	Design Pattern	Architecture
পরিসর	কয়েকটি Function বা Object	পুরো Application
প্রশ্ন	এই নির্দিষ্ট সমস্যার পরিচিত সমাধান কী?	অংশগুলো কীভাবে ভাগ, কে কার ওপর নির্ভর করে?
উদাহরণ	Strategy, Observer, Factory	Layered, Hexagonal, Event-driven
ভুল সিদ্ধান্তের মূল্য	স্থানীয়, সহজে বদলানো যায়	ব্যাপক, বদলানো ব্যয়বহুল

Pattern-এর বেশিরভাগ বর্ণনা Java-র মতো Class-নির্ভর ভাষার জন্য লেখা। JavaScript-এ First-class Function, Object Literal ও Closure থাকায় অনেক Pattern কয়েক লাইনে হয়ে যায়; এ অধ্যায়ে আমরা সেই হালকা রূপই ব্যবহার করব।

> **সতর্কতা:** Pattern একটি সমস্যার সমাধান, লক্ষ্য নয়। একটি মাত্র Fee-র নিয়মের জন্য Strategy, অথবা একটি মাত্র Listener-এর জন্য Event Bus অপ্রয়োজনীয় জটিলতা। নিয়ম: **একই ধরনের দ্বিতীয় বা তৃতীয় প্রয়োজন এলে** Pattern-এ রূপান্তর করুন (প্রায়ই "Rule of Three" বলা হয়)।

২১.৩ Factory: তৈরির নিয়ম এক জায়গায়

Factory একটি Function, যা Object তৈরি করে এবং তৈরির সব নিয়ম—যাচাই, Default মান, কোন ধরনের Object—নিজের ভেতরে রাখে। ব্যবহারকারী Code জানে না, জানতেও চায় না, ভেতরে কোন Implementation।

factory.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createNotifier(channel) {
  const notifiers = {
    sms: { send: (to, text) => `SMS ${to}: ${text.slice(0, 40)}` },
    email: { send: (to, text) => `Email ${to}: ${text}` },
    console: { send: (to, text) => `LOG ${to}: ${text}` },
  };
  if (!Object.hasOwn(notifiers, channel)) {
    throw new Error(`অজানা Channel: ${channel}`);
  }
  return notifiers[channel];
}

for (const channel of ['sms', 'email']) {
  const notifier = createNotifier(channel);
  console.log(notifier.send('rafi', 'আপনার Visa আবেদন জমা হয়েছে'));
}
try {
```

```
  createNotifier('fax');
} catch (error) {
  console.log(error.message);
}
```

Output

```
SMS rafi: আপনার Visa আবেদন জমা হয়েছে
Email rafi: আপনার Visa আবেদন জমা হয়েছে
অজানা Channel: fax
```

ব্যবহারকারী Code প্রতিটি Notifier-কে একইভাবে ডাকে (`send`)—এটিই Polymorphism, Class ছাড়াই। নতুন Channel যোগ করতে শুধু Factory বদলায়। Project-এর `createApplication` আরেক ধরনের Factory: এটি নিশ্চিত করে যে একটি আবেদন **কখনো অবৈধ অবস্থায় তৈরি হতে পারে না**—সব যাচাই পেরোলে তবেই Object।

২১.৪ Strategy: if/else-এর শিকলের বদলে বিনিময়যোগ্য নিয়ম

যখন একই কাজের কয়েকটি বিকল্প নিয়ম থাকে এবং নতুন নিয়ম যোগ হতেই থাকে, তখন প্রতিটি নিয়মকে একই আকারের একটি Object বা Function-এ রাখুন, এবং চলার সময় বেছে নিন। একে বলে **Strategy**।

strategy.js · JavaScript চলবে: Browser Console ও Node.js

```
const discountStrategies = {
  none: () => 0,
  earlyBird: ({ daysAhead, price }) => (daysAhead >= 60 ? price * 0.1 : 0),
  group: ({ travelers, price }) => (travelers >= 5 ? price * 0.15 : 0),
};

function finalPrice(order, strategyName) {
  const strategy = discountStrategies[strategyName] ?? discountStrategies.none;
  return order.price - Math.round(strategy(order));
}

const order = { price: 40000, travelers: 5, daysAhead: 75 };
for (const name of ['none', 'earlyBird', 'group', 'unknown']) {
  console.log(name.padEnd(10), finalPrice(order, name));
}

// নতুন নিয়ম: পুরোনো কোনো লাইন না বদলে যোগ।
discountStrategies.student = ({ price }) => price * 0.2;
console.log('student'.padEnd(10), finalPrice(order, 'student'));
```

Output

```
none       40000
earlyBird  36000
group      34000
unknown    40000
student    32000
```

`finalPrice` কোনো নিয়মের বিস্তারিত জানে না; তার কোনো `if (type === 'group')` নেই। এটি **Open/Closed Principle**-এর ব্যবহারিক রূপ: নতুন আচরণের জন্য খোলা, পুরোনো Code বদলানোর জন্য বন্ধ। একটি সতর্কতা: এখানে `unknown`-এর জন্য Default নিয়ম নেওয়া হয়েছে; ব্যবসার নিয়মে অজানা নাম অনেক সময় Error হওয়াই উচিত (Project-এর `policyFor` তাই করে), এবং Key খোঁজায় `??`-এর আগে `Object.hasOwn` দিয়ে উত্তরাধিকারের নাম আটকানো নিরাপদ (অধ্যায় ২০)।

২১.৫ Observer: Publisher জানে না কে শুনছে

Observer (বা Publish-Subscribe) Pattern-এ একটি অংশ ঘটনা ঘোষণা করে, এবং আগ্রহী অংশগুলো নিজেরাই নিবন্ধন করে। ঘোষণাকারী জানে না কতজন শুনছে, বা তারা কী করে। Browser-এর `addEventListener` এই Pattern-ই।

observer.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createEventBus() {
  const handlers = new Map();
  return {
    on(type, handler) {
      if (!handlers.has(type)) {
        handlers.set(type, new Set());
      }
      handlers.get(type).add(handler);
      return () => handlers.get(type).delete(handler);
    },
    emit(type, payload) {
      for (const handler of [...(handlers.get(type) ?? [])]) {
        handler(payload);
      }
    },
  };
}

const bus = createEventBus();
const stopEmail = bus.on('approved', (id) => {
  console.log(`Email: ${id} অনুমোদিত`);
});
bus.on('approved', (id) => console.log(`Report হালনাগাদ: ${id}`));
```

```
bus.emit('approved', 'VF-001');
stopEmail();
bus.emit('approved', 'VF-002');
```

Output

```
Email: VF-001 অনুমোদিত
Report হালনাগাদ: VF-001
Report হালনাগাদ: VF-002
```

`on` একটি Unsubscribe Function ফেরত দেয়—অধ্যায় ১৯-এর Memory Leak এড়ানোর অভ্যাস। Observer-এর মূল্য: প্রবাহ লুকিয়ে যায়। Code পড়ে বোঝা কঠিন যে `emit`-এর পর কী কী ঘটবে, এবং একটি Listener-এর Error পুরো `emit` থামিয়ে দিতে পারে। Project-এর Event Bus তাই প্রতিটি Handler আলাদা `try/catch`-এ চালায়। নিয়ম: Event ব্যবহার করুন "ঘটে গেছে" জানাতে (`application:changed`), "এটা করো" আদেশের জন্য নয়—আদেশের জন্য সরাসরি Function Call অনেক স্পষ্ট।

২১.৬ Repository, Adapter এবং Dependency Injection

Repository তথ্য কোথায় ও কীভাবে থাকে তা লুকায়: Service শুধু জানে `save`, `get`, `list`। পেছনে Memory, `localStorage`, না Server-এর API—তা Repository-র বিষয়। একটি বাইরের Interface-কে (যেমন `localStorage`-এর `getItem`/`setItem`) আমাদের চাওয়া Interface-এ রূপান্তর করাকে বলে **Adapter**।

Dependency Injection (DI) মানে একটি অংশ তার নির্ভরতা নিজে তৈরি বা Import করে না; বাইরে থেকে পায়। এতে Service-কে একটি নকল Repository দিয়ে Test করা যায়, এবং Browser ও Node.js-এ ভিন্ন Repository দিয়ে চালানো যায়।

di.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createGreetingService({ repository, clock }) {
  return {
    greet(id) {
      const user = repository.find(id);
      const hour = clock().getHours();
      const part = hour < 12 ? 'সুপ্রভাত' : 'শুভ সন্ধ্যা';
      return user ? `${part}, ${user.name}` : 'অজানা ব্যবহারকারী';
    },
  };
}

// Test-এর জন্য: স্থির তথ্য ও স্থির ঘড়ি।
const fakeRepository = { find: (id) => (id === 1 ? { name: 'Mina' } : null) };
const morning = () => new Date(2026, 9, 15, 9, 0);
const evening = () => new Date(2026, 9, 15, 19, 0);

const at = (clock) =>
```

```
  createGreetingService({ repository: fakeRepository, clock });
console.log(at(morning).greet(1));
console.log(at(evening).greet(1));
console.log(at(morning).greet(2));
```

Output

```
সুপ্রভাত, Mina
শুভ সন্ধ্যা, Mina
অজানা ব্যবহারকারী
```

এখানে কোনো DI Framework নেই—শুধু একটি Object Parameter। ঘড়িও একটি নির্ভরতা: `new Date()` ভেতরে লিখলে সকাল-সন্ধ্যার আচরণ Test করতে দিনের নির্দিষ্ট সময়ের জন্য অপেক্ষা করতে হতো। এখন একই Test যেকোনো সময়ে একই ফল দেয়।

সব নির্ভরতা কোথাও না কোথাও সত্যিই তৈরি ও জোড়া লাগাতে হয়। সেই একমাত্র জায়গার নাম **Composition Root**—সাধারণত Application-এর প্রবেশ File (Project-এ `main.mjs` ও `demo.mjs`)। বাকি সব Code শুধু Parameter পায়।

২১.৭ Module-Singleton: সুবিধা ও ঝুঁকি

ES Module একবারই চলে, এবং প্রতিটি `import` একই Module-এর একই অবস্থা পায়। তাই Module-এর স্তরে একটি Variable কার্যত একটি **Singleton**:

counter.mjs · JavaScript — চলবে: Node.js

```
let count = 0;
export const increment = () => {
  count += 1;
  return count;
};
```

main.mjs · JavaScript — চলবে: Node.js

```
import { increment } from './counter.mjs';

const again = await import('./counter.mjs');
increment();
console.log(again.increment(), again.increment === increment);
```

Output

```
2 true
```

দুটি ভিন্ন `import` একই `count` ভাগ করে। একটি Logger বা Configuration-এর জন্য এটি সুবিধাজনক। কিন্তু ঝুঁকিও এখানে: এক Test-এ বদলানো অবস্থা পরের Test-এ রয়ে যায়, এবং যে Code এমন Module Import করে, তাকে অন্য Implementation দিয়ে চালানো যায় না। তাই অবস্থাযুক্ত জিনিস (Repository, Event Bus) Project-এ Factory Function দিয়ে তৈরি এবং Inject করা হয়েছে, Module-এর স্তরে নয়।

২১.৮ State Machine: Workflow-কে তথ্য হিসেবে

একটি Visa আবেদন যেকোনো অবস্থা থেকে যেকোনো অবস্থায় যেতে পারে না: "অনুমোদিত" আবেদন আবার "জমা" হয় না। এই নিয়ম `if/else`-এ ছড়িয়ে লিখলে দ্রুত অসামঞ্জস্য তৈরি হয়। পরিবর্তে পুরো Workflow-কে একটি টেবিলে রাখুন: প্রতিটি অবস্থায় কোন কাজ চলে, এবং কাজের পর নতুন অবস্থা।

অবস্থা	কাজ	নতুন অবস্থা
draft	submit / withdraw	submitted / withdrawn
submitted	requestDocuments / approve / reject / withdraw	documents-requested / approved / rejected / withdrawn
documents-requested	resubmit / withdraw	submitted / withdrawn
approved, rejected, withdrawn	কোনোটি নয়	চূড়ান্ত

টেবিলটিই Code (অনুচ্ছেদ ২১.১০-এর `workflow.mjs`)। এর তিনটি সুবিধা: অবৈধ পরিবর্তন এক জায়গায় আটকায়; UI টেবিল থেকেই জানে কোন Button দেখাতে হবে (`availableActions`), তাই UI ও নিয়ম কখনো অমিল হয় না; এবং ব্যবসার মানুষ টেবিলটি পড়ে যাচাই করতে পারেন। এর চেয়ে জটিল প্রয়োজনে (শর্তযুক্ত পরিবর্তন, সময়সীমা, Nested অবস্থা) XState-এর মতো Library আছে।

২১.৯ Layered Architecture এবং নির্ভরতার দিক

Project-এর Code চারটি স্তরে:

স্তর	Folder	দায়িত্ব	নির্ভর করতে পারে
Domain	`src/domain`	ব্যবসার নিয়ম: আবেদন, Fee, Workflow	কিছুর ওপর নয়
Application	`src/app`	Use Case: তৈরি, কাজ, সারাংশ; Event	Domain
Infrastructure	`src/infra`	সংরক্ষণ: Memory, Web Storage	কোনো স্তর নয় (শুধু চুক্তি মানে)
UI	`src/ui`	পর্দায় আঁকা	Domain

স্তর	Folder	দায়িত্ব	নির্ভর করতে পারে
Composition Root	`src/main.mjs`, `demo.mjs`	সব জোড়া লাগানো	সব

মূল নিয়ম: **নির্ভরতা ভেতরের দিকে, ব্যবসার নিয়মের দিকে।** Domain জানে না Browser আছে কি না, `localStorage` আছে কি না; তাই একই Domain Node.js-এর Test-এ, Browser-এ এবং ভবিষ্যতে একটি Server-এ হুবহু চলে। Service Repository-র ওপর নির্ভর করে, কিন্তু কোনো নির্দিষ্ট Repository Import করে না—সে শুধু চুক্তি (`save`, `get`, `list`) জানে, এবং Composition Root একটি বাস্তব Repository পাঠায়। একে বলে **Dependency Inversion**। এই ধারণার বিস্তৃত রূপগুলোর নাম Hexagonal (Ports and Adapters) বা Clean Architecture।

নিয়ম শুধু লিখে রাখলে সময়ের সঙ্গে ভাঙে: কেউ তাড়াহুড়োয় Domain-এ একটি `localStorage` লিখে ফেলেন। Project-এর **Architecture Test** প্রতিটি File-এর `import` পড়ে নিয়মটি যাচাই করে, তাই নিয়ম ভাঙলে Test ব্যর্থ হয়।

২১.১০ Project: VisaFlow

VisaFlow একটি ভ্রমণ সংস্থার Visa আবেদনের Workflow Tracker। কর্মী আবেদন তৈরি করেন (আবেদনকারী, দেশ, Visa-র ধরন, যাত্রী, Express), এবং আবেদনটি জমা, কাগজ চাওয়া, অনুমোদন, প্রত্যাখ্যান বা প্রত্যাহারের পথে এগোয়। Fee আসে Visa-র ধরনের Strategy থেকে। একই Domain ও Service দুই Composition Root-এ চলে: Browser-এ (`localStorage`-এ সংরক্ষণ, DOM UI) এবং Node.js-এ (Memory-তে, Console-এ)।

> **নোট:** VisaFlow একটি শিক্ষামূলক Demo। Fee-গুলো কাল্পনিক; এতে Authentication, Authorization, Server বা ভাগ করা Database নেই, এবং `localStorage`-এর তথ্য ব্যবহারকারী নিজেই বদলাতে পারেন (অধ্যায় ২০)। বাস্তব System-এ Domain ও Service একই রেখে একটি Server-ভিত্তিক Repository Adapter ও Authorization যোগ করতে হবে—এই Architecture-এর সুবিধা ঠিক সেখানেই।

File-এর কাঠামো

Folder-এর কাঠামো

```
visa-flow/
├── index.html, style.css
├── demo.mjs                  Node.js Composition Root
├── src/
│   ├── main.mjs              Browser Composition Root
│   ├── domain/
│   │   ├── errors.mjs        DomainError
│   │   ├── fees.mjs          Fee-র Strategy ও Express Decorator
│   │   ├── workflow.mjs      State Machine
│   │   └── application.mjs   আবেদনের Factory
│   ├── app/
│   │   ├── events.mjs        Event Bus (Observer)
│   │   ├── ids.mjs           ক্রমিক ID
```

```
│   │   └── service.mjs       Use Case (DI)
│   ├── infra/
│   │   ├── memory-repository.mjs
│   │   └── storage-repository.mjs   Web Storage Adapter
│   └── ui/
│       └── view.mjs          DOM-এ আঁকা
└── tests/                    domain, service, repositories, architecture
```

Domain: errors.mjs ও fees.mjs

errors.mjs · JavaScript — চলবে: Node.js

```
// Domain-এর নিয়ম ভাঙলে এই Error; code দিয়ে UI বা Test সিদ্ধান্ত নেয়।
export class DomainError extends Error {
  constructor(code, message) {
    super(message);
    this.name = 'DomainError';
    this.code = code;
  }
}
```

fees.mjs · JavaScript — চলবে: Node.js

```
import { DomainError } from './errors.mjs';

// Strategy: প্রতিটি Visa-র ধরনের নিজস্ব Fee-র নিয়ম, একই আকারের Object-এ।
const tourist = {
  label: 'Tourist',
  maxTravelers: 6,
  fee: ({ travelers }) => 3000 * travelers,
};

const business = {
  label: 'Business',
  maxTravelers: 3,
  fee: ({ travelers }) => 5500 * travelers + 1000,
};

const student = {
  label: 'Student',
  maxTravelers: 1,
  fee: () => 2500,
};

export const FEE_POLICIES = Object.freeze({ tourist, business, student });

// একটি Strategy-কে মুড়ে আরেকটি: মূল নিয়ম না বদলে Express-এর ৪০% যোগ।
export const withExpress = (policy) => ({
  ...policy,
```

```
  fee: (input) => Math.round(policy.fee(input) * 1.4),
});

export function policyFor(visaType, express = false) {
  if (!Object.hasOwn(FEE_POLICIES, visaType)) {
    throw new DomainError('UNKNOWN_TYPE', `অজানা Visa-র ধরন: ${visaType}`);
  }
  const policy = FEE_POLICIES[visaType];
  return express ? withExpress(policy) : policy;
}
```

তিনটি Strategy একই আকারের: `label`, `maxTravelers`, `fee`। `withExpress` একটি **Decorator**: মূল Strategy-কে মুড়ে নতুন একটি দেয়, যার আচরণ একটু বাড়ানো, কিন্তু মূলটি অপরিবর্তিত। Express আলাদা Strategy না হয়ে Decorator হওয়ায় তিন ধরন × Express-এর জন্য ছয়টি Object লিখতে হয়নি। `Object.hasOwn` উত্তরাধিকারের নাম (`toString`) Visa-র ধরন হিসেবে আটকায়।

Domain: workflow.mjs

workflow.mjs · JavaScript চলবে: Node.js

```
import { DomainError } from './errors.mjs';

// State Machine: কোন অবস্থায় কোন কাজ চলে, এবং কাজের পর নতুন অবস্থা।
export const WORKFLOW = Object.freeze({
  draft: Object.freeze({ submit: 'submitted', withdraw: 'withdrawn' }),
  submitted: Object.freeze({
    requestDocuments: 'documents-requested',
    approve: 'approved',
    reject: 'rejected',
    withdraw: 'withdrawn',
  }),
  'documents-requested': Object.freeze({
    resubmit: 'submitted',
    withdraw: 'withdrawn',
  }),
  approved: Object.freeze({}),
  rejected: Object.freeze({}),
  withdrawn: Object.freeze({}),
});

// শুধু নিজের Key: "toString"-এর মতো উত্তরাধিকারের নাম কখনো কাজ নয়।
const movesFrom = (status) =>
  (Object.hasOwn(WORKFLOW, status) ? WORKFLOW[status] : {});

export const availableActions = (status) => Object.keys(movesFrom(status));

export const isFinal = (status) => availableActions(status).length === 0;
```

```
// নতুন Object ফেরত দেয়; পুরোনোটি অপরিবর্তিত (অধ্যায় ১৮)।
export function applyAction(application, action, now) {
  const moves = movesFrom(application.status);
  if (!Object.hasOwn(moves, action)) {
    throw new DomainError('INVALID_TRANSITION',
      `${application.status} অবস্থায় "${action}" করা যায় না`);
  }
  const status = moves[action];
  const entry = Object.freeze({ status, action, at: now.toISOString() });
  return Object.freeze({
    ...application,
    status,
    history: Object.freeze([...application.history, entry]),
  });
}
```

অনুচ্ছেদ ২১.৮-এর টেবিল এখানে হুবহু Object। `applyAction` নতুন Frozen Object ফেরত দেয় এবং ইতিহাসে একটি Entry যোগ করে—পুরোনো আবেদন অপরিবর্তিত, তাই কে কখন কী করেছে তা পুরোটা থেকে যায়। সময় (`now`) Parameter-এ আসে; Domain নিজে ঘড়ি পড়ে না।

Domain: application.mjs

application.mjs · JavaScript চলবে: Node.js

```
import { DomainError } from './errors.mjs';
import { policyFor } from './fees.mjs';

export const COUNTRIES = Object.freeze([
  'Thailand', 'Malaysia', 'Japan', 'Turkey',
]);

// Factory: যাচাই, স্বাভাবিকীকরণ ও হিসাব এক জায়গায়; ফল সব সময় বৈধ ও Frozen।
// ID নেওয়া হয় শুধু সব যাচাই পেরোলে, যাতে ব্যর্থ চেষ্টায় ID নষ্ট না হয়।
export function createApplication(input, { newId, now }) {
  const applicant = String(input.applicant ?? '').trim().replace(/\s+/g, ' ');
  if (applicant.length < 2 || applicant.length > 60) {
    throw new DomainError('INVALID_APPLICANT', 'আবেদনকারীর নাম ২-৬০ অক্ষর');
  }
  if (!COUNTRIES.includes(input.country)) {
    throw new DomainError('INVALID_COUNTRY', `অসমর্থিত দেশ: ${input.country}`);
  }
  const express = input.express === true;
  const policy = policyFor(input.visaType, express);
  const { travelers } = input;
  if (!Number.isInteger(travelers) || travelers < 1
    || travelers > policy.maxTravelers) {
    throw new DomainError('INVALID_TRAVELERS',
      `${policy.label} Visa-য় সর্বোচ্চ ${policy.maxTravelers} জন যাত্রী`);
```

```
  }
  return Object.freeze({
    id: newId(),
    applicant,
    country: input.country,
    visaType: input.visaType,
    travelers,
    express,
    fee: policy.fee({ travelers }),
    status: 'draft',
    history: Object.freeze([
      Object.freeze({
        status: 'draft', action: 'create', at: now.toISOString(),
      }),
    ]),
  });
}
```

Factory নাম স্বাভাবিক করে (বাড়তি ফাঁকা সরায়), দেশ Allowlist-এ মেলায়, Strategy থেকে যাত্রীর সীমা নেয় (Student Visa-য় একজন), এবং Fee হিসাব করে। `express === true` : শুধু আসল `true` ; `"yes"` বা `1` নয়। ID-র উৎস (`newId`) ও সময় বাইরে থেকে আসে, তাই Factory পরিবেশ-মুক্ত; এবং `newId()` ডাকা হয় শুধু সব যাচাই পেরোনোর পর, যাতে একটি ব্যর্থ চেষ্টা একটি ID নষ্ট না করে।

Application: events.mjs, ids.mjs ও service.mjs

events.mjs · JavaScript — চলবে: Node.js

```
// Observer: Publisher জানে না কে শুনছে; Subscriber-রা একে অপরকে চেনে না।
export function createEventBus({ onError = () => {} } = {}) {
  const handlers = new Map();

  function on(type, handler) {
    if (!handlers.has(type)) {
      handlers.set(type, new Set());
    }
    handlers.get(type).add(handler);
    return () => handlers.get(type).delete(handler);
  }

  function emit(type, payload) {
    // Copy-র ওপর Loop: কোনো Handler নিজেকে সরালেও ক্রম ঠিক থাকে।
    for (const handler of [...(handlers.get(type) ?? [])]) {
      try {
        handler(payload);
      } catch (error) {
        onError(error, type);
      }
    }
```

```
  }

  const count = (type) => handlers.get(type)?.size ?? 0;
  return { on, emit, count };
}
```

ids.mjs · JavaScript চলবে: Node.js

```
// ক্রমিক ID: VF-001, VF-002, ... (Test-এ নিশ্চিত ফলের জন্য Inject করা হয়)।
export function createSequence(prefix, start = 1) {
  let next = start;
  return () => `${prefix}-${String(next++).padStart(3, '0')}`;
}

// সংরক্ষিত ID-গুলোর সবচেয়ে বড় সংখ্যার পর থেকে। সংখ্যা গুনে শুরু করলে
// মাঝে ফাঁক থাকলে একটি পুরোনো ID আবার আসত, এবং আগের আবেদন মুছে যেত।
export function sequenceAfter(prefix, ids) {
  const numbers = ids.map((id) => Number(id.slice(prefix.length + 1)))
    .filter(Number.isInteger);
  return createSequence(prefix, Math.max(0, ...numbers) + 1);
}
```

service.mjs · JavaScript চলবে: Node.js

```
import { createApplication } from '../domain/application.mjs';
import { applyAction } from '../domain/workflow.mjs';
import { DomainError } from '../domain/errors.mjs';

// Application Service: Use Case-গুলো সমন্বয় করে। কোন Repository বা কোন UI,
// তা জানে না; সব নির্ভরতা Constructor-এ আসে (Dependency Injection)।
export function createVisaService(deps) {
  const { repository, events, nextId, clock = () => new Date() } = deps;

  function create(input) {
    const context = { newId: nextId, now: clock() };
    const application = createApplication(input, context);
    repository.save(application);
    const { id, status, fee } = application;
    events.emit('application:created', { id, status, fee });
    return application;
  }

  function act(id, action) {
    const current = repository.get(id);
    if (!current) {
      throw new DomainError('NOT_FOUND', `${id} নামে কোনো আবেদন নেই`);
    }
    const next = applyAction(current, action, clock());
    repository.save(next);
```

```
    events.emit('application:changed',
      { id, action, from: current.status, to: next.status });
    return next;
  }

  function summary() {
    const counts = {};
    let approvedFees = 0;
    for (const application of repository.list()) {
      counts[application.status] = (counts[application.status] ?? 0) + 1;
      if (application.status === 'approved') {
        approvedFees += application.fee;
      }
    }
    return { counts, approvedFees };
  }

  return { create, act, summary, list: () => repository.list() };
}
```

Service-এর প্রতিটি Use Case একই ছন্দে: পড়া, Domain-কে সিদ্ধান্ত নিতে দেওয়া, সংরক্ষণ, ঘোষণা। ব্যবসার কোনো নিয়ম এখানে নেই—সব Domain-এ। লক্ষ করুন, `act`-এ Domain Error ছুড়লে `save` ও `emit` কোনোটিই চলে না: ব্যর্থ কাজ কোনো চিহ্ন রেখে যায় না। Service জানে না কোন Repository বা কতজন Subscriber।

Infrastructure: দুটি Repository

memory-repository.mjs · JavaScript — চলবে: Node.js

```
// Repository-র চুক্তি: save(application), get(id) বা null, list() id-র ক্রমে।
// প্রতিটি Method Copy দেয়-নেয়, যাতে বাইরের পরিবর্তন ভেতরে না পৌঁছায়।
export function createMemoryRepository() {
  const items = new Map();
  return {
    save(application) {
      items.set(application.id, structuredClone(application));
    },
    get(id) {
      return items.has(id) ? structuredClone(items.get(id)) : null;
    },
    list() {
      return [...items.values()]
        .map((item) => structuredClone(item))
        .toSorted((a, b) => a.id.localeCompare(b.id));
    },
  };
}
```

storage-repository.mjs · JavaScript চলবে: Node.js

```
// একই চুক্তি, কিন্তু Web Storage-এ (localStorage বা Test-এর নকল Storage)।
// Adapter: Storage-এর getItem/setItem-কে Repository-র ভাষায় রূপান্তর।
export function createStorageRepository(storage, prefix = 'visaflow:') {
  const keys = () =>
    Array.from({ length: storage.length }, (_, i) => storage.key(i))
      .filter((key) => key.startsWith(prefix));

  function read(key) {
    try {
      return JSON.parse(storage.getItem(key));
    } catch {
      return null;
    }
  }

  return {
    save(application) {
      storage.setItem(prefix + application.id, JSON.stringify(application));
    },
    get(id) {
      return read(prefix + id);
    },
    list() {
      return keys()
        .map(read)
        .filter((item) => item !== null)
        .toSorted((a, b) => a.id.localeCompare(b.id));
    },
  };
}
```

দুটিই একই চুক্তি মানে, এবং দুটিই Copy দেয়-নেয় (`structuredClone`, JSON)—ফেরত পাওয়া Object বদলালে সংরক্ষিত তথ্য বদলায় না। Storage Adapter `localStorage` সরাসরি ব্যবহার না করে একটি `storage` Parameter নেয়; Browser-এ `localStorage`, Test-এ একটি ছোট নকল। নিজস্ব Prefix-এর বাইরের Key এবং ভাঙা JSON উপেক্ষিত, কারণ `localStorage` অন্য Code বা ব্যবহারকারী বদলাতে পারেন।

UI ও Browser-এর Composition Root

view.mjs · JavaScript চলবে: Browser

```
import { availableActions } from '../domain/workflow.mjs';

const LABELS = {
  submit: 'জমা দিন',
  requestDocuments: 'কাগজ চাই',
  resubmit: 'আবার জমা',
```

```
  approve: 'অনুমোদন',
  reject: 'প্রত্যাখ্যান',
  withdraw: 'প্রত্যাহার',
};

// View শুধু আঁকে; কোন কাজ চলবে তা State Machine বলে, কী ঘটবে তা onAction।
export function renderApplications(list, applications, onAction) {
  list.replaceChildren(...applications.map((application) => {
    const row = document.createElement('li');
    row.dataset.id = application.id;
    const text = document.createElement('span');
    text.textContent = `${application.id} · ${application.applicant} · `
      + `${application.country} ${application.visaType} · ৳${application.fee}`
      + ` · ${application.status}`;
    row.append(text);
    for (const action of availableActions(application.status)) {
      const button = document.createElement('button');
      button.type = 'button';
      button.className = `act-${action}`;
      button.textContent = LABELS[action] ?? action;
      button.addEventListener('click', () => onAction(application.id, action));
      row.append(button);
    }
    return row;
  }));
}

export function renderSummary(element, { counts, approvedFees }) {
  const parts = Object.entries(counts).map(([status, n]) => `${status} ${n}`);
  element.textContent = `${parts.join(', ') || 'কোনো আবেদন নেই'}`
    + ` · অনুমোদিত Fee ৳${approvedFees}`;
}
```

main.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Browser

```
// Composition Root (Browser): একমাত্র জায়গা যেখানে সব অংশ জোড়া লাগে।
import { createVisaService } from './app/service.mjs';
import { createEventBus } from './app/events.mjs';
import { createStorageRepository } from './infra/storage-repository.mjs';
import { renderApplications, renderSummary } from './ui/view.mjs';
import { sequenceAfter } from './app/ids.mjs';

const $ = (selector) => document.querySelector(selector);
const repository = createStorageRepository(localStorage);
const events = createEventBus({ onError: (error) => console.error(error) });
const service = createVisaService({
  repository,
  events,
  nextId: sequenceAfter('VF', repository.list().map((item) => item.id)),
```

```
});

function refresh() {
  renderApplications($('#applications'), service.list(), handleAction);
  renderSummary($('#summary'), service.summary());
}

function handleAction(id, action) {
  try {
    service.act(id, action);
  } catch (error) {
    $('#message').textContent = error.message;
  }
}

// ...
```

main.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

// তিনজন Subscriber, একে অপরকে না চিনে: Console Log, পর্দা, বার্তা।
events.on('application:created', (e) => {
  console.log(`নতুন ${e.id} ৳${e.fee}`);
});
events.on('application:changed', (e) => {
  console.log(`${e.id}: ${e.from} থেকে ${e.to}`);
});
for (const type of ['application:created', 'application:changed']) {
  events.on(type, refresh);
  events.on(type, () => {
    $('#message').textContent = '';
  });
}

$('#new-application').addEventListener('submit', (event) => {
  event.preventDefault();
  try {
    service.create({
      applicant: $('#applicant').value,
      country: $('#country').value,
      visaType: $('#visa-type').value,
      travelers: Number($('#travelers').value),
      express: $('#express').checked,
    });
    event.target.reset();
  } catch (error) {
    $('#message').textContent = error.message;
  }
```

```
});

refresh();
```

`view.mjs` কোন Button দেখাবে তা নিজে ঠিক করে না; `availableActions` State Machine থেকে জানায়। সব লেখা `textContent` দিয়ে। `main.mjs` একমাত্র File যেখানে `localStorage`, DOM ও Service একসঙ্গে দেখা যায়। Event-এর তিনজন Subscriber (Console Log, পর্দা নতুন করে আঁকা, পুরোনো বার্তা মোছা) একে অপরকে চেনে না; একটি নতুন Subscriber (যেমন একটি Notification) যোগ করতে শুধু একটি `events.on`।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>VisaFlow</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="src/main.mjs"></script>
</head>
<body>
  <h1>VisaFlow <small>শিক্ষামূলক Demo</small></h1>
  <form id="new-application">
    <label>আবেদনকারী <input id="applicant" required></label>
    <label>দেশ <select id="country">
      <option>Thailand</option><option>Malaysia</option>
      <option>Japan</option><option>Turkey</option>
    </select></label>
    <label>ধরন <select id="visa-type">
      <option value="tourist">Tourist</option>
      <option value="business">Business</option>
      <option value="student">Student</option>
    </select></label>
    <label>যাত্রী <input id="travelers" type="number" value="1"></label>
    <label><input id="express" type="checkbox"> Express</label>
    <button id="create">আবেদন তৈরি</button>
  </form>
  <p id="message" role="alert"></p>
  <p id="summary"></p>
  <ul id="applications"></ul>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 44rem;
```

```
  margin: 2rem auto;
  padding: 0 1rem;
}

form {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem 1rem;
  align-items: end;
}

#applications li {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
  margin-bottom: 0.5rem;
}

#message {
  color: #b91c1c;
}
```

Node.js-এর Composition Root: demo.mjs

demo.mjs · JavaScript — চলবে: Node.js

```
// Composition Root (Node.js): একই Service, কিন্তু Memory Repository
// ও নির্দিষ্ট ঘড়ি।
import { createVisaService } from './src/app/service.mjs';
import { createEventBus } from './src/app/events.mjs';
import { createSequence } from './src/app/ids.mjs';
import { createMemoryRepository } from './src/infra/memory-repository.mjs';

const events = createEventBus();
const service = createVisaService({
  repository: createMemoryRepository(),
  events,
  nextId: createSequence('VF'),
  clock: () => new Date('2026-10-15T10:00:00Z'),
});
events.on('application:changed', (e) => {
  console.log(`  ${e.id}: ${e.from} থেকে ${e.to}`);
});

const rafi = service.create({ applicant: '  Rafi   Ahmed ', country: 'Thailand',
  visaType: 'tourist', travelers: 2 });
const mina = service.create({ applicant: 'Mina Akter', country: 'Japan',
  visaType: 'business', travelers: 1, express: true });
```

```
const sara = service.create({ applicant: 'Sara', country: 'Turkey',
  visaType: 'student', travelers: 1 });
console.log(rafi.applicant, rafi.fee, mina.fee, sara.fee);

service.act(rafi.id, 'submit');
service.act(rafi.id, 'approve');
service.act(mina.id, 'submit');
service.act(mina.id, 'requestDocuments');
service.act(sara.id, 'withdraw');

for (const [id, action] of [[sara.id, 'submit'], ['VF-999', 'submit']]) {
  try {
    service.act(id, action);
  } catch (error) {
    console.log(`  ${error.code}: ${error.message}`);
  }
}
const { counts, approvedFees } = service.summary();
console.log(counts, approvedFees);
```

Output

```
Rafi Ahmed 6000 9100 2500
  VF-001: draft থেকে submitted
  VF-001: submitted থেকে approved
  VF-002: draft থেকে submitted
  VF-002: submitted থেকে documents-requested
  VF-003: draft থেকে withdrawn
  INVALID_TRANSITION: withdrawn অবস্থায় "submit" করা যায় না
  NOT_FOUND: VF-999 নামে কোনো আবেদন নেই
{ approved: 1, 'documents-requested': 1, withdrawn: 1 } 6000
```

একই Service, একই Domain, কিন্তু Repository Memory-তে এবং ঘড়ি স্থির। Rafi-র নামের বাড়তি ফাঁকা Factory সরিয়েছে; Mina-র Business Express Fee (৫৫০০ + ১০০০) × ১.৪ = ৯১০০; প্রত্যাহার করা আবেদন আর জমা দেওয়া যায় না, এবং অজানা ID-র জন্য স্পষ্ট `NOT_FOUND`।

Test-এর কৌশল

Terminal

```
cd visa-flow
node --test
node demo.mjs
python -m http.server 8000
```

শেষ লাইনটি Browser UI-র জন্য: ES Module `file://` থেকে চলে না, তাই একটি Static Server লাগে (অধ্যায় ১২); Python না থাকলে যেকোনো Static Server, এবং Browser-এ `http://localhost:8000`। `node --test` ৩১টি Test চালায়, চারটি ভাগে:

File	কী যাচাই করে	Test
`domain.test.mjs`	Factory, প্রতিটি Fee Strategy ও Decorator, সম্পূর্ণ ও অবৈধ Workflow, Immutability	৮
`service.test.mjs`	Use Case, Event-এর ক্রম, ব্যর্থতায় কোনো চিহ্ন বা নষ্ট ID নেই, ID-র ক্রম, DI-তে নকল Repository, Event Bus-এর Error বিচ্ছিন্নতা	৯
`repositories.test.mjs`	একই চার Contract Test দুটি Adapter-এ, এবং Storage-এর ভাঙা তথ্য	৯
`architecture.test.mjs`	প্রতিটি স্তরের Import-এর নিয়ম, Composition Root	৫

Contract Test-এর ধারণাটি গুরুত্বপূর্ণ: একই Test-এর তালিকা প্রতিটি Repository-তে চলে, তাই ভবিষ্যতে একটি API Repository যোগ করলে শুধু একটি লাইন যোগ করে প্রমাণ করা যায় যে সেটিও একই আচরণ করে। এছাড়া এই বইয়ের QA-তে আসল Chrome-এ UI-র একটি End-to-end যাচাই চলেছে: Form দিয়ে আবেদন তৈরি, Button দিয়ে Workflow এগোনো, অবৈধ ইনপুটের বার্তা, এবং Page আবার লোড করার পর `localStorage` থেকে তথ্য ফিরে আসা।

এই End-to-end যাচাই প্রথমবার চালানোর সময় দুটি আসল ত্রুটি ধরেছিল, যা ২৯টি Unit Test-এর কোনোটি ধরেনি। প্রথমত, Service যাচাইয়ের **আগে** `nextId()` ডাকত, তাই প্রতিটি ব্যর্থ চেষ্টা একটি ID নষ্ট করত (VF-001-এর পরে সরাসরি VF-004)। দ্বিতীয়ত, Page আবার লোড হলে ID-র ক্রম শুরু হতো "সংরক্ষিত আবেদনের সংখ্যা + ১" থেকে; ID-তে ফাঁক থাকলে এটি একটি বিদ্যমান ID আবার দিত, এবং নতুন আবেদন পুরোনোটিকে মুছে ফেলত। সমাধান দুটি এখন Code-এ (Factory-র `newId`, এবং `ids.mjs`-এর `sequenceAfter`), এবং প্রতিটির জন্য একটি নতুন Unit Test যোগ হয়েছে, যাতে ত্রুটি ফিরে না আসে। শিক্ষা: প্রতিটি স্তর আলাদাভাবে সঠিক হলেও, জোড়া লাগানোর জায়গায় (Composition Root ও স্তরের সীমানা) ত্রুটি লুকাতে পারে—তাই Testing Pyramid-এর ওপরের স্তরেও কিছু Test দরকার (অধ্যায় ১৬)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Browser-এ কিছুই আঁকা হয় না	Page `file://` থেকে খোলা	Static Server
Architecture Test ব্যর্থ	কোনো স্তর নিষিদ্ধ Import করছে	নির্ভরতা Inject করুন, Import নয়
Button দেখা যায় কিন্তু কাজ হয় না	Workflow-এ কাজটির নাম ভুল	`availableActions` ও `WORKFLOW` মেলান
পুরোনো তথ্য অদ্ভুত	`localStorage`-এ পুরোনো Format	Prefix বদলান বা Storage মুছুন
Event-এর পর পর্দা হালনাগাদ হয় না	Subscriber নিবন্ধিত নয়	`events.count(type)` দেখুন

২১.১১ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
একটি Function-এ DOM, নিয়ম ও Storage	কোনোটি আলাদা বদলানো বা Test করা যায় না	স্তরে ভাগ
প্রয়োজন ছাড়া Pattern	অপ্রয়োজনীয় জটিলতা	দ্বিতীয়-তৃতীয় প্রয়োজনে
Domain-এ `localStorage`, `fetch` বা `document`	Domain পরিবেশ-নির্ভর	Repository ও Inject
Service-এ ব্যবসার নিয়ম	নিয়ম ছড়িয়ে পড়ে	Domain-এ
ভেতরে `new Date()` বা `Math.random()`	Test অনিশ্চিত	ঘড়ি ও ID Inject
Status-এর `if/else` ছড়ানো	অসামঞ্জস্য	টেবিলভিত্তিক State Machine
Event দিয়ে আদেশ দেওয়া	প্রবাহ লুকায়	সরাসরি Function Call
Unsubscribe না করা	Memory Leak	`on`-এর ফেরত Function
অবস্থাযুক্ত Module-Singleton	Test একে অপরকে প্রভাবিত করে	Factory ও Inject
স্তরের নিয়ম শুধু Document-এ	সময়ের সঙ্গে ভাঙে	Architecture Test

২১.১২ Debugging নির্দেশিকা

১. **কোন স্তরে?** ভুল হিসাব হলে Domain-এর Unit Test; ভুল ক্রম হলে Service; ভুল প্রদর্শন হলে View। স্তর আলাদা থাকায় খোঁজার জায়গা ছোট।

২. **Event-এর প্রবাহ:** অস্থায়ীভাবে একটি Subscriber যোগ করুন যা প্রতিটি Event Log করে; কে আগে, কে পরে চলে দেখুন।

৩. **State Machine:** `INVALID_TRANSITION` বার্তায় বর্তমান অবস্থা ও কাজের নাম থাকে; টেবিলের সঙ্গে মেলান।

৪. **Repository:** একই Contract Test দুটি Adapter-এ চালান; একটিতে ব্যর্থ হলে সমস্যা সেই Adapter-এ।

৫. **Composition Root:** "undefined is not a function" প্রায়ই একটি নির্ভরতা Inject না করার ফল; Service-এর Parameter মিলিয়ে দেখুন।

২১.১৩ পেশাদার অভ্যাস

- **যা একসঙ্গে বদলায়, তা একসঙ্গে রাখুন; যা ভিন্ন কারণে বদলায়, তা আলাদা।**
- **নির্ভরতা ভেতরের দিকে;** Domain সবচেয়ে স্থির ও পরিবেশ-মুক্ত।
- **Interface ছোট রাখুন:** Repository-র তিনটি Method যথেষ্ট হলে দশটি নয়।

- **Composition-এ পছন্দ, Inheritance-এ সংযম।**
- **একটি Composition Root;** বাকি Code শুধু Parameter পায়।
- **Pattern-এর নাম Code-এ ব্যবহার করুন** (`createStorageRepository` , `withExpress`), যাতে পাঠক সঙ্গে সঙ্গে চেনেন।
- **Architecture-এর সিদ্ধান্ত লিখে রাখুন** (ছোট Architecture Decision Record): কী, কেন, কোন বিকল্প বাদ।
- **নিয়ম Test দিয়ে পাহারা দিন,** শুধু Document দিয়ে নয়।

২১.১৪ অনুশীলনী

প্রাথমিক স্তর

১. প্রতিটি Code কোন Pattern: (ক) `button.addEventListener('click', fn)` , (খ) `createApplication(input, context)` , (গ) `sort((a, b) => a.price - b.price)` -এ তুলনার Function দেওয়া, (ঘ) `withExpress(policy)` ।

২. VisaFlow-এর একই আকারে একটি `transit` Fee Strategy লিখুন: যাত্রীপ্রতি ১,২০০ টাকা, সর্বোচ্চ ৪ জন; Express Decorator দিয়ে ২ জনের Fee দেখান।

৩. একটি Support Ticket-এর জন্য অবস্থা ও কাজের টেবিল তৈরি করুন: open, in-progress, resolved, closed।

৪. Event Bus-এ `once(type, handler)` যোগ করুন, যা প্রথমবারের পরেই নিজে সরে যায়।

৫. VisaFlow-এ কেন শুধু `main.mjs` ও `demo.mjs` Infrastructure Import করে? দুই বাক্যে লিখুন।

মধ্যম স্তর

৬. একটি Logging Decorator Repository লিখুন: যেকোনো Repository মুড়ে প্রতিটি `save` ও `get` গণনা করে, এবং আচরণ অপরিবর্তিত রাখে।

৭. VisaFlow-এর Workflow থেকে একটি Mermaid-ধাঁচের লেখা Diagram তৈরি করুন (`draft --submit--> submitted` ধরনের লাইন), যাতে Diagram সব সময় Code-এর সঙ্গে মেলে।

৮. Contract Test-এর মতো একটি তৃতীয় Repository (একটি JSON String-এ সব আবেদন রাখে) লিখুন, এবং দেখান যে Service তার সঙ্গেও কাজ করে।

৯. একটি Audit Subscriber লিখুন, যা প্রতিটি `application:changed` Event একটি তালিকায় রাখে এবং ৩টির পরে নিজেকে Unsubscribe করে।

চ্যালেঞ্জ স্তর

১০. আবেদনের ইতিহাস থেকে "আগের অবস্থায় ফেরা" (Undo) প্রয়োগ করুন: শেষ Entry বাদ দিয়ে নতুন Object, কিন্তু চূড়ান্ত অবস্থা থেকে Undo নিষিদ্ধ।

১১. একটি Strategy Registry লিখুন, যা Runtime-এ নতুন Fee Strategy নিবন্ধন করে, কিন্তু আকার (`label`, `maxTravelers`, `fee`) যাচাই না হলে প্রত্যাখ্যান করে।

১২. নিচের জট পাকানো Function-কে Domain (বিশুদ্ধ হিসাব) ও বাকি অংশে ভাগ করুন, এবং বিশুদ্ধ অংশটি Browser ছাড়াই Test করুন: `function onSubmit() { const n = Number(document.querySelector('#n').value); const fee = n > 3 ? n * 2800 : n * 3000; localStorage.setItem('fee', fee); document.querySelector('#out').textContent = fee; }`

২১.১৫ অনুশীলনীর সমাধান

সমাধান ১ — (ক) Observer: Browser Publisher, আপনার Function Subscriber। (খ) Factory: যাচাইসহ তৈরি। (গ) Strategy: তুলনার নিয়ম Parameter হিসেবে বিনিময়যোগ্য। (ঘ) Decorator: একটি Strategy মুড়ে আচরণ বাড়ানো।

সমাধান ২

transit.mjs · JavaScript — চলবে: Node.js

```
import { withExpress } from '../../project/visa-flow/src/domain/fees.mjs';

const transit = {
  label: 'Transit',
  maxTravelers: 4,
  fee: ({ travelers }) => 1200 * travelers,
};

const two = { travelers: 2 };
console.log(transit.fee(two), withExpress(transit).fee(two));
console.log(withExpress(transit).maxTravelers);
```

Output

```
2400 3360
4
```

Decorator নতুন Strategy-র সঙ্গেও কোনো পরিবর্তন ছাড়াই কাজ করেছে, কারণ তা শুধু চুক্তি (`fee`) জানে। Project-এ স্থায়ীভাবে যোগ করতে `FEE_POLICIES`-এ একটি লাইন।

সমাধান ৩

অবস্থা	কাজ	নতুন অবস্থা
open	start / close	in-progress / closed
in-progress	resolve / pause	resolved / open
resolved	reopen / close	in-progress / closed
closed	কোনোটি নয়	চূড়ান্ত

সমাধান ৪

once.mjs · JavaScript চলবে: Node.js

```
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';

function once(bus, type, handler) {
  const off = bus.on(type, (payload) => {
    off();
    handler(payload);
  });
  return off;
}

const bus = createEventBus();
once(bus, 'ready', (n) => console.log(`প্রথমবার: ${n}`));
bus.emit('ready', 1);
bus.emit('ready', 2);
console.log(bus.count('ready'));
```

Output

```
প্রথমবার: 1
0
```

Event Bus না বদলে তার ওপর একটি ছোট Function—Composition।

সমাধান ৫ — Infrastructure পরিবেশ-নির্ভর (`localStorage` শুধু Browser-এ), তাই যে Code তা Import করে, সেটি সেই পরিবেশে আটকে যায়। Import শুধু Composition Root-এ থাকায় Domain ও Service যেকোনো পরিবেশে চলে এবং নকল দিয়ে Test করা যায়।

সমাধান ৬

logging-repo.mjs · JavaScript — চলবে: Node.js

```
import { createMemoryRepository }
  from '../../project/visa-flow/src/infra/memory-repository.mjs';

function withCounting(repository) {
  const counts = { save: 0, get: 0 };
  return {
    counts,
    save(item) {
      counts.save += 1;
      return repository.save(item);
    },
    get(id) {
      counts.get += 1;
      return repository.get(id);
    },
    list: () => repository.list(),
  };
}

const repository = withCounting(createMemoryRepository());
repository.save({ id: 'VF-001', status: 'draft' });
repository.get('VF-001');
repository.get('VF-002');
console.log(repository.counts, repository.list().length);
```

Output

```
{ save: 1, get: 2 } 1
```

সমাধান ৭

diagram.mjs · JavaScript — চলবে: Node.js

```
import { WORKFLOW } from '../../project/visa-flow/src/domain/workflow.mjs';

const lines = Object.entries(WORKFLOW).flatMap(([from, moves]) =>
  Object.entries(moves).map(([action, to]) => `  ${from} --${action}--> ${to}`));

console.log('stateDiagram');
console.log(lines.join('\n'));
```

Output

```
stateDiagram
  draft --submit--> submitted
  draft --withdraw--> withdrawn
  submitted --requestDocuments--> documents-requested
  submitted --approve--> approved
  submitted --reject--> rejected
  submitted --withdraw--> withdrawn
  documents-requested --resubmit--> submitted
  documents-requested --withdraw--> withdrawn
```

Diagram Code থেকে তৈরি, তাই Workflow বদলালে Diagram নিজেই বদলায়—হাতে আঁকা Diagram যেমন পুরোনো হয়ে যায়, এটি তেমন হয় না।

সমাধান ৮

json-repo.mjs · JavaScript চলবে: Node.js

```
import { createVisaService } from '../../project/visa-flow/src/app/service.mjs';
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';
import { createSequence } from '../../project/visa-flow/src/app/ids.mjs';

function createJsonRepository() {
  let json = '{}';
  const all = () => JSON.parse(json);
  return {
    save(item) {
      json = JSON.stringify({ ...all(), [item.id]: item });
    },
    get: (id) => (Object.hasOwn(all(), id) ? all()[id] : null),
    list: () => Object.values(all()).toSorted((a, b) => a.id.localeCompare(b.id)),
    size: () => json.length,
  };
}

const repository = createJsonRepository();
const service = createVisaService({
  repository, events: createEventBus(), nextId: createSequence('VF'),
});
service.create({
  applicant: 'Sara', country: 'Japan', visaType: 'student', travelers: 1,
});
service.act('VF-001', 'submit');
const rows = service.list().map((a) => `${a.id} ${a.status}`);
console.log(rows, repository.size() > 0);
```

Output

```
[ 'VF-001 submitted' ] true
```

Service-এর একটি লাইনও বদলাতে হয়নি। পুরো প্রমাণের জন্য এটিকে `repositories.test.mjs`-এর `adapters` তালিকায় যোগ করলে চারটি Contract Test এর ওপরও চলবে।

সমাধান ৯

audit.mjs · JavaScript চলবে: Node.js

```
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';

const bus = createEventBus();
const audit = [];
const stop = bus.on('application:changed', (event) => {
  audit.push(`${event.id}:${event.to}`);
  if (audit.length === 3) {
    stop();
  }
});

for (const to of ['submitted', 'documents-requested', 'submitted', 'approved']) {
  bus.emit('application:changed', { id: 'VF-001', to });
}
console.log(audit.join(' | '), bus.count('application:changed'));
```

Output

```
VF-001:submitted | VF-001:documents-requested | VF-001:submitted 0
```

সমাধান ১০

undo.mjs · JavaScript চলবে: Node.js

```
import { createApplication }
  from '../../project/visa-flow/src/domain/application.mjs';
import { applyAction, isFinal }
  from '../../project/visa-flow/src/domain/workflow.mjs';

function undo(application) {
  if (isFinal(application.status) || application.history.length < 2) {
    throw new Error(`${application.status} থেকে Undo করা যায় না`);
  }
  const history = application.history.slice(0, -1);
  const { status } = history.at(-1);
  return Object.freeze({ ...application, status, history });
```

```
}

const now = new Date('2026-10-15T10:00:00Z');
const draft = createApplication(
  { applicant: 'Rafi', country: 'Malaysia', visaType: 'tourist', travelers: 1 },
  { newId: () => 'VF-001', now });
const submitted = applyAction(draft, 'submit', now);
const asked = applyAction(submitted, 'requestDocuments', now);

console.log(undo(asked).status, undo(undo(asked)).status);
for (const app of [draft, applyAction(draft, 'withdraw', now)]) {
  try {
    undo(app);
  } catch (error) {
    console.log(error.message);
  }
}
```

Output

```
submitted draft
draft থেকে Undo করা যায় না
withdrawn থেকে Undo করা যায় না
```

ইতিহাস Immutable হওয়ায় Undo সহজ: শেষ Entry বাদ দিলেই আগের অবস্থা। চূড়ান্ত অবস্থা থেকে Undo নিষিদ্ধ করা একটি ব্যবসার সিদ্ধান্ত, তাই এটি Domain-এ থাকা উচিত।

সমাধান ১১

registry.mjs · JavaScript চলবে: Node.js

```
function createRegistry() {
  const strategies = new Map();
  return {
    register(name, strategy) {
      const valid = typeof strategy?.label === 'string'
        && Number.isInteger(strategy.maxTravelers) && strategy.maxTravelers > 0
        && typeof strategy.fee === 'function';
      if (!valid) {
        throw new TypeError(`${name}: label, maxTravelers ও fee প্রয়োজন`);
      }
      if (strategies.has(name)) {
        throw new Error(`${name} আগেই নিবন্ধিত`);
      }
      strategies.set(name, Object.freeze({ ...strategy }));
    },
    get: (name) => strategies.get(name) ?? null,
    names: () => [...strategies.keys()],
```

```
  };
}

const registry = createRegistry();
registry.register('transit',
  { label: 'Transit', maxTravelers: 4, fee: () => 1200 });
for (const [name, strategy] of [['bad', { label: 'Bad' }], ['transit', {}]]) {
  try {
    registry.register(name, strategy);
  } catch (error) {
    console.log(error.name, error.message);
  }
}
console.log(registry.names(), registry.get('transit').fee());
```

Output

```
TypeError bad: label, maxTravelers ও fee প্রয়োজন
TypeError transit: label, maxTravelers ও fee প্রয়োজন
[ 'transit' ] 1200
```

`Map` উত্তরাধিকারের Key-এর সমস্যা এড়ায়, এবং আকার যাচাই নিশ্চিত করে যে ভুল Strategy পরে, ব্যবহারের মুহূর্তে নয়, নিবন্ধনের সময়ই ধরা পড়ে।

সমাধান ৯২

refactor.mjs · JavaScript — চলবে: Node.js

```
// Domain: বিশুদ্ধ, Browser ছাড়াই Test করা যায়।
export function groupFee(travelers) {
  if (!Number.isInteger(travelers) || travelers < 1) {
    throw new RangeError('যাত্রী একটি ধনাত্মক পূর্ণসংখ্যা');
  }
  return travelers * (travelers > 3 ? 2800 : 3000);
}

// UI ও Storage: পাতলা আবরণ, নির্ভরতা Parameter-এ।
export function onSubmit({ read, save, show }) {
  const fee = groupFee(Number(read('#n')));
  save('fee', fee);
  show('#out', fee);
}

const shown = [];
const show = (_, value) => shown.push(value);
onSubmit({ read: () => '4', save: () => {}, show });
console.log([1, 3, 4].map(groupFee), shown);
```

Output

```
[ 3000, 9000, 11200 ] [ 11200 ]
```

Browser-এ `onSubmit`-কে `read: (s) => document.querySelector(s).value`, `save: (k, v) => localStorage.setItem(k, v)` ইত্যাদি দিয়ে ডাকা হবে। নিয়মটি এখন এক জায়গায়, নামযুক্ত, এবং Test-যোগ্য; অবৈধ ইনপুট (`NaN`) আর নিঃশব্দে `NaN` Fee হিসেবে সংরক্ষিত হয় না।

২১.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. একই কাজের একাধিক বিনিময়যোগ্য নিয়ম, চলার সময় বেছে নেওয়া—কোন Pattern?

ক) Observer

খ) Strategy

গ) Singleton

ঘ) Repository

প্রশ্ন ২. Factory Function-এর প্রধান সুবিধা?

ক) তৈরির নিয়ম ও যাচাই এক জায়গায়, ব্যবহারকারী Implementation জানে না

খ) Code দ্রুত চলে

গ) Memory কম লাগে

ঘ) Class ব্যবহার বাধ্যতামূলক হয়

প্রশ্ন ৩. Observer Pattern-এর একটি ঝুঁকি?

ক) Subscriber যোগ করা যায় না

খ) Publisher সব Subscriber-কে চেনে

গ) প্রবাহ লুকিয়ে যায় এবং Unsubscribe না করলে Leak

ঘ) একটির বেশি Subscriber থাকতে পারে না

প্রশ্ন ৪. Dependency Injection-এর মূল অর্থ?

ক) সব Module Global-এ রাখা

খ) Library Install করা

গ) সব Code এক File-এ

ঘ) নির্ভরতা নিজে তৈরি না করে বাইরে থেকে পাওয়া

প্রশ্ন ৫. Composition Root কী?

ক) Project-এর Root Folder

খ) একমাত্র জায়গা যেখানে সব নির্ভরতা তৈরি ও জোড়া লাগে

গ) সবচেয়ে বড় Class

ঘ) DOM-এর Root Element

প্রশ্ন ৬. Layered Architecture-এ Domain স্তর কীসের ওপর নির্ভর করা উচিত?

ক) কোনো বাইরের স্তরের ওপর নয়

খ) UI-এর ওপর

গ) `localStorage`-এর ওপর

ঘ) Infrastructure-এর ওপর

প্রশ্ন ৭. VisaFlow-এ `withdrawn` অবস্থার আবেদনে `submit` করলে কী হয়?

ক) আবেদন জমা হয়

খ) কিছুই ঘটে না, নিঃশব্দে

গ) `INVALID_TRANSITION` DomainError, কিছু সংরক্ষিত হয় না

ঘ) অবস্থা `draft` হয়

প্রশ্ন ৮. Contract Test-এর উদ্দেশ্য?

ক) একই চুক্তির প্রতিটি Implementation একই আচরণ করে কি না যাচাই

খ) আইনি চুক্তি যাচাই

গ) শুধু একটি Repository Test

ঘ) UI-এর রং যাচাই

প্রশ্ন ৯. ES Module-এর স্তরে রাখা অবস্থা (Module-Singleton)-এর ঝুঁকি?

ক) প্রতিটি Import নতুন Copy পায়

খ) Module একাধিকবার চলে

গ) Browser-এ কাজ করে না

ঘ) অবস্থা Test-এর মধ্যে ভাগ হয়ে যায় এবং বদলানো কঠিন

প্রশ্ন ১০. কখন একটি Design Pattern প্রয়োগ করা উচিত?

ক) সব সময়, যত বেশি তত ভালো

খ) একই ধরনের প্রয়োজন বারবার এলে, যখন তা জটিলতা কমায়

গ) শুধু বড় দলে

ঘ) কখনো নয়

২১.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–ক · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Fee-র নিয়ম বা ছাড়ের নিয়ম।
২	ক	`createApplication` অবৈধ আবেদন তৈরি হতে দেয় না।
৩	গ	তাই `on` Unsubscribe ফেরত দেয় এবং Handler আলাদা `try/catch`-এ।
৪	ঘ	কোনো Framework লাগে না; একটি Parameter-ই যথেষ্ট।
৫	খ	`main.mjs` ও `demo.mjs`।
৬	ক	তাই Domain সব পরিবেশে চলে।
৭	গ	`demo.mjs`-এর Output-এ দেখা যায়।
৮	ক	দুটি Repository-তে একই চারটি Test।
৯	ঘ	এক Test-এর পরিবর্তন অন্যটিতে রয়ে যায়।
১০	খ	Pattern সমাধান, লক্ষ্য নয়।

২১.১৮ অধ্যায়ের সারসংক্ষেপ

- Architecture-এর লক্ষ্য পরিবর্তন সস্তা রাখা: ভিন্ন কারণে বদলানো অংশ আলাদা।
- Pattern নামযুক্ত সমাধান; JavaScript-এ Function ও Object দিয়ে হালকা রূপে; প্রয়োজন এলে তবেই।
- Factory তৈরির নিয়ম লুকায় ও অবৈধ Object আটকায়; Strategy if/else-এর শিকল ভাঙে; Decorator আচরণ বাড়ায় মূল না বদলে।
- Observer ঘোষণাকারী ও শ্রোতাকে আলাদা রাখে; Unsubscribe ও Error-এর বিচ্ছিন্নতা জরুরি।
- Repository সংরক্ষণ লুকায়, Adapter বাইরের Interface রূপান্তর করে; Contract Test সব Implementation-কে এক মানে রাখে।
- Dependency Injection ও একটি Composition Root: বাকি Code শুধু Parameter পায়; ঘড়ি ও ID-ও নির্ভরতা।
- State Machine Workflow-কে তথ্যে রূপান্তর করে; UI সেখান থেকেই কাজের তালিকা পায়।
- নির্ভরতা ভেতরের দিকে; Domain পরিবেশ-মুক্ত; নিয়ম Architecture Test-এ পাহারা।

পরের অধ্যায়ে Testing আরও গভীরে: Unit, Integration ও End-to-end Test-এর কৌশল, Mocking, এবং Test-Driven Development।

২১.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Design Pattern	একটি পরিচিত সমস্যার নামযুক্ত, পুনর্ব্যবহারযোগ্য সমাধান।
Factory	Object তৈরির নিয়ম লুকানো Function।
Strategy	বিনিময়যোগ্য নিয়ম, একই আকারে।
Decorator	একটি Object মুড়ে আচরণ বাড়ানো।
Observer	ঘটনার ঘোষণা ও স্বাধীন শ্রোতা।
Repository	সংরক্ষণের বিস্তারিত লুকানো চুক্তি।
Adapter	একটি Interface-কে আরেকটিতে রূপান্তর।
Dependency Injection	নির্ভরতা বাইরে থেকে দেওয়া।
Composition Root	যেখানে সব নির্ভরতা জোড়া লাগে।
State Machine	অবস্থা ও বৈধ পরিবর্তনের টেবিল।
Layered Architecture	দায়িত্বভিত্তিক স্তর ও নির্ভরতার দিক।
Contract Test	একই চুক্তির সব Implementation-এ একই Test।

২১.২০ আরও পড়ুন

- **Refactoring.Guru — Design Patterns:** `https://refactoring.guru/design-patterns` — প্রতিটি Pattern-এর ব্যাখ্যা ও উদাহরণ।
- **Patterns.dev:** `https://www.patterns.dev/` — JavaScript-কেন্দ্রিক Design ও Rendering Pattern।
- **Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern:** `https://martinfowler.com/articles/injection.html` — DI-এর মূল প্রবন্ধ।
- **Alistair Cockburn — Hexagonal Architecture:** `https://alistair.cockburn.us/hexagonal-architecture/` — Ports and Adapters।
- **MDN — JavaScript modules:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules` — Module-এর আচরণ।

অধ্যায় ২২

২২

JavaScript Testing Mastery: Unit Testing, Integration Testing, Mocking, TDD এবং E2E

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- কোন আচরণ কোন স্তরের Test-এ (Unit, Integration, End-to-end) যাচাই করা উচিত, তা যুক্তি দিয়ে ঠিক করতে।
- Arrange–Act–Assert কাঠামো ও Table-driven Test দিয়ে পরিষ্কার, পড়তে সহজ Test লিখতে।
- Equivalence Partitioning ও Boundary Value Analysis দিয়ে কম Test-এ বেশি ত্রুটি ধরতে।
- Property-based Test দিয়ে হাজারো ইনপুটে একটি সাধারণ নিয়ম যাচাই করতে।
- Red–Green–Refactor চক্রে Test-Driven Development অনুশীলন করতে।
- `node:test`-এর `mock.fn`, `mock.method` ও `mock.timers` দিয়ে Network, সময় ও অপেক্ষা নিয়ন্ত্রণ করতে।
- স্থানীয় HTTP Server-এর বিরুদ্ধে Integration Test এবং আসল Browser-এ End-to-end Test লিখতে।
- Coverage Report পড়তে, এবং এর সীমা বুঝতে।
- একটি CI Pipeline-এ Test স্বয়ংক্রিয় করতে।
- একটি সম্পূর্ণ পরীক্ষিত **CurrencyDesk** Quote Service তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৬: `node:test`, `assert`, Test Double-এর ধারণা, Testing Pyramid।
- অধ্যায় ১৯: Seed-যুক্ত এলোমেলো সংখ্যা।
- অধ্যায় ২০: স্থানীয় HTTP Server-এর বিরুদ্ধে Test।
- অধ্যায় ২১: Dependency Injection, Contract Test।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার মুদ্রা-বিনিময় Counter-এর কথা ভাবুন। Software একজন গ্রাহককে বলে: ১,০০০ ডলার পাঠালে Fee কত, মোট কত দেবেন, এবং কত টাকা পাবেন। এখানে একটি পয়সার ভুলও গ্রাহকের অভিযোগ, হিসাবের গরমিল, বা নিয়ন্ত্রক সংস্থার প্রশ্ন ডেকে আনে। আর ভুলগুলো সাধারণত মাঝখানে লুকায় না; লুকায় **সীমায়** (৯৯৯.৯৯ বনাম ১,০০০.০০), **ব্যর্থতায়** (Rate Server বন্ধ), এবং **সময়ে** (Quote-এর মেয়াদ)।

অধ্যায় ১৬-এ আপনি Test লিখতে শিখেছেন। এই অধ্যায়ের প্রশ্ন আরেক ধাপ ওপরে: **কোন Test লিখবেন, কতগুলো, এবং কীভাবে নিশ্চিত হবেন যে সেগুলো আসলেই ত্রুটি ধরে?** উত্তরটি শুধু Tool নয়, একটি চিন্তার পদ্ধতি: ইনপুটকে শ্রেণিতে ভাগ করা, সীমা খোঁজা, নিয়ম হিসেবে আচরণ প্রকাশ করা, এবং বাইরের পৃথিবীকে (Network, ঘড়ি, Browser) নিয়ন্ত্রণে আনা।

২২.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`node:test`, `assert`, Subtest (অধ্যায় ১৬)	Table-driven Test, `tap` Output পড়া, `t.after` দিয়ে পরিষ্কার
Stub, Spy, Mock, Fake-এর ধারণা (অধ্যায় ১৬)	`mock.fn`, `mock.method`, `mock.timers`
Coverage-এর পরিচয় (অধ্যায় ১৬)	Coverage Report থেকে বাস্তব ফাঁক খুঁজে Test যোগ
Seed-যুক্ত এলোমেলো ইনপুট (অধ্যায় ১৯)	Property-based Testing
Contract Test (অধ্যায় ২১)	Equivalence Partitioning, Boundary Value Analysis
—	TDD-এর পূর্ণ চক্র, আসল Browser-এ E2E, CI Pipeline

২২.২ কোন স্তরে কী যাচাই করবেন

Test-এর স্তর আলাদা প্রশ্নের উত্তর দেয়। একই আচরণ সব স্তরে যাচাই করা অপচয়; ভুল স্তরে যাচাই করা ঝুঁকি।

স্তর	প্রশ্ন	গতি ও স্থিরতা	CurrencyDesk-এ উদাহরণ
Unit	একটি Function কি নিজের নিয়ম মানে?	মিলিসেকেন্ড, সবচেয়ে স্থির	`feeFor`, `parseAmount`
Integration	অংশগুলো কি একসঙ্গে ঠিকমতো কথা বলে?	সেকেন্ডের ভগ্নাংশ	API থেকে Rate Server পর্যন্ত HTTP
End-to-end (E2E)	ব্যবহারকারী কি আসল Browser-এ কাজটি করতে পারেন?	সেকেন্ড, সবচেয়ে ভঙ্গুর	Form পূরণ, Click, ফল দেখা

সাধারণ নির্দেশিকা: **হিসাবের প্রতিটি নিয়ম Unit স্তরে** (দ্রুত, অনেক ইনপুট), **প্রতিটি সংযোগ ও Error-এর পথ Integration স্তরে** (সংখ্যায় কম), এবং **শুধু সবচেয়ে গুরুত্বপূর্ণ ব্যবহারকারীর পথ E2E স্তরে** (দুই-একটি)। E2E দিয়ে Fee-র ২০টি সীমা যাচাই করলে Test ধীর ও ভঙ্গুর হয়; Unit Test দিয়ে "Browser-এ Button কাজ করে" প্রমাণ করা যায় না।

২২.৩ Arrange–Act–Assert এবং Table-driven Test

প্রতিটি Test তিন ধাপে পড়া সহজ: **Arrange** (প্রস্তুতি), **Act** (একটি কাজ), **Assert** (প্রত্যাশা)। একই আচরণের অনেক ইনপুট থাকলে ইনপুট ও প্রত্যাশাকে একটি টেবিলে রাখুন এবং Loop-এ Test তৈরি করুন: প্রতিটি সারি আলাদা নামে রিপোর্ট হয়, তাই কোনটি ব্যর্থ তা সঙ্গে সঙ্গে দেখা যায়।

aaa.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

function applyDiscount(priceMinor, percent) {
  if (!Number.isInteger(percent) || percent < 0 || percent > 50) {
    throw new RangeError('ছাড় ০ থেকে ৫০%');
  }
  return priceMinor - Math.round((priceMinor * percent) / 100);
}

const rows = [
  [10_000, 0, 10_000],
  [10_000, 10, 9_000],
  [999, 50, 499],
];

for (const [price, percent, expected] of rows) {
  test(`${price} পয়সায় ${percent}% ছাড়`, () => {
    // Arrange: টেবিলের সারি। Act:
    const result = applyDiscount(price, percent);
    // Assert:
    assert.equal(result, expected);
  });
}
```

```
test('৫০%-এর বেশি ছাড় প্রত্যাখ্যাত', () => {
  assert.throws(() => applyDiscount(10_000, 51), RangeError);
});
```

একটি Test File সরাসরি `node` দিয়ে চালালে `node:test` TAP Format-এ ফল ছাপে। নিচে তার মূল লাইনগুলো (প্রতিটি Test-এর ফল ও সারাংশ); সময়ের (`duration_ms`) লাইনগুলো প্রতিবার বদলায় বলে বাদ দেওয়া:

Output

```
ok 1 - 10000 পয়সায় 0% ছাড়
ok 2 - 10000 পয়সায় 10% ছাড়
ok 3 - 999 পয়সায় 50% ছাড়
ok 4 - ৫০%-এর বেশি ছাড় প্রত্যাখ্যাত
# tests 4
# pass 4
# fail 0
```

তৃতীয় সারিটি লক্ষ করুন: ৯৯৯-এর ৫০% = ৪৯৯.৫, `Math.round` করে ৫০০ ছাড়, ফল ৪৯৯। এমন একটি সারিই প্রমাণ করে যে Rounding-এর নিয়ম ঠিক আছে—টেবিলে এই ধরনের "অস্বস্তিকর" ইনপুট রাখুন।

২২.৪ Equivalence Partitioning এবং Boundary Value Analysis

সব ইনপুট পরীক্ষা করা অসম্ভব। তাই ইনপুটকে **সমতুল্য শ্রেণিতে** (Equivalence Partition) ভাগ করুন—যে ইনপুটগুলোর সঙ্গে Code একই আচরণ করার কথা—এবং প্রতিটি শ্রেণি থেকে কয়েকটি প্রতিনিধি নিন। `parseAmount`-এর শ্রেণি: বৈধ পূর্ণসংখ্যা, দশমিকসহ, কমাসহ, সামনে-পেছনে ফাঁকা; এবং অবৈধ: ফাঁকা, অক্ষর, ঋণাত্মক, তিন দশমিক, ভুল কমা, বৈজ্ঞানিক রূপ।

তারপর **সীমা** খুঁজুন। ত্রুটি সবচেয়ে বেশি থাকে শ্রেণির সীমায়: `<` বনাম `<=`, শুরু বনাম শেষ। **Boundary Value Analysis** প্রতিটি সীমার ঠিক নিচে, ঠিক সীমায় এবং ঠিক ওপরে Test নেয়। CurrencyDesk-এর Fee-র তিনটি স্তর (১,০০০ ও ১০,০০০ টাকায় সীমা) দেখুন। Project-এর প্রথম খসড়ায় দ্বিতীয় স্তরের ন্যূনতম Fee ছিল ১৫ টাকা:

fee-cliff.js · JavaScript চলবে: Browser Console ও Node.js

```
// খসড়া নিয়ম: ১,০০০ টাকার নিচে ২% (ন্যূনতম ৫০), তারপর ১.৫% (ন্যূনতম ১৫)।
function draftFee(minor) {
  if (minor < 100_000) {
    return Math.max(5_000, Math.round(minor * 0.02));
  }
  return Math.max(1_500, Math.round(minor * 0.015));
}

for (const minor of [99_999, 100_000, 100_001]) {
  const taka = (n) => (n / 100).toFixed(2);
  console.log(`${taka(minor)} পাঠালে Fee ${taka(draftFee(minor))}`);
}
```

Output

```
999.99 পাঠালে Fee 50.00
1000.00 পাঠালে Fee 15.00
1000.01 পাঠালে Fee 15.00
```

সীমার তিনটি Test একটি **ব্যবসার প্রশ্ন** তুলেছে, যা Code-এর ভুল নয়, নিয়মের ভুল: এক পয়সা বেশি পাঠালে গ্রাহক ৩৫ টাকা কম Fee দেন, তাই চতুর গ্রাহক ৯৯৯.৯৯-র বদলে ১,০০০ পাঠাবেন। এমন প্রশ্ন Test লেখার সময়ই ওঠে, এবং তার উত্তর দেয় ব্যবসা, Developer নয়। চূড়ান্ত Project-এ নিয়মটি বদলানো হয়েছে (অনুচ্ছেদ ২২.১৩)।

২২.৫ Property-based Testing

Example-based Test বলে "এই ইনপুটে এই ফল"। **Property-based Test** বলে "যেকোনো ইনপুটে এই নিয়ম সত্য", এবং অনেক এলোমেলো ইনপুটে তা যাচাই করে। Fee-র একটি স্বাভাবিক নিয়ম: **বেশি পাঠালে Fee কখনো কমে না**। খসড়া নিয়মের বিরুদ্ধে এই Property চালালে:

monotonic.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

function draftFee(minor) {
  if (minor < 100_000) {
    return Math.max(5_000, Math.round(minor * 0.02));
  }
  return Math.max(1_500, Math.round(minor * 0.015));
}

test('বেশি পাঠালে Fee কখনো কমে না', () => {
  let previous = 0;
```

```
  for (let minor = 1; minor <= 5_000_000; minor += 997) {
    const fee = draftFee(minor);
    assert.ok(fee >= previous, `${minor} পয়সায় Fee ${fee}, আগে ছিল ${previous}`);
    previous = fee;
  }
});
```

Error Message

```
not ok 1 - বেশি পাঠালে Fee কখনো কমে না
100698 পয়সায় Fee 1510, আগে ছিল 5000
# fail 1
```

Property-টি নিজেই সীমার কাছে প্রথম লঙ্ঘন খুঁজে বের করেছে—আমরা কোনো সীমা বলে দিইনি। এর শক্তি এখানেই: Property অজানা সীমাও ধরে। দুটি সতর্কতা: এলোমেলো ইনপুট ব্যবহার করলে **Seed** স্থির রাখুন (অধ্যায় ১৯), যাতে ব্যর্থ হলে একই ইনপুট আবার পাওয়া যায়; এবং Property সাধারণ ও সত্যিকারের নিয়ম হতে হবে, Implementation-এর হুবহু অনুলিপি নয়। CurrencyDesk-এর তিনটি Property: Round-trip (`format` তারপর `parse` আগের মান দেয়), Fee কখনো কমে না, এবং Fee সব সময় একটি নির্দিষ্ট পরিসরে। পূর্ণাঙ্গ Library (যেমন fast-check) ব্যর্থ ইনপুটকে ছোট করে (Shrinking) সবচেয়ে সরল উদাহরণ দেখায়; এখানে আমরা সেই Library ছাড়াই ধারণাটি দেখছি।

২২.৬ Test-Driven Development: Red, Green, Refactor

TDD-তে Test আগে লেখা হয়: **Red** (একটি ব্যর্থ Test, যা চাওয়া আচরণ বর্ণনা করে), **Green** (Test পাস করানোর সবচেয়ে সরল Code), **Refactor** (Test সবুজ রেখে Code পরিষ্কার)। এর মূল লাভ Test নয়, নকশা: আগে ব্যবহারকারীর দিক থেকে Function-কে ডাকতে হয় বলে Interface সরল হয়।

`formatAmount`-এর জন্য প্রথমে Test, এবং একটি ফাঁকা Function:

format.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('পয়সা থেকে হাজারের কমা ও দুই দশমিক', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
  assert.equal(formatAmount(5, 'USD'), '0.05 USD');
});
```

format.mjs · JavaScript — চলবে: Node.js

```
export function formatAmount() {
  return '';
}
```

Terminal

```
node format.test.mjs
```

চালালে (Red):

red.mjs · JavaScript — চলবে: Node.js

```
await import('./format.test.mjs');
```

Error Message

```
not ok 1 - পয়সা থেকে হাজারের কমা ও দুই দশমিক
expected: '1,234.56 BDT'
actual: ''
```

ব্যর্থ হওয়াটা জরুরি: যে Test কখনো ব্যর্থ হয়নি, সে যে আসলে কিছু যাচাই করে, তার প্রমাণ নেই। এবার Green—সবচেয়ে সরল সঠিক Code:

format.mjs · JavaScript — চলবে: Node.js

```
export function formatAmount(minor, currency) {
  const whole = Math.floor(minor / 100).toLocaleString('en-US');
  const cents = String(minor % 100).padStart(2, '0');
  return `${whole}.${cents} ${currency}`;
}
```

format.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('পয়সা থেকে হাজারের কমা ও দুই দশমিক', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
  assert.equal(formatAmount(5, 'USD'), '0.05 USD');
});
```

Output

```
ok 1 - পয়সা থেকে হাজারের কমা ও দুই দশমিক
# tests 1
# pass 1
# fail 0
```

পরের চক্রে একটি নতুন Test: ঋণাত্মক পরিমাণ (ফেরত টাকা)। এই Green সংস্করণে `-250` হতো `-3.-50` —আবার Red। Project-এর চূড়ান্ত `formatAmount` (অনুচ্ছেদ ২২.১৩) চিহ্ন আলাদা করে সেটি সামলায়; সেটিই Refactor-এর পর রূপ। প্রতিটি চক্র ছোট: একটি নতুন আচরণ, একটি নতুন Test।

২২.৭ node:test-এর Mock: Function, Method ও সময়

অধ্যায় ১৬-এ আপনি হাতে Test Double লিখেছেন। `node:test` নিজেই তিনটি হাতিয়ার দেয়:

- **`mock.fn(impl)`**: একটি Function, যা প্রতিটি ডাক মনে রাখে (`mock.calls`, `mock.callCount()`)।
- **`mock.method(object, 'name', impl)`**: একটি বিদ্যমান Object-এর Method সাময়িকভাবে বদলায়; Test শেষে `mock.restoreAll()` বা `t.mock` ব্যবহারে নিজে ফিরে আসে।
- **`mock.timers`**: `setTimeout`, `Date` ইত্যাদিকে নকল ঘড়িতে চালায়; `tick(ms)` দিয়ে সময় এগোয়, আসল অপেক্ষা ছাড়াই।

mocks.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

const notifier = { send: (to, text) => `SMS ${to}: ${text}` };

function remindLater(phone, delayMs) {
  const text = 'আপনার Quote-এর মেয়াদ শেষ হচ্ছে';
  setTimeout(() => notifier.send(phone, text), delayMs);
}

test('৬০ সেকেন্ড পরে ঠিক একটি SMS', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const send = t.mock.method(notifier, 'send', () => 'sent');

  remindLater('01712345678', 60_000);
  t.mock.timers.tick(59_999);
  assert.equal(send.mock.callCount(), 0);

  t.mock.timers.tick(1);
  assert.equal(send.mock.callCount(), 1);
  assert.deepEqual(send.mock.calls[0].arguments,
    ['01712345678', 'আপনার Quote-এর মেয়াদ শেষ হচ্ছে']);
});

test('Test শেষে আসল Method ফিরে আসে', () => {
  assert.equal(notifier.send('x', 'y'), 'SMS x: y');
});
```

Output

```
ok 1 - ৬০ সেকেন্ড পরে ঠিক একটি SMS
ok 2 - Test শেষে আসল Method ফিরে আসে
# tests 2
# pass 2
# fail 0
```

এক মিনিটের অপেক্ষা Test-এ কয়েক মিলিসেকেন্ডে শেষ, এবং ৫৯,৯৯৯ মিলিসেকেন্ডে কিছু না ঘটা—একটি সীমার Test—আসল ঘড়িতে প্রায় অসম্ভব হতো। দ্বিতীয় Test প্রমাণ করে `t.mock` পরিষ্কার করেছে: একটি Test-এর Mock অন্য Test-এ চুঁইয়ে পড়েনি।

কতটা Mock? যা ধীর, অনিশ্চিত বা আপনার নিয়ন্ত্রণের বাইরে (Network, ঘড়ি, এলোমেলো সংখ্যা, ইমেইল), তা নকল করুন। আপনার নিজের বিশুদ্ধ Function নকল করবেন না: তখন Test শুধু প্রমাণ করে যে Mock নিজের মতো আচরণ করে। Mock যত বেশি, Test তত Implementation-এর সঙ্গে বাঁধা; তাই নির্ভরতা Inject করুন (অধ্যায় ২১) এবং সীমানায় নকল করুন।

২২.৮ Integration Test: আসল HTTP, নিয়ন্ত্রিত পৃথিবী

Unit Test-এ নকল `fetch` প্রমাণ করে যে Rate Client নিজের নিয়ম মানে, কিন্তু প্রমাণ করে না যে API আসলে HTTP-তে সঠিক Status ও JSON পাঠায়। Integration Test-এর জন্য CurrencyDesk একটি **স্থানীয় নকল Rate Provider** তৈরি করে—একটি আসল HTTP Server, যার আচরণ Test নিয়ন্ত্রণ করে (`failNext` দিয়ে ইচ্ছামতো 503)। নিয়মগুলো:

- **Port 0:** Operating System একটি খালি Port দেয়; একাধিক Test বা Developer-এর মধ্যে সংঘর্ষ নেই।
- **শুধু `127.0.0.1`:** Test-এর Server বাইরে থেকে পৌঁছানো যায় না।
- **`t.after(() => desk.close())`:** Test ব্যর্থ হলেও Server বন্ধ হয়; নইলে Process ঝুলে থাকে।
- **কোনো বাইরের API নয়:** আসল বিনিময়-হারের Service-এর ওপর নির্ভর করলে Test তার Uptime, Rate Limit ও প্রতিদিন বদলানো মানের ওপর নির্ভর করবে।

পূর্ণ Test অনুচ্ছেদ ২২.১২-এ।

২২.৯ Async Test, Cleanup এবং Flaky Test

যে Test কখনো পাস কখনো ব্যর্থ, তাকে **Flaky** বলে। Flaky Test-এর চেয়ে খারাপ কিছু কম আছে: দল ব্যর্থতাকে উপেক্ষা করতে শেখে, এবং একদিন আসল ত্রুটিও উপেক্ষিত হয়।

কারণ	উদাহরণ	সমাধান
আসল সময়	"১০০ ms অপেক্ষা করে দেখি"	`mock.timers`, অথবা Promise-এর অপেক্ষা
ভাগ করা অবস্থা	এক Test-এর Cache অন্য Test-এ	প্রতিটি Test নিজের Object বা Server তৈরি করে

কারণ	উদাহরণ	সমাধান
Test-এর ক্রম	আগের Test-এর তৈরি তথ্যের ওপর নির্ভরতা	প্রতিটি Test স্বাধীন
বাইরের Network	আসল API	স্থানীয় নকল Server
`await` বাদ পড়া	Promise-এর Assertion Test শেষের পরে	`await assert.rejects(...)`
এলোমেলো ইনপুট	Seed ছাড়া `Math.random`	স্থির Seed
স্থির Port	দুটি Test একই Port চায়	Port 0

একটি সূক্ষ্ম ভুল: `assert.rejects(promise)` ফেরত দেয় একটি Promise; `await` না করলে Test আগেই "পাস" করে, এবং আসল ব্যর্থতা হারিয়ে যায়। নিয়ম: Async Test-এ প্রতিটি Promise-এর Assertion `await` করুন।

২২.১০ Coverage: কী বলে, কী বলে না

`node --test --experimental-test-coverage` প্রতিটি File-এর কত শতাংশ লাইন, শাখা (Branch) ও Function চলেছে তা দেখায়। CurrencyDesk-এ প্রথম চালানোর আসল Report-এর অংশ (এই বইয়ের লেখার সময়, Node.js 22.16.0):

Terminal

```
# file                 | line % | branch % | funcs % | uncovered lines
#  fees.mjs            | 100.00 |   100.00 |  100.00 |
#  money.mjs           |  94.59 |    92.31 |  100.00 | 21-22
#  quote.mjs           | 100.00 |   100.00 |   66.67 |
#  rates.mjs           | 100.00 |    96.67 |  100.00 |
```

Report দুটি আসল ফাঁক দেখিয়েছিল। `money.mjs`-এর ২১-২২ লাইন ("পরিমাণ খুব বড়") কোনো Test-এ চলেনি। আর `quote.mjs`-এ তিনটি Function-এর একটি— `isValid`, যা Quote-এর মেয়াদ যাচাই করে—একবারও ডাকা হয়নি: একটি ব্যবসার গুরুত্বপূর্ণ নিয়মের কোনো Test ছিল না। দুটির জন্য `tests/quote.test.mjs` যোগ করার পর এই File দুটি ১০০%।

কিন্তু উল্টোটি সত্য নয়: ১০০% Coverage মানে ত্রুটি নেই, তা নয়। খসড়া Fee-র নিয়মে প্রতিটি লাইন চলত, তবু ৩৫ টাকার "লাফ" ধরা পড়েছিল শুধু সীমার Test ও Property-তে। Coverage বলে **কোন Code-কে কোনো Test ছোঁয়নি**; বলে না Test ভালো কি না। তাই Coverage-কে লক্ষ্য না বানিয়ে ফাঁক খোঁজার যন্ত্র হিসেবে ব্যবহার করুন।

২২.১১ Mutation Testing: Test কি আসলেই ত্রুটি ধরে?

Coverage বলে কোন Code চলেছে; প্রশ্ন থেকে যায়, Code ভুল হলে Test কি ব্যর্থ হতো? **Mutation Testing** ঠিক এটি মাপে: Code-এ ছোট, ইচ্ছাকৃত ভুল (Mutant) ঢুকিয়ে Test চালানো হয়। কোনো Test ব্যর্থ হলে Mutant "ধরা পড়েছে" (Killed); সব Test পাস করলে Mutant "টিকে গেছে" (Survived)—অর্থাৎ সেই ধরনের ভুল আপনার Test-এর চোখ এড়িয়ে

যাবে। নিচে Fee-র Function-এর চারটি Mutant, এবং Project-এর Boundary Test-এর আটটি সারি:

mutation.js · JavaScript চলবে: Browser Console ও Node.js

```
const TIERS = [
  { from: 1, bp: 200, min: 5_000 },
  { from: 100_000, bp: 150, min: 5_000 },
  { from: 1_000_000, bp: 100, min: 15_000 },
];
const MAX = 50_000_000;

// mutant: Code-এ একটি ছোট, ইচ্ছাকৃত পরিবর্তন।
function makeFee(mutant = '') {
  return (minor) => {
    const over = mutant === 'max>=' ? minor >= MAX : minor > MAX;
    if (minor < 1 || over) {
      throw new RangeError('সীমার বাইরে');
    }
    const fits = (t) => (mutant === 'from>' ? minor > t.from : minor >= t.from);
    const tier = TIERS.findLast(fits) ?? TIERS[0];
    const min = mutant === 'cliff' && tier.from === 100_000 ? 1_500 : tier.min;
    const half = mutant === 'truncate' ? 0n : 5_000n;
    const percent = Number((BigInt(minor) * BigInt(tier.bp) + half) / 10_000n);
    return Math.max(min, percent);
  };
}

const rows = [[1, 5_000], [99_999, 5_000], [100_000, 5_000],
  [333_334, 5_000], [333_367, 5_001], [999_999, 15_000],
  [1_000_000, 15_000], [MAX, 500_000]];

const killedBy = (fee) => rows.find(([minor, expected]) => {
  try {
    return fee(minor) !== expected;
  } catch {
    return true;
  }
});

console.log(`আসল Code: ${killedBy(makeFee()) ? 'ব্যর্থ' : 'সব পাস'}`);
for (const mutant of ['cliff', 'truncate', 'max>=', 'from>']) {
  const row = killedBy(makeFee(mutant));
  console.log(`${mutant.padEnd(8)} ${row ? `ধরা পড়েছে (${row[0]} পয়সায়)` : 'টিকে
      গেছে'}`);
}
```

Output

```
আসল Code: সব পাস
cliff    ধরা পড়েছে (100000 পয়সায়)
truncate ধরা পড়েছে (333367 পয়সায়)
max>=    ধরা পড়েছে (50000000 পয়সায়)
from>    টিকে গেছে
```

তিনটি Mutant ধরা পড়েছে, এবং প্রতিটি ঠিক সেই সারিতে, যেটি তার জন্যই লেখা: পুরোনো "লাফ" (`cliff`) ১,০০০ টাকায়, Rounding বাদ দেওয়া (`truncate`) ৩,৩৩৩.৬৭-এ, এবং সর্বোচ্চ সীমার `>` থেকে `>=` সর্বোচ্চ পরিমাণে। কিন্তু স্তর বাছাইয়ের `>=` থেকে `>` টিকে গেছে। কারণ খুঁজলে দেখা যায়, চূড়ান্ত Fee-র টেবিল সীমায় অবিচ্ছিন্ন: ১,০০০ টাকায় প্রথম ও দ্বিতীয় স্তর দুটিই ৫০ টাকা দেয়, ১০,০০০ টাকায় দ্বিতীয় ও তৃতীয় স্তর দুটিই ১৫০। তাই এই Mutant আচরণ বদলায়ই না—একে বলে **Equivalent Mutant**, এবং এটি কোনো Test-এর দুর্বলতা নয়। টিকে যাওয়া প্রতিটি Mutant এভাবে বিশ্লেষণ করতে হয়: হয় একটি অনুপস্থিত Test, নয়তো একটি সমতুল্য পরিবর্তন।

বাস্তব Project-এ এই কাজ হাতে করা হয় না; StrykerJS-এর মতো Tool শত শত Mutant স্বয়ংক্রিয়ভাবে তৈরি করে এবং একটি "Mutation Score" দেয়। এটি ধীর (প্রতিটি Mutant-এ পুরো Test Suite), তাই সাধারণত গুরুত্বপূর্ণ Module-এ বা রাতের CI-তে চালানো হয়। এখানে আমরা Tool ছাড়াই ধারণাটি দেখলাম: **Coverage বলে Test কোথায় গেছে, Mutation বলে Test কী দেখেছে।**

২২.১২ End-to-end: আসল Browser-এ

E2E Test একজন ব্যবহারকারীর মতো আচরণ করে: Page খোলে, Form পূরণ করে, Button চাপে, পর্দায় ফল দেখে। এটি এমন ত্রুটি ধরে যা অন্য কোনো স্তর পারে না: ভুল `id` , Script লোড না হওয়া, JSON-এর Field-এর নাম Server ও Browser-এ অমিল।

পেশাদার দলে সাধারণত Playwright বা Cypress ব্যবহৃত হয়। এগুলো Install করতে হয়; এই বই কোনো Package Install না করে কাজ করে, তাই CurrencyDesk-এ একটি ছোট নিজস্ব Helper (`tests/support/browser.mjs`) আছে, যা Chrome-এর **DevTools Protocol** দিয়ে Headless Browser চালায়। Playwright ঠিক এই Protocol-ই ব্যবহার করে, অনেক বেশি সুবিধাসহ (স্বয়ংক্রিয় অপেক্ষা, একাধিক Browser, Trace, Screenshot)। বাস্তব Project-এ Playwright ব্যবহার করুন; এখানের Helper শুধু দেখায় ভেতরে কী ঘটে।

E2E Test-এর নিয়ম: আসল Server চালু করে, Browser শুধু সেটিতে যায়; কোনো বাইরের Site নয়; প্রতিটি Test নিজের তথ্য তৈরি করে; এবং `setTimeout` -এ নির্দিষ্ট সময় অপেক্ষার বদলে **একটি শর্তের জন্য অপেক্ষা** (Result-এর `data-state` বসা পর্যন্ত)—এটিই E2E Flakiness কমানোর সবচেয়ে বড় কৌশল। Chrome বা Edge না পাওয়া গেলে Test কারণসহ **Skip** হয়, ব্যর্থ নয়, যাতে Browser-হীন যন্ত্রেও বাকি Test চলে।

২২.১৩ Project: CurrencyDesk

CurrencyDesk একটি ভ্রমণ সংস্থার মুদ্রা-বিনিময় Counter-এর Quote Service: গ্রাহক কোন মুদ্রা থেকে কোন মুদ্রায় কত পাঠাতে চান দিলে, Service Fee, মোট দেয়, পাওয়া পরিমাণ এবং মেয়াদ জানায়। হার আসে একটি Rate Provider থেকে (এখানে স্থানীয় নকল Server)। Project-এর আসল বিষয়বস্তু তার Test: ৩৮টি, পাঁচ ধরনের।

> **নোট:** CurrencyDesk একটি শিক্ষামূলক Demo। হারগুলো কাল্পনিক নমুনা, Server শুধু `127.0.0.1` -এ চলে, এবং বাস্তব অর্থ-লেনদেনের জন্য প্রয়োজনীয় নিয়ন্ত্রণ (Authentication, Audit, আইনি অনুমোদন, আসল Provider-এর চুক্তি) এতে নেই।

File-এর কাঠামো

Folder-এর কাঠামো

```
currency-desk/
├── server.mjs              চালু করে (127.0.0.1)
├── package.json            npm test ও npm run coverage
├── .github/workflows/test.yml   CI
├── src/
│   ├── money.mjs           পয়সায় হিসাব, parse, format, convert
│   ├── fees.mjs            স্তরভিত্তিক Fee
│   ├── rates.mjs           Rate Client: Cache, Retry
│   ├── quote.mjs           Quote Service
│   ├── rate-server.mjs     স্থানীয় নকল Rate Provider
│   ├── api.mjs             HTTP API ও Static File
│   └── start.mjs           দুই Server জোড়া লাগানো
├── public/                 index.html, app.mjs
└── tests/                  ৭টি Test File, support/browser.mjs
```

src/money.mjs ও src/fees.mjs

money.mjs · JavaScript চলবে: Node.js

```
// টাকা সব সময় পূর্ণসংখ্যায় (Minor Unit: পয়সা বা সেন্ট)। 0.1 + 0.2-এর মতো
// Floating-point ভুল এখানে ঢোকে না।
const AMOUNT = /^(?:\d{1,3}(?:,\d{3})+|\d+)(?:\.\d{1,2})?$/;

export class MoneyError extends Error {
  constructor(message) {
    super(message);
    this.name = 'MoneyError';
  }
}

// "1,234.5" থেকে 123450। কমা শুধু সঠিক তিন অঙ্কের দলে।
export function parseAmount(text) {
```

```
  const clean = String(text ?? '').trim();
  if (!AMOUNT.test(clean)) {
    throw new MoneyError(`অবৈধ পরিমাণ: "${clean}"`);
  }
  const [whole, fraction = ''] = clean.replaceAll(',', '').split('.');
  const minor = Number(whole) * 100 + Number(fraction.padEnd(2, '0'));
  if (!Number.isSafeInteger(minor)) {
    throw new MoneyError('পরিমাণ খুব বড়');
  }
  return minor;
}

export function formatAmount(minor, currency) {
  const sign = minor < 0 ? '-' : '';
  const abs = Math.abs(minor);
  const whole = Math.floor(abs / 100).toLocaleString('en-US');
  return `${sign}${whole}.${String(abs % 100).padStart(2, '0')} ${currency}`;
}

// হার Micro Unit-এ (1 = 0.000001)। গুণফল BigInt-এ, অর্ধেক হলে ওপরে।
export function convert(minor, rateMicro) {
  const scaled = BigInt(minor) * BigInt(rateMicro) + 500_000n;
  return Number(scaled / 1_000_000n);
}
```

fees.mjs · JavaScript — চলবে: Node.js

```
// Fee-র স্তর। from ও MAX_AMOUNT পয়সায়; bp = Basis Point (100 bp = 1%)।
// প্রতিটি স্তরের নিচের সীমা অন্তর্ভুক্ত। min এমনভাবে বাছা যে বড় পরিমাণে
// Fee কখনো কমে না (স্তরের সীমায় "লাফ" নেই)।
export const FEE_TIERS = Object.freeze([
  Object.freeze({ from: 1, bp: 200, min: 5_000 }),
  Object.freeze({ from: 100_000, bp: 150, min: 5_000 }),
  Object.freeze({ from: 1_000_000, bp: 100, min: 15_000 }),
]);

export const MAX_AMOUNT = 50_000_000;

export function feeFor(minor) {
  if (!Number.isSafeInteger(minor) || minor < 1) {
    throw new RangeError('পরিমাণ অন্তত 0.01 হতে হবে');
  }
  if (minor > MAX_AMOUNT) {
    throw new RangeError('এক লেনদেনে সর্বোচ্চ 500,000.00');
  }
  const tier = FEE_TIERS.findLast((t) => minor >= t.from);
  const percent = Number((BigInt(minor) * BigInt(tier.bp) + 5_000n) / 10_000n);
  return Math.max(tier.min, percent);
}
```

টাকা পয়সায় পূর্ণসংখ্যা হিসেবে, হার Micro Unit-এ (১২১.৪৫ = ১২১,৪৫০,০০০), এবং গুণ `BigInt`-এ: ৯০ কোটি টাকার পয়সা × হার সাধারণ Number-এর নিরাপদ সীমা (`Number.MAX_SAFE_INTEGER`) ছাড়িয়ে যায়। চূড়ান্ত Fee-র টেবিলে দ্বিতীয় স্তরের ন্যূনতম ৫০ টাকা, তাই ১,০০০ টাকায় আর লাফ নেই; Property Test এখন পাস করে।

src/rates.mjs

rates.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
export class RateError extends Error {
  constructor(code, message, { retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = 'RateError';
    this.code = code;
    this.retryable = retryable;
  }
}

const CURRENCY = /^[A-Z]{3}$/;
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// সব বাইরের নির্ভরতা Inject করা: fetch, ঘড়ি, অপেক্ষা। Test-এ সবই বদলানো যায়।
export function createRateClient(options) {
  const { baseUrl, fetchFn = fetch, now = Date.now, sleep = wait } = options;
  const { ttlMs = 60_000, retries = 2 } = options;
  const cache = new Map();

  async function fetchOnce(pair) {
    let response;
    try {
      response = await fetchFn(`${baseUrl}/rates/${pair}`);
    } catch (error) {
      throw new RateError('NETWORK', 'Rate Server-এ পৌঁছানো যায়নি',
        { retryable: true, cause: error });
    }
    if (response.status === 404) {
      throw new RateError('UNKNOWN_PAIR', `${pair}-এর হার নেই`);
    }
    if (!response.ok) {
      throw new RateError('UPSTREAM', `Rate Server: ${response.status}`,
        { retryable: response.status >= 500 });
    }
    const { rateMicro } = await response.json();
    if (!Number.isSafeInteger(rateMicro) || rateMicro <= 0) {
      throw new RateError('BAD_DATA', 'Rate Server অবৈধ হার পাঠিয়েছে');
    }
    return rateMicro;
  }
```

```
// ...
```

rates.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

  async function getRate(from, to) {
    if (!CURRENCY.test(from) || !CURRENCY.test(to) || from === to) {
      throw new RateError('BAD_CURRENCY', 'মুদ্রা তিন অক্ষরের, এবং দুটি ভিন্ন');
    }
    const pair = `${from}-${to}`;
    const hit = cache.get(pair);
    if (hit && now() - hit.at < ttlMs) {
      return hit.rateMicro;
    }
    for (let attempt = 0; ; attempt += 1) {
      try {
        const rateMicro = await fetchOnce(pair);
        cache.set(pair, { rateMicro, at: now() });
        return rateMicro;
      } catch (error) {
        if (!error.retryable || attempt >= retries) {
          throw error;
        }
        await sleep(100 * 2 ** attempt);
      }
    }
  }

  return { getRate };
}
```

Testability-ই এই File-এর নকশা ঠিক করেছে: `fetchFn`, `now`, `sleep` তিনটিই Inject করা, Default আসল। Error-এর `retryable` চিহ্ন বলে কোনটি আবার চেষ্টাযোগ্য (Network, 5xx) আর কোনটি নয় (404, অবৈধ মুদ্রা, অবৈধ তথ্য)। Retry-র অপেক্ষা দ্বিগুণ হয় (১০০, ২০০ ms)। Cache-এর মেয়াদ এক মিনিট।

src/quote.mjs, src/rate-server.mjs ও src/api.mjs

quote.mjs · JavaScript চলবে: Node.js

```
import { parseAmount, convert } from './money.mjs';
import { feeFor } from './fees.mjs';

// একটি Quote: কত পাঠাবেন, Fee, মোট দেবেন, কত পাবেন, এবং কতক্ষণ বৈধ।
export function createQuoteService(deps) {
  const { rates, newId, now = Date.now, validityMs = 60_000 } = deps;
```

```
  async function quote({ from, to, amount }) {
    const minor = parseAmount(amount);
    const fee = feeFor(minor);
    const rateMicro = await rates.getRate(from, to);
    return {
      id: newId(),
      from,
      to,
      amount: minor,
      fee,
      payable: minor + fee,
      receives: convert(minor, rateMicro),
      rateMicro,
      expiresAt: now() + validityMs,
    };
  }

  const isValid = (q) => now() < q.expiresAt;
  return { quote, isValid };
}
```

rate-server.mjs · JavaScript — চলবে: Node.js

```
import { createServer } from 'node:http';

// একটি স্থানীয় নকল Rate Provider (Test ও Demo-র জন্য; বাস্তব Provider নয়)।
// controls দিয়ে Test ইচ্ছামতো ব্যর্থতা তৈরি করতে পারে।
export function createRateServer(table) {
  const controls = { failNext: 0, failStatus: 503, requests: 0 };
  const server = createServer((req, res) => {
    controls.requests += 1;
    const pair = /^\/rates\/([A-Z]{3}-[A-Z]{3})$/.exec(req.url)?.[1];
    if (controls.failNext > 0) {
      controls.failNext -= 1;
      res.writeHead(controls.failStatus).end();
    } else if (pair && Object.hasOwn(table, pair)) {
      res.writeHead(200, { 'content-type': 'application/json' });
      res.end(JSON.stringify({ pair, rateMicro: table[pair] }));
    } else {
      res.writeHead(404).end();
    }
  });
  return { server, controls };
}
```

api.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Node.js

```
import { createServer } from 'node:http';
import { readFile } from 'node:fs/promises';
```

```
import { MoneyError } from './money.mjs';

const PUBLIC = new URL('../public/', import.meta.url);
const STATIC = { '/': 'index.html', '/app.mjs': 'app.mjs' };
const TYPES = { html: 'text/html', mjs: 'text/javascript' };
const STATUS = { BAD_CURRENCY: 400, UNKNOWN_PAIR: 404 };

function send(res, status, data) {
  res.writeHead(status, { 'content-type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify(data));
}

// Error-এর ধরন থেকে HTTP Status: ব্যবহারকারীর ভুল 4xx, বাইরের সমস্যা 502।
function statusFor(error) {
  if (error instanceof MoneyError || error instanceof RangeError) {
    return 400;
  }
  return STATUS[error.code] ?? (error.name === 'RateError' ? 502 : 500);
}

// ...
```

api.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export function createApi({ quotes }) {
  return createServer(async (req, res) => {
    try {
      if (req.method === 'GET' && Object.hasOwn(STATIC, req.url)) {
        const file = STATIC[req.url];
        const type = TYPES[file.split('.').pop()];
        res.writeHead(200, { 'content-type': `${type}; charset=utf-8` });
        res.end(await readFile(new URL(file, PUBLIC)));
      } else if (req.method === 'POST' && req.url === '/api/quotes') {
        let body = '';
        for await (const chunk of req) {
          body += chunk;
        }
        send(res, 201, await quotes.quote(JSON.parse(body || '{}')));
      } else {
        send(res, 404, { error: 'পাওয়া যায়নি' });
      }
    } catch (error) {
      const status = error instanceof SyntaxError ? 400 : statusFor(error);
      const message = status === 500 ? 'Server-এ সমস্যা' : error.message;
      send(res, status, { error: message, code: error.code ?? null });
    }
```

```
  });
}
```

`quote` প্রথমে পরিমাণ যাচাই করে, তারপর Rate Server-এ যায়: অবৈধ ইনপুটে বাইরের Call-এর খরচ নেই (একটি Test এটি প্রমাণ করে)। API-র `statusFor` Error-এর ধরন থেকে HTTP Status ঠিক করে: ব্যবহারকারীর ভুল 400, অজানা মুদ্রা-জোড়া 404, Provider-এর সমস্যা 502 (Bad Gateway), অপ্রত্যাশিত Error 500 সাধারণ বার্তায়।

src/start.mjs ও server.mjs

start.mjs · JavaScript — চলবে: Node.js

```
import { createRateServer } from './rate-server.mjs';
import { createRateClient } from './rates.mjs';
import { createQuoteService } from './quote.mjs';
import { createApi } from './api.mjs';

export const SAMPLE_RATES = Object.freeze({
  'USD-BDT': 121_450_000,
  'BDT-USD': 8_234,
  'EUR-BDT': 131_900_000,
  'SAR-BDT': 32_380_000,
});

const listen = (server) => new Promise((resolve) => {
  server.listen(0, '127.0.0.1', () => resolve(server.address().port));
});

// দুটি স্থানীয় Server (নকল Rate Provider ও Quote API), এলোমেলো Port-এ।
export async function startDesk({ table = SAMPLE_RATES, retries = 2 } = {}) {
  const provider = createRateServer(table);
  const ratePort = await listen(provider.server);
  let counter = 0;
  const rates = createRateClient({
    baseUrl: `http://127.0.0.1:${ratePort}`, retries, sleep: async () => {},
  });
  const quotes = createQuoteService({
    rates, newId: () => `Q-${String(++counter).padStart(4, '0')}`,
  });
  const api = createApi({ quotes });
  const apiPort = await listen(api);
  const close = () => Promise.all([provider.server, api].map((s) =>
    new Promise((resolve) => s.close(resolve))));
  return { url: `http://127.0.0.1:${apiPort}`, provider, close };
}
```

server.mjs · JavaScript চলবে: Node.js

```
import { startDesk } from './src/start.mjs';

// শুধু 127.0.0.1-এ; হারগুলো কাল্পনিক নমুনা, বাস্তব বিনিময় হার নয়।
const desk = await startDesk();
console.log(`CurrencyDesk (শিক্ষামূলক Demo): ${desk.url}`);
console.log('বন্ধ করতে Ctrl+C চাপুন।');

process.on('SIGINT', async () => {
  await desk.close();
  process.exit(0);
});
```

`startDesk` একটি Composition Root (অধ্যায় ২১), যা Demo এবং প্রতিটি Integration ও E2E Test একইভাবে ব্যবহার করে: তাই Test আসল জোড়া লাগানো পরীক্ষা করে, একটি আলাদা Test-সংস্করণ নয়। Test-এ Retry-র অপেক্ষা শূন্য (`sleep: async () => {}`), যাতে ব্যর্থতার পথ দ্রুত চলে।

Browser UI

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CurrencyDesk</title>
  <script type="module" src="/app.mjs"></script>
</head>
<body>
  <h1>CurrencyDesk <small>শিক্ষামূলক Demo</small></h1>
  <form id="quote-form">
    <label>থেকে <input id="from" value="USD" maxlength="3"></label>
    <label>প্রতি <input id="to" value="BDT" maxlength="3"></label>
    <label>পরিমাণ <input id="amount" inputmode="decimal"></label>
    <button id="get-quote">Quote দেখুন</button>
  </form>
  <p id="result" role="status"></p>
</body>
</html>
```

app.mjs · JavaScript চলবে: Browser

```
const $ = (selector) => document.querySelector(selector);
const money = (minor, currency) => {
  const options = { minimumFractionDigits: 2 };
  return `${(minor / 100).toLocaleString('en-US', options)} ${currency}`;
```

```
};

$('#quote-form').addEventListener('submit', async (event) => {
  event.preventDefault();
  const body = {
    from: $('#from').value.trim().toUpperCase(),
    to: $('#to').value.trim().toUpperCase(),
    amount: $('#amount').value,
  };
  const response = await fetch('/api/quotes', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify(body),
  });
  const data = await response.json();
  $('#result').dataset.state = response.ok ? 'ok' : 'error';
  $('#result').textContent = response.ok
    ? `দেবেন ${money(data.payable, data.from)}`
      + ` (Fee ${money(data.fee, data.from)}),`
      + ` পাবেন ${money(data.receives, data.to)}`
    : data.error;
});
```

ফল বসানোর সময় `data-state` (`ok` বা `error`) চিহ্ন: E2E Test এটির জন্য অপেক্ষা করে, অনুমান করা সময়ের জন্য নয়।

Test: সীমা, Mock ও Property

fees.test.mjs · JavaScript — চলবে: Node.js

```
// Boundary Value Analysis: প্রতিটি সীমার ঠিক নিচে, ঠিক ওপরে ও ঠিক সীমায়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { feeFor, MAX_AMOUNT } from '../src/fees.mjs';

const cases = [
  ['সবচেয়ে ছোট বৈধ', 1, 5_000],
  ['প্রথম স্তরের শেষ (999.99)', 99_999, 5_000],
  ['দ্বিতীয় স্তরের শুরু (1,000.00)', 100_000, 5_000],
  ['1.5% যেখানে ন্যূনতমকে ছাড়ায় (3,333.34)', 333_334, 5_000],
  ['তার পরের পয়সা (3,333.67)', 333_367, 5_001],
  ['দ্বিতীয় স্তরের শেষ (9,999.99)', 999_999, 15_000],
  ['তৃতীয় স্তরের শুরু (10,000.00)', 1_000_000, 15_000],
  ['সর্বোচ্চ (500,000.00)', MAX_AMOUNT, 500_000],
];

for (const [name, minor, fee] of cases) {
  test(`Fee: ${name}`, () => {
    assert.equal(feeFor(minor), fee);
```

```
  });
}

test('সীমার বাইরে: 0, ঋণাত্মক, সর্বোচ্চের এক পয়সা বেশি, ভগ্নাংশ', () => {
  for (const minor of [0, -1, MAX_AMOUNT + 1, 1.5, Number.NaN]) {
    assert.throws(() => feeFor(minor), RangeError, String(minor));
  }
});
```

rates.test.mjs · JavaScript [নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code)] চলবে: Node.js

```
// Test Double: mock.fn দিয়ে নকল fetch, নকল ঘড়ি, এবং mock.timers।
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';
import { createRateClient } from '../src/rates.mjs';

const reply = (status, body = {}) =>
  new Response(JSON.stringify(body), { status });
const ok = (rateMicro) => reply(200, { rateMicro });

test('Cache: TTL-এর মধ্যে দ্বিতীয় অনুরোধ Network-এ যায় না', async () => {
  let time = 0;
  const fetchFn = mock.fn(async () => ok(121_450_000));
  const rates = createRateClient(
    { baseUrl: 'http://x', fetchFn, now: () => time });
  await rates.getRate('USD', 'BDT');
  time = 59_999;
  await rates.getRate('USD', 'BDT');
  assert.equal(fetchFn.mock.callCount(), 1);
  time = 60_000;
  await rates.getRate('USD', 'BDT');
  assert.equal(fetchFn.mock.callCount(), 2);
  assert.equal(fetchFn.mock.calls[0].arguments[0], 'http://x/rates/USD-BDT');
});

test('Retry: 503 দুবার, তৃতীয়বার সফল; অপেক্ষা 100 ও 200 ms', async () => {
  const responses = [reply(503), reply(503), ok(8_234)];
  const fetchFn = mock.fn(async () => responses.shift());
  const sleep = mock.fn(async () => {});
  const rates = createRateClient({ baseUrl: 'http://x', fetchFn, sleep });
  assert.equal(await rates.getRate('BDT', 'USD'), 8_234);
  assert.deepEqual(sleep.mock.calls.map((c) => c.arguments[0]), [100, 200]);
});

// ...
```

property.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Fee কখনো কমে না: বড় পরিমাণে Fee সমান বা বেশি', () => {
  let previous = 0;
  for (let minor = 1; minor <= MAX_AMOUNT; minor += 997) {
    const fee = feeFor(minor);
    assert.ok(fee >= previous, `${minor} পয়সায় Fee ${fee} < ${previous}`);
    previous = fee;
  }
});

// ...
```

`fees.test.mjs`-এর প্রতিটি সারি একটি সীমার নাম বলে, তাই ব্যর্থ হলে Report-ই বলে দেয় কোন সীমা ভেঙেছে। `rates.test.mjs`-এর Retry Test দুটি জিনিস প্রমাণ করে: তৃতীয় চেষ্টায় সফল, এবং অপেক্ষা ঠিক ১০০ ও ২০০ ms—কোনো আসল অপেক্ষা ছাড়াই।

Test: Integration ও E2E

integration.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Integration Test: আসল HTTP, আসল দুই Server (শুধু 127.0.0.1, এলোমেলো Port)।
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../src/start.mjs';

async function post(desk, body) {
  const response = await fetch(`${desk.url}/api/quotes`, {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: typeof body === 'string' ? body : JSON.stringify(body),
  });
  return { status: response.status, data: await response.json() };
}

test('Quote API', async (t) => {
  const desk = await startDesk();
  t.after(() => desk.close());

  await t.test('201: সম্পূর্ণ Quote, সব টাকা পয়সায়', async () => {
    const { status, data } = await post(desk,
      { from: 'USD', to: 'BDT', amount: '1,000' });
    assert.equal(status, 201);
    assert.equal(data.id, 'Q-0001');
    assert.equal(data.fee, 5_000);
    assert.equal(data.payable, 105_000);
```

```
    assert.equal(data.receives, 12_145_000);
    assert.ok(data.expiresAt > Date.now());
  });

// ...

  await t.test('502: Provider বারবার ব্যর্থ', async () => {
    desk.provider.controls.failNext = 3;
    const { status, data } = await post(desk,
      { from: 'EUR', to: 'BDT', amount: '5' });
    assert.equal(status, 502);
    assert.equal(data.code, 'UPSTREAM');
  });

// ...
```

e2e.test.mjs · JavaScript — চলবে: Node.js

```
// End-to-end: আসল Browser, আসল Server, ব্যবহারকারীর মতো Form পূরণ ও Click।
// Chrome বা Edge না থাকলে Skip (কারণসহ), ব্যর্থ নয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../src/start.mjs';
import { findBrowser, launch } from './support/browser.mjs';

const skip = findBrowser()
  ? false
  : 'Chrome বা Edge পাওয়া যায়নি (CHROME_PATH দিন)';

test('Browser-এ Quote', { skip, timeout: 60_000 }, async (t) => {
  const desk = await startDesk();
  const page = await launch();
  t.after(async () => {
    await page.close();
    await desk.close();
  });
  await page.open(desk.url);
  const waitForResult = () => page.eval(`new Promise((resolve) => {
    const tick = () => document.querySelector('#result').dataset.state
      ? resolve(document.querySelector('#result').textContent)
      : setTimeout(tick, 50);
    tick();
  })`);

  await page.type('#amount', '1,000');
  await page.click('#get-quote');
  assert.equal(await waitForResult(),
    'দেবেন 1,050.00 USD (Fee 50.00 USD), পাবেন 121,450.00 BDT');
```

```
  await page.eval("delete document.querySelector('#result').dataset.state");
  await page.type('#amount', '1.234');
  await page.click('#get-quote');
  assert.equal(await waitForResult(), 'অবৈধ পরিমাণ: "1.234"');
});
```

Integration Test একটি Parent Test-এর ভেতরে Subtest, যাতে দুই Server একবার চালু হয় এবং `t.after`-এ একবার বন্ধ হয়। `failNext = 2` দুটি 503-এর পর সফলতা দেয় (Retry পাস), `failNext = 3` সব চেষ্টা ব্যর্থ করে (502)। E2E Test আসল Browser-এ Form পূরণ করে দুটি পথ যাচাই করে: সফল Quote, এবং অবৈধ পরিমাণের বার্তা।

CI

test.yml · YAML — চলবে: Node.js

```yaml
# প্রতিটি Push ও Pull Request-এ Test (GitHub Actions)। এই বইয়ের QA-তে
# এই File-টি GitHub-এ চালানো হয়নি; একই Command স্থানীয়ভাবে চালানো হয়েছে।
name: CurrencyDesk tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: node --test
      - run: node --test --experimental-test-coverage
```

এই Workflow GitHub-এ প্রতিটি Push ও Pull Request-এ Test চালায়, যাতে ত্রুটিপূর্ণ Code Merge হওয়ার আগেই ধরা পড়ে। এই বইয়ের QA-তে File-টি GitHub-এ চালানো হয়নি (কোনো Repository-তে Push করা হয়নি); এর দুটি Command স্থানীয়ভাবে চালানো ও যাচাই করা হয়েছে। লক্ষ করুন, CI-র Ubuntu যন্ত্রে সাধারণত Chrome থাকে না, তাই E2E Test Skip হবে; সেখানে E2E চালাতে Browser Install-এর একটি ধাপ যোগ করতে হয়।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd currency-desk
node --test
node --test --experimental-test-coverage
node server.mjs
```

`node --test` ৩৮টি Test চালায়: `money` ৯, `fees` ৯, `rates` ৬, `property` ৩, `quote` ৩, `integration` ৭ (একটি Parent Test ও তার ছয়টি Subtest), এবং `e2e` ১। শেষে `# pass 38`, `# fail 0`। Coverage-সহ চালালে, এই বইয়ের লেখার সময় সব `src` File-এ লাইন Coverage ১০০%। `node server.mjs` একটি ঠিকানা দেখায়; Browser-এ খুলে Quote দেখুন।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
E2E Test "skipped"	Chrome বা Edge পাওয়া যায়নি	`CHROME_PATH` Environment Variable
Test শেষ হয় না, ঝুলে থাকে	কোনো Server বন্ধ হয়নি	`t.after`-এ `close()`
`assert.rejects` সত্ত্বেও ব্যর্থতা ধরা পড়ে না	`await` বাদ	`await assert.rejects(...)`
কখনো পাস, কখনো ব্যর্থ	আসল সময় বা ভাগ করা অবস্থা	`mock.timers`, প্রতিটি Test-এ নতুন Object
Coverage কম	কোনো শাখার Test নেই	Report-এর "uncovered lines" দেখুন

২২.১৪ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
শুধু "Happy Path" Test	ত্রুটি থাকে সীমা ও ব্যর্থতায়	Partition, Boundary, Error-এর পথ
Implementation-এর বিস্তারিত Test	Refactor করলেই Test ভাঙে	আচরণ (Input ও Output) Test
সবকিছু Mock	Test শুধু Mock-কে Test করে	শুধু সীমানায় Mock
আসল সময়ে অপেক্ষা (`sleep(500)`)	ধীর ও Flaky	`mock.timers`, শর্তের অপেক্ষা
এক Test-এ অনেক আচরণ	ব্যর্থ হলে কারণ অস্পষ্ট	একটি আচরণ, একটি Test
Test কখনো ব্যর্থ হতে দেখা হয়নি	হয়তো কিছুই যাচাই করে না	TDD-র Red ধাপ
Coverage-কে লক্ষ্য বানানো	অর্থহীন Test	Coverage দিয়ে ফাঁক খোঁজা
বাইরের API-তে Test	Network-নির্ভর, Flaky	স্থানীয় নকল Server
E2E দিয়ে সব যাচাই	ধীর, ভঙ্গুর	নিয়ম Unit-এ, পথ E2E-তে

২২.১৫ Debugging নির্দেশিকা

১. **একটি Test আলাদা চালান:** `node --test --test-name-pattern="Retry"`।

২. **ব্যর্থতার বার্তা পড়ুন:** `assert` `expected` ও `actual` দেখায়; Table-driven Test-এ Test-এর নামই ইনপুট বলে।
৩. **Flaky হলে বারবার চালান:** একটি Loop-এ ২০ বার; ব্যর্থ হলে সময়, ক্রম বা ভাগ করা অবস্থা খুঁজুন।
৪. **ঝুলে থাকা Test:** কোনো Server, Timer বা খোলা Connection বন্ধ হয়নি; `t.after` দেখুন।
৫. **Property ব্যর্থ:** বার্তায় ইনপুট ছাপুন, এবং সেই ইনপুট দিয়ে একটি Example-based Test যোগ করুন।
৬. **E2E ব্যর্থ:** Page-এর অবস্থা `eval` দিয়ে পড়ুন (`document.body.innerHTML`), Server-এর Log দেখুন।

২২.১৬ পেশাদার অভ্যাস

- **প্রতিটি Bug-এর জন্য প্রথমে একটি ব্যর্থ Test,** তারপর সমাধান।
- **Test-এর নাম একটি বাক্য,** যা আচরণ বর্ণনা করে।
- **Test দ্রুত রাখুন:** পুরো Unit Suite কয়েক সেকেন্ডে।
- **প্রতিটি Test স্বাধীন:** নিজের তথ্য, নিজের Server, যেকোনো ক্রমে চলে।
- **নির্ভরতা Inject করুন,** যাতে Test-এ সময়, Network ও এলোমেলোতা নিয়ন্ত্রণে থাকে।
- **CI-তে প্রতিটি Pull Request-এ Test;** লাল থাকলে Merge নয়।
- **Flaky Test সঙ্গে সঙ্গে ঠিক করুন অথবা সরান,** কখনো উপেক্ষা নয়।

২২.১৭ অনুশীলনী

প্রাথমিক স্তর

১. `isValidAge(age)` (১৮ থেকে ৬০, পূর্ণসংখ্যা)-এর জন্য Equivalence Partition ও Boundary Value-গুলোর তালিকা করুন।
২. প্রশ্ন ১-এর জন্য একটি Table-driven Test লিখুন।
৩. নিচের Test-এ কী ভুল? `test('rejects', () => { assert.rejects(load('x')); });`
৪. `mock.fn` দিয়ে একটি `log` Function নকল করুন, এবং যাচাই করুন যে `processOrder` এটিকে ঠিক একবার "done" দিয়ে ডাকে।
৫. Unit, Integration ও E2E—নিচের প্রতিটি কোন স্তরে: (ক) Fee-র সীমা, (খ) API-র 502, (গ) Button চাপলে ফল দেখা।

মধ্যম স্তর

৬. TDD-তে `formatAmount`-এর ঋণাত্মক পরিমাণের Test লিখুন (Red), তারপর Code (Green)।
৭. `mock.timers` দিয়ে একটি Debounce Function Test করুন: ৩০০ ms-এর মধ্যে তিনটি ডাকে Function একবার চলে।

৮. `parseAmount`-এর জন্য একটি Property লিখুন: যেকোনো বৈধ লেখার কমা মুছে ফেললেও একই মান।

৯. Rate Client-এ "৪২৯ Too Many Requests" কি Retry-যোগ্য হওয়া উচিত? একটি Test লিখে বর্তমান আচরণ দেখান।

চ্যালেঞ্জ স্তর

১০. Integration Test-এ প্রমাণ করুন যে একই মুদ্রা-জোড়ার দুটি Quote-এ Rate Server-এ একটিই অনুরোধ যায় (Cache)।

১১. একটি `retry(fn, { retries, sleep })` Helper TDD-তে লিখুন: প্রথমে তিনটি Test (সফল, শেষে সফল, সব ব্যর্থ), তারপর Code।

১২. E2E-তে নতুন একটি পথ যোগ করুন: অজানা মুদ্রা-জোড়ায় (USD থেকে JPY) বার্তা দেখা যায়।

২২.১৮ অনুশীলনীর সমাধান

সমাধান ১ — বৈধ শ্রেণি: ১৮ থেকে ৬০-এর পূর্ণসংখ্যা। অবৈধ শ্রেণি: ১৮-র কম, ৬০-এর বেশি, ভগ্নাংশ, সংখ্যা নয় (লেখা, `null`, `NaN`)। Boundary Value: ১৭, ১৮, ১৯, এবং ৫৯, ৬০, ৬১। প্রতিনিধি: ৩০ (বৈধের মাঝখান), ১৮.৫, `'30'`।

সমাধান ২

age.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

const isValidAge = (age) => Number.isInteger(age) && age >= 18 && age <= 60;

const rows = [[17, false], [18, true], [19, true], [30, true],
  [59, true], [60, true], [61, false], [18.5, false], ['30', false]];

for (const [age, expected] of rows) {
  test(`বয়স ${JSON.stringify(age)}: ${expected}`, () => {
    assert.equal(isValidAge(age), expected);
  });
}
```

Output

```
ok 1 - বয়স 17: false
ok 2 - বয়স 18: true
ok 3 - বয়স 19: true
ok 4 - বয়স 30: true
ok 5 - বয়স 59: true
ok 6 - বয়স 60: true
ok 7 - বয়স 61: false
ok 8 - বয়স 18.5: false
ok 9 - বয়স "30": false
# tests 9
# pass 9
# fail 0
```

সমাধান ৩ — `assert.rejects` একটি Promise ফেরত দেয়, কিন্তু `await` বা `return` নেই, এবং Test Function `async` নয়। Test সঙ্গে সঙ্গে শেষ হয় ও পাস করে; `load` যদি Reject না-ও করে, ব্যর্থতা হারিয়ে যায়। সমাধান: `test('rejects', async () => { await assert.rejects(load('x')); });`।

সমাধান ৪

log.test.mjs · JavaScript — চলবে: Node.js

```
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';

function processOrder(order, log) {
  const total = order.items.reduce((sum, item) => sum + item.price, 0);
  log('done');
  return total;
}

test('processOrder শেষে একবার "done" Log করে', () => {
  const log = mock.fn();
  const order = { items: [{ price: 50 }, { price: 25 }] };
  assert.equal(processOrder(order, log), 75);
  assert.equal(log.mock.callCount(), 1);
  assert.deepEqual(log.mock.calls[0].arguments, ['done']);
});
```

Output

```
ok 1 - processOrder শেষে একবার "done" Log করে
# tests 1
# pass 1
# fail 0
```

সমাধান ৫ — (ক) Unit: বিশুদ্ধ হিসাব, অনেক ইনপুট। (খ) Integration: API ও Provider-এর মধ্যে আসল HTTP। (গ) E2E: Browser, HTML, Script ও Server একসঙ্গে।

সমাধান ৬

format.mjs · JavaScript — চলবে: Node.js

```
export function formatAmount(minor, currency) {
  const sign = minor < 0 ? '-' : '';
  const abs = Math.abs(minor);
  const whole = Math.floor(abs / 100).toLocaleString('en-US');
  return `${sign}${whole}.${String(abs % 100).padStart(2, '0')} ${currency}`;
}
```

negative.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('ধনাত্মক পরিমাণ', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
});

test('ঋণাত্মক পরিমাণ (ফেরত টাকা)', () => {
  assert.equal(formatAmount(-250, 'USD'), '-2.50 USD');
});
```

Output

```
ok 1 - ধনাত্মক পরিমাণ
ok 2 - ঋণাত্মক পরিমাণ (ফেরত টাকা)
# tests 2
# pass 2
# fail 0
```

পুরোনো Green সংস্করণে দ্বিতীয় Test-এর ফল ছিল `-3.-50 USD` (Red)। চিহ্ন আলাদা করে পরম মানে কাজ করাই সমাধান—ঠিক Project-এর `formatAmount`।

সমাধান ৭

debounce.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
```

```
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

test('৩০০ ms-এর মধ্যে তিন ডাকে একবার, শেষ মান দিয়ে', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const search = t.mock.fn();
  const debounced = debounce(search, 300);
  debounced('ঢা');
  t.mock.timers.tick(100);
  debounced('ঢাকা');
  t.mock.timers.tick(299);
  assert.equal(search.mock.callCount(), 0);
  t.mock.timers.tick(1);
  assert.deepEqual(search.mock.calls.map((c) => c.arguments[0]), ['ঢাকা']);
});
```

Output

```
ok 1 - ৩০০ ms-এর মধ্যে তিন ডাকে একবার, শেষ মান দিয়ে
# tests 1
# pass 1
# fail 0
```

সমাধান ৮

commas.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { parseAmount, formatAmount }
  from '../../project/currency-desk/src/money.mjs';

test('কমা থাকুক বা না থাকুক, একই মান', () => {
  let seed = 8;
  for (let i = 0; i < 2_000; i += 1) {
    seed = (seed * 1_103_515_245 + 12_345) % 2_147_483_648;
    const minor = 1 + (seed % 50_000_000);
    const withCommas = formatAmount(minor, 'X').replace(' X', '');
    const plain = withCommas.replaceAll(',', '');
    assert.equal(parseAmount(plain), parseAmount(withCommas));
  }
});
```

Output

```
ok 1 - কমা থাকুক বা না থাকুক, একই মান
# tests 1
# pass 1
# fail 0
```

সমাধান ৯

too-many.test.mjs · JavaScript — চলবে: Node.js

```
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';
import { createRateClient } from '../../project/currency-desk/src/rates.mjs';

test('বর্তমান আচরণ: 429 Retry হয় না', async () => {
  const fetchFn = mock.fn(async () => new Response('', { status: 429 }));
  const rates = createRateClient({ baseUrl: 'http://x', fetchFn });
  await assert.rejects(rates.getRate('USD', 'BDT'),
    { code: 'UPSTREAM', retryable: false });
  assert.equal(fetchFn.mock.callCount(), 1);
});
```

Output

```
ok 1 - বর্তমান আচরণ: 429 Retry হয় না
# tests 1
# pass 1
# fail 0
```

Test বর্তমান আচরণ নথিভুক্ত করে: শুধু 5xx Retry-যোগ্য। ৪২৯ মানে "ধীরে চলুন"; সঙ্গে সঙ্গে Retry করলে অবস্থা খারাপ হয়, কিন্তু Server-এর `Retry-After` Header মেনে পরে চেষ্টা করা যুক্তিসংগত। আচরণ বদলাতে চাইলে আগে Test বদলান (Red), তারপর Code।

সমাধান ১০

cache.test.mjs · JavaScript — চলবে: Node.js

```javascript
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../../project/currency-desk/src/start.mjs';

test('একই জোড়ার দুটি Quote, Provider-এ একটি অনুরোধ', async (t) => {
  const desk = await startDesk();
  t.after(() => desk.close());
  const body = JSON.stringify({ from: 'USD', to: 'BDT', amount: '10' });
  for (let i = 0; i < 2; i += 1) {
    const url = `${desk.url}/api/quotes`;
    const response = await fetch(url, { method: 'POST', body });
    assert.equal(response.status, 201);
    await response.json();
  }
  assert.equal(desk.provider.controls.requests, 1);
});
```

Output

```
ok 1 - একই জোড়ার দুটি Quote, Provider-এ একটি অনুরোধ
# tests 1
# pass 1
# fail 0
```

`controls.requests` নকল Provider-এর নিজস্ব গণনা; আসল Server-এর ভেতরে না ঢুকেই Cache প্রমাণিত।

সমাধান ১১

retry.test.mjs · JavaScript — চলবে: Node.js

```javascript
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';

async function retry(fn, { retries = 2, sleep = async () => {} } = {}) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await fn(attempt);
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      await sleep(100 * 2 ** attempt);
    }
  }
```

```
}

test('প্রথমবারেই সফল', async () => {
  assert.equal(await retry(async () => 'ok'), 'ok');
});

test('দুবার ব্যর্থ, তৃতীয়বার সফল; অপেক্ষা 100, 200', async () => {
  const sleep = mock.fn(async () => {});
  const fn = async (attempt) => {
    if (attempt < 2) {
      throw new Error('fail');
    }
    return 'ok';
  };
  assert.equal(await retry(fn, { sleep }), 'ok');
  assert.deepEqual(sleep.mock.calls.map((c) => c.arguments[0]), [100, 200]);
});

test('সব ব্যর্থ: শেষ Error', async () => {
  const fn = mock.fn(async (attempt) => {
    throw new Error(`fail ${attempt}`);
  });
  await assert.rejects(retry(fn), /fail 2/);
  assert.equal(fn.mock.callCount(), 3);
});
```

Output

```
ok 1 - প্রথমবারেই সফল
ok 2 - দুবার ব্যর্থ, তৃতীয়বার সফল; অপেক্ষা 100, 200
ok 3 - সব ব্যর্থ: শেষ Error
# tests 3
# pass 3
# fail 0
```

TDD-তে তিনটি Test আগে লেখা হয়েছিল এবং `retry` না থাকায় তিনটিই ব্যর্থ হয়েছিল (Red); তারপর এই Code।

সমাধান ৬২

e2e-pair.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../../project/currency-desk/src/start.mjs';
import { findBrowser, launch }
  from '../../project/currency-desk/tests/support/browser.mjs';

test('অজানা মুদ্রা-জোড়ার বার্তা', { skip: !findBrowser() }, async (t) => {
```

```
  const desk = await startDesk();
  const page = await launch();
  t.after(async () => {
    await page.close();
    await desk.close();
  });
  await page.open(desk.url);
  await page.type('#to', 'JPY');
  await page.type('#amount', '10');
  await page.click('#get-quote');
  const text = await page.eval(`new Promise((resolve) => {
    const tick = () => document.querySelector('#result').dataset.state
      ? resolve(document.querySelector('#result').textContent)
      : setTimeout(tick, 50);
    tick();
  })`);
  assert.equal(text, 'USD-JPY-এর হার নেই');
});
```

Output

```
ok 1 - অজানা মুদ্রা-জোড়ার বার্তা
# tests 1
# pass 1
# fail 0
```

২২.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Fee-র সীমা ১,০০০ টাকা হলে Boundary Value Analysis কোন ইনপুট নেয়?

ক) ৫০০, ৫,০০০

খ) ৯৯৯.৯৯, ১,০০০.০০, ১,০০০.০১

গ) শুধু ১,০০০

ঘ) এলোমেলো ১০টি মান

প্রশ্ন ২. Equivalence Partitioning-এর উদ্দেশ্য?

ক) একই আচরণের ইনপুটগুলোকে শ্রেণিতে ভাগ করে প্রতিনিধি Test

খ) সব ইনপুট Test করা

গ) Test দ্রুত চালানো

ঘ) Coverage বাড়ানো

প্রশ্ন ৩. TDD-র সঠিক ক্রম?

ক) Code, Test, Refactor

খ) Refactor, Red, Green

গ) Red, Green, Refactor

ঘ) Green, Red, Refactor

প্রশ্ন ৪. `t.mock.timers.tick(1000)` কী করে?

ক) এক সেকেন্ড আসলেই অপেক্ষা করে

খ) Test-এর Timeout বাড়ায়

গ) আসল ঘড়ি বদলায়

ঘ) নকল ঘড়িতে সময় এগিয়ে নির্ধারিত Timer চালায়

প্রশ্ন ৫. ১০০% Line Coverage মানে কী?

ক) প্রতিটি লাইন অন্তত একটি Test-এ চলেছে

খ) কোনো ত্রুটি নেই

গ) সব ইনপুট Test হয়েছে

ঘ) Test-গুলো সঠিক

প্রশ্ন ৬. Flaky Test-এর একটি সাধারণ কারণ?

ক) Table-driven Test

খ) Port 0

গ) আসল সময়ের ওপর নির্ভরতা বা ভাগ করা অবস্থা

ঘ) `t.after`

প্রশ্ন ৭. Integration Test-এ বাইরের আসল Rate API-র বদলে স্থানীয় নকল Server কেন?

ক) আসল API ধীর নয়

খ) Test নিয়ন্ত্রিত, পুনরুৎপাদনযোগ্য, এবং ব্যর্থতা ইচ্ছামতো তৈরি করা যায়

গ) নকল Server বেশি সঠিক হার দেয়

ঘ) HTTP লাগে না

প্রশ্ন ৮. Property-based Test কী যাচাই করে?

ক) একটি নির্দিষ্ট ইনপুটের নির্দিষ্ট ফল

খ) শুধু Error

গ) UI-এর রং

ঘ) অনেক ইনপুটে একটি সাধারণ নিয়ম

প্রশ্ন ৯. E2E Test-এ নির্দিষ্ট সময় অপেক্ষার (`sleep(2000)`) বদলে কী ভালো?

ক) একটি শর্তের জন্য অপেক্ষা (যেমন Result-এর একটি চিহ্ন)

খ) আরও বেশি সময় অপেক্ষা

গ) Test বাদ দেওয়া

ঘ) Browser দ্রুত করা

প্রশ্ন ১০. কোন জিনিস Mock করা উচিত নয়?

ক) Network

খ) ঘড়ি

গ) আপনার নিজের বিশুদ্ধ হিসাবের Function, যাকে Test করছেন

ঘ) ইমেইল পাঠানো

২২.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–গ · ৭–খ · ৮–ঘ · ৯–ক · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	সীমার ঠিক নিচে, সীমায় ও ঠিক ওপরে; এখানেই "লাফ" ধরা পড়েছিল।
২	ক	কম Test-এ প্রতিটি আচরণ-শ্রেণি যাচাই।
৩	গ	ব্যর্থ Test আগে, তারপর সরল Code, তারপর পরিষ্কার।
৪	ঘ	আসল অপেক্ষা ছাড়াই সময়-নির্ভর আচরণ।
৫	ক	শুধু "চলেছে"; ঠিক ছিল কি না, তা বলে না।
৬	গ	তাই নকল ঘড়ি ও প্রতিটি Test-এ নতুন অবস্থা।
৭	খ	`failNext` দিয়ে 503 তৈরি করা যায়।
৮	ঘ	যেমন "Fee কখনো কমে না"।
৯	ক	দ্রুত Machine-এ অপচয় নেই, ধীর Machine-এ ব্যর্থতা নেই।
১০	গ	তখন Test শুধু Mock-কেই Test করে।

২২.২১ অধ্যায়ের সারসংক্ষেপ

- নিয়ম Unit-এ, সংযোগ ও Error-এর পথ Integration-এ, গুরুত্বপূর্ণ ব্যবহারকারীর পথ E2E-তে।
- Arrange–Act–Assert ও Table-driven Test: প্রতিটি সারি একটি নামযুক্ত Test।

- Equivalence Partitioning কম Test-এ বেশি আচরণ ঢাকে; Boundary Value Analysis সীমার ত্রুটি ও ব্যবসার প্রশ্ন ধরে।
- Property-based Test হাজারো Seed-যুক্ত ইনপুটে নিয়ম যাচাই করে এবং অজানা সীমাও খুঁজে পায়।
- TDD: Red, Green, Refactor; যে Test কখনো ব্যর্থ হয়নি, তার ওপর ভরসা নেই।
- `mock.fn`, `mock.method`, `mock.timers`: Network, Method ও সময় নিয়ন্ত্রণে; Mock শুধু সীমানায়।
- Integration Test: স্থানীয় Server, Port 0, `127.0.0.1`, `t.after`-এ পরিষ্কার।
- Coverage ফাঁক দেখায়, গুণমান নয়।
- E2E: আসল Browser, শর্তের অপেক্ষা, Browser না থাকলে Skip।
- CI প্রতিটি Pull Request-এ Test চালায়; Flaky Test কখনো উপেক্ষা নয়।

পরের অধ্যায়ে Performance-এর উন্নত কাজ: Profiling-এর হাতিয়ার, Browser Rendering, Virtual Scrolling এবং Web Worker।

২২.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Arrange–Act–Assert	প্রস্তুতি, কাজ, প্রত্যাশা: Test-এর তিন ধাপ।
Table-driven Test	ইনপুট ও প্রত্যাশার টেবিল থেকে তৈরি Test।
Equivalence Partition	একই আচরণ পাওয়া ইনপুটের শ্রেণি।
Boundary Value Analysis	শ্রেণির সীমার আশেপাশে Test।
Property-based Test	অনেক ইনপুটে একটি সাধারণ নিয়মের যাচাই।
TDD	Test আগে লিখে Code লেখার চক্র।
Mock Timers	নকল ঘড়িতে Timer চালানো।
Integration Test	একাধিক অংশ একসঙ্গে, আসল সংযোগে।
End-to-end Test	আসল Browser-এ ব্যবহারকারীর পথ।
Flaky Test	কখনো পাস, কখনো ব্যর্থ হওয়া Test।
Coverage	কোন Code কোনো Test-এ চলেছে তার পরিমাপ।
CI	প্রতিটি পরিবর্তনে স্বয়ংক্রিয় Build ও Test।

২২.২৩ আরও পড়ুন

- **Node.js — Test runner:** `https://nodejs.org/api/test.html` — `mock`, `mock.timers`, Coverage, Reporter।
- **Martin Fowler — The Practical Test Pyramid:** `https://martinfowler.com/articles/practical-test-pyramid.html` — স্তরভিত্তিক কৌশল।
- **Playwright — Getting started:** `https://playwright.dev/docs/intro` — পেশাদার E2E Testing।
- **fast-check — Property based testing:** `https://fast-check.dev/` — Shrinking-সহ Property Test।
- **GitHub Docs — Building and testing Node.js:** `https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs` — CI Workflow।

অধ্যায় ২৩

২৩

JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- অনুমানের বদলে Profiling দিয়ে খুঁজে বের করতে, সময় আসলে কোথায় যাচ্ছে।
- User Timing API (`performance.mark` , `performance.measure`) ও `PerformanceObserver` দিয়ে নিজের Code-এ মাপার বিন্দু বসাতে।
- Node.js-এর Inspector দিয়ে CPU Profile নিতে এবং তার Self Time পড়তে।
- Event Loop Delay মাপতে, এবং Heap-এর বৃদ্ধি দেখে Memory Leak শনাক্ত করতে।
- Browser-এর Rendering Pipeline (Style, Layout, Paint, Composite) বুঝে কোন CSS পরিবর্তন সস্তা, কোনটি দামি তা বলতে।
- Virtual Scrolling দিয়ে হাজারো সারির তালিকা অল্প DOM-এ দেখাতে।
- Web Worker-এ ভারী কাজ সরিয়ে Main Thread মুক্ত রাখতে, এবং Transferable Object-এর সুবিধা বুঝতে।
- একটি সম্পূর্ণ **TravelSearch Explorer** তৈরি করতে: ১২,০০০ Record, Index-ভিত্তিক খোঁজ, Worker ও Virtual List।

পূর্বশর্ত

- অধ্যায় ১৯: পরিমাপের পদ্ধতি (Warm-up, Median), Big-O, Long Task, Layout Thrashing, Garbage Collection, `WeakMap`।
- অধ্যায় ১০ ও ১৪: Event Loop, Debounce।
- অধ্যায় ১২: ES Module।
- অধ্যায় ২২: Property-based Test, Browser-এ E2E।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৯ একটি নীতি দিয়ে শেষ হয়েছিল: আগে মাপুন, তারপর Optimize করুন। কিন্তু একটি বাস্তব Application-এ শত শত Function, এবং "ধীর" অভিযোগটি প্রায়ই অস্পষ্ট: "খোঁজার সময় Page আটকে যায়"। কোন Function দায়ী? Rendering, না হিসাব? Memory কি ধীরে ধীরে বাড়ছে? এই প্রশ্নের উত্তর অনুমানে দেওয়া যায় না; দরকার যন্ত্র—**Profiler**—যা Program চলার সময় দেখে আসলে কী ঘটছে।

এই অধ্যায়ের দ্বিতীয় অংশ Browser-এর বিশেষ সমস্যা: হাজারো সারির তালিকা, এবং ভারী হিসাব যা Main Thread আটকে রাখে। দুটি কৌশল—Virtual Scrolling ও Web Worker—আধুনিক Web Application-এর অপরিহার্য হাতিয়ার, এবং দুটিই ভুলভাবে প্রয়োগ করা সহজ।

২৩.১ আগে যা শিখেছেন, এখন যা নতুন

এই বইয়ের অধ্যায় ১৯ ও ২৩—দুটিই Performance নিয়ে। পুনরাবৃত্তি এড়াতে এই অধ্যায় অধ্যায় ১৯-এর ধারণাগুলো সংক্ষেপে উল্লেখ করে এবং নতুন স্তরে যায়:

অধ্যায় ১৯-এ শেখা	এই অধ্যায়ে নতুন
`performance.now()`, Warm-up, Median	User Timing (`mark`, `measure`), `PerformanceObserver`
DevTools Performance Panel-এর ধারণা	Programmatic CPU Profile, Self Time-এর বিশ্লেষণ
Long Task, কাজ টুকরোয় ভাগ	Event Loop Delay-এর Histogram
GC ও Memory Leak-এর ধারণা	Heap মেপে Leak প্রমাণ
Layout Thrashing	Rendering Pipeline, `transform` বনাম Layout
বড় তালিকায় Virtualization-এর উল্লেখ	Virtual Scrolling-এর পূর্ণ Implementation ও Test
Web Worker-এর উল্লেখ	Module Worker, Message, Transferable

২৩.২ মাপার বিন্দু: User Timing API

`performance.now()` দিয়ে শুরু ও শেষ বিয়োগ করা কাজ করে, কিন্তু ফলটি শুধু একটি Variable-এ থাকে। **User Timing API** মাপকে নামযুক্ত Entry বানায়: `performance.mark(name)` একটি মুহূর্ত চিহ্নিত করে, `performance.measure(name, startMark, endMark)` দুটির মধ্যের সময় একটি Entry হিসেবে রাখে। Browser-এর DevTools এই Entry-গুলো Performance Panel-এর "Timings" সারিতে দেখায়, তাই আপনার নিজের কাজ Profile-এর ভেতরে নাম ধরে খুঁজে পাওয়া যায়।

user-timing.js · JavaScript — চলবে: Browser Console ও Node.js

```
performance.mark('filter-start');
const items = Array.from({ length: 50_000 }, (_, i) => i);
const even = items.filter((n) => n % 2 === 0);
performance.mark('filter-end');

const entry = performance.measure('filter', 'filter-start', 'filter-end');
console.log(entry.name, entry.entryType, even.length);
console.log(performance.getEntriesByName('filter').length, typeof entry.duration);
```

Output

```
filter measure 25000
1 number
```

`duration`-এর মান ছাপা হয়নি, কারণ তা প্রতিবার ও প্রতি যন্ত্রে আলাদা; এই বইয়ের স্বয়ংক্রিয় যাচাই শুধু স্থির অংশ মেলায় (অধ্যায় ১৯-এর নীতি: সময় কখনো Pass/Fail শর্ত নয়)। একই Code Browser ও Node.js দুটিতেই চলে।

`PerformanceObserver` Entry তৈরি হওয়ার সঙ্গে সঙ্গে জানায়, তাই মাপার Code ও Report-এর Code আলাদা রাখা যায়: যে Module কাজ করে সে শুধু `mark` / `measure` বসায়; অন্য কোথাও একজন Observer সেগুলো জমা বা পাঠায় (যেমন Analytics-এ)।

observer.mjs · JavaScript — চলবে: Node.js

```
import { PerformanceObserver, performance } from 'node:perf_hooks';

const seen = [];
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((entry) => seen.push(entry.name));
});
observer.observe({ entryTypes: ['measure'] });

for (const step of ['load', 'index', 'search']) {
  performance.mark(`${step}-start`);
  performance.measure(step, `${step}-start`);
}
```

```
setTimeout(() => {
  observer.disconnect();
  console.log(seen.join(' > '));
}, 0);
```

Output

```
load > index > search
```

Observer-এর Callback পরে, একসঙ্গে (Batch-এ) আসে—তাই ফল পড়া হয়েছে একটি `setTimeout`-এর ভেতরে। `measure`-এ শেষ চিহ্ন না দিলে "এখন" পর্যন্ত মাপা হয়।

২৩.৩ CPU Profile: সময় আসলে কোথায় যায়

User Timing বলে **কতক্ষণ**; Profiler বলে **কোথায়**। CPU Profiler প্রতি কয়েক মাইক্রোসেকেন্ডে দেখে নেয় এই মুহূর্তে কোন Function চলছে (Sampling)। যে Function-এ সবচেয়ে বেশি Sample, সেখানেই সবচেয়ে বেশি সময়। দুটি পরিমাপ গুরুত্বপূর্ণ:

পরিমাপ	অর্থ	কাজে লাগে
Self Time	Function নিজের Code-এ কাটানো সময়	সবচেয়ে ব্যয়বহুল লাইন খোঁজা
Total Time	নিজের ও তার ডাকা সব Function-এর সময়	কোন বড় কাজ দায়ী

Node.js-এ Profile নেওয়ার তিনটি উপায়: `node --cpu-prof script.mjs` (একটি `.cpuprofile` File লেখে, যা Chrome DevTools-এ খোলা যায়), `node --inspect` দিয়ে DevTools যুক্ত করা, অথবা Program-এর ভেতর থেকে `node:inspector`-এর Profiler। Project-এর `profile.mjs` তৃতীয়টি ব্যবহার করে, এবং `profile-report.mjs` Profile-এর JSON পড়ে Self Time-এর তালিকা বানায়। এই বইয়ের লেখার সময় একটি যন্ত্রে (Intel Core i5-6600, Windows 10, Node.js v22.16.0) এক চালানোর আসল Output:

Terminal

```
Node.js v22.16.0, win32; ১৫০টি করে খোঁজ
naive    median 8.293 ms
indexed  median 0.392 ms
সবচেয়ে বেশি Self Time (CPU Profile):
  68.6%  words (search.mjs:3)
  16.0%  (search.mjs)
  7.0%  naiveSearch (search.mjs:17)
  5.1%  (garbage collector) (native:0)
```

Profile একটি স্পষ্ট গল্প বলে: মোট সময়ের প্রায় দুই-তৃতীয়াংশ যায় `words`-এ—যে Function লেখাকে শব্দে ভাগ করে। কারণ সরল খোঁজ প্রতিটি প্রশ্নে ১২,০০০ Record-এর প্রতিটির লেখা **আবার** ভাগ করে। কোন Loop বা কোন তুলনা ধীর, তা অনুমান করলে আমরা হয়তো `startsWith` বা `filter` নিয়ে সময় নষ্ট করতাম। সমাধান তাই একটি: শব্দগুলো একবার ভাগ করে একটি **Index**-এ রাখা—Project-এর `buildIndex`—এবং Median ৮.৩ থেকে ০.৪ ms। সংখ্যাগুলো আপনার যন্ত্রে আলাদা হবে; অনুপাত ও Profile-এর গঠন দেখুন। "(garbage collector)" সারিটিও লক্ষ করুন: প্রতিবার নতুন Array তৈরি GC-কেও কাজ দেয়।

২৩.৪ Event Loop Delay এবং Memory

একটি Server-এ "ধীর" মানে প্রায়ই Event Loop আটকে থাকা: একটি Request-এর ভারী হিসাব চলার সময় বাকি সব Request অপেক্ষা করে। `monitorEventLoopDelay` প্রতিটি Event Loop চক্রের দেরি একটি Histogram-এ জমায়:

loop-delay.mjs · JavaScript — চলবে: Node.js

```
import { monitorEventLoopDelay } from 'node:perf_hooks';

const histogram = monitorEventLoopDelay({ resolution: 10 });
histogram.enable();

function blockFor(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {
    // ইচ্ছাকৃতভাবে Event Loop আটকে রাখা (শিক্ষামূলক প্রদর্শন)
  }
}

setTimeout(() => blockFor(200), 50);
setTimeout(() => {
  histogram.disable();
  const ms = (ns) => (ns / 1e6).toFixed(1);
  console.log(`median ${ms(histogram.percentile(50))} ms, `
    + `p99 ${ms(histogram.percentile(99))} ms, max ${ms(histogram.max)} ms`);
}, 500);
```

এর সংখ্যা প্রতিবার আলাদা, তাই এখানে কোনো যাচাই-করা Output Panel নেই; এই বইয়ের লেখার সময় এক চালানোর আসল ফল:

Terminal

```
median 15.9 ms, p99 200.1 ms, max 200.1 ms
```

Median প্রায় ১৬ ms—এই Windows যন্ত্রে Timer-এর স্বাভাবিক সূক্ষ্মতা (প্রায় ১৫.৬ ms)-র কাছাকাছি—কিন্তু p99 ও সর্বোচ্চ প্রায় ২০০ ms: সেই একটি আটকে থাকার সময়টুকু কোনো Request উত্তর পায়নি। Production-এ এই Histogram-এর p99 নিয়মিত Monitoring-এ পাঠানো হয়।

Browser-এ একই প্রশ্নের উত্তর দেয় **Long Task API**: Main Thread ৫০ ms-এর বেশি আটকে থাকলে একটি `longtask` Entry তৈরি হয়, যা `PerformanceObserver` দিয়ে ধরা যায়:

long-task.js · JavaScript — চলবে: Browser

```
const seen = [];
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((entry) => seen.push(entry.entryType));
});
observer.observe({ type: 'longtask', buffered: true });

setTimeout(() => {
  const end = performance.now() + 120;
  while (performance.now() < end) {
    // ১২০ ms Main Thread আটকে রাখা (শিক্ষামূলক প্রদর্শন)
  }
}, 50);

setTimeout(() => {
  observer.disconnect();
  console.log(`Long Task ধরা পড়েছে: ${seen.includes('longtask')}`);
}, 500);
```

Output

```
Long Task ধরা পড়েছে: true
```

১২০ ms-এর আটকে থাকা ৫০ ms-এর সংজ্ঞার অনেক ওপরে, তাই ফল স্থির। বাস্তব Website-এ এই Observer প্রকৃত ব্যবহারকারীর Browser থেকে Long Task-এর সংখ্যা ও সময় সংগ্রহ করে (Real User Monitoring)—Lab-এর একটি দ্রুত যন্ত্রে যা কখনো ধরা পড়ে না, ধীর ফোনে তা নিয়মিত। Google-এর **INP** (Interaction to Next Paint) পরিমাপ ঠিক এই সমস্যারই ফল মাপে: ব্যবহারকারীর Click বা চাপ থেকে পরের ছবি আঁকা পর্যন্ত সময়, যা Long Task থাকলে বেড়ে যায়। নতুন Chrome-এ আরও বিস্তারিত `long-animation-frame` Entry আছে, যা বলে দেয় কোন Script দায়ী।

Memory: Leak প্রমাণের নির্ভরযোগ্য উপায় হলো Garbage Collection চালিয়ে তারপর Heap মাপা—নইলে "Memory বেশি" মানে হয়তো GC এখনো চলেনি। নিচের **শিক্ষামূলক প্রদর্শন** দুটি একই রকম Cache তুলনা করে: একটি সীমাহীন, একটি সীমিত (অধ্যায় ১৯-এর LRU-র নীতি)। GC হাতে চালাতে এখানে V8-এর একটি Flag Program-এর ভেতর থেকে চালু করা হয়েছে—শুধু পরীক্ষার জন্য, Production-এ নয়।

heap.mjs · JavaScript — চলবে: Node.js

```
import { setFlagsFromString } from 'node:v8';
import { runInNewContext } from 'node:vm';

setFlagsFromString('--expose-gc');
const gc = runInNewContext('gc');
const heapMB = () => {
  gc();
```

```
  return process.memoryUsage().heapUsed / 1024 / 1024;
};

function fillCache(cache, limit) {
  for (let i = 0; i < 400; i += 1) {
    cache.set(`report-${i}`, new Array(10_000).fill(i));
    if (cache.size > limit) {
      cache.delete(cache.keys().next().value);
    }
  }
}

const before = heapMB();
const unbounded = new Map();
fillCache(unbounded, Infinity);
const afterUnbounded = heapMB();
const bounded = new Map();
fillCache(bounded, 20);
const afterBounded = heapMB();

const grewBy = (a, b) => b - a;
console.log(`সীমাহীন Cache-এ Heap ৩০ MB-র বেশি বেড়েছে: ${grewBy(before, afterUnbounded) >
    30}`);
console.log(`সীমিত Cache-এ বাড়তি ৫ MB-র কম: ${grewBy(afterUnbounded, afterBounded) <
    5}`);
console.log(unbounded.size, bounded.size);
```

Output

```
সীমাহীন Cache-এ Heap ৩০ MB-র বেশি বেড়েছে: true
সীমিত Cache-এ বাড়তি ৫ MB-র কম: true
400 20
```

এই তুলনা সময় নয়, Memory: GC-র পরে যা থেকে যায় তা পৌঁছানো যায় এমন Object, তাই ফল স্থির। ৪০০টি Report, প্রতিটি দশ হাজার সংখ্যার Array (প্রায় ৮০ KB), সীমাহীন Cache-এ সবই জীবিত; সীমিত Cache-এ মাত্র ২০টি। Browser-এ একই কাজ DevTools-এর Memory Panel-এ দুটি Heap Snapshot তুলনা করে করা হয় ("Comparison" View-তে কোন ধরনের Object বেড়েছে)।

২৩.৫ Browser Rendering Pipeline

Browser-এ একটি পরিবর্তন পর্দায় পৌঁছাতে পাঁচটি ধাপ পেরোতে পারে:

ধাপ	কাজ	কী ট্রিগার করে
JavaScript	আপনার Code	Event, Timer

ধাপ	কাজ	কী ট্রিগার করে
Style	কোন Element-এ কোন CSS	Class, Style বদল
Layout	প্রতিটি Element-এর আকার ও অবস্থান	`width`, `height`, `top`, Font, লেখা
Paint	Pixel আঁকা	রং, ছায়া, লেখা
Composite	স্তরগুলো জোড়া (প্রায়ই GPU-তে)	`transform`, `opacity`

যে পরিবর্তন যত আগের ধাপ ট্রিগার করে, তত দামি: `height` বদলালে Layout, Paint, Composite সবই; `transform` বদলালে সাধারণত শুধু Composite। তাই Animation ও সরানোর কাজে `transform` ও `opacity`। পার্থক্যটি মাপা যায় Layout-এর ফল দিয়ে:

pipeline.js · JavaScript চলবে: Browser

```
const box = document.createElement('div');
box.style.height = '50px';
const next = document.createElement('p');
next.textContent = 'পরের অংশ';
document.body.append(box, next);

const before = next.offsetTop;
box.style.transform = 'translateY(100px)';
const afterTransform = next.offsetTop;
box.style.height = '150px';
const afterHeight = next.offsetTop;

console.log(`transform-এর পর পরের অংশ সরেছে? ${afterTransform !== before}`);
console.log(`height-এর পর সরেছে? ${afterHeight !== before}, ${afterHeight -
    before}px`);
```

Output

```
transform-এর পর পরের অংশ সরেছে? false
height-এর পর সরেছে? true, 100px
```

`transform` বাক্সটিকে দৃশ্যত সরিয়েছে, কিন্তু Layout বদলায়নি—পরের অংশ যেখানে ছিল সেখানেই। `height` বদলানো পুরো Page-এর Layout নতুন করে হিসাব করিয়েছে। Project-এর Virtual List ঠিক এই কারণে সারিগুলো `transform: translateY(...)` দিয়ে সরায়। আরেকটি কার্যকর CSS: `content-visibility: auto`, যা পর্দার বাইরের অংশের Layout ও Paint পিছিয়ে দেয়।

২৩.৬ Virtual Scrolling

১২,০০০ সারি DOM-এ রাখলে প্রতিটি সারির জন্য Element, Style, Layout ও Memory—পর্দায় দেখা যায় মাত্র ১০টি। **Virtual Scrolling** শুধু দৃশ্যমান সারি (এবং দুই পাশে কয়েকটি বাড়তি, **Overscan**) আঁকে। কৌশলটির তিনটি অংশ:

১. **Spacer:** একটি খালি Element, যার উচ্চতা সব সারির মোট উচ্চতা (১২,০০০ × ৪৮px)। এতে Scrollbar-এর আকার ও অবস্থান সঠিক থাকে।

২. **হিসাব:** `scrollTop` থেকে প্রথম ও শেষ দৃশ্যমান সারির Index—একটি বিশুদ্ধ Function, তাই Browser ছাড়াই Test করা যায়।

৩. **অবস্থান:** আঁকা সারিগুলো `translateY(start × rowHeight)` দিয়ে সঠিক জায়গায়।

virtual-demo.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { visibleRange } from '../project/travel-search/src/virtual.mjs';

const list = { viewport: 480, rowHeight: 48, total: 12_000 };
for (const scrollTop of [0, 24, 48_000, 575_520]) {
  const { start, end } = visibleRange({ ...list, scrollTop });
  console.log(`scrollTop ${scrollTop}: সারি ${start} থেকে ${end - 1}, মোট ${end -
      start}`);
}
```

Output

```
scrollTop 0: সারি 0 থেকে 14, মোট 15
scrollTop 24: সারি 0 থেকে 15, মোট 16
scrollTop 48000: সারি 995 থেকে 1014, মোট 20
scrollTop 575520: সারি 11985 থেকে 11999, মোট 15
```

তালিকা ১২,০০০ বা দশ লাখ—DOM-এ সর্বোচ্চ ২০টি সারি। সীমাবদ্ধতা: এই সরল রূপ সব সারির উচ্চতা সমান ধরে; ভিন্ন উচ্চতার সারি, Keyboard দিয়ে চলাচল, Screen Reader-এর জন্য মোট সংখ্যা জানানো (`aria-rowcount`), এবং Browser-এর "Find in page" (যা না-আঁকা সারি খুঁজে পায় না)—এগুলো বাস্তব Data Grid Library সামলায়। তাই প্রশ্নটি সব সময় করুন: সত্যিই কি হাজার সারি দেখাতে হবে, নাকি Pagination বা ভালো খোঁজ যথেষ্ট?

২৩.৭ Web Worker: Main Thread মুক্ত রাখা

Browser-এ JavaScript, Layout, Paint এবং ব্যবহারকারীর Click—সবই সাধারণত একই **Main Thread**-এ। ভারী হিসাব সেখানে চললে Page জমে যায় (অধ্যায় ১৯-এ কাজ টুকরোয় ভাগ করে সমাধান দেখেছিলেন)। **Web Worker** একটি আলাদা Thread, যার নিজস্ব Global Scope: DOM, `window` বা `document` নেই, আছে `postMessage` ও `message` Event। Module Worker (`{ type: 'module' }`) সাধারণ `import` ব্যবহার করতে পারে।

main.mjs · JavaScript চলবে: Browser

```
const url = new URL('./worker.mjs', import.meta.url);
const worker = new Worker(url, { type: 'module' });

worker.addEventListener('message', ({ data }) => {
  console.log(`Worker-এর যোগফল: ${data}`);
  worker.terminate();
});

worker.postMessage(10_000_000);
console.log('postMessage সঙ্গে সঙ্গে ফিরেছে; Main Thread মুক্ত');
```

worker.mjs · JavaScript চলবে: Browser

```
addEventListener('message', ({ data: limit }) => {
  let sum = 0;
  for (let n = 1; n <= limit; n += 1) {
    sum += n;
  }
  postMessage(sum);
});
```

Chrome-এ চালালে:

Output

```
postMessage সঙ্গে সঙ্গে ফিরেছে; Main Thread মুক্ত
Worker-এর যোগফল: 50000005000000
```

ক্রমটিই মূল কথা: `postMessage` হিসাবের জন্য অপেক্ষা করে না, তাই Main Thread পরের লাইনে চলে যায় এবং হিসাবের সময় Click ও Scroll-এ সাড়া দিতে পারে। ফল আসে পরে, একটি Message হিসেবে।

Message-এর খরচ: `postMessage`-এর তথ্য **Structured Clone** হয়—পুরো একটি Copy। বড় তথ্যে এই Copy-ই ধীর হতে পারে। `ArrayBuffer`-এর মতো কিছু Object **Transfer** করা যায়: Copy নয়, মালিকানা হস্তান্তর, এবং পাঠানোর পর মূল Object ব্যবহার-অযোগ্য:

transfer.js · JavaScript চলবে: Browser Console ও Node.js

```
const buffer = new ArrayBuffer(8 * 1024 * 1024);

const copy = structuredClone(buffer);
console.log(`Clone-এর পরে: মূল ${buffer.byteLength}, Copy ${copy.byteLength}`);

const moved = structuredClone(buffer, { transfer: [buffer] });
console.log(`Transfer-এর পরে: মূল ${buffer.byteLength}, নতুন ${moved.byteLength}`);
```

Output

```
Clone-এর পরে: মূল 8388608, Copy 8388608
Transfer-এর পরে: মূল 0, নতুন 8388608
```

`worker.postMessage(data, [buffer])` একই নিয়মে কাজ করে। Project-এর Worker একটি ভিন্ন কৌশল ব্যবহার করে: তথ্য Main Thread-এ পাঠানোই হয় না—Worker নিজেই তৈরি ও Index করে, এবং শুধু ফল ফেরত পাঠায়। সীমাবদ্ধতা: প্রতিটি খোঁজে ফলের তালিকা (সর্বোচ্চ ১২,০০০টি ছোট Object) Clone হয়; আরও বড় তথ্যে শুধু ID বা শুধু দৃশ্যমান পাতা পাঠানো ভালো।

২৩.৮ Project: TravelSearch Explorer

TravelSearch Explorer একটি ভ্রমণ সংস্থার প্যাকেজ খোঁজার Page: ১২,০০০টি কাল্পনিক প্যাকেজ (Seed থেকে, তাই প্রতিবার একই), শব্দের শুরু দিয়ে খোঁজ ("syl tea"), দাম বা Rating-এ সাজানো, এবং একটি Virtual List। খোঁজ চলে একটি Web Worker-এ। অধ্যায় ১৯-এর PerfLab ছিল Node.js-এর Benchmark Harness; এটি একটি পূর্ণ Browser Application, যেখানে Profiling থেকে পাওয়া সিদ্ধান্ত (Index) প্রয়োগ করা হয়েছে এবং আচরণ আসল Browser-এ যাচাই করা হয়েছে।

> **নোট:** শিক্ষামূলক Demo। তথ্য কাল্পনিক; Virtual List সমান-উচ্চতার সারি ধরে নেয় এবং পূর্ণ Accessibility (Keyboard চলাচল, `aria-rowcount`) দেয় না।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-search/
├── index.html, style.css
├── serve.mjs               স্থানীয় Static Server (Worker-এর জন্য HTTP)
├── profile.mjs             Profiling Lab (User Timing ও CPU Profile)
├── src/
│   ├── data.mjs            Seed-যুক্ত ১২,০০০ Record
│   ├── search.mjs          সরল ও Index-ভিত্তিক খোঁজ
│   ├── virtual.mjs         দৃশ্যমান সারির হিসাব
│   ├── search-worker.mjs   Web Worker
│   ├── app.mjs             UI: Debounce, Virtual List, Worker
│   └── profile-report.mjs  CPU Profile থেকে Self Time
└── tests/                  search, virtual, e2e (+ support/browser.mjs)
```

src/data.mjs ও src/search.mjs

data.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// ১২,০০০টি কাল্পনিক ভ্রমণ-প্যাকেজ, Seed থেকে: প্রতিবার হুবহু একই তথ্য।
```

```
const PLACES = ['Dhaka', 'Sylhet', 'Bandarban', 'Rangamati', 'Khulna',
  'Kuakata', 'Srimangal', 'Rajshahi', 'Bogura', 'Saint Martin',
  'Kathmandu', 'Darjeeling', 'Bangkok', 'Kuala Lumpur', 'Dubai'];
const KINDS = ['Hotel', 'Resort', 'Tour', 'Cruise', 'Homestay'];
const WORDS = ['River', 'Hill', 'Lake', 'Garden', 'Sea', 'Tea', 'Royal',
  'Green', 'Golden', 'Silent'];

export function createRandom(seed) {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let t = Math.imul(state ^ (state >>> 15), state | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export function generateRecords(count = 12_000, seed = 23) {
  const random = createRandom(seed);
  const pick = (list) => list[Math.floor(random() * list.length)];
  return Array.from({ length: count }, (_, i) => {
    const place = pick(PLACES);
    const kind = pick(KINDS);
    return {
      id: i + 1,
      title: `${pick(WORDS)} ${pick(WORDS)} ${kind}`,
      place,
      kind,
      price: 1_500 + Math.floor(random() * 60) * 500,
      rating: Math.round((3 + random() * 2) * 10) / 10,
    };
  });
}
```

search.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Browser Console ও Node.js

```
// দুটি খোঁজ: সরল (প্রতিবার সব Record স্ক্যান) এবং Index-ভিত্তিক।
// দুটির ফল হুবহু এক হতে হবে; Test তা যাচাই করে।
const words = (text) => text.toLowerCase().split(/[^a-z0-9]+/).filter(Boolean);
const textOf = (r) => `${r.title} ${r.place} ${r.kind}`;

const SORTS = {
  relevance: (a, b) => a.id - b.id,
  price: (a, b) => a.price - b.price || a.id - b.id,
  rating: (a, b) => b.rating - a.rating || a.id - b.id,
};

function finish(records, { maxPrice = Infinity, sort = 'relevance' }) {
  return records.filter((r) => r.price <= maxPrice).sort(SORTS[sort]);
```

```
}

// প্রতিটি প্রশ্নের শব্দ কোনো Record-এর কোনো শব্দের শুরু হতে হবে।
export function naiveSearch(records, query, options = {}) {
  const terms = words(query);
  const hits = records.filter((r) => {
    const own = words(textOf(r));
    return terms.every((t) => own.some((w) => w.startsWith(t)));
  });
  return finish(hits, options);
}

// Index: সাজানো শব্দের তালিকা এবং প্রতিটি শব্দের Record-গুলো।
export function buildIndex(records) {
  const postings = new Map();
  for (const r of records) {
    for (const w of new Set(words(textOf(r)))) {
      if (!postings.has(w)) {
        postings.set(w, []);
      }
      postings.get(w).push(r);
    }
  }
  return { records, postings, sorted: [...postings.keys()].sort() };
}

// ...
```

search.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

// একটি Prefix-এর সব শব্দ: সাজানো তালিকায় Binary Search, তারপর পরপর।
function wordsWithPrefix(sorted, prefix) {
  let low = 0;
  let high = sorted.length;
  while (low < high) {
    const middle = (low + high) >>> 1;
    if (sorted[middle] < prefix) {
      low = middle + 1;
    } else {
      high = middle;
    }
  }
  const found = [];
  for (let i = low; i < sorted.length && sorted[i].startsWith(prefix); i += 1) {
    found.push(sorted[i]);
  }
  return found;
```

```
}

export function indexedSearch(index, query, options = {}) {
  let result = null;
  for (const term of words(query)) {
    const matches = new Set();
    for (const word of wordsWithPrefix(index.sorted, term)) {
      index.postings.get(word).forEach((r) => matches.add(r));
    }
    result = result === null
      ? matches
      : new Set([...result].filter((r) => matches.has(r)));
  }
  return finish(result === null ? index.records : [...result], options);
}
```

Index-এ প্রতিটি আলাদা শব্দের জন্য সেই শব্দযুক্ত Record-গুলোর তালিকা (**Inverted Index**, সার্চ ইঞ্জিনের মূল কাঠামো), আর শব্দগুলোর একটি সাজানো তালিকা। "syl" খুঁজতে সাজানো তালিকায় Binary Search দিয়ে প্রথম মিলের অবস্থান, তারপর পরপর যতক্ষণ শব্দ "syl" দিয়ে শুরু হয়। একাধিক শব্দে (`tea resort`) ফলগুলোর ছেদ (Intersection)। Index Record-গুলোর **Reference** রাখে, Copy নয়, তাই Memory দ্বিগুণ হয় না (একটি Test এটি যাচাই করে)। সরল খোঁজ (`naiveSearch`) রেখে দেওয়া হয়েছে উদ্দেশ্যমূলকভাবে: সেটি "সঠিক উত্তরের" মাপকাঠি।

src/virtual.mjs ও src/search-worker.mjs

virtual.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// Virtual Scrolling-এর হিসাব: কোন সারিগুলো আঁকতে হবে। বিশুদ্ধ Function, তাই
// Browser ছাড়াই Test করা যায়।
export function visibleRange(options) {
  const { scrollTop, viewport, rowHeight, total, overscan = 5 } = options;
  if (total === 0) {
    return { start: 0, end: 0 };
  }
  const first = Math.floor(Math.max(0, scrollTop) / rowHeight);
  const last = Math.ceil((Math.max(0, scrollTop) + viewport) / rowHeight);
  return {
    start: Math.max(0, first - overscan),
    end: Math.min(total, last + overscan),
  };
}

export const offsetOf = (index, rowHeight) => index * rowHeight;
```

search-worker.mjs · JavaScript — চলবে: Browser

```
// Web Worker: আলাদা Thread-এ তথ্য তৈরি, Index ও খোঁজ। DOM-এ প্রবেশ নেই;
```

```
// Main Thread-এর সঙ্গে শুধু Message।
import { generateRecords } from './data.mjs';
import { buildIndex, indexedSearch } from './search.mjs';

performance.mark('index-start');
const index = buildIndex(generateRecords());
performance.mark('index-end');
const built = performance.measure('index', 'index-start', 'index-end');
postMessage({ type: 'ready', total: index.records.length, ms: built.duration });

addEventListener('message', ({ data }) => {
  const started = performance.now();
  const results = indexedSearch(index, data.query, data.options);
  postMessage({
    type: 'results',
    requestId: data.requestId,
    rows: results,
    ms: performance.now() - started,
  });
});
```

Worker চালু হয়েই তথ্য তৈরি করে Index বানায় এবং একটি `ready` Message পাঠায়; তারপর প্রতিটি প্রশ্নে খুঁজে ফল ও সময় (`ms`) পাঠায়। User Timing Worker-এর ভেতরেও কাজ করে।

src/app.mjs

app.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
import { visibleRange, offsetOf } from './virtual.mjs';

const ROW_HEIGHT = 48;
const $ = (selector) => document.querySelector(selector);
const worker = new Worker(new URL('./search-worker.mjs', import.meta.url),
  { type: 'module' });
let rows = [];
let latestRequest = 0;
let frame = 0;
let timer;

// শুধু দৃশ্যমান সারি (এবং দুই পাশে কয়েকটি বাড়তি) DOM-এ।
function render() {
  frame = 0;
  const list = $('#list');
  const { start, end } = visibleRange({
    scrollTop: list.scrollTop, viewport: list.clientHeight,
    rowHeight: ROW_HEIGHT, total: rows.length,
  });
  $('#spacer').style.height = `${rows.length * ROW_HEIGHT}px`;
  $('#rows').style.transform = `translateY(${offsetOf(start, ROW_HEIGHT)}px)`;
```

```
  $('#rows').replaceChildren(...rows.slice(start, end).map((r) => {
    const item = document.createElement('li');
    item.textContent = `${r.title} · ${r.place} · ৳${r.price} · ${r.rating}`;
    return item;
  }));
}

function search() {
  latestRequest += 1;
  worker.postMessage({
    requestId: latestRequest,
    query: $('#query').value,
    options: { sort: $('#sort').value },
  });
}

// ...
```

app.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) — চলবে: Browser

```
// ...

worker.addEventListener('message', ({ data }) => {
  if (data.type === 'ready') {
    $('#status').textContent = `${data.total}টি প্যাকেজ প্রস্তুত`;
    search();
    return;
  }
  // ধীর পুরোনো উত্তর নতুন উত্তরকে মুছে দেবে না।
  if (data.requestId !== latestRequest) {
    return;
  }
  rows = data.rows;
  $('#list').scrollTop = 0;
  render();
  $('#count').textContent = `${rows.length}টি ফল · খোঁজ ${data.ms.toFixed(1)} ms`;
  $('#count').dataset.request = String(data.requestId);
});

$('#query').addEventListener('input', () => {
  clearTimeout(timer);
  timer = setTimeout(search, 150);
});
$('#sort').addEventListener('change', search);
// একটি Frame-এ একবারই আঁকা, যত Scroll Event-ই আসুক।
$('#list').addEventListener('scroll', () => {
  frame ||= requestAnimationFrame(render);
}, { passive: true });
```

চারটি Performance-এর সিদ্ধান্ত এখানে: খোঁজ Worker-এ; লেখার সময় ১৫০ ms Debounce (অধ্যায় ১৪); Scroll Event-এ সরাসরি না এঁকে `requestAnimationFrame`-এ, এবং একটি Frame-এ একবারই (`frame ||=`); আর প্রতিটি অনুরোধের একটি ক্রমিক নম্বর (`requestId`)—পুরোনো ধীর উত্তর পরে এলে তা উপেক্ষিত, যাতে "sy"-এর উত্তর "sylhet"-এর উত্তরকে মুছে না দেয়। এই শেষ সমস্যাটির নাম **Race Condition**, এবং Asynchronous খোঁজে এটি প্রায় সব সময় থাকে।

index.html ও style.css

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TravelSearch Explorer</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="src/app.mjs"></script>
</head>
<body>
  <h1>TravelSearch Explorer <small>শিক্ষামূলক Demo</small></h1>
  <p id="status" role="status">তথ্য তৈরি হচ্ছে...</p>
  <label>খুঁজুন <input id="query" placeholder="যেমন: sylhet tea"></label>
  <label>সাজান <select id="sort">
    <option value="relevance">ক্রমিক</option>
    <option value="price">দাম</option>
    <option value="rating">Rating</option>
  </select></label>
  <p id="count"></p>
  <div id="list">
    <div id="spacer"></div>
    <ul id="rows"></ul>
  </div>
</body>
</html>
```

style.css · CSS চলবে: Browser

```css
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 48rem;
  margin: 2rem auto;
  padding: 0 1rem;
}

/* Scroll-এর পাত্র: নির্দিষ্ট উচ্চতা, ভেতরে একটি লম্বা Spacer। */
#list {
  position: relative;
```

```
  height: 480px;
  overflow-y: auto;
  border: 1px solid #cbd5e1;
}

/* সারিগুলো Spacer-এর ওপরে; transform দিয়ে সরানো, Layout ছাড়াই। */
#rows {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  margin: 0;
  padding: 0;
  list-style: none;
  will-change: transform;
}

#rows li {
  height: 48px;
  line-height: 48px;
  padding: 0 0.75rem;
  border-bottom: 1px solid #e2e8f0;
  overflow: hidden;
  white-space: nowrap;
}
```

Profiling Lab: profile.mjs

profile.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// Profiling Lab (Node.js): User Timing দিয়ে প্রতিটি খোঁজের সময় এবং
// Inspector-এর CPU Profiler দিয়ে কোন Function সবচেয়ে বেশি সময় নেয়।
// সংখ্যাগুলো যন্ত্রভেদে বদলায়; এই Script কোনো Test-এর অংশ নয়।
import { PerformanceObserver, performance } from 'node:perf_hooks';
import { Session } from 'node:inspector/promises';
import { generateRecords } from './src/data.mjs';
import { buildIndex, naiveSearch, indexedSearch } from './src/search.mjs';
import { topFunctions } from './src/profile-report.mjs';

const durations = { naive: [], indexed: [] };
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((e) => durations[e.name].push(e.duration));
});
observer.observe({ entryTypes: ['measure'] });

function timed(name, fn) {
  performance.mark(`${name}-start`);
  fn();
  performance.measure(name, `${name}-start`);
```

```
}

// ...
```

profile.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const records = generateRecords();
const index = buildIndex(records);
const queries = ['sylhet', 'tea resort', 'go', 'river hill', 'dubai cruise'];

const session = new Session();
session.connect();
await session.post('Profiler.enable');
await session.post('Profiler.start');
for (let round = 0; round < 30; round += 1) {
  for (const q of queries) {
    timed('naive', () => naiveSearch(records, q));
    timed('indexed', () => indexedSearch(index, q));
  }
}
const { profile } = await session.post('Profiler.stop');
await new Promise((resolve) => setImmediate(resolve));
observer.disconnect();

const median = (list) => list.toSorted((a, b) => a - b)[list.length >> 1];
console.log(`Node.js ${process.version}, ${process.platform}; ১৫০টি করে খোঁজ`);
for (const [name, list] of Object.entries(durations)) {
  console.log(`${name.padEnd(8)} median ${median(list).toFixed(3)} ms`);
}
console.log('সবচেয়ে বেশি Self Time (CPU Profile):');
for (const f of topFunctions(profile, 4)) {
  console.log(`  ${f.percent}%  ${f.name}`);
}
```

profile-report.mjs · JavaScript চলবে: Node.js

```
// V8-এর CPU Profile (.cpuprofile-এর একই JSON) থেকে Self Time-এর তালিকা।
// প্রতিটি Sample মানে: সেই মুহূর্তে কোন Function চলছিল।
export function topFunctions(profile, limit = 5) {
  const counts = new Map();
  const byId = new Map(profile.nodes.map((node) => [node.id, node]));
  for (const id of profile.samples) {
    const { functionName, url, lineNumber } = byId.get(id).callFrame;
    const file = url.split('/').pop();
    const name = functionName
      ? `${functionName} (${file || 'native'}:${lineNumber + 1})`
      : `(${file || 'idle বা অন্যান্য'})`;
```

```
    counts.set(name, (counts.get(name) ?? 0) + 1);
  }
  const total = profile.samples.length;
  return [...counts]
    .sort((a, b) => b[1] - a[1])
    .slice(0, limit)
    .map(([name, n]) => ({ name, percent: ((n / total) * 100).toFixed(1) }));
}
```

এর আসল Output অনুচ্ছেদ ২৩.৩-এ। `profile.mjs` ইচ্ছা করে Automated Test-এর অংশ নয়: সময় যন্ত্রভেদে বদলায়।

Test: সঠিকতা আগে, তারপর গতি

search.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Optimization-এর প্রথম শর্ত: দ্রুত সংস্করণের ফল ধীর সংস্করণের হুবহু সমান।
// কোনো Test সময় মাপে না।
import test from 'node:test';
import assert from 'node:assert/strict';
import { generateRecords, createRandom } from '../src/data.mjs';
import { buildIndex, naiveSearch, indexedSearch } from '../src/search.mjs';

const records = generateRecords();
const index = buildIndex(records);
const ids = (list) => list.map((r) => r.id);

// ...

test('Property: ৩০০টি এলোমেলো প্রশ্নে দুই খোঁজ সমান', () => {
  const random = createRandom(99);
  const vocabulary = ['sy', 'tea', 'hill', 'resort', 'dha', 'g', 'cruise',
    'bang', 'lake', 'home', 'k', 'river', 'royal'];
  for (let i = 0; i < 300; i += 1) {
    const size = 1 + Math.floor(random() * 3);
    const q = Array.from({ length: size },
      () => vocabulary[Math.floor(random() * vocabulary.length)]).join(' ');
    const fast = ids(indexedSearch(index, q));
    assert.deepEqual(fast, ids(naiveSearch(records, q)), q);
  }
});

// ...
```

e2e.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

  await t.test('শেষ পর্যন্ত Scroll: শেষ Record আঁকা হয়', async () => {
    await page.eval(`(() => { const list = document.querySelector('#list');
      list.scrollTop = list.scrollHeight; })()`);
    await page.eval(
      'new Promise((r) => requestAnimationFrame(() => setTimeout(r, 50)))');
    const last = await page.eval(
      "document.querySelector('#rows li:last-child').textContent");
    assert.ok(last.startsWith(records.at(-1).title), last);
    assert.ok(await domRows() <= 20);
  });

// ...
```

Index একটি Optimization, এবং প্রতিটি Optimization-এর প্রথম শর্ত: ফল বদলাবে না। তাই Test-এর কেন্দ্র একটি Property (অধ্যায় ২২): ৩০০টি Seed-যুক্ত এলোমেলো প্রশ্নে দুই খোঁজ হুবহু সমান। E2E Test আসল Browser-এ তিনটি আচরণ যাচাই করে, কোনো সময় না মেপে: ১২,০০০ ফলেও DOM-এ সর্বোচ্চ ২০টি সারি, একদম নিচে Scroll করলে শেষ Record আঁকা হয়, এবং Worker-এর খোঁজ ও সাজানো Node.js-এর সরল খোঁজের সঙ্গে মেলে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-search
node --test
node profile.mjs
node serve.mjs
```

`node --test` ১৫টি Test চালায় (`search` ৫, `virtual` ৬, এবং `e2e` ৪—একটি Parent ও তিনটি Subtest); শেষে `# pass 15`, `# fail 0`। Chrome বা Edge না থাকলে E2E কারণসহ Skip হয়। `node serve.mjs` একটি স্থানীয় ঠিকানা দেখায় (Module Worker `file://` থেকে চলে না); DevTools-এর Performance Panel-এ Record করে লিখুন ও Scroll করুন, এবং Elements Panel-এ দেখুন `#rows`-এ কতগুলো `li`।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
"তথ্য তৈরি হচ্ছে…" থেকে নড়ে না	Page `file://` থেকে খোলা; Worker লোড হয়নি	`node serve.mjs`
Scroll-এ ফাঁকা জায়গা দেখা যায়	Overscan খুব কম বা সারির উচ্চতা CSS ও JS-এ আলাদা	`ROW_HEIGHT` ও CSS মেলান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
লেখার সময় পুরোনো ফল ফিরে আসে	`requestId` যাচাই নেই	পুরোনো উত্তর বাদ
Index-এর ফল সরল খোঁজ থেকে আলাদা	Tokenizing দুই জায়গায় ভিন্ন	একই `words` Function
Profile-এ "(idle)" বেশি	Program অপেক্ষা করছে, হিসাব নয়	I/O বা Timer দেখুন

২৩.৯ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Profile ছাড়া অনুমানে Optimize	ভুল জায়গায় সময়	আগে CPU Profile
Total Time দেখে Self Time ভুলে যাওয়া	দায়ী Function অস্পষ্ট	দুটিই দেখুন
GC না চালিয়ে Memory মাপা	বিভ্রান্তিকর সংখ্যা	GC-র পরে মাপ, Snapshot তুলনা
Animation-এ `top` / `height` বদলানো	প্রতি Frame-এ Layout	`transform`, `opacity`
হাজারো সারি DOM-এ	ধীর Rendering, বেশি Memory	Virtual Scrolling বা Pagination
Virtual List-এ উচ্চতা অনুমান ভুল	ফাঁকা জায়গা, লাফ	CSS ও JS-এ একই উচ্চতা
Worker-এ বিশাল তথ্য বারবার পাঠানো	Clone-এর খরচ	Worker-এ তথ্য রাখা, Transfer
Async খোঁজে অনুরোধের ক্রম না দেখা	পুরোনো ফল নতুনকে মুছে দেয়	`requestId`
Optimization-এর পর ফল যাচাই না করা	দ্রুত কিন্তু ভুল	সরল সংস্করণের সঙ্গে Property Test

২৩.১০ Debugging নির্দেশিকা

১. **লক্ষণ মাপুন:** কোন কাজ, কত সময়, কোন যন্ত্রে—User Timing দিয়ে নাম দিন।

২. **CPU Profile নিন:** `node --cpu-prof` বা DevTools-এর Performance Panel; Self Time অনুযায়ী সাজান।

৩. **Main Thread-এর Long Task:** Performance Panel-এ লাল কোণযুক্ত কাজ; সেটি কি Worker-এ যেতে পারে?

৪. **Rendering:** Panel-এ বেগুনি (Layout) ও সবুজ (Paint) ঘটনা; "Rendering" Tab-এ "Paint flashing" চালু করে দেখুন কী নতুন করে আঁকা হচ্ছে।

৫. **Memory:** দুটি Heap Snapshot, মাঝে কাজটি কয়েকবার; Comparison View-তে বাড়তে থাকা Object।

৬. **Server:** Event Loop Delay-এর p99; বেড়ে গেলে কোন Request চলছিল দেখুন।

২৩.১১ পেশাদার অভ্যাস

- **Profile আগে, Code পরে;** এবং পরিবর্তনের পর আবার Profile।
- **নিজের Code-এ নামযুক্ত মাপের বিন্দু** (User Timing), যাতে Profile-এ খুঁজে পাওয়া যায়।
- **প্রতিটি Optimization-এর সঙ্গে সঠিকতার Test,** সরল সংস্করণকে মাপকাঠি রেখে।
- **Main Thread-এ শুধু UI;** ভারী হিসাব Worker-এ।
- **Animation-এ শুধু `transform` ও `opacity`।**
- **বড় তালিকা আঁকার আগে প্রশ্ন করুন:** Pagination বা ভালো Filter কি যথেষ্ট?
- **ফলের সঙ্গে পরিবেশ লিখুন;** একটি সংখ্যা নয়, অনুপাত ও প্রবণতা।

২৩.১২ অনুশীলনী

প্রাথমিক স্তর

১. `performance.mark` ও `measure` দিয়ে একটি Array সাজানোর কাজের নামযুক্ত পরিমাপ তৈরি করুন, এবং Entry-র নাম ও ধরন ছাপুন।

২. Self Time ও Total Time-এর পার্থক্য একটি উদাহরণ দিয়ে ব্যাখ্যা করুন।

৩. নিচের কোনগুলো সাধারণত শুধু Composite ধাপ ট্রিগার করে: `width`, `transform`, `opacity`, `color`, `top`?

৪. `visibleRange` দিয়ে হিসাব করুন: ৬০০px উঁচু তালিকা, ৪০px সারি, ৫০০টি সারি, `scrollTop` ১০০০, Overscan ৩।

৫. Web Worker কেন `document` ব্যবহার করতে পারে না? এক বাক্যে।

মধ্যম স্তর

৬. `PerformanceObserver` দিয়ে শুধু ২০ ms-এর বেশি দীর্ঘ `measure` Entry-র নাম সংগ্রহ করুন (Node.js), নির্দিষ্টভাবে দীর্ঘ একটি কাজ দিয়ে।

৭. Project-এর `topFunctions` দিয়ে একটি ছোট Program-এর CPU Profile বিশ্লেষণ করুন, যেখানে একটি Function অন্যটির চেয়ে অনেক বেশি কাজ করে।

৮. একটি Search Worker-এ "অনুরোধ বাতিল" যোগের নকশা করুন: নতুন প্রশ্ন এলে পুরোনোর ফল পাঠানোই হবে না।

৯. Event Listener-এর একটি Leak তৈরি ও সমাধান করুন: প্রতিবার নতুন Object যুক্ত করা Listener, এবং `AbortController` দিয়ে পরিষ্কার—Heap মেপে দেখান।

চ্যালেঞ্জ স্তর

১০. ভিন্ন উচ্চতার সারির জন্য `visibleRange`-এর একটি সংস্করণ লিখুন: প্রতিটি সারির উচ্চতার Array থেকে Prefix Sum, তারপর Binary Search।

১১. Project-এর Index-এ ফলাফলকে Rating দিয়ে সাজানোর খরচ কমান: প্রতিবার সাজানোর বদলে Index তৈরির সময় একবার সাজিয়ে রাখার নকশা, এবং ফল যে বদলায় না তার Test।

১২. Worker থেকে ফলের পুরো Object-এর বদলে শুধু ID পাঠানোর একটি সংস্করণের নকশা করুন: Main Thread কীভাবে Record পাবে, এবং কী লাভ-ক্ষতি।

২৩.১৩ অনুশীলনীর সমাধান

সমাধান ১

sort-timing.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = Array.from({ length: 20_000 }, (_, i) => (i * 7919) % 20_000);

performance.mark('sort-start');
const sorted = numbers.toSorted((a, b) => a - b);
performance.mark('sort-end');

const [entry] = performance.getEntriesByName(
  performance.measure('sort', 'sort-start', 'sort-end').name);
console.log(entry.name, entry.entryType, sorted[0], sorted.at(-1));
```

Output

```
sort measure 0 19999
```

সমাধান ২ — ধরুন `buildReport` নিজে একটি Loop চালায় (১০ ms) এবং `formatRow`-কে ডাকে, যা মোট ৯০ ms নেয়। `buildReport`-এর Self Time ১০ ms, Total Time ১০০ ms; `formatRow`-এর Self ও Total দুটিই ৯০ ms (যদি সে অন্য কিছু না ডাকে)। Optimize করতে হবে `formatRow`-কে—Total Time একা দেখলে মনে হতো `buildReport` দায়ী।

সমাধান ৩ — `transform` ও `opacity` (Browser যখন সেগুলো আলাদা Composite স্তরে রাখে)। `width` ও `top` Layout ট্রিগার করে; `color` Paint।

সমাধান ৪

range.mjs · JavaScript — চলবে: Node.js

```
import { visibleRange } from '../../project/travel-search/src/virtual.mjs';

const range = visibleRange({
  scrollTop: 1_000, viewport: 600, rowHeight: 40, total: 500, overscan: 3,
});
console.log(range, range.end - range.start);
```

Output

```
{ start: 22, end: 43 } 21
```

প্রথম দৃশ্যমান সারি ১০০০ ÷ ৪০ = ২৫, শেষ (১০০০ + ৬০০) ÷ ৪০ = ৪০; Overscan ৩ দুই পাশে: ২২ থেকে ৪৩ (শেষটি বাদ), মোট ২১টি।

সমাধান ৫ — DOM Main Thread-এর Object এবং একসঙ্গে দুটি Thread থেকে বদলানো নিরাপদ নয়, তাই Worker DOM পায় না; সে Message পাঠায়, এবং Main Thread DOM বদলায়।

সমাধান ৬

long-measures.mjs · JavaScript — চলবে: Node.js

```
import { PerformanceObserver, performance } from 'node:perf_hooks';

const slow = [];
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry.duration > 5) {
      slow.push(entry.name);
    }
  }
});
observer.observe({ entryTypes: ['measure'] });

function work(name, ms) {
  performance.mark(name);
  const end = Date.now() + ms;
  while (Date.now() < end) {
    // নির্দিষ্ট সময় ব্যস্ত থাকা
  }
  performance.measure(name, name);
}
```

```
work('quick', 0);
work('report', 40);
setTimeout(() => {
  observer.disconnect();
  console.log(slow);
}, 0);
```

Output

```
[ 'report' ]
```

`quick` প্রায় শূন্য সময়, `report` অন্তত ৪০ ms; ২০ ms-এর সীমা থেকে দুটিই দূরে, তাই ফল স্থির। বাস্তবে এই ধরনের Observer ধীর কাজ Monitoring-এ পাঠায়।

সমাধান ৭

profile-two.mjs · JavaScript — চলবে: Node.js

```
import { Session } from 'node:inspector/promises';
import { topFunctions } from '../../project/travel-search/src/profile-report.mjs';

function heavy() {
  let total = 0;
  for (let i = 0; i < 3e7; i += 1) {
    total += Math.sqrt(i);
  }
  return total;
}

function light() {
  return Array.from({ length: 1_000 }, (_, i) => i).reduce((a, b) => a + b, 0);
}

const session = new Session();
session.connect();
await session.post('Profiler.enable');
await session.post('Profiler.start');
heavy();
light();
const { profile } = await session.post('Profiler.stop');
const [top] = topFunctions(profile, 1);
console.log(top.name.startsWith('heavy'));
```

Output

```
true
```

`heavy` কয়েক কোটি বার Loop চালায়, `light` এক হাজার; Profile-এ সবচেয়ে বেশি Sample `heavy`-তে—এটি সময়ের সীমা নয়, কাজের পরিমাণের বিশাল পার্থক্য, তাই ফল স্থির। নিজের যন্ত্রে `top.percent` ছাপিয়ে দেখুন।

সমাধান ৮ — Worker-এ প্রতিটি অনুরোধের `requestId` রাখুন এবং একটি `latest` Variable: নতুন Message এলে `latest = requestId`। খোঁজ যদি টুকরোয় চলে (প্রতি কয়েক হাজার Record-এর পর `await` একটি Task), প্রতিটি টুকরোর পর পরীক্ষা করুন `requestId === latest`; না হলে কাজ থামিয়ে কিছুই পাঠাবেন না। Project-এর মতো খোঁজ যদি এক টানে শেষ হয়, তাহলে বাতিল সম্ভব নয়, তাই Main Thread-এ পুরোনো উত্তর বাদ দেওয়াই (Project-এর `requestId` যাচাই) যথেষ্ট। আরও বড় কাজে `worker.terminate()` করে নতুন Worker তৈরি করাও একটি উপায়, যদিও তখন Index আবার বানাতে হয়।

সমাধান ৯

listener-leak.mjs · JavaScript — চলবে: Node.js

```
import { setFlagsFromString } from 'node:v8';
import { runInNewContext } from 'node:vm';

setFlagsFromString('--expose-gc');
const gc = runInNewContext('gc');
const heapMB = () => {
  gc();
  return process.memoryUsage().heapUsed / 1024 / 1024;
};

const bus = new EventTarget();
function openPanel(signal) {
  const bigState = new Array(20_000).fill('x');
  bus.addEventListener('refresh', () => bigState.length, { signal });
}

const start = heapMB();
for (let i = 0; i < 200; i += 1) {
  openPanel();
}
const leaked = heapMB() - start;

const controllers = [];
for (let i = 0; i < 200; i += 1) {
  const controller = new AbortController();
  openPanel(controller.signal);
  controllers.push(controller);
}
const beforeAbort = heapMB();
controllers.forEach((c) => c.abort());
await new Promise((resolve) => setTimeout(resolve, 0));
const freed = beforeAbort - heapMB();
```

```
console.log(`Signal ছাড়া: ২০ MB-র বেশি আটকে আছে: ${leaked > 20}`);
console.log(`abort()-এর পর ২০ MB-র বেশি মুক্ত: ${freed > 20}`);
```

Output

```
Signal ছাড়া: ২০ MB-র বেশি আটকে আছে: true
abort()-এর পর ২০ MB-র বেশি মুক্ত: true
```

প্রতিটি Listener-এর Closure ২০,০০০ ঘরের একটি Array ধরে রাখে, এবং Listener `bus`-এ যুক্ত থাকায় সেগুলো কখনো মুক্ত হয় না—২০০ বার Panel খোলা মানে প্রায় ৩০ MB। `signal`-সহ যুক্ত Listener `abort()`-এ সরে যায়, Closure পৌঁছানোর অযোগ্য হয়, এবং GC Memory ফেরত নেয়। একটি বাস্তব পর্যবেক্ষণ: এই বইয়ের পরীক্ষায় (Node.js 22.16.0) `abort()`-এর ঠিক পরে মাপলে Memory তখনো মুক্ত হয়নি; একটি Event Loop চক্র (`setTimeout(..., 0)`) পরে মুক্ত হয়েছে, তাই Code-এ সেই অপেক্ষা। আরেকটি সূত্র: প্রথম Loop চালানোর সময় Node.js নিজেই Terminal-এ সতর্ক করে—`MaxListenersExceededWarning: Possible EventTarget memory leak detected`—একই Event-এ দশটির বেশি Listener যুক্ত হলে। এই সতর্কবার্তা উপেক্ষা করবেন না; এটি প্রায়ই সত্যিকারের Leak।

সমাধান ১০

variable-rows.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function createVariableList(heights) {
  const tops = [0];
  for (const h of heights) {
    tops.push(tops.at(-1) + h);
  }
  // যে সারির শুরু y-এর ঠিক আগে বা y-এ, তার Index (Binary Search)।
  const rowAt = (y) => {
    let low = 0;
    let high = heights.length - 1;
    while (low < high) {
      const middle = (low + high + 1) >>> 1;
      if (tops[middle] <= y) {
        low = middle;
      } else {
        high = middle - 1;
      }
    }
    return low;
  };
  return {
    totalHeight: tops.at(-1),
    range: (scrollTop, viewport) =>
      ({ start: rowAt(scrollTop), end: rowAt(scrollTop + viewport - 1) + 1 }),
    offsetOf: (index) => tops[index],
  };
```

```
}

const list = createVariableList([40, 80, 40, 120, 40, 40, 200, 40]);
const range = JSON.stringify(list.range(100, 150));
console.log(list.totalHeight, range, list.offsetOf(3));
```

Output

```
600 {"start":1,"end":4} 160
```

Prefix Sum (`tops`) একবার হিসাব হয় O(n)-এ, তারপর প্রতিটি Scroll-এ খোঁজা O(log n)। ১০০ থেকে ২৪৯ পিক্সেল দেখা যায়: সারি ১ (৪০–১১৯), ২ (১২০–১৫৯), ৩ (১৬০–২৭৯)। বাস্তবে উচ্চতা আগে জানা থাকে না; Library-গুলো আনুমানিক উচ্চতা দিয়ে শুরু করে এবং আঁকার পর মেপে সংশোধন করে।

সমাধান ১১ — Index তৈরির সময় `rating`-এ সাজানো একটি Record-এর তালিকা ও প্রতিটি Record-এর অবস্থান (`rankById`) রাখুন। খোঁজার পর ফলকে তুলনা-ভিত্তিক `sort`-এর বদলে `rankById` দিয়ে সাজালে তুলনাগুলো সস্তা হয়; আর ফল যদি পুরো তালিকার বড় অংশ হয়, সাজানো তালিকা থেকে `filter` করাই O(n)। Test: Project-এর Property Test-টি `sort: 'rating'` দিয়ে চালান—নতুন সংস্করণ ও `naiveSearch` হুবহু সমান হতে হবে, বিশেষ করে সমান Rating-এ `id`-এর ক্রম। প্রথমে মাপুন: ১২,০০০ Record-এ এই Optimization-এর প্রয়োজন আছে কি না, Profile বলবে।

সমাধান ১২ — Worker শুধু ID-র একটি `Int32Array` পাঠায় এবং তার `buffer` Transfer করে—Clone-এর খরচ প্রায় শূন্য। Main Thread-এর কাছে Record থাকতে হবে: হয় শুরুতে একবার Worker থেকে সব Record পাঠানো (একবারের খরচ), নয়তো দৃশ্যমান সারির Record আলাদা Message-এ চাওয়া। লাভ: প্রতিটি খোঁজে ছোট Message। ক্ষতি: Main Thread-এ তথ্যের দ্বিতীয় Copy (Memory), অথবা Scroll-এর সময় অতিরিক্ত Message ও অপেক্ষা। ১২,০০০ ছোট Record-এ Project-এর সরল উপায় যথেষ্ট; লাখো Record-এ ID পাঠানোর নকশা যুক্তিসংগত—সিদ্ধান্ত মেপে নিন।

২৩.১৪ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `performance.measure('search', 'a', 'b')` কী করে?

ক) একটি নামযুক্ত Entry তৈরি করে, যাতে `a` থেকে `b` চিহ্নের মধ্যের সময়

খ) Function দ্রুত করে

গ) CPU Profile শুরু করে

ঘ) `a` ও `b` Function দুটি চালায়

প্রশ্ন ২. CPU Profile-এ "Self Time" কী?

ক) Program-এর মোট সময়

খ) Function নিজের Code-এ কাটানো সময়, ডাকা Function বাদে

গ) GC-র সময়

ঘ) Network-এর সময়

প্রশ্ন ৩. অধ্যায়ের Profile-এ সরল খোঁজের প্রধান খরচ কোথায় ছিল?

ক) `sort`-এ

খ) Network-এ

গ) প্রতিটি প্রশ্নে সব Record-এর লেখা আবার শব্দে ভাগ করায়

ঘ) DOM-এ

প্রশ্ন ৪. Memory Leak প্রমাণের সময় Heap মাপার আগে কী করা উচিত?

ক) Program বন্ধ

খ) Browser Refresh

গ) কিছুই না

ঘ) Garbage Collection চালানো বা Heap Snapshot নেওয়া

প্রশ্ন ৫. কোন CSS পরিবর্তন সাধারণত Layout ট্রিগার করে না?

ক) `transform`

খ) `height`

গ) `width`

ঘ) `top`

প্রশ্ন ৬. Virtual Scrolling-এ Spacer Element-এর কাজ কী?

ক) সারি আঁকা

খ) সব সারির মোট উচ্চতা ধরে রেখে Scrollbar সঠিক রাখা

গ) Worker চালানো

ঘ) খোঁজ করা

প্রশ্ন ৭. Web Worker কী করতে পারে না?

ক) `postMessage`

খ) Module `import`

গ) সরাসরি DOM বদলানো

ঘ) ভারী হিসাব

প্রশ্ন ৮. `ArrayBuffer` Transfer করার পর মূল Buffer-এর `byteLength`?

ক) একই থাকে

খ) দ্বিগুণ

গ) Error

ঘ) ০

প্রশ্ন ৯. Async খোঁজে `requestId` কেন?

ক) পুরোনো ধীর উত্তর নতুন উত্তরকে মুছে না দিতে

খ) খোঁজ দ্রুত করতে

গ) Memory কমাতে

ঘ) Worker বন্ধ করতে

প্রশ্ন ১০. Index-এর মতো Optimization-এর পর প্রথম যাচাই কী?

ক) Coverage ১০০%

খ) ফল সরল সংস্করণের সঙ্গে হুবহু মেলে

গ) Code ছোট

ঘ) Benchmark-এ ১০ গুণ দ্রুত

২৩.১৫ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–ক · ২–খ · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	ক	নামযুক্ত পরিমাপ, যা DevTools ও Observer দেখতে পায়।
২	খ	কোন লাইন ব্যয়বহুল, তা Self Time বলে।
৩	গ	`words` প্রায় দুই-তৃতীয়াংশ সময়; সমাধান Index।
৪	ঘ	নইলে অমুক্ত-কিন্তু-মৃত Object-ও গোনা হয়।
৫	ক	Composite ধাপেই সরানো যায়।
৬	খ	DOM-এ সারি কম, কিন্তু Scroll-এর আকার পূর্ণ তালিকার।
৭	গ	Worker-এ `document` নেই; Message দিয়ে Main Thread-কে জানায়।
৮	ঘ	মালিকানা হস্তান্তর; মূলটি Detached।
৯	ক	Race Condition প্রতিরোধ।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	খ	দ্রুত কিন্তু ভুল ফল অর্থহীন।

২৩.১৬ অধ্যায়ের সারসংক্ষেপ

- Profile আগে: User Timing বলে কতক্ষণ, CPU Profile বলে কোথায়; Self ও Total Time দুটিই দেখুন।
- `performance.mark` / `measure` নামযুক্ত পরিমাপ; `PerformanceObserver` সেগুলো আলাদা জায়গায় সংগ্রহ করে।
- Event Loop Delay-এর Histogram Server-এর আটকে থাকা ধরে; Memory মাপুন GC-র পরে।
- Rendering: JavaScript, Style, Layout, Paint, Composite; `transform` ও `opacity` সস্তা।
- Virtual Scrolling: Spacer, বিশুদ্ধ হিসাব, `translateY`; সীমাবদ্ধতা জেনে ব্যবহার।
- Web Worker Main Thread মুক্ত রাখে; Message Clone হয়, `ArrayBuffer` Transfer করা যায়।
- Async খোঁজে `requestId` দিয়ে Race Condition রোধ।
- প্রতিটি Optimization-এর আগে-পরে সঠিকতার Test; সময় কখনো Test-এর শর্ত নয়।

পরের অধ্যায়ে JavaScript থেকে TypeScript: Static Type কীভাবে অনেক ত্রুটি Code চালানোর আগেই ধরে, এবং কোথায় তার সীমা।

২৩.১৭ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
User Timing	`mark` ও `measure` দিয়ে নামযুক্ত পরিমাপ।
CPU Profile	Sampling দিয়ে কোন Function কত সময় চলেছে তার রেকর্ড।
Self Time	Function নিজের Code-এ কাটানো সময়।
Total Time	ডাকা Function-সহ মোট সময়।
Event Loop Delay	Event Loop-এর একটি চক্র কত দেরিতে চলেছে।
Heap Snapshot	একটি মুহূর্তে Memory-র সব Object-এর চিত্র।
Layout	Element-এর আকার ও অবস্থানের হিসাব।
Composite	আঁকা স্তরগুলো জোড়া লাগানো।
Virtual Scrolling	শুধু দৃশ্যমান সারি আঁকা।
Overscan	দৃশ্যমান অংশের বাইরে আঁকা বাড়তি সারি।

পরিভাষা	অর্থ
Web Worker	আলাদা Thread-এ চলা Script।
Transferable	Copy ছাড়াই মালিকানা হস্তান্তরযোগ্য Object।
Inverted Index	শব্দ থেকে সেই শব্দযুক্ত Record-এর তালিকা।

২৩.১৮ আরও পড়ুন

- **MDN — User Timing:** `https://developer.mozilla.org/en-US/docs/Web/API/Performance_API/User_timing` — `mark` ও `measure`।
- **Node.js — Performance measurement APIs:** `https://nodejs.org/api/perf_hooks.html` — `PerformanceObserver`, `monitorEventLoopDelay`।
- **web.dev — Rendering performance:** `https://web.dev/articles/rendering-performance` — Pipeline ও `transform`।
- **MDN — Using Web Workers:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers` — Worker ও Transferable।
- **Chrome DevTools — Fix memory problems:** `https://developer.chrome.com/docs/devtools/memory-problems` — Heap Snapshot।

অধ্যায় ২৪

২৪

JavaScript থেকে TypeScript: Static Typing, Type Safety এবং Professional Development Workflow

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- TypeScript কী নিশ্চিত করে এবং কী করে না—Compile-time Check ও Runtime Validation-এর পার্থক্য—স্পষ্টভাবে বলতে।
- `tsconfig.json` -এর Strict Setting দিয়ে একটি Project তৈরি করে `tsc` দিয়ে Type Check ও Build করতে।
- Type Inference, Literal Type, Union, `type` ও `interface` ব্যবহার করতে।
- `strictNullChecks` -এর অধীনে `undefined` ও `null` নিরাপদে সামলাতে।
- `unknown` দিয়ে Trust Boundary-তে তথ্য গ্রহণ করে Type Guard দিয়ে যাচাই করতে, এবং `any` ও `as` -এর ঝুঁকি চিনতে।
- Discriminated Union, `Result` Type ও `never` দিয়ে Exhaustive Check লিখতে।
- Generic Function ও Generic Class, এবং `keyof` , `Pick` , `Omit` , `Partial` , `Record` -এর মতো Utility Type ব্যবহার করতে।
- Branded Type দিয়ে একই ধরনের ভিন্ন অর্থের মান (টাকা বনাম সংখ্যা) আলাদা রাখতে।
- Compile-time Test (`@ts-expect-error`) ও Runtime Test দুটিই লিখতে।
- একটি সম্পূর্ণ **TypeSafe Claims** Service তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ৪–৭: Function, Object, Array, Destructuring।
- অধ্যায় ১১ ও ১৭: Validation, Trust Boundary।
- অধ্যায় ১৮: Result Type, Immutable হালনাগাদ।
- অধ্যায় ২১: State Machine, Repository, Dependency Injection।
- অধ্যায় ২২: `node:test`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

JavaScript-এ একটি Function যেকোনো মান গ্রহণ করে। `totalPrice(1500, '2')` কোনো আপত্তি ছাড়াই চলে, এবং ভুল ধরা পড়ে অনেক পরে—হয়তো গ্রাহকের Invoice-এ। দলে Code বড় হলে প্রশ্নগুলো জমতে থাকে: এই Function কী ফেরত দেয়? এই Field কি কখনো `undefined` হতে পারে? এই Status-এর তালিকায় নতুন একটি মান যোগ করলে কোন কোন `switch` হালনাগাদ করতে হবে?

TypeScript JavaScript-এর ওপর একটি Type-এর স্তর যোগ করে, এবং এই প্রশ্নগুলোর উত্তর Code চালানোর আগেই, Compiler-এর কাছ থেকে দেয়। আজ বেশিরভাগ বড় JavaScript Project—Framework, Library, Backend—TypeScript-এ লেখা। কিন্তু একটি ভুল ধারণাও ব্যাপক: "TypeScript থাকলে ইনপুট নিরাপদ"। তা নয়। TypeScript-এর সব Type Compile-এর পর মুছে যায়; Browser বা Server-এ যা চলে তা সাধারণ JavaScript। এই অধ্যায়ের কেন্দ্রীয় শিক্ষা তাই দুটি: Type দিয়ে আপনার নিজের Code-এর ভুল ধরুন, এবং বাইরে থেকে আসা তথ্য Runtime-এ যাচাই করুন।

২৪.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`typeof`, Runtime যাচাই (অধ্যায় ১১, ১৭)	Type Guard: যাচাই, যা Compiler-কেও Type জানায়
Result Object (অধ্যায় ১৮)	Discriminated Union হিসেবে `Result<T, E>`
টেবিলভিত্তিক State Machine (অধ্যায় ২১)	`Record<Status, ...>` ও `never` দিয়ে সম্পূর্ণতার নিশ্চয়তা
Repository-র চুক্তি (অধ্যায় ২১)	Generic `Repository<T, K>` Interface
JSDoc মন্তব্যে Type বর্ণনা	Compiler-যাচাই করা Type
—	`tsconfig.json`, Branded Type, Compile-time Test

২৪.২ Dynamic Typing-এর একটি চেনা ভুল, এবং TypeScript-এর উত্তর

Form থেকে আসা প্রতিটি মান লেখা (String)। নিচের JavaScript নীরবে ভুল করে:

dynamic.js · JavaScript চলবে: Node.js

```javascript
function totalPrice(price, quantity) {
  return price * quantity;
}

function addFee(total, fee) {
  return total + fee;
}

const quantityFromForm = '2';
const total = totalPrice(1500, quantityFromForm);
console.log(total, addFee(total, '50'));
```

Output

```
3000 300050
```

গুণ লেখাকে সংখ্যায় রূপান্তর করে "ঠিক" ফল দিয়েছে, কিন্তু যোগ লেখা জোড়া লাগিয়েছে: ৩,০৫০-এর বদলে ৩,০০,০৫০। একই Code TypeScript-এ, প্রতিটি Parameter-এর Type লিখে:

typed.ts · TypeScript চলবে: Node.js

```typescript
function totalPrice(price: number, quantity: number): number {
  return price * quantity;
}

const quantityFromForm = '2';
console.log(totalPrice(1500, quantityFromForm));
```

`tsc --strict` চালালে:

Error Message

```
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

ভুলটি Program চালানোর আগেই ধরা পড়েছে, এবং ঠিক কোন লাইনে তা বলা হয়েছে। সমাধানও স্পষ্ট হয়ে যায়: Form-এর লেখাকে সংখ্যায় রূপান্তর ও যাচাই করা— `Number(quantityFromForm)`, এবং `Number.isInteger` দিয়ে পরীক্ষা।

TypeScript কাজ করে এভাবে: আপনি `.ts` File লেখেন, Compiler (`tsc`) Type যাচাই করে, তারপর Type মুছে সাধারণ JavaScript তৈরি করে (**Type Erasure**)। Node.js ২২.৬ থেকে `--experimental-strip-types` দিয়ে `.ts` File সরাসরি চালানো যায়, কিন্তু সেটি শুধু Type **মুছে ফেলে**, যাচাই করে না—Type Check-এর জন্য `tsc` লাগবেই।

২৪.৩ Project-এর কাঠামো এবং `tsconfig.json`

একটি TypeScript Project-এ সাধারণত TypeScript নিজে (এবং Node.js-এর Type-এর জন্য `@types/node`) Development Dependency হিসেবে থাকে:

Terminal

```
npm install --save-dev typescript @types/node
npx tsc --init
```

> **নোট:** এই বইয়ের QA-তে কোনো Package Install করা হয়নি। এই কম্পিউটারে আগে থেকে Install করা একটি Global Tool-এর ভেতরে TypeScript 6.0.2 ও `@types/node` 25.6.0 পাওয়া গেছে; এই অধ্যায়ের সব Compile ও Test সেগুলো দিয়ে চালানো। আপনার Project-এ উপরের Command দিয়ে নিজস্ব Copy Install করুন।

Project-এর `tsconfig.json`-এ গুরুত্বপূর্ণ Setting:

tsconfig.json · JSON — চলবে: Node.js

```json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": false,
    "verbatimModuleSyntax": true,
    "types": ["node"],
    "rootDir": ".",
    "outDir": "dist",
    "sourceMap": true
  },
  "include": ["src", "tests"]
}
```

Setting	কাজ
`strict`	সব কঠোর Check একসঙ্গে, এর মধ্যে `strictNullChecks` ও `noImplicitAny`; নতুন Project-এ সব সময় চালু
`noUncheckedIndexedAccess`	`array[i]`-এর Type `T` অথবা `undefined`, কারণ Index-এ মান না-ও থাকতে পারে
`exactOptionalPropertyTypes`	ঐচ্ছিক Field-এ `undefined` স্পষ্টভাবে লেখা আর অনুপস্থিত থাকা আলাদা
`module` ও `moduleResolution`: `NodeNext`	Node.js-এর আসল ES Module নিয়ম; তাই Import-এ `.js` লেখা হয়
`verbatimModuleSyntax`	শুধু Type-এর Import (`import type`) Output থেকে মুছে যায়, বাকিগুলো হুবহু থাকে
`outDir`	Compile করা JavaScript `dist/`-এ

Import-এ `.js` প্রথমে অদ্ভুত লাগে (`import { toPaisa } from './money.js'`, অথচ File-টি `money.ts`), কিন্তু যুক্তিটি সরল: Node.js চালাবে Compile-এর পরের `money.js`, এবং TypeScript Import-এর পথ বদলায় না।

২৪.৪ Type Inference, Literal Type এবং Union

প্রতিটি Variable-এ Type লিখতে হয় না; TypeScript মান দেখে **অনুমান** (Infer) করে। Type লিখুন সীমানায়—Function-এর Parameter ও ফেরত মান, এবং Export করা জিনিসে।

inference.ts · TypeScript — চলবে: Node.js

```ts
let seats = 40;
const mode = 'bus';

type Mode = 'bus' | 'train' | 'air';
function fare(travelMode: Mode, km: number): number {
  const perKm = { bus: 2, train: 1.5, air: 9 }[travelMode];
  return Math.round(perKm * km);
}

seats = 'forty';
console.log(fare(mode, 220), fare('ship', 100));
```

Error Message

```
error TS2322: Type 'string' is not assignable to type 'number'.
error TS2345: Argument of type '"ship"' is not assignable to parameter
of type 'Mode'.
```

দুটি পর্যবেক্ষণ। `let seats = 40`-এর Type অনুমিত `number`, তাই পরে লেখা বসানো ভুল। আর `const mode = 'bus'`-এর Type শুধু `string` নয়, **Literal Type** `'bus'` — `const` কখনো বদলায় না—তাই এটি `Mode`-এ খাপ খায়। `Mode` একটি **Union**: তিনটির যেকোনো একটি, এর বাইরে কিছু নয়। Status, Mode, Category-র মতো সীমিত মানের তালিকার জন্য Union-ই সবচেয়ে নিরাপদ—বানান ভুল (`'trian'`) Compile-এই ধরা পড়ে।

`type` বনাম `interface`: দুটিই Object-এর আকার বর্ণনা করে। `interface` প্রসারিত (`extends`) ও পরে আবার ঘোষণা করে মেলানো যায়; `type` Union, Tuple ও জটিল রূপান্তরের জন্য। একটি সহজ নিয়ম: Object-এর চুক্তির জন্য `interface`, বাকি সবকিছুর জন্য `type`—এবং একটি Project-এ ধারাবাহিক থাকুন।

২৪.৫ null, undefined এবং Strict Mode

`strict`-এর সবচেয়ে মূল্যবান অংশ `strictNullChecks`: যে মান `undefined` হতে পারে, তাকে যাচাই না করে ব্যবহার করা যায় না। JavaScript-এর সবচেয়ে সাধারণ Runtime Error—"Cannot read properties of undefined"—এর বড় অংশ এভাবেই Compile-এ ধরা পড়ে।

nulls.ts · TypeScript — চলবে: Node.js

```ts
interface Traveler {
  name: string;
  passport?: string;
}

const travelers: Traveler[] = [{ name: 'Rafi', passport: 'BX1234567' }];
const found = travelers.find((t) => t.name === 'Mina');
console.log(found.name.toUpperCase());
console.log(travelers[0].passport.slice(0, 2));
```

Error Message

```
error TS18048: 'found' is possibly 'undefined'.
error TS2532: Object is possibly 'undefined'.
```

`find` কিছু না পেলে `undefined` দেয়, আর `passport?` ঐচ্ছিক। সমাধান—যাচাই, Optional Chaining বা Default মান:

nulls-fixed.ts · TypeScript চলবে: Node.js

```ts
interface Traveler {
  name: string;
  passport?: string;
}

const travelers: Traveler[] = [{ name: 'Rafi', passport: 'BX1234567' }];
const found = travelers.find((t) => t.name === 'Mina');
if (found) {
  console.log(found.name.toUpperCase());
}
console.log(found?.name ?? 'Mina নামে কেউ নেই');
console.log(travelers[0]?.passport?.slice(0, 2) ?? 'Passport নেই');
```

Output

```
Mina নামে কেউ নেই
BX
```

`if (found)`-এর ভেতরে TypeScript জানে `found` আর `undefined` নয়—একে বলে **Narrowing**। Non-null Assertion (`found!.name`) Compiler-কে চুপ করায়, কিন্তু Runtime-এ কিছুই যাচাই করে না; তাই এটি প্রায় সব সময় এড়িয়ে চলুন।

২৪.৬ unknown বনাম any: Trust Boundary

বাইরের তথ্য— `JSON.parse`-এর ফল, `fetch`-এর Body, Form, File—Compile-এর সময় অজানা। TypeScript দুটি Type দেয়, যার আচরণ বিপরীত:

- **`any`**: Type Check বন্ধ। যেকোনো কিছু করা যায়, এবং ভুল Runtime-এ পর্যন্ত লুকিয়ে থাকে।
- **`unknown`**: "জানি না"। যাচাই (Narrowing) না করে কিছুই করা যায় না।

any.ts · TypeScript চলবে: Node.js

```ts
const text = '{ "name": 42 }';
const data: any = JSON.parse(text);

try {
  console.log(data.name.toUpperCase());
} catch (error) {
  console.log(`Runtime-এ ধরা পড়ল: ${(error as Error).message}`);
}
```

Output

```
Runtime-এ ধরা পড়ল: data.name.toUpperCase is not a function
```

`any`-এর সঙ্গে Compiler কোনো আপত্তি করেনি; ভুলটি ধরা পড়েছে চলার সময়। একই কাজ `unknown`-এ:

unknown.ts · TypeScript চলবে: Node.js

```ts
const data: unknown = JSON.parse('{ "name": 42 }');
console.log(data.name.toUpperCase());
```

Error Message

```
error TS18046: 'data' is of type 'unknown'.
```

এখন Compiler যাচাই দাবি করছে। যাচাইয়ের সেরা হাতিয়ার **Type Guard**—একটি Function, যার ফেরত Type `value is T`: Runtime-এ পরীক্ষা করে, এবং `true` হলে Compiler-কেও Type জানায়:

guard.ts · TypeScript চলবে: Node.js

```ts
interface Customer {
  name: string;
  phone: string;
}

function isCustomer(value: unknown): value is Customer {
  if (typeof value !== 'object' || value === null) {
    return false;
  }
  const record = value as Record<string, unknown>;
  return typeof record.name === 'string' && typeof record.phone === 'string';
}

for (const text of ['{ "name": "Rafi", "phone": "017" }', '{ "name": 42 }']) {
  const data: unknown = JSON.parse(text);
  console.log(isCustomer(data) ? data.name.toUpperCase() : 'অবৈধ তথ্য');
}
```

Output

```
RAFI
অবৈধ তথ্য
```

`as` (Type Assertion) সবচেয়ে বিপজ্জনক: `JSON.parse(text) as Customer` Compiler-কে বলে "বিশ্বাস করো", এবং Compiler বিশ্বাস করে—কোনো যাচাই ছাড়াই। অধ্যায় ২০-এর নিয়ম এখানেও: **TypeScript Runtime-এ ইনপুটের নিরাপত্তা দেয় না।** Server-এ পৌঁছানো প্রতিটি তথ্য Runtime-এ যাচাই করতে হবে; TypeScript শুধু নিশ্চিত করে

যে যাচাইয়ের পর আপনার Code সঠিক Type ব্যবহার করছে। বড় Project-এ এই যাচাইয়ের জন্য Zod বা Valibot-এর মতো Library একবার লেখা Schema থেকে Validation ও Type দুটিই তৈরি করে।

২৪.৭ Discriminated Union, `Result` এবং Exhaustiveness

একটি Union-এর প্রতিটি সদস্যে একটি সাধারণ Literal Field (Discriminant, যেমন `kind` বা `ok`) থাকলে, সেই Field দেখে TypeScript বাকি Field জানে। Error-এর জন্য Exception-এর বদলে একটি `Result` Type (অধ্যায় ১৮) এখন Compiler-যাচাই করা:

result.ts · TypeScript — চলবে: Node.js

```ts
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };

type FareError =
  | { code: 'UNKNOWN_ROUTE'; route: string }
  | { code: 'SOLD_OUT'; date: string };

function quoteFare(route: string): Result<number, FareError> {
  if (route === 'DAC-CXB') {
    return { ok: true, value: 4_500 };
  }
  return { ok: false, error: { code: 'UNKNOWN_ROUTE', route } };
}

function message(error: FareError): string {
  switch (error.code) {
    case 'UNKNOWN_ROUTE':
      return `${error.route} পথ নেই`;
    case 'SOLD_OUT':
      return `${error.date}-এ আসন নেই`;
  }
}

for (const route of ['DAC-CXB', 'DAC-XYZ']) {
  const result = quoteFare(route);
  console.log(result.ok ? `৳${result.value}` : message(result.error));
}
```

Output

```
৳4500
DAC-XYZ পথ নেই
```

`result.ok` পরীক্ষার পর TypeScript জানে কোন শাখায় `value` আর কোন শাখায় `error` আছে; ভুল Field পড়লে Compile Error। `message`-এর `switch`-এ প্রতিটি `code`-এর ভিন্ন Field (`route`, `date`) নিরাপদে পড়া গেছে।

এখন একটি নতুন Error যোগ করা হলো (`PRICE_CHANGED`), কিন্তু কেউ `message` হালনাগাদ করতে ভুলে গেলেন। এই ভুল ধরতে `default` -এ একটি `never` Check:

exhaustive.ts · TypeScript চলবে: Node.js

```ts
type FareError =
  | { code: 'UNKNOWN_ROUTE'; route: string }
  | { code: 'SOLD_OUT'; date: string }
  | { code: 'PRICE_CHANGED'; newPrice: number };

function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত: ${JSON.stringify(value)}`);
}

function message(error: FareError): string {
  switch (error.code) {
    case 'UNKNOWN_ROUTE':
      return `${error.route} পথ নেই`;
    case 'SOLD_OUT':
      return `${error.date}-এ আসন নেই`;
    default:
      return assertNever(error);
  }
}

console.log(message({ code: 'SOLD_OUT', date: '2026-12-16' }));
```

Error Message

```
error TS2345: Argument of type '{ code: "PRICE_CHANGED"; newPrice: number; }'
is not assignable to parameter of type 'never'.
```

সব `case` সামলানো থাকলে `default` -এ `error` -এর Type `never` (আর কোনো সম্ভাবনা নেই), এবং `assertNever` সন্তুষ্ট। একটি `case` বাদ পড়লে বাকি সদস্যটি `never` -এ বসানো যায় না—Compiler ঠিক বলে দেয় কোনটি বাদ পড়েছে। নতুন Status বা Error যোগ করার পর এটিই "কোথায় কোথায় হালনাগাদ করতে হবে"-র তালিকা।

২৪.৮ Generics: একবার লেখা, অনেক Type-এ

Generic Function একটি **Type Parameter** নেয়, যাতে একই Code বিভিন্ন Type-এ কাজ করে, এবং Type-এর তথ্য হারায় না:

generics.ts · TypeScript চলবে: Node.js

```ts
function groupBy<T, K extends PropertyKey>(
  items: readonly T[], key: (item: T) => K,
) {
```

```
  const groups = {} as Record<K, T[]>;
  for (const item of items) {
    (groups[key(item)] ??= []).push(item);
  }
  return groups;
}

const tours = [
  { title: 'Sundarbans', days: 3, region: 'Khulna' },
  { title: 'Ratargul', days: 1, region: 'Sylhet' },
  { title: 'Jaflong', days: 1, region: 'Sylhet' },
];

const byRegion = groupBy(tours, (tour) => tour.region);
const byLength = groupBy(tours, (tour) => (tour.days > 1 ? 'long' : 'short'));
console.log(Object.keys(byRegion), byLength.short?.map((t) => t.title));
```

Output

```
[ 'Khulna', 'Sylhet' ] [ 'Ratargul', 'Jaflong' ]
```

`T` অনুমিত হয়েছে Tour-এর আকার থেকে, `K` হয়েছে `string` এবং দ্বিতীয় ডাকে `'long' | 'short'`। `K extends PropertyKey` একটি **Constraint**: Key হতে পারে শুধু String, Number বা Symbol। Generic Class-এর উদাহরণ Project-এর `MemoryRepository<T, K>`: যেকোনো Type-এর Repository, যার একটি `id` আছে।

২৪.৯ `keyof` এবং Utility Type

বিদ্যমান Type থেকে নতুন Type তৈরি করা যায়, ফলে একই আকার দুবার লিখতে হয় না, এবং মূলটি বদলালে সব আপনাআপনি বদলায়।

Utility	ফল	ব্যবহার
`keyof T`	T-এর সব Key-এর Union	নিরাপদ Field-এর নাম
`Pick<T, K>`	শুধু কয়েকটি Field	নতুন তৈরির ইনপুট
`Omit<T, K>`	কয়েকটি বাদে সব	সারাংশ, Public রূপ
`Partial<T>`	সব Field ঐচ্ছিক	আংশিক হালনাগাদ
`Required<T>`	সব Field বাধ্যতামূলক	Default পূরণের পর
`Readonly<T>`	সব Field অপরিবর্তনীয়	Immutable তথ্য
`Record<K, V>`	প্রতিটি K-এর জন্য একটি V	গণনা, Lookup টেবিল
`ReturnType<F>`	Function-এর ফেরত Type	Factory-র Type

keyof.ts · TypeScript — চলবে: Node.js

```ts
interface Tour {
  title: string;
  price: number;
  days: number;
}

function sortBy<T>(items: readonly T[], key: keyof T): T[] {
  const compare = (a: T, b: T) =>
    (a[key] < b[key] ? -1 : a[key] > b[key] ? 1 : 0);
  return [...items].sort(compare);
}

function update<T>(item: T, changes: Partial<T>): T {
  return { ...item, ...changes };
}

const tours: Tour[] = [
  { title: 'Sajek', price: 6_500, days: 2 },
  { title: 'Kuakata', price: 4_000, days: 2 },
];
console.log(sortBy(tours, 'price').map((t) => t.title));
console.log(update(tours[0]!, { price: 7_000 }));
sortBy(tours, 'prize');
```

Error Message

```
error TS2345: Argument of type '"prize"' is not assignable to parameter
of type 'keyof Tour'.
```

শেষ লাইনের বানান ভুল (`prize`) Compile-এ ধরা পড়েছে, কারণ `key`-এর Type `keyof Tour` = `'title' | 'price' | 'days'`। (ভুল লাইনটি সরালে বাকি Code চলে এবং দাম অনুযায়ী `['Kuakata', 'Sajek']` ছাপে।) Project-এ `NewClaim = Pick<Claim, 'employee' | 'items'>`, `ClaimSummary = Omit<Claim, 'items'> & { itemCount: number }` এবং `StatusCounts = Record<ClaimStatus, number>`।

২৪.১০ Branded Type: একই Type, ভিন্ন অর্থ

টাকার পরিমাণ (পয়সায়), রাতের সংখ্যা, মানুষের সংখ্যা—সবই `number`। কেউ ভুল করে পয়সার জায়গায় টাকা বা রাতের সংখ্যা পাঠালে সাধারণ Type ধরতে পারে না। **Branded Type** একটি অদৃশ্য চিহ্ন যোগ করে: Compile-এর সময় `Paisa` আর সাধারণ `number` আলাদা, Runtime-এ দুটিই সাধারণ সংখ্যা।

brand.ts · TypeScript — চলবে: Node.js

```ts
declare const brand: unique symbol;
type Paisa = number & { readonly [brand]: 'Paisa' };
```

```
function toPaisa(value: number): Paisa {
  if (!Number.isSafeInteger(value) || value < 0) {
    throw new RangeError(`অবৈধ পরিমাণ: ${value}`);
  }
  return value as Paisa;
}

function addFee(amount: Paisa, fee: Paisa): Paisa {
  return toPaisa(amount + fee);
}

const nights = 3;
console.log(addFee(toPaisa(120_000), toPaisa(5_000)));
console.log(addFee(toPaisa(120_000), nights));
```

Error Message

```
error TS2345: Argument of type 'number' is not assignable to parameter
of type 'Paisa'.
```

`Paisa` তৈরির একমাত্র দরজা `toPaisa`, যা Runtime-এ যাচাই করে—তাই একটি `Paisa` মান সব সময় যাচাই করা। এখানে `as`-এর ব্যবহার বৈধ, কারণ তার ঠিক আগের লাইনগুলোই প্রমাণ দিয়েছে। Project-এ `Paisa` ও `ClaimId` দুটিই Branded।

২৪.১১ পেশাদার Workflow: JavaScript থেকে ধাপে ধাপে TypeScript

বাস্তবে নতুন Project-এর চেয়ে বেশি দেখা যায় একটি চালু JavaScript Project, যাকে TypeScript-এ আনতে হবে। পুরোটা একবারে নতুন করে লেখা (Big-bang Rewrite) ঝুঁকিপূর্ণ: মাসের পর মাস দুটি সংস্করণ, এবং নতুন ভুল। নিরাপদ পথটি ধাপে ধাপে, প্রতিটি ধাপের শেষে Project চালু ও পরীক্ষিত।

প্রথম ধাপ: JavaScript-এই Type Check। একটি `.js` File-এর শুরুতে `// @ts-check` লিখলে, এবং JSDoc মন্তব্যে Type বর্ণনা করলে, TypeScript সেই JavaScript-ই যাচাই করে—কোনো File-এর নাম বা Build না বদলে:

discount.js · JavaScript চলবে: Node.js

```
// @ts-check

/**
 * @param {number} price
 * @param {number} percent
 * @returns {number}
 */
export function discount(price, percent) {
  return Math.round(price * (1 - percent / 100));
}

/** @type {{ title: string, days: number }} */
const tour = { title: 'Sajek', days: '2' };
console.log(discount('500', 10), tour);
```

`tsc --allowJs --checkJs --noEmit` চালালে:

Error Message

```
error TS2322: Type 'string' is not assignable to type 'number'.
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

এই File এখনো সাধারণ JavaScript, Browser বা Node.js-এ যেমন ছিল তেমনই চলে; কিন্তু দুটি ভুল— `days` -এ লেখা, আর `discount` -এ লেখা পাঠানো—এখন Editor ও `tsc` দুটিই দেখায়। অনেক দল এই ধাপেই থেকে যায়: Build Step ছাড়াই Type-এর বেশিরভাগ সুবিধা।

দ্বিতীয় ধাপ: File ধরে ধরে `.ts` । `tsconfig.json` -এ `allowJs: true` দিলে `.js` ও `.ts` একই Project-এ মিশে থাকতে পারে। রূপান্তর শুরু করুন "পাতা" থেকে—যে File অন্য কিছু Import করে না (যেমন `money.js`)—তারপর ধীরে ধীরে মূল Logic-এর দিকে। প্রতিটি File রূপান্তরের পর Test চালান।

তৃতীয় ধাপ: কঠোরতা বাড়ানো। পুরোনো Code-এ `strict` একবারে চালু করলে শত শত Error আসতে পারে। একটি একটি করে চালু করুন (`noImplicitAny` , তারপর `strictNullChecks`), এবং যে ভুল এখনই ঠিক করা যাচ্ছে না, সেখানে `any` নয়, বরং `// @ts-expect-error` এবং পাশে কারণ ও Issue-এর নম্বর লিখুন—পরে ভুলটি ঠিক হলে Compiler নিজেই মনে করিয়ে দেবে (`TS2578`)।

বাইরের Library-র Type। বেশিরভাগ আধুনিক npm Package নিজের Type (`.d.ts` File) সঙ্গে দেয়। পুরোনো JavaScript Package-এর Type প্রায়ই আলাদা `@types/...` Package-এ থাকে (যেমন এই অধ্যায়ের `@types/node`)। কোনোটিই না থাকলে নিজের Project-এ একটি ছোট `declare module 'name'` ঘোষণা লিখে শুধু ব্যবহৃত অংশের Type দিন। `.d.ts` File-এ শুধু Type থাকে, কোনো চলমান Code নয়।

দৈনন্দিন Workflow:

কোথায়	কী চলে	কেন
Editor	প্রতিটি টাইপে Type Check	তাৎক্ষণিক প্রতিক্রিয়া
Terminal	`tsc --noEmit --watch`	পুরো Project-এর অবস্থা
CI	`tsc --noEmit`, তারপর Test	Type ভাঙা Code Merge হয় না
Build	`tsc` (বা Bundler) দিয়ে `dist/`	Deploy বা Publish
Code Review	`any`, `as`, `!`, `@ts-expect-error`-এ প্রশ্ন	ফাঁকফোকর নিয়ন্ত্রণ

একটি গুরুত্বপূর্ণ পার্থক্য মনে রাখুন: Vite, esbuild বা Node.js-এর Type Stripping-এর মতো দ্রুত Tool সাধারণত শুধু Type **মুছে** JavaScript বানায়, যাচাই করে না। তাই এগুলো ব্যবহার করলেও Type Check-এর জন্য আলাদা একটি `tsc --noEmit` ধাপ CI-তে রাখতেই হবে।

২৪.১২ Project: TypeSafe Claims

TypeSafe Claims একটি ভ্রমণ সংস্থার কর্মীদের খরচ-দাবির (Expense Claim) Service: কর্মী হোটেল, যাতায়াত, খাবার ও Visa Fee-র খরচ জমা দেন; নীতির সীমা যাচাই হয়; দাবিটি খসড়া থেকে জমা, অনুমোদন বা প্রত্যাখ্যান এবং পরিশোধের পথে এগোয়। Booking নয়—এই Domain বেছে নেওয়া হয়েছে আগের অধ্যায়ের Booking Project-গুলোর পুনরাবৃত্তি এড়াতে।

> **নোট:** শিক্ষামূলক Demo। নীতির সীমা কাল্পনিক; তথ্য Memory-তে; `api.ts` একটি HTTP-ধাঁচের Adapter, আসল Network Server নয় (Server অধ্যায় ২৫-এ)।

File-এর কাঠামো

Folder-এর কাঠামো

```
typesafe-claims/
├── package.json, tsconfig.json
├── src/
│   ├── types.ts        Brand, ClaimItem Union, Claim, Utility Type, Result
│   ├── money.ts        toPaisa, sum, formatTaka
│   ├── policy.ts       assertNever, নীতির সীমা, বর্ণনা
│   ├── parse.ts        unknown থেকে NewClaim (Type Guard)
│   ├── workflow.ts     State Machine, Status-এর লেবেল
│   ├── repository.ts   Generic Repository
│   ├── service.ts      Use Case, ServiceError Union
│   ├── api.ts          HTTP-ধাঁচের Adapter
│   └── demo.ts
└── tests/              parse, service, api (Runtime);
                        types.check.ts (Compile-time)
```

src/types.ts ও src/money.ts

types.ts · TypeScript — চলবে: Node.js

```ts
// Branded Type: Compile-এর সময় সাধারণ number থেকে আলাদা, Runtime-এ number-ই।
declare const brand: unique symbol;
export type Brand<T, Name extends string> = T & { readonly [brand]: Name };

export type Paisa = Brand<number, 'Paisa'>;
export type ClaimId = Brand<string, 'ClaimId'>;

// প্রতিটি খরচের ধরন নিজস্ব Field-সহ: kind দেখে TypeScript বাকিটা জানে।
export type ClaimItem =
  | { kind: 'hotel'; city: string; nights: number; amount: Paisa }
  | { kind: 'transport'; mode: 'bus' | 'train' | 'air'; amount: Paisa }
  | { kind: 'meal'; people: number; amount: Paisa }
  | { kind: 'visa'; country: string; amount: Paisa };

export type ItemKind = ClaimItem['kind'];
export type ClaimStatus =
  'draft' | 'submitted' | 'approved' | 'rejected' | 'paid';

export interface Claim {
  readonly id: ClaimId;
  readonly employee: string;
  readonly items: readonly ClaimItem[];
  readonly status: ClaimStatus;
  readonly total: Paisa;
}

// Utility Type: বিদ্যমান Type থেকে নতুন Type, নকল ছাড়াই।
export type NewClaim = Pick<Claim, 'employee' | 'items'>;
export type ClaimSummary = Omit<Claim, 'items'> & { itemCount: number };
export type StatusCounts = Record<ClaimStatus, number>;

export type Result<T, E> =
  | { readonly ok: true; readonly value: T }
  | { readonly ok: false; readonly error: E };

export const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
export const fail = <E>(error: E): Result<never, E> => ({ ok: false, error });
```

money.ts · TypeScript — চলবে: Node.js

```ts
import type { Paisa } from './types.js';

// Branded মান তৈরির একমাত্র দরজা: Runtime যাচাই পেরোলে তবেই Paisa।
export function toPaisa(value: number): Paisa {
  if (!Number.isSafeInteger(value) || value < 0) {
    throw new RangeError(`অবৈধ পরিমাণ: ${value}`);
```

```
  }
  return value as Paisa;
}

export const sum = (values: readonly Paisa[]): Paisa =>
  toPaisa(values.reduce((total, value) => total + value, 0));

export function formatTaka(value: Paisa): string {
  const whole = Math.floor(value / 100).toLocaleString('en-US');
  return `৳${whole}.${String(value % 100).padStart(2, '0')}`;
}
```

`ClaimItem` চার সদস্যের একটি Discriminated Union: `kind: 'hotel'` হলে `city` ও `nights` আছে, `kind: 'meal'` হলে `people`। `ItemKind = ClaimItem['kind']` Union থেকে Discriminant-গুলোর তালিকা বের করে—হাতে আবার লিখতে হয় না। `ok` ও `fail` Generic Helper: `fail` ফেরত দেয় `Result<never, E>`, যা যেকোনো `Result<T, E>`-এ খাপ খায়।

src/policy.ts ও src/workflow.ts

policy.ts · TypeScript চলবে: Node.js

```
import type { ClaimItem, Paisa } from './types.js';
import { toPaisa } from './money.js';

// কোনো case বাদ পড়লে এখানে Compile Error: item তখন never নয়।
export function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত মান: ${JSON.stringify(value)}`);
}

// প্রতিটি খরচের সর্বোচ্চ অনুমোদিত পরিমাণ (কাল্পনিক নীতি)।
export function limitFor(item: ClaimItem): Paisa {
  switch (item.kind) {
    case 'hotel':
      return toPaisa(item.nights * 8_000_00);
    case 'transport':
      return toPaisa(item.mode === 'air' ? 25_000_00 : 3_000_00);
    case 'meal':
      return toPaisa(item.people * 1_000_00);
    case 'visa':
      return toPaisa(15_000_00);
    default:
      return assertNever(item);
  }
}

export function describe(item: ClaimItem): string {
  switch (item.kind) {
    case 'hotel':
```

```
      return `${item.city}-এ ${item.nights} রাত হোটেল`;
    case 'transport':
      return `${item.mode} যাতায়াত`;
    case 'meal':
      return `${item.people} জনের খাবার`;
    case 'visa':
      return `${item.country} Visa Fee`;
    default:
      return assertNever(item);
  }
}
```

workflow.ts · TypeScript — চলবে: Node.js

```ts
import type { ClaimStatus } from './types.js';
import { assertNever } from './policy.js';

export type Action = 'submit' | 'approve' | 'reject' | 'pay';

// Record<ClaimStatus, ...>: একটি নতুন Status যোগ করলে এখানে না লিখলে Compile Error।
const MOVES: Record<ClaimStatus, Partial<Record<Action, ClaimStatus>>> = {
  draft: { submit: 'submitted' },
  submitted: { approve: 'approved', reject: 'rejected' },
  approved: { pay: 'paid' },
  rejected: {},
  paid: {},
};

export function nextStatus(
  from: ClaimStatus, action: Action,
): ClaimStatus | null {
  return MOVES[from][action] ?? null;
}

export function statusLabel(status: ClaimStatus): string {
  switch (status) {
    case 'draft':
      return 'খসড়া';
    case 'submitted':
      return 'জমা দেওয়া';
    case 'approved':
      return 'অনুমোদিত';
    case 'rejected':
      return 'প্রত্যাখ্যাত';
    case 'paid':
      return 'পরিশোধিত';
    default:
      return assertNever(status);
```

```
  }
}
```

প্রতিটি `switch`-এর `default`-এ `assertNever`: একটি নতুন খরচের ধরন (যেমন `'insurance'`) যোগ করলে `limitFor` ও `describe` দুটিতেই Compile Error, যতক্ষণ না নতুন `case` লেখা হয়। `MOVES`-এর Type `Record<ClaimStatus, ...>`—নতুন Status যোগ করলে এই টেবিলেও Compile Error। অধ্যায় ২১-এর State Machine এখন Compiler-এর পাহারায়।

src/parse.ts: Trust Boundary

parse.ts · TypeScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```ts
import type { ClaimItem, NewClaim, Result } from './types.js';
import { ok, fail } from './types.js';
import { toPaisa } from './money.js';

export interface Issue {
  readonly path: string;
  readonly message: string;
}

// Type Guard: Runtime যাচাই, এবং সফল হলে TypeScript-কে Type জানায়।
const isRecord = (v: unknown): v is Record<string, unknown> =>
  typeof v === 'object' && v !== null && !Array.isArray(v);
const isText = (v: unknown): v is string =>
  typeof v === 'string' && v.trim().length > 0 && v.length <= 60;
const isCount = (v: unknown, max: number): v is number =>
  Number.isInteger(v) && (v as number) >= 1 && (v as number) <= max;
const isAmount = (v: unknown): v is number =>
  Number.isSafeInteger(v) && (v as number) > 0;
const MODES = ['bus', 'train', 'air'] as const;
const isMode = (v: unknown): v is (typeof MODES)[number] =>
  MODES.some((mode) => mode === v);

// ...
```

parse.ts · TypeScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```ts
// ...

function parseItem(raw: unknown, at: string, issues: Issue[]): ClaimItem | null {
  const report = (field: string, message: string): null => {
    issues.push({ path: field ? `${at}.${field}` : at, message });
    return null;
  };
  if (!isRecord(raw)) {
    return report('', 'একটি Object প্রয়োজন');
  }
```

```
  const { kind, amount } = raw;
  if (!isAmount(amount)) {
    return report('amount', 'পয়সায় ধনাত্মক পূর্ণসংখ্যা');
  }
  const paisa = toPaisa(amount);
  switch (kind) {
    case 'hotel': {
      const { city, nights } = raw;
      if (!isText(city)) {
        return report('city', 'শহরের নাম প্রয়োজন');
      }
      return isCount(nights, 30)
        ? { kind, city: city.trim(), nights, amount: paisa }
        : report('nights', '১ থেকে ৩০ রাত');
    }
    case 'transport':
      return isMode(raw.mode)
        ? { kind, mode: raw.mode, amount: paisa }
        : report('mode', 'bus, train অথবা air');
    case 'meal':
      return isCount(raw.people, 20)
        ? { kind, people: raw.people, amount: paisa }
        : report('people', '১ থেকে ২০ জন');
    case 'visa':
      return isText(raw.country)
        ? { kind, country: raw.country.trim(), amount: paisa }
        : report('country', 'দেশের নাম প্রয়োজন');
    default:
      return report('kind', 'অজানা খরচের ধরন');
  }
}

// ...
```

parse.ts · TypeScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// API-র সীমানা: যা আসে তা unknown; যাচাই পেরোলে তবেই NewClaim।
export function parseNewClaim(input: unknown): Result<NewClaim, Issue[]> {
  const issues: Issue[] = [];
  if (!isRecord(input)) {
    return fail([{ path: '', message: 'একটি JSON Object প্রয়োজন' }]);
  }
  const { employee, items } = input;
  if (!isText(employee)) {
    issues.push({ path: 'employee', message: 'কর্মীর নাম প্রয়োজন' });
  }
  if (!Array.isArray(items) || items.length < 1 || items.length > 20) {
```

```
    issues.push({ path: 'items', message: '১ থেকে ২০টি খরচ' });
  }
  const parsed = Array.isArray(items)
    ? items.map((raw, i) => parseItem(raw, `items[${i}]`, issues))
    : [];
  if (issues.length > 0 || !isText(employee)) {
    return fail(issues);
  }
  const valid = parsed.filter((item) => item !== null);
  return ok({ employee: employee.trim(), items: valid });
}
```

`parseNewClaim`-এর Parameter `unknown`: এই File-এর বাইরে কোথাও বাইরের তথ্য সরাসরি ব্যবহার হয় না। প্রতিটি Field একটি Type Guard দিয়ে যাচাই হয়, এবং `switch (kind)`-এর প্রতিটি `case`-এ TypeScript জানে `kind` কোন Literal—তাই `{ kind, city, nights, amount }` সঠিক Union সদস্য হিসেবে গৃহীত হয়। নতুন Object শুধু পরিচিত Field দিয়ে তৈরি: ইনপুটে `status: 'paid'` বা `approved: true` পাঠালেও তা কোথাও পৌঁছায় না (অধ্যায় ২০-এর Allowlist)। প্রতিটি ভুল একটি পথসহ (`items[1].mode`), যাতে UI সঠিক ঘরে বার্তা দেখাতে পারে।

src/repository.ts ও src/service.ts

repository.ts · TypeScript — চলবে: Node.js

```
// Generic Repository: যেকোনো Type T, যার একটি id আছে, এবং id-র Type K।
export interface Repository<T extends { readonly id: K }, K> {
  get(id: K): T | undefined;
  save(item: T): void;
  list(): readonly T[];
}

export class MemoryRepository<T extends { readonly id: K }, K>
implements Repository<T, K> {
  readonly #items = new Map<K, T>();

  get(id: K): T | undefined {
    return this.#items.get(id);
  }

  save(item: T): void {
    this.#items.set(item.id, item);
  }

  list(): readonly T[] {
    return [...this.#items.values()];
  }
}
```

service.ts · TypeScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```ts
import type {
  Claim, ClaimId, ClaimSummary, Paisa, Result, StatusCounts,
} from './types.js';
import { ok, fail } from './types.js';
import { sum } from './money.js';
import { limitFor } from './policy.js';
import { parseNewClaim, type Issue } from './parse.js';
import { nextStatus, type Action } from './workflow.js';
import type { Repository } from './repository.js';

// প্রতিটি সম্ভাব্য ব্যর্থতা একটি Type; ডাকা Code সবগুলো সামলাতে বাধ্য।
export type ServiceError =
  | { readonly code: 'VALIDATION'; readonly issues: readonly Issue[] }
  | { readonly code: 'OVER_LIMIT'; readonly index: number; readonly limit: Paisa }
  | { readonly code: 'NOT_FOUND'; readonly id: string }
  | { readonly code: 'INVALID_TRANSITION'; readonly from: Claim['status'];
      readonly action: Action };

export function createClaimService(
  repository: Repository<Claim, ClaimId>,
  newId: () => ClaimId,
) {
  function create(input: unknown): Result<Claim, ServiceError> {
    const parsed = parseNewClaim(input);
    if (!parsed.ok) {
      return fail({ code: 'VALIDATION', issues: parsed.error });
    }
    const { employee, items } = parsed.value;
    for (const [index, item] of items.entries()) {
      const limit = limitFor(item);
      if (item.amount > limit) {
        return fail({ code: 'OVER_LIMIT', index, limit });
      }
    }
    const total = sum(items.map((item) => item.amount));
    const claim: Claim = { id: newId(), employee, items, status: 'draft', total };
    repository.save(claim);
    return ok(claim);
  }

// ...
```

service.ts · TypeScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```ts
// ...

  function act(id: ClaimId, action: Action): Result<Claim, ServiceError> {
```

```
    const claim = repository.get(id);
    if (!claim) {
      return fail({ code: 'NOT_FOUND', id });
    }
    const status = nextStatus(claim.status, action);
    if (!status) {
      return fail({ code: 'INVALID_TRANSITION', from: claim.status, action });
    }
    const updated: Claim = { ...claim, status };
    repository.save(updated);
    return ok(updated);
  }

  function summary(): StatusCounts {
    const counts: StatusCounts = {
      draft: 0, submitted: 0, approved: 0, rejected: 0, paid: 0,
    };
    for (const claim of repository.list()) {
      counts[claim.status] += 1;
    }
    return counts;
  }

  const list = (): ClaimSummary[] => repository.list()
    .map(({ items, ...rest }) => ({ ...rest, itemCount: items.length }));

  return { create, act, summary, list };
}

export type ClaimService = ReturnType<typeof createClaimService>;
```

`ServiceError` চার রকম ব্যর্থতার Union, প্রতিটির নিজস্ব তথ্য। Service কোনো Exception ছোড়ে না; প্রতিটি ফল একটি `Result`, তাই ডাকা Code ব্যর্থতা উপেক্ষা করতে পারে না— `result.value` পড়তে হলে আগে `result.ok` যাচাই করতেই হবে। `ClaimService = ReturnType<typeof createClaimService>`: Factory-র ফেরত Type হাতে না লিখে তৈরি।

src/api.ts

api.ts · TypeScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import type { ClaimId } from './types.js';
import type { ClaimService, ServiceError } from './service.js';
import type { Action } from './workflow.js';
import { assertNever } from './policy.js';

export interface ApiRequest {
  readonly method: string;
  readonly path: string;
```

```
  readonly body?: unknown;
}

export interface ApiResponse {
  readonly status: number;
  readonly body: unknown;
}

// প্রতিটি ServiceError-এর HTTP Status; নতুন Error যোগ করলে Compile Error।
function statusOf(error: ServiceError): number {
  switch (error.code) {
    case 'VALIDATION':
    case 'OVER_LIMIT':
      return 422;
    case 'NOT_FOUND':
      return 404;
    case 'INVALID_TRANSITION':
      return 409;
    default:
      return assertNever(error);
  }
}

// ...
```

api.ts · TypeScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

const ACTIONS: readonly Action[] = ['submit', 'approve', 'reject', 'pay'];
const isAction = (v: string): v is Action => ACTIONS.some((a) => a === v);

// Transport Adapter: HTTP-এর মতো অনুরোধ থেকে Service; আসল Network Server নয়।
export function handle(service: ClaimService, request: ApiRequest): ApiResponse {
  const { method, path, body } = request;
  const action = /^\/claims\/([\w-]+)\/(\w+)$/.exec(path);
  const result = method === 'POST' && path === '/claims'
    ? service.create(body)
    : method === 'POST' && action?.[1] && action[2] && isAction(action[2])
      ? service.act(action[1] as ClaimId, action[2])
      : null;
  if (result === null) {
    return method === 'GET' && path === '/claims'
      ? { status: 200, body: service.list() }
      : { status: 404, body: { error: 'পাওয়া যায়নি' } };
  }
  return result.ok
    ? { status: path === '/claims' ? 201 : 200, body: result.value }
```

```
    : { status: statusOf(result.error), body: result.error };
}
```

`statusOf`-এর Exhaustive `switch` প্রতিটি `ServiceError`-কে একটি HTTP Status দেয়; নতুন Error যোগ করে এখানে ভুলে গেলে Compile Error। `isAction` URL-এর লেখাকে Type Guard দিয়ে `Action`-এ রূপান্তর করে। একমাত্র `as ClaimId` এখানে, কারণ Repository-তে খোঁজাই সেই ID-র যাচাই।

src/demo.ts

demo.ts · TypeScript — চলবে: Node.js

```ts
import type { ClaimId } from './types.js';
import { createClaimService } from './service.js';
import { MemoryRepository } from './repository.js';
import { handle } from './api.js';
import { formatTaka } from './money.js';
import { statusLabel } from './workflow.js';
import type { Claim } from './types.js';

let counter = 0;
const newId = () => `EC-${String(++counter).padStart(3, '0')}` as ClaimId;
const service = createClaimService(new MemoryRepository<Claim, ClaimId>(), newId);

// JSON.parse-এর ফল unknown-এর মতোই: বাইরের তথ্য, যাচাই না হওয়া পর্যন্ত।
const body: unknown = JSON.parse(`{
  "employee": "Mina Akter",
  "items": [
    { "kind": "hotel", "city": "Cox's Bazar", "nights": 2, "amount": 1200000 },
    { "kind": "transport", "mode": "bus", "amount": 180000 }
  ]
}`);

const created = handle(service, { method: 'POST', path: '/claims', body });
console.log(created.status, JSON.stringify(created.body).slice(0, 44));

const result = service.act('EC-001' as ClaimId, 'submit');
if (result.ok) {
  const { id, status, total } = result.value;
  console.log(id, statusLabel(status), formatTaka(total));
}

const bad = handle(service, {
  method: 'POST', path: '/claims',
  body: { employee: '', items: [{ kind: 'meal', people: 0, amount: 5 }] },
});
console.log(bad.status, JSON.stringify(bad.body));
```

```
const twice = handle(service, { method: 'POST', path: '/claims/EC-001/pay' });
console.log(twice.status, JSON.stringify(twice.body));
```

Terminal

```
npm run demo
```

Terminal

```
201 {"id":"EC-001","employee":"Mina Akter","item
EC-001 জমা দেওয়া ৳13,800.00
422 {"code":"VALIDATION","issues":[{"path":"employee","message":"কর্মীর নাম প্রয়োজন"},
    {"path":"items[0].people","message":"১ থেকে ২০ জন"}]}
409 {"code":"INVALID_TRANSITION","from":"submitted","action":"pay"}
```

এটি Compile করা `dist/src/demo.js` চালানোর আসল Output (এই বইয়ের QA-তে)। প্রথম লাইন সংক্ষিপ্ত (`slice(0, 44)`)।

Test: Compile-time ও Runtime

types.check.ts · TypeScript — চলবে: Node.js

```ts
// Compile-time Test: এই File কিছু চালায় না। প্রতিটি @ts-expect-error-এর পরের
// লাইনে Type Error থাকতেই হবে; না থাকলে tsc নিজেই ব্যর্থ হয়। অর্থাৎ Type-এর
// নিশ্চয়তাগুলো (Brand, Union, readonly) কেউ ভুল করে ঢিলে করলে Build ভাঙবে।
import type {
  Claim, ClaimId, ClaimItem, Paisa, StatusCounts,
} from '../src/types.js';
import { toPaisa } from '../src/money.js';

export function compileTimeChecks(claim: Claim): void {
  // @ts-expect-error সাধারণ number সরাসরি Paisa নয়; toPaisa দিয়ে যাচাই করতে হয়।
  const raw: Paisa = 500;

  // @ts-expect-error সাধারণ string ClaimId নয়।
  const id: ClaimId = 'EC-1';

  // @ts-expect-error hotel-এর nights বাধ্যতামূলক।
  const hotel: ClaimItem = { kind: 'hotel', city: 'Sylhet', amount: toPaisa(1) };

  const meal: ClaimItem = {
    kind: 'meal', people: 2, amount: toPaisa(1),
    // @ts-expect-error meal-এ city নেই (Union-এর ভুল সদস্য)।
    city: 'x',
  };

  // @ts-expect-error readonly Field বদলানো যায় না।
```

```
  claim.status = 'paid';

  // @ts-expect-error Record<ClaimStatus, number>-এ প্রতিটি অবস্থা লাগবে।
  const counts: StatusCounts = { draft: 0, submitted: 0 };

  void [raw, id, hotel, meal, counts];
}
```

parse.test.ts · TypeScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Runtime Test: TypeScript Runtime-এ নেই, তাই unknown ইনপুটের যাচাই Test করতে হয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { parseNewClaim } from '../src/parse.js';

// ...

test('প্রতিটি ভুলের পথ ও বার্তা', () => {
  const result = parseNewClaim({
    employee: '',
    items: [
      { kind: 'meal', people: 0, amount: 5 },
      { kind: 'transport', mode: 'rocket', amount: 10 },
      { kind: 'spa', amount: 10 },
      { kind: 'visa', country: 'Japan', amount: -1 },
      'text',
    ],
  });
  assert.equal(result.ok, false);
  if (!result.ok) {
    assert.deepEqual(result.error.map((i) => i.path), [
      'employee', 'items[0].people', 'items[1].mode', 'items[2].kind',
      'items[3].amount', 'items[4]',
    ]);
  }
});

// ...
```

দুই ধরনের Test, দুই ধরনের নিশ্চয়তা। `types.check.ts` কিছুই চালায় না; এর প্রতিটি `@ts-expect-error`-এর পরের লাইনে Type Error থাকতেই হবে, নইলে `tsc` নিজেই "Unused '@ts-expect-error' directive" বলে ব্যর্থ হয়। এই বইয়ের QA-তে এটি যাচাই করা হয়েছে: একটি লাইন ঠিক করে দিলে (`toPaisa(500)`) Build ভেঙেছে `error TS2578`-এ। অর্থাৎ কেউ ভবিষ্যতে `Paisa`-কে সাধারণ `number` বানিয়ে দিলে Build ব্যর্থ হবে। আর `parse.test.ts` Runtime-এ যাচাই করে যে প্রতিটি অবৈধ ইনপুট ধরা পড়ে—কারণ Type Runtime-এ নেই, এবং বাইরের তথ্য Type মানে না।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd typesafe-claims
npm install
npm run typecheck
npm test
```

`npm install` TypeScript ও `@types/node` আনে (এই বইয়ের QA-তে Install করা হয়নি; উপরের নোট দেখুন)। `npm run typecheck` (`tsc --noEmit`) শুধু Type যাচাই করে। `npm test` আগে `tsc` দিয়ে `dist/`-এ Build করে (এই ধাপে `types.check.ts`-এর Compile-time Test-ও চলে), তারপর `node --test "dist/tests/*.test.js"` ১১টি Runtime Test চালায়: `parse` ৪, `service` ৫, `api` ২; শেষে `# pass 11`, `# fail 0`।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find module './money.js'` (Compile)	Import-এ `.ts` বা Extension নেই	`.js` লিখুন (NodeNext)
`Cannot find module 'node:test'`	`@types/node` নেই	`npm install --save-dev @types/node`
Test File খুঁজে পায় না	Build হয়নি, বা `.ts` চালানোর চেষ্টা	`npm test` (আগে `tsc`)
`Unused '@ts-expect-error'`	একটি Type-এর নিশ্চয়তা ঢিলে হয়েছে	Type পরিবর্তনটি পুনর্বিবেচনা
`is possibly 'undefined'`	`noUncheckedIndexedAccess`	যাচাই বা `?.`

২৪.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
`any` দিয়ে Error চাপা	Type Check বন্ধ, ভুল Runtime-এ	`unknown` ও Type Guard
`JSON.parse(x) as T`	যাচাই ছাড়া বিশ্বাস	Runtime Validation
`strict` বন্ধ	`null`-এর ভুল ধরা পড়ে না	নতুন Project-এ সব সময় চালু
`!` দিয়ে `undefined` উপেক্ষা	Runtime Error থেকে যায়	Narrowing, `?.`, `??`
`switch`-এ `default: return ''`	নতুন সদস্যের জন্য নীরব ভুল	`assertNever`
সব জায়গায় Type লেখা	অপ্রয়োজনীয় শব্দ	Inference; সীমানায় স্পষ্ট Type
একই আকার বারবার লেখা	অমিল হয়ে যায়	`Pick`, `Omit`, `Partial`

ভুল	কেন সমস্যা	সমাধান
ধরে নেওয়া TypeScript ইনপুট নিরাপদ করে	Runtime-এ Type নেই	Server-এ যাচাই, অধ্যায় ২০
`strip-types`-কে Type Check ভাবা	শুধু মুছে ফেলে	CI-তে `tsc --noEmit`

২৪.১৪ Debugging নির্দেশিকা

১. **Error-এর Code পড়ুন:** `TS2345` (ভুল Argument), `TS2322` (ভুল মান বসানো), `TS18048` (সম্ভাব্য `undefined`), `TS18046` (`unknown`), `TS2578` (অপ্রয়োজনীয় `@ts-expect-error`)।

২. **Editor-এ Hover করুন:** প্রতিটি Variable-এর অনুমিত Type দেখায়; প্রত্যাশার সঙ্গে না মিললে সেখানেই ভুল।

৩. **দীর্ঘ Error-এর শেষ লাইন** সাধারণত আসল কারণ বলে।

৪. **Runtime Error কিন্তু Compile ঠিক:** কোথাও `any`, `as` বা `!`, অথবা বাইরের তথ্য যাচাই হয়নি।

৫. **`tsc --noEmit --watch`:** প্রতিটি সংরক্ষণে Type Check।

৬. **Source Map** (`sourceMap: true`): Compile করা JavaScript-এর Error Stack মূল `.ts` লাইনে নিয়ে যায়।

২৪.১৫ পেশাদার অভ্যাস

- **`strict` চালু, এবং `noUncheckedIndexedAccess`-ও।**
- **সীমানায় `unknown`, ভেতরে নির্দিষ্ট Type;** যাচাই এক জায়গায়।
- **`any`, `as` ও `!` Code Review-তে প্রশ্ন তোলার মতো;** প্রতিটির পাশে কারণ লিখুন।
- **Status, Mode, Category-র জন্য Literal Union,** `switch`-এ `assertNever`।
- **Error-এর জন্য Discriminated Union,** যাতে ডাকা Code প্রতিটি ক্ষেত্র সামলায়।
- **Type বারবার লিখবেন না:** Utility Type ও `ReturnType`।
- **CI-তে `tsc --noEmit`,** Test-এর আগে।
- **পুরোনো JavaScript Project ধাপে ধাপে:** `allowJs`, তারপর একটি একটি File।

২৪.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `function discount(price: number, percent: number): number` লিখুন, এবং দেখান `discount('500', 10)` Compile Error দেয়।

২. `type Seat = 'window' | 'aisle' | 'middle'`-এর একটি Variable-এ `'floor'` বসালে কী ঘটে? কেন?

৩. `const` ও `let`-এ Inference-এর পার্থক্য উদাহরণসহ লিখুন।

৪. `users.find(...)`-এর ফল নিরাপদে ব্যবহারের তিনটি উপায় লিখুন।

৫. `any` ও `unknown`-এর একটি পার্থক্য এক বাক্যে।

মধ্যম স্তর

৬. `isTour(value: unknown): value is Tour` Type Guard লিখুন (`title: string`, `days: number`), এবং দুটি JSON-এ পরীক্ষা করুন।

৭. একটি `Shape` Discriminated Union (`circle`, `rectangle`) ও Exhaustive `area` Function লিখুন।

৮. `pluck<T, K extends keyof T>(items: T[], key: K): T[K][]` লিখুন।

৯. TypeSafe Claims-এ একটি নতুন খরচের ধরন `insurance` (Field: `provider`) যোগ করলে কোন কোন File-এ Compile Error হবে? তালিকা করুন।

চ্যালেঞ্জ স্তর

১০. `ok` / `fail` দিয়ে একটি `parseAge(input: unknown): Result<number, string>` লিখুন এবং ফলাফল সামলান।

১১. একটি Generic `LookupCache<K, V>` Class লিখুন, যার `get` ফেরত দেয় `V | undefined`, এবং `getOr(key, fallback)`।

১২. `Readonly` ও `as const` দিয়ে একটি অপরিবর্তনীয় Config Object তৈরি করুন, এবং দেখান যে তার মান বদলানো Compile Error।

২৪.১৭ অনুশীলনীর সমাধান

সমাধান ১

discount.ts · TypeScript — চলবে: Node.js

```ts
function discount(price: number, percent: number): number {
  return Math.round(price * (1 - percent / 100));
}

console.log(discount(500, 10));
console.log(discount('500', 10));
```

Error Message

```
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

সমাধান ২ — Compile Error (`TS2322`): `'floor'` Union-এর তিনটি Literal-এর কোনোটি নয়। Union মানে "শুধু এগুলো", তাই বানান ভুল বা অজানা মান Code চালানোর আগেই ধরা পড়ে।

সমাধান ৩ — `let count = 5` -এর Type `number` (পরে বদলাতে পারে), `const count = 5` -এর Type Literal `5` । তেমনি `let mode = 'bus'` হলো `string` , `const mode = 'bus'` হলো `'bus'` । তাই `Mode` Union-এ পাঠাতে `const` কাজ করে, `let` করে না (যদি না `let mode: Mode = 'bus'` লেখা হয়)।

সমাধান ৪ — (ক) `if (user) { ... }` দিয়ে Narrowing; (খ) `user?.name ?? 'অজানা'` ; (গ) না পেলে স্পষ্ট Error: `if (!user) throw new Error('পাওয়া যায়নি')` , তার পরের লাইনে `user` নিশ্চিত। `user!.name` এড়িয়ে চলুন।

সমাধান ৫ — `any` সব Type Check বন্ধ করে, আর `unknown` যাচাই (Narrowing) না করা পর্যন্ত কিছুই করতে দেয় না।

সমাধান ৬

is-tour.ts · TypeScript — চলবে: Node.js

```ts
interface Tour {
  title: string;
  days: number;
}

function isTour(value: unknown): value is Tour {
  if (typeof value !== 'object' || value === null) {
    return false;
  }
  const record = value as Record<string, unknown>;
  return typeof record.title === 'string'
    && Number.isInteger(record.days) && (record.days as number) > 0;
}

const inputs = ['{ "title": "Sajek", "days": 2 }',
  '{ "title": "Sajek", "days": "2" }'];
for (const text of inputs) {
  const data: unknown = JSON.parse(text);
  console.log(isTour(data) ? `${data.title}: ${data.days} দিন` : 'অবৈধ');
```

```
}
```

Output

```
Sajek: 2 দিন
অবৈধ
```

সমাধান ৭

shapes.ts · TypeScript — চলবে: Node.js

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rectangle'; width: number; height: number };

function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত: ${JSON.stringify(value)}`);
}

function area(shape: Shape): number {
  switch (shape.kind) {
    case 'circle':
      return Math.PI * shape.radius ** 2;
    case 'rectangle':
      return shape.width * shape.height;
    default:
      return assertNever(shape);
  }
}

const shapes: Shape[] = [
  { kind: 'circle', radius: 1 },
  { kind: 'rectangle', width: 2, height: 3 },
];
console.log(shapes.map((s) => area(s).toFixed(2)));
```

Output

```
[ '3.14', '6.00' ]
```

সমাধান ৮

pluck.ts · TypeScript — চলবে: Node.js

```ts
function pluck<T, K extends keyof T>(items: readonly T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}

const tours = [{ title: 'Sajek', days: 2 }, { title: 'Ratargul', days: 1 }];
const titles: string[] = pluck(tours, 'title');
const days: number[] = pluck(tours, 'days');
console.log(titles, days);
```

Output

```
[ 'Sajek', 'Ratargul' ] [ 2, 1 ]
```

`T[K]` (Indexed Access Type) মানে "T-এর K Field-এর Type": `'title'`-এ `string`, `'days'`-এ `number` — তাই ফলের Type-ও সঠিক।

সমাধান ৯ — `types.ts`-এ Union-এ নতুন সদস্য যোগ করলে: `policy.ts`-এর `limitFor` ও `describe` (দুটি `assertNever`), এবং `parse.ts` —এখানে Compile Error নয়, কারণ অজানা `kind` `default`-এ যায়; তাই নতুন `case` যোগ না করলে Runtime-এ "অজানা খরচের ধরন"। শিক্ষা: Exhaustiveness Compiler-কে তখনই সাহায্য করে, যখন `switch` একটি Union-এর ওপর; `unknown` ইনপুটের `switch`-এর জন্য একটি Runtime Test লাগে (`parse.test.ts`-এ একটি নতুন সারি)।

সমাধান ১০

parse-age.ts · TypeScript — চলবে: Node.js

```ts
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };
const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
const fail = <E>(error: E): Result<never, E> => ({ ok: false, error });

function parseAge(input: unknown): Result<number, string> {
  const value = typeof input === 'string' ? Number(input.trim()) : input;
  if (typeof value !== 'number' || !Number.isInteger(value)) {
    return fail('বয়স পূর্ণসংখ্যা হতে হবে');
  }
  return value >= 0 && value <= 120 ? ok(value) : fail('বয়স ০ থেকে ১২০');
}

for (const input of ['34', 34.5, 200, null]) {
  const result = parseAge(input);
  console.log(result.ok ? `বৈধ: ${result.value}` : `ভুল: ${result.error}`);
}
```

Output

```
বৈধ: 34
ভুল: বয়স পূর্ণসংখ্যা হতে হবে
ভুল: বয়স ০ থেকে ১২০
ভুল: বয়স পূর্ণসংখ্যা হতে হবে
```

সমাধান ১১

cache.ts · TypeScript — চলবে: Node.js

```ts
class LookupCache<K, V> {
  readonly #entries = new Map<K, V>();

  set(key: K, value: V): this {
    this.#entries.set(key, value);
    return this;
  }

  get(key: K): V | undefined {
    return this.#entries.get(key);
  }

  getOr(key: K, fallback: V): V {
    return this.#entries.get(key) ?? fallback;
  }
}
```

```
const rates = new LookupCache<string, number>().set('USD-BDT', 121.45);
const rate: number = rates.getOr('EUR-BDT', 0);
console.log(rates.get('USD-BDT'), rate, rates.get('JPY-BDT'));
```

Output

```
121.45 0 undefined
```

`get`-এর ফেরত Type `number | undefined` হওয়ায় Caller-কে না-থাকার ক্ষেত্র ভাবতে হয়; `getOr` সেই সিদ্ধান্ত একবারে নেয়। লক্ষ করুন, `??` দিয়ে সংরক্ষিত `0` বা `''`-ও ঠিক থাকে (যা `||` দিলে হারিয়ে যেত)।

সমাধান ১২

config.ts · TypeScript — চলবে: Node.js

```
const CONFIG = {
  currency: 'BDT',
  limits: { hotelPerNight: 8_000, mealPerPerson: 1_000 },
  modes: ['bus', 'train', 'air'],
} as const;

type TravelMode = (typeof CONFIG.modes)[number];
const mode: TravelMode = 'air';

console.log(mode, CONFIG.limits.hotelPerNight);
CONFIG.limits.hotelPerNight = 20_000;
CONFIG.modes.push('ship');
```

Error Message

```
error TS2540: Cannot assign to 'hotelPerNight' because it is a read-only property.
error TS2339: Property 'push' does not exist on type
'readonly ["bus", "train", "air"]'.
```

`as const` সব স্তরে `readonly` ও Literal Type দেয়, তাই `modes` থেকে `'bus' | 'train' | 'air'` Union-ও তৈরি করা গেছে। মনে রাখুন, এটি শুধু Compile-time: Runtime-এ বদলানো আটকাতে `Object.freeze` লাগে (অধ্যায় ১৮)।

২৪.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. TypeScript-এর Type Runtime-এ কী হয়?

ক) Browser যাচাই করে

খ) Compile-এর পর মুছে যায় (Type Erasure)

গ) Node.js যাচাই করে

ঘ) ধীর হয়ে যায়

প্রশ্ন ২. Server-এ আসা JSON-এর জন্য কোন Type উপযুক্ত?

ক) `unknown`, তারপর Runtime যাচাই

খ) `any`

গ) `as Customer`

ঘ) `object`

প্রশ্ন ৩. `const mode = 'bus'`-এর অনুমিত Type?

ক) `string`

খ) `any`

গ) Literal `'bus'`

ঘ) `unknown`

প্রশ্ন ৪. `strictNullChecks` কী নিশ্চিত করে?

ক) কোনো মান কখনো `null` নয়

খ) Runtime-এ `null` আটকানো হয়

গ) `null` স্বয়ংক্রিয়ভাবে `0`

ঘ) `undefined` হতে পারে এমন মান যাচাই ছাড়া ব্যবহার করা যায় না

প্রশ্ন ৫. `switch`-এর `default`-এ `assertNever(x)` কেন?

ক) Union-এর কোনো সদস্য বাদ পড়লে Compile Error পেতে

খ) Runtime দ্রুত করতে

গ) `x`-কে `any` করতে

ঘ) Test এড়াতে

> **প্রশ্ন ৬.** Type Guard-এর ফেরত Type কোনটি?
>
> ক) `boolean | Type`
>
> খ) `value is Customer`
>
> গ) `Customer`
>
> ঘ) `never`

> **প্রশ্ন ৭.** `Partial<Tour>` কী?
>
> ক) Tour-এর প্রথম অর্ধেক Field
>
> খ) শুধু `readonly` Field
>
> গ) Tour-এর সব Field ঐচ্ছিক
>
> ঘ) Tour-এর একটি Copy

> **প্রশ্ন ৮.** Branded Type (`Paisa`)-এর লাভ কী?
>
> ক) Runtime-এ পরিমাণ যাচাই
>
> খ) Memory কম
>
> গ) দ্রুত গণনা
>
> ঘ) একই `number` হলেও ভিন্ন অর্থের মান ভুলে মেশানো Compile-এ ধরা

> **প্রশ্ন ৯.** `// @ts-expect-error`-এর পরের লাইনে কোনো Error না থাকলে?
>
> ক) `tsc` ব্যর্থ হয় (Unused directive)
>
> খ) কিছুই না
>
> গ) Runtime Error
>
> ঘ) Warning মাত্র

> **প্রশ্ন ১০.** Node.js-এর `--experimental-strip-types` কী করে?
>
> ক) Type যাচাই করে চালায়
>
> খ) Type মুছে চালায়, যাচাই করে না
>
> গ) TypeScript Install করে
>
> ঘ) JavaScript-কে TypeScript-এ রূপান্তর করে

২৪.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	তাই Runtime-এ ইনপুট যাচাই আলাদা দায়িত্ব।
২	ক	`any` ও `as` যাচাই ছাড়াই বিশ্বাস করে।
৩	গ	`const` বদলায় না, তাই সবচেয়ে নির্দিষ্ট Type।
৪	ঘ	Compile-time নিশ্চয়তা; Runtime-এ যাচাই আপনার Code-এ।
৫	ক	বাকি সদস্য `never`-এ বসানো যায় না।
৬	খ	Runtime-এ `boolean`, Compiler-এর কাছে Type-এর তথ্য।
৭	গ	আংশিক হালনাগাদের জন্য।
৮	ঘ	Runtime-এ সাধারণ সংখ্যাই; সুবিধাটি Compile-এ।
৯	ক	`TS2578`; তাই এটি Compile-time Test।
১০	খ	Type Check-এর জন্য `tsc` লাগবেই।

২৪.২০ অধ্যায়ের সারসংক্ষেপ

- TypeScript Compile-এর আগে ভুল ধরে, তারপর Type মুছে JavaScript দেয়; Runtime-এ কোনো Type নেই।
- `strict` ও `noUncheckedIndexedAccess` চালু রাখুন; `null` / `undefined` যাচাই বাধ্যতামূলক হয়।
- Inference বেশিরভাগ Type জানে; সীমানায় (Parameter, ফেরত, Export) স্পষ্ট Type।
- Literal Union সীমিত মানের তালিকা; Discriminated Union বিভিন্ন আকারের তথ্য ও Error।
- `assertNever` ও `Record<Union, ...>` দিয়ে সম্পূর্ণতা Compiler-এর পাহারায়।
- বাইরের তথ্য `unknown`, Type Guard দিয়ে যাচাই; `any`, `as`, `!` এড়িয়ে চলুন।
- Generics ও Utility Type: একবার লেখা, Type না হারিয়ে পুনর্ব্যবহার।
- Branded Type একই Runtime Type-এর ভিন্ন অর্থ আলাদা রাখে।
- Compile-time Test (`@ts-expect-error`) Type-এর নিশ্চয়তা পাহারা দেয়; Runtime Test বাইরের তথ্যের যাচাই।

পরের অধ্যায়ে Node.js দিয়ে Backend: REST API, Express, Layered Architecture, Authentication এবং একটি পরীক্ষিত Server।

২৪.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Static Typing	Code চালানোর আগে Type যাচাই।

পরিভাষা	অর্থ
Type Erasure	Compile-এর পর Type মুছে যাওয়া।
Inference	মান দেখে Type অনুমান।
Literal Type	একটি নির্দিষ্ট মানই Type (`'bus'`)।
Union	কয়েকটি Type-এর যেকোনো একটি।
Narrowing	যাচাইয়ের পর Type আরও নির্দিষ্ট হওয়া।
Type Guard	`value is T` ফেরত দেওয়া যাচাই Function।
Discriminated Union	একটি সাধারণ Literal Field-যুক্ত Union।
`never`	যে Type-এর কোনো মান নেই।
Generic	Type Parameter-যুক্ত Function বা Class।
Utility Type	বিদ্যমান Type থেকে নতুন Type (`Pick`, `Partial`)।
Branded Type	অদৃশ্য চিহ্নযুক্ত আলাদা Type।

২৪.২২ আরও পড়ুন

- **TypeScript Handbook:** `https://www.typescriptlang.org/docs/handbook/intro.html` — ভাষার পূর্ণ পরিচয়।
- **TSConfig Reference:** `https://www.typescriptlang.org/tsconfig/` — প্রতিটি Compiler Setting।
- **TypeScript — Narrowing:** `https://www.typescriptlang.org/docs/handbook/2/narrowing.html` — Type Guard ও Discriminated Union।
- **TypeScript — Utility Types:** `https://www.typescriptlang.org/docs/handbook/utility-types.html` — `Pick`, `Omit`, `Partial`।
- **Node.js — TypeScript support:** `https://nodejs.org/api/typescript.html` — Type Stripping ও তার সীমা।

অধ্যায় ২৫

২৫

JavaScript Backend Development with Node.js: REST APIs, Express, Databases, Authentication এবং Production Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Browser-এর JavaScript ও Server-এর Node.js-এর দায়িত্ব ও ঝুঁকির পার্থক্য বুঝতে।
- REST নীতিতে Resource, HTTP Method, Status Code ও URL নকশা করতে।
- Express-এ Route, Router, Middleware Chain এবং Error-handling Middleware লিখতে, এবং Express 4-এর Async Error-এর ফাঁদ এড়াতে।
- Route, Service ও Repository-তে ভাগ করা Layered Backend তৈরি করতে।
- Bearer Token দিয়ে Authentication এবং Role ও মালিকানাভিত্তিক Authorization প্রয়োগ করতে।
- Cursor Pagination, Filtering, `Idempotency-Key` ও `If-Match`-ভিত্তিক Optimistic Concurrency প্রয়োগ করতে।
- Request ID, Structured Log, Rate Limit, Health ও Readiness Check এবং Graceful Shutdown যোগ করতে।
- Repository-র চুক্তির পেছনে কীভাবে একটি আসল Database বসে, তা বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **LeadDesk API** তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৫: `fetch`, HTTP-এর মৌলিক ধারণা।
- অধ্যায় ১৬ ও ২২: `node:test`, Integration Test।
- অধ্যায় ২০: `node:http`, Validation, Authentication বনাম Authorization, Log Redaction।
- অধ্যায় ২১: Layered Architecture, Dependency Injection।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Website-এর "যোগাযোগ করুন" Form থেকে প্রতিদিন শত শত জিজ্ঞাসা (Lead) আসে: কে, কোথায় যেতে চান, কতজন। বিক্রয় দলের প্রতিজন Agent নিজের Lead-গুলো দেখেন ও এগিয়ে নেন, Manager সবকিছু দেখেন ও ভাগ করে দেন। এই তথ্যের কেন্দ্রে একটি **Backend API**: Website, Mobile App ও অফিসের Dashboard—সবাই এর সঙ্গে কথা বলে।

Backend-এর ভুল Frontend-এর ভুলের চেয়ে অনেক বেশি ক্ষতিকর। একটি Bug একজন ব্যবহারকারীর পর্দায় নয়, সবার তথ্যে প্রভাব ফেলে; একটি ধীর Endpoint পুরো Server আটকে দেয়; একটি Authorization-এর ফাঁক সব গ্রাহকের ফোন নম্বর ফাঁস করে। অধ্যায় ২০-এ আপনি `node:http` দিয়ে নিরাপত্তার মূল নিয়ন্ত্রণগুলো তৈরি করেছেন। এই অধ্যায় তার ওপর পেশাদার Backend-এর কাঠামো যোগ করে: Express, REST-এর নকশা, এবং এমন নির্ভরযোগ্যতার কৌশল, যা Network-এর অনিশ্চয়তায় ও একসঙ্গে অনেক ব্যবহারকারীর মধ্যে তথ্য সঠিক রাখে।

২৫.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`node:http`-এ হাতে Routing (অধ্যায় ২০)	Express: Router, Middleware Chain, Error Middleware
Cookie Session ও CSRF (অধ্যায় ২০)	API-র জন্য Bearer Token, Role ও Permission
404 দিয়ে অন্যের তথ্য লুকানো (অধ্যায় ২০)	Agent বনাম Manager-এর পূর্ণ অনুমতির নিয়ম
Retry (অধ্যায় ১৬, ২২)	Idempotency-Key: Retry-কে নিরাপদ করা
Immutable হালনাগাদ (অধ্যায় ১৮)	Version ও `If-Match`: একসঙ্গে দুজনের হালনাগাদ
Redaction-সহ Log (অধ্যায় ২০)	Request ID, প্রতিটি অনুরোধে একটি JSON Log
—	REST-এর নকশা, Cursor Pagination, Health/Ready, Graceful Shutdown

২৫.২ Browser JavaScript বনাম Node.js Server

একই ভাষা, ভিন্ন পৃথিবী:

বিষয়	Browser	Node.js Server
কার যন্ত্রে	ব্যবহারকারীর	আপনার নিয়ন্ত্রিত Server-এ
বিশ্বাস	কোনো কিছুই নয় (ব্যবহারকারী Code বদলাতে পারেন)	নিজের Code; কিন্তু প্রতিটি অনুরোধ অবিশ্বস্ত
একসঙ্গে কতজন	একজন	শত-হাজার, একই Process-এ
প্রবেশাধিকার	DOM, `localStorage`	File, Database, Secret, Network
একটি ধীর কাজের প্রভাব	একটি Tab জমে যায়	সব ব্যবহারকারী অপেক্ষা করেন
ভুলের প্রভাব	একজন ব্যবহারকারী	সব তথ্য, সব ব্যবহারকারী

দ্বিতীয় সারিটি সবচেয়ে গুরুত্বপূর্ণ: Server-এর প্রতিটি অনুরোধ বাইরের পৃথিবী থেকে আসে, তাই প্রতিটি Field যাচাই, প্রতিটি অনুমতি পরীক্ষা—Browser কী পাঠানোর "কথা", তা নয়, আসলে কী পাঠিয়েছে।

২৫.৩ REST: Resource, Method ও Status Code

REST (Representational State Transfer) একটি নকশার ধরন: API কতগুলো **Resource** (Lead, Tour, Customer) প্রকাশ করে, প্রতিটির একটি URL, এবং HTTP Method বলে কী করা হচ্ছে।

কাজ	Method ও URL	সফল Status	বৈশিষ্ট্য
তালিকা	`GET /v1/leads?status=new&limit=20`	200	Safe, Idempotent
একটি পড়া	`GET /v1/leads/L-00001`	200	Safe, Idempotent
তৈরি	`POST /v1/leads`	201, সঙ্গে `Location` Header	Idempotent নয়
আংশিক হালনাগাদ	`PATCH /v1/leads/L-00001`	200	
মুছে ফেলা	`DELETE /v1/leads/L-00001`	204	Idempotent

Safe মানে Server-এর অবস্থা বদলায় না; **Idempotent** মানে একই অনুরোধ একবার বা দশবার পাঠালে ফল একই। `POST` Idempotent নয়: দুবার পাঠালে দুটি Lead—অনুচ্ছেদ ২৫.৮-এ এর সমাধান। ব্যর্থতার Status-গুলো:

Status	অর্থ	LeadDesk-এ উদাহরণ
400	অনুরোধ ভুল	অবৈধ ফোন নম্বর, ভাঙা JSON
401	পরিচয় নেই	Token নেই বা ভুল

Status	অর্থ	LeadDesk-এ উদাহরণ
403	পরিচয় আছে, অনুমতি নেই	Agent মুছতে চাইছেন
404	নেই (অথবা আপনার জন্য নেই)	অন্য Agent-এর Lead
409	বর্তমান অবস্থার সঙ্গে সংঘাত	`won` থেকে আবার `contacted`
412	শর্ত মেলেনি	পুরোনো Version দিয়ে হালনাগাদ
413	অনুরোধ খুব বড়	১০ KB-র বেশি Body
429	অনেক বেশি অনুরোধ	Rate Limit
500	Server-এর অপ্রত্যাশিত ভুল	Bug; বিস্তারিত শুধু Log-এ

URL-এর শুরুতে `/v1`: API-র সংস্করণ। পরে কোনো Field-এর অর্থ বদলাতে হলে `/v2` চালু করা যায়, পুরোনো Client না ভেঙে।

২৫.8 Express: Route ও Middleware

`node:http`-এ প্রতিটি পথ ও Method নিজে মেলাতে হয় (অধ্যায় ২০-এর `route` Function)। **Express** এই কাজ সহজ করে: Route, URL-এর Parameter, Query, JSON Body এবং—সবচেয়ে গুরুত্বপূর্ণ—**Middleware**।

> **নোট:** Express একটি npm Package (`npm install express`)। এই বইয়ের QA-তে কোনো Package Install করা হয়নি; এই কম্পিউটারে আগে থেকে Install করা একটি Global Tool-এর ভেতরে Express 4.22.1 পাওয়া গেছে, এবং এই অধ্যায়ের সব Code ও Test সেটি দিয়ে চালানো। আপনার Project-এ `npm install` দিয়ে নিজস্ব Copy নিন।

express-basics.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const app = express();
app.use(express.json());
const tours = [
  { id: 1, title: 'Sajek', days: 2 },
  { id: 2, title: 'Kuakata', days: 3 },
];

app.get('/tours', (req, res) => {
  const maxDays = Number(req.query.maxDays ?? Infinity);
  res.json(tours.filter((tour) => tour.days <= maxDays));
});
app.get('/tours/:id', (req, res) => {
  const tour = tours.find((t) => t.id === Number(req.params.id));
  if (!tour) {
    res.status(404).json({ error: 'Tour নেই' });
```

```
    return;
  }
  res.json(tour);
});
app.post('/tours', (req, res) => res.status(201).json({ received: req.body }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
for (const path of ['/tours?maxDays=2', '/tours/2', '/tours/9']) {
  const response = await fetch(base + path);
  console.log(response.status, JSON.stringify(await response.json()));
}
const posted = await fetch(`${base}/tours`, {
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({ title: 'Ratargul' }),
});
console.log(posted.status, JSON.stringify(await posted.json()));
server.close();
```

Output

```
200 [{"id":1,"title":"Sajek","days":2}]
200 {"id":2,"title":"Kuakata","days":3}
404 {"error":"Tour নেই"}
201 {"received":{"title":"Ratargul"}}
```

`req.params.id` URL-এর অংশ, `req.query.maxDays` `?`-এর পরের অংশ, `req.body` JSON Body (`express.json()` পড়ে দিয়েছে)। লক্ষ করুন, তিনটিই সব সময় **লেখা** বা অবিশ্বস্ত তথ্য: `Number(...)` দিয়ে রূপান্তর, এবং বাস্তবে যাচাই।

Middleware একটি Function `(req, res, next)`: অনুরোধটি দেখে, কিছু করে, তারপর `next()` ডেকে পরেরটিকে দেয়—অথবা নিজেই উত্তর দিয়ে শৃঙ্খল থামিয়ে দেয়। ক্রমটি `app.use`-এর ক্রম:

middleware-order.mjs · JavaScript চলবে: Node.js

```
import express from 'express';

const app = express();
let order = [];
const step = (name) => (req, res, next) => {
  order.push(name);
  next();
};

app.use(step('requestId'), step('logger'));
app.use('/v1', (req, res, next) => {
```

```
  order.push('auth');
  if (!req.get('authorization')) {
    res.status(401).json({ error: 'Token নেই' });
    return;
  }
  next();
});
app.get('/v1/leads', (req, res) => {
  order.push('route');
  res.json([]);
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const url = `http://127.0.0.1:${server.address().port}/v1/leads`;
for (const headers of [{}, { authorization: 'Bearer demo' }]) {
  order = [];
  const response = await fetch(url, { headers });
  console.log(response.status, order.join(' > '));
}
server.close();
```

Output

```
401 requestId > logger > auth
200 requestId > logger > auth > route
```

Token ছাড়া অনুরোধে `auth` নিজেই 401 দিয়েছে এবং `next()` ডাকেনি, তাই Route পর্যন্ত পৌঁছায়নি—Middleware-এর ক্রমই নিরাপত্তা। একটি ভুল ক্রম (Route আগে, `auth` পরে) হলে সুরক্ষিত তথ্য Token ছাড়াই চলে যেত।

২৫.৫ Error Handling এবং Express 4-এর Async ফাঁদ

Express চারটি Parameter-এর Middleware `(err, req, res, next)`-কে **Error Handler** হিসেবে চেনে: কোনো Route `next(error)` ডাকলে বা Synchronous Error ছুড়লে নিয়ন্ত্রণ সরাসরি সেখানে যায়। কিন্তু Express 4 একটি `async` Function-এর **Rejected Promise** ধরে না—এবং আজকের প্রায় সব Route `async` (Database, Network):

async-errors.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

process.on('unhandledRejection', (error) => {
  console.log(`unhandledRejection: ${error.message}`);
});

const app = express();
const wrap = (handler) => (req, res, next) =>
```

```
  Promise.resolve(handler(req, res, next)).catch(next);

app.get('/unsafe', async () => {
  throw new Error('Database বন্ধ');
});
app.get('/safe', wrap(async () => {
  throw new Error('Database বন্ধ');
}));
app.use((err, req, res, next) => res.status(500).json({ error: 'Server-এ সমস্যা' }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
try {
  await fetch(`${base}/unsafe`, { signal: AbortSignal.timeout(500) });
} catch {
  console.log('/unsafe: ৫০০ ms-এ কোনো উত্তর আসেনি');
}
const safe = await fetch(`${base}/safe`);
console.log(`/safe: ${safe.status} ${JSON.stringify(await safe.json())}`);
server.closeAllConnections();
server.close();
```

Output

```
unhandledRejection: Database বন্ধ
/unsafe: ৫০০ ms-এ কোনো উত্তর আসেনি
/safe: 500 {"error":"Server-এ সমস্যা"}
```

`/unsafe`-এর Error কোনো Handler পায়নি: Client কোনো উত্তর পাননি (Timeout পর্যন্ত ঝুলে ছিলেন), এবং Process-এ একটি `unhandledRejection` ঘটেছে—Node.js-এর Default আচরণে এটি পুরো Server বন্ধ করে দেয়; এখানে শুধু প্রদর্শনের জন্য Listener দিয়ে ধরা হয়েছে। `wrap` Rejection ধরে `next(error)`-এ পাঠায়, এবং Error Handler একটি পরিষ্কার 500 দেয়। **Express 5** এই আচরণ নিজেই দেয় (Rejected Promise স্বয়ংক্রিয়ভাবে `next`-এ যায়); Express 4-এ `wrap` বাধ্যতামূলক।

LeadDesk-এর Error Handler সব Error-কে একই আকার দেয়—`{ error: { code, message, details, requestId } }`—যাতে Client একটি নিয়মেই সব ব্যর্থতা সামলাতে পারে। প্রত্যাশিত ব্যর্থতা (`AppError`: Validation, অনুমতি, সংঘাত) নিজের Status নিয়ে যায়; অপ্রত্যাশিত Error-এর বিস্তারিত শুধু Log-এ, Client পান সাধারণ 500 (অধ্যায় ২০)।

২৫.৬ Layered Architecture: Route, Service, Repository

অধ্যায় ২১-এর স্তরগুলো Backend-এ এভাবে বসে:

স্তর	LeadDesk-এ	দায়িত্ব	জানে না
Route (HTTP)	`routes.mjs`	URL, Header, Status Code, JSON	ব্যবসার নিয়ম, Database
Service	`service.mjs`	নিয়ম, অনুমতি, Idempotency, Version	HTTP, Express
Domain	`leads.mjs`	Lead-এর যাচাই ও অবস্থার পরিবর্তন	HTTP, সংরক্ষণ
Repository	`repository.mjs`	সংরক্ষণ ও খোঁজা	HTTP, অনুমতি

Route পাতলা: শুধু HTTP-এর ভাষাকে Service-এর ভাষায় রূপান্তর (`If-Match` Header থেকে `expectedVersion`)। ফলে Service একটি সাধারণ JavaScript Object, যাকে Express ছাড়াই Test করা যায়, এবং একই Service ভবিষ্যতে একটি Queue Worker বা Command-line Tool-ও ব্যবহার করতে পারে।

২৫.৭ Authentication ও Authorization: Token ও Role

Browser-এর জন্য অধ্যায় ২০-এ HttpOnly Cookie ব্যবহার করেছিলেন। অন্য Server, Mobile App বা Script থেকে আসা API অনুরোধে সাধারণত **Bearer Token**: `Authorization: Bearer <token>` Header। LeadDesk-এ নিয়মগুলো:

- Token Environment থেকে আসে, Code-এ নয়; Memory-তে থাকে শুধু Token-এর SHA-256 Hash। খোঁজা হয় Hash দিয়ে, তাই আসল Token Log বা Memory Dump-এ ফাঁস হয় না। (দীর্ঘ এলোমেলো Machine Token-এর জন্য দ্রুত Hash যথেষ্ট; মানুষের Password-এর জন্য scrypt, অধ্যায় ২০।)
- **Role**: `agent` বা `manager`।
- **Permission Matrix**, এক জায়গায় (`service.mjs`):

কাজ	Agent	Manager
Lead তৈরি	হ্যাঁ (নিজের নামে)	হ্যাঁ (কারও নামে নয়)
দেখা	নিজের ও কারও-না-দেওয়া	সব
অবস্থা বদলানো	শুধু নিজের	সব
অন্যকে দেওয়া (`assignee`)	না	হ্যাঁ
মুছে ফেলা	না	হ্যাঁ

অন্য Agent-এর Lead দেখতে চাইলে 404 (অস্তিত্বও লুকানো), দেখা যায় কিন্তু কাজের অনুমতি নেই এমন ক্ষেত্রে 403। Production-এ সাধারণত Token আসে একটি Identity Provider থেকে (OAuth 2.0, OpenID Connect), প্রায়ই JWT আকারে, মেয়াদসহ; Role-এর তথ্য তার ভেতরে। এই Demo-র স্থির Token শুধু শেখার জন্য।

২৫.৮ নির্ভরযোগ্যতা: Pagination, Idempotency, Concurrency

Cursor Pagination: `?page=5` (Offset) সরল, কিন্তু পড়ার মাঝে নতুন Lead যোগ হলে একই Lead দুবার আসে বা বাদ পড়ে, এবং বড় Offset Database-এ ধীর। **Cursor** বলে "এই ID-র পর থেকে": `GET /v1/leads?limit=20&after=L-00020`। Response-এ `next` থাকে পরের পাতার Cursor, শেষ পাতায় `null`।

Idempotency-Key: Mobile Network-এ একটি `POST` পাঠানো হলো, Server Lead তৈরিও করল, কিন্তু উত্তর পৌঁছানোর আগে সংযোগ কেটে গেল। App আবার পাঠাল—এখন দুটি Lead, এবং গ্রাহক দুজন Agent-এর কাছ থেকে দুবার ফোন পান। সমাধান: Client প্রতিটি নতুন কাজের জন্য একটি এলোমেলো Key তৈরি করে `Idempotency-Key` Header-এ পাঠায়, এবং Retry-তে **একই** Key। Server প্রথমবার Key ও ফল মনে রাখে; একই Key আবার এলে নতুন কিছু না করে আগের ফল ফেরত দেয় (Status 200)। একই Key-তে ভিন্ন Body এলে তা Client-এর Bug—409।

Optimistic Concurrency: দুজন একই Lead একসঙ্গে খুলেছেন। Rafi অবস্থা বদলে `contacted` করলেন, তারপর Manager তাঁর পুরোনো পর্দা থেকে `lost` পাঠালেন—Rafi-র পরিবর্তন নীরবে হারিয়ে যেত (**Lost Update**)। প্রতিটি Lead-এর একটি `version`; পড়ার সময় Server `ETag: "v2"` পাঠায়, হালনাগাদে Client `If-Match: "v2"` পাঠায়। Version না মিললে 412: "অন্য কেউ আগে বদলেছে, আবার পড়ে নিন"। "Optimistic" কারণ কোনো Lock নেই—সংঘাত বিরল ধরে নিয়ে শুধু হালনাগাদের মুহূর্তে যাচাই।

২৫.৯ Observability এবং Production-এর অভ্যাস

- **Request ID:** প্রতিটি অনুরোধের একটি ID (`x-request-id`), Response Header-এ, প্রতিটি Log-এ এবং প্রতিটি Error Body-তে। গ্রাহক Error-এর ID জানালে Log-এ ঠিক সেই অনুরোধ খুঁজে পাওয়া যায়।
- **Structured Log:** একটি অনুরোধ, একটি JSON লাইন: Method, Path, Status, সময়, ব্যবহারকারী। কখনো Header (Token) বা Body (ফোন নম্বর) নয়।
- **Health ও Readiness:** `/health` বলে Process জীবিত (Liveness); `/ready` বলে নতুন অনুরোধ নিতে প্রস্তুত (বন্ধ হওয়ার সময় 503)। Load Balancer ও Kubernetes-এর মতো ব্যবস্থা এগুলো দেখে Traffic পাঠায়।
- **Rate Limit:** প্রতি ব্যবহারকারী মিনিটে সর্বোচ্চ অনুরোধ, ছাড়ালে 429 ও `Retry-After`। (এখানে Memory-তে; একাধিক Server-এ Redis-এর মতো ভাগ করা Store লাগে।)
- **Configuration:** সব Setting Environment থেকে, শুরুতেই যাচাই, ভুল হলে স্পষ্ট বার্তায় থামা।
- **Graceful Shutdown:** Deploy বা Restart-এর সময় `SIGTERM` এলে: `ready` বন্ধ (নতুন Traffic আসা থামে), নতুন সংযোগ নেওয়া বন্ধ, চলমান অনুরোধ শেষ হতে দেওয়া, একটি সময়সীমার পর বাকি সংযোগ বন্ধ। নইলে প্রতিটি Deploy-এ কিছু ব্যবহারকারী আধা-সম্পূর্ণ অনুরোধ পান।
- **শুধু `127.0.0.1`:** Demo বাইরে থেকে পৌঁছানো যায় না। Production-এ Server সাধারণত একটি Reverse Proxy-র (Nginx, Cloud Load Balancer) পেছনে, যা HTTPS ও সংযোগের সীমা সামলায়।

২৫.১০ Database: Repository-র পেছনে

LeadDesk-এর Repository Memory-তে, যাতে কোনো Database ছাড়াই Project চলে ও Test হয়। কিন্তু প্রতিটি Method ইচ্ছা করে `async` এবং একটি ছোট চুক্তি মানে (`nextId` , `insert` , `get` , `replace` , `remove` , `list`) —অধ্যায় ২১-এর Repository Pattern। একটি PostgreSQL Adapter একই চুক্তি মানবে: `insert` হবে Parameterized `INSERT` , `replace(lead, expectedVersion)` হবে `UPDATE leads SET ... WHERE id = $1 AND version = $2` এবং বদলানো সারির সংখ্যা ০ হলে `false` —Optimistic Concurrency তখন Database নিজেই নিশ্চিত করে। Idempotency-র তথ্যও তখন Memory-র `Map` -এ নয়, একটি Table-এ, একটি Unique Constraint-সহ, যাতে একাধিক Server-এও কাজ করে। Service-এর একটি লাইনও বদলাতে হবে না। Database-এর বিস্তারিত—Schema, Constraint, Transaction, Index, Migration—পরের অধ্যায়ের বিষয়।

২৫.১১ Project: LeadDesk API

LeadDesk API একটি ভ্রমণ সংস্থার গ্রাহক-জিজ্ঞাসা (Lead) ব্যবস্থাপনার REST API, Express দিয়ে। এতে অনুচ্ছেদ ২৫.৩ থেকে ২৫.৯-এর প্রতিটি কৌশল প্রয়োগ ও পরীক্ষা করা হয়েছে।

> **সতর্কতা:** LeadDesk একটি **শিক্ষামূলক Demo**, Production-প্রস্তুত নয়। তথ্য ও Idempotency-র রেকর্ড Memory-তে (Restart-এ মুছে যায়); Token স্থির ও মেয়াদহীন; HTTPS নেই; Server শুধু `127.0.0.1` -এ। Production-এর জন্য Database (অধ্যায় ২৬), Identity Provider, HTTPS, ভাগ করা Rate Limit Store এবং স্বাধীন Security Review লাগবে।

File-এর কাঠামো

Folder-এর কাঠামো

```
lead-desk/
├── main.mjs                Server চালু, Demo Token, SIGINT/SIGTERM
├── package.json            express Dependency
├── src/
│   ├── config.mjs          Environment থেকে Configuration, Token-এর Hash
│   ├── errors.mjs          AppError ও Helper
│   ├── leads.mjs           Lead-এর যাচাই ও অবস্থার নিয়ম
│   ├── repository.mjs      Memory Repository (async চুক্তি)
│   ├── service.mjs         নিয়ম, অনুমতি, Idempotency, Version
│   ├── middleware.mjs      Request ID, Log, Authentication, Rate Limit
│   ├── error-handler.mjs   404 ও Error Handler
│   ├── routes.mjs          Express Router
│   ├── app.mjs             Middleware-এর ক্রম
│   └── server.mjs          HTTP Server ও Graceful Shutdown
└── tests/                  api, reliability, shutdown (+ helpers.mjs)
```

src/errors.mjs ও src/leads.mjs

errors.mjs · JavaScript — চলবে: Node.js

```
// সব প্রত্যাশিত ব্যর্থতা একটি Class-এ: HTTP Status, যন্ত্রে-পড়ার code, মানুষের বার্তা।
export class AppError extends Error {
  constructor(status, code, message, details = undefined) {
    super(message);
    this.name = 'AppError';
    this.status = status;
    this.code = code;
    this.details = details;
  }
}

export const badRequest = (message, details) =>
  new AppError(400, 'VALIDATION_FAILED', message, details);
export const unauthorized = () =>
  new AppError(401, 'UNAUTHENTICATED', 'বৈধ Bearer Token প্রয়োজন');
export const forbidden = () =>
  new AppError(403, 'FORBIDDEN', 'এই কাজের অনুমতি নেই');
export const notFound = (what = 'Resource') =>
  new AppError(404, 'NOT_FOUND', `${what} পাওয়া যায়নি`);
export const conflict = (code, message) => new AppError(409, code, message);
export const preconditionFailed = () =>
  new AppError(412, 'VERSION_MISMATCH', 'অন্য কেউ আগে বদলেছে; আবার পড়ে নিন');
```

leads.mjs · JavaScript — অংশ ১ / ২ (Part 1 of 2) — চলবে: Node.js

```
import { badRequest, conflict } from './errors.mjs';

// Lead: ভ্রমণ সংস্থার কাছে একজন সম্ভাব্য গ্রাহকের জিজ্ঞাসা।
export const STATUSES = ['new', 'contacted', 'quoted', 'won', 'lost'];
const NEXT = {
  new: ['contacted', 'lost'],
  contacted: ['quoted', 'lost'],
  quoted: ['won', 'lost'],
  won: [],
  lost: [],
};
const PHONE = /^01[3-9]\d{8}$/;

function check(problems, ok, field, message) {
  if (!ok) {
    problems.push({ field, message });
  }
}

// ...
```

leads.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function validateNewLead(body) {
  const b = typeof body === 'object' && body !== null ? body : {};
  const problems = [];
  const text = (v, max) =>
    typeof v === 'string' && v.trim().length > 0 && v.length <= max;
  check(problems, text(b.name, 80), 'name', 'নাম ১-৮০ অক্ষর');
  check(problems, typeof b.phone === 'string' && PHONE.test(b.phone), 'phone',
    '১১ অঙ্কের বাংলাদেশি মোবাইল নম্বর');
  check(problems, text(b.destination, 60), 'destination', 'গন্তব্য প্রয়োজন');
  check(problems, Number.isInteger(b.travelers) && b.travelers >= 1
    && b.travelers <= 50, 'travelers', 'যাত্রী ১ থেকে ৫০ জন');
  if (problems.length > 0) {
    throw badRequest('ইনপুট যাচাই ব্যর্থ', problems);
  }
  return {
    name: b.name.trim(), phone: b.phone, destination: b.destination.trim(),
    travelers: b.travelers,
  };
}

export function assertTransition(from, to) {
  if (!STATUSES.includes(to)) {
    throw badRequest(`অজানা অবস্থা: ${to}`);
  }
  if (!NEXT[from].includes(to)) {
    throw conflict('INVALID_TRANSITION', `${from} থেকে ${to} যাওয়া যায় না`);
  }
}
```

`validateNewLead` শুধু চারটি পরিচিত Field নেয় এবং নতুন Object তৈরি করে (Allowlist)—Body-তে `status: 'won'` বা `assignee` থাকলেও তা উপেক্ষিত। সব ভুল একসঙ্গে, Field-এর নামসহ (`details`)।

src/config.mjs ও src/repository.mjs

config.mjs · JavaScript চলবে: Node.js

```
import { createHash, randomBytes } from 'node:crypto';

export const hashToken = (token) =>
  createHash('sha256').update(token).digest('hex');

// Configuration আসে Environment থেকে; ভুল হলে শুরুতেই স্পষ্ট বার্তায় থামে।
// API_TOKENS-এর রূপ: "id:role:token,id:role:token" (role = agent বা manager)।
export function loadConfig(env = process.env) {
  const port = Number(env.PORT ?? 3025);
```

```
  if (!Number.isInteger(port) || port < 0 || port > 65535) {
    throw new Error(`PORT অবৈধ: ${env.PORT}`);
  }
  const users = new Map();
  for (const entry of (env.API_TOKENS ?? '').split(',').filter(Boolean)) {
    const [id, role, token] = entry.split(':');
    const validRole = ['agent', 'manager'].includes(role);
    if (!id || !validRole || !token || token.length < 24) {
      throw new Error(`API_TOKENS-এ অবৈধ Entry (id "${id}")`);
    }
    // Token নিজে Memory-তে নয়; শুধু তার Hash।
    users.set(hashToken(token), { id, role });
  }
  return {
    port,
    host: '127.0.0.1',
    users,
    rateLimit: { max: Number(env.RATE_LIMIT ?? 60), windowMs: 60_000 },
    shutdownGraceMs: Number(env.SHUTDOWN_GRACE_MS ?? 5_000),
  };
}

// Demo-র জন্য প্রতি চালানোয় নতুন এলোমেলো Token (Code-এ লেখা Token নয়)।
export function demoTokens() {
  const token = () => randomBytes(24).toString('base64url');
  return { manager: token(), rafi: token(), mina: token() };
}
```

repository.mjs · JavaScript — চলবে: Node.js

```
// Memory-ভিত্তিক Repository; প্রতিটি Method async, যাতে পরে Database Adapter
// (অধ্যায় ২৬) একই চুক্তিতে বসানো যায়। সব তথ্য Copy হিসেবে দেওয়া-নেওয়া।
export function createLeadRepository() {
  const leads = new Map();
  let sequence = 0;
  const copy = (lead) => structuredClone(lead);

  return {
    async nextId() {
      sequence += 1;
      return `L-${String(sequence).padStart(5, '0')}`;
    },
    async insert(lead) {
      leads.set(lead.id, copy(lead));
    },
    async get(id) {
      return leads.has(id) ? copy(leads.get(id)) : null;
    },
    // শর্তসাপেক্ষ হালনাগাদ: সংরক্ষিত version মিললে তবেই (Optimistic Concurrency)।
```

```
    async replace(lead, expectedVersion) {
      if (leads.get(lead.id)?.version !== expectedVersion) {
        return false;
      }
      leads.set(lead.id, copy(lead));
      return true;
    },
    async remove(id) {
      return leads.delete(id);
    },
    // Cursor Pagination: id-র ক্রমে, after-এর পরের limit-টি, এবং আরও আছে কি না।
    async list({ where = () => true, after = '', limit = 20 }) {
      const rows = [...leads.values()]
        .filter((lead) => lead.id > after && where(lead))
        .slice(0, limit + 1);
      const items = rows.slice(0, limit).map(copy);
      return { items, hasMore: rows.length > limit };
    },
  };
}
```

src/service.mjs

service.mjs · JavaScript (অংশ ১ / ৩ (Part 1 of 3)) চলবে: Node.js

```
import { validateNewLead, assertTransition, STATUSES } from './leads.mjs';
import {
  badRequest, forbidden, notFound, conflict, preconditionFailed,
} from './errors.mjs';

// অনুমতির নিয়ম এক জায়গায়। Agent দেখেন নিজের ও কারও-না-দেওয়া Lead;
// বদলাতে পারেন শুধু নিজেরটি। Manager সব।
const canSee = (user, lead) => user.role === 'manager'
  || lead.assignee === user.id || lead.assignee === null;
const canEdit = (user, lead) =>
  user.role === 'manager' || lead.assignee === user.id;

export function createLeadService({ repository, now = () => new Date() }) {
  const idempotency = new Map();

  async function create(user, body, idempotencyKey) {
    const input = validateNewLead(body);
    const key = idempotencyKey ? `${user.id}:${idempotencyKey}` : null;
    const fingerprint = JSON.stringify(input);
    const previous = key ? idempotency.get(key) : undefined;
    if (previous) {
      if (previous.fingerprint !== fingerprint) {
        throw conflict('IDEMPOTENCY_MISMATCH', 'একই Key, ভিন্ন অনুরোধ');
```

```
      }
      return { lead: await repository.get(previous.leadId), replayed: true };
    }
    const lead = {
      id: await repository.nextId(), ...input, status: 'new',
      assignee: user.role === 'agent' ? user.id : null,
      version: 1, createdAt: now().toISOString(),
    };
    await repository.insert(lead);
    if (key) {
      idempotency.set(key, { fingerprint, leadId: lead.id });
    }
    return { lead, replayed: false };
  }

// ...
```

service.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

  async function get(user, id) {
    const lead = await repository.get(id);
    if (!lead || !canSee(user, lead)) {
      throw notFound('Lead');
    }
    return lead;
  }

  async function list(user, { status, after = '', limit = '20' }) {
    const size = Number(limit);
    if (!Number.isInteger(size) || size < 1 || size > 100) {
      throw badRequest('limit ১ থেকে ১০০');
    }
    if (status !== undefined && !STATUSES.includes(status)) {
      throw badRequest(`অজানা status: ${status}`);
    }
    const where = (lead) =>
      canSee(user, lead) && (!status || lead.status === status);
    return repository.list({ where, after: String(after), limit: size });
  }

// ...
```

service.mjs · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

  async function update(user, id, patch, expectedVersion) {
```

```
    const lead = await get(user, id);
    if (!canEdit(user, lead)) {
      throw forbidden();
    }
    if (expectedVersion !== undefined && expectedVersion !== lead.version) {
      throw preconditionFailed();
    }
    const fields = Object.keys(patch ?? {});
    const unknown = fields.filter((k) => !['status', 'assignee'].includes(k));
    if (unknown.length > 0 || fields.length === 0) {
      throw badRequest('শুধু status বা assignee বদলানো যায়', unknown);
    }
    const next = { ...lead, version: lead.version + 1 };
    if ('status' in patch) {
      assertTransition(lead.status, patch.status);
      next.status = patch.status;
    }
    if ('assignee' in patch) {
      if (user.role !== 'manager') {
        throw forbidden();
      }
      const { assignee } = patch;
      next.assignee = typeof assignee === 'string' ? assignee : null;
    }
    if (!(await repository.replace(next, lead.version))) {
      throw preconditionFailed();
    }
    return next;
  }

  async function remove(user, id) {
    await get(user, id);
    if (user.role !== 'manager') {
      throw forbidden();
    }
    await repository.remove(id);
  }

  return { create, get, list, update, remove };
}
```

অনুমতির নিয়ম দুটি ছোট Function-এ (`canSee`, `canEdit`), এবং প্রতিটি কাজ সেগুলোর মধ্য দিয়ে যায়: তালিকার `where`-এও `canSee`, যাতে তালিকা দিয়েও অন্যের Lead বেরিয়ে না আসে। `update`-এ তিনটি স্তরের যাচাই: অনুমতি, `If-Match`-এর Version, এবং অবস্থার পরিবর্তনের নিয়ম; শেষে Repository-র শর্তসাপেক্ষ `replace`, যা দুটি একসঙ্গে আসা হালনাগাদের মধ্যে দ্বিতীয়টিকে ধরে।

src/middleware.mjs ও src/error-handler.mjs

middleware.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
import { randomUUID } from 'node:crypto';
import { AppError, unauthorized } from './errors.mjs';
import { hashToken } from './config.mjs';

// প্রতিটি অনুরোধের একটি ID: Response Header-এ ও প্রতিটি Log-এ।
export function requestId(req, res, next) {
  const incoming = req.get('x-request-id');
  req.id = incoming && /^[\w-]{1,64}$/.test(incoming) ? incoming : randomUUID();
  res.set('x-request-id', req.id);
  next();
}

// Structured Log: একটি অনুরোধ, একটি JSON লাইন। Header বা Body Log হয় না।
export const requestLogger = (log) => (req, res, next) => {
  const started = process.hrtime.bigint();
  res.on('finish', () => log({
    requestId: req.id, method: req.method, path: req.path,
    status: res.statusCode,
    user: req.user?.id ?? null,
    ms: Number(process.hrtime.bigint() - started) / 1e6,
  }));
  next();
};

// ...
```

middleware.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export const authenticate = (users) => (req, res, next) => {
  const [scheme, token] = (req.get('authorization') ?? '').split(' ');
  const valid = scheme === 'Bearer' && Boolean(token);
  const user = valid ? users.get(hashToken(token)) : undefined;
  if (!user) {
    next(unauthorized());
    return;
  }
  req.user = user;
  next();
};

// প্রতি ব্যবহারকারী নির্দিষ্ট সময়-জানালায় সর্বোচ্চ max অনুরোধ।
export function rateLimit({ max, windowMs, now = Date.now }) {
  const windows = new Map();
  return (req, res, next) => {
```

```
    const key = req.user?.id ?? req.ip;
    const current = windows.get(key);
    const window = current && now() - current.start < windowMs
      ? current : { start: now(), count: 0 };
    window.count += 1;
    windows.set(key, window);
    if (window.count > max) {
      const retry = Math.ceil((window.start + windowMs - now()) / 1000);
      res.set('retry-after', String(retry));
      next(new AppError(429, 'RATE_LIMITED', 'অনেক অনুরোধ; কিছুক্ষণ পরে চেষ্টা করুন'));
      return;
    }
    next();
  };
}
```

error-handler.mjs · JavaScript চলবে: Node.js

```
import { AppError, notFound } from './errors.mjs';

export const notFoundHandler = (req, res, next) => next(notFound('পথ'));

// Express চারটি Parameter-এর Middleware-কে Error Handler হিসেবে চেনে।
// সব Error একই আকারে: { error: { code, message, details?, requestId } }।
export const errorHandler = (log) => (err, req, res, next) => {
  let error = err;
  if (err.type === 'entity.parse.failed') {
    error = new AppError(400, 'INVALID_JSON', 'JSON পড়া যায়নি');
  } else if (err.type === 'entity.too.large') {
    error = new AppError(413, 'PAYLOAD_TOO_LARGE', 'অনুরোধ খুব বড়');
  } else if (!(err instanceof AppError)) {
    // অপ্রত্যাশিত: বিস্তারিত শুধু Server Log-এ, ব্যবহারকারী পান সাধারণ বার্তা।
    log({ requestId: req.id, level: 'error', message: err.message });
    error = new AppError(500, 'INTERNAL_ERROR', 'Server-এ সমস্যা হয়েছে');
  }
  if (res.headersSent) {
    next(err);
    return;
  }
  const { status, code, message, details } = error;
  const body = { code, message, details, requestId: req.id };
  res.status(status).json({ error: body });
};
```

Client চাইলে নিজের `x-request-id` পাঠাতে পারেন (যেমন Frontend-এর একটি ID), কিন্তু শুধু নিরাপদ অক্ষরে ও ৬৪ অক্ষর পর্যন্ত—নইলে Log-এ ক্ষতিকর লেখা ঢুকতে পারত। `express.json()`-এর দুটি নিজস্ব Error (`entity.parse.failed`, `entity.too.large`) Error Handler-এ নিজস্ব Code পায়।

src/routes.mjs, src/app.mjs ও src/server.mjs

routes.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
import express from 'express';

// Express 4 async Function-এর Rejected Promise নিজে ধরে না; এই Wrapper ধরে
// next(error)-এ পাঠায় (Express 5-এ এটি অন্তর্নির্মিত)।
const wrap = (handler) => (req, res, next) =>
  Promise.resolve(handler(req, res, next)).catch(next);

const etag = (lead) => `"v${lead.version}"`;
const versionFrom = (header) => {
  const match = /^"v(\d+)"$/.exec(header ?? '');
  return match ? Number(match[1]) : undefined;
};

export function leadRoutes(service) {
  const router = express.Router();

  router.post('/', wrap(async (req, res) => {
    const key = req.get('idempotency-key');
    const { lead, replayed } = await service.create(req.user, req.body, key);
    res.status(replayed ? 200 : 201)
      .location(`/v1/leads/${lead.id}`).set('etag', etag(lead)).json(lead);
  }));

// ...
```

routes.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

  router.get('/', wrap(async (req, res) => {
    const page = await service.list(req.user, req.query);
    const last = page.items.at(-1);
    res.json({ ...page, next: page.hasMore && last ? last.id : null });
  }));

  router.get('/:id', wrap(async (req, res) => {
    const lead = await service.get(req.user, req.params.id);
    res.set('etag', etag(lead)).json(lead);
  }));

  router.patch('/:id', wrap(async (req, res) => {
    const expected = versionFrom(req.get('if-match'));
    const { user, params, body } = req;
    const lead = await service.update(user, params.id, body, expected);
    res.set('etag', etag(lead)).json(lead);
  }));
```

```
  router.delete('/:id', wrap(async (req, res) => {
    await service.remove(req.user, req.params.id);
    res.status(204).end();
  }));

  return router;
}
```

app.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';
import {
  requestId, requestLogger, authenticate, rateLimit,
} from './middleware.mjs';
import { notFoundHandler, errorHandler } from './error-handler.mjs';
import { leadRoutes } from './routes.mjs';

// Middleware-এর ক্রমই নিরাপত্তা: ID ও Log সবার আগে, তারপর Body-র সীমা,
// তারপর Health (Token ছাড়া), তারপর Authentication, সীমা, Route, শেষে Error।
export function createApp(deps) {
  const { config, service, log = () => {}, state = { ready: true } } = deps;
  const app = express();
  app.disable('x-powered-by');
  app.use(requestId);
  app.use(requestLogger(log));
  app.use(express.json({ limit: '10kb' }));

  app.get('/health', (req, res) => res.json({ status: 'ok' }));
  app.get('/ready', (req, res) => {
    res.status(state.ready ? 200 : 503).json({ ready: state.ready });
  });

  app.use('/v1', authenticate(config.users), rateLimit(config.rateLimit));
  app.use('/v1/leads', leadRoutes(service));

  app.use(notFoundHandler);
  app.use(errorHandler(log));
  return app;
}
```

server.mjs · JavaScript — চলবে: Node.js

```
import { createServer } from 'node:http';
import { createApp } from './app.mjs';
import { createLeadService } from './service.mjs';
import { createLeadRepository } from './repository.mjs';

// সব অংশ জোড়া লাগিয়ে Server চালু; shutdown() নতুন সংযোগ বন্ধ করে,
```

```
// চলমান অনুরোধ শেষ হতে দেয়, এবং সময়সীমার পরে বাকি সংযোগ কেটে দেয়।
export async function startServer(config, { log = () => {}, service } = {}) {
  const state = { ready: true };
  const leads = service
    ?? createLeadService({ repository: createLeadRepository() });
  const app = createApp({ config, service: leads, log, state });
  const server = createServer(app);
  await new Promise((resolve) => {
    server.listen(config.port, config.host, resolve);
  });

  let closing = null;
  function shutdown() {
    closing ??= new Promise((resolve) => {
      state.ready = false;
      server.close(resolve);
      server.closeIdleConnections();
      const force = () => server.closeAllConnections();
      setTimeout(force, config.shutdownGraceMs).unref();
    });
    return closing;
  }

  const { port } = server.address();
  return { url: `http://${config.host}:${port}`, shutdown, state };
}
```

`createApp`-এর ক্রম পড়ুন যেন একটি অনুরোধ ওপর থেকে নিচে যাচ্ছে: ID, Log, Body-র সীমা, Token-ছাড়া Health, তারপর `/v1`-এর সব কিছুর আগে Authentication ও Rate Limit, তারপর Route, এবং যা মেলেনি তার জন্য 404, শেষে Error Handler। `shutdown()` একবারই চলে (`closing ??=`), Node.js-এর `server.close` (নতুন সংযোগ বন্ধ, চলমানগুলো শেষ হলে Callback), `closeIdleConnections` (অপেক্ষারত Keep-alive সংযোগ বন্ধ) এবং সময়সীমার পরে `closeAllConnections`।

main.mjs

main.mjs · JavaScript চলবে: Node.js

```
import { loadConfig, demoTokens } from './src/config.mjs';
import { startServer } from './src/server.mjs';

// API_TOKENS না দিলে Demo-র জন্য প্রতি চালানোয় নতুন Token (শুধু এই Terminal-এ দেখায়)।
let env = process.env;
if (!env.API_TOKENS) {
  const t = demoTokens();
  const tokens = [`manager:manager:${t.manager}`, `rafi:agent:${t.rafi}`,
    `mina:agent:${t.mina}`];
  env = { ...env, API_TOKENS: tokens.join(',') };
  console.log(`Demo Token (manager): ${t.manager}`);
```

```
  console.log(`Demo Token (rafi):    ${t.rafi}`);
  console.log(`Demo Token (mina):    ${t.mina}`);
}

const config = loadConfig(env);
const log = (entry) => console.log(JSON.stringify(entry));
const server = await startServer(config, { log });
console.log(`LeadDesk API (শিক্ষামূলক Demo, শুধু এই কম্পিউটারে): ${server.url}`);

for (const signal of ['SIGINT', 'SIGTERM']) {
  process.once(signal, async () => {
    console.log(`${signal}: নতুন অনুরোধ বন্ধ, চলমান অনুরোধ শেষ হওয়ার অপেক্ষা...`);
    await server.shutdown();
    console.log('বন্ধ হয়েছে।');
    process.exit(0);
  });
}
```

উদাহরণ অনুরোধ

`node main.mjs` চালালে Terminal-এ তিনটি Demo Token দেখায়। নিচের Command-গুলো উদাহরণ (Windows PowerShell-এ `curl.exe`); `<TOKEN>`-এর জায়গায় একটি Token বসান। এই বইয়ের QA-তে এগুলো হাতে চালানো হয়নি; একই অনুরোধগুলো স্বয়ংক্রিয় Test-এ যাচাই করা হয়েছে।

Terminal

```
curl -i -X POST http://127.0.0.1:3025/v1/leads \
  -H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \
  -H "Idempotency-Key: form-7f3a" \
  -d '{"name":"Karim Uddin","phone":"01712345678",
       "destination":"Sylhet","travelers":4}'
curl -i "http://127.0.0.1:3025/v1/leads?status=new&limit=20" \
  -H "Authorization: Bearer <TOKEN>"
curl -i -X PATCH http://127.0.0.1:3025/v1/leads/L-00001 \
  -H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \
  -H 'If-Match: "v1"' -d '{"status":"contacted"}'
```

Test

api.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Object-level Authorization: অন্য Agent-এর Lead দেখা যায় না', async (t) => {
  const { call } = await startTestServer(t);
  await call('POST', '/v1/leads', { as: 'rafi', body: newLead() });
  await call('POST', '/v1/leads',
    { as: 'manager', body: newLead({ name: 'Unassigned' }) });
```

```
  const other = await call('GET', '/v1/leads/L-00001', { as: 'mina' });
  assert.equal(other.status, 404);
  const minaList = await call('GET', '/v1/leads', { as: 'mina' });
  assert.deepEqual(minaList.body.items.map((l) => l.id), ['L-00002']);
  const assign = await call('PATCH', '/v1/leads/L-00002',
    { as: 'mina', body: { assignee: 'mina' } });
  assert.equal(assign.status, 403);
});

// ...
```

reliability.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Idempotency-Key: একই অনুরোধ দুবার, একটিই Lead', async (t) => {
  const { call } = await startTestServer(t);
  const headers = { 'idempotency-key': 'form-7f3a' };
  const send = () =>
    call('POST', '/v1/leads', { as: 'rafi', body: newLead(), headers });
  const first = await send();
  const retry = await send();
  assert.deepEqual([first.status, retry.status], [201, 200]);
  assert.equal(retry.body.id, first.body.id);
  const list = await call('GET', '/v1/leads', { as: 'rafi' });
  assert.equal(list.body.items.length, 1);
  const changed = await call('POST', '/v1/leads',
    { as: 'rafi', body: newLead({ travelers: 9 }), headers });
  assert.equal(changed.status, 409);
  assert.equal(changed.body.error.code, 'IDEMPOTENCY_MISMATCH');
});

// ...
```

shutdown.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('shutdown(): চলমান অনুরোধ সম্পূর্ণ, তারপর বন্ধ', async (t) => {
  let release;
  const gate = new Promise((resolve) => {
    release = resolve;
  });
  // নকল Service: list ধীর, যতক্ষণ না Test "release" করে।
  const service = { list: async () => {
    await gate;
    return { items: [], hasMore: false };
  } };
  const { server } = await startTestServer(t, { service });
```

```
  const auth = { authorization: `Bearer ${TOKENS.rafi}` };

  const inFlight = fetch(`${server.url}/v1/leads`, { headers: auth });
  await new Promise((resolve) => setTimeout(resolve, 100));
  const closed = server.shutdown();
  assert.equal(server.state.ready, false);

  const fresh = { headers: { connection: 'close' } };
  await assert.rejects(fetch(`${server.url}/health`, fresh));
  release();
  const response = await inFlight;
  assert.equal(response.status, 200);
  assert.deepEqual(await response.json(),
    { items: [], hasMore: false, next: null });
  await closed;
});

// ...
```

Shutdown Test-এর কৌশলটি লক্ষ করুন: একটি নকল Service, যার `list` একটি Promise-এর জন্য অপেক্ষা করে। Test একটি অনুরোধ পাঠায় (যা ঝুলে থাকে), তারপর `shutdown()` ডাকে, প্রমাণ করে `ready` বন্ধ এবং নতুন সংযোগ প্রত্যাখ্যাত, তারপর "ছেড়ে দেয়"—এবং পুরোনো অনুরোধটি সম্পূর্ণ উত্তর পায়। আসল সময়ের ওপর নির্ভরতা নেই (অধ্যায় ২২)।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd lead-desk
npm install
npm test
npm start
```

`npm install` Express আনে। `npm test` ১০টি Test চালায়: `api` ৪ (REST-এর পূর্ণ চক্র, প্রতিটি Error Status ও Body-র আকার, Object-level Authorization, Cursor Pagination ও Filter), `reliability` ৪ (Idempotency-Key, `If-Match` ও 412, Rate Limit ও `Retry-After`, Request ID ও Log), এবং `shutdown` ২ (চলমান অনুরোধসহ Graceful Shutdown, Configuration-এর ভুল); শেষে `# pass 10`, `# fail 0`। প্রতিটি Test নিজের Server চালায়, এলোমেলো Port-এ, শুধু `127.0.0.1`-এ। `npm start` Port ৩০২৫-এ চালু করে; Ctrl+C চাপলে Graceful Shutdown-এর বার্তা দেখায়।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find package 'express'`	`npm install` হয়নি	Project Folder-এ `npm install`
অনুরোধ ঝুলে থাকে, উত্তর আসে না	`async` Route-এ `wrap` নেই	`wrap` বা Express 5
সব অনুরোধে 401	Token-এর আগে `Bearer` নেই, বা ভুল Token	Header দেখুন
413	Body ১০ KB-র বেশি	ছোট করুন বা সীমা বিবেচনা
`EADDRINUSE`	Port ব্যস্ত	`PORT=3026 npm start`
Ctrl+C-এর পর দেরিতে বন্ধ	চলমান বা Keep-alive সংযোগ	স্বাভাবিক; সর্বোচ্চ `SHUTDOWN_GRACE_MS`

২৫.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Express 4-এ `async` Route, `wrap` ছাড়া	অনুরোধ ঝুলে থাকে, Process বন্ধ হতে পারে	`wrap` বা Express 5
Route-এ ব্যবসার নিয়ম ও SQL	পরীক্ষা ও পুনর্ব্যবহার কঠিন	Service ও Repository
Middleware-এর ভুল ক্রম	Authentication-এর আগেই Route	ক্রম নিয়ে Test
`req.body`-কে সরাসরি সংরক্ষণ	অনাকাঙ্ক্ষিত Field (`status`, `role`)	Allowlist যাচাই
সব ব্যর্থতায় 500 বা 200	Client বুঝতে পারে না	সঠিক Status ও একই আকারের Error
Offset Pagination বড় তালিকায়	ধীর, ডুপ্লিকেট বা বাদ	Cursor
Retry-তে নতুন Key	ডুপ্লিকেট রেকর্ড	একই `Idempotency-Key`
Version ছাড়া হালনাগাদ	Lost Update	`ETag` ও `If-Match`
Log-এ Header বা Body	Token ও ব্যক্তিগত তথ্য ফাঁস	শুধু নির্বাচিত Field
`process.exit()` সঙ্গে সঙ্গে	চলমান অনুরোধ কেটে যায়	Graceful Shutdown
Code-এ Token বা Password	Git-এ ফাঁস	Environment, Secret Manager

২৫.১৩ Debugging নির্দেশিকা

১. **Request ID দিয়ে শুরু করুন:** Error Body-র `requestId` দিয়ে Log-এ ঠিক সেই অনুরোধের লাইন।

২. `curl -i`: Status ও প্রতিটি Header দেখায় (`ETag`, `Retry-After`, `Location`)।

৩. **ঝুলে থাকা অনুরোধ:** কোনো Middleware `next()` ডাকেনি বা উত্তর দেয়নি, অথবা `async` Error ধরা পড়েনি।

৪. **401 বনাম 403 বনাম 404:** পরিচয়, অনুমতি, না অস্তিত্ব—অনুচ্ছেদ ২৫.৩-এর টেবিল।

৫. `node --inspect main.mjs`: Chrome DevTools-এ Breakpoint দিয়ে Server-এর Code ধাপে ধাপে চালান।

৬. **একটি অংশ আলাদা করুন:** Service-কে সরাসরি ডেকে দেখুন—সমস্যা HTTP-এ, না নিয়মে।

২৫.১৪ পেশাদার অভ্যাস

- **প্রতিটি Route পাতলা,** নিয়ম Service-এ, সংরক্ষণ Repository-তে।
- **প্রতিটি অনুরোধের Body Allowlist দিয়ে যাচাই,** Server-এ, সব সময়।
- **একই আকারের Error,** Request ID-সহ।
- **অনুমতির নিয়ম এক জায়গায়,** এবং প্রতিটি Role ও প্রতিটি কাজের জন্য Test।
- `POST`-এ **Idempotency-Key, হালনাগাদে Version।**
- **Health, Readiness ও Graceful Shutdown প্রথম দিন থেকে।**
- **Configuration Environment থেকে, শুরুতেই যাচাই।**
- **API-র সংস্করণ URL-এ (`/v1`),** এবং পরিবর্তন লিখিত (OpenAPI Specification)।

২৫.১৫ অনুশীলনী

প্রাথমিক স্তর

১. Express-এ `GET /health` লিখুন, যা `{ status: 'ok', uptime: <সেকেন্ড> }` দেয়।

২. নিচের প্রতিটির জন্য সঠিক Status লিখুন: (ক) নতুন Lead তৈরি, (খ) Token নেই, (গ) Agent মুছতে চাইছেন, (ঘ) একই Lead-এ পুরোনো Version।

৩. Safe ও Idempotent-এর পার্থক্য উদাহরণসহ লিখুন।

৪. একটি Middleware লিখুন, যা প্রতিটি Response-এ `x-response-time` Header বসায়।

৫. Express 4-এ `async` Route-এ Error কেন বিশেষ যত্ন চায়? দুই বাক্যে।

মধ্যম স্তর

৬. LeadDesk-এ `GET /v1/leads?destination=sylhet` Filter যোগ করুন (ছোট-বড় হাতের পার্থক্য ছাড়া), এবং একটি Test লিখুন।

৭. একটি `requireRole('manager')` Middleware লিখুন এবং একটি Route-এ ব্যবহার করুন।

৮. একটি ৪০০ ms-এর সময়সীমার Middleware লিখুন, যা ধীর Route-এ 503 দেয়।

৯. Cursor Pagination-এ ক্লায়েন্টের দিক থেকে সব পাতা পড়ার একটি Loop লিখুন (LeadDesk-এর বিরুদ্ধে)।

চ্যালেঞ্জ স্তর

১০. Idempotency-র রেকর্ড ২৪ ঘণ্টা পর মুছে যাওয়ার নকশা করুন (ঘড়ি Inject), এবং Test।

১১. Lead-এর জন্য একটি `POST /v1/leads/:id/notes` Sub-resource নকশা করুন: URL, Status, অনুমতি, এবং Service-এর Method।

১২. LeadDesk-এর Repository-র জন্য একটি `node:sqlite` Adapter লিখুন, যা একই চুক্তি মানে, এবং `replace`-এর Optimistic Concurrency SQL-এ প্রয়োগ করুন।

২৫.১৬ অনুশীলনীর সমাধান

সমাধান ১

health.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const app = express();
const started = Date.now();
app.get('/health', (req, res) => {
  res.json({ status: 'ok', uptime: Math.floor((Date.now() - started) / 1000) });
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const response = await fetch(`http://127.0.0.1:${server.address().port}/health`);
const body = await response.json();
console.log(response.status, body.status, Number.isInteger(body.uptime));
server.close();
```

Output

```
200 ok true
```

সমাধান ২ — (ক) 201, সঙ্গে `Location: /v1/leads/<id>`; (খ) 401; (গ) 403—পরিচয় আছে, Lead দেখাও যায়, কিন্তু মুছে ফেলার অনুমতি নেই; (ঘ) 412 (`If-Match` মেলেনি)।

সমাধান ৩ — Safe: Server-এর অবস্থা বদলায় না (`GET /v1/leads`)। Idempotent: একবার বা বহুবার পাঠালে একই ফল— `DELETE /v1/leads/L-1` দুবার পাঠালেও Lead একবারই মুছে যায় (দ্বিতীয়বার 404, কিন্তু অবস্থা একই)। সব Safe Method Idempotent, কিন্তু সব Idempotent Method Safe নয় (`DELETE`, `PUT`)। `POST` কোনোটিই নয়, তাই `Idempotency-Key` লাগে।

সমাধান ৪

response-time.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const responseTime = (req, res, next) => {
  const started = process.hrtime.bigint();
  const original = res.writeHead;
  res.writeHead = function writeHead(...args) {
    const ms = Number(process.hrtime.bigint() - started) / 1e6;
    res.setHeader('x-response-time', `${ms.toFixed(1)}ms`);
    return original.apply(this, args);
  };
  next();
};

const app = express();
app.use(responseTime);
app.get('/', (req, res) => res.json({ ok: true }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const response = await fetch(`http://127.0.0.1:${server.address().port}/`);
console.log(/^\d+\.\dms$/.test(response.headers.get('x-response-time')));
server.close();
```

Output

```
true
```

Header Response পাঠানোর ঠিক আগে বসাতে হয়; `finish` Event-এ বসালে দেরি হয়ে যেত, কারণ Header ততক্ষণে চলে গেছে। তাই `writeHead`-কে মুড়ে দেওয়া। সময়ের মান প্রতিবার আলাদা, তাই শুধু আকার যাচাই।

সমাধান ৫ — Express 4 একটি `async` Function-এর Rejected Promise ধরে না, তাই Error Handler-এ পৌঁছায় না: Client উত্তর পান না, এবং `unhandledRejection` Node.js-এর Default আচরণে পুরো Process বন্ধ করে। `wrap` (বা Express 5) Rejection-কে `next(error)`-এ পাঠায়।

সমাধান ৬ — `service.mjs`-এর `list`-এ: `const place = destination?.toLowerCase();` এবং `where`-এ `&& (!place || lead.destination.toLowerCase().includes(place))`। Test (`api.test.mjs`-এর ধাঁচে): দুটি ভিন্ন গন্তব্যের Lead তৈরি, তারপর `GET /v1/leads?destination=SYL` শুধু Sylhet-এরটি দেয়। মনে রাখুন, Query সব সময় লেখা: দৈর্ঘ্যের সীমা দিন (যেমন ৬০), এবং Database-এ গেলে Parameterized `LIKE` (অধ্যায় ২০-এর অনুশীলনী)।

সমাধান ৭

require-role.mjs · JavaScript চলবে: Node.js

```
import express from 'express';

const requireRole = (role) => (req, res, next) => {
  if (req.user?.role !== role) {
    res.status(403).json({ error: { code: 'FORBIDDEN' } });
    return;
  }
  next();
};

const app = express();
app.use((req, res, next) => {
  req.user = { id: 'x', role: req.get('x-demo-role') };
  next();
});
app.delete('/v1/leads/:id', requireRole('manager'),
  (req, res) => res.status(204).end());

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const url = `http://127.0.0.1:${server.address().port}/v1/leads/L-1`;
for (const role of ['agent', 'manager']) {
  const headers = { 'x-demo-role': role };
  const response = await fetch(url, { method: 'DELETE', headers });
  console.log(role, response.status);
}
server.close();
```

Output

```
agent 403
manager 204
```

এখানে `x-demo-role` Header শুধু প্রদর্শনের জন্য, Authentication-এর বদলে; বাস্তবে Role আসে যাচাই করা Token থেকে, কখনো Client-এর পাঠানো Header থেকে নয়। Route-স্তরের Role যাচাই সহজ ও স্পষ্ট, কিন্তু মালিকানার নিয়ম (Agent শুধু নিজের Lead) Service-এই থাকা উচিত।

সমাধান ৮

timeout.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const timeout = (ms) => (req, res, next) => {
  const timer = setTimeout(() => {
    if (!res.headersSent) {
      res.status(503).json({ error: { code: 'TIMEOUT' } });
    }
  }, ms);
  res.on('finish', () => clearTimeout(timer));
  res.on('close', () => clearTimeout(timer));
  next();
};

const app = express();
app.use(timeout(400));
app.get('/fast', (req, res) => res.json({ ok: true }));
app.get('/slow', (req, res) => {
  setTimeout(() => {
    if (!res.headersSent) {
      res.json({ ok: true });
    }
  }, 1_000);
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
for (const path of ['/fast', '/slow']) {
  console.log(path, (await fetch(base + path)).status);
}
server.closeAllConnections();
server.close();
```

Output

```
/fast 200
/slow 503
```

৪০০ ও ১,০০০ ms-এর পার্থক্য বড়, তাই ফল স্থির। লক্ষ করুন, 503 পাঠানোর পরও ধীর কাজটি পেছনে চলতেই থাকে (`headersSent` যাচাই তাই জরুরি)। আসল কাজ থামাতে `AbortSignal` কাজটির ভেতরে পৌঁছে দিতে হয় (যেমন Database Query বাতিল)।

সমাধান ৯

all-pages.mjs · JavaScript — চলবে: Node.js

```
import { loadConfig } from '../../project/lead-desk/src/config.mjs';
import { startServer } from '../../project/lead-desk/src/server.mjs';

const token = 'reader-token-for-the-pagination-demo';
const config = loadConfig({ PORT: '0', API_TOKENS: `boss:manager:${token}` });
const server = await startServer(config);
const headers = {
  authorization: `Bearer ${token}`, 'content-type': 'application/json',
};
for (let i = 1; i <= 7; i += 1) {
  const body = JSON.stringify({
    name: `Guest ${i}`, phone: '01812345678',
    destination: 'Bandarban', travelers: i,
  });
  await fetch(`${server.url}/v1/leads`, { method: 'POST', headers, body });
}

const ids = [];
let after = '';
let pages = 0;
do {
  const page = await (await fetch(`${server.url}/v1/leads?limit=3&after=${after}`,
    { headers })).json();
  ids.push(...page.items.map((lead) => lead.id));
  after = page.next;
  pages += 1;
} while (after);
console.log(pages, ids.length, ids.at(-1));
await server.shutdown();
```

Output

```
3 7 L-00007
```

Client শুধু `next` অনুসরণ করে; Cursor-এর ভেতরের গঠন তার জানার দরকার নেই, তাই Server পরে Cursor-এর রূপ বদলাতে পারে (যেমন তারিখ ও ID একসঙ্গে)।

সমাধান ১০ — `createLeadService`-এ `now` আগে থেকেই Inject করা; Idempotency-র রেকর্ডে `createdAt: now().getTime()` রাখুন, এবং খোঁজার সময় `now() - previous.createdAt > 24 * 3600 * 1000` হলে রেকর্ডটি মুছে নতুন অনুরোধ হিসেবে চালান। একটি পরিষ্কারের কাজ (`setInterval`, Server বন্ধে `clearInterval`) পুরোনো রেকর্ড মুছে Memory সীমিত রাখে। Test-এ একটি নকল ঘড়ি: প্রথম অনুরোধ, ঘড়ি ২৪ ঘণ্টা + ১ ms এগিয়ে, একই Key—এবার 201 ও নতুন ID। Database-এ এটি একটি `expires_at` Column ও নিয়মিত `DELETE`।

সমাধান ১১ — URL: `POST /v1/leads/:id/notes` (Note Lead-এর অংশ, তাই Sub-resource); সফল হলে 201 ও `Location: /v1/leads/:id/notes/:noteId`; Lead দেখা না গেলে 404; লেখা না থাকলে বা ১,০০০ অক্ষরের বেশি হলে 400। অনুমতি: Lead যিনি দেখতে পারেন, তিনিই Note যোগ করতে পারেন (`canSee`), কারণ Note ইতিহাস যোগ করে, কিছু বদলায় না। Service: `addNote(user, leadId, text)` — `get(user, leadId)` দিয়ে অনুমতি, তারপর `{ id, author: user.id, text, createdAt }` যোগ। Note কখনো সম্পাদনা নয়, শুধু যোগ (Audit-এর জন্য)।

সমাধান ১২

sqlite-repository.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

export function createSqliteLeadRepository(file = ':memory:') {
  const db = new DatabaseSync(file);
  db.exec(`CREATE TABLE IF NOT EXISTS leads (
    id TEXT PRIMARY KEY, version INTEGER NOT NULL, data TEXT NOT NULL)`);
  const sql = {
    count: db.prepare('SELECT COUNT(*) AS n FROM leads'),
    insert: db.prepare('INSERT INTO leads (id, version, data) VALUES (?, ?, ?)'),
    get: db.prepare('SELECT data FROM leads WHERE id = ?'),
    replace: db.prepare(
      'UPDATE leads SET version = ?, data = ? WHERE id = ? AND version = ?'),
    remove: db.prepare('DELETE FROM leads WHERE id = ?'),
    after: db.prepare('SELECT data FROM leads WHERE id > ? ORDER BY id'),
  };
  return {
    async nextId() {
      return `L-${String(sql.count.get().n + 1).padStart(5, '0')}`;
    },
    async insert(lead) {
      sql.insert.run(lead.id, lead.version, JSON.stringify(lead));
    },
    async get(id) {
      const row = sql.get.get(id);
      return row ? JSON.parse(row.data) : null;
    },
```

```
    async replace(lead, expected) {
      const data = JSON.stringify(lead);
      const { changes } = sql.replace.run(lead.version, data, lead.id, expected);
      return changes === 1;
    },
    async remove(id) {
      return sql.remove.run(id).changes === 1;
    },
    async list({ where = () => true, after = '', limit = 20 }) {
      const rows = sql.after.all(after)
        .map((r) => JSON.parse(r.data)).filter(where);
      return { items: rows.slice(0, limit), hasMore: rows.length > limit };
    },
  };
}

// ...
```

sqlite-repository.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

const repo = createSqliteLeadRepository();
const lead = { id: await repo.nextId(), name: 'Mina', status: 'new', version: 1 };
await repo.insert(lead);
const first = await repo.replace({ ...lead, status: 'contacted', version: 2 }, 1);
const stale = await repo.replace({ ...lead, status: 'lost', version: 2 }, 1);
console.log(first, stale, (await repo.get('L-00001')).status);
```

Output

```
true false contacted
```

`UPDATE ... WHERE id = ? AND version = ?`: Version মিললে একটি সারি বদলায় (`changes === 1`), না মিললে শূন্য—দ্বিতীয় হালনাগাদটি ধরা পড়েছে। এই Adapter-এর দুটি দুর্বলতা জেনে রাখুন: `nextId` সারির সংখ্যা গুনে ID দেয়, তাই মুছে ফেলার পর পুরোনো ID আবার আসতে পারে (অধ্যায় ২১-এর ত্রুটি!), এবং `list` সব সারি পড়ে Memory-তে Filter করে। পরের অধ্যায়ে Database-এর নিজস্ব হাতিয়ার— `AUTOINCREMENT` বা Sequence, `WHERE`-এ Filter, Index—দিয়ে এগুলোর সমাধান।

২৫.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. নতুন Resource সফলভাবে তৈরি হলে সঠিক Status?

ক) 200

খ) 201

গ) 204

ঘ) 302

প্রশ্ন ২. Express-এ Error-handling Middleware কীভাবে চেনা যায়?

ক) চারটি Parameter: `(err, req, res, next)`

খ) নাম `errorHandler`

গ) সবার আগে `app.use`

ঘ) `async` Function

প্রশ্ন ৩. Express 4-এ `async` Route-এ ধরা-না-পড়া Error-এর ফল?

ক) স্বয়ংক্রিয় 500

খ) স্বয়ংক্রিয় 404

গ) Client উত্তর পান না এবং `unhandledRejection` ঘটে

ঘ) কিছুই না

প্রশ্ন ৪. Token থাকা একজন Agent অন্য Agent-এর Lead মুছতে চাইলে LeadDesk কী দেয়?

ক) 401

খ) 500

গ) 200

ঘ) 404, কারণ সেই Lead তাঁর জন্য দৃশ্যমানই নয়

প্রশ্ন ৫. `Idempotency-Key`-এর উদ্দেশ্য?

ক) একই `POST` Retry-তে ডুপ্লিকেট রেকর্ড তৈরি না হওয়া

খ) Authentication

গ) Cache

ঘ) Rate Limit

প্রশ্ন ৬. `If-Match: "v1"` পাঠানো হালনাগাদে Server-এর Version ২ হলে?

ক) 200

খ) 412

গ) 409

ঘ) 404

প্রশ্ন ৭. Offset-এর বদলে Cursor Pagination-এর সুবিধা?

ক) কম Code

খ) Token লাগে না

গ) পড়ার মাঝে নতুন সারি যোগ হলেও ডুপ্লিকেট বা বাদ পড়ে না, এবং বড় তালিকায় দ্রুত

ঘ) সব পাতা একসঙ্গে

প্রশ্ন ৮. `/ready` Endpoint কখন 503 দেয়?

ক) কখনো না

খ) Token ভুল হলে

গ) Database-এ তথ্য না থাকলে

ঘ) Server বন্ধ হওয়ার প্রক্রিয়ায়, যাতে নতুন Traffic না আসে

প্রশ্ন ৯. Log-এ কী রাখা উচিত নয়?

ক) `Authorization` Header ও ব্যক্তিগত তথ্য

খ) Request ID

গ) Status Code

ঘ) সময়

প্রশ্ন ১০. Route, Service ও Repository আলাদা রাখার মূল লাভ?

ক) দ্রুত HTTP

খ) ব্যবসার নিয়ম HTTP ও Database থেকে স্বাধীন, তাই আলাদা Test ও বদলানো যায়

গ) কম File

ঘ) Express লাগে না

২৫.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	সঙ্গে `Location` Header।
২	ক	Express Parameter-এর সংখ্যা দেখে চেনে।
৩	গ	উদাহরণ ০৩-এ দেখানো; সমাধান `wrap` বা Express 5।
৪	ঘ	অস্তিত্ব লুকানো; নিজের দেখা Lead মুছতে চাইলে 403।
৫	ক	Retry-তে একই Key, Server আগের ফল ফেরত দেয়।
৬	খ	Precondition Failed: আবার পড়ে নিন।
৭	গ	Cursor বলে "এই ID-র পর থেকে", Offset নয়।
৮	ঘ	Load Balancer Traffic সরিয়ে নেয়।
৯	ক	Log-ও ফাঁস হয়; তাই শুধু নির্বাচিত Field।
১০	খ	Service Express ছাড়াই Test করা যায়।

২৫.১৯ অধ্যায়ের সারসংক্ষেপ

- Server-এ প্রতিটি অনুরোধ অবিশ্বস্ত; একটি ভুল সবার তথ্যে প্রভাব ফেলে।
- REST: Resource-ভিত্তিক URL, অর্থপূর্ণ Method ও Status, `/v1`-এ সংস্করণ।
- Express: Route, Router, Middleware-এর ক্রম; চার-Parameter-এর Error Handler; Express 4-এ `async` Route-এর জন্য `wrap`।
- Route পাতলা, নিয়ম Service-এ, সংরক্ষণ Repository-তে (`async` চুক্তি)।
- Bearer Token (Hash-এ সংরক্ষিত), Role, মালিকানা; অন্যের তথ্যে 404।
- Cursor Pagination, Idempotency-Key, Version ও `If-Match`—Network-এর অনিশ্চয়তা ও একসঙ্গে কাজের মধ্যে তথ্য সঠিক রাখে।
- Request ID, Structured Log, Rate Limit, Health/Ready, Graceful Shutdown।
- একই Repository-চুক্তির পেছনে Memory বা Database।

পরের অধ্যায়ে Database-এর গভীরে: Schema ও Constraint, SQL, Transaction ও Rollback, Index ও Query Plan, Migration এবং Backup।

২৫.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
REST	Resource ও HTTP Method-ভিত্তিক API-র নকশা।

পরিভাষা	অর্থ
Middleware	অনুরোধ-শৃঙ্খলের একটি ধাপ `(req, res, next)`।
Error Handler	`(err, req, res, next)` Middleware।
Safe Method	Server-এর অবস্থা বদলায় না।
Idempotent	একবার বা বহুবার পাঠালে একই ফল।
Bearer Token	`Authorization` Header-এ পাঠানো পরিচয়ের প্রমাণ।
Cursor Pagination	"এই অবস্থানের পর থেকে" পাতা।
Idempotency-Key	Retry-কে নিরাপদ করার Client-প্রদত্ত Key।
Optimistic Concurrency	Version মিলিয়ে হালনাগাদ, Lock ছাড়া।
ETag	একটি Resource-এর সংস্করণের চিহ্ন।
Request ID	একটি অনুরোধকে সব Log-এ চেনার ID।
Graceful Shutdown	চলমান কাজ শেষ করে বন্ধ হওয়া।

২৫.২১ আরও পড়ুন

- **Express — Guide:** `https://expressjs.com/en/guide/routing.html` — Routing ও Middleware।
- **Express — Error handling:** `https://expressjs.com/en/guide/error-handling.html` — Express 4 ও 5-এর পার্থক্যসহ।
- **MDN — HTTP response status codes:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Status` — প্রতিটি Status-এর অর্থ।
- **Stripe — Idempotent requests:** `https://docs.stripe.com/api/idempotent_requests` — Idempotency-Key-এর বাস্তব ব্যবহার।
- **Node.js — HTTP server.close():** `https://nodejs.org/api/http.html#serverclosecallback` — Graceful Shutdown-এর ভিত্তি।

অধ্যায় ২৬

২৬

Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Relational Model-এ তথ্য সাজাতে এবং Normalization দিয়ে পুনরাবৃত্তি ও অসামঞ্জস্য দূর করতে।
- Primary Key, Foreign Key, UNIQUE, CHECK ও NOT NULL Constraint দিয়ে Database-কেই নিয়মের পাহারাদার বানাতে।
- Transaction-এর ACID বৈশিষ্ট্য বুঝতে, এবং Rollback ও Savepoint ব্যবহার করতে।
- Isolation, Lock, Lost Update ও তার প্রতিরোধ বুঝতে।
- Index, `EXPLAIN QUERY PLAN` ও N+1 সমস্যা দিয়ে Query-র গতি বিশ্লেষণ করতে।
- সংস্করণভিত্তিক Migration লিখতে এবং চালানো Migration-এর পরিবর্তন ধরতে।
- Backup তৈরি ও যাচাই করতে।
- ORM (Prisma) ও Query Builder-এর সুবিধা ও সীমা বিচার করতে, এবং SQLite থেকে PostgreSQL-এ যাওয়ার পার্থক্য বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **TravelData Pro** Loyalty Ledger তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ২০: SQL Injection ও Parameterized Query, `node:sqlite`।
- অধ্যায় ২১: Repository Pattern।
- অধ্যায় ২২: Property-based Test, Integration Test।
- অধ্যায় ২৫: Repository-র `async` চুক্তি, Idempotency-Key, Optimistic Concurrency।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Loyalty Program: প্রতিটি Tour-এ গ্রাহক Point পান, Point দিয়ে হোটেলের রাত বা ছাড় নেন, পরিবারের সদস্যকে Point দেন। এখানে টাকার মতোই কঠোর নিয়ম: কোনো Point হাওয়া থেকে আসবে না, কোনো Point হারাবে না, কারও Balance ঋণাত্মক হবে না, এবং একই Tour-এর Point দুবার যোগ হবে না—Network বিভ্রাট, দুজন কর্মীর একসঙ্গে কাজ বা Server-এর হঠাৎ বন্ধ হওয়া সত্ত্বেও।

এই নিয়মগুলো শুধু JavaScript-এ লিখলে যথেষ্ট নয়। একটি Script Database-এ সরাসরি লিখতে পারে, দুটি Server একসঙ্গে একই Balance পড়তে পারে, একটি Process দুটি লেখার মাঝখানে বন্ধ হতে পারে। **Database** এই নিশ্চয়তাগুলো দেওয়ার জন্যই তৈরি: Constraint, Transaction, Lock। এই অধ্যায় শেখায় সেগুলো কীভাবে কাজে লাগাতে হয়—এবং কীভাবে Query-কে লাখো সারিতেও দ্রুত রাখতে হয়।

২৬.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Parameterized Query (অধ্যায় ২০)	Schema-র নকশা, Normalization, Constraint
Repository Pattern (অধ্যায় ২১, ২৫)	Transaction, Rollback, Savepoint
Optimistic Concurrency (অধ্যায় ২৫)	Lock, Isolation, Lost Update-এর কারণ
Idempotency-Key (অধ্যায় ২৫)	UNIQUE Constraint দিয়ে Database-স্তরের Idempotency
Big-O ও Profiling (অধ্যায় ১৯, ২৩)	Index, Query Plan, N+1
—	Migration, Backup, ORM, PostgreSQL-এর পার্থক্য

> **নোট:** এই অধ্যায়ের মূল পরিকল্পনা PostgreSQL ও Prisma-কেন্দ্রিক। এই বইয়ের QA-যন্ত্রে কোনো PostgreSQL Server বা Prisma ছিল না, এবং কোনো Package Install করা হয়নি। তাই সব চালানো Code Node.js-এর অন্তর্নির্মিত `node:sqlite` (SQLite) দিয়ে, যেখানে Constraint, Transaction, Lock, Index ও Query Plan—সবই আসল Database-এর আচরণ। PostgreSQL ও Prisma-র Code যেখানে দেখানো হয়েছে, সেখানে স্পষ্ট লেখা আছে যে তা চালানো হয়নি। দুই Database-এর পার্থক্য অনুচ্ছেদ ২৬.৯-এ।

২৬.২ Relational Model এবং Normalization

Relational Database-এ তথ্য থাকে Table-এ: প্রতিটি সারি একটি জিনিস, প্রতিটি Column একটি বৈশিষ্ট্য, এবং **Primary Key** প্রতিটি সারিকে আলাদা করে চেনায়। Table-গুলো যুক্ত থাকে **Foreign Key** দিয়ে।

একটি সাধারণ ভুল: সব তথ্য একটি বড় Table-এ। প্রতিটি লেনদেনের সারিতে গ্রাহকের নাম ও ফোন বারবার:

normalize.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE flat (entry INTEGER, name TEXT, phone TEXT, points INTEGER);
  INSERT INTO flat VALUES (1, 'Rafi', '01711111111', 500),
    (2, 'Rafi', '01711111111', 300), (3, 'Mina', '01822222222', 400);`);

// গ্রাহক ফোন বদলালেন; একটি সারিতে হালনাগাদ হলো, অন্যটিতে নয়।
db.exec("UPDATE flat SET phone = '01799999999' WHERE entry = 1");
const phones = db.prepare(
  "SELECT DISTINCT phone FROM flat WHERE name = 'Rafi'").all();
console.log('একই Rafi-র ফোন:', phones.map((r) => r.phone).join(', '));

db.exec(`CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT, phone TEXT);
  CREATE TABLE entries (id INTEGER PRIMARY KEY,
    customer_id INTEGER REFERENCES customers (id), points INTEGER);
  INSERT INTO customers VALUES (1, 'Rafi', '01711111111'),
    (2, 'Mina', '01822222222');
  INSERT INTO entries VALUES (1, 1, 500), (2, 1, 300), (3, 2, 400);
  UPDATE customers SET phone = '01799999999' WHERE id = 1;`);
const joined = db.prepare(`SELECT c.name, c.phone, SUM(e.points) AS total
  FROM customers c JOIN entries e ON e.customer_id = c.id
  GROUP BY c.id ORDER BY c.id`).all();
console.log(joined.map((r) => `${r.name} ${r.phone} ${r.total}`).join(' | '));
```

Output

```
একই Rafi-র ফোন: 01799999999, 01711111111
Rafi 01799999999 800 | Mina 01822222222 400
```

পুনরাবৃত্ত তথ্য একসময় অমিল হবেই (**Update Anomaly**): একই গ্রাহকের দুটি ফোন নম্বর। **Normalization** মানে প্রতিটি তথ্য একবারই, নিজের Table-এ: গ্রাহকের তথ্য `customers`-এ, লেনদেন `entries`-এ, এবং দুটি `JOIN` দিয়ে মেলানো। ফোন এখন এক জায়গায় বদলায়। নিয়মগুলোর আনুষ্ঠানিক নাম আছে (First, Second, Third Normal Form), কিন্তু মূল প্রশ্ন একটিই: **এই তথ্যের মালিক কোন Table?**

উল্টো দিকও আছে: Report-এর জন্য কখনো ইচ্ছাকৃতভাবে তথ্য পুনরাবৃত্তি করা হয় (**Denormalization**), যেমন TravelData-র `accounts.balance`—Ledger-এর যোগফলের একটি Copy, যাতে প্রতিবার হাজার সারি যোগ করতে না হয়। শর্ত: Copy-টি একই Transaction-এ হালনাগাদ, এবং একটি Audit Query নিয়মিত মিলিয়ে দেখে।

২৬.৩ Constraint: Database নিজেই নিয়ম পাহারা দেয়

JavaScript-এর যাচাই এড়িয়ে যাওয়া সম্ভব: একটি পুরোনো Script, একজন Developer-এর হাতে লেখা SQL, অথবা একটি Bug! **Constraint** Database-এর ভেতরের নিয়ম, যা কোনো পথেই এড়ানো যায় না:

constraints.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`PRAGMA foreign_keys = ON;
  CREATE TABLE customers (id INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE COLLATE NOCASE);
  CREATE TABLE accounts (
    customer_id INTEGER PRIMARY KEY REFERENCES customers (id),
    balance INTEGER NOT NULL
      CONSTRAINT balance_not_negative CHECK (balance >= 0));
  INSERT INTO customers VALUES (1, 'rafi@example.com');
  INSERT INTO accounts VALUES (1, 100);`);

const attempts = [
  "INSERT INTO customers VALUES (2, 'RAFI@example.com')",
  'INSERT INTO customers (id) VALUES (3)',
  'UPDATE accounts SET balance = balance - 500 WHERE customer_id = 1',
  'INSERT INTO accounts VALUES (99, 0)',
];
for (const sql of attempts) {
  try {
    db.exec(sql);
  } catch (error) {
    console.log(error.message);
  }
}
```

Output

```
UNIQUE constraint failed: customers.email
NOT NULL constraint failed: customers.email
CHECK constraint failed: balance_not_negative
FOREIGN KEY constraint failed
```

চারটি ভুল, চারটি Constraint। `COLLATE NOCASE` থাকায় বড় হাতের Email-ও একই গণ্য হয়েছে। Constraint-এর নাম (`balance_not_negative`) দিলে Error-বার্তায় নামটি আসে, যা Code ব্যবসার ভাষার বার্তায় রূপান্তর করতে পারে (Project-এর `translate`)। একটি SQLite-বিশেষ ফাঁদ: Foreign Key-র যাচাই Default বন্ধ; প্রতিটি সংযোগে `PRAGMA foreign_keys = ON` লাগে (PostgreSQL-এ সব সময় চালু)।

নিয়ম: **JavaScript-এ যাচাই ভালো বার্তার জন্য, Constraint নিশ্চয়তার জন্য।** দুটিই থাকুক।

২৬.৪ Transaction: সব, নয়তো কিছুই নয়

Point Transfer-এ দুটি লেখা: একজনের Balance কমানো, আরেকজনের বাড়ানো। প্রথমটির পরে Program বন্ধ হলে Point হাওয়া। **Transaction** কয়েকটি কাজকে একটি অবিভাজ্য একক বানায়, যার চারটি বৈশিষ্ট্য **ACID**:

বৈশিষ্ট্য	অর্থ
Atomicity	সব কাজ সফল, অথবা কোনোটিই নয়
Consistency	Transaction-এর পরে সব Constraint সত্য
Isolation	একসঙ্গে চলা Transaction একে অপরের আধা-কাজ দেখে না
Durability	`COMMIT`-এর পর তথ্য স্থায়ী, এমনকি বিদ্যুৎ গেলেও

transaction.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE accounts (id INTEGER PRIMARY KEY,
  balance INTEGER NOT NULL CHECK (balance >= 0));
  INSERT INTO accounts VALUES (1, 300), (2, 0);`);
const add = db.prepare('UPDATE accounts SET balance = balance + ? WHERE id = ?');
const balances = () => db.prepare('SELECT balance FROM accounts ORDER BY id')
  .all().map((r) => r.balance).join(' / ');

function transferUnsafe(from, to, points) {
  add.run(points, to);
  add.run(-points, from);
}

function transferSafe(from, to, points) {
```

```
  db.exec('BEGIN');
  try {
    add.run(points, to);
    add.run(-points, from);
    db.exec('COMMIT');
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}

for (const transfer of [transferUnsafe, transferSafe]) {
  db.exec('UPDATE accounts SET balance = CASE id WHEN 1 THEN 300 ELSE 0 END');
  try {
    transfer(1, 2, 500);
  } catch (error) {
    console.log(`${transfer.name}: ${error.message} | ${balances()}`);
  }
}
```

Output

```
transferUnsafe: CHECK constraint failed: balance >= 0 | 300 / 500
transferSafe: CHECK constraint failed: balance >= 0 | 300 / 0
```

Transaction ছাড়া দ্বিতীয় লেখা ব্যর্থ হলেও প্রথমটি থেকে গেছে: ৫০০ Point শূন্য থেকে তৈরি হয়েছে। Transaction-এ `ROLLBACK` দুটিকেই মুছে দিয়েছে। Project-এর `withTransaction` এই নকশাটি একবার লিখে সব জায়গায় ব্যবহার করে।

Savepoint একটি Transaction-এর ভেতরে আংশিক ফেরা: অনেকগুলো আইটেমের একটি ব্যর্থ হলে শুধু সেটি বাতিল, বাকিগুলো রেখে।

savepoint.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE gifts (name TEXT NOT NULL CHECK (length(name) >= 3));`);

const imported = [];
db.exec('BEGIN');
for (const name of ['Tea Garden Pass', 'X', 'Boat Ride']) {
  db.exec('SAVEPOINT item');
  try {
    db.prepare('INSERT INTO gifts VALUES (?)').run(name);
    db.exec('RELEASE item');
    imported.push(name);
  } catch {
```

```
    db.exec('ROLLBACK TO item');
    db.exec('RELEASE item');
  }
}
db.exec('COMMIT');
const saved = db.prepare('SELECT name FROM gifts').all().map((r) => r.name);
console.log(imported.length, saved.join(', '));
```

Output

```
2 Tea Garden Pass, Boat Ride
```

২৬.৫ Isolation, Lock এবং Lost Update

একাধিক সংযোগ একসঙ্গে একই তথ্যে কাজ করলে প্রশ্ন ওঠে: একটি Transaction অন্যটির পরিবর্তন কখন দেখবে? SQLite-এর WAL Mode-এ একটি পড়ার Transaction শুরু হওয়ার মুহূর্তের **Snapshot** দেখে, যতক্ষণ না শেষ হয়:

isolation.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const dir = mkdtempSync(join(tmpdir(), 'isolation-'));
const file = join(dir, 'demo.db');
const writer = new DatabaseSync(file);
writer.exec(`PRAGMA journal_mode = WAL;
  CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance INTEGER);
  INSERT INTO accounts VALUES (1, 100);`);
const reader = new DatabaseSync(file);
const read = (db) =>
  db.prepare('SELECT balance FROM accounts WHERE id = 1').get().balance;

reader.exec('BEGIN');
console.log('Reader শুরুতে:', read(reader));
writer.exec('UPDATE accounts SET balance = 900 WHERE id = 1');
console.log('Writer-এর COMMIT-এর পরে, Reader এখনো:', read(reader));
reader.exec('COMMIT');
console.log('Reader-এর নতুন Transaction:', read(reader));

reader.close();
writer.close();
rmSync(dir, { recursive: true, force: true });
```

Output

```
Reader শুরুতে: 100
Writer-এর COMMIT-এর পরে, Reader এখনো: 100
Reader-এর নতুন Transaction: 900
```

Reader-এর Report একটি সামঞ্জস্যপূর্ণ মুহূর্তের ছবি দেখায়, মাঝপথের পরিবর্তনে বিভ্রান্ত হয় না। SQLite একসঙ্গে একজনকেই লিখতে দেয়; দ্বিতীয় Writer অপেক্ষা করে (`busy_timeout`) বা "database is locked" পায়। PostgreSQL অনেক Writer একসঙ্গে চালায় এবং চারটি **Isolation Level** দেয় (Read Uncommitted, Read Committed—Default, Repeatable Read, Serializable): কঠোর Level বেশি নিশ্চয়তা দেয়, কিন্তু বেশি সংঘাত ও Retry।

Lost Update ঘটে যখন দুটি সংযোগ একই মান পড়ে, নিজের Code-এ হিসাব করে, তারপর লেখে: দ্বিতীয় লেখা প্রথমটিকে মুছে দেয়। Project-এর `concurrency.test.mjs` দুটি আসল সংযোগে এটি ঘটিয়ে দেখায় (১০০ ও ৫০ যোগের পর ফল ৫০), তারপর তিনটি প্রতিরোধ:

১. **হিসাব Database-এর ভেতরে:** `UPDATE accounts SET balance = balance + ?`—পড়া ও লেখা একটি পরমাণু কাজ।
২. **পড়া ও লেখা একই Transaction-এ, লেখার Lock-সহ:** SQLite-এ `BEGIN IMMEDIATE`, PostgreSQL-এ `SELECT ... FOR UPDATE`।
৩. **Optimistic Concurrency:** Version মিলিয়ে হালনাগাদ (অধ্যায় ২৫)।

Deadlock: দুটি Transaction একে অপরের Lock-এর জন্য অপেক্ষা করলে কেউই এগোয় না; Database একটিকে বাতিল করে। প্রতিরোধ: সব জায়গায় একই ক্রমে Lock নেওয়া (যেমন সব সময় ছোট ID আগে), Transaction ছোট রাখা, এবং বাতিল হলে পুরো Transaction Retry।

২৬.৬ Index, Query Plan এবং N+1

Index ছাড়া "এই গ্রাহকের সব লেনদেন" খুঁজতে Database প্রতিটি সারি পড়ে (**Full Table Scan**)—দশ লাখ সারিতে দশ লাখবার। **Index** একটি সাজানো কাঠামো (সাধারণত B-tree), যা সরাসরি সঠিক জায়গায় নিয়ে যায়—অধ্যায় ১৯-এর Binary Search-এর Database রূপ। Database একটি Query কীভাবে চালাবে, তা দেখায় `EXPLAIN QUERY PLAN`; Project-এর `queries.test.mjs` থেকে আসল ফল:

queries.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
const LEDGER_QUERY = `SELECT points FROM ledger_entries
  WHERE customer_id = ? ORDER BY created_at`;

test('Index থাকলে SEARCH, Index মুছলে পুরো Table SCAN ও আলাদা সাজানো', () => {
  const { db } = memoryStore();
  const withIndex = planOf(db, LEDGER_QUERY, 1).join(' | ');
  assert.match(withIndex,
    /SEARCH ledger_entries USING INDEX ledger_customer_time/);
  db.exec('DROP INDEX ledger_customer_time');
  const without = planOf(db, LEDGER_QUERY, 1).join(' | ');
  assert.match(without, /SCAN ledger_entries/);
  assert.match(without, /USE TEMP B-TREE FOR ORDER BY/);
});


// ...
```

Index-সহ: `SEARCH ledger_entries USING INDEX ledger_customer_time (customer_id=?)` — সরাসরি খোঁজ, এবং Index-এ সারিগুলো আগে থেকেই `created_at` অনুযায়ী সাজানো, তাই আলাদা সাজাতে হয় না। Index মুছে দিলে: `SCAN ledger_entries` (পুরো Table) এবং `USE TEMP B-TREE FOR ORDER BY` (আলাদা সাজানো)। Composite Index `(customer_id, created_at)`-এর Column-এর ক্রম গুরুত্বপূর্ণ: আগে যেটি দিয়ে খোঁজা হয়, তারপর যেটি দিয়ে সাজানো।

Index-এর মূল্যও আছে: প্রতিটি `INSERT` ও `UPDATE`-এ Index-ও হালনাগাদ হয়, এবং জায়গা লাগে। তাই প্রতিটি Column-এ নয়—যে Query সত্যিই ঘনঘন চলে এবং Plan-এ যার SCAN দেখা যায়, সেখানে।

N+1 সমস্যা Index দিয়ে সমাধান হয় না—এটি Query-র সংখ্যার সমস্যা, এবং ORM ব্যবহারে প্রায়ই অজান্তে ঘটে:

n-plus-one.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT);
  CREATE TABLE entries (customer_id INTEGER, points INTEGER);`);
for (let id = 1; id <= 50; id += 1) {
  db.prepare('INSERT INTO customers VALUES (?, ?)').run(id, `Guest ${id}`);
  db.prepare('INSERT INTO entries VALUES (?, ?), (?, ?)').run(id, 100, id, 50);
}

let queries = 0;
const run = (sql, ...params) => {
  queries += 1;
  return db.prepare(sql).all(...params);
};
```

```
const slow = run('SELECT id, name FROM customers').map((c) => ({
  ...c,
  total: run('SELECT SUM(points) AS t FROM entries WHERE customer_id = ?',
    c.id)[0].t,
}));
const nPlusOne = queries;

queries = 0;
const fast = run(`SELECT c.id, c.name, SUM(e.points) AS total
  FROM customers c JOIN entries e ON e.customer_id = c.id GROUP BY c.id`);
console.log(`N+1: ${nPlusOne}টি Query, JOIN: ${queries}টি Query`);
const totals = (rows) => JSON.stringify(rows.map((r) => r.total));
console.log(totals(slow) === totals(fast));
```

Output

```
N+1: 51টি Query, JOIN: 1টি Query
true
```

একই ফল, কিন্তু ৫১ বনাম ১টি Query। একই Process-এর SQLite-এ পার্থক্য ছোট মনে হয়; Network-এর ওপারের PostgreSQL-এ প্রতিটি Query-র যাওয়া-আসায় এক মিলিসেকেন্ড বা বেশি, তাই ৫০ জন গ্রাহকের পাতায় ৫০ গুণ দেরি। ORM-এ এর সমাধান সাধারণত "Include" বা "Eager Loading", যা ভেতরে JOIN বা একটি `IN (...)` Query চালায়।

২৬.৭ Pagination, Connection Pool ও Cache

- **Keyset Pagination:** `LIMIT 20 OFFSET 100000` Database-কে এক লাখ সারি পড়ে ফেলে দিতে বাধ্য করে। `WHERE id > ? ORDER BY id LIMIT 20` Index দিয়ে সরাসরি সেখানে যায় (অধ্যায় ২৫-এর Cursor; Project-এর `byCity`)।
- **Connection Pool:** PostgreSQL-এ প্রতিটি নতুন সংযোগ ব্যয়বহুল (Authentication, Memory), এবং Server-এর সংযোগের সংখ্যা সীমিত। একটি Pool কয়েকটি সংযোগ খোলা রাখে এবং অনুরোধগুলোর মধ্যে ভাগ করে (`pg.Pool`-এর `max`)। Transaction-এর সব Query একই সংযোগে চলতে হয়—তাই `pool.connect()` দিয়ে একটি সংযোগ নিয়ে, শেষে `release()`।
- **Cache:** বারবার পড়া, কম বদলানো তথ্য (Package-এর তালিকা) Memory বা Redis-এ। কিন্তু Balance-এর মতো তথ্য Cache থেকে দেখালে বাসি হতে পারে (অধ্যায় ১৯-এর সতর্কতা); টাকা ও Point-এর সিদ্ধান্ত সব সময় Database-এর বর্তমান মান থেকে।

২৬.৮ Migration ও Backup

Schema সময়ের সঙ্গে বদলায়: নতুন Column, নতুন Index, নতুন Table। হাতে Production Database-এ `ALTER TABLE` চালানো বিপজ্জনক—কোন Database-এ কোন পরিবর্তন হয়েছে, কেউ জানে না। **Migration** প্রতিটি পরিবর্তনকে একটি ক্রমিক File-এ লেখে (`001-init.sql`, `002-indexes.sql`), Git-এ রাখে, এবং একটি Table (`schema_migrations`) মনে রাখে কোনগুলো চলেছে। TravelData-র Runner আরও দুটি নিয়ম মানে: প্রতিটি Migration নিজের Transaction-এ (মাঝপথে ব্যর্থ হলে আধা-পরিবর্তন থাকে না), এবং চলে যাওয়া File পরে বদলালে Checksum দেখে থেমে যায়—পুরোনো Migration কখনো বদলাবেন না, নতুন একটি লিখুন।

Production-এ নিরাপদ Migration-এর অভ্যাস: নতুন Column প্রথমে Default-সহ বা ঐচ্ছিক (পুরোনো Code ভাঙে না), তারপর Code হালনাগাদ, তারপর পুরোনো Column সরানো (**Expand–Contract**)। বড় Table-এ Index তৈরি Table Lock করতে পারে (PostgreSQL-এ `CREATE INDEX CONCURRENTLY`)। আর প্রতিটি Migration-এর আগে Backup।

Backup যাচাই না হলে Backup নয়। TravelData-র `backupTo` চালু Database-এর একটি সামঞ্জস্যপূর্ণ Copy নেয় (SQLite-এর `VACUUM INTO`), তারপর Copy খুলে `integrity_check` চালায় এবং প্রতিটি Table-এর সারির সংখ্যা মেলায়। PostgreSQL-এ `pg_dump`, এবং বড় System-এ Point-in-time Recovery। নিয়মিত একটি Backup থেকে আলাদা জায়গায় **Restore করে দেখুন**—না দেখা পর্যন্ত আপনি জানেন না Backup কাজ করে কি না।

২৬.৯ SQLite থেকে PostgreSQL, এবং ORM

বিষয়	SQLite (এই অধ্যায়ের চালানো Code)	PostgreSQL
চলে	একটি File, Application-এর Process-এর ভেতরে	আলাদা Server, Network-এ
একসঙ্গে লেখা	একজন Writer	অনেক Writer, Row-level Lock
Parameter	`?`	`$1`, `$2`
Auto ID	`INTEGER PRIMARY KEY AUTOINCREMENT`	`GENERATED ALWAYS AS IDENTITY`
Foreign Key	`PRAGMA foreign_keys = ON` লাগে	সব সময় চালু
Constraint Error	বার্তার লেখা	নির্দিষ্ট Code (`23505` Unique, `23514` Check, `23503` FK)
Driver	`node:sqlite` (অন্তর্নির্মিত, Synchronous)	`pg` (Package, `async`, Pool)
উপযুক্ত	Embedded, Desktop, ছোট Server, Test	বেশিরভাগ Web Backend

Project-এর `src/postgres-ledger.mjs` একই Ledger-এর `pg` সংস্করণ: `$1` Parameter, Pool থেকে একটি সংযোগে `BEGIN` / `COMMIT` / `ROLLBACK`, এবং Unique Violation (`23505`)-কে Idempotency-র "আগেই হয়েছে" হিসেবে পড়া। এটি `node --check` দিয়ে Syntax যাচাই করা হয়েছে, কিন্তু **চালানো হয়নি** (PostgreSQL Server ছিল না)।

ORM (Object-Relational Mapper), যেমন Prisma, Table-কে Object-এর মতো ব্যবহার করতে দেয়। Prisma-তে একটি Schema File-এ Model লেখা হয় (`model Customer { id Int @id @default(autoincrement()) email String @unique ... }`), তা থেকে Migration ও একটি Type-যুক্ত Client তৈরি হয় (অধ্যায় ২৪-এর TypeScript-এর সুবিধা):

prisma-example.mjs · JavaScript — চলবে: Node.js

```
// উদাহরণ মাত্র: @prisma/client ও একটি Database প্রয়োজন। এই বইয়ের QA-তে
// Prisma Install বা চালানো হয়নি।
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

const rich = await prisma.customer.findMany({
  where: { account: { balance: { gte: 5_000 } } },
  include: { account: true },
  orderBy: { id: 'asc' },
  take: 20,
});

await prisma.$transaction(async (tx) => {
  await tx.account.update({
    where: { customerId: 1 }, data: { balance: { decrement: 500 } },
  });
  await tx.account.update({
    where: { customerId: 2 }, data: { balance: { increment: 500 } },
  });
});
console.log(rich.length);
```

ORM-এর সুবিধা	ORM-এর ঝুঁকি
Type-যুক্ত Query, কম Boilerplate	Generate হওয়া SQL অদৃশ্য; N+1 সহজে ঘটে
Migration-এর Tool	জটিল Report-এ অদক্ষ Query
Parameter স্বয়ংক্রিয় (Injection-নিরাপদ)	`$queryRawUnsafe`-এর মতো ফাঁক
Database বদলানো তুলনামূলক সহজ	Database-এর বিশেষ সুবিধা (Lock, CTE) কঠিন

পরামর্শ: ORM ব্যবহার করলেও SQL শিখুন, এবং ধীর Query-র আসল SQL ও Plan দেখুন। অনেক দল দুটি একসঙ্গে ব্যবহার করে: সাধারণ CRUD-এ ORM, জটিল Report-এ হাতে লেখা Parameterized SQL।

২৬.১০ Security ও Observability

- **Least Privilege:** Application-এর Database ব্যবহারকারী শুধু প্রয়োজনীয় Table-এ `SELECT/INSERT/UPDATE`; Migration-এর জন্য আলাদা, বেশি অধিকারের ব্যবহারকারী।
- **Secret:** `DATABASE_URL` Environment বা Secret Manager-এ, কখনো Git-এ নয় (অধ্যায় ২০)।
- **Parameterized Query সব সময়,** ORM-এর Raw Query-তেও।
- **Encryption:** Network-এ TLS; সংবেদনশীল Column (যেমন Passport নম্বর) আলাদাভাবে Encrypt করার কথা ভাবুন।
- **Observability:** ধীর Query-র Log (PostgreSQL-এর `log_min_duration_statement`), Pool-এর ব্যবহার, Lock-এর অপেক্ষা, এবং নিয়মিত Audit Query (TravelData-র `audit()`)।

২৬.১১ Project: TravelData Pro

TravelData Pro একটি ভ্রমণ সংস্থার গ্রাহক ও Loyalty Point-এর Data Layer: গ্রাহক নিবন্ধন (Account-সহ, একটি Transaction-এ), Point অর্জন, ব্যয় ও পরিবারে হস্তান্তর (Append-only Ledger, Idempotency Key), Report, Query Plan, Migration ও যাচাই-করা Backup। Project-এর নাম TravelData Pro; Domain হিসেবে Booking-এর বদলে Loyalty Ledger বেছে নেওয়া হয়েছে, যাতে আগের অধ্যায়গুলোর Booking Project-এর পুনরাবৃত্তি না হয় এবং Transaction-এর নিয়মগুলো স্পষ্ট দেখা যায়।

> **নোট:** শিক্ষামূলক Project। SQLite-এ চালানো ও পরীক্ষিত; PostgreSQL Adapter শুধু Syntax-যাচাই করা। কোনো আসল গ্রাহক বা কোম্পানির Database-এ যুক্ত নয়; সব Test নিজস্ব Memory বা Temp Database তৈরি করে।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-data/
├── demo.mjs
├── migrations/             001-init.sql, 002-indexes.sql, 003-tier.sql
├── src/
│   ├── db.mjs              সংযোগের Setting, withTransaction, Constraint চেনা
│   ├── migrate.mjs         Migration Runner (Checksum)
│   ├── errors.mjs          DataError, translate
│   ├── customers.mjs       গ্রাহক, Keyset Pagination, নিরাপদ খোঁজ
│   ├── ledger.mjs          earn, redeem, transfer, audit
│   ├── reports.mjs         Report ও Query Plan
│   ├── backup.mjs          VACUUM INTO ও যাচাই
│   └── postgres-ledger.mjs PostgreSQL সংস্করণ (চালানো হয়নি)
└── tests/                  schema, ledger, concurrency, queries (+ helpers)
```

Migration

001-init.sql · SQL — চলবে: Node.js

```sql
-- গ্রাহক, তাঁর Loyalty Account, এবং প্রতিটি Point-এর লেনদেন (Ledger)।
CREATE TABLE customers (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL
    CONSTRAINT name_length CHECK (length(trim(name)) BETWEEN 2 AND 80),
  email TEXT NOT NULL UNIQUE COLLATE NOCASE,
  phone TEXT NOT NULL UNIQUE
    CONSTRAINT phone_format CHECK (length(phone) = 11
      AND phone GLOB '01[3-9]*' AND phone NOT GLOB '*[^0-9]*'),
  city TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);

CREATE TABLE accounts (
  customer_id INTEGER PRIMARY KEY REFERENCES customers (id) ON DELETE CASCADE,
  balance INTEGER NOT NULL DEFAULT 0
    CONSTRAINT balance_not_negative CHECK (balance >= 0)
);

CREATE TABLE ledger_entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id INTEGER NOT NULL REFERENCES customers (id),
  points INTEGER NOT NULL CONSTRAINT points_not_zero CHECK (points <> 0),
  reason TEXT NOT NULL,
  idempotency_key TEXT UNIQUE,
  created_at TEXT NOT NULL
);
```

002-indexes.sql · SQL — চলবে: Node.js

```sql
-- প্রায়ই চলা Query-র জন্য Index: একজন গ্রাহকের লেনদেন সময়ের ক্রমে,
-- এবং শহর ধরে গ্রাহক।
CREATE INDEX ledger_customer_time ON ledger_entries (customer_id, created_at);
CREATE INDEX customers_city ON customers (city);
```

003-tier.sql · SQL চলবে: Node.js

```sql
-- নতুন Column, Default-সহ (পুরোনো সারিও বৈধ থাকে), এবং বিদ্যমান তথ্য থেকে
-- মান পূরণ (Data Migration)।
ALTER TABLE accounts ADD COLUMN tier TEXT NOT NULL DEFAULT 'silver'
  CONSTRAINT tier_known CHECK (tier IN ('silver', 'gold', 'platinum'));

UPDATE accounts SET tier = CASE
  WHEN balance >= 20000 THEN 'platinum'
  WHEN balance >= 5000 THEN 'gold'
  ELSE 'silver'
END;
```

তিনটি Table-এর প্রতিটি নিয়ম Constraint-এ: নামের দৈর্ঘ্য, ফোনের রূপ, Email ও ফোন অনন্য, Balance কখনো ঋণাত্মক নয়, Point শূন্য নয়, এবং প্রতিটি লেনদেনের Idempotency Key অনন্য। `003-tier.sql` দেখায় Production-বান্ধব পরিবর্তন: নতুন Column একটি Default-সহ (পুরোনো সারি বৈধ থাকে), তারপর বিদ্যমান তথ্য থেকে মান পূরণ (Data Migration)। লক্ষ করুন, `tier` এখানে একবারই হিসাব হয়; Balance বদলালে স্বয়ংক্রিয়ভাবে বদলায় না—বাস্তবে এটি একটি নিয়মিত কাজ বা Trigger-এর দায়িত্ব।

src/db.mjs, src/migrate.mjs ও src/errors.mjs

db.mjs · JavaScript চলবে: Node.js

```javascript
import { DatabaseSync } from 'node:sqlite';

// প্রতিটি সংযোগে একই Setting: Foreign Key চালু (SQLite-এ Default বন্ধ!),
// WAL Journal (পড়া ও লেখা একসঙ্গে), এবং ব্যস্ত থাকলে কতক্ষণ অপেক্ষা।
export function openDatabase(file = ':memory:', options = {}) {
  const { busyTimeoutMs = 2_000 } = options;
  const db = new DatabaseSync(file);
  db.exec('PRAGMA foreign_keys = ON');
  if (file !== ':memory:') {
    db.exec('PRAGMA journal_mode = WAL');
  }
  db.exec(`PRAGMA busy_timeout = ${Number(busyTimeoutMs)}`);
  return db;
}

// fn-এর সব কাজ একসঙ্গে সফল, নয়তো কিছুই নয় (Atomicity)।
// IMMEDIATE: শুরুতেই লেখার Lock, যাতে মাঝপথে অন্য Writer-এর সঙ্গে সংঘাত না হয়।
export function withTransaction(db, fn) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = fn();
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
```

```
    throw error;
  }
}

// SQLite-এর Constraint Error-কে অর্থপূর্ণ Code-এ রূপান্তর।
export function constraintOf(error) {
  const message = String(error?.message ?? '');
  if (message.includes('UNIQUE constraint failed')) {
    return { code: 'UNIQUE', column: message.split(': ').at(-1) };
  }
  if (message.includes('CHECK constraint failed')) {
    return { code: 'CHECK', column: message.split(': ').at(-1) };
  }
  if (message.includes('FOREIGN KEY constraint failed')) {
    return { code: 'FOREIGN_KEY', column: null };
  }
  return null;
}
```

migrate.mjs · JavaScript — চলবে: Node.js

```
import { readdirSync, readFileSync } from 'node:fs';
import { createHash } from 'node:crypto';
import { join } from 'node:path';
import { fileURLToPath } from 'node:url';
import { withTransaction } from './db.mjs';

const DEFAULT_DIR = fileURLToPath(new URL('../migrations/', import.meta.url));

// সংস্করণভিত্তিক Migration: প্রতিটি File একবারই, ক্রমে, নিজস্ব Transaction-এ।
// চলে যাওয়া File পরে বদলালে (Checksum আলাদা) থেমে যায়।
export function migrate(db, dir = DEFAULT_DIR) {
  db.exec(`CREATE TABLE IF NOT EXISTS schema_migrations (
    version TEXT PRIMARY KEY, checksum TEXT NOT NULL,
    applied_at TEXT NOT NULL)`);
  const rows = db.prepare(
    'SELECT version, checksum FROM schema_migrations').all();
  const applied = new Map(rows.map((row) => [row.version, row.checksum]));
  const files = readdirSync(dir)
    .filter((f) => /^\d{3}-.+\.sql$/.test(f)).sort();
  const ran = [];
  for (const file of files) {
    const sql = readFileSync(join(dir, file), 'utf8');
    const checksum = createHash('sha256').update(sql).digest('hex');
    const version = file.slice(0, 3);
    if (applied.has(version)) {
      if (applied.get(version) !== checksum) {
        throw new Error(`Migration ${file} চলার পরে বদলানো হয়েছে`);
      }
```

```
      continue;
    }
    withTransaction(db, () => {
      db.exec(sql);
      db.prepare('INSERT INTO schema_migrations VALUES (?, ?, ?)')
        .run(version, checksum, new Date().toISOString());
    });
    ran.push(file);
  }
  return ran;
}
```

errors.mjs · JavaScript চলবে: Node.js

```
import { constraintOf } from './db.mjs';

export class DataError extends Error {
  constructor(code, message) {
    super(message);
    this.name = 'DataError';
    this.code = code;
  }
}

// Database-এর Constraint Error থেকে ব্যবসার ভাষার Error। অজানা হলে যেমন ছিল।
export function translate(error, messages = {}) {
  const constraint = constraintOf(error);
  if (!constraint) {
    return error;
  }
  const key = constraint.column ?? constraint.code;
  const message = messages[key] ?? messages[constraint.code] ?? error.message;
  return new DataError(constraint.code, message);
}
```

`withTransaction` `BEGIN IMMEDIATE` ব্যবহার করে: Transaction শুরুতেই লেখার Lock নেয়, তাই পড়ার পরে লেখার মুহূর্তে অন্য Writer-এর সঙ্গে সংঘাত হয় না (একটি সাধারণ `BEGIN` পড়ার পরে লেখায় "উন্নীত" হতে গিয়ে ব্যর্থ হতে পারে)। `constraintOf` SQLite-এর বার্তা থেকে Constraint-এর ধরন ও নাম বের করে; `translate` সেগুলো ব্যবসার ভাষায় বদলায়।

src/customers.mjs

customers.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { withTransaction } from './db.mjs';
import { translate } from './errors.mjs';

const MESSAGES = {
```

```
  'customers.email': 'এই Email-এ আগেই একজন গ্রাহক আছেন',
  'customers.phone': 'এই ফোন নম্বরে আগেই একজন গ্রাহক আছেন',
  name_length: 'নাম ২ থেকে ৮০ অক্ষর',
  phone_format: '১১ অঙ্কের মোবাইল নম্বর',
};

// সব Query Parameterized; LIKE-এর বিশেষ চিহ্ন (% ও _) Escape করা।
const escapeLike = (text) => text.replace(/[\\%_]/g, (ch) => `\\${ch}`);

export function createCustomerStore(db) {
  const sql = {
    insert: db.prepare(
      'INSERT INTO customers (name, email, phone, city) VALUES (?, ?, ?, ?)'),
    account: db.prepare('INSERT INTO accounts (customer_id) VALUES (?)'),
    byEmail: db.prepare('SELECT * FROM customers WHERE email = ?'),
    page: db.prepare(`SELECT id, name FROM customers
      WHERE city = ? AND id > ? ORDER BY id LIMIT ?`),
    search: db.prepare(`SELECT id, name FROM customers
      WHERE name LIKE ? ESCAPE '\\' ORDER BY name, id LIMIT 20`),
  };

// ...
```

customers.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  // গ্রাহক ও তাঁর Account একসঙ্গে: একটি ব্যর্থ হলে দুটিই বাতিল।
  function create({ name, email, phone, city }) {
    try {
      return withTransaction(db, () => {
        const { lastInsertRowid } = sql.insert.run(name, email, phone, city);
        sql.account.run(lastInsertRowid);
        return Number(lastInsertRowid);
      });
    } catch (error) {
      throw translate(error, MESSAGES);
    }
  }

  // Keyset Pagination: "এই id-র পর থেকে", Index দিয়ে সরাসরি।
  const byCity = (city, { after = 0, limit = 20 } = {}) =>
    sql.page.all(city, after, limit).map((row) => ({ ...row }));

  const search = (text) =>
    sql.search.all(`%${escapeLike(text)}%`).map((row) => ({ ...row }));

  const findByEmail = (email) => {
    const row = sql.byEmail.get(email);
```

```
    return row ? { ...row } : null;
  };

  return { create, byCity, search, findByEmail };
}
```

src/ledger.mjs

ledger.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
import { withTransaction } from './db.mjs';
import { DataError, translate } from './errors.mjs';

const MESSAGES = {
  balance_not_negative: 'যথেষ্ট Point নেই',
  FOREIGN_KEY: 'গ্রাহক নেই',
};

// Ledger: প্রতিটি পরিবর্তন একটি নতুন সারি, কখনো মুছে বা বদলে নয় (Append-only)।
// balance তার যোগফলের একটি দ্রুত Copy, একই Transaction-এ হালনাগাদ।
export function createLedger(db, { now = () => new Date() } = {}) {
  const sql = {
    entry: db.prepare(`INSERT INTO ledger_entries
      (customer_id, points, reason, idempotency_key, created_at)
      VALUES (?, ?, ?, ?, ?)`),
    adjust: db.prepare(
      'UPDATE accounts SET balance = balance + ? WHERE customer_id = ?'),
    seen: db.prepare('SELECT 1 FROM ledger_entries WHERE idempotency_key = ?'),
    balance: db.prepare(
      'SELECT balance, tier FROM accounts WHERE customer_id = ?'),
    audit: db.prepare(`SELECT a.customer_id AS id, a.balance,
      COALESCE(SUM(e.points), 0) AS total
      FROM accounts a
      LEFT JOIN ledger_entries e ON e.customer_id = a.customer_id
      GROUP BY a.customer_id HAVING a.balance <> total`),
  };

  function post(customerId, points, reason, key) {
    sql.entry.run(customerId, points, reason, key, now().toISOString());
    if (sql.adjust.run(points, customerId).changes !== 1) {
      throw new DataError('NOT_FOUND', 'গ্রাহকের Account নেই');
    }
  }

// ...
```

ledger.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

  // একই key আগে দেখা গেলে কিছুই না করে duplicate: true (নিরাপদ Retry)।
  function run(key, steps) {
    if (!Number.isInteger(steps[0].points) || steps[0].points === 0) {
      throw new DataError('INVALID', 'Point শূন্য নয় এমন পূর্ণসংখ্যা');
    }
    try {
      return withTransaction(db, () => {
        if (key && sql.seen.get(`${key}:0`)) {
          return { duplicate: true };
        }
        steps.forEach((s, i) => post(s.customerId, s.points, s.reason,
          key ? `${key}:${i}` : null));
        return { duplicate: false };
      });
    } catch (error) {
      throw translate(error, MESSAGES);
    }
  }

  const earn = (customerId, points, reason, key) =>
    run(key, [{ customerId, points, reason }]);
  const redeem = (customerId, points, reason, key) =>
    run(key, [{ customerId, points: -points, reason }]);
  // এক গ্রাহক থেকে আরেকজনে: দুটি সারি, একটি Transaction—আধা-সম্পূর্ণ হয় না।
  const transfer = (from, to, points, key) => run(key, [
    { customerId: from, points: -points, reason: `transfer to ${to}` },
    { customerId: to, points, reason: `transfer from ${from}` },
  ]);

  const balanceOf = (customerId) => {
    const row = sql.balance.get(customerId);
    return row ? { ...row } : null;
  };
  // Invariant: প্রতিটি balance তার Ledger-এর যোগফলের সমান। মিল না থাকলে তালিকা।
  const audit = () => sql.audit.all().map((row) => ({ ...row }));

  return { earn, redeem, transfer, balanceOf, audit };
}
```

Ledger-এর তিনটি নকশার সিদ্ধান্ত। **Append-only:** কোনো লেনদেন বদলানো বা মোছা হয় না; ভুল হলে উল্টো একটি লেনদেন—হিসাবরক্ষণের প্রাচীন নিয়ম, যাতে পুরো ইতিহাস থাকে। **Idempotency Database-এ:** Key আগে দেখা হয়, এবং দুটি অনুরোধ একসঙ্গে এলেও Ledger-এর `UNIQUE` Constraint দ্বিতীয়টিকে আটকায়—Memory-র `Map` (অধ্যায় ২৫) একাধিক Server-এ যা পারে না। **Invariant ও Audit:** প্রতিটি Balance তার Ledger-এর যোগফলের সমান; `audit()` একটি Query-তে অমিল খোঁজে, এবং ৫০০টি এলোমেলো কাজের পরেও (Property Test) তালিকা খালি।

src/reports.mjs, src/backup.mjs ও src/postgres-ledger.mjs

reports.mjs · JavaScript — চলবে: Node.js

```
// Report-এর Query: JOIN, GROUP BY, এবং Query Plan দেখার সাহায্যকারী।
export function topCustomers(db, limit = 5) {
  return db.prepare(`SELECT c.name, c.city, a.balance, a.tier
    FROM customers c JOIN accounts a ON a.customer_id = c.id
    ORDER BY a.balance DESC, c.id LIMIT ?`)
    .all(limit).map((row) => ({ ...row }));
}

export function monthlyPoints(db, customerId) {
  return db.prepare(`SELECT substr(created_at, 1, 7) AS month,
      SUM(CASE WHEN points > 0 THEN points ELSE 0 END) AS earned,
      SUM(CASE WHEN points < 0 THEN -points ELSE 0 END) AS spent
    FROM ledger_entries WHERE customer_id = ?
    GROUP BY month ORDER BY month`).all(customerId).map((row) => ({ ...row }));
}

// EXPLAIN QUERY PLAN: Database Query-টি কীভাবে চালাবে (Index, না পুরো Table)।
export function planOf(db, sql, ...params) {
  return db.prepare(`EXPLAIN QUERY PLAN ${sql}`).all(...params)
    .map((row) => row.detail);
}
```

backup.mjs · JavaScript — চলবে: Node.js

```
import { existsSync } from 'node:fs';
import { DatabaseSync } from 'node:sqlite';

// চালু Database-এর একটি সামঞ্জস্যপূর্ণ Copy (VACUUM INTO), তারপর যাচাই:
// Backup খোলা যায়, ক্ষতিগ্রস্ত নয়, এবং প্রতিটি Table-এ সারির সংখ্যা মেলে।
export function backupTo(db, file) {
  if (existsSync(file)) {
    throw new Error(`${file} আগেই আছে; পুরোনো Backup মুছে যাবে না`);
  }
  db.prepare('VACUUM INTO ?').run(file);
  const copy = new DatabaseSync(file, { readOnly: true });
  try {
    const check = copy.prepare('PRAGMA integrity_check').get();
    const integrity = check.integrity_check;
    const counts = {};
    for (const table of ['customers', 'accounts', 'ledger_entries']) {
      const sql = `SELECT COUNT(*) AS n FROM ${table}`;
      const original = db.prepare(sql).get().n;
      const backedUp = copy.prepare(sql).get().n;
      if (original !== backedUp) {
        throw new Error(`${table}: ${original} বনাম ${backedUp} সারি`);
```

```
      }
      counts[table] = backedUp;
    }
    return { integrity, counts };
  } finally {
    copy.close();
  }
}
```

postgres-ledger.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// PostgreSQL-এর জন্য একই Ledger-এর চুক্তি (pg Driver, async)।
// এই বইয়ের QA-তে চালানো হয়নি: কোনো PostgreSQL Server ছিল না; শুধু Syntax যাচাই।
import pg from 'pg';

export const createPool = (connectionString) =>
  new pg.Pool({ connectionString, max: 10, idleTimeoutMillis: 30_000 });

// ...
```

postgres-ledger.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

// PostgreSQL-এর Error Code: 23505 Unique, 23514 Check, 23503 Foreign Key।
export function createPgLedger(pool) {
  async function run(key, steps) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      for (const [i, step] of steps.entries()) {
        await client.query(`INSERT INTO ledger_entries
          (customer_id, points, reason, idempotency_key, created_at)
          VALUES ($1, $2, $3, $4, now())`,
        [step.customerId, step.points, step.reason,
          key ? `${key}:${i}` : null]);
        const { rowCount } = await client.query(
          'UPDATE accounts SET balance = balance + $1 WHERE customer_id = $2',
          [step.points, step.customerId]);
        if (rowCount !== 1) {
          throw new Error('গ্রাহকের Account নেই');
        }
      }
      await client.query('COMMIT');
      return { duplicate: false };
    } catch (error) {
      await client.query('ROLLBACK');
      if (error.code === '23505' && key) {
        return { duplicate: true };
```

```
      }
      throw error;
    } finally {
      client.release();
    }
  }

  return {
    earn: (customerId, points, reason, key) =>
      run(key, [{ customerId, points, reason }]),
    redeem: (customerId, points, reason, key) =>
      run(key, [{ customerId, points: -points, reason }]),
  };
}
```

demo.mjs

demo.mjs · JavaScript চলবে: Node.js

```
import { mkdirSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase } from './src/db.mjs';
import { migrate } from './src/migrate.mjs';
import { createCustomerStore } from './src/customers.mjs';
import { createLedger } from './src/ledger.mjs';
import { topCustomers, monthlyPoints } from './src/reports.mjs';
import { backupTo } from './src/backup.mjs';

// শিক্ষামূলক Demo: Database File Operating System-এর Temp Folder-এ, প্রতিবার নতুন।
const dir = join(tmpdir(), 'travel-data-demo');
rmSync(dir, { recursive: true, force: true });
mkdirSync(dir);
const db = openDatabase(join(dir, 'travel.db'));

console.log('Migration:', migrate(db).join(', '));
const customers = createCustomerStore(db);
const now = () => new Date('2026-10-18T10:00:00Z');
const ledger = createLedger(db, { now });
const rafi = customers.create({ name: 'Rafi Ahmed', email: 'rafi@example.com',
  phone: '01712345678', city: 'Dhaka' });
const mina = customers.create({ name: 'Mina Akter', email: 'mina@example.com',
  phone: '01812345678', city: 'Sylhet' });

ledger.earn(rafi, 6_000, "Cox's Bazar tour", 'trip-1001');
ledger.earn(rafi, 6_000, "Cox's Bazar tour", 'trip-1001');
ledger.transfer(rafi, mina, 1_500, 'family-77');
try {
  ledger.redeem(mina, 9_000, 'Free hotel night');
```

```
} catch (error) {
  console.log(`${error.code}: ${error.message}`);
}

console.log(topCustomers(db).map((c) => `${c.name} ${c.balance}`).join(' | '));
console.log(JSON.stringify(monthlyPoints(db, rafi)));
console.log('Audit:', ledger.audit().length === 0 ? 'balance ও Ledger মেলে' :
    'অমিল!');
const { integrity, counts } = backupTo(db, join(dir, 'backup.db'));
console.log(`Backup: ${integrity}, ${JSON.stringify(counts)}`);
db.close();
```

Output

```
Migration: 001-init.sql, 002-indexes.sql, 003-tier.sql
CHECK: যথেষ্ট Point নেই
Rafi Ahmed 4500 | Mina Akter 1500
[{"month":"2026-10","earned":6000,"spent":1500}]
Audit: balance ও Ledger মেলে
Backup: ok, {"customers":2,"accounts":2,"ledger_entries":3}
```

একই Tour-এর Point (`trip-1001`) দুবার পাঠানো হলেও একবারই যোগ হয়েছে; পরিবারে ১,৫০০ Point হস্তান্তর; Mina-র ৯,০০০ Point ব্যয়ের চেষ্টা Constraint-এ আটকে Rollback। Audit মিলেছে, এবং Backup যাচাই করা।

Test: Transaction ও Concurrency

ledger.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('transfer পরমাণু: ব্যর্থ হলে দুই পক্ষই অপরিবর্তিত', () => {
  const { db, customers, ledger } = memoryStore();
  const [a, b] = [newCustomer(customers), newCustomer(customers)];
  ledger.earn(a, 1_000, 'tour');
  assert.throws(() => ledger.transfer(a, b, 5_000), { code: 'CHECK' });
  const both = () => [ledger.balanceOf(a).balance, ledger.balanceOf(b).balance];
  assert.deepEqual(both(), [1_000, 0]);
  assert.throws(() => ledger.transfer(a, 999, 100), { code: 'FOREIGN_KEY' });
  assert.equal(ledger.balanceOf(a).balance, 1_000);
  assert.equal(entries(db), 1);
  ledger.transfer(a, b, 400);
  assert.deepEqual(both(), [600, 400]);
});

// ...
```

concurrency.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('একজন লিখছেন: দ্বিতীয় Writer "database is locked" পান, পরে সফল', (t) => {
  const { first, second } = twoConnections(t);
  first.exec('BEGIN IMMEDIATE');
  assert.throws(() => second.exec('BEGIN IMMEDIATE'), /database is locked/);
  first.exec('COMMIT');
  second.exec('BEGIN IMMEDIATE');
  second.exec('COMMIT');
});

test('Lost Update: পড়ে, হিসাব করে, লেখা (দুই সংযোগে) একটি হালনাগাদ হারায়', (t) => {
  const { first, second, id } = twoConnections(t);
  const readA = balance(first, id);
  const readB = balance(second, id);
  first.prepare(SET).run(readA + 100, id);
  second.prepare(SET).run(readB + 50, id);
  assert.equal(balance(first, id), 50);
});

// ...
```

প্রথম Test দেখায় Transfer-এর পরমাণুতা: যথেষ্ট Point না থাকলে, বা প্রাপক না থাকলে, দুই পক্ষই অপরিবর্তিত এবং Ledger-এ কোনো আধা-সারি নেই। দ্বিতীয়টি একই Database File-এ দুটি আসল সংযোগ খোলে: একজন `BEGIN IMMEDIATE` ধরে রাখলে দ্বিতীয়জন "database is locked" পান; এবং পড়ে-হিসাব-করে-লেখা দুই সংযোগে চালালে ১০০ + ৫০-এর বদলে ফল ৫০—Lost Update প্রমাণিত, তারপর পরের দুটি Test-এ তার দুটি সমাধান।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-data
node --test
node demo.mjs
```

`node --test` ২১টি Test চালায়: `schema` ৬ (Migration-এর ক্রম ও Checksum, প্রতিটি Constraint, Foreign Key, ব্যর্থ create-এ কিছুই না থাকা, Data Migration), `ledger` ৬ (earn/redeem, Rollback, পরমাণু Transfer, Idempotency, অবৈধ Point, ৫০০ কাজের Property), `concurrency` ৪ (Lock, Lost Update ও দুটি সমাধান), এবং `queries` ৫ (Query Plan, Keyset Pagination, নিরাপদ খোঁজ, Report, Backup); শেষে `# pass 21`, `# fail 0`। `node:sqlite` একটি "ExperimentalWarning" দেখায়—স্বাভাবিক। Node.js 22.5 বা নতুন লাগে; কোনো Package লাগে না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Foreign Key ভুল সত্ত্বেও লেখা সফল	`PRAGMA foreign_keys` বন্ধ	প্রতিটি সংযোগে `openDatabase`
"database is locked"	অন্য সংযোগ লেখার Transaction ধরে আছে	Transaction ছোট, `busy_timeout`, Retry
Migration থেমে যায়: "চলার পরে বদলানো"	পুরোনো Migration File সম্পাদিত	পরিবর্তনটি নতুন Migration-এ
Query ধীর	Plan-এ SCAN	উপযুক্ত Index, তারপর আবার Plan
Audit-এ অমিল	কেউ Transaction ছাড়া বা সরাসরি SQL-এ Balance বদলেছে	Ledger-এর বাইরে লেখা বন্ধ
Windows-এ Test-এর Temp File মোছা যায় না	Database খোলা	আগে `close()`, তারপর মোছা

২৬.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
শুধু JavaScript-এ যাচাই	অন্য পথে ভুল তথ্য ঢোকে	Constraint
একাধিক লেখা Transaction ছাড়া	আধা-সম্পূর্ণ অবস্থা	`withTransaction`
পড়ে, Code-এ হিসাব, তারপর লেখা	Lost Update	`balance = balance + ?`, Lock, Version
প্রতিটি সারির জন্য আলাদা Query	N+1	JOIN, Eager Loading
প্রতিটি Column-এ Index	লেখা ধীর, জায়গা বেশি	Plan দেখে নির্বাচিত Index
`OFFSET` দিয়ে গভীর পাতা	প্রতিবার হাজারো সারি বাদ	Keyset Pagination
চলে যাওয়া Migration সম্পাদনা	Database-ভেদে ভিন্ন Schema	নতুন Migration
যাচাই-না-করা Backup	দরকারের দিন অকেজো	Restore করে দেখা
SQLite-এ Foreign Key ধরে নেওয়া	Default বন্ধ	`PRAGMA foreign_keys = ON`
ORM-এর Generate করা SQL কখনো না দেখা	লুকানো ধীর Query	Query Log ও Plan

২৬.১৩ Debugging নির্দেশিকা

১. **Constraint Error:** বার্তার Table, Column বা Constraint-এর নাম পড়ুন; Code-এর যাচাই আর Database-এর নিয়ম মেলে কি না দেখুন।
২. **ধীর Query:** `EXPLAIN QUERY PLAN` (PostgreSQL-এ `EXPLAIN ANALYZE`); SCAN ও TEMP B-TREE খুঁজুন।
৩. **Query-র সংখ্যা গুনুন:** একটি পাতায় কতগুলো Query চলে (N+1)।
৪. **Lock:** কোন Transaction দীর্ঘ সময় খোলা; Transaction-এর ভেতরে Network Call বা ব্যবহারকারীর অপেক্ষা নেই তো?
৫. **তথ্যের অমিল:** Audit Query; Ledger থেকে আবার হিসাব।
৬. **Migration:** `schema_migrations` Table দেখুন—কোনগুলো চলেছে, কখন।

২৬.১৪ পেশাদার অভ্যাস

- **প্রতিটি ব্যবসার নিয়ম, যতটা সম্ভব, একটি নামযুক্ত Constraint-এ।**
- **একসঙ্গে বদলাতে হবে এমন সব লেখা একটি Transaction-এ;** Transaction ছোট।
- **টাকা ও Point: Append-only Ledger, Integer, এবং Audit।**
- **Idempotency Key-র জন্য UNIQUE Constraint।**
- **প্রতিটি Schema পরিবর্তন একটি Migration,** Git-এ, Code Review-সহ।
- **Index Plan দেখে, মেপে।**
- **Backup স্বয়ংক্রিয়, এবং নিয়মিত Restore-এর অনুশীলন।**
- **Test-এ আসল Database (Memory বা Temp File),** নকল নয়—Constraint ও Transaction-এর আচরণ নকল করা যায় না।

২৬.১৫ অনুশীলনী

প্রাথমিক স্তর

১. `tours` Table-এর জন্য Constraint লিখুন: `price` ধনাত্মক, `days` ১ থেকে ৩০, `code` অনন্য।
২. Normalization-এর আগে-পরের দুটি Table-এর নকশা করুন: `bookings`-এ বারবার থাকা Tour-এর নাম ও দাম।
৩. ACID-এর প্রতিটি অক্ষর এক বাক্যে।
৪. `SELECT * FROM customers WHERE email = ?`-এর জন্য কোন Index? Plan দিয়ে যাচাই করুন।
৫. N+1 সমস্যা কী? এক বাক্যে, একটি উদাহরণসহ।

মধ্যম স্তর

৬. Ledger-এ `reverse(entryId)` যোগ করুন: একটি লেনদেনের উল্টো লেনদেন, মূলটি না বদলে।

৭. একটি Migration `004-referral.sql` লিখুন: `customers`-এ ঐচ্ছিক `referred_by` (অন্য গ্রাহকের Foreign Key)।

৮. "প্রতিটি শহরের মোট Point" Report-এর Query লিখুন।

৯. একটি Transaction-এর ভেতরে তিনটি গ্রাহক Import করুন, যার একটি অবৈধ; Savepoint দিয়ে শুধু সেটি বাদ দিন।

চ্যালেঞ্জ স্তর

১০. Lock না পেলে ৩ বার Retry করা `withRetry(fn)` লিখুন, এবং দুটি সংযোগে পরীক্ষা করুন।

১১. Ledger থেকে সব Balance নতুন করে হিসাব করার একটি "Repair" Query লিখুন, এবং কেন এটি শুধু জরুরি অবস্থায়—ব্যাখ্যা করুন।

১২. LeadDesk (অধ্যায় ২৫)-এর Repository-কে Migration ও Constraint-সহ SQLite-এ নেওয়ার নকশা করুন: Table, Index, Idempotency ও Version।

২৬.১৬ অনুশীলনীর সমাধান

সমাধান ১

tour-constraints.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE tours (
  code TEXT PRIMARY KEY,
  price INTEGER NOT NULL CONSTRAINT price_positive CHECK (price > 0),
  days INTEGER NOT NULL CONSTRAINT days_range CHECK (days BETWEEN 1 AND 30))`);

const tries = [
  ['SJK-2', 6500, 2], ['SJK-2', 7000, 3], ['KTA-0', 0, 2], ['LNG-31', 9000, 31],
];
for (const row of tries) {
  try {
    db.prepare('INSERT INTO tours VALUES (?, ?, ?)').run(...row);
    console.log(`${row[0]}: যোগ হয়েছে`);
  } catch (error) {
    console.log(`${row[0]}: ${error.message}`);
  }
}
```

Output

```
SJK-2: যোগ হয়েছে
SJK-2: UNIQUE constraint failed: tours.code
KTA-0: CHECK constraint failed: price_positive
LNG-31: CHECK constraint failed: days_range
```

সমাধান ২ — আগে: `bookings(id, customer, tour_name, tour_price, date)`—Tour-এর দাম বদলালে শত সারি হালনাগাদ, এবং পুরোনো ও নতুন দাম মিশে যায়। পরে: `tours(code PK, name, price)` এবং `bookings(id, customer_id FK, tour_code FK, price_paid, date)`। লক্ষ করুন, `price_paid` ইচ্ছাকৃতভাবে Booking-এ রাখা: এটি পুনরাবৃত্তি নয়, ঐতিহাসিক সত্য—বুকিংয়ের দিনের দাম, যা পরে Tour-এর দাম বদলালেও বদলাবে না।

সমাধান ৩ — Atomicity: সব বা কিছুই না। Consistency: শেষে সব Constraint সত্য। Isolation: অন্যদের আধা-কাজ দেখা যায় না। Durability: `COMMIT`-এর পর স্থায়ী।

সমাধান ৪

email-index.mjs · JavaScript — চলবে: Node.js

```javascript
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec('CREATE TABLE customers (id INTEGER PRIMARY KEY, email TEXT, name TEXT)');
const QUERY = 'SELECT * FROM customers WHERE email = ?';
const plan = () => db.prepare(`EXPLAIN QUERY PLAN ${QUERY}`)
  .all('a@b.c').map((row) => row.detail).join('');

console.log(plan());
db.exec('CREATE UNIQUE INDEX customers_email ON customers (email)');
console.log(plan());
```

Output

```
SCAN customers
SEARCH customers USING INDEX customers_email (email=?)
```

TravelData-র `email TEXT UNIQUE` নিজেই একটি Index তৈরি করে, তাই সেখানে আলাদা Index লাগে না।

সমাধান ৫ — একটি তালিকা আনতে ১টি Query, তারপর তালিকার প্রতিটি সারির জন্য আরও একটি করে Query (N টি)—যেমন ৫০ জন গ্রাহকের প্রত্যেকের Point আলাদা Query-তে; সমাধান JOIN বা একটি `IN (...)` Query (উদাহরণ ০৬)।

সমাধান ৬

reverse.mjs · JavaScript — চলবে: Node.js

```
import { openDatabase } from '../../project/travel-data/src/db.mjs';
import { migrate } from '../../project/travel-data/src/migrate.mjs';
import { createCustomerStore } from '../../project/travel-data/src/customers.mjs';
import { createLedger } from '../../project/travel-data/src/ledger.mjs';

const db = openDatabase();
migrate(db);
const id = createCustomerStore(db).create({ name: 'Sara Islam', email: 's@e.com',
  phone: '01912345678', city: 'Bogura' });
const ledger = createLedger(db);
ledger.earn(id, 700, 'wrong tour');

function reverse(entryId) {
  const entry = db.prepare('SELECT * FROM ledger_entries WHERE id = ?')
    .get(entryId);
  return ledger.earn(entry.customer_id, -entry.points,
    `reversal of #${entryId}`, `reverse-${entryId}`);
}

reverse(1);
reverse(1);
const rows = db.prepare(
  'SELECT points, reason FROM ledger_entries ORDER BY id').all();
console.log(rows.map((r) => `${r.points} (${r.reason})`).join(' | '));
console.log(ledger.balanceOf(id).balance, ledger.audit().length);
```

Output

```
700 (wrong tour) | -700 (reversal of #1)
0 0
```

মূল সারি অপরিবর্তিত; ইতিহাসে দেখা যায় কী ঘটেছিল এবং কীভাবে সংশোধন হয়েছে। `reverse-1` Key থাকায় দ্বিতীয়বার উল্টানো হয়নি।

সমাধান ৭

004-referral.sql · SQL — চলবে: Node.js

```
-- ঐচ্ছিক: পুরোনো সারি NULL থাকে, তাই কিছু ভাঙে না।
ALTER TABLE customers ADD COLUMN referred_by INTEGER
  REFERENCES customers (id) ON DELETE SET NULL;
CREATE INDEX customers_referred_by ON customers (referred_by);
```

apply.mjs · JavaScript — চলবে: Node.js

```
import { cpSync, mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { fileURLToPath } from 'node:url';
import { openDatabase } from '../../../project/travel-data/src/db.mjs';
import { migrate } from '../../../project/travel-data/src/migrate.mjs';

const dir = mkdtempSync(join(tmpdir(), 'mig-'));
const project = new URL('../../../project/travel-data/migrations/',
  import.meta.url);
cpSync(fileURLToPath(project), dir, { recursive: true });
cpSync(fileURLToPath(new URL('./004-referral.sql', import.meta.url)),
  join(dir, '004-referral.sql'));

const db = openDatabase();
console.log(migrate(db, dir).join(', '));
const columns = db.prepare('PRAGMA table_info(customers)').all()
  .map((c) => c.name);
console.log(columns.includes('referred_by'));
rmSync(dir, { recursive: true, force: true });
```

Output

```
001-init.sql, 002-indexes.sql, 003-tier.sql, 004-referral.sql
true
```

সমাধান ৮

city-report.sql · SQL — চলবে: Node.js

```
SELECT c.city, COUNT(*) AS customers, SUM(a.balance) AS points
FROM customers c
JOIN accounts a ON a.customer_id = c.id
GROUP BY c.city
ORDER BY points DESC;
```

`customers_city` Index (Migration ০০২) এই Query-র `GROUP BY`-তে সাহায্য করতে পারে; Plan দিয়ে যাচাই করুন।

সমাধান ৯ — উদাহরণ ০৪-এর ধাঁচ: `BEGIN`, প্রতিটি গ্রাহকের আগে `SAVEPOINT row`, সফল হলে `RELEASE row`, ব্যর্থ হলে `ROLLBACK TO row` ও `RELEASE row` এবং ভুলের তালিকায় যোগ, শেষে `COMMIT`। কিন্তু খেয়াল রাখুন, TravelData-র `customers.create` নিজেই `withTransaction` চালায়, এবং SQLite-এ Transaction-এর

ভেতরে আরেকটি `BEGIN` চলে না—তাই Import-এর জন্য Store-এ একটি আলাদা Function লাগবে, যা Transaction খোলে না, শুধু Savepoint ব্যবহার করে। এটি একটি বাস্তব নকশার প্রশ্ন: Transaction-এর সীমানা কে ঠিক করবে—Repository, না যে Use Case তাকে ডাকে? বড় Project-এ উত্তর সাধারণত দ্বিতীয়টি (Unit of Work)।

সমাধান ১০

retry.mjs · JavaScript চলবে: Node.js

```
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase, withTransaction }
  from '../../project/travel-data/src/db.mjs';

function withRetry(fn, { attempts = 3, onRetry = () => {} } = {}) {
  for (let i = 1; ; i += 1) {
    try {
      return fn();
    } catch (error) {
      if (!/database is locked/.test(error.message) || i >= attempts) {
        throw error;
      }
      onRetry(i);
    }
  }
}

const dir = mkdtempSync(join(tmpdir(), 'retry-'));
const a = openDatabase(join(dir, 'x.db'), { busyTimeoutMs: 0 });
const b = openDatabase(join(dir, 'x.db'), { busyTimeoutMs: 0 });
a.exec('CREATE TABLE t (n INTEGER)');

a.exec('BEGIN IMMEDIATE');
const retries = [];
const result = withRetry(() => withTransaction(b, () => {
  b.exec('INSERT INTO t VALUES (1)');
  return 'লেখা হয়েছে';
}), { onRetry: (i) => {
  retries.push(i);
  if (i === 2) {
    a.exec('COMMIT');
  }
} });
console.log(result, retries);
a.close();
b.close();
rmSync(dir, { recursive: true, force: true });
```

Output

```
লেখা হয়েছে [ 1, 2 ]
```

দুবার Lock-এ ব্যর্থ, তৃতীয়বার (অন্য সংযোগ ছেড়ে দেওয়ার পর) সফল। বাস্তবে Retry-এর মাঝে অপেক্ষা (Backoff, অধ্যায় ১৬) এবং শুধু Retry-যোগ্য Error-এ—যেমন এখানে Lock, কিন্তু Constraint কখনো নয়।

সমাধান ১১

repair.sql · SQL — চলবে: Node.js

```sql
-- জরুরি অবস্থায়, Backup-এর পরে, একটি Transaction-এ:
UPDATE accounts SET balance = (
  SELECT COALESCE(SUM(points), 0) FROM ledger_entries e
  WHERE e.customer_id = accounts.customer_id
);
```

এটি শুধু জরুরি অবস্থায়, কারণ: Audit-এ অমিল মানে কোথাও একটি Bug বা নিয়মবহির্ভূত লেখা আছে—Repair সেই কারণ লুকিয়ে ফেলে; বড় Table-এ এটি দীর্ঘ সময় Lock ধরে রাখে; এবং যদি Ledger-ই ভুল হয়, Repair ভুলটিকে Balance-এও ছড়িয়ে দেয়। আগে কারণ খুঁজুন, Backup নিন, তারপর Repair, এবং কী বদলাল তা Log করুন।

সমাধান ১২ — Table: `leads(id TEXT PRIMARY KEY, name, phone CHECK(...), destination, travelers CHECK (travelers BETWEEN 1 AND 50), status CHECK (status IN (...)), assignee, version INTEGER NOT NULL DEFAULT 1, created_at)` । Index: `(assignee, id)` ও `(status, id)` —তালিকা ও Filter-এর Keyset Pagination-এর জন্য। Idempotency: `idempotency_keys(user_id, key, fingerprint, lead_id, created_at, PRIMARY KEY (user_id, key))`, Lead তৈরি ও Key সংরক্ষণ একই Transaction-এ। Version: `UPDATE leads SET ..., version = version + 1 WHERE id = ? AND version = ?` এবং `changes === 0` হলে 412 (অধ্যায় ২৫-এর সমাধান ১২)। ID: `AUTOINCREMENT` থেকে, গণনা থেকে নয়। সবকিছু `001-init.sql`-এ, এবং Service-এর একটি লাইনও বদলায় না।

২৬.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Normalization-এর প্রধান লক্ষ্য?

ক) Query দ্রুত করা

খ) প্রতিটি তথ্য একবার রেখে পুনরাবৃত্তি ও অসামঞ্জস্য দূর করা

গ) Table-এর সংখ্যা কমানো

ঘ) Index তৈরি

প্রশ্ন ২. SQLite-এ Foreign Key যাচাইয়ের জন্য কী লাগে?

ক) `PRAGMA foreign_keys = ON`, প্রতিটি সংযোগে

খ) কিছুই না, সব সময় চালু

গ) একটি Index

ঘ) `UNIQUE`

প্রশ্ন ৩. Transaction-এর ভেতরে দ্বিতীয় লেখা ব্যর্থ হলে `ROLLBACK`-এর পর?

ক) প্রথম লেখা থেকে যায়

খ) Database ক্ষতিগ্রস্ত

গ) দুটি লেখাই বাতিল

ঘ) দ্বিতীয়টি আবার চেষ্টা হয়

প্রশ্ন ৪. Lost Update কেন ঘটে?

ক) Index নেই

খ) Backup নেই

গ) Constraint ভুল

ঘ) দুটি সংযোগ একই মান পড়ে, আলাদা হিসাব করে লেখে

প্রশ্ন ৫. `EXPLAIN QUERY PLAN`-এ `SCAN ledger_entries` মানে?

ক) পুরো Table পড়া হচ্ছে

খ) Index ব্যবহার হচ্ছে

গ) Query ব্যর্থ

ঘ) Cache থেকে

প্রশ্ন ৬. N+1 সমস্যার সাধারণ সমাধান?

ক) আরও Index

খ) JOIN বা Eager Loading-এ একসঙ্গে আনা

গ) Cache বন্ধ

ঘ) Transaction

প্রশ্ন ৭. চলে যাওয়া Migration-এ পরিবর্তন লাগলে?

ক) File সম্পাদনা

খ) Database মুছে নতুন করে

গ) হাতে `ALTER TABLE`

ঘ) একটি নতুন Migration লেখা

প্রশ্ন ৮. গভীর পাতায় `OFFSET`-এর বদলে কী ভালো?

ক) `WHERE id > ? ORDER BY id LIMIT ?` (Keyset)

খ) বড় `LIMIT`

গ) সব সারি একসঙ্গে

ঘ) `ORDER BY RANDOM()`

প্রশ্ন ৯. Idempotency-র সবচেয়ে নির্ভরযোগ্য Database-স্তরের নিশ্চয়তা?

ক) একটি `if`

খ) Key-তে `UNIQUE` Constraint

গ) Cache

ঘ) Index ছাড়া Table

প্রশ্ন ১০. Backup কখন নির্ভরযোগ্য বলা যায়?

ক) File তৈরি হলে

খ) প্রতিদিন হলে

গ) Restore ও যাচাই করে দেখার পর

ঘ) বড় হলে

২৬.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–ঘ · ৮–ক · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	উদাহরণ ০১-এর একই গ্রাহকের দুটি ফোন নম্বর।
২	ক	SQLite-এ Default বন্ধ; `openDatabase` চালু করে।
৩	গ	Atomicity; উদাহরণ ০৩-এ প্রমাণিত।
৪	ঘ	দ্বিতীয় লেখা প্রথমটিকে মুছে দেয়।
৫	ক	Index থাকলে SEARCH দেখা যায়।
৬	খ	৫১টির বদলে ১টি Query।
৭	ঘ	Checksum পরিবর্তন ধরে থামিয়ে দেয়।
৮	ক	Index দিয়ে সরাসরি পরের পাতায়।
৯	খ	একসঙ্গে আসা অনুরোধেও কাজ করে।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	গ	`backupTo` তাই খুলে যাচাই করে।

২৬.১৯ অধ্যায়ের সারসংক্ষেপ

- Normalization: প্রতিটি তথ্য একবার, তার মালিক Table-এ; Denormalization শুধু সচেতনভাবে, Audit-সহ।
- Constraint (PK, FK, UNIQUE, CHECK, NOT NULL) Database-কে নিয়মের পাহারাদার করে; নাম দিন।
- Transaction: ACID; একসঙ্গে বদলানো সব লেখা একটিতে; Savepoint দিয়ে আংশিক ফেরা।
- Isolation ও Lock; Lost Update প্রতিরোধে Database-এর ভেতরে হিসাব, Lock বা Version।
- Index ও Query Plan; N+1-এর বদলে JOIN; Keyset Pagination।
- Migration: ক্রমিক, Transaction-এ, Checksum-সহ; পুরোনোটি কখনো বদলাবেন না।
- Backup যাচাই না হলে Backup নয়।
- ORM সুবিধাজনক, কিন্তু SQL ও Plan জানা জরুরি; SQLite ও PostgreSQL-এর পার্থক্য জেনে Adapter।

পরের অধ্যায়ে এই বইয়ের যাত্রা আরও এগোবে; এ পর্যন্ত আপনি Browser থেকে Server, Test থেকে Database পর্যন্ত একটি পেশাদার JavaScript Application-এর প্রতিটি স্তর তৈরি করেছেন।

২৬.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Primary Key / Foreign Key	সারির অনন্য পরিচয় / অন্য Table-এর সারির উল্লেখ।
Constraint	Database-এর ভেতরের নিয়ম।
Normalization	পুনরাবৃত্তি দূর করে তথ্য সাজানো।
Transaction (ACID)	অবিভাজ্য কাজের একক: Atomicity, Consistency, Isolation, Durability।
Savepoint	Transaction-এর ভেতরে ফেরার বিন্দু।
Lost Update	একটি হালনাগাদ অন্যটিকে মুছে দেওয়া।
Index	দ্রুত খোঁজার সাজানো কাঠামো।
Query Plan	Database Query কীভাবে চালাবে তার বর্ণনা।
N+1	তালিকার প্রতিটি সারির জন্য আলাদা Query।
Migration	Schema পরিবর্তনের ক্রমিক File।
ORM	Table-কে Object হিসেবে ব্যবহারের Library।

পরিভাষা	অর্থ
Append-only Ledger	শুধু যোগ হয় এমন লেনদেনের তালিকা।

২৬.২১ আরও পড়ুন

- **Node.js — SQLite:** `https://nodejs.org/api/sqlite.html` — `node:sqlite`-এর API।
- **SQLite — Query Planning:** `https://www.sqlite.org/queryplanner.html` — Index ও Plan কীভাবে কাজ করে।
- **PostgreSQL — Transaction Isolation:** `https://www.postgresql.org/docs/current/transaction-iso.html` — Isolation Level।
- **PostgreSQL — Using EXPLAIN:** `https://www.postgresql.org/docs/current/using-explain.html` — Query Plan পড়া।
- **Prisma — Documentation:** `https://www.prisma.io/docs` — ORM, Migration ও Client।

অধ্যায় ২৭

২৭

Real-Time JavaScript Applications

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Short Polling, Long Polling, Server-Sent Events (SSE) ও WebSocket-এর পার্থক্য ও উপযুক্ত ব্যবহার বিচার করতে।
- Node.js-এ SSE Stream ও WebSocket Server তৈরি করতে, এবং Browser ও Node.js-এর WebSocket Client দিয়ে সংযোগ করতে।
- WebSocket-এর Lifecycle (Handshake, `open`, `message`, `close`), Close Code ও Heartbeat বুঝতে।
- Socket.IO-র Event, Room ও Acknowledgement-এর ধারণা বুঝতে, এবং কেন তা Raw WebSocket-এর সঙ্গে সরাসরি মেলে না তা ব্যাখ্যা করতে।
- একটি স্থির Event Envelope (eventId, type, version, sequence) নকশা ও যাচাই করতে।
- Command ও Event আলাদা করতে, এবং Transactional Outbox দিয়ে "Database-এ লেখা হলো কিন্তু Event গেল না" সমস্যা এড়াতে।
- At-most-once, At-least-once ও Exactly-once Effect বুঝতে; Duplicate, পুরোনো ও হারানো Event সামলাতে।
- Exponential Backoff ও Jitter দিয়ে পুনঃসংযোগ, এবং Bounded Buffer দিয়ে Backpressure সামলাতে।
- একাধিক Server Instance-এ Pub/Sub দিয়ে Event ছড়ানোর নকশা ও তার সীমা বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **TravelLive** Real-Time Booking Dashboard তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৩ ও ১৪: Promise, `async` / `await`, Event Loop।
- অধ্যায় ১৭ ও ২০: Browser-এর DOM Event, Input Validation, Authentication ও Authorization।
- অধ্যায় ২২: `node:test`, Integration Test।
- অধ্যায় ২৫: Node.js HTTP Server, Middleware, JSON Error চুক্তি, Idempotency।
- অধ্যায় ২৬: Transaction ও `node:sqlite`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার অফিস। Agent নাদিয়া একটি Booking নিশ্চিত করলেন। একই মুহূর্তে গ্রাহক রাফি তাঁর Phone-এ Booking-এর পাতা খুলে বসে আছেন, আর পাশের Desk-এ আরেকজন Agent একই Booking বাতিলের কথা ভাবছেন। রাফির Screen-এ যদি পাঁচ মিনিট পরেও "অপেক্ষমাণ" লেখা থাকে, তিনি ফোন করবেন; দ্বিতীয় Agent যদি পুরোনো অবস্থা দেখে সিদ্ধান্ত নেন, ভুল হবে।

Page বারবার Refresh করা বা প্রতি কয়েক সেকেন্ডে Server-কে জিজ্ঞেস করা সমাধান নয়—তাতে হাজারো অপ্রয়োজনীয় অনুরোধ যায়, আর তবুও দেরি হয়। **Real-Time** Application-এ Server নিজেই পরিবর্তনের খবর Client-এ পৌঁছে দেয়।

কিন্তু শুধু একটি Socket খুলে Message পাঠালেই কাজ শেষ নয়। Network কেটে যায় এবং ফিরে আসে; একই Event দুবার পৌঁছায়; কখনো একটি হারিয়ে যায়; ধীর Client-এর জন্য Server-এর Memory ভরে ওঠে; আর সংযুক্ত থাকা মানেই সবকিছু দেখার অনুমতি থাকা নয়। এই অধ্যায় এই প্রতিটি সমস্যা আসল Code ও আসল Network সংযোগ দিয়ে দেখায়, এবং শেষে একটি Browser Dashboard তৈরি করে যা এই সব পরিস্থিতিতে সঠিক থাকে।

২৭.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
HTTP Request ও Response (অধ্যায় ২৫)	দীর্ঘস্থায়ী সংযোগ: SSE Stream ও WebSocket
Token দিয়ে Authentication (অধ্যায় ২০)	সংযোগের প্রথম Message-এ Auth; প্রতিটি Subscribe ও Dispatch-এ অনুমতি
Idempotency-Key (অধ্যায় ২৫, ২৬)	Event-এর eventId দিয়ে Duplicate বাদ
Transaction (অধ্যায় ২৬)	Transactional Outbox
Promise ও Timer (অধ্যায় ১৩, ১৪)	Heartbeat, Backoff, Jitter
Integration Test (অধ্যায় ২২)	আসল WebSocket সংযোগে Test, আসল Browser-এ End-to-End

> **নোট:** WebSocket Server-এর জন্য এই অধ্যায় জনপ্রিয় `ws` Package ব্যবহার করে (`npm install ws`)। QA-যন্ত্রে নতুন কোনো Package Install করা হয়নি; আগে থেকে থাকা `ws` 7.5.10 দিয়ে সব Code চালানো হয়েছে, এবং Code শুধু সেই API ব্যবহার করে যা `ws` -এর ৭ ও ৮ দুই সংস্করণেই আছে। Client হিসেবে Node.js 22-এর অন্তর্নির্মিত `WebSocket` ও আসল Chrome Browser ব্যবহৃত হয়েছে। Socket.IO ও Redis QA-যন্ত্রে ছিল না: Socket.IO-র Code শুধু নকশা হিসেবে দেখানো হয়েছে (চালানো হয়নি), আর Redis Pub/Sub-এর আচরণ একটি ছোট in-memory অনুকরণ দিয়ে দেখানো হয়েছে—যেখানে যা, স্পষ্ট লেখা আছে।

২৭.২ Real-Time-এর চারটি পথ

Server থেকে Client-এ নতুন খবর পৌঁছানোর চারটি প্রচলিত কৌশল:

কৌশল	কীভাবে	কখন উপযুক্ত	সীমা
Short Polling	Client নির্দিষ্ট বিরতিতে বারবার HTTP অনুরোধ	কদাচিৎ পরিবর্তন, খুব সাধারণ Client	অপ্রয়োজনীয় অনুরোধ, বিরতির অর্ধেক গড় দেরি
Long Polling	Server উত্তর আটকে রাখে, খবর এলে বা সময় শেষে উত্তর	পুরোনো পরিবেশে বিকল্প	প্রতিটি খবরের পর নতুন অনুরোধ
SSE	একটি HTTP Response যা শেষ হয় না; Server Event লিখতে থাকে	Notification, Status Feed, Dashboard	একমুখী (Server থেকে Client), শুধু Text
WebSocket	HTTP থেকে Upgrade করা দ্বিমুখী স্থায়ী সংযোগ	Chat, যৌথ সম্পাদনা, Live Dashboard	সংযোগের জীবনচক্র ও Scaling নিজে সামলাতে হয়

Polling-এর খরচ সংখ্যায় দেখা যাক। ১০০০ জন Operator এক ঘণ্টা Dashboard খোলা রাখেন, আর সেই ঘণ্টায় আসল পরিবর্তন মাত্র ৪০টি:

polling-cost.mjs · JavaScript চলবে: Node.js

```
// ১০০০ জন Operator, ১ ঘণ্টা, আর ঘণ্টায় মাত্র ৪০টি আসল পরিবর্তন।
const clients = 1000;
const seconds = 60 * 60;
const changes = 40;

function polling(intervalSeconds) {
  const requests = clients * Math.floor(seconds / intervalSeconds);
  const useful = clients * changes;
  return { requests, wasted: requests - useful,
    averageDelay: intervalSeconds / 2 };
}

console.log('Polling ৫ সেকেন্ড:', polling(5));
console.log('Polling ৬০ সেকেন্ড:', polling(60));
// Push (SSE/WebSocket): প্রতিটি পরিবর্তন একবার প্রতিটি Client-এ।
console.log('Push:', { messages: clients * changes, averageDelay: 0 });
```

Output

```
Polling ৫ সেকেন্ড: { requests: 720000, wasted: 680000, averageDelay: 2.5 }
Polling ৬০ সেকেন্ড: { requests: 60000, wasted: 20000, averageDelay: 30 }
Push: { messages: 40000, averageDelay: 0 }
```

প্রতি ৫ সেকেন্ডের Polling-এ ৭ লাখ ২০ হাজার অনুরোধের ৬ লাখ ৮০ হাজারই অর্থহীন, তবু গড়ে আড়াই সেকেন্ড দেরি। বিরতি বাড়ালে অনুরোধ কমে, কিন্তু দেরি আধা মিনিট। Push-এ (SSE বা WebSocket) ঠিক যতগুলো পরিবর্তন, ততগুলো Message, এবং দেরি প্রায় শূন্য। তবু Polling-কে তুচ্ছ ভাববেন না: দিনে কয়েকবার বদলায় এমন তথ্যের জন্য, বা যেখানে Proxy দীর্ঘ সংযোগ কেটে দেয়, Polling সবচেয়ে সরল ও নির্ভরযোগ্য।

সিদ্ধান্তের নিয়ম: শুধু Server থেকে খবর আসবে? SSE দিয়ে শুরু করুন—এটি সাধারণ HTTP, Browser নিজেই পুনঃসংযোগ করে। দুই দিকেই ঘন ঘন Message? WebSocket। Room, স্বয়ংক্রিয় পুনঃসংযোগ ও Fallback একসঙ্গে চাই এবং দুই পাশেই একই Library ব্যবহার করতে পারবেন? Socket.IO।

২৭.৩ Server-Sent Events

SSE হলো একটি সাধারণ HTTP Response, যার `Content-Type` হলো `text/event-stream` এবং যা শেষ হয় না। প্রতিটি Event কয়েকটি `field: value` Line, আর একটি ফাঁকা Line (অর্থাৎ `\n\n`) Event শেষ করে। প্রধান Field চারটি: `data` (তথ্য), `event` (Event-এর নাম), `id` (Event-এর পরিচয়) ও `retry` (পুনঃসংযোগের আগে কত Millisecond অপেক্ষা)। `:` দিয়ে শুরু Line মন্তব্য—সংযোগ জীবিত রাখতে ব্যবহৃত হয়।

sse-basic.mjs · JavaScript চলবে: Node.js

```
import http from 'node:http';
```

```
// Server: সাধারণ HTTP Response, যা শেষ হয় না; প্রতিটি Event "\n\n" দিয়ে আলাদা।
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache' });
  const statuses = ['PENDING', 'CONFIRMED', 'TICKETED'];
  statuses.forEach((status, i) => setTimeout(() => {
    res.write(`id: ${i + 1}\nevent: status\ndata: {"status":"${status}"}\n\n`);
    if (i === statuses.length - 1) res.end();
  }, 20 * i));
});
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));

// Client: Browser-এ EventSource; এখানে fetch দিয়ে Stream পড়ে নিজেই ভাগ করা।
const res = await fetch(`http://127.0.0.1:${server.address().port}/`);
console.log('Content-Type:', res.headers.get('content-type'));
let buffer = '';
for await (const chunk of res.body.pipeThrough(new TextDecoderStream())) {
  buffer += chunk;
  const frames = buffer.split('\n\n');
  buffer = frames.pop();
  for (const frame of frames) {
    const fields = Object.fromEntries(frame.split('\n')
      .map((line) => [line.slice(0, line.indexOf(':')),
        line.slice(line.indexOf(':') + 2)]));
    console.log(`#${fields.id} ${fields.event}:`, JSON.parse(fields.data));
  }
}
server.close();
```

Output

```
Content-Type: text/event-stream
#1 status: { status: 'PENDING' }
#2 status: { status: 'CONFIRMED' }
#3 status: { status: 'TICKETED' }
```

Browser-এ এই ভাগ করার কাজ `EventSource` নিজেই করে: `new EventSource('/api/bookings/bk-1/stream')` এবং `source.addEventListener('status', handler)`। সংযোগ কাটলে Browser নিজেই আবার সংযোগ করে, এবং শেষ পাওয়া `id` একটি `Last-Event-ID` Header-এ পাঠায়—Server সেটি দেখে বাদ পড়া Event আবার পাঠাতে পারে। TravelLive Project ঠিক এটিই করে (অনুচ্ছেদ ২৭.১২)।

দুটি সীমা মনে রাখুন। প্রথমত, `EventSource` কোনো Custom Header (যেমন `Authorization`) পাঠাতে পারে না; Browser-এ Authentication সাধারণত Cookie দিয়ে হয়, যার জন্য একই Origin বা সঠিক CORS ও `withCredentials` লাগে। Token URL-এ বসাবেন না—URL Server Log ও Browser History-তে থেকে যায়। দ্বিতীয়ত, HTTP/1.1-এ একটি Browser একই Domain-এ সীমিত সংখ্যক (সাধারণত ছয়টি) সংযোগ খোলে; অনেক Tab-এ SSE খুললে এই সীমায় পৌঁছানো যায়। HTTP/2-তে এ সমস্যা প্রায় নেই।

২৭.৪ WebSocket: Handshake ও Lifecycle

WebSocket শুরু হয় একটি সাধারণ HTTP অনুরোধ দিয়ে, যাতে `Connection: Upgrade` ও `Upgrade: websocket` Header থাকে। Server রাজি হলে `101 Switching Protocols` দিয়ে উত্তর দেয়, এবং সেই TCP সংযোগ এরপর আর HTTP থাকে না—দুই পক্ষই যেকোনো সময় Frame পাঠাতে পারে। Browser সব সময় এই অনুরোধে `Origin` Header পাঠায়; Server-এর উচিত তা যাচাই করা।

Client-এর দিকে চারটি Lifecycle Event: `open`, `message`, `error`, `close`। `readyState`-এর মান 0 (CONNECTING), 1 (OPEN), 2 (CLOSING), 3 (CLOSED)। নিচের উদাহরণে Server `ws` Package দিয়ে, আর Client Node.js 22-এর অন্তর্নির্মিত `WebSocket`—যার API Browser-এর `WebSocket`-এর হুবহু এক:

ws-lifecycle.mjs · JavaScript — চলবে: Node.js

```
import http from 'node:http';
import WebSocketLib from 'ws';

// Server: 'ws' Package (npm install ws)। Client: Node.js 22-এর অন্তর্নির্মিত
// WebSocket, যা Browser-এর মতোই addEventListener ব্যবহার করে।
const server = http.createServer();
const wss = new WebSocketLib.Server({ server });
wss.on('connection', (socket) => {
  socket.send(JSON.stringify({ type: 'welcome' }));
  socket.on('message', (data) => {
    const msg = JSON.parse(String(data));
    socket.send(JSON.stringify({ type: 'echo', text: msg.text }));
    socket.close(1000, 'done');
  });
});
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));

const ws = new WebSocket(`ws://127.0.0.1:${server.address().port}`);
console.log('শুরুতে readyState =', ws.readyState);
ws.addEventListener('open', () => console.log('open, readyState =',
  ws.readyState));
ws.addEventListener('message', (event) => {
  const msg = JSON.parse(event.data);
  console.log('message:', msg);
  if (msg.type === 'welcome') ws.send(JSON.stringify({ text: 'হ্যালো' }));
});
ws.addEventListener('close', (event) => {
  console.log('close:', event.code, event.reason, 'clean =', event.wasClean);
  server.close();
});
```

Output

```
শুরুতে readyState = 0
open, readyState = 1
message: { type: 'welcome' }
message: { type: 'echo', text: 'হ্যালো' }
close: 1000 done clean = true
```

লক্ষ করুন, `new WebSocket(...)`-এর ঠিক পরে `readyState` 0: সংযোগ তখনো তৈরি হচ্ছে। এই অবস্থায় `send()` করলে Browser Error দেয়; তাই প্রথম Message সব সময় `open`-এর পরে।

সংযোগ বন্ধের সময় একটি **Close Code** ও কারণ পাঠানো হয়। কিছু Code Protocol-এর নির্ধারিত, আর 4000 থেকে 4999 পর্যন্ত Application নিজের অর্থে ব্যবহার করতে পারে:

Code	অর্থ	কে পাঠায়
1000	স্বাভাবিক বন্ধ	যেকোনো পক্ষ
1001	চলে যাচ্ছে (Page বন্ধ, Server Restart)	যেকোনো পক্ষ
1006	অস্বাভাবিক বন্ধ, Close Frame আসেনি (Network কাটা)	কেউ পাঠায় না; Client নিজে জানায়
1009	Message খুব বড়	গ্রহণকারী
1011	Server-এর অপ্রত্যাশিত সমস্যা	Server
4401, 4429	TravelLive-এ: Auth ব্যর্থ, Rate Limit	TravelLive Server

Production-এ সব সময় `wss://` (TLS-এর ওপর WebSocket) ব্যবহার করুন; `ws://` শুধু নিজের Computer-এ। প্রতিটি আসা Message-এর আকার সীমিত রাখুন, JSON-এর ধরন যাচাই করুন এবং প্রতি সংযোগে Message-এর হার সীমিত করুন। শুধু Client-এ যাচাই নিরাপত্তা দেয় না—যে কেউ Browser ছাড়াই একটি WebSocket Client লিখে যা খুশি পাঠাতে পারে।

২৭.৫ Socket.IO: Event, Room ও Acknowledgement

Socket.IO একটি Library, যা WebSocket-এর ওপর (প্রয়োজনে HTTP Long Polling-এ নেমে) নিজস্ব Protocol চালায়। এটি দেয়: নামযুক্ত Event (`socket.emit('booking:subscribe', data)`), **Room** (একদল Socket-কে একসঙ্গে Message), **Acknowledgement** (প্রাপক একটি Callback দিয়ে উত্তর দেয়), স্বয়ংক্রিয় পুনঃসংযোগ, এবং একাধিক Server-এ Room-এর Message পৌঁছাতে Adapter (যেমন Redis Adapter)।

গুরুত্বপূর্ণ: Socket.IO-র নিজস্ব Handshake, Frame-এর গঠন ও Event-এর Format আছে। তাই Socket.IO Client কোনো সাধারণ WebSocket Server-এ সংযোগ করতে পারে না, এবং Browser-এর সাধারণ `WebSocket` দিয়ে Socket.IO Server-এ কথা বলা যায় না—দুই পাশেই Socket.IO লাগে।

নিচের Code শুধু নকশা দেখানোর জন্য; QA-যন্ত্রে `socket.io` ছিল না বলে এটি চালানো হয়নি, শুধু Node.js-এর Syntax-যাচাই পার হয়েছে:

socketio-design.mjs · JavaScript — চলবে: Node.js

```
// শুধু নকশা দেখানোর জন্য; চালানো হয়নি। এই বইয়ের QA-যন্ত্রে socket.io নেই,
// এবং কোনো Package Install করা হয়নি। চালাতে: npm install socket.io
// (session-auth.mjs আপনার নিজের Session ও অনুমতি-যাচাইয়ের Module।)
import { createServer } from 'node:http';
import { Server } from 'socket.io';
import { authenticateSession, canViewBooking } from './session-auth.mjs';

const httpServer = createServer();
const io = new Server(httpServer, {
  cors: { origin: ['https://app.example.com'] },
  maxHttpBufferSize: 4 * 1024,
});

// Middleware: Connection-এর আগেই Session যাচাই।
io.use(async (socket, next) => {
  try {
    socket.data.user = await authenticateSession(socket.request);
    next();
  } catch {
    next(new Error('UNAUTHORIZED'));
  }
});

io.on('connection', (socket) => {
  // Acknowledgement: Client-এর কাছে ফলাফল ফেরত (ack Callback)।
  socket.on('booking:subscribe', async ({ bookingId } = {}, ack) => {
    if (typeof bookingId !== 'string') return ack?.({ ok: false });
    if (!await canViewBooking(socket.data.user.id, bookingId)) {
      return ack?.({ ok: false, code: 'FORBIDDEN' });
    }
    await socket.join(`booking:${bookingId}`);
    return ack?.({ ok: true });
  });
});

// Room-এ পাঠানো: ওই Room-এর সব Socket পায়, যে Instance-এই থাকুক
// (একাধিক Instance-এ Redis Adapter লাগবে)।
export function publish(event) {
  io.to(`booking:${event.bookingId}`).emit('booking:event', event);
}

httpServer.listen(3000);
```

তিনটি বিষয় খেয়াল করুন। এক, `io.use` Middleware সংযোগের আগেই Session যাচাই করে। দুই, Room-এ যোগ দেওয়ার আগে Server যাচাই করে ব্যবহারকারী Booking-টি দেখতে পারেন কি না—**Room-এর নাম গোপন রাখা অনুমতি নয়**; কেউ অনুমান করে `booking:bk-22` চাইতেই পারে। তিন, Room-এর সদস্যপদ একটি সংযোগের; পুনঃসংযোগ হলে Client-কে আবার Subscribe করতে হয়, এবং Server-কে আবার অনুমতি যাচাই করতে হয়। TravelLive এই একই নকশা `ws` দিয়ে নিজে তৈরি করে, যাতে প্রতিটি অংশ আপনি দেখতে ও পরীক্ষা করতে পারেন।

২৭.৬ Event Contract ও Versioning

Client ও Server যেন একই ভাষায় কথা বলে, তাই প্রতিটি Event-এর একটি স্থির খাম (**Envelope**) থাকা উচিত:

events.mjs · JavaScript — চলবে: Node.js

```
// Event Envelope: প্রতিটি Event-এর একই কাঠামো, যাতে Client নিশ্চিন্তে পড়তে পারে।
export const EVENT_VERSION = 1;

export function createEvent({ id, type, bookingId, sequence, payload, now }) {
  return Object.freeze({
    eventId: id,
    type,
    version: EVENT_VERSION,
    bookingId,
    sequence,
    occurredAt: now.toISOString(),
    payload: Object.freeze({ ...payload }),
  });
}

// Network থেকে আসা Event বিশ্বাস করবেন না: আকার ও ধরন যাচাই করুন।
export function isValidEvent(value) {
  return value !== null && typeof value === 'object'
    && typeof value.eventId === 'string' && value.eventId.length > 0
    && typeof value.type === 'string'
    && value.version === EVENT_VERSION
    && typeof value.bookingId === 'string'
    && Number.isInteger(value.sequence) && value.sequence > 0
    && typeof value.payload === 'object' && value.payload !== null;
}
```

প্রতিটি Field-এর আলাদা কাজ: `eventId` Duplicate চিনতে; `type` কোন ঘটনা (অতীতকালে নাম: `booking.status.changed`); `version` Payload-এর গঠন বদলালে পুরোনো ও নতুন Client আলাদা করতে; `bookingId` কোন Aggregate-এর Event; `sequence` সেই Booking-এর মধ্যে ক্রম ও ফাঁক ধরতে; `occurredAt` কখন ঘটেছে; এবং `payload` শুধু প্রয়োজনীয় তথ্য।

Network থেকে আসা Event-কে বিশ্বাস করার আগে যাচাই করুন:

envelope.mjs · JavaScript চলবে: Node.js

```
import { createEvent, isValidEvent }
  from '../project/travel-live/src/events.mjs';

const event = createEvent({ id: 'evt-102', type: 'booking.status.changed',
  bookingId: 'bk-22', sequence: 8, payload: { status: 'CONFIRMED' },
  now: new Date('2026-10-11T00:00:00Z') });
console.log(JSON.stringify(event, null, 2));

// Network থেকে আসা কয়েকটি সন্দেহজনক Message।
const incoming = [
  JSON.stringify(event),
  '{"type":"booking.status.changed","sequence":"8"}',
  JSON.stringify({ ...event, version: 2 }),
  JSON.stringify({ ...event, sequence: 0 }),
];
for (const text of incoming) {
  const value = JSON.parse(text);
  console.log(isValidEvent(value) ? 'গ্রহণ' : 'বাতিল', value.sequence,
    value.version);
}
```

Output

```
{
  "eventId": "evt-102",
  "type": "booking.status.changed",
  "version": 1,
  "bookingId": "bk-22",
  "sequence": 8,
  "occurredAt": "2026-10-11T00:00:00.000Z",
  "payload": {
    "status": "CONFIRMED"
  }
}
গ্রহণ 8 1
বাতিল 8 undefined
বাতিল 8 2
বাতিল 0 1
```

দ্বিতীয় Message-এ `sequence` একটি String—JSON-এ খুব সাধারণ ভুল। তৃতীয়টি `version` 2: এই Client সেটি বোঝে না, তাই অনুমান না করে বাতিল করে (বাস্তবে Log করে, এবং প্রয়োজনে পূর্ণ Snapshot চায়)। **Versioning-এর নিয়ম:** নতুন ঐচ্ছিক Field যোগ করা নিরাপদ, তাই version একই থাকতে পারে; কোনো Field-এর নাম বা অর্থ বদলালে বা সরালে version বাড়ান, এবং কিছু সময় দুই সংস্করণই পাঠান যতক্ষণ না সব Client হালনাগাদ হয়।

Payload-এ Password, Access Token, পূর্ণ ফোন নম্বর, কার্ডের তথ্য বা অপ্রয়োজনীয় আর্থিক বিবরণ পাঠাবেন না। Event অনেক জায়গায় যায়—Browser-এর DevTools, Log, Analytics—এবং যা একবার গেছে, তা ফেরানো যায় না।

২৭.৭ Event-Driven Architecture ও Transactional Outbox

দুটি শব্দ আলাদা রাখুন। **Command** একটি অনুরোধ: "Booking নিশ্চিত করুন" (`ConfirmBooking`)—এটি প্রত্যাখ্যাত হতে পারে। **Event** একটি ঘটে যাওয়া সত্য: "Booking নিশ্চিত হয়েছে" (`booking.confirmed`)—এটি বদলানো যায় না। Command আসে, যাচাই ও অনুমতি পরীক্ষা হয়, অবস্থা বদলায়, তারপর Event প্রকাশিত হয়। একই Event থেকে Notification, Analytics ও Audit—প্রত্যেকে নিজের মতো কাজ করে, Booking Service তাদের চেনেও না।

Folder-এর কাঠামো

```
Client Command -> API (Auth) -> Booking Service -> Database Transaction
                                                    |  Booking সারি
                                                    |  Outbox সারি
                                                    v
                                          Outbox Relay (আলাদা Worker)
                                                    |
                                                    v
                                        Broker বা WebSocket Gateway
                                                    |
                                                    v
                                          অনুমতিপ্রাপ্ত Client-গুলো
```

এখানে একটি সূক্ষ্ম বিপদ আছে: **Dual Write**। Service যদি প্রথমে Database-এ লেখে, তারপর Broker-এ Event পাঠায়, তাহলে মাঝখানে Process বন্ধ হলে Booking নিশ্চিত হলো কিন্তু কেউ জানল না। উল্টো ক্রমে করলে Event গেল কিন্তু Booking হয়তো Rollback হলো। **Transactional Outbox** এর সমাধান: Business পরিবর্তন ও একটি `outbox` সারি একই Transaction-এ লেখা হয়; একটি আলাদা Relay পরে Outbox থেকে পড়ে প্রকাশ করে।

outbox.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE bookings (id TEXT PRIMARY KEY, status TEXT NOT NULL);
  CREATE TABLE outbox (id INTEGER PRIMARY KEY, type TEXT, payload TEXT,
    published INTEGER NOT NULL DEFAULT 0);
  INSERT INTO bookings VALUES ('bk-1', 'PENDING');`);

// Business পরিবর্তন ও Outbox সারি একই Transaction-এ: দুটোই, নয়তো কোনোটিই নয়।
function confirm(id) {
  db.exec('BEGIN');
  try {
    db.prepare("UPDATE bookings SET status = 'CONFIRMED' WHERE id = ?").run(id);
    db.prepare('INSERT INTO outbox (type, payload) VALUES (?, ?)')
      .run('booking.confirmed', JSON.stringify({ bookingId: id }));
    db.exec('COMMIT');
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
```

```
  }
}

// ...
```

Relay ও একটি Idempotent Consumer:

outbox.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

// Relay: আলাদা Worker; Publish সফল হলে তবেই published = 1।
function relay(publish) {
  const rows = db.prepare('SELECT * FROM outbox WHERE published = 0').all();
  for (const row of rows) {
    publish(row);
    db.prepare('UPDATE outbox SET published = 1 WHERE id = ?').run(row.id);
  }
}

const seen = new Set();
const consumer = (row) => {
  if (seen.has(row.id)) return console.log('  duplicate বাদ:', row.id);
  seen.add(row.id);
  console.log('  Email পাঠানো হলো:', row.type, row.payload);
};

confirm('bk-1');
console.log('১ম Relay: Publish-এর পরে, published লেখার আগেই Crash');
try {
  relay((row) => { consumer(row); throw new Error('Crash'); });
} catch (error) {
  console.log('  ', error.message);
}
console.log('২য় Relay (Restart): একই সারি আবার, Consumer Idempotent');
relay(consumer);
console.log('বাকি:', db.prepare(
  'SELECT COUNT(*) AS n FROM outbox WHERE published = 0').get().n);
```

Output

```
১ম Relay: Publish-এর পরে, published লেখার আগেই Crash
  Email পাঠানো হলো: booking.confirmed {"bookingId":"bk-1"}
   Crash
২য় Relay (Restart): একই সারি আবার, Consumer Idempotent
  duplicate বাদ: 1
বাকি: 0
```

এই আউটপুট Outbox-এর আসল চরিত্র দেখায়। Relay Event প্রকাশ করার পরে কিন্তু `published = 1` লেখার আগে বন্ধ হলে, Restart-এর পর একই সারি আবার প্রকাশিত হয়—অর্থাৎ Outbox Event **হারায় না, কিন্তু Duplicate দিতে পারে** (At-least-once)। তাই Consumer-কে Idempotent হতে হয়: এখানে সে সারির `id` মনে রেখে দ্বিতীয় Email পাঠায় না। `node:sqlite` চালালে Node.js একটি "ExperimentalWarning" দেখায়; এটি স্বাভাবিক।

২৭.৮ Delivery Semantics, Ordering ও Duplicate

Message পৌঁছানোর তিনটি নিশ্চয়তা:

- **At-most-once:** একবার পাঠানো, Retry নেই। Duplicate হয় না, কিন্তু হারাতে পারে। (Live Cursor-এর অবস্থান, যেখানে পরেরটি আগেরটিকে অর্থহীন করে।)
- **At-least-once:** নিশ্চিত না হওয়া পর্যন্ত Retry। হারায় না, কিন্তু Duplicate আসতে পারে—তাই Consumer Idempotent। (Booking Event, Payment Notification।)
- **Exactly-once Effect:** Network-এর ওপর সত্যিকারের "ঠিক একবার পৌঁছানো" সাধারণভাবে অসম্ভব; কিন্তু At-least-once পৌঁছানো আর Idempotent প্রক্রিয়া—eventId, UNIQUE Constraint, Transaction—একসঙ্গে মিলে ফলাফল ঠিক একবারের মতো হয়।

ক্রম (Ordering): সব Event-এর একটি বিশ্বব্যাপী ক্রম ধরে নেবেন না; একাধিক Server, একাধিক সংযোগ ও Retry ক্রম এলোমেলো করে। যা দরকার তা হলো প্রতিটি Aggregate-এর (এখানে প্রতিটি Booking-এর) নিজস্ব `sequence`। Client শেষ দেখা sequence মনে রাখে, এবং প্রতিটি আসা Event-কে চার ভাগে ভাগ করে:

client-state.mjs · JavaScript — চলবে: Node.js

```
// Client-এর দিক: Duplicate বাদ, পুরোনো Event উপেক্ষা, ফাঁকে Snapshot চাই।
// Browser ও Node.js দুই জায়গাতেই চলে (কোনো Import নেই)।
export class BookingView {
  #state = new Map();
  #seen = new Set();

  constructor({ seenLimit = 500 } = {}) {
    this.seenLimit = seenLimit;
  }

  loadSnapshot({ bookingId, title, status, sequence }) {
    this.#state.set(bookingId, { title, status, sequence });
  }

  apply(event) {
    if (this.#seen.has(event.eventId)) return 'duplicate';
    const current = this.#state.get(event.bookingId);
    const last = current?.sequence ?? 0;
    if (event.sequence <= last) return 'stale';
    if (event.sequence !== last + 1) return 'gap';
    this.#remember(event.eventId);
```

```
    this.#state.set(event.bookingId, {
      title: current?.title ?? event.payload.title ?? '',
      status: event.payload.status,
      sequence: event.sequence,
    });
    return 'applied';
  }

  lastSequence(bookingId) {
    return this.#state.get(bookingId)?.sequence ?? 0;
  }

  get(bookingId) {
    return this.#state.get(bookingId);
  }

  #remember(eventId) {
    this.#seen.add(eventId);
    if (this.#seen.size > this.seenLimit) {
      this.#seen.delete(this.#seen.values().next().value);
    }
  }
}
```

নিয়মগুলো: আগে দেখা eventId হলে `duplicate`; sequence শেষ দেখাটির সমান বা কম হলে `stale` (পুরোনো); ঠিক পরেরটি হলে `applied`; আর মাঝে লাফ দিলে `gap` —কিছু হারিয়েছে, তাই এই Event প্রয়োগ না করে Server থেকে বাদ পড়াগুলো বা পূর্ণ Snapshot আনতে হবে। মনে রাখা eventId-এর Set সীমিত (`seenLimit`), নইলে দীর্ঘ সময় খোলা Tab-এর Memory বাড়তেই থাকবে; পুরোনো Duplicate তো sequence-এর নিয়মেই `stale` হিসেবে ধরা পড়ে।

dedupe-order.mjs · JavaScript চলবে: Node.js

```
import { BookingView } from '../project/travel-live/src/client-state.mjs';

const ev = (eventId, sequence, status) => ({ eventId, bookingId: 'bk-7',
  sequence, payload: { status } });

// Network-এ যা ঘটতে পারে: Retry-তে Duplicate, দেরিতে আসা, মাঝখানে হারানো।
const arrivals = [
  ev('e1', 1, 'PENDING'),
  ev('e2', 2, 'CONFIRMED'),
  ev('e2', 2, 'CONFIRMED'),
  ev('e4', 4, 'REFUNDED'),
  ev('e1', 1, 'PENDING'),
];
const view = new BookingView();
for (const event of arrivals) {
  console.log(`${event.eventId} (#${event.sequence}):`, view.apply(event));
}
```

```
console.log('দেখানো অবস্থা:', view.get('bk-7'));

// ফাঁক (#৩ নেই): Server থেকে Snapshot এনে তারপর এগোনো।
view.loadSnapshot({ bookingId: 'bk-7', title: 'Sajek', status: 'CANCELLED',
  sequence: 3 });
console.log('Snapshot-এর পরে #4:', view.apply(ev('e4', 4, 'REFUNDED')));
console.log('দেখানো অবস্থা:', view.get('bk-7'));
```

Output

```
e1 (#1): applied
e2 (#2): applied
e2 (#2): duplicate
e4 (#4): gap
e1 (#1): duplicate
দেখানো অবস্থা: { title: '', status: 'CONFIRMED', sequence: 2 }
Snapshot-এর পরে #4: applied
দেখানো অবস্থা: { title: 'Sajek', status: 'REFUNDED', sequence: 4 }
```

এখানে অবস্থার নামগুলো (যেমন REFUNDED) শুধু উদাহরণ; `BookingView` কোনো নির্দিষ্ট অবস্থা-যন্ত্র জানে না, সে শুধু ক্রম রক্ষা করে। #৪ প্রথমবার এসে ফাঁক দেখায়, তাই Screen #২-তেই থাকে—ভুল অবস্থা দেখানোর চেয়ে কিছুক্ষণ পুরোনো অবস্থা দেখানো ভালো। Snapshot আসার পর একই Event নির্বিঘ্নে প্রয়োগ হয়।

২৭.৯ Reconnect, Heartbeat ও Backpressure

পুনঃসংযোগ: সংযোগ কাটা স্বাভাবিক ঘটনা—Wi-Fi বদল, Laptop ঘুম, Server Deploy। সঙ্গে সঙ্গে, বারবার চেষ্টা করলে বিপদ: Server Restart-এর পর হাজার Client একই মুহূর্তে ফিরে এসে নতুন Server-কেও ফেলে দেয় (**Thundering Herd**)। সমাধান: প্রতি ব্যর্থতায় অপেক্ষা দ্বিগুণ (**Exponential Backoff**), একটি সর্বোচ্চ সীমা, এবং কিছুটা এলোমেলো (**Jitter**) যাতে Client-রা ছড়িয়ে পড়ে।

backoff.mjs · JavaScript চলবে: Node.js

```
// Exponential Backoff + Jitter: প্রতিবার অপেক্ষা দ্বিগুণ, সর্বোচ্চ ৩০ সেকেন্ড,
// আর ±২৫% এলোমেলো, যাতে হাজার Client একসঙ্গে ফিরে না আসে।
export function reconnectDelay(attempt, random = Math.random) {
  const capped = Math.min(30_000, 500 * 2 ** Math.min(attempt, 6));
  return Math.floor(capped * (0.75 + random() * 0.5));
}
```

`random` একটি Parameter, যাতে Test-এ নির্দিষ্ট মান দেওয়া যায়:

backoff-demo.mjs · JavaScript চলবে: Node.js

```
import { reconnectDelay } from '../project/travel-live/src/backoff.mjs';
```

```
// Test-যোগ্য করতে random বাইরে থেকে দেওয়া: 0.5 মানে Jitter শূন্য।
const middle = () => 0.5;
for (const attempt of [0, 1, 2, 3, 4, 5, 6, 10]) {
  console.log(`চেষ্টা ${attempt}: ${reconnectDelay(attempt, middle)} ms`);
}

// আসল Jitter: ১০০০ Client একসঙ্গে বিচ্ছিন্ন হলে ৩য় চেষ্টা কতটা ছড়ায়?
let seed = 27;
const random = () => {
  seed = (seed * 1_103_515_245 + 12_345) % 2_147_483_648;
  return seed / 2_147_483_648;
};
const delays = Array.from({ length: 1000 }, () => reconnectDelay(3, random));
console.log('৩য় চেষ্টা: সর্বনিম্ন', Math.min(...delays), 'ms, সর্বোচ্চ',
  Math.max(...delays), 'ms');
```

Output

```
চেষ্টা 0: 500 ms
চেষ্টা 1: 1000 ms
চেষ্টা 2: 2000 ms
চেষ্টা 3: 4000 ms
চেষ্টা 4: 8000 ms
চেষ্টা 5: 16000 ms
চেষ্টা 6: 30000 ms
চেষ্টা 10: 30000 ms
৩য় চেষ্টা: সর্বনিম্ন 3000 ms, সর্বোচ্চ 4997 ms
```

চেষ্টা ৬-এ হিসাব ৩২ সেকেন্ড হতো, কিন্তু সীমা ৩০ সেকেন্ড। আর ১০০০ Client-এর তৃতীয় চেষ্টা একই মুহূর্তে নয়, তিন থেকে পাঁচ সেকেন্ডে ছড়িয়ে। সফল সংযোগের পর `attempt` আবার শূন্য করতে ভুলবেন না।

Heartbeat: TCP সংযোগ কখনো নিঃশব্দে মরে যায়—Mobile Network বদলালে বা একটি NAT Router সংযোগ ভুলে গেলে কেউ Close Frame পাঠায় না। Server ভাবে Client আছে, আর তার জন্য Memory ও Subscription ধরে রাখে। WebSocket Protocol-এ **Ping** ও **Pong** Frame আছে: Server নির্দিষ্ট বিরতিতে Ping পাঠায়; পরের Ping-এর আগে Pong না এলে সংযোগ মৃত ধরে বন্ধ করে। Browser Pong স্বয়ংক্রিয়ভাবে পাঠায়, তাই Client-এ কিছু লিখতে হয় না। SSE-তে একই কাজ করে একটি মন্তব্য Line (`: ping`), যা মাঝের Proxy-কে জানায় সংযোগটি অলস নয়।

Backpressure: Server যত দ্রুত Message তৈরি করে, একটি ধীর Client (দুর্বল Mobile Network) তত দ্রুত নিতে পারে না। পাঠানো কিন্তু না-পৌঁছানো Data Server-এর Buffer-এ জমে (`ws` -এ `bufferedAmount`)। সীমাহীন Queue মানে একটি ধীর Client-ই পুরো Server-এর Memory শেষ করতে পারে। তিনটি প্রচলিত নীতি:

backpressure.mjs · JavaScript চলবে: Node.js

```
// ধীর Client-এর জন্য সীমিত Queue। তিনটি নীতি: পুরোনো ফেলা, একীকরণ, বিচ্ছিন্ন।
function createQueue({ limit, policy }) {
  const items = [];
  return {
```

```
    items,
    push(event) {
      if (policy === 'coalesce') {
        // একই Booking-এর পুরোনো Event থাকলে শুধু সর্বশেষটি রাখা।
        const i = items.findIndex((e) => e.bookingId === event.bookingId);
        if (i !== -1) items.splice(i, 1);
      }
      if (items.length < limit) return items.push(event) && 'queued';
      if (policy === 'drop-oldest' || policy === 'coalesce') {
        items.shift();
        items.push(event);
        return 'dropped-oldest';
      }
      return 'disconnect';
    },
  };
}

const burst = [1, 2, 3, 4, 5, 6].map((n) => ({
  bookingId: `bk-${n % 2 === 0 ? 'A' : 'B'}`, sequence: n }));
for (const policy of ['drop-oldest', 'coalesce', 'disconnect']) {
  const queue = createQueue({ limit: 3, policy });
  const results = burst.map((event) => queue.push(event));
  console.log(policy.padEnd(11), results.at(-1).padEnd(14),
    queue.items.map((e) => `${e.bookingId}#${e.sequence}`).join(' '));
}
```

Output

```
drop-oldest dropped-oldest bk-A#4 bk-B#5 bk-A#6
coalesce    queued         bk-B#5 bk-A#6
disconnect  disconnect     bk-B#1 bk-A#2 bk-B#3
```

`drop-oldest` পুরোনো ফেলে দেয়—Client-এ ফাঁক হবে, তাই তাকে Snapshot চাইতে হবে। `coalesce` একই Booking-এর পুরোনো Event-এর জায়গায় শুধু সর্বশেষটি রাখে—অবস্থা দেখানোর Dashboard-এর জন্য চমৎকার, কিন্তু প্রতিটি Event জরুরি হলে (যেমন Chat) চলবে না। `disconnect` ধীর Client-কে বিচ্ছিন্ন করে; Client পুনঃসংযোগে Replay চাইবে। TravelLive তৃতীয় নীতি নেয়, কারণ তার Replay ব্যবস্থা ইতিমধ্যে আছে।

২৭.১০ একাধিক Server: Pub/Sub ও Scaling

একটি Server-এ হাজার কয়েক সংযোগ সহজেই চলে। কিন্তু Load Balancer-এর পেছনে দুটি Instance রাখলে নতুন সমস্যা: রাফির Phone Instance A-তে, নাদিয়ার Laptop Instance B-তে। নাদিয়ার Command B-তে এলে A-র Client কীভাবে জানবে? উত্তর: একটি **Broker**। প্রতিটি Instance প্রাসঙ্গিক Channel-এ Subscribe করে; যেকোনো Instance

Publish করলে সবাই পায় এবং নিজের Client-দের পাঠায়। Redis Pub/Sub এর একটি জনপ্রিয় সমাধান, এবং Socket.IO-র Redis Adapter ভেতরে এটিই ব্যবহার করে।

QA-যন্ত্রে Redis ছিল না, তাই নিচের Broker একটি ছোট in-memory অনুকরণ—আসল Redis নয়—কিন্তু Redis Pub/Sub-এর মূল আচরণ হুবহু দেখায়:

pubsub-fanout.mjs · JavaScript চলবে: Node.js

```
// দুটি Gateway Instance, মাঝে একটি Broker। এখানে Broker একটি সাধারণ
// in-memory অনুকরণ, যা Redis Pub/Sub-এর মূল আচরণ দেখায়; আসল Redis নয়।
class PubSub {
  #channels = new Map();
  subscribe(channel, fn) {
    const set = this.#channels.get(channel) ?? new Set();
    set.add(fn);
    this.#channels.set(channel, set);
    return () => set.delete(fn);
  }
  publish(channel, message) {
    // শুধু এই মুহূর্তের Subscriber পায়; কোনো History জমা থাকে না।
    for (const fn of this.#channels.get(channel) ?? []) fn(message);
  }
}

function gateway(name, broker) {
  const local = new Map(); // এই Instance-এ যুক্ত Client
  return {
    connect(client, room) {
      local.set(client, room);
      return broker.subscribe(room, (msg) => {
        if (local.get(client) === room) console.log(`  ${name}/${client}:`, msg);
      });
    },
    publish: (room, msg) => broker.publish(room, msg),
  };
}

const broker = new PubSub();
const a = gateway('A', broker);
const b = gateway('B', broker);
a.connect('rafi-phone', 'booking:bk-1');
const leave = b.connect('agent-laptop', 'booking:bk-1');

console.log('Instance A থেকে Publish:');
a.publish('booking:bk-1', { sequence: 2, status: 'CONFIRMED' });
leave();
console.log('Agent offline থাকাকালে Publish:');
b.publish('booking:bk-1', { sequence: 3, status: 'CANCELLED' });
```

```
b.connect('agent-laptop', 'booking:bk-1');
console.log('Agent ফিরে এল; #3 আর আসবে না, Snapshot/Replay লাগবে');
```

Output

```
Instance A থেকে Publish:
  A/rafi-phone: { sequence: 2, status: 'CONFIRMED' }
  B/agent-laptop: { sequence: 2, status: 'CONFIRMED' }
Agent offline থাকাকালে Publish:
  A/rafi-phone: { sequence: 3, status: 'CANCELLED' }
Agent ফিরে এল; #3 আর আসবে না, Snapshot/Replay লাগবে
```

শেষ Line-টিই সবচেয়ে গুরুত্বপূর্ণ: **Redis Pub/Sub একটি টেকসই Queue নয়।** Message শুধু সেই মুহূর্তে সংযুক্ত Subscriber পায়; কেউ না থাকলে তা চিরতরে হারায়। তাই নির্ভরযোগ্য Replay-র জন্য তিনটির একটি লাগে: Database-এ Event History (TravelLive-এর `history`-র মতো, কিন্তু টেকসই); Redis Streams, Kafka বা NATS JetStream-এর মতো টেকসই Log, যেখানে Consumer নিজের অবস্থান মনে রাখে; অথবা অন্তত পুনঃসংযোগে Database থেকে পূর্ণ Snapshot।

আরও দুটি বাস্তব বিষয়। **Sticky Session:** Socket.IO-র Long Polling Fallback একই Instance-এ ফিরে আসা দরকার, তাই Load Balancer-এ Sticky Session চালু করতে হয়; খাঁটি WebSocket-এ সংযোগ নিজেই এক Instance-এ থাকে। **sequence কে দেবে?** কখনো Gateway নয়—দুটি Gateway আলাদা গুনবে। sequence আসবে Booking-এর মালিক Service থেকে, Database Transaction-এর ভেতরে, Outbox সারির সঙ্গে।

২৭.১১ নিরাপত্তা Checklist

১. **TLS:** Production-এ শুধু `wss://` ও `https://`।
২. **Origin:** WebSocket Upgrade-এ `Origin` Header একটি অনুমোদিত তালিকার সঙ্গে মেলান। এটি Cross-Site WebSocket Hijacking ঠেকায়, যেখানে একটি অচেনা Site আপনার ব্যবহারকারীর Cookie দিয়ে সংযোগ খোলে।
৩. **Authentication:** সংযোগের শুরুতে, অল্প সময়ের মধ্যে; না হলে বন্ধ। Token URL-এ নয়।
৪. **Authorization প্রতিবার:** প্রতিটি Subscribe-এ, এবং প্রতিটি Event পাঠানোর সময়ও—কারণ এর মধ্যে অনুমতি বাতিল হতে পারে (Logout, ভূমিকা বদল)।
৫. **Input:** Message-এর আকারের সীমা, JSON-এর ধরন যাচাই, প্রতি সংযোগে হারের সীমা, প্রতি সংযোগে Subscription-এর সংখ্যার সীমা।
৬. **তথ্য কমানো:** Event-এ শুধু যা দরকার; অস্তিত্ব ফাঁস না করতে অজানা ও নিষিদ্ধ Booking-এ একই উত্তর।
৭. **পর্যবেক্ষণ:** সক্রিয় সংযোগ, প্রত্যাখ্যাত Origin, Auth ব্যর্থতা, ধীর Client-এর বিচ্ছিন্নতা ও পুনঃসংযোগের হার মাপুন—হঠাৎ বাড়লে কিছু ভুল হচ্ছে।

২৭.১২ Project: TravelLive

TravelLive একটি Real-Time Booking Dashboard। গ্রাহক নিজের Booking-এর অবস্থা Live দেখেন; Agent সব Booking দেখেন এবং নিশ্চিত বা বাতিল করেন। এটি একটি in-memory TravelLive Simulator-এর সব নিয়ম রাখে—অবস্থা-যন্ত্র, Room-এর অনুমতি, sequence-ভিত্তিক Replay, Duplicate বাদ, সীমিত History, Backoff—এবং তার ওপর আসল Network যোগ করে: `ws` দিয়ে WebSocket Gateway, `node:http` দিয়ে SSE ও JSON API, এবং একটি Browser Dashboard।

ব্যবসার নিয়ম: `PENDING` থেকে `CONFIRMED` বা `CANCELLED`; `CONFIRMED` থেকে `CANCELLED`; `CANCELLED` শেষ অবস্থা। শুধু Agent নিশ্চিত করতে পারেন; মালিক ও Agent বাতিল করতে পারেন। গ্রাহক শুধু নিজের Booking দেখেন।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-live/
├── package.json          (একটি Dependency: ws)
├── README.md
├── main.mjs              Server চালু, SIGTERM-এ সুশৃঙ্খল বন্ধ
├── demo.mjs              এক Process-এ পুরো প্রবাহ
├── public/
│   ├── index.html        Browser Dashboard
│   └── app.mjs           WebSocket Client, Backoff, Replay
├── src/
│   ├── events.mjs        Event Envelope ও যাচাই
│   ├── live-hub.mjs      Domain: Booking, অনুমতি, History, Subscription
│   ├── client-state.mjs  Duplicate, পুরোনো ও ফাঁক চেনা
│   ├── backoff.mjs       Exponential Backoff + Jitter
│   ├── rate-limit.mjs    Token Bucket
│   ├── gateway.mjs       WebSocket: Origin, Auth, Protocol, Heartbeat
│   ├── sse.mjs           SSE Stream ও Last-Event-ID
│   ├── server.mjs        HTTP Route ও Static File
│   └── demo-data.mjs     Demo ব্যবহারকারী ও Booking
└── tests/                ১৮টি Test (Domain, Client, HTTP, SSE, WebSocket)
```

Domain (`live-hub.mjs`) কোনো Network জানে না; Gateway ও SSE দুটিই একই Hub ব্যবহার করে। এই বিভাজনের ফলে Domain-এর নিয়ম Network ছাড়াই Test করা যায়, আর ভবিষ্যতে Socket.IO বা অন্য Transport যোগ করলে Domain বদলায় না।

src/live-hub.mjs

প্রথম অংশ: অবস্থা-যন্ত্র (কোন অবস্থা থেকে কোথায় যাওয়া যায়), নিজস্ব Error, Private Field-এ রাখা অবস্থা, এবং ব্যবহারকারী যোগ ও বাতিল:

live-hub.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import { createEvent } from './events.mjs';

// Booking-এর অবস্থা বদলের নিয়ম। CANCELLED শেষ অবস্থা।
const TRANSITIONS = {
  PENDING: ['CONFIRMED', 'CANCELLED'],
  CONFIRMED: ['CANCELLED'],
  CANCELLED: [],
};

export class LiveError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

export class LiveHub {
  #bookings = new Map();
  #history = new Map();
  #subscribers = new Map();
  #users = new Map();
  #counter = 0;

  constructor({ historyLimit = 50, now = () => new Date() } = {}) {
    this.historyLimit = historyLimit;
    this.now = now;
  }

  addUser(user) {
    this.#users.set(user.id, { ...user, active: true });
  }

  revokeUser(userId) {
    const user = this.#users.get(userId);
    if (user) user.active = false;
    this.#recheck();
  }

// ...
```

`canView` একটি জায়গায় সিদ্ধান্ত নেয়—সক্রিয় ব্যবহারকারী, এবং Agent অথবা মালিক। তারপর নতুন Booking ও অবস্থা-বদল। ID ও মালিক Server ঠিক করে; Client কখনো নয়:

live-hub.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...
```

```
  canView(userId, bookingId) {
    const user = this.#users.get(userId);
    const booking = this.#bookings.get(bookingId);
    if (!user?.active || !booking) return false;
    return user.role === 'agent' || booking.ownerId === userId;
  }

  createBooking(actorId, title) {
    if (!this.#users.get(actorId)?.active) {
      throw new LiveError('UNAUTHORIZED', 'অজানা বা বাতিল ব্যবহারকারী');
    }
    const id = `bk-${this.#bookings.size + 1}`;
    const booking = { id, ownerId: actorId, title, status: 'PENDING',
      sequence: 0 };
    this.#bookings.set(id, booking);
    this.#history.set(id, []);
    return this.#record(booking, 'booking.created',
      { status: 'PENDING', title });
  }

  changeStatus(actorId, bookingId, status) {
    if (!this.canView(actorId, bookingId)) {
      throw new LiveError('FORBIDDEN', 'এই Booking বদলানোর অনুমতি নেই');
    }
    const booking = this.#bookings.get(bookingId);
    const actor = this.#users.get(actorId);
    if (status === 'CONFIRMED' && actor.role !== 'agent') {
      throw new LiveError('FORBIDDEN', 'শুধু Agent নিশ্চিত করতে পারেন');
    }
    if (!TRANSITIONS[booking.status].includes(status)) {
      throw new LiveError('INVALID_TRANSITION',
        `${booking.status} থেকে ${status} সম্ভব নয়`);
    }
    const from = booking.status;
    booking.status = status;
    return this.#record(booking, 'booking.status.changed', { from, status });
  }

// ...
```

`changeStatus` তিনটি জিনিস যাচাই করে—দেখার অনুমতি, ভূমিকা, এবং বৈধ অবস্থা-বদল—তারপর অবস্থা বদলায় ও Event রেকর্ড করে। এরপর Snapshot ও Subscribe:

live-hub.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  snapshot(userId, bookingId) {
    if (!this.canView(userId, bookingId)) {
```

```
      throw new LiveError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
    }
    const { id, title, status, sequence } = this.#bookings.get(bookingId);
    return { bookingId: id, title, status, sequence };
  }

  // Subscribe-এর সময় অনুমতি যাচাই; afterSequence-এর পরের Event Replay,
  // আর History-তে ফাঁক থাকলে পূর্ণ Snapshot।
  subscribe(userId, bookingId, listener, { afterSequence = 0,
    onClose = () => {} } = {}) {
    if (!this.canView(userId, bookingId)) {
      throw new LiveError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
    }
    const sub = { userId, listener, onClose };
    const subs = this.#subscribers.get(bookingId) ?? new Set();
    subs.add(sub);
    this.#subscribers.set(bookingId, subs);
    const history = this.#history.get(bookingId);
    const oldest = history[0]?.sequence ?? 1;
    const recovery = afterSequence + 1 < oldest
      ? { mode: 'snapshot', snapshot: this.snapshot(userId, bookingId) }
      : { mode: 'replay',
        events: history.filter((e) => e.sequence > afterSequence) };
    return { ...recovery, unsubscribe: () => subs.delete(sub) };
  }

// ...
```

Subscribe-এর উত্তর দুই রকম: History-তে প্রয়োজনীয় সব Event থাকলে `replay` (শুধু `afterSequence`-এর পরেরগুলো), আর History সীমিত বলে পুরোনোগুলো মুছে গেলে `snapshot`। শেষ অংশ:

live-hub.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  subscriberCount(bookingId) {
    return this.#subscribers.get(bookingId)?.size ?? 0;
  }

  #record(booking, type, payload) {
    booking.sequence += 1;
    this.#counter += 1;
    const event = createEvent({ id: `evt-${this.#counter}`, type,
      bookingId: booking.id, sequence: booking.sequence, payload,
      now: this.now() });
    const history = this.#history.get(booking.id);
    history.push(event);
    if (history.length > this.historyLimit) history.shift();
    this.#dispatch(event);
```

```
    return event;
  }

  // Dispatch-এর সময়ও অনুমতি আবার যাচাই: বাতিল ব্যবহারকারী কিছু পান না।
  #dispatch(event) {
    for (const sub of this.#subscribers.get(event.bookingId) ?? []) {
      if (this.canView(sub.userId, event.bookingId)) sub.listener(event);
      else this.#close(event.bookingId, sub);
    }
  }

  #recheck() {
    for (const [bookingId, subs] of this.#subscribers) {
      for (const sub of subs) {
        if (!this.canView(sub.userId, bookingId)) this.#close(bookingId, sub);
      }
    }
  }

  #close(bookingId, sub) {
    this.#subscribers.get(bookingId).delete(sub);
    sub.onClose('FORBIDDEN');
  }
}
```

`#record` sequence বাড়ায়, Event তৈরি করে, History-তে রাখে (সীমা ছাড়ালে সবচেয়ে পুরোনোটি বাদ) এবং পাঠায়। `#dispatch` প্রতিটি Subscriber-কে পাঠানোর **ঠিক আগে** আবার `canView` যাচাই করে: অনুমতি বাতিল হলে Subscription বন্ধ হয় এবং `onClose('FORBIDDEN')` জানানো হয়। `revokeUser` সঙ্গে সঙ্গে `#recheck` চালায়, তাই পরের Event-এর অপেক্ষাও করতে হয় না।

> **নোট:** এই Hub-এ অবস্থা-বদল ও History-তে লেখা একই Synchronous ধাপে ঘটে, তাই একটি Process-এর মধ্যে তারা কখনো আলাদা হয় না। একাধিক Process-এ এই নিশ্চয়তা পেতে Database Transaction ও Outbox লাগবে (অনুচ্ছেদ ২৭.৭)।

src/gateway.mjs

Gateway HTTP Server-এর `upgrade` Event ধরে। প্রথম কাজ: Path ও Origin যাচাই, তারপর `ws` -কে Handshake করতে দেওয়া। `maxPayload` 8 KB-র বড় Message-এ সংযোগ 1009 দিয়ে বন্ধ করে:

gateway.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import WebSocket from 'ws';
import { createBucket } from './rate-limit.mjs';

const CLOSE = { AUTH: 4401, FORBIDDEN_ORIGIN: 4403, RATE: 4429 };
```

```
// HTTP Server-এর 'upgrade' Event-এ WebSocket Gateway যুক্ত করে।
export function attachGateway(server, { hub, users, allowedOrigins,
  heartbeatMs = 30_000, authTimeoutMs = 3_000, maxBuffered = 64 * 1024 }) {
  const wss = new WebSocket.Server({ noServer: true, maxPayload: 4 * 1024 });
  const metrics = { connections: 0, rejectedOrigins: 0, slowConsumers: 0,
    protocolErrors: 0 };

  server.on('upgrade', (req, socket, head) => {
    const { pathname } = new URL(req.url, 'http://localhost');
    const origin = req.headers.origin;
    if (pathname !== '/live') return socket.destroy();
    // Browser সব সময় Origin পাঠায়; অচেনা Site-এর Page-কে ঢুকতে দেবেন না।
    if (origin && !allowedOrigins.includes(origin)) {
      metrics.rejectedOrigins += 1;
      socket.end('HTTP/1.1 403 Forbidden\r\n\r\n');
      return undefined;
    }
    return wss.handleUpgrade(req, socket, head,
      (ws) => wss.emit('connection', ws));
  });

// ...
```

প্রতিটি সংযোগের নিজস্ব অবস্থা: ব্যবহারকারী (Auth-এর আগে `null`), Subscription-এর তালিকা, এবং একটি Token Bucket (একসঙ্গে ২০টি, তারপর সেকেন্ডে ৫টি Message)। Auth-এর জন্য ৩ সেকেন্ড সময়। শেষে `send`: Buffer সীমা ছাড়ালে ধীর Client বিচ্ছিন্ন:

gateway.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

  wss.on('connection', (ws) => {
    metrics.connections += 1;
    const conn = { ws, user: null, subs: new Map(),
      take: createBucket({ capacity: 20, refillPerSecond: 5 }) };
    const authTimer = setTimeout(
      () => ws.close(CLOSE.AUTH, 'auth timeout'), authTimeoutMs);
    ws.on('message', (data) => {
      if (!conn.take()) return ws.close(CLOSE.RATE, 'rate limit');
      return handleMessage(conn, String(data), { hub, users, authTimer,
        send: (msg) => send(conn, msg) });
    });
    // ভুল Frame বা সীমা-ছাড়ানো Message-এ 'error'; ws নিজেই পরে বন্ধ করে।
    // Handler না থাকলে পুরো Process ভেঙে পড়ত।
    ws.on('error', () => { metrics.protocolErrors += 1; });
    ws.on('close', () => {
      clearTimeout(authTimer);
      for (const unsubscribe of conn.subs.values()) unsubscribe();
      metrics.connections -= 1;
```

```
    });
  });

  // Slow Client: Buffer সীমা ছাড়ালে Connection বন্ধ; Client পরে Replay চাইবে।
  function send(conn, message) {
    if (conn.ws.readyState !== WebSocket.OPEN) return;
    if (conn.ws.bufferedAmount > maxBuffered) {
      metrics.slowConsumers += 1;
      conn.ws.terminate();
      return;
    }
    conn.ws.send(JSON.stringify(message));
  }

// ...
```

`ws.on('error')` Handler ছোট, কিন্তু অপরিহার্য। Test লেখার সময় দেখা গেছে: এটি না থাকলে একটি বড় Message পাঠিয়েই যে কেউ পুরো Server Process ভেঙে দিতে পারত, কারণ `ws` সীমা-ছাড়ানো Message-এ সংযোগে একটি `error` Event তোলে, আর Node.js-এ Handler-হীন `error` Event পুরো Process থামিয়ে দেয়। পরের অংশ: Heartbeat, এবং Protocol-এর প্রথম ধাপ `handleMessage`। Auth-এর আগে অন্য যেকোনো Message-এ সংযোগ বন্ধ:

gateway.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  // Heartbeat: আগের Ping-এর Pong না এলে Connection মৃত ধরে নিন।
  const timer = setInterval(() => {
    for (const ws of wss.clients) {
      if (ws.isAlive === false) { ws.terminate(); continue; }
      ws.isAlive = false;
      ws.ping();
    }
  }, heartbeatMs);
  wss.on('connection', (ws) => {
    ws.isAlive = true;
    ws.on('pong', () => { ws.isAlive = true; });
  });
  server.on('close', () => clearInterval(timer));

  return { wss, metrics };
}

function handleMessage(conn, text, { hub, users, authTimer, send }) {
  let msg;
  try {
    msg = JSON.parse(text);
  } catch {
```

```
    return send({ type: 'error', code: 'BAD_JSON' });
  }
  if (msg.type === 'auth') {
    const user = users.get(msg.token);
    if (!user) return conn.ws.close(CLOSE.AUTH, 'invalid token');
    clearTimeout(authTimer);
    conn.user = user;
    return send({ type: 'auth.ok', userId: user.id });
  }
  if (!conn.user) return conn.ws.close(CLOSE.AUTH, 'auth required');
  if (msg.type === 'subscribe') return subscribe(conn, msg, hub, send);
  if (msg.type === 'unsubscribe') {
    conn.subs.get(msg.bookingId)?.();
    conn.subs.delete(msg.bookingId);
    return send({ type: 'ack', requestId: msg.requestId, ok: true });
  }
  return send({ type: 'error', code: 'UNKNOWN_TYPE' });
}

// ...
```

শেষ অংশ `subscribe`। প্রতিটি অনুরোধের উত্তরে `requestId`-সহ একটি `ack`, যাতে Client জানে কোন অনুরোধের ফল কী—Socket.IO-র Acknowledgement-এর সমতুল্য:

gateway.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

function subscribe(conn, msg, hub, send) {
  const { bookingId, requestId } = msg;
  const afterSequence = Number.isInteger(msg.afterSequence)
    ? msg.afterSequence : 0;
  if (typeof bookingId !== 'string' || conn.subs.has(bookingId)) {
    return send({ type: 'ack', requestId, ok: false, code: 'BAD_REQUEST' });
  }
  try {
    const result = hub.subscribe(conn.user.id, bookingId,
      (event) => send({ type: 'event', event }),
      { afterSequence, onClose: (reason) => {
        conn.subs.delete(bookingId);
        send({ type: 'unsubscribed', bookingId, reason });
      } });
    conn.subs.set(bookingId, result.unsubscribe);
    send({ type: 'ack', requestId, ok: true, mode: result.mode });
    if (result.mode === 'snapshot') {
      return send({ type: 'snapshot', snapshot: result.snapshot });
    }
    for (const event of result.events) send({ type: 'event', event });
    return undefined;
```

```
  } catch (error) {
    return send({ type: 'ack', requestId, ok: false, code: error.code });
  }
}
```

লক্ষ করুন, `subscribe`-এ প্রথমে `ack`, তারপর Replay-র Event বা Snapshot। Hub-এর `subscribe` Subscriber-কে নিবন্ধিত করে Replay ফেরত দেয় একই Synchronous ধাপে, তাই Replay ও Live Event-এর মাঝে কোনো Event হারানোর ফাঁক নেই।

src/sse.mjs

একই Hub, ভিন্ন Transport। Browser-এর `Last-Event-ID` থেকে `afterSequence`:

sse.mjs · JavaScript — চলবে: Node.js

```
// Server-Sent Events: একমুখী (Server থেকে Client) সাধারণ HTTP Stream।
// Browser পুনঃসংযোগে নিজেই Last-Event-ID পাঠায়; তা দিয়ে Replay।
export function streamBooking(req, res, { hub, user, bookingId,
  heartbeatMs = 15_000 }) {
  const lastId = Number.parseInt(req.headers['last-event-id'] ?? '0', 10);
  const write = (event) => res.write(
    `id: ${event.sequence}\nevent: ${event.type}\n`
    + `data: ${JSON.stringify(event)}\n\n`);
  const result = hub.subscribe(user.id, bookingId, write, {
    afterSequence: Number.isInteger(lastId) ? lastId : 0,
    onClose: () => res.end(),
  });
  res.writeHead(200, {
    'Content-Type': 'text/event-stream; charset=utf-8',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
  });
  res.write('retry: 3000\n\n');
  if (result.mode === 'snapshot') {
    res.write(`event: snapshot\ndata: ${JSON.stringify(result.snapshot)}\n\n`);
  } else {
    result.events.forEach(write);
  }
  // Comment Line (':' দিয়ে শুরু) Proxy-কে জানায় Connection জীবিত।
  const timer = setInterval(() => res.write(': ping\n\n'), heartbeatMs);
  req.on('close', () => {
    clearInterval(timer);
    result.unsubscribe();
  });
}
```

Response-এর Header লেখার আগে `hub.subscribe` ডাকা হয়, যাতে অনুমতি না থাকলে Error সাধারণ JSON 403 হিসেবে যায়। Client চলে গেলে (`req` 'close') Timer ও Subscription দুটিই পরিষ্কার—নইলে প্রতিটি বন্ধ Tab একটি Memory Leak রেখে যেত।

src/server.mjs

HTTP Route, JSON Body-র সীমা ও Error চুক্তি—অধ্যায় ২৫-এর মতোই `{ data }` অথবা `{ error: { code, message } }`:

server.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import http from 'node:http';
import { readFile } from 'node:fs/promises';
import { attachGateway } from './gateway.mjs';
import { streamBooking } from './sse.mjs';

const STATUS = { UNAUTHORIZED: 401, FORBIDDEN: 403, INVALID_TRANSITION: 409,
  BAD_REQUEST: 400, TOO_LARGE: 413, NOT_FOUND: 404 };
const STATIC = {
  '/': ['../public/index.html', 'text/html'],
  '/app.mjs': ['../public/app.mjs', 'text/javascript'],
  '/client-state.mjs': ['./client-state.mjs', 'text/javascript'],
  '/backoff.mjs': ['./backoff.mjs', 'text/javascript'],
};

class HttpError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

function json(res, status, body) {
  res.writeHead(status, { 'Content-Type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify(body));
}

async function readJson(req, limit = 1024) {
  let size = 0;
  const chunks = [];
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new HttpError('TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8') || '{}');
  } catch {
    throw new HttpError('BAD_REQUEST', 'অবৈধ JSON');
```

```
  }
}

// ...
```

Bearer Token থেকে ব্যবহারকারী, এবং প্রতিটি অপ্রত্যাশিত Error-এ 500—ভেতরের বার্তা Client-কে নয়, Log-এ:

server.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

export function createLiveServer({ hub, users, allowedOrigins,
  heartbeatMs, authTimeoutMs }) {
  const authenticate = (req) => {
    const token = /^Bearer (.+)$/.exec(req.headers.authorization ?? '')?.[1];
    const user = users.get(token);
    if (!user) throw new HttpError('UNAUTHORIZED', 'Token প্রয়োজন');
    return user;
  };

  const server = http.createServer(async (req, res) => {
    try {
      await route(req, res);
    } catch (error) {
      const status = STATUS[error.code];
      if (!status) console.error(error);
      json(res, status ?? 500, { error: { code: error.code ?? 'INTERNAL',
        message: status ? error.message : 'Server-এ সমস্যা' } });
    }
  });

// ...
```

Route-গুলো:

server.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  async function route(req, res) {
    const { pathname } = new URL(req.url, 'http://localhost');
    const [, api, resource, id, action] = pathname.split('/');
    if (req.method === 'GET' && STATIC[pathname]) {
      const [file, type] = STATIC[pathname];
      const body = await readFile(new URL(file, import.meta.url));
      res.writeHead(200, { 'Content-Type': `${type}; charset=utf-8` });
      return res.end(body);
    }
    if (req.method === 'GET' && pathname === '/health') {
```

```
      return json(res, 200, { data: { status: 'ok' } });
    }
    if (api !== 'api' || resource !== 'bookings') {
      throw new HttpError('NOT_FOUND', 'Route পাওয়া যায়নি');
    }
    const user = authenticate(req);
    if (req.method === 'POST' && !id) {
      const { title } = await readJson(req);
      if (typeof title !== 'string' || !title.trim()) {
        throw new HttpError('BAD_REQUEST', 'title প্রয়োজন');
      }
      const event = hub.createBooking(user.id, title.trim());
      return json(res, 201, { data: hub.snapshot(user.id, event.bookingId) });
    }
    if (req.method === 'POST' && action === 'status') {
      const { status } = await readJson(req);
      const event = hub.changeStatus(user.id, id, status);
      return json(res, 200, { data: event });
    }
    if (req.method === 'GET' && id && action === 'stream') {
      if (!hub.canView(user.id, id)) {
        throw new HttpError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
      }
      return streamBooking(req, res, { hub, user, bookingId: id });
    }
    if (req.method === 'GET' && id && !action) {
      return json(res, 200, { data: hub.snapshot(user.id, id) });
    }
    throw new HttpError('NOT_FOUND', 'Route পাওয়া যায়নি');
  }

// ...
```

server.mjs · JavaScript (অংশ 8 / 8 (Part 4 of 4)) চলবে: Node.js

```
// ...

  const gateway = attachGateway(server, { hub, users, allowedOrigins,
    heartbeatMs, authTimeoutMs });
  return { server, gateway };
}
```

অজানা Booking ও অন্যের Booking—দুটিতেই একই 403, যাতে কেউ ID অনুমান করে কোন Booking আছে তা জানতে না পারে।

Browser Dashboard

Page-টি ছোট। Token থাকে URL-এর `#` অংশে, যা Browser Server-এ পাঠায় না (শুধু শেখার সুবিধার জন্য; আসল প্রয়োগে Login ও Cookie):

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TravelLive</title>
  <link rel="icon" href="data:,">
  <style>
    body { font-family: system-ui, sans-serif; margin: 2rem; }
    #connection { font-weight: bold; }
    li[data-status="CONFIRMED"] { color: #0a7a3b; }
    li[data-status="CANCELLED"] { color: #b42318; }
  </style>
</head>
<body>
  <h1>TravelLive</h1>
  <p>সংযোগ: <span id="connection" role="status">বিচ্ছিন্ন</span></p>
  <ul id="bookings" aria-live="polite"></ul>
  <!-- Token থাকে URL-এর # অংশে (যেমন #token-rafi:bk-1); তা Server-এ যায় না। -->
  <script type="module" src="/app.mjs"></script>
</body>
</html>
```

`app.mjs` একই `client-state.mjs` ও `backoff.mjs` ব্যবহার করে যা Server-এর Test-এ পরীক্ষিত—Browser ও Node.js দুই জায়গায় একই ES Module। প্রথম অংশ: অবস্থা ও Render:

app.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Browser

```
import { BookingView } from '/client-state.mjs';
import { reconnectDelay } from '/backoff.mjs';

const [token, ...bookingIds] = location.hash.slice(1).split(/[:,]/);
const view = new BookingView();
const $ = (selector) => document.querySelector(selector);
let attempt = 0;

function render() {
  $('#bookings').replaceChildren(...bookingIds.map((id) => {
    const item = document.createElement('li');
    const booking = view.get(id);
    item.dataset.status = booking?.status ?? 'UNKNOWN';
    item.textContent = `${id}: ${booking?.title ?? ''} — `
```

```
      + `${booking?.status ?? 'অপেক্ষমাণ'} (#${booking?.sequence ?? 0})`;
    return item;
  }));
}

// ...
```

সংযোগ, Auth, Subscribe ও পুনঃসংযোগ। প্রতিবার Auth-এর পর প্রতিটি Booking-এর শেষ দেখা sequence পাঠানো হয়; Server শুধু বাদ পড়াগুলো পাঠায়। ফাঁক ধরা পড়লে Client নিজেই সংযোগ বন্ধ করে, এবং পুনঃসংযোগ তাকে ঠিক করে দেয়:

app.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

function connect() {
  const socket = new WebSocket(`ws://${location.host}/live`);
  socket.addEventListener('open', () => {
    socket.send(JSON.stringify({ type: 'auth', token }));
  });
  socket.addEventListener('message', ({ data }) => {
    const msg = JSON.parse(data);
    if (msg.type === 'auth.ok') {
      attempt = 0;
      $('#connection').textContent = 'সংযুক্ত';
      // পুনঃসংযোগে শেষ দেখা Sequence পাঠিয়ে শুধু বাদ পড়া Event চাওয়া।
      for (const bookingId of bookingIds) {
        socket.send(JSON.stringify({ type: 'subscribe', bookingId,
          requestId: bookingId, afterSequence: view.lastSequence(bookingId) }));
      }
    }
    if (msg.type === 'snapshot') view.loadSnapshot(msg.snapshot);
    if (msg.type === 'event' && view.apply(msg.event) === 'gap') {
      socket.close(); // ফাঁক: নতুন করে সংযোগ, Server Replay বা Snapshot দেবে
    }
    if (msg.type === 'ack' && !msg.ok) console.error('subscribe', msg.code);
    render();
  });
  socket.addEventListener('close', () => {
    const delay = reconnectDelay(attempt++);
    $('#connection').textContent = `বিচ্ছিন্ন; ${delay} ms পরে আবার চেষ্টা`;
    setTimeout(connect, delay);
  });
}

render();
connect();
```

demo.mjs

পুরো প্রবাহ এক Process-এ, আসল Network সংযোগে: রাফি Subscribe করেন, Agent নিশ্চিত করেন, মিনা অন্যের Booking চাইতে গিয়ে প্রত্যাখ্যাত হন, রাফি Offline থাকাকালে Agent বাতিল করেন, এবং রাফি ফিরে এসে শুধু বাদ পড়া Event পান:

demo.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// পুরো প্রবাহ এক Process-এ: Server চালু, দুই Client, HTTP দিয়ে অবস্থা-বদল।
import { createDemo } from './src/demo-data.mjs';
import { createLiveServer } from './src/server.mjs';
import { BookingView } from './src/client-state.mjs';
import { login } from './tests/helpers.mjs';

const { hub, users } = createDemo();
const { server } = createLiveServer({ hub, users, allowedOrigins: [] });
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));
const host = `127.0.0.1:${server.address().port}`;
const view = new BookingView();
const ack = (id) => (m) => m.type === 'ack' && m.requestId === id;

// নির্দিষ্ট Sequence পর্যন্ত অপেক্ষা, তারপর নতুন Event-গুলো ক্রমে প্রয়োগ।
async function applyUntil(client, sequence) {
  await client.next((m) => m.event?.sequence === sequence);
  for (const m of client.messages.splice(0)) {
    if (m.type !== 'event') continue;
    const result = view.apply(m.event);
    console.log(`  #${m.event.sequence} ${m.event.type}: ${result}`);
  }
}

async function rafiSubscribes(afterSequence) {
  const rafi = await login(host, 'token-rafi');
  rafi.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 's',
    afterSequence });
  console.log('Rafi subscribe:', (await rafi.next(ack('s'))).mode);
  return rafi;
}

const changeStatus = (token, status) => fetch(
  `http://${host}/api/bookings/bk-1/status`, { method: 'POST',
    headers: { Authorization: `Bearer ${token}` },
    body: JSON.stringify({ status }) }).then((res) => res.status);

// ...
```

demo.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

const rafi = await rafiSubscribes(0);
await applyUntil(rafi, 1);
console.log('Agent CONFIRMED:', await changeStatus('token-agent', 'CONFIRMED'));
await applyUntil(rafi, 2);

const mina = await login(host, 'token-mina');
mina.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 'm' });
console.log('Mina subscribe:', (await mina.next(ack('m'))).code);

rafi.socket.close();
await rafi.closed;
console.log('Rafi offline; Agent CANCELLED:',
  await changeStatus('token-agent', 'CANCELLED'));
const again = await rafiSubscribes(view.lastSequence('bk-1'));
await applyUntil(again, 3);
console.log('Rafi-র Screen:', view.get('bk-1'));

for (const client of [mina, again]) client.socket.close();
server.closeAllConnections();
server.close();
```

Output

```
Rafi subscribe: replay
  #1 booking.created: applied
Agent CONFIRMED: 200
  #2 booking.status.changed: applied
Mina subscribe: FORBIDDEN
Rafi offline; Agent CANCELLED: 200
Rafi subscribe: replay
  #3 booking.status.changed: applied
Rafi-র Screen: { title: "Cox's Bazar, ৩ দিন", status: 'CANCELLED', sequence: 3 }
```

দ্বিতীয়বার রাফি `afterSequence: 2` পাঠান, তাই Server শুধু #৩ পাঠায়—#১ ও #২ আবার আসে না। Demo প্রথম সংস্করণে Console-এ Event ছাপত যেই মুহূর্তে আসত; Network-এর সময়ের ওপর নির্ভর করে Line-গুলোর ক্রম কখনো বদলে যেত। এখন `applyUntil` নির্দিষ্ট Event পর্যন্ত অপেক্ষা করে, তারপর জমা Message ক্রমে প্রয়োগ করে—Real-Time Code-এর Test ও Demo-তে এটি একটি দরকারি অভ্যাস।

Test: আসল সংযোগে

Test-গুলো Port 0-তে (Operating System একটি খালি Port দেয়) আসল Server চালায় এবং Node.js-এর অন্তর্নির্মিত WebSocket Client দিয়ে সংযোগ করে। নিচে চারটি Test—অনুমতি বাতিল, অচেনা Origin, সীমা, এবং Heartbeat:

gateway.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
test('অনুমতি বাতিল হলে Server নিজেই Unsubscribe করে', async (t) => {
  const app = await startServer();
  t.after(app.close);
  const rafi = await login(app.host, 'token-rafi');
  rafi.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 'r' });
  await rafi.next(ack('r'));
  app.hub.revokeUser('u-rafi');
  assert.deepEqual(await rafi.next((m) => m.type === 'unsubscribed'),
    { type: 'unsubscribed', bookingId: 'bk-1', reason: 'FORBIDDEN' });
  rafi.socket.close();
});

test('অচেনা Origin-এর Upgrade: 403', async (t) => {
  const app = await startServer();
  t.after(app.close);
  const tryOrigin = (origin) => new Promise((resolve) => {
    const req = http.request(`http://${app.host}/live`, { headers: {
      Connection: 'Upgrade', Upgrade: 'websocket', Origin: origin,
      'Sec-WebSocket-Version': '13',
      'Sec-WebSocket-Key': 'dGhlIHNhbXBsZSBub25jZQ==' } });
    req.on('upgrade', (res, socket) => { socket.destroy(); resolve(101); });
    req.on('response', (res) => resolve(res.statusCode));
    req.end();
  });
  assert.equal(await tryOrigin('https://evil.example'), 403);
  assert.equal(await tryOrigin(ORIGIN), 101);
  assert.equal(app.gateway.metrics.rejectedOrigins, 1);
});

// ...

test('Heartbeat: Pong না দেওয়া Client বিচ্ছিন্ন হয়', async (t) => {
  const app = await startServer({ heartbeatMs: 50 });
  t.after(app.close);
  const zombie = new WebSocket(`ws://${app.host}/live`);
  zombie.pong = () => {}; // মৃত Client-এর ভান: Ping-এর উত্তর দেয় না
  await new Promise((resolve) => zombie.once('open', resolve));
  const code = await new Promise((resolve) => zombie.once('close', resolve));
  assert.equal(code, 1006);
  while (app.gateway.metrics.connections > 0) {
    await new Promise((resolve) => setTimeout(resolve, 10));
  }
});
```

Origin-এর Test-এ Node.js-এর `http.request` দিয়ে হাতে Upgrade অনুরোধ পাঠানো হয়, কারণ Browser বা অন্তর্নির্মিত WebSocket Client নিজের ইচ্ছামতো `Origin` বসাতে দেয় না। Heartbeat-এর Test-এ `ws` Library-র Client-এর `pong` Method খালি করে একটি "মৃত" Client বানানো হয়; Server দুই Heartbeat পরে তাকে বিচ্ছিন্ন করে, এবং Client দেখে 1006 (Close Frame ছাড়া বন্ধ)।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-live
npm install
npm test
npm run demo
npm start
```

`npm test` ১৮টি Test চালায়: `hub` ৪ (অবস্থা-যন্ত্র, অনুমতি, Replay ও Snapshot, Dispatch-এর সময় অনুমতি), `client-state` ৩ (Duplicate, পুরোনো ও ফাঁক; Backoff; Token Bucket), `http` ৩ (Health, API-র অনুমতি ও ভুল Input, SSE Replay), এবং `gateway` ৮ (আসল WebSocket সংযোগে Auth, অনুমতি, Replay, অনুমতি বাতিল, Origin, সীমা ও Heartbeat):

Output

```
# tests 18
# pass 18
# fail 0
```

`npm start`-এর পরে Browser-এ `http://localhost:3027/#token-rafi:bk-1,bk-2` খুলুন। bk-1 রাফির, তাই দেখা যায়; bk-2 মিনার, তাই Console-এ "subscribe FORBIDDEN" এবং তালিকায় "অপেক্ষমাণ"। এবার README-এর `curl` Command দিয়ে Agent হিসেবে bk-1 নিশ্চিত করুন—Page Refresh ছাড়াই অবস্থা বদলায়। Server বন্ধ করে আবার চালালে Page "বিচ্ছিন্ন; … ms পরে আবার চেষ্টা" দেখায় এবং নিজেই ফিরে আসে (তবে Memory-র অবস্থা Restart-এ মুছে যায়—README-র সীমাবদ্ধতা দেখুন)।

এই বইয়ের QA-তে Dashboard আসল Headless Chrome-এ চালানো হয়েছে: সংযোগ ও Auth, নিজের Booking-এর Replay, অন্যের Booking প্রত্যাখ্যান, Refresh ছাড়া Live Update, Server-এর দিক থেকে জোর করে বিচ্ছিন্ন, Backoff-সহ স্বয়ংক্রিয় পুনঃসংযোগ, এবং বিচ্ছিন্ন থাকাকালে ঘটা বাতিলের Replay—আটটি যাচাইয়ের সবগুলো পাস করেছে। প্রথম চালনায় একটি অপ্রত্যাশিত Console Error ধরা পড়েছিল: Browser `/favicon.ico` চেয়ে 404 পেয়েছিল; `index.html`-এ একটি খালি Icon Link যোগ করে তা দূর করা হয়েছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সংযোগ সঙ্গে সঙ্গে বন্ধ, Code 4401	Auth Message পাঠানো হয়নি বা ভুল Token	`open`-এর পরে প্রথমেই `auth`
Upgrade-এ 403	Page-এর Origin তালিকায় নেই	`allowedOrigins`-এ সঠিক Scheme, Host ও Port
Browser-এ "WebSocket connection failed"	ভুল Path বা Port, অথবা `wss`/`ws` গরমিল	`/live`, আর HTTPS Page-এ `wss://`
Live Update আসে না	Subscribe-এর `ack`-এ `ok: false`	Console-এ `ack`-এর `code` দেখুন
Reconnect-এর পর পুরোনো অবস্থা	`afterSequence` পাঠানো হয়নি বা Snapshot উপেক্ষিত	`lastSequence`, `snapshot` Message
সংযোগ প্রতি কয়েক মিনিটে কাটে	Proxy-র Idle Timeout	Heartbeat-এর বিরতি Timeout-এর চেয়ে কম
Server-এর Memory বাড়তেই থাকে	বন্ধ সংযোগের Subscription মুছছে না	`close`-এ Unsubscribe; `subscriberCount` দেখুন

২৭.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
সংযুক্ত মানেই অনুমোদিত ধরা	যে কেউ যেকোনো Room চাইতে পারে	প্রতিটি Subscribe-এ Server-এ অনুমতি
শুধু Subscribe-এর সময় অনুমতি যাচাই	Logout বা ভূমিকা বদলের পরেও Event যায়	Dispatch-এর সময়ও যাচাই
`send()` সফল মানেই Client পেয়েছে	Buffer-এ থাকতে পারে, সংযোগ কাটতে পারে	`ack`, sequence ও Replay
পুনঃসংযোগে কিছু হারায় না ভাবা	বিচ্ছিন্ন সময়ের Event হারায়	Last sequence পাঠিয়ে Replay বা Snapshot
eventId ছাড়া Event	Duplicate চেনা যায় না	স্থির Envelope
সঙ্গে সঙ্গে অসীম পুনঃসংযোগ	Server Restart-এর পর Thundering Herd	Backoff + Jitter + সর্বোচ্চ সীমা
সীমাহীন Send Queue	একটি ধীর Client Memory শেষ করে	Bounded Buffer, Coalesce বা Disconnect
`ws`-এর `error` Event-এ Handler নেই	একটি খারাপ Message-এ পুরো Process বন্ধ	প্রতিটি সংযোগে `on('error')`

ভুল	কেন সমস্যা	সমাধান
Token URL-এর Query-তে	Log ও History-তে ফাঁস	প্রথম Message বা Cookie
Redis Pub/Sub-কে টেকসই Queue ভাবা	Offline Subscriber Message হারায়	Database History বা টেকসই Log

২৭.১৪ Debugging নির্দেশিকা

১. **DevTools-এর Network Panel:** WS Filter দিয়ে সংযোগটি খুলুন; "Messages" Tab-এ প্রতিটি পাঠানো ও আসা Frame, আর বন্ধের Close Code দেখা যায়। SSE-র জন্য "EventStream" Tab।

২. **Close Code পড়ুন:** 1006 মানে Network বা Heartbeat; 1009 মানে বড় Message; 4xxx মানে আপনার Application-এর সিদ্ধান্ত।

৩. **Server-এর Metrics:** TravelLive-এ `gateway.metrics` —সক্রিয় সংযোগ, প্রত্যাখ্যাত Origin, ধীর Client ও Protocol Error। সংযোগ শুধু বাড়তে থাকলে কোথাও Close Handler-এ সমস্যা।

৪. **sequence দিয়ে খুঁজুন:** Client যে শেষ sequence দেখিয়েছে আর Server-এর History-র শেষ sequence মেলান; ফাঁক কোথায় তৈরি হলো তা বোঝা যায়।

৫. **Terminal থেকে পরীক্ষা:** Node.js-এর REPL-এ `new WebSocket('ws://localhost:3027/live')` খুলে হাতে Message পাঠান; SSE-র জন্য `curl -N -H "Authorization: Bearer token-rafi" http://localhost:3027/api/bookings/bk-1/stream`।

৬. **সময়-নির্ভর Bug:** Test-এ Heartbeat ও Auth Timeout ছোট করুন (TravelLive-এর `heartbeatMs`, `authTimeoutMs`), এবং অপেক্ষা করুন নির্দিষ্ট Message-এর জন্য, নির্দিষ্ট সময়ের জন্য নয়।

২৭.১৫ পেশাদার অভ্যাস

- **Event-এর একটি লিখিত চুক্তি রাখুন:** Envelope, প্রতিটি type-এর Payload ও version—Client ও Server দুই দল একই Document দেখে।
- **Domain ও Transport আলাদা:** নিয়ম একটি জায়গায় (Hub), WebSocket ও SSE শুধু বাহক।
- **Snapshot সব সময় একটি পথ:** Replay ব্যর্থ হলে Client-এর একটি নিশ্চিত পথ থাকা চাই।
- **At-least-once ধরে নিয়ে Consumer Idempotent।**
- **প্রতিটি সীমা স্পষ্ট:** Message-এর আকার, হার, Subscription-এর সংখ্যা, Buffer, History।
- **Graceful Shutdown:** Deploy-এর সময় সংযোগ বন্ধ করুন (1001), Client Backoff-সহ নতুন Instance-এ যাবে।
- **আসল সংযোগে Test, আর অন্তত একবার আসল Browser-এ:** Mock দিয়ে Close Code, Buffer বা Heartbeat-এর আচরণ ধরা পড়ে না।

২৭.১৬ অনুশীলনী

প্রাথমিক স্তর

১. ৫০০ জন ব্যবহারকারী প্রতি ১০ সেকেন্ডে Polling করেন, আর ঘণ্টায় ২০টি পরিবর্তন হয়। এক ঘণ্টায় কত অনুরোধ, কতগুলো অর্থহীন, আর গড় দেরি কত? একই কাজে SSE-র সুবিধা ও একটি সীমা লিখুন।

২. একটি Function `unsafeKeys(payload)` লিখুন, যা Payload-এর যেকোনো গভীরতায় `password`, `token`, `email`, `phone` বা `cardNumber` Key খুঁজে তার Path ফেরত দেয়।

৩. TravelLive-এ এই Close Code-গুলোর প্রতিটি কখন দেখা যায়: 1000, 1006, 1009, 4401, 4429?

৪. Socket.IO Client কেন একটি সাধারণ WebSocket Server-এ সংযোগ করতে পারে না? আর কখন Socket.IO বেছে নেওয়া যুক্তিসংগত?

মধ্যম স্তর

৫. At-least-once Delivery-র জন্য একটি Idempotent Notification Consumer লিখুন, যা সীমিত সংখ্যক eventId মনে রাখে। এর একটি দুর্বলতা দেখান।

৬. `BookingView`-এর ওপর একটি `receive(view, event, fetchSnapshot)` লিখুন: ফাঁক হলে Snapshot এনে আবার চেষ্টা করে।

৭. `node:test` দিয়ে তিনটি Test লিখুন: অন্য গ্রাহক Subscribe করতে পারেন না; মালিক ও Agent পারেন; অজানা Booking-ও একই Error দেয়।

৮. একটি SSE Frame Parser লিখুন, যা একাধিক `data` Line জোড়া লাগায় এবং `:` দিয়ে শুরু মন্তব্য Line বাদ দেয়।

চ্যালেঞ্জ স্তর

৯. Outbox Relay-তে Retry যোগ করুন: প্রতিটি ব্যর্থতায় `attempts` বাড়বে, তিনবারের পর সারিটি `dead` হবে। Broker প্রথমে বন্ধ, পরে চালু—এমন পরিস্থিতিতে দেখান।

১০. প্রতি সংযোগে সর্বোচ্চ Subscription-এর সংখ্যা সীমিত করুন (যেমন ২০), যাতে একটি Client হাজার Room-এ যোগ দিয়ে Memory শেষ করতে না পারে।

১১. TravelLive-কে তিনটি Instance-এ চালানোর নকশা করুন: Broker, sequence-এর উৎস, Replay-র History, এবং পুনঃসংযোগে অন্য Instance-এ গেলে কী হবে।

১২. TravelLive-এর একটি নিরাপত্তা পর্যালোচনা করুন: Production-এ যাওয়ার আগে অন্তত ছয়টি ফাঁক চিহ্নিত করুন এবং প্রতিটির সমাধান লিখুন।

২৭.১৭ অনুশীলনীর সমাধান

সমাধান ১ — অনুরোধ: ৫০০ × (৩৬০০ ÷ ১০) = ১,৮০,০০০। কাজের উত্তর: ৫০০ × ২০ = ১০,০০০; অর্থহীন ১,৭০,০০০। গড় দেরি বিরতির অর্ধেক, ৫ সেকেন্ড। SSE-তে ঠিক ১০,০০০ Message, দেরি প্রায় শূন্য, এবং Browser নিজেই পুনঃসংযোগ ও `Last-Event-ID` সামলায়। সীমা: একমুখী—Client-এর কিছু পাঠাতে আলাদা HTTP অনুরোধ লাগে; `EventSource` Custom Header পাঠাতে পারে না; আর HTTP/1.1-এ একই Domain-এ সংযোগের সংখ্যা সীমিত।

সমাধান ২

safe-payload.mjs · JavaScript — চলবে: Node.js

```
// Event-এ যা যাবে না: গোপন বা ব্যক্তিগত তথ্য। নিষিদ্ধ Key খোঁজা হয় গভীরেও।
const FORBIDDEN = new Set(['password', 'token', 'email', 'phone', 'cardNumber']);

export function unsafeKeys(value, path = 'payload') {
  if (value === null || typeof value !== 'object') return [];
  return Object.entries(value).flatMap(([key, inner]) => [
    ...(FORBIDDEN.has(key) ? [`${path}.${key}`] : []),
    ...unsafeKeys(inner, `${path}.${key}`),
  ]);
}

console.log(unsafeKeys({ status: 'CONFIRMED' }));
console.log(unsafeKeys({ status: 'CONFIRMED',
  customer: { name: 'Rafi', email: 'rafi@example.com' }, token: 'abc' }));
```

Output

```
[]
[ 'payload.customer.email', 'payload.token' ]
```

Recursion প্রতিটি Object-এর ভেতরে যায়, তাই লুকানো `customer.email`-ও ধরা পড়ে। এই Function Event প্রকাশের আগে একটি Test বা Development-কালীন যাচাই হিসেবে ভালো কাজ করে; তবে আসল সুরক্ষা হলো Payload তৈরিই করা শুধু অনুমোদিত Field থেকে (Allow-list), নিষিদ্ধ তালিকা থেকে নয়।

সমাধান ৩ — 1000: Client বা Server স্বাভাবিকভাবে বন্ধ করলে (যেমন Browser-এর Tab বন্ধ, `socket.close()`)। 1006: Close Frame ছাড়া সংযোগ শেষ—Network কাটা, অথবা Heartbeat-এ Pong না আসায় Server `terminate()` করলে; এটি কেউ পাঠায় না, Client নিজে জানায়। 1009: Message 8 KB-র বেশি (`maxPayload`)। 4401: Auth-এর আগে অন্য Message, ভুল Token, অথবা ৩ সেকেন্ডের মধ্যে Auth না আসা। 4429: Token Bucket খালি—খুব দ্রুত Message।

সমাধান ৪ — Socket.IO নিজের Handshake (প্রথমে একটি HTTP অনুরোধে Session ID), নিজের Packet-এর Format (Event-এর নাম, Acknowledgement-এর ID) এবং প্রয়োজনে Long Polling Transport ব্যবহার করে; সাধারণ WebSocket Server এগুলোর কোনোটিই বোঝে না, তাই দুই পাশে Socket.IO লাগে। এটি যুক্তিসংগত যখন Room, Acknowledgement, স্বয়ংক্রিয় পুনঃসংযোগ ও Multi-Instance Adapter তৈরি অবস্থায় চাই, এবং Client ও Server দুটিই আপনার নিয়ন্ত্রণে। অন্য দলের বা IoT Client-এর সঙ্গে কথা বললে, বা কোনো Standard Protocol লাগলে, সাধারণ WebSocket।

সমাধান ৫

idempotent-consumer.mjs · JavaScript — চলবে: Node.js

```
// At-least-once Delivery: একই Event একাধিকবার আসতে পারে।
// Consumer মনে রাখে কোন eventId-র কাজ হয়ে গেছে (সীমিত Memory-তে)।
function createConsumer(handler, { limit = 3 } = {}) {
  const done = new Set();
  return (event) => {
    if (done.has(event.eventId)) return 'skipped';
    handler(event);
    done.add(event.eventId);
    if (done.size > limit) done.delete(done.values().next().value);
    return 'handled';
  };
}

const sent = [];
const notify = createConsumer((e) => sent.push(`SMS ${e.bookingId}`));
const deliveries = ['e1', 'e2', 'e1', 'e3', 'e2', 'e4', 'e5', 'e1'];
for (const eventId of deliveries) {
  console.log(eventId, notify({ eventId, bookingId: `bk-${eventId}` }));
}
console.log('পাঠানো SMS:', sent.length);
```

Output

```
e1 handled
e2 handled
e1 skipped
e3 handled
e2 skipped
e4 handled
e5 handled
e1 handled
পাঠানো SMS: 6
```

শেষ Line-এ দুর্বলতা দেখা যায়: Memory-তে মাত্র তিনটি ID থাকে, তাই অনেক পরে আসা পুরোনো `e1` আবার SMS পাঠায়। আর Process Restart হলে সব ভুলে যায়। টেকসই সমাধান: Database-এ `processed_events(event_id PRIMARY KEY)`, এবং কাজ ও সেই সারি একই Transaction-এ (অধ্যায় ২৬-এর Idempotency)। যেখানে sequence আছে, সেখানে "শেষ প্রক্রিয়াকৃত sequence" রাখাই যথেষ্ট—তার চেয়ে ছোট সব বাদ।

সমাধান ৬

gap-recovery.mjs · JavaScript — চলবে: Node.js

```
import { BookingView } from '../../project/travel-live/src/client-state.mjs';

// ফাঁক ধরা পড়লে Server-এর Snapshot আনা; Snapshot-এর চেয়ে পুরোনো Event বাদ।
async function receive(view, event, fetchSnapshot) {
  const result = view.apply(event);
  if (result !== 'gap') return result;
  const snapshot = await fetchSnapshot(event.bookingId);
  view.loadSnapshot(snapshot);
  return `gap, snapshot #${snapshot.sequence}, ${view.apply(event)}`;
}

// Server-এর জায়গায় একটি নকল fetch; আসলে GET /api/bookings/:id।
const fakeFetch = async (bookingId) => ({ bookingId, title: 'Sajek',
  status: 'CONFIRMED', sequence: 5 });
const ev = (sequence, status) => ({ eventId: `e${sequence}`,
  bookingId: 'bk-9', sequence, payload: { status } });

const view = new BookingView();
console.log(await receive(view, ev(1, 'PENDING'), fakeFetch));
console.log(await receive(view, ev(6, 'CANCELLED'), fakeFetch));
console.log(await receive(view, ev(4, 'CONFIRMED'), fakeFetch));
console.log(view.get('bk-9'));
```

Output

```
applied
gap, snapshot #5, applied
stale
{ title: 'Sajek', status: 'CANCELLED', sequence: 6 }
```

#৬ এসে ফাঁক দেখায়; Snapshot #৫ পর্যন্ত নিয়ে আসে, তারপর #৬ বৈধভাবে প্রয়োগ হয়। দেরিতে আসা #৪ এখন `stale`—Snapshot তাকে ইতিমধ্যে অন্তর্ভুক্ত করেছে। Snapshot নিজেও যদি Event-এর চেয়ে পুরোনো হয় (একসঙ্গে আরও Event এলে), দ্বিতীয় `apply` আবার `gap` দেবে; আসল Code-এ তখন সংযোগ নতুন করে Replay চাওয়া নিরাপদ।

সমাধান ৭

subscribe.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { LiveHub } from '../../project/travel-live/src/live-hub.mjs';

function hubWithBooking() {
  const hub = new LiveHub();
  hub.addUser({ id: 'rafi', role: 'customer' });
  hub.addUser({ id: 'mina', role: 'customer' });
  hub.addUser({ id: 'nadia', role: 'agent' });
  const { bookingId } = hub.createBooking('rafi', 'Kuakata');
  return { hub, bookingId };
}

test('অন্য গ্রাহক Subscribe করতে পারেন না', () => {
  const { hub, bookingId } = hubWithBooking();
  assert.throws(() => hub.subscribe('mina', bookingId, () => {}),
    { code: 'FORBIDDEN' });
  assert.equal(hub.subscriberCount(bookingId), 0);
});

test('মালিক ও Agent পারেন', () => {
  const { hub, bookingId } = hubWithBooking();
  hub.subscribe('rafi', bookingId, () => {});
  hub.subscribe('nadia', bookingId, () => {});
  assert.equal(hub.subscriberCount(bookingId), 2);
});

test('অজানা Booking-ও FORBIDDEN: অস্তিত্ব ফাঁস হয় না', () => {
  const { hub } = hubWithBooking();
  assert.throws(() => hub.subscribe('rafi', 'bk-999', () => {}),
    { code: 'FORBIDDEN' });
});
```

Output

```
ok 1 - অন্য গ্রাহক Subscribe করতে পারেন না
ok 2 - মালিক ও Agent পারেন
ok 3 - অজানা Booking-ও FORBIDDEN: অস্তিত্ব ফাঁস হয় না
# tests 3
# pass 3
# fail 0
```

তৃতীয় Test নিরাপত্তার একটি সূক্ষ্ম নিয়ম রক্ষা করে: "নেই" ও "অনুমতি নেই"-এর উত্তর আলাদা হলে আক্রমণকারী ID অনুমান করে বুঝতে পারত কোন Booking আছে।

সমাধান ৮

sse-parser.mjs · JavaScript — চলবে: Node.js

```
// SSE Frame পড়া: একাধিক data Line জোড়া লাগে, ':' দিয়ে শুরু Line মন্তব্য।
export function parseFrame(frame) {
  const event = { type: 'message', data: [], id: null };
  for (const line of frame.split('\n')) {
    if (line === '' || line.startsWith(':')) continue;
    const colon = line.indexOf(':');
    const field = colon === -1 ? line : line.slice(0, colon);
    const value = colon === -1 ? '' : line.slice(colon + 1).replace(/^ /, '');
    if (field === 'data') event.data.push(value);
    if (field === 'event') event.type = value;
    if (field === 'id') event.id = value;
  }
  return event.data.length ? { ...event, data: event.data.join('\n') } : null;
}

const stream = ': ping\n\n'
  + 'id: 7\nevent: booking.status.changed\ndata: {"a":1,\ndata: "b":2}\n\n'
  + 'data: সাধারণ বার্তা\n\n';
for (const frame of stream.split('\n\n')) {
  const event = parseFrame(frame);
  if (event) console.log(event.id, event.type, event.data.replace('\n', ' '));
  else if (frame) console.log('(মন্তব্য বা ফাঁকা Frame বাদ)');
}
```

Output

```
(মন্তব্য বা ফাঁকা Frame বাদ)
7 booking.status.changed {"a":1, "b":2}
null message সাধারণ বার্তা
```

Specification অনুযায়ী একাধিক `data` Line একটি `\n` দিয়ে জোড়া লাগে, `:`-এর পরের একটি Space বাদ যায়, আর `event` না থাকলে type হয় `message`। আসল Stream-এ একটি Frame কয়েকটি Chunk-এ ভাগ হয়ে আসতে পারে, তাই উদাহরণ ০২-এর মতো Buffer রেখে `\n\n` পর্যন্ত অপেক্ষা করতে হয়।

সমাধান ৯

outbox-retry.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE outbox (id INTEGER PRIMARY KEY, type TEXT NOT NULL,
  attempts INTEGER NOT NULL DEFAULT 0,
```

```
  state TEXT NOT NULL DEFAULT 'pending');
  INSERT INTO outbox (type) VALUES ('booking.confirmed'), ('poison.event');`);

// প্রতিটি ব্যর্থতায় attempts বাড়ে; ৩ বারের পরে 'dead' (মানুষ দেখবেন)।
function relay(publish, maxAttempts = 3) {
  const rows = db.prepare(
    "SELECT * FROM outbox WHERE state = 'pending' ORDER BY id").all();
  for (const row of rows) {
    try {
      publish(row);
      db.prepare("UPDATE outbox SET state = 'sent' WHERE id = ?").run(row.id);
    } catch {
      const attempts = row.attempts + 1;
      const state = attempts >= maxAttempts ? 'dead' : 'pending';
      db.prepare('UPDATE outbox SET attempts = ?, state = ? WHERE id = ?')
        .run(attempts, state, row.id);
    }
  }
}

let brokerUp = false;
const publish = (row) => {
  if (!brokerUp || row.type === 'poison.event') throw new Error('fail');
};
for (const round of [1, 2, 3]) {
  if (round === 2) brokerUp = true; // Broker দ্বিতীয় দফায় ফিরে এল
  relay(publish);
  const rows = db.prepare('SELECT id, attempts, state FROM outbox').all();
  console.log(`দফা ${round}:`,
    rows.map((r) => `${r.id}:${r.state}/${r.attempts}`).join(' '));
}
```

Output

```
দফা 1: 1:pending/1 2:pending/1
দফা 2: 1:sent/1 2:pending/2
দফা 3: 1:sent/1 2:dead/3
```

প্রথম দফায় Broker বন্ধ, দুটিই ব্যর্থ। দ্বিতীয় দফায় সাধারণ Event যায়, কিন্তু "poison" Event—যা সব সময় ব্যর্থ হয়—তৃতীয়বারে `dead`। এটি না থাকলে একটি খারাপ সারি চিরকাল Retry হতো এবং পেছনের সারিগুলো আটকে রাখত। বাস্তবে Retry-এর মাঝে Backoff, `dead` সারির জন্য Alert, এবং কারণ ঠিক করার পর হাতে আবার চালানোর একটি পথ রাখুন।

সমাধান ১০

subscription-limit.mjs · JavaScript — চলবে: Node.js

```
import { LiveHub, LiveError } from '../../project/travel-live/src/live-hub.mjs';

// প্রতি Connection-এ সর্বোচ্চ max-টি Subscription: একটি Client হাজার
// Room-এ যুক্ত হয়ে Server-এর Memory শেষ করতে পারবে না।
function limitedSubscriber(hub, userId, max) {
  const active = new Map();
  return {
    subscribe(bookingId, listener) {
      if (active.size >= max) {
        throw new LiveError('LIMIT', `সর্বোচ্চ ${max}টি Subscription`);
      }
      const result = hub.subscribe(userId, bookingId, listener);
      active.set(bookingId, result.unsubscribe);
      return result;
    },
    unsubscribe(bookingId) {
      active.get(bookingId)?.();
      active.delete(bookingId);
    },
  };
}

const hub = new LiveHub();
hub.addUser({ id: 'nadia', role: 'agent' });
hub.addUser({ id: 'rafi', role: 'customer' });
for (let i = 0; i < 3; i += 1) hub.createBooking('rafi', `Tour ${i + 1}`);

const conn = limitedSubscriber(hub, 'nadia', 2);
for (const id of ['bk-1', 'bk-2', 'bk-3']) {
  try {
    conn.subscribe(id, () => {});
    console.log(id, 'subscribed');
  } catch (error) {
    console.log(id, error.code, error.message);
  }
}
conn.unsubscribe('bk-1');
conn.subscribe('bk-3', () => {});
console.log('bk-3 subscribed after unsubscribe; bk-1 count =',
  hub.subscriberCount('bk-1'));
```

Output

```
bk-1 subscribed
bk-2 subscribed
bk-3 LIMIT সর্বোচ্চ 2টি Subscription
bk-3 subscribed after unsubscribe; bk-1 count = 0
```

TravelLive-এর Gateway-তে এটি সবচেয়ে সহজে যায় `subscribe` Function-এ: `conn.subs.size >= 20` হলে `ack`-এ `ok: false, code: 'LIMIT'`। Unsubscribe করলে জায়গা খালি হয়, তাই বৈধ ব্যবহারকারী আটকে যান না।

সমাধান ১১ — Broker: Redis (Pub/Sub দ্রুত Fan-out-এর জন্য) অথবা টেকসই Log (Redis Streams, NATS JetStream, Kafka)। sequence-এর উৎস: Booking Service, Database Transaction-এর ভেতরে (`UPDATE bookings SET sequence = sequence + 1 ... RETURNING sequence`), Outbox সারির সঙ্গে একসঙ্গে—কোনো Gateway sequence তৈরি করে না। History: Database-এর `booking_events` Table (Outbox-ই হতে পারে), যেখান থেকে যেকোনো Instance Replay দিতে পারে; পুরোনো Event মুছলে Snapshot। প্রতিটি Instance শুধু তার সংযুক্ত Client-দের Room-এর Channel-এ Subscribe করে। পুনঃসংযোগে Client অন্য Instance-এ গেলে কোনো সমস্যা নেই, কারণ Client নিজের শেষ sequence পাঠায় এবং History সব Instance-এর জন্য এক জায়গায়—এই কারণেই অবস্থা Gateway-এর Memory-তে রাখা হয় না।

সমাধান ১২ — (১) স্থির Demo Token: Login, মেয়াদযুক্ত ও বাতিলযোগ্য Session (অধ্যায় ২০)। (২) `ws://` ও `http://`: TLS-যুক্ত Reverse Proxy-র পেছনে `wss://`। (৩) সব অবস্থা এক Process-এর Memory-তে: Database, Outbox ও টেকসই History। (৪) প্রতি IP বা প্রতি ব্যবহারকারী সংযোগের সংখ্যার সীমা নেই: Gateway বা Proxy-তে সীমা। (৫) প্রতি সংযোগে Subscription-এর সীমা নেই: সমাধান ১০। (৬) Audit Log নেই: কে কখন কোন অবস্থা বদলাল, তা টেকসই Log-এ। (৭) Metrics শুধু Memory-তে: Prometheus বা অনুরূপ ব্যবস্থায় প্রকাশ, এবং অস্বাভাবিক হারে Alert (অধ্যায় ২৮)। (৮) Session শেষ হলে খোলা সংযোগ চলতে থাকে: মেয়াদ শেষে `revokeUser`-এর মতো পুনর্যাচাই ও সংযোগ বন্ধ।

২৭.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোনটি দ্বিমুখী, স্থায়ী সংযোগ?

ক) Short Polling

খ) WebSocket

গ) Server-Sent Events

ঘ) `localStorage`

প্রশ্ন ২. SSE-তে Browser পুনঃসংযোগের সময় কোন Header দিয়ে শেষ পাওয়া Event জানায়?

ক) `Authorization`

খ) `If-None-Match`

গ) `Last-Event-ID`

ঘ) `Origin`

প্রশ্ন ৩. একটি Booking-এর Room-এ যোগ দেওয়ার অনুমতি কোথায় যাচাই করা উচিত?

ক) Browser-এর JavaScript-এ

খ) Room-এর নাম গোপন রেখে

গ) শুধু সংযোগের সময় একবার

ঘ) Server-এ, প্রতিটি Subscribe-এ এবং Event পাঠানোর সময়ও

প্রশ্ন ৪. Event-এর `eventId`-এর প্রধান কাজ কী?

ক) Duplicate চেনা

খ) Event-এর ক্রম নির্ধারণ

গ) Payload-এর সংস্করণ জানানো

ঘ) ব্যবহারকারী চেনা

প্রশ্ন ৫. Client-এ আসা Event-এর sequence শেষ দেখা sequence-এর চেয়ে দুই বেশি। সঠিক পদক্ষেপ?

ক) প্রয়োগ করা, কারণ নতুন

খ) বাদ দেওয়া, কারণ Duplicate

গ) প্রয়োগ না করে Replay বা Snapshot চাওয়া

ঘ) Page Reload করা

প্রশ্ন ৬. Redis Pub/Sub সম্পর্কে কোনটি সঠিক?

ক) Offline Subscriber-এর জন্য Message জমা রাখে

খ) শুধু সেই মুহূর্তে সংযুক্ত Subscriber Message পায়

গ) Exactly-once নিশ্চিত করে

ঘ) Message ৩০ দিন রাখে

প্রশ্ন ৭. Transactional Outbox কোন সমস্যার সমাধান করে?

ক) Database-এ লেখা ও Event প্রকাশের মাঝে অসামঞ্জস্য (Dual Write)

খ) ধীর Query

গ) Duplicate Event

ঘ) WebSocket-এর Origin যাচাই

প্রশ্ন ৮. Server Restart-এর পর হাজার Client-এর জন্য সঠিক পুনঃসংযোগ নীতি?

ক) সঙ্গে সঙ্গে, বারবার

খ) কখনো আবার চেষ্টা নয়

গ) প্রতি ১ সেকেন্ডে স্থির বিরতিতে

ঘ) Exponential Backoff, সর্বোচ্চ সীমা ও Jitter

প্রশ্ন ৯. একটি ধীর Client-এর জন্য Server-এর Send Buffer বাড়তেই থাকলে কোনটি সঠিক নয়?

ক) Buffer-এর সীমা ছাড়ালে বিচ্ছিন্ন করা

খ) একই Booking-এর পুরোনো Event একীকরণ

গ) সীমাহীন Queue রেখে অপেক্ষা

ঘ) পুরোনো Event ফেলে Client-কে Snapshot চাওয়ানো

প্রশ্ন ১০. At-least-once Delivery-তে কী ঘটতে পারে, তাই Consumer-কে Idempotent হতে হয়?

ক) একই Event একাধিকবার পৌঁছানো

খ) কোনো Event না পৌঁছানো, এবং তা নিশ্চিত

গ) সব Event-এর বিশ্বব্যাপী ক্রম

ঘ) শূন্য দেরি

২৭.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–গ · ৬–খ · ৭–ক · ৮–ঘ · ৯–গ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	SSE একমুখী, Polling প্রতিবার নতুন অনুরোধ; WebSocket দুই দিকে একই সংযোগে।
২	গ	TravelLive-এর `sse.mjs` এই Header থেকেই `afterSequence` নেয়।
৩	ঘ	Room-এর নাম অনুমানযোগ্য, আর অনুমতি পরে বাতিল হতে পারে (Hub-এর `#dispatch`)।
৪	ক	ক্রম ও ফাঁকের জন্য `sequence`, সংস্করণের জন্য `version`।
৫	গ	মাঝের Event হারিয়েছে; `BookingView` একে `gap` বলে।
৬	খ	উদাহরণ ০৮-এ Offline Agent #৩ আর পায়নি।
৭	ক	দুটি লেখা একই Transaction-এ; Relay পরে প্রকাশ করে।
৮	ঘ	Jitter Client-দের ছড়িয়ে দেয়; উদাহরণ ০৭-এ ৩ থেকে ৫ সেকেন্ড।
৯	গ	সীমাহীন Queue একটি Client দিয়েই Server-এর Memory শেষ করতে পারে।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	ক	Retry মানেই Duplicate-এর সম্ভাবনা; উদাহরণ ০৬-এর দ্বিতীয় Relay।

২৭.২০ অধ্যায়ের সারসংক্ষেপ

- Real-Time-এর চার পথ: Polling সরল কিন্তু অপচয়ী; SSE একমুখী HTTP Stream; WebSocket দ্বিমুখী স্থায়ী সংযোগ; Socket.IO তার ওপর Room, Acknowledgement ও পুনঃসংযোগসহ নিজস্ব Protocol।
- WebSocket: HTTP Upgrade, চারটি Lifecycle Event, Close Code; Origin যাচাই, প্রথম Message-এ Auth, আকার ও হারের সীমা।
- স্থির Event Envelope: eventId (Duplicate), sequence (ক্রম ও ফাঁক), version (বিবর্তন); Payload-এ শুধু প্রয়োজনীয় তথ্য।
- Command অনুরোধ, Event ঘটে যাওয়া সত্য; Transactional Outbox Dual Write-এর ফাঁক বন্ধ করে, কিন্তু At-least-once দেয়।
- Exactly-once Effect = At-least-once পৌঁছানো + Idempotent প্রক্রিয়া।
- পুনঃসংযোগে শেষ sequence পাঠিয়ে Replay; History না থাকলে Snapshot।
- Backoff + Jitter, Heartbeat, এবং Bounded Buffer দিয়ে Backpressure।
- একাধিক Instance-এ Pub/Sub দিয়ে Fan-out; কিন্তু Pub/Sub টেকসই নয়, sequence আসে মালিক Service থেকে।
- অনুমতি প্রতিটি Subscribe-এ এবং প্রতিটি Dispatch-এ।

পরের অধ্যায়ে আপনি এই ধরনের Application Docker-এ প্যাক করবেন, CI/CD দিয়ে স্বয়ংক্রিয়ভাবে যাচাই ও Deploy করবেন, এবং Health Check, Log ও Metrics দিয়ে Production-এ পর্যবেক্ষণ করবেন—TravelLive-এর Graceful Shutdown ও Metrics সেখানে কাজে লাগবে।

২৭.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Polling / Long Polling	বারবার অনুরোধ / খবর না আসা পর্যন্ত আটকে রাখা অনুরোধ।
Server-Sent Events (SSE)	Server থেকে Browser-এ একমুখী HTTP Event Stream।
WebSocket	HTTP Upgrade থেকে তৈরি দ্বিমুখী স্থায়ী সংযোগ।
Close Code	WebSocket বন্ধের কারণের সংখ্যা।
Event Envelope	প্রতিটি Event-এর স্থির কাঠামো।
Command / Event	কাজের অনুরোধ / ঘটে যাওয়া সত্য।

পরিভাষা	অর্থ
Transactional Outbox	Business পরিবর্তন ও প্রকাশিতব্য Event একই Transaction-এ লেখা।
At-least-once / At-most-once	হারায় না কিন্তু Duplicate সম্ভব / Duplicate নেই কিন্তু হারাতে পারে।
Idempotent Consumer	একই Event বারবার পেলেও ফল একবারের মতো।
Replay / Snapshot	বাদ পড়া Event আবার পাঠানো / বর্তমান পূর্ণ অবস্থা।
Exponential Backoff, Jitter	ক্রমশ দ্বিগুণ অপেক্ষা, এলোমেলো বিচ্যুতি।
Heartbeat (Ping/Pong)	সংযোগ জীবিত কি না নিয়মিত যাচাই।
Backpressure	ধীর গ্রাহকের কারণে জমে ওঠা Data সামলানো।
Pub/Sub, Broker	Channel-এ প্রকাশ ও গ্রহণ; মাঝের Message-বাহক।

২৭.২২ আরও পড়ুন

- **MDN — WebSocket API:** `https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API` — Browser-এর WebSocket ও Close Code।
- **MDN — Using server-sent events:** `https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events` — `EventSource` ও Stream-এর Format।
- **RFC 6455 — The WebSocket Protocol:** `https://www.rfc-editor.org/rfc/rfc6455` — Handshake, Frame ও Close Code।
- **ws — README:** `https://github.com/websockets/ws` — Node.js-এর WebSocket Server, Heartbeat-এর উদাহরণ।
- **Socket.IO — Documentation:** `https://socket.io/docs/v4/` — Room, Acknowledgement ও Redis Adapter।
- **Redis — Pub/Sub:** `https://redis.io/docs/latest/develop/interact/pubsub/` — Delivery-র নিশ্চয়তা ও সীমা।

অধ্যায় ২৮

২৮

Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Development ও Production পরিবেশের পার্থক্য এবং Twelve-Factor App-এর মূল নীতিগুলো ব্যাখ্যা করতে।
- Environment থেকে Configuration পড়ে শুরুতেই যাচাই করতে (Fail Fast), এবং গোপন মান Log থেকে লুকাতে।
- পুনরুৎপাদনযোগ্য Build Artifact, Lockfile, Commit SHA ও Image Digest-এর ভূমিকা বুঝতে।
- একটি নিরাপদ Dockerfile লিখতে: নির্দিষ্ট Base Image, Non-root ব্যবহারকারী, `.dockerignore`, HEALTHCHECK এবং Exec Form CMD।
- Docker Compose দিয়ে Production-এর মতো Local পরিবেশ সাজাতে।
- একটি CI Workflow লিখতে এবং Docker ছাড়াই Deploy File যাচাই করতে।
- Blue/Green, Rolling ও Canary Rollout তুলনা করতে, এবং সংখ্যা দিয়ে Canary-র সিদ্ধান্ত নিতে।
- Liveness ও Readiness আলাদা করতে এবং Graceful Shutdown বাস্তবায়ন করতে।
- Structured Log, Prometheus Format-এ Metrics ও Trace দিয়ে Application পর্যবেক্ষণ করতে।
- SLI, SLO, Error Budget ও Burn Rate হিসাব করে কাজে লাগানো Alert নকশা করতে।
- Secret, Supply Chain, Backup, RPO/RTO ও Incident Response-এর মূল অভ্যাস প্রয়োগ করতে।
- **DeployReady** নামে একটি Production-উপযোগী, পরীক্ষিত API সাজাতে।

পূর্বশর্ত

- অধ্যায় ২০: Secret, Token ও নিরাপদ Error বার্তা।
- অধ্যায় ২২: `node:test`, Integration Test।
- অধ্যায় ২৫: Node.js HTTP Server, Middleware, JSON Error চুক্তি, Graceful Shutdown-এর প্রথম পরিচয়।
- অধ্যায় ২৬: Migration ও Backup।
- অধ্যায় ২৭: দীর্ঘস্থায়ী সংযোগ ও Metrics-এর প্রয়োজন।
- Terminal-এ Command চালানোর অভ্যাস। Docker জানা আবশ্যক নয়।

এই অধ্যায় কেন গুরুত্বপূর্ণ

আপনার Computer-এ `npm start` চালালে Server চালু হয়—কাজ শেষ মনে হয়। কিন্তু একটি ভ্রমণ সংস্থার Operations দল যখন সকাল আটটায় হোটেল Booking যাচাইয়ের কাজ খোলেন, তখন প্রশ্নগুলো অন্য: নতুন সংস্করণ Deploy করতে গিয়ে কি চলমান অনুরোধ কেটে যাবে? কোন সংস্করণ এখন চলছে? ভুল Configuration নিয়ে Server চালু হয়ে চুপচাপ ভুল কাজ করছে না তো? রাত দুটোয় Error বাড়লে কে জানবে, আর কীভাবে বুঝবে কোন অনুরোধে সমস্যা? Database নষ্ট হলে কতটা তথ্য হারাবে, আর কতক্ষণে সেবা ফিরবে?

Deployment শুধু Code Server-এ তোলা নয়; এটি একটি পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য এবং ফেরানো-যোগ্য প্রক্রিয়া। এই অধ্যায় সেই প্রক্রিয়ার প্রতিটি অংশ আসল Code দিয়ে দেখায়—Configuration থেকে Container, CI থেকে Canary, Health Check থেকে Error Budget—এবং শেষে একটি API তৈরি করে যা Production-এর এই প্রশ্নগুলোর উত্তর দিতে প্রস্তুত।

২৮.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`process.env` পড়া (অধ্যায় ২৫)	পুরো Configuration শুরুতেই যাচাই, সব ভুল একসঙ্গে
Graceful Shutdown-এর পরিচয় (অধ্যায় ২৫)	Drain, Readiness 503, Timeout ও Container-এর Grace Period
Test (অধ্যায় ২২)	CI Pipeline, Smoke Test, Canary বিশ্লেষণ
Error-এর নিরাপদ বার্তা (অধ্যায় ২০)	Structured Log, Request ID, গোপন মান লুকানো
Metrics-এর প্রয়োজন (অধ্যায় ২৭)	Prometheus Format, Histogram, SLO, Burn Rate
Backup (অধ্যায় ২৬)	RPO, RTO, Restore Drill, Incident Response

> **নোট:** QA-যন্ত্রে Docker CLI ছিল, কিন্তু Docker Daemon চালু ছিল না; তাই `docker build` ও `docker compose up` চালানো হয়নি। পরিবর্তে Dockerfile একটি স্থির যাচাই Script দিয়ে, আর `compose.yaml` ও CI Workflow দুটি আলাদা YAML Parser দিয়ে যাচাই করা হয়েছে। CI Workflow GitHub-এ চালানো হয়নি—শুধু তার গঠন ও তার ডাকা প্রতিটি Script-এর অস্তিত্ব পরীক্ষিত। Application নিজে, তার Health Check, Shutdown ও Smoke Test আসলেই চালানো ও পরীক্ষিত।

২৮.২ Localhost থেকে Production: কী বদলায়

বিষয়	Local Development	Production-এর প্রত্যাশা
Configuration	`.env` বা হাতে লেখা মান	Platform-এর Configuration ও Secret Store, শুরুতেই যাচাই
Process	একটি Terminal	Supervisor বা Container Platform, স্বয়ংক্রিয় Restart
Data	Memory বা Local File	টেকসই Storage, Backup, Restore-এর অনুশীলন
Network	`localhost`	DNS, TLS, Load Balancer, Firewall
ব্যর্থতা	Developer নিজে দেখেন	Log, Metrics, Alert, Incident প্রক্রিয়া
Release	হাতে Restart	CI, ধাপে ধাপে Rollout, Rollback
সংস্করণ	"আমার Computer-এর Code"	নির্দিষ্ট Commit SHA ও Image Digest

এই সারণির প্রতিটি সারি একটি প্রশ্ন: "এটি ব্যর্থ হলে কীভাবে জানব, আর কীভাবে ফিরে যাব?" বাকি অধ্যায় সেই প্রশ্নের উত্তর।

২৮.৩ Twelve-Factor ও Configuration

Twelve-Factor App একটি প্রচলিত নীতিমালা, যার কয়েকটি এই অধ্যায়ের কেন্দ্রে: Configuration Code-এর বাইরে, Environment-এ (III); Process Stateless, অবস্থা থাকে Database-এ (VI); দ্রুত চালু ও সুশৃঙ্খলভাবে বন্ধ হওয়া (IX); Development ও Production যতটা সম্ভব একই (X); এবং Log একটি Event Stream, যা Process শুধু Standard Output-এ লেখে (XI)।

Configuration পড়ার সবচেয়ে বিপজ্জনক ভুল: `Number(process.env.PORT)` লিখে এগিয়ে যাওয়া। `PORT=abc` হলে মান `NaN`, আর Server হয়তো একটি এলোমেলো Port-এ চালু হয়। তার চেয়ে ভালো—শুরুতেই সব যাচাই, এবং ভুল থাকলে চালুই না হওয়া (**Fail Fast**):

config.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
// Twelve-Factor: সব Configuration Environment থেকে; শুরুতেই যাচাই।
// ভুল থাকলে সব ভুল একসঙ্গে জানিয়ে Process চালুই হবে না (Fail Fast)।
const LEVELS = ['debug', 'info', 'warn', 'error'];

function integer(env, name, fallback, min, max, errors) {
  const raw = env[name] ?? String(fallback);
  const value = Number(raw);
  if (!/^\d+$/.test(raw) || value < min || value > max) {
    errors.push(`${name} হতে হবে ${min} থেকে ${max}-এর পূর্ণসংখ্যা`);
  }
  return value;
}

// ...
```

config.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export function loadConfig(env = process.env) {
  const errors = [];
  const nodeEnv = env.NODE_ENV ?? 'development';
  const config = {
    nodeEnv,
    port: integer(env, 'PORT', 3000, 0, 65535, errors),
    logLevel: env.LOG_LEVEL ?? 'info',
    shutdownTimeoutMs: integer(env, 'SHUTDOWN_TIMEOUT_MS', 10_000,
      1, 120_000, errors),
    drainDelayMs: integer(env, 'DRAIN_DELAY_MS', 0, 0, 60_000, errors),
    version: env.APP_VERSION ?? '0.0.0-dev',
    commit: env.GIT_SHA ?? 'unknown',
    opsToken: env.OPS_TOKEN,
  };
  if (!['development', 'test', 'production'].includes(nodeEnv)) {
    errors.push('NODE_ENV হতে হবে development, test বা production');
  }
  if (!LEVELS.includes(config.logLevel)) {
    errors.push(`LOG_LEVEL হতে হবে ${LEVELS.join(', ')}`);
  }
  if (nodeEnv === 'production' && (config.opsToken ?? '').length < 24) {
    errors.push('Production-এ OPS_TOKEN কমপক্ষে ২৪ অক্ষরের হতে হবে');
  }
  if (errors.length) {
    throw new Error(`Configuration ভুল:\n- ${errors.join('\n- ')}`);
  }
  return Object.freeze(config);
}
```

```
// Log বা Error-এ Config দেখাতে হলে গোপন মান লুকিয়ে।
export function describeConfig(config) {
  return { ...config, opsToken: config.opsToken ? '[REDACTED]' : undefined };
}
```

তিনটি সিদ্ধান্ত লক্ষ করুন। এক, ভুলগুলো একটি Array-তে জমে, তাই Operator একবারে সব ঠিক করতে পারেন—একটি ঠিক করে Deploy, তারপর পরেরটি, এমন নয়। দুই, Production-এ `OPS_TOKEN` বাধ্যতামূলক এবং দীর্ঘ; Development-এ ঐচ্ছিক। তিন, `describeConfig` গোপন মান লুকিয়ে দেয়, যাতে "চালু হলো, এই Configuration দিয়ে" Log-এ Token ফাঁস না হয়।

config-fail-fast.mjs · JavaScript চলবে: Node.js

```
import { loadConfig, describeConfig }
  from '../project/deploy-ready/src/config.mjs';

const environments = {
  'Local, কিছুই দেওয়া নেই': {},
  'ভুল টাইপ': { PORT: 'abc', LOG_LEVEL: 'verbose' },
  'Production, Token নেই': { NODE_ENV: 'production', PORT: '8080' },
  'Production, ঠিকঠাক': { NODE_ENV: 'production', PORT: '8080',
    OPS_TOKEN: 'k'.repeat(32), GIT_SHA: '9f2c1ab' },
};

for (const [name, env] of Object.entries(environments)) {
  try {
    const config = describeConfig(loadConfig(env));
    console.log(`${name}: চালু`, { port: config.port,
      env: config.nodeEnv, token: config.opsToken });
  } catch (error) {
    console.log(`${name}: থামল\n  ${error.message.replace(/\n/g, '\n  ')}`);
  }
}
```

Output

```
Local, কিছুই দেওয়া নেই: চালু { port: 3000, env: 'development', token: undefined }
ভুল টাইপ: থামল
  Configuration ভুল:
  - PORT হতে হবে 0 থেকে 65535-এর পূর্ণসংখ্যা
  - LOG_LEVEL হতে হবে debug, info, warn, error
Production, Token নেই: থামল
  Configuration ভুল:
  - Production-এ OPS_TOKEN কমপক্ষে ২৪ অক্ষরের হতে হবে
Production, ঠিকঠাক: চালু { port: 8080, env: 'production', token: '[REDACTED]' }
```

`.env` File Git-এ না রাখা জরুরি, কিন্তু তা একা Secret Management নয়। Production-এ গোপন মান থাকে Platform-এর Secret Store-এ (Cloud Provider-এর Secret Manager, Kubernetes Secret, বা Vault), যেখানে কে পড়তে পারে তা নিয়ন্ত্রিত, পরিবর্তনের ইতিহাস থাকে, এবং মান নিয়মিত বদলানো (Rotation) যায়।

২৮.৪ Build Artifact, Lockfile ও পুনরুৎপাদনযোগ্যতা

একটি পরিচ্ছন্ন Release-প্রবাহ: Source Code, তারপর স্বয়ংক্রিয় যাচাই, তারপর একটি **অপরিবর্তনীয় Build Artifact** (যেমন Docker Image), এবং সেই একই Artifact প্রথমে Staging-এ, পরে Production-এ। Production-এর জন্য আলাদা করে আবার Build করবেন না—তাহলে যা পরীক্ষা হয়েছে আর যা চলছে, তা এক থাকে না।

- **Lockfile** (`package-lock.json`) প্রতিটি Dependency-র হুবহু সংস্করণ ও Checksum ধরে রাখে। CI ও Image-এ `npm ci` ব্যবহার করুন: এটি শুধু Lockfile মেনে Install করে, আর Lockfile ও `package.json` না মিললে থেমে যায়। `npm install` Lockfile হালনাগাদ করতে পারে।
- **Release-এর পরিচয়:** Git Commit SHA এবং Image Digest (`sha256:...`)। `latest` Tag একটি চলমান লক্ষ্য: আজ এক Image, কাল আরেক। Rollback-এর জন্য জানতে হবে আগে ঠিক কোনটি চলছিল।
- **একই Runtime:** Node.js-এর Major Version, Operating System ও Native Module—এগুলোও Artifact-এর অংশ। Container এর বড় সুবিধা।

DeployReady-এর কোনো Runtime Dependency নেই, তাই Lockfile নেই। বাস্তব Project-এ Lockfile Git-এ রাখুন এবং Dependency-র নিরাপত্তা Scan CI-তে চালান।

২৮.৫ Docker: Image, Container ও Dockerfile

Image হলো একটি File System-এর স্তরে স্তরে সাজানো ছবি, সঙ্গে চালানোর নির্দেশ—অপরিবর্তনীয়। **Container** হলো সেই Image থেকে চালু হওয়া একটি বিচ্ছিন্ন Process। এক Image থেকে অনেক Container চলতে পারে। **Dockerfile** Image তৈরির নির্দেশমালা; প্রতিটি নির্দেশ একটি স্তর (Layer), আর যে স্তর বদলায়নি, Docker তা আগের Build থেকে আবার ব্যবহার করে (Layer Cache)।

DeployReady-এর Dockerfile:

Terminal

```
cat Dockerfile
```

Output

```
# syntax=docker/dockerfile:1
# Production-এ Tag-এর বদলে Digest দিয়ে Pin করুন: node:22-alpine@sha256:<digest>
FROM node:22-alpine

ENV NODE_ENV=production
```

```
WORKDIR /app

# প্রথমে যা কম বদলায় (package.json), পরে Code: Layer Cache কাজে লাগে।
# এই Project-এ কোনো Runtime Dependency নেই। থাকলে এখানে:
#   COPY package-lock.json ./  এবং  RUN npm ci --omit=dev
COPY --chown=node:node package.json ./
COPY --chown=node:node main.mjs healthcheck.mjs ./
COPY --chown=node:node src ./src

# Root হিসেবে নয়: Image-এ আগে থেকে থাকা 'node' ব্যবহারকারী।
USER node
EXPOSE 3000

HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
  CMD ["node", "healthcheck.mjs"]

# Exec Form: node নিজেই PID 1, তাই SIGTERM সরাসরি পায়।
# "npm start" ব্যবহার করবেন না; npm Signal ঠিকমতো পৌঁছে দেয় না।
STOPSIGNAL SIGTERM
CMD ["node", "main.mjs"]
```

প্রতিটি সিদ্ধান্তের কারণ:

নির্দেশ	কেন
`FROM node:22-alpine`	ছোট Base Image, নির্দিষ্ট Major Version; Production-এ Digest দিয়ে Pin
`package.json` আগে, Code পরে	Code বদলালেও Dependency-র স্তর Cache থেকে আসে
নির্দিষ্ট File `COPY`	`COPY . .` অপ্রয়োজনীয় ও গোপন File-ও ঢুকিয়ে দিতে পারে
`USER node`	Container-এর ভেতরে কেউ ঢুকলেও Root ক্ষমতা পায় না
`HEALTHCHECK`	Docker নিজেই জানে Container সুস্থ কি না
Exec Form `CMD ["node", ...]`	`node` PID 1, তাই Platform-এর SIGTERM সরাসরি পায়

`npm start` দিয়ে চালালে PID 1 হয় `npm`, এবং `npm` SIGTERM Application-এ পৌঁছে দেয় না বা দেরিতে দেয়—ফলে Graceful Shutdown কখনো ঘটে না, Platform কিছুক্ষণ পরে জোর করে মেরে ফেলে। Alpine-এ Native Module-এর সামঞ্জস্য (musl libc) যাচাই করতে হয়; সমস্যা হলে `node:22-slim` (Debian-ভিত্তিক)।

`.dockerignore` Build-এর সময় কোন File Docker-এ পাঠানো হবে না তা বলে:

Terminal

```
cat .dockerignore
```

Output

```
node_modules
npm-debug.log
.git
.github
.env
.env.*
!.env.example
tests
coverage
*.md
compose.yaml
Dockerfile
```

`node_modules` বাদ যায় কারণ Host-এর (যেমন Windows-এর) Native Module Linux Container-এ কাজ নাও করতে পারে, আর Build Context অকারণে বড় হয়। `.env` ও `.git` বাদ যায় কারণ গোপন মান বা পুরো ইতিহাস Image-এ ঢুকে যেতে পারে—আর Image একবার Registry-তে গেলে তা ফেরানো কঠিন।

Docker-এর `HEALTHCHECK` যে Script চালায়, তা Image-এ আগে থেকে থাকা Node.js দিয়েই লেখা, যাতে `curl` বা `wget` আলাদা করে লাগে না:

healthcheck.mjs · JavaScript চলবে: Node.js

```javascript
// Docker HEALTHCHECK: Container-এর ভেতর থেকে Liveness যাচাই।
// curl বা wget-এর ওপর নির্ভর নয়; Image-এ Node.js তো আছেই।
const port = process.env.PORT ?? '3000';
try {
  const res = await fetch(`http://127.0.0.1:${port}/health/live`,
    { signal: AbortSignal.timeout(2000) });
  process.exit(res.ok ? 0 : 1);
} catch {
  process.exit(1);
}
```

এটি সরাসরি চালানো হয়নি এই Listing থেকে, কারণ এটি একটি চলমান Server চায়; Project-এর Test এটি আসল Server-এর বিপরীতে চালিয়ে দেখে যে চালু Server-এ Exit Code 0 এবং বন্ধ Port-এ 1।

Security-র ভারসাম্য: ছোট Image আক্রমণের সুযোগ কমায়, কিন্তু Image Scanning, হালনাগাদ Base Image এবং Dependency পর্যালোচনার বিকল্প নয়।

২৮.৬ Docker Compose: Local-এ Production-এর অনুকরণ

Compose একটি YAML File-এ এক বা একাধিক Service-এর বর্ণনা দেয়—Build, Port, Environment, Restart নীতি, বন্ধের সময়সীমা। Development-এ ও ছোট Deployment-এ কার্যকর; একাধিক Server-এ ছড়ানো, Autoscaling ও উচ্চ Availability-র জন্য Kubernetes বা Managed Container Platform লাগে।

compose.yaml · YAML — চলবে: Node.js

```
services:
  api:
    build: .
    image: deploy-ready:local
    # শুধু নিজের Computer থেকে পৌঁছানো যাবে, Network-এর অন্যদের থেকে নয়।
    ports:
      - "127.0.0.1:3000:3000"
    # গোপন মান (OPS_TOKEN) .env-এ; .env কখনো Git-এ নয় (.env.example দেখুন)।
    env_file: .env
    environment:
      NODE_ENV: production
      PORT: "3000"
      DRAIN_DELAY_MS: "5000"
      SHUTDOWN_TIMEOUT_MS: "15000"
    # Drain (৫ সে.) + Shutdown Timeout (১৫ সে.) < Grace Period (৩০ সে.)
    stop_grace_period: 30s
    restart: unless-stopped
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
```

তিনটি জিনিস বিশেষভাবে দেখুন। **Port** শুধু `127.0.0.1`-এ: `"3000:3000"` লিখলে Docker সব Network Interface-এ খোলে, আর Docker নিজের Firewall নিয়ম যোগ করে বলে Host-এর Firewall অনেক সময় তা আটকায় না। **Secret** আসে `env_file` থেকে, Compose File-এ নয়। আর **সময়ের হিসাব**: Drain (৫ সেকেন্ড) + Shutdown Timeout (১৫ সেকেন্ড) = ২০ সেকেন্ড, যা `stop_grace_period` (৩০ সেকেন্ড)-এর কম। উল্টো হলে Docker Application-এর Shutdown শেষ হওয়ার আগেই SIGKILL পাঠাবে। `read_only`, `cap_drop` ও `no-new-privileges` Container-এর ভেতরের ক্ষমতা আরও কমায়।

২৮.৭ CI: প্রতিটি পরিবর্তন যাচাই

Continuous Integration মানে প্রতিটি Push ও Pull Request-এ স্বয়ংক্রিয়ভাবে Test, Syntax ও নিরাপত্তা যাচাই, এবং Image Build। কোনো ধাপ ব্যর্থ হলে Merge বা Release আটকে যায়। GitHub Actions-এর Workflow:

ci.yml · YAML অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

# সবচেয়ে কম অনুমতি: Code পড়া ছাড়া কিছু নয়।
permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      # Production Pipeline-এ Tag-এর বদলে পূর্ণ Commit SHA দিয়ে Pin করুন।
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm test
      - run: npm run check

# ...
```

দ্বিতীয় Job Image Build করে, চালায় এবং Smoke Test দিয়ে যাচাই করে। Token প্রতিবার নতুন করে তৈরি হয়, কোথাও সংরক্ষিত নয়:

ci.yml · YAML অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```yaml
# ...

  image:
    needs: test
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - name: Build image (tag = commit SHA)
```

```
        run: docker build -t deploy-ready:${{ github.sha }} .
      - name: Run image and smoke test
        run: |
          export OPS_TOKEN="$(openssl rand -hex 16)"
          docker run -d --name api -p 3000:3000 \
            -e OPS_TOKEN -e GIT_SHA=${{ github.sha }} \
            deploy-ready:${{ github.sha }}
          for i in $(seq 1 20); do
            curl -fs http://127.0.0.1:3000/health/ready && break
            sleep 1
          done
          node scripts/smoke.mjs http://127.0.0.1:3000
      - name: Logs on failure
        if: failure()
        run: docker logs api
```

নিরাপত্তার অভ্যাসগুলো: `permissions: contents: read` —Workflow-এর Token শুধু Code পড়তে পারে; `timeout-minutes` —আটকে থাকা Job চিরকাল চলে না; `concurrency` —একই Branch-এ নতুন Push এলে পুরোনো চালনা বাতিল; `needs: test` —Test পাস না করলে Image-এর কাজ শুরুই হয় না। Third-party Action (`actions/checkout@v4`) Tag দিয়ে উল্লেখ করা—Tag সরানো যায়, তাই Production Pipeline-এ পূর্ণ Commit SHA দিয়ে Pin করুন। মনে রাখবেন, CI সবুজ মানে Production-এর জন্য প্রস্তুত নয়; এটি শুধু বলে যে আপনার জানা যাচাইগুলো পার হয়েছে।

Docker ছাড়াও Deploy File-এর সাধারণ ভুল ধরা যায়। `npm run check` এই Script চালায়:

check-deploy.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
// Docker ছাড়াই Deploy File-এর সাধারণ ভুল ধরা (CI-তে Build-এর আগে)।
import { readFileSync } from 'node:fs';

const read = (file) => readFileSync(new URL(`../${file}`, import.meta.url),
  'utf8');
const problems = [];
const rule = (ok, message) => { if (!ok) problems.push(message); };

// Dockerfile: '\' দিয়ে ভাঙা Line জোড়া লাগিয়ে প্রতিটি Instruction আলাদা।
const steps = read('Dockerfile').replace(/\\\n/g, ' ').split('\n')
  .map((line) => line.trim()).filter((line) => line && !line.startsWith('#'))
  .map((line) => {
    const [op, ...rest] = line.split(/\s+/);
    return { op: op.toUpperCase(), args: rest.join(' ') };
  });
const find = (op) => steps.filter((s) => s.op === op);
const index = (op) => steps.findIndex((s) => s.op === op);

rule(find('FROM').length > 0 && !/:latest\b/.test(find('FROM')[0].args)
  && /:/.test(find('FROM')[0].args), 'FROM-এ নির্দিষ্ট Tag বা Digest');
```

```
rule(find('USER').some((s) => !/^(root|0)$/.test(s.args)),
  'USER: root ছাড়া অন্য ব্যবহারকারী');
rule(index('USER') !== -1 && index('USER') < index('CMD'),
  'USER থাকতে হবে CMD-এর আগে');
rule(find('CMD').at(-1)?.args.startsWith('['), 'CMD Exec Form-এ (JSON Array)');
rule(!/npm/.test(find('CMD').at(-1)?.args ?? ''), 'CMD-এ npm নয়, সরাসরি node');
rule(find('HEALTHCHECK').length === 1, 'একটি HEALTHCHECK');
rule(!steps.some((s) => ['ENV', 'ARG'].includes(s.op)
  && /(SECRET|TOKEN|PASSWORD|KEY)\s*=/i.test(s.args)),
  'ENV বা ARG-এ কোনো গোপন মান নয়');
rule(!find('COPY').some((s) => /^\.\s+\.$/.test(s.args.replace(/--\S+\s*/g, ''))),
  'COPY . . নয়; শুধু প্রয়োজনীয় File');

// ...
```

check-deploy.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const ignore = read('.dockerignore').split('\n').map((l) => l.trim());
for (const entry of ['node_modules', '.git', '.env']) {
  rule(ignore.includes(entry), `.dockerignore-এ ${entry}`);
}

const compose = read('compose.yaml');
rule(/"127\.0\.0\.1:\d+:\d+"/.test(compose), 'Compose Port শুধু 127.0.0.1-এ');
rule(/stop_grace_period:/.test(compose), 'Compose-এ stop_grace_period');
rule(!/OPS_TOKEN:/.test(compose), 'Compose File-এ Token-এর মান নয়');

for (const p of problems) console.log(`FAIL ${p}`);
console.log(`Instruction: ${steps.length}, সমস্যা: ${problems.length}`);
process.exit(problems.length ? 1 : 0);
```

Output

```
Instruction: 11, সমস্যা: 0
```

এটি একটি পূর্ণ Linter নয় (যেমন `hadolint`), কিন্তু এই Project-এর নিয়মগুলো—Non-root, Exec Form, HEALTHCHECK, গোপন মান নয়, `.dockerignore`—প্রতিটি পরিবর্তনে স্বয়ংক্রিয়ভাবে রক্ষা করে।

২৮.৮ CD: Staging, Rollout ও Rollback

Continuous Delivery-তে প্রতিটি সফল Pipeline একটি Release-যোগ্য Artifact তৈরি করে, আর Production-এ যাওয়ার সিদ্ধান্ত মানুষের (একটি Approval)। **Continuous Deployment**-এ সেই সিদ্ধান্তও স্বয়ংক্রিয়। একটি সাধারণ প্রবাহ:

Folder-এর কাঠামো

```
Pull Request -> Test -> Image Build -> Security Scan
  -> Staging Deploy -> Smoke Test -> Approval
  -> Production Rollout (ধাপে ধাপে) -> পর্যবেক্ষণ -> প্রয়োজনে Rollback
```

কৌশল	কীভাবে	সুবিধা	মূল্য
Rolling	Instance-গুলো একে একে নতুন সংস্করণে	অতিরিক্ত Server লাগে না	কিছুক্ষণ দুই সংস্করণ একসঙ্গে; Rollback ধীর
Blue/Green	পুরো নতুন পরিবেশ, তারপর Traffic এক ধাক্কায় বদল	তাৎক্ষণিক Rollback	দ্বিগুণ Infrastructure
Canary	প্রথমে অল্প Traffic (যেমন ৫%) নতুন সংস্করণে, মেপে বাড়ানো	ক্ষতি সীমিত, আসল Traffic-এ যাচাই	Metrics ও স্বয়ংক্রিয় বিশ্লেষণ লাগে

Canary-র সিদ্ধান্ত অনুভূতিতে নয়, সংখ্যায়:

canary-decision.mjs · JavaScript — চলবে: Node.js

```
// Canary: নতুন সংস্করণে অল্প Traffic; পুরোনোর (Baseline) সঙ্গে তুলনা করে
// এগোনো, অপেক্ষা বা ফিরে যাওয়া। সিদ্ধান্ত সংখ্যা দিয়ে, অনুমানে নয়।
function decide(baseline, canary, { minRequests = 500, maxErrorRatio = 1.5,
  maxLatencyMs = 50 } = {}) {
  if (canary.requests < minRequests) return 'wait: যথেষ্ট Traffic নেই';
  const errorRate = (w) => w.errors / w.requests;
  const ratio = errorRate(canary) / Math.max(errorRate(baseline), 0.0001);
  if (ratio > maxErrorRatio) {
    return `rollback: Error ${ratio.toFixed(1)} গুণ`;
  }
  if (canary.p95Ms - baseline.p95Ms > maxLatencyMs) {
    return `rollback: p95 ${canary.p95Ms - baseline.p95Ms} ms বেশি`;
  }
  return 'promote: পরের ধাপে (যেমন ৫% থেকে ২৫%)';
}

const baseline = { requests: 95_000, errors: 95, p95Ms: 180 };
const cases = {
  'প্রথম ৫ মিনিট': { requests: 120, errors: 0, p95Ms: 170 },
  'সুস্থ': { requests: 5_000, errors: 6, p95Ms: 190 },
  'Error বেশি': { requests: 5_000, errors: 40, p95Ms: 185 },
  'ধীর': { requests: 5_000, errors: 4, p95Ms: 320 },
};
for (const [name, canary] of Object.entries(cases)) {
  console.log(`${name}: ${decide(baseline, canary)}`);
}
```

Output

```
প্রথম ৫ মিনিট: wait: যথেষ্ট Traffic নেই
সুস্থ: promote: পরের ধাপে (যেমন ৫% থেকে ২৫%)
Error বেশি: rollback: Error 8.0 গুণ
ধীর: rollback: p95 140 ms বেশি
```

প্রথম ক্ষেত্রে ১২০টি অনুরোধে শূন্য Error কিছুই প্রমাণ করে না—তাই অপেক্ষা। "সুস্থ" Canary-র Error Rate ০.১২%, Baseline-এর ০.১%-এর কাছাকাছি, তাই এগোনো। শেষ দুটিতে হয় Error, নয় দেরি স্পষ্টভাবে খারাপ—ফিরে যাওয়া।

Database Migration-এর সতর্কতা: Application Rollback সহজ, কিন্তু Database-এর পরিবর্তন সহজে ফেরে না। Rollout-এর সময় পুরোনো ও নতুন দুই সংস্করণ একই Database ব্যবহার করে, তাই প্রতিটি Migration পশ্চাৎ-সামঞ্জস্যপূর্ণ হতে হবে। নিয়ম **Expand/Contract**: প্রথমে যোগ (নতুন Column, ঐচ্ছিক), তারপর Code নতুনটি ব্যবহার করে, তারপর পুরোনো Data স্থানান্তর, এবং অনেক পরে—সব পুরোনো সংস্করণ চলে গেলে—পুরোনো Column মোছা।

২৮.৯ Liveness, Readiness ও Graceful Shutdown

দুটি Health Endpoint দুটি ভিন্ন প্রশ্নের উত্তর দেয়:

- **Liveness** (`/health/live`): "Process কি জীবিত?" ব্যর্থ হলে Platform Container Restart করে। তাই এখানে Database যাচাই করবেন না—Database বন্ধ হলে সব Container একসঙ্গে Restart হয়ে অবস্থা আরও খারাপ হবে।
- **Readiness** (`/health/ready`): "এখন কি Traffic নিতে পারি?" ব্যর্থ হলে Load Balancer এই Instance-এ নতুন অনুরোধ পাঠানো বন্ধ করে, কিন্তু Restart করে না। এখানে প্রয়োজনীয় Dependency সময়সীমা দিয়ে যাচাই করা যায়, এবং Shutdown শুরু হলে এটি 503 দেয়।

Server-এর জীবনচক্র:

server.mjs · JavaScript — চলবে: Node.js

```
import http from 'node:http';
import { setTimeout as sleep } from 'node:timers/promises';
import { createApp } from './app.mjs';

// HTTP Server-এর জীবনচক্র: চালু করা এবং সুশৃঙ্খলভাবে বন্ধ করা।
export async function startServer({ config, logger, metrics, store }) {
  const app = createApp({ config, logger, metrics, store });
  const server = http.createServer(app.handle);
  server.keepAliveTimeout = 5_000;
  server.headersTimeout = 10_000;
  server.requestTimeout = 30_000;
  await new Promise((resolve) => server.listen(config.port, resolve));
  const { port } = server.address();
  logger.info('server.started', { port, version: config.version,
    commit: config.commit });
```

```
  let closing;
  // ১) Readiness 503, ২) Load Balancer-এর জন্য অপেক্ষা, ৩) নতুন সংযোগ বন্ধ,
  // ৪) চলমান অনুরোধ শেষ হতে দেওয়া, ৫) সময় পেরোলে জোর করে বন্ধ।
  function shutdown(reason) {
    closing ??= (async () => {
      app.state.draining = true;
      logger.info('shutdown.started', { reason,
        inFlight: app.state.inFlight });
      await sleep(config.drainDelayMs);
      let forced = false;
      const closed = new Promise((resolve) => server.close(resolve));
      server.closeIdleConnections();
      const timer = setTimeout(() => {
        forced = true;
        server.closeAllConnections();
      }, config.shutdownTimeoutMs);
      await closed;
      clearTimeout(timer);
      logger.info('shutdown.complete', { forced });
      return { forced };
    })();
    return closing;
  }

  return { server, port, app, shutdown };
}
```

Shutdown-এর পাঁচ ধাপের কারণ: Platform SIGTERM পাঠানোর মুহূর্তে Load Balancer হয়তো তখনো এই Instance-এ অনুরোধ পাঠাচ্ছে। তাই প্রথমে Readiness 503, তারপর `drainDelayMs` অপেক্ষা যাতে Load Balancer তা লক্ষ করে এই Instance-কে তালিকা থেকে সরায়। এরপর `server.close()` নতুন সংযোগ নেওয়া বন্ধ করে, আর অলস Keep-Alive সংযোগগুলো বন্ধ হয়। চলমান অনুরোধ শেষ হলে `close`-এর Callback আসে। কোনো অনুরোধ আটকে থাকলে Timeout-এর পর জোর করে সব বন্ধ—এবং `forced: true` জানায় যে কিছু অনুরোধ অসমাপ্ত রইল, তাই Exit Code 1। `closing ??=` নিশ্চিত করে যে দ্বিতীয় Signal (অধৈর্য Operator-এর দ্বিতীয় Ctrl+C) একই প্রক্রিয়া আবার শুরু করে না।

`keepAliveTimeout`, `headersTimeout` ও `requestTimeout` সীমা দেয় যাতে একটি ধীর বা দুষ্ট Client সংযোগ চিরকাল ধরে রাখতে না পারে। Load Balancer-এর পেছনে থাকলে `keepAliveTimeout` Load Balancer-এর Idle Timeout-এর চেয়ে বেশি রাখুন, নইলে মাঝে মাঝে "Connection reset" Error দেখা যায়।

২৮.১০ Logs, Metrics ও Traces

পর্যবেক্ষণের (**Observability**) তিনটি সংকেত: **Log**—নির্দিষ্ট ঘটনার বিবরণ; **Metrics**—সময়ের সঙ্গে সংখ্যার প্রবণতা; **Trace**—একটি অনুরোধ একাধিক Service-এর মধ্য দিয়ে কোন পথে গেল। Google SRE-এর চারটি **Golden Signal**: Latency, Traffic, Error, Saturation।

Structured Log: "User rafi logged in" ধরনের মুক্ত বাক্য মানুষ পড়তে পারে, কিন্তু Log-ব্যবস্থা খুঁজতে বা গুনতে পারে না। প্রতি Line-এ একটি JSON Object হলে "শেষ এক ঘণ্টায় কতগুলো `auth.failed`" বা "এই requestId-এর সব Line" এক Query-তে পাওয়া যায়:

logger.mjs · JavaScript — চলবে: Node.js

```
// Structured Log: প্রতি Line একটি JSON Object, যাতে Log-ব্যবস্থা খুঁজতে
// ও গুনতে পারে। গোপন Key-গুলোর মান কখনো লেখা হয় না।
const LEVELS = { debug: 10, info: 20, warn: 30, error: 40 };
const SECRET = /authorization|password|token|secret|cookie/i;

function redact(value) {
  if (Array.isArray(value)) return value.map(redact);
  if (value === null || typeof value !== 'object') return value;
  return Object.fromEntries(Object.entries(value).map(([key, inner]) =>
    [key, SECRET.test(key) ? '[REDACTED]' : redact(inner)]));
}

export function createLogger({ level = 'info', base = {},
  write = (line) => process.stdout.write(`${line}\n`),
  now = () => new Date() } = {}) {
  const min = LEVELS[level];
  const log = (lvl) => (event, fields = {}) => {
    if (LEVELS[lvl] < min) return;
    write(JSON.stringify({ time: now().toISOString(), level: lvl, event,
      ...redact(base), ...redact(fields) }));
  };
  return {
    debug: log('debug'), info: log('info'),
    warn: log('warn'), error: log('error'),
    child: (fields) => createLogger({ level, write, now,
      base: { ...base, ...fields } }),
  };
}
```

structured-log.mjs · JavaScript — চলবে: Node.js

```javascript
import { createLogger } from '../project/deploy-ready/src/logger.mjs';

// স্থির সময়, যাতে আউটপুট প্রতিবার এক।
let tick = 0;
const now = () => new Date(Date.UTC(2026, 9, 18, 10, 0, tick++));
// আসলে প্রতিটি Log একটি JSON Line; এখানে পড়ার সুবিধায় Object আকারে ছাপা।
const write = (line) => console.log(JSON.parse(line));
const logger = createLogger({ level: 'info', now, write,
  base: { service: 'deploy-ready' } });

const request = logger.child({ requestId: 'req-7f3a' });
request.debug('cache.miss', { key: 'jobs' }); // level info: লেখা হবে না
request.info('job.created', { jobId: 42, priority: 'high' });
request.warn('auth.failed', { authorization: 'Bearer s3cr3t', ip: '10.0.0.8' });
logger.error('db.timeout', { query: 'list-jobs', durationMs: 5003 });
```

Output

```
{
  time: '2026-10-18T10:00:00.000Z',
  level: 'info',
  event: 'job.created',
  service: 'deploy-ready',
  requestId: 'req-7f3a',
  jobId: 42,
  priority: 'high'
}
{
  time: '2026-10-18T10:00:01.000Z',
  level: 'warn',
  event: 'auth.failed',
  service: 'deploy-ready',
  requestId: 'req-7f3a',
  authorization: '[REDACTED]',
  ip: '10.0.0.8'
}
{
  time: '2026-10-18T10:00:02.000Z',
  level: 'error',
  event: 'db.timeout',
  service: 'deploy-ready',
  query: 'list-jobs',
  durationMs: 5003
}
```

`debug` Line লেখা হয়নি কারণ Level `info`। `child` Logger প্রতিটি Line-এ `requestId` বসায়, তাই একটি অনুরোধের সব ঘটনা একসঙ্গে খোঁজা যায়। `authorization`-এর মান কখনো লেখা হয় না। Log-এ Password, Token, পূর্ণ কার্ড নম্বর বা অপ্রয়োজনীয় ব্যক্তিগত তথ্য রাখবেন না—Log অনেক মানুষ ও অনেক ব্যবস্থার হাতে যায়, আর দীর্ঘ সময় থাকে।

Metrics: Prometheus একটি বহুল ব্যবহৃত Metrics ব্যবস্থা, যা নির্দিষ্ট বিরতিতে Application-এর `/metrics` Endpoint থেকে একটি সহজ Text Format পড়ে। **Counter** শুধু বাড়ে (মোট অনুরোধ); **Gauge** বাড়ে-কমে (চলমান অনুরোধ); **Histogram** মানগুলোকে Bucket-এ গোনে, যা থেকে p95-এর মতো Percentile হিসাব করা যায়:

metrics.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// Prometheus Text Exposition Format-এ Metrics: Counter ও Histogram।
const BUCKETS = [0.005, 0.01, 0.05, 0.1, 0.5, 1, 5];
const HIST = 'http_request_duration_seconds';

const labelText = (labels) => Object.entries(labels)
  .map(([key, value]) => `${key}="${String(value).replace(/"/g, '\\"')}"`)
  .join(',');

export function createMetrics() {
  const requests = new Map();
  const durations = new Map();
  const gauges = new Map();

// ...
```

metrics.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  return {
    observeRequest({ method, route, status, seconds }) {
      const key = labelText({ method, route, status });
      requests.set(key, (requests.get(key) ?? 0) + 1);
      const hkey = labelText({ method, route });
      const h = durations.get(hkey)
        ?? { counts: BUCKETS.map(() => 0), sum: 0, count: 0 };
      BUCKETS.forEach((le, i) => { if (seconds <= le) h.counts[i] += 1; });
      h.sum += seconds;
      h.count += 1;
      durations.set(hkey, h);
    },
    setGauge(name, value) {
      gauges.set(name, value);
    },
    render() {
      const lines = ['# HELP http_requests_total HTTP অনুরোধের সংখ্যা',
```

```
        '# TYPE http_requests_total counter'];
      for (const [key, n] of requests) {
        lines.push(`http_requests_total{${key}} ${n}`);
      }
      lines.push(`# HELP ${HIST} অনুরোধের সময় (সেকেন্ড)`,
        `# TYPE ${HIST} histogram`);
      for (const [key, h] of durations) {
        BUCKETS.forEach((le, i) => lines.push(
          `${HIST}_bucket{${key},le="${le}"} ${h.counts[i]}`));
        lines.push(`${HIST}_bucket{${key},le="+Inf"} ${h.count}`,
          `${HIST}_sum{${key}} ${h.sum}`,
          `${HIST}_count{${key}} ${h.count}`);
      }
      for (const [name, value] of gauges) {
        lines.push(`# TYPE ${name} gauge`, `${name} ${value}`);
      }
      return `${lines.join('\n')}\n`;
    },
  };
}
```

metrics-format.mjs · JavaScript — চলবে: Node.js

```
import { createMetrics } from '../project/deploy-ready/src/metrics.mjs';

const metrics = createMetrics();
const seen = [
  ['GET', 200, 0.003], ['GET', 200, 0.04], ['GET', 200, 0.3],
  ['POST', 201, 0.02], ['POST', 422, 0.004], ['GET', 500, 1.7],
];
for (const [method, status, seconds] of seen) {
  metrics.observeRequest({ method, route: '/jobs', status, seconds });
}
// শুধু GET-এর Histogram ও সব Counter দেখানো (পুরো আউটপুট দীর্ঘ)।
const text = metrics.render().split('\n')
  .filter((line) => !line.includes('method="POST",route="/jobs",le'))
  .join('\n');
console.log(text.trimEnd());
```

Output

```
# HELP http_requests_total HTTP অনুরোধের সংখ্যা
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/jobs",status="200"} 3
http_requests_total{method="POST",route="/jobs",status="201"} 1
http_requests_total{method="POST",route="/jobs",status="422"} 1
http_requests_total{method="GET",route="/jobs",status="500"} 1
# HELP http_request_duration_seconds অনুরোধের সময় (সেকেন্ড)
# TYPE http_request_duration_seconds histogram
```

```
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.005"} 1
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.01"} 1
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.05"} 2
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.1"} 2
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.5"} 3
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="1"} 3
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="5"} 4
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="+Inf"} 4
http_request_duration_seconds_sum{method="GET",route="/jobs"} 2.043
http_request_duration_seconds_count{method="GET",route="/jobs"} 4
http_request_duration_seconds_sum{method="POST",route="/jobs"} 0.024
http_request_duration_seconds_count{method="POST",route="/jobs"} 2
```

Histogram-এর Bucket **ক্রমযোজিত**: `le="0.5"` মানে ০.৫ সেকেন্ড বা তার কম—চারটি GET-এর তিনটি। Label-এর মান সীমিত রাখুন: Route-এর নাম (`/jobs`), কখনো আসল Path (`/jobs/9381`) বা User ID নয়, নইলে প্রতিটি নতুন মান একটি নতুন Time Series তৈরি করে Metrics-ব্যবস্থার Memory শেষ করে (**High Cardinality**)।

Trace: একটি অনুরোধ Gateway থেকে Booking Service, সেখান থেকে Payment Service ও Database-এ গেলে, প্রতিটি ধাপ একটি **Span**, আর সব Span একটি Trace ID-তে যুক্ত। W3C-র `traceparent` Header এই ID এক Service থেকে অন্যটিতে বয়ে নেয়। OpenTelemetry হলো Trace, Metrics ও Log সংগ্রহের একটি মুক্ত মান, যা প্রায় সব পর্যবেক্ষণ-ব্যবস্থা বোঝে। DeployReady-এর `x-request-id` এর সবচেয়ে সরল রূপ: একটি ID, যা Response Header, Access Log ও Error Log—তিন জায়গায় একই।

২৮.১১ SLI, SLO ও Alerting

- **SLI** (Service Level Indicator): একটি মাপা সংখ্যা, যেমন সফল অনুরোধের অনুপাত, বা ৩০০ ms-এর কমে উত্তর পাওয়া অনুরোধের অনুপাত।
- **SLO** (Service Level Objective): লক্ষ্য, যেমন "৩০ দিনে ৯৯.৯% অনুরোধ সফল"।
- **SLA** (Service Level Agreement): গ্রাহকের সঙ্গে চুক্তি, ভঙ্গ হলে ক্ষতিপূরণ। SLA সাধারণত SLO-এর চেয়ে শিথিল।
- **Error Budget:** ১০০% থেকে SLO বাদ দিলে যা থাকে—৯৯.৯%-এ ০.১%। Budget বাকি থাকলে দ্রুত নতুন Feature ছাড়া যায়; শেষ হলে নির্ভরযোগ্যতার কাজ আগে।

slo-budget.mjs · JavaScript — চলবে: Node.js

```javascript
// SLO: ৩০ দিনে ৯৯.৯% অনুরোধ সফল। Error Budget = বাকি ০.১%।
const slo = 0.999;
const month = { total: 4_200_000, failed: 2_900 };
const lastHour = { total: 6_000, failed: 90 };

const sli = (w) => (w.total - w.failed) / w.total;
const budget = month.total * (1 - slo);
const pct = (x) => `${(x * 100).toFixed(3)}%`;
```

```
console.log('৩০ দিনের SLI:', pct(sli(month)));
console.log('Error Budget:', Math.round(budget), 'টি ব্যর্থ অনুরোধ');
console.log('খরচ হয়েছে:', pct(month.failed / budget));

// Burn Rate: এই হারে চললে Budget কত গুণ দ্রুত শেষ হবে।
const burnRate = (1 - sli(lastHour)) / (1 - slo);
console.log('শেষ ১ ঘণ্টার Error Rate:', pct(1 - sli(lastHour)));
console.log('Burn Rate:', burnRate.toFixed(1));
console.log('Budget শেষ হবে প্রায়', Math.round(30 / burnRate * 24), 'ঘণ্টায়');
console.log(burnRate >= 14.4 ? 'Page: এখনই কাউকে জাগান'
  : burnRate >= 1 ? 'Ticket: কাজের সময়ে দেখুন' : 'ঠিক আছে');
```

Output

```
৩০ দিনের SLI: 99.931%
Error Budget: 4200 টি ব্যর্থ অনুরোধ
খরচ হয়েছে: 69.048%
শেষ ১ ঘণ্টার Error Rate: 1.500%
Burn Rate: 15.0
Budget শেষ হবে প্রায় 48 ঘণ্টায়
Page: এখনই কাউকে জাগান
```

মাসের হিসাবে SLI ৯৯.৯৩১%—লক্ষ্যের ওপরে, তবু Budget-এর ৬৯% খরচ। আসল খবর শেষ ঘণ্টায়: Error Rate ১.৫%, যা অনুমোদিত হারের ১৫ গুণ (**Burn Rate**)। এই হারে পুরো মাসের Budget দুই দিনে শেষ। Google SRE Workbook-এর প্রচলিত নিয়মে এক ঘণ্টার Burn Rate ১৪.৪ বা বেশি হলে সঙ্গে সঙ্গে কাউকে জাগানো (Page), আর ধীর খরচে শুধু একটি Ticket। প্রতিটি একক Error-এ Page করলে Alert-ক্লান্তি আসে—মানুষ Alert উপেক্ষা করতে শেখে, আর আসল বিপদও উপেক্ষিত হয়। প্রতিটি Alert-এর সঙ্গে একটি Runbook থাকা উচিত: কী দেখতে হবে, কী করতে হবে।

২৮.১২ Secret, Supply Chain ও Infrastructure-এর নিরাপত্তা

- **Secret:** Code, Image, Frontend Bundle বা Log-এ নয়; Secret Store-এ, ন্যূনতম অনুমতিতে, নিয়মিত Rotation-সহ। স্বল্পমেয়াদি Credential ও Workload Identity (Platform নিজেই Service-কে পরিচয় দেয়) সবচেয়ে ভালো।
- **Supply Chain:** Lockfile, `npm ci`, Dependency Scan, Image Scan, এবং **SBOM** (Software Bill of Materials—Image-এ ঠিক কোন Package আছে তার তালিকা)। CI-এর Third-party Action Commit SHA দিয়ে Pin।
- **Network:** TLS সর্বত্র; Database শুধু Application-এর Network থেকে পৌঁছানো যায়; `/metrics` শুধু অভ্যন্তরীণ Network-এ।
- **Least Privilege:** CI-এর Token, Container-এর ব্যবহারকারী, Cloud-এর Role—প্রত্যেকে শুধু যা দরকার।

২৮.১৩ Backup, Disaster Recovery ও Incident Response

RPO (Recovery Point Objective): সর্বোচ্চ কতটা তথ্য হারানো সহনীয়—প্রতি ৬ ঘণ্টায় Backup হলে সবচেয়ে খারাপ ক্ষেত্রে ৬ ঘণ্টার তথ্য। **RTO** (Recovery Time Objective): কতক্ষণে সেবা ফিরতে হবে। দুটিই ব্যবসার সিদ্ধান্ত, এবং ছোট করতে খরচ বাড়ে (Continuous Backup, একাধিক Region)। অধ্যায় ২৬-এর শিক্ষা আবার: **যে Backup থেকে Restore করে দেখা হয়নি, তা Backup নয়।** নিয়মিত Restore Drill করুন এবং সময় মাপুন—সেটিই আপনার আসল RTO।

Incident Response-এর ধাপ: শনাক্ত (Alert বা গ্রাহক), দায়িত্ব নেওয়া (একজন Incident Lead), ক্ষতি থামানো (Rollback প্রায়ই সবচেয়ে দ্রুত), যোগাযোগ (Status Page, গ্রাহক), পুনরুদ্ধার, এবং শেষে **Postmortem**—দোষারোপহীন বিশ্লেষণ: কী ঘটেছিল, কেন ধরা পড়েনি, কী বদলাবে যাতে আবার না ঘটে।

২৮.১৪ Project: DeployReady Travel Operations API

DeployReady একটি ছোট API, যেখানে Operations দল কাজ (Job) তৈরি ও দেখে—যেমন "হোটেল Booking যাচাই"। Domain ইচ্ছাকৃতভাবে সরল; Project-এর আসল বিষয় তার চারপাশ: Configuration, Log, Metrics, Health, Shutdown, Container ও CI। Node.js-এর অন্তর্নির্মিত `http` দিয়ে লেখা, কোনো npm Package নেই। **এটি আসল Booking বা Payment Backend নয়;** Data Memory-তে থাকে।

File-এর কাঠামো

Folder-এর কাঠামো

```
deploy-ready/
├── package.json          Script: start, test, check, smoke
├── README.md             চালানো, Environment Variable, সীমাবদ্ধতা
├── main.mjs              Config যাচাই, Signal ধরা, Server চালু
├── healthcheck.mjs       Docker HEALTHCHECK
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .env.example          .env-এর নমুনা (আসল .env Git-এ নয়)
├── .github/workflows/ci.yml
├── scripts/
│   ├── check-deploy.mjs Deploy File-এর স্থির যাচাই
│   └── smoke.mjs         Deploy-এর পরের Smoke Test
├── src/
│   ├── config.mjs        Configuration ও Fail Fast
│   ├── logger.mjs        Structured Log
│   ├── metrics.mjs       Prometheus Format
│   ├── jobs.mjs          Domain ও Memory Store
│   ├── app.mjs           Route, Request ID, Error চুক্তি
│   └── server.mjs        জীবনচক্র ও Graceful Shutdown
└── tests/                ১৪টি Test
```

API

Method	Path	Auth	ফল
GET	`/health/live`	না	200 যতক্ষণ Process চলছে
GET	`/health/ready`	না	200; Shutdown বা Dependency সমস্যায় 503
GET	`/version`	না	Version ও Commit
GET	`/metrics`	Token	Prometheus Text Format
GET	`/api/jobs`	না	Job-এর তালিকা
POST	`/api/jobs`	Token	201; ভুল Input-এ 422

src/jobs.mjs

Domain ও একটি Memory Storeı Store-এর Method-গুলো `async`, যাতে পরে Database বসালে বাকি Code বদলাতে না হয়:

jobs.mjs · JavaScript — চলবে: Node.js

```
// Domain: Travel Operations-এর কাজ (যেমন "হোটেল Booking যাচাই")।
// Storage Interface async, যাতে পরে Database বসানো যায় (অধ্যায় ২৬)।
export const PRIORITIES = ['low', 'normal', 'high'];

export class ValidationError extends Error {
  constructor(details) {
    super('Input যাচাইয়ে ব্যর্থ');
    this.details = details;
  }
}

export function validateJob(input) {
  const details = [];
  const title = typeof input?.title === 'string' ? input.title.trim() : '';
  const priority = input?.priority ?? 'normal';
  if (title.length < 3 || title.length > 120) {
    details.push('title: ৩ থেকে ১২০ অক্ষর');
  }
  if (!PRIORITIES.includes(priority)) {
    details.push(`priority: ${PRIORITIES.join(', ')}`);
  }
  if (details.length) throw new ValidationError(details);
  return { title, priority };
}

export function createMemoryJobStore({ now = () => new Date() } = {}) {
  const jobs = [];
  return {
```

```
    async list() {
      return jobs.map((job) => ({ ...job }));
    },
    async create(input) {
      const job = { id: jobs.length + 1, ...validateJob(input),
        status: 'open', createdAt: now().toISOString() };
      jobs.push(job);
      return { ...job };
    },
    async ping() {
      return true;
    },
  };
}
```

src/app.mjs

প্রথম অংশ: নিজস্ব HTTP Error, Route-এর তালিকা এবং JSON পড়া—Content-Type যাচাই (415), আকারের সীমা (413) ও ভাঙা JSON (400):

app.mjs · JavaScript অংশ ১ / ৫ (Part 1 of 5) চলবে: Node.js

```
import { randomUUID } from 'node:crypto';
import { ValidationError } from './jobs.mjs';

class HttpError extends Error {
  constructor(status, code, message) {
    super(message);
    this.status = status;
    this.code = code;
  }
}

const ROUTES = {
  'GET /health/live': 'live',
  'GET /health/ready': 'ready',
  'GET /version': 'version',
  'GET /metrics': 'metrics',
  'GET /api/jobs': 'listJobs',
  'POST /api/jobs': 'createJob',
};

async function readJson(req, limit = 8 * 1024) {
  if (!/^application\/json\b/.test(req.headers['content-type'] ?? '')) {
    throw new HttpError(415, 'UNSUPPORTED_MEDIA_TYPE', 'JSON পাঠান');
  }
  let size = 0;
  const chunks = [];
```

```
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new HttpError(413, 'TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new HttpError(400, 'BAD_JSON', 'অবৈধ JSON');
  }
}

// ...
```

Dependency যাচাইয়ের সময়সীমা, এবং Token যাচাই। Development-এ `OPS_TOKEN` না থাকলে যাচাই হয় না; Production-এ Config নিজেই Token বাধ্যতামূলক করে:

app.mjs · JavaScript অংশ ২ / ৫ (Part 2 of 5) চলবে: Node.js

```
// ...

function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((resolve, reject) => {
    timer = setTimeout(() => reject(new Error('timeout')), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

export function createApp({ config, logger, metrics, store,
  readyTimeoutMs = 1000 }) {
  const state = { draining: false, inFlight: 0 };

  const requireOps = (req) => {
    if (!config.opsToken) return; // শুধু development-এ Token ছাড়া
    if (req.headers.authorization !== `Bearer ${config.opsToken}`) {
      throw new HttpError(401, 'UNAUTHORIZED', 'Token প্রয়োজন');
    }
  };

// ...
```

প্রতিটি Route-এর কাজ। Readiness-এ Dependency (`store.ping()`) এক সেকেন্ডের বেশি অপেক্ষা করলে 503:

app.mjs · JavaScript অংশ ৩ / ৫ (Part 3 of 5) চলবে: Node.js

```
// ...

  const handlers = {
```

```
    live: async () => [200, { status: 'live' }],
    ready: async () => {
      if (state.draining) return [503, { status: 'draining' }];
      try {
        await withTimeout(store.ping(), readyTimeoutMs);
        return [200, { status: 'ready' }];
      } catch {
        return [503, { status: 'dependency-unavailable' }];
      }
    },
    version: async () => [200, { version: config.version,
      commit: config.commit }],
    metrics: async (req) => {
      requireOps(req);
      metrics.setGauge('http_in_flight_requests', state.inFlight);
      return [200, metrics.render(), 'text/plain; version=0.0.4'];
    },
    listJobs: async () => [200, { data: await store.list() }],
    createJob: async (req) => {
      requireOps(req);
      return [201, { data: await store.create(await readJson(req)) }];
    },
  };

// ...
```

প্রতিটি অনুরোধের সাধারণ কাজ: Request ID (Client-এর পাঠানোটি নিরাপদ হলে তা-ই, নইলে নতুন), Response শেষ হলে Access Log ও Metrics:

app.mjs · JavaScript অংশ ৪ / ৫ (Part 4 of 5) চলবে: Node.js

```
// ...

  async function handle(req, res) {
    const started = process.hrtime.bigint();
    const incoming = req.headers['x-request-id'];
    const requestId = /^[\w-]{8,64}$/.test(incoming ?? '')
      ? incoming : randomUUID();
    const { pathname } = new URL(req.url, 'http://localhost');
    const name = ROUTES[`${req.method} ${pathname}`];
    // Metrics-এর Label-এ আসল Path নয়, Route-এর নাম: সীমিত সংখ্যক মান।
    const route = name ? pathname : 'unmatched';
    state.inFlight += 1;
    res.setHeader('x-request-id', requestId);
    res.on('finish', () => {
      state.inFlight -= 1;
      const seconds = Number(process.hrtime.bigint() - started) / 1e9;
      metrics.observeRequest({ method: req.method, route,
        status: res.statusCode, seconds });
```

```
      logger.info('http.request', { requestId, method: req.method, route,
        status: res.statusCode, durationMs: Math.round(seconds * 1000) });
    });
    try {
      if (!name) throw new HttpError(404, 'NOT_FOUND', 'Route পাওয়া যায়নি');
      const [status, body, type] = await handlers[name](req);
      send(res, status, body, type);
    } catch (error) {
      sendError(res, error, requestId);
    }
  }

// ...
```

Error চুক্তি: যাচাইয়ের ভুলে 422 ও কারণের তালিকা; জানা HTTP Error-এ তার নিজের Status; আর অপ্রত্যাশিত Error-এ Client পায় শুধু একটি সাধারণ বার্তা ও Request ID, পুরো Stack যায় Log-এ:

app.mjs · JavaScript অংশ ৫ / ৫ (Part 5 of 5) চলবে: Node.js

```
// ...

  function sendError(res, error, requestId) {
    if (error instanceof ValidationError) {
      return send(res, 422, { error: { code: 'VALIDATION',
        message: error.message, details: error.details } });
    }
    if (error instanceof HttpError) {
      return send(res, error.status, { error: { code: error.code,
        message: error.message } });
    }
    logger.error('http.unhandled', { requestId, message: error.message,
      stack: error.stack });
    return send(res, 500, { error: { code: 'INTERNAL',
      message: 'Server-এ সমস্যা', requestId } });
  }

  return { handle, state };
}

function send(res, status, body, type = 'application/json; charset=utf-8') {
  res.writeHead(status, { 'Content-Type': type });
  res.end(typeof body === 'string' ? body : JSON.stringify(body));
}
```

গ্রাহক যখন বলেন "Error এসেছিল", তিনি Request ID দিতে পারেন, আর Operator সেই ID দিয়ে Log-এ পুরো ঘটনা খুঁজে পান—Stack Trace গ্রাহকের Screen-এ না দেখিয়েই।

main.mjs

main.mjs · JavaScript চলবে: Node.js

```
import { loadConfig, describeConfig } from './src/config.mjs';
import { createLogger } from './src/logger.mjs';
import { createMetrics } from './src/metrics.mjs';
import { createMemoryJobStore } from './src/jobs.mjs';
import { startServer } from './src/server.mjs';

let config;
try {
  config = loadConfig();
} catch (error) {
  console.error(error.message);
  process.exit(1); // ভুল Config নিয়ে কখনো চালু নয়
}

const logger = createLogger({ level: config.logLevel,
  base: { service: 'deploy-ready', version: config.version } });
logger.info('config.loaded', describeConfig(config));
const { shutdown } = await startServer({ config, logger,
  metrics: createMetrics(), store: createMemoryJobStore() });

// Container Platform বন্ধ করার আগে SIGTERM পাঠায়; Ctrl+C পাঠায় SIGINT।
for (const signal of ['SIGTERM', 'SIGINT']) {
  process.once(signal, async () => {
    const { forced } = await shutdown(signal);
    process.exit(forced ? 1 : 0);
  });
}
process.on('unhandledRejection', async (reason) => {
  logger.error('process.unhandledRejection', { message: String(reason) });
  await shutdown('unhandledRejection');
  process.exit(1);
});
```

`main.mjs` এই Listing থেকে চালানো হয়নি, কারণ এটি থামে না (একটি Server); Project-এর Test একে ভুল Config দিয়ে চালিয়ে দেখে যে Exit Code 1 এবং stdout ফাঁকা, আর QA-র Deploy যাচাই একে Production Mode-এ চালিয়ে Smoke Test করে।

scripts/smoke.mjs

Deploy-এর পরে, Traffic দেওয়ার আগে, পাঁচটি মৌলিক যাচাই। একটিও ব্যর্থ হলে Exit Code 1, যাতে Pipeline Rollback করতে পারে:

smoke.mjs · JavaScript চলবে: Node.js

```
// Deploy-এর পরে Smoke Test: node scripts/smoke.mjs https://staging.example.com
```

```
// কোনো ধাপ ব্যর্থ হলে Exit Code 1, যাতে Pipeline থেমে Rollback করতে পারে।
const base = process.argv[2] ?? 'http://127.0.0.1:3000';
const token = process.env.OPS_TOKEN;
const auth = token ? { Authorization: `Bearer ${token}` } : {};

const checks = [
  ['liveness', () => fetch(`${base}/health/live`), 200],
  ['readiness', () => fetch(`${base}/health/ready`), 200],
  ['version', () => fetch(`${base}/version`), 200],
  ['create job', () => fetch(`${base}/api/jobs`, { method: 'POST',
    headers: { 'Content-Type': 'application/json', ...auth },
    body: JSON.stringify({ title: 'Smoke test job', priority: 'low' }) }), 201],
  ['list jobs', () => fetch(`${base}/api/jobs`), 200],
];

let failed = 0;
for (const [name, run, expected] of checks) {
  try {
    const res = await run();
    const ok = res.status === expected;
    if (!ok) failed += 1;
    console.log(`${ok ? 'PASS' : 'FAIL'} ${name}: ${res.status}`);
  } catch (error) {
    failed += 1;
    console.log(`FAIL ${name}: ${error.message}`);
  }
}
process.exit(failed ? 1 : 0);
```

Test: Graceful Shutdown

Shutdown-এর আচরণ শুধু পড়ে বিশ্বাস করা কঠিন, তাই Test একটি "ধীর Upload" তৈরি করে—Header ও Body-র কিছু অংশ পাঠিয়ে থেমে থাকা একটি অনুরোধ—এবং তার মাঝখানে Shutdown শুরু করে:

shutdown.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Drain: Readiness 503, চলমান অনুরোধ সফলভাবে শেষ', async () => {
  const app = await start({ env: { DRAIN_DELAY_MS: '200' } });
  const upload = slowUpload(app.port);
  await new Promise((resolve) => setTimeout(resolve, 50));
  const done = app.shutdown('SIGTERM');
  const ready = await call(app.base, '/health/ready');
  assert.deepEqual([ready.status, ready.body], [503, { status: 'draining' }]);
  upload.finish();
  assert.equal(await upload.response, 201);
  assert.deepEqual(await done, { forced: false });
  assert.equal(app.shutdown('again'), done); // দ্বিতীয় Signal-এ একই কাজ
```

```
  const events = app.lines.map((l) => l.event);
  assert.ok(events.indexOf('shutdown.started')
    < events.indexOf('shutdown.complete'));
});

test('Timeout: শেষ না হওয়া অনুরোধ জোর করে বন্ধ', async () => {
  const app = await start({ env: { SHUTDOWN_TIMEOUT_MS: '150' } });
  const upload = slowUpload(app.port);
  upload.response.catch(() => {}); // সংযোগ কাটা প্রত্যাশিত
  await new Promise((resolve) => setTimeout(resolve, 50));
  const started = Date.now();
  assert.deepEqual(await app.shutdown('SIGTERM'), { forced: true });
  assert.ok(Date.now() - started >= 140);
  await assert.rejects(upload.response);
});
```

প্রথম Test প্রমাণ করে: Shutdown শুরুর পর Readiness 503, অথচ চলমান Upload সফলভাবে 201 পায় এবং Shutdown `forced: false` দিয়ে শেষ হয়। দ্বিতীয়টি প্রমাণ করে: Upload কখনো শেষ না হলে ১৫০ ms পর সংযোগ জোর করে বন্ধ এবং `forced: true`। Test লেখার সময় একটি শিক্ষণীয় ভুল ধরা পড়েছিল: Health Check Script `spawnSync` দিয়ে চালালে Test নিজেই ব্যর্থ হতো, কারণ Synchronous Spawn একই Process-এর Event Loop আটকে রাখে, আর সেই Process-এর Server উত্তর দিতে পারে না। সমাধান: Asynchronous `spawn`।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd deploy-ready
npm test
npm run check
npm start
```

`npm test` ১৪টি Test চালায়: `config` ৪ (Default, সব ভুল একসঙ্গে, Production-এর Token, ভুল Config-এ Process-এর Exit Code 1), `observability` ২ (Log-এর গঠন ও লুকানো মান, Prometheus Format), `app` ৫ (Health ও Request ID, Input-এর প্রতিটি ভুল, Metrics-এর অনুমতি ও Label, অপ্রত্যাশিত Error, Dependency-র Timeout), এবং `shutdown` ৩ (Drain, জোর করে বন্ধ, HEALTHCHECK Script):

Output

```
# tests 14
# pass 14
# fail 0
```

QA-তে আরও যা চালানো হয়েছে: `compose.yaml` ও `ci.yml` দুটি YAML Parser (`yaml` 2.8.3 ও `js-yaml` 3.14.2) দিয়ে Parse—দুটির ফল হুবহু এক; Compose-এর Port, Secret, সময়ের হিসাব ও Hardening; CI-এর অনুমতি, Job-এর নির্ভরতা ও Timeout, এবং Workflow-এর ডাকা প্রতিটি npm Script ও File-এর অস্তিত্ব। এরপর `main.mjs`

Production Mode-এ চালু করে Smoke Test—পাঁচটিই পাস, আর Token ছাড়া চালালে Smoke Test ঠিকভাবে ব্যর্থ (Exit Code 1); Startup Log-এ Token-এর বদলে `[REDACTED]`। **চালানো হয়নি:** `docker build`, `docker compose up` (Docker Daemon চালু ছিল না) এবং GitHub-এ Workflow।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Process চালুর সঙ্গে সঙ্গে Exit Code 1	Configuration ভুল	stderr-এ "Configuration ভুল"-এর তালিকা
Container বারবার Restart	Liveness ব্যর্থ, বা Process Crash	`docker logs`, HEALTHCHECK-এর Port
Deploy-এর সময় কিছু অনুরোধে 502	Readiness-এর আগেই Traffic, বা Drain নেই	`DRAIN_DELAY_MS`, Readiness Probe
Container বন্ধ হতে ঠিক ১০ বা ৩০ সেকেন্ড লাগে, তারপর হঠাৎ	SIGTERM Application-এ পৌঁছায় না	Exec Form CMD, `npm start` নয়
`/metrics`-এ 401	Token নেই বা ভুল	`Authorization: Bearer <OPS_TOKEN>`
Metrics-ব্যবস্থার Memory বাড়ছে	Label-এ আসল Path বা ID	Route-এর নাম Label-এ
Windows-এ বানানো Image Linux-এ চলে না	Host-এর `node_modules` কপি	`.dockerignore`, Image-এ `npm ci`

২৮.১৫ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
`.env` বা Private Key Git-এ	ইতিহাসে চিরকাল থেকে যায়	`.gitignore`, Secret Store, ফাঁস হলে Rotation
শুধু `latest` Tag	কোনটি চলছে, কোনটিতে ফিরব—অজানা	Commit SHA Tag ও Image Digest
Liveness-এ Database যাচাই	Database বন্ধ হলে সব Container Restart	Liveness সরল, Dependency Readiness-এ
Graceful Shutdown নেই	প্রতিটি Deploy-এ কিছু অনুরোধ কাটে	Drain, `server.close`, Timeout
`CMD npm start`	SIGTERM পৌঁছায় না	`CMD ["node", "main.mjs"]`
Root হিসেবে Container	আক্রমণকারী বেশি ক্ষমতা পায়	`USER node`

ভুল	কেন সমস্যা	সমাধান
শুধু HTTP 200 দেখে "সব ঠিক"	ধীরগতি বা আংশিক ব্যর্থতা চোখে পড়ে না	Latency Histogram, Error Rate, SLO
প্রতিটি Error-এ Page	Alert-ক্লান্তি	Burn Rate-ভিত্তিক Alert, Runbook
Memory-তে Session নিয়ে একাধিক Replica	Instance বদলালে ব্যবহারকারী Logout	Session Store (Database বা Redis)
Backup আছে, Restore কখনো চেষ্টা হয়নি	দরকারের দিন অকেজো	নিয়মিত Restore Drill

২৮.১৬ Debugging নির্দেশিকা

১. **প্রথমে সংস্করণ:** `/version` থেকে কোন Commit চলছে নিশ্চিত হন; সমস্যা কি নতুন Deploy-এর পরে শুরু?
২. **Request ID ধরে খুঁজুন:** গ্রাহক বা Response Header থেকে ID নিয়ে Log-এ সেই ID-র সব Line।
৩. **Metrics-এ প্রবণতা:** Error Rate ও p95 কখন থেকে বেড়েছে; কোন Route-এ; Traffic কি স্বাভাবিকের চেয়ে বেশি (Saturation)?
৪. **Health আলাদা করে দেখুন:** Live 200 কিন্তু Ready 503 মানে Process ঠিক আছে, Dependency বা Drain-এ সমস্যা।
৫. **Container-এ:** `docker ps` (Health অবস্থা), `docker logs`, `docker inspect` (Exit Code, OOMKilled)।
৬. **Local-এ পুনরুৎপাদন:** একই Image Digest ও একই Environment Variable (গোপন মান ছাড়া) দিয়ে চালান।
৭. **সন্দেহ হলে Rollback, পরে তদন্ত:** গ্রাহকের ক্ষতি থামানো আগে; কারণ খোঁজা পরে, ঠান্ডা মাথায়।

২৮.১৭ পেশাদার অভ্যাস

- **একবার Build, সবখানে Deploy:** Staging ও Production-এ একই Image Digest।
- **Configuration শুরুতেই যাচাই, গোপন মান কখনো Log-এ নয়।**
- **প্রতিটি Release-এর পরিচয়:** Commit SHA Log, Metrics ও `/version`-এ।
- **Health দুই ভাগে:** Liveness সরল, Readiness সত্যিকারের প্রস্তুতি।
- **Shutdown-এর সময়ের হিসাব লিখে রাখুন:** Drain + Timeout < Grace Period।
- **SLO আগে, Alert পরে:** ব্যবহারকারীর অভিজ্ঞতা মাপুন, CPU নয়।
- **প্রতিটি Alert-এর Runbook, প্রতিটি Incident-এর Postmortem।**
- **যা যাচাই হয়নি, তা স্পষ্ট লিখুন:** এই অধ্যায়ের README যেমন বলে Docker Build চালানো হয়নি।

২৮.১৮ অনুশীলনী

প্রাথমিক স্তর

১. Development ও Production পরিবেশের পাঁচটি পার্থক্য লিখুন, প্রতিটির জন্য একটি করে Production-এর ব্যবস্থা।

২. `PORT=abc` হলে Server-এর চালু হওয়া কেন ভুল? DeployReady এ ক্ষেত্রে কী করে?

৩. Image ও Container-এর পার্থক্য লিখুন। Dockerfile-এ `package.json` কেন Code-এর আগে কপি করা হয়?

৪. `.dockerignore`-এ `node_modules`, `.env` ও `.git` রাখার কারণ লিখুন।

মধ্যম স্তর

৫. তিনটি পরিস্থিতিতে Liveness ও Readiness কী উত্তর দেবে: Database বন্ধ; Shutdown শুরু; Event Loop পুরো আটকে গেছে।

৬. `node:test` দিয়ে লিখুন: ফাঁকা, শুধু Space-এর ও দুই অক্ষরের `title`-এ 422 এবং কোনো Job তৈরি হয় না; `title` না দিলেও 422।

৭. একটি Log File থেকে নির্দিষ্ট Request ID-র সব ঘটনা ক্রমে বের করার Function লিখুন; ভাঙা Line উপেক্ষা করুন।

৮. Rolling, Blue/Green ও Canary তুলনা করুন: কোনটি কখন বেছে নেবেন?

চ্যালেঞ্জ স্তর

৯. ৪,০০০ অনুরোধের ৩,৯৮০টি সফল। Success Rate, ৯৯.৯% SLO-তে অনুমোদিত ব্যর্থতা, Budget-এর খরচ ও Burn Rate হিসাব করুন।

১০. DeployReady-তে PostgreSQL যোগ করার পরিকল্পনা লিখুন: Store, Migration, Readiness, Secret, Test ও Rollback।

১১. একটি Readiness যাচাই লিখুন যা Dependency-কে সময়সীমা দেয় এবং ফল দুই সেকেন্ড Cache করে।

১২. এই অধ্যায়ের CI Workflow-কে আরও শক্ত করার অন্তত পাঁচটি প্রস্তাব দিন।

২৮.১৯ অনুশীলনীর সমাধান

সমাধান ১ — (১) Configuration: Local-এ `.env`; Production-এ Platform-এর Environment ও Secret Store, শুরুতেই যাচাই। (২) Process: Terminal; Production-এ Container Platform, HEALTHCHECK ও স্বয়ংক্রিয় Restart। (৩) Data: Memory; Production-এ টেকসই Database, Backup ও Restore Drill। (৪) Network: `localhost`; Production-এ TLS, Load Balancer, শুধু প্রয়োজনীয় Port খোলা। (৫) ব্যর্থতা জানা: Developer চোখে দেখেন; Production-এ Structured Log, Metrics, SLO-ভিত্তিক Alert ও Runbook।

সমাধান ২ — `Number('abc')` হলো `NaN`; Node.js-এ `listen(NaN)` একটি এলোমেলো খালি Port বেছে নিতে পারে, অথবা অন্য অপ্রত্যাশিত আচরণ—Server "চালু", কিন্তু Load Balancer যে Port-এ পাঠায় সেখানে কেউ নেই, এবং কেউ কারণ বোঝে না। DeployReady-এর `loadConfig` `PORT`-কে শুধু অঙ্কের String হিসেবে ও ০–৬৫৫৩৫ সীমায় গ্রহণ করে; নইলে সব ভুলের তালিকা stderr-এ লিখে Exit Code 1 (উদাহরণ ০১ ও `config.test.mjs`-এর শেষ Test)। Platform তখন Deploy ব্যর্থ দেখায় এবং পুরোনো সংস্করণ চলতে থাকে।

সমাধান ৩ — Image একটি অপরিবর্তনীয় Template (স্তরে সাজানো File System ও চালানোর নির্দেশ); Container তার একটি চলমান Instance, নিজস্ব Process ও লেখার স্তরসহ। Docker প্রতিটি নির্দেশের স্তর Cache করে, আর কোনো স্তর বদলালে তার পরের সব আবার তৈরি হয়। `package.json` (ও Lockfile) কম বদলায়, Code প্রায় প্রতিবার; তাই আগে `package.json` ও `npm ci`, পরে Code কপি করলে Dependency Install-এর ধীর স্তর Cache থেকে আসে।

সমাধান ৪ — `node_modules`: Host-এর Native Module (যেমন Windows-এ Build করা) Linux Container-এ চলে না, আর Build Context বিশাল হয়; Image-এ `npm ci` করে নতুন করে Install করুন। `.env`: গোপন মান Image-এ ঢুকে যায়, আর Image Registry-তে গেলে যে-কেউ Pull করে পড়তে পারে। `.git`: পুরো ইতিহাস, পুরোনো Commit-এ ভুলে রাখা Secret-সহ, Image-এ চলে যায়, এবং আকার বাড়ে।

সমাধান ৫

পরিস্থিতি	Liveness	Readiness	কেন
Database বন্ধ	200	503	Restart করে লাভ নেই; শুধু Traffic সরানো
Shutdown শুরু	200	503	চলমান কাজ শেষ হোক, নতুন না আসুক
Event Loop আটকে	উত্তর নেই (Timeout)	উত্তর নেই	Process অকেজো; Liveness ব্যর্থ, তাই Restart

তৃতীয় ক্ষেত্রেই Liveness-এর আসল মূল্য: একটি অসীম Loop বা Synchronous ভারী কাজে Process জীবিত কিন্তু অকেজো, আর শুধু Restart-ই তাকে ফেরাতে পারে।

সমাধান ৬

empty-title.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { start, call, json, TOKEN }
  from '../../project/deploy-ready/tests/helpers.mjs';

test('ফাঁকা বা শুধু Space-এর title: 422 এবং কারণ', async (t) => {
  const app = await start();
  t.after(() => app.shutdown('test'));
```

```
  for (const title of ['', '   ', 'ab']) {
    const res = await call(app.base, '/api/jobs', { method: 'POST',
      token: TOKEN, ...json({ title }) });
    assert.equal(res.status, 422);
    assert.deepEqual(res.body.error.details, ['title: ৩ থেকে ১২০ অক্ষর']);
  }
  const list = await call(app.base, '/api/jobs');
  assert.deepEqual(list.body.data, []);
});

test('title না দিলেও 422, Server ভাঙে না', async (t) => {
  const app = await start();
  t.after(() => app.shutdown('test'));
  const res = await call(app.base, '/api/jobs', { method: 'POST',
    token: TOKEN, ...json({ priority: 'low' }) });
  assert.equal(res.status, 422);
});
```

Output

```
ok 1 - ফাঁকা বা শুধু Space-এর title: 422 এবং কারণ
ok 2 - title না দিলেও 422, Server ভাঙে না
# tests 2
# pass 2
# fail 0
```

শুধু Status নয়, কারণের তালিকা এবং "কোনো Job তৈরি হয়নি"—দুটিই যাচাই করা হয়। Test Project-এর `helpers.mjs` আবার ব্যবহার করে, তাই আসল Server Port 0-তে চালু হয়।

সমাধান ৭

trace-request.mjs · JavaScript — চলবে: Node.js

```
// একজন গ্রাহক বলছেন "Error এসেছিল, Request ID: req-b2"। Log থেকে খুঁজে বের করা।
// নিচের Text একটি Log File-এর মতো: প্রতি Line একটি JSON, একটি Line ভাঙা।
const t = (ms) => `2026-10-18T09:00:00.0${ms}Z`;
const logText = [
  { time: t(10), level: 'info', event: 'http.request', requestId: 'req-a1',
    status: 200, durationMs: 12 },
  { time: t(20), level: 'warn', event: 'db.slow', requestId: 'req-b2',
    durationMs: 4100 },
  { time: t(30), level: 'error', event: 'http.unhandled',
    requestId: 'req-b2', message: 'pool exhausted' },
  'not json at all',
  { time: t(40), level: 'info', event: 'http.request', requestId: 'req-b2',
    status: 500, durationMs: 4210 },
].map((e) => (typeof e === 'string' ? e : JSON.stringify(e))).join('\n');
```

```
function trace(text, requestId) {
  return text.split('\n').flatMap((line) => {
    try {
      const entry = JSON.parse(line);
      return entry.requestId === requestId ? [entry] : [];
    } catch {
      return []; // Structured নয় এমন Line বাদ
    }
  });
}

for (const entry of trace(logText, 'req-b2')) {
  const extra = entry.message ?? (entry.status
    ? `${entry.status} in ${entry.durationMs} ms` : `${entry.durationMs} ms`);
  console.log(`${entry.time.slice(11, 23)} ${entry.level.padEnd(5)}`,
    `${entry.event}: ${extra}`);
}
```

Output

```
09:00:00.020 warn  db.slow: 4100 ms
09:00:00.030 error http.unhandled: pool exhausted
09:00:00.040 info  http.request: 500 in 4210 ms
```

তিনটি Line মিলে গল্পটি স্পষ্ট: Database ধীর হলো, Connection Pool শেষ হলো, এবং অনুরোধটি চার সেকেন্ডের বেশি পরে 500 দিল। Request ID ছাড়া এই তিনটি Line হাজারো Line-এর মধ্যে আলাদা করা প্রায় অসম্ভব।

সমাধান ৮ — **Rolling:** অতিরিক্ত Infrastructure নেই, ছোট ও পশ্চাৎ-সামঞ্জস্যপূর্ণ পরিবর্তনে উপযুক্ত; কিন্তু কিছুক্ষণ দুই সংস্করণ একসঙ্গে চলে, আর Rollback-ও আরেকটি Rolling Deploy। **Blue/Green:** বড় বা ঝুঁকিপূর্ণ Release, যেখানে তাৎক্ষণিক Rollback চাই এবং দ্বিগুণ পরিবেশের খরচ বহন করা যায়; Database উভয় পরিবেশে এক, তাই Migration সামঞ্জস্যপূর্ণ হতেই হবে। **Canary:** অনেক Traffic ও ভালো Metrics থাকলে সেরা—আসল ব্যবহারকারীর ছোট অংশে ঝুঁকি সীমিত রেখে সংখ্যা দিয়ে সিদ্ধান্ত (উদাহরণ ০৫); কিন্তু অল্প Traffic-এ পরিসংখ্যান দুর্বল।

সমাধান ৯

success-rate.mjs · JavaScript চলবে: Node.js

```
// 8,000 অনুরোধে ৩,৯৮০ সফল। SLO ৯৯.৯% হলে কী দাঁড়ায়?
const total = 4000;
const ok = 3980;
const slo = 0.999;

const successRate = ok / total;
const allowedFailures = total * (1 - slo);
const failures = total - ok;

console.log('Success Rate:', `${(successRate * 100).toFixed(1)}%`);
console.log('SLO অনুযায়ী অনুমোদিত ব্যর্থতা:', allowedFailures.toFixed(1));
console.log('আসল ব্যর্থতা:', failures);
console.log('Budget-এর', `${((failures / allowedFailures) * 100).toFixed(0)}%`,
  'খরচ; Burn Rate', ((1 - successRate) / (1 - slo)).toFixed(1));
console.log(successRate >= slo ? 'SLO পূরণ' : 'SLO পূরণ হয়নি');
```

Output

```
Success Rate: 99.5%
SLO অনুযায়ী অনুমোদিত ব্যর্থতা: 4.0
আসল ব্যর্থতা: 20
Budget-এর 500% খরচ; Burn Rate 5.0
SLO পূরণ হয়নি
```

8,000 অনুরোধে ৯৯.৯% মানে মাত্র ৪টি ব্যর্থতার অনুমতি; ২০টি হয়েছে—Budget পাঁচ গুণ খরচ। ৯৯.৫% শুনতে ভালো, কিন্তু SLO-র প্রেক্ষিতে খারাপ। আর ছোট Window-তে (8,000 অনুরোধ) একটি খারাপ মিনিটই ফল বদলে দেয়; তাই SLO সাধারণত ২৮ বা ৩০ দিনের Window-তে মাপা হয়।

সমাধান ১০ — (১) **Store:** `createPostgresJobStore(pool)` একই `list`, `create`, `ping` চুক্তিতে (অধ্যায় ২৬-এর Adapter-এর মতো); `ping` চালায় `SELECT 1`। (২) **Migration:** `001-jobs.sql` (Table, CHECK Constraint, Index); Deploy-এর আগে আলাদা ধাপে চলে, Application চালুর সময় নয়; প্রতিটি পশ্চাৎ-সামঞ্জস্যপূর্ণ (Expand/Contract)। (৩) **Readiness:** `store.ping()` ইতিমধ্যে সময়সীমায় যাচাই হয়, তাই Database বন্ধ হলে 503। (৪) **Secret:** `DATABASE_URL` Secret Store থেকে; `loadConfig`-এ Production-এ বাধ্যতামূলক, আর `describeConfig`-এ লুকানো। (৫) **Test:** Unit Test Memory Store দিয়ে; Integration Test একটি আসল PostgreSQL-এ (CI-তে Service Container)। (৬) **Shutdown:** `server.close`-এর পরে `pool.end()`। (৭) **Rollback:** Application-এর আগের Image চলতে পারে কারণ Migration শুধু যোগ করেছে; প্রথম Deploy-এর আগে Backup ও Restore Drill।

সমাধান ১১

cached-readiness.mjs · JavaScript চলবে: Node.js

```
// Readiness: Dependency-র যাচাই সময়সীমিত, আর ফল কিছুক্ষণ Cache।
// প্রতি সেকেন্ডে আসা Probe যেন Database-কে নিজেই চাপে না ফেলে।
export function createReadiness(check, { timeoutMs = 500, cacheMs = 2000,
  now = Date.now } = {}) {
  let cached = null;
  let calls = 0;
  async function probe() {
    if (cached && now() - cached.at < cacheMs) return cached.ready;
    calls += 1;
    let timer;
    const timeout = new Promise((resolve) => {
      timer = setTimeout(() => resolve(false), timeoutMs);
    });
    const ready = await Promise.race([
      check().then(() => true, () => false), timeout,
    ]).finally(() => clearTimeout(timer));
    cached = { ready, at: now() };
    return ready;
  }
  return { probe, calls: () => calls };
}

let clock = 0;
let healthy = true;
const db = () => (healthy ? Promise.resolve()
  : new Promise(() => {})); // অসুস্থ Database: কখনো উত্তর দেয় না
const readiness = createReadiness(db, { timeoutMs: 50, now: () => clock });

console.log('t=0:', await readiness.probe());
clock = 1000;
healthy = false;
console.log('t=1s (Cache থেকে):', await readiness.probe());
clock = 2500;
console.log('t=2.5s (নতুন যাচাই, Timeout):', await readiness.probe());
console.log('আসল যাচাই হয়েছে', readiness.calls(), 'বার');
```

Output

```
t=0: true
t=1s (Cache থেকে): true
t=2.5s (নতুন যাচাই, Timeout): false
আসল যাচাই হয়েছে 2 বার
```

এক সেকেন্ডে Database অসুস্থ হলেও Cache-এর কারণে Readiness তখনো `true` —এটি ইচ্ছাকৃত মূল্য: সর্বোচ্চ দুই সেকেন্ড দেরিতে জানা, বিনিময়ে প্রতি Probe-এ Database-এ একটি Query না পাঠানো। অসুস্থ Database কখনো উত্তর দেয় না, তাই Timeout ছাড়া Probe নিজেই আটকে যেত। `clock` Inject করা, যাতে Test-এ সত্যিকারের অপেক্ষা লাগে না।

সমাধান ১২ — (১) প্রতিটি Action পূর্ণ Commit SHA দিয়ে Pin, এবং Dependabot বা অনুরূপ ব্যবস্থায় হালনাগাদ। (২) `npm ci` -এর জন্য Lockfile, এবং `npm audit` বা অন্য Dependency Scan। (৩) Image Scan (যেমন Trivy) ও SBOM তৈরি; গুরুতর দুর্বলতায় ব্যর্থ। (৪) Image Registry-তে Push শুধু `main` Branch থেকে, শুধু সেই Job-এ `packages: write` অনুমতি। (৫) Cloud-এ Deploy-এর জন্য দীর্ঘমেয়াদি Key-এর বদলে OIDC-ভিত্তিক স্বল্পমেয়াদি Credential। (৬) Pull Request-এ Fork থেকে আসা Code-কে Secret দেখতে না দেওয়া। (৭) Branch Protection: CI পাস ও Review ছাড়া Merge নয়।

২৮.২০ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Readiness ব্যর্থ হলে সাধারণত কী ঘটে?

ক) Container Restart হয়

খ) Load Balancer এই Instance-এ নতুন Traffic পাঠানো বন্ধ করে

গ) Image মুছে যায়

ঘ) Database Restart হয়

প্রশ্ন ২. Dockerfile-এ `CMD ["node", "main.mjs"]` কেন `CMD npm start` -এর চেয়ে ভালো?

ক) দ্রুত Build হয়

খ) Image ছোট হয়

গ) `node` PID 1 হয়ে SIGTERM সরাসরি পায়

ঘ) npm Container-এ থাকে না

প্রশ্ন ৩. DeployReady-এ `PORT=abc` দিলে কী হয়?

ক) Port 3000-এ চালু হয়

খ) এলোমেলো Port-এ চালু হয়

গ) একটি Warning দিয়ে চালু হয়

ঘ) ভুলের তালিকা দেখিয়ে Exit Code 1

প্রশ্ন ৪. Production-এ Secret কোথায় রাখা উচিত?

ক) Platform-এর Secret Store-এ, নিয়ন্ত্রিত অনুমতি ও Rotation-সহ

খ) Git-এ, Private Repository হলে

গ) Docker Image-এর ENV-এ

ঘ) Frontend-এর Configuration File-এ

প্রশ্ন ৫. Compose-এ Drain ৫ সেকেন্ড ও Shutdown Timeout ১৫ সেকেন্ড হলে `stop_grace_period` কেমন হওয়া উচিত?

ক) ৫ সেকেন্ড

খ) ২০ সেকেন্ডের বেশি

গ) ১০ সেকেন্ড

ঘ) শূন্য

প্রশ্ন ৬. Metrics-এর Label-এ আসল Path (যেমন `/jobs/9381`) রাখলে কী সমস্যা?

ক) Metrics ভুল হয়

খ) Prometheus পড়তে পারে না

গ) অসংখ্য Time Series তৈরি হয়ে Memory শেষ হয় (High Cardinality)

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৭. Canary Rollout-এ কী করা হয়?

ক) নতুন সংস্করণে প্রথমে অল্প Traffic, মেপে ধাপে বাড়ানো

খ) সব Traffic একসঙ্গে নতুন সংস্করণে

গ) Test বাদ দিয়ে দ্রুত Deploy

ঘ) পুরোনো সংস্করণ মুছে ফেলা

প্রশ্ন ৮. RPO কী বোঝায়?

ক) সেবা ফেরার সর্বোচ্চ সময়

খ) সর্বোচ্চ সহনীয় তথ্যহানি

গ) Request-এর গড় সময়

ঘ) Backup-এর আকার

প্রশ্ন ৯. SLO ৯৯.৯%, আর শেষ এক ঘণ্টায় Error Rate ১.৫%। Burn Rate কত?

ক) ১.৫

খ) ০.১

গ) ৯৯.৯

ঘ) ১৫

প্রশ্ন ১০. অপ্রত্যাশিত Error-এ DeployReady Client-কে কী পাঠায়?

ক) পুরো Stack Trace

খ) Database-এর Error বার্তা

গ) সাধারণ বার্তা ও Request ID; বিস্তারিত শুধু Log-এ

ঘ) কিছুই না, সংযোগ কেটে দেয়

২৮.২১ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–ক · ৮–খ · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Restart Liveness-এর কাজ; Readiness শুধু Traffic সরায়।
২	গ	npm Signal ঠিকমতো পৌঁছায় না, ফলে Graceful Shutdown ঘটে না।
৩	ঘ	`loadConfig` সব ভুল জমিয়ে Throw করে; `main.mjs` Exit Code 1 দেয়।
৪	ক	Git ও Image-এর ইতিহাস মোছা কঠিন; Secret Store নিয়ন্ত্রিত ও Rotation-যোগ্য।
৫	খ	৫ + ১৫ = ২০; Grace Period কম হলে Platform আগেই SIGKILL পাঠায়।
৬	গ	প্রতিটি নতুন Label-মান একটি নতুন Time Series; তাই Route-এর নাম।
৭	ক	উদাহরণ ০৫-এর মতো সংখ্যা দেখে এগোনো বা ফিরে যাওয়া।
৮	খ	সময়ের সীমা RTO; RPO হলো কতটা তথ্য হারানো সহনীয়।
৯	ঘ	১.৫% ÷ ০.১% = ১৫; উদাহরণ ০৪-এ এই হারে Page।
১০	গ	`app.test.mjs`-এর Test দেখায় Response-এ "exploded" নেই, Log-এ আছে।

২৮.২২ অধ্যায়ের সারসংক্ষেপ

- Production মানে পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য ও ফেরানো-যোগ্য Release, শুধু চালু Server নয়।
- Configuration Environment থেকে, শুরুতেই পুরো যাচাই; ভুল হলে চালুই নয়; গোপন মান কখনো Log-এ নয়।

- একবার Build, একই Image সবখানে; Commit SHA ও Digest দিয়ে পরিচয়; Lockfile ও `npm ci`।
- Dockerfile: নির্দিষ্ট Base, Cache-বান্ধব ক্রম, `.dockerignore`, Non-root, HEALTHCHECK, Exec Form CMD।
- Compose: Local-এ শুধু `127.0.0.1`, Secret `env_file`-এ, Drain + Timeout < Grace Period।
- CI: ন্যূনতম অনুমতি, Timeout, Test-এর পরে Image, Smoke Test; Action SHA দিয়ে Pin।
- Rolling, Blue/Green, Canary; Canary-র সিদ্ধান্ত সংখ্যায়; Migration পশ্চাৎ-সামঞ্জস্যপূর্ণ (Expand/Contract)।
- Liveness সরল, Readiness সত্যিকারের প্রস্তুতি; Graceful Shutdown-এর পাঁচ ধাপ।
- Structured Log ও Request ID; Prometheus Counter, Gauge, Histogram; সীমিত Label; Trace ও OpenTelemetry।
- SLI, SLO, Error Budget, Burn Rate; কাজে লাগে এমন Alert ও Runbook।
- Secret Store, Supply Chain, Least Privilege; RPO, RTO, Restore Drill, দোষারোপহীন Postmortem।

পরের অধ্যায়ে দৃষ্টি একটি Service থেকে পুরো ব্যবস্থায় যাবে: কখন Monolith ভাঙবেন, Service-গুলো কীভাবে কথা বলবে, একাধিক Service জুড়ে একটি Booking কীভাবে সম্পূর্ণ বা সম্পূর্ণ বাতিল হবে (Saga), এবং ব্যর্থতার মধ্যেও ব্যবস্থা কীভাবে নির্ভরযোগ্য থাকবে।

২৮.২৩ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Twelve-Factor App	Cloud-উপযোগী Application-এর বারোটি নীতি।
Fail Fast	ভুল Configuration-এ শুরুতেই থেমে যাওয়া।
Build Artifact	পরীক্ষিত, অপরিবর্তনীয় Release-বস্তু (যেমন Image)।
Image / Container	অপরিবর্তনীয় Template / তার চলমান Instance।
Image Digest	Image-এর বিষয়বস্তুর অনন্য Hash।
CI / CD	প্রতিটি পরিবর্তনের স্বয়ংক্রিয় যাচাই / স্বয়ংক্রিয় Release-প্রক্রিয়া।
Rolling, Blue/Green, Canary	Release ছড়ানোর তিনটি কৌশল।
Liveness / Readiness	Process জীবিত কি না / Traffic নিতে প্রস্তুত কি না।
Graceful Shutdown	চলমান কাজ শেষ করে সুশৃঙ্খলভাবে বন্ধ হওয়া।
Structured Log	প্রতি Line একটি যন্ত্র-পাঠযোগ্য Object।
Counter, Gauge, Histogram	শুধু বাড়ে / বাড়ে-কমে / Bucket-এ গোনা Metrics।
Trace, Span	একটি অনুরোধের পুরো পথ / তার একটি ধাপ।
SLI, SLO, SLA	মাপা সংখ্যা / লক্ষ্য / চুক্তি।

পরিভাষা	অর্থ
Error Budget, Burn Rate	অনুমোদিত ব্যর্থতা / তা খরচের গতি।
RPO, RTO	সহনীয় তথ্যহানি / সেবা ফেরার সময়সীমা।
SBOM	Software-এ থাকা সব Component-এর তালিকা।

২৮.২৪ আরও পড়ুন

- **The Twelve-Factor App:** `https://12factor.net/` — মূল নীতিমালা।
- **Docker — Building best practices:** `https://docs.docker.com/build/building/best-practices/` — Dockerfile লেখার পরামর্শ।
- **Docker — Compose file reference:** `https://docs.docker.com/reference/compose-file/` — Compose-এর প্রতিটি Key।
- **GitHub — Security hardening for GitHub Actions:** `https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions` — অনুমতি, Pin ও Secret।
- **Prometheus — Exposition formats:** `https://prometheus.io/docs/instrumenting/exposition_formats/` — Text Format।
- **OpenTelemetry — Concepts:** `https://opentelemetry.io/docs/concepts/` — Trace, Metrics, Log।
- **Google SRE Workbook — Alerting on SLOs:** `https://sre.google/workbook/alerting-on-slos/` — Burn Rate-ভিত্তিক Alert।
- **Node.js — Process signal events:** `https://nodejs.org/api/process.html#signal-events` — SIGTERM ও SIGINT।

অধ্যায় ২৯

২৯

Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Functional ও Non-functional Requirement আলাদা করে লিখতে, এবং Capacity Planning-এ সংখ্যা দিয়ে অনুমান করতে (Little's Law-সহ)।
- Monolith, Modular Monolith ও Microservices-এর সুবিধা-অসুবিধা বিচার করে উপযুক্তটি বেছে নিতে।
- Domain-Driven Design-এর Bounded Context দিয়ে Service-এর সীমানা ও Data-র মালিকানা ঠিক করতে।
- Synchronous (HTTP) ও Asynchronous (Event) যোগাযোগের মূল্য বুঝতে।
- একাধিক Service জুড়ে একটি কাজের জন্য Saga Pattern—Orchestration ও Choreography—প্রয়োগ করতে, এবং Compensation লিখতে।
- Idempotency Key ও Request Fingerprint দিয়ে Duplicate ও Conflict সামলাতে।
- CAP-এর আসল অর্থ বুঝে কোথায় দৃঢ় Consistency আর কোথায় Eventual Consistency যথেষ্ট তা ঠিক করতে।
- Timeout, Retry (Backoff + Jitter), Circuit Breaker, Bulkhead, Outbox ও Inbox প্রয়োগ করতে।
- Crash-এর পরে টেকসই Log থেকে অসমাপ্ত কাজ নিরাপদে শেষ করতে।
- **TravelSaga** নামে একটি পরীক্ষিত Booking Orchestrator তৈরি করতে, যেখানে আলাদা HTTP Service, টেকসই Saga Log ও একটি Failure Lab আছে।

পূর্বশর্ত

- অধ্যায় ২৫: Node.js HTTP Server, Idempotency-Key।
- অধ্যায় ২৬: Transaction, UNIQUE Constraint, `node:sqlite`।
- অধ্যায় ২৭: Event Envelope, At-least-once, Transactional Outbox, Backoff ও Jitter।
- অধ্যায় ২৮: Health Check, Structured Log, Metrics ও SLO।

এই অধ্যায় কেন গুরুত্বপূর্ণ

রাফি একটি Cox's Bazar Package-এ দুটি আসন Book করছেন। কাজটি তিন ধাপের: আসন ধরা, টাকা কাটা, Booking নিশ্চিত করা। একটি Application ও একটি Database হলে তিনটিই এক Transaction-এ—সব হবে, নয়তো কিছুই না। কিন্তু প্রতিষ্ঠান বড় হলে আসনের হিসাব রাখে একটি দল ও একটি Service, টাকার হিসাব আরেক দল ও আরেক Service (প্রায়ই একটি বাইরের Payment Gateway), প্রত্যেকের নিজস্ব Database।

এখন নতুন প্রশ্ন: আসন ধরা হলো, কিন্তু টাকা কাটা ব্যর্থ—আসন কে ফেরত দেবে? টাকা কাটার অনুরোধ পাঠানো হলো, কিন্তু উত্তর আসার আগেই Network কেটে গেল—টাকা কি কাটা হয়েছে? আবার চেষ্টা করলে কি দুবার কাটবে? আর Orchestrator ঠিক মাঝখানে বন্ধ হয়ে গেলে, Restart-এর পরে কোথা থেকে শুরু করবে?

Distributed System-এ এই প্রশ্নগুলো ব্যতিক্রম নয়, প্রতিদিনের ঘটনা। এই অধ্যায় সেগুলোর উত্তর দেয় নকশার নীতি দিয়ে, এবং প্রতিটি উত্তর আসল HTTP Service, আসল Timeout ও আসল Crash দিয়ে পরীক্ষা করে।

২৯.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
এক Database-এ Transaction (অধ্যায় ২৬)	একাধিক Service জুড়ে Saga ও Compensation
Idempotency-Key (অধ্যায় ২৫)	Key + Fingerprint; প্রতিটি অংশগ্রহণকারী Service-ও Idempotent
Outbox (অধ্যায় ২৭)	Saga Log, Outbox ও Inbox একসঙ্গে
Backoff ও Jitter (অধ্যায় ২৭)	Timeout, Circuit Breaker, Bulkhead, Retry Amplification
Health ও Metrics (অধ্যায় ২৮)	Capacity Planning, Little's Law
—	Monolith বনাম Microservices, Bounded Context, CAP

> **নোট:** TravelSaga-র Inventory ও Payment দুটি সত্যিকারের আলাদা HTTP Server, প্রত্যেকের নিজস্ব Port ও Data, কিন্তু QA-র সুবিধার জন্য একই Node.js Process-এ চলে। Payment একটি অনুকরণ, আসল Payment Gateway নয়। Orchestrator-এর Saga Log সত্যিকারের SQLite Database (`node:sqlite`), এবং Crash-এর Test আসল Database File ব্যবহার করে। কোনো Message Broker (Kafka, RabbitMQ) ব্যবহার হয়নি; Event-এর প্রবাহ in-memory অনুকরণে দেখানো হয়েছে।

২৯.২ System Design আসলে কী

System Design মানে শুধু Box ও তীর আঁকা নয়। এটি একটি ব্যবসার প্রয়োজনকে একটি নির্ভরযোগ্য, নিরাপদ, রক্ষণযোগ্য, পর্যবেক্ষণযোগ্য এবং খরচ-সচেতন ব্যবস্থায় রূপান্তরের প্রক্রিয়া। শুরু হয় দুই ধরনের Requirement দিয়ে।

Functional Requirement—ব্যবস্থা কী করবে:

- গ্রাহক Package খুঁজতে ও Booking-এর অনুরোধ পাঠাতে পারবেন।
- আসন ধরা হবে; টাকা কাটা সফল হলে Booking নিশ্চিত।
- যেকোনো ধাপ ব্যর্থ হলে আগের ধাপগুলো ফিরিয়ে নেওয়া হবে।
- একই অনুরোধ একাধিকবার এলে একবারই কাজ হবে।

Non-functional Requirement—কতটা ভালোভাবে করবে: Availability, Latency, নিরাপত্তা, টেকসই Data, Scalability, Audit, Disaster Recovery ও খরচ। প্রতিটিকে মাপযোগ্য করুন। "দ্রুত হতে হবে" নয়, "৯৫% অনুরোধ ৩০০ ms-এর মধ্যে"—এবং তা প্রতিশ্রুতি দেওয়ার আগে Traffic-এর ধরন ও প্রতিটি Dependency-র Latency মাপুন। ৩০০ ms কোনো সার্বজনীন মান নয়; একটি বাইরের Payment Gateway নিজেই এক সেকেন্ড নিতে পারে।

২৯.৩ Capacity Planning: অনুমান নয়, হিসাব

capacity.mjs · JavaScript — চলবে: Node.js

```
// Capacity Planning: অনুমান নয়, হিসাব। সংখ্যাগুলো শুরুর অনুমান, পরে মেপে বদলাবে।
const dailyRequests = 86_400;
const peakMultiplier = 20; // সন্ধ্যার Sale-এ গড়ের ২০ গুণ
const latencySeconds = 0.25; // একটি অনুরোধে গড় সময়
const queriesPerRequest = 4;
const cacheHitRate = 0.6;

const avgRps = dailyRequests / (24 * 60 * 60);
const peakRps = avgRps * peakMultiplier;
// Little's Law: একসঙ্গে চলমান অনুরোধ = আগমনের হার x গড় সময়
const concurrent = peakRps * latencySeconds;
const dbQps = peakRps * queriesPerRequest * (1 - cacheHitRate);

console.log({ avgRps, peakRps, concurrent, dbQps });
// Retry Amplification: Payment ব্যর্থ হলে প্রতিটি অনুরোধ ৩ বার চেষ্টা করে
console.log('ব্যর্থতার সময় Payment-এ অনুরোধ/সেকেন্ড:', peakRps * 3);
```

Output

```
{ avgRps: 1, peakRps: 20, concurrent: 5, dbQps: 32 }
ব্যর্থতার সময় Payment-এ অনুরোধ/সেকেন্ড: 60
```

দিনে ৮৬,৪০০ অনুরোধ মানে গড়ে সেকেন্ডে মাত্র একটি—কিন্তু গড় দিয়ে Server কিনবেন না। Peak-এ সেকেন্ডে ২০টি। **Little's Law** বলে: একসঙ্গে চলমান কাজ = আগমনের হার × প্রতিটির গড় সময়; তাই Peak-এ একসঙ্গে ৫টি অনুরোধ চলবে, যা Connection Pool ও Thread-এর মাপ ঠিক করতে কাজে লাগে। Cache ৬০% Query বাঁচালে Database-এ সেকেন্ডে ৩২টি Query। আর শেষ Line একটি সতর্কবার্তা: Payment ব্যর্থ হলে প্রতিটি অনুরোধ তিনবার চেষ্টা করলে, ঠিক যখন Payment দুর্বল, তখনই তার ওপর চাপ তিন গুণ (**Retry Amplification**)।

এগুলো শুরুর অনুমান। Payload-এর আকার, Cache-এর আসল Hit Rate, বাইরের Service-এর Latency—সব মেপে সংখ্যা হালনাগাদ করুন, আর Load Test দিয়ে যাচাই করুন।

২৯.৪ Monolith, Modular Monolith ও Microservices

- **Monolith:** একটি Deploy-যোগ্য Application। Deploy ও Debug সহজ, এক Database-এ Transaction সহজ। কিন্তু ভেতরের সীমানা দুর্বল হলে সবকিছু সবকিছুর সঙ্গে জড়িয়ে যায়।
- **Modular Monolith:** একটিই Deploy, কিন্তু ভেতরে স্পষ্ট Module—প্রতিটির নিজস্ব Data, শুধু Public Interface দিয়ে যোগাযোগ।
- **Microservices:** প্রতিটি Service আলাদাভাবে Deploy ও Scale হয়, প্রায়ই আলাদা দলের মালিকানায়। বিনিময়ে আসে Network-এর ব্যর্থতা, ছড়ানো Data, Distributed Tracing ও অনেক বেশি Operations।

বিষয়	Modular Monolith	Microservices
Deploy	একটি একক	অনেক একক, আলাদা সময়ে
Transaction	এক Database-এ সহজ	Service জুড়ে Saga লাগে
ব্যর্থতা	Function Call ব্যর্থ হয় না	প্রতিটি ডাকে Timeout, Retry
পর্যবেক্ষণ	তুলনামূলক সহজ	Distributed Tracing জরুরি
দলের স্বাধীনতা	মাঝারি	সঠিক সীমানায় বেশি
Operations-এর খরচ	কম	অনেক বেশি

Modular Monolith-এর সীমানা Code-এ কেমন দেখায়:

modular-monolith.mjs · JavaScript চলবে: Node.js

```
// Modular Monolith: একটি Deploy, কিন্তু প্রতিটি Module শুধু তার Public
// Interface দেয়; অন্য Module-এর Data সরাসরি ছোঁয়া যায় না।
function createInventoryModule(seats) {
  let available = seats; // Private: বাইরে থেকে দেখা বা বদলানো যায় না
  return Object.freeze({
    reserve(count) {
      if (count > available) return { ok: false, reason: 'SOLD_OUT' };
      available -= count;
      return { ok: true };
    },
    release(count) {
      available += count;
    },
    available: () => available,
  });
}

function createBookingModule({ inventory }) {
  const bookings = [];
  return Object.freeze({
    book(customer, seats) {
      const result = inventory.reserve(seats); // শুধু Interface দিয়ে
      if (!result.ok) return result;
      bookings.push({ customer, seats });
      return { ok: true, bookingNo: bookings.length };
    },
  });
}

const inventory = createInventoryModule(3);
const booking = createBookingModule({ inventory });
console.log(booking.book('Rahim', 2));
console.log(booking.book('Mina', 2));
```

```
console.log('বাকি আসন:', inventory.available());
try {
  inventory.available = () => 99; // অন্য Module-এর ভেতরে হাত দেওয়ার চেষ্টা
} catch (error) {
  console.log('আটকানো হলো:', error.constructor.name);
}
```

Output

```
{ ok: true, bookingNo: 1 }
{ ok: false, reason: 'SOLD_OUT' }
বাকি আসন: 1
আটকানো হলো: TypeError
```

Booking Module আসনের সংখ্যা জানে না, শুধু `reserve` ডাকে। `Object.freeze` ও Closure অন্য Module-কে ভেতরে হাত দিতে দেয় না—ES Module Strict Mode-এ চলে বলে চেষ্টা করলে TypeError। এই সীমানা পরিষ্কার থাকলে পরে Inventory-কে আলাদা Service-এ বের করা প্রায় যান্ত্রিক কাজ: Function Call-এর জায়গায় HTTP Call।

পরামর্শ: প্রমাণিত প্রয়োজন—আলাদাভাবে Scale করার চাহিদা, স্বাধীন দল, ভিন্ন Release-এর গতি—না থাকলে Modular Monolith দিয়ে শুরু করুন। Microservices স্বয়ংক্রিয় উন্নতি নয়; এটি একটি মূল্য, যা বিশেষ সমস্যার সমাধানে দেওয়া হয়।

২৯.৫ Domain-Driven Design ও Bounded Context

Bounded Context একটি ব্যবসায়িক এলাকা, যার নিজস্ব ভাষা ও নিয়ম। ভ্রমণ-ব্যবসায়: Catalog, Inventory, Booking, Payment, Notification। একই শব্দের অর্থ Context-ভেদে বদলায়: Booking-এর "নিশ্চিত" আর Payment-এর "Captured" এক নয়; Inventory-র "Reservation" Booking-এর "Booking" নয়। সীমানা টানুন ব্যবসার সক্ষমতা ধরে, Database Table ধরে নয়।

Folder-এর কাঠামো

```
Client / Web App
       |
   API Gateway (Auth, Rate Limit)
       |
 +-----+-----------+------------+
 | Catalog      Booking      Identity |
 +-----------------+------------+
                   |
          Saga Orchestrator ---- Saga Log (SQLite)
              /          \
        Inventory       Payment
              \          /
               Event Bus
                   |
              Notification
```

এটি একটি ধারণাগত চিত্র, কোনো Deploy-করা কাঠামো নয়। মূল নিয়ম **Data-র মালিকানা**: প্রতিটি Service শুধু নিজের Database-এ লেখে। Booking Service কখনো Inventory-র Table-এ সরাসরি `UPDATE` চালাবে না—তাহলে Inventory-র নিয়ম (আসন ঋণাত্মক নয়) পাশ কাটানো যায়, আর Inventory দল নিজের Schema বদলাতে পারে না। অন্যের Data চাইলে তার API বা তার প্রকাশিত Event।

২৯.৬ Synchronous বনাম Asynchronous যোগাযোগ

Synchronous (HTTP): অনুরোধ পাঠিয়ে উত্তরের অপেক্ষা। সরল, আর উত্তর সঙ্গে সঙ্গে পাওয়া যায়। কিন্তু ডাকা Service ধীর বা বন্ধ হলে ডাকারীও ধীর বা বন্ধ; একটি অনুরোধের পথে যত Service, Availability তত কম (তিনটি ৯৯.৯% Service পরপর মানে প্রায় ৯৯.৭%)।

Asynchronous (Event): Event প্রকাশ করে এগিয়ে যাওয়া; আগ্রহীরা নিজের সময়ে প্রক্রিয়া করে। Service-গুলো সময়ে বিচ্ছিন্ন, একটি বন্ধ থাকলেও অন্যরা চলে। বিনিময়ে আসে Delivery-র নিশ্চয়তা, ক্রম, Duplicate, Schema-র বিবর্তন, Dead-letter Queue—আর "কাজ শেষ হয়েছে কি না" বোঝা কঠিন হয়।

Event-এর নামে সংস্করণ রাখা (`booking.confirmed.v1`) বা Envelope-এ `version` Field (অধ্যায় ২৭)—যেকোনোভাবে নিয়ম একই: নতুন ঐচ্ছিক Field যোগ নিরাপদ; নাম বা অর্থ বদলানো নতুন সংস্করণ।

২৯.৭ Distributed Transaction ও Saga

দুটি আলাদা Database-এর পরিবর্তন একটি সাধারণ Transaction-এ Commit করা যায় না। Two-Phase Commit (2PC) তাত্ত্বিকভাবে আছে, কিন্তু সব অংশগ্রহণকারী (বিশেষত বাইরের Payment Gateway) তা সমর্থন করে না, এবং সমন্বয়কারী ব্যর্থ হলে সবাই আটকে থাকে। **Saga** ব্যবহারিক বিকল্প: প্রতিটি ধাপ নিজের Service-এ একটি স্থানীয়

Transaction, আর পরের ধাপ ব্যর্থ হলে আগের ধাপগুলোর **Compensation**—একটি যুক্তিসংগত উল্টো কাজ (আসন ছাড়া, Refund)।

Folder-এর কাঠামো

```
সফল পথ:       Booking অনুরোধ -> আসন ধরা -> টাকা কাটা -> নিশ্চিত -> Event
আসন নেই:      আসন ধরা ব্যর্থ (409) -> প্রত্যাখ্যান (টাকায় হাতই পড়েনি)
টাকা ব্যর্থ:  আসন ধরা -> টাকা কাটা ব্যর্থ (402) -> আসন ছাড়া -> প্রত্যাখ্যান
অনিশ্চিত:     আসন ধরা -> টাকা কাটা Timeout -> Refund -> আসন ছাড়া -> প্রত্যাখ্যান
ক্ষতিপূরণ ব্যর্থ:  ... -> আসন ছাড়া ব্যর্থ -> MANUAL_REVIEW (মানুষের হাতে)
```

Compensation সব সময় নিখুঁত "Undo" নয়। পাঠানো Email ফেরানো যায় না, শুধু একটি সংশোধনী পাঠানো যায়; Refund-এ গ্রাহক কয়েক দিন টাকা ছাড়া থাকেন। তাই ধাপের ক্রম সাজান যাতে সহজে ফেরানো যায় এমন কাজ আগে (আসন ধরা), কঠিনটি পরে (টাকা কাটা), আর একেবারে ফেরানো যায় না এমন কাজ (Email) সবশেষে, সব নিশ্চিত হওয়ার পর।

Orchestration বনাম **Choreography:** Orchestration-এ একটি কেন্দ্রীয় সমন্বয়কারী প্রতিটি ধাপ ডাকে—প্রবাহ এক জায়গায় পড়া ও Debug করা যায়, TravelSaga এটিই করে। Choreography-তে কোনো পরিচালক নেই; প্রতিটি Service Event শুনে নিজের কাজ করে:

choreography.mjs · JavaScript চলবে: Node.js

```
// Choreography: কোনো কেন্দ্রীয় পরিচালক নেই। প্রতিটি Service Event শুনে
// নিজের কাজ করে এবং নতুন Event ছাড়ে। Compensation-ও Event দিয়ে।
const handlers = new Map();
const log = [];
const on = (type, fn) => handlers.set(type, [...(handlers.get(type) ?? []),
  fn]);
const emit = (type, data) => {
  log.push(type);
  for (const fn of handlers.get(type) ?? []) fn(data);
};

let seats = 5;
const declined = new Set(['Karim']);

on('booking.requested', (b) => (seats >= b.seats
  ? (seats -= b.seats, emit('seats.reserved', b))
  : emit('booking.rejected', { ...b, reason: 'SOLD_OUT' })));
on('seats.reserved', (b) => (declined.has(b.customer)
  ? emit('payment.failed', b) : emit('payment.captured', b)));
on('payment.captured', (b) => emit('booking.confirmed', b));
on('payment.failed', (b) => {
  seats += b.seats; // Inventory নিজেই শুনে আসন ফেরত দেয়
  emit('booking.rejected', { ...b, reason: 'PAYMENT_DECLINED' });
});

for (const customer of ['Rahim', 'Karim']) {
```

```
  log.length = 0;
  emit('booking.requested', { customer, seats: 2 });
  console.log(`${customer}: ${log.join(' > ')}`);
}
console.log('বাকি আসন:', seats);
```

Output

```
Rahim: booking.requested > seats.reserved > payment.captured > booking.confirmed
Karim: booking.requested > seats.reserved > payment.failed > booking.rejected
বাকি আসন: 3
```

Choreography-তে Service-গুলো একে অপরকে চেনে না, শুধু Event চেনে—যোগসূত্র কম। কিন্তু পুরো প্রবাহ কোথাও লেখা নেই; তা ছড়িয়ে আছে বহু Handler-এ। পাঁচ-ছয়টি ধাপের পরে "Booking এখন কোন অবস্থায়, আটকে আছে কোথায়" বোঝা কঠিন হয়। ছোট, স্থির প্রবাহে Choreography; বহু ধাপ, Timeout ও Compensation-যুক্ত ব্যবসায়িক প্রবাহে Orchestration সাধারণত সহজ।

২৯.৮ Idempotency: Key ও Fingerprint

Network Timeout মানে ব্যর্থতা নয়—শুধু "উত্তর জানা নেই"। Client আবার চেষ্টা করবেই, তাই প্রতিটি অবস্থা-বদলানো অনুরোধ Idempotent হতে হবে। দুটি অংশ: **Idempotency Key** বলে "এটি কোন যৌক্তিক অনুরোধ" (এখানে `bookingId`), আর **Fingerprint** বলে "ঠিক কী চাওয়া হয়েছিল"। একই Key ও একই Fingerprint হলে আগের ফল; একই Key কিন্তু ভিন্ন Fingerprint হলে এটি Client-এর Bug—409 Conflict, নীরবে পুরোনো ফল দেওয়া নয়।

fingerprint.mjs · JavaScript — চলবে: Node.js

```
import { fingerprintOf } from '../project/travel-saga/src/orchestrator.mjs';

// একই Idempotency Key, তিনটি অনুরোধ। Fingerprint বলে কোনটি সত্যিই "একই"।
const first = { bookingId: 'trip-7', customer: 'Rahim', seats: 2,
  amount: 1_250_000 };
const reordered = { amount: 1_250_000, seats: 2, customer: 'Rahim',
  bookingId: 'trip-7' };
const changed = { ...first, amount: 125_000 };

const short = (request) => fingerprintOf(request).slice(0, 12);
console.log('প্রথম:      ', short(first));
console.log('ক্রম বদলানো:', short(reordered), '(একই অনুরোধ)');
console.log('দাম বদলানো: ', short(changed), '(Conflict)');
```

Output

```
প্রথম:        f6b104d2fcb3
ক্রম বদলানো: f6b104d2fcb3 (একই অনুরোধ)
দাম বদলানো:  f29354afe76f (Conflict)
```

Fingerprint-এর আগে Key-গুলো সাজানো হয়, তাই JSON-এ Field-এর ক্রম বদলালেও একই অনুরোধ একই Hash পায়। TravelSaga-তে Idempotency দুই স্তরে: Orchestrator `bookingId` ও Fingerprint SQLite-এ রাখে (PRIMARY KEY), আর প্রতিটি অংশগ্রহণকারী Service-ও নিজে Idempotent—Inventory `reservationId`-তে, Payment `chargeId`-তে। এই দ্বিতীয় স্তরই Crash-এর পরে নিরাপদে আবার চালানো সম্ভব করে (অনুচ্ছেদ ২৯.১২)। একাধিক Replica থাকলে Process-এর Memory-র Lock যথেষ্ট নয়; Database-এর UNIQUE Constraint-ই চূড়ান্ত পাহারাদার।

২৯.৯ Consistency, CAP ও Data-র মালিকানা

CAP-কে প্রায়ই "তিনটির যেকোনো দুটি বেছে নিন" বলে সরলীকরণ করা হয়—ভুলভাবে। আসল কথা: Network বিভাজন (**Partition**) চলাকালে একটি ব্যবস্থাকে বেছে নিতে হয়—হয় কিছু অনুরোধ প্রত্যাখ্যান করে Consistency রাখবে (CP), নয়তো সব অনুরোধের উত্তর দিয়ে সাময়িক অসামঞ্জস্য মেনে নেবে (AP)। Partition না থাকলে দুটিই পাওয়া যায় (যদিও Latency-র মূল্যে)।

partition.mjs · JavaScript — চলবে: Node.js

```
// Network Partition চলাকালে দুটি Replica: Consistency না Availability?
// শেষ আসনটি দুই শহরের দুই Agent একই সময়ে বিক্রি করতে চান।
function replica(name, seats) {
  return { name, seats, sold: [] };
}

function sell(r, agent, { partitioned, mode }) {
  if (partitioned && mode === 'CP') {
    return `${r.name}: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)`;
  }
  if (r.seats < 1) return `${r.name}: আসন নেই`;
  r.seats -= 1;
  r.sold.push(agent);
  return `${r.name}: ${agent}-এর কাছে বিক্রি`;
}

for (const mode of ['AP', 'CP']) {
  const dhaka = replica('Dhaka', 1);
  const ctg = replica('Chattogram', 1);
  console.log(`${mode}:`);
  console.log('  ', sell(dhaka, 'Agent-A', { partitioned: true, mode }));
  console.log('  ', sell(ctg, 'Agent-B', { partitioned: true, mode }));
  // Partition সারল; দুই Replica মিলিয়ে দেখা
  const sold = [...dhaka.sold, ...ctg.sold];
```

```
  console.log(`  মিলনের পর বিক্রি: ${sold.length} (আসন ছিল 1)`,
    sold.length > 1 ? 'Overbooking!' : 'ঠিক আছে');
}
```

Output

```
AP:
  Dhaka: Agent-A-এর কাছে বিক্রি
  Chattogram: Agent-B-এর কাছে বিক্রি
  মিলনের পর বিক্রি: 2 (আসন ছিল 1) Overbooking!
CP:
  Dhaka: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)
  Chattogram: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)
  মিলনের পর বিক্রি: ০ (আসন ছিল 1) ঠিক আছে
```

কোনটি সঠিক, তা ব্যবসা ঠিক করে। শেষ আসনের মতো দুর্লভ সম্পদে Overbooking ব্যয়বহুল—তাই Inventory দৃঢ় সমন্বয় চায় (একটি মালিক Database, Transaction, Lock), এমনকি তাতে কিছু অনুরোধ প্রত্যাখ্যাত হলেও। কিন্তু Loyalty Point-এর হিসাব, Analytics বা Notification কয়েক সেকেন্ড দেরিতে হলে কোনো ব্যবসায়িক নিয়ম ভাঙে না—সেখানে **Eventual Consistency** (সময়ের সঙ্গে সব Replica এক অবস্থায় পৌঁছায়) সস্তা ও যথেষ্ট।

২৯.১০ নির্ভরযোগ্যতার Pattern

- **Timeout:** প্রতিটি Network ডাকে। Timeout ছাড়া একটি ধীর Service ধীরে ধীরে ডাকারীর সব সংযোগ ও Memory দখল করে।
- **Retry (Backoff + Jitter):** শুধু সাময়িক ব্যর্থতায় (5xx, Timeout, 429), শুধু Idempotent কাজে, সীমিত সংখ্যায়, আর শুধু একটি স্তরে। 4xx (যেমন Card প্রত্যাখ্যাত) আবার চেষ্টা করলে বদলায় না।
- **Circuit Breaker:** একটি Dependency বারবার ব্যর্থ হলে কিছুক্ষণ তাকে ডাকাই বন্ধ ("খোলা")—দ্রুত ব্যর্থতা, আর অসুস্থ Service-কে সেরে ওঠার সুযোগ। নির্দিষ্ট সময় পরে একটি পরীক্ষামূলক ডাক ("আধা-খোলা"); সফল হলে আবার স্বাভাবিক ("বন্ধ")।
- **Bulkhead:** প্রতিটি Dependency-র জন্য আলাদা সীমিত সম্পদ, যাতে একটির ধীরগতি সবাইকে ডুবিয়ে না দেয় (জাহাজের জলরোধী কক্ষের মতো)।
- **Dead-letter Queue:** বারবার ব্যর্থ Message আলাদা জায়গায়, তদন্ত ও হাতে আবার চালানোর জন্য।
- **Transactional Outbox:** অবস্থা-বদল ও Event একই Transaction-এ (অধ্যায় ২৭)।
- **Inbox:** Consumer-এর দিকে Event-এর ID-তে UNIQUE, যাতে At-least-once Delivery-তে কাজ একবারই।

TravelSaga-র HTTP Client প্রথম তিনটি একসঙ্গে বাস্তবায়ন করে। প্রথম অংশ: Error-এর ধরন ও Circuit Breaker-এর অবস্থা-যন্ত্র:

http-client.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
// Service-এর মধ্যে HTTP ডাক: Timeout, সীমিত Retry (Backoff + Jitter)
```

```
// এবং Circuit Breaker। 4xx-এ Retry নয়: সেটি ব্যবসার উত্তর, সাময়িক সমস্যা নয়।
import { setTimeout as delay } from 'node:timers/promises';

export class HttpError extends Error {
  constructor(status, body) {
    super(`HTTP ${status}${body?.error ? ` ${body.error}` : ''}`);
    this.status = status;
    this.body = body;
    this.retryable = status >= 500 || status === 429;
  }
}

export class CircuitOpenError extends Error {
  constructor() {
    super('Circuit open: Dependency-কে বিশ্রাম দেওয়া হচ্ছে');
    this.code = 'CIRCUIT_OPEN';
  }
}

// ...
```

http-client.mjs · JavaScript (অংশ ২ / ৩ (Part 2 of 3)) চলবে: Node.js

```
// ...

export function createBreaker({ failureThreshold = 3, resetMs = 1000,
  now = Date.now } = {}) {
  let state = 'closed';
  let failures = 0;
  let openedAt = 0;
  return {
    get state() {
      if (state === 'open' && now() - openedAt >= resetMs) state = 'half-open';
      return state;
    },
    before() {
      if (this.state === 'open') throw new CircuitOpenError();
    },
    success() {
      state = 'closed';
      failures = 0;
    },
    failure() {
      failures += 1;
      if (state === 'half-open' || failures >= failureThreshold) {
        state = 'open';
        openedAt = now();
      }
    },
```

```
  };
}

// ...
```

`state` একটি Getter: খোলা অবস্থায় `resetMs` পেরোলে নিজেই "half-open" হয়। Half-open অবস্থায় একটি ব্যর্থতাই আবার খুলে দেয়। এবার ডাক নিজে:

http-client.mjs · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

export const backoffMs = (attempt, random = Math.random, base = 50) =>
  Math.floor(random() * base * 2 ** attempt); // Full Jitter

export async function callJson(url, { method = 'GET', body, timeoutMs = 500,
  retries = 2, breaker, random = Math.random } = {}) {
  for (let attempt = 0; ; attempt += 1) {
    breaker?.before();
    try {
      const res = await fetch(url, { method,
        headers: body ? { 'Content-Type': 'application/json' } : {},
        body: body ? JSON.stringify(body) : undefined,
        signal: AbortSignal.timeout(timeoutMs) });
      const data = await res.json().catch(() => ({}));
      if (res.ok) {
        breaker?.success();
        return { status: res.status, ...data };
      }
      throw new HttpError(res.status, data);
    } catch (error) {
      if (error instanceof HttpError && !error.retryable) {
        breaker?.success(); // Service সুস্থ, শুধু উত্তর "না"
        throw error;
      }
      breaker?.failure();
      if (attempt >= retries) throw error;
      await delay(backoffMs(attempt, random));
    }
  }
}
```

চারটি সিদ্ধান্ত লক্ষ করুন: `AbortSignal.timeout` প্রতিটি চেষ্টাকে সময়সীমা দেয়; 4xx-এ Breaker-এ সাফল্য গোনা হয় (Service সুস্থ, শুধু উত্তর "না") এবং কোনো Retry নেই; Breaker খোলা থাকলে Network-এ যাওয়াই হয় না; আর **Full Jitter**—অপেক্ষা শূন্য থেকে সর্বোচ্চের মধ্যে এলোমেলো—যাতে একসঙ্গে ব্যর্থ হওয়া অনেক Client একসঙ্গে ফিরে না আসে।

২৯.১১ পর্যবেক্ষণ ও নিরাপত্তা

একাধিক Service জুড়ে একটি অনুরোধ Debug করতে **Distributed Trace** লাগে: একটি Trace ID প্রতিটি Service-এ যায় (W3C `traceparent` Header), প্রতিটি Log Line-এ থাকে, এবং OpenTelemetry-র মতো ব্যবস্থা পুরো পথ একটি সময়রেখায় দেখায় (অধ্যায় ২৮)। Saga-র ক্ষেত্রে আরেকটি শক্তিশালী হাতিয়ার হলো Saga Log নিজেই: TravelSaga-র `saga_log` প্রতিটি Booking-এর প্রতিটি অবস্থা-বদল সময়সহ রাখে—"কেন এই Booking প্রত্যাখ্যাত" প্রশ্নের উত্তর সেখানেই।

নিরাপত্তায়: Service থেকে Service-এর ডাকও Authenticate করুন (mTLS বা স্বল্পমেয়াদি Token)—"ভেতরের Network" মানেই বিশ্বস্ত নয়। প্রতিটি Service শুধু যা দরকার তার অনুমতি পায়। Trace ও Log-এ Card নম্বর, Password বা ব্যক্তিগত তথ্য নয়। আর Audit-এর জন্য টাকার প্রতিটি নড়াচড়া একটি অপরিবর্তনীয় Log-এ।

২৯.১২ Project: TravelSaga Booking Orchestrator

TravelSaga একটি Booking-এর তিনটি অংশগ্রহণকারীকে সমন্বয় করে: **Inventory Service** (আসন), **Payment Service** (টাকা) এবং **Orchestrator** (Saga), যার নিজস্ব SQLite Log। দুটি Service সত্যিকারের আলাদা HTTP Server; Orchestrator তাদের সঙ্গে শুধু HTTP-তে কথা বলে, কোনো Function বা Data ভাগ করে না। টাকা Integer পয়সায় (১২,৫০০ টাকা = 1250000), Floating-point-এর গোলমাল এড়াতে।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-saga/
├── package.json
├── README.md            চালানো, Failure Lab, সীমাবদ্ধতা
├── demo.mjs             সাতটি পরিস্থিতি
├── src/
│   ├── services/
│   │   ├── http-util.mjs   ছোট JSON Server
│   │   ├── inventory.mjs   আসন; reservationId-তে Idempotent
│   │   └── payment.mjs     টাকা (অনুকরণ); chargeId-তে Idempotent
│   ├── http-client.mjs     Timeout, Retry, Circuit Breaker
│   ├── saga-store.mjs      SQLite: অবস্থা, ইতিহাস, Outbox
│   ├── orchestrator.mjs    Saga, Compensation, Resume
│   └── system.mjs          সব একসঙ্গে চালু
└── tests/                  ১৪টি Test
```

অংশগ্রহণকারী Service

Inventory: প্রতিটি কাজ `reservationId` দিয়ে Idempotent। একই ID দ্বিতীয়বার এলে আসন আবার কমে না, আগের ফল (`replay: true`)। আসন ছাড়াও Idempotent: দ্বিতীয়বার `released: false`। `faults` Failure Lab-এর জন্য—পরের কয়েকটি অনুরোধে 503:

inventory.mjs · JavaScript — চলবে: Node.js

```
import { serveJson } from './http-util.mjs';

// Inventory Service: আসনের মালিক। প্রতিটি কাজ reservationId দিয়ে Idempotent,
// তাই Orchestrator নিশ্চিন্তে Retry বা Crash-এর পরে আবার ডাকতে পারে।
export async function startInventory({ seats }) {
  const state = { available: seats, reservations: new Map() };
  const faults = { failNext: 0 };

  function route(method, [resource, id], body) {
    if (method === 'GET' && resource === 'stock') {
      const active = [...state.reservations.values()].filter((r) => r.active);
      return [200, { available: state.available,
        reserved: active.reduce((sum, r) => sum + r.seats, 0) }];
    }
    if (method === 'POST' && resource === 'reservations') {
      if (faults.failNext > 0) {
        faults.failNext -= 1;
        return [503, { error: 'UNAVAILABLE' }];
      }
      const existing = state.reservations.get(body.reservationId);
      if (existing) return [200, { ...existing, replay: true }];
      if (!Number.isInteger(body.seats) || body.seats < 1) {
        return [400, { error: 'INVALID_SEATS' }];
      }
      if (state.available < body.seats) return [409, { error: 'SOLD_OUT' }];
      state.available -= body.seats;
      const reservation = { reservationId: body.reservationId,
        seats: body.seats, active: true };
      state.reservations.set(body.reservationId, reservation);
      return [201, { ...reservation }];
    }
    if (method === 'DELETE' && resource === 'reservations') {
      const reservation = state.reservations.get(id);
      if (!reservation?.active) return [200, { released: false }];
      reservation.active = false;
      state.available += reservation.seats;
      return [200, { released: true }];
    }
    return [404, { error: 'NOT_FOUND' }];
  }

  const server = await serveJson(route);
  return { ...server, state, faults };
}
```

Payment একই নীতিতে, `chargeId` দিয়ে। একই chargeId কিন্তু ভিন্ন Amount এলে 409—আসল Payment Gateway-ও এভাবেই কাজ করে। `delayMs` সবচেয়ে বিপজ্জনক পরিস্থিতি তৈরি করে: টাকা কাটা হয়ে গেছে, কিন্তু উত্তর দেরিতে আসে, তাই Client Timeout পায়:

payment.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { serveJson } from './http-util.mjs';

// Payment Service (অনুকরণ, আসল Gateway নয়)। chargeId দিয়ে Idempotent:
// একই chargeId ও একই Amount-এ আগের ফল, ভিন্ন Amount-এ 409।
export async function startPayment() {
  const charges = new Map();
  const faults = { failNext: 0, delayMs: 0, decline: new Set() };

// ...
```

payment.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  function route(method, [resource, id, action], body) {
    if (method === 'GET' && resource === 'ledger') {
      const all = [...charges.values()];
      const sum = (status) => all.filter((c) => c.status === status)
        .reduce((total, c) => total + c.amount, 0);
      return [200, { captured: sum('CAPTURED'), refunded: sum('REFUNDED'),
        attempts: all.length }];
    }
    if (method === 'POST' && resource === 'charges' && !id) {
      if (faults.failNext > 0) {
        faults.failNext -= 1;
        return [503, { error: 'UNAVAILABLE' }];
      }
      const old = charges.get(body.chargeId);
      if (old && old.amount !== body.amount) {
        return [409, { error: 'IDEMPOTENCY_CONFLICT' }];
      }
      if (!old) {
        const declined = faults.decline.has(body.customer);
        charges.set(body.chargeId, { chargeId: body.chargeId,
          amount: body.amount, status: declined ? 'DECLINED' : 'CAPTURED' });
      }
      const charge = charges.get(body.chargeId);
      // delayMs: টাকা কাটা হয়ে গেছে, কিন্তু উত্তর দেরিতে; Client Timeout পায়।
      if (charge.status === 'DECLINED') return [402, { ...charge }];
      return [old ? 200 : 201, { ...charge }, faults.delayMs];
    }
    if (method === 'POST' && resource === 'charges' && action === 'refund') {
      const charge = charges.get(id);
```

```
      if (charge?.status !== 'CAPTURED') return [200, { refunded: false }];
      charge.status = 'REFUNDED';
      return [200, { refunded: true }];
    }
    return [404, { error: 'NOT_FOUND' }];
  }

  const server = await serveJson(route);
  return { ...server, charges, faults };
}
```

src/saga-store.mjs

Orchestrator-এর টেকসই স্মৃতি। তিনটি Table: `sagas` (বর্তমান অবস্থা ও অনুরোধ), `saga_log` (প্রতিটি অবস্থা-বদলের ইতিহাস), `outbox` (প্রকাশিতব্য Event)। `BEGIN IMMEDIATE` লেখার Lock শুরুতেই নেয়, যাতে দুই সংযোগ একই Saga একসঙ্গে শুরু করতে না পারে:

saga-store.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// Orchestrator-এর নিজস্ব টেকসই Log: প্রতিটি Saga-র অবস্থা, ইতিহাস ও Outbox।
// অবস্থা-বদল, ইতিহাস ও Event একই Transaction-এ লেখা হয়।
export const TERMINAL = new Set(['CONFIRMED', 'REJECTED', 'MANUAL_REVIEW']);

export function openSagaStore(path = ':memory:', now = () => new Date()) {
  const db = new DatabaseSync(path);
  db.exec(`CREATE TABLE IF NOT EXISTS sagas (id TEXT PRIMARY KEY,
      fingerprint TEXT NOT NULL, request TEXT NOT NULL,
      status TEXT NOT NULL, reason TEXT, updated_at TEXT NOT NULL);
    CREATE TABLE IF NOT EXISTS saga_log (saga_id TEXT NOT NULL,
      status TEXT NOT NULL, reason TEXT, at TEXT NOT NULL);
    CREATE TABLE IF NOT EXISTS outbox (id INTEGER PRIMARY KEY,
      saga_id TEXT NOT NULL, type TEXT NOT NULL, payload TEXT NOT NULL,
      published INTEGER NOT NULL DEFAULT 0);`);
  const row = (r) => r && { ...r, request: JSON.parse(r.request) };
  const get = (id) => row(db.prepare('SELECT * FROM sagas WHERE id = ?').get(id));

  function inTransaction(fn) {
    db.exec('BEGIN IMMEDIATE');
    try {
      const result = fn();
      db.exec('COMMIT');
      return result;
    } catch (error) {
      db.exec('ROLLBACK');
      throw error;
    }
  }
```

```
// ...
```

`start` নতুন হলে তৈরি করে, আগে থাকলে যা আছে তা-ই ফেরত দেয়—এটিই Idempotency-র ভিত্তি। `transition` অবস্থা, ইতিহাস ও Event—তিনটি একসঙ্গে, একটি Transaction-এ:

saga-store.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

  return {
    get,
    // নতুন হলে তৈরি; আগে থাকলে যা আছে তা-ই (Idempotency-র ভিত্তি)।
    start(id, fingerprint, request) {
      return inTransaction(() => {
        const existing = get(id);
        if (existing) return { created: false, saga: existing };
        const at = now().toISOString();
        db.prepare(`INSERT INTO sagas (id, fingerprint, request, status,
          updated_at) VALUES (?, ?, ?, 'STARTED', ?)`)
          .run(id, fingerprint, JSON.stringify(request), at);
        db.prepare('INSERT INTO saga_log VALUES (?, ?, NULL, ?)')
          .run(id, 'STARTED', at);
        return { created: true, saga: get(id) };
      });
    },
    transition(id, status, reason = null, event = null) {
      inTransaction(() => {
        const at = now().toISOString();
        db.prepare(`UPDATE sagas SET status = ?, reason = ?, updated_at = ?
          WHERE id = ?`).run(status, reason, at, id);
        db.prepare('INSERT INTO saga_log VALUES (?, ?, ?, ?)')
          .run(id, status, reason, at);
        if (event) {
          db.prepare(`INSERT INTO outbox (saga_id, type, payload)
            VALUES (?, ?, ?)`).run(id, event.type, JSON.stringify(event.payload));
        }
      });
    },
    history: (id) => db.prepare(
      'SELECT status, reason FROM saga_log WHERE saga_id = ? ORDER BY rowid')
      .all(id),
    unfinished: () => db.prepare('SELECT * FROM sagas').all().map(row)
      .filter((saga) => !TERMINAL.has(saga.status)),
    pendingEvents: () => db.prepare(
      'SELECT id, saga_id, type FROM outbox WHERE published = 0 ORDER BY id')
      .all(),
    markPublished: (id) => db.prepare(
      'UPDATE outbox SET published = 1 WHERE id = ?').run(id),
```

```
    close: () => db.close(),
  };
}
```

src/orchestrator.mjs

প্রথম অংশ: Validation ও Fingerprint। Validation Saga শুরুর আগেই—ভুল অনুরোধ কোনো Service-কে বিরক্ত করে না:

orchestrator.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import { createHash } from 'node:crypto';
import { callJson, createBreaker, HttpError } from './http-client.mjs';
import { TERMINAL } from './saga-store.mjs';

export class SagaError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

function validate(input) {
  const { bookingId, customer, seats, amount } = input ?? {};
  const problems = [];
  if (!/^[\w-]{3,64}$/.test(bookingId ?? '')) problems.push('bookingId');
  if (typeof customer !== 'string' || !customer.trim()) problems.push('customer');
  if (!Number.isInteger(seats) || seats < 1 || seats > 9) problems.push('seats');
  // টাকা পয়সায় (Integer): ১২,৫০০ টাকা = 1250000
  if (!Number.isInteger(amount) || amount <= 0) problems.push('amount');
  if (problems.length) {
    throw new SagaError('VALIDATION', `ভুল Field: ${problems.join(', ')}`);
  }
  return { bookingId, customer: customer.trim(), seats, amount };
}

// একই অনুরোধ চেনার আঙুলের ছাপ: Field-এর ক্রম যা-ই হোক, একই Hash।
export const fingerprintOf = (request) => createHash('sha256')
  .update(JSON.stringify(Object.keys(request).sort()
    .map((key) => [key, request[key]])))
  .digest('hex');

// ...
```

`book`: অনুরোধ Validation, Saga শুরু বা আগেরটি খুঁজে পাওয়া, Fingerprint মিলানো, শেষ হয়ে থাকলে আগের ফল, নইলে কাজ। **Single-flight**: একই Process-এ একই Booking-এর একসঙ্গে আসা পাঁচটি অনুরোধ একটিই কাজ ভাগ করে নেয়:

orchestrator.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

export function createOrchestrator({ store, inventoryUrl, paymentUrl,
  call = callJson, hooks = {}, callOptions = {} }) {
  const breakers = { inventory: createBreaker(), payment: createBreaker() };
  const inFlight = new Map();
  const inventory = (path, opts) => call(`${inventoryUrl}${path}`,
    { breaker: breakers.inventory, ...callOptions, ...opts });
  const payment = (path, opts) => call(`${paymentUrl}${path}`,
    { breaker: breakers.payment, ...callOptions, ...opts });

  const view = (id) => {
    const saga = store.get(id);
    return { bookingId: id, status: saga.status, reason: saga.reason };
  };

  function finish(id, status, reason, request) {
    const type = { CONFIRMED: 'booking.confirmed', REJECTED: 'booking.rejected',
      MANUAL_REVIEW: 'booking.needs_review' }[status];
    store.transition(id, status, reason, { type, payload: { bookingId: id,
      customer: request.customer, seats: request.seats, reason } });
    return view(id);
  }

  async function book(input) {
    const request = validate(input);
    const id = request.bookingId;
    const { saga } = store.start(id, fingerprintOf(request), request);
    if (saga.fingerprint !== fingerprintOf(request)) {
      throw new SagaError('IDEMPOTENCY_CONFLICT',
        'একই bookingId, কিন্তু ভিন্ন অনুরোধ');
    }
    if (TERMINAL.has(saga.status)) return view(id);
    return singleFlight(id, () => run(id, request));
  }

  // একই Process-এ একই Booking-এর একাধিক ডাক একটিই কাজ ভাগ করে নেয়।
  function singleFlight(id, work) {
    if (!inFlight.has(id)) {
      inFlight.set(id, work().finally(() => inFlight.delete(id)));
    }
    return inFlight.get(id);
  }

// ...
```

Saga-র সামনের পথ। প্রতিটি ধাপের আগে অবস্থা Log-এ লেখা হয়, তাই Crash-এর পরে জানা যায় কোথায় থেমেছিল। ব্যর্থতার ধরন অনুযায়ী আলাদা সিদ্ধান্ত: 409 মানে আসন নেই (কিছু ফেরানোর নেই); 402 মানে প্রত্যাখ্যাত (টাকা কাটা হয়নি, শুধু আসন ফেরত); আর Timeout বা 5xx মানে **অনিশ্চিত**—টাকা কাটা হয়ে থাকতেও পারে, তাই Refund-ও:

orchestrator.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  // সামনের প্রতিটি ধাপ Idempotent (একই reservationId ও chargeId), তাই
  // Crash-এর পরে শুরু থেকে আবার চালানো নিরাপদ।
  async function run(id, request) {
    store.transition(id, 'RESERVING');
    try {
      await inventory('/reservations', { method: 'POST',
        body: { reservationId: id, seats: request.seats } });
    } catch (error) {
      if (error instanceof HttpError && error.status === 409) {
        return finish(id, 'REJECTED', 'SOLD_OUT', request);
      }
      // জানা নেই আসন ধরা হয়েছে কি না: ছেড়ে দেওয়ার চেষ্টা (Idempotent)।
      return compensate(id, 'INVENTORY_UNAVAILABLE', request, false);
    }
    await hooks.after?.('reserved', id);
    store.transition(id, 'CHARGING');
    try {
      await payment('/charges', { method: 'POST', body: { chargeId: id,
        amount: request.amount, customer: request.customer } });
    } catch (error) {
      const declined = error instanceof HttpError && error.status === 402;
      // Timeout মানে ব্যর্থতা নয়: টাকা কাটা হয়ে থাকতে পারে, তাই Refund-ও।
      return compensate(id, declined ? 'PAYMENT_DECLINED'
        : 'PAYMENT_UNCERTAIN', request, !declined);
    }
    await hooks.after?.('charged', id);
    return finish(id, 'CONFIRMED', null, request);
  }

// ...
```

Compensation উল্টো ক্রমে, এবং নিজেও ব্যর্থ হতে পারে—তখন `MANUAL_REVIEW`, একটি `booking.needs_review` Event সহ, যাতে একজন মানুষ দেখেন। Error গিলে ফেলা হয় না। শেষে `resume`, Restart-এর পরে:

orchestrator.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  // Compensation উল্টো ক্রমে: আগে Refund, তারপর আসন ছাড়া।
```

```
  async function compensate(id, reason, request, mayBeCharged) {
    store.transition(id, 'COMPENSATING', reason);
    try {
      if (mayBeCharged) {
        await payment(`/charges/${id}/refund`, { method: 'POST' });
      }
      await inventory(`/reservations/${id}`, { method: 'DELETE' });
      return finish(id, 'REJECTED', reason, request);
    } catch (error) {
      // ক্ষতিপূরণও ব্যর্থ: লুকানো নয়, মানুষের হাতে।
      return finish(id, 'MANUAL_REVIEW', `${reason}; ${error.message}`, request);
    }
  }

  // Restart-এর পরে: অসমাপ্ত প্রতিটি Saga শেষ করা।
  async function resume() {
    const results = [];
    for (const saga of store.unfinished()) {
      const work = saga.status === 'COMPENSATING'
        ? () => compensate(saga.id, saga.reason, saga.request, true)
        : () => run(saga.id, saga.request);
      results.push(await singleFlight(saga.id, work));
    }
    return results;
  }

  return { book, resume, breakers, status: view };
}
```

`resume`-এর সরলতা লক্ষ করুন: অসমাপ্ত Saga-কে শুধু শুরু থেকে আবার চালানো। এটি নিরাপদ কারণ প্রতিটি সামনের ধাপ Idempotent—আসন আগে ধরা থাকলে Inventory আগের ফল দেয়, টাকা আগে কাটা থাকলে Payment আগের ফল দেয়। Idempotency না থাকলে Crash-Recovery-র জন্য জটিল "কোন ধাপ সত্যিই হয়েছিল" তদন্ত লাগত।

src/system.mjs ও demo.mjs

`startSystem` দুটি Service আলাদা Port-এ চালু করে, Saga Store খোলে এবং Orchestrator তৈরি করে:

system.mjs · JavaScript — চলবে: Node.js

```
import { startInventory } from './services/inventory.mjs';
import { startPayment } from './services/payment.mjs';
import { openSagaStore } from './saga-store.mjs';
import { createOrchestrator } from './orchestrator.mjs';

// পুরো ব্যবস্থা এক জায়গায় চালু: দুটি আলাদা HTTP Service (প্রত্যেকের নিজস্ব
// Port ও নিজস্ব Data) এবং একটি Orchestrator, যার নিজস্ব SQLite Log।
export async function startSystem({ seats = 10, dbPath = ':memory:',
  hooks, callOptions } = {}) {
  const inventory = await startInventory({ seats });
```

```
  const payment = await startPayment();
  const store = openSagaStore(dbPath);
  const orchestrator = createOrchestrator({ store, hooks, callOptions,
    inventoryUrl: inventory.url, paymentUrl: payment.url });
  const json = (url) => fetch(url).then((res) => res.json());
  return {
    inventory, payment, store, orchestrator,
    stock: () => json(`${inventory.url}/stock`),
    ledger: () => json(`${payment.url}/ledger`),
    async close() {
      await Promise.all([inventory.close(), payment.close()]);
      store.close();
    },
  };
}
```

Demo সাতটি পরিস্থিতি চালায়। Retry-র Jitter-এ স্থির মান (`random: () => 0.5`) দেওয়া, যাতে আউটপুট প্রতিবার এক:

demo.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) — চলবে: Node.js

```
import { startSystem } from './src/system.mjs';

const system = await startSystem({ seats: 10,
  callOptions: { timeoutMs: 150, retries: 1, random: () => 0.5 } });
const { orchestrator, inventory, payment } = system;

async function attempt(label, request) {
  try {
    const result = await orchestrator.book(request);
    console.log(`${label}: ${result.status}${result.reason
      ? ` (${result.reason})` : ''}`);
  } catch (error) {
    console.log(`${label}: ${error.code}`);
  }
}

const trip = { bookingId: 'trip-101', customer: 'Rahim', seats: 2,
  amount: 1_250_000 };
await attempt('১. সাধারণ Booking', trip);
await attempt('২. একই অনুরোধ আবার', trip);
await attempt('৩. একই ID, ভিন্ন দাম', { ...trip, amount: 1 });

// ...
```

demo.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) — চলবে: Node.js

```
// ...
```

```
payment.faults.decline.add('Karim');
await attempt('8. Card প্রত্যাখ্যাত', { bookingId: 'trip-102',
  customer: 'Karim', seats: 3, amount: 900_000 });

inventory.faults.failNext = 1;
await attempt('৫. Inventory সাময়িক 503', { bookingId: 'trip-103',
  customer: 'Mina', seats: 1, amount: 450_000 });

await attempt('৬. আসন নেই', { bookingId: 'trip-104', customer: 'Rafi',
  seats: 8, amount: 100_000 });

payment.faults.delayMs = 300; // টাকা কাটে, কিন্তু উত্তর আসে Timeout-এর পরে
await attempt('৭. Payment-এর উত্তর অনিশ্চিত', { bookingId: 'trip-105',
  customer: 'Nadia', seats: 1, amount: 300_000 });
payment.faults.delayMs = 0;

console.log('Inventory:', await system.stock());
console.log('Payment:', await system.ledger());
console.log('Outbox-এর Event:');
for (const event of system.store.pendingEvents()) {
  console.log(`  ${event.saga_id} ${event.type}`);
}
console.log('trip-105-এর ইতিহাস:', system.store.history('trip-105')
  .map((h) => h.status).join(' > '));
await system.close();
```

Output

```
১. সাধারণ Booking: CONFIRMED
২. একই অনুরোধ আবার: CONFIRMED
৩. একই ID, ভিন্ন দাম: IDEMPOTENCY_CONFLICT
8. Card প্রত্যাখ্যাত: REJECTED (PAYMENT_DECLINED)
৫. Inventory সাময়িক 503: CONFIRMED
৬. আসন নেই: REJECTED (SOLD_OUT)
৭. Payment-এর উত্তর অনিশ্চিত: REJECTED (PAYMENT_UNCERTAIN)
Inventory: { available: 7, reserved: 3 }
Payment: { captured: 1700000, refunded: 300000, attempts: 4 }
Outbox-এর Event:
  trip-101 booking.confirmed
  trip-102 booking.rejected
  trip-103 booking.confirmed
  trip-104 booking.rejected
  trip-105 booking.rejected
trip-105-এর ইতিহাস: STARTED > RESERVING > CHARGING > COMPENSATING > REJECTED
```

হিসাব মিলিয়ে দেখুন। শুধু trip-101 (২ আসন) ও trip-103 (১ আসন) নিশ্চিত, তাই ১০টির ৩টি ধরা। Payment-এ ১২,৫০০ + ৪,৫০০ = ১৭,০০০ টাকা কাটা; trip-105-এর ৩,০০০ টাকা কাটা হয়েছিল (অনিশ্চিত উত্তরের পেছনে), তারপর Refund। চারটি Charge-এর চেষ্টা: 101, 102 (প্রত্যাখ্যাত), 103, 105; trip-104 আসন না পেয়ে Payment পর্যন্ত যায়ইনি। প্রতিটি শেষ

হওয়া Saga Outbox-এ একটি Event রেখেছে।

Test: Crash ও Recovery

সবচেয়ে কঠিন পরিস্থিতি: Orchestrator ঠিক টাকা কাটার পরে, কিন্তু "নিশ্চিত" লেখার আগে মারা গেল। Test `hooks.after` দিয়ে ঠিক সেই মুহূর্তে একটি Error ছুড়ে Crash অনুকরণ করে, তারপর একই Database File দিয়ে একটি নতুন Orchestrator তৈরি করে `resume` চালায়:

recovery.test.mjs · JavaScript — চলবে: Node.js

```
// Orchestrator ঠিক ভুল মুহূর্তে মারা গেলে: টেকসই Log থেকে Restart-এর পরে শেষ করা।
import test from 'node:test';
import assert from 'node:assert/strict';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { startSystem } from '../src/system.mjs';
import { openSagaStore } from '../src/saga-store.mjs';
import { createOrchestrator } from '../src/orchestrator.mjs';

class Crash extends Error {}

for (const crashPoint of ['reserved', 'charged']) {
  test(`'${crashPoint}'-এর পরে Crash, Restart-এ resume: একবারই কাটা`,
    async (t) => {
      const dir = mkdtempSync(join(tmpdir(), 'saga-'));
      const dbPath = join(dir, 'saga.db');
      const sys = await startSystem({ seats: 5, dbPath,
        hooks: { after: (step) => {
          if (step === crashPoint) throw new Crash(step);
        } } });
      t.after(async () => {
        await sys.close();
        rmSync(dir, { recursive: true, force: true });
      });
      const request = { bookingId: 'trip-9', customer: 'Rahim', seats: 2,
        amount: 800_000 };
      await assert.rejects(sys.orchestrator.book(request), Crash);
      const before = sys.store.get('trip-9').status;
      assert.equal(before, crashPoint === 'reserved' ? 'RESERVING'
        : 'CHARGING');

      // "নতুন Process": একই Database File, নতুন Orchestrator, কোনো Hook নেই।
      const store = openSagaStore(dbPath);
      const restarted = createOrchestrator({ store,
        inventoryUrl: sys.inventory.url, paymentUrl: sys.payment.url });
      const [result] = await restarted.resume();
      assert.equal(result.status, 'CONFIRMED');
      assert.deepEqual(await sys.ledger(),
```

```
        { captured: 800_000, refunded: 0, attempts: 1 });
      assert.deepEqual(await sys.stock(), { available: 3, reserved: 2 });
      assert.deepEqual(await restarted.resume(), []); // আর কিছু বাকি নেই
      store.close();
    });
}
```

দুটি Crash-বিন্দুতেই ফল এক: Booking নিশ্চিত, টাকা ঠিক একবার কাটা (`attempts: 1`), আসন ঠিক একবার ধরা। দ্বিতীয় `resume` কিছুই করে না।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-saga
npm test
npm run demo
```

`npm test` ১৪টি Test চালায়: `participants` ২ (দুটি Service-এর Idempotency, Conflict, প্রত্যাখ্যান ও Refund), `http-client` ৪ (Retry, 4xx-এ Retry নয়, Timeout, Circuit Breaker-এর চার অবস্থা), `saga` ৬ (সফল পথ ও Outbox, প্রত্যাখ্যান, আসন নেই, একসঙ্গে পাঁচটি Duplicate ও Conflict, অনিশ্চিত Payment, Compensation ব্যর্থ), এবং `recovery` ২ (দুটি Crash-বিন্দু):

Output

```
# tests 14
# pass 14
# fail 0
```

Test লেখার সময় একটি শিক্ষণীয় ব্যর্থতা: প্রথম চালনায় Saga-র ছয়টি Test-ই "ভুল Field: bookingId" দিয়ে ব্যর্থ হয়েছিল। কারণ Code-এ নয়, Test-এ— `'t1'`-এর মতো ID দুই অক্ষরের, অথচ Validation কমপক্ষে তিন অক্ষর চায়। Validation ঠিক কাজ করছিল; Test-এর ID বদলানো হয়েছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই Booking-এ দুবার টাকা	অংশগ্রহণকারী Idempotent নয়, বা প্রতিবার নতুন chargeId	chargeId = bookingId
IDEMPOTENCY_CONFLICT অপ্রত্যাশিত	Client Retry-তে Body বদলাচ্ছে (যেমন সময় যোগ)	Retry-তে হুবহু একই Body
Saga RESERVING বা CHARGING-এ আটকে	Orchestrator মাঝপথে বন্ধ হয়েছিল	চালুর সময় `resume()`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সব ডাক CIRCUIT_OPEN	Dependency বারবার ব্যর্থ	Dependency-র Health; `resetMs` পরে Half-open
MANUAL_REVIEW জমছে	Compensation ব্যর্থ	`saga_log` দেখে হাতে মিলানো; কারণ ঠিক করা
হঠাৎ Payment-এ অনেক বেশি অনুরোধ	একাধিক স্তরে Retry	Retry এক স্তরে, Breaker

২৯.১৩ Failure Lab

README-র সারণির প্রতিটি পরীক্ষা নিজে চালান এবং ফল ব্যাখ্যা করুন:

১. `payment.faults.decline.add('Karim')` দিয়ে প্রত্যাখ্যান; Inventory-র `stock` আগের মতো কি না দেখুন।
২. আসনের চেয়ে বেশি চেয়ে Sold-out; Payment-এর `ledger`-এ কোনো চেষ্টা নেই তো?
৩. একই Booking দুবার পাঠিয়ে আসনের সংখ্যা অপরিবর্তিত কি না দেখুন।
৪. একই ID কিন্তু ভিন্ন `amount` —IDEMPOTENCY_CONFLICT।
৫. `Promise.all` দিয়ে একসঙ্গে পাঁচটি Duplicate।
৬. Compensation-এর মাঝে Inventory বন্ধ করে MANUAL_REVIEW দেখুন, এবং ভাবুন একজন Operator কী তথ্য পেলে তা হাতে মিলাতে পারবেন।

২৯.১৪ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Timeout-কে নিশ্চিত ব্যর্থতা ধরে আবার টাকা কাটা	দ্বিগুণ Charge	Idempotent chargeId; অনিশ্চিত হলে Refund বা যাচাই
Microservices মানেই Scalability ভাবা	Network-এর খরচ, জটিলতা	প্রমাণিত প্রয়োজনে, Modular Monolith আগে
অন্য Service-এর Table সরাসরি বদলানো	নিয়ম পাশ কাটানো, Schema আটকে যায়	শুধু API বা Event
At-least-once Event-এ Idempotent নয় এমন Consumer	একই কাজ বারবার	Inbox, UNIQUE
Outbox ছাড়া অবস্থা-বদলের পরে Event প্রকাশ	মাঝে Crash হলে অমিল	একই Transaction-এ Outbox

ভুল	কেন সমস্যা	সমাধান
প্রতিটি স্তরে Retry, Jitter নেই	Retry Storm	এক স্তরে Retry, Backoff + Jitter, Breaker
Compensation ব্যর্থতা গিলে ফেলা	অদৃশ্য অমিল, টাকা আটকে	MANUAL_REVIEW, Alert
Trace Context না পাঠানো	কোন Service-এ দেরি, বোঝা যায় না	`traceparent`, OpenTelemetry
Log-এ Card বা ব্যক্তিগত তথ্য	গোপনীয়তা ভঙ্গ	Redaction, Data কমানো
না মেপে SLA প্রতিশ্রুতি	রাখা যায় না এমন কথা	আগে মাপা, তারপর SLO, তারপর SLA

২৯.১৫ Debugging নির্দেশিকা

১. **Saga Log আগে:** `history(bookingId)`—কোন ধাপ পর্যন্ত গেছে, কোন কারণে থেমেছে।
২. **প্রতিটি অংশগ্রহণকারীর নিজের দৃষ্টি:** Inventory-র Reservation ও Payment-এর Ledger—Saga যা বলছে তার সঙ্গে মিলছে?
৩. **অনিশ্চিত অবস্থা খুঁজুন:** COMPENSATING বা CHARGING-এ পুরোনো Saga মানে Resume চলেনি।
৪. **Trace ID:** একই অনুরোধ কোন Service-এ কত সময় নিল।
৫. **Breaker-এর অবস্থা ও Retry-র সংখ্যা Metrics-এ:** হঠাৎ বাড়লে Dependency অসুস্থ।
৬. **নিয়মিত Reconciliation:** দিনের শেষে Payment Gateway-এর তালিকা ও নিজের Booking মিলিয়ে দেখা—যে অমিল কোনো Alert ধরেনি, তা এখানে ধরা পড়ে।

২৯.১৬ পেশাদার অভ্যাস

- **সীমানা ব্যবসা ধরে, Data-র একজন মালিক।**
- **প্রতিটি Network ডাকে Timeout; Retry শুধু Idempotent কাজে, এক স্তরে।**
- **প্রতিটি অবস্থা-বদলানো API Idempotent, Key + Fingerprint-সহ।**
- **Saga-র প্রতিটি ধাপের আগে টেকসই Log; চালুর সময় Resume।**
- **ফেরানো সহজ কাজ আগে, ফেরানো যায় না এমন কাজ সবশেষে।**
- **Compensation-এর ব্যর্থতা মানুষের কাছে পৌঁছায়।**
- **Architecture-এর সিদ্ধান্ত লিখে রাখুন (ADR):** কী বেছেছেন, কোন বিকল্প, কেন।
- **Architecture Review-এর Checklist:** Requirement ও Traffic-এর অনুমান লিখিত; সীমানা ও Data-র মালিকানা; Auth ও Secret; Idempotency, Retry, Timeout ও Compensation; Event-এর Schema, Outbox ও Inbox; Monitoring, Alert, Trace ও Runbook; Backup, DR ও Reconciliation; Load, Integration, Chaos ও Security Test-এর পরিকল্পনা।

২৯.১৭ অনুশীলনী

প্রাথমিক স্তর

১. দিনে ৪,৩২,০০০ অনুরোধ, Peak গড়ের ১০ গুণ, প্রতিটি ২০০ ms। গড় ও Peak RPS, একসঙ্গে চলমান অনুরোধ, এবং প্রতি Instance ২০টি সামলালে কতগুলো Instance (একটি অতিরিক্তসহ)?

২. Modular Monolith ও Microservices-এর তিনটি Trade-off লিখুন।

৩. Booking, Payment ও Inventory—প্রতিটি Bounded Context কোন Data-র মালিক, আর কোন শব্দ তার নিজস্ব অর্থে ব্যবহার করে?

৪. TravelSaga-তে Card প্রত্যাখ্যাত হলে Saga Log-এ কোন অবস্থাগুলো ক্রমে আসবে? অনিশ্চিত Payment-এ কোনটি আলাদা?

মধ্যম স্তর

৫. Key ও Fingerprint দিয়ে একটি Idempotent Handler লিখুন: একই অনুরোধে আগের ফল, ভিন্ন Body-তে 409।

৬. তিন স্তরে প্রতিটি নিজে তিনবার Retry করলে একটি অনুরোধ নিচের Service-এ কতগুলো ডাক তৈরি করে? সমাধান কী?

৭. `booking.confirmed` Event-এর জন্য একটি Inbox-ভিত্তিক Consumer লিখুন, যা Loyalty Point একবারই যোগ করে।

৮. Inventory কেন দৃঢ় সমন্বয় চায়, অথচ Notification Eventual Consistency মেনে নিতে পারে? একটি করে ব্যর্থতার উদাহরণ দিন।

চ্যালেঞ্জ স্তর

৯. উদাহরণ ০৩-এর Choreography-তে "Payment অনিশ্চিত" পথ যোগ করার নকশা করুন। Orchestration-এর তুলনায় কী কঠিন হলো?

১০. একটি Bulkhead লিখুন: প্রতি Dependency-তে সর্বোচ্চ তিনটি একসঙ্গে ডাক, বাকিরা সঙ্গে সঙ্গে প্রত্যাখ্যাত।

১১. TravelSaga-র Outbox-এর জন্য একটি Relay লিখুন: ক্রম রক্ষা করে প্রকাশ, ব্যর্থ হলে পরের দফায়।

১২. TravelSaga-কে Production-এ নেওয়ার একটি Architecture প্রস্তাব লিখুন।

২৯.১৮ অনুশীলনীর সমাধান

সমাধান ১

capacity-answer.mjs · JavaScript — চলবে: Node.js

```
// ৪,৩২,০০০ দৈনিক অনুরোধ; Peak গড়ের ১০ গুণ; প্রতি অনুরোধ ২০০ ms।
const daily = 432_000;
const avg = daily / 86_400;
const peak = avg * 10;
const concurrent = peak * 0.2; // Little's Law

console.log(`গড়: ${avg} RPS, Peak: ${peak} RPS`);
console.log(`একসঙ্গে চলমান অনুরোধ (Peak): ${concurrent}`);
// প্রতি Instance নিরাপদে ২০টি একসঙ্গে সামলায়; একটি নষ্ট হলেও চলতে হবে (N+1)
const perInstance = 20;
const instances = Math.ceil(concurrent / perInstance) + 1;
console.log(`Instance: ${instances} (একটি অতিরিক্ত, ব্যর্থতার জন্য)`);
```

Output

```
গড়: 5 RPS, Peak: 50 RPS
একসঙ্গে চলমান অনুরোধ (Peak): 10
Instance: 2 (একটি অতিরিক্ত, ব্যর্থতার জন্য)
```

একসঙ্গে ১০টি অনুরোধ একটি Instance-ই সামলাতে পারে (২০ সীমা), কিন্তু একটি নষ্ট হলে সেবা বন্ধ—তাই N+1 নিয়মে দুটি। Deploy-এর সময় একটি বন্ধ থাকে, সেটিও মনে রাখুন।

সমাধান ২ — (১) **Deploy:** Microservices-এ প্রতিটি দল স্বাধীনভাবে Release করে; বিনিময়ে অনেক Pipeline, সংস্করণের সামঞ্জস্য ও Operations। (২) **Scaling:** শুধু ব্যস্ত অংশটি (যেমন Search) আলাদাভাবে বাড়ানো যায়; বিনিময়ে প্রতিটি ডাক Network-এ—Latency ও ব্যর্থতার নতুন উৎস। (৩) **Data:** Modular Monolith-এ এক Transaction-এ একাধিক Module-এর পরিবর্তন সহজ; Microservices-এ Saga, Compensation ও Eventual Consistency।

সমাধান ৩

Context	মালিকানার Data	নিজস্ব ভাষা
Booking	Booking, তার অবস্থা ও ইতিহাস	"নিশ্চিত", "প্রত্যাখ্যাত", "অনুরোধ"
Payment	Charge, Refund, Ledger	"Captured", "Declined", "Refunded"
Inventory	আসন, Reservation	"Available", "Reserved", "Released"

Booking-এর "নিশ্চিত" ঘটে কেবল Payment-এর "Captured" ও Inventory-র "Reserved"—দুটির পরে; তিনটি শব্দ তিন Context-এর, এবং কোনোটিই অন্যের Table-এ লেখা হয় না।

সমাধান ৪ — প্রত্যাখ্যানে: STARTED, RESERVING, CHARGING, COMPENSATING (কারণ PAYMENT_DECLINED), REJECTED। অনিশ্চিত Payment-এ অবস্থার ক্রম একই, কিন্তু কারণ PAYMENT_UNCERTAIN, এবং COMPENSATING-এর সময় আসন ছাড়ার আগে একটি Refund-ও পাঠানো হয়—কারণ টাকা কাটা হয়ে থাকতে পারে। প্রত্যাখ্যানে Refund পাঠানো হয় না, কারণ 402 নিশ্চিত করে টাকা কাটা হয়নি (`compensate`-এর `mayBeCharged`)।

সমাধান ৫

idempotency.mjs · JavaScript চলবে: Node.js

```
import { createHash } from 'node:crypto';

// Key ও Fingerprint একসঙ্গে: Key বলে "কোন অনুরোধ", Fingerprint বলে "কী চাওয়া"।
const sha = (value) => createHash('sha256')
  .update(JSON.stringify(value)).digest('hex');
const saved = new Map(); // আসলে: Database Table, Key-তে UNIQUE

function handle(key, body, work) {
  const fingerprint = sha(body);
  const old = saved.get(key);
  if (old && old.fingerprint !== fingerprint) {
    return { status: 409, body: { error: 'IDEMPOTENCY_CONFLICT' } };
  }
  if (old) return { ...old.response, replay: true };
  const response = work(body);
  saved.set(key, { fingerprint, response });
  return response;
}

let charges = 0;
const charge = (body) => {
  charges += 1;
  return { status: 201, body: { charged: body.amount } };
};

console.log(handle('k-1', { amount: 500 }, charge));
console.log(handle('k-1', { amount: 500 }, charge));
console.log(handle('k-1', { amount: 900 }, charge));
console.log('আসলে টাকা কাটা হয়েছে:', charges, 'বার');
```

Output

```
{ status: 201, body: { charged: 500 } }
{ status: 201, body: { charged: 500 }, replay: true }
{ status: 409, body: { error: 'IDEMPOTENCY_CONFLICT' } }
আসলে টাকা কাটা হয়েছে: 1 বার
```

Production-এ `saved` একটি Database Table, Key-তে UNIQUE, আর কাজ ও সারি একই Transaction-এ; নইলে দুটি Replica একই মুহূর্তে "আগে নেই" দেখে দুজনেই কাজ করবে।

সমাধান ৬

retry-storm.mjs · JavaScript — চলবে: Node.js

```javascript
// তিন স্তর (Gateway, Booking, Payment Client), প্রতিটি নিজে ৩ বার চেষ্টা করে।
// নিচের Service পুরো বন্ধ থাকলে একটি ব্যবহারকারী-অনুরোধ কতগুলো ডাক তৈরি করে?
function callsToBottom(layers, attemptsPerLayer) {
  return attemptsPerLayer ** layers;
}

for (const attempts of [1, 2, 3]) {
  const calls = callsToBottom(3, attempts);
  console.log(`প্রতি স্তরে ${attempts} চেষ্টা: ${calls}টি ডাক`);
}
// সমাধান: Retry শুধু এক স্তরে (সাধারণত নিচের Client-এ), বাকিরা দ্রুত ব্যর্থ।
console.log('শুধু নিচের স্তরে ৩ চেষ্টা:', 1 * 1 * 3, 'টি ডাক');
// আর Circuit Breaker খুললে পরের অনুরোধগুলো নিচে পৌঁছায়ই না।
const peakRps = 200;
console.log('৩^৩ হলে বন্ধ Service-এ অনুরোধ/সেকেন্ড:', peakRps * 27);
```

Output

```
প্রতি স্তরে 1 চেষ্টা: 1টি ডাক
প্রতি স্তরে 2 চেষ্টা: ৪টি ডাক
প্রতি স্তরে 3 চেষ্টা: 27টি ডাক
শুধু নিচের স্তরে ৩ চেষ্টা: 3 টি ডাক
৩^৩ হলে বন্ধ Service-এ অনুরোধ/সেকেন্ড: 5400
```

গুণফল সূচকীয়: সেকেন্ডে ২০০ অনুরোধ একটি অসুস্থ Service-এ ৫,৪০০ হয়ে আঘাত করে—সেরে ওঠার কোনো সুযোগ নেই। সমাধান: Retry শুধু এক স্তরে; ওপরের স্তরগুলো দ্রুত ব্যর্থ; Circuit Breaker; এবং একটি **Retry Budget** (মোট অনুরোধের, যেমন, ১০%-এর বেশি Retry নয়)।

সমাধান ৭

inbox.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// Inbox: Consumer-এর দিকে Deduplication। Event-এর ID-তে PRIMARY KEY, আর
// কাজ ও Inbox-এর সারি একই Transaction-এ; তাই Duplicate কখনো দুবার নয়।
const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE inbox (event_id TEXT PRIMARY KEY);
  CREATE TABLE loyalty (customer TEXT PRIMARY KEY, points INTEGER);
  INSERT INTO loyalty VALUES ('Rahim', 0);`);

function onBookingConfirmed(event) {
  db.exec('BEGIN');
  try {
    db.prepare('INSERT INTO inbox VALUES (?)').run(event.id);
    db.prepare('UPDATE loyalty SET points = points + ? WHERE customer = ?')
      .run(event.seats * 100, event.customer);
    db.exec('COMMIT');
    return 'processed';
  } catch (error) {
    db.exec('ROLLBACK');
    if (error.code === 'ERR_SQLITE_ERROR' && /UNIQUE/.test(error.message)) {
      return 'duplicate';
    }
    throw error;
  }
}

const event = { id: 'evt-trip-101', customer: 'Rahim', seats: 2 };
for (const delivery of [1, 2, 3]) {
  console.log(`Delivery ${delivery}:`, onBookingConfirmed(event));
}
console.log({ ...db.prepare('SELECT * FROM loyalty').get() });
```

Output

```
Delivery 1: processed
Delivery 2: duplicate
Delivery 3: duplicate
{ customer: 'Rahim', points: 200 }
```

Inbox-এর সারি ও Point যোগ একই Transaction-এ: দ্বিতীয় Delivery-তে PRIMARY KEY ভঙ্গ হয়, Transaction Rollback হয়, Point দ্বিতীয়বার যোগ হয় না। শুধু "আগে দেখেছি কি" যাচাই করে তারপর আলাদাভাবে লিখলে দুটি Consumer-এর মাঝে Race থাকত।

সমাধান ৮ — Inventory: শেষ আসন দুজনের কাছে বিক্রি হলে একজন গ্রাহক বিমানবন্দরে গিয়ে আসন পান না—টাকা ফেরত, ক্ষতিপূরণ, সুনামহানি (উদাহরণ ০৪-এর AP Overbooking)। তাই এক মালিক, Transaction বা Lock, এবং Partition-এ প্রত্যাখ্যান মেনে নেওয়া। Notification: নিশ্চিতকরণের SMS দশ সেকেন্ড বা এক মিনিট দেরিতে গেলে কোনো ব্যবসায়িক নিয়ম ভাঙে না; ব্যর্থ হলে Retry করা যায়। তাই At-least-once Event ও Idempotent Consumer যথেষ্ট—দৃঢ় সমন্বয়ের খরচ অপ্রয়োজনীয়।

সমাধান ৯ — Payment Service অনিশ্চিত উত্তরে `payment.uncertain` Event ছাড়বে। কিন্তু কে Refund করবে? Payment নিজেই (সে জানে কী কাটা হয়েছে), তারপর `payment.refunded`; Inventory শুনবে `payment.refunded` ও `payment.failed` দুটিই, এবং আসন ছেড়ে `seats.released`; Booking শুনবে `seats.released` এবং প্রত্যাখ্যান লিখবে। কঠিন যা হলো: (১) প্রবাহ এখন চারটি Handler-এ ছড়ানো, কোথাও এক জায়গায় লেখা নেই; (২) "Refund-ও ব্যর্থ হলে কী" প্রশ্নের মালিক কেউ নয়; (৩) একটি Event হারালে Booking চিরকাল অসমাপ্ত, আর তা ধরতে আলাদা Timeout-পর্যবেক্ষক লাগে। Orchestrator-এ এই তিনটিই এক Function-এ (`compensate`) ও এক Log-এ।

সমাধান ১০

bulkhead.mjs · JavaScript — চলবে: Node.js

```
// Bulkhead: প্রতিটি Dependency-র জন্য আলাদা সীমিত "কক্ষ"। ধীর Payment
// সব সংযোগ দখল করলেও Inventory-র ডাক আটকে যায় না।
function createBulkhead(limit) {
  let active = 0;
  let rejected = 0;
  return {
    async run(task) {
      if (active >= limit) {
        rejected += 1;
        throw new Error('BULKHEAD_FULL');
      }
      active += 1;
      try {
        return await task();
      } finally {
        active -= 1;
      }
    },
    stats: () => ({ active, rejected }),
  };
}

const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const payment = createBulkhead(3);
const inventory = createBulkhead(3);
```

```
// Payment ধীর: ১০টি ডাক একসঙ্গে, প্রতিটি ১০০ ms।
const slow = Array.from({ length: 10 }, () =>
  payment.run(() => wait(100)).catch((e) => e.message));
const fast = await inventory.run(async () => 'Inventory: উত্তর এল');
console.log(fast);
const results = await Promise.all(slow);
console.log('Payment:', results.filter((r) => r === 'BULKHEAD_FULL').length,
  'টি সঙ্গে সঙ্গে প্রত্যাখ্যাত,', results.filter((r) => r === undefined)
    .length, 'টি সম্পন্ন');
console.log(payment.stats());
```

Output

```
Inventory: উত্তর এল
Payment: 7 টি সঙ্গে সঙ্গে প্রত্যাখ্যাত, 3 টি সম্পন্ন
{ active: 0, rejected: 7 }
```

ধীর Payment-এর দশটি ডাকের তিনটি চলে, সাতটি সঙ্গে সঙ্গে প্রত্যাখ্যাত (দীর্ঘ অপেক্ষার বদলে দ্রুত ব্যর্থতা), আর Inventory-র ডাক নিজের কক্ষে বাধাহীন। বাস্তবে প্রত্যাখ্যানের বদলে একটি ছোট, সীমিত Queue-ও রাখা যায়।

সমাধান ১১

outbox-relay.mjs · JavaScript — চলবে: Node.js

```
import { openSagaStore } from '../../project/travel-saga/src/saga-store.mjs';

// Relay: Outbox থেকে Event নিয়ে Broker-এ; সফল হলে তবেই published।
async function relayOnce(store, publish) {
  let sent = 0;
  for (const event of store.pendingEvents()) {
    try {
      await publish(event);
      store.markPublished(event.id);
      sent += 1;
    } catch {
      break; // ক্রম রক্ষা: ব্যর্থ হলে পরেরগুলো পরের দফায়
    }
  }
  return sent;
}

const store = openSagaStore();
const request = { bookingId: 'trip-1', customer: 'Rahim', seats: 2,
  amount: 100 };
for (const id of ['trip-1', 'trip-2', 'trip-3']) {
  store.start(id, 'fp', { ...request, bookingId: id });
  store.transition(id, 'CONFIRMED', null, { type: 'booking.confirmed',
    payload: { bookingId: id } });
```

```
}

let brokerDown = true;
const broker = [];
const publish = async (event) => {
  if (brokerDown && event.saga_id === 'trip-2') throw new Error('down');
  broker.push(event.saga_id);
};
console.log('দফা ১:', await relayOnce(store, publish), 'টি পাঠানো');
brokerDown = false;
console.log('দফা ২:', await relayOnce(store, publish), 'টি পাঠানো');
console.log('Broker পেল:', broker.join(', '));
console.log('বাকি:', store.pendingEvents().length);
```

Output

```
দফা ১: 1 টি পাঠানো
দফা ২: 2 টি পাঠানো
Broker পেল: trip-1, trip-2, trip-3
বাকি: 0
```

প্রথম দফায় trip-2-এ ব্যর্থতা, তাই trip-3-ও অপেক্ষা করে—ক্রম রক্ষা। এর মূল্য: একটি "বিষাক্ত" Event পুরো Queue আটকে দিতে পারে; তাই কয়েকবার ব্যর্থতার পর Dead-letter-এ সরানো (অধ্যায় ২৭-এর সমাধান ৯)। ক্রম যদি শুধু একই Booking-এর মধ্যে জরুরি হয়, তবে প্রতিটি `saga_id` আলাদা Queue হিসেবে চালানো যায়।

সমাধান ১২ — (১) **Process:** Inventory, Payment-Adapter ও Orchestrator আলাদা Container, প্রত্যেকের নিজস্ব Database (PostgreSQL)। (২) **Payment:** আসল Gateway-এর Idempotency-Key ব্যবহার; অনিশ্চিত উত্তরে আগে Gateway-কে জিজ্ঞাসা ("এই Key-র Charge আছে?"), তারপর সিদ্ধান্ত। (৩) **Saga Log:** PostgreSQL-এ; একাধিক Orchestrator-এর মধ্যে একই Saga দুবার না চলে তাই Row Lock বা Lease (`locked_until`)। (৪) **Resume:** একটি Background Worker নিয়মিত পুরোনো অসমাপ্ত Saga খোঁজে। (৫) **Outbox Relay:** আলাদা Worker, Broker-এ (যেমন RabbitMQ বা Kafka) প্রকাশ; Consumer-রা Inbox-সহ। (৬) **Reconciliation:** প্রতিদিন Gateway-এর Settlement File ও নিজের Ledger মিলানো। (৭) **পর্যবেক্ষণ:** প্রতিটি অবস্থায় Saga-র সংখ্যা, অবস্থায় কাটানো সময়, MANUAL_REVIEW-এর হার—Alert-সহ; Trace ID সব Service-এ। (৮) **নিরাপত্তা:** Service-এর মধ্যে mTLS, Payment-এর Secret Secret Store-এ, Card-এর তথ্য কখনো নিজের ব্যবস্থায় নয় (Gateway-এর Token)।

২৯.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Saga কী?

ক) একটি CSS Library

খ) স্থানীয় Transaction-এর ধারা, ব্যর্থতায় Compensation-সহ

গ) একটি Encryption পদ্ধতি

ঘ) Browser-এর Storage

প্রশ্ন ২. একই Idempotency Key কিন্তু ভিন্ন Body এলে সঠিক উত্তর কী?

ক) নতুন কাজ করা

খ) আগের ফল নীরবে ফেরত দেওয়া

গ) 409 Conflict

ঘ) 500 Error

প্রশ্ন ৩. Payment-এর ডাকে Timeout হলে TravelSaga কী ধরে নেয়?

ক) টাকা কাটা হয়নি

খ) টাকা কাটা হয়েছে, Booking নিশ্চিত

গ) কিছুই না, চুপচাপ অপেক্ষা

ঘ) অনিশ্চিত: Refund ও আসন ছাড়া

প্রশ্ন ৪. Circuit Breaker খোলা থাকলে কী ঘটে?

ক) ডাক Network-এ না গিয়েই দ্রুত ব্যর্থ হয়

খ) ডাক দ্বিগুণ দ্রুত যায়

গ) Service Restart হয়

ঘ) Retry-এর সংখ্যা বাড়ে

প্রশ্ন ৫. কোন HTTP উত্তরে Retry করা উচিত নয়?

ক) 503

খ) 402 (Card প্রত্যাখ্যাত)

গ) Timeout

ঘ) 429

প্রশ্ন ৬. Network Partition চলাকালে শেষ আসন বিক্রির জন্য সাধারণত কোনটি নিরাপদ?

ক) দুই Replica-ই বিক্রি করুক (AP)

খ) পরে দেখা যাবে

গ) সমন্বয় না হলে প্রত্যাখ্যান (CP)

ঘ) এলোমেলো একটি

প্রশ্ন ৭. TravelSaga-তে Crash-এর পরে অসমাপ্ত Saga শুরু থেকে আবার চালানো নিরাপদ কেন?

ক) অংশগ্রহণকারী Service-গুলো reservationId ও chargeId দিয়ে Idempotent

খ) Crash কখনো ঘটে না

গ) SQLite নিজেই Undo করে

ঘ) Payment সব সময় ব্যর্থ হয়

প্রশ্ন ৮. তিন স্তরের প্রতিটি নিজে তিনবার চেষ্টা করলে নিচের Service-এ একটি অনুরোধ কতগুলো ডাক হতে পারে?

ক) ৩

খ) ৯

গ) ৬

ঘ) ২৭

প্রশ্ন ৯. Bounded Context কী নির্দেশ করে?

ক) একটি TCP Port

খ) একটি ব্যবসায়িক এলাকা, তার নিজস্ব ভাষা, নিয়ম ও Data

গ) একটি Git Branch

ঘ) Memory-র আকার

প্রশ্ন ১০. Compensation-ও ব্যর্থ হলে সঠিক আচরণ কী?

ক) Error লুকিয়ে রাখা

খ) Booking নিশ্চিত দেখানো

গ) MANUAL_REVIEW-এ রেখে মানুষকে জানানো

ঘ) পুরো Database মুছে ফেলা

২৯.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	প্রতিটি ধাপ নিজের Service-এ; ব্যর্থতায় আগের ধাপের যুক্তিসংগত উল্টো কাজ।
২	গ	একই Key-তে ভিন্ন চাওয়া Client-এর Bug; উদাহরণ ০৫ ও সমাধান ৫।
৩	ঘ	Timeout মানে "জানা নেই"; Demo-র trip-105-এ টাকা কাটা হয়েছিল, পরে Refund।
৪	ক	Test-এ খোলা অবস্থায় Service-এ ডাকের সংখ্যা বাড়েনি।
৫	খ	প্রত্যাখ্যান একটি ব্যবসায়িক উত্তর; আবার চেষ্টায় বদলায় না।
৬	গ	উদাহরণ ০৪: AP-তে Overbooking, CP-তে কেউ পায়নি কিন্তু ভুলও হয়নি।
৭	ক	আগে হয়ে থাকা ধাপ আবার ডাকলে আগের ফল আসে; Recovery Test-এ attempts ১।
৮	ঘ	৩ × ৩ × ৩ = ২৭; সমাধান ৬।
৯	খ	যেমন Payment-এর "Captured" আর Booking-এর "নিশ্চিত" আলাদা ধারণা।
১০	গ	`booking.needs_review` Event ও Saga Log মানুষকে তথ্য দেয়।

২৯.২১ অধ্যায়ের সারসংক্ষেপ

- System Design শুরু হয় Functional ও মাপযোগ্য Non-functional Requirement দিয়ে; Capacity-র হিসাবে Peak ও Little's Law।
- Modular Monolith প্রায়ই সঠিক শুরু; Microservices একটি মূল্য, যা নির্দিষ্ট প্রয়োজনে দেওয়া হয়।
- Bounded Context ব্যবসা ধরে; প্রতিটি Data-র একজন মালিক, অন্যরা শুধু API বা Event দিয়ে।
- Synchronous সরল কিন্তু যুক্ত; Asynchronous বিচ্ছিন্ন কিন্তু জটিল।
- Saga: স্থানীয় ধাপ ও Compensation; ফেরানো সহজ কাজ আগে; Orchestration বনাম Choreography।
- Idempotency: Key + Fingerprint, এবং প্রতিটি অংশগ্রহণকারী Idempotent—তাতেই Crash-Recovery সরল।
- Timeout মানে অনিশ্চিত, ব্যর্থতা নয়।
- CAP: Partition-এ Consistency বা Availability; দুর্লভ সম্পদে দৃঢ় সমন্বয়, বাকিতে Eventual।
- Timeout, এক-স্তরের Retry, Circuit Breaker, Bulkhead, Outbox, Inbox, Dead-letter।
- টেকসই Saga Log, Resume, MANUAL_REVIEW ও Reconciliation।

পরের ও শেষ অধ্যায়ে এই বইয়ের সব দক্ষতা এক জায়গায় আসবে: একটি পূর্ণাঙ্গ Full-Stack Capstone—TravelMaster Pro—যেখানে Authentication, Validation, Database Transaction, Idempotency, Test, Browser Client ও Deployment-এর প্রস্তুতি একসঙ্গে, আর তার সঙ্গে একটি পেশাদার Career Roadmap।

২৯.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Functional / Non-functional Requirement	কী করবে / কতটা ভালোভাবে করবে।
Little's Law	একসঙ্গে চলমান = আগমনের হার x গড় সময়।
Modular Monolith / Microservices	একটি Deploy, ভেতরে স্পষ্ট Module-সীমানা / আলাদাভাবে Deploy ও Scale-যোগ্য Service।
Bounded Context	নিজস্ব ভাষা ও নিয়মের ব্যবসায়িক এলাকা।
Saga / Compensation	স্থানীয় Transaction-এর ধারা / আগের ধাপের যুক্তিসংগত উল্টো কাজ।
Orchestration / Choreography	কেন্দ্রীয় সমন্বয় / Event-নির্ভর স্বতন্ত্র প্রতিক্রিয়া।
Fingerprint	অনুরোধের বিষয়বস্তুর Hash।
CAP / Eventual Consistency	Partition-এ Consistency ও Availability-র মধ্যে বেছে নেওয়া / সময়ের সঙ্গে সব Replica এক অবস্থায় পৌঁছানো।
Circuit Breaker / Bulkhead	বারবার ব্যর্থ Dependency-কে সাময়িক বিশ্রাম / প্রতিটি Dependency-র জন্য আলাদা সীমিত সম্পদ।
Inbox	Consumer-এর দিকে Event-এর Deduplication।
Reconciliation	দুটি ব্যবস্থার হিসাব নিয়মিত মিলিয়ে দেখা।

২৯.২৩ আরও পড়ুন

- **Microservices.io — Saga pattern:** `https://microservices.io/patterns/data/saga.html` — Orchestration ও Choreography।
- **Microsoft Azure Architecture Center — Saga:** `https://learn.microsoft.com/en-us/azure/architecture/patterns/saga` — Compensation-এর নকশা।
- **Martin Fowler — Microservices:** `https://martinfowler.com/articles/microservices.html` — মূল ধারণা ও মূল্য।
- **Martin Fowler — MonolithFirst:** `https://martinfowler.com/bliki/MonolithFirst.html` — কেন Monolith দিয়ে শুরু।
- **AWS Builders' Library:** `https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/` ও `https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/` — Timeout, Jitter ও নিরাপদ Retry।
- **Stripe — Idempotent requests:** `https://docs.stripe.com/api/idempotent_requests` — একটি আসল Payment API-র Idempotency।

অধ্যায় ৩০

৩০

JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি ব্যবসার প্রয়োজন থেকে Functional ও Non-functional Requirement, API চুক্তি এবং বাস্তবায়নের পরিকল্পনা তৈরি করতে।
- Frontend, Backend, Database ও নিরাপত্তা একসঙ্গে ভেবে একটি পূর্ণাঙ্গ Application তৈরি করতে।
- Trust Boundary-তে Input যাচাই, নিরাপদ Password ও Session, Server-নির্ধারিত দাম, Idempotency এবং Transaction-ভিত্তিক Inventory বাস্তবায়ন করতে।
- প্রতিটি Requirement-কে Code ও Test-এর প্রমাণের সঙ্গে যুক্ত করতে (Traceability)।
- Unit, Integration, Concurrency, Browser End-to-End Test এবং Type Check দিয়ে একটি Quality Gate সাজাতে।
- Architecture Decision Record (ADR) লিখতে এবং Production-এর জন্য একটি সৎ উন্নয়ন-পরিকল্পনা তৈরি করতে।
- ১০০ নম্বরের মানদণ্ডে নিজের কাজ মূল্যায়ন করতে।
- ১২ সপ্তাহের একটি পেশাদার Roadmap, Portfolio ও Interview-এর প্রস্তুতি পরিকল্পনা করতে।
- **TravelMaster Pro** নামে একটি পূর্ণাঙ্গ, পরীক্ষিত Full-Stack Capstone তৈরি করতে।

পূর্বশর্ত

এই অধ্যায় পুরো বইয়ের ওপর দাঁড়িয়ে। বিশেষভাবে কাজে লাগবে:

- অধ্যায় ১৭: Form ও DOM Event; অধ্যায় ২০: Authentication, Password Hash, Security Header।
- অধ্যায় ২২: Test-এর স্তর; অধ্যায় ২৪: TypeScript ও JSDoc Type।
- অধ্যায় ২৫: HTTP API-র নকশা ও Error চুক্তি; অধ্যায় ২৬: SQLite, Constraint ও Transaction।
- অধ্যায় ২৮: Deployment ও Monitoring; অধ্যায় ২৯: Idempotency ও Consistency।

এই অধ্যায় কেন গুরুত্বপূর্ণ

উনত্রিশটি অধ্যায়ে আপনি আলাদা আলাদা দক্ষতা শিখেছেন: Variable থেকে Closure, DOM থেকে Event Loop, HTTP থেকে Database Transaction, Test থেকে Deployment, Real-Time থেকে Saga। কিন্তু চাকরির প্রথম দিনে কেউ বলবে না "একটি Closure লিখুন"। বলবে: "গ্রাহকরা Package Book করতে পারবেন—দুবার ক্লিক করলে দুটি Booking যেন না হয়, শেষ আসন যেন দুজনকে বিক্রি না হয়, আর কেউ যেন অন্যের Booking বাতিল করতে না পারে।"

এই অধ্যায় সেই কাজটিই করে। একটি সম্পূর্ণ Application—**TravelMaster Pro**—যেখানে প্রতিটি Requirement একটি Code-এর অংশে এবং একটি Test-এ প্রমাণিত। আর শেষে একটি প্রশ্ন: এরপর কী? কীভাবে এই দক্ষতাকে একটি পেশায় রূপ দেবেন।

৩০.১ JavaScript Mastery মানে কী

Syntax মুখস্থ রাখা Mastery নয়। Mastery হলো Correctness, রক্ষণযোগ্যতা, নিরাপত্তা, পর্যবেক্ষণযোগ্যতা, Performance, Accessibility ও পণ্য-ভাবনা—এগুলো মিলিয়ে একটি বাস্তব সমস্যার সমাধান করা, এবং কেন এভাবে করলেন তা ব্যাখ্যা করতে পারা। একজন পেশাদার Developer প্রতিটি সিদ্ধান্তে জিজ্ঞাসা করেন: এটি ব্যর্থ হলে কী হবে? কীভাবে জানব? কীভাবে প্রমাণ করব যে এটি কাজ করে?

স্তর	প্রধান দক্ষতা	প্রমাণ	বইয়ের অধ্যায়
ভিত্তি	Variable, Function, Object, Async	ছোট Utility ও Unit Test	১-১৪
Application	DOM, API, Validation, State	ব্যবহারযোগ্য UI ও Error অবস্থা	১৫-১৮
Backend	HTTP, Auth, Database	লিখিত API ও Integration Test	২০, ২৫, ২৬
Engineering	Type, Test, নিরাপত্তা, CI	পুনরুৎপাদনযোগ্য Build, Code Review	২১-২৪, ২৮
Architecture	Scale, পর্যবেক্ষণ, ব্যর্থতার ধরন	ADR, চিত্র, Incident-পরিকল্পনা	২৭-২৯
পেশাদার	সহযোগিতা ও যোগাযোগ	Portfolio, README, সিদ্ধান্তের ব্যাখ্যা	৩০

৩০.২ Capstone: TravelMaster Pro-র Requirement

একটি ভ্রমণ সেবায় ব্যবহারকারী Package দেখবেন, Account খুলবেন, Login করবেন, Booking করবেন, নিজের Booking দেখবেন ও বাতিল করবেন।

Functional Requirement

১. Public Catalog-এ Package-এর নাম, দাম (টাকা) ও খালি আসন।
২. নিবন্ধনে নাম, Email ও Password যাচাই।
৩. Login-এর পরে একটি স্বল্পমেয়াদি Bearer Session Token।
৪. Booking-এর দাম **Server-এর Catalog** থেকে হিসাব; Client-এর পাঠানো দাম বিশ্বাস করা যাবে না।
৫. একই Idempotency Key-তে একই অনুরোধ আবার এলে দ্বিতীয়বার আসন কমবে না।
৬. নিজের Booking দেখা ও বাতিল করা যাবে; অন্যের Booking-এ প্রবেশ নয়।
৭. বাতিলে আসন ঠিক একবার ফেরত।
৮. Test-এ সফল পথ, ব্যর্থ পথ ও প্রান্তিক ক্ষেত্র।

Non-functional Requirement

- Error-এ Password বা ভেতরের Stack Trace কখনো নয়।
- JSON Body-র আকার সীমিত; অবৈধ JSON-এ 400।
- Token-এর মেয়াদ শেষে 401।
- সব HTTP Response একই চুক্তিতে: `{ data }` অথবা `{ error: { code, message } }`।
- Domain ও Transport স্তর আলাদা।
- একসঙ্গে আসা Booking-এ Overbooking নয়।
- README-তে Setup, সীমাবদ্ধতা ও Production-এর ঘাটতি স্পষ্ট।

> **নোট:** এই Capstone-এর প্রথম পরিকল্পনায় সব অবস্থা একটি Process-এর Memory-তে ছিল এবং কোনো Browser UI ছিল না। এই বইয়ে TravelMaster Pro আসল SQLite Database (`node:sqlite`) ব্যবহার করে, Inventory Transaction-এ সুরক্ষিত, এবং একটি Browser Frontend আছে যা QA-তে আসল Headless Chrome-এ চালানো হয়েছে। **Payment অংশ বাস্তবায়িত নয়**—Booking সরাসরি নিশ্চিত হয়; এটি অনুচ্ছেদ ৩০.১০-এ স্পষ্টভাবে চিহ্নিত।

৩০.৩ Architecture: যা বানানো হয়েছে, আর যা লক্ষ্য

Folder-এর কাঠামো

```
Browser (public/app.mjs)
        |
        | HTTP + JSON + Bearer Token   (Production: HTTPS)
        v
  Transport: src/http.mjs   Route, JSON, Status, Security Header
        |
        v
  Domain: src/service.mjs   Validation, Auth, দাম, Idempotency, মালিকানা
        |
        v
  SQLite: src/db.mjs        users, sessions, packages, bookings, audit_events
                            CHECK, UNIQUE, FOREIGN KEY, Transaction

বানানো: একটি Process, একটি SQLite File (WAL Mode)
লক্ষ্য:  একাধিক Instance, PostgreSQL, Payment Gateway, Redis-এ Rate Limit
```

এই বিভাজনের কারণ: Domain (`service.mjs`) HTTP-র কিছুই জানে না, তাই তার প্রতিটি নিয়ম Network ছাড়াই Test করা যায়; Transport (`http.mjs`) কোনো ব্যবসার নিয়ম জানে না, শুধু অনুবাদ করে; আর Database-এর Constraint শেষ পাহারাদার—Code-এ Bug থাকলেও Schema অবৈধ অবস্থা আটকায়।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-master-pro/
├── package.json        start, demo, test, check, typecheck
├── README.md           চালানো, API, Requirement থেকে প্রমাণ, সীমাবদ্ধতা
├── tsconfig.json       checkJs + strict (JSDoc Type)
├── main.mjs            Environment পড়া, Server চালু, Signal
├── demo.mjs            HTTP দিয়ে প্রতিটি নিয়ম
├── public/             index.html, app.mjs, styles.css
├── src/
│   ├── errors.mjs      Error Code ও HTTP Status
│   ├── validation.mjs  Trust Boundary
│   ├── passwords.mjs   scrypt, Salt, সমান-সময়ের তুলনা
│   ├── db.mjs          Schema, Seed, Transaction
│   ├── service.mjs     Domain
│   ├── http.mjs        Transport
│   └── server.mjs      সব জোড়া লাগানো
└── tests/              ১৭টি Test
```

API চুক্তি

Method	Path	Auth	প্রত্যাশিত ফল
GET	`/health`	না	200
GET	`/api/packages`	না	200 Catalog
POST	`/api/register`	না	201; ভুল Input 400; একই Email 409
POST	`/api/login`	না	200 Token; ভুল 401; বারবার ভুলে 429
POST	`/api/logout`	হ্যাঁ	200, Session বাতিল
POST	`/api/bookings`	হ্যাঁ	201 নতুন; একই Key 200; ভিন্ন অনুরোধ বা আসন নেই 409
GET	`/api/bookings`	হ্যাঁ	200 নিজের Booking
PATCH	`/api/bookings/:id/cancel`	হ্যাঁ	200; অন্যের বা অজানা 404

প্রথম নকশায় Logout Token যাচাই না করেই Session বাতিল করত; এখানে Logout-এর আগে Token যাচাই হয়, অবৈধ Token-এ 401।

৩০.৪ মূল সিদ্ধান্ত ১: Trust Boundary-তে যাচাই

`typeof travelers === 'number'` যথেষ্ট নয়:

trust-boundary.mjs · JavaScript — চলবে: Node.js

```
import { validateBooking } from '../project/travel-master-pro/src/validation.mjs';

// typeof দিয়ে যাচাই যথেষ্ট নয়: এগুলো সবই "number"।
const suspicious = [2, Number.NaN, Infinity, 1.5, -3, 0, 11, '2'];
for (const travelers of suspicious) {
  const naive = typeof travelers === 'number';
  let strict;
  try {
    validateBooking({ packageId: 'BT-4D', travelers,
      idempotencyKey: 'demo-key-0001' });
    strict = 'গ্রহণ';
  } catch (error) {
    strict = `বাতিল (${error.details.join(', ')})`;
  }
  const shown = typeof travelers === 'string' ? `'${travelers}'`
    : String(travelers);
  console.log(shown.padEnd(8), `typeof: ${naive}`.padEnd(14), strict);
}
```

Output

```
2         typeof: true   গ্রহণ
NaN       typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
Infinity  typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
1.5       typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
-3        typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
0         typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
11        typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
'2'       typeof: false  বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
```

`NaN`, `Infinity`, `1.5` ও `-3` —সবগুলোর `typeof` "number"। ১.৫ জন যাত্রীর দাম বা ঋণাত্মক যাত্রীর জন্য আসন **বাড়ানো**—এগুলো শুধু তাত্ত্বিক নয়, আসল আক্রমণ। Validation Module:

validation.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// @ts-check
import { AppError } from './errors.mjs';

/** @param {unknown} value */
const text = (value) => (typeof value === 'string' ? value.trim() : '');

/**
 * Trust Boundary: বাইরে থেকে আসা প্রতিটি মান এখানে যাচাই ও স্বাভাবিক হয়।
 * @param {any} input
 * @returns {{ name: string, email: string, password: string }}
 */
export function validateRegistration(input) {
  const name = text(input?.name);
  const email = text(input?.email).toLowerCase();
  const password = typeof input?.password === 'string' ? input.password : '';
  const details = [];
  if (name.length < 2 || name.length > 60) details.push('name: ২-৬০ অক্ষর');
  // শুধু মৌলিক Format; Email-এর মালিকানা যাচাই আলাদা কাজ (Verification Link)।
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || email.length > 254) {
    details.push('email: সঠিক Format');
  }
  if (password.length < 12 || password.length > 128) {
    details.push('password: ১২-১২৮ অক্ষর');
  }
  if (details.length) throw new AppError('VALIDATION', 'Input ঠিক নয়', details);
  return { name, email, password };
}

// ...
```

Email-এর Regular Expression শুধু মৌলিক Format দেখে; Email-টি সত্যিই ব্যবহারকারীর কি না, তা জানার একমাত্র উপায় একটি Verification Link। Email ছোট হাতের অক্ষরে স্বাভাবিক করা হয়, তাই `RAFI@EXAMPLE.COM` দিয়ে দ্বিতীয় Account খোলা যায় না। Password-এ জটিলতার নিয়মের চেয়ে দৈর্ঘ্য বেশি কার্যকর—১২ অক্ষরের সীমা। Booking-এর যাচাই:

validation.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

/**
 * @param {any} input
 * @returns {{ packageId: string, travelers: number, idempotencyKey: string }}
 */
export function validateBooking(input) {
  const packageId = text(input?.packageId);
  const travelers = input?.travelers;
  const idempotencyKey = text(input?.idempotencyKey);
  const details = [];
  if (!/^[A-Z0-9-]{2,20}$/.test(packageId)) details.push('packageId');
  // NaN, Infinity, ভগ্নাংশ, ঋণাত্মক, String: সবই বাদ।
  if (!Number.isInteger(travelers) || travelers < 1 || travelers > 10) {
    details.push('travelers: ১-১০ পূর্ণসংখ্যা');
  }
  if (!/^[\w-]{8,64}$/.test(idempotencyKey)) details.push('idempotencyKey');
  if (details.length) throw new AppError('VALIDATION', 'Input ঠিক নয়', details);
  return { packageId, travelers, idempotencyKey };
}
```

প্রতিটি `// @ts-check` ও JSDoc Type (`@returns`) TypeScript Compiler-কে এই File-গুলো strict Mode-এ যাচাই করতে দেয়, কোনো Build ধাপ ছাড়াই (অধ্যায় ২৪)। Validation-এর পরে Domain জানে `travelers` একটি `number`—আর Compiler তা নিশ্চিত করে।

৩০.৫ মূল সিদ্ধান্ত ২: Password ও Session

passwords.mjs · JavaScript চলবে: Node.js

```
// @ts-check
import { randomBytes, scryptSync, timingSafeEqual } from 'node:crypto';

// scrypt: ইচ্ছাকৃতভাবে ধীর ও Memory-ভারী, তাই চুরি যাওয়া Hash থেকে
// অনুমান করে Password বের করা ব্যয়বহুল। প্রতিটি ব্যবহারকারীর আলাদা Salt।
const PARAMS = { N: 16384, r: 8, p: 1 };
const KEY_LENGTH = 32;

/** @param {string} password */
export function hashPassword(password) {
```

```
  const salt = randomBytes(16);
  const hash = scryptSync(password, salt, KEY_LENGTH, PARAMS);
  return `scrypt$${salt.toString('base64')}$${hash.toString('base64')}`;
}

/**
 * @param {string} password
 * @param {string} stored
 */
export function verifyPassword(password, stored) {
  const [scheme, salt, hash] = stored.split('$');
  if (scheme !== 'scrypt' || !salt || !hash) return false;
  const expected = Buffer.from(hash, 'base64');
  const actual = scryptSync(password, Buffer.from(salt, 'base64'),
    expected.length, PARAMS);
  // সমান সময়ে তুলনা: কতগুলো Byte মিলল, তা সময় দেখে বোঝা যায় না।
  return timingSafeEqual(actual, expected);
}

// অজানা Email-এও একই পরিমাণ কাজ, যাতে উত্তরের সময় দেখে বোঝা না যায়
// কোন Email নিবন্ধিত।
export const DUMMY_HASH = hashPassword('not-a-real-password-for-timing');
```

password-hash.mjs · JavaScript — চলবে: Node.js

```
import { hashPassword, verifyPassword }
  from '../project/travel-master-pro/src/passwords.mjs';

const password = 'long-password-123';
const first = hashPassword(password);
const second = hashPassword(password);

// একই Password, ভিন্ন Salt, তাই ভিন্ন Hash: দুই ব্যবহারকারীর একই
// Password থাকলেও Database দেখে তা বোঝা যায় না।
console.log('গঠন:', first.split('$').map((part, i) => (i === 0 ? part
  : `${part.length} অক্ষর`)).join(' | '));
console.log('দুটি Hash এক?', first === second);
console.log('সঠিক Password:', verifyPassword(password, first));
console.log('ভুল Password:', verifyPassword('long-password-124', first));
console.log('Hash-এ Password আছে?', first.includes(password));
```

Output

```
গঠন: scrypt | 24 অক্ষর | 44 অক্ষর
দুটি Hash এক? false
সঠিক Password: true
ভুল Password: false
Hash-এ Password আছে? false
```

Stored Hash-এ তিনটি অংশ: পদ্ধতির নাম, Salt ও Hash। পদ্ধতির নাম রাখা হয় যাতে ভবিষ্যতে Argon2id-এ গেলে পুরোনো Hash চেনা যায় এবং পরের Login-এ নতুন পদ্ধতিতে হালনাগাদ করা যায়।

Session: Login-এর পর ৩২ Byte-এর এলোমেলো Token Client পায়, কিন্তু Database-এ থাকে শুধু তার SHA-256 Hash ও মেয়াদ। Database চুরি গেলেও Hash থেকে Token ফেরানো যায় না, তাই চোর কারও হয়ে Login করতে পারে না। প্রতিটি অনুরোধে Token-এর Hash খুঁজে মেয়াদ যাচাই; Logout সেই সারি মুছে দেয়। ভুল Password ও অজানা Email—দুটিতেই একই বার্তা ("Email বা Password ভুল") এবং একই পরিমাণ কাজ (`DUMMY_HASH`), যাতে কেউ বার্তা বা সময় দেখে বুঝতে না পারে কোন Email নিবন্ধিত। পাঁচবার ভুলের পরে ১৫ মিনিট 429—Password অনুমান (Credential Stuffing) ধীর করতে।

৩০.৬ মূল সিদ্ধান্ত ৩: Database, দাম, Idempotency ও Inventory

Schema-ই শেষ পাহারাদার:

db.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
// @ts-check
import { DatabaseSync } from 'node:sqlite';

// Schema-ই শেষ পাহারাদার: Code-এ Bug থাকলেও seats কখনো ঋণাত্মক নয়,
// আর একই ব্যবহারকারীর একই idempotency_key দুটি Booking হতে পারে না।
const SCHEMA = `
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS sessions (
  token_hash TEXT PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  expires_at INTEGER NOT NULL);
CREATE TABLE IF NOT EXISTS packages (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  price_bdt INTEGER NOT NULL CHECK (price_bdt > 0),
  seats INTEGER NOT NULL CHECK (seats >= 0));
CREATE TABLE IF NOT EXISTS bookings (
  id INTEGER PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  package_id TEXT NOT NULL REFERENCES packages(id),
  travelers INTEGER NOT NULL CHECK (travelers BETWEEN 1 AND 10),
  total_bdt INTEGER NOT NULL CHECK (total_bdt > 0),
  status TEXT NOT NULL CHECK (status IN ('CONFIRMED', 'CANCELLED')),
  idempotency_key TEXT NOT NULL,
  created_at TEXT NOT NULL,
```

```
  UNIQUE (user_id, idempotency_key));
CREATE INDEX IF NOT EXISTS bookings_by_user ON bookings (user_id, id);
CREATE TABLE IF NOT EXISTS audit_events (
  id INTEGER PRIMARY KEY,
  type TEXT NOT NULL,
  user_id INTEGER,
  detail TEXT NOT NULL,
  at TEXT NOT NULL);`;

// ...
```

`CHECK (seats >= 0)` মানে Code-এ Bug থাকলেও আসন ঋণাত্মক হতে পারে না—Database নিজেই লেখা প্রত্যাখ্যান করে। `UNIQUE (user_id, idempotency_key)` মানে একই ব্যবহারকারীর একই Key দুটি Booking হতে পারে না, এমনকি দুটি Server একসঙ্গে চেষ্টা করলেও।

db.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

export const SEED_PACKAGES = [
  ['BT-4D', 'Bandarban Trek, 8 দিন', 12_500, 12],
  ['CXB-3D', "Cox's Bazar Beach, ৩ দিন", 9_800, 20],
  ['SYL-2D', 'Sylhet চা-বাগান, ২ দিন', 6_500, 2],
  ['SDB-3D', 'Sundarbans Boat Safari, ৩ দিন', 15_500, 8],
];

/** @param {string} [path] */
export function openDatabase(path = ':memory:') {
  const db = new DatabaseSync(path);
  db.exec('PRAGMA foreign_keys = ON; PRAGMA busy_timeout = 5000;');
  if (path !== ':memory:') db.exec('PRAGMA journal_mode = WAL;');
  db.exec(SCHEMA);
  const insert = db.prepare(`INSERT OR IGNORE INTO packages
    (id, name, price_bdt, seats) VALUES (?, ?, ?, ?)`);
  for (const row of SEED_PACKAGES) insert.run(...row);
  return db;
}

// ...
```

`foreign_keys` প্রতিটি সংযোগে চালু করতে হয় (অধ্যায় ২৬); `busy_timeout` অন্য সংযোগ লিখছে এমন সময়ে সঙ্গে সঙ্গে ব্যর্থ না হয়ে অপেক্ষা করতে দেয়; WAL Mode পড়া ও লেখা একসঙ্গে চলতে দেয়।

db.mjs · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

/**
```

```
 * BEGIN IMMEDIATE: লেখার Lock শুরুতেই, তাই দুটি সংযোগ একই সঙ্গে
 * "আসন আছে" দেখে দুজনেই বিক্রি করতে পারে না।
 * @template T
 * @param {DatabaseSync} db
 * @param {() => T} work
 * @returns {T}
 */
export function transaction(db, work) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = work();
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}
```

কেন "পড়ে, তারপর লেখা" বিপজ্জনক: দুটি অনুরোধ একসঙ্গে শেষ দুটি আসনের দুটি করে চাইলে কী হয়:

race-condition.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// দুটি অনুরোধ একসঙ্গে শেষ ২টি আসনের ২টি করে চায়।
// ভুল পথ: পড়া, তারপর (await-এর ফাঁকে) লেখা।
const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE packages (id TEXT PRIMARY KEY, seats INTEGER);
  INSERT INTO packages VALUES ('SYL-2D', 2);`);
const reset = () => db.exec("UPDATE packages SET seats = 2");
const seats = () => db.prepare('SELECT seats FROM packages').get().seats;
const pause = () => new Promise((resolve) => setTimeout(resolve, 10));

async function checkThenAct(want) {
  const available = seats();
  await pause(); // যেমন Payment Gateway-কে জিজ্ঞাসা, বা অন্য কোনো await
  if (available < want) return 'SOLD_OUT';
  db.prepare('UPDATE packages SET seats = ? WHERE id = ?')
    .run(available - want, 'SYL-2D');
  return 'BOOKED';
}

async function conditionalUpdate(want) {
  await pause();
  // যাচাই ও কমানো একটি Statement-এ: মাঝখানে কেউ ঢুকতে পারে না।
  const { changes } = db.prepare(`UPDATE packages SET seats = seats - ?
    WHERE id = ? AND seats >= ?`).run(want, 'SYL-2D', want);
  return changes ? 'BOOKED' : 'SOLD_OUT';
}
```

```
for (const strategy of [checkThenAct, conditionalUpdate]) {
  reset();
  const results = await Promise.all([strategy(2), strategy(2)]);
  console.log(`${strategy.name}: ${results.join(', ')}; বাকি আসন ${seats()}`);
}
```

Output

```
checkThenAct: BOOKED, BOOKED; বাকি আসন 0
conditionalUpdate: BOOKED, SOLD_OUT; বাকি আসন 0
```

প্রথম পদ্ধতিতে দুজনেই "২টি আছে" পড়েছেন, তারপর দুজনেই বিক্রি করেছেন—চারটি আসন বিক্রি হলো দুটির জায়গায়, অথচ শেষে আসন শূন্য দেখায়, তাই ভুলটি চোখেও পড়ে না। দ্বিতীয় পদ্ধতিতে `WHERE seats >= ?` যাচাই ও কমানো একই Statement-এ, তাই দ্বিতীয়জন সঠিকভাবে SOLD_OUT পান।

src/service.mjs

প্রথম অংশ: Import, Token-এর Hash ও Booking-এর রূপান্তর:

service.mjs · JavaScript · অংশ ১ / ৬ (Part 1 of 6) · চলবে: Node.js

```
// @ts-check
import { createHash, randomBytes } from 'node:crypto';
import { AppError } from './errors.mjs';
import { transaction } from './db.mjs';
import { hashPassword, verifyPassword, DUMMY_HASH } from './passwords.mjs';
import { validateRegistration, validateBooking } from './validation.mjs';

/** @typedef {import('node:sqlite').DatabaseSync} Database */
/** @typedef {{ id: number, name: string, email: string }} User */

/** @param {string} token */
const sha256 = (token) => createHash('sha256').update(token).digest('hex');

/** @param {any} row */
const toBooking = (row) => ({ id: row.id, packageId: row.package_id,
  travelers: row.travelers, totalBdt: row.total_bdt, status: row.status,
  createdAt: row.created_at });

/**
 * Domain Service: সব ব্যবসার নিয়ম এখানে; HTTP-র কিছু জানে না।
 * @param {{ db: Database, now?: () => number, tokenTtlMs?: number,
 *   maxFailedLogins?: number }} options
 */
export function createTravelService({ db, now = Date.now,
  tokenTtlMs = 30 * 60_000, maxFailedLogins = 5 }) {
  /** @type {Map<string, { count: number, since: number }>} */
```

```
  const failures = new Map();
  const iso = () => new Date(now()).toISOString();

// ...
```

Audit ও নিবন্ধন। Password-এর Hash তৈরি হয় Database-এ লেখার আগে; একই Email-এ UNIQUE Constraint ভঙ্গ হলে একটি বোধগম্য Error:

service.mjs · JavaScript অংশ ২ / ৬ (Part 2 of 6) চলবে: Node.js

```
// ...

  /**
   * @param {string} type
   * @param {number | null} userId
   * @param {object} detail
   */
  const audit = (type, userId, detail) => db.prepare(
    'INSERT INTO audit_events (type, user_id, detail, at) VALUES (?, ?, ?, ?)')
    .run(type, userId, JSON.stringify(detail), iso());

  /** @param {any} input @returns {User} */
  function register(input) {
    const { name, email, password } = validateRegistration(input);
    const passwordHash = hashPassword(password);
    try {
      const { lastInsertRowid } = db.prepare(`INSERT INTO users
        (name, email, password_hash, created_at) VALUES (?, ?, ?, ?)`)
        .run(name, email, passwordHash, iso());
      audit('user.registered', Number(lastInsertRowid), {});
      return { id: Number(lastInsertRowid), name, email };
    } catch (error) {
      if (/UNIQUE/.test(String(error))) {
        throw new AppError('EMAIL_TAKEN', 'এই Email আগেই নিবন্ধিত');
      }
      throw error;
    }
  }

// ...
```

Login: ভুলের সীমা, অজানা Email-এও Hash-যাচাই, এবং Database-এ শুধু Token-এর Hash:

service.mjs · JavaScript অংশ ৩ / ৬ (Part 3 of 6) চলবে: Node.js

```
// ...

  /** @param {any} input */
  function login(input) {
```

```
    const email = typeof input?.email === 'string'
      ? input.email.trim().toLowerCase() : '';
    const password = typeof input?.password === 'string' ? input.password : '';
    const recent = failures.get(email);
    if (recent && recent.count >= maxFailedLogins
      && now() - recent.since < 15 * 60_000) {
      throw new AppError('TOO_MANY_ATTEMPTS', 'কিছুক্ষণ পরে চেষ্টা করুন');
    }
    /** @type {any} */
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
    // অজানা Email-এও একই Hash-যাচাই: সময় দেখে কিছু বোঝা যায় না।
    const ok = verifyPassword(password, user?.password_hash ?? DUMMY_HASH);
    if (!user || !ok) {
      const entry = recent && now() - recent.since < 15 * 60_000
        ? recent : { count: 0, since: now() };
      failures.set(email, { ...entry, count: entry.count + 1 });
      audit('login.failed', user?.id ?? null, {});
      throw new AppError('UNAUTHORIZED', 'Email বা Password ভুল');
    }
    failures.delete(email);
    // Database-এ শুধু Token-এর Hash: Database ফাঁস হলেও Token ব্যবহারযোগ্য নয়।
    const token = randomBytes(32).toString('base64url');
    const expiresAt = now() + tokenTtlMs;
    db.prepare('INSERT INTO sessions VALUES (?, ?, ?)')
      .run(sha256(token), user.id, expiresAt);
    audit('login.succeeded', user.id, {});
    return { token, expiresAt: new Date(expiresAt).toISOString(),
      user: { id: user.id, name: user.name, email: user.email } };
  }

// ...
```

service.mjs · JavaScript অংশ 8 / ৬ (Part 4 of 6) চলবে: Node.js

```
// ...

  /** @param {string | undefined} token @returns {User} */
  function authenticate(token) {
    /** @type {any} */
    const row = token && db.prepare(`SELECT u.id, u.name, u.email,
      s.expires_at FROM sessions s JOIN users u ON u.id = s.user_id
      WHERE s.token_hash = ?`).get(sha256(token));
    if (!row || row.expires_at <= now()) {
      throw new AppError('UNAUTHORIZED', 'Login প্রয়োজন');
    }
    return { id: row.id, name: row.name, email: row.email };
  }

  /** @param {string | undefined} token */
```

```
  function logout(token) {
    const user = authenticate(token);
    db.prepare('DELETE FROM sessions WHERE token_hash = ?')
      .run(sha256(/** @type {string} */ (token)));
    audit('logout', user.id, {});
  }

  function listPackages() {
    return db.prepare('SELECT id, name, price_bdt AS priceBdt, seats '
      + 'FROM packages ORDER BY id').all().map((row) => ({ ...row }));
  }

// ...
```

`authenticate`-এ `expires_at <= now()` —ঠিক মেয়াদের মুহূর্তেই অবৈধ। এবার Capstone-এর হৃদয়, `createBooking`। পুরোটা একটি `BEGIN IMMEDIATE` Transaction-এ: আগে একই Key-র Booking খোঁজা; থাকলে অনুরোধ মিলিয়ে আগেরটি ফেরত (`replayed: true`) বা KEY_CONFLICT; না থাকলে শর্তযুক্ত UPDATE; আসন না কমলে Package আছে কি না দেখে SOLD_OUT বা NOT_FOUND; তারপর Catalog-এর দামে Booking:

service.mjs · JavaScript অংশ ৫ / ৬ (Part 5 of 6) চলবে: Node.js

```
// ...

  /** @param {number} userId @param {any} input */
  function createBooking(userId, input) {
    const { packageId, travelers, idempotencyKey } = validateBooking(input);
    return transaction(db, () => {
      /** @type {any} */
      const existing = db.prepare(`SELECT * FROM bookings
        WHERE user_id = ? AND idempotency_key = ?`).get(userId, idempotencyKey);
      if (existing) {
        if (existing.package_id !== packageId
          || existing.travelers !== travelers) {
          throw new AppError('KEY_CONFLICT',
            'একই idempotencyKey, কিন্তু ভিন্ন অনুরোধ');
        }
        return { booking: toBooking(existing), replayed: true };
      }
      // শর্তযুক্ত UPDATE: যাচাই ও কমানো একটি Statement-এ, তাই Race নেই।
      const { changes } = db.prepare(`UPDATE packages SET seats = seats - ?
        WHERE id = ? AND seats >= ?`).run(travelers, packageId, travelers);
      if (changes === 0) {
        const found = db.prepare('SELECT 1 FROM packages WHERE id = ?')
          .get(packageId);
        throw found ? new AppError('SOLD_OUT', 'যথেষ্ট আসন নেই')
          : new AppError('NOT_FOUND', 'Package পাওয়া যায়নি');
      }
      // দাম সব সময় Server-এর Catalog থেকে; Client-এর পাঠানো দাম উপেক্ষিত।
```

```
      /** @type {any} */
      const pkg = db.prepare('SELECT price_bdt FROM packages WHERE id = ?')
        .get(packageId);
      const { lastInsertRowid } = db.prepare(`INSERT INTO bookings
        (user_id, package_id, travelers, total_bdt, status, idempotency_key,
        created_at) VALUES (?, ?, ?, ?, 'CONFIRMED', ?, ?)`)
        .run(userId, packageId, travelers, pkg.price_bdt * travelers,
          idempotencyKey, iso());
      audit('booking.created', userId, { bookingId: Number(lastInsertRowid) });
      const row = db.prepare('SELECT * FROM bookings WHERE id = ?')
        .get(lastInsertRowid);
      return { booking: toBooking(row), replayed: false };
    });
  }

// ...
```

তালিকা ও বাতিল। বাতিলের Query-তে `AND user_id = ?` —মালিকানা যাচাই আলাদা ধাপ নয়, Query-রই অংশ, তাই ভুলে বাদ পড়ার সুযোগ নেই। আগে বাতিল হয়ে থাকলে কিছুই না বদলে একই ফল:

service.mjs · JavaScript অংশ ৬ / ৬ (Part 6 of 6) চলবে: Node.js

```
// ...

  /** @param {number} userId */
  function listBookings(userId) {
    return db.prepare('SELECT * FROM bookings WHERE user_id = ? ORDER BY id')
      .all(userId).map(toBooking);
  }

  /** @param {number} userId @param {number} bookingId */
  function cancelBooking(userId, bookingId) {
    return transaction(db, () => {
      /** @type {any} */
      const row = db.prepare(`SELECT * FROM bookings
        WHERE id = ? AND user_id = ?`).get(bookingId, userId);
      // অন্যের Booking-ও "পাওয়া যায়নি": অস্তিত্বের তথ্যও ফাঁস নয়।
      if (!row) throw new AppError('NOT_FOUND', 'Booking পাওয়া যায়নি');
      if (row.status === 'CANCELLED') return toBooking(row); // একবারই ফেরত
      db.prepare("UPDATE bookings SET status = 'CANCELLED' WHERE id = ?")
        .run(bookingId);
      db.prepare('UPDATE packages SET seats = seats + ? WHERE id = ?')
        .run(row.travelers, row.package_id);
      audit('booking.cancelled', userId, { bookingId });
      return toBooking({ ...row, status: 'CANCELLED' });
    });
  }

  return { register, login, authenticate, logout, listPackages,
```

```
    createBooking, listBookings, cancelBooking };
}
```

৩০.৭ Transport: src/http.mjs

http.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
// @ts-check
import http from 'node:http';
import { readFile } from 'node:fs/promises';
import { AppError } from './errors.mjs';

/**
 * @typedef {ReturnType<typeof import('./service.mjs').createTravelService>}
 *   Service
 */
/** @typedef {[number, unknown]} Result */

const STATIC = {
  '/': ['index.html', 'text/html'],
  '/app.mjs': ['app.mjs', 'text/javascript'],
  '/styles.css': ['styles.css', 'text/css'],
};

// Browser-কে কঠোর নিয়ম: শুধু নিজের Script ও Style, অন্য Site-এ Frame নয়।
const SECURITY_HEADERS = {
  'X-Content-Type-Options': 'nosniff',
  'Referrer-Policy': 'no-referrer',
  'Content-Security-Policy': "default-src 'self'; script-src 'self'; "
    + "style-src 'self'; connect-src 'self'; img-src 'self' data:; "
    + "frame-ancestors 'none'; base-uri 'none'; form-action 'self'",
};

// ...
```

Content-Security-Policy Browser-কে বলে শুধু নিজের Domain-এর Script ও Style চালাতে—কেউ কোনোভাবে একটি `<script>` ঢুকিয়ে দিলেও (XSS) Browser তা চালাবে না। `frame-ancestors 'none'` অন্য Site-কে এই Page Frame-এ রাখতে দেয় না (Clickjacking)।

http.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

/** @param {http.IncomingMessage} req @param {number} limit */
async function readJson(req, limit) {
  if (!/^application\/json\b/.test(req.headers['content-type'] ?? '')) {
    throw new AppError('UNSUPPORTED_MEDIA_TYPE', 'JSON পাঠান');
```

```
  }
  let size = 0;
  const chunks = [];
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new AppError('TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new AppError('BAD_JSON', 'অবৈধ JSON');
  }
}

/** @param {http.IncomingMessage} req */
const bearer = (req) => /^Bearer ([\w-]+)$/
  .exec(req.headers.authorization ?? '')?.[1];

// ...
```

Route-এর তালিকা। Booking তৈরিতে আগে Authentication, তারপর Body পড়া—পরিচয়হীন কারও Input পড়ার কাজই হয় না। একই Key-তে দ্বিতীয় অনুরোধে Status 200, প্রথমবারে 201—Client চাইলে পার্থক্য বুঝতে পারে:

http.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

/**
 * Transport স্তর: Route, JSON, Status ও Header। ব্যবসার নিয়ম Service-এ।
 * @param {{ service: Service, bodyLimit?: number,
 *   log?: (event: string, fields: object) => void }} options
 */
export function createHttpServer({ service, bodyLimit = 4096,
  log = () => {} }) {
  const user = (/** @type {http.IncomingMessage} */ req) =>
    service.authenticate(bearer(req));
  const body = (/** @type {http.IncomingMessage} */ req) =>
    readJson(req, bodyLimit);

  /** @type {[string, RegExp, (req: http.IncomingMessage,
   *   params: string[]) => Promise<Result>][]} */
  const routes = [
    ['GET', /^\/health$/, async () => [200, { status: 'ok' }]],
    ['GET', /^\/api\/packages$/, async () => [200, service.listPackages()]],
    ['POST', /^\/api\/register$/,
      async (req) => [201, service.register(await body(req))]],
    ['POST', /^\/api\/login$/,
      async (req) => [200, service.login(await body(req))]],
```

```
    ['POST', /^\/api\/logout$/, async (req) => {
      service.logout(bearer(req));
      return [200, { loggedOut: true }];
    }],
    ['GET', /^\/api\/bookings$/,
      async (req) => [200, service.listBookings(user(req).id)]],
    // আগে Authentication, তারপর Body: পরিচয় ছাড়া কোনো Input পড়া নয়।
    ['POST', /^\/api\/bookings$/, async (req) => {
      const { id } = user(req);
      const { booking, replayed } = service.createBooking(id, await body(req));
      return [replayed ? 200 : 201, booking];
    }],
    ['PATCH', /^\/api\/bookings\/(\d+)\/cancel$/, async (req, [bookingId]) =>
      [200, service.cancelBooking(user(req).id, Number(bookingId))]],
  ];

// ...
```

Static File, Route মেলানো এবং Error চুক্তি। `AppError` তার নিজের Status ও Code পায়; বাকি সব 500, সাধারণ বার্তা, আর বিবরণ শুধু Log-এ। প্রতিটি API Response-এ `Cache-Control: no-store`, যাতে কোনো Proxy বা Browser Cache ব্যক্তিগত তথ্য জমা না রাখে:

http.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  return http.createServer(async (req, res) => {
    const { pathname } = new URL(req.url ?? '/', 'http://localhost');
    try {
      const file = req.method === 'GET'
        && STATIC[/** @type {keyof typeof STATIC} */ (pathname)];
      if (file) {
        const content = await readFile(
          new URL(`../public/${file[0]}`, import.meta.url));
        res.writeHead(200, { ...SECURITY_HEADERS,
          'Content-Type': `${file[1]}; charset=utf-8` });
        return void res.end(content);
      }
      for (const [method, pattern, handler] of routes) {
        const match = pattern.exec(pathname);
        if (match && req.method === method) {
          const [status, data] = await handler(req, match.slice(1));
          return send(res, status, { data });
        }
      }
      throw new AppError('NOT_FOUND', 'Route পাওয়া যায়নি');
    } catch (error) {
      if (error instanceof AppError) {
        const { code, message, details } = error;
```

```
        return send(res, error.status, { error: { code, message,
          ...(details.length ? { details } : {}) } });
      }
      // অপ্রত্যাশিত: পুরো বিবরণ Log-এ, Client পায় শুধু সাধারণ বার্তা।
      log('http.unhandled', { path: pathname, error: String(error),
        stack: error instanceof Error ? error.stack : undefined });
      return send(res, 500, { error: { code: 'INTERNAL',
        message: 'Server-এ সমস্যা' } });
    }
  });
}

/**
 * @param {http.ServerResponse} res
 * @param {number} status
 * @param {object} body
 */
function send(res, status, body) {
  res.writeHead(status, { ...SECURITY_HEADERS,
    'Content-Type': 'application/json; charset=utf-8',
    'Cache-Control': 'no-store' });
  res.end(JSON.stringify(body));
}
```

`server.mjs` তিনটি স্তর জোড়া লাগায় (Composition Root), আর `main.mjs` Environment থেকে Port ও Database-এর Path পড়ে:

server.mjs · JavaScript — চলবে: Node.js

```
// @ts-check
import { openDatabase } from './db.mjs';
import { createTravelService } from './service.mjs';
import { createHttpServer } from './http.mjs';

/**
 * Database, Service ও HTTP একসঙ্গে জোড়া লাগানো (Composition Root)।
 * @param {{ port?: number, host?: string, dbPath?: string,
 *   now?: () => number, tokenTtlMs?: number,
 *   log?: (event: string, fields: object) => void }} [options]
 */
export async function startServer({ port = 0, host = '127.0.0.1',
  dbPath = ':memory:', now, tokenTtlMs, log } = {}) {
  const db = openDatabase(dbPath);
  const service = createTravelService({ db, now, tokenTtlMs });
  const server = createHttpServer({ service, log });
  await new Promise((resolve) => server.listen(port, host, () => resolve(0)));
  const address = /** @type {import('node:net').AddressInfo} */ (
    server.address());
  return {
```

```
    url: `http://${host}:${address.port}`,
    server, service, db,
    close: () => new Promise((resolve) => {
      server.closeAllConnections();
      server.close(() => {
        db.close();
        resolve(0);
      });
    }),
  };
}
```

৩০.৮ Frontend: public/app.mjs

Frontend একটি সাধারণ ES Module, কোনো Framework ছাড়া। প্রথম অংশ: অবস্থা, Error-এর বন্ধুত্বপূর্ণ বার্তা এবং সব API ডাকের একটি Function:

app.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Browser

```
// Token শুধু Memory-তে (Page Reload-এ Logout)। localStorage-এ রাখলে কোনো XSS
// Script তা পড়তে পারত; Production-এ HttpOnly Cookie + CSRF সুরক্ষা ভালো।
const state = { token: null, user: null, pendingKey: null, busy: false };
const $ = (selector) => document.querySelector(selector);
const MESSAGES = {
  SOLD_OUT: 'দুঃখিত, এত আসন খালি নেই।',
  EMAIL_TAKEN: 'এই Email আগেই নিবন্ধিত।',
  KEY_CONFLICT: 'আগের অনুরোধের সঙ্গে মিলছে না; আবার চেষ্টা করুন।',
  TOO_MANY_ATTEMPTS: 'অনেকবার ভুল হয়েছে; কিছুক্ষণ পরে চেষ্টা করুন।',
};

function say(text, isError = false) {
  $('#message').textContent = text;
  $('#message').classList.toggle('error', isError);
}

// সব API ডাক এক জায়গায়: Token, JSON, Error-এর বার্তা।
async function api(path, { method = 'GET', body } = {}) {
  let res;
  try {
    res = await fetch(path, { method, headers: {
      ...(body && { 'Content-Type': 'application/json' }),
      ...(state.token && { Authorization: `Bearer ${state.token}` }) },
    body: body && JSON.stringify(body) });
  } catch {
    throw new Error('Network সমস্যা; সংযোগ দেখে আবার চেষ্টা করুন।');
  }
  const payload = await res.json();
```

```
  if (res.ok) return payload.data;
  const { code, message, details } = payload.error;
  // Login থাকা অবস্থায় 401 মানে Session শেষ; Login Form-এ তা ভুল Password।
  if (code === 'UNAUTHORIZED' && state.token) {
    signOut();
    throw new Error('Session শেষ হয়েছে; আবার Login করুন।');
  }
  const extra = details?.length ? ` (${details.join(', ')})` : '';
  throw new Error(MESSAGES[code] ?? `${message}${extra}`);
}

// ...
```

Token শুধু একটি JavaScript Variable-এ— `localStorage`-এ নয়। কারণ: কোনো XSS Script `localStorage` পড়তে পারে। মূল্য: Page Reload করলে আবার Login। Production-এ ভালো বিকল্প HttpOnly Cookie (JavaScript পড়তেই পারে না), সঙ্গে CSRF সুরক্ষা। আরেকটি বিষয়: Login অবস্থায় 401 মানে Session শেষ, কিন্তু Login Form-এ 401 মানে ভুল Password—দুটিকে আলাদা করা হয়েছে। প্রথম সংস্করণে দুটিই "Session শেষ হয়েছে" দেখাত; Browser End-to-End Test এই ভুল ধরেছে।

app.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Browser

```
// ...

const taka = (n) => `${n.toLocaleString('en-IN')} টাকা`;

async function loadPackages() {
  const packages = await api('/api/packages');
  $('#packages').replaceChildren(...packages.map((p) => {
    const item = document.createElement('li');
    item.textContent = `${p.name}: ${taka(p.priceBdt)}, আসন ${p.seats}`;
    return item;
  }));
  const select = $('#book select');
  const chosen = select.value;
  select.replaceChildren(...packages.map((p) => new Option(
    `${p.name} (${taka(p.priceBdt)})`, p.id, false, p.id === chosen)));
}

async function loadBookings() {
  const bookings = await api('/api/bookings');
  $('#bookings').replaceChildren(...bookings.map((b) => {
    const item = document.createElement('li');
    item.textContent = `#${b.id} ${b.packageId}, ${b.travelers} জন, `
      + `${taka(b.totalBdt)}, ${b.status} `;
    item.classList.toggle('cancelled', b.status === 'CANCELLED');
    if (b.status === 'CONFIRMED') {
      const cancel = document.createElement('button');
      cancel.type = 'button';
```

```
      cancel.textContent = 'বাতিল';
      cancel.dataset.id = b.id;
      item.append(cancel);
    }
    return item;
  }));
}

// ...
```

DOM-এ সব লেখা `textContent` দিয়ে, কখনো `innerHTML` নয়—Package-এর নাম বা Server-এর বার্তায় HTML থাকলেও তা Code হিসেবে চলবে না।

app.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Browser

```
// ...

function render() {
  $('#who').textContent = state.user
    ? `${state.user.name} হিসেবে Login` : 'অতিথি হিসেবে দেখছেন';
  $('#logout').hidden = !state.user;
  $('#account').hidden = Boolean(state.user);
  $('#booking').hidden = !state.user;
}

function signOut() {
  Object.assign(state, { token: null, user: null, pendingKey: null });
  render();
}

// প্রতিটি Form-এর জন্য একই কাঠামো: বার্তা মোছা, কাজ, Error দেখানো।
function onSubmit(selector, work) {
  $(selector).addEventListener('submit', async (event) => {
    event.preventDefault();
    say('');
    try {
      await work(new FormData(event.target), event.target);
    } catch (error) {
      say(error.message, true);
    }
  });
}

onSubmit('#register', async (form, element) => {
  await api('/api/register', { method: 'POST',
    body: Object.fromEntries(form) });
  element.reset();
  say('নিবন্ধন সম্পন্ন। এবার Login করুন।');
});
```

```
onSubmit('#login', async (form, element) => {
  const session = await api('/api/login', { method: 'POST',
    body: Object.fromEntries(form) });
  Object.assign(state, { token: session.token, user: session.user });
  element.reset();
  render();
  say(`স্বাগতম, ${session.user.name}।`); // আগে বার্তা: পরে এলে নতুন বার্তা মুছত
  await loadBookings();
});

// ...
```

Booking Form-এ দুই স্তরের Double-submit সুরক্ষা। Client-এ: `busy` থাকলে দ্বিতীয় Submit উপেক্ষিত, Button নিষ্ক্রিয়। Server-এ: `pendingKey` একবার তৈরি হয় এবং সফল না হওয়া পর্যন্ত একই থাকে—তাই Network-এ উত্তর হারিয়ে গেলে ব্যবহারকারী আবার চাপলে একই Key যায়, আর Server আগের Booking ফেরত দেয়। Form-এ কিছু বদলালে নতুন অনুরোধ, নতুন Key:

app.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Browser

```
// ...

// Double Submit: busy থাকলে দ্বিতীয়টি উপেক্ষিত। Network ব্যর্থ হয়ে আবার
// চেষ্টা করলে একই pendingKey যায়, তাই Server-ও দ্বিতীয় Booking করে না।
onSubmit('#book', async (form) => {
  if (state.busy) return;
  state.busy = true;
  $('#book-button').disabled = true;
  state.pendingKey ??= crypto.randomUUID();
  try {
    const booking = await api('/api/bookings', { method: 'POST', body: {
      packageId: form.get('packageId'),
      travelers: Number(form.get('travelers')),
      idempotencyKey: state.pendingKey } });
    state.pendingKey = null;
    say(`Booking #${booking.id} নিশ্চিত: ${taka(booking.totalBdt)}`);
    await Promise.all([loadBookings(), loadPackages()]);
  } finally {
    state.busy = false;
    $('#book-button').disabled = false;
  }
});

// Form-এর কিছু বদলালে নতুন অনুরোধ, তাই নতুন Key।
$('#book').addEventListener('input', () => { state.pendingKey = null; });

$('#bookings').addEventListener('click', async (event) => {
  const id = event.target.dataset?.id;
```

```
  if (!id) return;
  try {
    await api(`/api/bookings/${id}/cancel`, { method: 'PATCH' });
    say(`Booking #${id} বাতিল হয়েছে।`);
    await Promise.all([loadBookings(), loadPackages()]);
  } catch (error) {
    say(error.message, true);
  }
});

$('#logout').addEventListener('click', async () => {
  await api('/api/logout', { method: 'POST' }).catch(() => {});
  signOut();
  say('Logout হয়েছে।');
});

render();
loadPackages().catch((error) => say(error.message, true));
```

HTML-এ প্রতিটি Input-এর `<label>`, বার্তার জায়গায় `role="status"` ও `aria-live="polite"` (Screen Reader বার্তা পড়ে শোনায়), এবং Keyboard দিয়ে সব কাজ করা যায় কারণ সব Control আসল `<button>`, `<input>` ও `<select>`। CSS-এ একটি Media Query ৭২০ Pixel-এর কম চওড়ায় দুই কলাম থেকে এক কলামে নামায়।

৩০.৯ Test-এর পিরামিড ও Quality Gate

স্তর	কী যাচাই করে	কোথায়	সংখ্যা
Unit	Validation-এর প্রতিটি প্রান্ত	`validation.test.mjs`	৩
Domain	দাম, Idempotency, Sold-out, মালিকানা, বাতিল	`service.test.mjs`	৫
নিরাপত্তা	Hash, একই বার্তা, Rate Limit, মেয়াদ, Logout	`auth.test.mjs`	৫
Integration (HTTP)	পুরো প্রবাহ, Status, Error চুক্তি, Header	`http.test.mjs`	৩
Concurrency	চারটি Thread, একই Database, Overbooking নয়	`concurrency.test.mjs`	১
Type	strict Mode-এ JSDoc Type	`npm run typecheck`	৭ File
Browser E2E	আসল Chrome-এ Form, Double-submit, Mobile	QA Script	১৪ যাচাই

Idempotency-র Domain Test:

service.test.mjs · JavaScript [নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code)] চলবে: Node.js

```
// ...

test('Idempotency: একই Key ও অনুরোধে একই Booking, আসন একবার কমে', () => {
  const { service, user, seats } = domain();
  const { user: rafi } = user('Rafi');
  const first = service.createBooking(rafi.id, order('key-retry-01'));
  const retry = service.createBooking(rafi.id, order('key-retry-01'));
  assert.equal(retry.booking.id, first.booking.id);
  assert.deepEqual([first.replayed, retry.replayed], [false, true]);
  assert.equal(seats('BT-4D'), 10);
  assert.throws(() => service.createBooking(rafi.id,
    order('key-retry-01', { travelers: 3 })), { code: 'KEY_CONFLICT' });
  // অন্য ব্যবহারকারীর একই Key আলাদা অনুরোধ
  const { user: mina } = user('Mina');
  assert.notEqual(service.createBooking(mina.id, order('key-retry-01'))
    .booking.id, first.booking.id);
});

// ...
```

আর সবচেয়ে কঠিন প্রমাণ—সত্যিকারের একসঙ্গে লেখা। চারটি `worker_threads`, প্রত্যেকে নিজস্ব Database সংযোগ খুলে একই File-এ দশবার করে Booking চেষ্টা করে, অথচ আসন মাত্র ২০টি:

concurrency.test.mjs · JavaScript চলবে: Node.js

```
// চারটি Thread, প্রত্যেকের নিজস্ব সংযোগ, একই SQLite File: ৪০টি চেষ্টা,
// আসন মাত্র ২০। ঠিক ২০টি Booking হতে হবে, একটিও বেশি নয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { Worker } from 'node:worker_threads';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase } from '../src/db.mjs';
import { PASSWORD } from './helpers.mjs';

test('একসঙ্গে Booking-এ Overbooking হয় না', async (t) => {
  const dir = mkdtempSync(join(tmpdir(), 'tmp-'));
  const dbPath = join(dir, 'race.db');
  openDatabase(dbPath).close(); // Schema ও Seed: CXB-3D-তে ২০ আসন
  t.after(() => rmSync(dir, { recursive: true, force: true }));

  const workers = [0, 1, 2, 3].map((index) => new Promise((resolve, reject) => {
    const worker = new Worker(new URL('./fixtures/booking-worker.mjs',
      import.meta.url), { workerData: { dbPath, index, attempts: 10,
      password: PASSWORD } });
    worker.once('message', resolve);
```

```
    worker.once('error', reject);
  }));
  const results = await Promise.all(workers);
  const booked = results.reduce((sum, r) => sum + r.booked, 0);
  const soldOut = results.reduce((sum, r) => sum + r.soldOut, 0);
  assert.deepEqual([booked, soldOut], [20, 20]);

  const db = openDatabase(dbPath);
  const { seats } = db.prepare(
    "SELECT seats FROM packages WHERE id = 'CXB-3D'").get();
  const { sold } = db.prepare(`SELECT SUM(travelers) AS sold FROM bookings
    WHERE package_id = 'CXB-3D' AND status = 'CONFIRMED'`).get();
  db.close();
  assert.deepEqual([seats, sold], [0, 20]);
});
```

ঠিক ২০টি Booking, ২০টি SOLD_OUT, শেষে আসন শূন্য এবং বিক্রীত যাত্রীর যোগফল ২০। এটি সম্ভব হয়েছে তিন স্তরের সুরক্ষায়: `BEGIN IMMEDIATE` (লেখার Lock শুরুতেই), শর্তযুক্ত UPDATE, এবং শেষ পাহারাদার `CHECK (seats >= 0)`।

Test পাস করা মানে পুরো নিরাপত্তা নিরীক্ষা বা Production-এর নির্ভুলতা প্রমাণিত নয়; এর অর্থ শুধু এই যে লিখিত Requirement-গুলো লিখিত পরিস্থিতিতে কাজ করে।

৩০.১০ Project: TravelMaster Pro চালানো ও প্রমাণ

Terminal

```
cd travel-master-pro
npm test
npm run demo
npm start
```

Demo HTTP দিয়ে প্রতিটি ব্যবসার নিয়ম দেখায়:

demo.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// HTTP দিয়ে পুরো Capstone-এর প্রতিটি নিয়ম, এক Process-এ (Memory Database)।
import { startServer } from './src/server.mjs';

const app = await startServer();
async function call(path, { method = 'GET', token, body } = {}) {
  const res = await fetch(`${app.url}${path}`, { method, headers: {
    ...(token && { Authorization: `Bearer ${token}` }),
    ...(body && { 'Content-Type': 'application/json' }) },
  body: body && JSON.stringify(body) });
  const json = await res.json();
```

```
  return { status: res.status, ...json };
}
const show = (label, { status, data, error }) => {
  const detail = error?.code ?? `${data.status}, ${data.totalBdt} টাকা`;
  console.log(`${label}: ${status} ${detail}`);
};

const people = {};
for (const name of ['Rafi', 'Mina']) {
  const email = `${name.toLowerCase()}@example.com`;
  await call('/api/register', { method: 'POST',
    body: { name, email, password: 'long-password-123' } });
  people[name] = (await call('/api/login', { method: 'POST',
    body: { email, password: 'long-password-123' } })).data.token;
}

// ...
```

demo.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const order = { packageId: 'BT-4D', travelers: 2, price: 1,
  idempotencyKey: 'demo-key-0001' };
const book = (token, body) => call('/api/bookings', { method: 'POST',
  token, body });
const first = await book(people.Rafi, order);
show('৪. Booking (Client price 1 পাঠিয়েছে)', first);
show('২. একই Key আবার', await book(people.Rafi, order));
show('৩. একই Key, ভিন্ন যাত্রী', await book(people.Rafi,
  { ...order, travelers: 3 }));
show('8. Token ছাড়া', await book(undefined, order));
show('৫. SYL-2D-তে ৩ জন', await book(people.Rafi,
  { packageId: 'SYL-2D', travelers: 3, idempotencyKey: 'demo-key-0002' }));
const cancelPath = `/api/bookings/${first.data.id}/cancel`;
show('৬. Mina বাতিল করতে চান', await call(cancelPath,
  { method: 'PATCH', token: people.Mina }));
show('৭. Rafi বাতিল করেন', await call(cancelPath,
  { method: 'PATCH', token: people.Rafi }));
const seats = (await call('/api/packages')).data
  .find((p) => p.id === 'BT-4D').seats;
console.log('BT-4D-এ আসন:', seats);
await app.close();
```

Output

```
১. Booking (Client price 1 পাঠিয়েছে): 201 CONFIRMED, 25000 টাকা
২. একই Key আবার: 200 CONFIRMED, 25000 টাকা
৩. একই Key, ভিন্ন যাত্রী: 409 KEY_CONFLICT
৪. Token ছাড়া: 401 UNAUTHORIZED
৫. SYL-2D-তে ৩ জন: 409 SOLD_OUT
৬. Mina বাতিল করতে চান: 404 NOT_FOUND
৭. Rafi বাতিল করেন: 200 CANCELLED, 25000 টাকা
BT-4D-এ আসন: 12
```

Client `price: 1` পাঠিয়েছিল, কিন্তু দাম ২ × ১২,৫০০ = ২৫,০০০ টাকা—Catalog থেকে। Mina-র জন্য অন্যের Booking "পাওয়া যায়নি" (404), "নিষিদ্ধ" (403) নয়—যাতে অস্তিত্বের তথ্যও ফাঁস না হয়। শেষে আসন আবার ১২: Booking-এ ২টি কমেছিল, বাতিলে ২টি ফিরেছে, দ্বিতীয় অনুরোধে কিছুই বদলায়নি।

Output

```
# tests 17
# pass 17
# fail 0
```

QA-তে যা চালানো হয়েছে: ১৭টি Test (Node.js 22.16.0); Demo; TypeScript 6.0.2 দিয়ে strict Type Check—`src/`-এর সাতটি File-এ শূন্য Error, এবং প্রমাণ হিসেবে একটি ইচ্ছাকৃত Type-ভুল সঠিকভাবে ধরা পড়েছে; এবং আসল Headless Chrome 155-এ Frontend-এর ১৪টি যাচাই: Catalog, নিবন্ধন, ভুল Password-এর বার্তা, Login, Server-এর দামে Booking, দুটি দ্রুত Submit-এ ঠিক একটি Booking, Catalog-এর আসন হালনাগাদ, Sold-out বার্তা, বাতিল ও আসন ফেরত, ৩৭৫ Pixel চওড়ায় কোনো আনুভূমিক Scroll নেই, Logout, এবং কোনো Console Error বা CSP লঙ্ঘন নেই।

Feature-এর সৎ হিসাব:

Feature	অবস্থা
Catalog, নিবন্ধন, Login, Logout	বাস্তবায়িত ও পরীক্ষিত
Server-নির্ধারিত দাম, Idempotency, Inventory Transaction	বাস্তবায়িত ও পরীক্ষিত (Concurrency-সহ)
নিজের Booking, বাতিল, মালিকানা	বাস্তবায়িত ও পরীক্ষিত
Token-এর মেয়াদ, Login-এর ভুলের সীমা	বাস্তবায়িত ও পরীক্ষিত (সীমা শুধু এক Process-এ)
Browser Frontend	বাস্তবায়িত; আসল Chrome-এ পরীক্ষিত
Payment, Refund, Webhook	**বাস্তবায়িত নয়**
Email যাচাই, Password Reset	**বাস্তবায়িত নয়**
HTTPS, Docker, CI, Monitoring	এই Project-এ নয় (অধ্যায় ২৮-এ আলাদাভাবে)

Feature	অবস্থা
Screen Reader-এ আসল পরীক্ষা	**করা হয়নি**; শুধু Semantic HTML ও ARIA

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`node:sqlite` পাওয়া যায় না	Node.js 22.5-এর পুরোনো	Node.js হালনাগাদ
"database is locked"	দীর্ঘ Transaction বা অন্য Process	Transaction ছোট; `busy_timeout`
Login-এর পর Reload করলে আবার Login	Token Memory-তে (নকশা অনুযায়ী)	স্বাভাবিক; অনুচ্ছেদ ৩০.৮
Browser Console-এ CSP Error	Inline Script বা Style যোগ করা হয়েছে	আলাদা File-এ সরান
বারবার 429	পাঁচবার ভুল Password	১৫ মিনিট পরে
Restart-এর পর Data নেই	`DB_PATH=:memory:`	File Path দিন

৩০.১১ Production-এর জন্য উন্নয়ন-পরিকল্পনা

১. **PostgreSQL:** একই Schema; `SELECT ... FOR UPDATE` বা শর্তযুক্ত UPDATE; Connection Pool (অধ্যায় ২৬)।

২. **Migration:** সংস্করণভিত্তিক File, Expand/Contract (অধ্যায় ২৬, ২৮)।

৩. **Payment:** Booking-এর অবস্থা PENDING_PAYMENT; Gateway-এর Idempotency-Key; Webhook-এর Signature যাচাই ও Replay সুরক্ষা; নির্দিষ্ট সময়ে Payment না হলে আসন ছাড়া; Refund ও প্রতিদিনের Reconciliation (অধ্যায় ২৯)।

৪. **Session:** HttpOnly, Secure, SameSite Cookie; CSRF Token; Token Rotation; Rate Limit একটি ভাগ-করা Store-এ।

৫. **পর্যবেক্ষণ:** Request ID, Structured Log, Latency ও Error-এর Metrics, SLO-ভিত্তিক Alert (অধ্যায় ২৮)।

৬. **Delivery:** Docker Image, CI, Staging, ধাপে ধাপে Rollout, Backup ও Restore Drill, Rollback (অধ্যায় ২৮)।

৭. **গোপনীয়তা:** ন্যূনতম তথ্য, সংরক্ষণের মেয়াদ, প্রবেশাধিকার নিয়ন্ত্রণ, Secret Manager।

৮. **Accessibility:** আসল Screen Reader ও শুধু-Keyboard ব্যবহারে পরীক্ষা।

কখনো এই Demo Server সরাসরি Internet-এ একটি আসল ভ্রমণ-Booking সেবা হিসেবে চালাবেন না।

৩০.১২ Architecture Decision Record

ADR একটি ছোট Document: কোন প্রেক্ষাপটে কী সিদ্ধান্ত, কোন বিকল্প বাদ গেল, এবং তার মূল্য কী। এক বছর পরে নতুন সহকর্মী জানতে চাইবেন "কেন SQLite?"—উত্তর লেখা থাকবে।

ADR-001: Database হিসেবে SQLite (`node:sqlite`)। প্রেক্ষাপট: পাঠককে কোনো Package বা Server Install ছাড়াই আসল Transaction ও Constraint দেখাতে হবে। সিদ্ধান্ত: Node.js-এর অন্তর্নির্মিত SQLite, WAL Mode, `BEGIN IMMEDIATE`। বিকল্প: in-memory Map (Transaction ও Constraint নেই, তাই বাদ); PostgreSQL (আলাদা Server লাগে)। মূল্য: একটি Server-এই সীমিত; একাধিক Instance-এ PostgreSQL লাগবে, তবে Schema ও Query প্রায় একই থাকবে।

ADR-002: Token Browser-এর Memory-তে। প্রেক্ষাপট: XSS-এর ঝুঁকি ও সরলতা। সিদ্ধান্ত: Bearer Token শুধু JavaScript Variable-এ, কঠোর CSP-সহ। বিকল্প: `localStorage` (XSS পড়তে পারে, বাদ); HttpOnly Cookie (সবচেয়ে নিরাপদ, কিন্তু CSRF সুরক্ষা লাগে—Production-এর পরিকল্পনায়)। মূল্য: Reload-এ আবার Login।

ADR-003: Framework ছাড়া Transport ও Frontend। প্রেক্ষাপট: শেখার Capstone, যেখানে প্রতিটি স্তর দৃশ্যমান হওয়া জরুরি। সিদ্ধান্ত: `node:http` ও সাধারণ DOM। মূল্য: বড় দলে বা বড় UI-তে Express/Fastify ও একটি UI Framework সময় বাঁচাবে; Domain Service যেহেতু HTTP থেকে আলাদা, সেই বদল সীমিত।

৩০.১৩ ১০০ নম্বরের মূল্যায়ন

ক্ষেত্র	নম্বর	যা দেখা হয়
Requirement ও User Flow	১০	প্রান্তিক ক্ষেত্র স্পষ্ট
JavaScript-এর নকশা	১৫	Module, Validation, স্তর-বিভাজন
API ও Authentication	১৫	সঠিক Status, মালিকানা
Data-র সামঞ্জস্য	১৫	Idempotency, Inventory, Transaction
Test ও Debugging	১৫	স্বয়ংক্রিয় ও অর্থপূর্ণ Test
নিরাপত্তা ও গোপনীয়তা	১০	হুমকি ও প্রতিরোধ
Deployment ও Monitoring-এর পরিকল্পনা	১০	প্রস্তুতির Checklist
Documentation ও যোগাযোগ	১০	README, ADR, Demo

ব্যাখ্যা: ৮৫-১০০ শক্তিশালী Capstone; ৭০-৮৪ ভালো ভিত্তি; ৫০-৬৯ নির্দিষ্ট উন্নতি দরকার; ৫০-এর নিচে আগে Requirement ও Test। এটি শেখার মানদণ্ড, কোনো সরকারি সনদ নয়। নিজের কাজ মূল্যায়নের একটি সাহায্যকারী:

`rubric.mjs` · JavaScript — চলবে: Node.js

```
// ১০০ নম্বরের মূল্যায়ন: প্রতিটি ক্ষেত্রে নিজের নম্বর দিন, তারপর ব্যাখ্যা।
```

```
const rubric = [
  ['Requirement ও User Flow', 10],
  ['JavaScript-এর নকশা', 15],
  ['API ও Authentication', 15],
  ['Data-র সামঞ্জস্য', 15],
  ['Test ও Debugging', 15],
  ['নিরাপত্তা ও গোপনীয়তা', 10],
  ['Deployment ও Monitoring-এর পরিকল্পনা', 10],
  ['Documentation ও যোগাযোগ', 10],
];

function assess(scores) {
  const total = rubric.reduce((sum, [area, max]) => {
    const given = scores[area] ?? 0;
    if (given < 0 || given > max) throw new RangeError(`${area}: 0-${max}`);
    return sum + given;
  }, 0);
  const band = total >= 85 ? 'শক্তিশালী Capstone'
    : total >= 70 ? 'ভালো ভিত্তি'
      : total >= 50 ? 'নির্দিষ্ট উন্নতি দরকার' : 'Requirement ও Test আগে';
  const weakest = rubric.map(([area, max]) => [area, (scores[area] ?? 0) / max])
    .sort((a, b) => a[1] - b[1])[0][0];
  return { total, band, weakest };
}

console.log(assess({ 'Requirement ও User Flow': 9, 'JavaScript-এর নকশা': 13,
  'API ও Authentication': 13, 'Data-র সামঞ্জস্য': 14, 'Test ও Debugging': 13,
  'নিরাপত্তা ও গোপনীয়তা': 7, 'Deployment ও Monitoring-এর পরিকল্পনা': 5,
  'Documentation ও যোগাযোগ': 8 }));
```

Output

```
{
  total: 82,
  band: 'ভালো ভিত্তি',
  weakest: 'Deployment ও Monitoring-এর পরিকল্পনা'
}
```

সবচেয়ে দুর্বল ক্ষেত্রটি নম্বরের অনুপাতে বের করা হয় (৫/১০ = ৫০%), মোট নম্বরে নয়—উন্নতির পরের পদক্ষেপ সেখান থেকেই।

৩০.১৪ পেশাদার Roadmap: ১২ সপ্তাহ

সপ্তাহ	মনোযোগ	যা তৈরি করবেন
১-২	ভিত্তি পুনরালোচনা, Async, Git	Test-সহ একটি Utility Library

সপ্তাহ	মনোযোগ	যা তৈরি করবেন
৩-৪	DOM, Accessibility, API ডাক	Responsive Package Catalog
৫-৬	Node.js, Auth, REST	Booking API
৭-৮	SQL, Transaction, ORM	টেকসই Booking প্রবাহ
৯-১০	Test, TypeScript, নিরাপত্তা	CI-তে পরীক্ষিত Refactor
১১	Docker, Deploy, Monitoring	Staging Demo
১২	Portfolio, README, Interview	Case Study ও Walkthrough

Portfolio: শুধু Screenshot নয়। প্রতিটি Project-এর জন্য লিখুন: সমস্যা কী ছিল; Architecture ও কেন; কোন Trade-off নিলেন; কীভাবে প্রমাণ করলেন (Test, Demo); কী সীমাবদ্ধতা আছে; এবং আবার করলে কী অন্যভাবে করতেন। TravelMaster Pro-র README-র "Requirement থেকে প্রমাণ" ও "যা নেই" অংশ এর একটি নমুনা। Git-এর ইতিহাস পরিচ্ছন্ন রাখুন—ছোট, অর্থপূর্ণ Commit।

Interview-এর প্রস্তুতি: Event Loop ও Microtask; Closure ও Scope; Async Error সামলানো; Authentication বনাম Authorization; SQL Transaction ও Race Condition; Caching ও তার Invalidation; Idempotency; এবং একটি ছোট System Design ("একটি Booking ব্যবস্থা নকশা করুন")। প্রতিটি উত্তরে একটি নিজের Code-এর উদাহরণ দিতে পারলে—"আমার Capstone-এ এই Race Condition এভাবে ধরেছিলাম"—তা যেকোনো মুখস্থ সংজ্ঞার চেয়ে শক্তিশালী।

শেখা চালিয়ে যাওয়া: প্রতিটি নতুন প্রযুক্তি শেখার আগে নিজেকে জিজ্ঞাসা করুন: এটি কোন সমস্যার সমাধান করে? এর মূল্য কী? এটি ছাড়া কীভাবে করতাম? আর একটি Open Source Project-এ ছোট অবদান—একটি Documentation সংশোধন বা একটি Bug-এর Test—আসল দলের সঙ্গে কাজের অভ্যাস তৈরি করে।

৩০.১৫ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Client-এর পাঠানো দাম বিশ্বাস	যে কেউ ১ টাকায় Booking	দাম সব সময় Server-এর Catalog থেকে
`typeof` দিয়ে সংখ্যা যাচাই	NaN, Infinity, ভগ্নাংশ পার হয়	`Number.isInteger` ও সীমা
পড়ে, তারপর লেখা (Inventory)	একসঙ্গে অনুরোধে Overbooking	শর্তযুক্ত UPDATE, Transaction, CHECK
Password সাধারণ লেখায় বা দ্রুত Hash-এ	ফাঁস হলে সব Account বিপদে	scrypt বা Argon2id, প্রতিজনের Salt
Token সরাসরি Database-এ	Database ফাঁস মানে Session চুরি	Token-এর Hash

‹‹‹‹

ভুল	কেন সমস্যা	সমাধান
অন্যের Booking-এ 403	কোন ID আছে তা ফাঁস	404, এবং Query-তেই মালিকানা
Double-submit শুধু Button নিষ্ক্রিয় করে ঠেকানো	Network Retry-তে তবু দুটি	Idempotency Key, Server-এ UNIQUE
`innerHTML` দিয়ে Server-এর লেখা বসানো	XSS	`textContent`
Token `localStorage`-এ	XSS Script পড়ে নেয়	Memory বা HttpOnly Cookie
"Test পাস, তাই Production-প্রস্তুত"	নিরাপত্তা, Scale, Operations অপরীক্ষিত	সৎ সীমাবদ্ধতার তালিকা ও পরিকল্পনা

৩০.১৬ Debugging নির্দেশিকা

১. **Response-এর `error.code` আগে:** VALIDATION হলে `details`; UNAUTHORIZED হলে Token ও মেয়াদ; KEY_CONFLICT হলে একই Key-তে ভিন্ন Body।

২. **`audit_events` Table:** কে কখন Login করল, কোন Booking তৈরি বা বাতিল হলো—সময়ের ক্রমে।

৩. **আসনের হিসাব মিলানো:** `seats` + CONFIRMED Booking-এর যাত্রীর যোগফল = প্রাথমিক আসন। না মিললে কোথাও Transaction-এর বাইরে লেখা হয়েছে।

৪. **Browser-এর DevTools:** Network Tab-এ Request ও Response; Console-এ CSP লঙ্ঘন।

৫. **Domain আলাদা করে:** Bug HTTP-তে না Domain-এ? `service` সরাসরি ডেকে দেখুন—Test-এর `domain()` Helper-এর মতো।

৬. **সময়-নির্ভর Bug:** ঘড়ি Inject করা (`now`), তাই মেয়াদ বা Rate Limit-এর Bug সঙ্গে সঙ্গে পুনরুৎপাদন করা যায়।

৩০.১৭ পেশাদার অভ্যাস

- **প্রতিটি Requirement একটি Test-এ প্রমাণিত, প্রতিটি Test একটি Requirement-এর জন্য।**
- **Trust Boundary-তে যাচাই, ভেতরে Type-এর নিশ্চয়তা।**
- **Database-এর Constraint শেষ পাহারাদার।**
- **নিরাপত্তার ডিফল্ট কঠোর:** CSP, `no-store`, সাধারণ Error-বার্তা, ন্যূনতম তথ্য।
- **সিদ্ধান্ত ADR-এ, সীমাবদ্ধতা README-তে।**
- **যা করা হয়নি তা স্পষ্ট বলুন**—সৎ "বাস্তবায়িত নয়" একটি মিথ্যা "সম্পূর্ণ"-এর চেয়ে অনেক বেশি পেশাদার।

৩০.১৮ অনুশীলনী

প্রাথমিক স্তর

১. `TEST@EXAMPLE.COM` দিয়ে নিবন্ধনের পর `test@example.com` দিয়ে আবার নিবন্ধন করলে কী হয় এবং কেন?

২. Client Booking-এর Body-তে `{ "price": 1 }` পাঠালে মোট দাম কত হয়? কোন Line এটি নিশ্চিত করে?

৩. একই Idempotency Key ও একই Body দুবার পাঠালে কী হয়: Status, Booking ID, আসন?

৪. Token ছাড়া `GET /api/bookings` পাঠালে Response কী, এবং কেন Body পড়ার আগেই তা ঘটে?

মধ্যম স্তর

৫. একই Key-তে `travelers` বদলে পাঠালে 409 কেন, নীরবে আগের ফল নয় কেন?

৬. ব্যবহারকারী A-র Booking ব্যবহারকারী B বাতিল করতে চাইলে 404 কেন, 403 নয় কেন?

৭. একই Booking দুবার বাতিল করলে আসন কতবার বাড়ে? Code-এর কোন অংশ তা নিশ্চিত করে?

৮. Test লিখুন: মেয়াদের এক Millisecond আগে Token বৈধ, ঠিক মেয়াদে অবৈধ; এবং মেয়াদোত্তীর্ণ Token-এ Booking হয় না।

চ্যালেঞ্জ স্তর

৯. Schema-র প্রতিটি Constraint কোন Requirement রক্ষা করে? একটি নতুন Constraint প্রস্তাব করুন।

১০. Production-এ যাওয়ার আগে অগ্রাধিকার অনুযায়ী পাঁচটি কাজ লিখুন এবং কেন এই ক্রম।

১১. Catalog-এ দামের সীমা, শুধু খালি আসনের Package এবং Keyset Pagination যোগ করুন।

১২. Payment-এর জন্য "Reservation Expiry" নকশা করুন: Booking PENDING_PAYMENT অবস্থায় ১৫ মিনিট আসন ধরে রাখবে, তারপর ছেড়ে দেবে।

৩০.১৯ অনুশীলনীর সমাধান

সমাধান ১ — দ্বিতীয় নিবন্ধন 409 EMAIL_TAKEN পায়। `validateRegistration` Email-কে `trim().toLowerCase()` করে, তাই দুটিই `test@example.com` হিসেবে Database-এ যায়, আর `users.email`-এর UNIQUE Constraint দ্বিতীয়টি প্রত্যাখ্যান করে। `register` সেই Constraint-এর Error ধরে বোধগম্য বার্তা দেয়। স্বাভাবিক না করলে একই মানুষ দুটি Account পেতেন, আর Login-এ কোন Account—তা বড়-ছোট হাতের অক্ষরের ওপর নির্ভর করত (`auth.test.mjs`-এর দ্বিতীয় Test)।

সমাধান ২ — ২৫,০০০ টাকা (BT-4D, ২ জন × ১২,৫০০)। `validateBooking` শুধু `packageId`, `travelers` ও `idempotencyKey` ফেরত দেয়— `price` Field Domain-এ পৌঁছায়ই না। আর `createBooking`-এ মোট দাম `pkg.price_bdt * travelers`, যেখানে `pkg` Database-এর Catalog থেকে পড়া। Demo-র প্রথম Line এবং `service.test.mjs`-এর প্রথম Test এটি প্রমাণ করে।

সমাধান ৩ — প্রথমবার 201, দ্বিতীয়বার 200; Booking ID একই; আসন একবারই কমে। Transaction-এর শুরুতে `(user_id, idempotency_key)` দিয়ে খোঁজা হয়, পাওয়া গেলে অনুরোধ মিলিয়ে আগের Booking `replayed: true` সহ ফেরত, এবং HTTP স্তর তা দেখে 200 দেয়। Database-এর UNIQUE Constraint নিশ্চিত করে যে দুটি একসঙ্গে আসা অনুরোধেও দুটি সারি হবে না।

সমাধান ৪ — 401, Body: `{ "error": { "code": "UNAUTHORIZED", "message": "Login প্রয়োজন" } }`। Route Handler-এ প্রথম কাজ `user(req)`, যা `service.authenticate` ডাকে; Token না থাকলে সেখানেই AppError। Body পড়ার আগে Authentication মানে পরিচয়হীন কেউ বড় Body পাঠিয়ে Server-এর সময় ও Memory নষ্ট করতে পারে না।

সমাধান ৫ — একই Key মানে Client দাবি করছে "এটি সেই আগের অনুরোধ", অথচ বিষয়বস্তু ভিন্ন—এটি Client-এর Bug (যেমন প্রতিটি Retry-তে নতুন Form-মান কিন্তু পুরোনো Key)। নীরবে আগের ফল দিলে ব্যবহারকারী ভাববেন তিনজনের Booking হয়েছে, আসলে হয়েছে দুজনের। 409 KEY_CONFLICT স্পষ্টভাবে অমিল জানায়। TravelMaster-এর Frontend Form-এ কিছু বদলালেই `pendingKey` মুছে দেয়, তাই সাধারণ ব্যবহারে এটি ঘটে না।

সমাধান ৬ — 403 বলত "এই Booking আছে, কিন্তু আপনার নয়"। তাহলে কেউ ID ১, ২, ৩... দিয়ে চেষ্টা করে জানতে পারত কতগুলো Booking আছে, কোনগুলো সক্রিয়—ব্যবসার তথ্য ও গোপনীয়তা ফাঁস (Enumeration)। 404 বলে "আপনার জন্য এমন কিছু নেই", যা সত্য। Query-তে `WHERE id = ? AND user_id = ?` থাকায় অন্যের Booking Domain-এর চোখে অস্তিত্বহীন।

সমাধান ৭ — একবার। `cancelBooking` Transaction-এ সারি পড়ে; `status` আগে থেকেই CANCELLED হলে কোনো UPDATE না করে একই ফল ফেরত দেয়। আসন ফেরত দেওয়া ও অবস্থা বদলানো একই Transaction-এ, তাই দুটি একসঙ্গে আসা বাতিল-অনুরোধেও দ্বিতীয়টি প্রথমটির Commit-এর পরে CANCELLED দেখবে (`BEGIN IMMEDIATE`-এর Lock)।

সমাধান ৮

token-expiry.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { domain } from '../../project/travel-master-pro/tests/helpers.mjs';
```

```
test('মেয়াদের এক মুহূর্ত আগে বৈধ, ঠিক মেয়াদে অবৈধ', () => {
  const { service, user, clock } = domain({ tokenTtlMs: 30 * 60_000 });
  const { token } = user('Rafi');
  clock.now += 30 * 60_000 - 1;
  assert.equal(service.authenticate(token).name, 'Rafi');
  clock.now += 1;
  assert.throws(() => service.authenticate(token), { code: 'UNAUTHORIZED' });
});

test('মেয়াদ শেষে Booking-এর চেষ্টাও 401-এর পথে', () => {
  const { service, user, clock } = domain({ tokenTtlMs: 1000 });
  const { token } = user('Mina');
  clock.now += 5000;
  assert.throws(() => {
    const me = service.authenticate(token); // HTTP স্তর আগে এটিই ডাকে
    service.createBooking(me.id, { packageId: 'BT-4D', travelers: 1,
      idempotencyKey: 'late-key-0001' });
  }, { code: 'UNAUTHORIZED' });
});
```

Output

```
ok 1 - মেয়াদের এক মুহূর্ত আগে বৈধ, ঠিক মেয়াদে অবৈধ
ok 2 - মেয়াদ শেষে Booking-এর চেষ্টাও 401-এর পথে
# tests 2
# pass 2
# fail 0
```

ঘড়ি Inject করা বলে Test-এ ৩০ মিনিট অপেক্ষা লাগে না। সীমার দুই পাশে (এক Millisecond আগে ও ঠিক মেয়াদে) পরীক্ষা করা Boundary Testing-এর মূল নিয়ম— `<` বনাম `<=` ধরনের ভুল এখানেই ধরা পড়ে।

সমাধান ৯ — `users.email UNIQUE`: এক Email এক Account (FR2)। `sessions.token_hash PRIMARY KEY` ও `expires_at`: স্বল্পমেয়াদি Session (FR3)। `packages.price_bdt CHECK (> 0)`: শূন্য বা ঋণাত্মক দামে বিক্রি নয় (FR4)। `packages.seats CHECK (>= 0)`: Overbooking নয় (FR5, Concurrency)। `bookings.user_id`, `package_id` FOREIGN KEY: অস্তিত্বহীন ব্যবহারকারী বা Package-এ Booking নয়। `travelers CHECK (BETWEEN 1 AND 10)`: Validation-এর দ্বিতীয় স্তর। `status CHECK (IN ...)`: অজানা অবস্থা নয়। `UNIQUE (user_id, idempotency_key)`: Idempotency (FR5)। প্রস্তাব: `CHECK (total_bdt % travelers = 0)` নয় (দাম বদলালে ভাঙবে); বরং একটি `price_at_booking` Column, যাতে পরে Catalog-এর দাম বদলালেও Booking-এর দাম ইতিহাসে ঠিক থাকে।

সমাধান ১০ — (১) **Payment** (PENDING_PAYMENT, Webhook যাচাই, Expiry): এটি ছাড়া ব্যবসাই চলে না। (২) **HTTPS ও Cookie-ভিত্তিক Session, CSRF:** Token Network-এ খোলা থাকলে বাকি সব নিরাপত্তা অর্থহীন। (৩) **PostgreSQL ও Backup/Restore Drill:** টেকসই Data এবং হারালে ফেরানোর প্রমাণিত পথ। (৪) **ভাগ-করা Rate Limit ও Monitoring (Log, Metrics, Alert):** আক্রমণ ও ব্যর্থতা দেখতে পাওয়া। (৫) **CI ও ধাপে ধাপে Deploy, Rollback:** প্রতিটি পরিবর্তন নিরাপদে। ক্রমের যুক্তি: আগে যা ছাড়া সেবা অর্থহীন বা বিপজ্জনক, তারপর যা ব্যর্থতা থেকে রক্ষা করে, তারপর যা পরিবর্তনকে নিরাপদ করে।

সমাধান ১১

catalog-filter.mjs · JavaScript — চলবে: Node.js

```
import { openDatabase } from '../../project/travel-master-pro/src/db.mjs';

// Catalog-এ দামের সীমা, শুধু খালি আসনের Package, এবং Keyset Pagination।
// সব মান Parameter-এ (?), তাই SQL Injection-এর সুযোগ নেই।
function listPackages(db, { maxPrice = Number.MAX_SAFE_INTEGER,
  onlyAvailable = false, limit = 2, after = '' } = {}) {
  if (!Number.isInteger(limit) || limit < 1 || limit > 50) {
    throw new RangeError('limit: ১-৫০');
  }
  const rows = db.prepare(`SELECT id, price_bdt AS price, seats FROM packages
    WHERE price_bdt <= ? AND (? = 0 OR seats > 0) AND id > ?
    ORDER BY id LIMIT ?`)
    .all(maxPrice, onlyAvailable ? 1 : 0, after, limit + 1)
    .map((row) => ({ ...row }));
  const page = rows.slice(0, limit);
  return { page, next: rows.length > limit ? page.at(-1).id : null };
}

const db = openDatabase();
db.exec("UPDATE packages SET seats = 0 WHERE id = 'SYL-2D'");
let cursor = '';
do {
  const { page, next } = listPackages(db, { maxPrice: 13_000,
    onlyAvailable: true, limit: 1, after: cursor });
  console.log(page.map((p) => `${p.id} ${p.price}`).join(' | '),
    next ? `(পরের পাতা: ${next}-এর পর)` : '(শেষ)');
  cursor = next;
} while (cursor);
```

Output

```
BT-4D 12500 (পরের পাতা: BT-4D-এর পর)
CXB-3D 9800 (শেষ)
```

একটি বেশি সারি (`limit + 1`) চেয়ে জানা যায় পরের পাতা আছে কি না, আলাদা COUNT Query ছাড়াই। Keyset (`id > ?`) OFFSET-এর মতো গভীর পাতায় ধীর হয় না (অধ্যায় ২৬)। API-তে এটি `GET /api/packages?maxPrice=13000&available=1&after=BT-4D` হবে, আর Query String-এর প্রতিটি মান একই কঠোরতায় যাচাই করতে হবে।

সমাধান ১২ — Booking তৈরিতে অবস্থা `PENDING_PAYMENT` ও `hold_expires_at = now + 15 মিনিট`; আসন তখনই কমে (শর্তযুক্ত UPDATE, আগের মতো)। Payment Gateway-এর Webhook এলে (Signature যাচাই, Event ID-তে Inbox) একটি Transaction-এ `PENDING_PAYMENT` থেকে `CONFIRMED` —শুধু যদি তখনো PENDING থাকে। একটি Background Job প্রতি মিনিটে `WHERE status = 'PENDING_PAYMENT' AND hold_expires_at < now` খুঁজে প্রতিটিকে Transaction-এ `EXPIRED` করে আসন ফেরত দেয়। সূক্ষ্ম ক্ষেত্র: মেয়াদ শেষের পরে Payment-এর Webhook এলে—আসন হয়তো অন্যকে বিক্রি হয়েছে—তাই স্বয়ংক্রিয় Refund ও গ্রাহককে জানানো। Status-এর CHECK-এ নতুন দুটি মান যোগ হবে (একটি Migration)।

৩০.২০ MCQ: নিজেকে যাচাই করুন

> **প্রশ্ন ১.** Booking-এর দাম Server-এ হিসাব করার প্রধান কারণ?
>
> ক) UI দ্রুত হয়
>
> খ) Client-এর পাঠানো মান বদলানো যায়, তাই বিশ্বাসযোগ্য নয়
>
> গ) CSS সহজ হয়
>
> ঘ) Cache ভালো কাজ করে

> **প্রশ্ন ২.** Idempotency Key-এর উদ্দেশ্য?
>
> ক) একই অনুরোধ বারবার এলেও কাজ একবারই
>
> খ) SEO উন্নতি
>
> গ) ছবি ছোট করা
>
> ঘ) Login দ্রুত করা

> **প্রশ্ন ৩.** Authentication ও Authorization-এর পার্থক্য?
>
> ক) দুটি একই
>
> খ) Network বনাম Disk
>
> গ) পরিচয় যাচাই বনাম অনুমতি যাচাই
>
> ঘ) HTML বনাম CSS

প্রশ্ন ৪. একসঙ্গে আসা অনুরোধে Overbooking ঠেকাতে কী দরকার?

ক) `setTimeout`

খ) `localStorage`

গ) একটি `alert`

ঘ) শর্তযুক্ত UPDATE বা Lock-সহ Database Transaction

প্রশ্ন ৫. TravelMaster-এ মেয়াদোত্তীর্ণ Token-এ কী উত্তর আসে?

ক) 401

খ) 404

গ) 409

ঘ) 200

প্রশ্ন ৬. অন্যের Booking বাতিলের চেষ্টায় TravelMaster 404 দেয় কেন?

ক) Server-এর Bug

খ) Booking-এর অস্তিত্বের তথ্যও ফাঁস না করতে

গ) 403 HTTP-তে নেই

ঘ) Booking মুছে ফেলা হয়েছে

প্রশ্ন ৭. Database-এ Token-এর বদলে তার Hash রাখার সুবিধা?

ক) Query দ্রুত

খ) Database ছোট

গ) Database ফাঁস হলেও চুরি করা মান দিয়ে Login করা যায় না

ঘ) Token-এর মেয়াদ বাড়ে

প্রশ্ন ৮. ADR কী লিখে রাখে?

ক) Architecture-এর সিদ্ধান্ত, বিকল্প ও তার মূল্য

খ) Animation

গ) শুধু CSS

ঘ) Invoice

প্রশ্ন ৯. Frontend-এ Server-এর লেখা DOM-এ বসাতে কোনটি নিরাপদ?

ক) `innerHTML`

খ) `document.write`

গ) `eval`

ঘ) `textContent`

> **প্রশ্ন ১০.** Portfolio-র Case Study-তে সবচেয়ে জরুরি কী?
>
> ক) শুধু Screenshot
>
> খ) শুধু রং
>
> গ) সমস্যা, সিদ্ধান্ত, প্রমাণ ও সীমাবদ্ধতা
>
> ঘ) শুধু শিরোনাম

৩০.২১ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ক · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Demo-তে Client `price: 1` পাঠালেও ২৫,০০০ টাকা।
২	ক	একই Key-তে দ্বিতীয় অনুরোধ আগের Booking ফেরত দেয়, আসন কমে না।
৩	গ	Login পরিচয় দেয়; মালিকানা যাচাই অনুমতি ঠিক করে।
৪	ঘ	উদাহরণ ০২ ও Concurrency Test: চার Thread-এও ঠিক ২০টি।
৫	ক	`authenticate`-এ `expires_at <= now()` হলে UNAUTHORIZED।
৬	খ	403 বললে ID অনুমান করে কোন Booking আছে তা জানা যেত।
৭	গ	Hash থেকে আসল Token ফেরানো যায় না।
৮	ক	যেমন ADR-001: কেন SQLite, কোন বিকল্প, কী মূল্য।
৯	ঘ	`textContent` লেখাকে লেখা হিসেবেই রাখে, Code হিসেবে চালায় না।
১০	গ	নিয়োগকর্তা দেখতে চান আপনি কীভাবে ভাবেন ও প্রমাণ করেন।

৩০.২২ অধ্যায়ের সারসংক্ষেপ

- Mastery মানে Correctness, নিরাপত্তা, রক্ষণযোগ্যতা ও পর্যবেক্ষণ মিলিয়ে সমস্যা সমাধান, এবং সিদ্ধান্তের ব্যাখ্যা।
- Requirement থেকে API চুক্তি, প্রতিটি Requirement একটি Code ও একটি Test-এ প্রমাণিত।
- Trust Boundary-তে কঠোর যাচাই; `typeof` যথেষ্ট নয়।
- scrypt ও Salt, সমান-সময়ের তুলনা, একই Error-বার্তা, Token-এর Hash, মেয়াদ ও Rate Limit।
- দাম Server-এর; Idempotency Key ও UNIQUE; শর্তযুক্ত UPDATE ও Transaction; Constraint শেষ পাহারাদার।
- Domain ও Transport আলাদা; একটি Error চুক্তি; CSP ও `no-store`।
- Frontend: `textContent`, Memory-তে Token, দুই স্তরের Double-submit সুরক্ষা, Accessibility-র ভিত্তি।

- Test-এর পিরামিড: Unit থেকে Concurrency, Type Check ও আসল Browser।
- ADR, সৎ সীমাবদ্ধতা, Production-এর পরিকল্পনা, ১০০ নম্বরের মূল্যায়ন ও ১২ সপ্তাহের Roadmap।

৩০.২৩ বই শেষ, শেখা নয়

এই ত্রিশটি অধ্যায়ের লক্ষ্য ছিল আপনাকে নিজে Engineering-এর সিদ্ধান্ত নিতে সক্ষম করা। আপনি একটি নাম দিয়ে শুরু করেছিলেন—একটি Variable, একটি অভিবাদন—আর শেষ করলেন এমন একটি ব্যবস্থায়, যেখানে চারটি Thread একসঙ্গে শেষ আসন চাইলেও হিসাব মেলে। পথে শিখেছেন যে ভালো Code শুধু কাজ করে না; সে জানায় কখন কাজ করছে না, কেন, এবং কীভাবে ঠিক করতে হবে।

প্রযুক্তি বদলাবে: নতুন Framework, নতুন Runtime, নতুন হাতিয়ার আসবে। কিন্তু এই বইয়ের মূল প্রশ্নগুলো বদলাবে না—Input কি বিশ্বাসযোগ্য? একসঙ্গে দুজন এলে কী হবে? ব্যর্থ হলে কে জানবে? কীভাবে প্রমাণ করব? এই প্রশ্নগুলো যেকোনো নতুন প্রযুক্তিতে নিয়ে যান। শুভকামনা।

৩০.২৪ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Capstone	শেখা সব দক্ষতা একত্র করা চূড়ান্ত Project।
Trust Boundary	যেখানে বাইরের অবিশ্বস্ত তথ্য ব্যবস্থায় ঢোকে।
Traceability	Requirement থেকে Code ও Test পর্যন্ত যোগসূত্র।
Salt	প্রতিটি Password-এর জন্য আলাদা এলোমেলো মান।
Timing-safe Comparison	তুলনার সময় দেখে কিছু বোঝা যায় না এমন তুলনা।
Session Token	Login-এর পরে পরিচয় বহনকারী গোপন মান।
Enumeration	ID অনুমান করে কী আছে তা খুঁজে বের করা।
Conditional Update	যাচাই ও পরিবর্তন একই Statement-এ।
Content-Security-Policy	কোন উৎসের Script বা Style চলবে, তার নিয়ম।
Composition Root	যেখানে সব স্তর জোড়া লাগানো হয়।
ADR	Architecture-এর সিদ্ধান্তের লিখিত নথি।
Quality Gate	Release-এর আগে যে যাচাইগুলো পার হতে হয়।

৩০.২৫ আরও পড়ুন

- **OWASP — Password Storage Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html` — scrypt, Argon2id ও Parameter।
- **OWASP — Session Management Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html` — Token, Cookie ও মেয়াদ।
- **OWASP — Authorization Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html` — মালিকানা যাচাই ও IDOR।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP` — CSP-র প্রতিটি Directive।
- **Node.js — SQLite:** `https://nodejs.org/api/sqlite.html` — `node:sqlite`-এর API।
- **Michael Nygard — Documenting Architecture Decisions:** `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions` — ADR-এর মূল প্রস্তাব।
- **web.dev — Learn Accessibility:** `https://web.dev/learn/accessibility` — Accessible UI-র ভিত্তি।

পরিশিষ্ট ক — পরিভাষা-কোষ

প্রতিটি অধ্যায়ের শেষের পরিভাষার সারণি এখানে একত্র করা হয়েছে, বর্ণানুক্রমে (ইংরেজি পরিভাষা আগে)। একই পরিভাষা একাধিক অধ্যায়ে থাকলে প্রথম অধ্যায়ের সংজ্ঞা দেওয়া হয়েছে, আর ডান কলামে সব অধ্যায়ের নম্বর। মোট ৩৫২টি পরিভাষা।

পরিভাষা	অর্থ	অধ্যায়
AbortController	চলমান কাজ বাতিলের নিয়ন্ত্রক।	১০
Accumulator	Loop জুড়ে ফল জমা রাখার Variable।	৪, ৬
Adapter	একটি Interface-কে আরেকটিতে রূপান্তর।	২১
ADR	Architecture-এর সিদ্ধান্তের লিখিত নথি।	৩০
AggregateError	একাধিক ত্রুটি একসঙ্গে বহনকারী Error।	১৬
Aggregation	একটি দলের সংখ্যা, মোট, সর্বোচ্চ ইত্যাদি।	১৮
Anchor	অবস্থান: `^`, `$`, `\b`।	১৭
Append-only Ledger	শুধু যোগ হয় এমন লেনদেনের তালিকা।	২৬
`aria-live`	লেখা বদলালে Screen Reader-কে তা পড়তে বলার Attribute।	৯
Arrange–Act–Assert	প্রস্তুতি, কাজ, প্রত্যাশা: Test-এর তিন ধাপ।	২২
Array	ক্রমানুসারে একাধিক মান রাখার মান।	৪
Array Literal	`[ ]` দিয়ে লেখা Array।	৬
Arrow Function	`=>` দিয়ে লেখা সংক্ষিপ্ত Function।	৫
Assignment / Reassignment	Variable-এ মান রাখা / নতুন মান রাখা।	২
`async` / `await`	Promise-এর ওপর Synchronous-এর মতো দেখতে লেখার ধরন।	১০
Asynchronous	অপেক্ষার কাজ পেছনে চলে, Program এগিয়ে যায়, ফল পরে আসে।	১০
At-least-once / At-most-once	হারায় না কিন্তু Duplicate সম্ভব / Duplicate নেই কিন্তু হারাতে পারে।	২৭
Authentication	পরিচয় যাচাই।	২০
Authorization	নির্দিষ্ট কাজের অনুমতি যাচাই।	২০
Backpressure	ধীর গ্রাহকের কারণে জমে ওঠা Data সামলানো।	২৭
Backreference	আগের Group-এর মিলের পুনরাবৃত্তি।	১৭
Batch	একসঙ্গে প্রক্রিয়া করা কাজের দল।	১৮
Bearer Token	`Authorization` Header-এ পাঠানো পরিচয়ের প্রমাণ।	২৫
Big-O	ইনপুটের সঙ্গে কাজের বৃদ্ধির হার।	১৯
Blob	কাঁচা তথ্যের টুকরো, একটি ধরনসহ।	১৫
Block	`{ }`-এর ভেতরের Code।	২
Boundary Test	সীমার দুই পাশের মান দিয়ে Test।	১৬
Boundary Value Analysis	শ্রেণির সীমার আশেপাশে Test।	২২
Bounded Context	নিজস্ব ভাষা ও নিয়মের ব্যবসায়িক এলাকা।	২৯

পরিভাষা	অর্থ	অধ্যায়
Branded Type	অদৃশ্য চিহ্নযুক্ত আলাদা Type।	২৪
Breakpoint	DevTools-এ Program থামানোর জায়গা।	১১
Browser API	Browser-এর দেওয়া সুবিধা, ভাষার অংশ নয়।	১৫
Bubbling	Event-এর DOM গাছ বেয়ে ওপরে ওঠা।	৯
Bug / Debugging	Program-এর ভুল / ভুল খুঁজে ঠিক করার কাজ।	১
Build Artifact	পরীক্ষিত, অপরিবর্তনীয় Release-বস্তু (যেমন Image)।	২৮
Call	Function চালানো।	৫
Call Stack	এই মুহূর্তে চলমান Function-গুলোর স্তূপ।	১০
Callback	অন্য Function-কে Argument হিসেবে পাঠানো Function।	৫, ১০
CAP / Eventual Consistency	Partition-এ Consistency ও Availability-র মধ্যে বেছে নেওয়া / সময়ের সঙ্গে সব Replica এক অবস্থায় পৌঁছানো।	২৯
Capstone	শেখা সব দক্ষতা একত্র করা চূড়ান্ত Project।	৩০
Capturing Group	মিলে যাওয়া অংশ ধরে রাখা বন্ধনী।	১৭
`cause`	নতুন ত্রুটির ভেতরে রাখা মূল ত্রুটি।	১১
Character Class	বন্ধনীর ভেতরের যেকোনো একটি অক্ষর।	১৭
CI	প্রতিটি পরিবর্তনে স্বয়ংক্রিয় Build ও Test।	২২
CI / CD	প্রতিটি পরিবর্তনের স্বয়ংক্রিয় যাচাই / স্বয়ংক্রিয় Release-প্রক্রিয়া।	২৮
Circuit Breaker / Bulkhead	বারবার ব্যর্থ Dependency-কে সাময়িক বিশ্রাম / প্রতিটি Dependency-র জন্য আলাদা সীমিত সম্পদ।	২৯
Class	একই ধরনের Object তৈরির ছাঁচ।	১৩
Close Code	WebSocket বন্ধের কারণের সংখ্যা।	২৭
Closure	তৈরির জায়গার Variable মনে রাখা Function।	১৪
Code Point	Unicode-এর একটি অক্ষর-নম্বর।	৮
Code Unit	JavaScript String-এর সংরক্ষণের একক (UTF-16)।	৮
Collator	ভাষার নিয়মে লেখা তুলনাকারী।	১৮
Command / Event	কাজের অনুরোধ / ঘটে যাওয়া সত্য।	২৭
Comment	মানুষের জন্য লেখা টীকা, যা চালানো হয় না।	১
CommonJS	Node.js-এর পুরোনো Module-ব্যবস্থা (`require`)।	১২
Comparison Operator	দুটি মান তুলনা করে Boolean দেয়।	৩
Compensation	ব্যর্থতার পর আগের ধাপের প্রভাব উল্টে দেওয়া।	১৬
Composite	আঁকা স্তরগুলো জোড়া লাগানো।	২৩
Composition	অন্য Object-কে অংশ হিসেবে রেখে আচরণ গঠন (has-a)।	১৩, ১৪
Composition Root	যেখানে সব নির্ভরতা জোড়া লাগে।	২১, ৩০
Computed Key	`[ ]`-এর Expression থেকে তৈরি Key।	৭
Conditional Update	যাচাই ও পরিবর্তন একই Statement-এ।	৩০
Console	Developer-এর জন্য বার্তা ও Error দেখানোর জায়গা।	১

পরিভাষা	অর্থ	অধ্যায়
Constraint	Database-এর ভেতরের নিয়ম।	২৬
Constructor	Instance তৈরির সময় চলা প্রাথমিক Code।	১৩
Content-Security-Policy	কোন উৎসের Script বা Style চলবে, তার নিয়ম।	৩০
Contract Test	একই চুক্তির সব Implementation-এ একই Test।	২১
CORS	অন্য Origin-কে Response পড়ার অনুমতির নিয়ম।	২০
Counter	Loop কতবার চলেছে তা ধরে রাখার Variable।	৪
Counter, Gauge, Histogram	শুধু বাড়ে / বাড়ে-কমে / Bucket-এ গোনা Metrics।	২৮
Coverage	কোন Code কোনো Test-এ চলেছে তার পরিমাপ।	২২
CPU Profile	Sampling দিয়ে কোন Function কত সময় চলেছে তার রেকর্ড।	২৩
CSP	কোন উৎসের Script চলবে তার Browser-নীতি।	২০
CSRF	অন্য Site থেকে ব্যবহারকারীর Cookie-সহ অনুরোধ করানো।	২০
Currying	Argument এক এক করে নেওয়ার রূপান্তর।	১৪
Cursor Pagination	"এই অবস্থানের পর থেকে" পাতা।	২৫
Custom Error	নিজের তৈরি Error-এর ধরন।	১১
Data Model	একটি জিনিসের তথ্যের গঠন ও নিয়মের নকশা।	৭
Data Type	মানের ধরন, যা ঠিক করে মানটি নিয়ে কী করা যাবে।	২
Dead Code Elimination	অব্যবহৃত ফলের হিসাব বাদ দেওয়া।	১৯
Declaration	একটি Variable তৈরি করা।	২
Decorator	একটি Object মুড়ে আচরণ বাড়ানো।	২১
Deep Copy	সব স্তরসহ সম্পূর্ণ আলাদা Copy।	৭
Deep Freeze	সব স্তরে অপরিবর্তনীয় করা।	১৮
Default Export	File-এর প্রধান Export; একটিই।	১২
Default Parameter	Argument না দিলে যে মান ব্যবহৃত হয়।	৫
`defer`	HTML পড়া শেষে Script চালানোর নির্দেশ।	৯
Dependency Injection	বাইরের নির্ভরতা Parameter হিসেবে দেওয়া।	১২, ২১
Design Pattern	একটি পরিচিত সমস্যার নামযুক্ত, পুনর্ব্যবহারযোগ্য সমাধান।	২১
Destructuring	Object বা Array থেকে মান আলাদা Variable-এ নেওয়া।	৭
Discriminated Union	একটি সাধারণ Literal Field-যুক্ত Union।	২৪
DOM	Browser-এর তৈরি Page-এর কাঠামো, যা JavaScript পড়তে ও বদলাতে পারে।	১, ৯
Dynamic Import	`import()` দিয়ে চলার সময় Module আনা।	১২
Dynamic Typing	Type মানের সঙ্গে থাকে; একই Variable-এ ভিন্ন Type-এর মান রাখা যায়।	২
Element	শুরুর Tag থেকে শেষের Tag পর্যন্ত HTML-এর একটি অংশ।	১
Element / Index	Array-র একটি মান / তার অবস্থানের নম্বর (শূন্য থেকে শুরু)।	৪, ৬
Encapsulation	অবস্থা লুকিয়ে শুধু Method দিয়ে বদলানোর নীতি।	১৩
End-to-end Test	আসল Browser-এ ব্যবহারকারীর পথ।	২২

পরিভাষা	অর্থ	অধ্যায়
Enumeration	ID অনুমান করে কী আছে তা খুঁজে বের করা।	৩০
Equivalence Partition	একই আচরণ পাওয়া ইনপুটের শ্রেণি।	২২
Error Budget, Burn Rate	অনুমোদিত ব্যর্থতা / তা খরচের গতি।	২৮
Error Code	ত্রুটির স্থির, যন্ত্রপাঠ্য পরিচয়।	১৬
Error Handler	`(err, req, res, next)` Middleware।	২৫
ES Module	`import` ও `export` ব্যবহারকারী নিজস্ব Scope-এর File।	১২
ES2015 (ES6)	JavaScript-এর বড় পরিবর্তনের সংস্করণ; তারপর প্রতি বছর একটি।	১২
Escape Sequence	`\` দিয়ে শুরু হওয়া বিশেষ অক্ষরের লিখিত রূপ।	৮
Escaping	বিশেষ অক্ষরকে আক্ষরিক অর্থ দেওয়া।	১৭
ETag	একটি Resource-এর সংস্করণের চিহ্ন।	২৫
Event	ব্যবহারকারীর কাজ বা Browser-এর ঘটনা, যেমন Click বা Submit।	১, ৯
Event Delegation	Parent-এ একটি Listener দিয়ে সন্তানদের Event সামলানো।	৯
Event Envelope	প্রতিটি Event-এর স্থির কাঠামো।	২৭
Event Listener	Event ঘটলে চলার জন্য দেওয়া Function।	৯
Event Loop	Stack খালি হলে Queue থেকে পরের কাজ এনে চালানোর ব্যবস্থা।	১০
Event Loop Delay	Event Loop-এর একটি চক্র কত দেরিতে চলেছে।	২৩
Event Object	Listener-এ আসা Event-এর তথ্য।	৯
Expected Error	পরিবেশ বা ব্যবহারকারীর কারণে, যা স্বাভাবিকভাবে সামলানো যায়।	১১
Explicit / Implicit Conversion	নিজে করা রূপান্তর / JavaScript-এর নিজে থেকে করা রূপান্তর।	২
Exponential Backoff	প্রতিবার ব্যর্থতার পর অপেক্ষা দ্বিগুণ করে আবার চেষ্টা।	১০
Exponential Backoff, Jitter	ক্রমশ দ্বিগুণ অপেক্ষা, এলোমেলো বিচ্যুতি।	২৭
Expression	যে Code থেকে একটি মান পাওয়া যায়।	৩
Factory	Object তৈরির নিয়ম লুকানো Function।	২১
Fail Fast	ভুল Configuration-এ শুরুতেই থেমে যাওয়া।	২৮
Fallthrough	`switch`-এ `break` না থাকায় পরের `case`-এ চলে যাওয়া।	৩
Feature Detection	ব্যবহারের আগে API আছে কি না যাচাই।	১৫
`filter`	শর্ত মেলা Element নিয়ে নতুন Array।	৬
`finally`	সফল বা ব্যর্থ, সব ক্ষেত্রে চলা অংশ।	১১
`find`	শর্ত মেলা প্রথম Element।	৬
Fingerprint	অনুরোধের বিষয়বস্তুর Hash।	২৯
First-Class Function	মান হিসেবে ব্যবহারযোগ্য Function।	১৪
Flag	Regex-এর আচরণ বদলানোর চিহ্ন: `g`, `i`, `u`।	৮
Flaky Test	কখনো পাস, কখনো ব্যর্থ হওয়া Test।	২২
`flatMap`	Map করে এক স্তর সমতল করা।	১৮
Frontend / Backend	Application-এর ব্যবহারকারী-মুখী অংশ / Server-এ চলা অংশ।	১
Function	নাম দেওয়া, পুনর্ব্যবহারযোগ্য একগুচ্ছ নির্দেশনা।	১

পরিভাষা	অর্থ	অধ্যায়
Function Declaration	`function name() { }` দিয়ে তৈরি Function।	৫
Function Expression	Variable-এ রাখা Function।	৫
Function Factory	নতুন Function তৈরি করে ফেরত দেওয়া Function।	১৪
Functional / Non-functional Requirement	কী করবে / কতটা ভালোভাবে করবে।	২৯
Garbage Collection	অব্যবহৃত Object-এর মেমোরি স্বয়ংক্রিয়ভাবে ফিরিয়ে নেওয়া।	২, ১৯
Generic	Type Parameter-যুক্ত Function বা Class।	২৪
Getter / Setter	Property-র মতো পড়া ও লেখা, ভেতরে Code চলে।	১৩
Graceful Shutdown	চলমান কাজ শেষ করে বন্ধ হওয়া।	২৫, ২৮
Grapheme	পাঠকের চোখে একটি অক্ষর।	৮
Greedy / Lazy	যতটা সম্ভব বেশি / কম নেওয়া।	১৭
Guard Clause	Function-এর শুরুতে অগ্রহণযোগ্য অবস্থা ধরে `return` করা।	৩, ১২
Heap Snapshot	একটি মুহূর্তে Memory-র সব Object-এর চিত্র।	২৩
Heartbeat (Ping/Pong)	সংযোগ জীবিত কি না নিয়মিত যাচাই।	২৭
Higher-Order Function	Function নেয় বা ফেরত দেয় এমন Function।	১৪
History API	Reload ছাড়া URL বদলানো ও Back সামলানো।	১৫
Hoisting	কিছু Declaration-এর, লেখার লাইনের আগেই পরিচিত থাকা।	৫
Idempotency	একই অনুরোধ বারবার এলেও ফল একবারের মতো।	১৬
Idempotency-Key	Retry-কে নিরাপদ করার Client-প্রদত্ত Key।	২৫
Idempotent	একবার বা বহুবার পাঠালে একই ফল।	২৫
Idempotent Consumer	একই Event বারবার পেলেও ফল একবারের মতো।	২৭
IDOR	ID বদলে অন্যের Object-এ পৌঁছানো।	২০
Image / Container	অপরিবর্তনীয় Template / তার চলমান Instance।	২৮
Image Digest	Image-এর বিষয়বস্তুর অনন্য Hash।	২৮
Immutable	তৈরির পর বদলানো যায় না।	৮
Inbox	Consumer-এর দিকে Event-এর Deduplication।	২৯
Index	Key থেকে সারিতে দ্রুত পৌঁছানোর `Map`।	১৮, ২৬
Inference	মান দেখে Type অনুমান।	২৪
Infinite Loop	যে Loop কখনো থামে না।	৪
Inheritance	`extends` দিয়ে আরেকটি Class-এর বৈশিষ্ট্য নেওয়া (is-a)।	১৩
Inner / Left Join	উভয় পাশে মেলা সারি / বাম পাশের সব সারি।	১৮
Input Validation	ব্যবহারের আগে বাইরে থেকে আসা মান যাচাই করা।	৩
Instance	Class থেকে `new` দিয়ে তৈরি Object।	১৩
Integration Test	একাধিক অংশ একসঙ্গে, আসল সংযোগে।	২২
Invariant	যে নিয়ম Program-এ সব সময় সত্য থাকার কথা।	১৬
Inverted Index	শব্দ থেকে সেই শব্দযুক্ত Record-এর তালিকা।	২৩
Iteration	Loop-এর ভেতরের Code-এর একবার চলা।	৪

পরিভাষা	অর্থ	অধ্যায়
JIT	চলার সময় ঘনঘন Code-কে অপ্টিমাইজড Machine Code-এ রূপান্তর।	১৯
JSON	তথ্যকে String হিসেবে প্রকাশের মান-নির্ধারিত রূপ।	৭
Key / Value	Property-র নাম / তার মান।	৭
Layered Architecture	দায়িত্বভিত্তিক স্তর ও নির্ভরতার দিক।	২১
Layout	Element-এর আকার ও অবস্থানের হিসাব।	২৩
Layout Thrashing	লেখা-পড়া পর্যায়ক্রমে বারবার Layout।	১৯
Lazy Loading	প্রয়োজনের মুহূর্তে আনা।	১৯
Lexical Scope	Scope ঠিক হয় Code কোথায় লেখা তা থেকে।	৫
Literal Type	একটি নির্দিষ্ট মানই Type (`'bus'`)।	২৪
Little's Law	একসঙ্গে চলমান = আগমনের হার x গড় সময়।	২৯
Live Binding	Import করা নাম মূল Variable-এর বর্তমান মান দেখায়।	১২
Liveness / Readiness	Process জীবিত কি না / Traffic নিতে প্রস্তুত কি না।	২৮
Logical Error	Program চলে, কিন্তু ফল ভুল।	১১
Logical Operator	`&&`, `||`, `!`: শর্ত জোড়া লাগায় বা উল্টে দেয়।	৩
Long Task	৫০ মিলিসেকেন্ডের বেশি Main Thread আটকানো কাজ।	১৯
Lookaround	পাশে কী আছে তার শর্ত, অক্ষর না নিয়ে।	১৭
Loop	একই Code বারবার চালানোর ব্যবস্থা।	৪
Lost Update	একটি হালনাগাদ অন্যটিকে মুছে দেওয়া।	২৬
LRU Cache	সবচেয়ে দীর্ঘ অব্যবহৃত Entry বাদ দেওয়া সীমিত Cache।	১৯
`map`	প্রতিটি Element রূপান্তর করে নতুন Array।	৬
Mass Assignment	অনাকাঙ্ক্ষিত ঘর ইনপুট থেকে ফলে ঢুকে পড়া।	১৭
Median / p95	মাঝের মান / ৯৫% মান যার নিচে।	১৯
Memoization	ফল মনে রেখে পুনরায় হিসাব এড়ানো।	১৪
Memory Leak	অপ্রয়োজনীয় Object জীবিত থেকে যাওয়া।	১৯
Method	একটি মানের সঙ্গে যুক্ত Function, যেমন `array.push()`।	৬, ৭
Method Chaining	একটি Method-এর ফলের ওপর সরাসরি আরেকটি Call করা।	৬
Microtask	Promise-এর Callback; প্রতিটি Task-এর পরে সবগুলো চলে।	১০
Middleware	অনুরোধ-শৃঙ্খলের একটি ধাপ `(req, res, next)`।	২৫
Migration	পুরোনো গঠনের তথ্য নতুন গঠনে রূপান্তর।	১৫, ২৬
Mock Timers	নকল ঘড়িতে Timer চালানো।	২২
Modular Monolith / Microservices	একটি Deploy, ভেতরে স্পষ্ট Module-সীমানা / আলাদাভাবে Deploy ও Scale-যোগ্য Service।	২৯
Mutating / Non-mutating	যে Method মূলটি বদলায় / যেটি নতুন মান দেয়।	৬
Mutation	একটি বিদ্যমান Array বা Object-কে বদলে ফেলা।	৬
N+1	তালিকার প্রতিটি সারির জন্য আলাদা Query।	২৬
Named Export	নির্দিষ্ট নামে Export।	১২

পরিভাষা	অর্থ	অধ্যায়
Named Group	নামযুক্ত Capturing Group।	১৭
Namespace Import	`import * as` দিয়ে সব Export একটি Object-এ।	১২
NaN	Not a Number: অর্থহীন সংখ্যা-হিসাবের ফল।	২
Narrowing	যাচাইয়ের পর Type আরও নির্দিষ্ট হওয়া।	২৪
Nested Loop	একটি Loop-এর ভেতরে আরেকটি Loop।	৪
`never`	যে Type-এর কোনো মান নেই।	২৪
NodeList	`querySelectorAll`-এর ফল; Array-র মতো, কিন্তু Array নয়।	৯
Normalization	পুনরাবৃত্তি দূর করে তথ্য সাজানো।	২৬
Object	একাধিক তথ্য একসঙ্গে রাখার মান।	২
Object Literal	`{ }` দিয়ে লেখা Object।	৭
Object URL	Memory-র Blob-এর জন্য সাময়িক `blob:` ঠিকানা।	১৫
Observer	ঘটনার ঘোষণা ও স্বাধীন শ্রোতা।	২১
Off-by-One Error	Loop একবার বেশি বা একবার কম চলার ভুল।	৪
Operator / Operand	যে চিহ্ন কাজ করে / যে মানের ওপর কাজ হয়।	৩
Operator Precedence	কোন Operator আগে হিসাব হবে, তার নিয়ম।	৩
Optimistic Concurrency	Version মিলিয়ে হালনাগাদ, Lock ছাড়া।	২৫
Optional Chaining	`?.`: মান না থাকলে থেমে `undefined` দেওয়া।	৭
Orchestration / Choreography	কেন্দ্রীয় সমন্বয় / Event-নির্ভর স্বতন্ত্র প্রতিক্রিয়া।	২৯
ORM	Table-কে Object হিসেবে ব্যবহারের Library।	২৬
Output Encoding	প্রসঙ্গ অনুযায়ী লেখাকে শুধু তথ্য হিসেবে নিরাপদ করা।	২০
Overscan	দৃশ্যমান অংশের বাইরে আঁকা বাড়তি সারি।	২৩
Parameter	Function চালানোর সময় দেওয়া মান যে নামে পাওয়া যায়।	১
Parameter / Argument	Function-এর গ্রহণের নাম / Call-এ দেওয়া মান।	৫
Partial Application	কিছু Argument আগে বসিয়ে নতুন Function।	১৪
Pivot	দুই মাত্রার Grouping-এর সারণি।	১৮
Polling / Long Polling	বারবার অনুরোধ / খবর না আসা পর্যন্ত আটকে রাখা অনুরোধ।	২৭
Polymorphism	একই Method-এ বিভিন্ন Class-এর ভিন্ন আচরণ।	১৩
Primary Key / Foreign Key	সারির অনন্য পরিচয় / অন্য Table-এর সারির উল্লেখ।	২৬
Primitive	একটি সরল মান: string, number, boolean, undefined, null, bigint, symbol।	২
Private Field	`#` দিয়ে শুরু, Class-এর বাইরে অগম্য Field।	১৩
Program	Computer-এর জন্য লেখা ধাপে ধাপে নির্দেশনা।	১
Promise	একটি অপেক্ষার কাজের ভবিষ্যৎ ফলের প্রতিনিধি Object।	১০
Property	Object-এর একটি নাম-মানের জোড়া।	৭
Property-based Test	অনেক ইনপুটে একটি সাধারণ নিয়মের যাচাই।	২২
Prototype	যে Object থেকে আরেকটি Object Property উত্তরাধিকারসূত্রে পায়।	১৩

পরিভাষা	অর্থ	অধ্যায়
Prototype Chain	Property খোঁজার জন্য Prototype থেকে Prototype-এ শিকল।	১৩
Prototype Pollution	`Object.prototype`-এ অনাকাঙ্ক্ষিত Property লেখা।	২০
Pub/Sub, Broker	Channel-এ প্রকাশ ও গ্রহণ; মাঝের Message-বাহক।	২৭
Pure Function	একই Argument-এ একই ফল; বাইরের কিছু বদলায় না।	৫, ১৪
Quality Gate	Release-এর আগে যে যাচাইগুলো পার হতে হয়।	৩০
Quantifier	কতবার: `*`, `+`, `?`, `{m,n}`।	১৭
Query Plan	Database Query কীভাবে চালাবে তার বর্ণনা।	২৬
Quota	প্রতি Website-এর Storage-সীমা।	১৫
Race Condition	কাজ শেষের ক্রমের ওপর নির্ভর করে ভুল ফল।	১০
Reconciliation	দুটি ব্যবস্থার হিসাব নিয়মিত মিলিয়ে দেখা।	২৯
Redaction	Log থেকে সংবেদনশীল তথ্য মুছে ফেলা।	২০
ReDoS	Backtracking-এর বিস্ফোরণে Program আটকে যাওয়া।	১৭
`reduce`	পুরো Array থেকে একটি মান।	৬
Refactoring	আচরণ না বদলে Code-এর গঠন উন্নত করা।	১২
Reference	মেমোরিতে একটি Object-এর ঠিকানা।	২
Referential Transparency	Expression-কে ফল দিয়ে প্রতিস্থাপনযোগ্যতা।	১৮
Regression Test	পুরোনো Bug না ফেরার Test।	১৬
Regular Expression	লেখার Pattern বর্ণনার ভাষা।	৮
Reliability	সব পরিস্থিতিতে Program-এর সঠিক ও প্রত্যাশিত আচরণ।	১৬
Remainder	ভাগ করার পর অবশিষ্ট অংশ (`%`)।	৩
Render	State থেকে Page-এর অংশ তৈরি করা।	৯
Replay / Snapshot	বাদ পড়া Event আবার পাঠানো / বর্তমান পূর্ণ অবস্থা।	২৭
Repository	সংরক্ষণের বিস্তারিত লুকানো চুক্তি।	২১
Request ID	একটি অনুরোধকে সব Log-এ চেনার ID।	২৫
Response	`fetch`-এর ফল: Status, Header এবং Body।	১০
REST	Resource ও HTTP Method-ভিত্তিক API-র নকশা।	২৫
Result Type	সফল মান বা ব্যর্থতা বহনকারী মান।	১৮
Return Value	Function যে মান ফেরত দেয়।	৫
Rolling, Blue/Green, Canary	Release ছড়ানোর তিনটি কৌশল।	২৮
RPO, RTO	সহনীয় তথ্যহানি / সেবা ফেরার সময়সীমা।	২৮
Runtime	যে Software JavaScript Code চালায়, যেমন Browser বা Node.js।	১
Runtime Error	চলার সময় ঘটা ত্রুটি।	১১
Safe Method	Server-এর অবস্থা বদলায় না।	২৫
Saga / Compensation	স্থানীয় Transaction-এর ধারা / আগের ধাপের যুক্তিসংগত উল্টো কাজ।	২৯
Salt	প্রতিটি Password-এর আলাদা এলোমেলো মান।	২০, ৩০
Savepoint	Transaction-এর ভেতরে ফেরার বিন্দু।	২৬

পরিভাষা	অর্থ	অধ্যায়
SBOM	Software-এ থাকা সব Component-এর তালিকা।	২৮
Scope	Program-এর যে অংশ থেকে একটি Variable ব্যবহার করা যায়।	২, ৫
Scope Chain	নাম খোঁজার পথ: ভেতর থেকে বাইরের দিকে।	৫
Secure Context	`https://` বা `localhost`; কিছু API শুধু এখানে।	১৫
Selector	CSS-এর নিয়মে Element চেনার লেখা: `#id`, `.class`, `tag`।	৯
Self Time	Function নিজের Code-এ কাটানো সময়।	২৩
Server-Sent Events (SSE)	Server থেকে Browser-এ একমুখী HTTP Event Stream।	২৭
Session Token	Login-এর পরে পরিচয় বহনকারী গোপন মান।	৩০
`sessionStorage`	Tab-এর আয়ু পর্যন্ত টেকা Storage।	১৫
`Set`	প্রতিটি মান একবারই থাকে এমন সংগ্রহ।	১৫
Shadowing	ভেতরের একই নামের Variable বাইরেরটিকে ঢেকে দেওয়া।	৫
Shallow Copy	শুধু প্রথম স্তর Copy; ভেতরের Object-এর ঠিকানা ভাগাভাগি হয়।	৭
Short-Circuit	ফল নিশ্চিত হয়ে গেলে বাকি অংশ হিসাব না করা।	৩
Shorthand Property	Key ও Variable একই নামে হলে একবার লেখা।	৭
Side Effect	বাইরের অবস্থা বদলানো বা বাইরে কিছু দেখানো।	৫, ১৪
SLI, SLO, SLA	মাপা সংখ্যা / লক্ষ্য / চুক্তি।	২৮
Slug	Web Address-এর উপযোগী, হাইফেন দিয়ে জোড়া শব্দ।	৮
Spread	`...`: একটি Array বা Object-এর বিষয়বস্তু নতুনটিতে ছড়িয়ে দেওয়া।	৬
SQL Injection	ইনপুট SQL-এর অংশ হয়ে যাওয়া।	২০
Stable Sort	সমান উপাদানের মূল ক্রম রক্ষাকারী সাজানো।	১৮
Stack Trace	Error পর্যন্ত পৌঁছাতে কোন Function থেকে কোনটি Call হয়েছে, তার তালিকা।	১, ১১
State	Application-এর বর্তমান তথ্য, যা থেকে Page আঁকা হয়।	৯
State Machine	অবস্থা ও বৈধ পরিবর্তনের টেবিল।	২১
Statement	একটি সম্পূর্ণ নির্দেশ।	১
Static	Class-এর নিজের সদস্য, Instance-এর নয়।	১৩
Static Typing	Code চালানোর আগে Type যাচাই।	২৪
Sticky Flag	ঠিক `lastIndex` থেকেই মেলানো।	১৭
`storage` Event	অন্য Tab-এ Storage বদলালে আসা Event।	১৫
Strategy	বিনিময়যোগ্য নিয়ম, একই আকারে।	২১
Strict Equality	`===`: Type এবং মান দুটোই মেলায়।	২
String	Quotation Mark-এ ঘেরা লেখা।	১
Structural Sharing	নতুন ও পুরোনো অবস্থার অপরিবর্তিত অংশ ভাগাভাগি।	১৮
Structured Log	যন্ত্রপাঠ্য গঠনের (যেমন JSON) Log লাইন।	১৬, ২৮
Stub / Spy / Mock / Fake	নির্দিষ্ট উত্তর / ডাক লেখা / প্রত্যাশা যাচাই / কার্যকর বিকল্প।	১৬
Syntax	ভাষার গঠনগত নিয়ম।	১

পরিভাষা	অর্থ	অধ্যায়
Syntax Error	ব্যাকরণের ভুল; Program চলে না।	১১
Table-driven Test	ইনপুট ও প্রত্যাশার টেবিল থেকে তৈরি Test।	২২
Task	Timer, Event-এর মতো Callback; একবারে একটি চলে।	১০
TDD	Test আগে লিখে Code লেখার চক্র।	২২
Template Literal	Backtick-এর String, যাতে `${ }` দিয়ে মান বসানো যায়।	৮
Terminal	লিখে Command দেওয়ার জানালা।	১
Ternary Operator	`শর্ত ? মান১ : মান২`।	৩
Test Double	Test-এ আসল নির্ভরতার নিয়ন্ত্রিত বিকল্প।	১৬
Testing Pyramid	Unit, Integration, E2E-এর অনুপাতের নির্দেশিকা।	১৬
`this`	Method Call-এর সময় যে Object দিয়ে Call করা হয়েছে।	৭
Threat Model	কী রক্ষা, কার কাছ থেকে, কীভাবে—তার বিশ্লেষণ।	২০
Throttle	নির্দিষ্ট সময়ে সর্বোচ্চ একবার চালানো।	১৪
`throw`	ত্রুটি ছোড়া; স্বাভাবিক প্রবাহ থামে।	১১
Timing-safe Comparison	তুলনার সময় দেখে কিছু বোঝা যায় না এমন তুলনা।	৩০
Tokenization	লেখাকে শব্দে ভাগ করা।	৮
Top-level `await`	Module-এর বাইরের স্তরে `await`।	১২
Total Time	ডাকা Function-সহ মোট সময়।	২৩
Trace Table	প্রতিটি Iteration-এ Variable-এর মান লিখে রাখার টেবিল।	৪
Trace, Span	একটি অনুরোধের পুরো পথ / তার একটি ধাপ।	২৮
Traceability	Requirement থেকে Code ও Test পর্যন্ত যোগসূত্র।	৩০
Transaction (ACID)	অবিভাজ্য কাজের একক: Atomicity, Consistency, Isolation, Durability।	২৬
Transactional Outbox	Business পরিবর্তন ও প্রকাশিতব্য Event একই Transaction-এ লেখা।	২৭
Transferable	Copy ছাড়াই মালিকানা হস্তান্তরযোগ্য Object।	২৩
Trust Boundary	যেখানে বাইরের তথ্য Program-এ ঢোকে।	১১, ২০, ৩০
Truthy / Falsy	শর্তে যে মান সত্য / মিথ্যা হিসেবে গণ্য হয়।	৩
Twelve-Factor App	Cloud-উপযোগী Application-এর বারোটি নীতি।	২৮
Type Conversion	এক Type-এর মানকে অন্য Type-এ বদলানো।	২
Type Erasure	Compile-এর পর Type মুছে যাওয়া।	২৪
Type Guard	`value is T` ফেরত দেওয়া যাচাই Function।	২৪
Unexpected Error	Program-এর ভুল; ডেভেলপারকে ঠিক করতে হয়।	১১
Unicode Normalization	একই লেখার ভিন্ন ভিন্ন Code Point-রূপকে একটি আদর্শ রূপে আনা।	৮
Union	কয়েকটি Type-এর যেকোনো একটি।	২৪
URLSearchParams	URL-এর Query অংশ পড়া ও লেখার Object।	১৫
User Timing	`mark` ও `measure` দিয়ে নামযুক্ত পরিমাপ।	২৩
Utility Type	বিদ্যমান Type থেকে নতুন Type (`Pick`, `Partial`)।	২৪

পরিভাষা	অর্থ	অধ্যায়
Variable	একটি নাম, যার সঙ্গে একটি মান যুক্ত।	১
Virtual Scrolling	শুধু দৃশ্যমান সারি আঁকা।	২৩
Web Worker	আলাদা Thread-এ চলা Script।	২৩
WebSocket	HTTP Upgrade থেকে তৈরি দ্বিমুখী স্থায়ী সংযোগ।	২৭
Wrapper	মূল Function-কে মুড়ে নতুন আচরণ যোগ করা Function।	১৪
XSS	ব্যবহারকারীর লেখার মাধ্যমে Page-এ ক্ষতিকর Code চালানোর আক্রমণ।	৮, ২০

পরিশিষ্ট খ — Companion Project-এর তালিকা

প্রতিটি অধ্যায়ের Project Companion Code-এ `chapter-NN/project/<নাম>/` Folder-এ, আর অধ্যায়ের উদাহরণ ও সমাধান `chapter-NN/examples/` -এ। প্রতিটি Project-এর README-তে বিস্তারিত চালানোর নিয়ম ও সীমাবদ্ধতা আছে।

অধ্যায়	Project	ধরন	চালানো	npm Package
১	`interactive-greeting-app`	Browser	`node --test`; Local Server-এ `index.html`	নেই
২	`student-result-calculator`	Node.js	`node --test`	নেই
৩	`shopping-discount-calculator`	Node.js	`node --test`	নেই
৪	`cart-analyzer`	Node.js	`node --test`	নেই
৫	`invoice-calculator`	Node.js	`node --test`	নেই
৬	`inventory-dashboard`	Node.js	`node --test`	নেই
৭	`student-management`	Node.js	`node --test`	নেই
৮	`content-search-utility`	Node.js	`node --test`	নেই
৯	`task-manager`	Browser	`node --test`; Local Server-এ `index.html`	নেই
১০	`user-directory-explorer`	Browser	`node --test`; Local Server-এ `index.html`	নেই
১১	`order-validation`	Browser	`node --test`; Local Server-এ `index.html`	নেই
১২	`modular-shopping-cart`	Browser	`node --test`; Local Server-এ `index.html`	নেই
১৩	`booking-lab`	Node.js	`node --test`	নেই
১৪	`sales-function-lab`	Node.js	`node --test`	নেই
১৫	`bookmark-vault`	Browser	`node --test`; Local Server-এ `index.html`	নেই
১৬	`booking-guard`	Node.js	`node --test`	নেই
১৭	`form-shield`	Node.js	`node --test`	নেই
১৮	`ledger-flow`	Node.js	`node --test`	নেই
১৯	`perf-lab`	Node.js	`node --test`	নেই
২০	`secure-trip`	Node.js	`node --test`	নেই
২১	`visa-flow`	Browser	`node --test`; Local Server-এ `index.html`	নেই
২২	`currency-desk`	Node.js	`npm test`; `npm start`	নেই
২৩	`travel-search`	Browser	`node --test`; Local Server-এ `index.html`	নেই

অধ্যায়	Project	ধরন	চালানো	npm Package
২৪	`typesafe-claims`	Node.js	`npm test`	`@types/node`, `typescript`
২৫	`lead-desk`	Node.js	`npm test`; `npm start`	`express`
২৬	`travel-data`	Node.js	`node --test`	নেই
২৭	`travel-live`	Node.js	`npm test`; `npm start`	`ws`
২৮	`deploy-ready`	Node.js	`npm test`; `npm start`	নেই
২৯	`travel-saga`	Node.js	`npm test`	নেই
৩০	`travel-master-pro`	Node.js	`npm test`; `npm start`	`@types/node`, `typescript`

পরিশিষ্ট গ — যাচাই-পরিবেশ

এই বইয়ের Code যে পরিবেশে চালিয়ে দেখা হয়েছে, এবং কী চালানো হয়নি—সংক্ষেপে।

বিষয়	সংস্করণ বা অবস্থা
Operating System	Windows 10 Pro (x64)
Node.js	22.16.0
Browser	Google Chrome 155 (Headless, Chrome DevTools Protocol দিয়ে)
TypeScript	6.0.2, @types/node 25.6.0 (অধ্যায় ২৪, ৩০)
npm Package (Install না করে, আগে থেকে থাকা কপি)	express 4.22.1 (অধ্যায় ২৫), ws 7.5.10 (অধ্যায় ২৭), yaml 2.8.3 ও js-yaml 3.14.2 (অধ্যায় ২৮-এর যাচাই)
Database	SQLite (Node.js-এর `node:sqlite`)

প্রতিটি অধ্যায়ের Project-এর স্বয়ংক্রিয় Test (`node --test`, শেষ যাচাইয়ে পাস করা Test-এর সংখ্যা): অধ্যায় ১: ৮; অধ্যায় ২: ১৪; অধ্যায় ৩: ১৬; অধ্যায় ৪: ১৫; অধ্যায় ৫: ১৪; অধ্যায় ৬: ১৬; অধ্যায় ৭: ১৬; অধ্যায় ৮: ১৬; অধ্যায় ৯: ১৬; অধ্যায় ১০: ১৮; অধ্যায় ১১: ১৬; অধ্যায় ১২: ২৪; অধ্যায় ১৩: ২০; অধ্যায় ১৪: ২২; অধ্যায় ১৫: ২১; অধ্যায় ১৬: ২৪; অধ্যায় ১৭: ২৪; অধ্যায় ১৮: ২২; অধ্যায় ১৯: ২০; অধ্যায় ২০: ২৪; অধ্যায় ২১: ৩১; অধ্যায় ২২: ৩৮; অধ্যায় ২৩: ১৫; অধ্যায় ২৪: ১১; অধ্যায় ২৫: ১০; অধ্যায় ২৬: ২১; অধ্যায় ২৭: ১৮; অধ্যায় ২৮: ১৪; অধ্যায় ২৯: ১৪; অধ্যায় ৩০: ১৭। মোট ৫৫৫টি।

চালানো হয়নি (সংশ্লিষ্ট অধ্যায়ে চিহ্নিত): PostgreSQL ও Prisma-র Code (অধ্যায় ২৬); Socket.IO ও Redis (অধ্যায় ২৭; Redis-এর আচরণ in-memory অনুকরণে); `docker build`, `docker compose up` এবং GitHub Actions Workflow (অধ্যায় ২৮); কোনো আসল Payment Gateway (অধ্যায় ২৯, ৩০)। macOS, Linux, Firefox ও Safari-তে আলাদা পরীক্ষা করা হয়নি।

পরিশিষ্ট ঘ — তথ্যসূত্র ও আরও পড়ুন

প্রতিটি অধ্যায়ের "আরও পড়ুন" তালিকা একত্রে, প্রথম যে অধ্যায়ে এসেছে সেই অনুসারে সাজানো; একই ঠিকানা একাধিক অধ্যায়ে থাকলে শুধু একবার, বন্ধনীতে অন্য অধ্যায়গুলোর নম্বর। মোট ১৪৮টি সূত্র। প্রকাশের সময় ঠিকানাগুলো বদলে থাকতে পারে।

অধ্যায় ১

- **MDN Web Docs — JavaScript Guide:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide` — JavaScript ভাষার ধাপে ধাপে নির্দেশিকা।
- **MDN Web Docs — Learn Web Development:** `https://developer.mozilla.org/en-US/docs/Learn_web_development` — HTML, CSS ও JavaScript একসঙ্গে শেখার পাঠক্রম।
- **Node.js Documentation:** `https://nodejs.org/en/learn` — Node.js Install করা এবং ব্যবহার শুরুর নির্দেশনা।
- **Visual Studio Code Documentation:** `https://code.visualstudio.com/docs` — Editor এবং Integrated Terminal ব্যবহারের নির্দেশনা।
- **Chrome DevTools Documentation:** `https://developer.chrome.com/docs/devtools` — Console এবং অন্যান্য Developer Tool-এর ব্যবহার।

অধ্যায় ২

- **MDN — Grammar and types:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types` — Variable ও Type-এর পূর্ণ বিবরণ।
- **MDN — JavaScript data types and data structures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures` — প্রতিটি Type-এর বিস্তারিত।
- **MDN — typeof:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof` — `typeof`-এর সব সম্ভাব্য ফল।
- **MDN — Equality comparisons and sameness:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Equality_comparisons_and_sameness` — `==` এবং `===`-এর তুলনা।

অধ্যায় ৩

- **MDN — Expressions and operators:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators` — সব Operator-এর বিবরণ।
- **MDN — if...else:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/if...else` — শর্তযুক্ত Statement।

- **MDN — switch:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/switch` — `switch`, `break` এবং Fallthrough।
- **MDN — Operator precedence:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence` — পূর্ণ Precedence তালিকা।
- **MDN — Falsy:** `https://developer.mozilla.org/en-US/docs/Glossary/Falsy` — Falsy মানের সম্পূর্ণ তালিকা।

অধ্যায় ৪

- **MDN — Loops and iteration:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration` — সব ধরনের Loop-এর বিবরণ।
- **MDN — for:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for` — `for` Statement-এর পূর্ণ নিয়ম।
- **MDN — while:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/while` — `while` Statement।
- **MDN — break:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/break` — `break`-এর ব্যবহার।

অধ্যায় ৫

- **MDN — Functions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions` — Function-এর পূর্ণ নির্দেশিকা।
- **MDN — Arrow function expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions` — Arrow Function-এর নিয়ম ও সীমা।
- **MDN — Default parameters:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters` — Default Parameter।
- **MDN — Scope:** `https://developer.mozilla.org/en-US/docs/Glossary/Scope` — Scope-এর সংজ্ঞা।
- **MDN — Hoisting:** `https://developer.mozilla.org/en-US/docs/Glossary/Hoisting` — Hoisting-এর ব্যাখ্যা।

অধ্যায় ৬

- **MDN — Array:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array` — সব Array Method-এর তালিকা।
- **MDN — Indexed collections:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Indexed_collections` — Array-র নির্দেশিকা।
- **MDN — Array.prototype.map():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map` — `map`-এর বিবরণ।
- **MDN — Array.prototype.reduce():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce` — `reduce`-এর বিবরণ ও উদাহরণ।

- **MDN — for...of:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...of` — `for...of` Statement।

অধ্যায় ৭

- **MDN — Working with objects:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects` — Object-এর পূর্ণ নির্দেশিকা।
- **MDN — Destructuring:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring` — Destructuring-এর সব রূপ।
- **MDN — Optional chaining:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining` — `?.` -এর নিয়ম।
- **MDN — this:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this` — `this` -এর আচরণ।
- **MDN — JSON:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON` — `JSON.stringify` ও `JSON.parse`।

অধ্যায় ৮

- **MDN — String:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String` — সব String Method-এর তালিকা।
- **MDN — String.prototype.normalize():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/normalize` — Unicode Normalization।
- **MDN — Intl.Segmenter:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter` — চোখে দেখা অক্ষর ও শব্দে ভাগ করা।
- **MDN — Regular expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions` — Regex-এর নির্দেশিকা। (অধ্যায় ১৭-এও)
- **OWASP — Cross Site Scripting (XSS):** `https://owasp.org/www-community/attacks/xss/` — XSS আক্রমণের ব্যাখ্যা।

অধ্যায় ৯

- **MDN — Introduction to the DOM:** `https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction` — DOM-এর মূল ধারণা।
- **MDN — Introduction to events:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events` — Event, Bubbling এবং Delegation।
- **MDN — Event bubbling:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling` — Propagation-এর বিস্তারিত।

- **MDN — Window.localStorage:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage` — Storage-এর নিয়ম ও সীমা।
- **web.dev — Learn Accessibility:** `https://web.dev/learn/accessibility` — সবার জন্য ব্যবহারযোগ্য Web Page। (অধ্যায় ৩০-এও)

অধ্যায় ১০

- **MDN — Introducing asynchronous JavaScript:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing` — Asynchronous ধারণার ভূমিকা।
- **MDN — Using promises:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises` — Promise-এর বিস্তারিত নির্দেশিকা।
- **MDN — Using the Fetch API:** `https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch` — `fetch`, Response এবং ত্রুটি।
- **MDN — AbortController:** `https://developer.mozilla.org/en-US/docs/Web/API/AbortController` — বাতিল ও সময়সীমা।
- **Node.js — The Node.js Event Loop:** `https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick` — Node.js-এর Event Loop-এর বিবরণ।

অধ্যায় ১১

- **MDN — Control flow and error handling:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling` — `try`, `catch`, `throw` ও `finally`।
- **MDN — Error:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error` — Error-এর ধরন, `cause` এবং নিজের Error।
- **MDN — Window: unhandledrejection event:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/unhandledrejection_event` — ধরা না পড়া Promise-এর ত্রুটি।
- **Chrome DevTools — Debug JavaScript:** `https://developer.chrome.com/docs/devtools/javascript` — Breakpoint ও Step-এর ব্যবহার।
- **OWASP — Input Validation Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html` — Trust Boundary-তে যাচাইয়ের নীতি। (অধ্যায় ১৭-এও)

অধ্যায় ১২

- **MDN — JavaScript modules:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules` — Browser-এ Module-এর পূর্ণ নির্দেশিকা। (অধ্যায় ২১-এও)
- **MDN — import:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import` — Import-এর সব রূপ।

- **Node.js — ECMAScript modules:** `https://nodejs.org/api/esm.html` — Node.js-এ ES Module, `import.meta`।
- **MDN — structuredClone():** `https://developer.mozilla.org/en-US/docs/Web/API/Window/structuredClone` — Deep Copy।
- **MDN — Object.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy` — দলে ভাগ করা।

অধ্যায় ১৩

- **MDN — Classes:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes` — Class-এর সম্পূর্ণ রেফারেন্স।
- **MDN — Private properties:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_properties` — `#` Field ও Method।
- **MDN — Inheritance and the prototype chain:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain` — Prototype Chain-এর গভীর ব্যাখ্যা।
- **MDN — extends:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends` — Inheritance ও `super`।
- **MDN — Function.prototype.bind():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind` — `this` স্থির করা।

অধ্যায় ১৪

- **MDN — Closures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures` — Closure-এর বিস্তারিত ও প্রচলিত ভুল।
- **MDN — Function.prototype.call():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call` — `call`, `apply`, `bind`।
- **MDN — Map:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map` — Key-Value সংগ্রহ।
- **Wikipedia — Pure function:** `https://en.wikipedia.org/wiki/Pure_function` — Pure Function-এর সংজ্ঞা ও উদাহরণ।
- **Node.js — Mocking timers in tests:** `https://nodejs.org/api/test.html#class-mocktimers` — `mock.timers` দিয়ে Timer-নির্ভর Test।

অধ্যায় ১৫

- **MDN — URL:** `https://developer.mozilla.org/en-US/docs/Web/API/URL` — `URL` ও `URLSearchParams`।
- **MDN — Web Storage API:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API` — `localStorage`, `sessionStorage` ও `storage` Event।

- **MDN — Storage quotas and eviction criteria:** `https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria` — Storage-এর সীমা ও মুছে যাওয়া।
- **MDN — History API:** `https://developer.mozilla.org/en-US/docs/Web/API/History_API` — `pushState`, `replaceState`, `popstate`।
- **MDN — Blob:** `https://developer.mozilla.org/en-US/docs/Web/API/Blob` — Blob, File ও Object URL।

অধ্যায় ১৬

- **Node.js — Test runner:** `https://nodejs.org/api/test.html` — `describe`, Hook, Mock, Coverage। (অধ্যায় ২২-এও)
- **Node.js — Assert:** `https://nodejs.org/api/assert.html` — সব Assertion।
- **MDN — AggregateError:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError` — একাধিক ত্রুটি।
- **MDN — new.target:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new.target` — Constructor-এর ভেতরে ডাকার Class।
- **Martin Fowler — Test Double:** `https://martinfowler.com/bliki/TestDouble.html` — Stub, Mock, Fake-এর সংজ্ঞা।

অধ্যায় ১৭

- **MDN — Lookahead assertion:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Lookahead_assertion` — Lookaround।
- **MDN — Unicode character class escape:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Unicode_character_class_escape` — `\p{...}`।
- **OWASP — Regular expression Denial of Service:** `https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS` — ReDoS-এর ব্যাখ্যা।

অধ্যায় ১৮

- **MDN — Array.prototype.flatMap():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flatMap` — `flatMap`।
- **MDN — Map.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy` — Key-র ধরন রেখে Grouping।
- **MDN — Intl.Collator:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator` — ভাষাভিত্তিক তুলনা।
- **MDN — Array.prototype.sort():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort` — Stability ও Comparator।

- **Immer — Introduction:** `https://immerjs.github.io/immer/` — Immutable হালনাগাদের একটি জনপ্রিয় Library।

অধ্যায় ১৯

- **MDN — Performance.now():** `https://developer.mozilla.org/en-US/docs/Web/API/Performance/now` — উচ্চ-নির্ভুল সময় মাপা।
- **web.dev — Optimize long tasks:** `https://web.dev/articles/optimize-long-tasks` — Main Thread-এ কাজ ভাগ করা।
- **MDN — Memory management:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_management` — Garbage Collection ও Reachability।
- **Chrome DevTools — Analyze runtime performance:** `https://developer.chrome.com/docs/devtools/performance` — Performance Panel।
- **web.dev — Web Vitals:** `https://web.dev/articles/vitals` — LCP, INP, CLS।

অধ্যায় ২০

- **OWASP Top 10:** `https://owasp.org/www-project-top-ten/` — Web Application-এর প্রধান ঝুঁকি।
- **OWASP Cheat Sheet Series:** `https://cheatsheetseries.owasp.org/` — XSS, Session, Password Storage, CSRF-এর ব্যবহারিক নির্দেশিকা।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP` — CSP-র Directive।
- **MDN — Set-Cookie:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie` — Cookie-র Flag।
- **Node.js — crypto.scrypt:** `https://nodejs.org/api/crypto.html#cryptoscryptpassword-salt-keylen-options-callback` — Password Hashing।

অধ্যায় ২১

- **Refactoring.Guru — Design Patterns:** `https://refactoring.guru/design-patterns` — প্রতিটি Pattern-এর ব্যাখ্যা ও উদাহরণ।
- **Patterns.dev:** `https://www.patterns.dev/` — JavaScript-কেন্দ্রিক Design ও Rendering Pattern।
- **Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern:** `https://martinfowler.com/articles/injection.html` — DI-এর মূল প্রবন্ধ।
- **Alistair Cockburn — Hexagonal Architecture:** `https://alistair.cockburn.us/hexagonal-architecture/` — Ports and Adapters।

অধ্যায় ২২

- **Martin Fowler — The Practical Test Pyramid:** `https://martinfowler.com/articles/practical-test-pyramid.html` — স্তরভিত্তিক কৌশল।
- **Playwright — Getting started:** `https://playwright.dev/docs/intro` — পেশাদার E2E Testing।
- **fast-check — Property based testing:** `https://fast-check.dev/` — Shrinking-সহ Property Test।
- **GitHub Docs — Building and testing Node.js:** `https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs` — CI Workflow।

অধ্যায় ২৩

- **MDN — User Timing:** `https://developer.mozilla.org/en-US/docs/Web/API/Performance_API/User_timing` — `mark` ও `measure`।
- **Node.js — Performance measurement APIs:** `https://nodejs.org/api/perf_hooks.html` — `PerformanceObserver`, `monitorEventLoopDelay`।
- **web.dev — Rendering performance:** `https://web.dev/articles/rendering-performance` — Pipeline ও `transform`।
- **MDN — Using Web Workers:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers` — Worker ও Transferable।
- **Chrome DevTools — Fix memory problems:** `https://developer.chrome.com/docs/devtools/memory-problems` — Heap Snapshot।

অধ্যায় ২৪

- **TypeScript Handbook:** `https://www.typescriptlang.org/docs/handbook/intro.html` — ভাষার পূর্ণ পরিচয়।
- **TSConfig Reference:** `https://www.typescriptlang.org/tsconfig/` — প্রতিটি Compiler Setting।
- **TypeScript — Narrowing:** `https://www.typescriptlang.org/docs/handbook/2/narrowing.html` — Type Guard ও Discriminated Union।
- **TypeScript — Utility Types:** `https://www.typescriptlang.org/docs/handbook/utility-types.html` — `Pick`, `Omit`, `Partial`।
- **Node.js — TypeScript support:** `https://nodejs.org/api/typescript.html` — Type Stripping ও তার সীমা।

অধ্যায় ২৫

- **Express — Guide:** `https://expressjs.com/en/guide/routing.html` — Routing ও Middleware।
- **Express — Error handling:** `https://expressjs.com/en/guide/error-handling.html` — Express 4 ও 5-এর পার্থক্যসহ।

- **MDN — HTTP response status codes:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Status` — প্রতিটি Status-এর অর্থ।
- **Stripe — Idempotent requests:** `https://docs.stripe.com/api/idempotent_requests` — Idempotency-Key-এর বাস্তব ব্যবহার। (অধ্যায় ২৯-এও)
- **Node.js — HTTP server.close():** `https://nodejs.org/api/http.html#serverclosecallback` — Graceful Shutdown-এর ভিত্তি।

অধ্যায় ২৬

- **Node.js — SQLite:** `https://nodejs.org/api/sqlite.html` — `node:sqlite`-এর API। (অধ্যায় ৩০-এও)
- **SQLite — Query Planning:** `https://www.sqlite.org/queryplanner.html` — Index ও Plan কীভাবে কাজ করে।
- **PostgreSQL — Transaction Isolation:** `https://www.postgresql.org/docs/current/transaction-iso.html` — Isolation Level।
- **PostgreSQL — Using EXPLAIN:** `https://www.postgresql.org/docs/current/using-explain.html` — Query Plan পড়া।
- **Prisma — Documentation:** `https://www.prisma.io/docs` — ORM, Migration ও Client।

অধ্যায় ২৭

- **MDN — WebSocket API:** `https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API` — Browser-এর WebSocket ও Close Code।
- **MDN — Using server-sent events:** `https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events` — `EventSource` ও Stream-এর Format।
- **RFC 6455 — The WebSocket Protocol:** `https://www.rfc-editor.org/rfc/rfc6455` — Handshake, Frame ও Close Code।
- **ws — README:** `https://github.com/websockets/ws` — Node.js-এর WebSocket Server, Heartbeat-এর উদাহরণ।
- **Socket.IO — Documentation:** `https://socket.io/docs/v4/` — Room, Acknowledgement ও Redis Adapter।
- **Redis — Pub/Sub:** `https://redis.io/docs/latest/develop/interact/pubsub/` — Delivery-র নিশ্চয়তা ও সীমা।

অধ্যায় ২৮

- **The Twelve-Factor App:** `https://12factor.net/` — মূল নীতিমালা।
- **Docker — Building best practices:** `https://docs.docker.com/build/building/best-practices/` — Dockerfile লেখার পরামর্শ।
- **Docker — Compose file reference:** `https://docs.docker.com/reference/compose-file/` — Compose-এর প্রতিটি Key।

- **GitHub — Security hardening for GitHub Actions:** `https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions` — অনুমতি, Pin ও Secret।
- **Prometheus — Exposition formats:** `https://prometheus.io/docs/instrumenting/exposition_formats/` — Text Format।
- **OpenTelemetry — Concepts:** `https://opentelemetry.io/docs/concepts/` — Trace, Metrics, Log।
- **Google SRE Workbook — Alerting on SLOs:** `https://sre.google/workbook/alerting-on-slos/` — Burn Rate-ভিত্তিক Alert।
- **Node.js — Process signal events:** `https://nodejs.org/api/process.html#signal-events` — SIGTERM ও SIGINT।

অধ্যায় ২৯

- **Microservices.io — Saga pattern:** `https://microservices.io/patterns/data/saga.html` — Orchestration ও Choreography।
- **Microsoft Azure Architecture Center — Saga:** `https://learn.microsoft.com/en-us/azure/architecture/patterns/saga` — Compensation-এর নকশা।
- **Martin Fowler — Microservices:** `https://martinfowler.com/articles/microservices.html` — মূল ধারণা ও মূল্য।
- **Martin Fowler — MonolithFirst:** `https://martinfowler.com/bliki/MonolithFirst.html` — কেন Monolith দিয়ে শুরু।
- **AWS Builders' Library:** `https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/` ও `https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/` — Timeout, Jitter ও নিরাপদ Retry।

অধ্যায় ৩০

- **OWASP — Password Storage Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html` — scrypt, Argon2id ও Parameter।
- **OWASP — Session Management Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html` — Token, Cookie ও মেয়াদ।
- **OWASP — Authorization Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html` — মালিকানা যাচাই ও IDOR।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP` — CSP-র প্রতিটি Directive।
- **Michael Nygard — Documenting Architecture Decisions:** `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions` — ADR-এর মূল প্রস্তাব।